游戏编程精粹 3

GAME PROGRAMMING Gems 3

内附光盘

Dante Treglia 编
张磊 译

人民邮电出版社
POSTS & TELECOMMUNICATIONS PRESS

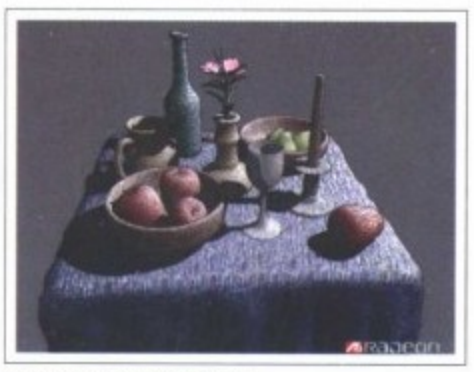
游戏编程精粹3

本书是“游戏编程精粹”系列的最新一卷，充满了即学即用的大师技巧、创意、建议和代码，并且提供了成功商业游戏中的许多解决方案。本书覆盖了游戏开发的所有关键阶段，融合了70位顶尖高手的开发心得，每章还由一位该领域的专家编辑把关，以确保内容的原创性、准确性和实用性。

除了涉及必不可少的数学、图形学、编程技术、音频处理和人工智能领域，本书还包含了最新的网络和多玩家游戏技术，所有的代码由C和C++编写，都能立刻被读者运用到自己的项目中去。

对于游戏开发者来说，本著作无疑是必备的参考资料。

© NVIDIA CORPORATION 2002



© ATI RESEARCH 2002

ISBN 7-115-10870-6

9 787115 108708 >

ISBN7-115-10870-6/TP·3189
定价：85.00 元（附光盘）

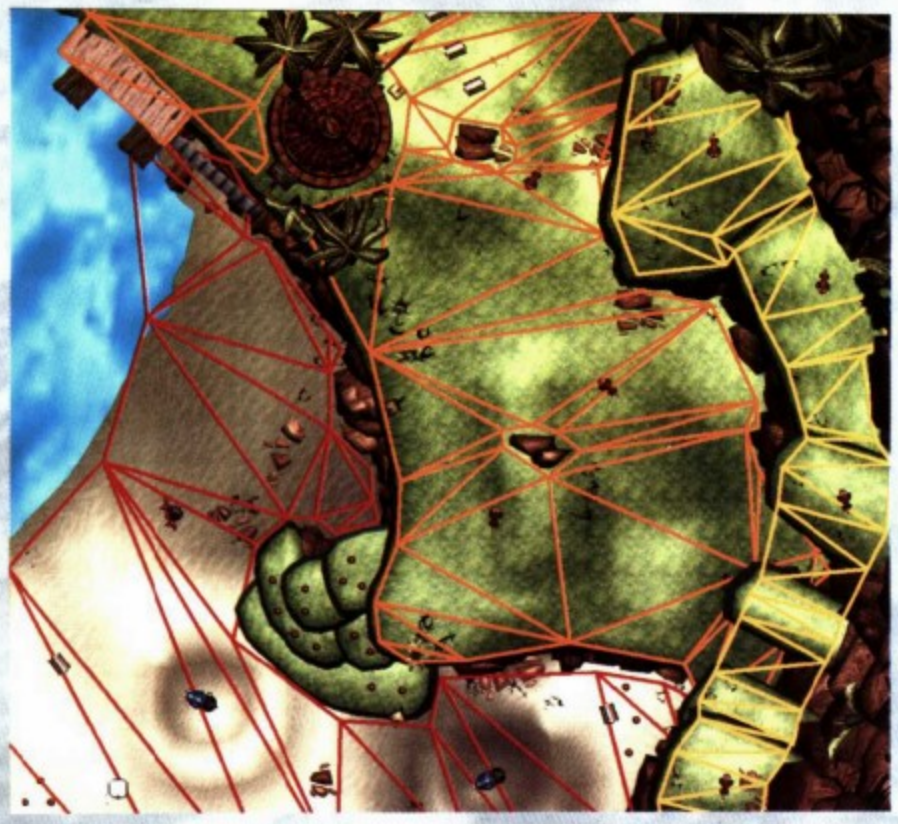
人民邮电出版社
http: // www.ptpress.com.cn

Download at www.Pin5i.Com

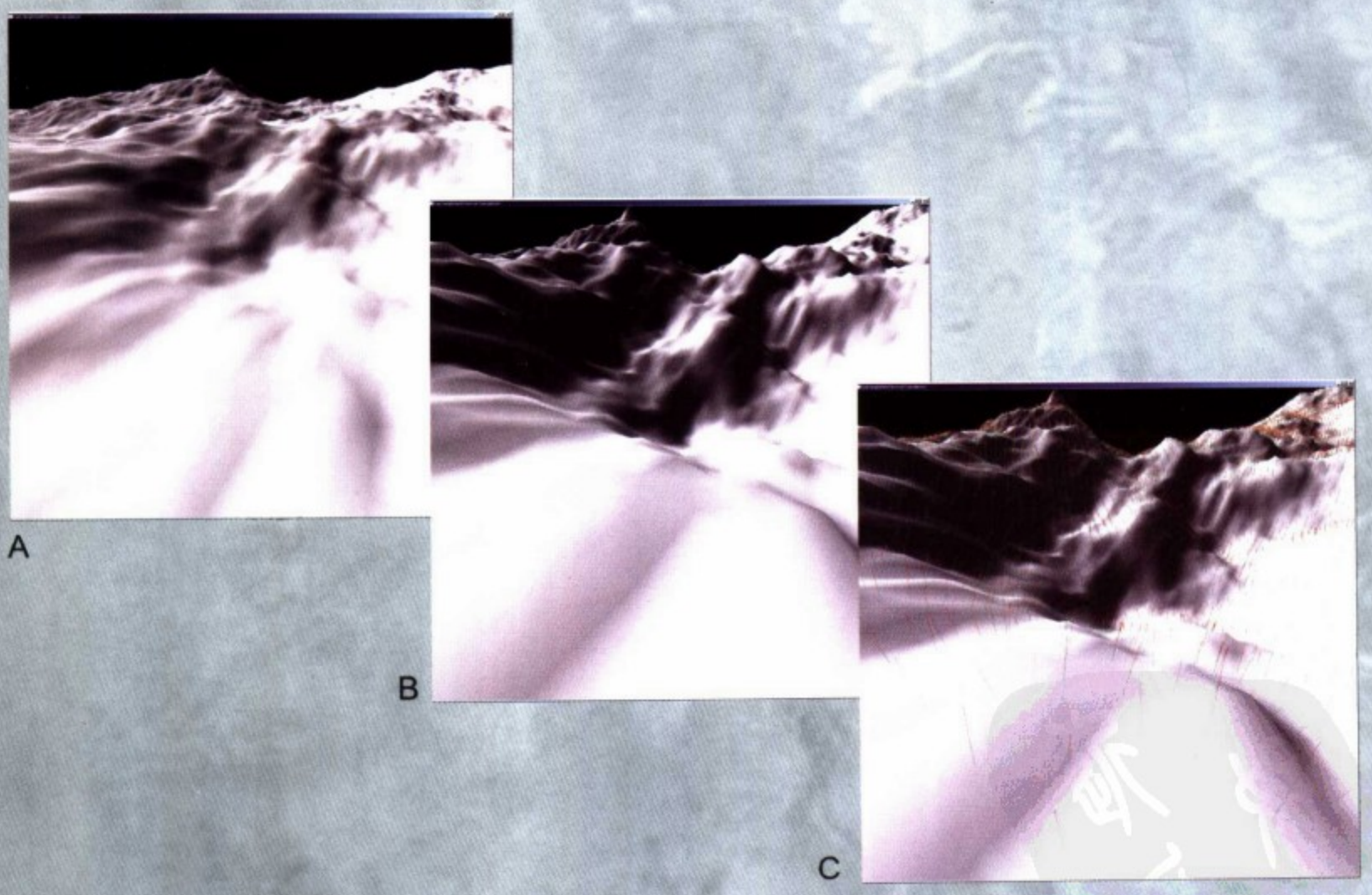
彩图1　自然的选择，饼状菜单的演进：*The Sims* 中的饼状菜单

彩图2　蝎子坦克的网格（A）加上了受限的骨节结构（B）并采用逆向运动以达到紫色效应点（C）。（B）中的红、绿、蓝弧线说明了在每个关节的*x*-、*y*-、*z*-轴上旋转角是如何受限的。Jason Weber 提供

Download at www.Pin5i.com

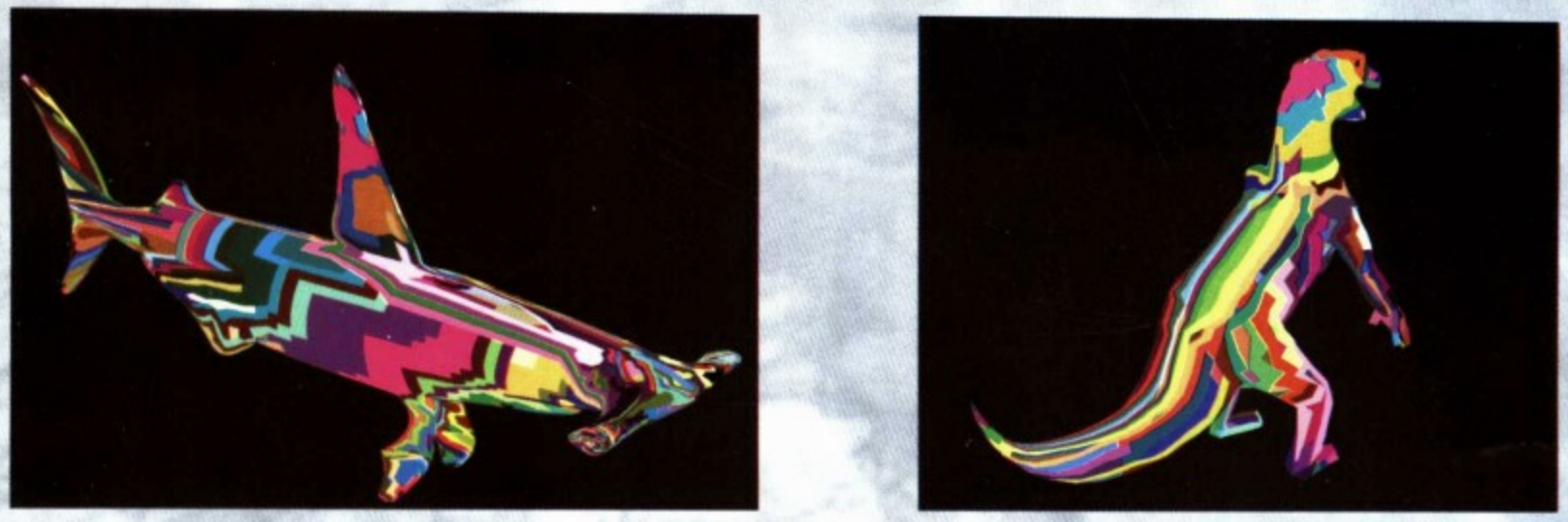


彩图3 游戏*Jak and Daxter: The Precursor Legacy*中的俯视图，其中的背景为游览网格。生物使用这个游览网格，能够智能地了解地形，并表现出多种复杂的移动行为



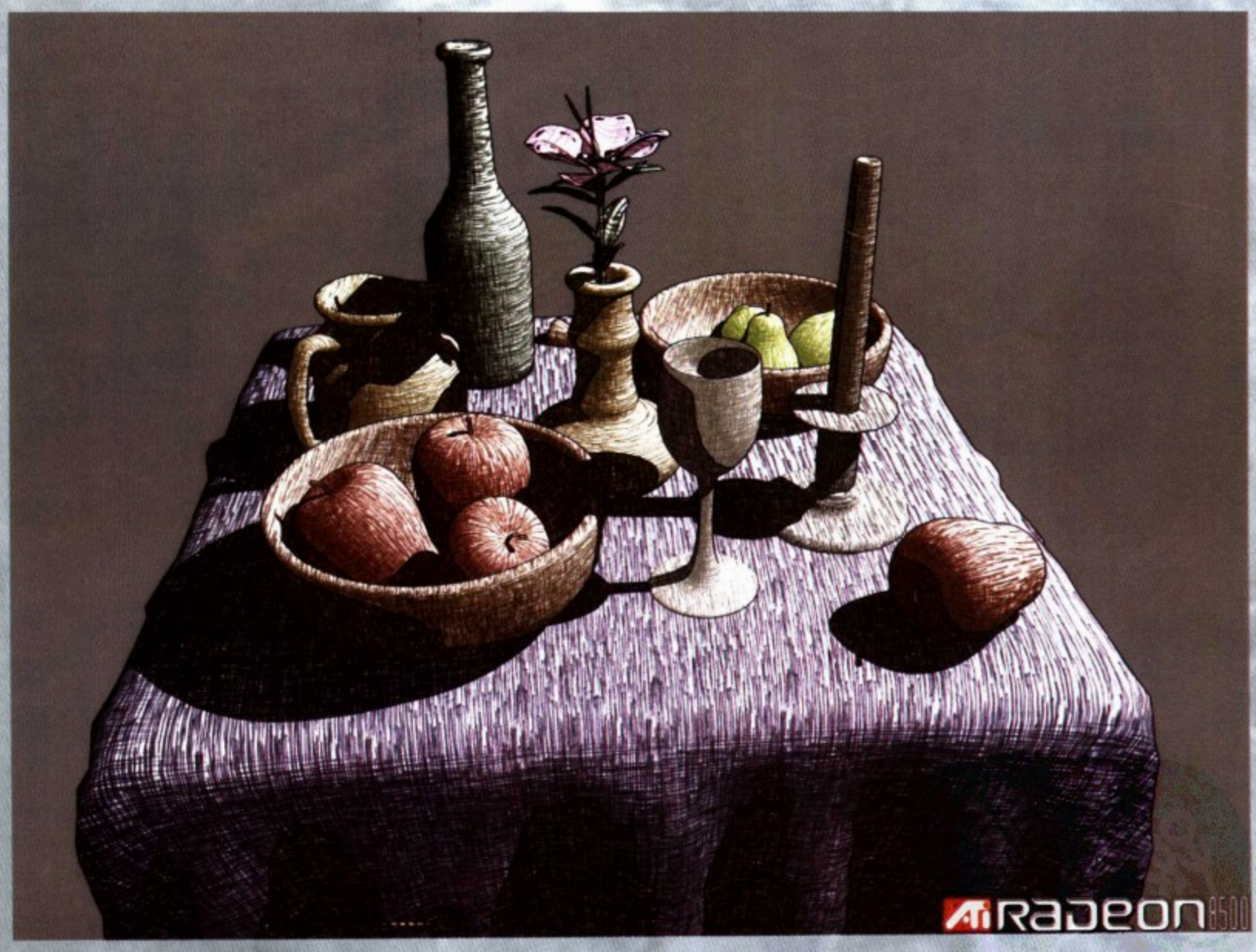
彩图4 （A）1/8高度的分块地形，光照公式中使用了顶点法线对地形的细节进行了高亮处理；（B）以全高度渲染的分块地形，随着高程场的增强，法线开始分散，光照效果更加明显；（C）同样的地形，其中每个顶点都计算出了法线，法线是以绿底红顶的线段来表示的

Download at www.Pin5i.Com



A　　B

彩图5　本图显示的是三角形条带：（A）为鲨鱼的；（B）为龙的。每种颜色都代表了一个新的条带。鲨鱼模型的条带上平均顶点数目为20；龙模型则有243个条带，平均顶点数目为27个

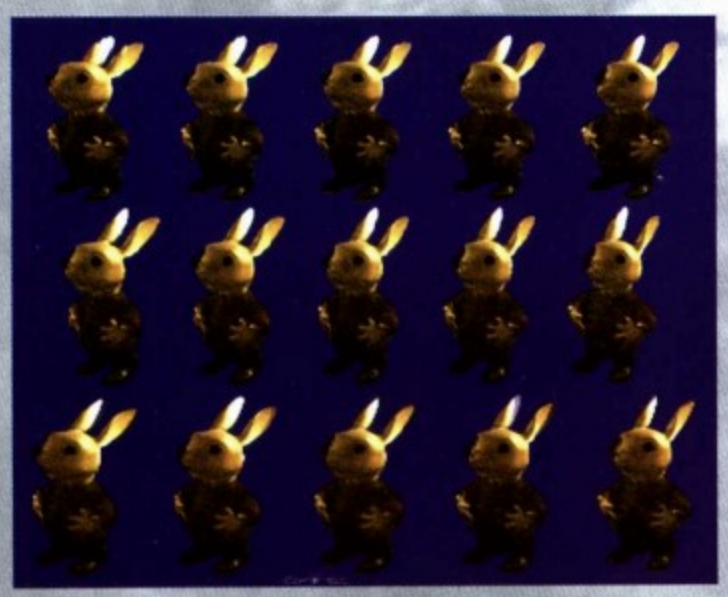


彩图6　这个实时截取的演示图形中有一个静态光源，其中有预先计算好的阴影体。其轮廓是由一个像素着色器计算出来的，使用的是边缘检测法（edge-detection）

Download at www.Pin5i.Com

A B C

彩图7 带有指定权重输出的网格（A）用来自动生成新的权重值（B），然后在关节处使用更小的骨节进行增强（C）。几乎完全消除了基本蒙皮算法中缩小与伸展的缺陷

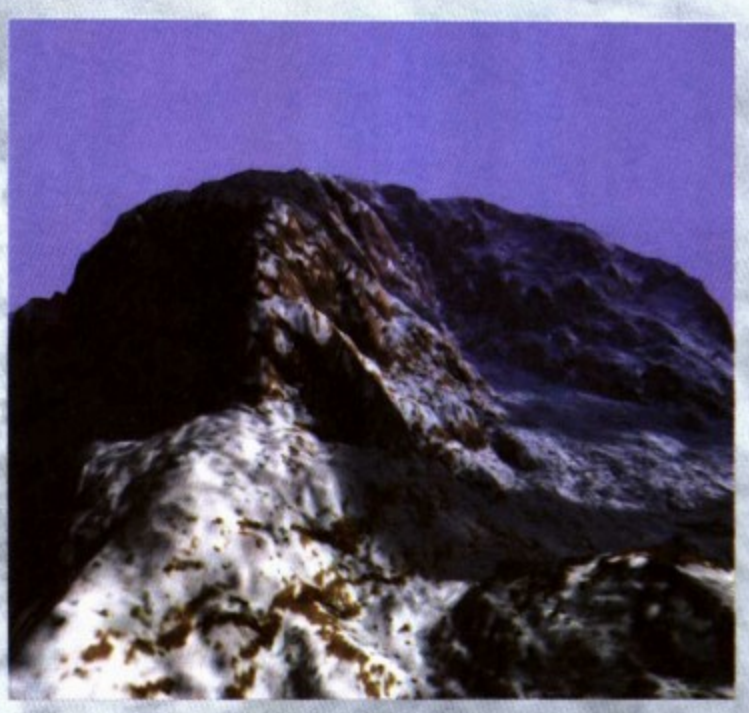


彩图8 由2D图像源生成伪3D图像的3种方法：双图像高程地图法（第1行）；经程序自动图像变换法（第2行）；穿插纹理法（第3行）

彩图9 对头部模型高分辨率的网格仿真，使用了经Max4插件生成的法向地图（参见Oscar Blasco的《使用法向地图进行曲面模拟》一文）。这些模型是在对象空间中使用法向地图，经软Phong照明渲染出来的

Download at www.Pin5i.Com

A



B

C

D

彩图 10 （A～D）具有照片效果的实时变换光照下的地形光照

Download at www.Pin5i.Com

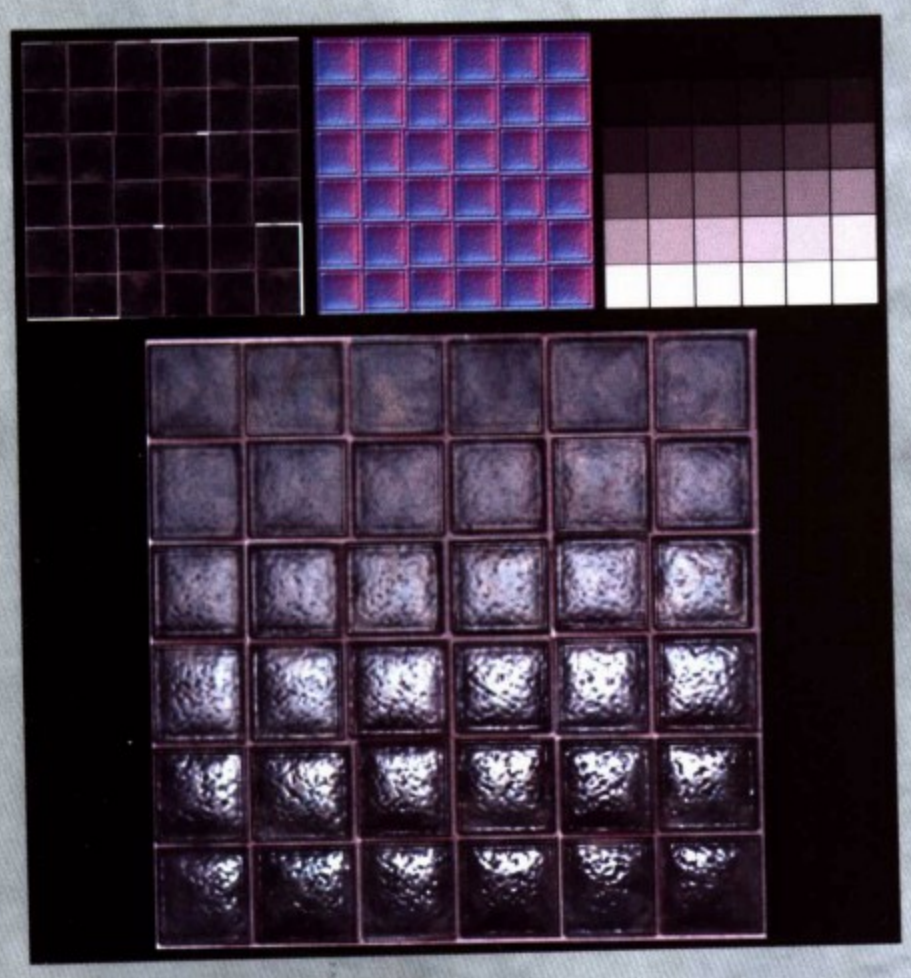
彩图 11　使用一个立体图得到具有阴影的云层运动

A　　B

彩图 12　由程序自动生成纹理的两个例子：（A）一个茶壶上的3D噪声纹理；（B）加上 alpha 渲染的3D噪声纹理

Download at www.Pin5i.Com

彩图 13　NVIDIA 的 *Wolfman* 中展示了实时的具有立体感的皮毛，它是基于一个全运动人物模型渲染的。此处的皮毛使用了"壳鳍"（shells and fins）技术，为身体上的皮毛使用了 8 个同心壳，同时还用鳍几何模型改善轮廓，每帧中有 100 000 个三角形。程序使用了顶点着色器，通过矩阵调色板蒙皮让人物产生运动，共有 61 个骨节，每个顶点有 4 个骨节。程序使用了像素着色器通过一个逐像素的非等测光照模型对皮毛采取光照，每个面都通过阴影地图进行了全面的阴影处理



彩图 14　基础图（左上角）、一个凸凹贴图（上中）与一个镜面指数图（右上角）混合得到下面的图像，它具有不同的镜面指数

Download at www.Pin5i.Com

彩图 15　脑部（左图）与牛（右图）上的光晕纹理着色器

彩图 16　一块布上不同的法线密度函数产生的效果

Download at www.Pin5i.Com

内容提要

本书汇集了近 70 篇最新的游戏编程大师的技术文章。这些文章都来自于实际经验的积累，各有独到之处，依其所属领域不同，全书划分为通用编程技术、数学技巧、人工智能、图形、网络和多玩家游戏、音频处理六章，覆盖了当今游戏开发中的所有关键技术领域。

本书适合游戏开发专业人员阅读。对于入门级的读者，本书指出了您将要面临的各方面挑战，并提供大量的参考资料和资源助您提高专业知识和技术；对于专家级的读者，本书中实用的新思想与新技巧将帮助您节省大量游戏开发的宝贵时间。

Download at www.Pin5i.Com

光盘内容介绍

本书配套光盘包含可移植的C & C++代码（大多数适用于所有平台，少数需要在 Windows/DirectX 平台上）、本书部分技术示例、DirectX 8.1 SDK、OpenGL 工具包（GLUT）、GLSetup 最新 1.0.0.121 完全版，以及本书彩图的高分辨率版本。

LIMITED WARRANTY AND DISCLAIMER OF LIABLITY

THE CD WHICH ACCOMPANIES THE BOOK MAY BE USED ON A SINGLE PC ONLY. THE LICENSE DOES NOT PERMIT THE USE ON A NETWORK (OF ANY KIND). YOU FURTHER AGREE THAT THIS LICENSE GRANTS PERMISSION TO USE THE PRODUCTS CONTAINED HEREIN, BUT DOES NOT GIVE YOU RIGHT OF OWNERSHIP TO ANY OF THE CONTENT OR PRODUCT CONTAINED ON THIS CD. USE OF THIRD PARTY SOFTWARE CONTAINED ON THIS CD IS LIMITED TO AND SUBJECT TO LICENSING TERMS FOR THE RESPECTIVE PRODUCTS.

CHARLES RIVER MEDIA, INC. ("CRM") AND/OR ANYONE WHO HAS BEEN INVOLVED IN THE WRITING, CREATION OR PRODUCTION OF THE ACCOMPANYING CODE ("THE SOFTWARE") OR THE THIRD PARTY PRODUCTS CONTAINED ON THE CD OR TEXTUAL MATERIAL IN THE BOOK, CANNOT AND DO NOT WARRANT THE PERFORMANCE OR RESULTS THAT MAY BE OBTAINED BY USING THE SOFTARE OR CONTENTS OF THE BOOK. THE AUTHOR AND PUBLISHER HAVE USED THEIR BEST EFFORTS TO ENSURE THE ACCURACY AND FUNCTIONALITY OF THE TEXTUAL MATERIAL AND PROGRAMS CONTAINED HEREIN; WE HOWEVER, MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, REGARDING THE PERFORMANCE OF THESE PROGRAMS OR CONTENTS. THE SOFTWARE IS SOLD "AS IS" WITHOUT WARRANTY (EXCEPT FOR DEFECTIVE MATERIALS USED IN MANUFACTURING THE DISK OR DUE TO FAULTY WORKMANSHIP).

THE AUTHOR, THE PUBLISHER, DEVELOPERS OF THIRD PARTY SOFTWARE, AND ANYONE INVOLVED IN THE PRODUCTION AND MANUFACTURING OF THIS WORK SHALL NOT BE LIABLE FOR DAMAGES OF ANY KIND ARISING OUT OF THE USE OF (OR THE INABILITY TO USE) THE PROGRAMS, SOURCE CODE, OR TEXTUAL MATERIAL CONTAINED IN THIS PUBLICATION. THIS INCLUDES, BUT IS NOT LIMITED TO, LOSS OF REVENUE OR PROFIT, OR OTHER INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THE PRODUCT.

THE SOLE REMEDY IN THE EVENT OF A CLAIM OF ANY KIND IS EXPRESSLY LIMITED TO REPLACEMENT OF THE BOOK AND/OR CD-ROM, AND ONLY AT THE DISCRETION OF CRM.

THE USE OF "IMPLIED WARRANTY" AND CERTAIN "EXCLUSIONS" VARY FROM STATE TO STATE, AND MAY NOT APPLY TO THE PURCHASER OF THIS PRODUCT.

Download at www.Pin5i.Com

序

Mark DeLoura
madsax@satori.org

各位读者，大家好，欢迎阅读本书——《游戏编程精粹 3》！感谢你选择了这本游戏开发领域中最新的一部优秀作品。在整套系列丛书中，我们一直坚信，如果能群策群力，集思广益，那么我们的读者就可以学习到更丰富的知识，把握住更先进的信息。在此，我十分荣幸能为大家介绍本书的编辑，Dante Treglia。他是任天堂（Nintendo）公司的主力软件工程师，负责为游戏开发人员提供支持。Dante 为此书倾注心血，在材料的研究与组织上精益求精，其成果读者马上就可以在本书中看到。Dante 还邀请了一批领域专家参与这项工作，他们作为本书各章的编辑与顾问，为 Dante 提供了大力支持——若是想对他们以及本书的智慧结晶有更多的了解，请参阅 Dante 的前言。

展望未来

“游戏编程精粹”（*Game Programming Gems*）系列丛书的目标是鼓励游戏开发人员将自己的知识拿出来，与大家共同分享。公开的标准与代码构成了一个坚实的基础，使得游戏开发的技术专家们能够分享自己的经验，改进各自的工作。但是随着业界的发展，风险也随之上升，因此人们倾向于保护自己的专利，有意隐藏自己的接口。正值本书出版之际，GameCube、XBox 与 PlayStation2 的王朝已到来。有谁能预测出下一代游戏机到底会有多么复杂呢？若是只能管中窥豹，无法知道其具体的工作机制，在这些平台上开发游戏会带来多大的风险呢？又应该如何根据不同平台的异同之处进行优化呢？随着实时系统的日益复杂化，信息共享、源码开放的呼声也在日益高涨。

我们身处的行业蒸蒸日上，这无疑让大家欢欣鼓舞。我们可以自豪地将营收状况公之于众，甚至可以与好莱坞的盈利相匹敌。但是我们未来的方向何在？未来给我们准备的最终礼物又是什么呢？当前，整个业界的走向由两大趋势组成：一个是主流平台正在向网络娱乐设备的方向靠拢，这与现在简单的单机游戏形成了鲜明对比；另一个则是游戏老玩家群的形成，对其而言，游戏已经成为了生命中不可分割的一部分，而且随着年龄的增大，他们的品位也逐渐成熟。这个玩家群要求更成熟的情节，虽然他们仍

Download at www.Pin5i.Com

有可能喜欢简单的打斗游戏、射击游戏或是平台游戏，但是相对来说，复杂的故事情节，例如杀出重围（*Deus Ex*），与宏大的场景，例如模拟人生（*The Sims*）以及侠盗猎车 III（*Grand Theft Auto 3*），则受到了越来越多的关注和喜爱。

最初，人们只能到音乐会与剧院里才能欣赏到音乐与电影。随着技术的发展，这些娱乐活动逐渐能为每个人唾手可得了，它们通过收音机与电视，伴随着广告进入了家庭。在今天，我们已经对随手可得的免费音乐和电影习以为常了，而在未来，游戏也会以同样的方式进入家庭。

使之成为可能的技术直到最近才浮现出来。前面提及的三个主流游戏平台都正在推出自己的网络经营策略。在这场角逐中，宽带接入是一个重要的限制条件。真正的成功不会在一夜之间自动降临，甚至有可能不会在这一轮的三大平台的竞争中实现。但是最终，技术的发展将使之梦想成真，随着宽带接入的普及，游戏将尾随音乐与电影进入家庭，当然了，随之而来的还会有一大堆广告。

技术的走向

但是现在的游戏又将走向何方？游戏引擎与平台遵从甚至超越了摩尔定律，而且这种疯狂增长的复杂性正在逐渐成为业界的负担。它将增加游戏开发的周期，极大地增加开发时间与开发成本。其影响是很明显的：某些发行商每年都会将自己的产品改头换面一番，不管三七二十一，强行将之发布给玩家（其口头禅就是“来年会更好！”）；越来越多的游戏已经开始使用中间件，以缩短推向市场的时间；开发队伍越来越臃肿，因而，软件工程也正在悄悄地进入到游戏的领域中来；发行商买断开发人员，以用有限的成本完全占有作品；一流游戏与二流游戏之间的品质也可说是天壤之别。整个说来，冒风险是越来越难了。这种情况带来的结果就是，游戏要么是非常有趣而独特的，要么就是剽窃而来的，半生不熟。

这将给我们的游戏开发带来什么影响呢？从短期看来，上述所有趋势无疑还在继续。但是接入家庭的在线宽带游戏将极大地改变我们的开发内容与开发方法。随着宽带的到来，你可能会发现自己的技术被锁定在某个特定的平台或是特定的 ISP 上。在此时，我们需要快速推出新的标准，以确保不会造成平台供应商垄断的局面，限制发展的机会。宽带空间应该与当今的因特网一样开放，而平台供应商也应大力支持自己平台的开发社区，保持平台的开放，鼓励开发人员创造出更优秀的游戏来。

当你真正在为大众市场制作游戏的时候，你将不仅仅注重技术，而会更加看重故事情节。此外，你还应该让玩家参与到游戏的开发过程中，这么一来，游戏就会更有吸引力。你还可以对外发布自己的工具，让玩家为游戏制作自己的内容，这样玩家就会更加喜欢这个游戏，同时也促进了下一代开发人员的成长。在每个人心灵的深处都有一个梦幻世界，每个人都梦想着将它亲手创造出来，在梦的天空翱翔，与大家分享他们梦中的世界。从这个角度上来看，身处游戏业界中的我们无疑就是少数的幸运儿了，因为我们有机会能实现自己的梦想——若是你让玩家也同时拥有这个机会，那将是多么美好的事情啊。

Download at www.Pin5i.Com

读者的责任

读者最重要的责任就是要将自己的经验贡献出来，与所有的职业开发人员以及那些正在成长的开发人员共同分享。我们已经有幸拥有了很多高水平的业界会议、Internet 论坛以及邮件列表。只要稍作思考，我们就会明白——若说知识就是力量，那么不管什么人，如果在你创造一流游戏的时候，他却对你加以限制，不让你获得必需的信息，那么，他也无异于是在限制你的发展，阻碍你创造未来的梦想。一定要要求那些游戏平台供应商公开源码，一定要确保未来尽在自己掌握之中。

虽然有一些商业性的控制因素是很自然的，但是我们一定要保有选择的自由。作为专业人士，我们创造的东西应该让大众喜欢，而不应该局限在一个自认是圈内人的小圈子里面。你创作的源泉是自己的思想。一旦你必须要按照上头的指示办事，要听从发行商的命令开发，要服从游戏平台供应商的指挥，或是受到广告商的限制，那你创作出来的东西还能真正算是自己的东西么？它或许只是一个无味的市场产品，抑或是一个按部就班、行规蹈矩的开发经历？

要在游戏开发社区中保持活跃，也要付出努力在业界拼搏。我们的游戏业界前程远大，其未来一片光明。一定要随时做好准备，积极主动，最终，我们将使自己的梦想成真。

尾声

看到“游戏编程精粹”系列的成长过程，真是令人备感欣慰。你或许已经发现，在最初《游戏编程精粹 1》一书中的几位作者已经重返此书了。同时书中的某些作者也即将推出自己的专著。若是读者对此系列丛书中的文章尚感满意，我鼓励你继续去查看他们的著作，因为他们正是本系列丛书的来源。

但是也请不要忘记了，贡献出自己的经验，与大家一同分享。

Download at www.Pin5i.Com

前　言

Dante Treglia
Nintendo of America, Inc.
treglia@yahoo.com

欢迎各位来到《游戏编程精粹 3》的世界！我们已经出版了一整套系列丛书，这套丛书面向高级游戏开发人员，在许多游戏开发技术上提供了宝贵而有深度，同时又是实际可用的经验之谈。此书正是本系列丛书中的第 3 卷。我们向大家介绍资深游戏开发人员的知识，其目的就是为了鼓励游戏开发界的同仁们共同努力，以达到最终目标，创造出更优秀的作品。我们希望读者通过学习书中的技巧，能节省自己的时间，同时也能深受鼓舞，投身于此项事业，创造出更新颖、更有趣的游戏来。

如果你是一个初学者，本书为你提供了一套覆盖面极广的资料，通过它们，你可以得知当今游戏开发中的进展与面临的挑战。我们同时还提供了各种参考文献，以帮助读者得到相应的资料，对提及的话题有更深的了解。如果你是一个富有经验的游戏开发人员，本书的真正价值就在于它能帮助你节省很多时间，少走许多弯路。本书的各位作者都花费了数月的时间来组织他们的思想、代码与相应的说明，以在相应的问题上辅以具体的材料，提出自己的真知灼见。因此即使你不是相关问题方面的专家，也可以很容易地吸取他们的智慧。这样，你就可以省下更多的时间，着手为自己的游戏添加更眩目的特性了。

随着前两卷的出版，本系列丛书已经在游戏业界中得到了认同，这从我们收到的 215 篇征稿上面就可以看出来，作者们都希望与同仁一同分享自己的经验。而这些稿件的质量与数量也同样令人感到惊喜，因此要在其中做出抉择真是一项非常困难的任务。为了保证本书的质量，我们邀请了各个方面的专家来参与评审，以保证这些文章真正是读者，即职业游戏开发人员所需要的。《游戏编程精粹 3》各章的编辑分别是：

- 通用编程技术——Kim Pallister，来自 Intel 公司；
- 数学技巧——John Byrd，来自 Electronic Arts；
- 人工智能——Steve Woodcock，来自 Wyrd Wyrk；
- 图形——Jeff Lander，来自 Darwin 3D, LLC.；
- 网络和多玩家游戏——Andrew Kirmse，来自 LucasArts Entertainment Company；
- 音频处理——Scott Patterson，来自 Next Generation Entertainment。

Download at www.Pin5i.Com

我们挑选文章的标准是要有长远的保存价值、能与实际紧密结合、视角独特，并且生动有趣。我们主要关注的是那些能给游戏开发人员带来长远效益的问题。本书很多文章所提供的宝贵内容都是读者可以立即采用，得到立竿见影的效果的；另一些文章则较为复杂，牵涉甚广，它们提供了必要的细节与知识，以帮助读者将之融入真实的游戏开发过程之中。

章节说明

带有网络互联、支持多玩家特性的 PC 游戏市面上已经有很多了——随手拈来，就有无尽的任务（*Everquest*）、阿斯龙的召唤（*Asheron's Call*）、网络创世纪（*Ultima Online*）、反恐精英（*Counter-Strike*）以及虚幻锦标赛（*Unreal Tournament*）。正当本书出版之时，三大主流游戏机制造商（Sony、Nintendo 与 Microsoft）都正在忙于向大众市场推出自己的联网游戏，紧随 Sega 公司具有联网功能的 Dreamcast。与此同时，随着手机技术的发展与市场的普及，无线游戏的市场也正在涌现。其相关技术上至基于模板的对象序列化，下至网络监控与仿真器，甚至还有无线游戏。

前些天，当我正在与一个朋友看电视的时候，看到一个商业广告，在里面有一只可以与人对话的狗。随后，那位朋友对我说："嘿！这个动画制作得还真够酷的。"就在那时，我正半梦半醒的时候，我才意识到自己刚才真是没有注意到，原来说话的竟是一只狗呢。无疑，我当时已经完全沉浸到了动画与图形的世界中去。我真地梦想能有那么一天，我们创造出来的游戏能达到这种如梦如幻的效果，能让自己沉醉其中。随着图形处理器的更新换代，我们离这个目标越来越近。而在这些使得梦想成真的技术中，就包括了顶点着色器与像素着色器。在本书的图形一章中，我们精选了很多文章来帮助读者营造出一个更真实的环境，创造出更真实的人物。其话题包含了：高层的可编程顶点着色器，它能帮你充分挖掘现代图形芯片的能力；法线分布函数（NDF）以帮你创造有趣的表面效果；另外还有使用纹理作为查找函数以基于每个像素轻松实现高级的图形算法。

网络与图形是游戏编程中非常专业化的两个领域。然而，为了使用它们，我们还需要创建游戏的框架。下面就让我们进入通用编程技术一章吧！在这一章中，我们探讨了方方面面的话题，从游戏设计管理的技巧一直到高效的调试技术。在这一章中，每个游戏开发人员都可以获益极多。

每个算法的背后都隐藏着数学的身影。在教学技巧一章中，我们囊括了一组极佳的数学文章。你可以在其中找到一些通用的数学方法，例如快速 2 基对数与三角函数的处理；以及某些更具专业性的问题，例如有约束条件的逆向动力学与摩擦仿真处理。最妙的一点就是，这些作者已经为你把问题后面所有数学背景相关的问题都给解决了。

每个游戏的背后都隐藏着人工智能（AI）的身影。人工智能一章中覆盖了众多的话题——有如何改善寻径算法的技巧，也有探讨寻径算法与碰撞模型之间关系的文章。读者可以轻易地使用这里谈到的技术来创建一个更聪明的游戏。

视觉效果与游戏智能只是游戏的一部分，耳朵在我们的感官中也占有重要的一席之地。你曾经听过真实的枪响么？亲自听一下比在电影里听到的效果要真实得多。风帽的爆炸声足以使你的身体自动进入警觉状态。为什么？因为我们的耳朵是一个非常精细的器官，它可以提醒我们是否遇到了危险、危险的方向以及危险的距离。因此，作为一个音频开发人员，我

Download at www.Pin5i.Com

鼓励你去在自己的游戏中创造出吸引玩家的音响效果。相应的音频处理一章中谈到了相关的话题，能帮助你达到这个目的，帮你创建 3D 音响效果，帮你使用随机算法生成极其真实的音响效果。

结论

能为读者献上这本《游戏编程精粹 3》，我确实感到莫大的荣幸。我希望读者们能从此书谈及的话题中受益，同时也能通过类似的方法与业界的同仁分享自己的经验。我们身处的业界发展一日千里，动态万千，规模达数 10 亿美元。让我们一起努力，继续共同推进它的发展吧（不是要我们忙于玩游戏啊）！

我们都能记住自己经验中的每个心得。我希望在此处能为读者献上一个小小的程序[1]，它也是我的一个小小发现：

```
main(k){float i,j,r,x,y=-16;while(puts(""),y++<15)for(x
=0;x++<84;putchar(" .:-;!/>)|&IH%*#"[k&15]))for(i=k=r=0;
j=r*r-i*i-2+x/25,i=2*r*i+y/10,j*j+i*i<11&&k++<111;r=j);}
```

—ken Perlin,http://mrl.nyu.edu/~perlin/

译者注[1]此程序是一个绘制 Mandelbrot 图的程序，这是分形（fractal）艺术的一部分。

Download at www.Pin5i.Com

致　　谢

首先，我要向为本书撰写文章的67位作者表示感谢。他们都愿意将自己学到的知识与得来不易的经验与大家分享，这种精神不仅是本书得以产生的原因，也是整个系列丛书产生的第一驱动力。本书中的很多作者都是再度，甚至是三度出手了。在此，谨向大家表示感谢。特别需要提到的就是各章的编辑们，他们的工作本来就已经相当繁重，能再为本书抽出时间进行审核，这也令我感到由衷的感激。

我在此特别感谢 Mark DeLoura，是他将我送出作为此书的编辑，为我提供了大量的帮助，并在整个过程中鼓励我前进（甚至在最后的两个周末飞到西雅图来帮助我编辑这些文章）。出版者 Jenifer Niles 也与我合作愉快。我十分感谢她的指导与及时的反馈。我同时还要向 Jessica Leppaluoto 致以特别的谢意，她帮助我组织、准备并编辑了此书。如果没有她的支持，我肯定是完成不了这项任务的。

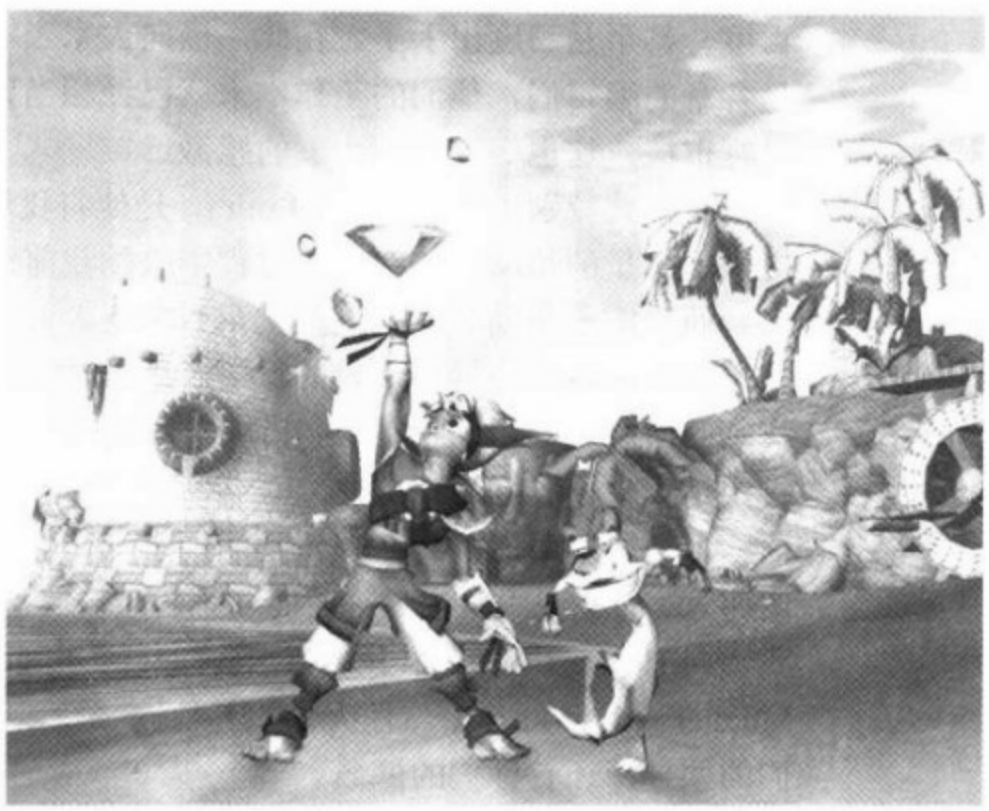
另外要感谢的是我在任天堂的同事们，他们支持了我，也鼓励了我。谢谢大家，在我睡眼惺忪、猛喝咖啡的时候容忍我的一切所作所为。在此，我谨向 Howard Cheng、Ramin Ravanpey、Eugene Kwon、Tian Lim、Carl Mueller、Jim Merrick 与任天堂的其他同仁致以谢意，感谢你们的友谊、你们的领导，还有你们的慷慨。

最后，我要感谢我的家人给我的爱与支持。妈妈，谢谢您！

Download at www.Pin5i.Com

关于封面图案

Bob Rafei
Naughty Dog



本书的封面图案是从游戏《杰克和达斯特：旧世界的遗产》（*Jak and Daxter:The Precursor Legacy*）中截取出来的，这是一个由 Sony Cumputer Entertainment 公司发行，由 Naughty Dog 公司开发的角色动作游戏，其目标平台是 playStation 2，其发行日期是 2001 年 12 月。游戏中的几何模型使用 Maya 创建、组合而成，其中包含了不同层次的分辨率。水面使用了动画几何模型、动画纹理以及环境地图反射等几项技术。这是从一个获胜后的场景中截取出来的 Jak 和 Daxter 庆祝的场面，它是由人物获得了 7 个能量球触发的一个事件（在这里用宝珠代替了），它们是藏在 Sentinel Beach 中的。*Jak and Daxter* 的游戏模型使用了 IK 结构、顶点成形工具（为了生成面部造型）以及可运动的 uv 映射瞳孔，这样就可以得到完全的面部表情和漂亮的舞步了，正如图中显示的一样。我们使用了一个微粒系统（particle system）以用来创建一组丰富的视觉增强效果——从踢起的灰尘颗粒到火球、蝴蝶以及图中显示的条纹状的光照效果。

Download at www.Pin5i.Com

作者简介

Thor Alexander

thor@hardcodedgames.com

Thor Alexander 在过去的 10 年中致力于为游戏界创造更为可信的智能人物角色。最近，他创建了 Hard Coded Games 公司（位于德克萨斯州奥斯汀），为在线游戏创造具有艺术效果的人工智能和机器学习。在此之前，他在 Electronic Arts、微软还有 Xatrix Entertainment 担任过资深人工智能开发和设计人员，同时他还是 Asgard Interactive 的创始人之一，并且是 Harbinger Technologies 的 CEO。他是 hyperSim 智能人物系统和 GoCap（Game Observation Capture）的发明者，后者是一个机器学习过程，它可以通过观察真人玩游戏的过程来训练人工智能的人物。

Jason Beardsley

jbeardsley@ncaustin.com

Jason 自从 1996 年以来就在为在线多玩家的游戏编写网络和服务器程序。他在麻省理工学院（MIT）和 Binghamton 大学获得了计算机科学的学位。现在，Jason 在 NCsoft/Austin 工作，致力于下一代在线游戏。

Oscar Blasco

oscar@asidesoft.com

Oscar 从孩童时代起就开始编程了，并一直坚持自学，提高自己的技能。他在 Crytek Studios 工作，致力于角色开发，同时担当研发程序员。现在他主要关注的是完成学业和其在 Aside 软件公司中新的 Titan2 引擎上的开发工作。

Ben Board

ben_boaid@yahoo.com

Ben 走上编程这条路主要得归功于其父亲，一个喜欢 ZX81、BBC Model B 与 Amstrad 8086 个人电脑的人。随着乳牙而去的是 Ben 的 BASIC

Download at www.Pin5i.Com

编程经历，从 12 岁起，他就开始学习 C，直到他在剑桥大学学到了 OOP。经过一次与现实世界不那么愉快的接触以后，Ben 找到了自己的人生目标，他在 1997 年加入了 Bullfrog Production 的 Theme 小组，编写了 *Theme Park World* 的人工智能代码。在 2000 年后半年，他和别人一起组建了 Dogfish Entertainment 公司，担任了公司处女作的主力开发员。但是命运受挫，2002 年初公司倒闭了。不过失败是成功之母，他现在很自豪能为 Godalming 最好的游戏 *Big Blue Box* 出力，完成 *Project Ego* 的后期制作，他认为这个游戏甚至比 3D 怪兽迷宫还要好。

Martin Brownlow

mbrownlow@shiny.com

Martin10 岁时就在朋友的 ZX81 上开始编程。之后， Martin 开始在英国的 Virtuality 公司工作，编写虚拟现实街机的游戏。三年之后，他搬到了美国，开始为 Shiny Entertainment 公司工作。在那里，他经手开发了好几个游戏，包括 *MDK* 与 *Sacrifice*，现在他正在忙于 *Matrix* 视频游戏。

Phil Burk

philburk@softsynth.com

Phil Burk 既是一个计算机编程人员，也是一个设计人员，他精通的领域是人机交互与试验型的音乐系统。刚开始的时候，他制作的是家造模拟合成器与计算机，主要关注于吉他的处理。1981 年，在 Mills 大学的现代音乐中心，他开始与 Larry Polansky 以及 David Rosenboom 合作。他们一同开发了 HMSL（Hierarchical Music Specification Language，结构化音乐规范语言），全世界的作曲家都通过它来进行实时计算机音乐的开发。HMSL 是一个基于 Forth 的面向对象的合成语言，支持 MIDI 与基于 DSP 的合成。后来 Phil 进入了 3DO 公司，为一个视频游戏平台开发了第一个基于 DSP 的声音合成器。在 CagEnt，3DO 的一个子公司，他设计了一个定制的、RISC 风格的 DSP（数字信号处理器），专门用来处理基于单元生成器的合成与音效。现在，他正在开发 JSyn，这是一套针对 Java 的合成 API，它可以让作曲家将交互的电脑音乐片断添加到网页上去。Phil 曾在很多项目中当过咨询顾问，包括为数字电视 ASIC 设计音频模块以及使用 Java 开发 MIDI 工具和与电话相关的应用程序。

John Byrd

jbyrd@well.com

John Byrd 现在在 Electronic Arts 公司工作，在 *Freekstyle* 项目中担任资深项目经理。在此之前，他在 Sega 公司担任开发人员的技术总监，在那里，他创建了 Sega Dreamcast (Katana) 的 SDK，支持了数以百计的游戏开发项目。他也在 3DO 担任过资深工程师的职位。John 是 Bosslevel（一个网上游戏业界论坛）的创始人。John 在哈佛得到了计算机科学学位，并能弹得一手好的班卓琴。

Download at www.Pin5i.Com

Drew Card

dcard@ati.com

Drew Card 在 ATI 工作，是 3D 应用研究组的软件工程师，他的主攻方向是基于着色器的渲染技术的应用。他参与了 SDK 的开发，同时在演示引擎上也做出过贡献。Drew 是南卡罗来纳大学的研究生。

Christopher Christensen

cchristensen@naughtydog.com

1982 年，Christopher 在一台 Apple II+机器上开始了自己的编程生涯。1994 年，在获得了计算机工程硕士学位之后，他找到了一份工作，在 Interplay Productions 公司开发 PC 机上的角色扮演游戏。从那以后，他就一头扎进了游戏业界。今天，他在加利福尼亚州圣莫尼卡市的 Naughty Dog 公司担任程序员。

Wagner Corrêa

wtcorrea@research.att.com

Wagner Corrêa 是普林斯顿大学计算机科学系的博士。他在计算机图形方面已研究了近 10 年，其硕士论文是关于面向对象的游戏引擎。Wagner 已经发表了好些学术论文，其中有两篇被近期的 SIGGRAPH 收录。他是普林斯顿大学里图形与几何模型研究组的一名成员，现在正在开发一套真实世界环境的游览系统。

Mark DeLoura

madsax@satori.org

Mark 是“游戏编程精粹”系列从书的创始人。他现在任职于索尼（美国）计算机娱乐公司，职位是开发人员协调经理，正在努力引导开发人员达到 SIMD 的理想境界。他也曾在任天堂（美国）公司当过主力软件工程师，“耕耘”代码，亦有过在 *Game Developer* 杂志担任主编，剪裁文章的经历。Mark 现在仍然是虚拟现实的“铁杆球迷”，他仍将实时在线，在多玩家第一人称视角游戏里面与你对战。

Thomas Demachy

thomas@abunth.com

当 Thomas 还是一个小孩的时候，他母亲就在他的枕头下面发现过程序清单。学习了几年科学与高性能计算之后，他离开了阳光明媚的法国里维埃拉地区，开始在巴黎附近的 Titus Interactive 工作室担任程序员。最近，他参与了基于 PlayStation 2 开发的 *Robocop* 的项目。除

Download at www.Pin5i.com

了视频游戏以外，他主要的兴趣大都与大海有关，例如航行或者是在世界的各个岛屿之间旅游。

Mike Ducker

mike@ducker.org.uk

Mike 16 岁才开始编程，开始一直用 Turbo Pascal，直到在埃塞克斯大学的人工智能学位攻读期间发现了 C++。毕业以后，正在他茫然不知该选择什么职业的时候，天上掉下了一个工作馅饼，他在 1997 年的晚期在 Anco Software 公司找到了第一份游戏开发的工作。做了 3 年图形用户界面、图形与人工智能的开发以后，Mike 重返校园，进入了苏塞克斯大学，攻读人工生命的硕士学位。之后他作为人工智能程序员重返游戏界，主要在 Dogfish Entertainment 公司与 Ben Board 一起工作，直到公司倒闭。在进入 Lionhead 工作室之前，他与 Richard Evans 一起工作，开发着一个迄今尚未公布的游戏。

Eddie Edwards

eddie@tinyted.net

Eddie 在剑桥大学克里斯特学院学的是数学。在 1994 年进入视频游戏的领域给 *Wolfenstein 3D* 做移植之前，他一直在软件工程的领域里工作。从那以后，他加入过好些工作室，参与过好几个游戏的制作，包括（最近的）Naughty Dog 公司的 *Jak and Daxter*。他现在正在英国筹建自己的工作室。

Charles Farris

charlesf@vr1.com

Charles Farris 在 VR1 Entertainment（Boulder，CO）公司担任资深软件工程师。自从 1998 年加入游戏业界以后，Farris 就当过 *Hired Guns* 与 *Nightcaster* 的人工智能指导，现在他正在忙于 *Nightcaster II* 的人工智能系统。他获得过佛罗里达理工学院的海洋工程与应用数学的学士学位，在科罗拉多矿业大学获得了应用数学的硕士学位。

Tom Forsyth

tomf@muckyfoot.com

自从在他的 ZX Spectrum 上看过 *Elite* 之后，Tom 就一直沉迷于 3D 图形，而且总是可以把硬件的最后一滴油水给轧出来。他为 Spectrum、Sinclair QL、Atari ST、Sega 32X、Saturn、Dreamcast 与 PC 都编写过绘制三角形的程序，以至后来他一看到它们就恶心。谢天谢地，现在总算可以让硬件来帮我们画这些玩意了。自从在 3Dlabs 度过了编写 3D 图形卡驱动程序的惬意的两年生涯之后，Tom 转到了 Mucky Foot Productions 公司——这是一个在英国 Guildford 的游戏公司，现在正沉迷于位移地图（displacement maps）与 *Wesley Snipes* 皮肤的 BRDF。

Download at www.Pin5i.Com

David Fox

dfox@citycom.com

David 为 Next Game 公司服务，致力于创建 Web 游戏与无线多玩家游戏。他是好几本畅销书（这些书是与因特网技术有关的）的作者，而且也经常在 Gamelan、Salon.com 与 Developer.com 上面发表文章。过去 3 年中，David 一直在 Sun 微系统公司的 JavaOne 大会上发表关于 Java 游戏编程主题的讲话，在过去两年中，他还曾经获得过 Motorola-Nextel Developer's Challenge 的奖励。

Byon Garrabrant

byon@byon.com

Byon Garrabrant 从 1988 年开始成为职业游戏开发员，他刚开始的工作是将街机的游戏移植到 PC 上面去。Byon 在 Interplay 公司开发了 *Castles*、*Castles II* 与 *Conquest of the New World* 游戏。在 1997 年他加入了 Westwood 工作室，参与领导开发了 *Command & Conquer: Renegade*。

Miguel Gomez

kikomu@seanet.com

Miguel Gomez 是 SAAM 研究所的一个软件工程师，是一个医学研究建模与分析软件的开发人员。他以前的工作经历包括 *PGA Tour '96*（Electronic Arts 公司）的图形与物理编程，在 Activision 公司开发 *Hyperblade*，在微软开发 *Baseball 3D*，以及在 Psygnosis 开发 *Destruction Derby 64*，他在华盛顿大学获得了物理学士学位与应用数学的硕士学位。

Robin Green

robin_green@playstation.sony.com

Robin 在游戏业界工作了 7 年了。起先是在 Electronic Arts（英国）公司，然后进入了 Bullfrog Productions 的研发部门，最后加入了索尼（美国）计算机娱乐公司的研发部门。他为 *Dungeon Keeper 2* 与 *Sim Theme Park* 做出过技术贡献与支持，另外还在 SIGGRAPH 2000 上发表过控制行为（steering behaviors）的教材。同时他也是 C++ Language Police（Const Correctness 分部）的一名全职成员。

Jim Greer

jamesfgreer@mindspring.com

Jim Greer 在 1991 年加入了 Origin Systems 公司，投身于游戏业界，在那里他参加了 *Ultima VII* 与 *Ultima VIII* 的开发。随后，在 1995 年，他在德克萨斯的奥斯汀与人一同创建了一个游戏

Download at www.Pin5i.Com

工作室，为 Activision 公司开发了 *Netstorm: Islands at War* 游戏。*Netstorm* 是一个在线的实时策略游戏，据 C-Net 的游戏评论者说，享有“有史以来无人购买的游戏之最”的美誉。自从那以后，他就开始为 shockwave.com 与 ea.com 制作网上游戏，此外，他还参与了 *Shadow Garden*，这是一个交互式艺术的项目（请参见网站 http://www.mine-control.com，上面有更多的相关信息），同时他还是一个扑克竞赛迷。

Søren Hannibal

sorenhan@yahoo.com

11 岁的时候，Søren 就在一台 Commodore 64 机上开始了自己的编程生涯，在接下来的 8 年中，他把大部分晚上都花在开发演示程序上了，频繁参加了各个演示场景。1993 年，他的爱好最终成为了他的职业生涯，当时他从丹麦搬了出来，加入了游戏业界。自从那时起，他在 Core Design 公司与 Scavenger 公司当过主力程序员，同时也得到了自己的学士学位。现在，他正在兴致勃勃地在 Shiny Entertainment 公司为自己的第二部作品开发人物技术（character technology），这是一个得到 *Matrix* 电影授权的游戏。

Michael Harvey

michael.harvey@intel.com

Mike 是英特尔实验室里面图形与 3D 技术方面的资深软件工程师。现在，他正在研制下一代基于 Web 的 3D 引擎中的仿真技术。

Brian Hawkins

winterdark@sprynet.com

Brian Hawkins 在卡内基·梅隆大学获得了数学学士与计算机科学学士的学位，然后在 Justsystem 匹兹堡研究中心投入计算机图形方面的研究。两年之后，他希望能自主开发一个成品，于是便横跨美国大陆，到了洛杉矶，加入了 Activision 公司，在这里他是 *Star Trek: Armada* 游戏的核心人员与用户界面指导。此外，他还参与过 *Civilization: Call To Power and Call To Power 2* 的开发。Brian 现在是 Seven 工作室 *Defender* 游戏的主力开发人员，此游戏是其经典版的一次重新开发。他的兴趣爱好非常广泛，你可以与他海阔天空地大侃。

Garin Hiebert

ghiebert@creativelabs.com

Garin Hiebert 是创新实验室公司的一名开发人员，在那里他参与了多个项目。他曾经参与过几个游戏中的音频编码开发，是 OpenAL 的一个积极参与者。他一直在开发苹果机上的 OpenAL 代码库，进行跨平台的代码测试，同时也在编写 OpenAL 开发人员指南。

Download at www.Pin5i.Com

Dan Higgins

dan@stainlesssteelstudios.com

身为 *Empire Earth* 中人工智能小组的开发成员之一，Dan 深感自豪，因为这个小组里面有天才的 Bob Scott 与聪明的 Chad Dawson。他的曾为历史频道、艺术与娱乐频道以及传记频道开发高性能的搜索引擎。在此之前，Dan 做过一些 COM 集成架构方面的开发，但是只有在喝了几杯之后，他才会带泪回忆当时的经历。在 *Empire Earth* 中，他负责的是寻径、地形分析、电脑玩家的军队以及动物的人工智能。他是 Frostburg 州立大学（马里兰）的计算机科学研究生毕业，对 C++、游戏、STL 以及优化都有着强烈的兴趣。

Greg Hjelstrom

greg@westwood.com

Greg Hjelstrom 自从 1995 年以来就在 Westwood 工作室工作了，他参与制作的游戏包括 *Command & Conquer Tiberian Sun* 等。他最近的工作是参与领导 *Command & Conquer. Renegade* 游戏开发。

Naty Hoffman

naty@westwood.com

Naty Hoffman 在 Westwood 工作室是一个资深图形开发员，在那里他花了 5 年的时间为 *Earth and Beyond*，这个巨量多玩家空间游戏制作漂亮的空间爆炸（space explosions）。在此之前，他在英特尔做微处理器的工作，在那里他是带有 MMX 指令的奔腾处理器的架构师，而且为 MMX、SSE 与 SSE2 这些扩展指令集做出了很多贡献。他曾在 GDC 上发言，也曾在 *Game Developer* 杂志上面发表过文章。

Don Hopkins

xardox@mindspring.com

Don Hopkins 是一个程序员兼用户界面设计员，他曾经在多个平台上研究和开发过交互式图形系统。Don 曾在马里兰大学并行处理实验室、异质系统实验室、人机交互实验室、图灵研究所、卡内基·梅隆大学计算机科学系、Kaleida 实验室以及 Interval Research 公司，担任过临时研究开发人员与用户界面的开发人员。此外，他还当过商业软件开发人员，实现过 *The Sims* 的人物动画系统与用户界面，将 *SimCity* 移植到了 Unix 平台上，用 ScriptX 为 Kaleida 实验室创建了交互式的电视与 Web 的开发工具包，与 David Levitt 一同为 Levity 和 Interval Research 开发与使用过实时可视化数据流的开发语言。Levitt 与 Hopkins 建立了一家叫做 ConnectedMedia 的新公司，在那里他们正在开发一个称为 ConnectedTV 的消费产品，这是一个个性化的电视娱乐指南，另外还带有为 Palm 与其他设备准备的电视遥控装置。

Download at www.Pin5i.Com

Kenneth Hurley

khurley@nvidia.com

Kenneth 于 1985 年在一家叫做 Dynamix 的公司开始了他的游戏生涯。他也曾经在 Activision、Electronic Arts 与英特尔工作过，现在则在 NVIDIA 公司从事开发人员关系协调的工作。当前他的工作是开发与研究以及指导开发人员如何使用 NVIDIA 的新技术。在游戏业界，他参与开发的游戏有 *Sword of Kadash* (Atari ST)、*Rampage* (PC, Amiga, Apple II)、*Copy II ST*、*Chuck Yeager's Air Combat Simulator* (PC)、*The Immortal* (PC)，以及 *Wing Commander III* (PlayStation)。在 NVIDIA 的这段时间里面，他参与过以下包与演示系统的开发：NVASM (Geforce3 顶点/像素着色器的汇编器)、NVTune (NVIDIA 的性能分析工具包)、DX7 的折射演示、 Minnaert 光照演示、粒子物理的演示、刷过了金属色效果、云层覆盖的演示以及具有照片效果的表面演示。他在马里兰大学获得了计算机科学的学士学位。

Pete Isensee

peteis@xbox.com

Pete 从事开发生涯已经快整整 10 年了。他参与发行的游戏有 CD-ROM 上发布的动作类与冒险类的游戏，也有一般的网上游戏，其工作过的平台有 PC 机、MAC 机与 XBox。他现在是 Xbox 高级技术小组中的一员。他获得过计算机工程的学位，而且远在 C++的模板流行之前，他就是一个模板高手了。

John Isidoro

jisidoro@csa.bu.edu

John Isidoro 是 ATI 里 3D 应用研究组的成员，同时也在波士顿大学攻读计算机科学的博士学位。在 ATI，他参与开发了 Radeon 8500 技术的演示与屏幕保护程序以及很多其他的应用程序。他的研究兴趣是实时图形、着色器开发、基于图像的渲染、多视图重建、非固定化的区域跟踪以及通过新颖的方法来使用图形硬件。他在 ICCV'98、CVPR'00、《游戏编程精粹 2》与 *ShaderX*（Wordware 出版公司）上都发表过文章。

Jan Kautz

kautz@mpi-sb.mpg.de

Jan 正在德国 Saarbrücken 的 Max-Planck-Institut für Informatik 攻读博士学位，其主要研究领域是使用图形硬件实现交互式的真实光照与着色。

Paul Kelly

paul_kelly2000@yahoo.com

Download at www.Pin5i.Com

Kenneth Hurley

khurley@nvidia.com

Kenneth 于 1985 年在一家叫做 Dynamix 的公司开始了他的游戏生涯。他也曾经在 Activision、Electronic Arts 与英特尔工作过，现在则在 NVIDIA 公司从事开发人员关系协调的工作。当前他的工作是开发与研究以及指导开发人员如何使用 NVIDIA 的新技术。在游戏业界，他参与开发的游戏有 *Sword of Kadash* (Atari ST)、*Rampage* (PC, Amiga, Apple II)、*Copy II ST*、*Chuck Yeager's Air Combat Simulator* (PC)、*The Immortal* (PC)，以及 *Wing Commander III* (PlayStation)。在 NVIDIA 的这段时间里面，他参与过以下包与演示系统的开发：NVASM (Geforce3 顶点/像素着色器的汇编器)、NVTune (NVIDIA 的性能分析工具包)、DX7 的折射演示、 Minnaert 光照演示、粒子物理的演示、刷过了金属色效果、云层覆盖的演示以及具有照片效果的表面演示。他在马里兰大学获得了计算机科学的学士学位。

Pete Isensee

peteis@xbox.com

Pete 从事开发生涯已经快整整 10 年了。他参与发行的游戏有 CD-ROM 上发布的动作类与冒险类的游戏，也有一般的网上游戏，其工作过的平台有 PC 机、MAC 机与 XBox。他现在是 Xbox 高级技术小组中的一员。他获得过计算机工程的学位，而且远在 C++的模板流行之前，他就是一个模板高手了。

John Isidoro

jisidoro@csa.bu.edu

John Isidoro 是 ATI 里 3D 应用研究组的成员，同时也在波士顿大学攻读计算机科学的博士学位。在 ATI，他参与开发了 Radeon 8500 技术的演示与屏幕保护程序以及很多其他的应用程序。他的研究兴趣是实时图形、着色器开发、基于图像的渲染、多视图重建、非固定化的区域跟踪以及通过新颖的方法来使用图形硬件。他在 ICCV'98、CVPR'00、《游戏编程精粹 2》与 *ShaderX*（Wordware 出版公司）上都发表过文章。

Jan Kautz

kautz@mpi-sb.mpg.de

Jan 正在德国 Saarbrücken 的 Max-Planck-Institut für Informatik 攻读博士学位，其主要研究领域是使用图形硬件实现交互式的真实光照与着色。

Paul Kelly

paul_kelly2000@yahoo.com

Download at www.Pin5i.Com

Paul Kelly 在中佛罗里达大学获得了计算机科学的硕士学位。他曾经参加过 *NCAA Football '99*、*Die Hard Trilogy 2: Viva Las Vegas* 以及 *Duke Nukem: Land of the Babes* 这些游戏的开发。他在开发方面的兴趣是 3D 图形与人工智能。在不编程序的时候，Paul 比较喜欢踢足球。

Andrew Kirmse

ark@alum.mit.edu

Andrew 是 *Meridian 59* (1996)的指导者与发明者之一，也是 *Star Wars: Starfighter* (2001)的图形开发人员。他在麻省理工学院(MIT)获得了物理、数学与计算机科学的学位。Andrew 现在在 LucasArts Entertainment 公司工作。

James Klosowski

jklosow@us.ibm.com

James Klosowski 是 IBM Thomas J. Watson 研究中心的一名研究员。他的主要研究方向是计算机图形、视觉与计算几何应用。1992 年，James 在 Fairfield 大学获得了计算机科学与数学学士的学位，在 1994 年与 1998 年，他分别获得了纽约州立大学石溪分校的应用数学硕士与博士的学位。他在计算机图形方面的兴趣包括大型数据的交互视觉、碰撞检测、体渲染以及自适应网络图形。最近，他主要在研究可视裁减、复杂几何模型的简化以及分布式数据的并行渲染。

Adam Lake

adam.t.lake@intel.com

Adam Lake 是英特尔架构实验室(Hillsboro, OR)的一名资深软件工程师。在加入英特尔之前，他在 Chapel Hill 的北卡罗莱纳(UNC)大学获得了计算机科学的硕士学位，其方向是计算机图形与虚拟现实。他在使用 Microsoft Visual Studio 进行 C++开发与 GNU C/C++开发上面具有超过 10 年的经验。在进入 UNC-Chapel Hill 学习之前，他在 Los Alamos 国家实验室工作，具体是在应用理论物理与计算科学方法（XCM）组里。Adam 在 Evansville 大学获得了计算机科学的学士学位与数学的第二学位。在那里，他的工作是为物理学家开发计算机辅助设计的应用程序，程序的名字叫“Justine”。你可以通过网址 http://www.cs.unc.edu/~lake/vitae.html 获得一些关于他的信息。

Jeff Lander

jeffl@darwin3D.com

Jeff Lander 是 Darwin 3D 公司的创始者，此公司的目标是将实时 3D 图形推向更广的市场。Jeff 在视频游戏、电视与电影舞台领域当程序员的历史已经长达 10 年了，在这些地方他

Download at www.Pin5i.Com

开发了很多实时图形应用程序。Darwin 3D 有很多游戏与娱乐方面的客户，包括 Activision、MGM Animation、QuantumWorks Corporation，以及 Rhythm 与 Hues 工作室。Jeff 曾在 *Game Developer* 杂志上大量投稿，也曾经在很多业界贸易展览会与会议上面发表过演讲。

William Leeson

Wleeson@indigo.ie

自从 1996 年开始，William 就以自己在全局照明（global illumination）与并行渲染方面的论文让学术界感到震惊了。他在 2001 年从都柏林的三一学院获得了博士学位，在其后的几年里，他一直在研究人物动画。

Eric Lengyel

lengyel@terathon.com

Eric Lengyel 是 Terathon Entertainment 公司（位于加利福尼亚的 Sacramento）的创始人之一，也是该公司的技术总监。他同时还是 *Mathematics for 3D Game Programming & Computer Graphics*（Charles River Media 出版公司，2002）的作者。Eric 在维吉尼亚理工学院获得了自己的数学硕士学位。

Frank Luchs

frank@visiomedia.com

早在 1983 年，Frank Luchs 就已经为 Atari 计算机编写了自己的第一个音乐程序，开始了“自己音乐+开发”的双面生涯。他参与的项目包括为电影与电视制作音乐，也包括为定制的应用程序与多媒体软件进行音响的设计与开发。他曾经创作过数以百计的歌曲、小曲以及电影中的插曲（例如德国电影中著名的犯罪片 *Tatort* 的插曲）。他是 Visiomedia 软件公司的创始人之一，此公司的目标是制作虚拟乐器。在 Visiomedia 的时候，他设计了 Sphinx Modular 媒体系统，这是软件合成器 Saccara、Chephren 与 Cheops 的基础。Sphinx 的一个轻量级版本成为了本书中开放的源码。Frank 现在正在德国慕尼黑的一个电影中心工作。

Carl S. Marshall

Carl.S.Marshall@intel.com

Carl S. Marshall 是英特尔实验室 3D 图形与技术小组的一位资深软件工程师。他在 Clemson 大学获得了计算机科学的硕士学位，在那里他的研究方向是虚拟现实。他曾经在《游戏编程精粹 2》中发表过文章，也曾经在 Shockwave3D 图形引擎的 NPR 与其他部分做过工作。现在，Carl 的研究兴趣正在转向智能寻径与实时的、具有照片效果或非照片效果的 3D 图形算法。

Download at www.Pin5i.com

James McNeill

james_mcneill@ameritech.net

James McNeill 是一位顾问程序员，其专研方向是 3D 图形，现在，他住在伊利诺斯的芝加哥。他曾经在 Sinister Games 这家位于北卡罗来纳州的 Ubisoft 公司工作过，曾经在 Westwood 工作室工作过，在那里他为 PC 游戏 *Blade Runner* 开发了图形的部分。游戏玩家们应该在 Westwood 的一些游戏里面听过他的声音，他曾经为 *Dune 2000* 的 House Atreides 步兵配过音，还在 *Command & Conquer 2* 中也为很多车辆配过音。

Mike Milliger

mikem@2015.com

Mike 最近这两年一直在 *Medal of Honor: Allied Assault* 游戏的诺曼底登陆战中苦苦挣扎，在他眼前晃悠的尽是蓝色的漩涡、拿刀的美工、带着猴子的漫画家——更别说还落下了睡觉流口水的毛病。现在，他可成为了 2015 的主力程序员，正在开发他们的下一个第一人称视角射击类游戏。给他发个 email，和他在游戏里开战吧。

Jack Moffitt

jack@xiph.org

Jack Moffitt 于 1998 年晚期创建了 Icecast 项目，而且在好几个早期因特网广播创业公司充当过重要角色，其中包括 Green Witch Internet Radio 与 iCast，它是一个 CMGi 公司。他被广泛认为是流媒体业界的先驱专家之一。他在软件开发、数据库、负载管理与扩展、流媒体、系统管理、架构以及安全方面的经验堪称丰富。Jack 还是 Xiph.org 组织的执行总监。Xiph.org 是一个非赢利性质的技术与研究组织，其关注的方向是互操作性、无专利限制的多媒体技术与标准，例如 Icecast 流媒体服务器与 Ogg Vorbis 编解码器的项目。

Aaron Nicholls

aaron_feedback@hotmail.com

Aaron Nicholls 是在微软公司华盛顿州雷特蒙德市的一名主力开发员。从年轻时起，他就特别喜欢图形编程、人工智能、物理仿真，再加上他对伟大的游戏的爱好，导致了他最终对游戏开发产生了兴趣。最近几年，他在软件的国际化中应用到了自己的外语技能。Aaron 曾在《游戏编程精粹 2》上发表过文章，他期待着读者的回应与反馈。

Chris Oat

coat@ati.com

Download at www.Pin5i.Com

Chris Oat 是 ATI 中 3D 应用研究组的一名软件工程师，在那里他正在研究针对实时 3D 图形应用程序的最新渲染技术。他当前关注的是针对 PC 游戏的像素着色器与顶点着色器。Chris 是波士顿大学的研究生。

Kim Pallister

kim.pallister@intel.com

Kim Pallister 是英特尔软件与解决方案小组中的一名技术市场经理。他现在主要关注的是实时 3D 图形技术与游戏的开发。

Scott Patterson

scottp@tonebyte.com

Scott Patterson 是《游戏编程精粹 2》的一名作者，现在则是本书，《游戏编程精粹 3》中第 6 章音频处理的编辑，而且也是一名辅助编辑。在过去的 12 年中，他曾经做过游戏开发中的音频系统、图形系统以及逻辑系统，此外还为音频开发与图形开发制作过工具和其具体内容。过去，他曾经在 Naughty Dog、Midway 与 Microprose 工作过。现在，他正在为 Next Generation Entertainment 公司设计与开发一个游戏系统，并在其中担当研发小组长。他在《游戏编程精粹 3》中的工作得到了妻子 Alison、儿子 Nick 与女儿 Grace 给予的耐心、爱与支持。幸运的是，他们也都喜欢计算机带来的乐趣。

Steve Rabin

steve@aiwisdom.com

Steve Rabin 已经在任天堂（美国）公司工作了 10 年，是一个游戏业界的老手了。他曾经为三个发布的游戏开发人工智能的代码，而且是《游戏编程精粹 1》与《游戏编程精粹 2》的作者之一。他是《游戏编程精粹 2》中人工智能一章的编辑，也是 *AI Game Programming Wisdom*（Charles River Media 出版公司，2002）的创始者与主编。Steve 在游戏开发者会议上曾经就人工智能发表过演讲。他从华盛顿大学获得了计算机工程的学位，其专业方向是机器人。

Justin Randall

jrandall@soe.sony.com

Justin Randall 现在是索尼在线娱乐公司（位于德克萨斯州奥斯汀）的一名程序员。他正在开发游戏 *Star Wars Galaxies*，这是一个属于 *Star Wars* 系列的巨量多玩家角色扮演游戏。他曾经在数个已经发布了的游戏中工作过，包括实时策略游戏与第一人称视角的动作游戏。

Download at www.Pin5i.Com

Eric Robert

eric.robert@videotron.ca

Eric 是 Ubi Soft Entertainment 公司核心技术小组中的一名软件工程师，此小组位于加拿大的蒙特利尔市。现在，他正在开发一个下一代的多平台引擎。有些时候，他喜欢在源代码里面加上一些看起来很像程序错误的“特性”，以测试一下同事的技术。

Gabriel Rohweder

grohwed@hotmail.com

Gabriel 在 13 岁的时候用苹果机的 BASIC 编出了自己的第一个游戏。几年以后，他正式从 Ball State 大学获得了计算机科学的学位。紧巴巴地过了一阵数据库程序员与网络工程师的日子之后，他立刻就明白了，自己马上可以圆上儿时的梦想，成为游戏业界中的一员了。他现在加入了微软，是 DirectPlay 开发小组中的一名软件设计工程师。

Greg Seegert

gseegert@alum.wpi.edu

Greg Seegert 是 Stainless Steel 工作室的一名程序员，在那里他曾为知名的游戏 *Empire Earth* 做出过很多贡献。在不编程的时候，Greg 会思考编程、阅读编程书籍，甚至会梦到编程。自从 10 岁开始起，他就开始编程了，虽然直到现在 Greg 还没有在社会中成熟起来，但是他现在依然决定继续编程。在此期间，他希望自己能成为真正的高手。

Jason Shankel

JShankel@maxis.com

Jason Shankel 自从 1992 年以来就是一名职业游戏程序员了。现在，他是 Electronic Arts/Maxis 的一名资深软件工程师，但是他希望自己有一天能成为一个真正的男孩。

Cláudio Silva

csilva@research.att.com

Cláudio Silva 是 AT&T 实验室研究部的一名资深技术员。他当前的研究兴趣主要集中在建构可扩展显示的架构与算法、大型数据集的渲染技术、3D 扫描与图形硬件的算法上。Claudio 从纽约大学石溪分校获得了自己的计算机科学博士学位。当还是一个学生的时候，他曾在 Sandia 国家实验室工作过，在那里他开发出了大型可视化算法与处理巨型数据集的工具。Claudio 在国际会议与期刊上发表论文超过 40 篇，而且曾在不同的会议上课，包括 ACM SIGGRAPH、Eurographics 与 IEEE Visualization。

Download at www.Pin5i.Com

Zack Booth Simpson

zsimpson@sprynet.com

Zack Booth Simpson 于 1991 年加入了 Origin/Electronic Arts 公司，成为了 Ultima 中的一名程序员，在 1995 年他离开的时候，已经是技术总监了（他后来变成了一名研究员）。1995 年，他与他人一同创建了 Titanic Entertainment 公司，此公司发布过 *NetStorm: Islands at War* 游戏。1997 年，Zack 退出了游戏业界，现在喜欢创作一些交互式的艺术作品、环球旅行，以及教授数学课程。他的主页是 http://www.totempole.net。

Greg Snook

gregsn@microsoft.com

Greg 在游戏业界已经工作 8 年多了，他曾经在 Viacom New Media、Kinesoft、TerraGlyph，以及 Past Tree 工作过，为他们制作过游戏。现在他在微软的 Bungie 工作室工作，制作机密的 Xbox 游戏，你至少要灌他三杯好酒才能从他嘴里套出话来。Greg 也常常为自己倾注了心血的史诗般的角斗士这个作品而感叹，其中一幕正是《游戏编程精粹 2》的封底插图。

William van der Sterren

william@cgf-ai.com

William van der Sterren 为计算机游戏与仿真开发战术式的人工智能技术。他是 CGF-AI 公司的创建者之一，也是该公司的开发人员与顾问。William 曾经在游戏开发者大会上发表过演讲，也曾经在 *AI Programming Wisdom*（Charles River Media 出版公司，2002）与“游戏编程精粹”系列丛书中发表过文章。William 曾经在嵌入式系统与防御仿真的领域作过研究科学家，他现在是一个下一代游戏机上游戏的人工智能部分的主力开发员。

Jan Svarovsky

jan@svarovsky.com

Jan 的编程生涯已经有将近 20 年了。他是在 1995 年从英国的剑桥大学毕业，加入 Bullfrog 公司的，在那里他进入了研发部，编写人工智能的程序、试验性的 3D 引擎、网络程序、还有两个最早的 3D 加速器的 PC 移植版。他在 Bullfrog 的两个技术领先的项目中充当着主力开发员或是唯一开发员的角色。之后，他加入了 Mucky Foot Productions 公司，成为了 *StarTopia* 的主力开发人员，这是一个在 2001 年 6 月发布的游戏。他的网页是 http://www.svarovsky.com/jan。

Download at www.Pin5i.Com

Dante Treglia

treglia@yahoo.com

Dante 现在还记得年少时的日子，那时他还在使用自己的 Atari 400，用 BASIC 语言画出一个个模糊的点。几年之后，他发现自己可以使用 Mathematica 创作出眩目的图形，而不必在屏幕上费劲地画点了——其结果就是，他一下子从乔治亚大学获得了数学学士学位。但是，面临了现实世界之后，他立刻发现自己走错了路，于是决定到 Clemson 大学去学习编程。在 Clemson，他花费了很多时间带着 5 磅重的连着价值百万美元的虚拟现实硬件头套显示器，查看 OpenGL 的代码。获得了硕士学位以后，Dante 重返现实世界，幸运地得到了任天堂（美国）公司的一份工作。他现在是开发技术支持的一名主力软件工程师，同时还在继续自己充满兴趣的虚拟现实研究——这次使用的设备可就便宜多了。

Alex Vlachos

Alex@Vlachos.com

Alex Vlachos 自从 1998 年起就是 ATI 公司 3D 应用研究小组中的一名资深软件工程师了，他主要关注 3D 引擎的开发。Alex 是 ATI 图形演示与屏幕保护程序的主力开发员，而且他一直在编写 3D 引擎以用来展示下一代的硬件特性。此外，他还开发了 N-贴片（一种曲面表示形式，它是微软 DirectX8 中的一部分），也称为 PN 三角形或者是 TRUFORM。在加入 ATI 之前，他是 Spacetec IMC 的一名软件工程师，其任务是开发 SpaceOrb 360，一个具有 6°自由度的游戏控制器。他曾经在《游戏编程精粹 1》、《游戏编程精粹 2》、ACM 交互式 3D 图形研讨会以及 *ShaderX* (Wordware Publishing, Inc.)上发表过文章。Alex 是波士顿大学的研究生毕业。他的网页是 http://alex.vlachos.com。

Carlo Vogelsang

cvogelsang@creativelabs.com

Carlo Vogelsang 自从 1998 年初期就开始开发游戏了。他曾经在许多公司开发过音乐与音频以及游戏的其他部分，这些公司包括：Epic MegaGames、Digital Extremes, Triumph Studios, Orange Games 以及 Secret Level。Carlo 现在在创新实验室工作。

Jason Weber

jason.p.weber@intel.com

Jason Weber 最开始在军方研究实验室从事 3D 夜视景地形仿真开发包的编程工作，后来他做出了具有动画效果的树木与草丛。现在，他在英特尔图形与 3D 技术小组工作，开发在 Macromedia's Shockwave3D 中使用到的运动与骨架变形技术。Jason 曾经在 SIGGRAPH 与游戏开发者大会上面发表过文章。他的网页是 http://www.imonk.com/baboon。

Download at www.Pin5i.Com

Stephen White

swhite@naughtydog.com

Stephen White 是 Naughty Dog 的开发总监，具有 15 年之久的在多种平台上开发视频游戏的职业生涯。Stephen 的著名之作有 *Jak and Daxter: The Precursor Legacy*、*Crash Bandicoot: Warped*、*Crash Bandicoot: Cortex Strikes Back*、*Brilliance*，以及 *Deluxe Paint ST*。他特别喜欢制作视频游戏，而且曾经参与过视频游戏开发的方方面面。

Steven Woodcock

ferretman@gameai.com

Steven Woodcock 在人工智能方面的背景知识是从 18 年的弹道导弹防御工事建筑与大量的实时战争游戏以及仿真的制作中得来的。他在 www.gameai.com 网址上保留了一张网页专门用来宣传游戏中的人工智能，而且他也是此课题上许多论文与出版物的作者。他现在为许多签订的合同制作游戏中的人工智能，并在游戏开发者大会上帮助他人调整游戏中的人工智能策略，他还是此领域几本书籍与杂志的作者与技术编辑之一。

Thomas Young

thomas.young@bigfoot.com

Thomas 多年前在一台 Amiga 机器上开发硬件游戏的时候牺牲了一颗牙齿。从 Sussex 大学获得了一个人工智能的学位以后，他加入了英国谢菲尔德的 Gremlin Interactive 公司，成为了一名人工智能程序员。这些年来他在人工智能方面的两个研究兴趣是让人物理解其环境中的障碍物以及让人物的走动更加真实。2000 年，他离开了 Gremlin（现为 Infogames，Sheffield House），成为了一名独立签约者，同时创建了一个公司，在中间件许可证的形式下发送自己复杂的寻径系统（http://www.pathengine.com）。

Mark Zarb-Adami

Mark@muckyfoot.com

Mark Adami 从 10 岁就开始编写游戏了，当时他只有一台配有 BASIC 解释器的 Atari 800，这是他的圣诞礼物，不过电脑上没有安装游戏。8 年后，他在剑桥大学学习计算机科学，之后曾在 Bullforg 公司工作，现在任职于 Mucky Foot 公司。他的游戏作品有 *Syndicate Wars* 和 *Urban Chaos*。

Download at www.Pin5i.Com

目　　录

第 1 章　通用编程技术

简介 …… 2
Kim Pallister
1.1　调度游戏中的事件 …… 4
Michael Harvey，Carl S. Marshall
1.1.1　调度器的组成 …… 5
1.1.2　一个简单的调度器 …… 8
1.1.3　高级概念 …… 10
1.1.4　结论 …… 11
1.1.5　参考文献 …… 12
1.2　一个基于对象组合的游戏架构 …… 13
Scott Ratterson
1.2.1　游戏开发的各个阶段 …… 13
1.2.2　游戏架构设计 …… 14
1.2.3　游戏架构实现 …… 17
1.2.4　源代码 …… 20
1.2.5　参考文献 …… 21
1.3　让 C 中的宏重现光辉 …… 23
Steve Rabin
1.3.1　声明 …… 23
1.3.2　第 1 个宏技巧：把枚举值转化为字符串 …… 23
1.3.3　第 2 个宏技巧：利用二进制表达式得到编译期常量 …… 25
1.3.4　第 3 个宏技巧：给标准断言添加描述性注释 …… 26
1.3.5　第 4 个宏技巧：编译期断言 …… 26
1.3.6　第 5 个宏技巧：得到一个数组里面的元素个数 …… 27
1.3.7　第 6 个宏技巧：在一个字符串中间加入__LINE__ …… 27
1.3.8　第 7 个宏技巧：防止进入无限循环 …… 28
1.3.9　第 8 个宏技巧：小型的特制语言 …… 29
1.3.10　第 9 个宏技巧：简化类接口 …… 30
1.3.11　结论 …… 33
1.3.12　参考文献 …… 33
1.4　平台无关的函数绑定代码生成器 …… 34
Allen Pouratian
1.4.1　年轻与智慧 …… 34

Download at www.Pin5i.Com

1.4.2 概要 …… 35
1.4.3 细节 …… 36
1.4.4 脚本 …… 38
1.4.5 网络 …… 38
1.4.6 结论 …… 39
1.4.7 参考文献 …… 39
1.5 基于句柄的智能指针 …… 40
Brian Hawkins
1.5.1 用法 …… 40
1.5.2 句柄 …… 41
1.5.3 智能指针 …… 42
1.5.4 结论 …… 43
1.5.5 参考文献 …… 43
1.6 定制 STL 分配器 …… 44
Pete Isensee
1.6.1 一个范例 …… 44
1.6.2 分配器的基础 …… 45
1.6.3 分配器的要求 …… 45
1.6.4 缺省的分配器对象 …… 49
1.6.5 编写自己的分配器 …… 49
1.6.6 潜在的用途 …… 51
1.6.7 分配器状态数据 …… 51
1.6.8 一些建议 …… 52
1.6.9 实现细节 …… 52
1.6.10 结论 …… 52
1.6.11 参考文献 …… 53
1.7 立即存盘 …… 54
Martin Brownlow
1.7.1 为何如此困难 …… 54
1.7.2 SAVEMGR 类 …… 55
1.7.3 SAVEOBJ 类 …… 55
1.7.4 数据类型与扩展 …… 56
1.7.5 重载缺省函数 …… 56
1.7.6 一个简单的例子 …… 57
1.7.7 结论 …… 58
1.8 自动列表设计模式 …… 59
Ben Board
1.8.1 实现 …… 59
1.8.2 实现时的注意事项 …… 61

Download at www.Pin5i.Com

1.8.3 结论 …… 63
1.9 浮点异常处理 …… 64
Søren Hannibal
1.9.1 为什么要崩溃 …… 64
1.9.2 你的程序处理浮点异常么 …… 65
1.9.3 异常的类型 …… 65
1.9.4 代码 …… 65
1.9.5 调试浮点错误 …… 66
1.9.6 结论 …… 66
1.10 使用 UML 开发一个配合设计的游戏引擎 …… 67
Thomas Demachy
1.10.1 对象就在游戏之中 …… 67
1.10.2 动态的类——正如动态的棋子 …… 70
1.10.3 协作与迭代 …… 72
1.10.4 实现上的问题 …… 73
1.10.5 结论 …… 74
1.10.6 参考文献 …… 75
1.11 使用 Lex 和 Yacc 分析自定义数据文件 …… 76
Paul Kelly
1.11.1 Lex …… 77
1.11.2 Yacc …… 77
1.11.3 优点与缺陷 …… 77
1.11.4 Yacc 和 Lex 中的交互 …… 78
1.11.5 针对游戏子系统的自定义数据文件 …… 79
1.11.6 把数据输出工具与 Lex 和 Yacc 结合起来 …… 80
1.11.7 一个完整的例子 …… 80
1.11.8 结论 …… 84
1.11.9 如何得到 Flex 和 Bison …… 84
1.11.10 参考文献 …… 84
1.12 为世界市场开发游戏 …… 85
Aaron Nicholls
1.12.1 市场潜力 …… 85
1.12.2 门面事，先处理——显示和输入 …… 86
1.12.3 字符集 …… 88
1.12.4 界面和设计方面的考虑 …… 90
1.12.5 本地化 …… 93
1.12.6 设计和规划中的考虑 …… 94
1.12.7 测试 …… 95
1.12.8 结论 …… 98

Download at www.Pin5i.Com

1.12.9 参考文献 …… 98
1.13 3D 游戏中的实时输入和用户界面 …… 99
Greg Seegert
1.13.1 实现用户界面 …… 99
1.13.2 指定用户界面元素 …… 100
1.13.3 本地化问题 …… 101
1.13.4 输入系统 …… 102
1.13.5 鼠标与操纵杆 …… 103
1.13.6 在处理延迟方面用户界面的作用 …… 104
1.13.7 结论 …… 105
1.13.8 参考文献 …… 105
1.14 自然的选择：饼状菜单的演化 …… 106
Don Hopkins
1.14.1 Feng GUI 的饼状菜单 …… 106
1.14.2 对饼状菜单的研究与评估 …… 107
1.14.3 饼状菜单插件 …… 108
1.14.4 未来发展方向 …… 112
1.14.5 走进 *SimCity* 中的城镇 …… 112
1.14.6 *The sims* 中的起居室 …… 114
1.14.7 结论 …… 115
1.14.8 参考文献 …… 115
1.15 轻量级的、基于规则的日志记录 …… 117
Brian Hawkins
1.15.1 规则 …… 117
1.15.2 调试标志 …… 117
1.15.3 配置文件 …… 118
1.15.4 可配置的标志值 …… 119
1.15.5 日志记录 …… 119
1.15.6 用法 …… 121
1.15.7 结论 …… 122
1.15.8 参考文献 …… 122
1.16 日志服务 …… 123
Eric Robert
1.16.1 管理信息 …… 123
1.16.2 系统层次 …… 124
1.16.3 Journal 接口 …… 127
1.16.4 创建日志服务 …… 129
1.16.5 结论 …… 132
1.16.6 参考文献 …… 132

Download at www.Pin5i.Com

1.17 实时的层次化性能评测 …… 133
Greg Hjeistrom，Byon Garrabrant
1.17.1 性能评测树 …… 134
1.17.2 用法 …… 134
1.17.3 实现 …… 136
1.17.4 结论 …… 139
1.17.5 参考文献 …… 139

第 2 章 数学技巧

简介 …… 142
John Byrd
2.1 对数与随机数生成的 2 基快速函数 …… 144
James McNeill
2.1.1 整数的 2 基对数 …… 144
2.1.2 位掩码与随机数生成 …… 144
2.1.3 函数是如何工作的 …… 146
2.1.4 参考文献 …… 146
2.2 使用分数矢量得到更精确的几何图形 …… 147
Thomas Young
2.2.1 问题 …… 147
2.2.2 一个解决方法：分数矢量 …… 150
2.2.3 使用分数矢量 …… 151
2.2.4 数字的范围 …… 152
2.2.5 实现上的细节 …… 153
2.2.6 结论 …… 154
2.2.7 参考文献 …… 154
2.3 三角函数的更多近似计算方法 …… 155
Robin Green
2.3.1 衡量误差 …… 155
2.3.2 正弦与余弦函数 …… 156
2.3.3 多项式逼近 …… 163
2.3.4 有关收敛性的注意事项 …… 167
2.3.5 结论 …… 168
2.3.6 参考文献 …… 168
2.4 四元数的压缩 …… 169
Mark Zarb-Adami
2.4.1 四元数 …… 169
2.4.2 三个最小数方法 …… 169
2.4.3 极点方法 …… 170

Download at www.Pin5i.com

2.4.4 实现 …… 171
2.4.5 性能 …… 171
2.4.6 结论 …… 172
2.4.7 答谢 …… 172
2.4.8 参考文献 …… 172
2.5 受限的逆向运动学 …… 173
Jason Weber
2.5.1 骨节层次 …… 173
2.5.2 循环坐标推演 …… 174
2.5.3 旋转限制 …… 175
2.5.4 调整每个骨节，同时保持限制 …… 176
2.5.5 结论 …… 178
2.5.6 参考文献 …… 178
2.6 针对物理建模的单元自动机 …… 180
Tom Forsyth
2.6.1 CA 基础 …… 180
2.6.2 八叉树 …… 183
2.6.3 实际的物理 …… 183
2.6.4 核心处理模型 …… 184
2.6.5 气体 …… 185
2.6.6 水流 …… 185
2.6.7 流速 …… 186
2.6.8 热量 …… 187
2.6.9 火焰 …… 189
2.6.10 动态更新速率 …… 190
2.6.11 结论 …… 191
2.6.12 参考文献 …… 192
2.7 在动态仿真中处理摩擦 …… 193
Miguel Gomez
2.7.1 库仑摩擦力 …… 193
2.7.2 数值方法 …… 197
2.7.3 一个三维公式 …… 200
2.7.4 几何图形问题 …… 201
2.7.5 结论 …… 202
2.7.6 参考文献 …… 202

第 3 章 人工智能

简介 …… 204
Steven WoodCock

Download at www.Pin5i.Com

3.1 经 GoCap 优化过的机器学习 …… 206
Thor Alexander
3.1.1 GoCap 架构一览 …… 206
3.1.2 训练开车 …… 208
3.1.3 学习规则 …… 209
3.1.4 结论 …… 213
3.1.5 参考文献 …… 213
3.2 区域游览：对寻径模式的扩展 …… 214
Ben Board，Mike Ducker
3.2.1 辞旧 …… 215
3.2.2 迎新 …… 216
3.2.3 分而治之 …… 219
3.2.4 路径遍历 …… 221
3.2.5 对此模式的扩展 …… 225
3.2.6 结论 …… 225
3.2.7 参考文献 …… 225
3.3 基于函数指针的内嵌式有限状态机 …… 226
Charles Farris
3.3.1 什么是有限状态机 …… 226
3.3.2 FSM 的实现 …… 227
3.3.3 实现 CFSM …… 229
3.3.4 使用 CFSM …… 233
3.3.5 结论 …… 235
3.3.6 参考文献 …… 236
3.4 在 RTS 中的地形分析——一个隐藏的重要因素 …… 237
Daniel Higgins
3.4.1 区域 …… 237
3.4.2 凸包 …… 241
3.4.3 重要的匹配器 …… 244
3.4.4 关隘 …… 247
3.4.5 进行地形分析 …… 250
3.4.6 结论 …… 251
3.4.7 参考文献 …… 251
3.5 一个针对 AI 代理、对象，以及任务的可扩展触发器系统 …… 252
Steve Rabin
3.5.1 触发器系统简介 …… 252
3.5.2 对象自有的触发器系统 …… 253
3.5.3 定义条件 …… 253

Download at www.Pin5i.Com

3.5.4 使用布尔逻辑组合条件 …… 253
3.5.5 定义响应 …… 255
3.5.6 求取触发器的值 …… 255
3.5.7 一次性触发与载入次数 …… 256
3.5.8 使用标志和计数器将触发器结合起来 …… 257
3.5.9 触发器系统与脚本语言的对比 …… 258
3.5.10 局限性 …… 259
3.5.11 结论 …… 259
3.5.12 参考文献 …… 259
3.6 基于 A*算法的战术式寻径 …… 260
William van der Sterren
3.6.1 有风险的 A* …… 261
3.6.2 对于有缺陷路径的战术式改良 …… 263
3.6.3 暴露时间与对敌人建模 …… 263
3.6.4 威胁并不仅仅是静态的 …… 265
3.6.5 更战术化的改进 …… 266
3.6.6 性能 …… 266
3.6.7 有效的火力线以及视野的探测 …… 267
3.6.8 扩展的 A*算法的代价 …… 268
3.6.9 ASE 程序 …… 269
3.6.10 结论 …… 269
3.6.11 参考文献 …… 270
3.7 快速游览网格的方法 …… 271
Stephen White，Christopher Christensen
3.7.1 静态障碍与动态障碍 …… 271
3.7.2 游览网格 …… 271
3.7.3 门户 …… 273
3.7.4 建表 …… 275
3.7.5 其他的门户相关问题 …… 276
3.7.6 表示生物 …… 277
3.7.7 动态障碍 …… 278
3.7.8 在静态障碍与动态障碍之间进行游览 …… 280
3.7.9 有关游览网格的其他想法 …… 280
3.7.10 结论 …… 281
3.8 在寻径与碰撞之间选择一种关系 …… 282
Thomas Young
3.8.1 在碰撞控制下的运动 …… 282
3.8.2 对于寻径的碰撞模型 …… 282
3.8.3 方法 1：具有容错性的 AI …… 283

Download at www.Pin5i.Com

3.8.4 方法 2：在无障碍空间一个子集内的寻径 …… 285
3.8.5 方法 3：使用寻径器本身处理人物碰撞 …… 287
3.8.6 实现沿路的运动 …… 289
3.8.7 结论 …… 290
3.8.8 参考文献 …… 290

第 4 章 图形

简介 …… 294
Jeff Lander
4.1 消除 T 形连接与重新三角化 …… 297
Eric Lengyel
4.1.1 T 形连接的消除 …… 298
4.1.2 重新三角化 …… 299
4.1.3 实现 …… 300
4.1.4 结论 …… 301
4.2 快速高程场法线的计算 …… 302
Jason Shankel
4.2.1 一个任意网格上的法线 …… 302
4.2.2 高程场法线 …… 303
4.2.3 结论 …… 305
4.2.4 例子程序 …… 305
4.2.5 参考文献 …… 306
4.3 快速计算面片法线 …… 307
Martin Brownlow
4.3.1 定义 …… 307
4.3.2 传统方法 …… 307
4.3.3 相关问题 …… 308
4.3.4 一个更简单的方法 …… 308
4.3.5 其他的优点 …… 308
4.3.6 此方法的精确度有多大 …… 309
4.3.7 结论 …… 309
4.3.8 参考文献 …… 309
4.4 快速、简单的遮蔽剪裁 …… 310
Wagner T. Corrêa，Princeton University
4.4.1 可见性问题 …… 310
4.4.2 PLP 算法 …… 311
4.4.3 cPLP 算法 …… 312
4.4.4 讨论 …… 312
4.4.5 实验结果 …… 313

Download at www.Pin5i.Com

4.4.6 结论 …… 314
4.4.7 参考文献 …… 314
4.5 三角形条带的创建、优化以及渲染 …… 316
Carl S. Marshall
4.5.1 三角形条带 …… 316
4.5.2 三角形条带的创建 …… 317
4.5.3 优化 …… 320
4.5.4 渲染 …… 321
4.5.5 倾向于缓存的三角形条带 …… 321
4.5.6 连续分层细节的三角形条带 …… 321
4.5.7 结论 …… 322
4.5.8 参考文献 …… 322
4.6 针对复杂数据集计算优化阴影体 …… 323
Alex Vlachos，Drew Card
4.6.1 前期工作 …… 323
4.6.2 算法 …… 323
4.6.3 优化算法 …… 325
4.6.4 参考文献 …… 326
4.7 针对人物运动的表面细分 …… 327
William Leeson
4.7.1 各种细分模式 …… 327
4.7.2 骨节的层次化结构以及顶点积累缓冲 …… 331
4.7.3 优化 …… 332
4.7.4 系统集成 …… 334
4.7.5 源代码 …… 335
4.7.6 结论 …… 336
4.7.7 参考文献 …… 336
4.8 改良的骨节变换计算 …… 337
Jason Weber
4.8.1 背景知识 …… 337
4.8.2 简单的方法 …… 338
4.8.3 添加骨节 …… 339
4.8.4 改变权重 …… 340
4.8.5 系统集成与优化 …… 342
4.8.6 结论 …… 344
4.8.7 参考文献 …… 344
4.9 针对真实人物运动的架构 …… 345
Thomas Young
4.9.1 问题：针对任意目标的运动 …… 345

Download at www.Pin5i.Com

4.9.2 问题：运动之间的平滑过渡 …… 347
4.9.3 解决问题的一个架构：局部修正器与独立的插值系数 …… 348
4.9.4 应用：处理任意目标的运动 …… 349
4.9.5 位移修正器 …… 350
4.9.6 应用：变换 …… 351
4.9.7 其他细节 …… 351
4.9.8 结论 …… 352
4.9.9 参考文献 …… 352
4.10 可编程顶点着色器的编译器 …… 353
Adam Lake
4.10.1 可编程顶点着色器 …… 353
4.10.2 编译器 …… 355
4.10.3 编译器的组成部分 …… 355
4.10.4 结论 …… 359
4.10.5 致谢 …… 359
4.10.6 参考文献 …… 359
4.11 画板光束 …… 361
Brian Hawkins
4.11.1 矩阵 …… 361
4.11.2 顶点 …… 362
4.11.3 UV 映射 …… 363
4.11.4 结论 …… 363
4.12 针对等测引擎的 3D 技术 …… 364
Greg Snook
4.12.1 进入第三个维度 …… 365
4.12.2 方法 1：画板越多，效果越好 …… 365
4.12.3 方法 2：变换纹理 …… 366
4.12.4 方法 3：垂直插值的纹理 …… 368
4.12.5 结论 …… 369
4.12.6 参考文献 …… 369
4.13 使用法向地图进行曲面模拟 …… 370
Oscar Blasco
4.13.1 法向地图 …… 370
4.13.2 整个过程的纵览 …… 371
4.13.3 数据准备 …… 371
4.13.4 投影线 …… 372
4.13.5 得到细节信息 …… 373
4.13.6 后处理 …… 374
4.13.7 已知的问题 …… 374

Download at www.Pin5i.Com

4.13.8 其他方法 …… 375
4.13.9 结论 …… 375
4.13.10 致谢 …… 376
4.13.11 参考文献 …… 376
4.14 动态的、具有照片效果的地形光照 …… 377
Naty Hoffman，Kenny Mitchell
4.14.1 背景知识 …… 377
4.14.2 解的分类 …… 379
4.14.3 日照：地平角、椭圆阴影以及 PTM …… 379
4.14.4 天空光照：辐射透过量的近似与分块 …… 381
4.14.5 活动的云层阴影 …… 382
4.14.6 基于视频的解决方案 …… 384
4.14.7 非地形对象 …… 385
4.14.8 结论 …… 385
4.14.9 参考文献 …… 385
4.15 立体图光照技术 …… 387
Kenneth L. Hurley
4.15.1 立体图的物理属性 …… 387
4.15.2 如何与立体图进行数据交换 …… 388
4.15.3 使用立体图进行渲染 …… 388
4.15.4 对云层进行编码 …… 389
4.15.5 在一个立体图中对光源进行编码 …… 392
4.15.6 在立体图中渲染散射光照 …… 392
4.15.7 将日夜循环编码进立体图中 …… 393
4.15.8 结论 …… 393
4.15.9 参考文献 …… 393
4.16 程序纹理 …… 394
Mike Milliger
4.16.1 参数与函数 …… 394
4.16.2 进入游戏世界 …… 395
4.16.3 硬件加速 …… 397
4.16.4 结论 …… 398
4.16.5 致谢 …… 399
4.16.6 参考文献 …… 399
4.17 独一无二的纹理 …… 400
Tom Forsyth
4.17.1 程序纹理 …… 400
4.17.2 智能纹理缓存 …… 401
4.17.3 合成模型 …… 401

Download at www.Pin5i.Com

4.17.4 层的映射与变换 …… 401
4.17.5 层的源与过滤器 …… 402
4.17.6 合成方法 …… 402
4.17.7 对数字的控制 …… 403
4.17.8 动态纹理 …… 403
4.17.9 可扩展性 …… 404
4.17.10 使用 CPU 还是图形芯片进行合成运算 …… 405
4.17.11 演示程序 …… 405
4.17.12 结论 …… 406
4.17.13 参考文献 …… 406
4.18 使用纹理作为查找表进行逐像素光照计算 …… 407
Alex Vlachos, John Isidoro, Chris Oat
4.18.1 不使用立体图进行 h 归一化（$n.h/h.h$ 映射）的镜面逐像素光照 …… 407
4.18.2 使用一个$(n.h)^k$图的逐像素镜面指数 …… 409
4.18.3 色彩偏移的光晕 …… 411
4.18.4 拥有正确的逐像素衰减的逐像素点光照 …… 412
4.18.5 拥有正确的逐像素衰减的逐像素聚光灯与方向性光照 …… 413
4.18.6 结论 …… 415
4.18.7 参考文献 …… 415
4.19 使用手工制作的着色模型进行渲染 …… 416
Jan Kautz
4.19.1 着色模型 …… 416
4.19.2 基于微表面的着色模型 …… 417
4.19.3 NDF 着色 …… 417
4.19.4 使用 NDF 的凸凹贴图 …… 419
4.19.5 扩展 …… 420
4.19.6 结论 …… 420
4.19.7 参考文献 …… 420

第 5 章 网络和多玩家游戏

简介 …… 424
Andrew Kirmse
5.1 将实时策略游戏中的延迟最小化 …… 425
Jim Greer, EA.com, Zachary Booth Simpson, Mine Control
5.1.1 帧锁定与事件锁定 …… 425
5.1.2 时间同步 …… 429
5.1.3 结论 …… 431
5.1.4 参考文献 …… 431
5.2 实时策略网络协议 …… 432

Download at www.Pin5i.com

Jan Svarovsky
5.2.1 其他的协议 …… 432
5.2.2 我们的协议 …… 433
5.2.3 精炼 …… 435
5.2.4 有用的模块 …… 437
5.2.5 在 StarTopia 中容易犯的错误 …… 438
5.2.6 示例游戏 …… 439
5.2.7 结论 …… 439
5.2.8 参考文献 …… 439
5.3 一个针对巨量多玩家游戏的灵活的仿真架构 …… 440
Thor Alexander
5.3.1 架构一览 …… 440
5.3.2 支持类 …… 441
5.3.3 核心类 …… 443
5.3.4 管理器与工厂 …… 446
5.3.5 把它们都组合起来 …… 448
5.3.6 结论 …… 451
5.3.7 参考文献 …… 451
5.4 对多玩家游戏进行扩展 …… 452
Justin Randall
5.4.1 改善游戏公平度的策略 …… 452
5.4.2 设计可扩展的服务器 …… 454
5.4.3 分布负载 …… 459
5.4.4 优化 …… 461
5.4.5 结论 …… 463
5.4.6 参考文献 …… 464
5.5 基于模板的对象序列化 …… 465
Jason Beardsley
5.5.1 现存的解决方案 …… 465
5.5.2 可移植性 …… 467
5.5.3 Serializer 类 …… 468
5.5.4 扩展与优化 …… 473
5.5.5 未来的工作 …… 475
5.5.6 结论 …… 475
5.5.7 参考文献 …… 475
5.6 安全套接字 …… 476
Pete Isensee
5.6.1 IPSec …… 476
5.6.2 警告 …… 477

Download at www.Pin5i.Com

5.6.3 安全连接 …… 477
5.6.4 包格式 …… 478
5.6.5 发送数据 …… 479
5.6.6 接收数据 …… 480
5.6.7 示例实现 …… 481
5.6.8 CryptoAPI …… 483
5.6.9 性能 …… 483
5.6.10 安全性 …… 484
5.6.11 结论 …… 484
5.6.12 参考文献 …… 485
5.7 一个网络监控与模拟工具 …… 486
Andrew Kirmse
5.7.1 界面 …… 486
5.7.2 网络监控 …… 487
5.7.3 TCP 模拟 …… 487
5.7.4 UDP 模拟 …… 487
5.7.5 主机带宽模拟 …… 488
5.7.6 结论 …… 488
5.8 使用 DirectPlay8.1 创建多玩家游戏 …… 489
Gabriel Rohweder
5.8.1 DirectPlay 内幕 …… 489
5.8.2 数据传输 …… 490
5.8.3 可重入的回调函数 …… 493
5.8.4 使用 DirectPlay 发送语音 …… 496
5.8.5 相关资源 …… 499
5.9 使用 Java 微型版开发无线游戏 …… 500
David Fox
5.9.1 网络特性 …… 500
5.9.2 Java 微型版 …… 501
5.9.3 J2ME 网络精髓 …… 502
5.9.4 HTTP 的限制 …… 503
5.9.5 优化数据包 …… 504
5.9.6 从服务器获取图像 …… 505
5.9.7 结论 …… 506
5.9.8 参考文献 …… 507

第 6 章 音频处理

简介 …… 510
Scott Patterson

Download at www.Pin5i.Com

6.1 使用 Ogg Vorbis 进行音频压缩 …… 512
Jack Moffitt
6.1.1 心理声学压缩 …… 512
6.1.2 使用压缩的情况 …… 514
6.1.3 使用 Ogg 的代码示例 …… 515
6.1.4 结论 …… 518
6.1.5 参考文献 …… 518
6.2 创建一个美妙的 3D 音频环境 …… 519
Garin Hiebert
6.2.1 3D 音频的核心概念 …… 519
6.2.2 有效地使用你的音频引擎 …… 520
6.2.3 实现 …… 521
6.2.4 结论 …… 522
6.2.5 参考文献 …… 522
6.3 使用轴对齐的边界框设置音障 …… 524
Carlo Vogelsang
6.3.1 问题 …… 524
6.3.2 解决方法 …… 525
6.3.3 实现 …… 528
6.3.4 结论 …… 528
6.3.5 参考文献 …… 528
6.4 使用双二次共振滤波器 …… 529
Phil Burk
6.4.1 数字滤波器的工作原理 …… 529
6.4.2 IIR 与 FIR 滤波器 …… 530
6.4.3 双二次滤波器的实现 …… 530
6.4.4 改变变量 …… 531
6.4.5 避免异常情况 …… 531
6.4.6 控制滤波器 …… 532
6.4.7 计算此滤波器的系数 …… 532
6.4.8 低通滤波器 …… 533
6.4.9 高通滤波器 …… 533
6.4.10 带通滤波器 …… 533
6.4.11 将滤波器进行串联 …… 534
6.4.12 将滤波器进行并联 …… 534
6.4.13 软件 …… 534
6.4.14 结论 …… 534
6.4.15 参考文献 …… 534
6.5 语音压缩与音效的线性预测编码 …… 535

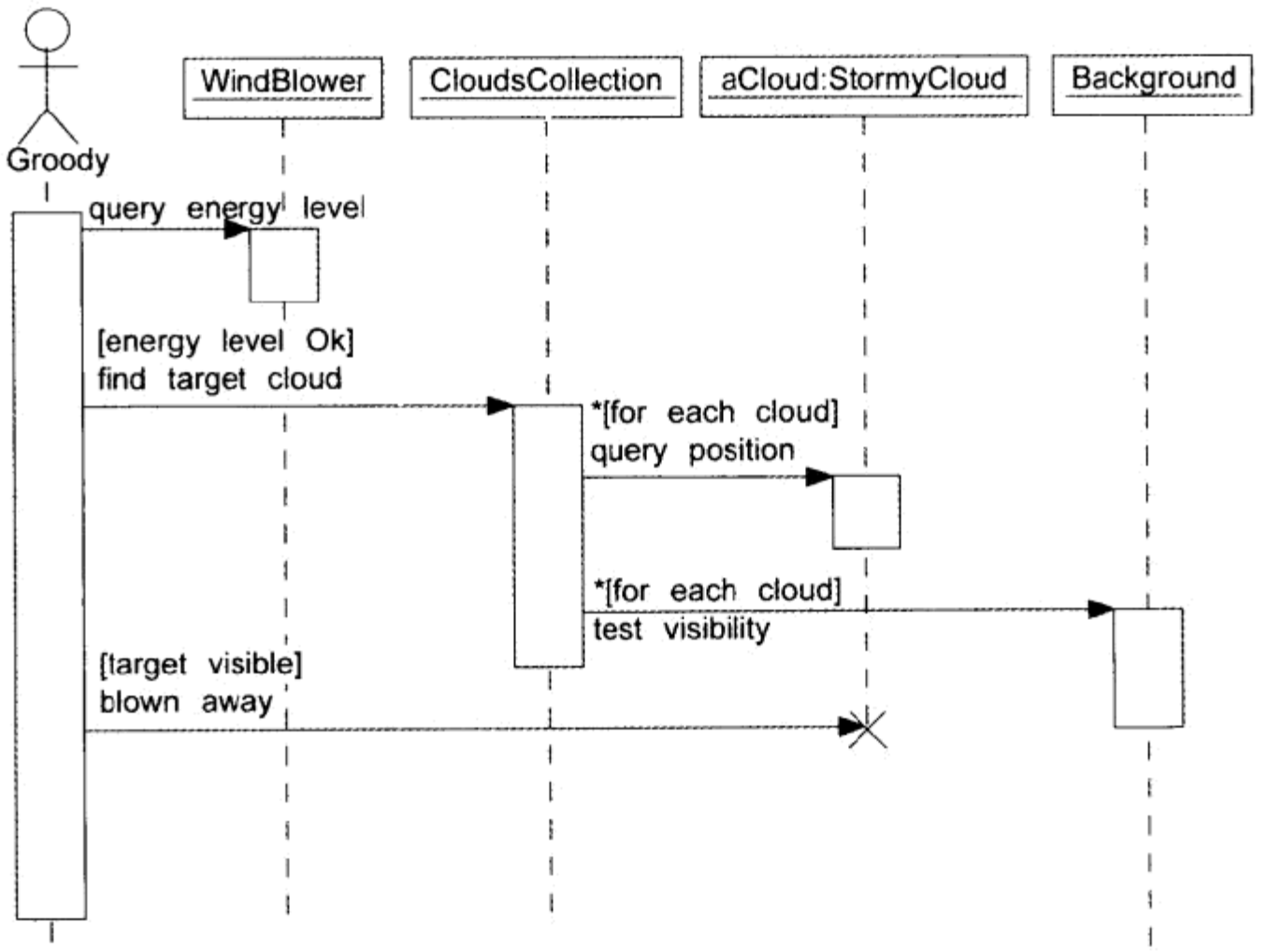
Download at www.Pin5i.Com

Eddie Edwards
6.5.1 对语音建模 …… 536
6.5.2 软件仿真 …… 537
6.5.3 取代声带 …… 539
6.5.4 控制重新合成器 …… 540
6.5.5 增加扬声器的深度 …… 541
6.5.6 对数据编码 …… 541
6.5.7 速度 …… 542
6.5.8 实验 …… 542
6.5.9 参考文献 …… 542
6.6 复杂声音的随机合成方法 …… 543
Phil Burk
6.6.1 线性同余算法 …… 543
6.6.2 噪声种类 …… 544
6.6.3 软件示例 …… 545
6.6.4 软件 …… 549
6.6.5 结论 …… 549
6.6.6 参考文献 …… 549
6.7 针对游戏的实时模块化音频处理 …… 550
Frank Luchs
6.7.1 模块化音频处理 …… 550
6.7.2 通过程序生成声音 …… 551
6.7.3 Sphinx MMOS 系统 …… 551
6.7.4 处理器 …… 552
6.7.5 模块文件简介 …… 552
6.7.6 模块文件的应用 …… 554
6.7.7 源代码 …… 556
6.7.8 结论 …… 557
6.7.9 参考文献 …… 557
索引 …… 559

Download at www.Pin5i.Com

第 1 章

通用编程技术



Download at www.Pin5i.Com

简介

Kim Pallister
英特尔公司
Kim.Pallister@intel.com

还是几年前出席一个会议的时候，我参加了 Ken Perlin 主办的一个讲座，在那里我听到了一件趣事。他讲了一个故事。一次，他和其他几个图形专家在自己最喜欢的高级语言话题上争了起来：

“C++是最好的高级语言！”一个人争执道。

“Java 是最好的高级语言！”另一个人说道。

“最好的高级语言，”又是一个人说，“是研究生。”

当然了，Ken 把大部分听众都给逗笑了——除了一些比较学究气一点的人以外，但是过了好一阵以后他们也开始大笑了。

真是一个有趣的笑话；但是如果再在这些话上多想想的话，你就会觉得它还是有些真实性的。编程只不过是要发明一种方法（在某些时候，如果你足够幸运的话，可能是一个天才式的方法）来解决问题并向计算机发出指令，以让它们使用这种方法来处理问题。不管这种问题很小可以用汇编语言来解决，或是问题较大需要使用 C++语言——或是一个很大的问题需要使用英语，通过一个“研究生编译器”来实现——其本质都是一样的。

在有了这一点认识以后，我们就随之面临了一个两难的问题，那就是应该为《游戏编程精粹 3》中这一章的文章选择什么级别的问题和解决方法。

是应该关注比较底层的经验——那些精巧的函数里的东西，例如应该使用什么方法在 *N* 条指令内实现某个功能，或是考虑如何利用 CPU 以让其帮你实现呢？还是应该关注那些高层的问题？或者我们是否要聚焦于业界高手（他们就是这些文章的来源）在引擎设计或是工具开发这些领域上的关键的点拨？

面对这些问题的时候，我们使用了历经时间考验的抽象化的方法。不管其层次、角度，或是其问题所在的领域，那些真正宝贵的东西是在处理问题中所表现出来的独到的洞察力。这种洞察力可以让读者在初遇问题的时候能成功地解决，而对于那些老手而言，则在下次遇到同样问题的时候为他们的宝库增加了一个新的工具。

请思考下面这句经常被引用的话：

我们称计算机编程为一种艺术，因为它将千锤百炼的知识应用到了世界，因为它需要技巧与天才，更重要的是因为它可以创造美的对象。

——Donald E. Knuth, *The Art of Computer Programming*, 1974

Download at www.Pin5i.Com

对 Knuth 的这种观点我们也可以做一个抽象。任何人，只要他做过某种层次上编程的工作，不管是高度优化的汇编语言编程还是大型的系统开发，在某种程度上，他就是找到了解决问题的一条途径或是一个方案，它不仅正确，而且优美。也正是此种发现使得阿基米德在从澡堂里跳出来的时候大叫“找到了!”，产生了此条举世名言。或者，以我们的现代语言来表达：“嘿，老兄，这真是太酷了!”显然，此种精神古今如一。

本章的文章覆盖了相当大的主题。我们讨论了一些高层话题，例如架构设计和保存游戏。我们开发方面的主题从 STL 分配器，到一些 C 语言的宏，再到浮点异常的处理。我们关注了如何使用设计辅助手段和工具的话题，例如 UML 以及 Lex 和 Yacc 以使得复杂游戏的开发更容易一些。我们甚至还有一些文章覆盖了诸如本地化、用户界面设计和实现这样的话题。最后，还有那些由于我们经常忽视而导致严重后果的领域，它们包括了如何更好地调试游戏和对游戏进行性能评测的问题。

这些主题覆盖很广，但是它们都有一个共同点，那就是它们都体现了作者的洞察力——这就是把它们收进本章的原因。现在就让我们继续前进，尽情遨游吧!

Download at www.Pin5i.Com

1.1 调度游戏中的事件

Michael Harvey，Carl S. Marshall
英特尔实验室
michael.harvey@intel.com
carl.s.marshall@intel.com

如果没有很清楚地了解了一个游戏中的事件——动画的更新、物体的碰撞，诸如此类的事情——是如何组织和执行的话，管理它们将是一件麻烦的事情。本节将谈谈一个调度器可以如何为游戏的架构提供组织性和灵活性。

开始我们将描述什么是调度器并说明为什么它很有用，然后将以调度器开发中的高级课题结束本节。在本书附送的 CD-ROM 上面有一个简单的调度器的源代码。

随着计算机游戏的日益复杂化，实时事件和仿真(simulation)在今天的游戏架构里已经成为了事实上的标准。我们需要一种方法来管理和执行一帧里面的的多个事件或者是帧里一个时间片上事件的多次触发。一个调度器可以使用很灵活的方式来管理游戏的事件，同时也能增强游戏的模块化，提供更好的可扩展性。

一个调度器可以有效地帮助以下游戏技术的实现，它们包括物理仿真、人物运动、碰撞检测、游戏中的人工智能、渲染[1]。在所有这些技术中有一个关键的方面，那就是时间。在不同的时间里，当数百个不同的物体和过程都需要更新的时候，这些仿真技术里的很多种都将变得非常复杂。举个例子，为了能更新物体的运动，一个物理仿真就需要为每个物体把时间分成很小的、相互独立的片断[Bourg01]。当采用更高的时间精度时，仿真就会在精确度上大大提高。在这种情况下，很多物体和时间片都是由同一段调度代码管理的，为了防止调度成为瓶颈，效率就成了一个至关重要的因素。

调度器的另外一个重要的能力在于它能够动态地增加和删除物体。这可以使得新的实体能平滑地加入到游戏里面去，和其他游戏里面的实体一起参加仿真，然后在不需要的时候也能从调度过程里面把它们删除掉。

译者注[1] 渲染（render）在这里指的是把数据经过一个过程的处理（例如视频或者音频的解码）最后显示出来（视频）或是播放出来（音频）。

Download at www.Pin5i.Com

1.1.1　调度器的组成

调度器的基本组件包括任务管理器、事件管理器和时钟（参见图 1.1.1）。通过这些组件，调度器就能生成基于时间或者是基于帧的事件，然后调用相应的事件处理器。在本节中，我们把事件处理器称为任务。

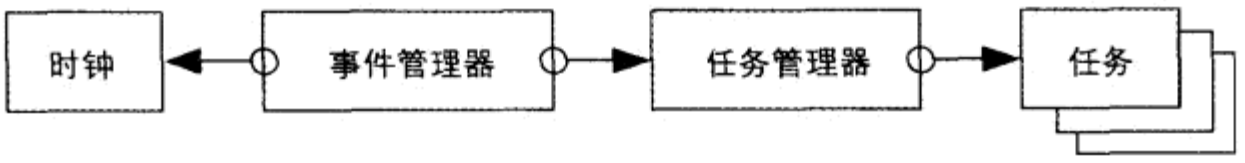


图 1.1.1　基本的调度器架构

1. 任务管理器

任务管理器处理任务的注册和组织。每个任务都有一个包含了一个管理器可以调用的回调函数的接口。任务管理器维护了一个任务列表，其中包含了每一个任务的调度信息——例如开始时间、执行频率、持续时间、优先级和其他必要的属性。它也可能包含一个用户数据的指针或者性能统计信息。

2. 事件管理器

事件管理器是调度器的核心部分。任务管理器里面的每一个任务都定义了一个或多个其需要处理的事件。一个事件指的是一个任务需要被执行的时间。例如在图 1.1.2 中，Task1 在时间 10 和 15 设定了事件。事件管理器的责任就是要产生必须的事件以执行相应的任务。

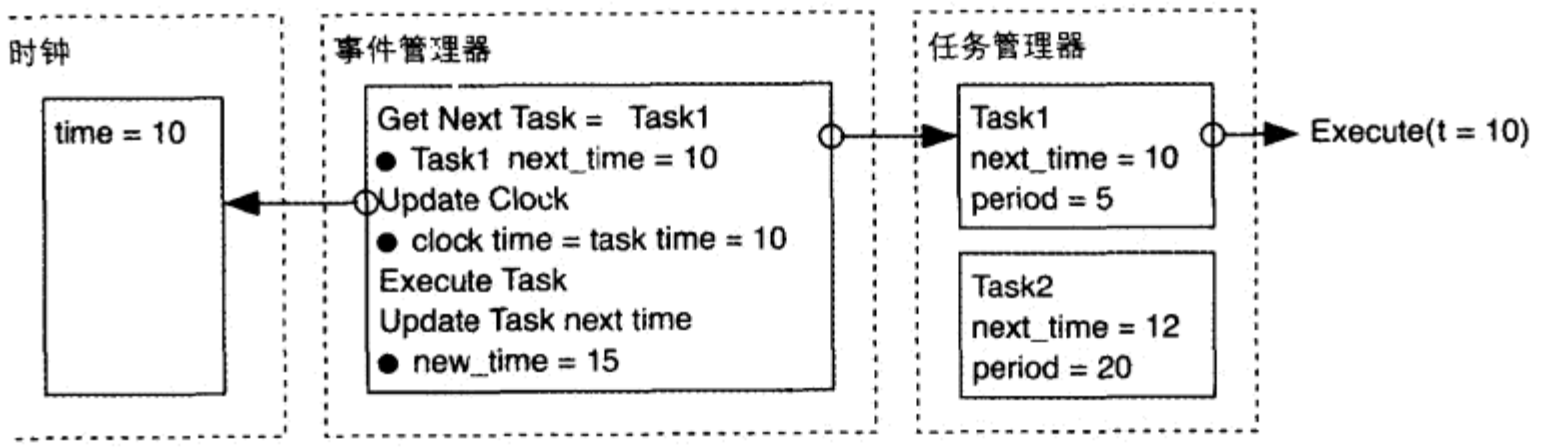


图 1.1.2　事件处理

3. 真实时间与虚拟时间

一个使用真实时间的调度器在概念上是很简单的——时间管理器不停地进行循环，查看一个真实时间的时钟，每当目标时间到达的时候它就会触发一个事件。在一个真实时间的系统里面，延时会有很大的影响。如果一个任务拖的时间太长，它就有可能与下一个任务的开始交叠了。由于每个任务都在它设定的时间发生，在任务之间的时间实际上在调度器中就被浪费了。

Download at www.Pin5i.Com

从任务的角度来看，时间只不过是一个数字。时间可以进行比较，而且流逝的时间可以根据比较的数字被计算出来。调度器可以通过操纵这个数字设置一个给定的时间或者是流逝的时间，而这可以和真实的时间完全独立——可能一眨眼几个小时就过去了，也可能时间暂停了。这就是虚拟时间的基本概念。

虚拟时间非常有用，因为它能让调度器在最方便的时候来执行任务，而不是被真实的时间牵着鼻子走。它可以对一系列的事件进行快进、停止、记录和重放的操作。这也使得调试实时应用程序方便多了，因为此时可以每次前进一个时间片。

一个虚拟事件的调度器会把时间分成帧。任务在帧之间以批处理的方式执行，在虚拟时间里运行，然后在每帧渲染出来的时候与真实的时间进行同步。如果帧率足够高，这就会造成一种真实时间的假象。然而，每帧间的几十 ms 对于计算机来说已经是一个大数目了，特别是在它效率比较高的时候。通过把所有的任务集中在一个块里面批量运行，剩余的时间就可以用来做些其他的工作了（参见可扩展性一段）。延时的问题几乎可以完全解决。

在本节中我们使用仿真时间这个词来指代虚拟时间，这是因为所有的仿真都使用它作为参照系。如果仿真时间停住了，那么仿真也会停住。当它继续走的时候，仿真中的物体就不会察觉任何不连续性。在仿真开始的时候仿真时间被置为零。

任务的执行是按顺序的，仿真时间就在任务执行中得到更新。举一个例子，假设每一帧长度为 20ms（参见图 1.1.3）。如果我们在 51ms 和 54ms 有事件发生，那么它们将在第二帧里面处理。除非第二帧结束，不然事件管理器是不会知道它的长度的。因此在第三帧开始的时候它将查看真实时间而发现这是第 60ms。现在它就可以处理第二帧里安排执行的任务了。Task1 是第一个任务，它是在 51ms 的地方的，但是仿真时间仍然是在第二帧的开始处。此时时钟就会被置为 51ms 然后开始执行 Task1。当 Task1 执行完了以后，时钟被设置到下一个事件的时间 54ms 处，然后 Task2 执行。在这帧里面再也没有其他的任务了，因此时钟就被设置到了这一帧的结尾（60ms），然后这一帧就被渲染出来了（如果你使用了非显示缓冲区（offscreen buffer），这一帧也许已经被渲染出来了，现在只是把图像复制到屏幕上面去而已）。任何没有被使用到的时间都可以用来进行额外的处理（参看可扩展性一段，应该如何使用这些多余的处理时间）。

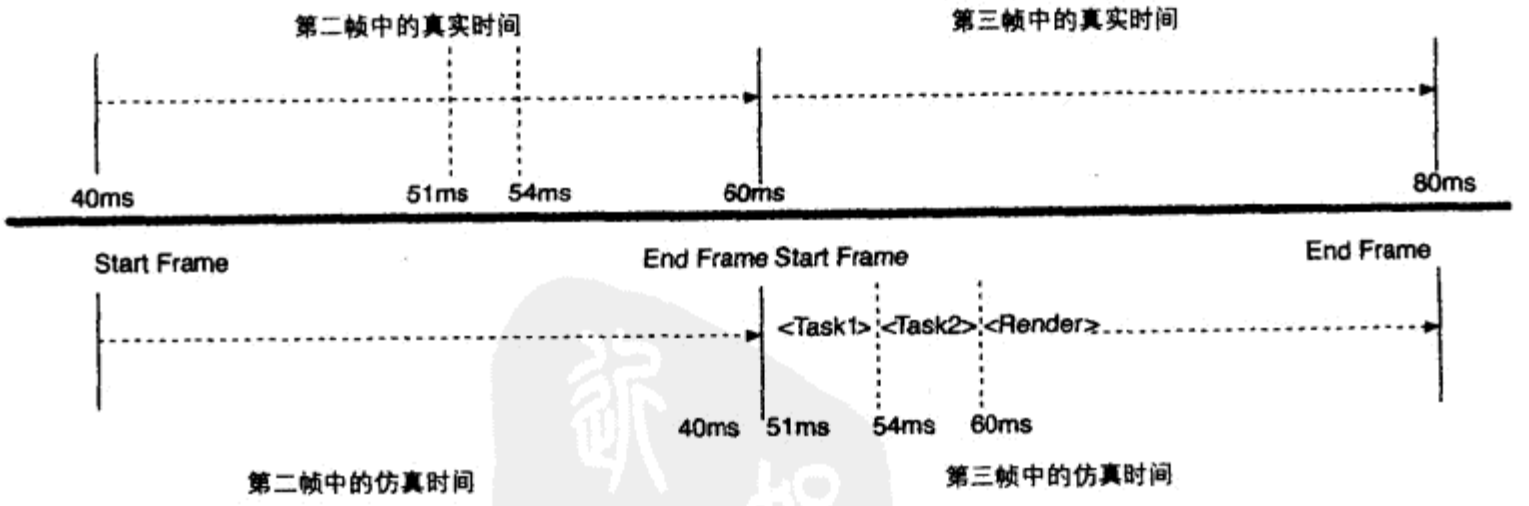


图 1.1.3 第二帧中真实时间与仿真时间的差异

Download at www.Pin5i.Com

在这种模式下面，任务的运行和每帧的渲染总会比真实的时间稍微慢一点儿。但是这对于用户来说是察觉不到的，而且这也使我们在变帧率时也能工作。如果帧率下降了，调度器可以做一些补偿以使得仿真看起来是以一个不变的速率运行的。如果帧率不能改变，调度器可以预知每帧的开始和结束时间，这样就可以事先处理好事件了。然而，如果机器的负荷非常重的话，调度器也不能补偿了，游戏就会变得更慢。

4. 事件的类型

帧事件是最简单的事件，它每 N 帧或者每帧（N=1）发生一次。它们一般在渲染的开始或者结束的时候发生。在另一方面，时间事件则是在仿真时间中发生的，也不一定是和帧同步的。举一个例子，一个时间事件可以独立于帧渲染的速度固定每 10ms 发生一次。也可以把时间事件和帧事件混合起来，例如，我们可以在每帧开始的 10ms 以后产生一个事件，或者在每帧产生 5 个事件，而事件之间的仿真时间相等。

5. 时钟

调度器的时钟组件是用来跟踪真实时间、当前的仿真时间和帧数的。时钟的精确度决定了整个仿真的精确度——一个 1ns 精度的时钟比一个 1ms 精度的时钟要精确得多。对于大多数需求来说，1ms 精度的时钟就足以满足要求了。如果需要更高的精度，开发人员需要使用 1ms 的硬件级时钟，然后根据需要把真实时间分得更细。也可以使用浮点时钟，不过使用的时候要很小心，要注意应该如何应付舍入误差。

6. 顺序

时间管理器负责事件的排序和产生。由于任务是由事件触发的，这里就有了一个自然的排序。举个例子，我们先定义两个任务：

Task1：每 5ms 从 5 运行到 15，普通优先级。

Task2：每 4ms 从 11 运行到 19，高优先级（参看 1.1.1 表）。

表 1.1.1　　任务执行顺序

时　间	任　务
5ms	Task1
10ms	Task1
11ms	Task2
15ms	Task2
15ms	Task1
19ms	Task2

在某些情况下，多个任务可能会设置在同一个时间运行。在这个例子中，Task1 和 Task2 都在时间点 15 上运行。由于 Task2 相比 Task1 有较高的优先级，它将先被执行。如果优先级相等或者系统没有使用优先级，它们就轮流运行。优先级对于基于帧的任务的排序也是有用的。

Download at www.Pin5i.Com

7. 任务管理器的细节

ON THE CD

由于有可能有数百个任务需要管理，任务管理器必须运作得非常聪明。使用蛮力搜索来查找下一个任务显然是很低效的。有很多方法可以实现这一点，本书附送的 CD-ROM 的例子程序使用的是排序列表的方法。任务将按照它们下次执行的时间进行排序存放在一个列表里面——列表的开头总是下一个需要执行的任务。事件管理器只需要查看第一个任务就知道下一个要发生的事件是什么了。当一个事件发生的时候，最前面的任务被弹出来然后执行，它下次执行的时间被更新，根据这个更新的下次执行时间它将再插入到列表中去。

除了可以避免耗时的搜索以外，这种方法还有个好处，那就是会频繁执行的任务将会停留在列表的最前面一段（就像一个高速缓存一样），不那么频繁运行的任务则会待在后面在适当的时间才冒出来。

我们经常需要动态地更改一个已经注册的任务的属性，这可能会牵涉到更改它的优先级、周期、持续时间或者甚至要求在它还没有结束的时候就将它删除。为了要能更新任务属性，我们必须使用一个外部的方法来找到它。可以使用一个唯一的注册 ID 来标识列表中的一个任务。

1.1.2 一个简单的调度器

ON THE CD

现在我们已经对调度器的多个相关概念和组件进行了讨论，下面就要演示如何创建和使用一个简单的调度器了。例子中的源代码在 CD-ROM 上面可以找到。供演示的调度器（Scheduler、Clock 和 ITask）也可以作为库来使用。另外还提供了两个客户程序（sample.exe 和 win.exe）。

1. 设计

调度器的设计主要集中在两个组件上面——调度器引擎本身和 ITask 插件接口（参看图 1.1.4）。

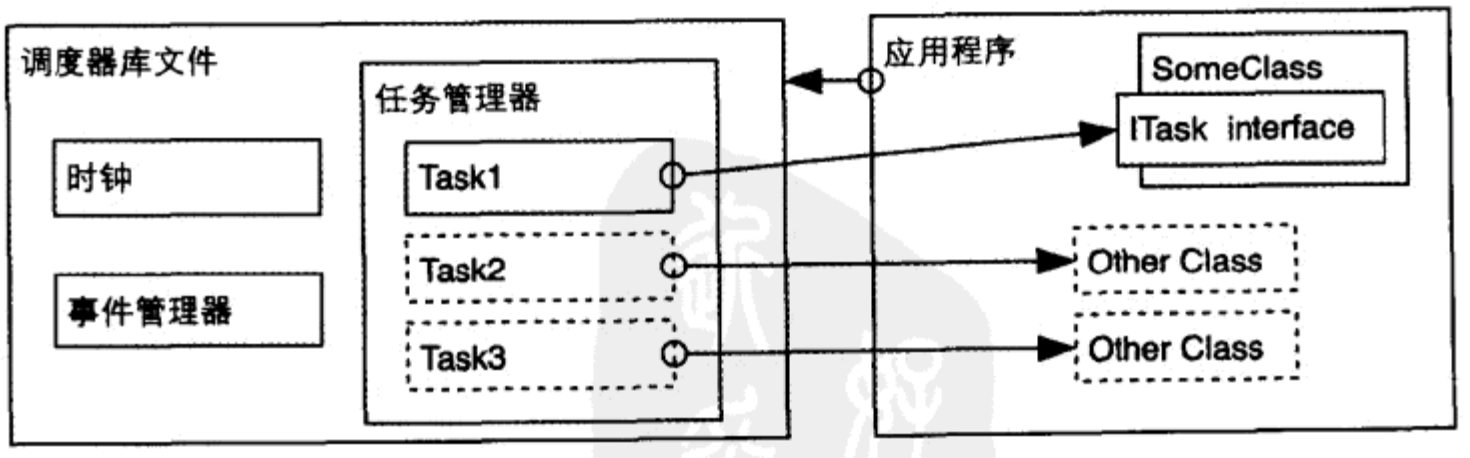


图 1.1.4 调度器的组件与客户端应用程序的关系

Download at www.Pin5i.Com

要使得调度器运行起来，必须要有一个调用它的程序。在一个非图形界面的程序里面，这只要求把它放在一个循环里面然后执行就可以了：

```
while (running)
    scheduler.ExecuteFrame();
```

有两种方法把调度器集成在一个消息驱动的图形界面（例如 Windows）上。第一种方法是修改消息循环来处理消息和调用调度器。这是一个最容易想到的方法，但是它有个缺点，就是当窗口大小改变的时候调度器会停止工作。

第二种方法是创建一个 Windows 时钟，利用时钟消息来调用调度器。由于时钟消息并不会被窗口的拖动打断，调度器就可以在后台连续运行了。在 CD-ROM 上提供了一个这样的例子程序（win）演示如何在一个 Windows 应用程序里面使用调度器。下面我们来更仔细地分析一下这个应用程序。

Windows 核心代码相当简单。调度器响应 WM_TIMER 消息运行，它有两种要调度的事件：一个用于每隔 15ms 就更新球的位置；而另一个是渲染事件，它把所有的球都写到一个非显示缓冲区里面去。然后 Windows 会在需要的时候把非显示缓冲区里面的内容复制到屏幕上去。仿真和渲染都没有牵涉到任何特别的 Windows 编程。这个例子演示了如何使用多个仿真任务和如何动态地增加删除任务。

游戏经常会有帧率变化的情况发生，这取决于系统的能力和负荷，但是运动物体的速度却能保持不变。此处两个例子程序的帧率是固定的，但提供的调度器可支持这种常量速率技术。这中间的关键就在于 `Clock.Update()` 方法，它对真实时间进行采样，然后通过流逝时间对仿真时间进行更新。如果一个物体在 60ms 里面移动了 N 个单位，我们就不用管系统究竟是渲染了两个 30ms 的帧还是三个 20ms 的帧了——物体在真实的时间里面移动了相同的距离，因此速度是一个常量。如果你希望要仿真物体的速度随着帧率的增减而增减，就需要改变 `Clock.Update()` 方法以使得它以固定的间隔前进，而不是读取真实时间的时钟了。

那么，到底调度器是怎么管理时间的呢？我们需要注册一些事件来看看它们是如何工作的。

2. 调度

调度任务的第一步就是要设定它究竟是时间事件，帧事件，还是渲染事件。下面的代码将设置一个从第 200 帧开始的事件，此事件每 3 帧运行一次，然后在第 210 帧之前结束（开始帧 200 + 持续时间 10）：

```
scheduler.Schedule(TASK_FRAME, 200, 3, 10,
pSomeHandler, pUserPointer, &id);
```

这个任务将会在 200 帧、203 帧、206 帧和 209 帧运行。当结束了最后一轮运行以后，任务就超时了，然后被任务管理器删除掉。设置持续时间为 0 的任务是永久性的，永不超时。在某些情况下面，你可能希望在一个任务结束以前就删除它，或者是可能想要手工结束一个永久性任务。要做到这一点，可以以它的任务 ID 为参数调用 `Terminate()` 函数。

帧是怎样更新的呢？每次当调度器的 `ExecuteFrame()` 方法被调用的时候，它首先会调用 `Clock.BeginFrame()`，这个方法会更新帧数，计算新帧的开始时间和结束时间，从而开始一

Download at www.Pin5i.Com

个新的帧。更新了帧数以后，它会执行所有的时间事件，将仿真时间设置为帧结束的时间，执行所有的帧事件，最后执行渲染任务（在这个例子程序里面，渲染任务是一种特殊的帧任务，它没有开始时间、周期和持续时间，它总是每帧执行一次）。

整个仿真过程可以用 Run()和 Stop()方法来停止或者重新启动。当调度器停止了以后再被重新启动的时候，它会计算流逝的时间，并把它从整个仿真时间里面减掉。停止的时候，调度器然仍然在调度渲染和帧事件，但是时间事件被挂起了。

1.1.3 高级概念

有很多种方法可以改进调度器或者是让它使用得更好，可扩展性、仿真和多线程是其中的几种。

1. 可扩展性

游戏开发中的一个常见问题就是可扩展性。游戏应该能利用所有可用的处理能力得到更好的效果，同时它也要能在较差的系统上正常工作，此时需要密集型计算的效果可以被减弱或者是完全关闭。游戏应该能在多任务环境里面表现正常——如果一个游戏占用了大部分 CPU 的话，它就可能会使得系统定期的对用户输入没有响应。如果操作系统在后台启动了维护性的任务的话，这也会把游戏给拖慢。理想状态下，游戏应该能动态地适应系统环境。

如果能收集到性能统计数据，那就已经成功一半了。由于调度器要处理所有的过程，它本身就有可能变成一个瓶颈，因此它是一个统计性能数据的好地方。

正如前面所说的，时钟会记下当前的时间，并把它和最后一帧相比得到流逝的时间。调度器也可以由比较开始时间和结束时间得到每个任务消耗的时间。这些信息可以用来和任务通信，也可以用来决定是否需要运行一个任务以及运行的频率。

提供扩展性的方法之一是要使用时间预算——越强劲就可以得到越多的时间。一个任务可能在每帧都有一个时间预算。调度器记录下总的预算和总的运行时间，只有当任务的当前预算超过真实时间的时候才会执行它。举个例子，一个任务可能在每帧有 2ms 的时间预算，但是它在一个慢速的 CPU 上面要运行 3ms。调度器就会每三帧跳过一帧以使得任务预算不超支（参见表 1.1.2）。

表 1.1.2 每个任务的时间预算

帧　序	时 间 预 算	实 际 时 间
第 1 帧	2	3
第 2 帧	4	6
第 3 帧	6	—
第 4 帧	8	9

某些任务可能会有一个时间预算的门限值——如果它们超过了预算，它们不会只在部分时间运行，而是根本不运行。通过把所有任务的时间加起来的方法，调度器也可以得知进行整个仿真需要的时间。在处理时间和帧长之间的差值就是空闲时间。理想状态下，游戏应该能利用所有可用的时间以改善游戏的效果，但同时也不应该超过可用的时间，不然帧率就会下降了。调度器可以通过增加任务来利用空闲时间，或者是删除任务以减少负荷（当然，必

Download at www.Pin5i.com

须要有个方法来判断哪些任务是可以安全地增加或者删除掉的）。向任务提供了系统总体利用率统计数据以后，任务就能够根据收到的数据进行自我调整，以使用更多或者更少的时间。最好要预留 5%到 10%的空闲时间，这样如果在真正的处理时间上出现了小的波动也不会使得系统变慢。

其他可以提供可扩展性的选择包括增加或减少时间预算、对空闲任务的调度（它们只在空闲的时候运行）、垃圾收集或者其他的维护性工作、增强图形效果或者人工智能的改进。当进行此类管理的时候，一定要避免进入极端之间的振荡状态[1]。要做到这一点，可以把调整量限制为较小的增量，避免大的跳跃，或者利用以前调整效果的统计分析来改善估测。

2. 仿真

调度器可以用来驱动仿真系统。为了实现动画和碰撞检测，大多数仿真引擎都将时间分成独立的小片。本节所说的调度器对于此类仿真系统来说非常适合。

举一个简单的月球登陆器的例子，登陆器有垂直速度和前进速度。每过一段时间重力就会增加。如果我们使用人工智能系统来操作登陆器的垂直喷射器，那么在每个时间片上，人工智能系统就会对速度进行一次采样，调整喷射参数作为补偿，从而能够有控制降落进行。这些时间片必须足够小以让人工智能系统及时反应——不然登陆器在得到有效的响应之前就撞到地面了。对于碰撞检测来说，你也会希望使用小的时间片以便让登陆器接触地面的时候就知道，而不会整个地穿过地面。

3. 多线程

调度器也可以管理子线程的执行[Caster01，Dawson01]。有很多原因会促使你这么做，例如某些任务在一个连续的过程里会比在一系列分离的事件里工作得更好[Otaegui01]。这些任务可以被编成一个线程，而调度器则可以控制这些线程能运行多长时间。这种方法可以真正做到抢先式多任务，并同时加上了时间预算的限制。

多处理器系统慢慢地变得越来越常见了，在不久的将来很有可能多处理器会变成一个标准特性。能利用多处理器的游戏在性能上将打败那些为单个 CPU 写的游戏。利用多处理器有个容易的方法，那就是把它变成多线程的，让操作系统来做把线程分配到各个处理器的工作。

一个多 CPU 调度器能同时激活数个线程以让它们同时运行。它也可以将事件处理器分配到特定的线程里面去，这样多个事件就可以被同时处理了。

1.1.4　结论

要使用调度器有诸多原因——可移植性、灵活性和对仿真的支持。一个高质量的调度器应该灵活而有效。本节谈及了调度器的一些基本概念，提供了一个调度器的示例，还展示了如何把它集成到传统的和图形界面的应用程序里去。在下一个游戏里使用调度器来帮你组织事件吧。

译者注[1] 振荡状态指的是动态系统的状态变化得太频繁，表现得不稳定，这一般是没有使用双门限值或者是调整的数量太大导致的。例如一个恒温箱如果只有一个 27℃的门限，当温度低于 27℃的时候就加热，高于 27℃的时候就不加热，它就很容易陷入振荡状态。因为刚加热超过了 27℃就停止加热的话马上又会降到 27℃以下又进行短暂的加热……如此反复，形成一个振荡。

Download at www.Pin5i.Com

1.1.5 参考文献

[Bourg01] Bourg, David M., *Physics for Game Developers*, O'Reilly, 2001.

[Carter01] Carter, Simon "Managing AI with Micro-Threads," *Game Programming Gems 2*, Charles River Media, Inc., 2001.

[Dawson01] Dawson, Bruce "Micro-Threads for Game Object AI," *Game Programming Gems 2*, Charles River Media, Inc., 2001.

[Llopis01] Llopis, Noel, "Programming with Abstract Interfaces," *Game Programming Gems 2*, Charles River Media, Inc.,2001.

[Mirtich00] Mirtich, Brian, "Timewarp Rigid Body Simulation," *Computer Graphics Proceedings*, SIGGRAPH 2000: pp.193～200.

[Otaegui01] Otaegui, Javier, "Linear Programming Model for Windows-Based Games," *Game Programming Gems 2*, Charles River Media, Inc., 2001.

Download at www.Pin5i.Com

1.2　一个基于对象组合的游戏架构

Scott Patterson
Next Generation Entertainment
scott P@tonebyte.com
scott@gameframework.com

本节将要展示一个基于对象组合的游戏设计架构，然后阐述使用它的优点和其设计思想。我们将要说明为何此种架构对实现游戏会有帮助。

此游戏架构可以作为一个参考为开发人员自己的游戏所用。你可以在其上创建拥有你所需要效果的新系统，也可以增加拥有你所需要的行为的新任务。

在此处，我们谈及的开发架构实际上可以说成是一个系统，此系统由一组对象合作组成，而且能提供某种服务。一个应用程序架构则是指能为创建应用程序提供必要服务的一组类的集合。本节的目标就是要找出为了开发一个游戏的应用程序，我们到底需要创建什么样的架构。

使用架构来创建应用程序有诸多有力的理由。一个主要原因就是可以让系统先跑起来。毕竟，时间就是金钱。架构一般说来已经包含了一些内建的特性、一致的行为表现和结构，还有成熟的用于对象访问、对象所有权和对象生命期的规则。

在本节我们将首先总结一下游戏开发的各个阶段，以对需要做的工作有个总体的把握。然后我们将讨论游戏架构设计中的一些问题。最后我们将展示本书附送的 CD-ROM 上的一个游戏架构实现的大体情况。

1.2.1　游戏开发的各个阶段

随着开发阶段的不同，对游戏架构的需求也会有所不同。表 1.2.1 列出了一个典型的游戏开发阶段列表和每个阶段的一般目标。

表 1.2.1　游戏开发中的典型目标

阶　段	目　标
概念阶段	美感与功能性设计。角色，情节与任务概念的创建
原型阶段	通过概念性的演示来展示玩游戏时的关键因素。技术演示

Download at www.Pin5i.Com

续表

阶　段	目　标
可玩性阶段	至少一个任务或者一个难度，从头玩到尾的演示
生产阶段	完成所有任务和难度的设计和实现
产品包装阶段	不同游戏模式和显示的集成。包括情节片断、玩家培训、任务/难度的选择、游戏的输赢、状态和积分、暂停和重新开始、选项和配置
测试阶段	解决设计和实现中的问题。解决兼容性问题。不同驱动的集成
发布阶段	将游戏在第一个平台上面发布。欢庆晚会
移植阶段	不同语言的版本。不同平台的版本

在早期阶段（概念、原型、可玩性阶段），架构的主要焦点将集中在如何让程序先跑起来上面。在这些阶段，与创建概念性的演示对比，架构有可能看起来不那么重要。然而，如果架构的设计没有为以后阶段的游戏开发着想的话，以后你就会发现自己已经没有时间来做游戏的重构了。

在生产阶段，查看器和编辑器一类的工具将很有用。查看器可以用来让开发人员看到自己的内容在游戏中会是什么样子的。编辑器可以使得开发人员能对游戏的各个方面进行调整。虽然此类的工具可以和游戏应用本身分离开来，但是一般会要求把查看器和编辑器的功能集成到游戏中。要做到这一点，我们必须同时为顾客和开发人员编码。其目标是要创建如此的一个架构，它能增加共享组件的特性，也能把它们的开发相对独立出来。

在产品包装阶段，我们必须把所有的游戏模式和显示集成为一个无缝的产品。这个集成过程有时候会为日程延迟或者原设计方案的修改所累。如果有一个架构能管理这些游戏模式和显示，甚至管理它们之间的转化的话，那它就会让这个过程进行得更加顺利。

在测试阶段，我们可能需要能在不同的模式或某些点进入游戏。如此就需要我们的架构拥有此类的灵活性，能把设置这些模式和进入点的工作变得容易起来。我们也有可能需要在游戏中能切换不同的驱动。如果我们的架构把游戏和某种固定的视频技术绑定了，那么切换视频驱动就不可能了。不管我们是否需要提供驱动切换的功能，我们至少要在架构中间加入日志功能以协助兼容性测试。

当我们到达发布阶段以后，游戏开发小组可能还要继续进行游戏的移植，或者会有其他的开发人员来做移植工作。不管如何，如果我们的架构移植到其他平台很困难的话，这就会造成延迟。我们希望移植小组的时间是花在为每个平台开发新特性和特性增强上面，而不是花在研究如何让它跑起来的问题上面。

1.2.2 游戏架构设计

现在我们对开发游戏所需要做的工作已经有了一个概念，下面就要看看创建一个架构中的设计问题了。我们将讨论平台相关性、游戏相关立性、对象组合、继承、基于帧的编码、基于函数的编码、操作顺序、对象生命期，还有任务的集成。

1. 平台独立性与平台相关性

游戏经常会使用很多独立于操作系统和平台技术的概念。这些概念决定了玩家在游戏中

Download at www.Pin5i.Com

和更深层次上的愉悦感。从另一个方面来说，游戏一般也会使用特定的硬件特性来增强游戏的表现力。这种表现力会使游戏拥有一种令人沉浸于其中的感觉。

架构是不应该依赖于操作系统和其技术的。我们可以为架构设计一套独立于平台的系统接口，而不是依赖于平台的系统接口。虽然接口本身是平台独立的，但是我们可以使用工厂系统来创建特定平台的具体实现。

我们游戏中概念性的工作和平台相关的东西距离越远，也就越容易在转化的时候把平台相关的代码给替换掉。因此，我们创建一个游戏架构的部分目标就是要在任何可能的时候把下面的事情变得更容易一点，那就是要把游戏的高层次概念和操作系统还有平台技术给独立开来。

2. 游戏独立性与游戏相关性

我们如果想让架构在多个游戏里面都能使用，那就需要让架构拥有游戏独立性。然而，如果我们只是要在一个游戏的几个平台上面使用架构的话，那也可以让部分架构和游戏有一些相关性。

举个例子，如果游戏控制了一个特定类型的角色，根据角色状态的不同，它将表现出很多动态的视觉效果，我们的渲染代码可能需要访问游戏的相关状态，然后决定是否此角色需要渲染。这种做法可以减少系统接口的调用，从而简化代码并使速度提高。

3. 对象组合与对象继承

要想使架构的结构良好，方法之一就是使用模板方法这个设计模式，然后只创建一个应用类的派生类实例。如果这样做，我们就是要把类的初始化和销毁代码设计成其派生类可以重载的算法。这种设计模式被称为是基于类的，因为它使用了类继承的方法来得到不同的行为。

另外一个设计应用程序初始化和销毁步骤（不使用继承）的方法是将其步骤分成一系列任务来处理。一个任务系统类可以用来调整任务的执行，一个资源系统类可以用来定位和管理任务列表和任务对象。现在我们使用的是基于对象的模式，因为我们使用了对象组合的方法，而其中我们的资源系统则成了这些对象的中介者。

使用这样的任务系统意味着我们可以使用对象组合而不是继承的方法来创建一个架构。我们的任务系统通过任务的接口控制任务对象，而我们的任务则通过调用对象接口来工作。这同时也意味着我们的架构不会有一个倒置的控制结构，而这往往是使用模板方法所固有的特性。取而代之的是我们的任务对象会来控制软件。或许这个架构可说成是一个对象架构，而不是一个类架构，但毫无疑问它是一个架构。

《设计模式》[GoF94]这本书上讨论了很多对象组合的优点，也着重提出了面向对象设计的两个原则：

- 对于接口编程，而不是对实现编程。
- 尽量采用对象组合而不是类继承。

4. 基于帧的和基于函数的设计

有很多类型的软件对于帧定时并不关心，在这些软件中函数可能要花费数秒，数分钟或

Download at www.Pin5i.Com

者更长的时间才能完成。这种基于函数的操作比基于帧的操作要更容易编程实现。

绝大多数游戏都必须有音频和视频方面的响应，还要在每帧里面计算许多动画和细节。每当一幅视频图像以及一段音效缓冲被渲染的时候，我们也就创建了一帧。游戏可能以每秒50帧或者60帧的速度进行渲染。这种基于帧的操作要求游戏分在很小的时间片断上面执行。如果要进行一个耗时操作（超过了1/60s），它就必须被分成更小的片断或者是作为一个后台任务来运行。

对于我们的架构来说，基于帧的操作需要一个帧系统类，这个类可以通知我们的任务系统类什么时候应该调用与帧同步的任务。我们的任务系统也要能处理非帧同步的任务，这些任务我们称之为“异步任务”。

由于帧系统控制了什么时候应该调用帧同步的任务，我们也可以提供手工播放每帧和选择特定帧率的功能了。这既可以帮助检查动画播放效果的细节，也对其他的调试和测试有用。

5. 动态和静态操作顺序

有很多类型的软件操作需要以一种特定的顺序执行。对于这些操作，我们必须以预定的顺序来调用函数，或者使用预定的顺序把任务提交到我们的任务系统里面去。这是一个静态操作顺序的例子。

游戏总是由一组相互关联的显示和它们之间的转移组成的，它们之间一般没有预定的顺序。取而代之的是由玩家来决定下一步将会发生什么。这就是一个动态操作顺序的例子。

我们让任务对象能访问任务系统和提交新任务，这样我们的架构就提供了一个动态操作的顺序。

6. 动态和静态对象生命期（以及所有权）

在一个层次化的系统里面，我们可能会发现父类会拥有一些为其派生类所使用的对象，而派生类也会创建一些其父类一无所知的对象。通常这些对象的生命周期是由层次化的结构内定的，而且继承的子类也无法动态地创建和删除它们。在这种意义上，这些对象的生命期就是静态的。

在一个基于对象组合的架构里面，想要知道什么时候一个对象拥有另一个对象就比较乱了。为了解决这个问题，我们可以将对象所有权的管理责任分给资源系统来做。如此一来，每个任务都可以随时按需连接系统资源而不必承担管理的责任了。我们把对象所有权的管理责任交给资源系统，把对象访问的责任交给对象，这样就可以避免所有权方面的混淆。

我们可以在资源系统里面增加任务命令来使得这些对象的生命期动态化。我们可以要求资源系统以集合的方式取出和存入一批对象。例如，在需要载入对象的任务开始之前，我们可以发出一个取出集合的命令，在这些任务结束了以后，我们可以再发出一个存入集合的命令。另外一个方法就是我们在应用程序的整个生命期都持有对象以时刻备用。

7. 任务的水平集成和垂直集成

在一个层次化的系统中间，有可能看上去总有某些对象在控制着我们，而且我们与其他

Download at www.Pin5i.Com

对象的关系是固定不变的，感觉任务就像是垂直组织起来的，任何系统高层的改变都会对系统底层的部分带来深远的影响。

在一个对象组合的系统里面，我们的程序结构看上去比较扁平，而且我们与其他对象的关系也更加动态化，感觉任务更像是水平式组织的，对于系统里某些对象的改变对于其他的系统里面的对象影响很小或者根本没有。

1.2.3　游戏架构实现

现在我们应该对怎样实现架构满足上面的需求有一个总体概念了。此架构由系统和任务组成。有种特殊的被称为“帧播放器”的任务可以用来提供高层的音视频渲染和逻辑的控制。

1. 系统

`System_t` 类包含了指向我们系统要使用的纯接口[Stroustrup97]的指针。为了进行动态的系统切换和把访问系统的平台独立的代码与平台相关的代码分离开，我们就可以选择使用纯指针来访问系统。在表 1.2.2 里列出了所有由 `System_t` 类提供的接口。

表 1.2.2　　System_t 类的所有接口

系　统	简　介
LogSys_t	处理游戏中所有的消息纪录，可以选择的输出方式包括文本框或文件
ErrorSys_t	处理所有的错误消息和状态
TimeSys_t	提交时间信息
FactorySys_t	使用工厂 ID 创建对象
ResourceSys_t	使用实例 ID 管理对象实例
TaskSys_t	管理任务的执行和控制
WindowSys_t	提供窗口系统的管理和控制
FrameSys_t	提供帧同步服务和控制
InputSys_t	提供输入设备的管理和控制
VisualSys_t	提供视觉系统的管理和控制
AudioSys_t	提供音频系统的管理和控制
NetworkSys_t	提供网络系统的管理和控制

每个系统都有一个 `Init(System_t *pSystem)` 和一个 `Shutdown()` 方法。把 `System_t` 的指针传递给对象后，它们就可以访问任何系统接口了。在代码中加入 `System_t` 类的声明并不会使程序在编译的时候产生依赖于系统类的代码，因为系统都是通过指针来访问的，这只需要在源代码前面声明一下就可以了。这一点非常重要，因为架构的很多类里面都用到了 `System_t` 类的指针。对于良好的物理设计[Lakos96]来说，减少物理相关性是一个重要的目标。

由于每个系统都被声明为了一个纯接口，完全隐蔽了实现细节，因此我们可以动态地选择系统实现，当然前提是这些实现不是静态相关地连接在一起的。把相互依赖的实现分解成动态载入的组件是包模式[Noble01]的一个例子。

Download at www.Pin5i.Com

在 CD-ROM 上面的源代码演示了如何动态地切换虚拟系统。这是通过动态连接库实现的，每个库都提供了 VisualSys_t 接口的一个不同实现。下面就是一个例子，演示了如何控制视觉系统的切换：

```
FactorySys_t *pFS = m_pSystems->GetFactorySys();
pFS->DeleteVisualSys( m_pSystems->GetVisualSys() );
pFS->SetVisualSysDriverID( m_nVisualSysDriverID );
m_pSystems->SetVisualSys( pFS->CreateVisualSys() );
```

2. 任务

TasksSys_t 类提供了一个任务系统的接口。通过使用 Post_TaskCommand 函数，我们可以将一个任务设置为帧同步的任务或者是异步任务。唯一的区别就在于任务被调用的时间：当帧系统通知下一帧应该开始的时候，帧同步任务才会被调用；而异步任务则在任务系统的每次循环里面被调用。

```
// get the task system
TaskSys_t *pTaskSys = m_pSystems->GetTaskSys();
// get the resource system
ResourceSys_t *pResSys = m_pSystems->GetResourceSys();
// remove the current asynchronous task
pTaskSys->Post_TaskCommand( ASYNC_REMOVE, this );
// get the new task to start
Task_t *pTask = pResSys->GetTask( INSTANCE_ID_TASK_INTRO );
// push back the new frame-synchronized task
pTaskSys->Post_TaskCommand( FRAMESYNC_PUSH_BACK, pTask );
```

在此处可以看到，我们可以通过任何任务的实例 ID 调用资源系统的 GetTask 函数来访问这个任务。而当任务通过其 Connect(System_t *pSystem)函数连接到系统的时候，任务就可以得到相应的 System_t 的指针了。

3. 分层

在整个游戏开发过程中，通常有很多工作都是与视觉渲染设计有关的，而渲染可以使用分层系统在一个高层次上进行管理。当屏幕上的内容以层的模式来渲染的时候，分层系统的优点就体现出来了。例如，在一个 3D 游戏中，我们可能将环境作为一个层来渲染，而将游戏中的物体分为另一个层，然后是一个信息显示[1]作为第三层。我们希望能将新的层叠加在场景上，让其相应地改变视频、音效和输入处理逻辑。

为了控制视觉显示、音频数据和输入处理逻辑的各个层，我们引进了一种特殊的称之为 FramePlayer_t 的任务。这个任务需要和帧同步，且能管理音视频层(AVLayer_t)对象和逻辑(LogicLayer_t)对象。

音视频层对象在每帧中以如下的方式被调用：

```
// Update AV Layers
```

译者注[1] 所谓的信息显示层（heads-up display layer）指的是在游戏中屏幕上显示玩家当前统计数据的层，例如在这一层上面可以显示玩家的分数和生命值等。这个说法来自军队，原来是指战机飞行员驾驶的时候可以将敌机的信息显示在其视野中，这样飞行员就可以不用低头看仪表而保持抬头状态了（keep his head up）。

Download at www.Pin5i.Com

```
for( each audio-visual layer (forward-order)  )
{
   AVLayer_t *pAVL = contents of iterator;
   pAVL->Update();
}

// Begin Render Visual
if( m_pSystems->GetVisualSys()->BeginRender() )
{
for( each audio-visual layer (forward-order) )
   {
      AVLayer_t *pAVL = contents of iterator;
      pAVL->RenderVisual();
   }
   m_pSystems->GetVisualSys()->EndRender();
}
// End Render Visual

// Begin Render Audio
if( m_pSystems->GetAudioSys()->BeginRender() )
{
   for( each audio-visual layer (forward-order) )
   {
      AVLayer_t *pAVL = contents of iterator;
      pAVL->RenderAudio();
   }
   m_pSystems->GetAudioSys()->EndRender();
}
// End Render Audio
```

我们可以看到每个音视频层在更新的时候首先会调用 `Update()`。这就是依据音视频对象运动的状态更新它们的时候。在这个更新调用以后，我们就会渲染音视频了。

逻辑层对象在每帧中以如下的方式调用：

```
// Update Logic Layers
for( each logic layer (reverse-order) )
{
   LogicLayer_t *pLL = contents of iterator;
   pLL->Update();
   if( pLL->IsExclusive() ) break;
}
```

我们可以看到，每个逻辑层的更新只要简单地调用 `Update()` 就可以了。对于音视频层来说更新就是意味着处理动画，对于逻辑层来说则是处理游戏逻辑和玩家的输入。

由于逻辑层是反过来处理的，而且如果把它们标识为独占的时候，处理就会停止，因此在 `m_LogicLayerPtrList` 里面的最后一个逻辑层可以改写前面逻辑层的结果[1]。所以如果增

译者注[1] 这段话有点曲折。它可以如下举例说明：当一个用户在界面上处理一个多层菜单的时候，最后一层菜单的结果显然是决定性的。而在处理这个菜单的时候，它就被标志为了独占的（exclusive），此时其他的菜单都不会被处理。而所谓反过来处理则是指的逻辑层的出现顺序和处理优先级显然不一样，最后出来的结果才是最重要的。

Download at www.Pin5i.Com

加一个逻辑层的话，我们就可以完全改变用户输入处理的方式了，其中包括游戏菜单、查看器和编辑器。与之形成对比的是，如果我们添加一个新的音视频层的话，我们只不过是添加了一套新的可看和可听的东西而已。

正如任务系统有任务命令一样，FrameLayer_t 类也有层命令。下面是一个例子，展示了如何发布一个层命令：

```
AVLayer_t *pAVL;
LogicLayer_t *pLL;
// get the resource system
ResourceSys_t *pResSys = m_pSystems->GetResourceSys();
// get the task to modify
Task_t *pTask = pResSys->GetTask(INSTANCE_ID_TASK_INTRO);
// we know it is a frame player
FramePlayer_t *pFP = (FramePlayer_t *)pTask;
// push back an audio-visual layer
pAVL = pResSys->GetAVLayer(INSTANCE_ID_AVLAYER_INTRO);
pFP->Post_AVLayerCommand(PUSH_BACK, pAVL);
// push back a logic layer
pLL = pResSys->GetLogicLayer(INSTANCE_ID_LOGICLAYER_INTRO);
pFP->Post_LogicLayerCommand(PUSH_BACK, pLL);
```

此处我们可以使用任何音视频层的实例 ID 调用资源系统的 GetAVLayer 函数得到此层。类似地，我们也可以使用任何逻辑层的实例 ID 调用资源系统的 GetLogicLayer 函数得到此逻辑层。而当层通过其 Connect(System_t *pSystem)函数连接到系统的时候，它就可以得到相应 System_t 的指针了。

对于此分层系统，我们现在可以动态地改变音视频层。所有在产品包装阶段提及的不同的游戏模式和显示都可以使用层命令和任务命令来实现。例如，在需要的时候我们可以为一个信息显示层添加一个音视频层。类似的，我们也可以为浮动菜单添加相应的音视频层和逻辑层。当创建新的模式和显示的时候，我们就可以选择到底是要使用层命令来改变自己的帧播放器任务，还是使要用前面说的任务命令来在不同的帧播放器中切换。

最后，在转化中有一个复杂的层和任务的操作。一个游戏显示上的物体（第一个音视频层）可以逐渐的被另外一个游戏显示（第二个音视频层）的物体覆盖住。当这种转化结束的时候，只有第二个层可以被看见，这时第一个层就应该从音视频处理过程中被删除掉。

1.2.4 源代码

在 CD-ROM 上面有游戏架构实现的源代码和一些其他的文档。本书的代码着重于游戏架构的概念，而不是着眼于游戏技术的概念。在网址 http://www.gameframework.com 有更新的源代码。图 1.2.1 说明了源代码里面模式、任务和层的实现。

Download at www.Pin5i.com

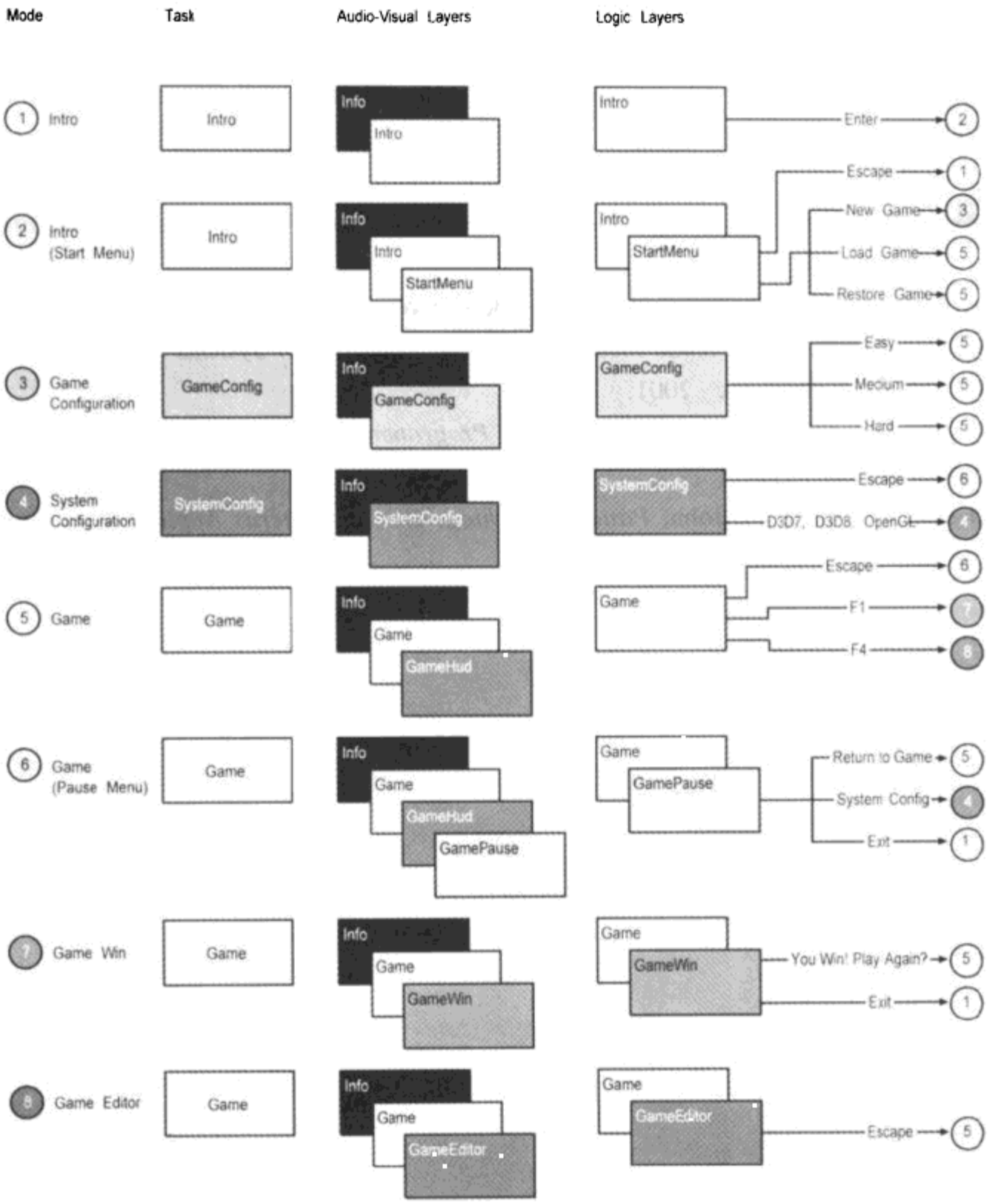


图 1.2.1　使用游戏架构：一个通过任务、音视频层与逻辑层实现游戏中各种状态的例子

1.2.5　参考文献

[Boer00] Boer, James, "Object-Oriented Programming and Design Techniques," *Game Programming Gems*, Charles River Media, Inc., 2000: pp.8～19.

[Boer00] Boer, James, "Using the STL in Game Programming," *Game Programming Gems*, Charles River Media, Inc., 2000: pp.41～55.

[GoF94] Gamma, E., et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison Wesley Longman, Inc., 1994.

Download at www.Pin5i.Com

[Lakos96] Lakos, John, *Large-Scale C++ Software Design*, Addison Wesley Longman, Inc., 1996.

[Llopis01] Liopis, Noel, "Programming with Abstract Interfaces," *Game Programming Gems 2*, Charles River Media, Inc., 2001: pp.20～27.

[Meyers96] Meyers, Scott, *More Effective C++*, Addison Wesley Longman, Inc., 1996.

[Meyers98] Meyers, Scott, *Effective C++*, Second Edition, Addison Wesley Longman, Inc., 1998.

[Meyers01] Meyers, Scott, *Effective STL*, Addison Wesley Longman, Inc., 2001.

[Noble01] Noble, James, *Small Memory Software: Patterns For Systems with Limited Memory*, Addison Wesley Longman, Inc., 2001.

[Stroustrup97] Stroustrup, Bjarne, *The C++ Programming Language*, Third Edition, Addison Wesley Longman, Inc., 1997.

[Vlissides98] Vlissides, John, *Pattern Hatching: Desing Patterns Applied*, Addison Wesley Longman, Inc., 1998.

Download at www.Pin5i.Com

1.3　让 C 中的宏重现光辉

Steve Rabin
任天堂（美国）公司
steve@aiwisdom.com

C 语言中底层的宏（#define）是一个强大的工具，但是它却被人严重误解了。很多人还有一些书籍都认为它过时了，而且用之有害，早就被 C++中的内联函数给取而代之了。不幸的是，这种说法太片面了，它没有考虑到宏拥有的独特功能。可悲的是，很多 C++的书籍都没有讲清楚宏的基本特性，甚至有的根本就不承认这一点。然而，在 C 的经典书籍[Kerni ghan88]中，我们仍可以找到有关这方面充分的阐述。

本质上来说，宏只是对编译器的预处理器做了一些指示，进行了一些聪明的文本替换工作。因此，类型检查或者任何其他的安全性检查在这一步都不会发生，这就是给很多开发人员添加麻烦的地方。为了避免这一点，有一条基本原则，那就是不要使用宏来创建有函数功能的行为或者常量。如果想要知道宏可能产生的错误，请参考[Dalton01]、[Hyman99]和[McConnell93]。

本节并不是来讨论宏的这些问题，我们的目标是要表明宏在很多方面都是有用而被需要的。宏能够在真正编译之前进行一些有趣的文本替换，这就是下面很多技巧可以利用到的地方。请注意，在下面的例子中间出现的代码在本书附送的 CD-ROM 上面都有。

1.3.1　声明

使用宏的时候很容易让开发人员的编程习惯变坏。但是，这并不是本节关注的焦点。虽然各人的理解不同，但是公认的看法认为最好不要利用宏给 C/C++添加新的语言特性。应该让任何一个加入项目的新成员都能以轻松读懂你原来的程序，因此，在你阅读这些技巧的时候，请一定要记住衡量一下其中每一个的利弊。一定要谨慎地使用这些例子，或许这才能使你从本节获得最大的收获。

1.3.2　第 1 个宏技巧：把枚举值转化为字符串

有两个特殊的操作符可以让你在一个宏里面进行巧妙的文本替换。第一个是#，它的作用是给其后的参数添加一对双引号。如下所示：

Download at www.Pin5i.Com

```
#define CaseEnum(a)   case(a): LogMsgToFile(#a, id, time)

switch( msg_passed_in )
{
   CaseEnum( MSG_YouWereHit );
        ReactToHit();
        break;

   CaseEnum( MSG_GameReset );
        ResetLogic();
        break;
}
```

在编译器进行了预处理以后，上面的代码就会变成如下形式：

```
switch( msg_passed_in )
{
   case( MSG_YouWereHit ):
      LogMsgToFile( "MSG_YouWereHit", id, time );
      ReactToHit();
      break;

   case( MSG_GameReset ):
      LogMsgToFile( "MSG_GameReset", id, time );
      ResetLogic();
      break;
}
```

利用这个宏的模式，你可以轻松而可靠地把枚举值转化为有意义的输出，例如字符串，输出到日志文件里或者显示到屏幕上。如果不使用这样的技巧，你就需要自己做一个枚举值一字符串的查找表了。不过，查找表一般来说维护性很差，而且也不可靠。另一个解决方案是完全使用字符串，而不使用枚举值，但是在游戏里面适不适合使用字符串比较进行匹配还很难说。因此，这个宏技巧是一个既快速又可靠的安全解决方案。

另外一个此操作符的用法是可以把一组枚举放在一个头文件里面，然后在使用的时候把它放在不同的列表里面以同时得到枚举和字符串。下面的代码展示了具体的作法：

```
// data.h

DATA(MSG_YouWereHit)
DATA(MSG_GameReset)
DATA(MSG_HealthRestored)

// data.cpp

#define DATA(x) x,

enum GameMessages
{
   #include "data.h"
```

Download at www.Pin5i.Com

```
};

#undef DATA
#define DATA(x) #x, // make enums into strings

static const char* GameMessageNames[] =
{
    #include "data.h"
};

#undef DATA
```

1.3.3　第 2 个宏技巧：利用二进制表达式得到编译期常量

另外一个特殊操作符是##。这个操作符能让你把两个参数连接起来，如下所示：

```
#define cat(a,b)   a ## b

value = cat( 1, 2 );
```

预处理器会把上面的语句转化为：

```
value = 12;
```

虽然这个简单的例子并没有什么实际用处(仅仅是概念性演示用)，但是可以看见##操作符确实有一些很有趣的用处。下面的技巧就可以让你利用二进制表达式创建编译期的常量。这里特别有趣的是所有下面的代码都是在预处理器里面处理的，没有产生一行运行时的代码(特别感谢 Jeff Grills，是他提供了这些代码实现)。

下面是用法：

```
const int nibble = BINARY1(0101);       // 0x5
const int byte   = BINARY2(1010,0101);  // 0xa5

// 0xa5a5a5a5
const int dword = BINARY8(1010,0101,1010,0101,1010,0101,1010,0101);
```

下面是宏的源代码：

```
#define HEX_DIGIT_0000 0
#define HEX_DIGIT_0001 1
#define HEX_DIGIT_0010 2
#define HEX_DIGIT_0011 3
#define HEX_DIGIT_0100 4
#define HEX_DIGIT_0101 5
#define HEX_DIGIT_0110 6
#define HEX_DIGIT_0111 7
#define HEX_DIGIT_1000 8
#define HEX_DIGIT_1001 9
#define HEX_DIGIT_1010 a
```

Download at www.Pin5i.Com

```
#define HEX_DIGIT_1011 b
#define HEX_DIGIT_1100 c
#define HEX_DIGIT_1101 d
#define HEX_DIGIT_1110 e
#define HEX_DIGIT_1111 f

#define HEX_DIGIT(a)        HEX_DIGIT_ ## a

#define BINARY1H(a)         (0x ## a)
#define BINARY1I(a)         BINARY1H(a)
#define BINARY1(a)          BINARY1I(HEX_DIGIT(a))

#define BINARY2H(a,b)       (0x ## a ## b)
#define BINARY2I(a,b)       BINARY2H(a,b)
#define BINARY2(a,b)        BINARY2I(HEX_DIGIT(a), HEX_DIGIT(b))

#define BINARY8H(a,b,c,d,e,f,g,h) (0x##a##b##c##d##e##f##g##h)
#define BINARY8I(a,b,c,d,e,f,g,h) BINARY8H(a,b,c,d,e,f,g,h)
#define BINARY8(a,b,c,d,e,f,g,h)  BINARY8I(HEX_DIGIT(a), \
   HEX_DIGIT(b), HEX_DIGIT(c), HEX_DJGIT(d), HEX_DIGIT(e), \
   HEX_DIGIT(f), HEX_DIGIT(g), HEX_DIGIT(h))
```

1.3.4 第 3 个宏技巧：给标准断言添加描述性注释

标准 windows 断言（在 assert.h 中定义的）本来就是一个宏。它非常有用，甚至对于一个好的软件工程来说是不可或缺的——但是它仍然可以改进。最大的改进是可以给它添加一个描述性的字符串，这样一来当一个断言被触发的时候就可以显示一条有意义的消息了。

利用如下的宏，你可以很容易地对标准的断言进行扩展以加入一个描述性字符串：

```
#define assertmsg(a,b) assert( a && b )
```

使用方法演示：

```
assertmsg( time > 0, "Trigger::Set - The arg time must be > 0" );
```

当 time 小于或者等于零的时候，断言会被触发，内嵌的消息就会被显示出来成为断言失败的一部分。你可以在书籍[Rabin00]里得到关于此宏和其他断言技巧更多的信息。

1.3.5 第 4 个宏技巧：编译期断言

有时候，你可能需要一次程序创建（build）活动在编译期时，如果某个特定条件没有满足，就编译失败。虽然各个项目大小不一，但是一般说来这可能会节省很多时间并减少很多麻烦。下面的宏就可以使你在编译期检验一条语句，在效果上相当于一个编译期的断言。

```
#define cassert(expn) typedef char __C_ASSERT__[(expn)?1:-1]
```

例如，如果你在开发一个跨平台的软件，那你可能需要在编译期检查是否所有的枚举类

Download at www.Pin5i.com

型都是和一个无符号整型占据的内存是一样大小的。假设这个枚举类型叫 MyEnum，你就可以这样检查：

```
cassert( sizeof(MyEnum) == sizeof(unsigned int) );
```

如果 cassert 探测到了一个值为 false 的语句，它就会在编译期失败，因为此时它试图定义一个具有负数大小的字符数组，而定义一个负数长的数组则会产生一个编译错误，从而就能立即停止创建了。

1.3.6　第 5 个宏技巧：得到一个数组里面的元素个数

有时候程序需要知道一个数组里面元素的个数，然而，并没有直接的方法可以做到这一点。这是因为这个属性并不是单独存储的——而是在数组初始化的时候才知道的。

要想得到一个数组里面元素的个数，只需要把数组占有的内存大小除以数组里面一个元素的大小就知道了。举个例子，如果整个数组的大小是 120 个字节，而每一个元素拥有 12 个字节，则数组里必然有 10 个元素。每个元素的大小，以字节计，是在编译期就可以知道的，下面的宏就能很简洁地得到数组中元素的个数。

```
#define NumElm(array) (sizeof(array) / sizeof((array)[0]))
```

1.3.7　第 6 个宏技巧：在一个字符串中间加入__LINE__

缺省情况下 C 中已经有一些很有用的宏了，它们包括：

```
__LINE__     //在此变量出现的地方的程序行号
__FILE__     //程序的文件名
__DATE__     //程序编译的日期
__TIME__     //程序编译的时间
```

这些宏主要的用处在于可以用来记录帮助调试的信息。例如，当程序里面添加了一个断言的时候，它就利用了__FILE__和__LINE__这两个宏。万一断言被触发了，这些值就会被显示出来，这样一来，你就可以知道出现问题的具体的文件和所在行数了。

由于这些信息通常是用来打印字符串的调试信息的，下面就列出了一些利用__FILE__字符串和__LINE__整数的宏，它们把它俩给合在了一起：

```
#define _QUOTE(x) # x
#define QUOTE(x) _QUOTE(x)
#define __FILELINE__ __FILE__ "(" QUOTE(__LINE__) ")"
```

这个技巧对于微软的 Visual C++没有用，这是因为 VC 对于__LINE__宏的处理和其他的编译器不太一样，它不是简单地用一个整数代替__LINE__宏，而是使用一个(__LINE_Var + offset)来代替的。此处__LINE_Var 是一个内部变量，它代表了一个函数开始的行号，而 offset 则是真正的从函数开始算起到当前代码行的偏移量。因此，对于 VC 来说，这个宏有可能会被展开为以下结果：

Download at www.Pin5i.Com

```
"c:\project\main.cpp((__LINE_Var+5))"
```

不管怎么说，这个宏对于 Metrowerks CodeWarrior 和 SN Systems ProDG（一个基于 gcc 的编译器）都得到了理想的结果。

1.3.8 第 7 个宏技巧：防止进入无限循环

可能出现无限循环的地方，通常是你代码中的一些特定部分，尤其是那些依赖于外部数据或脚本的部分。一个实用的保护方法就是创建一个计数器，在每次循环的时候它都增加 1，当它一旦达到某个极限的时候就触发一个断言。这就是下一个宏技巧正要做的事情，唯一的不同是它可以以一种透明的方式加入代码，在进行发行版编译时可选择不对它进行编译。

这个技巧需要创建一个叫做“while_limit”的宏，它表现得和一个 while 循环一样，不过，它有另外一个参数用来定义循环的次数，当超过了这个数的时候，断言就被触发。例如，你可以看看下面的代码：

```
while_limit( node != 0, 1000 ) {
    //some work
    node = node->next;
}
```

在上面的例子中间，如果循环超过了 1000 次的话，一个断言就会被触发了。利用断言取代一个无限循环的无休止的等待，测试人员就可以更容易找到程序错误了。另外一个好处是如果测试人员选择忽略这个断言的话，则 while_limit 宏可以跳出这个循环继续进行下面正常的程序流程。

下面是宏的源代码：

```
static bool while_assert( bool a )
{
    assert( a && "while_limit: exceeded iteration limit" );
    return( a );
}

#define UNIQUE_VAR(x) safety_limit ## x
#define _while_limit(a,b,c) \
    assert(b>0 && "while_limit: limit is zero or negative");\
    int UNIQUE_VAR(c) = b; \
    while(a && while_assert(--UNIQUE_VAR(c)>=0))
#define while_limit(a,b) _while_limit(a,b,__COUNTER__)
```

请注意在上面我们使用了一个叫__COUNTER__的宏。这个__COUNTER__宏展开为一个整数，初始化为 0，每使用一次就增加 1。这就可以让我们每次使用 while_limit 宏的时候都可以创建一个唯一的变量了。这个唯一的变量可以用来跟踪循环次数，当第一次使用 while_limit 宏的时候，这个变量的名字将是 safely_limit0（第二次将是 safely_limit1）。这种唯一命名的技巧是很必要的，因为 while_limit 宏可能会被多次使用，这保证了它在各个地方展开的时候不至于产生变量名冲突（如果产生多重定义则会产生编译器错误）。

Download at www.Pin5i.Com

__COUNTER__宏不是一个 ANSI C 的标准宏，因此不是所有的编译器都支持。不过，微软的 Visual C++和 Metrowerks CodeWarrior 都支持它。对于 SN Systems ProDG 和其他基于 gcc 的编译器来说有另外一个方法，那就是可以使用__LINE__来创建一个唯一的变量名。下面的代码对于 SN Systems ProDG 和 Metrowerks CodeWarrior 都是适用的（两种 while_limit 对 Metrowerks CodeWarrior 都是可用的）。

```
static bool while_assert( bool a )
{
    assert( a && "while_limit: exceeded iteration limit" );
    return( a );
}

#define _UNIQUE_VAR(x) safety_limit ## x
#define UNIQUE_VAR(x) _UNIQUE_VAR(x)
#define while_limit(a,b) \
    assert(b>0 && "while_limit: limit is zero or negative"); \
    int UNIQUE_VAR(__LINE__) = b; \
    while(a && while_assert(--UNIQUE_VAR(__LINE__)>=0))
```

1.3.9　第 8 个宏技巧：小型的特制语言

宏是相当强大的，而此技巧或许就能全面表现出宏的威力。利用宏文本替换的能力，你可以为自己创建一个小型的、特制的语言，把它直接编译到 C/C++中间去。在你作结论之前，请记住我们的目标仍然是要得到一个易读的、维护性好的和可调试的代码。

在“Implementing a State Machine Language”（*AI Game Programming Wisdom*[Rabin02]）这篇文章中有一个宏语言应用的例子，它使得状态机的创建标准化了，并且使得编程风格更良好。其结果是，它使得状态机的创建容易了，使代码的易读性也增强了。下面就是使用宏语言的状态机的一个例子：

```
BeginStateMachine

    State( STATE_Wander )
        OnEnter
            // C++ code for state entry
        OnUpdate
            // C++ code executed every tick
        OnExit
            // C++ code for state clean-up

    State( STATE_Attack )
        OnEnter
            // C++ code for state entry

EndStateMachine
```

虽然上面的代码看起来好像是一个全新的脚本语言，但实际上它只是把 6 个宏的关键字

Download at www.Pin5i.Com

给编译进 C/C++代码了而已。这样做的好处在于你可以随意地把 C/C++代码加在其结构中去，而且调试也很容易，因为调试器的能力完整无缺。一旦你理解了状态机的行为，你就将明白宏语言只是把不必要的细节给隐藏了起来，这样开发人员就可以简单而自然地在结构里面添加代码了。

这 6 个宏的关键字定义如下（OnEvent 是一个辅助性的东西——并不被直接使用的）：

```
#define BeginStateMachine if(state < 0){if(0){
#define EndStateMachine   return(true);}}else{assert(0); \
                          return(false);}return(false);
#define State(a)          return(true);}} \
                          else if(a == state){if(0){
#define OnEvent(a)        return(true);}else if(a == event){
#define OnEnter           OnEvent(EVENT_Enter)
#define OnUpdate          OnEvent(EVENT_Update)
#define OnExit            OnEvent(EVENT_Exit)
```

如果你希望对这种宏脚本语言有更多的了解，请参阅[Rabin02]以得到更多的相关说明。

1.3.10 第 9 个宏技巧：简化类接口

C++的目标之一就是要把类的声明和其定义分离开来。这是非常有用的，这可以使开发人员看到一个类暴露的接口——它的声明，这通常是放在一个头文件里面的——而不用了解其对相应功能的具体实现（类的定义，这通常放在一个.CPP 文件里面）。

不幸的是，C++实现类声明与类定义分离的方法最后导致大量额外的工作，那就是每个非内联函数的标识（函数名及其参数）都需要写两次，一次是在类声明里面，一次是在类定义里面。

```
// class declaration in Elmo.h
class Elmo
{
   void TickleMe(int x, int y = 0);
};

// class definition in Elmo.cpp
void Elmo::TickleMe(int x, int y)
{
   // actual implementation of Elmo::TickleMe() goes here
}
```

由于 Elmo::TickleMe 的标识在两个地方都出现了，因此每次我们要改变此方法的参数的时候——或者改变它的函数名，或者把它加上 const 修饰符——在两个地方都需加以改动。

通常这没有多少工作量。在大多数情况下面，一个方法只属于一个类，如果只需要在两个地方进行改动实在也没有什么。然而，在某些情况下 C++语言的这个特性就会给你带来极大的工作量。而且在一些情况下，当你改变一个函数标识的时候很容易引进一些小错误——

Download at www.Pin5i.Com

一些编译器无法给你提出警告的错误。

举个例子来说，假设我们有一个叫做 BaseClass 的基类。有三个从 BaseClass 继承而来的子类——D1、D2 和 D3。BaseClass 声明了一个虚函数 Foo()。BaseClass::Foo()在 BaseClass 中有一个缺省的实现，它不是一个纯虚函数。更进一步，我们假设 D3 重载了 Foo()函数，这样 D3 的 Foo()就和 BaseClass 的 Foo()有所不同了。

现在，假设我们给 BaseClass::Foo()添加一个参数，而忘记了给 D3::Foo()同时做相应的更新。糟糕的是，编译器也会忘记 BaseClass::Foo()和 D3::Foo()之间的联　系——它会认为它们是两个完全不同的函数，以为 D3::Foo()和 BaseClass::Foo()是完全独立的。在我们想把 Foo()当成一个虚函数在 BaseClass 上来调用的时候，如果对象的类型是 D3，则 D3::Foo()不会被调用，这和我们原来的意图就完全不同了。

理想情况是我们希望一个函数的标识只在一个地方存在。要是能够只声明 BaseClass::Foo()一次，然后在 BaseClass 和 D3 中实现的时候不用再额外声明 D3::Foo()那就太好了。

在效率方面也有一些争议：在 C++中使用继承的时候我们经常会使用很多浅层次的类继承，一个父类往往有一堆子类。与深层次的包罗万象的继承方法相比，这种方法要有用得多，因为后者需要人为地把很多互不相关的功能集成到一个单独的、扩展庞大的类继承家族里面来。

对于浅继承的方式来说，我们只是把开始的父类声明为一个接口——就是说，它声明了一些虚函数，而其中大部分是纯虚函数。大多数情况下，我们经常会在这个类家族里面生成很多类，其中一个是基类，而其他的类都从这个基类派生的。问题就在于我们需要在所有的派生类里面重新声明所有的需要重载实现的方法。如果我们的基类有 10 个函数，而我们从这个基类派生了 10 个类的话，那就需要额外做 100 个函数声明才行，但实际上它们只不过是在说，“我只不过是实现了那个基类里面声明的方法而已”。所有这些额外的代码都会让头文件变得很难维护，而且难以阅读。

我们假设现在已经有了一个叫 Creature 的类，而我们希望从这个基类派生出不同的子类来。Creature 定义了三个纯虚函数，如下所示：

```
class Creature
{
    public:
        virtual std::string GetName() const = 0;
        virtual int GetHitPoints() const = 0;
        virtual float GetMaxVelocity() const = 0;
};
```

此外，我们还假设要从这个 Creature 基类派生出几个不同的子类来（SnowCrab、NordicYeti 和 SnowshoeBandit）。现在我们三个方法的每一个——GetName()、GetHitPoints()和 GetMaxVelocity()——都会在 7 个地方存在了：在 Creature 类的声明中有一次，在 SnowCrab、NordicYeti 和 SnowshoeBandit 的声明和定义中各出现一次。

然后假设我们要做一个小小的改动，对 GetHitPoints()方法进行修改，把其返回值由一个 int 改成一个 float。如此一来我们就要在七个地方修改 GetHitPoints()方法——Creature.h、

Download at www.Pin5i.Com

SnowCrab.h、SnowCrab.cpp、NordicYeti.h、NordicYeti.cpp、SnowshoeBandit.h 和 SnowshoeBandit.cpp。

一种处理这种情况的简单方法就是把这些方法进行包装，改成所谓接口宏的形式。这是一个简单声明了 Creature 所有方法的宏，如下所示：

```
#define INTERFACE_Creature(terminal)                    \
   public:                                               \
      virtual std::string GetName() const ##terminal    \
      virtual int GetHitPoints() const ##terminal       \
      virtual float GetMaxVelocity() const ##terminal

#define BASE_Creature          INTERFACE_Creature(=0;)
#define DERIVED_Creature       INTERFACE_Creature(;)
```

这样做的好处在于现在我们就可以大大简化 Creature 类的声明，还有所有从它派生的类的声明了：

```
// Creature.h
class Creature
{
   BASE_Creature;
};

// Skeleton.h
class SnowCrab
   : public Creature
{
   DERIVED_Creature;
};

// NordicYeti.h
class NordicYeti
   : public Creature
{
   DERIVED_Creature;
};

// etc.
```

现在，不管我们什么时候想要改动 Creature 的方法，我们都不需要再去改动 SnowCrab.h、NordicYeti.h 或者 SnowshoeBandit.h——我们需要做的工作只是要改动这个接口宏而已（Creature.h 中的 INTERFACE_Creature）。我们仍然需要手工改动每个.cpp 文件中间的实现，但是由于函数声明已经变动了，如果我们忘记了修改实现的话，编译器会发现这个错误并提示我们进行修改。

更进一步说，我们的类声明也变得更容易阅读了。当我们查看 NordicYeti.h 中 NordicYeti 类的声明的时候，我们看到的是 DERIVED_Creature 和一些专门针对 NordicYeti 类的方法，与其他 Creature 的子类毫不相关。这就可以立刻提醒我们什么方法是专属于 NordicYeti 的，

Download at www.Pin5i.Com

而如果要想要看 Creature 类有关方法的声明的话，我们就应该在 Creature.h 中间查看，这才是它真正进行声明的地方。

需要注意的是，这里的接口宏与其他语言，例如 Java 或者 C#，里面的接口概念是完全不同的。在这些语言里面，你可以将接口看成是一个对象——你可以传递它的引用（reference），可以调用它的任何方法。然而，一个接口宏只是对一套相关函数声明包装的一个方法，它隐藏了 C++语言结构强加的需要你来维护的不必要的代码复制。

特别感谢 Paul Tozour 提供了这个宏技巧。

1.3.11　结论

我希望这些给出的例子能让你对宏能有个新的认识，能为你自己的工具箱添加一些解决方法。请记住一定要小心权衡其中的代价和好处。一定要使你的代码既容易理解，又十分健壮。只要能够巧妙地使用这些宏，它们就能让你编程的时候更容易一点，也能让程序的错误更少一点。

1.3.12　参考文献

[Dalton01] Dalton, Peter, "Inline Functions Versus Macros", *Game Programing Gems 2*, Charles River Media, Inc., 2001.

[Hyman99] Hyman, Michael and Phani Vaddadi, *Mike and Phani's Essential C++ Techniques*, APress, 1999.

[Kernighan88] Kernighan, Brian, and Dennis Ritchie, *The C Programming Language*, Prentice Hall, 1988.

[McConnell93] McConnell, Steve, *Code Complete: A Pratical Handbook of Software Construction*, Microsoft Press, 1993.

[Rabin00] Rabin, Steve, "Squeezing More Out of Assert", *Game Programming Gems*, Charles River Media, Inc., 2000.

[Rabin02] Rabin, Steve, "Implementing a State Machine Language", *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

Download at www.Pin5i.Com

1.4 平台无关的函数绑定代码生成器

Allen Pouratian
Sony Computer Entertainment RTime
allenp@csua.berkeley.edu

一个自动化的函数绑定工具能够对你的C代码进行扫描，得到函数原型，给每个函数赋予一个唯一的整数值，生成相应的代码以根据函数名得到对应于整数值的散列值，然后生成一个条件判断（switch）代码把这些整数值和调用函数的代码给绑定起来。

一旦绑定工具生成的代码编译进了你的程序里面去以后，此接口就形成了一个脚本引擎的核心，或是成为一个网络RPC的执行部分，这样就可以避免通常是繁琐而容易出错的维护性的开发工作了。

本文是《游戏编程精粹1》（*Game Programming Gems*）中的[Bilas00]的一个扩展。

1.4.1 年轻与智慧

当我们还是小孩子，正在捣鼓自己的第一台计算机的时候，我们还不知道如何不通过一次重新创建（rebuild）就能生成自己的软件。之后，当我们懂得了更多知识而成长为少年以后，就知道使用命令行参数了。又过了一段时间，我们开始学会使用配置文件。如果你是个内行的话，你甚至会自己开发一个工具生成配置文件了。

现在我们已经长大了很多，我们将要设计一个工具来生成代码，在自己的程序里面根据函数名来调用任意一个函数。例如，与使用硬编码载入一个游戏的难度相比，我们最好能使用脚本来做此工作。还有，假如设计人员要求在你的枪战场景里面增加一类新的敌人，如果能自动生成在其他机器上调用此新函数的代码岂不是很好？

1. Cygwin

在Red Hat公司的热心人们推出其Cygwin开发包之前，要想编写出这样的代码可不容易。Cygwin的库能使得大多数POSIX[1]的代码在Windows9x/NT/2000/XP环境下面运行。因此，再加上德高年劭的编译器生

译者注[1] POSIX即指可移植操作系统接口（Portable Operating System Interface）。后面的X是为了让它像UNIX一点。它是一个IEEE制定的操作系统API的标准，它主要是参考Unix类系统设计的，也主要在Unix（包含BSD系列和Linux）上使用，不过Windows也有POSIX接口，虽然性能和功能都不是很好。

Download at www.Pin5i.Com

成工具 Lex 和 Yacc，还有因特网上能找到的免费的 C 语言的文法和规范的资料，我们就几乎有了一套完整的，在所有支持 GNU 的平台上的，对于分析 C 语言的结构体、类型定义以及函数原型所需要的一切了——此处完整的意思是指有了所需的足够条件以完整地进行代码移植任务。这是由于对 GNU 的支持每周都在加强，还有就是我们总是可以随处下载到 Lex 和 Yacc。

2. 有关本文的前身

本文的前身（[Bilas00]）已经彻底地讨论了与函数绑定机制有关的各种问题。我们进行了两次尝试性设计，这些尝试简单但是没有什么实际用处，而且令人头痛。通过这些前车之鉴，我们得到了一个特定于 Windows、x86 架构、Visual C++ 6.0 的自动化而精致的解决方案，特别是通过编译器产生的输出文件，我们可以在运行时将那些与编译期 DLL 的输出函数对应的函数转化为一个表，然后通过这个表，根据函数的文本形式的名称，我们可以查找出相关联的函数 ID 并执行它们。

1.4.2　概要

如果你曾详细阅读 Lex 中 C 的规范[Degener95]的话，你将注意到里面没有包含`#include`和`#define`。这些东西是 C 的预处理器来处理的，然后预处理器将把结果交给编译器。幸好 gcc 的开发人员将预处理器提了出来做成了单独的叫做“cpp”的工具。

下面就是我们要做的工作了。首先，得到一个你希望输出其功能的 C 的模块，然后用 cpp 来处理它。接下来，在此工具的第一个阶段，得到 cpp 的生成结果，若要用它，你需要使用 Lex 与 Yacc 开发的工具对其进行处理，抽取出其中的函数原型。然后，在此工具的第二阶段，为每一个提取出来的函数原型分配一个唯一的 ID，生成在函数原型和 ID 中间的映射表的代码。在第三阶段，我们还需要生成更多的代码以用来得到一个函数，此函数可以将函数名解析为相应的 ID，推荐使用一个自动前移的散列链表。最后，我们的工具将针对上述的 ID 生成一个条件判断代码，可以用来为每个硬编码的函数生成相应的参数进行调用。经过这么一个过程，工具产生的 C 代码就可以把如何调用函数的事情交给编译器来处理了，而这是与平台相关的。只需要把工具生成的代码编译和连接进来，你就可以运行程序了。

一旦我们使用 Lex 和 Yacc 得到了自己的读取函数原型的工具，我们就需要利用它来产生一些高效的用来绑定函数名和其 ID 的 C 代码。令人惊异的是，使用一个自动前移的散列链表机制[Zobel01]要比所有的树都要快至少三倍，包括自适应的扩展树（splay tree）[Sleator85]。正如 Paredo 原则所说的那样，80%的时间里需要的只是 20%的数据，最经常访问到的元素总是放在链表的顶部。

在把我们工具产生的 C 代码编译连接进了自己的游戏以后，我们就可以坐享其成了。我们可以在任何脚本文件上调用我们的脚本执行代码，还可以为我们的服务器开发一个新函数，而且在客户端上不需要进行任何编程做转化或者激活就可以在服务器上执行它。

请记住，如果你只是想要在 Perl、Python，或者 Tcl/Tk 的脚本中调用自己的 C、C++，或者 Objective-C 的函数的话，那请访问 www.swig.com 网站。在更多的语言里面绑定 C/C++/Objective-C 的代码的工作仍在进行中。

Download at www.Pin5i.Com

1.4.3 细节

由于经常需要给编译器编写标识符分析器，因此 Mike Lesk 和 Eric Schmit（贝尔实验室）决定编写一个工具来简化这项任务。如果想要确认一下 Lex 确实会大大减少你的工作量，那就请参阅相应的 O'Reilly 的书籍[Levin95]。

我们使用正则表达式[Borsodi01]来表达一个标识符，使用 C 的代码来对每个表达式进行处理。Lex 则负责麻烦的活，那就是产生相应的 C 的代码，在找到一个标识符的时候将之提取出来并调用你写的 C 代码。下面的 Lex 代码是用来识别历史上一些著名的科学家和数学家的：

```
%{
/* Put any C code here. It need not be a comment. */
%}

%% [\t ]+ /* ignore white space */ ;

Newton | newton    { printf("Issac Newton\n") ; }
Pascal | pascal    { printf("Blaise Pascal\n") ; }
Pasteur | pasteur  { printf("Louis Pasteur\n") ; }

[a-zA-Z]+ {printf("%s: don't recognize \n", yytext); }

\&.|\n { ECHO; /* normal default anyway */ }

%%

main()
{
    yylex();
}

/* Again, any C code of yours can go here */
```

在%{和%}的空白地段之间，我们可以在词法分析器产生的 C 代码上面加入任意的 C 代码。接下来，我们将想要提取出来的标识符放置在两个%%之间，紧接下来的就是在找到了一个词以后用大括号包住的我们想要执行的 C 代码了。在结尾的%%以后，我们还可以接着加入任何需要的 C 代码。

不用为如何使用 Lex 感到害怕，因为你在每一个正则表达式后面的 C 代码中仍然可以调用函数、声明变量和给变量赋值。不过请确定把 Lex 的库文件加入了自己的程序里。如果你是用 Unix 的 make 工具的话，大多数情况下你只要在调用连接器的时候加入-ll 命令行参数就可以了。

再加上 Yacc

20 世纪 70 年代贝尔实验室里的工程师们是一群“牛人”，因此他们还编写了 Yacc 以减

Download at www.Pin5i.com

轻编写解析器的负担。正如英语文法标明了英语句子的语法一样，我们也给 C 编写了文法，这样 Yacc 就能把一行 C 代码和一行 Ada 代码给分辨开了。下面就是一个简单的 Yacc 的文法，可以用来分析最简单的英语句子：

```
%{
/* Put any C code here. It need not be a comment. */
#include <stdio.h>
%}

%token NOUN VERB PRONOUN

%%
sentence: subject VERB {printf("Baby talk!\n"); }
    ;

subject: NOUN | PRONOUN ;

%%

#define ERROR_RETURN( intReturnValue, expression,
stringExplanation ) \
    if( expression ) \
    { \
        printf("Function Failure: %s in %s at line
        %d\n", stringExplanation, \
        __FILE__, __LINE__ ) ; \
        return intReturnValue ; \
    } \

extern FILE *yyin;

main(int argc, char* argv[])
{
    FILE *fp ;

    ERROR_RETURN( 1, argc != 2 , "main-->wrong number of arguments") ;

    fp = fopen(argv[1], "r" ) ;
    ERROR_RETURN( 2, !fp , "main-->Invalid input
file") ;

    yyin = fp ;

    while(!feof(yyin))
    {
        yyparse();
    }

    fclose( fp ) ;
}
```

Download at www.Pin5i.com

```
yyerror(s)
char *s;
{
    fprintf(stderr, "%s\n", s);
}
```

正如其近亲 Lex 一样，Yacc 也允许在生成的 C 文件中%{和}%之间插入任意 C 代码。在这里，除了 C 的注释以外，我们还加入了载入 stdio.h 文件的 C 代码。在%token 以后的单词指明了此文法需要识别的标识符。如果我们同时使用配合着 Yacc 使用 Lex 话，这些宏定义在你的 Lex 和 Yacc 中就都是一样的了。

在%%之间，我们对什么是文法中的产品（productions）作了一个描述，在此处 Yacc 需要拥有一个无歧义对其文法的语法定义。一般情况下，开发人员会在每个希望处理的产品后面加上相应的需要执行的语句。在结尾的%%之后，我们又可以在 Yacc 生成的 C 代码后面加入所需要的 C 代码了。对于 Yacc 来说我们不需要像 Lex 一样连接一个 Yacc 的库。

1.4.4 脚本

一旦你输出了载入游戏难度的函数以后，在开发过程中就可以使用一个脚本以提供深受欢迎的灵活性。假设现在需要以一个 WAV 文件取代一段触发的声音，或者改变任何事物的属性而不需要重新创建程序，只要把此过程做得直观一点以便让内容制作者能处理此过程，你就可以利用这段时间去寻找另一个内存泄露的错误了。假设在发布前一个星期有人说游戏中的某个难度场景太暗了——此时由于他们只需在你脚本的基础上调整一个参数，你就会觉得进度的压力小多了。

人工智能中的指令

在 Gamasutra 的一篇文章里面，Charles Guy 讨论了要使用形式化指令给人工智能进行自动化的建模。因此，这种输出函数的功能再一次使得事务的处理只需要一个非专业人员。人工智能需要人的参与才会表现得令人满意，而从长远的角度看来，使用脚本来提供这种行为可以提高效率并减少麻烦。

1.4.5 网络

一旦你将函数绑定的功能加入到了自己的 RPC 系统中，就不需要再在扩展 API 的时候为扩展功能而发愁了。假设我们现在在设计一个基于对等用户的四人游戏，在此游戏中如果 A 玩家发射了一个导弹，则在电脑 A、B、C 和 D 上都必须调用 fire-missile()函数。如果电脑 A 上调用函数机制的基础是使用的通用 ANSI C 的变长参数列表的话（此处指的是前面提到的工具生成的函数原型 ID 表，还包含了编码后的函数参数列表），那么在有了新函数的时候，我们就不需要做额外的工作来在网络的另一端上转化和执行它们了。

Download at www.Pin5i.Com

1.4.6　结论

利用工具产生的代码来输出和绑定函数可以为我们在手工为脚本和 RPC 引擎绑定函数时节省很多工作。以前我们需要开发人员来修改代码以改动函数映射表，现在只需要设计人员或者艺术人员就可以了，这就可以大大加快迭代设计的周期了。

1.4.7　参考文献

[Bilas00] Bilas, Scott, "A Generic Function-Binding Interface," *Game Programming Gems*, Charles River Media, Inc., 2000.

[Borsodi01] Borsodi, Jan, "*Regular Expressions Explained*," 在网址 http://www.zez.org/article/articleprint/11/上有在线资料, 2000.

[Cygwin01] http://www.cygwin.com.

[Degener95] Degener, Jutta, "*ANSI C Grammar, Lex Specification*," 在网址 http://www.lysator.liu.se/c/ANSI-C-grammar-1.html 上有在线资料, 1995.

[Degener95] Degener, Jutta, "*ANSI-C Yacc Grammar*," 在网址 http://www.lysator.liu.se/c/ANSI-C-grammar-y.html 上有在线资料, 1995.

[Guy99] Guy, Charles, "*A Modular Framework for Artificial Intelligence Based on Stimulus Response Directives*," 在网址 http://www.gamasutra.com/features/19991110/guy_01.htm 上有在线资料。

[Levine95] Levin, Mason, Brown, *Lex & Yacc*, O'Reilly & Associates, 1995.

[Rabin00] Rabin, Steve, "*The Magic of Data-Driven Design*," Game Programming Gems, Charles River Media Inc., 2000.

[Sleator85] Sleator, Tarjan, "*Self-adjusting Binary Search Trees*," JACM, Vol.32, No.3, July 1985: pp 652～686.

[Zobel01] Zobel, Heinz, Williams, "*In-Memory Hash Table for Accumulating Text Vocabularies*," 在网址 http://goanna.cs.rmit.edu.au/~hugh/zhw-ipl.html 上有在线资料。

Download at www.Pin5i.Com

1.5 基于句柄的智能指针

Brian Hawkins
Seven Studios
winterdark@sprynet.com

在游戏中的任何时候，一个对象都有可能被销毁而遗留下一些指向原对象的空置的指针，从而导致严重的后果。通常我们可以使用句柄来防止这些失效的指针被使用到。不过如果对对象的每一个动作都用上句柄的话则会带来比较大的开销。为了减少此种开销，一般我们可以把句柄转化为指针，但是这种做法将导致产生出很多偶然性的指针而没有办法来跟踪它的使用情况。一个更好的解决方法是把句柄封装在一个智能指针中，这既能保持原有的语法，又能得到更好的安全性和可调试性。

智能指针是C++的一个类，它的语法与一般的指针是一样的，但是它提供了一些通常情况下一般指针没有的功能，最常用的一个功能就是对对象数据所有权的管理。有很多智能指针的实现方法，每种都有自己的所有权管理策略（深度复制、写时复制、引用计数，还有销毁式复制），在当前任务中使用智能指针的时候一定要慎重考虑应该采用哪种方法。Andrei Alexandrescu 在 *Modern C++ Design*[Alexandrescu01]一书中对许多智能指针的特性做了很好的总结。

现在让我们来考虑一下什么时候应该使用基于句柄的智能指针。首先，对象应该有一个明确的所有者来控制什么时候销毁它——例如，假设只有场景管理器可以销毁游戏中的对象。如果没有明确的所有者能决定什么时候销毁对象的话，使用引用计数可能会是一个更好的方法。其次，对象销毁的时间应该是非确定的。游戏中一个被玩家杀死的对象就是一个非确定销毁时间的例子。如果可以确定一个对象销毁的时间，那一般就不需要使用句柄以免增加开销了。最后，只有在多个对象需要分别保存指向一个对象的指针的时候，基于句柄的智能指针才是必要的，例如四个玩家都同时引用了一个车辆对象的时候。不然的话，很有可能可以找到另外一种开销更小的解决方案。

1.5.1 用法

现在既然你已经知道什么时候和为什么要使用基于句柄的智能指针了，我们就可以继续讨论如何安全而高效地实现它们的细节了。不过首先，我们需要描述一下基于句柄的指针是如何使用的，这可以帮助我们理解此

Download at www.Pin5i.Com

实现究竟要达到什么样才算成功。

第一步是要创建此对象实例然后对得到的哑指针（dumb pointer）（哑指针一般指的是语言内建的指针类型）赋予一个智能指针。由于我们再也不需要哑指针了，因此此操作可以用一行代码完成：

```
t_HandlePointer<class> l_handle(new class);
```

接下来，为对象实例设定的所有者可以将此句柄传给任何一个想要引用此实例的人：

```
t_HandlePointer<class> l_handleCopy = l_handle;
```

当对象的生命期结束了以后，此实例的所有者就将销毁此对象了。由于所有者拥有的是一个智能指针，因此我们需要一个函数而不能用 delete 操作符来做这件事情：

```
g_Destroy(l_handle);
```

使用一个句柄指针几乎和使用一个哑指针完全一样。举个例子，句柄指针也可以使用->或者*操作符来解引用(dereference)。我们也可以和检验一个空的哑指针一样来检验此句柄指针是否为空：

```
if(l_handle) {
    l_handle->m_Member();
    (*l_handle).m_Member();
}
```

如果有可能对象已经被销毁，从而导致句柄失效的话，那么这种检验就一定要进行。

1.5.2　句柄

句柄是基于句柄的智能指针的核心部分，因此实现它的时候一定要保证高效。对句柄主要的操作是将之转换为指针，并检验它的有效性。把句柄转换为指针的一个很高效的作法就是把指针保存在一个数组里面，把数组的下标变成句柄的一部分。这样一个转化就由两步组成了：首先是从句柄得到下标；然后是在数组里面根据下标找到指针。

只有在句柄本身有效的时候以上的转换才是有效的。因此我们需要一种方法来判断下标所在位置的内容是否和转换使用的句柄是一致的。要做到这一点，我们需要给每个句柄一个唯一的标识，把它和相应的指针都保存在数组里面。当句柄失效的时候，指针和此标识值都将从数组里面删除。以后再试图使用句柄的时候就会得知它已经失效了，因为此时标识值已经不在数组中对应的位置上了。图 1.5.1 展示了使用此方法时一种可能的布局。

虽然有好几种方法可以生成一个的唯一标识，但是对于句柄来说一个简单的方法就足以应付了。只要在初始化的时候设置此值为 1，然后当需要一个新的标识值的时候就每次增加 2。这么做的好处在于 0 始终不会成为一个有效的标识值，因为此时最低位总是为 1。请注意此处我们没有检查唯一标志值是否重复。只要在任何时候都能保证唯一标志值的位数足够大，以能容纳下此时现存的对象，那么在同一位置产生同样标识值的可能性就小得可以省略掉了。如果能够保证唯一标识值一直增长的话，那么这种很小的可能性都会完全消除，虽然此时它也限制了游戏中对象的总数目。

Download at www.Pin5i.Com

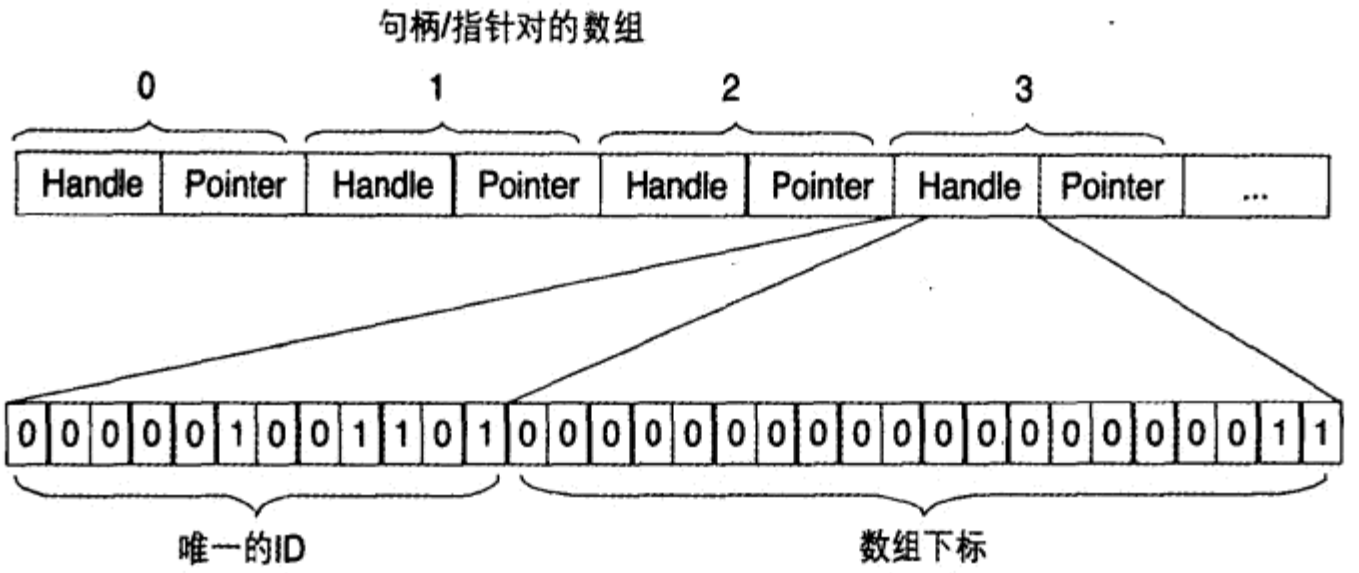


图 1.5.1 从句柄得到指针的一个示例性的布局

1.5.3 智能指针

智能指针封装了句柄，以在使用句柄的时候能提供与一般指针相类似的语法。由于已经有一些很好的书籍和文章描述了如何实现智能指针的所有细节了，因此我们将主要关注那些特定于基于句柄的智能指针的问题。这些问题包括建构、销毁、解引用，还有有效性检验。在 CD-ROM 上提供的例子实现里包含了一个完整可用的范例。

1. 建构与销毁

从哑指针建构智能指针需要创建一个句柄。建构函数将哑指针传递给句柄管理器以用来创建此句柄。然后，此句柄和可用来进行有效性检验和解引用的哑指针就会存储到智能指针里面去。

句柄管理器一般是一个单实例（Singleton）类[GoF95]——即此类只有一个全局的实例。它可以为句柄/指针对提供一个中心点，为以后的句柄查询备用。管理器同时还保证了每个指针只有一个句柄，不过这一点会带来一些性能上的损失。当然这种检验也可以免去，前提就是当对象一创建就立刻给哑指针赋予一个基于句柄的智能指针，而且所有接下来的操作都是通过对智能指针或是智能指针的拷贝进行的。

一般情况下，对象是用 delete 关键字来销毁的，但是这对于智能指针来说是不可能的。一个替换的方法就是在智能指针上添加一个成员函数，此函数可以用来调用 delete 删除其哑指针。但是这会使代码令人混淆，因为此时用户将可以在智能指针上面同时使用->和.操作符。另一个更好的方法是可以把智能指针当作参数传递给一个友元函数以用来销毁哑指针。一旦对象被销毁，我们应该马上通知句柄管理器让其知道句柄已经无效了。

2. 有效性检验

如果对象有可能已经被销毁的话，在一个智能指针被解引用之前就应该检验一下其有效性。在检验基于句柄的智能指针的有效性之时，我们会使用到两个重要的条件——相等和不相等。在内部实现的时候，两种检验都必须查询句柄管理器以得知句柄有效与否。

Download at www.Pin5i.Com

不相等的检验只需要重载!操作符就可以了，但是相等的检验则需要多做一点工作，以防止智能指针出现古怪的行为。必须创建一个简单的类禁止 delete 操作符被使用，方法是声明但是不定义此操作符，然后在句柄有效的时候通过转换操作符返回一个指针，其指向的是此类的一个全局的实例。否则，转换函数就应该返回一个空指针。若想知道为什么此方法比较优异，读者可以在 *Modern C++ Design*[Alexandrescu01]里面得到更多的信息。

3. 解引用

智能指针最有用的特性就是可以对其使用->和*操作符。这就是为什么要在智能指针内部保存一个哑指针拷贝的原因，只有这样，我们才能快速地返回一个对象的指针或是引用。这些操作符必须极其简单而且高效，因为它们将是基于句柄的智能指针最常用的特性。这些函数的调试版本还应该检验句柄的有效性，以此作为增加的一层保护，以防止调用者在解引用之前没有检查此智能指针而出错。

1.5.4　结论

当针对正确的对象使用基于句柄的智能指针的时候，它将是一个有效而有力的工具。句柄和智能指针的结合简化了对象的管理，同时对所有的开发人员提供了相似的语法。

我们讨论了与使用句柄的智能指针相关的特定问题，但是对智能指针来说还有大量的资料可供参考。*Modern C++ Design*[Alexandrescu01]和 *More Effective C++*[Meyers96]都包含了智能指针实现和使用方面的很多有用的信息。

1.5.5　参考文献

[Alexandrescu01] Alexandrescu, Andrei, *Modern C++ Design*, Addison Wesley, 2001.

[GoF95] Gamma, Erich, et al., *Design Patterns*, Addison Wesley, 1995.

[Meyers96] Meyers, Scott, *More Effective C++*, Addison Wesley, 1996.

Download at www.Pin5i.Com

1.6 定制 STL 分配器

Pete Isensee
微软公司
pkisensee@msn.com

很多游戏都使用自己定制的分配策略，为自己提供定制的 malloc 和 free 函数或者重载全局操作符 new 和 delete。C++STL（标准模板库）提供了强大的扩展功能，它能让你对 C++中的容器对象，包括 string，使用自己定制的分配和释放策略。本节就讲述了如何创建一个定制的分配器以将之集成进游戏的代码里面去。

1.6.1 一个范例

先举一个例子，假设你的游戏创建了一个列表，对它进行了一些操作，然后将之丢在一边——所有的这些操作都是在一个函数里面进行的：

```
std::list<int> MyList;
for( int i = 0; i < 100; ++i ) // build the list
   MyList.push_back( i );
MyList.reverse(); // manipulate the list
```

如果此函数在游戏里每过一个循环就被调用一次，那么你将会发现分配和释放列表元素的效率极低，因而将会导致你不再使用 STL 的列表了。假设你能对列表进行定制，使得所有的内存都从栈上分配而不是堆上分配。那么你就不需要再释放这些内存，因为当栈中的对象离开其定义域的时候栈里面的内存就会被自动释放。假设使用了定制分配器，那么基于栈的列表其代码可能会和下面相似：

```
// Create custom allocator object
const size_t nStackSize = 100 * 12;
unsigned char Stack[ nStackSize ];
StackAlloc<int> sa( Stack, nStackSize );

// Tell the list about the custom allocator
std::list<int, StackAlloc<int> > MyList( sa );

// This code is the same
for( int i = 0; i < 100; ++i ) // build the list
   MyList.push_back( i );
MyList.reverse(); // manipulate the list
```

Download at www.Pin5i.Com

StackAlloc 是一个定制的分配器。此 StackAlloc 类型在列表的声明中充当的是一个模板参数的角色，而且此 StackAlloc 对象被传递到了列表的建构函数中去。在此例中，栈的大小被设置为刚好能容纳 100 个节点。在大多数 STL 的实现中，一个列表的结点由一个对象和两个指针组成。在随书附送的 CD-ROM 上面有 StackAlloc 实现的完整代码。

从性能的角度上来说，这种特制的优化大概能使代码比第一个版本的代码快上两到三个数量级，这要视编译器和平台而定。显然，定制的分配器在性能上提供了极大的优越性，而使用分配器的好处就在于你可以在代码里面以任何适合自己游戏的方式来利用一个分配技术实现之。

1.6.2　分配器的基础

STL 容器类包括 vector、deque、list、map、set、multimap，还有 multiset。而 basic_string 对象也可以算是一个容器类。所有的 STL 容器类都要分配内存以存储其容纳的对象。这些容器类可以使用你提供的定制的分配器，这样以便你定义应该如何管理内存。例如，list 是如下定义的：

```
template <typename T, class Allocator = allocator<T> >
class list { ... };
```

此 Allocator 参数是一个缺省的模板参数。在一般情况下，它没有被特别指定，此时，使用的是缺省分配器 allocator<T>。在下面我们还将对缺省分配器做更多的细节讨论。

```
std::list<int> IntList; // uses default allocator<int>
```

1.6.3　分配器的要求

所有的分配器对象都需要满足 C++标准 20.1.5 节[Cpp98]里面申明的最低要求。这些要求并非难以满足，而且也主要只涉及到程式化的一些代码。不过，最好我们能对分配器类的所有部分都先了解一下。

1. 类型定义

所有的分配器对象都需要包含以下的类型定义。本节的这些类型定义几乎对于所有的分配器都是一样的。不过 C++标准也允许你实现不同的定义，例如，以下的 pointer 的类型定义就可以被声明成远指针（far pointers），或者其他的对于相应平台有意义的指针类型。在实际情况中，一般不太可能定义其他的类型，特别是 C++没有什么方法可以定义引用类型，除了那个古老而好用的 T&以外。因此基本精神就是：不要改变下面的这些类型定义，除非你非常清楚其后果。

```
typedef size_t    size_type;
```

Download at www.Pin5i.Com

```
typedef ptrdiff_t difference_type;
typedef T*        pointer;
typedef const T*  const_pointer;
typedef T&        reference;
typedef const T&  const_reference;
typedef T         value_type;
```

2. 建构与复制

分配器同时也是普通的 C++对象，因此必须要有某种方法来创建它们、复制它们和销毁它们。下面就是建构函数和析构函数的声明：

```
allocator() throw();
allocator( const allocator& ) throw();
template <typename U>
   allocator( const allocator<U>& ) throw();
~allocator() throw();
```

请注意那些不同寻常的模板成员建构函数，它们可以利用其他类型的分配器来创建此分配器。某些编译器尚不支持模板成员函数，因此在你实现的时候可能没有这种建构函数。同时也请注意 C++标准要求不管是建构函数还是析构函数都不能抛出异常。

3. 辅助函数

分配器对象必须提供数量很少的一些辅助函数。最有意思的辅助函数就是 max_size 了，它返回通过 allocate 函数可以可靠地分配的对象的最大数目。

```
pointer address( reference r ) const
{
   return &r;
}

const_pointer address( const_reference c ) const
{
   return &c;
}

size_type max_size() const
{
   return numeric_limits<size_t>::max() / sizeof(T);
}
```

4. 分配

allocate 函数是分配器对象中的两个重要函数之一，它也是定制分配策略的焦点所在。C++标准规定 allocate 必须返回一个指向一个原始内存块（raw memory）的指针，此内存块必须足够大以能容纳 n 个大小为 T 的对象，其中 n 总是大于零的。此函数并不创建这些对象。第二个参数主要是一个提示，它指向另一个内存块以用来改进引用的局部性（locality）。此参

Download at www.Pin5i.Com

数一般不会使用。

```
pointer allocate( size_type n, const void* pHint );
```

下面就是此函数的一种符合标准的实现：

```
pointer allocate( size_type n, const void* )
{
    assert( n > 0 );
    return pointer( malloc( n * sizeof(T) ) );
}
```

5. 释放

deallocate 函数是分配器对象第二重要的函数。C++标准规定此函数必须释放 n 个大小为 T 的对象占用的内存，其中含有这些对象的内存块是一个非空指针 p。此指针 p 必须是通过调用同一个分配器的 allocate 函数得到的，而且此函数不能抛出异常。

```
void deallocate( pointer p, size_type n );
```

下面就是此函数的一种符合标准的实现：

```
void deallocate( pointer p, size_type )
{
    assert( p != NULL );
    free( p );
}
```

6. 就地（In-Place）建构与析构

请注意，与 new 和 delete 不一样，allocate 和 deallocate 并不创建或是销毁对象——它们只与原始的内存打交道。出于性能方面的考虑，分配器一般是独立于这些操作的。例如，vector::reserve()函数会分配原始内存，但是只会创建 vector::size()个对象，当且仅当 vector::resize()函数被调用的时候这些没有被初始化的对象才会被创建。

分配器的 construct 函数可以使得对象在已分配的内存上就地创建。你不能在 C++里面直接调用一个建构函数，但是定位 new 操作符提供这种功能。定位 new 操作符实际上并不分配内存——它只是在已分配的内存上调用对象的建构函数而已。

```
void construct( pointer p, const_reference c )
{
    // placement new operator
    new( reinterpret_cast<void*>(p) ) T(c);
}
```

destroy 函数可以用来显式地销毁一个对象而不释放其内存。你可以在 C++里面直接调用一个析构函数，而它与 destroy 函数做的是完全一样的事情。

```
void destroy( pointer p )
```

Download at www.Pin5i.Com

```
{
    // call destructor directly
    (p)->~T();
}
```

C++标准对这些函数作了很精确的定义。它们必须和此处叙述的表现完全一致。这些函数真正的目的是要把形式上的复杂性对容器类给隐藏起来。

7. 重绑定（rebind）

假设你已经为一个整数列表定义了一个自己的分配器。

```
list<int, MyAlloc<int> > IntList;
```

你或许会以为每当列表新加入一个元素的时候 MyAlloc<int>::allocate 就会被调用一次，若是如此那就错了。大多数容器类（除了 vector 以外）并不真正为其所存储的对象分配内存，它们只会分配节点，每个节点都含有一个对象，还有一个或多个指针。这意味着分配器必须采用某种方法把类型 T 转化为节点类型，而不管这个节点类型是什么类型。此方法就叫做重绑定。

```
template <typename U>
struct rebind
{
    typedef allocator<U> other;
};
```

重绑定只不过是让 allocator<T>来为其他类型 U 的对象进行分配而已。例如，对于一个处理类型 T 的分配器 a 来说，你就可以利用下面的表达式分配一个类型为 U 的对象：

```
T::rebind<U>::other(a).allocate( 1, NULL );
```

如果你不了解这些难懂的与模板相关的细节，那也没关系，重要的是你要记住为某种类型定义的分配器要能为其他类型（以及大小）的对象进行分配。另一个需要知道的要点就是 rebind 要求编译器支持成员模板函数才能正常工作。对于那些要与不满足此条件的编译器打交道的 STL 实现，它们必须自己提供一套分配节点的方法。举个例子，老版本的 Dinkumware STL 库[Dinkum]提供了_Charalloc 函数以支持不符合此条件的微软 Visual C++。

8. 比较

C++标准在对分配器进行比较的问题上提及了两个要求。首先，它要求两个分配器相等的条件是当且仅当其中任一个分配器分配的内存能被另一个分配器释放。第二，它规定 STL 实现的时候可以假设分配器是可互换的，而且在比较的时候可以总是认为它们是等价的。如果这些声明让你觉得有点自相矛盾的话，那是因为它们本来就有点儿矛盾。标准在此处要表达的意思是说分配器的比较是与 STL 实现相关的。某些 STL 实现允许对分配器进行比较，而另一些则在比较的时候总是认为它们是等价的。

```
template <typename T1, typename T2>
```

Download at www.Pin5i.Com

```
bool operator==( const allocator<T1>&,
                 const allocator<T2>& ) throw();+
template <typename T1, typename T2>
bool operator!=( const allocator<T1>&,
                 const allocator<T2>& ) throw();
```

在很多情况下面，只要假设分配器都是可以互换的就行了。这种情况中比较经典的是分配器中没有任何数据成员。在另一些情况下面，你可能真的需要对分配器进行比较（如果想对此问题作更多的了解，请参见下面的分配器状态数据小节）。

1.6.4　缺省的分配器对象

C++标准要求所有的 STL 实现都包含一个缺省的称之为“allocator<T>”的分配器对象，当你没有使用定制分配器的时候，这就是真正被使用的分配器了。换个角度说，这有可能就是现在你的代码里使用的分配器，因此我们很有必要对它作一番了解。

缺省分配器的定义出现在 C++标准的 20.4.1 节[Cpp98]。你可以在自己使用的 STL 版本的<memory>头文件（或者<memory>里面包含的头文件）里找到缺省分配器的定义。

在标准里规定缺省分配器必须调用 new 操作符来分配内存。如果没有足够的内存，它应该抛出一个 bad_alloc 异常。一个典型的实现缺省 allocate 函数的代码是这样的：

```
pointer allocate( size_type n, const void* )
{
   assert( n > 0 );
   return pointer( operator new( n * sizeof(T) ) );
}
```

标准里规定缺省的 deallocate 函数必须这样调用 delete 操作符：delete(p)。一个典型的实现缺省的 deallocate 函数的代码大致是这样的：

```
void deallocate( pointer p, size_type )
{
   assert( p != NULL );
   operator delete( p );
}
```

正如你会想到的一样，缺省分配器只不过是对 new 和 delete 做了一层封装而已。在实现的时候可以凭你喜欢自由地在缺省分配器里作一些额外的优化。你可以对自己使用的 STL 版本作一番检查，看看其缺省分配器是如何实现的。

1.6.5　编写自己的分配器

正如上面提过的，每个分配器对象都必须满足 C++标准提出的最低要求。为了能让它有用，它同样必须在各种特定于编译器的限制下也能工作。如果你要满足所有的这些要求（和限制），最好的方法就是把缺省分配器从<memory>里面拷贝出来，然后定制自己的 allocate 和 deallocate 函数，或许还有建构函数、析构函数，以及比较操作符。

Download at www.Pin5i.Com

下面是一个在简介里提及的 StackAlloc 分配器的部分实现。完整的源代码在 CD-ROM 上有。其中的 allocate 函数只是简单地返回了一个堆栈上尚未使用的下一个内存块，然后更新了一下已分配字节数。如果栈空间已经耗尽了，它就会抛出一个 bad_alloc 异常。而 deallocate 函数根本不需要做任何事情，因为栈空间是会被自动回收的。所有的这些函数都是内联的模板函数，因此在发布版本中 deallocate 实际上只是一个空操作而已。

```
template <typename T>
class StackAlloc
{
public:

    // boilerplate typedefs here . . .

    // critical ctor
    StackAlloc( unsigned char* pStack,
      size_t nMaxBytes ) throw()
    :
       mpStack( pStack ),
       mBytesAllocated( 0 ),
       mMaxBytes( nMaxBytes )
    {
    }

    // other ctors, dtor . . .

    // utility functions . . .

    // construct, destroy, rebind . . .

    pointer allocate( size_type n, const void* )
    {
       void* pRaw = mpStack + mBytesAllocated;
       mBytesAllocated += ( n * sizeof(T) );

       if( mBytesAllocated+1 > mMaxBytes )
          throw std::bad_alloc();

       return pointer(pRaw);
    }

    void deallocate( pointer p, size_type )
    {
       assert( p != NULL );
    }

    // member data . . .
};
```

Download at www.Pin5i.Com

1.6.6　潜在的用途

什么时候你会需要自己编写分配器呢？如果在对自己的游戏进行了性能评测以后，你认为缺省分配器是瓶颈所在的话，这就可以考虑提供一个定制的分配器了。如果你在整个游戏中都使用定制的策略而且不希望有任何对 new 操作符的调用，但是还是想使用 STL 容器类的时候，也可以考虑提供一个定制的分配器。

分配策略相当多，要想在这里把它们都列出来是不可能的，不过表 1.6.1 列出了一些精选的策略，当你实现自己的分配器的时候，可以对照考虑一下。

表 1.6.1　　分配策略

类　型	描　述
固定大小的缓冲池	所有内存的分配都是一样大小的；减少了每次分配的内存浪费
共享内存	分配使用的是共享内存。参见[Stroustrup97]中的一个例子
多个堆	分配使用不同的堆，视分配大小和类型而定
单线程的	分配和释放线程不安全；在单线程代码里才有用
垃圾回收	调用释放的时候并不释放内存；调用垃圾回收函数的时候才释放内存
基于栈的策略	所有的内存都是在栈上面的。对于生命期短的容器类而言比较有用
静态内存	分配的内存存在于程序的数据区（静态内存）里面
从不删除	调用释放的时候绝不释放内存；当程序退出的时候才回收内存
一次性删除	调用释放的时候并不释放内存；通过定制的函数来释放内存
边界对齐策略	为了满足某些条件，内存边界总是对齐分配。例如在使用页对齐内存或者是在 SSE 中使用指令对齐内存的时候
调试	分配记录、检查内存泄漏、检查内存覆盖（overwrite）情况、峰值分配大小等等

1.6.7　分配器状态数据

正如上面提及的一样，STL 实现可能会允许分配器之间互相等价。这就意味着除非你的特定实现支持可比较的分配器，不然如果分配器存储有任何一种针对特定对象（per-object）的数据或者状态的时候就危险了。如果你需要编写一个在多种实现里使用的分配器，那么唯一安全的方法就是不要在其内含有任何对象的数据。此限制会带来一个好处，那就是如此实现的分配器一般不会造成空间的浪费；其缺点是使用此种分配器有可能会使你的任务更难完成。

要想知道你的 STL 实现是否支持针对对象的数据保存，一个最简单的方法就是检查一下 `list::splice` 的实现。顾名思义，此 `splice` 函数就是将一个列表与另一个列表通过移动指针的方法连接起来，这是非常快的一个操作。现在，假设两个列表的分配器是不一样的——例如，它们可能使用了不同的堆。此时 `Splice` 函数就不能简单地交换指针了，因为其产生的列表将有一部分结点在一个堆上面，而另一些节点在另一个堆上面。如此生成的列表将不知道如何删除其包含的一些节点了！

在上面的列表的 splice 函数中，如果针对分配器比较的结果产生了两个不同的处理过程，则很有可能你的 STL 实现支持针对对象的数据保存。不然，有可能你的实现认为所有的分配器都是等价的。

Download at www.Pin5i.Com

1.6.8 一些建议

下面列出了一些编写定制分配器时的建议：

- 从<memory>中把缺省分配器复制出来，然后改写 `allocate` 和 `deallocate` 函数。如果你的分配器有状态数据，则还要更新建构函数和析构函数。不要忘记了要实现全局的比较函数。
- 一定要记住使用 `allocator<T>`的时候并不意味着只分配 `sizeof(T)`大小的内存。Rebind 函数还允许分配其他类型（因此也是其他大小）的对象。
- 不要忘记了类型定义，特别是处理与模板有关的代码的时候。对使用定制分配器的分配器和容器类使用类型定义。如果你需要作一些改变，那么只需要改变类型定义就可以了。
- 在创建拥有针对对象数据或者有内部状态的分配器的时候一定要小心。你的 STL 实现可能并不支持这种分配器。检查一下你的列表的 `splice` 函数，看看你的实现是否支持针对对象数据的分配器。
- 限于现有的编译器技术水平，并非所有的编译器都是符合 C++标准的。如果有什么疑问的话，检查一下<memory>里面的缺省分配器的实现以得知其实现是如何绕过其编译器限制的。
- 除非必需，否则不要添加一个定制分配器。分配器架构最大的优点就是你只需要改动很少的一段代码就能替换掉原分配器了。

1.6.9 实现细节

在 CD-ROM 上的代码是针对微软的 Visual C++6 和 Visual Studio.NET 编写的。它对 Dinkumware 3.08 和 3.10 版本的 STL 实现做了测试。这些版本都支持针对对象的数据，允许使用真正的比较函数。

例子中的分配器提供了多种策略的实现，包括多个堆的实现、基于栈的实现，还有静态分配器。还有一个分配器只是简单地封装了一下 malloc 和 free 而已。

1.6.10 结论

编写一个分配器只需要拷贝一些程式化的代码和自制很少的关键成员函数。分配器有一些有趣的特性，只有对这些特性有了一个实际的了解才能理解分配器是如何工作的。另外重要的一点是不同的 STL 实现可能会不支持针对对象数据的分配器，而且可能提供了自制的成员函数以避免编译器的问题。把这些复杂性抛到一边来说，定制分配器能极大地改善性能，它肯定是值得了解的。

Download at www.Pin5i.Com

1.6.11　参考文献

[Alexandrescu01] Alexandrescu, Andrei, *Modern C++ Design*, Addison Wesley, 2001.

[Austen98] Austern, Matt, "What are Allocators Good For?," *C/C++ Users Journal*, 在网址 http://www.cuj.com/experts/1812/austern.htm 上有在线资料，May, 1998.

[Austern99] Austern, Matt, *Generic Programming and the STL*, Addison Wesley, 1999.

[Cpp98] ISO/IEC 14882, *ANSI C++ Standard*, August, 1998.

[Dinkum] Plauge, P.J., et al., "Dinkumware Standard Template Library," 在网址 http://www.dinkumware.com 上有在线资料。

[Josuttis99] Joscuttis, Nocolai, *The C++ Standard Library: A Tutorial and Reference*, Addison Wesley, 1999.

[Meyers01] Meyers, Scott, *Effective STL*, Addison Wesley, 2001.

[Plauger01] Plauger, P.J., et al., *The C++ Standard Template Library*, Prentice Hall, 2001.

[Stroustrup97] Stroustrup, Bjarne, *The C++ Programming Language*, Third Edition, Addison Wesley, 1997.

Download at www.Pin5i.Com

1.7 立即存盘

Martin Brownlow
Shiny Entertainment
mbrownlow@shiny.com

最近很多游戏发布的时候都缺少了一个至关重要的特性，与其他游戏特性的缺少相比，这个产生的用户抱怨是最多的。是什么东西会产生这么严重的影响呢？不是别的，就是在游戏的任何地方都能存盘的特性。在任何需要的时候都能保存游戏，这已经成为了游戏玩家的一个共识；而一旦游戏缺乏了这个特性，大家就都会牢骚很大，抱怨个不停。

然而，某些游戏是可以不需要这种功能的。如果你要求在其中的任何地方都能保存游戏的话，某类游戏就根本没有意义了。你能想象在泡泡龙（Bubble Bobble）中随心所欲在任何时候保存游戏么？不幸的是，现代的大部分游戏并不是可以逃避此特性的幸运儿。

可能有人会说如果允许在任何地方保存游戏的话，游戏的难度就会大大降低，这样将影响或是严重损坏一个游戏的耐玩度。这个理由对于不实现此功能来说倒是说得过去。大多数游戏采用的取代方法是只能允许在某些特定的地方才能存盘，此时很容易得到游戏的一些信息，简化了存盘的实现。不幸的是，这个方法一般都会对整个游戏产生很坏的影响。玩家会感到被骗了，而且经常会觉得这个特性是故意被省掉的以让他们不停地玩一个 20 分钟的片段来增加游戏的时间。有个更好的方法，那就是允许游戏在任何地方存盘，但是在游戏中某些关键的地方则要去掉这个功能，例如在和关底老板打的时候。

1.7.1 为何如此困难

要在游戏中的任何地方都能存盘是一个让人头痛的任务——对于外行来说它看起来好像很简单，但是一旦你深入进去着手来做的时候，你就会发现它会涉及到一些难题。

在一个游戏中，即使采取了很高明的对象重用策略，内存也会被分成碎片，而对象列表也会被搞得乱七八糟。因此，在任何时刻几乎都不可能搞清楚是否在内存中的某个地方某个对象正在被创建。解决此问题的一个方法是把游戏中的所有指针都换成句柄，而它们则可以利用一个查找表找出相应的对象来。不过，如果你没有使用句柄的话，还有另一个方法，那

Download at www.Pin5i.Com

就是在存盘的时候把所有的指针都转化为句柄，然后在载入的时候再把句柄转化为指针。值得注意的是，这种方法要求在存盘之前与载入之后做一些额外的工作。

还有一个大问题就是判断每个对象应该保存些什么信息。一旦决定了以后，就需要对每个对象类型都编写一些函数以进行数据的读写。由于大多数游戏里都有很多种类型的对象，因此这可能会是一个繁重的工作。此外，一旦有了任何数据结构的变化，所有的存盘记录就都没用了，而且还要编写新的代码。这就是为什么存盘功能往往被延期处理直到项目收尾才做的原因。在最坏的情况下，存盘的代码将没有被正常测试或是由于要按期结束游戏开发的压力而被完全忽略掉了。

我们需要的是给此项工作添加一些自动化的特性。理想情况下，我们只需要给出每个对象的类需要存盘的数据元素和类型，然后给出一个需要保存的对象的列表，以后就是游戏存盘管理器的任务了。在本节中我们将讨论一个 SAVEMGR 类的架构，它能以上述的自动化过程来管理游戏的存盘。在 CD-ROM 上有这个类的文件。

1.7.2　SAVEMGR 类

设计 SAVEMGR 类的目的是为了让游戏存盘函数的编写更加简易和快捷。一旦实现了此类，游戏存盘的过程就很简单了。只要对所有需要存盘的对象调用 AddSaveObject()函数，然后再调用 Save()函数保存游戏就可以了。而载入游戏甚至更加简单——只要调用一下 Load()游戏就被载入进来。

SAVEMGR 将创建一个列表，把那些通过 AddSaveObject()函数加进来的要存盘的对象都放入此列表中。当 Save()被调用的时候，SAVEMGR 将遍历一遍列表，然后在每个对象上调用其 Save()成员函数。当此对象的一个指针需要被保存的时候，SAVEMGR 将在列表里查找出相应的对象来。对象在列表中的位置（从 1 开始，0 是为空指针保留的）将被作为其 ID，将代替其指针被保存。值得注意的是如果被一个指针指向的对象不在保存列表里面的话，游戏就无法正常存盘了。

当读取一个文件的时候，SAVEMGR 首先将读出文件里面对象的个数。然后对每个对象，它再读出其类的 ID，接下来调用 SAVEMGR::MakeObject()函数以产生一个此类型的空对象。其后，SAVEMGR 再调用对象的 Load()方法以读出对象的数据。当所有的对象都通过此方法被载入了以后，SAVEMGR 就会对每个对象调用其 PostLoad()函数，它是用来将每个对象的 ID 转化为相应对象的指针的。

1.7.3　SAVEOBJ 类

每个存盘的类都应该从 SAVEOBJ 类派生出来，而且必须要实现其纯虚函数 GetSaveID()与 GetSaveData()。另外，一个类可以选择是否需要重载 Save()、Load()，和 PostLoad()虚函数，但是如果需要重载则一定要记住在函数返回之前要调用其基类的相应函数。

它是怎样得知每个对象需要保存什么数据的呢？这就是 GetSaveData()函数起作用的地方了。对每个类来说，都需要定义一个静态数组以指明需要写入存盘文件的数据元素。此数

Download at www.Pin5i.Com

组定义了每个数据的偏移量、类型和长度。GetSaveData()函数会返回一个针对此类的数组的指针。为了简单起见，SAVEMGR 类对于一般的类型定义了几个宏，它们包括指针、分配后的数据，还有相邻的数据块。此外，考虑到类的层次性，另外还有一个宏以使得一个类能得到其基类存盘数据的描述。

GetSaveID()成员函数必须返回每个类的唯一的 ID。此 ID 可以被自定义的 SAVEMGR::MakeObject()工厂函数（factory function）在载入的时候创建正确类型的类实例。

1.7.4 数据类型与扩展

SAVEMGR 类可以通过创建存盘表的宏内建地支持几种不同数据类型的存盘。通过计算给定数据的偏移量和长度，这些宏能自动填充 SAVERECORD 结构体。缺省支持的类型有相邻的通用数据块、指针、分配后的内存，还有基类的存盘表。这些都是通过 SAVEDATA 宏、SAVEPTR 宏、SAVEALLOC 宏，还有 SAVEBASE 宏在存盘表里一一列出的。

SAVEDATA 宏有两个参数：第一个是类的类型；第二个是成员变量的名字。举个例子来说，在 PLAYER 类里面的 mat 成员变量就可以用 SAVEDATA(PLAYER, mat)来声明。类似的，SAVEPTR 宏也是需要类的类型和成员变量的名字作为参数的。

SAVEALLOC 宏需要三个参数——类的类型、内存指针对应的成员变量，还有一个指明分配内存长度的成员变量。

最后，SAVEBASE 宏需要一个参数——一个指向其基类的存盘表的指针。例如，对一个从 OBJECT 类派生出来的 PLAYER 类来说，其存盘表中可能就有这么一项：SAVEBASE(OBJECT::obj_savetable)。如果 OBJECT 类又是从另一个类派生出来的，假设其为 MASTEROBJ 类，那么 OBJECT 类的存盘表中就将包含 SAVEBASE(MASTEROBJ::mobj_savetable)这么一项了。如此一来，则 PLAYER 类就能自动地保存其基类 OBJECT 的数据元素，还有 OBJECT 的基类 MASTEROBJ 的数据元素了。

这些宏的工作过程如下：首先它们找到给定数据元素的偏移量和大小，然后将这些信息写入存盘表中。而 SAVEALLOC 宏则略有不同。这个宏不仅要保存指向分配数据的指针变量的偏移量，还需要保存那个指明了分配内存大小的变量的偏移量。经常我们需要保存一些特定于当前游戏状态的数据类型。举个例子，我们可能需要保存一个改变了的资源。为了做到这一点，我们还需要定义一些宏，而且要在 SAVEMGR 的成员函数 WriteData()、ReadData()，还有 CorrectData()里面添加处理这些宏的代码。对于上面的例子来说，WriteData()必须要将资源指针转化为一个可还原的形式（例如资源 ID），而 ReadData()则必须将之给转化回来。

1.7.5 重载缺省函数

在某些情况下，可能会要求不要把某些数据存盘，因为它是需要从其他的数据给导出来的。这时候我们就需要重载 Save()、Load()，还有 PostLoad()函数了。

当一个对象被保存的时候，存盘管理器会调用 SAVEOBJECT::Save()函数。而它又会调用 SAVEMGR::SaveData()函数，此函数是真正用来写数据的。如果在保存之前需要做一些工

Download at www.Pin5i.Com

作的话（例如要从一个矩阵里导出欧拉角[1]），此时我们必须重载 Save()函数。需要注意的是一定不要忘记了调用基类的 Save()函数以用来写盘。

同样地，Load()函数也可以被重载。不过，只有在你调用了基类的 Load()函数以后类里面才会有数据的。最后，PostLoad()函数也可以被重载以在所有对象载入了以后对每个对象调用。它主要的目的就是要一个对象里所有的指针还原回来，但是它也要把所有没有保存的数据给恢复出来（例如，从欧拉角里面导出一个矩阵出来）。

1.7.6　一个简单的例子

下面的代码展示了如何保存一个简单的类：

```
class PLAYER : public SAVEOBJ
{
public:
    /* constructors/members omitted */
    int       GetSaveID();
    SAVERECORD *GetSaveData();

protected:
    /* This is the table returned by GetSaveData() */
    static SAVERECORD player_savedata[];

    /* This is the data for the PLAYER class */
    MATRIX    mat;
    NTT       *targetNTT;
};

/* The data saved for class PLAYER */
SAVERECORD PLAYER::player_savedata[] =
{
    SAVEDATA(PLAYER,mat),
    SAVEPTR(PLAYER,targetNTT),
    SAVEDONE()
};

SAVERECORD *PLAYER::GetSaveData()
{   /* return the data table for PLAYER */
    return player_savedata;
}

int PLAYER::GetSaveID()
{   /* return a unique ID for this class */
    return PLAY_ID;
}
```

译者注[1] 欧拉角是指的三维空间里一个旋转动作的（x,y,z）三个方向上的角度。我们可以把欧拉角转化为一个旋转矩阵，经过此矩阵的变换，一个点的坐标就可以变换成旋转后的坐标，显然我们也可以从旋转矩阵得到欧拉角。

Download at www.Pin5i.Com

```
/*****************************************
    This is the class factory function.
    It takes a classID and makes an
    object of the correct type
*****************************************/
SAVEOBJ *SAVEMGR::MakeObject( int classID )
{
    switch( classID )
    {
    case PLAY_ID:
        return new PLAYER();
    }
    return NULL;
}
```

PLAYER 类是从 SAVEOBJECT 基类里面派生出来的，它定义了继承下来的纯虚函数 GetSaveID()和 GetSaveData()。GetSaveID()返回 PLAY_ID 值，它可以用来在以后的工厂函数 SAVEMGR::MakeObject()中创建一个正确类型的类实例。GetSaveData()函数返回的是一个 PLAYER 类的存盘表的指针，它定义了需要存盘和载入的数据。存盘表只定义了两个需要保存的成员变量——mat 变量和 targetNTT 指针。

1.7.7 结论

游戏存盘和载入是一项艰苦的工作，它需要细心的数据转化和管理。不过，只要保持小心并使用一点自动化功能，这个任务就能变得简单了，即只需要给每种类型的类的数据维护一张表就够了。在 CD-ROM 上的代码为存盘/载入管理器提供了一个框架。你可以为之添加一些增强的特性。例如，输出流可以再加入一个压缩层或是加密层的处理。你或许还想为每个文件添加头信息以更快地解析文件，或是为之添加一个快照以在文件管理器显示出来。甚至还可以对 SAVERECORD 做一些扩展以得到向后的兼容性。

Download at www.Pin5i.Com

1.8　自动列表设计模式

Ben Board
Dogfish Entertainment, Ltd.
Ben_board@yahoo.com

一个 C++游戏开发人员会发现他经常需要选择性地访问一组所有类型为 T 的对象（假设其为 CAIPedestrian 类），在此处 T 的祖先类是类型 B（让我们称之为 CAIObject）。由于在游戏里一般会有一个列表，里面包含了所有从 B 类型派生而来的对象，因此要想在其中找到数目相对较少的 T 类型的对象，往往需要进行一番费时的搜索。

一个可行的取代方法是创建一个单独的列表，其中只包含了类型为 T 的对象，这样就能独立快速地访问这些对象了。然而，这种方法还有一些问题：

- 我们必须决定此列表本身存储在什么地方。
- 我们必须保证每个对象一创建就加入进列表，而一旦销毁就要从列表中删除掉。
- 必须防止列表元素胡乱地添加或是胡乱地删除。

要创建这样的列表需要在好几个地方添加数行代码，而这很容易导致程序错误，特别是在使用复制和粘贴的时候，或是有可能在其中忘记了添加某一步骤。一个理想的解决方案应该让程序员能够把一个类标志为“须加入列表”，然后，不需要其他的编程，这个类本身就可以创建一个列表，智能地把它存储起来，而且能保证此列表正好包容了创建列表本身的实例。本节就提供了这样的一个解决方案，它使用了一个称为自动列表（Autolists）的设计模式，此模式拥有上述的所有特性，极大地消除了引进程序错误的可能性，而且对每个想要加入列表的类来说只要添加半行代码就足够了。

1.8.1　实现

假设有这么一个类 CListMe。我们希望此类的每个实例在创建的时候加入一个特定的新的列表，并在销毁的时候能自动从列表中删除，而且我们希望能很容易地访问此列表，当然必须要使用合适的面向对象方式来访问了。

自动列表可以通过一个简单的 C++模板类，TAutolist<T>，来达到此目的。TAutolist<T>有以下几种关键的属性：

Download at www.Pin5i.Com

- 它有一个静态的私有成员变量，其类型是指向 T 对象指针的列表。
- 在创建的时候，它把 this 指针转化为 T*类型，然后把自己加入列表。
- 在销毁的时候，它在列表中找到相应的项然后将之删除。
- TAutolist 只暴露了一个只读的接口，这样就提供了一组有用的查询函数而不至于使得列表被直接改变了。

要使得一个 CListMe 类能够被标志为“须加入列表”，只需要让它继承 TAutolists<TListMe>的公共接口就行：

```
class CListMe : public TAutolists<CListMe>
{
    ...
    // no further references to TAutolists required
};
```

由于 CListMe 的建构函数必须调用其基类（即 TAutolists<TListMe>）的建构函数，因此此类的 this 指针（被转化为 CListMe 的指针）就被添加到了列表中去了。当此对象被销毁的时候，逆过程就会发生，指针就会被从列表中删除，而这些都是在其基类的私有函数里做的。在这两种情况下，派生类都不需要为此作其他的工作。

对列表的访问是通过 TAutolists 接口进行的——没有提供直接访问列表对象本身的方法。下面就是一个自动列表使用的典型的例子：

```
CListMe* pLM = TAutolists<CListMe>::GetAutolistFirst()

while (pLM)
{
    // use pLM here

    // finally:
    pLM = TAutolists<CListMe>::GetAutolistNext();
}
```

GetAutolistFirst()和 GetAutolistNext()都返回一个 T*类型的指针，它指向的是一个存在的有效 T 对象或是一个空指针，除非到了列表的结尾。只要能保证列表元素的删除满足上述的条件的话，几乎不可能会出现返回的非空 T*指针指向一个不存在的对象的危险。对于开发人员来说，根本就不需要记住什么东西了。

需要注意此处的显式限定符，即 GetAutolistFirst()和 GetAutolistNext()函数前面 TAutolists<CListMe>::的前缀。在此处严格说来它们不是必要的，因为在这里只有一个函数有这样的名字（下面我们将对此情况作更多的讨论）。

对这么一种实现要想把它用错了倒是比较困难的——它只有两个接口函数，而且都不需要参数。我们可以实现一个使用迭代器（iterator）的系统，但是如果多暴露一个接口类型，就需要在自动列表的循环里多声明一个变量，对列表的查找增加了额外的参数，而且这只会使用到迭代器中的一小部分列表相关的功能而丢掉其他大部分有用的功能了——这对我们前面易于使用的无错的模式来说反而是一个退步。

Download at www.Pin5i.com

1.8.2　实现时的注意事项

现在让我们讨论一下实现时的细节问题。

1. 代价

使用一个自动列表的代价是很小的：只需要一个静态的列表对象，其中的元素每个都对应一个加入列表的对象（正如其他的列表一样），每个列表还有一个静态的迭代器和一个为 TAutolists 的析构虚函数产生的虚函数表项。

2. 嵌套的迭代

在示例的实现中，我们是不允许列表中有嵌套的迭代的。如果当 GetAutolistFirst() 在一个列表上被调用的同时另一个迭代器正在调用栈上被处理着，则此迭代器就会被破坏了。此问题产生的原因是因为一个迭代器在开始新的迭代的时候会被置为 NULL 值，但是如果真的需要使用嵌套访问的话，那可以把 TAutolists 类中唯一的静态迭代器给替换成一个数组（或者列表）的迭代器，以在 GetAutolistFirst/Next() 中管理一个迭代器“堆栈”。

3. 不使用建构函数的自动列表

第二个需要明确的问题是某些游戏会避免使用建构函数和析构函数，而以自己的 Initialize 和 Shutdown 方法来替换，它们将分别在紧接着创建后和刚好在销毁前被显式调用，这样程序可以对对象生命期中的那些重要阶段获得更多的控制。在这种情况下，我们可能需要去掉此模式中一些自动化的特性，具体就是要把对列表进行添加和删除的工作放到单独的函数里面去，比如说 InitializaAutolists() 和 ShutdownAutolists()，然后要求需要进行列表的对象显式地调用这些方法——这一步骤开发人员必须记住，但是比起它带来的好处来说，这是微不足道的一个代价。

4. 向下类型转化（Downward Casting）

可能有人会对在 TAutolists 的建构函数里面做的向下类型转化提出异议。我们也认识到了这一点，但是觉得它不会引起什么实际的问题，这是因为此转化在理论上显然是类型安全的，涉及的编译器不会报错，而且在对其添加、删除和查询列表中也不应该有什么问题。如果利用运行时的类型识别（RTTI），使用一个动态转化代替静态转化的话，则有可能会减轻一点开发人员的担忧。但是实际上是不应该发生问题的。

5. 其他的存储方法

从此模式的名称上来看，它暗示了链表是唯一的（或者是推荐的）用来存储类实例的方法。其实不然。举个例子说，可能还可以定义一个 TAutolists 的姐妹类 TAutomaps，它可以把每个新对象存储在一个 STL 映射类里。实际上，针对实际需要的类的数量和对数据访问的优化要求，你可以创建一系列的类以使用不同的对象集合保存的方法。我们采用（STL）链表只不过是为了简单起见。

Download at www.Pin5i.Com

6. 多继承的使用

在此模式中我们使用了有争议性的多继承（MI），我们需要对此来做一番探讨。为了使得一个已有基类的类被标志为“须加入列表”，它就必须把 TAutolists 作为它的第二个基类。这一点并没有什么大不了的，与 MI 相关的问题是在此模式的使用中遇到的。

使用 MI 引起的最大问题就是名字冲突——在一个继承结构里面的某类可能会从不同的基类继承出两个或者多个同名的函数来。这个问题可以通过仔细的命名规则来解决，我们可以把公共接口 Get*()类函数的名称中都加上 Autolist 这个词。这样在继承结构的其他地方找到它的可能性就很小了。当然，一个名字和自己总是会冲突的。考虑一下此种可能发生却有用的情况，此时 CBase 类是 CDerived 类的基类，而两者都将使用自动列表：

```
class CBase : public TAutolists<CBase>
{
    ...
}

class CDerived : public CBase, public
    TAutolists<CDerived>
{
    ...
}
```

在 CBase 的成员函数中，即使不使用通常的显式限定符，你也可以通过自动列表的接口访问到属于此类的其他成员：

```
void CBase::ExamineOtherCBases()
{
   CBase *pBase = GetAutolistFirst();
   // not strictly necessary to write
   // TAutolists<CBase>::GetFirst(), because this
   // way is unambiguous
}
```

然而，对于 CDerived 类来说情况就不是这样了，它继承了两个名称为 GetAutolistFirst()的函数（一个从它自己的 TAutolists 继承而来，另一个从 CBase 继承而来）。此时，我们就需要指明范围了：

```
void CDerived::ExamineOtherDerived ()
{
   CDerived *pDerived = GetAutolistFirst();
   // syntax error - ambiguous call

   CDerived *pDerived =
      TAutoLists<CDerived>::GetAutolistFirst();
   // no error, and arguably clearer syntax
}
```

在我们的项目里，我们强行规定对所有的自动列表，不管其范围是否具有二义性，都需

Download at www.Pin5i.Com

要显式地指明其有问题的范围。这一方面是为了加强可读性，另一方面是为了培养出一个好习惯。

1.8.3　结论

自动列表设计模式的目的是要减轻通常游戏开发中跟踪所有同一特定类类型的实例，同时还要保证开发人员不需要对每种类型手工维护一个列表（一个容易导致程序错误，需要好几步工作的过程）。

此目的是通过一个模板类，`TAutolists`，达到的。当把它作为类 T 的基类的时候，它就将新建一个包含指向 T 的指针的列表，而且它将其实现隐藏在一个简单的接口后面，使得每个 T 的新实例在创建的时候会被加入此列表，在销毁的时候会从此列表删除——所有这些只需要添加半行的代码就够了。自动列表可以代替你来保管对象。此 `TAutolists<>`类的定义可以在 CD-ROM 上找到。

Download at www.Pin5i.Com

1.9 浮点异常处理

Søren Hannibal
Shiny Entertainment
sorenhan@yahoo.com

大多数开发 Windows 应用程序的开发包在创建的应用程序里面浮点异常都是自动处理的，这使得处理器对于任何可能发生的浮点异常都保持沉默。本节将向你展示如何使得在一个浮点错误产生的时候让自己的程序崩溃，这只需要添加三行代码就够了。本节还将给你一些提示以帮你找到错误发生的位置。

1.9.1 为什么要崩溃

为什么这些浮点错误一定要暴露出来呢？对此问题至少有三个很好的解释：

- **跨平台的兼容性**：由于其他的平台或者操作系统可能对浮点错误不会如此宽容，因此任何可能引起浮点异常的代码都有可能在某些平台上崩溃，而在另一些上运行正常。
- **性能**：视处理器设计而定，一条引起异常的代码可能会比一条正常执行的代码更慢一些。还有，由于很多操作在处理非正常或是无限大浮点数的时候表现不一，代码可能会运行得很慢甚至停止。举个例子，下面的代码当 x 是无限大的时候将会停止，因为无限大的一半还是无限大：

```
int counter=0;
while(x>1.0f)
{
    x=x/2.0f;
    counter++;
}
```

- **避免粗心大意**：虽然大多数浮点错误都是无关紧要的，但是它们的后面可能隐藏着一些更大的问题，往往这些问题会在项目结束前那个晚上的 2:00 浮现出来。

一定要记住，本节的目的不是想为开发人员的程序引进错误，我们的目标只是要把那些本来已经存在的程序错误给暴露出来。因此，最好每个开发人员都能使用本节中的方法来找出和修复那些导致浮点异常的代码。

Download at www.Pin5i.Com

1.9.2　你的程序处理浮点异常么

某些编译器（例如微软的 Visual Sudio 6）缺省是禁止浮点异常处理的。因此，每次在改变开发环境的时候你都应该做一个测试。下面就是一小段代码，当你把它放在一个程序里面的时候，它就会产生一个被零除的错误。在放置这段测试代码的时候你一定要注意，要把它放在程序的核心处，而不是放在刚开始的地方，因为某些库函数，例如 DirectX8 的 `IDirect3DDevice8::Reset()`，可能会在每一次被调用的时候禁止浮点异常处理。

```
volatile float x=1.0f;
volatile float y=0.0f;
for(int i=0;i<10;i++)
{
    x=x/y;
}
```

此段代码使用了 `volatile` 以防止编译器把被零除的代码给优化掉了。当你把测试代码放进程序的时候，很容易看出来你的程序究竟是忽略了浮点异常，还是会导致崩溃。

1.9.3　异常的类型

在打开浮点异常处理的时候，最重要的问题就是应该使用什么类型的异常。Windows 系统规定只能访问 6 种不同的异常：

- _EM_INVALID——非法异常。此异常的处理应该被打开。
- _EM_ZERODIVIDE——被零除异常。此异常的处理也应该被打开。
- _EM_OVERFLOW——溢出异常。有可能需要被打开，除非应用程序需要处理无限大的数字才禁止。
- _EM_INEXACT——对于不精确结果的异常。此异常的处理绝对不能打开，因为大多数浮点运算都会因为有点不精确而设置此标志。
- _EM_UNDERFLOW——下溢异常。当一个操作产生的结果浮点寄存器无法容纳的时候此异常就会产生，而结果就会被舍入为零。此异常处理不应该被打开，因为，举个例子来说，两个垂直矢量的点积在有点不太精确的情况下会产生一个极小的数。
- _EM_DENORMAL——非正常浮点数（denormal）异常。非正常浮点数是一种稍微有点不同的浮点数，而且它是 IEEE754 标准里面定义的最小的浮点数。一个很小的数在变成零之前首先会成为一个非正常浮点数。因此，非正常浮点数是不可避免的。通常情况下，此异常处理不应该被打开。然而，非正常浮点数的处理要比正常浮点数慢得多，因此如果你在测试代码的时候把这个标志设置上了，就可以发现哪些地方可以改进以避免非正常浮点数了。

1.9.4　代码

要打开一个浮点异常处理，其代码是很简单的。程序首先会读出原来的控制寄存器中的

Download at www.Pin5i.com

内容，修改它们，接着再将其写回去。代码如下：

```
#include <float.h>
void enableFPExceptions()
{
    int i = _controlfp (0,0);
    i &= ~(EM_ZERODIVIDE|EM_OVERFLOW|EM_INVALID);
    _controlfp (i,MCW_EM);
}
```

此处的难点在于应该将此代码放置到程序的什么地方去。如上所述，把此段代码放进程序以后，一定要记住对其进行测试。同样的，此测试代码应该放置在程序的核心部分，而不是在打开了异常处理的地方之后。请注意改变浮点异常处理的标志是很耗 CPU 资源的，因此不能在每帧里面都这么做。如果你知道什么库函数会改写控制寄存器，那最好在此库函数调用之前恢复寄存器的内容，然后在调用结束后再设置回来。

在发布程序里的异常处理

你应该考虑一下在自己发布版本的程序里是否要打开浮点异常的处理。对此两方面都有自己的理由，而答案则取决于你程序的类型。对于一个游戏来说，可能最好是禁止掉所有的异常处理。一旦程序关闭，一个浮点错误大概不会造成什么长期的影响。在另一方面则有理由在一个游戏不同关卡的编辑器中打开异常处理，因为有可能在内存里的某关卡数据被破坏了以后又被存盘。

1.9.5 调试浮点错误

好了，现在你打开了浮点异常处理，而且程序也崩溃了。但是发生崩溃时的代码似乎看上去没什么错。那么究竟是什么地方出了毛病呢？

不幸的是，崩溃不是出错后立即发生的。由于现代的处理器是多级流水线的，一个浮点错误将在下一条浮点指令的第一级流水线上被发现，而此指令甚至有可能与发生错误的代码根本不在一个函数里面。因此，最好不要查看源代码级的调试输出信息，而要进入反汇编模式查看真正的浮点寄存器的内容。当你需要定位那些（一开始的时候）看上去无错的程序错误之时请记住这一点。

1.9.6 结论

有三个理由需要对浮点异常进行检查——跨平台兼容性、性能，还有就是要避免粗心大意。检查浮点异常的代码很简单，但是在放置这些代码的时候要注意放置地方，而且一定要对其进行仔细的测试。由于延迟的多流水线的问题，调试信息可能会令人感到迷惑，但是如果不看源代码级的调试信息而查看反汇编信息的话，则此任务会变得容易一点。

没有理由不检查浮点错误，而此做法将可能使程序更稳定、更高效。只要使用了本节提供的信息，你就可以更快地写出更好的代码了。

Download at www.Pin5i.com

1.10　使用 UML 开发一个配合设计的游戏引擎

Thomas Demachy
Titus Interactive Studio
tdemachy@titus.fr

我们都知道，作为开发人员，我们有时候不得不仅仅由于游戏引擎无法进行处理而抛弃一个美妙的游戏设计主意。在另一方面来说，当游戏设计人员对全新的天气控制系统表示不满仅仅由于游戏是发生在地下的时候，我们也会感到失望。

我们都碰到过这种情况，一般都是出于以下原因：那就是游戏设计和引擎开发是并行的任务，即使在设计者和开发者之间有频繁的沟通，但他们各自对游戏的想法都自有一套。

本节将探讨一下游戏设计小组和开发小组之间的协同工作，其方法是通过使用一个通用的的图形语言——统一建模语言（UML）。通过在同一个模型上共同工作，设计者和开发者将把注意力放在游戏上，而且仅仅是游戏上，如此就减少了延期和开发的成本。

1.10.1　对象就在游戏之中

面向对象编程（OOP）是根据现实世界来建模的。让我们来看看一个对象——它是唯一的和可被标识出来的，因此可以使用属性来描述它，例如其形状与颜色。此外，任何一个对象都和世界互相作用，因此也有自己特定的行为。

这两点观察得出的结论就是面向对象方法的基础。这些方法已经成功地在软件行业使用超过 20 年了，而且有很多语言实现了它——举几个例子，SmallTalk、Java，还有 Visual BASIC。

出于种种原因，OOP 最近刚刚随着 C++作为游戏开发的语言而进入游戏界。举个例子，[GPG01]列出了很多由于使用 C++带来的好处而产生的成果。然而，C++不仅仅是在 C 上面多了加号而已，它使用了全新的模式，还有一个有力的口号：“通过对象来思考”。

1．软件和游戏设计共进

最近，游戏设计面临的最大挑战就是如何超越单线索的或是多线索的叙事性情节。在游戏的场景中每个用户的交互都被限制得死死的，此种情况太多了，这导致玩家根本无法自由选择自己想走的路线。

Download at www.Pin5i.Com

在一个名为“游戏设计的未来”[Smith01]的讲座中，Harvey Smith（Ion Storm）对面向对象的游戏设计颇为称道，因为通过它可以得到一个“全局的一致性[1]”（以及其他的优点）。他论证道，通过使用从其他的类继承其性质得到的类对象，我们就可以得到一个更真实的游戏世界。听上去是不是很耳熟？大概是吧。

2. 协同工作的方案

由于游戏设计变得更加面向对象了，现在我们就应该使用同一种语言来协同工作。统一建模语言最早是在20世纪90年代中期引进的，它就可以促进协同工作。UML是一个图形建模的工具箱，它主要是用在软件设计上的。但是从一开始起它的目的就是想将其应用到任何一种设计之中。本节并不想成为一部UML的教材，这方面的资源已经很多了，不管是书籍还是网站，在参考文献中列出了其中的几个。

3. 整个世界就是一个大舞台

在选择设计工具的时候，一定要考虑到游戏设计动态的一面。如果世界果真是一个舞台的话，那么建模语言就需要自己的演员了。对于UML来说情况正是如此。在游戏的情节中，你可以标出每个人物及其动作。从UML的角度来看，一个演员就是一个充当某种角色的实体，例如玩家、一个非玩家人物、一个车辆、……，甚至可以是一个天气控制系统。它引进了参与者、用例、顺序流图，还有协作流图。

4. Groody简介

游戏设计人员和开发人员都使用游戏的情节作为起始点。假设你现在正在做一个2维平台上的游戏，其名为Groody。其情节相当简单，大致如此：Groody是一个在滚动的背景中奔跑的英雄，他可以从一块白云跳到另一块去，但是必须要用吹风机将雨云给吹走。如果Groody被一块雨云给撞着了，他就可能会得一场感冒。

5. 设定需要的用例

第一步是要标出动作和其参与者。对于Groody来说我们可以标出三个能发起动作的参与者——游戏的玩家、Groody和雨云。请注意游戏玩家与其操纵的虚拟人物是两个不同的参与者——你可不会像游戏中的英雄那样在自己的屏幕上奔跑。然后我们需要为情节设计其主要的行为。对于UML来说，这些行为可以被形式化为用例。用例是非常重要的，因为它们定义了整个系统应该如何建构。在图1.10.1中，参与者是用小人表示的，用例是用椭圆表示的。这些对象中的连线代表的是通信，支持的动作有“使用”和“扩展”，它们将几个用例联系了起来。在此图中，Run Through Level的用例就对白云使用了Jump的用例。每个用例都必须使用文本来描述，它应该包括正常的控制流程、其他可选的操作、特殊情况，等等。

译者注[1] 英文原文是global consistency。此处这句话比较突兀，解释一下。根据原文的意思大致说来，Smith是说原来游戏中每个物体都直接和另一个物体交互，例如子弹打玻璃的时候原来必须要专门编一段代码来做这件事，很麻烦，而且维护性差。如果玩家用子弹不小心打到了瓦罐而开发人员没有编子弹打瓦罐的程序的话，就会发生子弹无法打破瓦罐的荒谬现象。但是如果使用了面向对象的方法，将了弹从一个IDamager接口继承下来，可以被打的物体从IDamagable继承下来，就可以以通用的方式处理一类问题了，这样就不会出现子弹无法打破瓦罐的不一致的情况。

Download at www.Pin5i.com

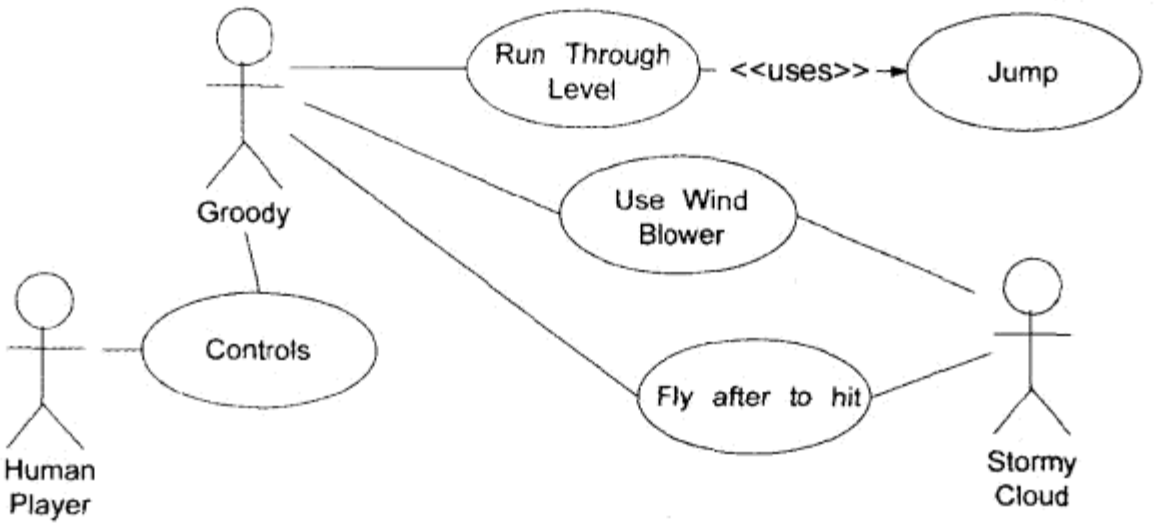


图 1.10.1　Groody 的用例图

6. 细化动作

用例对于快速地创建一个有关设计概念的草案是非常好的，但是它们仍然太抽象了，可能会在更细的概念上产生误解。在一个参与者的用例关系中，每次产生一个新的参与者的实例，它就会执行一个用例的实例，也可称其为“情节”。此情节可以在时间上更加细化，以序列流图的形式表现出来（参见图 1.10.2）。

一个序列流图标识了对象之间动态的交互。如果我们现在将焦点集中到 Use Wind Blower 的用例上，就可以得到下面一个典型的情节：

- Groody 查看一下是否自己仍有可以使用吹风机的能量。
- 对于游戏中的每块云，检查一下是否有一条清楚的视线（Line Of Sight, SOL）。
- 在清楚的视线上的第一块云将被击中化为稀薄的空气。

对每一个用例都定义了几个序列流图。所有这些情节的总和就描述出了一个用例。

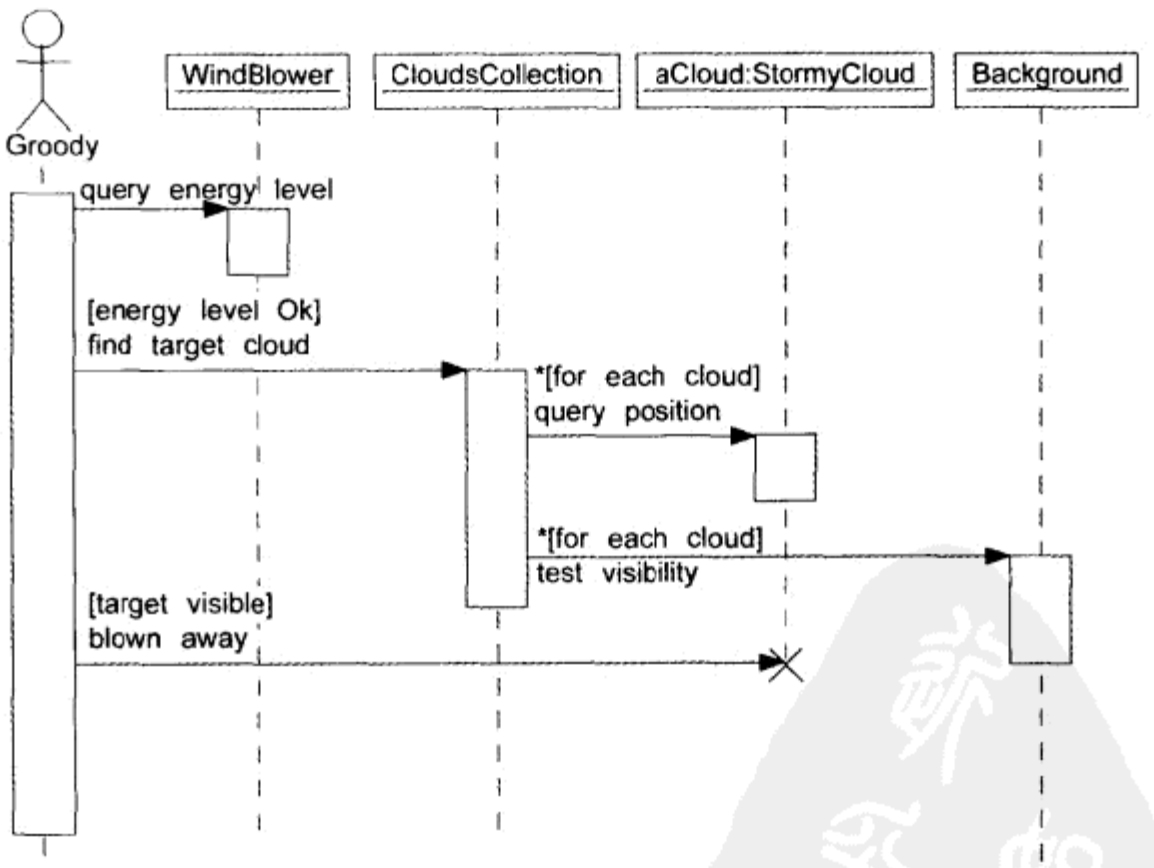


图 1.10.2　Use Wind Blower 用例的序列图（时间方向向下）

Download at www.Pin5i.Com

7. 从序列流图到协作流图

序列流图和协作流图是一个镜子的两面——它们表达的是完全一样的意思，唯一的区别就在于序列流图是基于时间的，而协作流图是基于对象的（参见图 1.10.3）。UML 的魔力就在这个简单的转化之中，它就是模型中动态部分和静态部分的联系纽带。

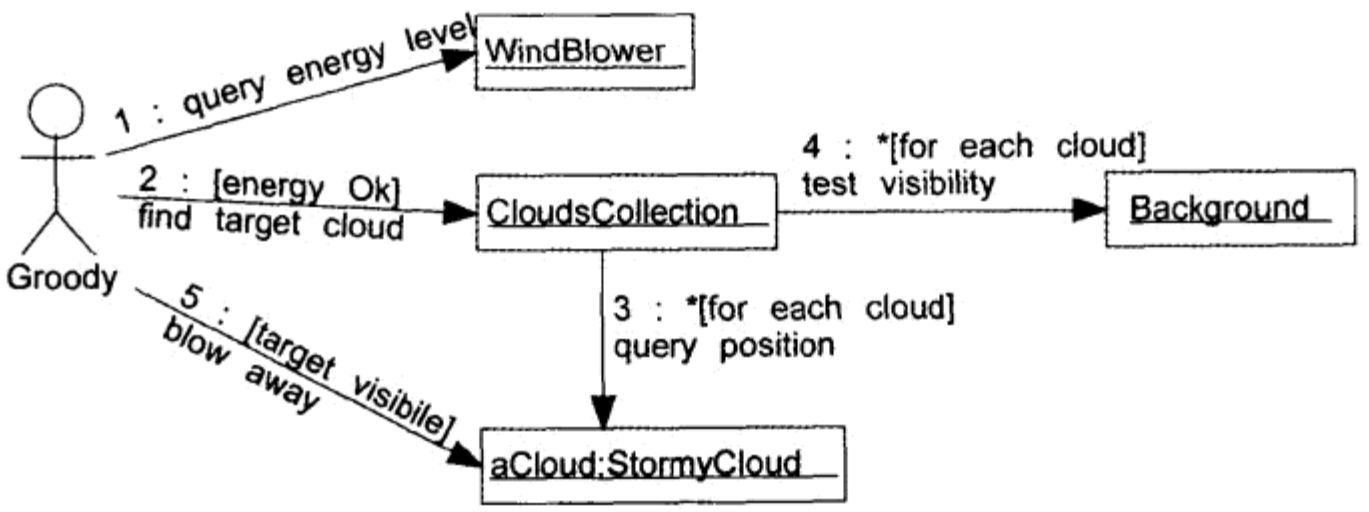


图 1.10.3 Use Wind Blower 用例的协作图

1.10.2 动态的类——正如动态的棋子

围棋游戏在某种意义上有点自相矛盾。双方互相攻击以夺取疆界，其方法本质上是放置一些绝不会移动的棋子。其动态的行为由其静态的棋子产生。我们至今为止主要关注的是对象动态的一面及其之间的关系。然而，在这个游戏的引擎里面，动态的行为是基于对象的属性产生的。

下面，我们将关注此游戏世界静态方面的描述，还有就是 UML 中使用最广的图，那就是类图。

1. 讨论类的两个方式

在更加现实一点的情况中，现代的游戏需要管理上千个对象，每个对象都会和其他的一组对象进行交互。类泛化（以及类继承）是面向对象的机制中最有力的一个。通过把对象设计为类和创建一个层次化的结构，我们就极大地简化了系统的复杂度。

一般情况下，继承从动态分析中是得不出来的，因为继承只是一种对象间属性的联系。在类设计的过程中要在开发者和游戏设计者之间达到有效的协作，一个极重要的方面就是“词汇表”（参见图 1.10.4）。两者都必须拥有同样的面向对象的流图，但是开发人员使用的是“关联”、“聚合”，或者“泛化”这样的术语，而游戏设计人员使用的则是“使用”、“拥有”，或是“是一种”。这些词虽然不同，但是这些概念的图形表达都是一致的。

2. 创建一个对象层次结构

我们可以从协作流图开始。类之间的消息在此处将被转化为发起消息的类的操作，它将在类之间传递。相互通信的类之间是关联的关系，除非这种关系是不对称的。在这种情况下，

Download at www.Pin5i.Com

我们称之为聚合，即如果一个对象是另一个的一部分，而且所有者销毁的时候会导致被所有者也销毁的话，那么它们之间就是聚合的关系。

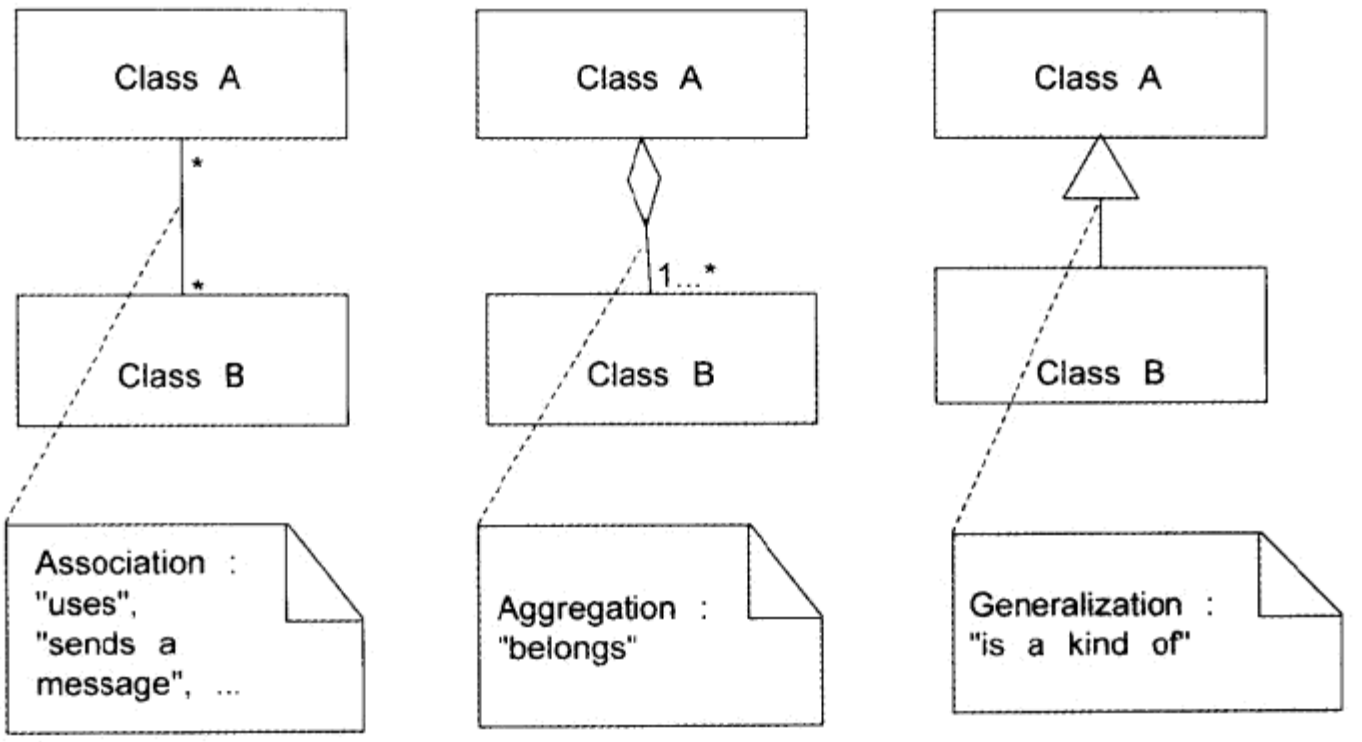


图 1.10.4　每个关联的注释

在协作流图中（图 1.10.3），我们把消息定义为发起消息的类的操作。图 1.10.5 显示的就是从一个用例分析转化而来的类图。

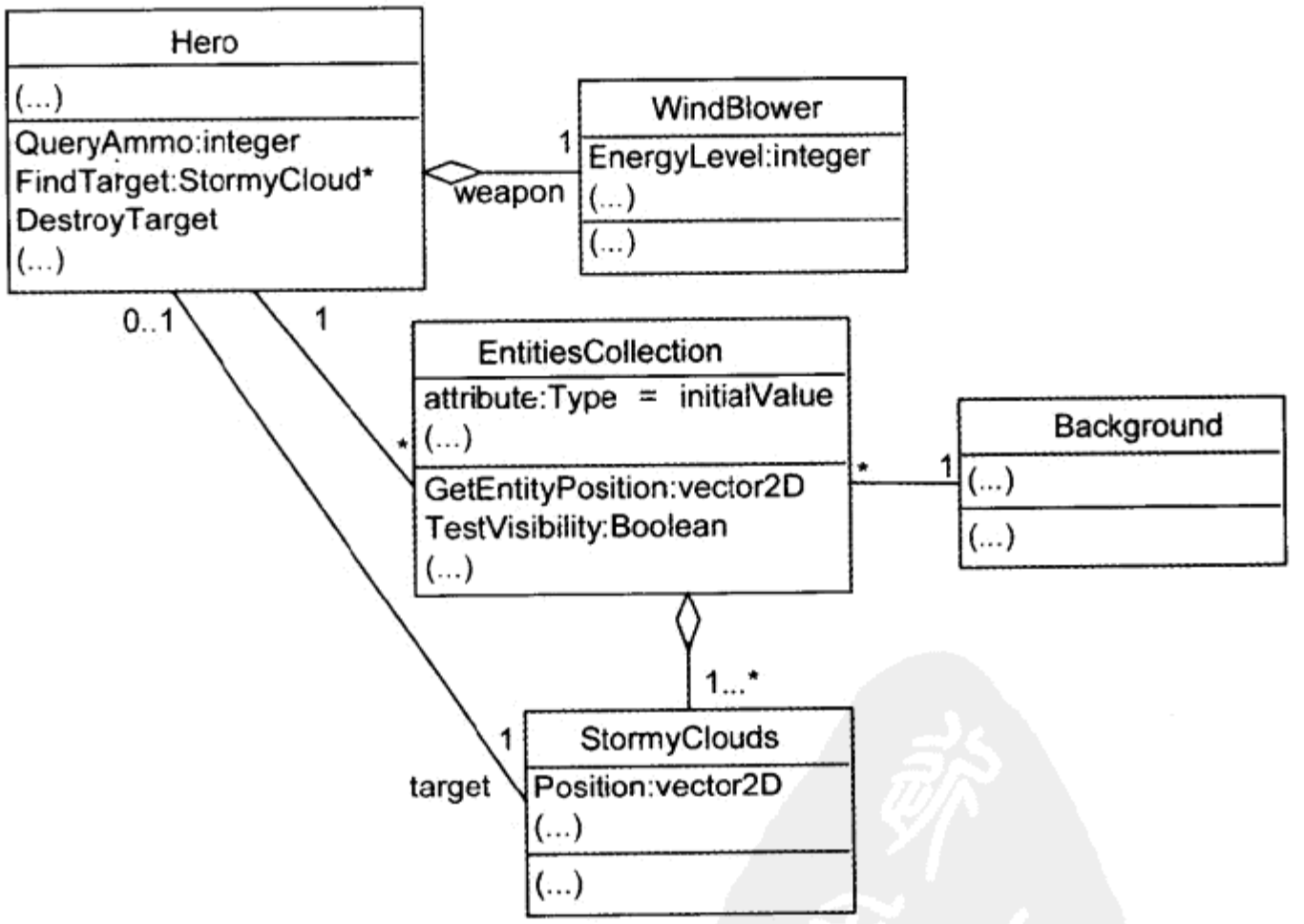


图 1.10.5　经过一个用例分析后的类图（没有画全）

Download at www.Pin5i.Com

1.10.3 协作与迭代

除了可以简化游戏设计者和开发者之间的通信以外，UML 还有更大的用途。隐含于 UML 之中的主要思想就是迭代式的建模过程。这意味着不需要（而且甚至根本不用去想）在开始引擎实现之前就要把整个游戏的设计做完。此过程是增量式的，是一个游戏设计人员和开发人员交互的过程。对于计算机辅助软件工程的工具（例如后面参考文献中列出的那些）的使用可以使得整个过程更顺利。实际上，其中的好些工具都有代码生成和逆向工程[1]的功能。因此，一个开发人员可以先开发然后在需要的时候再去修改设计。

由于其图形工具的本质，一个 UML 模型的修改是很简单的。例如，当引进一种新敌人的时候，假设说是蛇，你可能希望创建一个抽象敌人类，然后将云类从其派生出来。这种修改只要求对类图作一点小改动就行了（参见图 1.10.6）。我们必须引进抽象的敌人类，创建一个蛇派生类，然后利用泛化连线把雨云类与敌人类连起来以表示其继承关系。

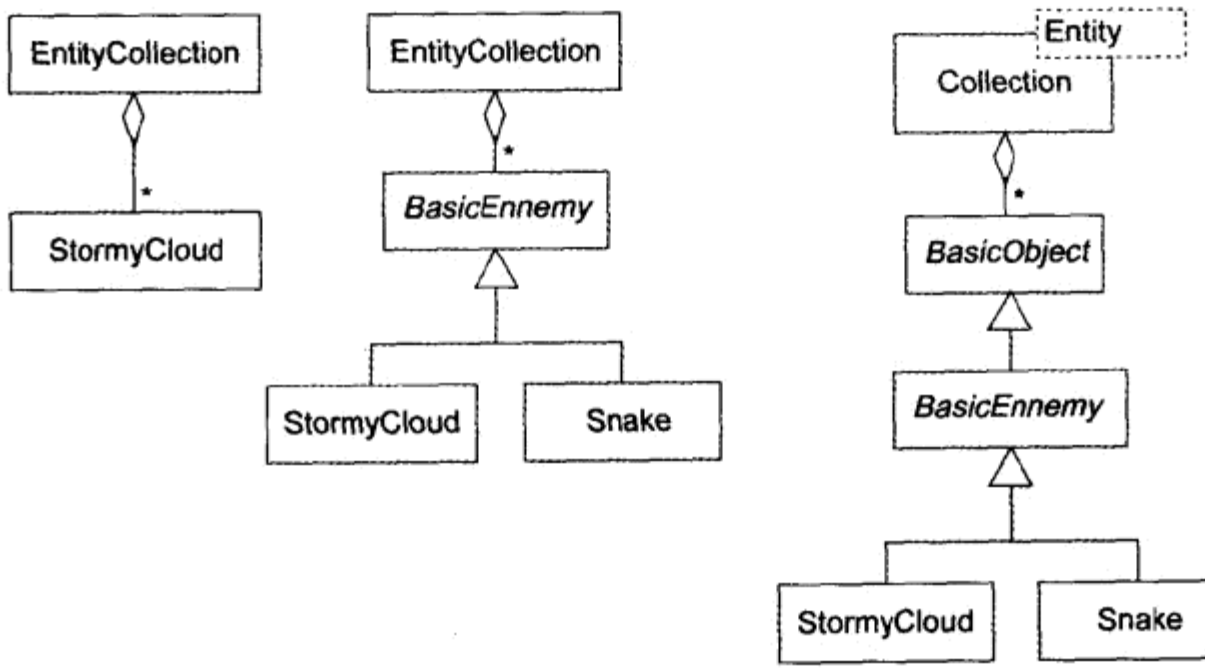


图 1.10.6 对类图进行增量式的修改

当进行源代码实现的时候，它要求进行继承关系的修改，这和使用其他设计技术是一样的。然而，如果你使用的是一个 UML 计算机辅助软件工程的工具，这种代码的修改将会自动进行。

我们可以不仅仅修改类的继承关系，还可以修改其他任何东西。在序列流图（图 1.10.2）中，Groody 参与者负责找到 StormyCloud 目标。如果出于实现的某些原因或是想增加代码的重用性，我们可以引进一个战斗处理类，用来在射击者和其目标之间建立一个接口（参见图 1.10.7）。新的情节将是这样的：

- Groody 通知战斗处理类他将开始射击了。
- 战斗处理类确认是否在吹风机里还有足够的能量。

译者注[1] 逆向工程即指把代码转化回设计的过程。

Download at www.Pin5i.Com

- 战斗处理类在 Groody 的视线里找到一个雨云。
- 当瞄准了一个雨云以后，战斗处理类将通知雨云其已经被击中化为稀薄的空气了。

此新的情节将产生一个新的序列流图（图 1.10.7），也从而将产生一个新的协作流图。这个新的类，CombatResolver，只要简单地插入到类图中就可以了。由此可见，实现也可以对游戏设计产生作用。只要使用了一个计算机辅助软件工程的工具，整个过程只需要几分钟就可以完成。

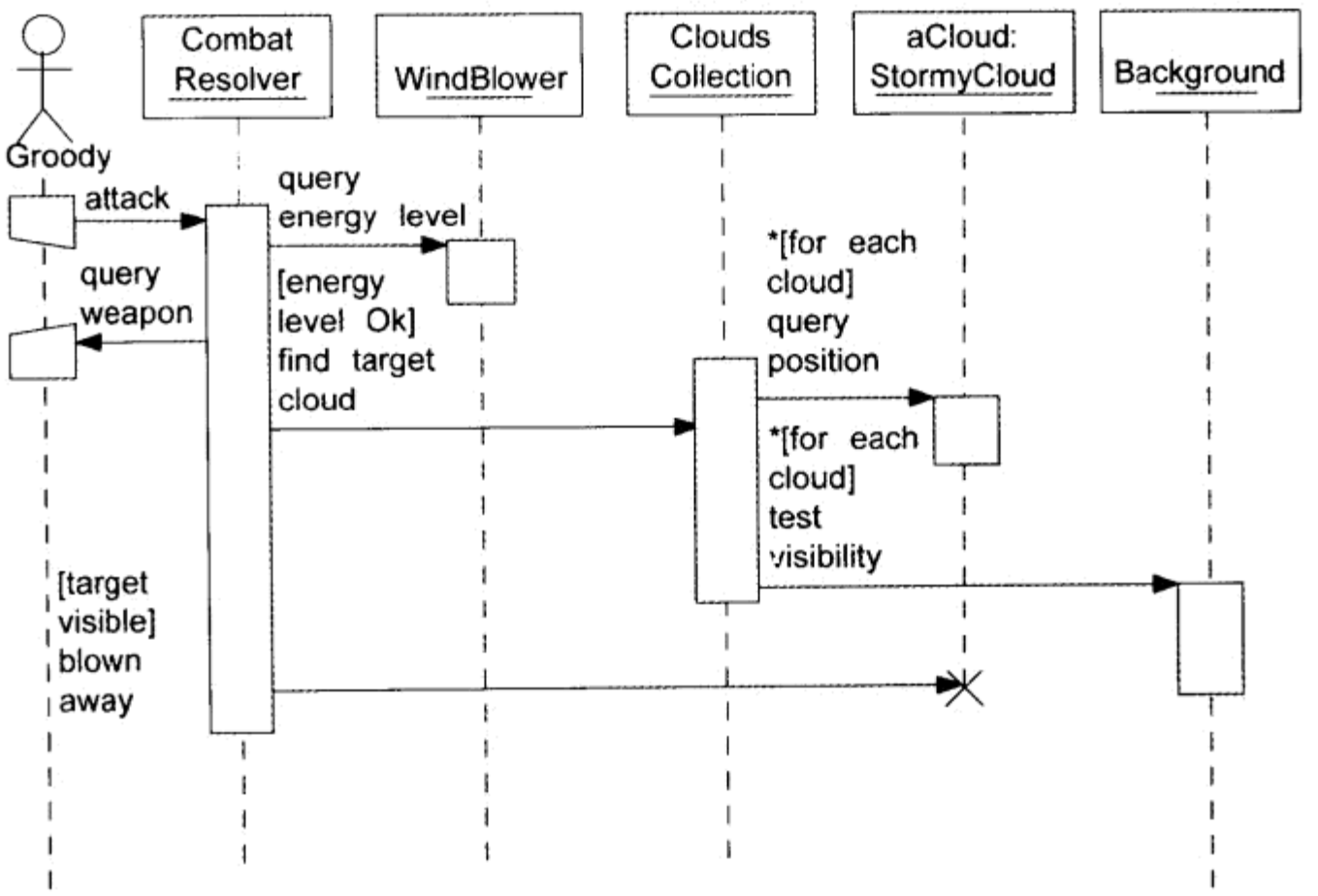


图 1.10.7　经图 1.10.2 修改而来的序列图

1.10.4　实现上的问题

在本节中我们没有写任何代码。这表现了 UML 完全独立于语言的能力。然而，我们可以依据类图在任何时候开始实现。如果你使用了一个 UML 工具，它一般都可以自动地把设计转化为源代码。

1. 硬编码与脚本化

由于 UML 可以被转化为很多种语言，因此你可以选择一下什么可以被硬编码进来，而哪些又可以使用脚本来描述。一般情况下，在开发的早期就会决定此问题。如果使用 UML 设计的话，函数和类都可以放到硬编码里或是脚本里。

如果你使用的是一种面向对象的脚本语言，例如嵌入式的 Python[Python02]，你同样可以使用 UML 来生成此游戏脚本的代码。实际上，还有一种非常适合于行为与人工智能的 UML 流图——状态流图，如图 1.10.8 所示。状态流图在人工智能的状态机中使用很广泛[Dybsand00]。它能展示出一个对象所能经历的不同状态以及其条件转化。

Download at www.Pin5i.Com

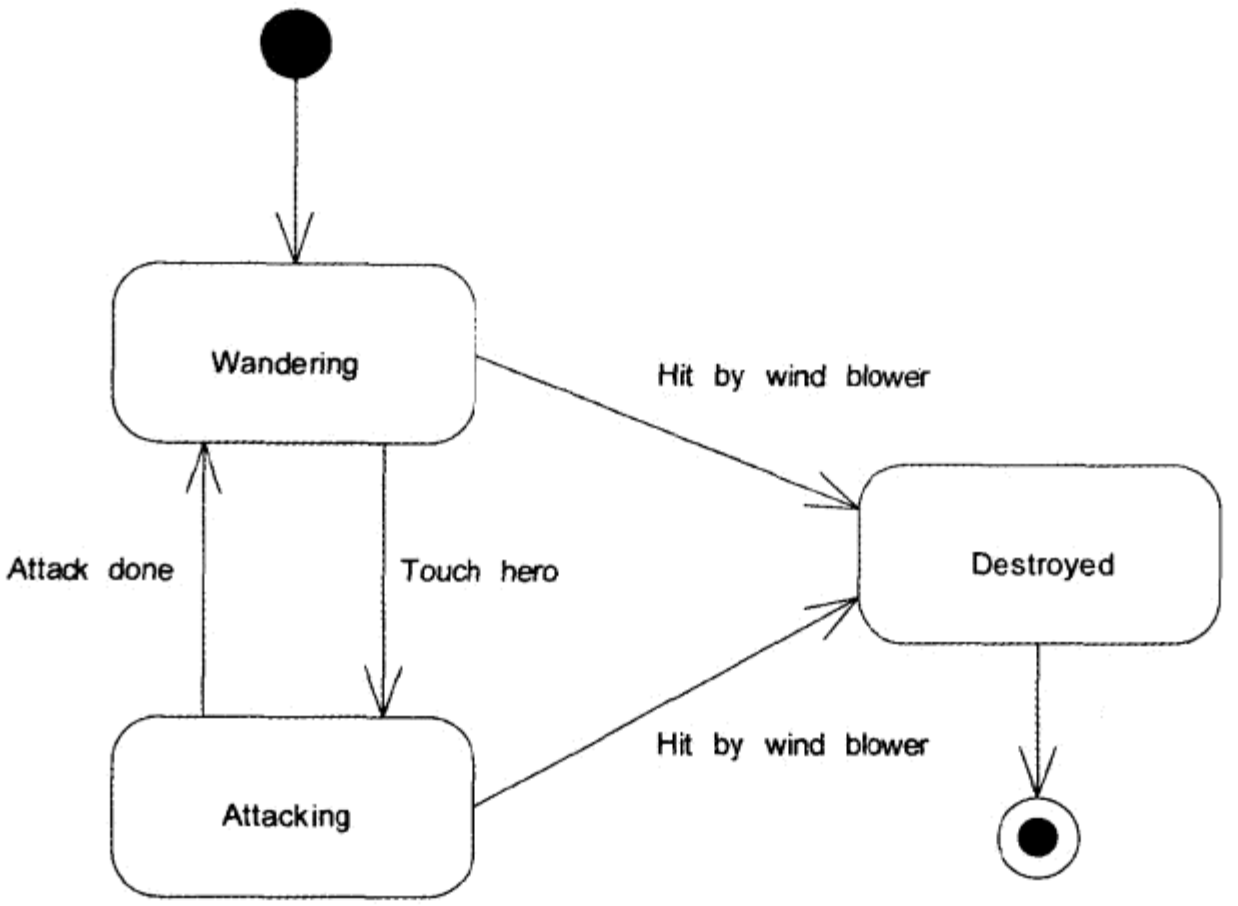


图 1.10.8 Stormy Cloud 类的状态图

2. 在你的项目里使用 UML

有好几本书，如[Muller97]，描述了在很多语言中把类图转化为代码的一些准则。然而，最好的方法就是使用一种能支持整个迭代过程的计算机辅助软件工程的工具，它能生成你选择的语言的代码，甚至还能对代码进行逆向工程。这样，你就有了所有所需的工具，能将 UML 模型和代码始终保持一致了。

现在最流行的 UML 工具是 Together ControlCenter[Together02]和 Rational Rose [Rational02]。后者在 *Game Developer Magazine* 上曾经被讨论过[Sari01]，这表示了它正在进入游戏开发界。另外还有一个开源（open-source）的计算机辅助软件工程的建模工具，ArgoUML[Argo02]，但是由于它是基于 Java 开发的，因此尚还没有实现完整的代码自动生成功能。

1.10.5 结论

本节提出了一些在游戏开发中如何使用 UML 的技术。通过它，我们可以看到使用建模语言可以帮助游戏设计人员创建面向对象的设计，而且可以帮助加强开发人员和设计人员之间的协作。UML 最近的版本（1.3）中提出了很多可以让你的小组协作使用的特性。

通过使用一个计算机辅助软件工程工具，你可以在面向对象方法的基础上增量式地工作，同时还能保持对代码的完全控制，并且有更好的可维护性、更好的文档，以及游戏引擎更强的可重用性。

Download at www.Pin5i.Com

1.10.6　参考文献

[Argo02] ArgoUML Project, 在网址 http://www.argouml.org 上有在线资料。

[Dybsand00] Dybsand, Eric, "A Finite-State Machine Class," *Game Programming Gems*, Charles River Media, Inc., 2000: pp. 237～248.

[GPG01] DeLoura, Mark, *Game Programming Gems 2*, Charles River Media, Inc., 2001.

[Muller97] Mullter, Pierre-Alain, *Instant UML*, Wrox Press, Inc., 1997.

[Python02] Python 2.2 Script Language, 在网址 http://www.python.org 上有在线资料。

[Rational02] Rational Software Corporation, http://www.rational.com.

[Sari01] Sari, Jonathan, "Product　Review: Rational Rose," *Geme Developer Magazine*, July 2001: pp. 10～11.

[Smith01] Smith, Harvey, "The Future of Game Design: Moving Beyond Deus Ex and Other Dated Paradigms," 在网址 http://www.igda.org/Endeavors/Articles/hsmith_intro.htm 上有在线资料。

[Together02] TogetherSoft Corporation, http://www.togethersoft.com.

Download at www.Pin5i.Com

1.11 使用 Lex 和 Yacc 分析自定义数据文件

Paul Kelly
paul_kelly2000@yahoo.com

大多数游戏引擎子系统都非常复杂——它们都需要很多数据来对此子系统的行为进行配置。最好我们能将此数据与代码分离开，在游戏初始化的时候载入进来就可以了。数据的格式应该易于修改，这可以通过一个自定义的数据文件来办到。自定义数据文件主要包含数据的描述和其数据。通过自定义数据文件将代码与数据分离有以下几点好处：

- 为子系统创建一个自定义数据文件能促进对数据的组织，这样就可以很容易地使用一个文本编辑器来修改了。
- 艺术人员和游戏设计人员可以借此改变子系统的行为以对游戏进行调整。
- 数据管理独立于代码。如果数据和代码保存在一起，则开发人员就必须维护数据，这将会成为项目里的一个大瓶颈。

自定义数据文件可以作为一个工具的输入，此工具将把数据的文本描述作为输入将之转化为二进制的形式，然后二进制的文件就可以在游戏初始化的时候直接载入内存。此工具的转化需要使用一个分析器，然而，如果为所有的游戏子系统都编写一个定制的分析器的话工作量就太大了。为什么要重新发明轮子呢？！Lex 和 Yacc，一套可以让开发人员为某个游戏子系统创建其“编程语言”的应用程序开发工具包，可以为你完成大部分的工作。Lex 和 Yacc 可以从文本文件里面抽取出数据然后将数据传给工具以进行更多的处理。本节将展示如何在一个工具里面使用 Lex 和 Yacc 以用来处理自定义数据文件，这些文件将通过此工具被转化为某个游戏子系统的二进制数据。

本节中的各个小节将分别描述如何为一个自定义的数据文件创建词法和语法分析器过程中的各个步骤，以及如何制作一个简单的数据文件格式。开始两个小节将对 Lex 和 Yacc 的基本功能作一个大致的描述。第三小节将谈及如何把两者联合起来以分析数据文件。请参考[Levine92]以得到更完全的关于 Lex 和 Yacc 的资料。

最后两个小节覆盖了使用 Lex 和 Yacc 生成游戏数据的方法：第一个方法主要讨论针对某个游戏子系统从一个自定义数据文件生成游戏数据；第二个方法讲的是如何从中间输出的数据文件生成游戏可载入的数据。最后我们将给出一个范例。本节中的例子都是在非 GNU 的 Flex 和 GNU 的 Bison（而不是 Lex 和 Yacc）上测试通过的。参看本节结尾的如何得到 Flex

Download at www.Pin5i.Com

和 Bison 以得知如何获取这些应用程序。

1.11.1　Lex

Lex 生成的是一个词法分析器（也称为词法器或是扫描器）的源文件。一般说来，一个词法器首先得到一个文本文件，然后将其文本中的字符组合起来形成独立的称为标识符的块，其意义由词法器指定。这些标识符然后将被送到语法分析器[Aho86]。在我们这里，此语法分析器是由 Yacc 创建的。

对于每个 Lex 文件来说都有三个区域：一个 C 代码区域，其后是词法规则区域，紧接着又是一个 C 代码区域。第一个部分用来加入头文件，放置常量、宏、全局变量、函数原型，以及其他词法器和最后的（C 代码）区域要使用到的声明。第二个区域定义了词法器。一个词法器由一个正则表达式来说明，此表达式将用来识别输入中的标识符。最后一个区域是放置代码的地方，举个例子，这些代码可以是用来处理输入的辅助性函数。

1.11.2　Yacc

Yacc 是一个语法分析器生成工具。Yacc 生成的是一个语法分析器（还有可能有一个头文件，其中有标识符的定义，可以被 Lex 生成的代码使用）的源文件。一般说来，一个语法分析器将把词法分析器产生的标识符组合成有特定意义的短语。每个短语可以和一个动作联系起来。在我们使用自定义数据文件的例子里，这些动作被用来判断文本文件应该如何转化为二进制文件。

一个 Yacc 文件的格式和一个 Lex 的格式差不多。第一个区域含有 C 代码，它一般用来加入头文件，放置常量、宏、全局变量、函数原型，以及其他语法分析器和第二个 C 代码区域要使用到的声明。第二个区域含有语法分析器的定义。这些语法分析器的定义由产品规则和相应的动作组成，当一个产品匹配的时候这些动作就会被触发了。最后一个区域是另一个 C 代码区域，它包含了语法分析器需要使用的代码，用来报错和其他输入处理的函数都放在这里。

1.11.3　优点与缺陷

那么，使用 Lex 来编写词法器和 Yacc 来编写语法分析器有什么优点呢？[Levine92]指出即使是一个较小的用 C 编写的用来处理简单命令语言的词法器也是同样功能 Lex 程序长度的 3 倍。这也就意味着 C 版本的程序需要 3 倍的调试。当使用 Lex 的时候，我们只需要调试正则表达式或者是其他用来进行字符串匹配以从文本得出标识符的代码。对 Yacc 来说情况也是一样的。

当然了，使用利用 Lex 生成的词法器和利用 Yacc 生成的语法分析器也有它们的缺陷，这种缺陷对于两者来说也是一样的：它们产生的代码会比你自己写的代码长很多，这是由于 Lex 和 Yacc 中处理其他功能的代码所导致——如果没有用到此功能的话会造成空间的浪费；Lex 和 Yacc 产生的代码运行会比较慢，这是因为对于由 Lex 产生的词法器和 Yacc 产生的语法分析器来说，想要对其进行优化会比对一个自己写的词法器与语法分析器作优化要更困难。

Download at www.Pin5i.Com

1.11.4 Yacc 和 Lex 中的交互

我们现在已经了解 Yacc 和 Lex 是怎样独立工作的了，它们怎样协同工作以为某个工具生成分析器呢？正如图 1.11.1 中所示，由 Lex 和 Yacc 产生的源文件将被编译并连接进一个使用它们的工具里去。此工具将把 Yacc 中的全局变量 `yyin` 当成一个自定义数据文件的指针。然后，此工具将调用 `yyparse()` 开始对输入的文本文件进行分析（参见表 1.11.4 中一个工具的 `main()` 函数的例子）。Yacc 生成的源代码利用对词法器函数（由 Lex 生成的）的调用得到输入（此自定义数据文件）中的标识符。通过使用 Yacc 的 `%union` 和另一个 Yacc 中的全局变量 `yylval`，数据就从 Lex 传到了 Yacc。数据从 Lex 到 Yacc 到工具的传送，利用的都是全局变量。这些全局变量的值都是在语法分析器的动作中被赋值的（参见图 1.11.2）。上面给出的 Lex 和 Yacc 的介绍是一个很简单的描述。若想对 Lex 和 Yacc 做更多的了解，请参看[Levine92]。

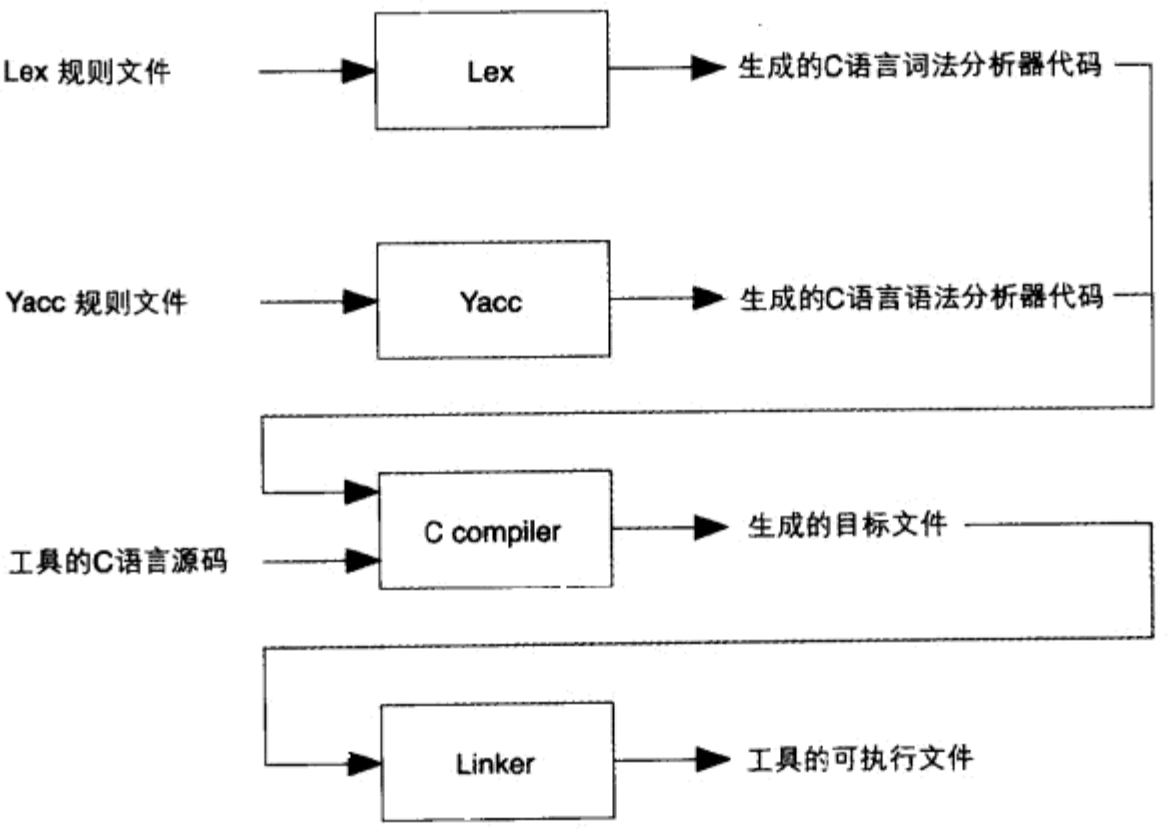


图 1.11.1 根据 Lex 与 Yacc 规则文件生成一个工具的全过程

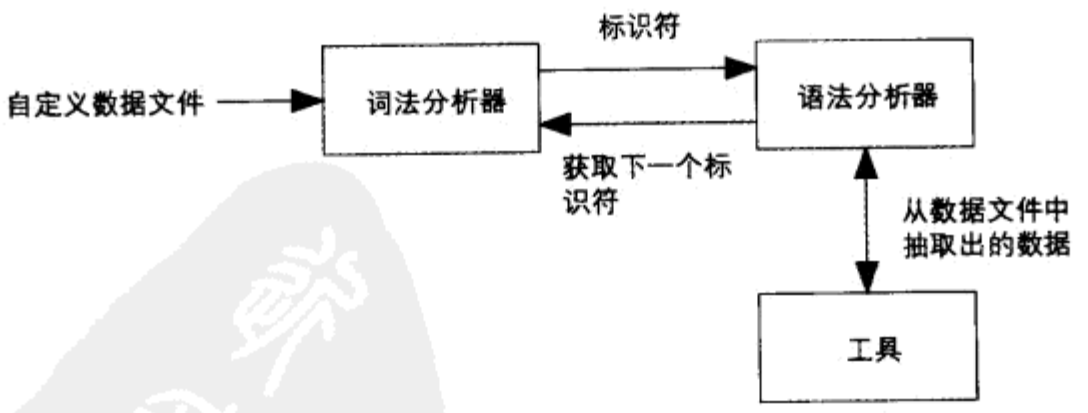


图 1.11.2 词法分析器、语法分析器与使用它们的工具之间的关系

Download at www.Pin5i.Com

1.11.5　针对游戏子系统的自定义数据文件

我们可以使用一个自定义数据文件来指定一个游戏子系统的的数据。自定义数据文件的格式可能会很难定义，然而，大多数游戏数据都有某种结构性。此结构可以用来形成一个数据文件的格式，而此格式既对设计有意义，也能让用户明了。最好先从简单而可用的自定义数据文件的格式开始做起。例如，假设一个游戏有一个武器系统，一个武器可能会有静态数据和动态数据。举几个例子来说，静态的武器数据将包括一个子弹夹里面的弹药上限、弹药的类型和发射频率。静态数据就可以用一个自定义数据文件来配置。表 1.11.1（在 CD-ROM 上有）就是一个武器系统的自定义数据文件的例子，还有此自定义数据文件 Lex 规则的文件，以及此自定义数据文件的 Yacc 规则的文件。

```
START_AMMO
    AMMO 9MM
        DAMAGE   5
    END
    AMMO 50CAL
        DAMAGE   15
    END
END

START_WEAPON
    WEAPON MP5
        AMMO_CLIP   30   /* num bullets in clip  */
        AMMO_TYPE   9MM /* in millimeters       */
        FIRE_RATE   600 /* in rounds per second */
    END

    WEAPON DESERT_EAGLE
        AMMO_CLIP   10
        AMMO_TYPE   50CAL
        FIRE_RATE   150
    END
END
```

此自定义数据文件必须有良好的结构，以便能做出一个 Yacc 规则来唯一地标识出文件中的每个元素。在上面的例子中，我们主要通过给每个小节加上开始和结束符号来到达此目的。更复杂的格式将使用类似的结构，但是可能会内嵌更多的子结构（由一对开始和结束符号包围的区域）。

我们可以使用类似的自定义数据文件、Lex 规则文件，还有 Yacc 规则文件处理启动数据、人工智能参数、各种库存量数据、玩家的属性、车辆的属性，还有其他一些可以利用自定义数据文件的数据。这些系统的数据文件将和武器系统的数据文件相类似。

一个自定义数据文件可以被创建以用在菜单或者 HUD 系统数据上，此时这些元素里面

Download at www.Pin5i.Com

的数据可能会是动态的，但是这些元素本身一般则是不变的。可以为菜单创建一整套规则，将之放在数据文件里，用来创建游戏中所有的屏幕显示。如果你们熟悉 Windows 开发的话，其文件格式将和 Windows 的资源文件很相像。这些规则将描述菜单屏幕上每个元素的位置和其可能的值的范围。

1.11.6 把数据输出工具与 Lex 和 Yacc 结合起来

Lex 和 Yacc 也可以用来分析如 3DMax 此类工具输出的文本数据。如果能把模型、地形和动画数据先输出来作为文本文件以进行人工检查的话，有时候可能会更容易在其中找出错误。然后可以编写一个分析器把文本数据转化为游戏初始化的时候可以载入的二进制文件。此类分析器在本书的 CD-ROM 上有一个例子。Packer 程序将读入一个.md3 的 QuakeIII 模型文件的文本描述，然后会将其转化为二进制形式。Unpacker 在 CD-ROM 上也有，它用来将一个.md3 的二进制文件转化为文本形式。

1.11.7 一个完整的例子

现在我们将提供一个例子（在 CD-ROM 也有），它使用了表 1.11.1 里面武器系统的自定义数据文件格式。首先，我们看看表 1.11.2（在 CD-ROM 也有）里面的 Lex 规则。每当一个新的标识符发现的时候，此词法器代码（由 Lex 生成的）就将被语法分析器（由 Yacc 生成）调用。正如你所看到的，词法器只是简单地返回每一个它找到的标识符而已。如果输入文本是“START_WEAPON”，那么程序就将返回一个 TKN_START_WEAPON 的常量给 Yacc 中被调用出来处理此标识符的函数。Yacc 的函数不断要求得到标识符，直到得到一个完整的语法匹配。除了可以返回标识符的类型以外，语法分析器动作中使用到的值还可以通过 yylval 返回给 Yacc 的函数。

```
%{
/* C code section for Lex specification */
/*** INCLUDES ***/
#include "weapon_y.h"  /* include token macros
                       generated by yacc */

#include <stdlib.h>
#include <string.h>

/*** PROTOTYPES ***/
void RemoveComment (void);
%}

/*** LEXER SPECIFICATION ***/
%%
START_WEAPON      {    return TKN_START_WEAPON; }
START_AMMO        {    return TKN_START_AMMO; }
```

Download at www.Pin5i.Com

```
END                 {    return TKN_END; }
WEAPON              {    return TKN_WEAPON; }
AMMO                {    return TKN_AMMO; }
AMMO_CLIP           {    return TKN_AMMO_CLIP; }
AMMO_TYPE           {    return TKN_AMMO_TYPE; }
FIRE_RATE           {    return TKN_FIRE_RATE; }
DAMAGE              {    return TKN_DAMAGE; }

[0-9]+          {    yylval.integer = atoi (yytext);
                     return TKN_INTEGER; }

"/**"               {    RemoveComment (); }

[a-zA-Z0-9]*[a-zA-Z][_a-zA-Z0-9]*
{   strcpy (yylval.string, yytext);
    return TKN_IDENTIFIER; }

%%
/* C code section for Lex specification */

/* function to remove a comment from the input data file. */
void RemoveComment (void)
{
    int   c1 = 0, c2 = input();

    for (;;)
    {
        if (c2 == EOF)
        {
            break;
        }
        if (c1 == '*' && c2 == '/')
        {
            break;
        }
        c1 = c2;
        c2 = input();
    }
}
```

接下来，我们看看表 1.11.3 中的 Yacc 的规则。Yacc 的规则由一个 Backus-Navr 范式（BNF）文法[Aho86]和内嵌的动作组成。此 Yacc 规则的开头是词法器返回的值，由%union{}符号包了起来。接着是词法器可能返回的标识符的类型。这些常量将在头文件里面定义（我们需要在使用 Yacc 的时候加上一个命令行参数以生成此头文件，而此头文件的名字必须被 Lex 的规则文件所包含）。Yacc 规则的第二个区域是语法分析器的定义。当一个语法分析匹配产生了以后，与此匹配相关联的任何动作都会执行。例如，当“WEAPAN MP5”进行分析的时候，动作 `strcpy(weapon_tbl[weapon_cnt], $2->string);`就会执行，然后分析会继续进行，直到整个输入文件处理完毕（假设没有错误产生的话）。此动作规定要将产品规则

Download at www.Pin5i.Com

的第二个标识符的对应字符串值（由 Lex 返回的）复制到武器列表里面去。此处的$2 表示的是这个标识符和要求 Lex 返回的数据域。这是一个联合体的域，因此一定要特别小心！在此处，$2->string 表示的就是字符串“MPS”。

```
%{
/*** INCLUDES ***/
#include "weapon.h"
#include <string.h>
#include <stdio.h>

/*** GLOBAL VARIABLES ***/
extern char *yytext;
%}

/*** TOKENS ***/
// return types for tokens
%union
{
    int     integer;     // for INTEGER token
    char    string[80];  // for IDENTIFIER token
}

// used by the lexer as return values so that the
// parser knows which tokens have been found
%token TKN_START_WEAPON TKN_START_AMMO TKN_END
%token TKN_WEAPON TKN_AMMO
%token TKN_AMMO_CLIP TKN_AMMO_TYPE TKN_FIRE_RATE
%token TKN_DAMAGE

// tokens that can return a value from Lex to Yacc via // yylval
%token <integer> TKN_INTEGER
%token <string> TKN_IDENTIFIER
%%

/*** YACC SPECIFICATION ***/
// weapon_data is the start symbol
weapon_data:      /* lamda rule - empty rule */
                | weapon_data weapon_section
                | weapon_data ammo_section

weapon_section:   TKN_START_WEAPON weapon TKN_END
weapon:
    /* lamda rule - empty rule */
    |  weapon TKN_WEAPON TKN_IDENTIFIER
          { strcpy (weapon_tbl[weapon_cnt], $3); }
       weapon_attribute TKN_END
          { weapon_cnt++; }
weapon_attribute:
    /* lamda rule - empty rule */
    | weapon_attribute ammo_clip
```

Download at www.Pin5i.Com

```
    | weapon_attribute ammo_type
    | weapon_attribute fire_rate

ammo_clip:
    TKN_AMMO_CLIP TKN_INTEGER
        {weapon [weapon_cnt].ammo_clip = $2;}
ammo_type:
    TKN_AMMO_TYPE TKN_IDENTIFIER
        {strcpy (weapon [weapon_cnt].ammo_type, $2);}
fire_rate:
    TKN_FIRE_RATE TKN_INTEGER
        {weapon [weapon_cnt].fire_rate = $2;}

ammo_section:    TKN_START_AMMO ammo TKN_END
ammo:
    /* lamda rule - empty rule */
    | ammo TKN_AMMO TKN_IDENTIFIER
            { strcpy (ammo_tbl[ammo_cnt], $3);}
      ammo_attribute TKN_END
            { ammo_cnt++;}
ammo_attribute:
    /* lamda rule - empty rule */
    | ammo_attribute TKN_DAMAGE TKN_INTEGER
        {ammo [ammo_cnt].damage = $3;}
%%
int yyerror (const char *msg)
{
    printf ("%s at '%s'\n", msg, yytext);
    return 0;
}
```

表 1.11.4 展示了一个工具里面 main() 函数大致的代码，此工具使用的是 Lex 和 Yacc 生成的词法器与语法分析器。

```
#include "y.tab.h"

WEAPON_DATA weapon;

int main (void)
{
    FILE *in;

    in = fopen ("weapon.data", "r");
    yyin = in;
    yyparse();

    /* process data in weapon and write out as binary
       data */

    return 0;
}
```

Download at www.Pin5i.com

1.11.8 结论

本节展示了可以怎样使用Lex和Yacc来为自定义数据文件创建分析器。利用Lex和Yacc，你可以生成一个强大的分析器。花一点功夫把这些工具给搞清楚，此时你花费的时间将在未来为你节省十倍的时间。

1.11.9 如何得到Flex和Bison

Flex（可以在GNU网址上找到的一个Lex的版本）和Bison（一个GNU的Yacc的版本）都可以免费从GNU的网址[GNU02]上得到。参看[Flex02]和[Bison02]以得知如何从FTP站点上进行下载。

1.11.10 参考文献

[Aho86] Aho, Alfred, et al., *Compilers: Priciples, Techniques, and Tools*, Addison Wesley, 1986.

[Bison02] Free Software Foundation, March 2002. Bison Flex在站点ftp://ftp.gnu.org/gnu/bison/上可以下载到。

[Boer01] Boer, James, "A Flexible Text Parsing System," *Game Programming Gems 2*, Charles River Media, Inc., 2001.

[Flex02] Free Software Foundation, March 2002. Flex在站点ftp://ftp.gnu.org/gnu/non-gnu/flex/上可以下载到。

[GNU02] Free Software Foundation, "GNU's Not Unix!-the GNU Project and the Free Software Foundation(FSF)," 在网址http://www.gnu.org/上有在线资料，March 2002.

[Levine92] Levine, John, et al., *Lex & Yacc*, O'Reilly & Associates, Inc., 1992.

Download at www.Pin5i.Com

1.12　为世界市场开发游戏

Aaron Nicholls
微软公司
aaron_feedback@hotmail.com

为多语言市场开发游戏通常是到了后期才考虑的一个问题，这种情况直到最近才有所改变。当核心语言的产品已经准备好要发布（或者快要发布）的时候，支持其他语言的编码工作才会开始，而其后马上就是相应的测试和本地化的工作。这样生产的产品一般都会延期，而且在语言支持方面也非常的贫乏。然而，在最近的几年里，很多公司都开始为多语言市场同步发行游戏了，而例如 *StarCraft* 和 *Diablo 2* 这样的游戏在全世界范围拥有了很大的销售额。

为了要为一个游戏占领全方位的市场，理解与全球化相关的技术上的障碍和过程是十分重要的。此外，把全球化的技术和整个开发过程集成起来也同样是必须的。在本节中，我们将要探讨一下与面向全世界游戏开发相关的设计、开发和测试上的问题，还有解决这些问题的一些方法。

1.12.1　市场潜力

传统上来说，很多游戏公司趋向于只关注它们的本地市场，而忽略了一个高质量游戏的全球市场潜力。很多游戏在美国市场上赢得了好评，获得了很大的销售额，但是没有人去把它们引入到新的市场上去。在另一些情形下，一个游戏在全球范围内发行，但是游戏却是全部使用的英语——只有手册被翻译过来了。某些特定的游戏迷们，还有那些熟悉外语的热情的游戏玩家可能会努力来玩一个并非其母语的游戏，然而由于越来越多的人开始使用 PC 和游戏机，游戏开发就不应该只面向那些老玩家，也应该适应新玩家了。

在过去的几年中，已经有几个游戏给我们展现了国际游戏市场之大了。例如，在 2000 年和 2001 年中，韩国就被证实是一个很大的市场。*StartCraft* 和 *Diablo 2* 的发行量都很大，都在世界范围内销售了 300 万套，而其中有约 1/3 的销售额是在韩国，即每个游戏都在那里销售了超过 100 万套。这些游戏在某种程度上成为了一种文化现象，游戏中的角色做成了玩具并印刷到了薯条袋上。

这种成功的现象不仅仅出现在销往世界的美国游戏上。在 2002 年 1 月，销售额排行第一的巨量多玩家在线角色扮演游戏（massively multiplayer, online role-playing game）并非 Ultima Online、Asheron's Call 或是 Everquest,

Download at www.Pin5i.Com

而是韩国的游戏 Lineage，它销售了 400 万套——而且它甚至还尚未在美国和欧洲上市。有了这些成功案例之后，就很容易理解在设计开始的时候需要考虑到全世界市场的重要性了。

1.12.2 门面事，先处理——显示和输入

当处理国际性游戏开发的时候，最先碰到的一个挑战就是要把对应的语言显示出来。此外，你还可能要为游戏的每个语言版本提供不同的输入设置和定制方案。

1. 字体

首先，你需要为所有必须显示的字符提供所需的字体。虽然在某些游戏中仍然使用位图字体，但是很多游戏已经开始为 ANSI 字符集和 Unicode 字符集的显示分别使用 TrueType 和 OpenType 字体了。Unicode 字体包含了多种语言的字符，但是它们产生的文件也会大得多。

值得庆幸的是，Windows 和 Mac OS 最新的版本都支持多种语言的输入和显示，当然也包含了相应的字体。然而，游戏一般会使用自己独有的字体（在安装/卸除中使用的字体可能除外），而且在获取或者开发字体的时候考虑游戏面向的市场是十分重要的。此外，要牢牢记住，虽然一个 8 号字体对英文的显示也许非常清楚，但是对一个繁体汉字的显示就可能很不清晰了。你设计游戏的时候一定要相当灵活，能考虑到这一点才行。

2. 分行和排序

如果你的游戏支持多行文本的输入和显示，可能产生的一个问题就是对于多行文本的分行。对于英文和其他欧洲文字来说，分行问题相对比较简单，但是对于很多其他语言来说，事情就不那么容易了。举个例子，在中文和日文中，一般说来，空格并不是用来区分一个句子里面单独的单词的。取而代之的是句子由没有空格的字符组成，虽然也是可以被打断的（但是边界是以两个字节计）。对于这种情况，我们可能就需要使用更先进的规则来决定什么时候换行和回滚文字了。当处理多字节文本的时候，你可以采用以下的策略：一行可以在双字节字符之前、之后或者中间打断。然而，如果需要在单字节字符中间进行换行的话，你应该在最后一个空格或者双字节字符处换行。

另外，除了换行的问题，在不同的语言之间排序规则也是不一样的，而使用字符编码的顺序进行排序一般是不够的。幸运的是，在操作系统里一般都定义了这样的规则，这些规则以系统 API 的形式提供开发人员使用。

3. 输入法编辑器（IME）

当你能让字符正常显示了以后，还应该能正常处理输入的问题。虽然一些游戏只允许英文输入，但是当要存盘、和其他玩家聊天或者要临时做一些游戏中的记录（对于那些提供了这样功能的游戏而言）的时候，这就不太好了。很多玩家都不是使用英语作为母语的，而要求输入仅限于英语则会让玩家分神，阻碍玩家完全沉浸到游戏中去。虽然提供欧洲语言输入

Download at www.Pin5i.Com

的支持对于英文输入支持只需要作很小的改动，但其他的语言，例如韩文、中文和日文，可能都需要投入大量的工作。

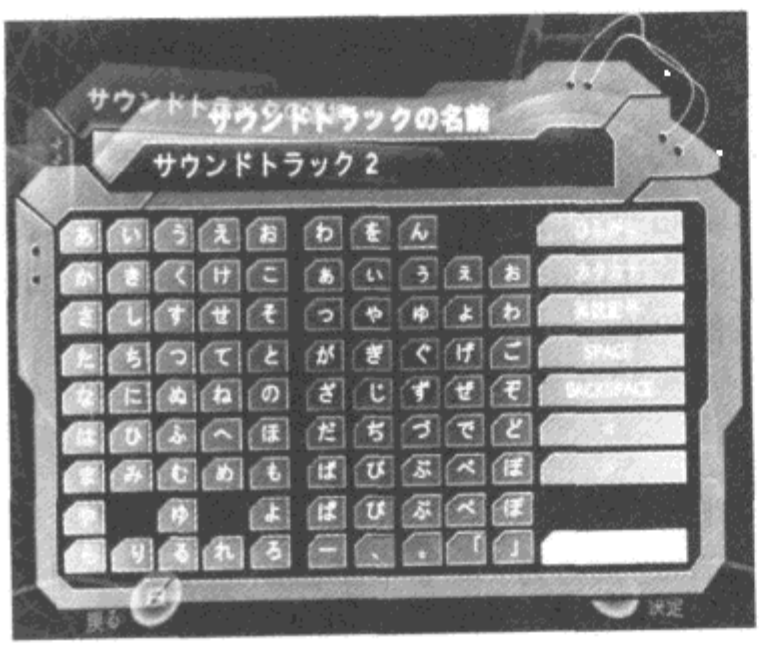
支持这些语言的输入在实现上的第一个问题就是，它们中的每一种都有上千个字符——对于一个键盘来说实在是太多了。当开发电脑游戏的时候，还有可能实现一个或者直接使用现成的输入法编辑器（Input Method Editor, IME）。一个 IME 是一个程序，它能把键盘扫描码（还有最近才开始使用的笔驱动或者鼠标驱动的输入）转化为字符编码。

举个例子，如果一个用户需要输入日文“sushi”，他会首先打开 IME（通常使用一个用户自定义键的组合，或者日文键盘上的一个专用键来激活 IME）。IME 被激活了以后，在用户输入的时候，它同时截取英文的“s-u-s-h-i”或者相应的日文的发音。然后 IME 会向用户给出对应于这个发音的一系列可选的词。当用户选择了相应的词（或者字符串）的时候，真正对应于这个日文输入的字符编码就会被传递给相应的应用程序。如果选择在自己的游戏里面支持 IME 输入的话，你就应该熟悉一下对应游戏平台的 IME 消息代码和结构（参考文献中列出了 IME 相关信息的 Web 连接）。

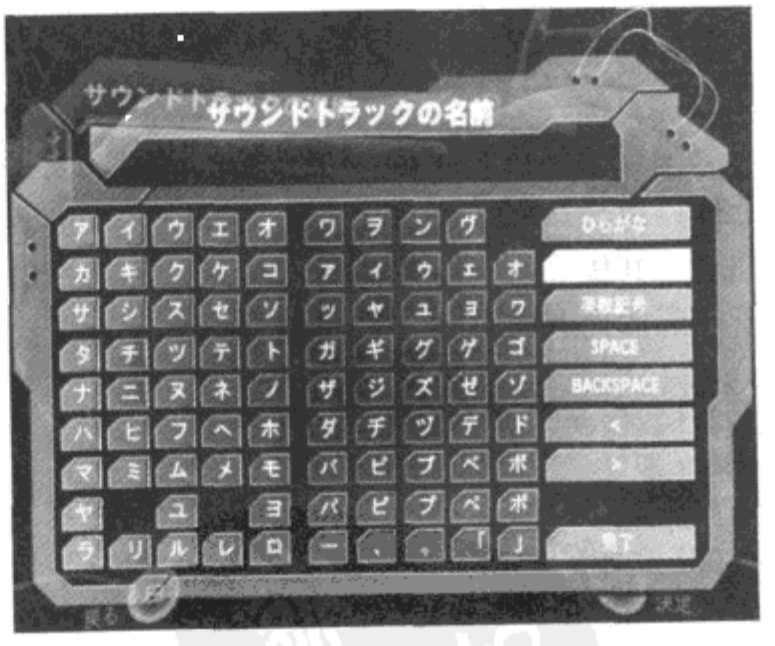
4. 不使用 IME 的输入

不幸的是，IME 并不总是会有的。考虑到用户界面设计和开发中的限制，可能实现一个 IME 或者使用现成的 IME 都不现实。对于游戏机来说这种情况司空见惯。然而，只要对游戏使用的语言有一些基本的知识，就有可能向用户提供某种程度上的输入支持。

例如在一个日文游戏里面，为了要让用户能够用日文输入玩家名称，可以提供一个虚拟的屏幕上面的键盘进行输入，这个键盘上只有片假名和平假名（统称为假名）字符。这是两种发音字符，都可以用来表现日文里面的任何一个字符。虽然每种字母都可以完全表现任何一个日文字符音，但是每个都有自己的用处，有点像在英文里面大写字母和小写字母的关系。实现一个用来输入日文片假名或者平假名的键盘和一个输入英文大小写字母的键盘在屏幕上占用大约同样大小的空间。在图 1.12.1 中显示了这样的一个键盘，这个例子是从微软日本版 XBox 的用户界面中截取出来的。当然，开发人员还需要衡量这种做法的优点与它对测试和开发造成的影响。

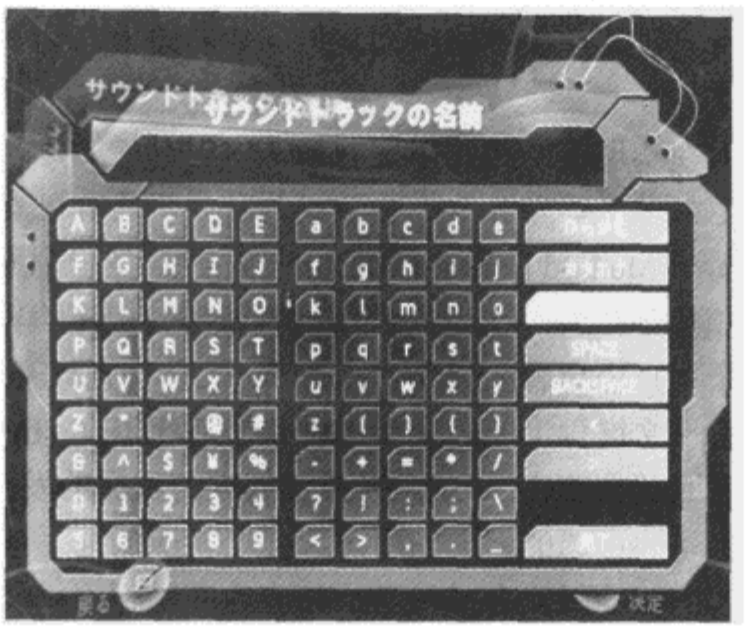


A



B

Download at www.Pin5i.com



C

图 1.12.1 （A）为日文平假名字符的屏幕输入键盘（B）为日文片假名字符的屏幕输入键盘
（C）为英文字符的屏幕输入键盘。这些是微软 Xbox 中的截图。这些屏幕截图得到了微软公司的授权许可

5. 单位与显示的格式

开发一个游戏的时候，要保证用户界面和游戏本身分离开是很重要的。要想使游戏让用户用起来觉得很习惯，方法之一就是给用户的界面要对应用户习惯，要使用用户可能偏爱的日期、时间和数字的格式。如果保存文件的时候的格式是 DD/MM/YY、而显示给用户的格式却是 YY/MM/DD 的话，用户就会被搞混淆。同样的问题在处理数字和时间格式（应该是 1 234.56 还是 1234.56 呢？）的时候也会出现，还有就是在速度、重量和距离的单位上也要考虑到。

虽然大多数时候只要选择使用目标市场上最常用的缺省格式就可以了，但是一定要记住某些操作系统（特别是 Windows 和 MacOS）可以让用户自己选择这些格式的很多细节。此外，一些游戏机生产厂家把格式对于不同的地区进行了硬编码。如果你的游戏能将这些设置读取出来的话，你就可以自动地让用户使用他自己最喜欢的格式了。

1.12.3 字符集

当在一个游戏里面实现对不同语言的支持的时候，往往有几种不同的字符编码模式可以选择。虽然在某些平台上面有逐渐使用 Unicode 的趋势，但是对于每一个字符编码模式来说都是有其优点与缺点的。一般说来，你选择的模式应该取决于要使用的平台、目标语言、遗留代码的问题和其他一些没有为世界化作好准备的代码。

1. 单字节字符集(SBCS)

除了字符的显示以外，还有一个问题需要处理，那就是字符编码的问题。在处理传统的 ASCII 文本的时候，储存一个字符只需要一个字节。这就称为“单字节字符集”，它经常用来对英语和其他的欧洲语言进行编码。

然而，即使是对于单字节字符集来说，也有很多种编码方法。标准 ASCII 码使用 7 个位

Download at www.Pin5i.Com

来存储字符，第 8 个位原来是用来做奇偶校验的。这也就是说它能存储 128 个字符，对于英语文本、标点符号和少数几个控制字来说这已经足够了。糟糕的是，这对于很多其他的欧洲语言来说都是不够的，因为它们还需要表达重音符、除英文 26 个字母以外的其他字母，还有一些在英语里面一般不会用到的符号。

为了要能表示这些额外的语言，涌现出了几个 ASCII 码的扩展建议。由于奇偶校验再也不需要了，因此整个 8 位都可以利用上了，字符 0～127 还可以用来和原来 ASCII 码的表示一致，128～255 则可以用来存储额外的字符、标点符号和符号。例如，在 Windows 里面，这些字符是使用的一种被称为 ISO-8859 的 ANSI 字符集来表示的。然而，为了要表现不同的语言，又开发了多编码页（或称为对字符的数字编码）。一般说来，SBCS 的编码页仅仅在高端的 128～255 号字符上面有不同的定义。然而，由于单字节字符集最多只能表示 256 个字符，它仍然不能用来表示甚至仅仅是全欧洲的语言，当然也不能表示字符多得多的语言，例如中文、日文和韩文（它们一般统称为 CJK）了。

2. 双字节与多字节字符集(DBCS/MBCS)

为了表达更多的字符，人们开发了更大的字符集。从名字上面就能看出，一个双字节字符集使用两个字节表示一个字符，而多字节字符集使用变长的多个字节来表示一个字符。这其中的一个例子就是日文的 Shift-JIS 编码模式，它是用一个或者两个字节来表示一个字符的。

在 Shift-JIS 系统里面，0x00 到 0x7F 的单字节字符编码是专门为 ASCII 字符保留的，0xA0 到 0xDF 是为单字节日本假名字符所用的。双字节编码由一个头字节和一个尾字节符组成，头字节处在单字节字符所在的范围之外的地方，这也就是说它占用了 0x80 到 0x9F 以及 0xE0 到 0xFC 范围的字节。这样的系统可以使用这种扩展编码的方法对成千上万的字符进行编码，但是开发这样的系统就很麻烦了，因为字符的长度是不定的，这就需要在字符计算、文字编排和光标的移动方面进行算法的改进。此外，由于不同的 MBCS 系统的头字节的范围是不一样的，你编程的时候就要对不同的亚洲语言考虑不同处理。虽然对于 Shift-JIS 这样的系统来说，双字节字符集这个术语也经常被使用。但是在 C/C++标准里面 MBCS 则是其专用的术语，而且此术语也更加恰当。

3. 为不同字节大小的字符集编程

当处理不同 SBCS 数据的时候，编码页虽然是要考虑的问题，但是这里字符和字节却通常是一一对应的关系，不要求什么代码的改动。内存分配、光标移动、搜索、编辑和换行也保持着比较简单的特性。然而，如果是对 MBCS 或者 DBCS 数据编程的话，必须要牢牢记住以下事实：那就是一个字符并不总是对应一个字节的，反过来也是一样。

让我们从为 DBCS/MBCS 的字符串分配内存开始谈起，如果把字节和字符搞混了的话，这就会带来灾难性的后果。例如，如果一个玩家的名字是用 16 字节的一个字符串来表示的，而输入算法使用字符而不是字节来计算用户输入长度，此时就可能产生一个缓冲区溢出的错误，导致系统不稳定，甚至导致安全问题的出现。此外，界面上还要以字符而不是字节为单位来处理光标的移动、编辑和搜索。

特别要指出的是搜索是另一个容易出问题的地方。举个例子，当分析一个文件路径的时候，必须要分清楚斜线符或者反斜线符与用来表示一个双字节字符或者多字节字符的尾字节，

Download at www.Pin5i.Com

即使它们可能使用同样的字符编码。如果没有考虑到这一点，则会导致文件载入的错误、游戏存盘的错误和其他的错误，而这些错误只会在使用特定语言的特定字符时才会重现。

由于这种错误太难以定位了（而且经常在发行后被毫无关联的不同用户发现），所以在测试的时候就需要测试小组理解这一点，对这些情况进行测试以保证它们不会在最终的产品中出现。首先，必须要保证在开发过程中所有的字符串搜索和分析算法都考虑到了 DBCS 和 MBCS 的情况。另外，要挑选出这样一套“危险字符”，并把它们加入到所有的字符串算法的测试中，这样就能在产品发布阶段之前有效地检测出这种问题了。

4. MBCS 特定的问题

在 MBCS 系统中间，编辑甚至更为复杂。举个例子，即使只是敲了一个退格删除键就会导致一个字符串分析算法的触发，以用来决定到底需要删除多少个字节。首先，我们并不能马上知道在光标前面的字节究竟表示的是一个单独的字节还是一个多字节字符的一部分。要知道这一点，我们可以每次倒退一个字符直到碰到了此字符串的开头或者是一个肯定是某个单字节字符或者多字节字符的串的开头。直到此时，我们才能得知到底需要删除那个字符里的多少个字节了。

在最坏的情况下，可能我们需要分析到字符串的开头。此时，可能从整个字符串头开始分析效率还高一些，特别是当字符串比较短的时候。但是，我们还有另外一个选择。很多系统都有 MBCS 相关的函数来处理类似的问题。例如在 Windows 中，CharPrev、CharNext 和 IsDBCSLeadByte。这些函数可以使得这个过程变得更简单，因为这些功能已经被它们封装成一个简单的系统调用了。

5. Unicode

值得庆幸的是，有一种方法可以使用一种编码集来支持多种语言，同时也不需要处理编码页和变长字符的问题。Unicode 标准使用了一种定长两个字节的格式定义了大约 40 000 个字符，它可以在一个编码页中支持主要语言大部分情况下显示需要的所有字符、符号和代码。虽然此标准同样有扩展，可以用来支持数百万的字符和极多的语言（其中甚至有 Klington 语言！），但是双字节的 Unicode 已足以满足绝大多数游戏的需要了。而且它既灵活，又有通用的本质特性，这可以让它成为一个满足很多真正的、面向世界的游戏需要的编码系统。如果想要获得关于 Unicode 的更多的信息，可以参考[Unicode02]或者最新出版的标准。在本书写作的时候，最新的版本是 Unicode 标准 3.0。

1.12.4 界面和设计方面的考虑

当开发一个面向多个市场游戏的时候，除了要考虑以上提到的字符输入、编码和显示的问题以外，还有几个其他方面有关游戏界面和设计的问题必须要考虑，它们包括设备输入和视频输出，还有用户界面设计和游戏的内容。

1. 视频输出

当为游戏机开发游戏的时候，一定要牢牢记住视频标准。虽然在美国、加拿大、墨西哥

Download at www.Pin5i.Com

和日本使用 NTSC 标准，它每秒播放 30 帧（场频则是 60 帧/秒），但在其他国家中主要使用的是 PAL/SECAM 制式，它每秒播放 25 帧（场频 50 帧/秒）。由于有这种情况，在渲染前的视频可能需要同时提供两种格式。此外，最后要保证你的游戏不要和一种特定的标准保持同步渲染或者同步动画。

2. 键盘输入

在开发一个面向全球市场的 PC 游戏的时候有一个常见的麻烦，那就是键盘输入的问题。虽然游戏杆和 gamepad 在大多数情况下都是标准化的，但是各个地区的键盘则有可能各不相同。某些游戏只能处理预先设想好的一套键盘扫描码（代表了一个键在键盘上的特定位置），或者虚拟键盘码，或者虚拟键（一般代表的是键上表示的字符）。如果想要游戏在多个语言市场上都受欢迎，一定要保证对于不同的键盘布局它都能工作正常。

这个问题在玩家使用不同的键盘布局玩游戏的时候就会产生。在最佳情况下，他们可能会发现在游戏中无法把某些键定义为功能键。在最坏的情况下，键的名称会被搞混或者无法在游戏里面发挥作用，这是一种非常令人恼火的情况。例如，在美国的英文键盘上的右括号“]”的扫描码对应的是德国键盘上的“+”和法国键盘上的“$”。除此之外，虚拟键的高端上的某些编码在不同的语言里面是不一样的，特别是 0xba～0xbf、0xc0、0xdb～0xde 和 0xe2。如果想要了解扫描码、虚拟键和 ASCII 字符编码的更多的细节，请参看表 1.12.1。幸运的是，如果使用更高级的输入函数，例如 Windows 的 DirectInput 的话，就可以把你和此处某些实现上的细节分离开。它提供了一个抽象层，还包含了一些用来把编码转化为键的函数。

表 1.12.1　　虚拟键、键盘图案，以及对应扫描码的 ASCII 码

	英语			德语			法语		
扫描码	虚拟键	图案	ASCII	虚拟键	图案	ASCII	虚拟键	图案	ASCII
0x0c	0xbd	-	0x2d	0xdb	ß	0xdf	0xdb	)	0x29
0x0d	0xbb	=	0x3d	0xdd	`	0xb4	0xbb	=	0x3d
0x1a	0xdb	[	0x5b	0xba	ü	0xfc	0xdd	^	0x5e
0x1b	0xdd	]	0x5d	0xbb	+	0x2b	0xba	$	0x24
0x27	0xba	;	0x3b	0xc0	ö	0xf6	0x4d	m	0x6d
0x28	0xde	'	0x27	0xde	ä	0xe4	0xc0.	ù	0xf9
0x29	0xc0	`	0x60	0xdc	^	0x5e	0xde	?	0xb2
0x2b	0xdc	\	0x5c	0xbf	#	0x23	0xdc	*	0x2a
0x32	0x4d	m	0x6d	0x4d	m	0x6d	0xbc	,	0x2c
0x33	0xbc	,	0x2c	0xbc	,	0x2c	0xbe	;	0x3b
0x34	0xbe	.	0x2e	0xbe	.	0x2e	0xbf	:	0x3a
0x35	0xbf	/	0x2f	0xbd	-	0x2d	0xdf	!	0x21
0x56	0xe2	\	0x5c	0xe2	<	0x3c	0xe2	<	0x3c

①：德国键盘中没有。

②：法国键盘中没有。

③：美国键盘中没有。

3. 独立于语言的用户界面设计

先不管一个游戏里使用的字符编码和输入法系统，还有一个通用性的游戏开发问题，那

Download at www.Pin5i.com

就是用户界面设计和文本布局。在本地化的过程中，当把不同语言的资源载入游戏的时候，这是一个非常棘手的问题。就算你的游戏引擎和用户界面设计得很好，能处理很多字符类型，从经验上来看你在第一次进行本地化的过程中肯定会遇到显示大小改变或者重叠的问题的。

当开发一个用户界面的时候，最好能在用户界面和数据存储的时候多留一些空间，这是因为某些语言可能会使用比较长的字符串或者不同的字体。举个例子，把一个英文的字符串翻译成德文往往会变得比以前长很多。如果一个用户界面在创建的时候只考虑到了英文，其中的图形按钮或者用户界面元素只能刚刚放下英文字符串，那对德文来说或许就不可能产生同样的界面了，除非进行修改或者进行大量的文字简化。

一个经常被引用的准则是，当使用英文作为原始语言的时候，开发人员应该在用户界面上预留30%到50%的位置给字符串作为扩展用。除此之外，短字符串有可能需要更大的扩展比例，特别是如果想要从英文产品做成德文版的时候。这样一来，当开始做那些需要更长字符串的语言版本的本地化工作时，你的用户界面就不太可能需要作改动了。

此外，可能还需要能容纳更高的字体，这样才能把比较精细的中文或者日文给清楚地显示出来，特别是在游戏机上，它们使用的电视显示设备一般分辨率比较低，也没有点锐化渲染。如果用户界面没有为这种改变预留空间的话，用户可能就会看到一个不友好的界面或者不易看清的界面。另外一个可能发生的问题是重叠，在不同语言之间由于尺寸和格式的差异，文本字符串可能会不适当地重叠到一起。虽然字符串的重叠和间断可以被游戏中用户界面的手工测试发现，但是也可以让这个过程自动化，编程检测可能的字符串的问题，在形成产品之前把它们发现出来。

4. 独立于文化的游戏设计

除了要保证界面是为了目标语言进行设计的之外，还要保证你的游戏不要在与文化相关的方面作任何不成立的假设。举个例子，如果你的游戏中的角色沿路的左边驾驶的话，美国玩家就会惊异地发现车辆居然从他们的右边迎面驶来，除非你能在开始就告诉他们这一点。还有，如果你在用户界面上有个“呼叫”按钮使用了一个图标表示它的功能，那最好能使用一个相对通用的图标，而不要使用一个老式的转盘电话的图标或者是传统的英国电话亭作图标。

5. 文化和政治敏感问题

最后一个问题是文化和政治敏感性的问题。虽然很多游戏的场景是设置在一个虚幻或者科学幻想环境中的，但是游戏本身可是在真实世界里发行的，一不小心它就会违背文化传统或者是政府标准。举个例子，如果你要开发一个有暴力主题的游戏的话，那就可能需要考虑把调子降低一点，或者是提供一个过滤的方法去掉可能的侵犯性的内容，这要视你针对的市场而定。

除此之外，一定要记住游戏希望销售的市场所在的政治环境。例如，如果你的游戏显示了一个有争议地区的地图，那就一定要小心，不要触犯了目标市场所在地的玩家或者官员们。除此之外，当处理有争议的地区时，不要把它们称之为国家。国旗是另一个敏感的问题，这是由于它们可能变动比较频繁（而且一个地区也可能会有类似的问题）。最后，你一定要小心保证自己的游戏不要触犯到了当前的政府、地方或者民族。当制作一个面向全世界的游戏的

Download at www.Pin5i.Com

时候，多考虑一下游戏内容可能带来的政治后果是很重要的，不然的话，你可能就会冒导致游戏被目标市场的玩家抵制甚至被完全禁止的风险。

1.12.5　本地化

当你已经为自己的游戏开发好了相应语言的显示、输入和数据处理的功能以后，还有一个问题——你的游戏还没有真正变成目标语言的游戏。本地化是一个内容翻译的过程，同时它还涉及到要为另一种语言市场作软件生产的准备而调整界面和内部设置。在理想情况下，它应该不涉及到代码的改动。在设计和开发过程中应该小心谨慎，以保证本地化过程进展顺利。

1. 硬编码的字符串

本地化过程的最大障碍极有可能就在于硬编码的那些字符串和资源。开发软件的时候有一个标准的经验做法，那就是把所有的字符串都保存在一个资源文件里面，和源代码分开。这会使得访问和修改游戏里的文本变得很容易。如果这样的字符串被放在了源代码里面而不是资源文件里面的话，要找到它们或者是修改它们都是很难的，虽然如果这些字符串在产品的生命期里面保持不变的话，这可能不是个问题。

在本地化产品的时候这个问题变得更加严重了，因为显示出错误语言的文本是根本不能接受的。举个例子来说，在 Windows 里面，很多系统路径在很多产品里面都被本地化了，但也不是所有的情况都做了类似的处理。在这种情况下，任何时候要使用系统路径（和对这种字符串进行内存分配）都要对系统进行查询，而不是把它们硬编码进来。为了防止硬编码问题阻碍本地化的进程，最好建立并遵循一套适当的编码标准。

2. 过度本地化

虽然本地化对于一个成功的产品来说很重要，但是防止所谓“过度本地化”的情况出现也同样重要。在这种情况下，本地化的过程做得太多了，以至于把不应该本地化的资源都修改了。例如，游戏里面物体的内部名称就不应该本地化，因为它们没有暴露给用户看到，如果改变它们名称的话则可能使得依赖于它们的代码无法运行。为了防止过度本地化的情况发生，所有的资源和字符串都应该加上一个标志，表示它是否应该被本地化。除此之外，当对对话或者是媒体进行本地化的时候，你可能需要在本地化之前把内容给记录下来，这样以保证不管是什么语言，玩家都会有同样的体验。举个例子，如果一个角色在美国版本里说一口很烂的英语，那么在本地化的法语版本里面，这个角色的剧本和嗓音也一样要能刻画出这个人物来，使得他的法语也不太流利。

3. 本地化媒体

对于非文本的资源来说本地化的工作就更复杂了。例如，如果游戏里面有一些包含着文字的图形资源，期望本地化人员利用合适的软件和技巧来对它们进行编辑和本地化显然是不明智的。取代方法是，所有图形里内嵌的文本都应该保存在一个单独的层里和一个字符串的表关联起来以方便本地化。这也使得修改原语言产品文本的工作变得容易了。除此之外，图

Download at www.Pin5i.Com

形应该足够大以保证本地化过程中可能产生的的文本换行或者字体变化有足够的空间。

不仅如此，在制作音效或者视频的时候，要记住本地化的产品可能会运行更长的时间，特别是如果原语言的版本中包含快速说过的文字的时候。因此，最好要保证游戏里面的音效/视频剪辑足够长以适应变长的本地化版本，也要保证玩游戏的时候不要被这些（不同语言版本之间的）差异影响。除此之外，如果你的目标是多个语言市场，还要记住全动态视频（FMV）本地化要比本地化使用游戏引擎的视频剪辑昂贵得多。在项目的早期就应该决定 FMV 和游戏内部视频剪辑的使用比例。一个避免这个问题的变通的方法是仅仅本地化每个视频剪辑说明对白的字幕，这可以大大减少媒体同步中的问题，但是你需要进行一番权衡，究竟是要采用这些便易之处，还是要避免这种不完全本地化有可能对用户的游戏体验带来的潜在影响。

本地化的另外一个问题就是要保证游戏在不同的语言中有同样的感受。举个例子，如果你选择把游戏中的音效或者视频进行了本地化，最好也要保证原来角色的性格和嗓音尽量不变。如果一个角色说话听起来神经质而急躁，那这种特点一定要在所有的语言里都表现出来。另外，如果游戏中有一个角色是一个纽约匪徒，那么在游戏的日文版本中，你就需要挑选一个配音演员能够使用现代的大阪黑帮那种模式化的嗓音说话，前提是如果这样做符合设计中的人物类型的话。最后，一定要记住，如果你在游戏里面使用了一个名人的形象，有可能在国际市场上的其他地方并没有多少人认得他或者她。

4. 字符串和音效视频的连接

最后一个关注的要点是媒体和字符串的连接。由于在句法结构和语法上的不同，把多个字符串连接起来组成一个句子的过程可能会充满了问题。在此种情况下，如果游戏中间非要使用字符串连接的话，请注意在早期就考虑到这个问题以保证晚期本地化的成功。对于音效和视频来说，连接的问题就更大了，在本地化这样一个产品的时候，一定要格外的小心。

1.12.6 设计和规划中的考虑

一旦你理解了开发一个面向世界市场游戏的核心问题，下面就需要把这种理解集成到设计和开发过程中了。对于同一个游戏的多个语言版本的规划来说有几种可选的方案，首先是可以从开始就选出一些小组来关注核心语言产品，以便项目后期制作本地化版本。这种方法会带来几个问题。

首先，本地化的版本可能会被大大的推迟，从而会丧失一次成功发布会带来的兴奋之情。此外，这种方法会导致本地化和测试的工作量大增，而且可能会严重影响其他语言版本中的功能。然而，如果游戏开始就是为一个特定的市场开发的，只不过后来才对其他语言市场有了浓厚兴趣的话，这种方法就是唯一的方法了。

1. 在成品上修改游戏

如果游戏已经发布了或者是已经在开发过程的延期了，此时你想为它加上多语言版本支持的话，其回旋余地就太小了。首先，要把整个游戏在项目的后期转化为 Unicode 相当昂贵，耗费时间，而且容易出错。因此通常情况下不要做这种事情。然而，可以为多语言提供一个中等层次上的支持，这样同时也不会增加太多的开发和测试的工作量。

Download at www.Pin5i.Com

首先，有可能在减弱多语言支持的同时仍能提供需要的功能。例如说，你可以选择在游戏的用户界面上只支持欧洲语言，但是在安装引擎和文件系统上提供更广泛的支持。另外一个方法是，可以在输入方面只支持英文，但是通过修改游戏引擎，在保存和显示方面可以使用更多的语言，这个方案用户也许是可以接受的。最后，你还可以修改游戏以支持 MBCS（例如 Shift-JIS），但是这有可能会大大增加开发工作量，很容易引进程序错误。除此之外，跨语言的功能有可能会被作一些限制，例如聊天。决定性的因素应该在其涉及到的代价/时间和期望向用户提供的游戏体验上取得一个平衡。

2. 从头做起

虽然上面的方法是可行的，而且已经被使用了很多次，但是成功的全球化应该是在一开始就设计好的。首先，与从一个成品开发相比，从头以正确的方法来开发会便宜得多。其次，如果游戏所有的版本都使用同一套代码的话，因为始终只有一个平台需要维护和支持，开发也就不会那么复杂，售后支持的负担也会比较小。

从长远的角度来看，如果能把全球化和本地化集成到核心的设计过程中间，项目的成本会更低，开发会更快。虽然这种集成在第一次可能会引起一些延期，但是它可以大大加快你游戏多语言版本的发行速度。此外，你还可以同时发布这些版本，而且，如果要把这些版本放到一个安装媒介上，这还可以降低制造成本和售后支持的成本。

1.12.7　测试

到此为止，本节的焦点主要集中在设计和实现上。然而，同样的国际化的标准不仅适用于开发，也适用于测试，这一点是很重要的。对于不同本地化版本的测试并非无事生非。如果你的游戏是面向世界市场的，而且使用的是同一套代码，那么一个完全的测试只需要对一个语言走一遍就够了。对于游戏其他语言版本的测试可以仅集中在这些版本的本地化和其特有的功能上面。下面是几个在处理国际化游戏中需要受到特殊关注的测试方面的内容。

1. 我看不懂游戏——该如何测试呢?

测试一个本地化游戏的最初障碍之一就是语言难关。如果你和大多数公司一样，挑选测试人员的准则是他们的技术才干，往往在多语言方面的能力就不会考虑那么多。然而，一个优秀的测试小组能够在不用学习另一种语言情况下也能测试出大多数功能性的错误来。

首先，你的测试人员必须对游戏有非常深刻的了解。如果已经对原语言版本进行了很长时间的测试，他们就应该能即使闭着眼睛也能把游戏走完了。只要每件事情都按计划工作正常，测试人员就应该对此感到十分熟悉。其中一个例外可能是警告和错误信息。如果它们在测试阶段就被本地化了，此时可能就需要给测试人员一张列表，上面记载了常见的错误和警告信息，还有短语（“没有找到”、“没有足够的空间”，等等）和它们相应的本地化的产物。

除此以外，如果在你成员中有一个讲这种语言的人而且能够随叫随到，在问题出现的时候，他就能帮忙快速定位问题所在了。只要有了一些经验，测试人员应该对本地化产品的常见消息和已知问题相当熟悉，因此只有新的或是比较罕见的问题才会引起注意。然而，游戏里面的验证性质和其他性质的测试可能会需要一些相当了解对应语言的人。很多技术招聘、

Download at www.Pin5i.Com

人员配备、合同签订方面的公司在寻找和提供具有相应母语的测试人员方面具有相当的经验。一旦本地化版本做好了以后，你可能会发现找一两个对于目标语言很流利的测试人员然后与其签合同会很划得来。此外，你还可以把本地化的测试工作外包给一些国际化公司来做。

2. 危险字符

一旦测试小组已经准备好要测试面向世界市场的产品时，你就需要找出最高风险所在的地方，然后把这个发现和测试过程结合起来。一个可以入手的地方是所谓的“危险字符”，即那些由于其用途或在编码页中的位置而导致其有可能在应用程序中造成大量问题的字符。特别是当程序原来是为 SBCS 准备的，而后来转化到了支持 MBCS/DBCS 的时候，危险字符的测试就非常重要了。当处理这种代码的时候，通常很容易发现搜索或者分析算法会只把字符串的范围限制在单字节字符串上。

多数操作系统会把一些字符作为保留字符，它们不能在其文件路径中使用，例如斜线符和反斜线符，还有管道符（即“|”）。如果文件名和路径分析算法是基于单字节算法的，从而就会产生一个常见的对某些完全合法的文件名误认为是错误的问题。例如，如果用户输入了一个中文文件名，其中一个双字节字符编码刚好在第二个字节上是 0x5C（ASCII 码中间的退格删除符），一个单字节的算法就可能会把这当成是一个反斜线符，得到一个以为文件路径不正常的错误分析结果。

另外，当处理 MBCS 编码页的时候经常会发现错一个字的错误。这种错误是在比较的时候漏掉了或者多算了字符产生的，例如把大于等于的运算当成了一个大于运算。举个例子，在 Shift-JIS 系统中间，0xDF 这个字符值表示的是假名里面最大的单字节编码，而 0xE0 表示的是 DBCS 系统中第二段头字节的开始。在这种情况下，可能需要对边界值（此处是 0xDF 和 0xE0）做一个测试，以保证你的代码里面没有这种错一个字的错误。

为了产生一个危险字符的列表，你应该评估对应的语言/编码页，然后标志出下面的字符：

- 任何一个文件名/路径的分析器中可能认为的非法字符，例如管道符、反斜线符、括号、文件删除标志，诸如此类的东西。
- 如果使用的是 MBCS 字符集，要检查每一个字符范围开始和结束的边界值。
- 最小的和最大可能使用到的字符值，还有在它们范围之外多一个值的字符（这些字符应该被标为非法字符）。

你应该首先测试这些值以保证合法的和非法的值都能被正确处理。此外，对于任何一个 SBCS 值，你应该定义一个 MBCS 字符，这些字符把对应的 SBCS 字节作为尾字节，这样你可以确认自己的文件/路径分析算法不会在这些值上面出错。为了达到最高的效率，你应该创建一个通用的包含每种非法字符串各一个的测试字符串，把这个字符串在每个可能的地方都输入进去。如果你看不懂目标语言，你也应该找一个对应的本地人以得知字符串是什么样的，应该怎么输入它。此外，如果你的游戏使用的是 Unicode 而且允许多语言输入的话，一个优秀的测试字符串还应该包含由多种语言组成的文本（最好有危险字符）。

3. 缓冲区分配

在开发面向世界市场游戏时的另一个常见问题是由缓冲区分配引起的。当没有搞清楚字符与字节之间关系的时候，这种错误就会经常发生。例如，如果一个字符串使用 16 个字节来

Download at www.Pin5i.Com

分配，如果字符是双字节的话，它就只能包含 8 个字符了。测试人员应该知道不同字符串的缓冲区的限制（这些应该被仔细地记录在游戏规范中，还要说明这些限制究竟是以字节作单位还是以字符作单位的），对它们进行边界测试，其中要包含单字节和双字节测试，如果可能，还有多字节测试。

4. 硬件配置

在测试本地化游戏中可能碰到的另一个问题是硬件配置与输入的问题。首先在国际范围来说，有很多不同的键盘在使用着，而且某些市场上（例如韩国和日本）可能在它们的键盘上有其自己特定语言的键。其次，在世界范围内可能还有其他的硬件上的差异，例如 PC 上面本地化了的驱动和游戏机上不同的缺省硬件配置/控制。当为这些市场进行测试的时候，重要的是要考虑到其主流配置（这可能影响到你支持的硬件配置）然后对这样的设备进行测试，以保证其和目标市场是兼容的。

5. 系统配置

面向世界市场的系统配置是自成体系的。对于测试来说，你需要关注的两件事是系统的不兼容性和用户的设置。由于系统不兼容问题的严重性，它们一般在开发早期就会被发现，相比之下，用户设置就往往被忽略了。这些包括但是不仅限于时间、日期和单位的格式。在这个方面，PC 机和某些游戏机可以做不同程度的定制，而你也应该进行测试以保证自己的游戏能尊重用户的个人偏好。

6. 输入测试

对于一个面向世界市场的产品来说，另外一个需要特别测试的方面是编辑和输入。特别是有两个功能（如果你的游戏里面有的话）需要被测试：一个是光标的移动；一个是文本编辑。光标移动测试一般包括文本选择和在文本里面光标的移动，而编辑测试则包含删除、退格删除、复制、剪切和粘贴这样的功能。为了要测出这些方面的毛病，你可以创建一个由不同双字节（和单字节，如果有的话）字符混合而成的字符串，以确保光标移动、选择和编辑都是基于字符的。不管在任何情况下都不能把光标移到字符中间去，或者把一个多字节字符的头字节或者尾字节给切断了。

7. 用户界面和本地化测试

当进行用户界面和本地化测试的时候，必须要由一个非母语人员来进行一次合理性测试，检查切断，大小变化和其他方面的视觉上的问题。然而，很多其他的问题需要一个母语人员来进行观察，它们包含语境、字符串顺序和翻译测试。语境和字符串顺序的测试是要验证用户界面的内容（多数时候是文本）并保证字符串出现在适当的位置上面，还要保证能充分理解所有的文本，而且对于当时的语境要翻译得当。当一个字符串由于一个变长的组件必须要分成两个或多个部分的时候，字符串顺序的问题就可能出现了。由于在不同的语言里面有不同的句法结构和顺序，在本地化和测试中都要小心谨慎，以保证所有的资源都已经适当地在语境中表现出来了。

翻译测试主要关注翻译本身的正确性，而且由于在不同的语言中表达同一个消息有其困

Download at www.Pin5i.Com

难之处，这往往会带来一些翻译上的模糊性。如果你的公司是在内部进行的本地化工作，那就最好在测试过程中把本地化小组加入进去，因为他们对内容和目标语言都很熟悉。此外，在本地化过程中进行适当的检查也能在翻译之前找出原始文本的一些错误，这样就能在问题到达测试小组之前就把它们改过来了。

1.12.8 结论

正如你看到的，要开发一个面向世界市场的游戏，我们需要考虑到好几个问题。在本节中，我们尝试着列出了与此过程相关的主要因素，还提出了与游戏开发相关的对这些问题的估测和解决方法。虽然此过程可能会很复杂，但是如果你能把全球化和本地化集成到自己的设计过程中的话，那就能在更大的市场上赢利，得到一个更好的设计，还有更深刻的游戏体验了。

1.12.9 参考文献

[Aliprand00] Aliprand, Joan, *The Unicode Standard*, Version 3.0, Addison Wesley Publishing Co., 2000.

[Dmoz02] Open Directory Project, "*Open Directory —— Help Central*," 在网址 http://www.dmoz.org/Computers/Software/Globalization/上有在线的资料。此 Open Directory Project 的软件全球化的页面上有几乎与所有相关网站的连接。 March 2002.

[DrIntl02] Dr. International, *Developing International Software*, Microsoft Press, 2002.

[Kano95] Kano, Nadine, *Developing International Software*, Microsoft Press, 1995.整本书在网址 http://www.microsoft.com/globaldev/dis_v1/disv1.asp 上都有。

[Lunde98] Lunde, Ken, and Gigi Estabrook, *CJKV Information Processing*, O'Reilly, 1998.

[MocrosoftGlobalDev02] Microsoft Corporation, "*Microsoft:Global Software Development!*," 在网址 http://www.microsoft.com/globaldev 上有在线资料。微软的全球软件开发页面上包含了一步接一步的全球化的规则、对常见问题的解决方法、还有指向其他网站的连接。March 2002.

[MicrosoftIME02] Microsoft Corporation, "*Microsoft Global Input Method Editors(IMEs) Further Enhance East Asian Text Input*," 在网址 http://www.microsoft.com/Windows/ie/downloads/recommended/ime/上面有在线资料，这是微软为 Windows 2000 之前的系统做的 IME 下载页面。March 2002.

[Shmitt00] Shmitt, David, *International Programming for Microsoft Windows*, Microsoft Press, 2000.

[Unicode02] Unicode Consortium. Unicode 的主页，在网址 http://www.unicode.org 上有在线的资料，包含有一般的信息、相关标准的文档、编码图、还有会议记录。 March 2002.

Download at www.Pin5i.Com

1.13　3D 游戏中的实时输入和用户界面

Greg Seegert
Stainless Steel Studios
gseegert@alum.wpi.edu

为游戏开发一个安全、快速，并且能及时响应的用户界面和输入系统是一个经常被人们所忽视的开发任务，但是在一个成功的游戏里面，这是一个极其重要的元素。在本节中，我们将探讨各种实现方面的细节、技巧和优化。它们能帮助你尽可能创建一个最好的用户界面和输入系统。我们还将讨论在减少网络延迟方面用户界面和输入系统能起什么作用。本节假设你对 DirectX 有一个基本程度的熟悉，而此中的示例代码和其他的代码都是使用 DirectX8 实现的，虽然其中的概念对于任何 3D API 都适用。

1.13.1　实现用户界面

当创建一个 3D 游戏的时候，最好能使用已有硬件的功能来实现对 2D 元素的渲染。如果想画出用户界面元素，我们只需在创建完世界、视图，以及投影矩阵以后把平面的多边形渲染出来就可以了。下面的示例代码说明了设置 Direct3D 来二维地渲染多边形的方法：

```
// an LPDIRECT3DDEVICE8 variable named "d3dDevice"
// has been previously instantiated.

D3DXMATRIX tempMatrix;

// make an identity matrix
D3DXMatrixIdentity(&tempMatrix);

// set dx8's world and view matrices
d3dDevice->SetTransform(D3DTS_WORLD, &tempMatrix);
d3dDevice->SetTransform(D3DTS_VIEW,  &tempMatrix);

// set the projection matrix using
// the width and height of the viewport
tempMatrix._11 =  2.0f / width;
tempMatrix._41 = -1.0f;
tempMatrix._22 = -2.0f / height;
tempMatrix._42 =  1.0f;
```

Download at www.Pin5i.Com

```
d3dDevice->SetTransform(D3DTS_PROJECTION,&tempMatrix);
```

当我们创建好了世界、视图，以及投影矩阵以后，必须在屏幕上渲染出两个三角形。这两个三角形将组成我们的多边形，我们可以把它加上颜色或者纹理以作为用户界面上的背景、按钮，或者任何其他的 2D 组件。首先，当相应的用户界面对象被实例化了以后，创建一个由指定的两个三角形的顶点组成的顶点缓冲区。将每个顶点的坐标设置在你想要的屏幕象素坐标系的位置上。接下来，对每一个帧调用 `DrawPrimitive()` 以把顶点缓冲区作为一个三角形列表给渲染出来[MSDN101]。使用屏幕坐标系的方法简化了对你想要开发的任何 2D 用户界面元素的封装的任务。同样需要指出的是 DirectX8 提供了一个方法，可以用来把一个小精灵在屏幕坐标系里渲染出来，这可能会有用处[MSDN201]。如果想要使用 OpenGL 达到此目的，则另有一篇绝妙的文章可以参看，即“Using 3D Hardware for 2D Sprite Effect”[McCuskey00]。

随着时间的流逝，用户界面也越来越复杂了。最终用户希望它有丰富的功能，而开发人员的任务就是要为用户提供这些特性。表 1.13.1 列出了在任何一个健壮的用户界面里应该实现的一般的控件。

表 1.13.1 建议实现的用户界面元素

• 背景	• 编辑框	• 滑块条
• 按钮	• 列表框	• 静态文本
• 复选框（Check Box）	• 单选框（Radio Button）	• 提示
• 下拉框	• 滚动条	• 页或是表单

1.13.2 指定用户界面元素

一旦代码编完了，而且各种用户界面元素也经过了测试，就该把它们放到游戏里去了。有很多方法可以做到这一点的。每个屏幕显示和元素都可以被硬编码进游戏里。然而，如果使用这种方法，就连最简单的用户界面的改变都需要开发人员来参与。理想情况下，我们希望能够把用户界面放到一个外部的文件里面去，以使得设计人员能够在根本不需要编程的情况下也能随心所欲地创建和修改界面。

XML：它再也不仅仅是微软的了

XML 是可扩展标志语言（eXtensible Markup Language）的缩写。只要你对 HTML 的语法有所了解，你就会很容易接受 XML 了。实际上，HTML 可以看成是 XML 的一个“老表兄”。XML 的真正力量就在于它的可扩展性——本质上来说，它允许你创建一个自己的标志语言。XML 的骨架是 DTD（Document Type Definition，文档类型定义）。DTD 指明了一个特定的 XML 实例中的结构和语法。DTD 可以被用来验证是否对应的 XML 文件符合正确的语法。XML 标准是相当完善的，而且现在也有很多资源可以帮你编写和分析一个 XML 文件[W3C102]。由于 XML 本质上是面向对象和数据驱动的，因此它不仅仅是一个理想的指定用户界面的选择，而且实际上对游戏引擎中所有的数据驱动的元素都是。

Download at www.Pin5i.Com

下面的代码片段是从一个虚拟 XML 文件中提取出来的用于指定一个用户界面的部分。正如你看到的那样，XML 文件逻辑清晰地指明了一个用户界面中的元素，其说明的方式不仅易于分析，而且也易于读取和编辑。

```
<!-- This will create a UI screen with a background,
     a title, list box, and an ok button. These are
     all user-defined tags -->
<UISCREEN NAME="screen0" BACKGROUND='screen0_bg.tga'>
   <UITEXT NAME="text0" LEFT="10%" RIGHT="90%"
         TOP='10%" BOTTOM="25%">
      <UITEXTITEM>This is the title.</UITEXTITEM>
   </UITEXT>
   <UILISTBOX NAME="list0" LEFT="30%" RIGHT="70%"
           TOP="40%" BOTTOM="70%">
      <UITEXTITEM>This is item 1.</UITEXTITEM>
      <UITEXTITEM>This is item 2.</UITEXTITEM>
      <UITEXTITEM>%1001%</UITEXTITEM>
   </UILISTBOX>
   <UIBUTTON NAME="button0" LEFT="40%" RIGHT="60%"
          TOP="80%" BOTTOM='90%">
      <UITEXTITEM>OK</UITEXTITEM>
      <UIEVENT ONCLICK="PushScreen"
             PARAMETER1="screen1"/>
   </UIBUTTON>
</UISCREEN>
```

以上代码片段的 DTD 声明了 XML 中允许出现的标志、它们的名字，以及它们下面可以内嵌的标志。一个 DTD 可以用来验证任意数量的 XML 文件。以上所示的例子和相应的 DTD 都可以在 CD-ROM 上找到。

在这个例子中，<UITEXTITEM>标志声明了一个文本，其中的%value%表示的是字符串表中的一项。每个用户界面实体的位置都用百分比来标明。使用百分比可以保证用户界面在各种分辨率下都能正常显示。而<UIEVENT>标志则声明了对于特定的用户界面实体来说什么事件是有效的，还有就是当这些事件发生的时候应该采取什么动作。举个例子来说，当单击按钮“button0”的时候，用户界面的代码就会把 screen1 作为参数来执行 PushScreen 这个动作。可以如下实现这些触发动作，那就是使用函数指针或者是使用一个 C++的类，而此类则是从一个需要一个字符串的抽象的基类派生下来的。然后此动作就可以被映射到一个相应的命名对象上去了，接下来就可以使用此特定的参数执行代码。

要想解析一个 XML 文件，你既可以自己写一个，也可以使用定制的解析函数。实际上现在在不同的平台上有很多可以免费得到的支持 XML 解析的 SDK[W3C202]。对于 Windows 开发人员来说，微软提供了 XML Core Services(MSXML)4.0 SDK，而它包含了可以由 C 以及 C++接口访问的全面的 XML 解析支持。

1.13.3　本地化问题

随着游戏逐渐在大小和领域上的扩大，其目标用户群也在扩大。为了得到最大的市场覆

Download at www.Pin5i.Com

盖率，我们就必须要设计一个能很容易本地化或是转化为其他语言的游戏。用户界面是此项工作最为关注的地方，因为翻译后的文本最终将在各个屏幕上显示出来。以下的准则在用户界面和游戏引擎本身的开发过程中都应该被常常提起。

- 绝对不要对任何以后可能要翻译的文本进行硬编码。取代方法是使用一个字符串表。
- 使用 MBCS 或是 Unicode 字符串。
- 在字符串连接的时候要格外小心。
- 对所有向用户显示的文本处理其 WM_CHAR 消息。
- 设计用户界面控件的时候要能适应以后可能出现的大字符串。
- 使用支持多字节语言的字体。

最重要的准则就是绝对不要对文本进行硬编码。取代的方法就是使用一个字符串表来放置所有的文本（当然，除了调试信息以外）。还有一点也很重要，那就是要在整个游戏中使用 MBCS（多字节字符串）或者是 Unicode（宽字符）。如果你不这么做的话，在某些语言中就会出现混乱的文字了。标准模板库（STL）提供了一个宽字符版本的 `string` 类，对应地其名字就是 `wstring` 了。在连接字符串的时候一定要小心。在其他的语言里面句子的组成是不一样的，因此同样重要的是要把句子的格式也放在字符串表里。另外，绝对不要使用 DirectInput 处理键盘输入，那样会直接给用户显示成文本。取代方法是要使用 `WM_CHAR` 消息。当处理 `WM_CHAR` 消息的时候，Windows 将基于本地语言做相应的转化工作。设计用户界面上的控件以使其适应大字符串或者将其进行扩展以适应显示字符串的大小也是很重要的任务。一个英语短语在翻译成另一种语言的时候很容易就变得有原来的两倍长。最后，使用的字体一定要精心选择，以使其能显示多字节语言。某些系统字体能支持多字节语言，它们就是很好的选择。然而，如果你坚持使用自定义的字体，一个巧妙的解决方案就是在字符串表里面指明其需要使用什么字体，然后，本地化的小组就能为翻译的语言选择相应的字体了，如果必要的话。

1.13.4 输入系统

用户对游戏的体验远不仅仅是取决于背景以及按钮上使用的纹理。你的主要目标应该包括保证所有的用户输入都能及时响应，而且要如用户所愿地进行响应。现在我们将探讨一下为使键盘、鼠标，还有操纵杆达到此目的的几个有用的技巧。

1. 键盘

DirectInput 可以使用两种方法从不同的输入系统得到数据：直接访问式与缓冲式。直接访问的数据将返回当前设备的一个快照（snapshot）；而缓冲式数据则由一系列事件组成，这些事件是在上次缓冲式数据调用以后发生的。对键盘而言，直接访问模式返回一个代表了当前键盘按键状态的 256 字节长的数组。如果一个特定字符的高位被置为了 1，则此键就处于按下的状态。由于其简单性和其与 Win32 和 `Getkeyboard State()` 函数的相似性，且由于开发人员希望在任意时刻都能准确得到当前键盘的状态，直接访问模式对很多 Windows 开发人员都很有吸引力。虽然这种方法可以用，但是好像只有在游戏处理很少的一些键盘命令的时候才有用，例如向左、向右、向上、向下，还有射击。对于飞行模拟游

Download at www.Pin5i.Com

戏、RTS 游戏，还有其他可能拥有上百个热键组合的游戏来说又怎么办呢？如果在各个热键中进行循环查看其是否被按下了，在每一次更新的时候检查 GetDeviceState()返回的数组，这就太低效了。处理一个按键释放的事件将需要同时保存当前键盘状态与上次更新时的键盘状态。此外，这种方法无法处理此种情况：用户在输入系统的两次更新之间按下和释放了一个或者多个键。

2. 如何保证不丢失输入

幸运的是，DirectInput 的缓冲数据模式提供了一个可行的取代方案。我们将不调用 GetDeviceState()函数，而是调用 GetDeviceData()函数。这个函数将返回从上次调用 GetDeviceData()以来发生的键盘事件的一个列表。每个键盘事件都记录了按键的按下或是释放的状态、究竟是哪一个键，还提供了一个高精度的时间以记录事件发生的时间。不管输入系统更新的频率是高还是低，我们都将精确地得知到底什么事件按什么顺序发生了。

下一步就是要把输入事件给封装起来，然后将其放置在一个输入队列里以待游戏引擎处理。我们将维护三个队列："刚按下"事件、"按下"事件，还有"刚释放"事件。把输入分别放在三个队列中就可以让我们根据按键的状态来触发相应的动作。例如，按住一个向前箭头的按键可以在一个赛车游戏中给车进行加速，而按"A"键则可以换档。其他考虑的要素还有那些"修饰型"（Modifier）的按键。这些按键，包括 CTRL、SHIFT，还有 ALT，对于 DirectInput 来说与其他的键没有什么不同的。这就是我们想要的效果，因为我们可能希望在这些特殊键被按下的时候触发相应的事件。然而，我们也可以将这些键看成是修饰键。例如，CTRL-A 可以看成一个单独的键而与 CTRL 键和 A 键分开。由于此原因，输入系统将维护一个当前修饰键的状态，它由一组位标志组成，当相应的修饰键被按下或是被释放的时候其标志位将被更新。因此，每个封装的输入事件都将由此事件的按键和当前的修饰键状态组成。将按键和当前修饰键的状态封装成一个 16 位的值可以使我们得到一个唯一的键组合，以便于排序和比较。

当每次输入系统更新的时候，我们就将给相应的队列添加输入事件了。当我们要处理这些事件的时候，游戏引擎将遍历一次这些队列。每个我们需要检查的热键都将保存成一个输入事件至热键的映射。这样一来定位相应热键指针（如果存在的话）的查找和对游戏引擎代码的执行就会很快了。

不管输入系统更新得有多快，我们现在已经能清楚地知道用户按了什么键。此外，这种处理输入的方法可以看成是一个重要的优化，因为我们现在只需要快速处理一个小得多的用户真正的输入列表，而不用对每一个可能的键组合进行检查了。请参看 CD-ROM 以得到更多的实现上的细节。

1.13.5　鼠标与操纵杆

鼠标和操纵杆可以和键盘完全一样地从缓冲式输入中获益。这次不是按键处理，我们将把鼠标和按钮点击的事件给封装起来进行处理。然而，对于鼠标和操纵杆的轴向移动来

Download at www.Pin5i.Com

说我们就不需要对之进行缓冲处理了。在游戏引擎执行的间歇中，当处理缓冲的轴向输入的时候，鼠标指针或是操纵杆的控制有可能会在屏幕上到处跳动。简单的解决方法就是同时使用缓冲式输入和直接访问输入。缓冲式输入用来处理按钮和点击，而直接访问数据则来处理轴。

一个平滑的、及时响应的指针

在游戏中一个鼠标的实现通常是将其放在它自己的线程里处理。这是一个好方法，因为这样鼠标就可以独立于其他的游戏引擎正在进行的处理而进行更新和渲染了。然而，鼠标线程可能会被暂停以挤出时间供物理、人工智能，还有图形使用更多的处理器周期。虽然这很合理，但是暂停鼠标线程的处理会在某些配置的机器上导致出人意料的行为。如果游戏是以60fps（帧/秒）运行的，一个每秒只更新 10 次的鼠标指针就会显得迟滞而且响应缓慢。在此种情况下，解决的方法是要把鼠标轴的更新与按钮的更新分离开。这样，鼠标的位置就可以更新了，而鼠标指针将在每帧里都渲染一次，可以得到尽可能平滑的效果。

1.13.6 在处理延迟方面用户界面的作用

随着网络游戏越来越受人欢迎，用户对在线游戏的速度和体验的要求也越来越高。有时，用户的期望值简直是不现实。一个使用 28.8kbit/s 调制解调器玩复杂的网络游戏的用户会希望自己游戏的平滑度和那些直接通过局域网连接的用户一样。对于网络代码来说只能对其进行有限的优化和调整——出于网络连接的不可预测性来说。如果用户传输数据的速度不够快的话，那就真的没有什么办法能达到这个目标了。

解决此问题的最好方法就是对用户把网络的延迟给隐藏起来。某些最成功的在线游戏，例如 *Quake* 和 *Half-Life*，都使用了一种叫做客户端预测的方法。本质上来说，在客户的机器上仿真是尽快地运行的，此时客户机会预测游戏中其他敌人的位置和动作。当从服务器收到了网络传来的更新以后，客户机就会进行更正，就好像其没有与服务器同步好一样。

对于用户界面和输入的所有方面都可以使用同样的方法。游戏中的每个反馈，包括用户界面按钮的更新或是对单元的选择，都应该在客户机上立刻执行。举个例子，如果用户按下了扳机的按钮，在网络消息送到服务器的同时枪就应该立刻开火。如果用户发送了一条聊天的消息，它就应该在屏幕上马上显示出来。如果用户点击了一个按钮创建另一个单元，屏幕上的（单元）计数器就应该马上增加。但是，如果命令不能执行，或是被服务器拒绝了，或是网络数据包丢失了，那该怎么办呢？

对于此问题有很多可用的解决方案。可以过一段时间就把用户界面和游戏世界的状态比较一下，当需要的时候就进行纠正。为了能及早防止这些错误产生，在客户机上可以进行服务器端的命令和动作的确认，尽管这会导致代码重复和增加额外的处理。最后，可以在游戏中实现一个可靠的消息传递。这可以使得客户端能更新用户界面，确保服务器最终会收到此命令。然而，此方法的缺陷也显而易见。在一个网络丢包率高的环境中，客户端就会被迫多次重发一个消息。再加上网络延迟，此方法将加重而不是减轻问题的严重性。

Download at www.Pin5i.Com

1.13.7　结论

我希望本节的这些想法和提示会对你有用。虽然我们只是匆匆地过了一遍，但是如果把这些要点记住了，它就可以帮助你在自己的游戏里创建一个快速、安全，而健壮的用户界面和输入系统。CD-ROM 上有一个完整的键盘输入系统，它可以很容易地加到任何游戏中。此输入系统实现了前面说到的快速的缓冲输入方法。此外，此输入系统易于本地化，还可以从一个外部文件里读入热键的赋值，这就可以使用完全定制的热键了。同样在CD-ROM 上还有一个简单的C++的 XML 解析器和此处用到的 XML 例子文件。

1.13.8　参考文献

[McCurkey00] McCurkey, Mason, "Using 3D Hardware for 2D Sprite Effects," Game Programming Gems, Charles River Media, Inc., 2000.

[MSDN101] msdn.microsoft.com,"IDirect3DDevice8::DrawPrimitive," 在网址 http://www.msdn.microsoft.com/library/en-us/dx8_c/directx_cpp/Graphics/Reference/CPP/D3D/Interfaces/IDirect3DDevice8/DrawPrimitive.asp 上有在线资料，2002.

[MSDN201] msdn.microsoft.com, "ID3DXSprite::Draw," 在网址 http://www.msdn.microsoft.com/library/en-us/dx8_c/directx_cpp/Graphics/Reference/CPP/D3D/Interfaces/ ID3DXSprite/Draw.asp 上有在线资料，2002.

[MSDN301] msdn.microsoft.com, "MSXML 4.0 RTM," 在网址 http://www.msdn.microsoft.com/downloads/sample.asp?url=/MSDN-FILES/027/001/766/msdncompositedoc.xml 上有在线资料，2002.

[W3C102] World Wide Web Consortium, "Extensible Markup Language," 在网址 http://ww.w3c.org/XML/上有在线资料，2002.

[W3C202] World Wide Web Consortium, "Extensible Markup Language(Software)," 在网址 http://ww.w3c.org/XML/上有在线资料，2002.

Download at www.Pin5i.Com

1.14 自然的选择：饼状菜单的演化

Don Hopkins
Don@DonHopkins.com

饼状菜单（pie menus）是一种自然有效的用户界面中的技巧——通过饼状划分的菜单项可进行有方向性的选择。鼠标初始化的时候会停在非选择性饼状菜单的中间地带，另外每个划分的选项都很大，而且互相接近，但是处在不同的方位上。饼状菜单对于新用户来说极其易于使用：你只要在弹出来的方向菜单里选一个就行了。对于有经验的玩家来说，它们则极为高效：一旦你对每个菜单项的方向都搞熟了以后，你就可以不用看菜单，快速而可靠地“鼠行直下”了。Fitts 定律[1][Fitts54]可以解释饼状菜单的优点——它们拥有快速选择的速度，而且选择错误的概率较低，这是因为它们的菜单项较大，且每个菜单项之间间隔较小。

界面设计的演化并不仅是由理论驱动的，实践也在其中发挥了作用。我们将对现实世界中的少数几个例子进行一番探讨，考察它们的成功之处和失败之处，这样不仅可以避免重复劳动，而且还将在未来给我们以灵感。此处提到的例子都是想要激起你在高一个层次上的思考，也是要使你能设计出好玩的、高效的，而且是可靠的用户界面。

1.14.1 Feng GUI 的饼状菜单

用户界面设计并不仅仅是艺术性的创造，也不仅是对界面设计准则和理论的生搬硬套，它是要去探索，去发现一种自然高效的方法来解决问题，同时还要在互相冲突的一组限制条件里面进行权衡。其结果也总是各不相同的，因为采取的权衡和限制条件都是不同的。然而其中的很多原理则都是通用的。

“FengGUI”寻求去理解人的因素和在实现上的动态流程。在整个界面中，它在总体的高度上控制了玩家的注意焦点和操作。Fitts 定律对于科学地分析性能和错误率都是很有用的，但是它却不能计算出人的因素的作用。FengGUI 力图避免发生不幸的偶发事件（如 2000 年佛罗里达总统选举中的“蝶形选票”[2]风波一样），而且力图把它们扼杀在摇篮里。

译者注[1] Fitts 定律如下：MT = a + b log2(2A/W)，其中 MT 为移动的时间，a 与 b 是系数，A 为起始点与目标的距离，W 为目标的尺寸。

译者注[2] 在 2000 年美国总统选举中，佛罗里达棕榈滩选票设计为蝶形选票（butterfly ballot），选票中间有一条孔印，候选人的名字交叉分列在其两侧。事后部分人认为这种容易出错的设计导致了很多戈尔的选票被误投给了布坎南。因此这个选票风波可以看成是由于用户界面设计不当引起的。

Download at www.Pin5i.Com

当设计一个饼状菜单的时候，先想想 Martha Stewart[1]是如何把一束鲜花化为美丽的花束的。你必须要在给定的条件下进行工作，同时力求表现出视觉效果、对称性和种种关系，最后以达到一个令人满意的效果，既美观，又容易记住。

如果想要创建令人印象深刻的饼状菜单的子菜单树结构，你应该模仿一下 Alexander Calder[2]的创建移动结构（mobile structure）的过程。此项任务不仅要求你对科学的工程原则有完全的了解，而且还需要你有美学判断力和空间平衡感。

Doug Engelbart，那个发明了鼠标，并在用户界面设计上具有先锋意识的人，深信人类工具的共同演化应该是基于在更广泛的现实里各种应用的使用而得出来的。因此不要再空谈饼状菜单了——使用它们，评估它们的性能，改善它们吧！

1.14.2　对饼状菜单的研究与评估

方向性菜单选择的主要思想已经有好长一段时间的历史了，它以不同的形式和名称存在着。其实现在已经有了很多方向性菜单的例子，而其演化的历史则可以参见[PieMenu02]。

人们已经在它们和其他用户界面方法的有效性问题上做了很多研究。Gordon Kurtenbach 和 Bill Buxon（多伦多大学）已经做了很多试验型的实验和控制性的实验，他们制作了菜单和各种不同的输入设备，并且从实验中得出了一些有趣的结果。在 AliaslWaveFront[3]，他们把它成功地运用到了 Maya 中，这是一个高端的 3D 动画的制作环境，这样用户就可以设计他们自己的菜单以定制其环境了。在其过程中，他们研究了从新手到老手的学习曲线。他们发现可以把学习曲线分为三个行为阶段：

（1）新手点击菜单，等待其显示出来，查找想要的标签，移动鼠标，然后单击以选择高亮的菜单项。

（2）中级用户能记住其方位，点击菜单，移向预想的方向，等待菜单弹出来高亮显示其菜单项，然后释放鼠标按键确认选择。

（3）老手仅仅是按下鼠标按键，移向预想的方向，然后不加犹豫地释放鼠标按键。

由于新手、中级用户和老手的物理移动速度是一样的，因此从上面可以看出饼状菜单可以在不知不觉中把你训练成一个老手。每次在进行一个选择的时候，你就温习了一次老手“鼠行直下”的动作。中间等级就如一个学习曲线中的自动电梯一般，它能帮助新手，训练他们的技巧，增强他们“鼠行直下”的信心，使他们成长为老手。到了最后，即使不用看屏幕，你也可以在无意识的情况下移动得很快。

对上述的情况，Jaron Lanier（VPL 研究院）有句话说得好：“意识或已将之遗忘，但身体却可牢牢记住。”即使你的意识已经忘记了相应菜单项的名字，饼状菜单利用了你身体的能力来记住肌肉的运动和方向。

使用的输入设备的特性对选择的速度和错误率都有极大的影响。比起轨迹球来说，鼠标的速度更快，错误率更低，但是输入笔却比鼠标还要快速和准确。

最大的可用宽度（菜单项的个数）和深度（子菜单的级数）是由应用程序可以容忍的最

译者注[1] 此人为同名公司的主席，是一个装饰专家。
译者注[2] 此人被誉为 20 世纪最富天才的雕塑家之一。
译者注[3] AliaslWaveFront 为出品 Maya 的公司。

Download at www.Pin5i.Com

大错误率决定的。核电站使用的界面就应该只有一级菜单，而且只能有 2 个或是 4 个菜单项。这种选择才是最可靠的。一个例如 *SimCity* 的游戏或是 Maya 的编辑器就可以不受这个限制，即可以使用更深的菜单和更多的菜单项，因为即使选错了想要恢复也很容易。

有经验的用户认为在一级菜单中只有 2 个、4 个或是 6 个菜单项的时候才是不容易出错的，而 8 个菜单项则将非常可靠。Kurtenbach 和 Buxton 对之进行了测量，他们发现在 4 级菜单 4 个菜单项的时候，以及 8 个菜单项 2 级菜单的时候错误率都低于 10%。

增加饼状菜单里菜单项的个数会对选择的速度和错误率造成明显的负面影响，但是此关系并不是简单的线性关系。偶数数目的菜单项易于选择，也容易记住，因为会有比较多的菜单项分布在轴向上或是对称分布。在轴向上的菜单项比不在其上的更容易选择，因此最好把常用的选项放在北边、南边、东边和西边上，把不很常用的选项放在对角线上。

这种奇数和偶数的对比在比较 7 个和 8 个菜单项，以及 11 个和 12 个菜单项的时候最为明显。8 个和 12 个菜单项特别易于使用，因为此时的方位分布在人的感觉上更熟悉，在物理上则有更多菜单项分布在轴向上。随着菜单项个数的增加，添加一个菜单项带来的负面影响也随之减小了。因此，对于 11 个和 7 个，甚至是 3 个菜单项的情况来说，最好再加上一个菜单项，这样可以使其数目更好，更加偶数化。

当设计嵌套饼状菜单的时候，在其深度和宽度之间的权衡似乎没有什么定论。因此，最好根据菜单项的意义来决定其位置。可以使用每层有多个菜单项的浅层菜单，也可以使用每层菜单项比较少，但是层次比较多的菜单。

值得注意的是，某些菜单当使用线性菜单的时候会有更好的效果。大多数线性菜单和其子菜单在设计的时候都没有利用饼状菜单的方位感这个优点，但太多菜单项的饼状菜单则太大，太笨拙了。为了解决这些问题，有人开发了可定制的饼状菜单让用户可以自行配置，并将菜单加上滚动和分页的功能，这样就能处理任意数目的菜单项了。

1.14.3 饼状菜单插件

有了组件技术，例如 ActiveX 和动态 HTML 行为[1]，我们就可以实现通用的、容易重用的插件式用户界面组件了。饼状菜单可以提供用于配置的语言、属性表，以及针对特定目的的编辑器，这样就能让设计人员和用户不经过编程就可创建和定制自己的菜单了。

ActiveX（也可称为 COM 和 OLE）是一个由微软开发的组件技术。我们开发了一个公开源码的 ActiveX 饼状菜单组件，能插入到任何 OLE 控件容器里面去，它们包括 Internet Explorer、Visual Basic、Visual C++、还有许多其他的工具和应用程序。此组件可以很容易地利用脚本语言，例如 Visual Basic 和 JavaScript 创建出来，而且它们还拥有属性表可以用来配置选项，编辑和预览饼状菜单。

ActiveX 的饼状菜单提供了很多属性和方法以用来控制其外观和行为。你可以通过编写脚本来控制其属性，调用其方法，还可以处理在鼠标移动中产生的事件回调，这样就可以对其进行定制了。然而，与动态 HTML 比较起来（参见图 1.14.2），它们图形方面的能力还是相当差的。

译者注[1] 动态 HTML 行为（Dynamic HTML Behavior）是微软提供的一种定制网页中组件的技术。详见 MSDN。

Download at www.Pin5i.Com

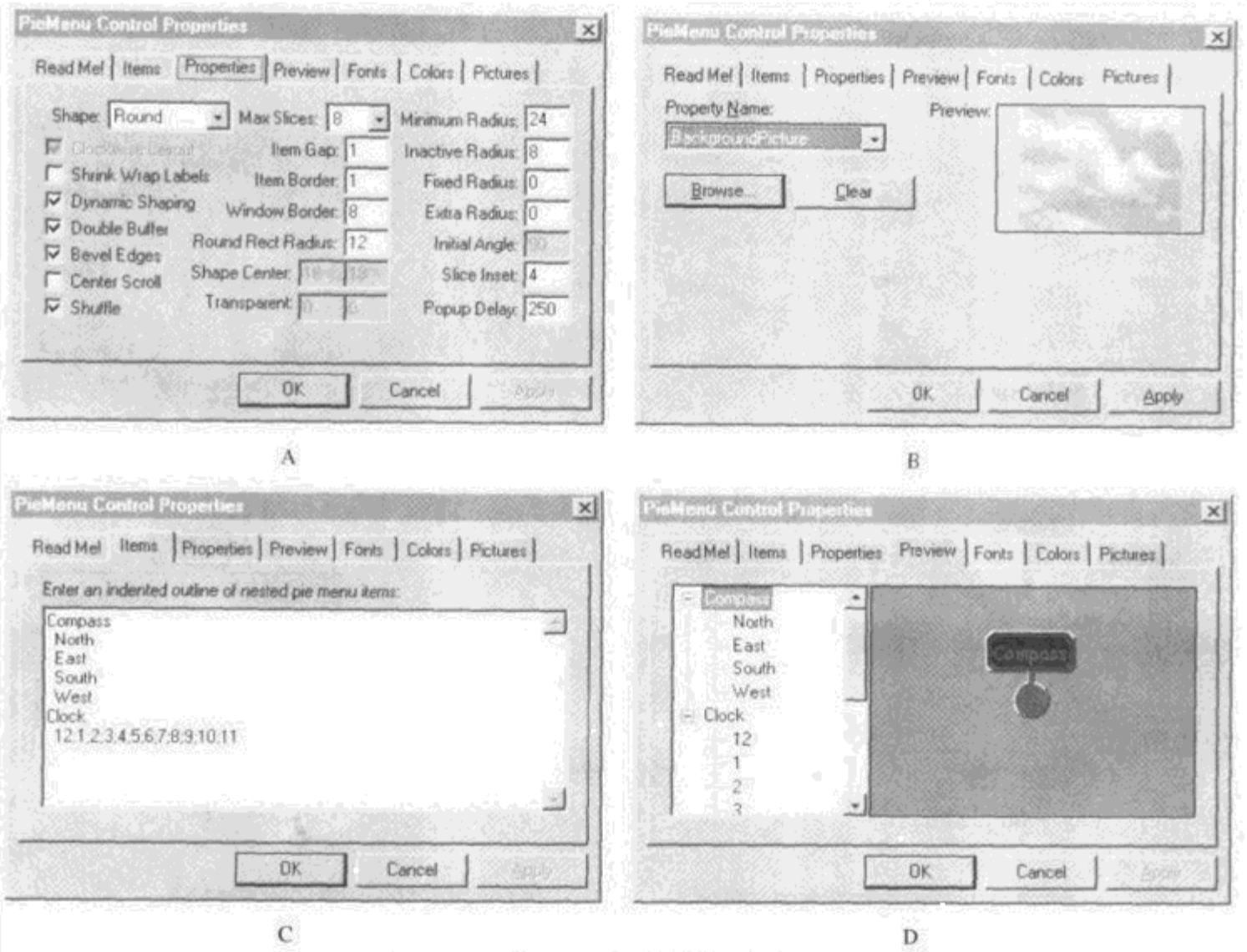


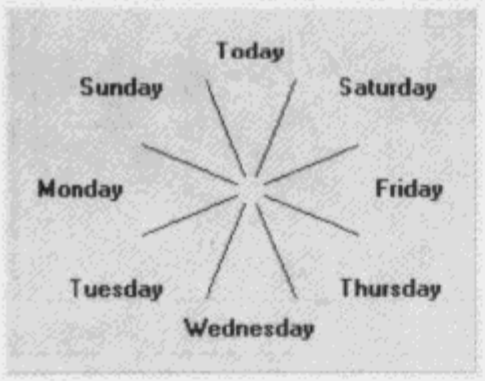
图 1.14.1　使用一个示例程序编辑控件的属性

为 Internet Explorer 编写的开源的 JavaScript 饼状菜单可以令人满意地解决这个问题[JavaScript02]。它们与网络浏览器结合得很紧密，而且它有个优势，那就是可以使用浏览器的所有特性。它们可以很容易地通过 XML 来进行完全的配置，而且也拥有很好的灵活性，因为你可使用动态 HTML 来设定它的外观。这些菜单使用方便，对于网页设计人员来说，他们可以将之加入静态页面中；对于 Web 服务器开发人员来说，他们可以将之应用到动态在线服务中，这是因为它们是使用模块化的“动态 HTML 行为组件”来实现的。

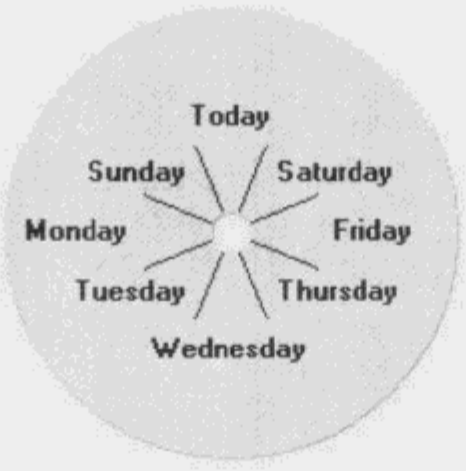
JavaScript 饼状菜单的规格是通过 XML 指定的，因此任何人都可以使用文本编辑器来编写规格。另外程序也可以动态地从数据库里生成它们。JavaScript 饼状菜单组件的代码与网页和 XML 的饼状菜单规格是不一样的。你可以在一个网页里使用 JavaScript、VBScript 或是其他的语言来编写事件处理函数以定制其行为。它们可以提供一个丰富的、动态的图形式反馈，这是因为脚本可以访问饼状菜单和网页，而且可以在运行时对动态 HTML 进行修改。

使用 XML 来指定饼状菜单的规格有很多优点。它的格式与实现是独立的，因此同样的饼状菜单可以在不同的平台上使用。通过使用标准的 XML 处理工具，例如 XSLT 和分布式的 XML 数据库，Web 服务器和浏览器可以将应用程序特定的 XML 格式自动转化为饼状菜单。例如，基于一个拥有“宠物小精灵”属性的 XML 数据库以及指向相应动画的连接，一个 XSLT 的样式表就可以动态地生成一个拥有“宠物小精灵”饼状菜单的网页了。

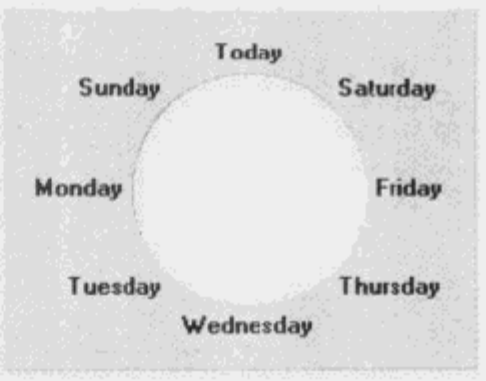
Download at www.Pin5i.Com



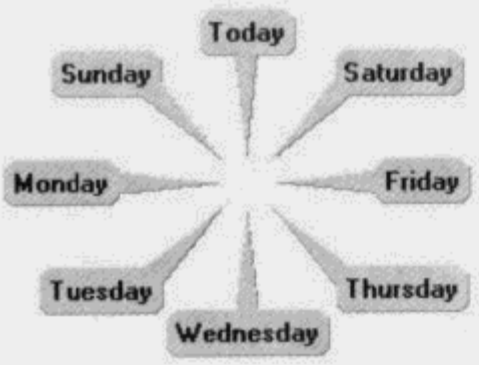
A



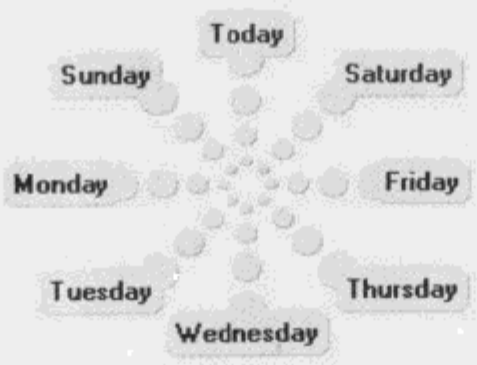
B



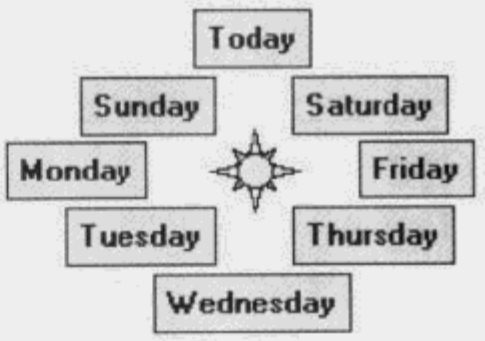
C



D



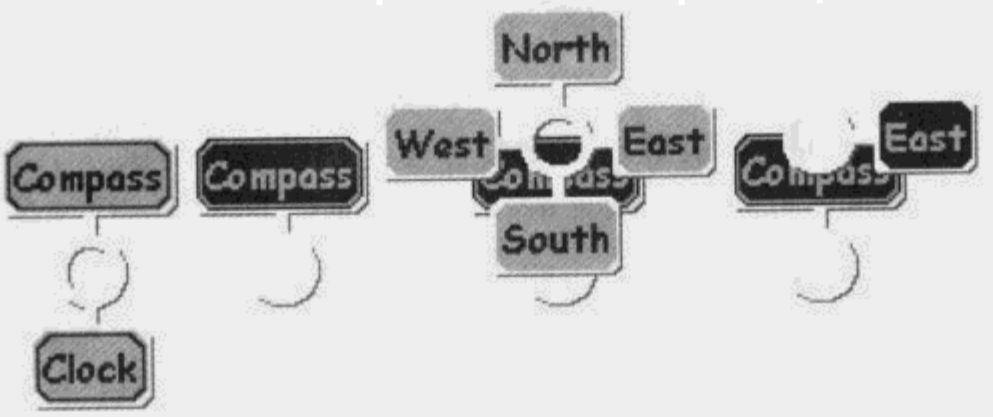
E



F

G

H



I

图 1.14.2 通过动态 HTML 实现的饼状菜单

Download at www.Pin5i.Com

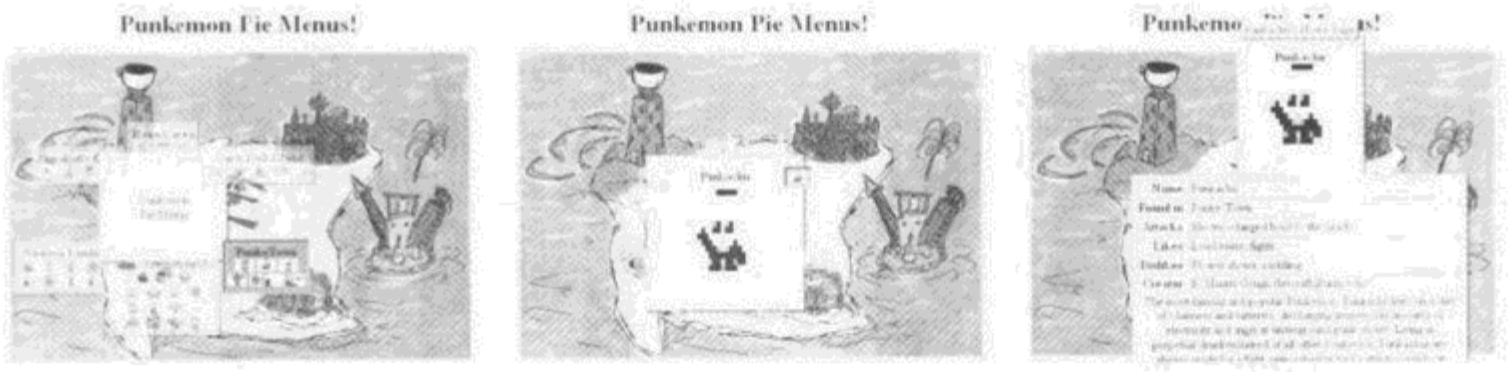


图 1.14.3　宠物小精灵的例子

ON THE CD

XML 饼状菜单的模板可以使得编辑器能自动地验证、创建和编辑饼状菜单。此饼状菜单的模板在 CD-ROM 和[PieSchema02]上都有。图 1.14.4 中显示了一个作为范例的编辑器，在网上可以通过[PieEditor02]得到它。

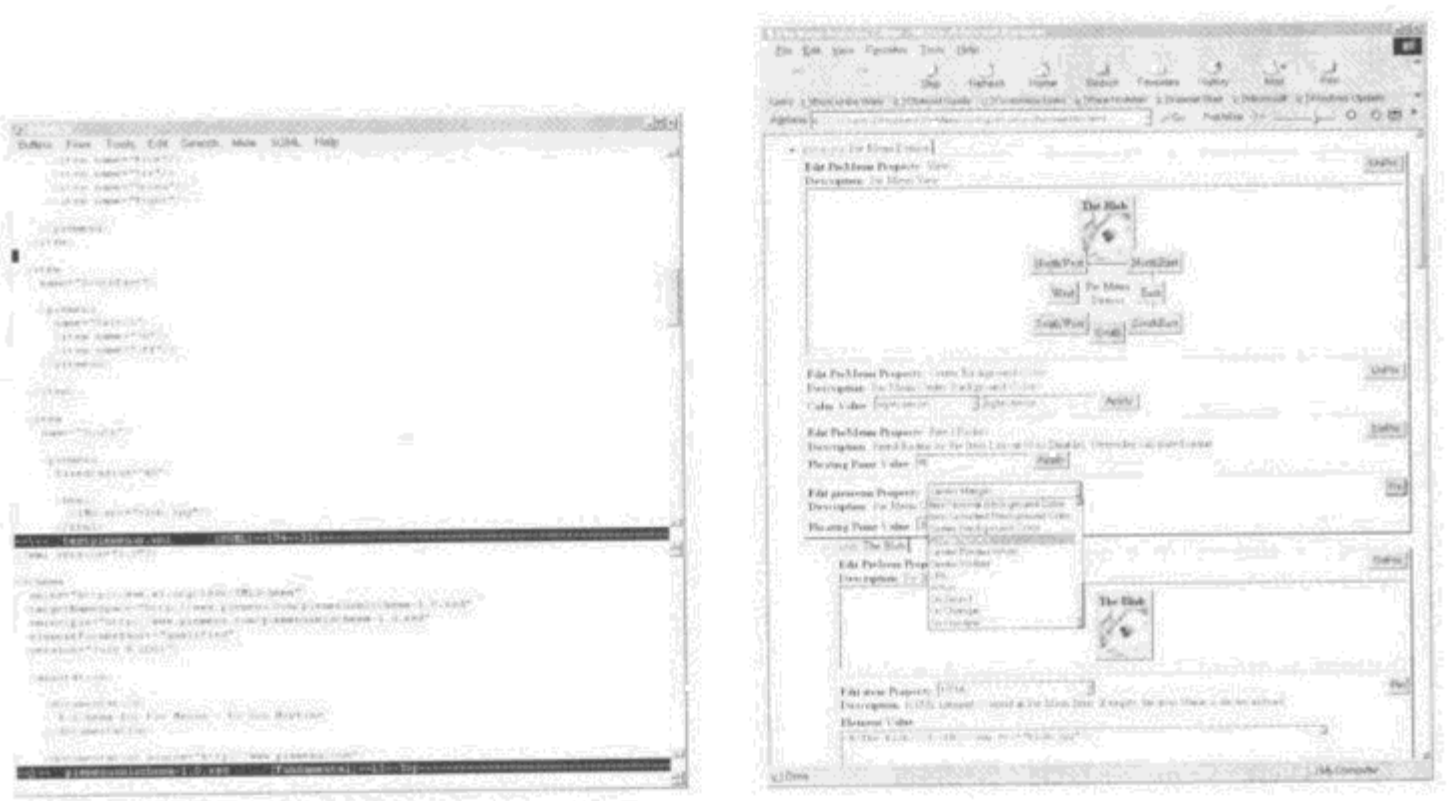


图 1.14.4　饼状菜单的模板与编辑器

Fasteroids[Fasteroids01]既是一个实时的视频游戏，也是一个试验型的用户界面实验。通过它，你可以比较一下饼状菜单和线性菜单。JavaScript 饼状菜单也支持老式的线性菜单风格，而且通过用户的指示，为了实验的目的，它们还可以记录下选择所花费的时间。*Fasteroids* 在饼状菜单和线性菜单中来回切换（参见图 1.14.5），然后提示你选择某个特定的菜单项以释放一颗星星。它将平均的选择时间和错误率记录下来并将之显示出来，这样你就可以自行对饼状菜单和线性菜单做比较了。

Download at www.Pin5i.Com

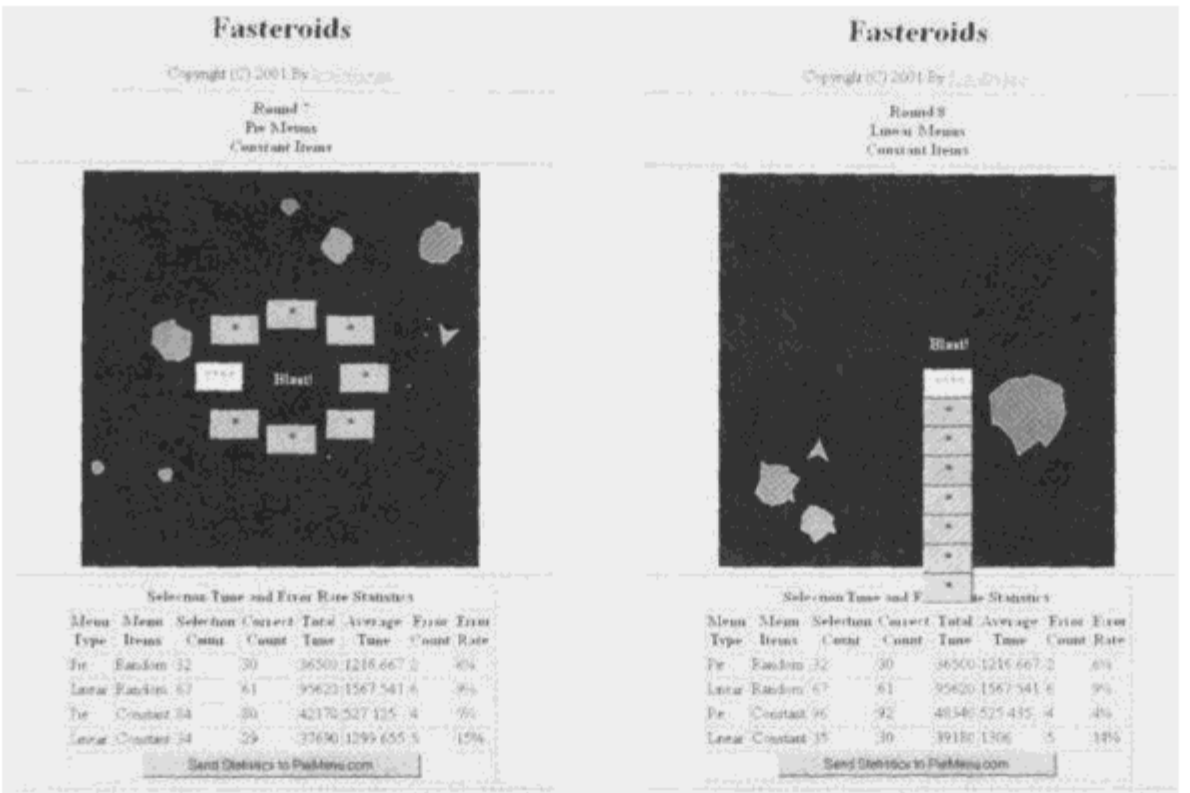


图 1.14.5 *Fasteroids* 游戏与实验

1.14.4 未来发展方向

饼状菜单对于掌上设备，例如 Palm Pilot 和 Pocket PC 的触摸屏有很好的效果。“指上菜单”可以很容易通过手指来使用，因此也不需要输入笔了。我们开发的一个产品 ConnectedTV 将把你的手持设备变为一个你可以定制的娱乐指南和遥控器，你只需要把它放在一只手里，通过手指就可以进行操作了。ConnectedTV 可以让你通过指上菜单设置自己个性化的电视节目单、过滤器，以及节目搜索导航；可以让你在节目描述信息和电影评论中来回切换，发出红外遥控指令切换电视的频道以及对其他的设备进行控制。

足够灵敏可以检测出重力方向的、快速而便宜的动作传感器马上就将被植入进消费型电子设备中，例如便携式电话、掌上电脑、遥控器，以及游戏。动作传感器将使以下技术的实现更为方便，包括单手滚屏、拨号、地图漫游、点击饼状菜单、连续手势识别，以及其他的令人眼花缭乱的交互技术。

1.14.5 走进 *SimCity* 中的城镇

1991 年，我们第一次将 *SimCity* 移植到了 Unix 下。它使用饼状菜单（参见图 1.14.6）来快速选择 *SimCity* 中的编辑工具，比起原来 *SimCity* 中的命令栏来说这个方法要快多了。

我们对菜单项保持不变的静态饼状菜单进行了仔细的设计，以便得到更好的易用性和更美观的界面。我们将 *SimCity* 的工具栏转化为了一组方便的饼状菜单，饼状菜单中的图标的摆放模式与原有的工具栏的模式保持一致，因此它们便于上手，也很容易使用。

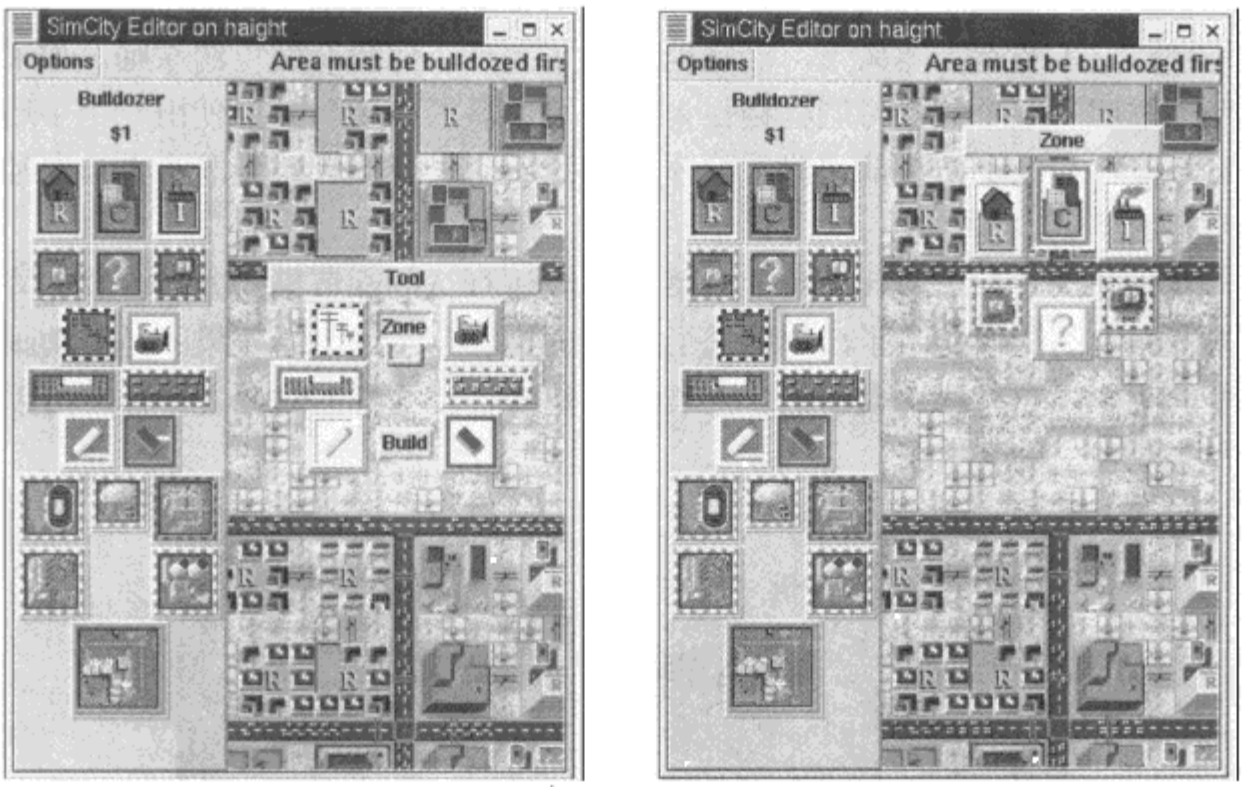
弹出式的饼状菜单可以让你快速地切换工具而不必在地图和工具栏中来回切换。通过快速点击饼状菜单和其子菜单上相应的方位，你很快就会学会鼠行直下的。托 Moore 定律[1]的福，

译者注[1] Moor's Law，此定律认为每一年半或是两年芯片的晶体管数量就会翻一番。

Download at www.Pin5i.com

你现在运行的 *SimCity* 要快得多了，它是一个策略型游戏，世间方一秒，游戏已十年。托 Fitts 定律的福，饼状菜单可以让你不用多看就鼠行直下，让你在 *SimCity* 中能赶上飞快流逝的时光，这样可以使你不用在线性菜单上浪费几个世纪的时间来作选择了。

与原来线性的 *SimCity* 工具图标比起来，*SimCity* 工具栏和饼状菜单的图标大小和形状都不一样。它们的大小和形状与相应工具的价钱和功能有关。小图标表示的是便宜的工具，例如停车场或是推土机；大图标表示的是昂贵的工具，例如电站或是机场；长图标表示的是线形的工具，例如公路或是铁路；方形的工具表示的是方形的建筑，例如居民区或是消防站。这种古怪图标设计的目的是为了让玩家能更容易地把它们记住和分辨出来（参见图 1.14.6）。



A　　B

C

图 1.14.6　*SimCity* 的饼状菜单

Download at www.Pin5i.Com

1.14.6 *The Sims* 中的起居室

The Sims 中的饼状菜单混合使用了降低饱和度、降低亮度，以及 Alpha 渲染技术，以增强菜单边缘的效果（参见图 1.14.7 以及彩图 1）。我们这样做的目的是为了让饼状菜单不要挡住了后面的事物。你可以透过饼状菜单看到其后的实时动画仍然在继续进行。当前选中人物的头部被置于饼状菜单的中央，而且随着鼠标方位的变化，其头部的方向也相应变化，始终对着当前选择的菜单项。

不过，我们需要把头部和其他的景物分离开，否则，那看上去就像一个巨大的脑袋正在房中飘浮一样，这不免有点煞风景，而且也违反了“最小化惊奇度”的原则。如果只是简单地画出一个不透明的菜单背景可能会挡住太多的菜单后的景物。如果使用一个半透明的菜单背景也仍然不能在视觉上把头部和背景给分开。这看上去会令人感到既难看又混乱，一点也没有干脆明亮的感觉。

因此，我们使用了一个取代简单地对菜单背景进行 Alpha 渲染的方法。我们降低了对比度，这使得背景上的图像变暗了，也降低了背景的饱和度。达到的效果就是给动画背景加上了一层灰度的阴影，它有一个柔和的、毛质感的边缘，在这样的对比之下你就可以很容易地看到头部和菜单项上的文字了。

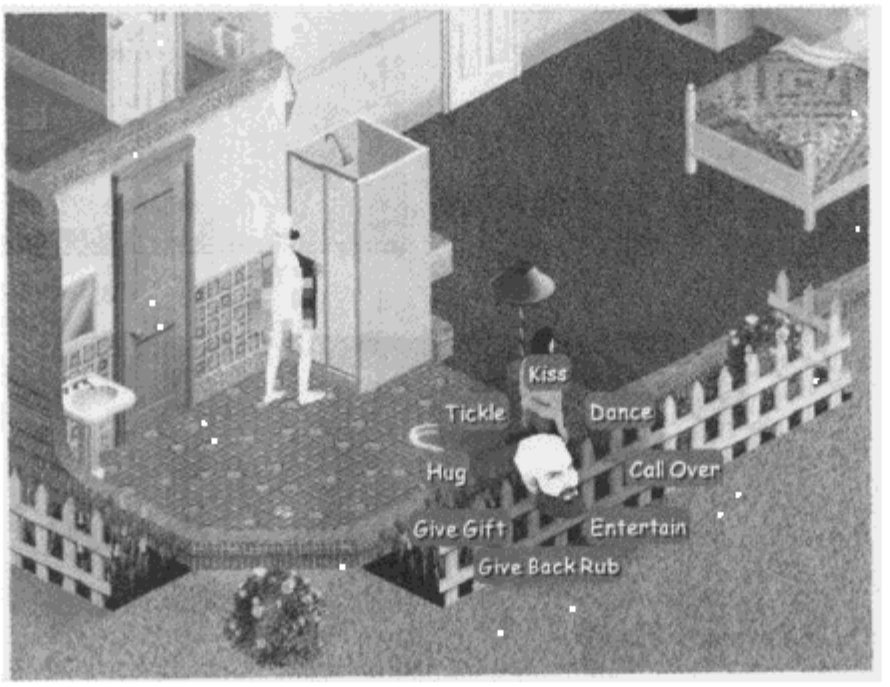


图 1.14.7 *The Sims* 中的饼状菜单

我们没有在饼状菜单上勾画圆角，取而代之的是让灰度阴影实现一个渐变，表示当前激活的饼状菜单的目标区域并没有局限在一个圆形区域中。其文字在饼状菜单的中心部分显示出来，其后还有一个高对比度的投射阴影，因此它们是很容易看到的。

在其中央的动画头部对比起饼状菜单的背景来说需要一种锐化的、明亮的观感。因此，此阴影效果将使用 Z 缓冲区对菜单中央的头部进行裁减，保持其强对比和明亮度。在视觉上，它看起来有种突出的感觉，这就可以把用户界面和游戏场景给分离开了，同时也不必画出分隔线或是造成不必要的视觉混乱。

Download at www.Pin5i.Com

1.14.7　结论

饼状菜单得益于 Fitts 定律自然的结果。它们并不是唯一的，也不是最好的使用此效果的用户界面的技术。然而，相对于当前流行的线性菜单来说，它们则是一个重要的改进，同时它们也奠定了未来开发更优秀用户界面的一个基础。

我们证实了饼状菜单相对于线性菜单的优越之处就在于其更高的速度和更低的错误率。同时它们也有潜力通过多种途径设计得更仔细且达到更便捷的自动化功能，这将增加它们对于许多应用程序的价值。

电脑游戏和手持消费型电子设备正在改变用户界面设计的方法，这是因为它们拥有全新的、非同寻常的需求。实时游戏则需要一个快速、反应灵敏、迷人的用户界面。手持电脑和电话在真实世界的环境中必须要能够满足大多数人的使用要求，因此它要求一个极可靠，易用性极高的用户界面。

设计一个优秀的用户界面需要在很多互相矛盾的要求和准则下进行权衡。最重要的是不要浪费用户的时间和注意力——你要把它们当成是自己最稀缺、最宝贵的资源。不要被那些五花八门的图标给迷惑了——如果失败了，那就退一步看看究竟是怎么一回事。一定要牢记用户的目的、其意识模型，以及其物理上的动作。

要做好放弃最初设计的心理准备。每天都要使用你自己的系统。依据实验测试和用户的反馈信息，以不断改进设计。每个应用程序和用户都有自己不同的需求，从而需要在不同的时候进行不同的取舍。

设计一个好的饼状菜单需要通过思考和努力，这就像写出一首好诗一样。对于每个菜单上的菜单项数目作一个限制，将之进行分组以得到易于记忆的、排列均衡的子菜单。将菜单项以自然的方式进行排列以表现其间功能上的联系和物理上的关联。当使用其他技术更为合适的时候，就不要固执于使用饼状菜单，比如需要使用滑块条、滚动列表、键盘，或是手写识别等的时候。如果要让系统更加易于使用的话，最好为完成同一个任务提供多种方法。

FengGUI 试图把真实生活中的经验、实验性研究的成果、理论上的原理结合起来，以期将之应用到设计一个高效可靠的用户界面任务上。“蝶形选票”事件就表明了一个设计上有缺陷的用户界面可以对现实世界造成多么大的影响。通过努力，通过好的 FengGUI 进行用户界面设计，你就可以改善人们的生活，对世界作出贡献了。

1.14.8　参考文献

[Fasteroids02] Hopkins, Don, *Fasteroids*, 在网址 http://www.PieMenu.com/fasteroids.html 处有在线资料，March 2002.

[Fineman01]Fineman, Howard, “Unsettled Scores,” *Newsweek*, September 17, 2001.

[Fitts54] Fitts, P.M., “The Information Capacity of the Human Motor System in Controlling the Amplitude of Movement,” *Journal of Experimenal Psychology*, 1954: Vol. 47, pp. 381～391.

[JavaScript02] Hopkins, Don, “Open Source JavaScript Pie Menus,” 在网址 http://www.PieMenus.com/JavaScriptPieMenus.html 处有在线资料， March 2002.

Download at www.Pin5i.Com

[PieEditor02] Hopkins，Don，“Pie Menu Schema Editor，”在网址 http://www.PieMenus.com/piemenuschemaeditor.html 处有在线资料，March 2002.

[PieMenu02] Hopkins, Don, “Pie Menu Central,” 在网址 http://www.piemenus.com/上有在线资料，March 2002.

[PieSchema02] Hopkins, Don, “Pie Menu XML Schema,” 在网址 http://www.PieMenus.com/piemenuxmlschema-1.0.xsd 处有在线资料， March 2002.

[Punkemon02] Hopkins, Don, “Punkemon Pie Menus,” 在网址 http://www.PieMenus.com/punkemon.xml 处有在线资料，March 2002.

Download at www.Pin5i.Com

1.15　轻量级的、基于规则的日志记录

Brian Hawkins
Seven Studios
winterdark@sprynet.com

现代的调试器都非常强大而实用，但是还有一些事情它们是做不到的。而这就是一个简单的，轻量级的日志系统可以发挥作用的地方。在无法使用调试器的时候我们可以使用日志记录，这对于游戏设计人员和测试人员来说这是一件常事。日志记录也能让你检查当前游戏的状态和类似的信息，与此同时游戏仍可以在正常的操作下运行。

为了达到这个目的，本节引入了一个高效的日志系统，它在编译期和运行时都是可配置的。整个系统将被放入一个库里面，这样就可以通过使用规则（policy）来保持编译期的灵活性了[Alexandrescu01]，而且也能在最后的游戏中将日志记录的调用给去掉。运行时的配置可以不经过重新编译就对日志数据进行修改。这可以使开发人员共享一份代码，但是可以只在日志中记录他们感兴趣的内容。这样一来，我们不需要去掉日志记录的调用就可以把代码加入源码控制系统了，因为这些调用以后可能还会有用。在本节中我们将主要关注性能因素。

1.15.1　规则

规则就是一个类的接口，它可以在编译期作为一个模板参数进行绑定。规则与特征（trait）类似，后者在标准模板库（STL）和策略（Strategy）设计模式[GoF95]中得到了使用。如此，使用基于规则类的最终用户就可以创建一个符合特定规则的类。这个类在不需要改变原始类代码的情况下可以被作为模板参数传递出去以对一个基于规则类的实例进行配置。这使得规则在创建一个健壮的库时就很有用了，此概念将会在整个日志系统的设计中得到使用。关于规则和规则设计你可以从 *Modern C++ Design*[Alexandrescu01]中得到更多的信息。

1.15.2　调试标志[1]

调试标志器是用来提供运行时的配置的。这些标志器可以被看成布尔

译者注[1] 对于本节，如果对 traits 没有什么了解读起来会比较吃力。读者可以到网上搜索一下 Alexandrescu 的文章和书籍，这样不仅对 traits 将有所了解，而且由于此文的规则（policy）概念也是他提出的，因此也将对本文的理解极有帮助。

Download at www.Pin5i.Com

类型的数据，用来判断是否要执行各种调试操作。当设计一个标志器的时候，我们需要做许多决策。这些决策可以分为可转化为规则的三大类选择——初始化、赋值和存储的类型，这就可以让使用标志器的用户来做最后的决定了。下面我们就将实现这些对日志记录有用的选择。

1. 初始化

最重要的决策就是如何进行初始化。初始化的规则接口如下：

```
class t_InitializationPolicy
{
    typedef /*type*/ t_Type;
    static bool m_Convert(t_Type i_value);
};
```

此接口的第一部分定义了用来初始化的参数类型。此类型将被用来传递到标志器的建构函数中去。由于它定义为 t_Type，因此调试标志器可以访问到此类型。接口的第二部分是一个函数，其唯一的参数是 t_Type 类型，函数将把它转化为一个布尔值。此函数必须被定义为静态函数，这样我们就不必为了访问它而先创建一个此类的实例了。有了这两个部分，调试标志器的建构函数实现时就可以得到一个参数，将之转化为一个布尔值，然后保存其布尔值。

2. 赋值

赋值规则的接口与初始化的接口是一样的。我们把这两个接口分开以便在初始化和赋值的时候使用不同的类型。这样一来，两者主要的区别就在于赋值规则需要实现=操作符，而初始化规则则用来实现建构函数。

3. 布尔存储类型

最后的决策是要决定布尔存储类型。此规则的主要用途是要使此标志器成为一个常量的或者可改的布尔标志。如果将之定义为常量类型，就不能对其赋值了；将其定义为一个正常的布尔值则可以对之赋值。此外，我们也可以提供对更复杂的类型的支持，只要它们能被转化为布尔类型，而且可以从布尔类型转化而来的话。此规则的接口如下所示：

```
class t_BooleanPolicy
{
    typedef /*type*/ t_Boolean;
};
```

1.15.3 配置文件

对调试标志器而言，如果能使用一个配置文件对其进行初始化的话，那就太好了，因为这样不需要重新编译就可以得到调试信息。出于这个目的，我们创建了一个单实例类[GoF95]，提供由单个配置文件进行的初始化。

Download at www.Pin5i.Com

1. 初始化

第一步就是要通过分析配置文件以初始化配置的单实例类。我们将基于配置文件的内容创建一个使用 `std::vector` 或是类似数组类的排序字符串数组。一旦含有可用字符串的数组进行了排序以后，我们就能使用二分法搜索，例如 `std::binary_search`[Meyers01]，来找出一个特定的字符串了。由于大多数调试标志器是在应用程序启动的时候被初始化的，因此它不会对性能造成什么影响。即便如此，二分法搜索也应该保证高效。

2. 使用 PIMPL

配置文件的公共接口很简单，也不应该改变，但是其私有实现则会有多种选择，而且可能会出于种种原因而改变。PIMPL（或称为私有实现）的设计模式[Sutter00]正好可以用来把公共接口和私有实现分开。公共类包含有一个指向私有实现类的指针，而且公共类将把所有的函数调用都转给私有实现处理。这样一来私有实现就可以放在一个单独的头文件或是源文件里面了。如果以后需要改变私有实现，对公共接口的用户也不会造成任何影响。这种模式还防止了公共接口的用户对私有实现细节的依赖。

1.15.4　可配置的标志值

调试标志器和配置类的功能可以合并到一起得到单独的配置标志器。要想做到这一点，最好的方法就是为调试配制器提供一个使用单实例配置类的初始化规则。上面 `t_Type` 的定义可以变成一个字符串，而转化函数可以调用配置的单实例类把此字符串转化为一个布尔值。

1.15.5　日志记录

日志类将使用几种规则把所有的组件合在一起以得到完整的日志系统。

1. 标志器规则

是否需要禁止一个日志的实例要视其标志器规则而定。标志器的规则如下所示：

```
class t_FlagPolicy
{
    typedef /*type*/ t_InitializationType;
    typedef /*type*/ t_AssignmentType;
    t_FlagPolicy(t_InitializationType i_value);
    t_FlagPolicy& operator=(t_AssignmentType i_value);
    operator bool() const;
};
```

此接口的开始两个部分是对建构函数与=操作符的类型定义。标志器规则必须支持使用初始化类型的建构函数和使用赋值类型的赋值操作符。值得注意的是此规则与前面谈及的规则有所不同，因为日志类内部将要创建一个此标志器的实例，因此它需要一个建构函数。最后，此标志器规则必须支持对布尔类型的转化。前面描述的调试标志器就满足此接口要求。

Download at www.Pin5i.Com

2. <<操作符

我们可以使用两种方法记录日志消息：变长参数列表或是<<操作符。虽然变长参数列表是传统的日志记录操作的方法，但是此方法将带来以下潜在的问题：

- 缺乏参数检查。
- 缺乏类型安全。
- 缺乏可扩展性。
- 不支持用户自定义类的类型。

即使在最乐观的情况下，这些问题也会导致日志系统中产生垃圾信息。在最坏的情况下，日志将破坏游戏甚至使之崩溃。例如，如果在输出格式中定义了一个字符串，然而在参数列表里面却没有它的话，程序就会访问到一个非法内存地址。在很多平台上，即使只是读取某些范围地址的数据也会抛出异常，因此如果此非法地址刚好落在了这样的地址范围中就将使游戏崩溃。

<<操作符可以解决以上所有列出的问题，不过它又会引进一个新问题来。变长参数列表的语法只需要使用一个函数调用就够了，但是<<操作符则需要进行不定量的函数调用。我们需要一个缓冲区来保存每个相继的调用结果直到整个日志收集全了以待处理为止。

两相权衡，<<操作符仍然是最佳的选择，因此我们将定义一个缓冲区规则来处理缓冲区的问题。

3. 定义一个缓冲区规则

一个用来决定日志字符串如何存储直到被处理的缓冲区实现，如下面的缓冲区规则所示：

```
class t_BufferPolicy
{
   typedef /*type*/ t_Type;
   static const string m_ToString(const t_Type &i_buffer);
   static void m_Clear(t_Type &o_buffer);
};
```

接口的第一部分是缓冲区类的类型，此实例将被每个日志类创建。需注意的是此缓冲区类型必须支持所有需要被日志记录的<<操作符的实例。这样，通过使用模板成员函数，日志类就可以将所有的<<操作符的调用转到缓冲区规则来处理了：

```
template <typename t_Type>
   t_LogImplementation& operator<<(t_Type i_value)
{ m_buffer << i_value; return(*this); }
```

虽然我们也可以实现一个新的类，但是在大多数情况下，使用 std::stringstream 类将是最好的选择。这个标准字符串流也支持其他两个缓冲区规则的函数。

m_ToString 函数将把缓冲区中的内容转化为一个标准的字符串。此标准字符串接下来将被传到处理规则（下面将要谈到的）。而 m_Clear 函数则用来在处理了缓冲区以后将之清空。

Download at www.Pin5i.Com

4. 性能

性能问题中最重要的是那些被禁止掉的日志实例。被使用的日志是不需要讨论的，因为它们已经有了 I/O 性能造成的瓶颈了。对于禁止的日志来说，最多只需要对其做一个布尔检查和一个分支处理就可以了。这就要求我们不能使用函数来记录日志，因为这将需要对函数的所有参数进行赋值。由于重载操作符实际上只不过是调用更加方便的函数而已，因此它们也不能在此处使用。

取而代之的是，我们要回过头来使用一个值得信赖的 C 中的范例——逻辑与运算&&。我们可以把布尔检查放在&&的前面，把日志函数放在后面，这样当布尔检查返回 false 的时候，整个日志调用就被短路[1]了。需要两步来达到此结果。首先，为了简单起见，定义一个宏：

```
#define LOG(type) (type) && (type)
```

请注意一定要使用宏来确保使用的是逻辑与的行为。如果使用模板函数，那它就将产生一个函数调用，从而改变了我们使用逻辑与的初衷了。其次，要为那些只有在内部标志值为 true 的时候才调用处理函数的日志类定义一个 bool 操作符的内联实现：

```
operator bool() { return(m_flag && m_Dispatch()); }
```

这样，对于被禁止的标志来说，扩展的结果就将只有一个测试和一个分支了。你有可能会对其中的处理调用感到疑惑，因为在 LOG 宏中有两个布尔类型的转化，因此看上去它好像被调用了两次。是的，它对每一条日志调用了两次，但是第一次调用不会进行任何处理，因为此时缓冲区是空的。

5. 处理规则

一旦缓冲区被填充了，而且调用了日志类的 bool 操作符，此字符串就被抽取出来了，然后缓冲区将被清空。字符串接着被传递到由处理规则定义的处理函数中：

```
class t_DispatchPolicy
{
   static void m_Dispatch(const string &i_string);
};
```

此函数可以用来做任何数量的事情，包括将信息输出到控制台，或者写入文件，或是将消息显示到屏幕上，或是所有上述的操作。

1.15.6　用法

使用此日志类的第一步就是创建一个此类的实例。由于使用了模板，如果能为各类模板参数定义一组方便实用的类型，那一般都会比较有帮助。例如：

```
typedef t_FlagImplementation<
```

译者注[1] 短路（short-circuited）是指在一个由多个条件决定的布尔值可以在开头几个条件处就被决定，这样就不需要处理后面的条件了。例如 if(p && p->FunctionA())中如果 p 为空则程序不需要往下走就可以跳过这个 if 判断了，这就是一个短路。

Download at www.Pin5i.Com

```
    t_FlagConfigurationPolicy,
    t_FlagBooleanMutatorPolicy,
    t_FlagMutablePolicy> t_ConfigurationFlag;
typedef t_LogImplementation<
    t_ConfigurationFlag,
    t_StandardOutDispatchPolicy,
    t_StringStreamBufferPolicy> t_Log;
```

如此，从配置文件初始化的实例就可以被创建了：

```
t_Log LOG_MEMORY("MEMORY");
t_Log LOG_SCRIPT("SCRIPT");
```

日志信息的记录与使用标准 I/O 流库极其相似。下面就是一个日志记录的例子：

```
LOG(LOG_MEMORY) << "Memory used is " << l_memoryUsage
    << "\n";
```

我们也可以在以后再改变日志实例的状态，例如：

```
LOG_SCRIPT = false;
```

1.15.7 结论

通过本节中提及的信息，你可以创建一个灵活而高效的日志记录库。通过使用规则，你可以在不改变库代码的情况下对其进行扩展。此处和 CD-ROM 上都列出了几个基本的规则，但是针对特定的实现仍有很多新的规则可以写。我们鼓励你去把它们发掘出来。

1.15.8 参考文献

[Alexandrescu01] Alexandrescu, Andrei, *Modern C++ Design*, Addison-Wesley, 2001.

[GoF95] Gamma, Erich, et al. Richard Helm, Ralph Richard, Johnson, Ralph, and John Vlissides, John, *Design Patterns*, Addison-Wesley, 1995.

[Meyers01] Meyers, Scott, *Effective STL*, Addison-Wesley, 2001.

[Sutter00] Sutter, Herb, *Exceptional C++*, Addison-Wesley, 2000.

Download at www.Pin5i.Com

1.16　日志服务

Eric Robert
Ubi Soft Entertainment, Inc.,
eric.robert@videotron.ca

调试交互式的程序是一件很麻烦的事情。很多事情都有可能出现错误但却不会导致系统的崩溃，或者不会被代码审核给找出来。如果再加上经常会涉及到的时间因素，事情就更糟了。

一些状态机在某些环境下时不时地就会出一些出人意料的错误。而导致出错的那部分逻辑有可能在时间和代码位置上都与发现问题的地方相差甚远。在最终发现错误之前有可能会经过许多的中间步骤。这就会使得开发人员很难发现错误，将会大量增加调试的时间。如果没有任何执行历史的记录，问题就必须被不断地重现直到调试的迭代过程缩小到一个很窄的范围，能对其真实的情况有一个彻底清楚的了解为止。

开发人员一般倾向于使用自己的调试器对代码进行跟踪。如果这种做法不可行的话，他们将会在可能需要的地方加上一些调试代码以生成原始的跟踪信息。这是一个既耗时又繁重的工作。从另一个角度来说，在源代码中加的调试信息只会反映程序的当前状态。出错的地方有可能离调试的地方有十万八千里远，而且也很难再跟踪到。由于调试器一般只会提供很多停止运行以监控内存的有效手段，因此它们对于交互式应用程序的调试来说也没什么作用。我们需要的是在不打断程序的情况下跟踪程序的流程。

因此，我们需要的解决方案要以一种灵活而高效的方法来提供隐藏在当前程序中的实时信息。这就是本节的焦点所在。

1.16.1　管理信息

关于实时调试的一个重要方面就是决定需要向用户提供多少信息。在此问题上必须澄清的一点就是过犹不及。过多的信息就会把系统拖慢，而且使得用户根本没法查看。显然，信息太少的话也一样是没什么用的。

要想达到一个令人满意的平衡点，那就需要在正确的时间提供正确的调试信息。因此，用户也需要积极参与进选择信息源的活动中来。如果能控制此系统，让它只汇报某个特定系统或子系统的信息，那就将使其成为一个有用的工具。这样一来，日志服务就可以对需要进行调试的过程进行模拟，使用一种分而治之的方法来帮助发现和修正错误了。

因此，此系统应该由一系列的逻辑状态（on/off），以及与之关联的每

Download at www.Pin5i.Com

个系统和子系统组成，这些系统的记录功能可以被激活或是被禁止。在创建的时候要使得它能很方便地对成组的状态进行控制。例如，如果需要禁止整个系统的日志功能，我们就不需要对每个子系统来分别进行处理：

```
Journaling::setDisable("/Core/Loading/...");
```

另一个需要考虑的方面是应该把这些信息汇报输出到什么地方去。用户可能希望把数据直接显示在屏幕上，直接写入一个文件，或是如果监控机在远程的话，甚至写到网络上去。此种灵活性在控制台程序里面是最受欢迎的，因为对其而言调试信息一般不太多。

如果能提供一个可扩展的架构，能在其上开发和集成不同的输出位置，系统的可用度就会大大提高了。

1.16.2 系统层次

我们的日志服务是基于三个类来建构的：Switch 类、SwitchBox 类和 Journal 类。我们可以使用这些基本组件来创建出很多具体的服务。

Switch 类是用户主要使用的接口。它包含有一个逻辑状态，还含有一个与之相关联的被 Journal 类封装的输出位置。每个 Switch 都被一个 SwitchBox 所有，而后者是 SwitchBox 树结构的一部分。用户给每个元素赋予一个字符串，这样以便于使用路径命名的方式来访问它们。

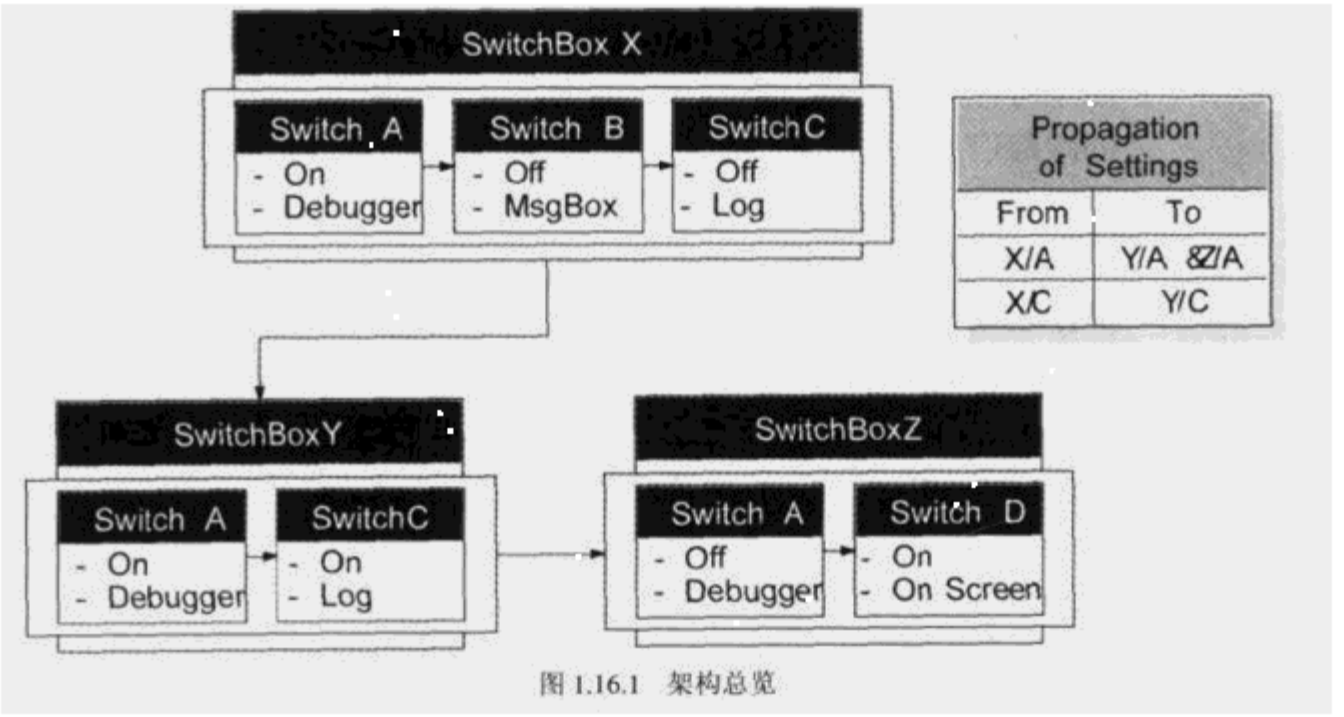


图 1.16.1 架构总览

```
mySwitch = Journaling::getSwitch("/Core/Loading/Trace");
```

在此层次结构中，每个 SwitchBox 都是一个根结点，而每个 Switch 则是一个叶节点。因此在上面的例子当中，“Core”和“Loading”都是 SwitchBox，而“Trace”则是一个 Switch。

我们这么做的目的是为了将 Switch 和特定系统或是子系统的特定日志汇报功能联系起来。这样，我们只需要对相应 Switch 的当前状态作一个简单的查询就能控制程序中一整块区域的日志汇报了。

```
if(mySwitch->getState()) {
```

Download at www.Pin5i.Com

```
    mySwitch->getJournal() << "Some Trace";
}
```

在展示代码之前必须明确的是，在设计此系统的时候，我们认为日志服务是一个基础服务。因此，我们没有使用任何的动态内存分配（由于内存管理器有可能最终会使用这些服务，这就会导致一个依赖性上的循环）。另外，我们认为整个日志系统应该在程序的入口之前（main）就准备好了，这是因为此服务可能会被用来跟踪全局对象的创建过程。由于全局对象的创建顺序是不可移植的行为，因此我们需要的是可以按需进行初始化的对象。

首先，用户将声明其自己的 SwitchBox 和 Switch 的单实例派生类对象，以便在服务中使用。两个类的初始化过程都使用的是一样的方法。在第一次对此单实例对象进行访问的时候，此对象将使用一个静态的工厂方法（factory method）创建自己，然后会将自身插入到其所有者的单向链表里去。请注意对于这些任务的处理是有相应的宏的，不过下面展示的是其生成的展开后的代码。

```
// Header file
class SwitchSample : public Switch
{
    typedef Root OwnerType;

    static Switch * ourInstance;
    static Switch * staticFactory();

public:

    SwitchSample();
    static Switch * getInstance() {
        if(ourInstance) {
            return ourInstance;
        }

        return staticFactory();
    }
};


// Source file
SwitchSample::SwitchSample() :
    Switch(OwnerType::getInstance(),
           "My name",
        "My description") {}

Switch * SwitchSample::staticFactory() {
    static SwitchSample instance;
    ourInstance = &instance;
    return ourInstance;
}
```

Download at www.Pin5i.Com

```
Switch * SwitchSample::ourInstance =
   SwitchSample::getInstance();
```

在此处使用一个辅助性的宏会更方便一点。

```
DECLARE_SWITCH(SwitchSample, Root);
IMPLEMENT_SWITCH(SwitchSample,
       "My name",
       "My description");
```

此单向列表实现的时候没有使用任何动态内存分配，它是将每个插入的元素直接作为表的结点来处理的。因此，两个类[1]都提供了一个指向下一个列表元素的连接（称为 brothe 这样就可以实现列表的插入了。

```
void SwitchBox::addSwitch(Switch * item) {
   if(mySwitches) {
      item->myBrothers = mySwitches;
   }

   mySwitches = item;
}
```

通过使用此类结构，我们可以进行层次化的属性设置。这样一来，我们只要设置了一系统的 Switch，那么此系统的所有子系统属性就都被设置了。由于每个实体都是和根结联系在一起的，这样也就提供了一种方法可以用来列出和管理系统里面的所有实体。

如此，我们也就提供了一个方法能同时对整个 SwitchBox 树进行很多的操作。一个称 Journaling 的类实现了其中的许多操作。

```
// Disable everything at once.
Journaling::disableAll();

// Print the whole tree.
std::cout << Journaling::getRoot();
```

从实例定义的方法中可以看出，如果 SwitchBox 树结构中某些分支要在 main 之前创的话，那么其初始化工作将被强制进行。只有在进行了此过程以后整个树才能使用。因此即使此系统可以用在全局变量上，这个使用范围也是有限的。

每个 SwitchBox 使用了两个使用前向迭代器的单向链表。一个存储是 Switch 用的，一个是存储其 SwitchBox 子节点用的。它们都支持增量操作符和*操作符以用来返回相应列表元素的引用。因此，在上面的例子中，如果一个叫“Trace”的 Switch 加入到了现在叫“Core”的 SwitchBox 中间，则改变“Core/Trace”的状态就会影响到更下层，从而会相地更新“Core/Loading/Trace”。

```
void Switch::setState(bool state, bool recursive) {
   myState = state;
```

译者注[1] 此处的两个类是下面代码中的 mySwitches 和 item。

Download at www.Pin5i.com

```
    if(recursive) {
        myOwner->propagateState(this);
    }
}

void SwitchBox::propagateState(Switch * source) {
    SwitchBox::Iterator i = getChildrenBegin();

    while(i != getChildrenEnd()) {
        SwitchBox * node = &(*i++);
        node->setState(source);
    }
}

void SwitchBox::setState(Switch * source) {
    Switch::Iterator i = getSwitchesBegin();

    while(i != getSwitchesEnd()) {
        Switch * leaf = &(*i++);

        if(!strcmp(leaf->myName, source->myName)) {
            leaf->myState = source->myState;
        }
    }

    propagateState(source);
}
```

在这个现成的层次结构的基础上，我们还可以加上其他的服务。这些服务可以包括某种广义的使用通配符或者是正则表达式进行的名字匹配。不过，仅凭此处提出的基本功我们就应该可以有效地利用此系统并通过它来控制信息流了。

提供的代码中也同样实现了 Journal 的层次化控制，以及利用 SwitchBox 树中的目录名搜索的功能。如果要得到一个更强大的实现，那么至少还需要提供一个控制台命令以便能实时地修改系统的 Switch 状态。或许还需要一个单独的用户界面以便在不支持键盘程调试系统上使用。

3 Journal接口

witch 实例的输出位置是利用 Journal 接口与之关联上的。简单说来，它提供了一种对设备进行格式化输出的方法。由于 Journal 的实例将被共享使用，因此我们对此资源进用计数的管理。

实现此功能的方法之一是利用标准库中的 std::iostream 来进行。由于它的设计是可扩因此我们可以很容易地通过它来建构我们自己的流。作为一个范例，下面的代码就给一个（特定于 Win32 平台的）调试流。

对不熟悉 std::iostream 设计的人来说，其设计可以分为两层。第一层实现了所有利用和操作符进行格式化的功能。另一层则提供了底层的缓冲系统。这种结构上的分离给

Download at www.Pin5i.Com

予了我们一些方便，可以让我们针对不同的输出位置创建自己的缓冲系统，同时还能使用标准的格式化操作。我们的调试流是使用一种略异于 std::streambuf 的方式来建构的。由于我们只需要一个输出流，因此（在缺省情况下）缓冲的输入部分是被禁止掉的。

第一步，我们需要创建一个 std::streambuf。我们使用了一个大小固定的缓冲区以进行缓冲，另外就只需要实现两个虚函数了。第一个用来将缓冲区中的数据利用 Win32 调用将其输出到调试器中。需要注意的是我们需要保留一个字符用来放置作为结束符的空字符。第二个函数用来进行溢出的处理。

```
class DebuggerStreamBuf : public std::streambuf
{
   char myBuf[512];

public:

   DebuggerStreamBuf() {
      setp(myBuf, myBuf + sizeof(myBuf) - 1);
   }

protected:

   virtual int sync() {
      size_t len = pptr() - pbase();

      if(len > 0) {
         myBuf[len] = 0;
         OutputDebugString(myBuf);
         setp(myBuf, myBuf + sizeof(myBuf) - 1);
      }

      return 0;
   }

   virtual int_type overflow(int_type c) {
      sync();

      if(c != traits_type::eof()) {
         myBuf[0] = c;
         pbump(1);
         return c;
      }

      return traits_type::not_eof(c);
   }
};
```

另外，Journal 还提供了两个额外的虚函数以用来标志一个汇报事件的开始与结束。对于调试器的日志来说，我们只使用了结束标志，它提供了与调试器的同步功能。因此，在实现此日志的时候，相应的 std::ostream 子系统需要使用上面的 std::streambuf 来初始化。

Download at www.Pin5i.Com

```
class DebuggerJournal : public Journal
{
    DebuggerStreamBuf myStreamBuf;

public:

    DebuggerJournal() {
        init(&myStreamBuf);
    }

    virtual char endReport() {
        *this << std::endl << std::flush;
        return 0;
    }
};
```

我们可以使用此方法来实现更强大的日志。在提供的代码示例中，我们添加了一个使用 Win32 消息对话框的日志。这样，我们就可以得到用户提供的反馈，然后再作相应的处理了。

其他的实现可以用来对系统进行扩展，还有可以用来添加更多类型的日志。例如，可能一个汇报事件会需要使用多个输出位置。然而，由于 Switch 类只包含有一个 Journal 的实例，因此我们就需要设计一个新的能处理此种情况的 Journal 了。解决此问题最快的方法就是把格式化后的字符串直接转给放在一个 std::vector 中的所有其他的 Journal 实例去处理。

```
virtual int sync() {
    size_t len = pptr() - pbase();
    if(len > 0) {
        myBuf[len] = 0;
        for(int i = 0; i < myJournals.size(); ++i) {
            *(myJournals[i]) << myBuf;
        }

        setp(myBuf, myBuf + sizeof(myBuf) - 1);
    }

    return 0;
}
```

1.16.4　创建日志服务

通过使用上一小节中讲过的用来控制信息流的系统，我们在下面给出了一些典型的日志服务的例子。

1. 信息汇报

第一个服务只是把数据输出到相应的日志中去。它展示了使用此系统所需要做的一些极其基本的工作。

Download at www.Pin5i.Com

```
#define REPORT(Leaf, Expression)                          \
   do {                                                   \
      Switch * leaf = Leaf::getInstance();                \
                                                          \
      if(leaf->getState()) {                              \
         Journal& journal = *leaf->getJournal();          \
                                                          \
         journal.beginReport();                           \
         journal << Expression;                           \
         journal.endReport();                             \
      }                                                   \
   } while(0)
```

请注意，所有的服务都是被设计成宏来使用的。这样就可以进行条件编译，可以在特定的版本中去掉部分或是所有的信息汇报功能了。

2. 跟踪信息

日志的另外一个用武之地就是进行堆栈跟踪。能知道现在程序正在执行什么函数（在不打断执行的情况下）是非常有用的。特别当调试优化过的版本或是无法得到相关调试信息的时候就更是如此。

下面展示的方法很简单，它将在整个过程中对整个调用堆栈进行跟踪。每个方法都应该包含一些在程序运行过程中更新堆栈信息的代码。如果某个函数没有这些代码的话，它的相关信息就不会被显示出来了。日志记录将汇报（如果允许的话）每个函数的进入和退出事件。然而，由于即使在没有输出信息的时候这些堆栈也是正确有效的，因此后台维护性的代码总是会被执行的。

```
void foo() {
   TRACE(TraceSwitch); // Add the stack trace here.
   ...
}
```

此实现只是简单地把当前堆栈上的函数名和相应的 Switch 给输出出来。由于很多编译器现在都提供一个预定义好的可以给出当前函数名的宏，我们在这里就可以用上它了。为了区分同名的函数，我们将使用函数签名来代替名称。要注意的是我们需要保留一个指向当前父节点的指针，以使得我们能够回溯到当前堆栈的上一层去。

```
class StackTracer
{
   StackTracer * myParent;
   Switch * myLeaf;

   const char * myName;

public:

   StackTracer(Switch * leaf, const char * name) :
      myLeaf(leaf), myName(name) {
```

Download at www.Pin5i.Com

```
        myParent = ourParent;

        reportEvent("Enter -- ");
        ourParent = this;
    }

    ~StackTracer() {
        reportEvent("Leave -- ");
        ourParent = myParent;
    }

protected:

    void reportEvent(const char * prefix) {
        ...
    }

    static StackTracer * ourParent;
};
```

通过使用特定于编译器的对函数签名的定义，我们就可以很容易地定义一个辅助性的宏了。请注意，我们使用了一个预定义的，在微软 Visual Studio .NET 之前的编译器中没有的宏。因此，某些编译器可能不支持此特性。一些其他的编译器，例如 MetroWorks CodeWarrier，就可以通过使用 GCC 的扩展选项__PRETTY_FUNCTION__来支持此特性，它们做的是完全一样的事情。

```
#define TRACE(Leaf)    \
    StackTracer        \
    stackTop(Leaf::getInstance(), __FUNCSIG__)
```

在任一时刻，这个虚拟的调用堆栈[1]都会被用来输出程序中当前的位置。例如，当一个意料之外的情况发生的时候，下一个服务就可以输出调用堆栈信息了。通过这种方法，比起原来只有文件名和行号的情况，用户就可以得到更有价值的信息了。

3. 交互式的输出

最后，为了得到交互式输出的纵览，首先我们将解释一下常见的断言（assertion）宏。一般说来，如果断言失败，程序将显示一个消息对话框以提示用户，让其选择放弃运行此应用程序、调试程序或者简单地忽略此错误。断言处理的这种逻辑和 Journal 不能混起来，因为它可以被其他任何的交互式服务所用。因此，随着以上跟踪服务中 endReport 事件处理的不同，汇报时的行为也会不同。它应该只与用户的反馈进行通信以对之进行适当的处理。下面代码中所有使用到的值都在 Journal 接口里列举了出来。

```
#define ASSERT(Leaf, Condition)                \
    do {                                       \
```

译者注[1]：此处的“虚拟堆栈”指的是 StackTracer。由于每个 StackTracer 都有一个指向其上一层的 myParent 指针，因此它也相当于一个调用堆栈，记录了调用的顺序和位置信息。

Download at www.Pin5i.com

```
        Switch * leaf = Leaf::getInstance();          \
                                                      \
        if(leaf->getState() && !(Condition)) {        \
            char result =                             \
                displayAssert(leaf->getJournal(),     \
                    #Condition,                       \
                    __FILE__,                         \
                    __LINE__);                        \
                                                      \
            if(result == Journal::ABORT) {            \
                System::Terminate();                  \
            }                                         \
            if(result == Journal::BREAK) {            \
                System::Break();                      \
            }                                         \
        }                                             \
    } while(0)
```

需要注意的是，这将会产生一些很有趣的事情。如果我们此时又使用了另一个 Journal，那么就可以在没有交互的情况下运行程序了。断言仍然可以被捕获和汇报，但是这时候就没有等待用户选择的对话框了。

1.16.5 结论

小结一下，此处我们展示了一个简单且可扩展的系统，它用来控制实时信息流，且拥有一些基本工具对之进行汇报。然而我们必须澄清的是，信息的质量和如何把它进行适当的解析以得到有用的各个部分是开发人员的责任。必须要记住，如果你在进行这些信息汇报上投入了时间，那么在调试的时候你就会节省更多的时间。

1.16.6 参考文献

[Reeves01] Reeves, Jack, "The (B)Leading Edge: Using IOStream, Part I," The C/C++ User's Journal Experts Forum, 在网址 http://www.cuj.com/experts/1901/reeves.htm 上有在线资料, January 2001.

[Stroustrup00] Stroustrup, Bjarne, *The C++ Programming Language Special Edition*, Addison Wesley, 2000.

Download at www.Pin5i.com

1.17　实时的层次化性能评测

Greg Hjeistrom，Byon Garrabrant
Westwood Studios
greg@westwood.com
byon@byon.com

在大多数游戏的开发之中，一个主要的目标就是要使代码的性能达到最佳效果。要想达到此目的，最关键的就是要知道你需要在什么地方进行优化。我们都听说过各种版本的一句古老谚语：“一个程序有 90%的时间花在了 10%的代码之上。”如果想发现这 10%需要优化的代码究竟在什么地方，性能评测（profiling）就是一个无价之宝了。

一般我们使用的是两种性能评测的策略——采样与显式的（explicit）计时。一个采样性能评测器的工作原理就是要在程序运行的时候对指令指针的位置进行频繁的采样。此种方法将生成巨量的原始数据，然后这些原始数据将被进行处理以生成性能评测数据。采样性能评测器一般都能很精确地指出究竟是在哪一行代码耗费了大多数的时间。很多商业性能评测器都是使用这种方法的，因为这样就不需要改动应用程序的代码了。糟糕的是，想要在实时的情况下进行采样通常是不现实的．因为在收集和处理上的开销太大了。

另一个常用的性能评测方法就是显式地记录下代码块执行的时间。接下来，这些测量结果可以被实时地显示出来，这样就可以帮助找到瞬时的（transient）性能问题了。这一点是很重要的，因为游戏中的瓶颈往往和很多因素有关。如果在用户玩游戏和触发不同事件的时候能看到性能评测数据的变化情况，那就会相当有用了。另外，这些评测数据一般都是按逻辑组织好了的。例如，你可以纪录下代码花费在 AI、渲染，以及物理处理[1]上的时间。然后在任何时候，你都可以看到究竟是代码中的哪个子系统耗费了大部分时间。这种性能评测有个唯一的缺陷，那就是对于代码处理的时间究竟花费在了什么地方，它仅给出了一个大体的分析。例如，一旦你知道了 AI 运行得特别慢，通常你就需要使用一个采样性能评测器了，或是临时增加更多的时间记录信息，这样你才能知道为何 AI 会运行得如此之慢。

理想情况下，我们希望同时得到两者的优点——实时的性能评测，同时还需要如采样评测器一样尽量详细的信息。本节将探讨以下的一个系统，

译者注[1]　例如物体的碰撞和摩擦等方面的处理。

Download at www.Pin5i.com

它虽然没有采样性能评测器那样详细，但是也能高效地处理逻辑上组织好的层次化的上千个采样。系统将定时更新一个性能评测树，它可以被实时地浏览，而且我们可以利用它来找出CPU 的时间究竟花在了什么地方去。

1.17.1 性能评测树

性能评测树是一个由性能评测节点构成的多叉树。多叉树也是数据结构中的一种树，不过它可以有任意数目的子节点。此树的拓扑结构由应用程序中性能评测宏的位置决定。由于树中的每个节点一般说来都只有有限几个子节点，因此对其的搜索和显示比较高效。

每个性能评测节点对应于一段代码中单独的一个显式的计时采样。每个节点将跟踪此段代码执行耗费的总时间，以及此段代码被执行的总次数。在另一个采样中进行的采样对应了那个采样的一个子节点。不管在什么时候，一旦进行了一次性能评测调用，在当前的节点上就会增加一个新的子节点或是重用一个子节点。

让我们来看一个现实中的例子，先来看看在 *Command & Conquer Renegade* 中此性能评测系统是如何使用的。*Regenade* 总共拥有超过 1000 个性能评测节点（参见图 1.17.1）。不过，此处的性能评测效率很高，因为平均算来，一个节点只有 2 个子节点，即使在最坏的情况下也只有 15 个子节点。而我们下面就将说道，此算法的代价是与一个节点的子节点数目成正比的。

1.17.2 用法

如果想要使用此性能评测系统，那就需要在代码的关键点处放置 PROFILE 宏。正如下面将要谈到的，在代码中放置一个 PROFILE 宏就会生成一个性能评测节点,此节点将负责对相应的代码段进行计时。有个较好的方法就是在应用程序中每个子系统的开头处放上一个 PROFILE 宏，然后用户还可以对它进行细化。

```
void My_Function(void)
{
    PROFILE("My_Function")
    ...
}
```

最好能把大的流程分成几个独立的采样来处理。这是很容易的，只需要在函数里面加上额外的定域符（scoping brackets）就可以了，然后每个域里面都加上自己的 PROFILE 宏。下面就是一个例子：

```
void BigFunction(void)
{
    {
        PROFILE("BigFunction Part 1")
        ...
    }
    {
```

Download at www.Pin5i.Com

```
        PROFILE("BigFunction Part 2")
        ...
    }
}
```

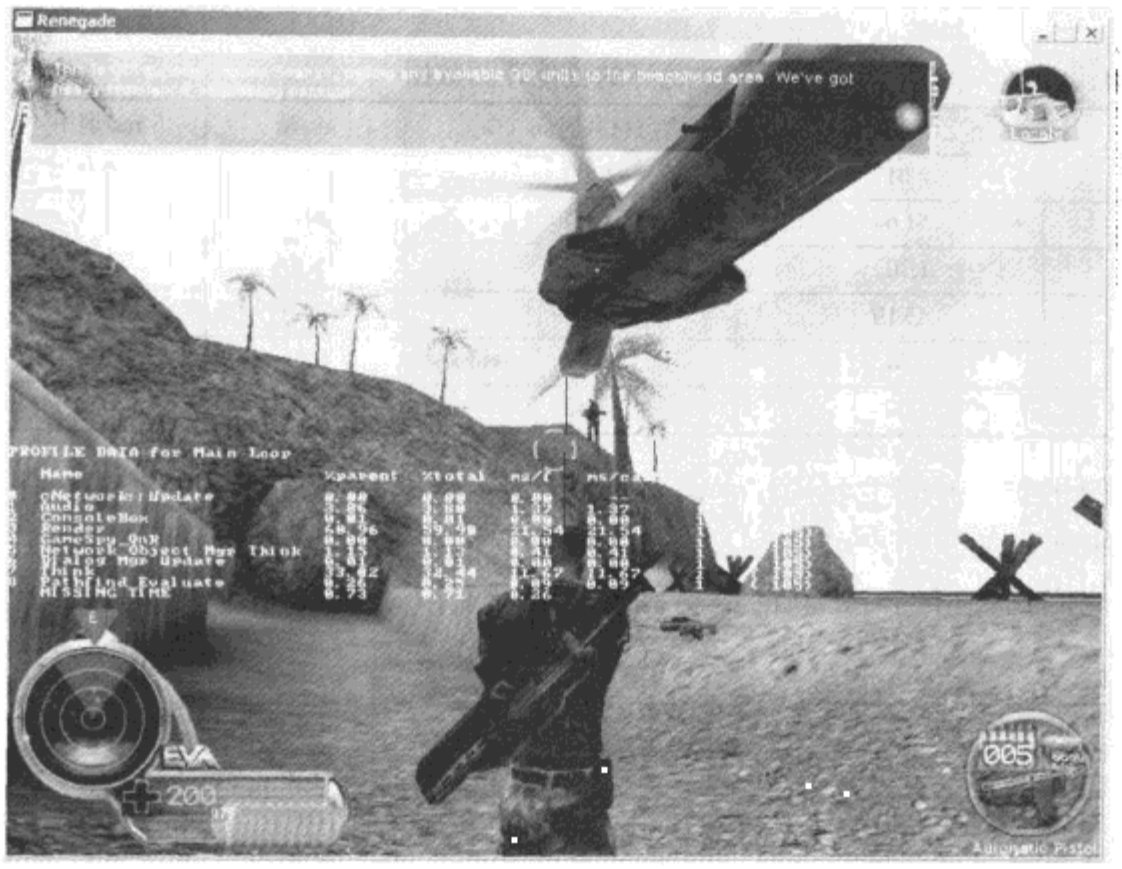


图 1.17.1　这是 *Command & Conguer Renegade* 中实时层次化性能评测器运行的截图。只有一个树节点显示了出来，但用户可以实时地对此树进行游览

同样有用的是，可以将一个类层次中的成员函数性能评测的结构“扁平化”。下面的例子中展示了如何可以得到一个重载函数在特定某层中耗费的总时间，即使它会被许多不同的派生类给调用。在这个例子中，耗费在 BasePhysice::Timestep() 中的时间在 CarPhysics::Timestep() 中并没有计算，所有耗费在 BasePhysics::Timestep() 中的时间都会被合起来以得到一个单一的性能评测采样，即使它会被其他的派生类调用（假设它们都使用这种性能评测的方法）。

```
void CarPhysics::Timestep(void)
{
    {
    PROFILE("CarPhysics::Timestep");
    ...
    }
    BasePhyics::Timestep();
}
```

浏览性能评测数据

此性能评测系统将生成大量的数据，因此用户必须找到一个方法以方便对之进行操纵。

Download at www.Pin5i.Com

我们提供了一个迭代器以用来对此树进行操纵和显示当前节点子节点的统计数据。通过使用此系统，用户可以在性能评测树中上下走动以实时地跟踪瓶颈，另外还可以将焦点集中到占用了大量时间的代码段上。例如，在表 1.17.1 中我们就列出了 Renegade 主循环中具有代表性的性能评测采样。

表 1.17.1 主循环的性能评测数据

名称	占用父节点时间（%）	占用总共时间（%）	ms/帧	ms/调用	调用/帧
0－音频	2.01	2.00	0.35	0.35	1
1－渲染	51.64	51.40	8.95	8.99	1
2－网络	1.70	1.69	0.30	0.30	1
3－思考	43.19	42.99	7.49	7.49	1
4－寻径	0.04	0.04	0.01	0.01	1
没有记录的	1.42	1.42			

在这个例子中，时间主要用在了渲染和思考上。对树的操纵是通过给每个子节点赋予一个数字索引来实现的。用户可以输入父节点或是其任一子节点。假设用户想调查一下渲染节点的情况。他们就会得到一个新的信息显示，如表 1.17.2 所示。

表 1.17.2 渲染的性能评测数据

名称	占用父节点时间（%）	占用总共时间（%）	ms/帧	ms/调用	调用/帧
0－线程切换	16.39	8.58	1.47	1.47	1
1－后渲染处理	0.28	0.14	0.02	0.02	1
2－渲染收尾	8.04	4.21	0.72	0.72	1
3－对话框管理	0.09	0.01	0.01	0.01	1
4－渲染游戏	49.55	25.95	4.44	4.44	1
5－渲染启动	1.70	0.89	0.15	0.15	1
6－阴影生成	22.83	11.96	2.05	2.05	1
没有记录的	1.12	0.09			

由于渲染游戏占用了此节点的大部分时间，因此用户可以再深入一层以得到关于此节点更详细的信息。只要当前你感兴趣的节点还有子节点，这个过程就可以无限进行下去。

此性能评测系统中所有的统计数据都代表了运行时的总和与平均值。利用清零的特性以得到历史数据是很有用的。例如，当帧率下降的时候，我们经常会把此性能评测器清零来得到一个更精确的测量数据，以得知当时代码究竟在做什么。当此性能评测器清零的时候，树并没有被销毁，只不过是每个节点的数据被清零了。

1.17.3 实现

实现依赖于采样时间的精确性。在此，我们使用了 Pentium CPU 的 64 位循环计数器。每当 CPU 执行一条指令，此计数器就自动加一。在性能评测采样的开头保存一下此计数器的值，然后在此性能评测采样的结束时再得到此计数器的值，把两者相减，你就可以很精确地计算出花费的时间了。然后我们再把此数除以 CPU 时钟的速度就可以得到消耗的时间了。我们的性能评测节点使用的是一个浮点变量来保存时间总和的。

Download at www.Pin5i.Com

1. CProfileSample

这是一个很小的 C++的类，它只有一个任务，那就是在其建构函数中调用 CProfileManager 的 Start_Profile()方法，在其析构函数中调用 Stop_Profile()方法。这样一来，在一个对象的域里面，性能评测采样的开始和结束的任务就自动化了。为了进一步简化此用法，我们可以使用 PROFILE 宏来自动创建一个 CProfileManager 对象，然后在发布版本中可以很容易地把性能评测的代码给去掉。

```
class    CProfileSample {
public:
     CProfileSample(const char * name)
     {
          CProfileManager::Start_Profile(name);
     }
     ~CProfileSample(void)
     {
          CProfileManager::Stop_Profile();
     }
};

#define PROFILE(name) CProfileSample __profile(name)
```

2. CProfileManager

这个性能评测管理器是性能评测系统的外部接口。它维护了一个性能评测树中的 CurrentNode 指针，其对应的是当前正在执行的代码中的域。它还有一个方法，用来访问性能评测树并将其数据显示出来。

Start_Profile()方法用来开始一个性能评测采样。通过比较当前节点的名字和要求进行性能评测的名字，可以检测出递归调用。当找到了一个与给定名字匹配的子节点的时候，此节点就成为了当前结点。在搜索中引入的额外开销是与当前节点的直接子节点数目成正比的。一般说来，这是一个小数目，很少会超过 10。此外，由于对所有的性能评测采样的名字使用的都是静态字符串，在查找一个特定的性能评测采样时，我们就可以使用指针比较而不是较慢的字符串比较法了。

如果出现了子节点中没有匹配名字的情况，一个新节点就会被创建出来并连接到树上去。要注意的是只有在第一次通过一个特定的代码段时才会创建节点。在任何情况下，此节点的 Call()方法都会被调用以开始计时。

```
void CProfileManager::Start_Profile(const char * name)
{
     if (name != CurrentNode->Get_Name()) {
          CurrentNode=CurrentNode->Get_Sub_Node(name);
     }
     CurrentNode->Call();
}
```

Stop_Profile()方法是用来结束一个性能评测采样的。它的第一个任务就是要在当前的

Download at www.Pin5i.Com

节点上调用 Return()方法以结束和记录计时。由于性能评测管理器维护了当前被采样的节点，因此在此操作中不会产生额外的搜索开销。假设我们现在没有在一个递归函数中间被调用，父节点就变为了当前节点。

```
void CProfileManager::Stop_Profile( void )
{
    // go to parent unless recursed
    if (CurrentNode->Return()) {
        CurrentNode = CurrentNode->Get_Parent();
    }
}
```

性能评测管理器余下的方法是一些访问性的函数或是执行某些简单的管理功能的函数。例如，应用程序每帧将调用一个 Increment_Frame_Counter()方法以便于对每个性能评测采样计算每帧的调用次数。Get_Iterator()和 Release_Iterator()方法分别是用来提供和销毁一个用于访问性能评测树的迭代器。

3. CProfileNode

CProfileNode 是性能评测系统内部使用费的 C++类。它保存了一个代码段里面所耗费的总时间和此代码段的调用次数的数值。当 CPU 在执行相应节点所在域里的代码时，此节点还存储了开始时间。

在开始一个性能评测采样的时候，性能评测管理器将调用此节点的 Call()方法。此方法仅是用来增加一下调用计数的，而且如果我们正在进行递归调用的话，还将记录其开始时间。

```
void CProfileNode::Call( void )
{
    TotalCalls++;
    if (RecursionCounter++ == 0) {
        Profile_Get_Ticks(&StartTime);
    }
}
```

在每个性能评测采样的结尾，Return()方法将被调用。在检查了是否在进行递归调用以后，系统将计算出花费的时间，并将其加到 TotalTime 变量上去。此函数还将返回一个值表示代码现在是否在被递归调用，这样性能评测管理器才能知道是否此节点的处理已经结束，如果结束了则要返回到其父节点上了。

```
bool CProfileNode::Return( void )
{
    if (--RecursionCounter== 0 && TotalCalls != 0 ) {
        __int64 time;
        Profile_Get_Ticks(&time);
        time-=StartTime;
        TotalTime += (float)time / Tick_Rate();
    }
    return ( RecursionCounter == 0 );
}
```

Download at www.Pin5i.Com

4. CProfileIterator

这个对象提供了一个简易的方法以用来浏览性能评测树。它包含了用来操纵此树的方法和显示一个特定节点内容的方法。典型地说，一个节点和其直接子节点将向用户显示出来。对于每个子节点来说，我们可以提供好几个统计量：耗费的总时间、总的调用次数、每帧的调用次数，还有每帧中花费的时间。

1.17.4　结论

以上系统具有实时对代码进行层次化的性能评测的能力，而且能高效地对上千段代码进行采样，因而我们发现这已经是一个对代码优化来说很有用的工具了。我们希望读者能使用此系统对自己的代码进行改善。你将在附送的 CD-ROM 上找到这些类的完整的实现。

1.17.5　参考文献

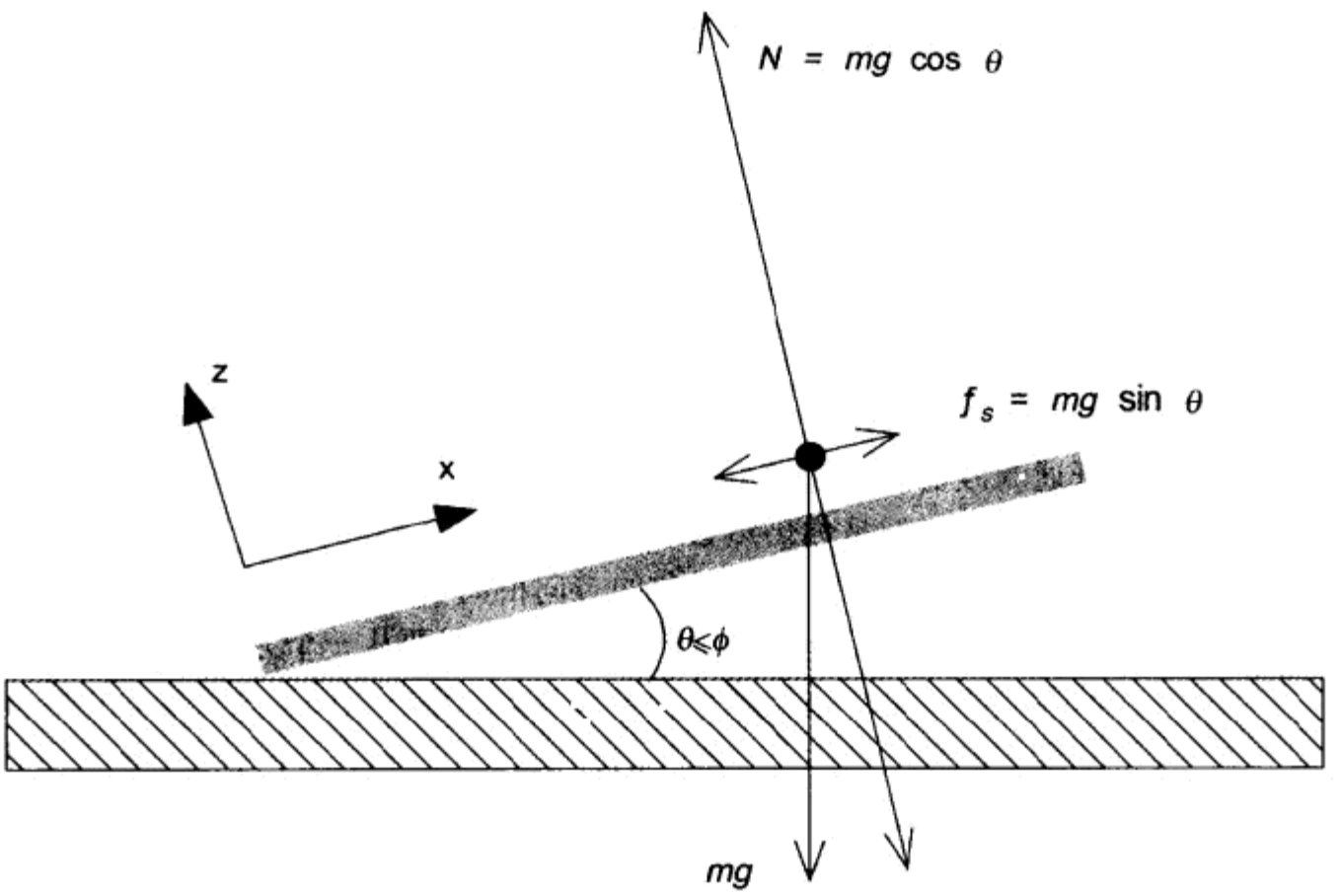
[GPG00] Rabin, Steve, *Game Programming Gems*, Charles River Media, Inc., 2000.

Download at www.Pin5i.Com

Download at www.Pin5i.Com

第 2 章

数学技巧



Download at www.Pin5i.Com

简介

John Byrd
jbrrd@well.com

为什么我们要关心游戏开发中更深层次的数学问题呢？说到底，优化不就是要把嵌在一起的循环代码分离开，或是要尽量利用高度缓存处理顶点数据这样的事情么？此外，现代的硬件几乎可以为你处理一切 3D 数学运算和渲染！我们关心它的原因很简单，一旦我们忽视了数学，程序就不会正确地工作了。

过时的 3D 游戏机，例如 3DO、Sega Saturn 以及 Sony PlayStation 都使用仿射纹理。这给第一人称视角的射击游戏带来了很大的麻烦，直到 1996 年出现了 Nintendo64，给游戏带来了能正确处理视角的纹理之后，情况才有了好转。为了渲染出能正确处理视角的纹理，我们必须在渲染流水线上进行一些工作，在 uv 空间里去解二阶偏微分方程。此处的数学确实很讨厌，但是这是必须要完成的任务。

假设你需要对一个数字音频信号进行重采样，将其实时地从 44.100kHz 降为 11.025kHz。因此，每 4 个采样就丢掉 3 个采样，将其结果送至 DAC[1]中。然而，出于某种原因，得到的音频听上去好像不对。在其中出现了一些原来信号里面没有的泛音。或许你需要在重采样之前使用一个低通滤波器，这样就可以避免出现 Nyquist 错误了。这里涉及的数学也是挺麻烦的，但是你仍然必须要完成此任务。

在欧拉坐标系中表示对象的方向很简单，与我们在大学里学到的方法是一样的。然而，在真正的游戏中，我们需要从一个方向转到另一个方向。在欧拉空间里进行这种操作会导致对象出现剧烈的摇摆现象，就好像出现了一种我们称之为“平衡锁”的神秘的力量一样。不过，如果我们使用四元数空间的话，调换方向就容易得多了——这里涉及的数学同样可怕，但是你仍然必须要完成此任务。

讲述游戏中数学的文章在此章里面已经达到了相当的高度，因为它与当今一般的硬件特性紧密相关。在往昔的岁月里，一个单精度乘法就需要耗费数毫秒才能完成，实现所有的超越函数都是如此的困难。这种情况下，在游戏里面进行这样昂贵的操作是一定要使用查找表的。当今，在大部分硬件上，几乎可以肯定你至少可以在一条流水线上同时进行四个乘法，或许还可以加上一个加法。在本章中，你将从 Green 处学习到如何利用这些

译者注[1] DAC：Digital Audio Codec。数字音频编解码器。

Download at www.Pin5i.Com

硬件的特性实现某些可怕的超越函数。你的寻径算法是否由于精度不够而常常失败呢？如果如此，请参看 Young 的算法，它针对 128 位系统进行了优化。把所有的四元数保存在内存里面是不是很困难？通过 Adami 的方法，不需要多少工作量你就可以将一个四元数压缩到 32 位了。

游戏开发人员对数学的依赖性就如制作人对游戏开发人员的依赖性一样大。如果你仅仅满足于在大学数学课中学到的二重积分与泰勒级数，你还是可以做一个称职的游戏开发人员，但是在选择算法的时候就一定要很小心了，因为这时候它们实际上是取代了你所应该做的工作了。

Download at www.Pin5i.Com

2.1 对数与随机数生成的 2 基快速函数

James McNeill
James_mcneill@ameritech.net

在本节中，我们将讨论 3 个用来计算与整数的 2 基（base-2）数相关的函数。它们简单明了、效率较高，而且对于任何 32 位的输入来说都是正确的。这比起其对应的浮点函数来说好得多了。

2.1.1 整数的 2 基对数

在游戏开发过程中，有的时候我们需要计算一个整数对 2 的对数。此整数的对数是用实数的对数来计算的，然后再将其结果向高或向低取整得到所需的最接近的整数。在以下几种例子中我们需要计算一个整数对 2 的对数，例如对一个纹理图像的维数进行舍入以得到最接近的 2 的指数幂，对一段数据进行填充以得到最接近的 2 的指数幂以进行快速傅立叶变换，或是得到一个四叉树层数的数目以便分隔一个网格。

虽然可以使用例如 `int(floor(log(n)/log(2)))` 或是 `int(ceil(log(n)/log(2)))` 这样的表达式来计算对数，但是浮点除法可能会产生下溢，从而将得到不正确的结果。我们用上面的表达式编了一个简单的测试程序，在一个英特尔处理器的机器上，当 n=65536 的时候就产生了这种错误。另外还有一些环境，在其中虽然没有浮点运算功能，我们仍然需要这么一种计算整数的对数的方法。

为了避免这些问题，我们使用了在本节中将谈到的两个仅用到整数的函数。在整个 32 位数的输入范围之内，它们都能提供正确的结果，而且比起对应的浮点方法来说要高效得多。表达式 log2le(n)，它代表的是“$\log_2$ 小于或等于 n”，将找出满足 $2^x \leqslant n$ 条件的最大的非负整数 x。表达式 log2ge(n)，它表示的是“$\log_2$ 大于或等于 n”，将找到满足 $2^x \geqslant n$ 条件的最小的非负整数 x。这些函数的代码在 CD-ROM 上面都可以找到。

2.1.2 位掩码与随机数生成

本节提供的第三个辅助性函数 `bitmask(n)`，将用来计算掩码（mask），在此掩码中所有从 1 到 n，且包含 n 本身的数占据的位都将被设置为 1。

Download at www.Pin5i.Com

bitmask()函数的一个用途是可以用来生成在某一范围内均匀分布的随机整数。先看一个例子，假设有个函数 rand()可以生成从范围 0 到 32767 均匀分布的随机整数，但是我们需要一系列在[0,2]范围内的随机数。一般的方法是使用一个取模操作符将源域映射到目标域上去：

```
int randomNum = rand() % 3;
```

正如在[Booth97]里说到的一样，此种方法将会带来好几个问题。取模操作符一般都相当的慢，而且此结果的分布也不是很均匀。这是因为[0,2]这个定义域并不能把域[0,32767]均匀地分开，randomNum 的值并不是均匀分布的。0 和 1 分别有 10 923 个值映射到它们，但是 2 却只有 10 922 个值映射到它。当然，我们承认这个差异并不很大，但是偏差可能会变得更大。例如，如果我们使用这种方法在范围[0,32766]中生成随机数的话，零的几率就比其他任何数都要大一倍了。

如果想要得到均匀分布的结果，一个方法就是先生成随机数，然后将它们丢弃掉，直到遇到了落在想要的范围内的数字：

```
do { randomNum = rand() } while ( randomNum > 2 );
```

现在我们的例子就要花费相当长的时间了，这是因为 rand()生成的数落在此范围之外的概率比落在其中的概率要大得多。不过，经过修改以后，此方法就会快得多了。首先，使用取模操作符，不过使用的模要能把 rand()的域分成整数的均匀的几份，然后可以通过它来得到一个尽可能接近目标范围的中间结果。接下来，丢弃掉任何不在目标范围内的中间结果：

```
do { randomNum = rand() % 4 } while ( randomNum > 2 );
```

由于大多数随机数生成器都有一个 2 次幂的范围，因此我们可以使用位掩码来代替取模操作符以提高速度：

```
do { randomNum = rand() & 3 } while ( randomNum > 2 );
```

这就是我们的 bitmask()函数的用武之地了。它将计算出在此算法中使用的必要的位数，这样我们就可以编写一个通用的函数用来在任意的范围中生成随机数了：

```
unsigned random( unsigned range )
{
    if ( range < 2 )
        return 0;
    unsigned mask = bitmask(range-1);
    unsigned n;
    do { n = rand() & mask; } while (n >= range);
    return n;
}
```

假设 rand()函数的值是随机分布的，循环迭代次数的期望值将比 2 小，而且随着中间范围和目标范围的接近，这个值将逐渐逼近于 1。这比起其取代方法来说要快多了，而且也更加可靠。

Download at www.Pin5i.Com

2.1.3 函数是如何工作的

这些函数是专门针对 32 位整数的。这就将返回值的个数限制在了 32 或是 33 个（视函数而定）。函数 log2le()的返回值范围在 0 到 31 之间，log2ge()函数的返回值则在 0 到 32 之间，而 bitmask()返回的位掩码则将在 0 到 0xFFFFFFFF 之间。

有一个可以考虑的算法就是把这 32 或 33 个返回值保存在一个数组里面，当需要正确结果的时候就对此数组进行二分查找。此处的函数使用了这个方法，但是并没有保存数组，这是因为这些数组的各项在需要的时候很容易计算出来。只使用了极少的几个寄存器变量以作为结果。最后，这些循环都没有使用嵌套以提高速度。

每个函数的实现都可以在 CD-ROM 上找到。

2.1.4 参考文献

[Booth97] Booth, Rick, *Inner Loops: A Sourcebook For Fast 32-bit Software Development*, Addison-Wesley Developers Press, 1997: pp. 223～224.

Download at www.Pin5i.Com

2.2　使用分数矢量得到更精确的几何图形

Thomas Young
PathEngine
thomas@pathengine.com

故事是发生在项目收尾阶段的。一个特别聪明的测试人员发现在某关中的某个特定位置上寻径会出错，导致敌人陷入此地无法退出。还有一个人则注意到如果跳到另一个角落则你会从世界的边缘滑落出去。在经过了数日紧张的调试之后，你最终发现了问题的根源——近似计算带来的误差。

在游戏开发中，很少有比舍入误差更为隐蔽、更为阴险的误差了。也许你会在系统中加上一些代码来捕获这些误差。你将检查误差范围，写下一大堆特殊处理的代码。不幸的是，这往往只是把精确度的问题转化为了一组其他的麻烦问题。我们应该采用多大的误差范围？我们怎么保证这些代码一定是正确的？最后，当结果出来了以后谁能指出我们代码中的问题呢？

本节主要考虑的是结构固体几何图形以及寻径中的例子，这些例子中交点上的近似处理会造成算法的错误。我们提供了一些技术以避免使用近似来得到这些交点。这样，不用让算法更加复杂，我们就可以消除误差了。

2.2.1　问题

当一个多边形渲染到屏幕像素上面的时候，如果绿色值在最低有效位上错了一位，用户是不会察觉的。即使屏幕坐标偏移了一到两个像素，或许也没什么关系，只要没有视觉上的错误，例如在多边形中出现了裂缝。

从另一方面来说，在游戏开发中间有些很关键的算法，即使在数学计算中出现了很小的舍入误差，它也会对此极其敏感。特别是当一个算法需要重用隐含舍入误差的计算结果时，就会出现问题。对于一个复杂的算法来说，通常只有在针对某些数据结构的限制条件成立时，算法才能正确进行。或许只有这些条件成立，算法才能正常终止。

当我们将分数值表现为浮点数或是定点数的时候，经常会引入一些会影响到算法最终结果的舍入误差。我们是不可能使用有限精度的浮点数或是定点数来精确表示 1/3 的。在结构固体几何图形（CSG）中和可视点寻径中就会产生这种问题。

Download at www.Pin5i.Com

1. 构造式的固体几何图形

下面让我们考虑一下二维网格中布尔运算的问题。为了能对网格进行高效的处理，我们需要对此网格进行某些有效性验证。例如，我们可以要求网格的所有面都是凸的。只有这种有效性验证通过了，算法才能正确应用到对象上。

作为布尔操作过程的一部分，我们可能会发现有两个面重叠的现象，并以在边相交的地方引入一个新顶点的办法来解决这个问题（参见图 2.2.1）。而在相交点的近似运算则可能导致生成非凸的几何图形（参见图 2.2.2）。

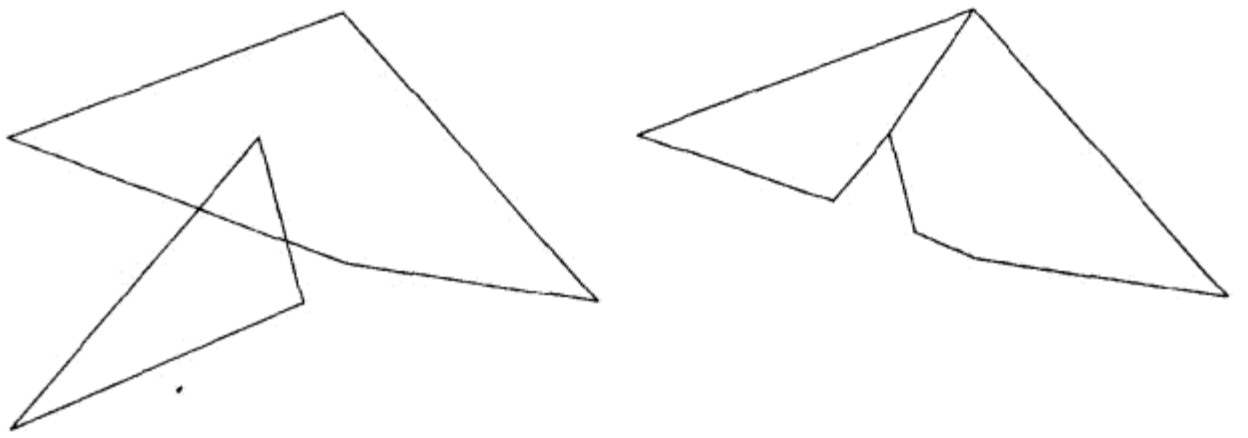


图 2.2.1 二维空间的布尔减法运算

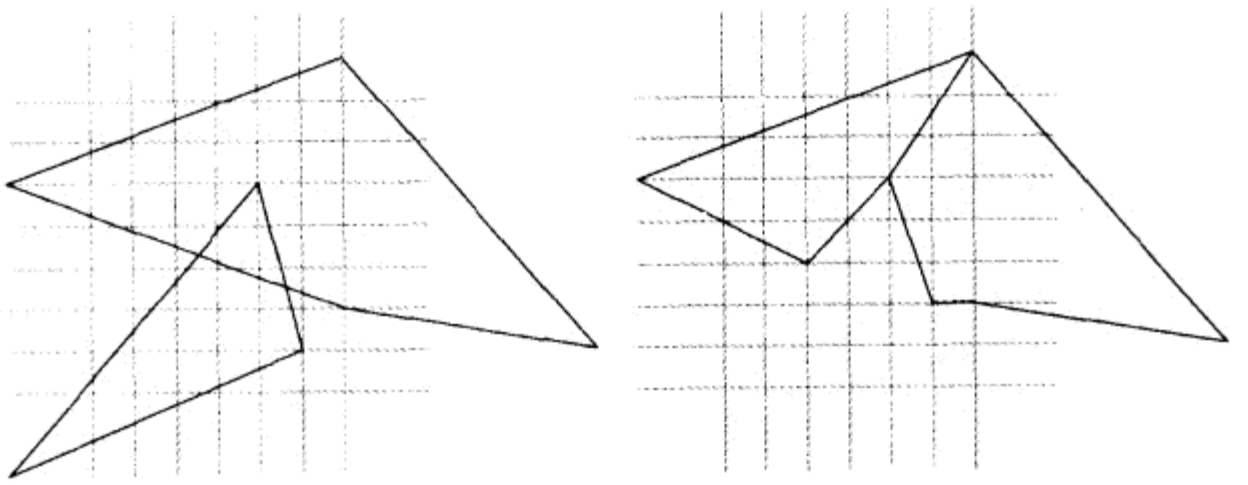


图 2.2.2 由于近似产生的非凸面

在三维结构固体几何图形中，在相交的面上生成边或是在面上相交的边上生成顶点的时候，也会发生同样的问题。

2. 寻径

[Young01]中描述了如何使用轮廓区域对可视点寻径进行优化。每个可视点都对应了环境中的一个角落。从源点的角度上来看，如果一个点处在轮廓上的话，它就将被连接上。

图 2.2.3 展示了在一个相交处的近似是如何造成寻径失败的。一个轮廓区域的边界与另一条线相交，后者可能是一个扩展的外部边界，另一个区域的边界，或是一个门户。在相交点的近似就意味着我们的源点会被以为是处在此轮廓区域以外，因此就没有生成到这个可视

Download at www.Pin5i.Com

点的一条连接。从源点到目标点的连线便会与障碍物的边界相交，从而导致碰撞。其结果就是寻径器没有生成一条到达目标区域的路径。

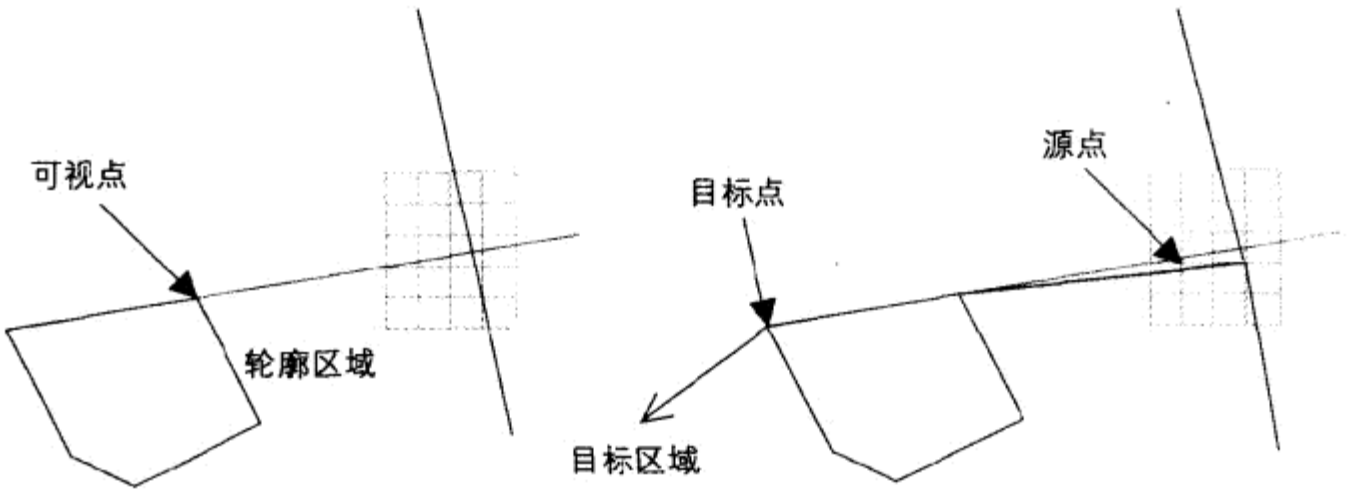


图 2.2.3　由于源点被错误地放在了轮廓区域的外部导致了寻径失败

针对此问题的一个解决方法就是把轮廓区域的边界线看成是无限长的直线。为了检测出源点是否在一个给定的区域里面，我们将测试每一条直线。然而，如果能知道当一个点移去的时候什么边会被遍历，以此跟踪一个点在什么区域的话，速度就会快得多了。如果我们的寻径器不得不与重叠的地形打交道，那么这种遍历还可以用来描绘出不同层上的几何图形。

为了进行遍历，我们需要能判定一个交点究竟是在一个遍历的哪一边。在图 2.2.4 中，遍历是结束于轮廓区域之内的，因为交点在遍历直线的右边。

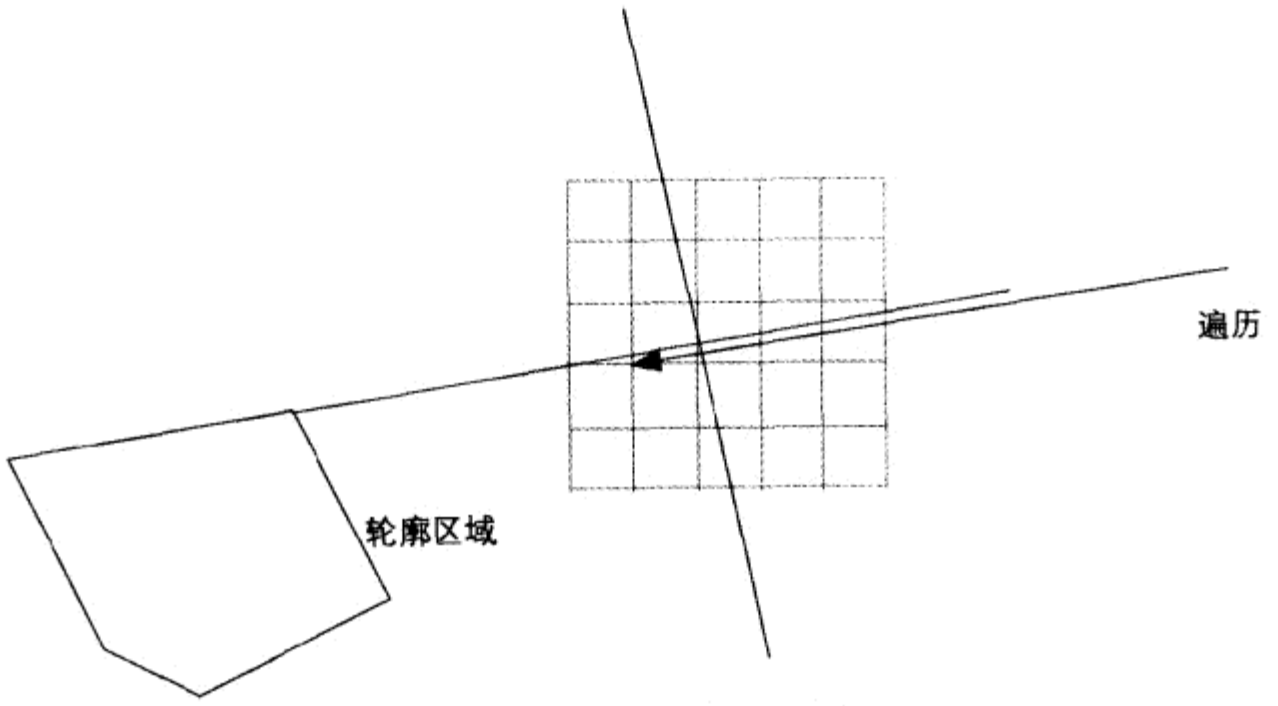


图 2.2.4　对一个轮廓区域进行遍历

3. 为什么不仅仅需要浮点数?

浮点数的表现形式只能给我们以更加接近 0 的精确度。在此数范围的边界上面，实际上我们得到的精确度比一个整数形式的还要小，因为浮点数还需要一些位来存储指数位。浮点数的表现形式暗示的是类似于图 2.2.5 表现的网格，但是其粒度更小。因此，问题仍然存在，因为结果仍然是要近似到一个网格上的。

Download at www.Pin5i.Com

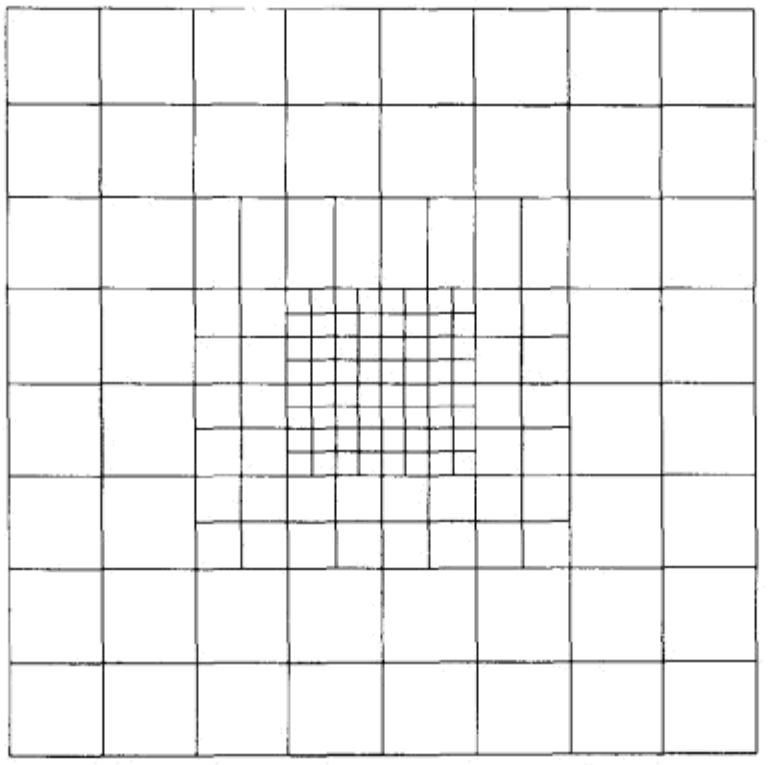


图 2.2.5 一个浮点网格

甚至是加上了任意的精度也依然解决不了这个问题，例如，不管是用多少个十进制的数位，我们也无法精确地将 1/3 给表示出来。不管我们的网格有多小，某些误差出现的频率仍然保持不变。除非误差几率变成了无限小，不然试图减少误差出现的频率只会给我们带来更麻烦的问题。

2.2.2 一个解决方法：分数矢量

我们可以不用近似来表示相交点，这样就解决了这个问题。我们提议使用分数矢量作为达到此目标的一个方法。

1. 2D 线条的相交

假设在二维空间里有两条无限延伸的直线，它们分别用起点与终点表示为 S_1A_1 与 S_2A_2。只要这两条线不平行，它们就会相交，因而我们可以使用交点 I 来表示延某条线的轴向上的分数距离（参见图 2.2.6 与图 2.2.1）。

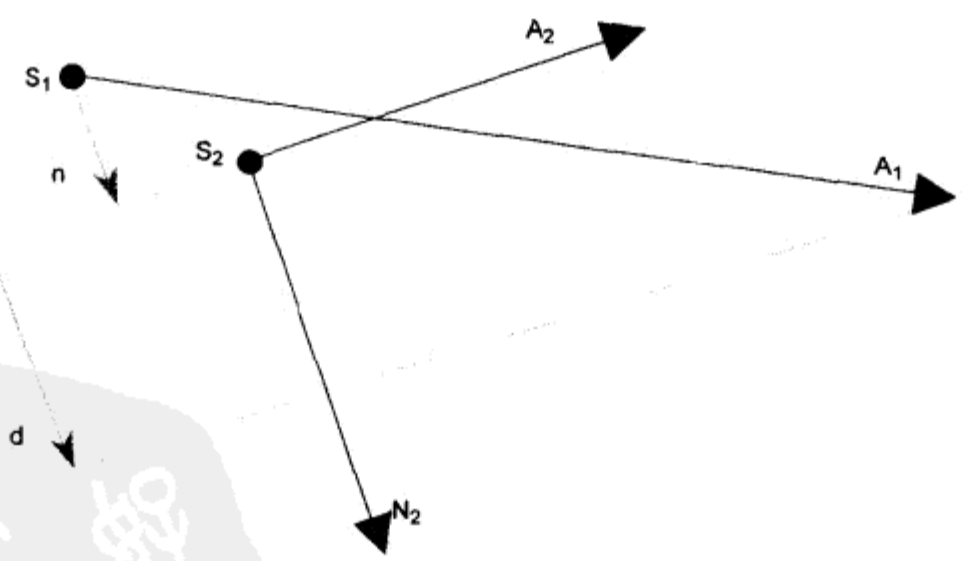


图 2.2.6 将交点表示成一个分数矢量

Download at www.Pin5i.Com

$$I = S_1 + \frac{nA_1}{d} \tag{2.2.1}$$

其中的分母 d 与分子 n 是由与矢量 N_2 的点积决定的，它是一个与第二条线垂直的矢量（方程 2.2.2 与方程 2.2.3）。这些值与图 2.2.6 中箭头所示的长度成正比例。

$$n = (S_2 - S_1) \cdot N_2 \tag{2.2.2}$$

$$d = A_1 \cdot N_2 \tag{2.2.3}$$

2. 一条 3D 线条与一个平面的相交

我们很容易对上述的方法进行扩展以处理一条 3D 线条与一个平面的相交。在这种情况下面，我们可以与平面的法线进行点积以得到延三维直线方向上交点对应的分数距离。

2.2.3　使用分数矢量

我们可以像使用一般的分数一样来使用分数矢量。最重要的是，当我们使用分数矢量的时候，多重乘法可以让我们避免进行显式的除法进行某些操作。将除法消除了以后，我们就可以在没有近似的情况下进行需要的几何图形操作了。涉及到整数值的相加、相减和相乘的操作只会产生整数结果。

1. 检测相交点在线段的哪一侧

对于一条起于 S 轴向 A 的无限直线来说，若想判定一个点 P 是否在其右侧，我们可以检查方程 2.2.4 中的不等式是否成立。此方程假设 x 的正方向在 y 的正方向的右侧：

$$(P_x - S_x)\, A_y < (P_y - S_y)\, A_x \tag{2.2.4}$$

为了将此方程应用到一个代表了某分数矢量的点上面，我们只需利用一个分数来表示 P（方程 2.2.5），然后将每一项都乘以 d_p 以消去除法就行了，这样我们就得到了方程 2.2.6 中的不等式。这就给出了一种方法，通过它我们就可以实现在寻径区域里面所需的遍历了。

$$P = B + \frac{C}{d_p} \tag{2.2.5}$$

$$(B_x d_p + C_x - S_x d_p)\, A_y < (B_y d_p + C_y - S_y d_p) A_x \tag{2.2.6}$$

2. 扩展到其他操作

我们很容易就可以将此方法进行扩展，得到针对分数矢量的更通用的操作。为了比较个点 P 与另一个点 Q（它也是使用一个分数矢量来表示的，如方程 2.2.7 所示），我们将这两个点化为同一个分母，使用 $d_p d_q$ 乘以它们。P 与 Q 相加的分数矢量的结果如方程 2.2.8 所示。

$$Q = D + \frac{E}{d_q} \tag{2.2.7}$$

$$P + Q = B + D + \frac{d_q C + d_p E}{d_q d_q} \tag{2.2.8}$$

Download at www.Pin5i.Com

3. 另一个遍历的方法——使用相交的顺序

另一个解决我们遍历问题的方法是可以在一个通用的轴向上对两个交点的距离进行比较。图 2.2.7 描述了这个遍历问题。我们需要判定区域边界（S_1A_1 与 S_2A_2）相交点究竟在遍历线 S_3A_3 的哪一边。如果我们已知遍历线从右向左穿过了边界线 S_1A_1，那么我们就可以比较延 A_2 轴向上的分数距离了。当边界顶点在遍历线的右侧时，则方程 2.2.9 中的不等式成立。

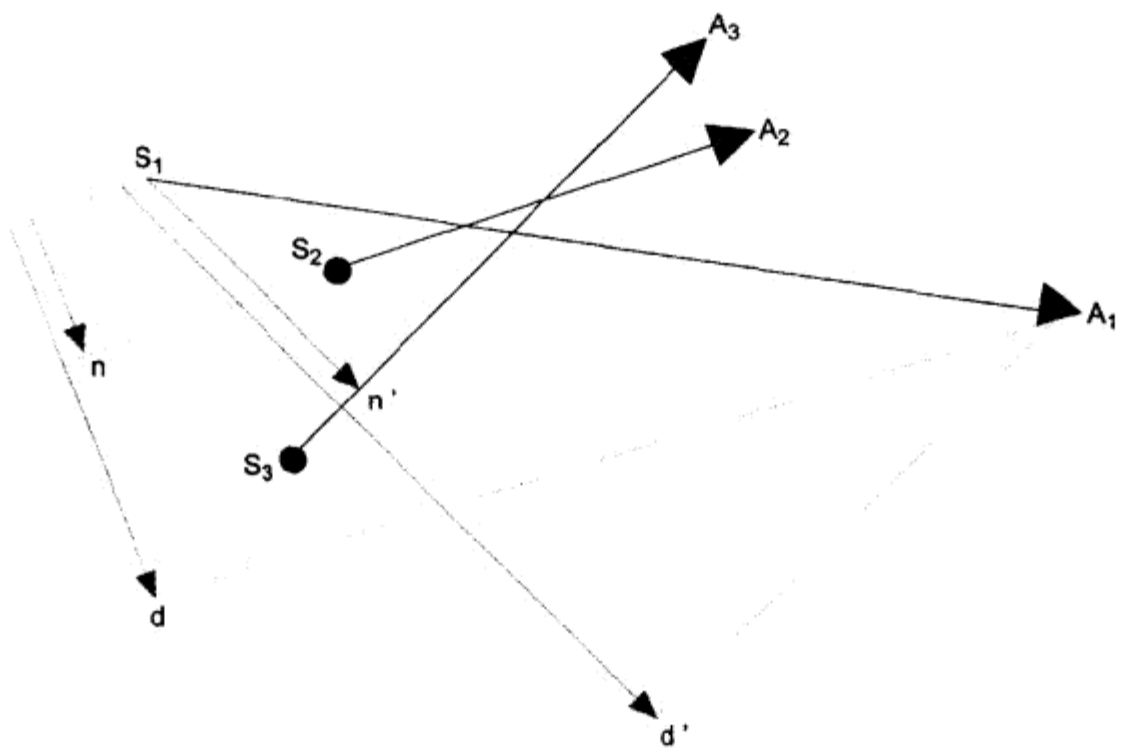


图 2.2.7 确定交点的顺序

$$nd' > nd' \tag{2.2.9}$$

2.2.4 数字的范围

通过使用整数坐标和消除除法，我们就可以不使用近似，不需要额外的精度要求来进行几何图形的操作了。然而，我们必须仔细考察数字的范围，以防止溢出的可能。必须支持的范围要取决于我们几何图形中坐标使用的数字范围，还有对此几何图形采取的精确操作。

1. 得到范围

让我们假设在自己的几何图形中，所有点的坐标都在范围[0, r]以内，且矢量是由从终点到起点的一个减法得到的。

由对点坐标进行减法得到的矢量范围应该在[–r, r]之间。在方程 2.2.1 中的分子与分母都是由这些矢量的点及形成的，因此在二维情况下它们的范围应该在$[-2r^2,2r^2]$之间。对于三维的情况，三维矢量的点及其范围是$[-3r^2,3r^2]$。如果对方程 2.2.6 中的二维分析进行扩展，我们将在不等式的两边得到$[-6r^4,6r^4]$的范围。对于方程 2.2.9，其范围是$[-8r^4,8r^4]$。

2. 将范围适应到整数数据类型

假设我们有 128 位来表示$-8r^4$到$8r^4$的范围，那么我们将得到针对 r 的方程 2.2.10。解出

Download at www.Pin5i.com

不等式，将结果舍入到整数以后就可以得到 r 的范围了。

$$8r^4=2^{127}-1 \tag{2.2.10}$$

$$r=2^{31}-1 \tag{2.2.11}$$

点和矢量可以存储在 32 位值里，点积的结果是 64 位值，点积的积则是 128 位值。对于不同的限制或是不同的所需计算来说，则需要选择一套不同的范围，因此，还有一套不同的数据类型。

3. 管理范围的限制

对不同阶段的计算定义不同的数据类型可以帮助管理这些不同的范围。例如，点和矢量应该使用不同的类型来实现，当然还有点积与点积的乘积。通过这种方法，我们就可以使用编译器来检查是否在一个计算中使用了正确的数据类型。

最好在调试版本中进行范围检查。如此，只有当数据类型需要从其他尚未定义范围的数据类型中建构出来的时候才进行范围检查，这就使得其对性能的影响最小化了。这样一来，我们就可以确保尽可能少地发生这种建构了。

在编译期，可能会难以得知在复杂操作中需要使用什么样的范围。在这种情况下面，或是仅仅为了预先计算出一个取值范围，我们可以使用一个整数类，它可以按需分配内存以表现任意大小的整数。现在有很多开发包可以完成这个任务，读者可以参见[GMP02]以及[Haible02]。

2.2.5　实现上的细节

1. 处理大型整数

对于有 64 位寄存器大小的平台来说，处理大型整数应该不是问题。对于这些平台，两个 64 位值的乘法将在芯片中进行，生成一个保存在两个寄存器中的 128 位值。128 位寄存器值对的加法、减法以及比较都不是问题。

对于具有 32 位寄存器大小的平台来说，64 位整数的相乘就需要稍微大一点的开销了。但是如果使用汇编语言来进行合适处理的话，它的代价也不会太大。一定小小心处理你的编译器中提供的标准 64 位处理的扩展，因为它们生成的代码可能会比你想象的运行得要慢一点。如果 64 位整数的相乘代价太大了，那么替代方法就是减小其范围，这样，只需要 32 位的乘法就可以了。

2. 应该使用何种遍历?

在实现遍历的时候，我们可以选择方程 2.2.6 或是方程 2.2.9 中的任一个。让我们来考虑一下这些方程在实现的时候所需的因素。

对于方程 2.2.6 来说，我们可以预先计算出 $\mathbf{B}d_p$，将此结果与我们的分数表达式保存在一起。现在，检查不等式就只需要 4 个乘法了。然而，我们的中间结果却需要进行一个点积与点坐标的乘法才能获得。因此对于方程 2.2.11 中给出的 r 来说，其中的两个乘法需要对 128 位的值进行。比较理想的情况是，我们避免这么大的数字的相乘，因为它们将带来比较大的

Download at www.Pin5i.Com

性能上的损失。

一个解决方法就是减小允许使用的范围，这样针对方程 2.2.6 两个最后的乘法就可以使用 64 位值来进行了。这是最快的方法。另一个办法就是使用方程 2.2.9，不过这需要进行 6 次针对 64 位值的乘法。

3. 优化

上面得出的范围是理论上的极限，在实际情况中它们几乎是不会达到的。如果想冒冒险，你可以使用比理论值更少的位来存储中间结果，而且有可能永远不会溢出。另一个方法就是检查是否发生了溢出，当溢出发生的时候就使用不同的代码处理。在目标处理器上的分支预测可以帮助改善在这种情况下的性能。在运行期的大部分时间里面是不会发生溢出的，因此我们必须设计好自己的代码，以便预测算法能正确地处理分支情况。

基于动态整数类的分数矢量会导致一些很大的整数，使程序运行很慢。在这种情况下，我们可能需要减小牵涉到的数值。我们可以找出分数中分母与分子的最大公约数以达到此目的。然后，我们就可以让分母与分子同时除以这个数值了。

一个更简单的优化方法是检查一下看看一个相交点是否精确地落在了一个网格节点上，若是，则可以像对普通的点一样来对其进行处理。但是只有当节省的性能能抵消检查的代价的时候才是值得的。

2.2.6 结论

分数矢量的数学基础并不复杂，但是它们提供了一个巧妙的方法可以在不引进近似计算误差的情况下处理相交的点。这样在实现几何图形算法的时候，如果把近似造成的错误消除的话，则可以减轻不少负担了。

在使用分数矢量的时候很重要的一点就是要避免产生溢出。如果我们在不同的计算阶段使用了不同的数据类型，编译器就可以帮助我们进行处理。如果我们已知自己需要进行什么几何图形查询，我们可以将分数矢量的概念抽象成一个类或是一组函数了。

这种改变的代价可以很小，小到大概只需要多花费一点乘法以及多一点额外需要操作的位而已。加入了这些额外的计算以后，你就再也不需要对一个近似的结果进行特殊情况的处理了。

2.2.7 参考文献

[GMP02] GMP, "GMP", 对应网址为 http://www.swox.com/gmp/, January 2002.

[Haible02] Haible, Bruno, "CLN – Class Library for Numbers," 对应网址为 http://www.ginac.de/CLN/, January 2002.

[Young01] Young, Thomas, "Optimizing Points-of-Visibility Pathfinding," *Game Programming Gems 2*, Charles River Media, Inc., 2001.

Download at www.Pin5i.Com

2.3　三角函数的更多近似计算方法

Robin Green
Sony Computer Entertainment America
robin_green@playstation.sony.com

编写数学函数库的艺术与科学近 10 年来一直没什么变化，很多写自 20 世纪 70 年代和 80 年代的计算机科学的参考书现在还在普遍使用。而且由于数学是一门通用的学问，因此一般都认为这些书是关于此课题的最后定论了。与此同时，硬件却在发展，指令流水线的级数增加了，内存访问的速度越来越快了，乘法器和求方根单元比以前要便宜得多，而更专业化的硬件则在使用单精度浮点数了。现在应该再从基础开始，对我们每日使用的数学函数的实现作一番复查。只需要一点训练，再加上一点思考，我们就可以针对自己特定的游戏相关问题进行优化了，有时候还会比通用硬件的实现效率更高呢。

2.3.1　衡量误差

在检查函数的实现之前，我们首先需要的就是一套工具，通过它们我们才能衡量出究竟这些实现有多精确。衡量误差显而易见的方法就是可以先得到一个高精度的相应函数的实现版本（一般都是一个缓慢而且麻烦的实现，涉及到无穷级数的计算），然后再用得到的数值减去我们的的近似值。此方法被称之为绝对误差测量：

$$error_{abs}=|f_{actual}-f_{approx}| \tag{2.3.1}$$

对精确度来说这是一个很好的衡量方法，但是它并不能指出一个误差的重要性。如果函数的返回值是 38000 的话，一个等于 3 的误差是可以接受的，但是如果返回值应该是 0.008 的时候，一个 3 的误差就太过分了。我们还需要得到相对误差：

$$erro_{rrel}=1-\frac{f_{approx}}{f_{actual}} \quad 当\ f_{actual}\neq 0 \tag{2.3.2}$$

当检查想对误差的时候，一个等于 0 的误差表示没有误差。此近似值是精确的。对于像 `sin()` 和 `cos()` 这样值域在[-1.0, 1.0]的函数来说，相对误差就没什么意思了。但是对于 `tan()` 这样有一个很大值域的函数来说，相对误差就是一个很重要的标志精确度的手段了。

Download at www.Pin5i.Com

2.3.2 正弦与余弦函数

在下面大多数例子中，我们主要考查的是正弦与余弦函数，但是其中很多多项式的技巧只需要做一点改动，对其他的函数也是适用的，例如指数函数、对数函数，还有反正切函数。

1. 振荡滤波器

如果有人问你什么是计算一个角度的正弦与余弦函数的最快方法，可以告诉他们你只需要使用两条指令就够了。这种方法被称为振荡滤波器法，它需要得到一个角度的上一次计算的结果，而且需要假定整个计算序列相邻的角度之间的差值为常数（参见图 2.3.1）。

```
int       N = 64;
float     PI = 3.14159265;
float     a = 2.0f*sin(PI*N);
float     c = 1.0f;
float     s = 0.0f;

for(i=0; i<N; ++i) {
   output_sine   = s;
   output_cosine = c;
   c = c - s*a;
   s = s + c*a;
   ...
}
```

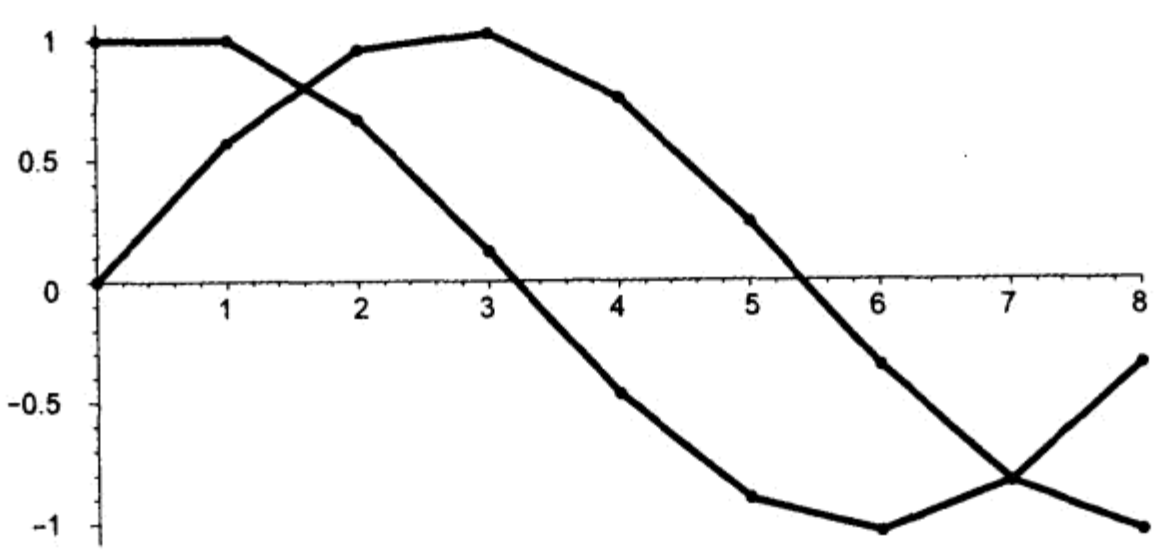


图 2.3.1 通过振荡滤波器生成的正弦波与余弦波—在 3π/2 上迭代了 8 次

请注意，用来计算 s 的 c 值是从上一行代码的计算中得到的新值。这个计算公式可以被转化为单精度的浮点运算，此时对 a 的乘法可以用一个移位运算来代替（例如，乘以 1/8 的运算可以被转化为一个右移 3 位的运算）。

如果你想将使用此方法得到的值填充到一个查找表中以便以后使用的话，那么一定要特别注意 a（步长）以及 c 和 s 的初始值。这种方法要求系统得到前面计算出的值作为其反馈，

Download at www.Pin5i.Com

因此 s 和 c 的初始值将影响波形最终的振幅（例如，如果开始时 s=0.5 而且 c=0.5，则其结果的峰值将是 0.7）。虽然这种方法可以快速地生成类似正弦函数波形的信号，但是你可不能指望通过一个循环里的计算值就可以得到一个精确的结果。例如，假设我们希望把一个圆周的 1/4 分成 7 份：

```
N = 7;
a = 0.5*sin(PI*N);
...
```

第 7 次迭代和第 8 次（由零开始算起）迭代的结果如表 2.3.1 所示。

表 2.3.1　　测试振荡滤波器的精确度

迭代次数	正弦函数	余弦函数
7	1.004717676	0.158172209
8	0.9917460469	-0.0597931103
……	……	……
27	1.000000000	-0.000000002

此函数的结束点与正确值 1.0 和 0.0 相差甚远（参见表 2.3.1 与图 2.3.2）。然而，如果我们对此表进行扩展使用 27 个计算值以生成整个循环的数值，我们将发现 s 和 c 的值可以精确到小数点后 9 位的精度。如果迭代的次数继续增加，那么误差将继续减少，不过仍然不能使之消除。显然，使用此近似方法可以生成一个长序列的类正弦函数的波形，特别是对整个循环来说。但是对小角度的精确运算来说它就不行了。

2. Goertzel 算法

对同一问题我们有另外一个更快的算法，此即 Goertzel 算法，它使用了两个上次运算的结果并将之更新（那就是说，它是一个二阶滤波器）。通过它，我们就可以计算出在序列 x_n=sin(a + n*b)中的一系列正弦与余弦函数的值，其中 n 是一个整数：

```
float cb = 2 * cos(b);
float s2 = sin(a + b);
float s1 = sin(a + 2*b);
float c2 = cos(a + b);
float c1 = cos(a + 2*b);
float s,c;

for(i=0; i<N; ++i) {
    s = cb * s1 - s2;
    c = cb * c1 - c2;
    s2 = s1;
    c2 = c1;
    s1 = s;
    c1 = c;
    output_sine   = s;
    output_cosine = c;
    ...
}
```

Download at www.Pin5i.Com

此方法在运行的时候只比上一个方法稍微多一点开销而已，但它初始化的开销大多了。不过，如果能在编译期把初始化工作完成的话，此算法还是很高效的（参见图 2.3.3）。

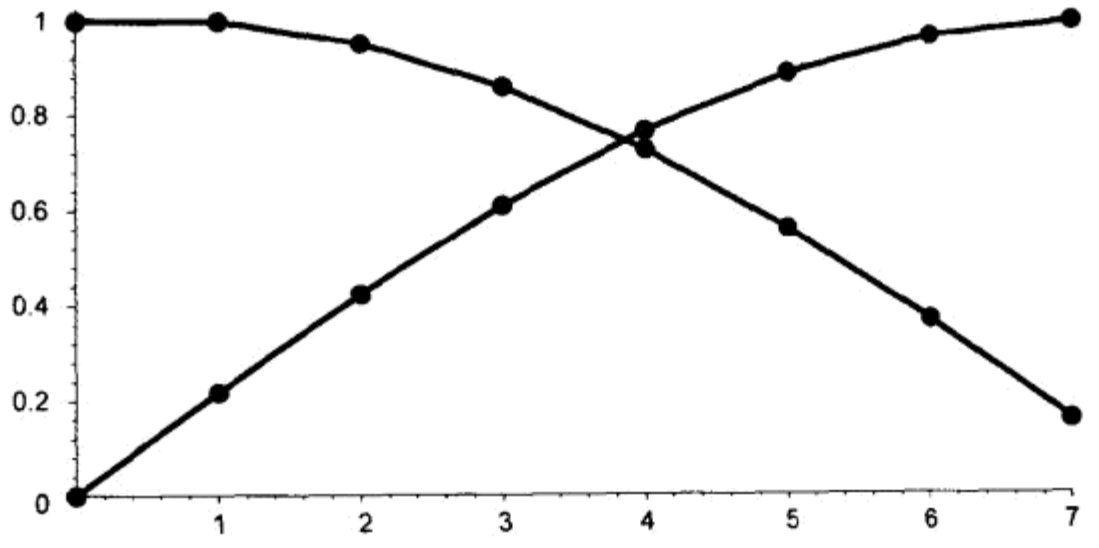


图 2.3.2 振荡滤波器的 1/4 圆周测试，在π/2 上迭代了 7 次

由于它是一个二阶算法，因此此算法中有些容易让人出错的地方。由于 s 和 c 的值是由上两步计算值得到的，所以此算法得到的结果实际上是比预期结果多了 3 个步长迭代之后的数值。为了在这一点上进行补偿，我们需要对此序列进行仔细的初始化，从 a 的初始值中减去 3 个步长：

```
// step = N steps over 2*PI radians
float b = 2.0f*PI/N;
// minus three steps from origin
float a = 0.0f - 3.0f * b;
...
```

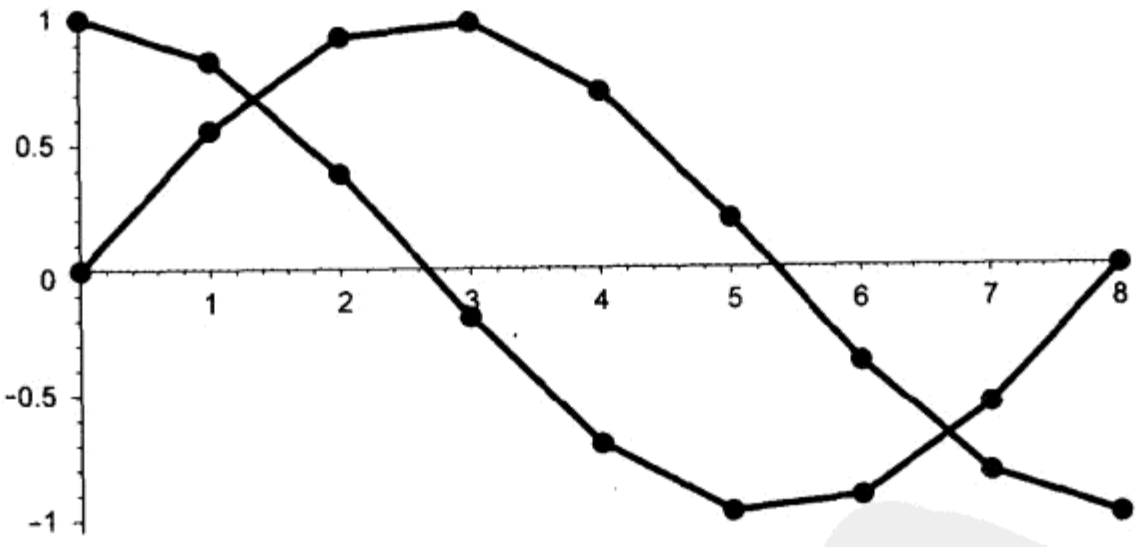


图 2.3.3 Goertzels 算法：在 3π/2 上迭代了 8 次的正弦波与余弦波

在增加了这些改变以后，再在四分之一圆周上作 Goertzel 算法的测试，我们将发现它可以成功地通过此项测试，其生成的结果要比振荡滤波器在整个循环上的结果还要精确得多。

Download at www.Pin5i.Com

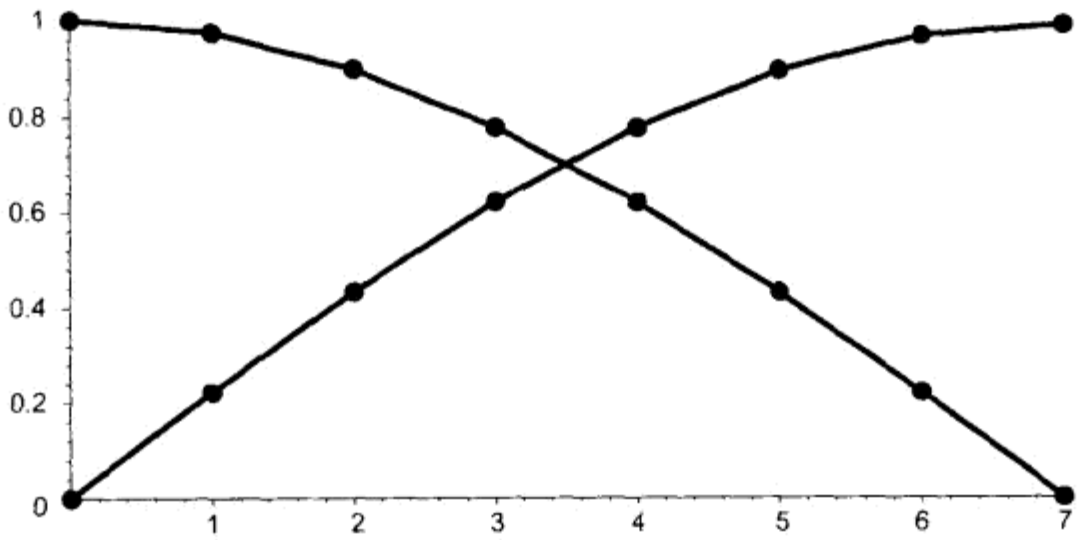


图 2.3.4　Goertzels 1/4 圆周测试，在π/2 上迭代了 7 次

3. 基于表驱动的方法

由于时钟频率越来越快，而内存访问的延迟越来越大，使用正弦与余弦函数表的方法已经不再流行，而且在某些环境里不再是最快的方法了。对于那些提供了矢量单元和紧耦合的快速 RAM 的新架构来说，它们还可以对比较小的表的访问提供一个时钟的常数访问时间，因此此项技术还不能被抛弃掉，对我们来说还可以使用一段时间。

此方法的原理就是要预先计算出一个函数每隔一定步长上点的数值，并将之放在表里面，然后有输入的时候就从表里查找出对应的函数值，得到与输入最近的两个值，然后对其进行线性插值（参见图 2.3.5）。实际上，我们是在以空间换时间。

为了加快两个采样点之间的线性插值（lerp）的速度，我们可以事先计算出两个相邻采样点之间差值，每查找一次就保存一个差值，特别是在 SIMD 的机器上面，在此机器上一次查找通常一次就可以载入一个有四个浮点数的矢量。

$$\begin{aligned}\sin(x) &\approx \text{table}[i] + \Delta * (\text{table}[i+1] - \text{table}[i]) \\ &\approx \text{table}[i] + \Delta * \text{gradient}[i]\end{aligned} \tag{2.3.3}$$

得到了这些差值以后，一个 lerp 运算就被转化为一个相乘然后相加的过程了。

使用表格带来了下面一个问题：我们需要采用多少个点才能得到小数点后 N 位数的精确度呢？基于表使用 16 个采样点的正弦函数波形如图 2.3.5 中所示，其绝对误差图如图 2.3.6 所示。

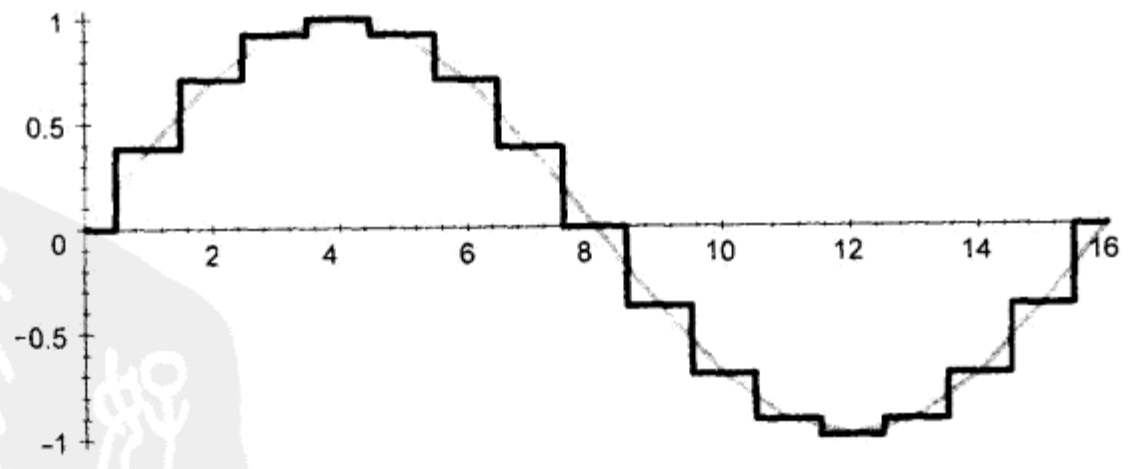


图 2.3.5　表驱动的正弦波（16 个采样点）有线性插值及没有时的情况

Download at www.Pin5i.com

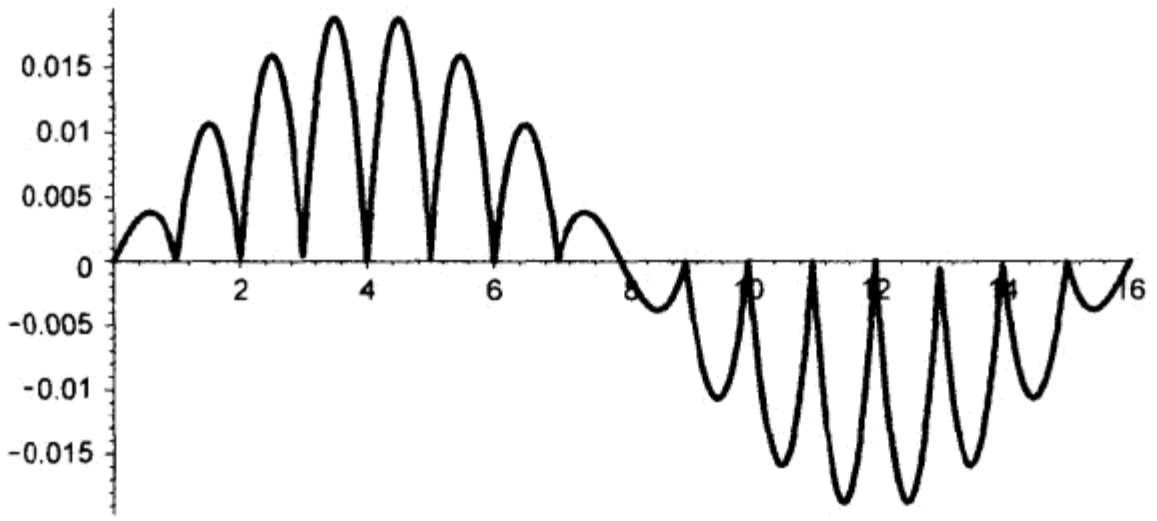


图 2.3.6 由 16 个采样构成，经线性插值得到的正弦表的绝对误差

最大的误差（或称极值误差）是在函数曲率的最大点处达到的——实际上是当两个采样点刚好跨越了曲线的顶点的时候。当步长值为 $\Delta x=x_{n+1}-x_n$ 的时候相应的最大误差值可用下面的公式计算出来：

$$E=1-\cos\left(\frac{\Delta x}{2}\right) \tag{2.3.4}$$

对于一个使用了在一个周期上 16 个采样点的表来说，最大的相对误差就是 `1-cos(π/16)=0.0192147`，在最坏的情况下将给我们带来小数点后两位的误差。把这个问题反过来问的话，如果已知了要求的精度，那我们需要多少个表项呢？我们把不等式反过来看就行了。例如，如果想要在 `sin(x)` 上得到 1%的误差，我们只需要采样 23 个点就行了：

$$\begin{aligned} E&=1\% \\ 1-\cos(\pi/N)&<1\% \\ \cos(\pi/N)&>0.99 \\ N&>\pi/\arccos(0.99) \\ &\approx 22.19 \end{aligned} \tag{2.3.5}$$

通过采用一个称之为范围缩减（range reduction）的过程，我们可以每 45 度角进行一次采样就可以得出整个周期内的序列值，这意味着我们只需要一个只有 23/8=3 个表项的表就可以了。方程 2.3.4 将给出此误差的上限值，而这个上限值是几乎绝不会达到的。对于一个稍微小一些的上限值，你可以对 `arccos()` 使用小角度上的近似，由于很有可能 π/N 为一个很小的角度，因此你可以得到下面的极限值：

$$N=\frac{\pi}{\sqrt{2E}} \tag{2.3.6}$$

利用方程 2.3.6 计入各种误差后，我们就可以得知什么时候可以使用表，而什么地方又必须要使用更高精度的方法了。参见表 2.3.2 中的例子。

表 2.3.2 给定精度下逼近 `sin(x)` 所需要的表的大小

	E	360 度范围	45 度范围
1%的精确度	0.01	23	3
0.1%的精确度	0.001	71	9
0.01%的精确度	0.0001	223	28

Download at www.Pin5i.Com

续表

	E	360 度范围	45 度范围
1.0 度	0.01745	17	3
0.1 度	0.001745	54	7
8 位整数	2^{-7}	26	4
16 位整数	2^{-15}	403	51
24 位浮点数	10^{-5}	703	88
32 位浮点数	10^{-7}	7025	880
64 位浮点数	10^{-17}	无穷大	8.7e＋8

4. 范围缩减与重构

正弦与余弦函数的定义域是无穷大。每个输入值在域[-1, 1]上都有一个对应的输出值，而且每隔 2π个周期波形就会重复一次。我们将输入值除以 2π，将此结果裁减得尽量与 0 接近（即要将之转化为一个整数），然后要将其减去整数倍的 2π。此过程，至少对于正弦函数与余弦函数而言，被称为加性范围缩减（additive range reduction），请参看图 2.3.7。

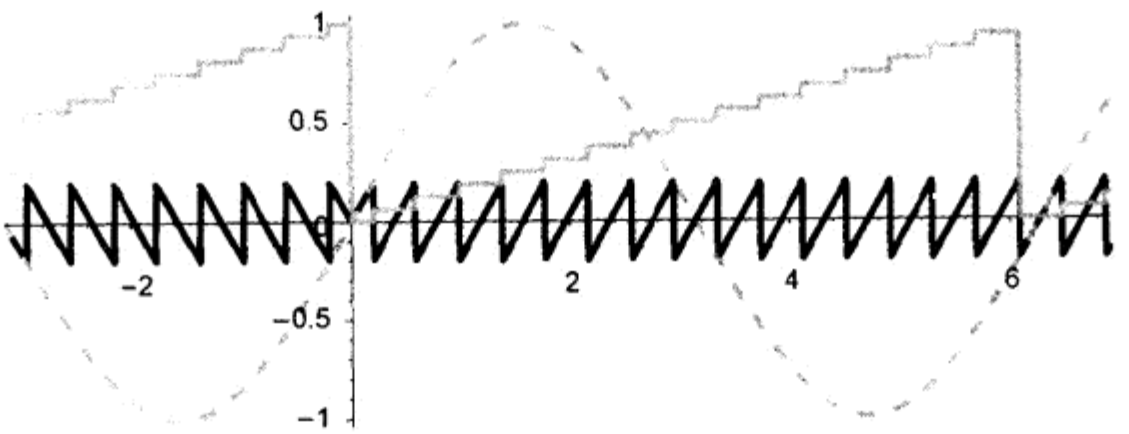


图 2.3.7　C=2π/16 时的加性范围缩减

```
const float C = 2*PI;
const float invC = 1/C;

int k = (int)(x*invC);
y = x - (float)k * C;
...
```

在上个例子中，k 表示的是我们需要从原数中减去的 2π的整数倍，然而，如果我们继续对此数利用一个周期的分数倍使用范围缩减的方法，那么通过 k 的小数部分我们就可以得知原数是属于哪一个象限里面的了。为什么要经历这么一个过程呢？其原因始于下面的三角函数公式：

$$\sin(A+B) = \sin(A)\cos(B) + \cos(A)\sin(B)$$
$$\cos(A+B) = \cos(A)\cos(B) + \sin(A)\sin(B) \qquad (2.3.7)$$

假设我们最终得出 y∈ [0..π/2]的结论，那么这就意味着我们的周期可以被分为四段，而 k mod 4 就可以得出究竟此数是属于哪一段的了。如果将方程 2.3.7 的左右分别相乘，我们将

Download at www.Pin5i.Com

发现 sin(B) 和 cos(B) 将合并为常量 0 或者 1 了，然后我们将得到四种特殊的情况：

$$\sin(y+0*\pi/2) = \sin(y)$$
$$\sin(y+1*\pi/2) = \cos(y)$$
$$\sin(y+2*\pi/2) = -\cos(y)$$
$$\sin(y+3*\pi/2) = -\sin(y) \qquad (2.3.8)$$

如此代码就将相应变为：

```
float table_sin(float x) {
    const float CONVERT = (2.0f * TABLE_SIZE) / PI;
    const float PI_OVER_TWO = PI/2.0f;
    const float TWO_OVER_PI = 2.0f/PI;

    int k = int(x * TWO_OVER_PI);
    float y = x - float(k)*PI_OVER_TWO;
    float index = y * CONVERT;
    switch(k&3) {
        case 0:  return sin_table(index);
        case 1:  return sin_table(TABLE_SIZE-index);
        case 2:  return -sin_table(TABLE_SIZE-index);
        default: return -sin_table(index);
    }
    return(0);
}
```

为什么只计算最初四个象限的值呢？如果想要得到更多象限的值，我们需要对最终的结果进行重构，方法就是要更仔细地利用方程 2.3.7，然后使用内联常量或是一个表来计算：

```
...
s = sin_table(y);
c = cos_table(y);
switch(k&15) {
    case 0:   return s;
    case 1:   return s * 0.923880f + c * 0.382685f;
    case 2:   return s * 0.707105f + c * 0.707105f;
    case 3:   return s * 0.382685f + c * 0.923880f;
    case 4:   return c;
    case 5:   return s * -0.382685f + c * 0.923880f;
    ...
}
```

值得注意的是，为了得到正弦的结果，我们还需要同时逼近正弦和余弦函数。只需要再多做一点点工作，我们同样也能同时重构出余弦函数的结果。而一个能同时返回一个输入角的正弦和余弦函数结果的函数，在传统 FORTRAN 语言的数学函数库里面就有一个，通常被称之为 sincos()。

你将会发现大多数库在设计其数学函数库的时候都会使用到范围缩减、逼近，以及重构的方法，这种编程的模式会反复重现。在下一个小节里面，我们将生成一个优化过的多项式级数以代替这里使用的表查找与 lerp 的方法。

Download at www.Pin5i.Com

2.3.3　多项式逼近

在初次学习逼近函数的时候，我们一般是从高中的泰勒（Talor）级数开始的。通过使用一系列的微分，我们可以把超越函数分解为一个无穷级数和的形式——例如：

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \cdots \tag{2.3.9}$$

如果我们拥有无穷的时间和无限的存储量，那么这个就是我们需要的终极解答了。但是由于我们的时间有限，而且内存很少，现在就先把此级数算到 9 次幂，保留五位有效数字再说吧：

$$\begin{aligned}\sin(x) &\approx x - \frac{1}{6}x^3 + \frac{1}{120}x^5 - \frac{1}{5040}x^7 + \frac{1}{362880}x^9 \\ &= x - 0.16667x + 0.0083333x^5 - 0.00019841x^7 + 0.0000027557x^9\end{aligned} \tag{2.3.10}$$

这是一个经典的无穷级数——它的各项正负相反而且系数会急剧减小（1/x!将极快地逼近 0），这两个特征都表明此级数将很快逼近其真实值。这里的问题是出在近似产生的误差上的。如果你把此函数的绝对误差给画下来（参见图 2.3.8），你将发现它在临近泰勒展开点的小角度的条件下是极其精确的，但是随着 x 离 0 越来越远，其误差几乎是在指数增长。如果使用更多的级数项，此误差将会减小，但是它的代价也会增大，将使你冒更大的数字计算误差的风险，而且每加一项就意味着又要在程序里面添加一个载入/相乘-相加的操作序列。我们需要在整个域里都有极好的精确度，而且需要使用尽量少的项数得到此结果。

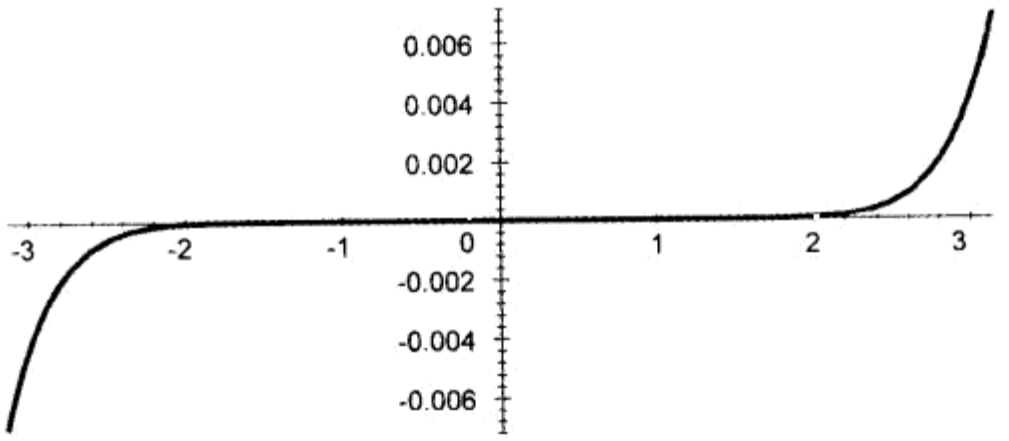


图 2.3.8　在[–π，π]定义域上泰勒级数的绝对误差

是否可以考虑减小输入范围呢？如果你减小了需要逼近的正弦函数的定义域，那么当然了，我们也减小了误差，因为误差的来源减少了！除了可以减小输入的范围以外，我们还可以在想要逼近的范围的中心点上进行泰勒展开。这将把总的误差减小为原来的一半，但是却需要原来双倍数量的系数——现在我们就需要从 0 到 9 计算每次幂了，而不是原来的每两个才计算一次幂。实际上我们可以使用比泰勒级数更快的方法来进行多项式的逼近。

1. 最大值最小化多项式

泰勒展开的最大误差太大了。如果我们能够找到一种方法把此误差的一部分给平均到整

Download at www.Pin5i.Com

个域上的每个数该有多好啊。事实上，依据切比雪夫发现的一个定理，我们现在可以给出每种逼近的一个特定多项式，此多项式将在每处都有同样大小的误差——就是说我们把最大误差给最小化了，它就被称为最大值最小化多项式。它拥有以下特性：

- 对于一个 N 次多项式的逼近，其误差曲线将改变 N+1 次符号。
- 误差曲线将 N+2 次达到最大误差。

计算出这些多项式逼近的方法被称为是 Remez 交换算法，它是通过生成一系列的线性方程来达到此目的的。例如：

$$\sin(x) - a + bx_n + cx_n^2 = 0, \quad x_n \in [a..b] \tag{2.3.11}$$

通过解出这些方程，我们就可以得到所需的系数 a、b 和 c 了，最大误差是被保存在 x_n 里面返回的。这种高度技巧性的优化问题对于浮点运算的精确度特别敏感，而且较难编程实现，因此我们会找一些专业人士来帮忙解决。数值数学包，例如 Mathematica 和 Maple，都有此种必需的环境，它们都能表现巨型数字，而且还有那些必需的数值运算工具以得到精确的结果。

计算一个最大值最小化多项式需要参数如下所示：

- 需要逼近的函数。
- 逼近所在的函数定义域。
- 逼近的幂次数。
- 一个权重函数用来对逼近进行调整以将绝对误差（此时权重为 1）或是相对误差最小化。

下面让我们来找出一个在[0，π/4]定义域上对 sin(x)逼近的 7 次多项式。首先，我们来看看在 x=0 点处 sin(x)的泰勒级数展开是什么样的。主要是为了看看这个多项式大概会是什么样的。其结果指出此多项式的第一个系数为 1，而且只有奇数次幂：

$$\sin(x) \approx x - 0.166666667x^3 + 0.00833333333x^5 - 0.000198412698x^7 \tag{2.3.12}$$

缺省情况下得到的原始的最大值最小化多项式将使用所有可用次幂的所有系数以最小化误差，这将导致产生一些非常小的，看起来很怪的系数，还将产生一些不必要的项。我们将此问题转化为一个在下面的表达式中找出多项式的问题：

$$\sin(x) \approx x + x^3 P(x^2) \tag{2.3.13}$$

首先，我们给出最大值最小化的不等式，以表示我们希望在多项式 P 中对相对误差进行最小化：

$$\left| \frac{\sin(x) - x - x^3 P(x^2)}{\sin(x)} \right| \leqslant error \tag{2.3.14}$$

上下都除以 x^3 可以得到：

$$\left| \frac{\frac{\sin(x)}{x^3} - \frac{1}{x^2} - P(x^2)}{\frac{\sin(x)}{x^3}} \right| \leqslant error \tag{2.3.15}$$

我们还希望结果中只有奇数次幂，因此，我们用 $y=x^2$ 来代入此式：

Download at www.Pin5i.Com

$$\left|\frac{\frac{\sin\left(\sqrt{y}\right)}{y^{3/2}}-\frac{1}{y}-P(y)}{\frac{\sin\left(\sqrt{y}\right)}{y^{3/2}}}\right| \leqslant error \qquad (2.3.16)$$

这样一来，我们就把问题变成了寻找对下面的函数进行最大值最小化多项式逼近的问题：

$$P(y)=\frac{\sin\left(\sqrt{y}\right)}{y^{3/2}}-\frac{1}{y} \qquad (2.3.17)$$

其中还有一个权重函数：

$$W(y)=\frac{y^{3/2}}{\sin(\sqrt{y})} \qquad (2.3.18)$$

为了能在具有任意精度的 Mathematica 或是 Maple 的环境中对此函数得到一个正确的评估，我们就需要（有点讽刺性）把第一个表达式展开为泰勒级数，并且要使其级数足够多，至少要超过我们要求的精度范围，这样才能防止使用特别编写的、任意精度的正弦函数来对之进行评估：

$$P(y)=-\frac{1}{6}+\frac{y}{120}-\frac{y^2}{5040}+\frac{y^3}{362880}-\frac{y^4}{39916800}+\frac{y^5}{6227020800}-\cdots \qquad (2.3.19)$$

我们最后一个任务就是要对逼近的定义域进行转化。由于我们使用了 $y=x^2$，因此我们的定义域[0，π/4]就将转化为[0，π^2/16]了。使用最大值最小化函数在其上运行后，我们可以得到如下的二阶的结果：

$$P(y)=-0.166666546+0.00833216076y-0.000195152832y^2 \qquad (2.3.20)$$

将此结果再次代入方程 2.3.12 以后，我们将得到以下的最终结果：

$$\sin(x)\approx x-0.166666546x^3+0.00833216076x^5-0.000195152832x^7 \qquad (2.3.21)$$

为了把这些系数重构为单精度浮点数，我们只需要记录开头的 9 个有效数字就行了（参看下面的证明），这样我们可以得出图 2.3.9，其上画出了绝对误差与相对误差曲线，其中绝对误差的最大值在 x=0.785 的地方达到了 2.59e-9。

下面是一个不严格的证明。在单精度数中，[10, 2] = [1000, 1024]范围中的数在小数点左边有 10 位，在小数点右边有 14 位。因此，此范围一共有$(2^{10}-10^3)2^{14}$=393 216 个可用其表达的数值。如果我们用的是 8 位的 10 进制数，那么我们一共可以表达$(2^{10}-10^3)10^8$=240 000 个数值，因而我们需要 9 位 10 进制数字才能重构出正确的 2 进制数来。在数轴上的相类似的构造则显示其需要 6 到 9 位数字。欲知详情，请参见[Goldberg91]。

2. 对浮点数进行优化

我们可以使用在上面的消去系数的方法来强制数字转化为可用机器表达的形式。一定要记住，像 1/10 这样的数字使用有限个 2 进制数字是无法精确表达的。我们可以使用上述强制性得到系数的方法应用到将数字转化为可用机器表达的浮点数这方面来。请注意所有的浮点数都是有理数，我们可以将第二个系数强制转化为一个单精度的浮点数：

Download at www.Pin5i.Com

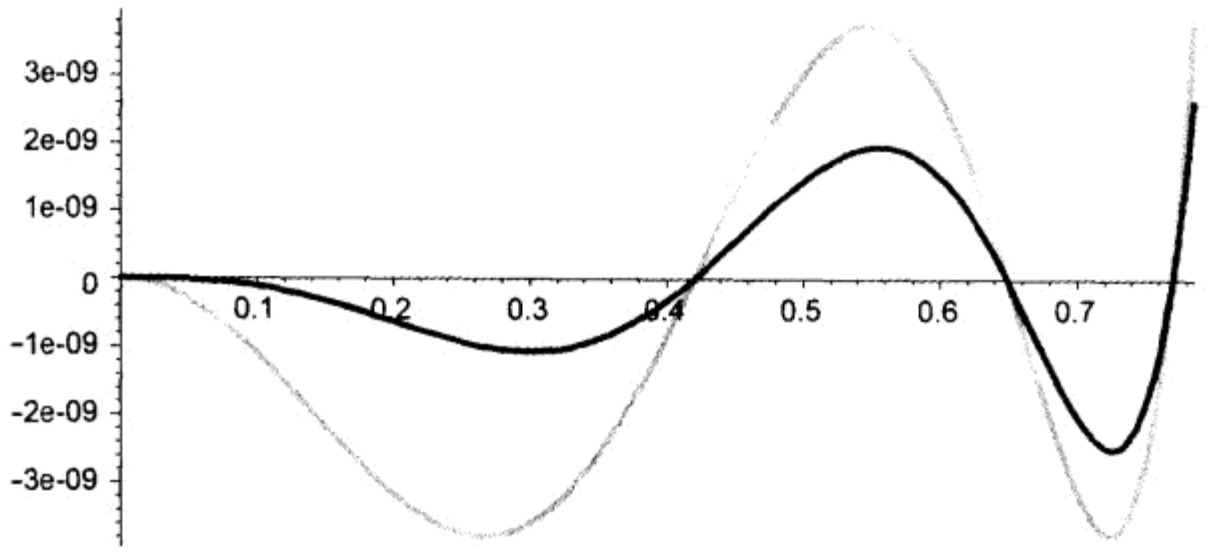


图 2.3.9 [0，π/4]域上的逼近的绝对误差与相对误差

$$k = -\frac{2796201}{2^{24}} = -0.166666567325592041015625000 \tag{2.3.22}$$

现在我们有自己的常量了，下面就来对多项式进行优化以将之合并进去。首先，我们来定义一个我们想要得到的多项式的最终形式：

$$\sin(x) = x + kx^3 + x^5 P(x^2) \tag{2.3.23}$$

下面我们给出最大值最小化的不等式来：

$$\left|\frac{\sin(x) - x - kx^3 - k^5 P(x^2)}{\sin(x)}\right| \leqslant error \tag{2.3.24}$$

首先我们将此式除以 k^5，继而把 $y=x^2$ 代入进去，然后为了 P(y)将之解出来，最后我们发现还必须要计算以下式子的最大最小值：

$$P(y) = \frac{\sin\left(\sqrt{y}\right)}{y^{5/2}} - \frac{1}{y^2} + \frac{k}{y} \tag{2.3.25}$$

还有一个权重函数是：

$$W(y) = \frac{y^{5/2}}{\sin(\sqrt{y})} \tag{2.3.26}$$

将此方程解出来进行变量代入以后，我们将得到一个在[0, π/4]域上的 7 次幂优化后的多项式，其最大误差为 3.39e-8，此误差发生在 x=0.557 的地方，但是我们的结果具有更好的单精度浮点数上的精确度。

$$\sin(x) \approx x - \frac{3796201}{2^{24}}x^3 + 0.00833220803x^5 - 0.000195168955x^7 \tag{2.3.27}$$

在[0，π/4]域上，针对浮点数优化后的逼近的绝对误差与相对误差如图 2.3.10 所示。

Download at www.Pin5i.Com

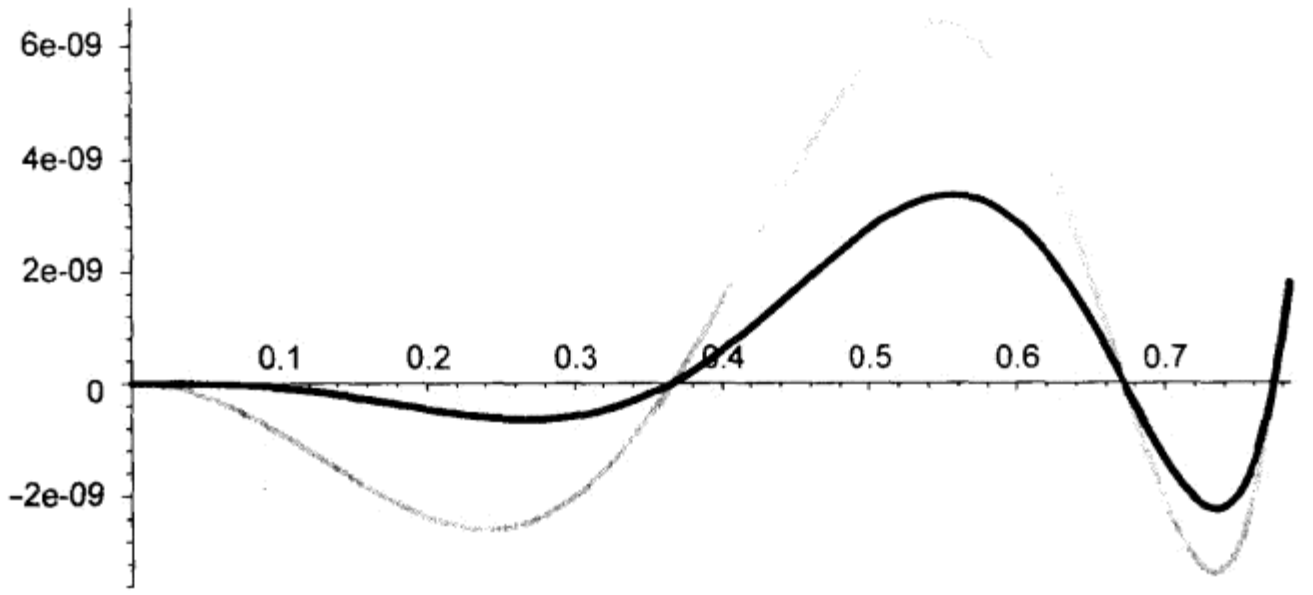


图 2.3.10　在[0，π/4]域上经浮点优化后逼近的绝对误差与相对误差

2.3.4　有关收敛性的注意事项

我们究竟应该使用多少项多项式级数来逼近函数以达到给定的精确度要求呢？我们能够很容易就计算出一个逼近能精确到多少位。首先，我们计算出给定域上的最大误差（这个数值应该很小，典型的值例如 5e-4），然后使用 ln(error)/ln(2)可以得到此值的 2 基对数，它会返回一个负数，表示的是我们需要使用多少位才能表示出此数值的小数点后的数来。利用这种方法我们可以得出一个列表，其中包含了几个重要函数的对应值，通过此表我们将发现一些有趣的结果（参见表 2.3.3）。

表 2.3.3　　在[0, 1]域上精确度的有效位数与最大值最小化多项式的次数

	2	3	4	5	6	7	8
e^x	6.8	10.8	15.1	19.8	24.6	29.6	34.7
sin(x)	7.8	12.7	16.1	21.6	25.5	31.3	35.7
ln(1 + x)	8.2	11.1	14.0	16.8	19.6	22.3	25.0
arctan(x)	8.7	9.8	13.2	15.1	17.2	21.2	22.3
tan(x)	4.8	6.9	8.9	10.9	12.9	14.9	16.9
arcsin(x)	3.4	4.0	4.4	4.7	4.9	5.1	5.3
$\sqrt{x}$	3.9	4.4	4.8	5.2	5.4	5.6	5.8

首先，从表中我们可以看出我们使用多项式来逼近 exp(x)和 sin(x)是相当有效的，因为每增加一个次幂我们就可以增加 4 位的精确度。对一个 24 位的单精度浮点数来说，我们只需要 6 次或是 7 次多项式就可以了。通过这个表还可以看出使用多项式来逼近 $\sqrt{x}$ 实在是太糟了。每增加一个次幂只能增加半位的精确度——这就是为什么求平方根没有简单而快速的算法的原因。我们必须使用牛顿算法进行范围缩减，还需要给出一个精确的初始值估计才能对之进行计算。另一个令人吃惊的是 tan(x)函数。不管怎样，它只不过是 sin(x)/cos(x)，难道不是么？像这样的有理函数使用一般的多项式逼近是不行的，它需要其他不同的工具包来对付。

Download at www.Pin5i.Com

2.3.5 结论

在本文献中我们并没有涉及到使用高精度函数来编写低精度数学函数的任务。但是随着可编程 DSP、矢量单元、高速硬件（不过也是有限的），还有更高深的投影模型的越来越广泛的使用，对编写自己的数学函数的需求也随之越来越重要了。

在开始介绍多项式逼近的时候一般都会谈到它们的精确程度将有多高，这种针对精确度的困惑贯穿于整个过程中，直到我们将每个浮点数的最后一位都给分析透了为止。为什么要怀疑它的精确度呢？因为你可以通过它来使用更少的系数以建构高精度的多项式，同样，你也可以使用这种技术来建构微型的、低精度的逼近多项式。仅仅通过两个常量和范围缩减，你可得到的精确度将是惊人的。希望本节能给予你信心，让你利用数学包来创建一些你自己的高速函数。

2.3.6 参考文献

[Cody80] Cody & Waite, *Software Manual for the Elementary Functions*, Prentice Hall, 1980.

[Crenshaw00] Crenshaw, Jack W., *Math Toolkit for Real-Time Programming*, CMP Books, 2000.

[DSP] The Music DSP Source Code. 在网址 http://www.smart-electronix.com 上有在线资料。

[Goldberg91] Goldberg, Steve, "What Every Computer Scientist Should Know About Floating Point Arithmetic," *ACM Computing Surveys*, Vol. 23, No. 1, March 1991.

[Hart68] Hart, J.F., *Computer Approximations*, John Wiley & Sons, 1968.

[Moshier89] Moshier, Stephen L., *Methods and Programs for Mathematical Functions*, Prentice Hall, 1989.

[Muller97] Muller, J.M., *Elementary Functions: Algorithms and Implementations*, Birkhaüser, 1997.

[Ng92] Ng, K. C., "Argument Reduction for Huge Arguments: Good to the Last Bit," SunPro Report, July 1992.

[Story00] Story, S. and Tang, P.T.P, "New Algorithms for Improved Transcendental Functions on IA-64," Intel Report, 2000.

[Tang89] Tang, Ping Tak Peter, "Table Driven Implementation of the Exponential Function in IEEE Floating Point Arithmetic," ACM Transactions on Mathematical Software, Vol. 15, No. 2 June 1989.

[Tang90] Tang, Ping Tak Peter, "Table Driven Implementation of the Logarithm Function in IEEE Floating Point Arithmetic," ACM Transactions on Mathematical Software, Vol 16, No. 2 December 1990.

[Tang91] Tang, Ping Tak Peter, "Table Lookup Algorithms for Elementary Functions and Their Error Analysis," Proceedings of 10th Symposium on Computer Arithmetic, 1991.

[Upstill90] Upstill, S., *The Renderman Companion*, Addison Wesley, 1990.

Download at www.Pin5i.Com

2.4　四元数的压缩

Mark Zarb-Adami
Muckyfoot Productions
mark@muckyfoot.com

当今的很多游戏都使用了大量的动画数据，而对每个动画帧使用的大量内存被骨节的旋转占用了。一般说来，旋转数据是使用四元数来存储的。在本节中，我们将给出并比较几个将四个浮点四元数压缩到一个 32 位的四元数中的方法。

2.4.1　四元数

四元数 $Q(x, y, z, w)$可以用来表示一个旋转矩阵。如果我们将所有的旋转矩阵都看成是一个代表了围绕轴 $A(x, y, z)$旋转 θ 角的转动，那么对应此旋转的四元数就是：

$$Q = (sX, sY, sZ, c)$$

其中：

$$s = \sin(\theta)$$

$$c = \cos(\theta)$$

很重要的一点就是四元数 $Q(x, y, z, w)$以及四元数 $Q'(-x, -y, -z, -w)$代表的是同样的旋转，这是因为围绕轴 A 进行的 θ 角的旋转等价于一个围绕 -A 轴进行的- θ 角的旋转。此外，还请注意本节中我们讨论的所有四元数都是归一化的，这就是说对于每一个四元数 $Q(x, y, z, w)$都有：

$$x^2 + y^2 + z^2 + w^2 = 1$$

2.4.2　三个最小数方法

我们可以将一个四元数的每个元素都量化为一个字节对之进行压缩。这是很快的方法，但是它也是不精确的方法。由于四元数是归一化的，因此我们可以改进四元数的精确度，把其中一个元素给删除掉。只要有了三个元素，我们就可以计算出第四个了。现在，我们就必须决定要把哪一个元素给去掉了。如果我们消去了最大的一个元素，那么我们就只需要存储三个较小的数字了！实际上，一个四元数中最小的三个元素其绝对值都肯定不会大于 $1/\sqrt{2}$。想知道这是为什么吗？让我们来看看下文的推理。一个归一化四元数的第二大元素只有当四元素中的两个元素有最大值，而且

Download at www.Pin5i.Com

另外两个元素为零的时候才会有最大值，其形式将为 $Q(u, v, 0, 0)$。如果我们将此四元数归一化，则 v 就必然等于 $1/\sqrt{2}$。

我们不需要保留最大元素的符号，因为我们可以确保它总是一个正数。如果它不是正数，只要如前所述地将其取相反数就行了。

2.4.3 极点方法

由于我们可以使用两个角度，*yaw* 与 *pitch*，来保存一个方向矢量(*x*, *y*, *z*)，因此我们也可以将一个四元数保存为(*yaw*, *pitch*, *w*)。请注意，使用 *yaw* 和 *pitch* 保存(*x*, *y*, *z*)将失去它的长度信息，但是我们可以通过 *w* 来恢复出正确的长度，因为四元数是归一化的。如果我们能确保 *w* 总是正数，那么我们就不需要保存其符号了。再一次，如果 *w* 是负数，我们只要取四元数的相反数就行了。

使用 *yaw* 和 *pitch* 来保存一个方向矢量的问题就在于编码后的矢量不是均匀分布在球面上，而是集中在两极的。当矢量的 *pitch* 为π/2，而且矢量直指极点的时候，我们不希望保存 *yaw*。然而，当矢量的坡度为零，矢量处于赤道位置的时候，我们希望能存储更多的 *yaw* 值。假设我们希望使用 *n* 个位对方向矢量进行编码，一个方法就是把点均匀分布在整个球面上，在一个查找表中存储每一点的(*x*, *y*, *z*)或是(*yaw*, *pitch*)，然后在表中放入一个 *n* 位长的索引。虽然对于比较小的 *n* 来说这个方法还可以，但是查找表的尺寸却会很快增长到过大的地步。

不过，我们不使用查找表也可以有一个有效的方法对方向矢量进行编码。首先，我们分别存储 *x*, *y* 以及 *z* 的符号，然后可以假设 *x*, *y* 以及 *z* 都是正数。这样我们就把问题局限在球面的 1/8 上了。下一步就是找出在这个 1/8 球面上的点数，如图 2.4.1 所示。

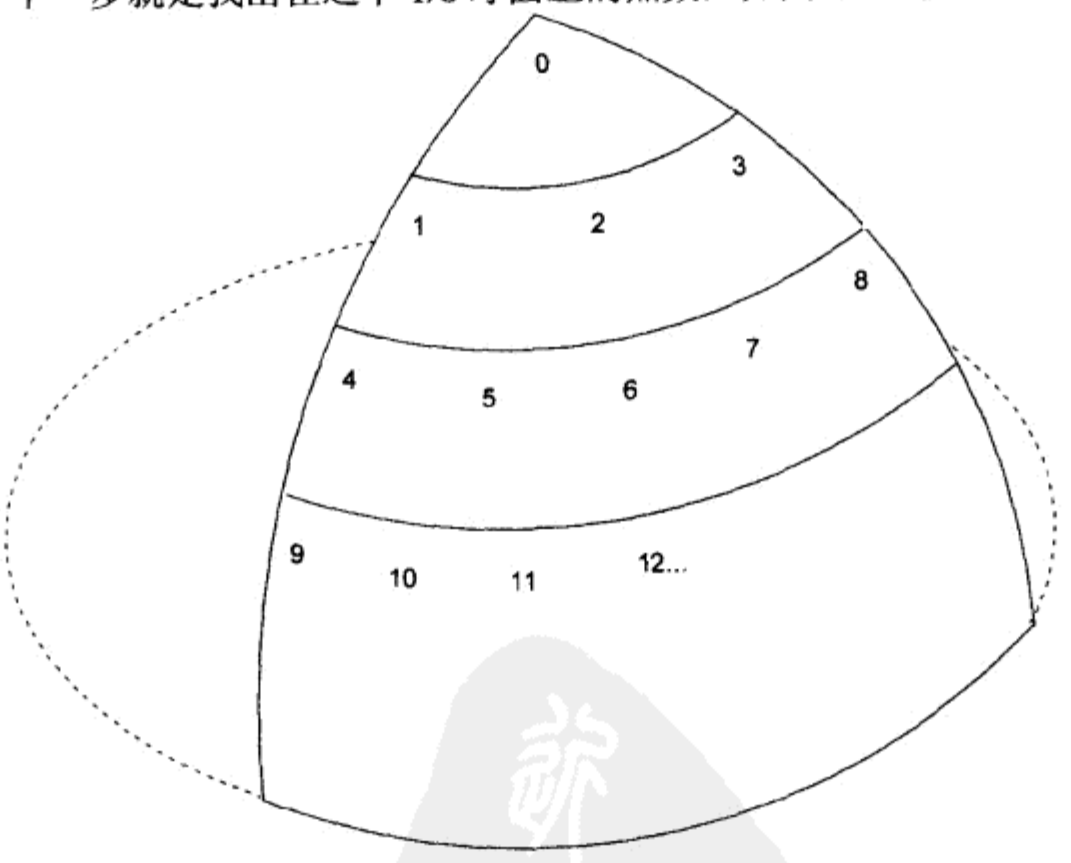


图 2.4.1 对 1/8 的球面编号

请注意在途中出现在右手边处的平方数，它可以帮我们找到一个编码数字 *e* 的行号和列号，如下所示：

Download at www.Pin5i.Com

```
row = floor(sqrt(e));
colum = e - row * row;
```

现在，如果我们让 *pitch* 成为 *e* 的行号，而且让 *yaw* 成为 *e* 的列号，我们就可以有效地存储和读出这些值了。请注意我们是如何做到在极点不浪费任何 *yaw*，而在赤道处则有足够多的 *yaw* 的。

2.4.4　实现

接下来，让我们探讨一下已经讨论过的方法应该如何实现。

1. 三个最小数

三个最小数的方法可以很巧妙地将一个四元数存储到 32 位中。我们需要两个位来保存可推算出来的（最大的）元素所在的位置，剩下的给最小的三个元素每个分配 10 个位。由于我们已知保存的每个元素都是在$[-\sqrt{2}, \sqrt{2}]$范围之内，因此我们可以将这些值插入[0, 1023]中，这样它们就可以用正数来表示了。

2. 极点

试验了几种位分配的方法以后，我们发现当给 *w* 分配 11 个位的时候压缩比最大。我们需要额外的 3 个位保存 *x*、*y* 和 *z* 的符号，剩下 18 个位保存 *yaw* 和 *pitch*。如果我们使用一个数对 *yaw* 和 *pitch* 进行编码，那么我们就可以在 512 行中保存 2^{18} 个数值。或者，我们可以为 *yaw* 分配 9 位，为 *pitch* 分配 9 位。我们知道它们都是处在[0, π/2]范围里面的，因此我们可以将这些值插入[0, 511]中去，将之保存为整数。

把 *yaw* 和 *pitch* 转化为一个矢量需要对 *yaw* 和 *pitch* 进行正弦和余弦计算，因此我们推荐使用一个快速的多项式逼近[Edwards00]以计算出这些值来。另外，由于 *yaw* 和 *pitch* 总是处在[0, π/2]范围里面的，我们可以针对这一特定范围进行更好的逼近。我们可以在 π/4 的地方进行泰勒级数的展开（而不是 0），选取在[0, π/4]范围里面的值作为拉格朗日级数的采样点。只要使用一个到 5 次方的泰勒级数，或是使用一个到 4 次方或 5 次方的拉格朗日级数就足够了。4 次方的拉格朗日级数会稍微快一点，但是你会丧失一点精确度。此外，5 次方的拉格朗日级数比 4 次方的泰勒级数的精确度要高得多。

我们的游戏 *Blade II* 里面使用的动画数据是进行了层次化存储的，这样一来，大部分四元数就只有一个很小的旋转角度 θ。由于 *w* 就是 $\cos(\theta/2)$，因此对此数据而言，*w* 通常与 1 很接近。请注意，即使四元数的角度是随机分布的，这个近似也是成立的，不过其适用范围会更小一点，这是由 cosine 函数本身决定的。因此，为了对接近于 1 的数值得到更高的精确度，我们将存储 $\sqrt{1-w}$ 而不是 *w*，然后在解压的时候把 *w* 再恢复回来。

2.4.5　性能

为了对每个压缩方法的性能进行量化，我们使用了 *Blade II* 里面的四元数作为测试数据。

Download at www.Pin5i.Com

我们将每个四元数 Q 进行压缩与解压，这样就得到了另一个四元数 Q'。我们将 Q 与 Q'转化为矩阵，使用这些得到的矩阵对很多点进行了旋转。接下来，对每个点，我们再计算出使用 Q 与 Q'得出的点之间的距离。这些距离就将作为压缩方法的误差了。此外，我们还测量了每个方法的压缩速度。图 2.4.2 展示了每个压缩方法针对我们层次化数据的相对性能。极点方法使用的近似方法是一个 5 次方的拉格朗日级数。

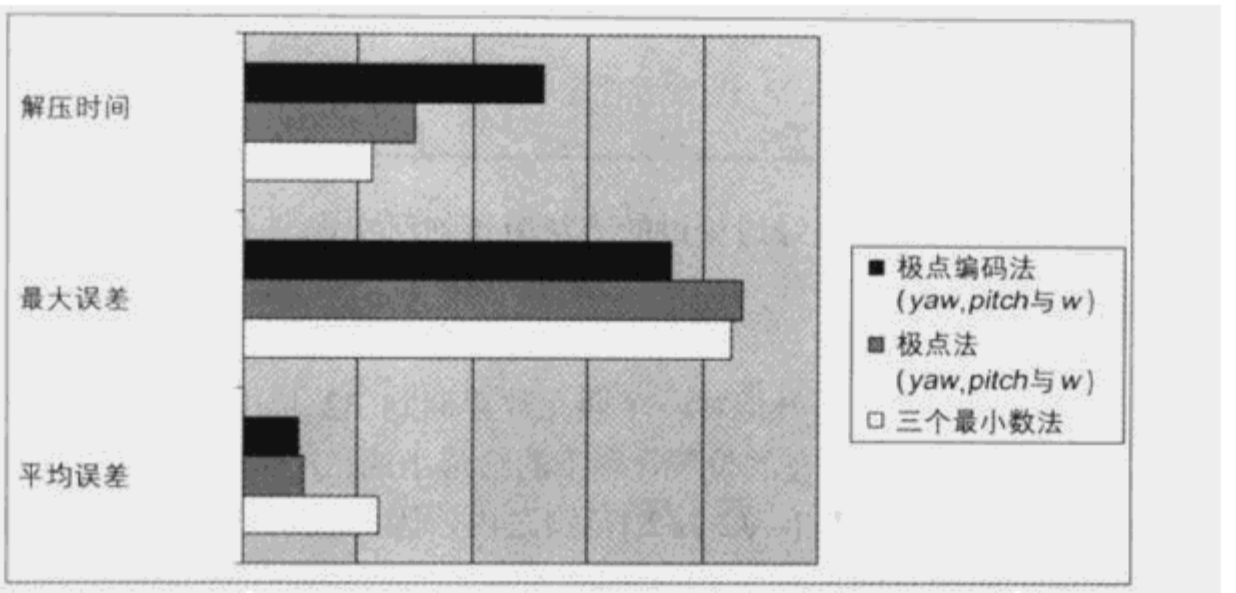


图 2.4.2 使用每个压缩方法对 *Blade II* 的四元数数据进行压缩的相对性能

2.4.6 结论

对每个压缩方法真正的测量应该是建立在对你游戏进行了细心观察的基础之上的。对于 *Blade II* 来说，最坏的情况是积累的误差最终会影响到骨架，特别是如果人物正在手持一个较长的杆状物（例如散弹枪）的时候！而使用极点方法比使用三个最小数的方法则有明显的改善。

2.4.7 答谢

我在此处向 Jan Svarosky 与 Mike Diskett 表示谢意，是他们帮我形成了本节中的这些思想。

2.4.8 参考文献

[Edwards00] Edwards，Eddie，“Polynomial Approximations to Trigonometric Functions,” *Game Programming Gems*, Charles Rier Media, Inc., 2000.

[Svarovsky00] Svarovsky, Jan, “Quaternions for Game Programming,” *Game Programming Gems*, Charles River Media, Inc., 2000.

Download at www.Pin5i.Com

2.5 受限的逆向运动学

Jason Weber
英特尔公司
jason.p.weber@intel.com

当前交互应用程序里面使用的动画有很多都依赖于事先存储下来的动作捕获或是手工制作的数据片断。虽然可以对这种动画进行后续加工美化，但是在重复的使用之下玩家还是很容易看出端倪，而吸引用户的关键点之一就是要不停地给之以崭新而独特的游戏环境。

为了在运行时提供独特的环境，我们首先需要一个抽象的方法以对一个网格进行控制和变换，例如可以使用内嵌的骨架。接下来，我们需要根据网格附近的事件或是事物对此骨架生成动作。通过正向运动学，我们可以很容易地调整骨节的角度，就像处理一个连接在一起的艺术家使用的木头模型一样。然而，我们同样需要进行逆向运算以从期望的端点位置，例如手脚，得到相应的角度。逆向运动学（IK）就提供了一种快速而稳定的方法，通过它，我们就可以让端骨节以一个可移动的“效应点”为准得到任意一个骨架中各个骨节的位置了。提供了一个实时的解决方法以后，我们就可以让人物对不可预测的环境进行及时与独特的反应了。

在本节，我们阐述了一个知名的称为“循环坐标推演”的方法，另外还演示了应该如何基于关节的物理限制对角度进行控制。我们将主要关注于3ds max 的格式。

2.5.1 骨节层次

骨架结构基本上来说就是一组有次序的转换，正如一幕幕走景一样。在每个转换中间，我们都将定义一个骨节长度，实际上它就是延变换的 x 轴方向上的一段位移而已。在缺省情况下，每个子骨节的原点都处在父骨节的终点上。我们可以允许使用任意的位移量，但是在大部分情况下位移量为零，因为一般来说骨节都是一节节连起来的。

为了清晰起见，我们将不会特别提到模型空间，在这个空间里面放置了所有的人物。我们将把根骨节移动的空间称之为世界空间，虽然在实际情况下，它可能只是一个在更大场景之下的节点而已。在每个骨节层次的转换中，相对于其父骨节，此转换将被称为一个局部转换。

至此，我们将使用四元数对所有的旋转进行处理，因为对它们进行插值的时候，其结果可以保持连续性[Bobick98]。一个四元数是一个复数在

Download at www.Pin5i.Com

四维上的扩展。因此，正如一个复数可以用来表示二维的旋转运算一样，$w+x\mathbf{i}+y\mathbf{j}+z\mathbf{k}$(**i*i = j*j = k*k = −1, ij = k = −ji**)可以用来有效地表示三维旋转。牵涉到四元数的转换，还有使用四元数的运算时，其数学计算就比较麻烦了，但是你也可以得到一些较小的库以及很清楚的示例程序[Flipcode98]。我们使用右手螺旋系的单位四元数，其形式为(w, x, y, z)，而其中(1, 0, 0, 0)表示的就是一个没有旋转的单位四元数。这些值与角度/轴坐标格式是类似的，其表示的就是一个非单位矢量(x, y, z)旋转了 $2\arccos(w)$弧度。

我们使用了一个参考坐标以描述骨架的层次结构，在此坐标中骨架处在一个没有变形的网格中（3ds max 中的 BiPad™称其为“figure mode”）。如果骨节的运动偏移了参考坐标，那么此运动可以用来将网格变换到任意的位置。此变换技术不在本节描述范围之内[Weber02]，因此在这里，我们只需要知道 IK 算法或是插值的运动数据在每一帧都提供了一个基于骨节索引的世界坐标下的变换就行了。当我们得知了一帧中所有相对于父骨节的四元数值之后，整个骨节结构就已经被遍历了一次，我们就可以从简单的四元数到四元数的连接和四元数矩阵的转换之中生成世界的变换了。

2.5.2 循环坐标推演

IK 系统将试图对一组连续的骨节进行旋转，以满足端骨节处在一个可移动的控制点上，此点被称为效应点。我们有很多复杂而昂贵的方法用来生成逆向运动学上的解，但是幸运的是还有一种相当简单的算法，它不仅快速，而且非常稳定，称为“循环坐标推演”（CCD），此方法在[Lander98]中得到了很好的阐述。我们将在所有的旋转操作中使用四元数。

在每次循环之中，我们都将从链上最深的子骨节开始处理。此骨节将相对于其原点进行旋转，这样以使其直接指向效应点。接下来，此骨节的父骨节将针对其原点进行旋转，以使得此父骨节的原点到新旋转的子骨节端点的连线指向效应点。然后对每个骨节都进行上述的处理，以使得每个骨节的原点到同一个端骨节端点的连线进行旋转指向效应点。如果对此骨节链进行多次循环处理，那么得到的每个骨节的旋转角度就会更连续平滑。即使效应点是不可达到的，此方法也可以得到一个稳定的，而不是振荡或是不稳定的解，此解将企图达到效应点。

使用此方法的时候，有可能其结果会使得最深的子骨节特别灵活，例如它可能会弯曲很大的角度去捡一个球。我们可以对此解进行修改，只允许某些骨节在每次循环中旋转部分角度，但是这将导致循环的次数增大以得到一个稳定的解。如果是捡一个球，一般最好是让 IK 效应器对腕骨节起作用，而不是对指骨节起作用，然后再在手上使用一个通用的捡物体的方法。

图 2.5.1 显示了一个推演中的步骤。X 符号表示的是效应点的位置，点划线表示的是当前原点到端点的角度。每个当前的骨节都将旋转以尽力使点划线接近期望的虚线。

如果一个骨节与多个受影响的端骨节有关，那么此骨节可以先变换出一个角度以满足每个解，然后再使用一个给定的权重值以得到一个综合结果。

要想得到一个可接受的解所需要的循环次数，那就要看每个新帧是在何处开始求解的。在传统的方法中，你可以在每个新帧开始的时候将每个骨节设置到一个缺省的位置。此方法可能需要进行大约 10 次的循环以得到一个新解。然而，如果效应点的位置偏移了一点，那么解就会发生巨大的变化。例如，如果你将效应点移到了垂直于手背的地方，那么此方法的结

Download at www.Pin5i.Com

果可能会是翻过肩来得到此效应点，而不是转过来得到它。一个取代方法就是从上一帧得到的结果继续下去求解。这样，我们甚至可以做到每帧只有一次循环。通过使用这种增量方法，求解的时候就不会在帧之间进行幅度很大的改动，而将会得到比较连续的解。

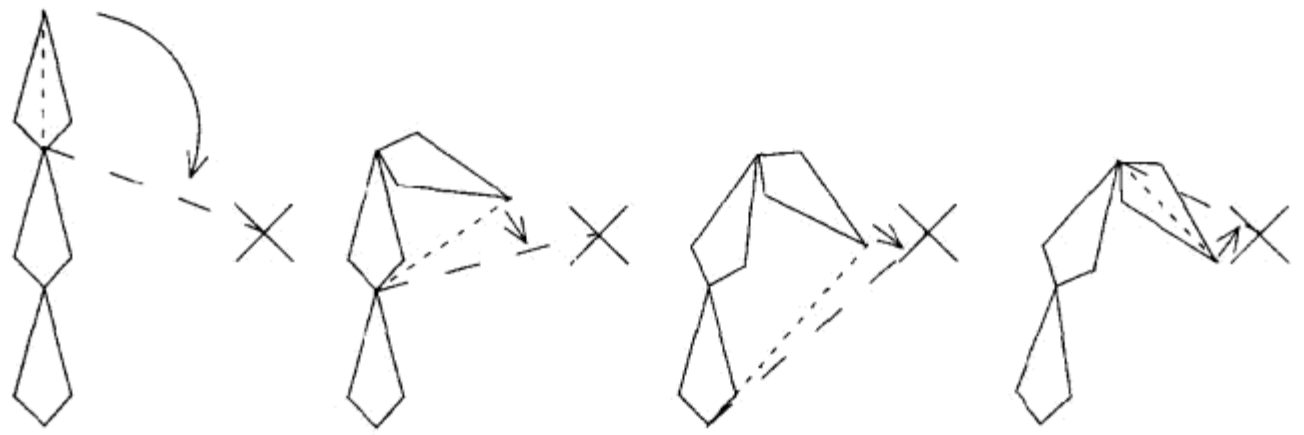


图 2.5.1　循环坐标推演的步骤。X 符号代替的是效应点的位置

在生成真实效果动作的时候，有一个重要的因素，那就是要对角速度加以限制。若想做到这一点，我们需要在世界坐标系里面得到前一帧的四元数，拿它与本帧的四元数进行比较。我们将差值转换成角度/坐标系的形式，对角度加一个最大值的限制。3 弧度左右每秒的速度是一个比较好的缺省值，如果此值更小的话，动作会更缓慢。而对角加速度加以限制则没有什么意义，因为大部分生物结构都可以在一帧的时间里面加速到最大值。

请参见[Welman93]，这样可以得到有关 CCD 以及其他逆向运动学的更深入的阐述。

2.5.3　旋转限制

上小节得到的 IK 解与物理对象而言仍有缺陷，因为我们没有考虑到真实情况下角度的限制条件。如何对限制条件进行定义与应用，这是由编辑包输出的形式决定的。因此，我们需要定义两个需要遵循的转换。

1. 欧拉角

虽然我们的旋转是利用四元数进行的，但是艺术创作人员是使用欧拉角对角度加以限制的，欧拉角是另一种角度的表现形式，它通过连续的三个角度来表示。例如，欧拉角可以用航向角、高度角与倾斜角来表示一个飞机的方向，在此处我们在使用航向角对飞机进行了旋转以后将再使用高度角对之旋转，然后使用倾斜角对之进行处理。如果其中某个角度接近 90° 的话，那么就有可能发生平衡锁定（gimbal lock）的问题，因为此时两个旋转轴可能会合二为一，将变为冗余的。在这种情况下，我们就丧失了第三个自由度。

从一个欧拉角的表现形式转换为一个四元数或是矩阵只需要连续进行三次旋转变换就行。不过，如何还原为欧拉角则没有明确的定义，而是有很多方法。在这里仅提一下，在参考文献[Flipcode98]中有这么一个转换的例子。

2. 世界坐标系里面的限制

开发人员可能希望某个特定骨节的角度限制是针对其父骨节而定的。虽然有时的确如

Download at www.Pin5i.Com

此，但是 3ds max 使用的是世界坐标系来定义这些角度限制值的，这就意味着，如果一个父骨节相对于世界的参考变换不是与世界坐标系的轴向一致的话，那么相对于此父骨节局部坐标的子骨节的角度限制也就不可能实现了。正是出于此点考虑，3ds max 的手册上推荐在参考位置（它们的 figure mode 中）中受限的骨节应该与世界的轴向保持一致。不幸的是，我们无法将世界坐标系里面的限制条件简单地转换到骨节的局部坐标系里面来。为了重现艺术创作人员的意图，我们必须绝对遵循以上的规范，在需要的时候进行相互之间的转换。

2.5.4 调整每个骨节，同时保持限制

下面的伪代码描述了将 IK 应用到一个特定骨节上的整个过程。此过程是对整个结构进行的，其顺序是子骨节优先序。如果需要对照，读者可以参阅 CD-ROM 上的文件 GPGCharacter.cpp 中的 GPGCharacter::KineBone()函数。变量 world_relative 的值为 true（你可能注意到了，如果它的值是 false 的话，代码就会简单多了）。

每个骨节都包含了一个列表，里面是它需要参与的所有效应点的求解过程。对每个有一个效应点的骨节来说，通常都会有数个骨节参与求解以将此骨节指向效应点。请注意，我们将这种与一个效应点相关的参与求解称为一个“效应（effection）”。有一个效应并不就是说此特定的骨节需要达到此效应点，只不过意味着它有可能需要帮助其下的骨节达到效应点而已。例如，上臂可以帮助手得到一个球。

浮点数 *scaler* 是用来消除多个效应点的影响的。在我们当前的代码中，它是平均分配的，这样的话，如果一个骨节有一个与两个效应点有关的效应，那么每个效应点最终产生的效果只占总效果的一半而已。

```
Vector3 effected = current end of effected bone in world space displaced in local X
 by the bone length
Vector3 effector = current position of relevant control point in world space
Vector3 current = effected, reverse-transformed to local space
Vector3 desired = effector, reverse-transformed to local space

if the difference between current and desired is small, skip to next bone

normalize current and desired

compute a quaternion delta that would rotate current to desired

sum all the scaled deltas from the multiple effections and store in change

if velocity-limiting is on, limit change to the max per-frame angle allowed; this should
 be adjusted to the magnitude of the timestep

rotate the bone by the quaternion delta

Quaternion global_rot = parent's world rotation * this bone's local rotation (this is
 the bone's current world transform)
```

Download at www.Pin5i.Com

```
Quaternion parent_delta = parent's reference rotation * inverse of parent's current
 world rotation (this is how much the parent has deviated from reference with respect
 to the world)

Quaternion global_delta = parent_delta * global_rot * inverse of parent reference world
 rotation (this is the bone rotation in world-aligned axes adjusted for parent rotation)

convert Quaternion global_delta to Euler euler
```

下一段代码将对角度加上真正的限制。它将对每个轴上的 euler 变量进行处理。“Active”表示其可以移动。“Limited”表示此角度将受到限制。为了避免平衡锁定，当 *y* 角接近正负 90° 的时候（大概是在一个弧度的 5%范围内），我们就忽略 *x* 和 *z* 角。我们将保持 *x* 与 *z* 的值，直到 *y* 角进入了安全的范围内以后再变换。

请注意 *bias* 变量的用法。当一个可能的解其范围远远超出了限制值以后，它可以用来防止解处于振荡状态跑到负方向上面去。如果 *bias* 为 0，那就意味着骨节正在倾向于最小角度的限制；为 1 则意味着它在倾向于最大角度的限制；为 2 就意味着不偏不倚。当考虑这些限制值的时候，可以将之想象成一个在圆中间的饼状块，其角度就是最大角与最小角。通常情况下，如果角度超出了此范围，你将希望得到一个最接近的值（最小值或是最大值）。然而，如果计算得到的解是超出此饼状块的，但是又几乎与两个限制值距离相等，那么有可能解就会在最大值与最小值之间来回振荡。因此，我们加入了一个 *bias* 值来处理在饼外抖动的情况。在此时，其偏向的限制值应该比另一个值接近得多。在我们的代码中进行大小的比较之前，我们给偏向的值加上了一个 10° 的额外值。

```
if active and not limited, skip this axis (any angle is fine)

if not active, set the angle to the reference value; skip to next axis

X,Z axes only: if Y angle is near 90 degrees, set angle to last frame's value, and skip
 to next axis (gimbal lock avoidance)

if current angle is within limits, reset bias to 2 (no preference), and skip to next
 axis

mindiff = minimum - angle
maxdiff = angle - maximum

adjust mindiff and maxdiff to be in range of 0 to 2*PI

if there is a bias preference, adjust angles to make flipping between solutions less
 desirable by adding the 10 degrees to the opposite min/max diff variable

if maxdiff<mindiff, set angle to max and bias to 1
if maxdiff>mindiff, set angle to min and bias to 0

store modified euler in case there is a gimbal lock next frame
```

Download at www.Pin5i.Com

```
convert euler to Quaternion global_delta

Quaternion constrained = inverse of parent reference world rotation * global_delta *
 parent reference world rotation

set bone rotation to constrained

recompute cached current world transform for this bone and all descendents
```

图 2.5.2 展示了一个人物是如何取到其头部 2/3m 之后的一个效应点的。在没有限制的时候，求解将把胳膊越过肩膀呈直线直取效应点，加上了限制以后，胳膊就被局限在其物理极限以内了。此外，还请参看彩图 2，里面显示了一个蝎子坦克网格里面是如何使用此技术的。

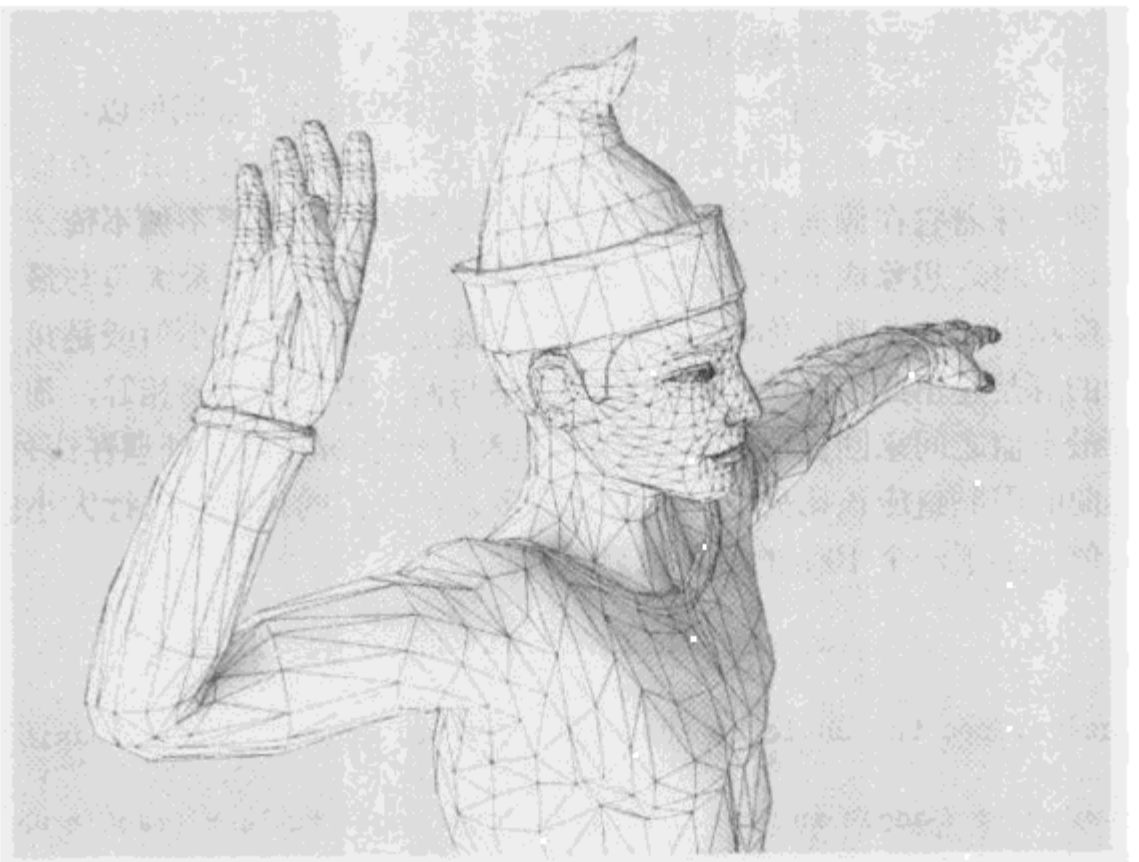


图 2.5.2 向后取一个效应点

2.5.5 结论

在一个骨架运动中使用逆向运动学将得到某种程度上的自动化，此效果通过事先制作或是捕获的运动数据是很难生成的。如果能实时地生成这些动作，那么人物就可以在一个不断变化的环境中反应自如了。

在本节的例子中，我们使用了固定的限制值对角度加以限制，如果再在限制值上加一些弹性，我们就可以减少一些可能产生的机械化的动作了。

2.5.6 参考文献

[Bobick98] Bobick, Nick, "Rotating Objects Using Quanternions," *Game Developer*

Download at www.Pin5i.com

Magazine, February 1998: pp.34-42. 此外对应网址为 http://www.gdmag.com/.

[Flipcode98] 无署名，“The Matrix and Quaternions FAQ,” 对应网址为 http://www.flipcode.com/documents/matrfaq.html, December 1998.

[Lander98] Lander, Jeff, “Making Kine More Flexible,” *Game Developer Magazine*, November 1998: pp.15～22.

[Weber02] Weber, Jason, “Improved Bones Deformation,” *Game Programming Gems 3*, Charles River Media, Inc., 2002.

[Welman93] Welman, Chris, “Inverse Kinematics and Geometric Constraints for Articulated Figure Manipulation,“ Masters Thesis, Simon Fraser University, September 1993.

作者将尽力在 http://www.imonk.com/baboon/bones 维护一个长期的文档以及提供某些相关资源的连接网址。

Download at www.Pin5i.Com

2.6 针对物理建模的单元自动机

Tom Forsyth
Mucky Foot Productions, Ltd.
tomf@muckyfoot.com

在很多当前的游戏中，环境大部分是静态的。在游戏中的物体仅局限于小的、个别的物体，例如车辆与人；或有时候会是一些大的、机械式的物体；或是指定好的对象。在某些情况下，容器里面的水平面会按照脚本进行运动，但是此运动也仅限于一个水平面的上下移动，而且玩家根本不能与其直接交互。

就当前游戏的水平来看，下面的效果一般都不太真实或是根本不能进行模拟：

- 火，扩散的火焰将可燃物点燃，并造成破坏。
- 水，可以放在容器里，流过管道，被抽出来（具有真实效果地），有物体在其中走过，有重物在其中下沉，可漫过容器，或是漫过地板，流下斜坡。
- 油，结合了水的流动性和燃烧物质，例如木头的可燃性。
- 具有真实效果伤害半径的爆炸，在屋内比在屋外造成的伤害更大，冲击波真实地经过各个角落的效果。
- 造成空气上升的热，导致气流的对流，可以利用通风扇将其吹散，甚至可能会带有一些气味。
- 随着气流扩散的烟尘，它们由火焰或是烟雾弹造成，使视线变模糊，导致人们呼吸困难。
- 可以被破坏、摧毁、点火、移动，或是能提供一些对爆炸与攻击进行防护的墙壁以及环境。

其中的某些特性在游戏中出现过，但是其一般是受脚本控制的，而且也相当有限。通常情况下，它们在真正的游戏中起的作用很小，而且看起来斧凿的痕迹太重——当然了，它们本来也是假造的。但是如果使用单元自动机（CA）来仿真这些效果，就会得到动态得多、真实得多的行为了，而且也能在游戏里面得到新的游戏玩法与策略。退一万步来说，它们也会带来更真实的效果、更好的图形渲染，因此还将使得用户更加沉浸于游戏之中。

2.6.1 CA 基础

单元自动机（CA）以及其近邻有限元分析（FEA）以及计算流体动力

Download at www.Pin5i.Com

学（CFD）[CFD]，已经在对气体和水流建模、热量分布、建筑压力与张力，还有很多真实世界的其他方面上，在许多应用中得到了采用。然而，在学术研究与商业建模中主要感兴趣的是精确度，但对于游戏开发人员来说，主要关心的只是要事物看上去足够好，而且运行起来也足够快，而且在绝大多数情况下面，仿真都能得到极大的简化，并保证绝大多数人看起来仍然会有相当好的效果。

CA 的基本概念很简单：整个世界被划分为固定大小的单元组成的网，每个单元都有一些不同的数字与之关联以表示出它的状态，一般一个单元里面的值有气压、温度、水量、水或是空气流动的方向等。

在每个游戏循环里面，每个单元都会进行处理。它将把自己与相邻的单元进行比较，根据不同的规则，在它们之间的差异将导致单元及其相邻单元的状态发生变化。最出名的 CA 之一就是“Conway 的生命游戏”[Conway]。这是一个极其简单的 CA。它只有一个位来表示其状态——表示这个单元是否为空——以及几条极其简单的规则来依照相邻单元的状态改变状态。尽管如此，这个简单的模型能带来一些非常复杂的行为。

在游戏中，CA 使用的规则是基于各种物理模型而来的，它用来决定在相邻单元之间转移的热量、气量、水量，或是烟尘含量。如果我们能很快地对足够数目的单元执行这些规则，那么水就会流下山岗，归于大地，微热的空气就会形成气体的对流，极热的空气就会开始燃烧，而且，反过来又会被燃烧物加热。

在一个三维数组的立体单元里面，我们有三种相邻单元的定义：

- 与中心单元共享一个面的 6 个单元。
- 以上 6 单元再加上 12 个与中心单元有共享边的单元。
- 以上 18 单元，再加上与中心单元有共享顶点的 8 个单元。

令人惊异的是，不管我们使用哪种定义，最终针对物理仿真使用的规则得到的结果几乎完全相同。当然了，第一个版本是最简单的，而且它只有一种相邻单元，而不是三种。出于这个理由，如果只考虑与中心单元共享一个面的 6 个单元作为相邻单元，那就会容易得多了。

首先，我们要选定一个 CA 单元的物理大小。对于有人体尺寸大小的游戏来说，我们决定使用半米见方的立体。如果立方体再大一点的话，一个 CA 单元的空气就无法塞入一个狭窄的通道里面了。如果使用更小的单元，我们会得到更高的分辨率，得到更小的管道、更狭窄的间隙，等等——但是额外的空间和处理代价也是很大的。不同大小的游戏自然会要求不同大小的 CA 单元尺寸。然而，由于大部分游戏是人体尺寸大小的，为了方便起见，在本节中我们的例子中将假设单元的尺寸为半米。

在一个人体尺寸大小的游戏中，另一个重要的因素就是如何对较薄的墙进行建模。很多房屋内部的墙壁和房门都只有几厘米厚。它们可以挡住水流、减慢火速，还能停止烟雾和气体的扩散，因此我们必须要有某种方法来对之建模。如果使用传统的方法，利用很多小的单元来建模，将墙壁占据的单元划为实心态，则我们就需要使用低于 10cm 厚的单元，这就要求得到 125 倍于前的单元数量——这是一个非常昂贵的代价。

我们有两种解决方案。一种方法是首先由 *X-Com* 系列游戏采用 CA 用法，它令人印象深刻，而且也相当新颖[Xcom]，此方法将单元之间的面建模为实体，就和对单元的建模一样。这样一来，墙壁、地板，以及天花板一般就可以处于两个单元之间，延平面展开了。这种方法很有效，不过它也意味着，现在，我们就需要处理两种不同的类了——充满了整个立方体

Download at www.Pin5i.Com

的对象（例如，岩石、灰尘、家具，或是高大的植物）以及落于两个单元之间的对象（例如，墙壁、地板、矮小的植被，或是房门）。在对物质和其交互进行建模的代码中，这将最终产生各种特殊情况处理的代码，而且会在两种类型处理中产生大量的代码复制（意大利面条式的代码）。不过，如果这种模型适合的话，那么还是可以对之进行考虑的，而且它也很直观——对象的内部处理形式与外部渲染的形式很匹配。

另一个方法就是保持方法的通用性，同时也要避免使用大量的小型单元。在这里，我们将不会使用固定位置的网格来摆放立方体单元，而是允许单元之间的边缘依照其包含的内容稍微偏移一点。这样一来，一面薄墙就可以嵌入到半米的方形面里面去了，而每个方形的面都是在一个单元里面的。由于墙只有几厘米厚，因此我们可以把相邻的单元进行扩张以占据额外的空间。即使单元不是按照网格对齐也没有关系，因为 CA 处理的代码本身不知道，也不关心其处理对象的表现形式。站在 CA 的物理角度上来看，每个对象依旧是半米厚的。大部分的区别仅仅发生在对象的渲染代码里面，而不是在处理 CA 的代码里面。渲染代码必须保证水在墙壁的网格里流动，而不是沿着 CA 单元的边缘流动，如果是沿着后者流动的话，水与墙壁之间就会有一段距离了。需要进行类似扩散处理的都是具有体积性质的效果，例如烟雾、火焰和流水。当需要画出具有这么一种效果的单元时，渲染器需要对每个相邻的单元进行检查，看看其多边体的形状是否比标准的半米立方体要小。如果是的话，它就对那些具有体积性质效果的尺寸进行扩张，使之充满整个单元空间。

在这种模式下面，一个一米宽、带有一层薄薄木墙的走廊会使用一组“木质”单元、一组“气体”单元，然后又是一组“木质”单元来表示。由于这些单元与其他单元的中心都是半米的距离，因此视觉上墙到墙之间的距离仍然是一米。当然了，在世界的图形显示上面，这些木质单元就根本不是单元，而只不过是一个几厘米厚的平面而已。在进行任何碰撞检测的时候，我们都将使用这种表现形式。但是对于使用 CA 建构的世界来说，这种表现形式与其他形式也没有什么区别。因为，它对实体进行处理的时候是不考虑其形状的，而对于玩家来说，他们则不会察觉渲染物（一个一米宽的间隙）与建模物（一个半米的间隙）之间的差别的。再次声明，如果不需要这种精确性的话，我们将牺牲精确以获得更高的速度。

下一个需要考虑的因素对玩游戏有一些影响——使用被动场景与使用主动场景之间的区别。

1. 被动场景

在这个系统里面，凡是涉及到 CA 的地方，其场景都是惰性的——它根本不会被 CA 的动作所影响。在两种表现形式中它是最简单的，但是它仍然能够使用离散的对象，制造出例如单独的一个油桶和箱子在一条河上漂流或是爆炸或是被加热的时候燃烧的效果。

由于 CA 是只知道单元而不是多边形的，因此此场景必须被转化为单元的表现形式——这通常是预处理的一步。这些单元将仅仅被标志为一个限制 CA 动作的立方体而已。当然了，场景一般是一组任意多边形的集合，而且也不会按照单元的边界对齐。但是由于使用 CA 建模的对象而言我们根本不考虑其形体，因此实际上这也没有什么区别。只要每个固定的多边形墙壁能被转化为一组连续的 CA 墙壁单元，水就不会流出去，也就不会露出马脚了。

即使是在这样的系统中，我们仍然要对能打开的门以及其他能运动或是移动的对象进行特殊处理。当门被打开的时候，它们应该删除（滑动式的门）或是移动（转门）那些代表它

Download at www.Pin5i.Com

们的固体单元，这样流水或是火焰就可以通过它们了。

2. 主动场景

这种方法远为通用，远为优越，在此方法中我们将更进一步，同时使用 CA 对场景进行建模。这就会引入一个“完全可分解的世界”的概念了，而这是很多设计人员都认为这将是游戏界中出现的下一场革命，虽然这个概念在电脑游戏中并非一个真正的新事物[XCom]。

在这个系统中，场景再也不是由惰性物质组成的单元，而是使用其真正属性进行建模的了，例如温度、可燃性等。当使用 CA 建模的单元根据 CA 的物理规则改变其状态的时候，图形渲染引擎也会随之改变相应的多边形对象渲染的方式（例如，被熏黑了，被损坏了，等等）。

在后者的情况下，图形引擎可以是“Geo-Mod”类型的[RedFaction]，也可以给对象加上一个特殊标志，表示它是可分解的，同时还要提供一个备用的，这个物体被破坏后的图形表现形式。

2.6.2　八叉树

如果你正在考虑如何实现这些 CA 方法，那么很快，你就会注意到即使是对一个中等大小的关卡来说，存储半米尺寸的单元也会消耗大量的内存，而且其需要的处理与内存上的开销也非常严重。要想高效地处理这些问题，那就要记住不要存储或是处理那些没有参与当前活动的单元——它们一般是不会动的墙壁以及在标准气温气压（STP）下的空气。

对于这种情况来说，使用八叉树是最好的存储策略，特别是一个动态分配的八叉树。在任何一个八叉树的实现中，一定要记住在一个 CA 中最常见的操作就是“找出离我最近的单元”，因此在实现八叉树的时候一定要对这个方面进行优化。如果进行了这个操作但是却没有得到八叉树中的任何相邻单元，则我们就认为相邻的单元都是处在 STP 的空气中。相应的物理仿真就开始进行了。如果发现一个“丢失”的单元其状态明显不是在 STP 之下，则我们就会创建一个带有新属性的单元，将之插入八叉树中。当返回的单元其状态在 STP 的一定范围之内的时候，我们就将之从八叉树中删除掉，不再对其进行处理。

含有 CA 单元的八叉树还可以当成是一个通用的八叉树来使用。很多游戏都使用了八叉树进行碰撞检测和可视性裁减的优化，这样我们就可以使用八叉树来同时满足这个要求，存储一些不是与 CA 直接相关的对象了。我们可以针对搜索算法进行一番很容易的调整，让八叉树变化为一个“松散八叉树”[Ulrich00]，它比传统上的八叉树要多出好几个优点。但是在进行 CA 方面的处理时，原先的行为也不会改变，因为所有的 CA 单元都是按常数距离对齐的，而且都有固定的尺寸。

2.6.3　实际的物理

在编写 CA 物理处理函数的时候，你要记住的一件主要的事情就是要保持事物的简洁性。使用简化过的物理架构编写出很简单的函数是非常容易的，而且在玩家看来它仍将非常真实。只要质量与能量守恒的基本概念没有被违反——这些经常被忽略了——其他的代码都是锦上

Download at www.Pin5i.Com

添花，只不过是为了让仿真更稳定一点。

在实现上我们遇到的主要问题就是要找出一个优良而简单的模型，同时它还要具有各种物理特性。大部分的标准实现方法都是应用 Navier-Strokes 方程，对各种物质进行处理，以尽可能少的误差来将它们实现出来。这将使得代码非常复杂，而且大部分学术研究与商业文献都是以减少误差为主题的。对于游戏而言，我们需要的是简洁性，而不是精确度。在大部分情况下面，我们只需要在文献中找出如何实现大致感觉正确的效果就行了。

2.6.4 核心处理模型

对大部分由 CA 仿真的属性来说，其处理都是类似的。为了对这些通用的方法进行一番阐述，下面我们给出了一个非常简单的流体仿真，它的目标是为了在给出的空间里面造成一个均匀的气压分布。虽然这个模型极其简单，但是对于气体和流体建模来说，它仍是很有用的。

```
for ( neigh = each neighbor cell )
{
    if ( neigh->Material->IsInert() ) continue;
    float DPress = cell->Pressure - neigh->Pressure;
    float Flow = cell->Material->Flow * DPress;
    Flow = clamp ( Flow,
        cell->Pressure / 6.0f,
        -neigh->Pressure / 6.0f );
    cell->NewPressure -= Flow;
    neigh->NewPressure += Flow;
}
```

代码中的 clamp()操作是为了防止 NewPressure 成为负数。除以 6 是因为周围一共有 6 个相邻的单元。在实际情况中，即使是衰减也需要保持其稳定性，防止产生小的振荡，例如水面上的涟漪，变成不真实的振荡。

传统上来看，一旦所有的单元都通过此方法处理完毕以后，NewPressure 就会被复制到 Pressure 值中去了。这种双缓冲机制是必要的，不要在函数结束的时候把 Pressure 给直接覆盖了就好。否则，在各个单元更新的方向上气压传输的速度就会非常快（有时候几乎是瞬时的），但是在反方向上传输的则会慢得多。这样就将在热力分布、水流以及其他的处理中产生明显的不对称性。

对每个单元进行两次访问会造成严重的性能下降，尤其是第二次访问仅仅是进行一次简单的复制而已，而且这也会受到大部分现代 CPU 的内存带宽的限制。一个更好的方法是把一个单元处理的最后一次的遍数保存下来。当在以后对这个单元进行处理的时候，我们就会检查这个遍数。如果这个值比当前的遍数要小，则需要进行复制。虽然这种代码好像比较古怪，但是它比对所有的单元进行扫描还是要快得多了。这样代码就变成下面的形式：

```
if ( cell->Turn != CurrentTurn )
{
    cell->Turn = CurrentTurn;
    cell->Pressure = cell->NewPressure;
}
```

Download at www.Pin5i.Com

```
for ( neigh = each neighbor cell )
{
    if ( neigh->Material->IsInert() ) continue;
    if ( neigh->Turn != CurrentTurn )
    {
        neigh->Turn = CurrentTurn;
        neigh->Pressure = neigh->NewPressure;
    }
    // same physics code as before
}
```

2.6.5　气体

对于均匀分布的气体来说，这个简单的模型是完全适用的。乍看上去，在一个游戏里面气体不会频繁的建模。但是实际上它是最重要的一种效果了——那就是爆炸以及它们对物体造成的影响。一个爆炸就是一块物质在极短的时间里面产生了极大量的气体。我们可以使用下面的步骤对之建模：首先，找到离爆炸中心最近的 CA 单元；其次，给此单元加上一个极大数值的气压；再次，让 CA 把此气压扩散到整个世界中去。对周围事物的损害是通过很高的绝对压强或是很高的压力差造成的——实际上，两者都会对不同的事物造成不同的损害。但是对于游戏而言，这种复杂的考虑是不必要的。

这种建模方法比传统方法要好的地方就是视线的处理可以自动化。在一个受限空间里面的爆炸威力要比在一个开放空间里面的大得多，这是因为扩散压力的空间比较小。此外，它还显示出仅使用视界是无法判断是否会受到爆炸伤害的——在某种程度上，爆炸可以绕过墙角和障碍物。

由于这种流体仿真的方法对于人眼来说是看不出破绽的，同时随着爆炸还可以带起一些碎片和小的物体。你也不用担心由于碎片奔向错误的方向或是穿过固体的墙壁而导致产生不真实性。

2.6.6　水流

水流的处理只比气体要复杂一点点而已。最明显的区别就是气体的扩散会带来单元气压的改变，而水只会停留在容器的底部，且不能被压缩。

实际上，要仿真水中压力的传播，最简单的方法是对水进行一点点压缩。这就意味着压力可以被存储为单元中额外的一部分水，它是超出了单元能够容纳的那部分体积之外的。事实上，需要进行压缩的量是非常小的——只需要每个单元每立方体长度 1%的水就够了。在一个静态的水体中，其单元一般每个都包含 1.00 升（0.001 立方米）的水，在顶上的单元包含的是 1.00 升的水，在其下的单元会有 1.01 升的水，在更下面的单元会有 1.02 升的水，依此类推。这么微小的压力对于玩家来说是感觉不到的，但是我们就有足够的余地对液体所有的属性进行处理了。例如，通过一个管道连接的两个容器的水面最终会一样高，即使我们正在把水倒入其中的一个容器里，它也会通过管道流到另一个容器中的。

```
if ( neighbor cell is above this one )
```

Download at www.Pin5i.Com

```
{
   if ( ( cell->Mass < material->MaxMass ) ||
        ( neigh->Mass < material->MaxMass ) )
   {
      Flow = cell->Mass - material->MaxMass;
   }else{
      Flow = cell->Mass - neigh->Mass
           - material->MaxCompress;
      Flow *= 0.5f;
   }
}
else if ( neighbor cell is below this one )
{
   if ( ( cell->Mass < material->MaxMass ) ||
        ( neigh->Mass < material->MaxMass ) )
   {
      Flow = material->MaxMass - neigh->Mass;
   }else{
      Flow = cell->Mass - neigh->Mass
    + material->MaxCompress;
      Flow *= 0.5f;
   }
}
else    // neighbor is on same level
{
   Flow = ( cell->Mass - neigh->Mass ) * 0.5f;
}
```

此处的 Flow 值接下来就会被根据水流的最大速度进行调整和缩小，使得某些水流比其他的部分显得粘滞，防止任何最终得到的质量变成负数。

水流模型处理中的两种分支对应的是不同的情况。第一种情况是两个单元种的一个没有被填满水——例如在一个水体的表面或是水正在溅开或是掉下来（例如，在瀑布中）。此处的行为是很简单的——水将流下来填满两者中较低水面的单元直到 MaxMass 值为止——这是一个单元体积里面可以填充的最大水量。在上一个例子中间，此质量就是一升水。

第二中情况是两个单元都充满了水，或是有点儿溢出了，例如在水体中间的时候。在这里，水流就必须要判断一下上一个单元比下一个单元是否刚好多出 MaxCompress 体积的水。MaxCompress 就是由于压缩而额外可以填充进来的水。在上个例子中，它就是 0.01 升的水量。

2.6.7 流速

到此为止，气体和水流的模型中都忽略了一个对任何液体与气体来说都很重要的性质——流速。我们仅仅考虑了两个单元中的压力差，而且使用了这个压力差来对物质进行移动。如果对于相对静态的环境来说，我们想得到稳定状态的话，这是可以的，例如均匀的气压或是需要进行一致化的水面。很多游戏仅仅在游戏交互中使用了这些简单的属性和实现它们需要的真实性。

不过，在真实生活中水和气体都是有动量的（它等于速度乘以质量），而压力差仅会影

Download at www.Pin5i.Com

响到单元之间的流速，而不会唯一确定此速度。在对波浪、流动的河流，以及气流进行建模的时候，把其动量或是速度存储下来是很重要的。虽然即使没有考虑动量，河流仍然可以流动，但是此时河流会有一个明显可见的至少有 10° 的斜坡，这看上去将极为奇怪。

在每一个处理步骤中对动量或是流速进行建模时，就像前面那样，质量差决定了压力的梯度。但是，我们不会直接改变单元中的质量，现在压力差只会改变单元之间的流速。接下来流速才会改变单元的质量。此时的代码会更复杂一点，因为流速是一个三维的矢量，而不是一个像质量一样的标量。

我们有两种处理流速的方法：第一种是把一个流速矢量看成是通过单元中心的流速。这可能是最直观的模型了——流速和质量都按照其单元的中心进行计算。然而，在这种情况下面，流速是受到相邻单元之间压力差的影响的，而它反过来又会影响质量是如何从一个单元流向另一个单元的。请注意此处将产生一个奇特的结果：对于一个特定的单元来说，存储在其中心的流速矢量不受其中心的质量影响，而是受相邻的单元影响，而且其流速也不会改变此单元的质量，只会改变其相邻单元的质量。这个结果有些奇怪，而且在某些情况下面，将会导致一些古怪的行为。

如果将流速矢量的每个分量看成是相邻节点之间的流速就更实用了——其相邻流量从当前节点指向下一个节点的相对正方向。这么一来，在(x, y, z)位置的单元存储的流速矢量 **F** 就是指有一个流速 F_x 从(x, y, z)单元指向$(x+1, y, z)$单元；F_y 则是由单元(x, y, z)指向单元$(x, y+1, z)$的流速；对于 F_z 也有同样的结果。现在 **F** 的意义不那么直观，但是其物理模型更为合理。实际上，这是最常见的模型。不过，只要对各种参量进行适当的调整，两种模型都可以用在仿真上。

在此模型中最重要的一步就是要仔细地控制振荡。这种模型不仅支持波浪效果，而且很容易产生这种效果。我们必须引入一个简单的摩擦系数来对流速进行衰减，否则波浪就会越来越高而不会衰减下去了。在这种情况下，液体或是气体就会产生一些非常古怪的行为。

值得注意的是虽然流速最重要的应用之一是河流，但是在大部分人体尺寸大小的游戏中，大面积的水体，例如湖泊和河流，由与其体积太大，一般说来在游戏交互中就不会被经常用到了。不管玩家做什么，它们的行为几乎保持不变，而且即使发生了变化，其变化也是相当有限的。一般说来它们不需要一个 CA 那样的灵活性，而且如果使用更常规的方法来处理，它会更容易建模，也更容易渲染。我们可以使用预先计算好的动画网格、碰撞模型和脚本事件。不过，仍然有很多其他种类的游戏，尺寸更大，而且希望使用 CA 来对河流进行仿真。

2.6.8　热量

热量在环境中的传播，不管热是由燃烧的对象或是其他来源产生的，都是以三种形式进行：热传导、热对流和热辐射。

1. 传导

传导是最容易仿真的机制。相邻的单元将互相传递热能，最终它们将达到一样的温度。这中间的过程是很复杂的，因为同样的热量在不同的物质上会产生不同的效果——这被称为特定的热容量（SHC），一般它是使用 J/kg℃作为单位。如果一个由水组成的单元（其具有较

Download at www.Pin5i.Com

高的热容量，因此难以被加热起来）与一个较冷的有同样质量的铁组成的单元（其具有较低的热容量）相邻的话，最终的热平衡温度将与水的原始温度更加接近，而不会是两个温度的平均值。这是因为当等量的热量从水流向铁的时候，水降低的温度比铁上升的温度要少得多。

请注意上面的例子对于同样质量的不同物质是成立的。不过，由于铁的密度要比水大得多，因此，对于同样的体积来说，它们的热容量就非常接近了。

```
// Find current heat capacities.
float HCCell = cell->material->SHC * cell->Mass;
float HCNeigh = neigh->material->SHC * neigh->Mass;
float EnergyFlow = neigh->Temp - cell->Temp;
// Convert from heat to energy
if ( EnergyFlow > 0.0f )
   EnergyFlow *= HCNeigh;
else
   EnergyFlow *= HCCell;
// A constant according to cell update speed.
// Usually found by trial and error.
EnergyFlow *= ConstantEnergyFlowFactor;
neigh->Temp -= EnergyFlow / HCNeigh;
cell->Temp += EnergyFlow / HCCell;
// Detect and kill oscillations.
if (((EnergyFlow>0.0f)&&(neigh->Temp<cell->Temp))||
   ((EnergyFlow<=0.0f)&&(neigh->Temp>cell->Temp)))
{
   float TotalEnergy = HCCell * cell->Temp +
                       HCNeigh * neigh->Temp;
   float AverageTemp = TotalEnergy /
                       ( HCCell + HCNeigh );
   cell->Temp = AverageTemp;
                       neigh->Temp = AverageTemp;
}
```

如果两个具有相似热容量的物质紧紧相连的话，最后的代码是必不可少的。在这种情况下，两者的温度将会来回振荡，最终失去控制。物理上正确的解决方法是将热的传输在时间上集成起来考虑。不过，这种方法得到的结果是一个加权的平均温度，它实际上是系统最终将要达到的温度。这种结果不太精确，但是非常自然，而且执行速度很快。重要的是，它遵循了能量守恒定律，因此，任何错误都将是暂时。其长期的状态将与真实效果是一样的。

2. 对流

对流是热量上升的现象。较热的流体区域，例如气体或水，比较冷的区域密度更低，因此它们有上升的趋势。在对水或是气体建模的时候，只要我们考虑到了温度，那么这种仿真就可以集成进去了。如果使用了某种流速模型，则流速除了会被相邻单元的压力差影响以外，还会被其相对温度所影响。否则，对流机制就会产生错误，虽然其效果仍然可以用火焰小节中描述的技术给伪装出来。

Download at www.Pin5i.Com

3. 辐射

热的物体会产生辐射。它们发射不同波长的光，这些光延直线行走，接触到其他物质的表面，这样就会使之温度上升。在物理上来看，这种效果是非常重要的，但不幸的是，对之进行建模也同样的麻烦。每种热源都必须从自身辐射出很多光线，而且这些光线将使任何接触到的物体升温。

辐射热量的建模与现代游戏中创建光照图的辐射建模是非常相似的。就算只进行粗糙的建模，在运行时对两者进行建模仍是非常昂贵的，虽然我们可以使用某些高明的技术进行大量的近似以改善辐射热量建模的速度。即使采用了这些算法，对于一个游戏而言，就算仅仅对辐射热量的某部分进行建模也依然会造成很大的开销。这些算法都非常复杂，而且它们也很难统一到我们前面所说的对其他物理属性建模使用的标准的单元到单元的交互。出于以上任何一个理由，我们都将不再对此话题在此处进行进一步讨论。

2.6.9　火焰

燃烧物质的物理现象通常都是非常复杂的。多个部分同时以不同的速度和热量燃烧，而且在此过程中还会牵涉到物质的不同形态变化。

为了实时地进行此运算，在此过程中使用到的物质模型需要被裁减到最小可以容忍的范围。而且对于每种物质来说，我们都需要选出不同的模型以着重强调其主要性质。

在考虑到的很多模型中，如果想经过最小的努力就能得到最好的效果，那么最终的方法可能是某种二次指数曲线的近似法。此曲线可以显示出在单位时间内一个物质在给定温度下燃烧的时候会释放出多少热能来。无论火焰温度有多高，此值有一个最大能量的上限。但是即使是在相对较低的温度下，也会释放出很多热量来。这可以解释为什么在开放环境中的火焰开始时很小，接下来会迅速增长到某个尺寸，然后不会继续增长，却仍然能燃烧很长一段时间，而不论是否有充足的燃料。它们只不过是无法产生足够的热量以弥补对环境丧失的热量而已（这是与温度成比例的）。

```
float Temp = cell->Temp - material->Flashpoint;
// Damage the cell.
CellDamage = Temp * material->BurnRate;
float Burn;
// Convert to actual burning value.
if ( Temp > material->MaxBurn * 2 )
    Burn = material->MaxBurn;
else
    Burn = ( 1.0f - ( 0.25f * Temp /material->MaxBurn ) ) * Temp;
ASSERT ( Burn <= material->MaxBurn );
ASSERT ( Burn >= 0.0f );
// And heat the cell up from the burning.
cell->Temp += Burn * material->BurnTemp;
```

请注意对一个单元造成的损坏是与其真实温度成正比，而不是与燃烧产生的热量成正比

Download at www.Pin5i.Com

的。这样当在低温下燃烧的物质暴露在高温环境中的时候，会对其造成更为严重的损坏。我们只要改变一下MaxBurn与BurnTemp因数就可以对任何事物的燃烧进行模拟了——纸张、木头、油料、火药，或是爆炸物。

当然了，火焰的主要特性之一就是它很热，因此它是依赖于前面讨论的三种热量传递模型的。在真实世界的火焰中，对流和辐射对其行为都是极其重要的。对流使得火焰在垂直方向上比在水平方向上伸展得更快，后者例如在地板上的延伸，这样在燃烧的建筑物中就很容易形成明显的火墙现象了。辐射将导致火焰集中在角落处，使得火焰将首先在房间的角落进行扩展。

不幸的是，正如前述，对辐射热量的建模是非常困难的，而对流虽然稍微直接一点，但也需要对火源附近的大量气体单元进行建模与更新，而这是很昂贵的操作。如果我们能找到一些捷径，不使用这些开销甚大的操作就能对某些特性进行仿真就太好了。

针对对流效果来说，我们可以使用一条捷径，那就是热传导在垂直方向上容易很多。在真实世界中，一堵燃烧的墙避会使得贴近的空气升温，而此空气会上升，使得墙壁高处的空气变热。这样火焰就更容易在垂直方向上扩张了。使用了这个捷径以后，热传导就会变得不对称了。在上述的模型中，我们将对与 1 个单元相邻的 6 个单元使用同一个因数——ConstantEnergyFactor——进行热传导的处理。然而现在的方案则是当与高处的单元进行热传导的时候，此数值就会增大，但是与低处的单元进行热传导的时候此数值就会减小。

找出热辐射的捷径则困难得多，但是我们可以使用与处理辐射性相同的技术找出一个房间的哪些部分更容易由于辐射的反馈热量而导致着火，至少这种预先计算的方法是可能的。一个可能性就是计算出半球型封闭区间[Hemispherical01]，然后使用它来得到火焰产生的热量——通常都是在边缘和角落的地方。

在其中，一个不是很明显的因素就是这些捷径要比任何真正的解决方法控制得更好。在真实世界中，对流是著名的混沌系统，一点点条件的变化就将导致产生完全不同的流体模式。这样围绕此效果设计游戏交互就真的会非常麻烦了。游戏以及关卡设计人员一般都要求游戏具有高度的可控性与可预测性，这样才能仔细地营造出玩家体验到的各种事物。上面提到的捷径在其行为上都是高度可预测的，而且是线性的，在游戏中，这比绝对的真实性要好得多了。

2.6.10 动态更新速率

在此处仿真的某些物理属性要求相当高的更新速度以保持其真实性。对于一个单元，在一个更新周期里面，我们需要以最高的速度进行物理属性从一个单元传播到另一个单元的处理。火焰可能会传播得比较快——其速度是以 m/s 或是更大的单位来计算的。从一个容器里面流出水的速度可能会更大——以每秒十米的量级运动。而爆炸则需要非常高的更新速度——真实世界中的爆炸冲击波是以声速传播的，大概是 340m/s。

对以上的所有效果进行仿真就意味着我们需要有每秒高达 680 个周期的更新速度。这个速度太可怕了，对于一个正常尺寸的游戏世界来说，似乎当前还没有任何平台可以支持这么高的更新速率。

如果使用了对 STP 单元不存储与不处理的策略，对于那些不需要快速更新以保持其真实

Download at www.Pin5i.com

性的单元而言，我们可以通过八叉树来降低其更新速率。

当一个单元被处理的时候，它将根据当前的状态判定到底需要多快的更新速度才能保持一个良好的仿真。涉及到爆炸的单元需要较高的更新速率；拥有流水、燃烧物，或是高温的单元需要中等更新速率；只有静态水的单元只需要较低更新速率；至于那些在与环境温度一致的场景中，我们则根本不需要对其进行处理。

接下来我们就在单元里保存这个处理速度，将其传给八叉树结构，对每一级都作上记号，这样我们就可以以其子节点要求的最高更新速度进行处理了。更新函数可以在八叉树的根节点开始处理，对树节点进行递归处理。在每一层上，它都会判断一下是否当前节点需要对每个字节点进行处理。

有一点需要注意，那就是只有当更新速率是使用 2 的整数次方进行量化的时候，此系统才发挥作用。例如，如果某个子节点要求每 3 个周期更新一次，但是父节点则要求每 2 个周期更新一次，这样每隔 6 个周期就会出现子节点要求更新但是父节点不需要更新的现象。由于遍历算法采用的是预先判定法，因此子节点在这一轮上面就不会得到更新，最终将导致子节点每 6 个周期才更新一次。而如果使用 2 的整数次方进行量化，我们就可以解决这个问题，而且得到少许额外的存储效率。

这种变长更新速率会导致一个明显的结果，那就是要求物理处理函数能同样应付变长更新速率。直到此时，所有的代码都认为自己是在固定速度下面运行的，而我们也调整了相应的物理常数以针对这个给定的速度得到较好的结果。使用了变长更新速率以后，我们就需要在各种方程中把给定周期考虑进去以得到正确的物理行为。不过，使用变长更新速率的目的之一就是要得到一个适当的更新速率，这样就可以确保在更新时间间隔之内其行为相对稳定了。

有了这样的假设以后，集成就容易多了。我们只需要把指定的行为流速或是速率与上次更新之间的时间相乘就行了。这会带来一点点额外的开销，但是由于每秒更新的单元数大大减少了，因此处理时间与要求的存储带宽会相应减少，反而更高效。

当其数学运算比较简单的时候，在很多种情况下进行这种集成会更容易一些。这样，其带来的额外的精确度就可以使得更新速率变得更低，使得速度更为改善。

当相邻的单元更新速度差异极大的时候，使用变长更新速率可能会带来意想不到的错误。由于某个单元比相邻的单元更新得远为频繁，这样在其他的单元进行反映之前它又会发生变化了。其解决方法就是限制相邻单元的更新速率之差。每当处理一个单元的时候，除了改变相邻单元的温度、热量以及其他信息以外，还必须确保相邻单元的更新速度至少要是自己更新速度的 1/4。我们纯粹是通过实验得到的这个上限值。使用 2 作为上限值将导致过多的单元毫无意义地被相邻的事件所影响，而实际上对于它们来说，可能根本没有任何值得进行处理的事情。如果使用 8 或是更大的数值作为上限值，则会导致较多可能的错误，而又没有极大地减少处理上的开销。对于通过实验找到的这些任意上限值来说，你应该在自己的游戏里进行试验得到最佳的数值。

2.6.11 结论

使用 CA 方法可以得到在当今游戏里面很少出现的许多真实世界中的效果及场景。通过

Download at www.Pin5i.Com

它们，不需要受到预先脚本的限制，玩家就可以完全地、灵活地，而且符合逻辑地与之交互了。通过它们，我们将得到更多的创新、更多的玩家参与、更大的实验自由度、更真实的渲染效果，最终也就是在游戏世界里面有更好的沉浸感，让其更加真实，而不仅仅是一堆多边形实体的堆砌。

2.6.12 参考文献

[CFD] 对于计算流体动力学这个领域来说有数不清的参考文献。不幸的是，大部分文献需要研究生级的物理知识，或是对于 3D 微积分要有非常好的基础。在这些文献之中，综合性较强的是 http://www.efunda.com/formulae/fluids/overview.cfm.

[Conway] http://www.dmoz.corg/Computers/Artificial_List/Cellular_Automata/Conway's_Game of_ Life/.

[Hemispherical01] "Advanced Shading and Lighting," presentation at Meltown 2001: pp. 22 ～ 35. 对应网址为 *http://www.microsoft.com/mscorp/corpevents/meltdown2001/ppt/DXGLighting.ppt*.

[LorensenClien87] Lorensen, W.E. and Cline, H. E., "Marching Cubes: A High Resolution 3D Surface Reconstruction Algorithm," *Computer Graphics Proceedings*(SIGGRAPH 1987), Vol.21, No.4: pp.163～169.

[RedFaction] *Red Faction*, 由 Vlotition, Inc.开发，由 THQ, Inc.发布, 2000.

[Ulrich00] Ulrich, Thatcher, "Loose Octrees," *Game Programming Gems*, Charles River Media, Inc., 2000.

[Xcom] *X-Com, UFO Defense*(U.S) or *X-Com: Enemy Unknown*(U.K), Microprose, 1994, http://www.codogames.com/UFOUnknownDefense.htm.

Download at www.Pin5i.Com

2.7　在动态仿真中处理摩擦

Miguel Gomez
kikomu@seanet.com

你现在正处在一个运动场上，手上拿着一块木头，你站在一个斜坡的底部，将这块木头尽力地推上坡去。木块滑到了斜坡的中间，转了一个方向，接着就滑下斜坡的底部了。

你将木块放在处于平衡状态的杠杆支点上，抓住其一端，当你慢慢地将杠杆倾斜起来的时候，开始木块还能保持静止，接着就开始滑下杠杆了。随着你把斜度增大，木块滑行的速度也越来越大，最终将滑到你的脚下。

你的摩擦物理模型能正确地处理以上的各个情况么？如果能完成，除此之外，它在进行处理工作的时候需不需要使用各种奇怪的数值以及进行误差检验呢？

摩擦力的库仑模型（Coulomb model）是机械系统的仿真的一个重要方面。不过，标准的数值计算模式是不行的，因为对于速度而言，摩擦力是一个离散量。本节将采用一些简单的一维示例提供一个对摩擦力库仑模型的直观阐述，并说明在仿真摩擦系统的时候会出现的数值计算问题。我们将提出一个三维情况下的公式，并提出一个简单的数值方法用来计算一个不可旋转的物体在一个常数坡度表面上滑动与粘连产生的轨迹。此外，我们还探讨了将此方法扩展到曲面和多边形表面时需要注意的重要事项。

2.7.1　库仑摩擦力

有两种库仑摩擦力：一种是滑动摩擦力（或运动摩擦力），它是作用在有相对运动的两个物体表面之间的；另一种是静摩擦力（也称为粘滞力），它的方向是物体表面的切向，位置在接触面的静态区域上，其大小等于作用在物体上欲使其运动的力。下面的例子描述了静摩擦和滑动摩擦之间的本质区别。

1. 一个存在滑动摩擦的系统

在我们的第一个例子中，假设有一个木块放在一个平面上。在时间 $t=0$ 的时候，它的初始速度是 $v_0>0$，其方向是延 x 轴的正方向，如图 2.7.1 所示（请注意，一般说来，速度是一个矢量。但由于我们处理的是一维的情况，因此标量也是可以的）。实验结果表明此木块的速度将以常数率递减，直到最终在某个时刻 t_s 静止下来，此后它就将保持静止了。此外，减速的速率与

Download at www.Pin5i.Com

物体的质量 m 是独立的，与重力加速度 g 成正比。如果我们的木块其初始速度 $v_0<0$ 是延 x 轴负方向的，摩擦力也同样将使其减速，直至最后使其停止。我们可以得出结论，那就是摩擦力总是与运动速度 v 的方向相反，而且当 $v=0$ 的时候摩擦力将不发生作用。这就暗示了，在一维系统，正确的滑动摩擦力公式是如下所示的：

$$f_d = \begin{cases} -\operatorname{sgn}(v)\mu_d N & \text{when } v \neq 0 \\ 0 & \text{when } v = 0, \end{cases} \tag{2.7.1}$$

其中μ_d是滑动摩擦系数，N 是法向压力的大小。这个力是表面对物体在法线方向产生的力（在这个例子中是延 z 的正方向）。方程 2.7.1 组成了滑动库仑摩擦力[Stewart00]的一个一维公式。虽然我们的木块是仅存在于 x 和 z 方向上的，但是摩擦力仅仅是用一个标量值表示的，这样我们在这个简单的例子中就可以将摩擦力看成是一维的力了。

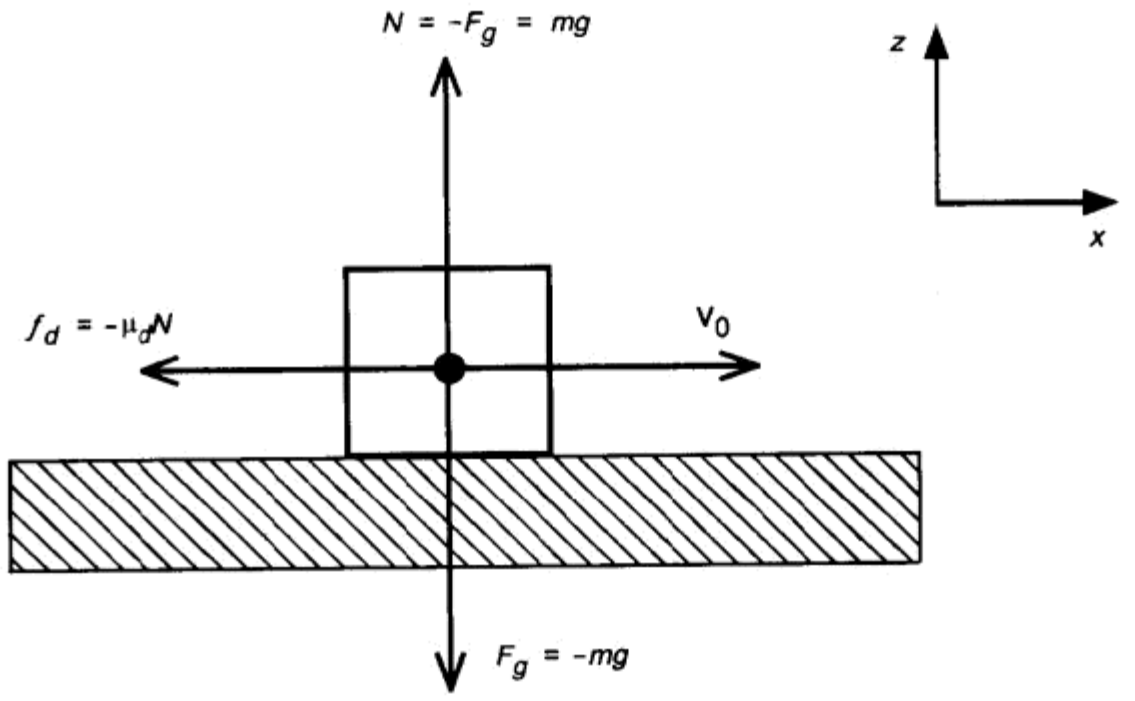


图 2.7.1 在平面上运动的木块将均匀减速，直至停止。这个减速度正比于重力加速度与滑动摩擦系数μ_d

在此例中我们假设木块在 z 方向上没有运动。因此，在此方向上最终的合力是等于零的，且 N 必须在大小上等于重力 mg，但是在方向上应该刚好相反。根据牛顿第二定律，我们知道 $x(t)$，这个以时间 t 为变量表示的木块在 x 方向上的坐标满足以下微分方程：

$$\frac{dx(t)}{dt} = v(t) \tag{2.7.2a}$$

$$\frac{d^2x(t)}{dt^2} = \frac{dv(t)}{dt} = -\operatorname{sgn}(v(t))\mu_d g \tag{2.7.2b}$$

再次假设 $v_0>0$。由于在停止之前 v 的符号不会改变，因此 sgn(v)就可以被忽略了，这样我们就可以使用方程 2.7.2b 直接得到一个 $v(t)$的显式表达式：

$$\int_{v_0}^{v(t)} dv = -\int_0^t \mu_d g \, dt \Rightarrow v(t) = v_0 - \mu_d g t. \tag{2.7.3}$$

如果木块初始的 x 坐标 $x_0=0$，那么再代入方程 3 我们就可以得到：

Download at www.Pin5i.Com

$$\int_0^{x(t)} dx = \int_0^t v_0 - \mu_d g t \, dt \Rightarrow x(t) = v_0 t - \frac{1}{2}\mu_d g t^2. \tag{2.7.4}$$

这样我们就可以得到木块停止的精确时间和位置，它们分别是：

$$t_s = \frac{v_0}{\mu_d g} \tag{2.7.5a}$$

与

$$x_s = x(t_s) = \frac{1}{2}\frac{v_0^2}{\mu_d g} \tag{2.7.5b}$$

对于所有 $t \geq t_s$ 而言，木块都是处在停止状态的，换句话说，其解如下所示：

$$x(t) \equiv x_s, \text{ and } v(t) \equiv 0. \tag{2.7.6a, b}$$

木块随时间变化的位置、速度和加速度都在图 2.7.2 里面显示了出来。请注意，在 t_s 处，木块的加速度有一个跳变导致的不连续点（一个数值的瞬变）。在我们开发计算木块轨迹的方法时必须考虑到这个不连续点。

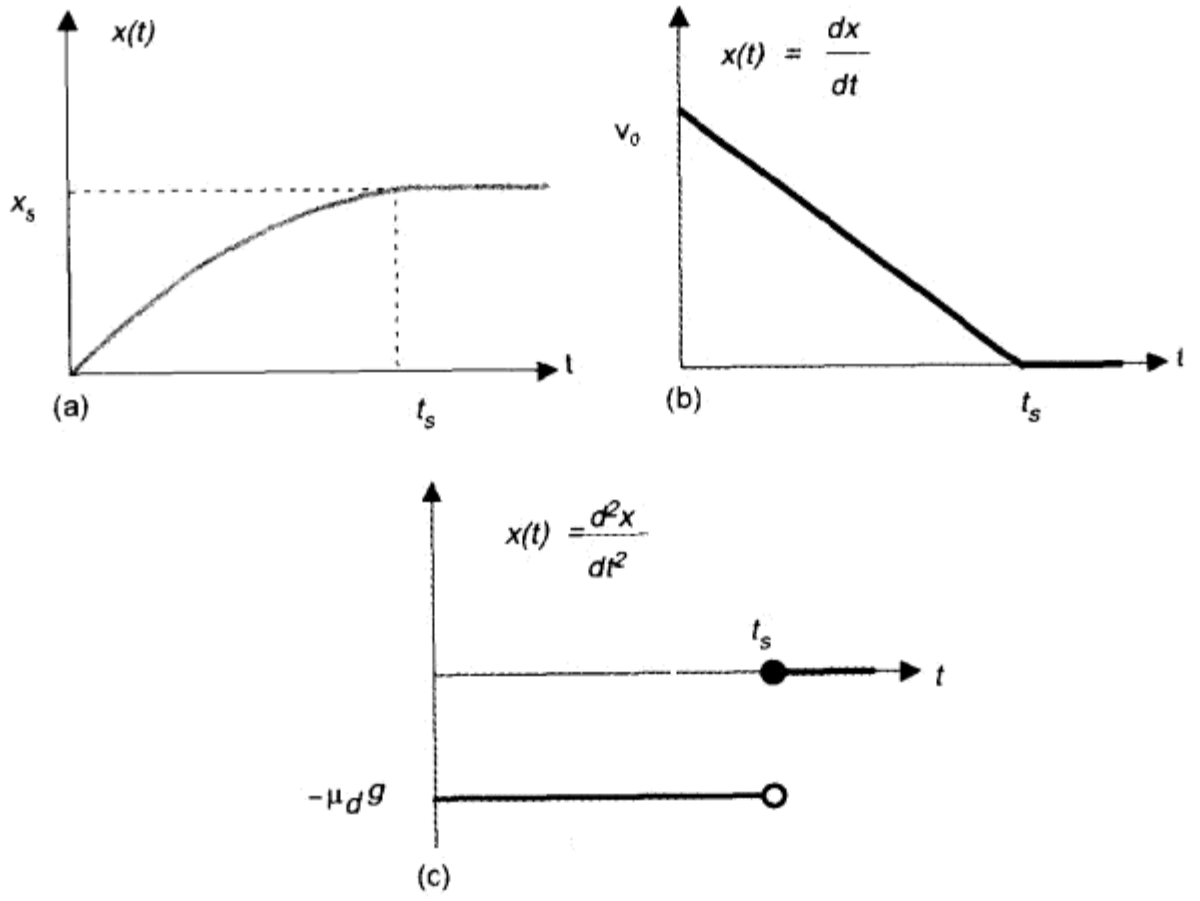


图 2.7.2　一个木块在平面上滑动，受摩擦力作用下的位置（a），速度（b）与加速度（c）的曲线。在时间点 t_s 上，速度变为 0 的同时加速度立刻为 0

摩擦系数是由表面的类型以及接触面的其他条件决定的，请参见[Beer92]，在其中你可以得到一些常见表面的静摩擦系数。

2. 一个存在静摩擦力的系统

假设现在我们的木块放置在一个具有倾角 $\theta > 0$ 的平台上面，为了简单考虑，令 x 轴与平台的表面平行，z 轴则是平台的法向。假设木块初始是静止的，如图 2.7.3 所示。对于大部

Download at www.Pin5i.com

分（如果不是全部的话）表面来说，都存在一个角度$\phi_s>0$，如果倾角小于此角度，木块将保持静止；如果大于此角度则将开始滑动。因此，你可以把静摩擦力看成是一种约束力，它必须满足：

$$f_s \leqslant \mu_s N, \tag{2.7.7}$$

其中μ_s是静摩擦力系数。

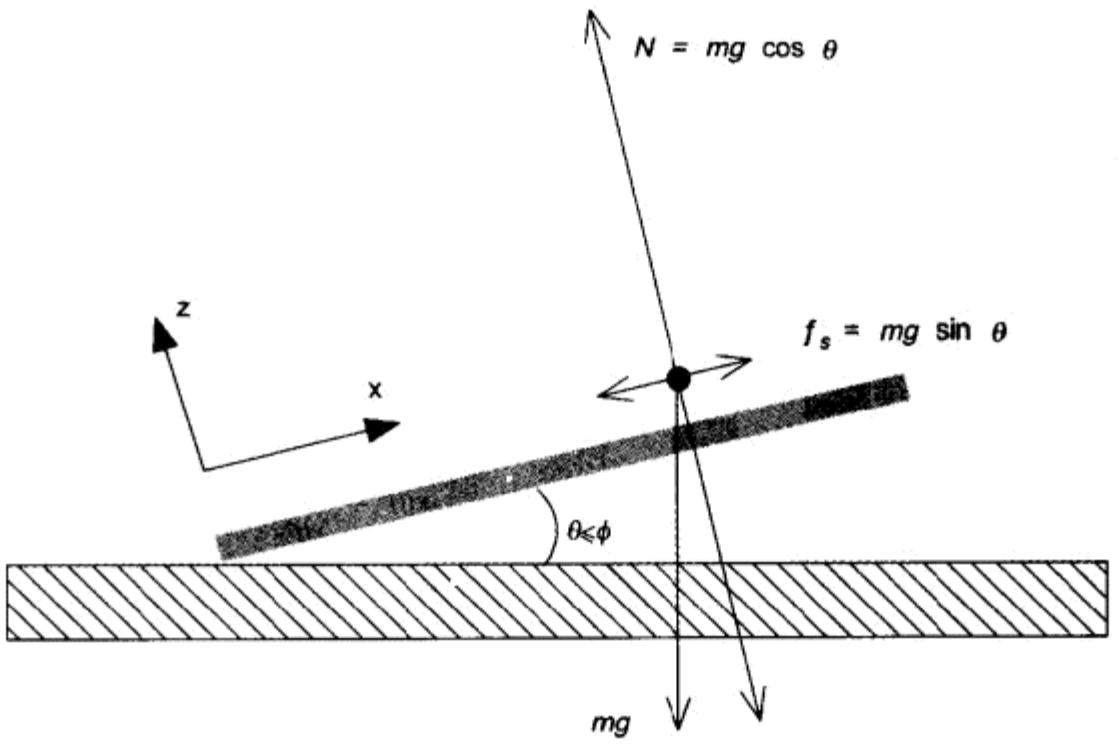


图 2.7.3 静摩擦防止木块滑下斜坡

首先，假设平台的坡度为$\theta < \phi_s$，这样模块就将是静止的了。这就是说摩擦力大小上等于重力在 x 方向上的分量，但是方向相反。当$\theta = \phi_s$ 的时候，静摩擦力将达到其最大值（[Pfeiffer92]的作者称这种状态为“摩擦饱和”），这样就有：

$$f_s = \mu_s N. \tag{2.7.8}$$

这就意味着下面的关系成立：

$$\frac{f}{N} = \frac{mg\sin\phi_s}{mg\cos\phi_s} \Rightarrow \mu_s = \tan\phi_s. \tag{2.7.9}$$

如果平台的倾角$\theta > \phi_s$，那么阻止运动所需要的力就会超过静摩擦的最大值，这样木块就开始滑动了。

3. 滑动摩擦与静摩擦之间的转化

现在，假设木块正在以 $v_0>0$ 的初速度延斜坡上行。基于以上的推理，如果斜坡的倾角$\theta \leqslant \phi_s$，那么木块将滑上斜坡，以一个常数率减速直至最后停止保持静止状态；从另一方面来说，如果$\theta > \phi_s$，那么静摩擦力将小于重力的分量，因此木块将滑到一个最大高度，然后又滑下来。

同时请注意，如果$\mu_d < \tan\phi < \mu_s$，那么木块停止的时候将是一个不稳定状态，即使是极

Download at www.Pin5i.Com

小的动作都将使得木块开始滑动。

2.7.2　数值方法

我们希望在一个实际的仿真中实现上述概念。在给定了一个滑行在表面上的物体之后，我们必须找出某些方法来计算它在每一帧中的位置以将之显示出来。

好像看上去我们可以直接使用上面推导出来的 $x(t)$的显式公式来计算在给定显示时刻我们想得到的位置。不幸的是，这种方法只对平面适用，对一个更通用的曲面来说，我们是找不到一个显式公式的。因此我们必须使用逐帧计算法来得到在下一个时间段中的位置。

首先，我们将实现一个最简单的摩擦处理，然后看看在实现游戏代码的时候，为何加速度的不连续性会成为一个问题。接下来，我们将对自己的摩擦力处理模型稍作修改以解决加速度的问题。最后，我们将提出一个更好的全面数学模型以解决这些问题。

1. 第一个方法：欧拉法

让我们来考虑一下第一个例子：一个木块在一个平面上滑动的例子。对于时间 t，如果已知了位置 $x(t)$与速度 $v(t)$，那么我们就可以计算出下一个时刻 $t+h$ 的位置与速度了，下面就是计算方法：

$$x(t+h)=x(t)+hv(t), \tag{2.7.10a}$$

$$v(t+h)=v(t)-h\operatorname{sgn}(v(t))\mu_d g \tag{2.7.10b}$$

这个直观的计算法被称为欧拉法[Gerald97]。由于它的简单和计算速度很快，因此很多游戏都使用了这种方法。

令 x_0=0，v_0=1，m=1，μ_d=1，g=9.81 以及 h=0.02。我们可以得到一个精确解，那就是木块将在时间 t_s=0.10194s 之后停止在位置 x_s=0.05097m 之处。表 2.71 中给出了欧拉法对于这个初始值的一组解：

表 2.7.1　　对于平面上的滑行木块由欧拉法得到的位置与速度解

t	$x(t)$	$v(t)$
0.00	0.00000	1.0000
0.02	0.02000	0.8038
0.04	0.03608	0.6076
0.06	0.04823	0.4114
0.08	0.05646	0.2152
0.10	**0.06076**	**0.0190**
0.12	**0.06114**	**- 0.1772**
0.14	**0.05760**	**0.0190**
0.16	**0.05798**	**- 0.1772**
0.18	**0.05443**	**0.0190**

注意：此数值将永不收敛，这是由于在 v=0 的时候加速度的不连续性导致的。

数值计算的结果表明木块在 0.1s 以后在 0.061m 的地方几乎就要停止了。由于欧拉法的精确度是一阶的，因此我们会得到一些误差。最要紧的不仅仅是 v 永不收敛到 0，还有就是

Download at www.Pin5i.Com

它会围绕 0 作不对称的振荡，这样实际上在 $t>t_s$ 之后 x 就会以常速率递减了。这样在动画中就会显示出木块来回在原点的附近滑动！

可能你会想，只要当 v 小于某个门限值 $v_c>0$ 的时候就让其等于 0，这个问题不就解决了么？不幸的是，如果使用了这种奇怪的数值使一个特定的系统表现正确是一个没有规律的事情。在一个牵涉到不同质量的数个物体与摩擦系数的仿真中，要找出其门限值是非常费时费力，需要经过很多测试。实际上，这需要对每个物体在每个可能运动的表面类型上的摩擦力进行测试。这样我们只是解决了一个问题，同时又引入了另一个更麻烦的问题而已。我们在下面就会发现，在处理滑动摩擦力与静摩擦力转换的问题上，速度门限值常会带来更大的麻烦。

2. 第二个方法：重新计算摩擦力

再回想一下，欧拉法其实对于这种问题是不适合的。当 v 很小的时候，我们的计算方法将导致 v 超过 0 并且改变符号。这样接下来加速度也改变了方向，导致 v 又一次变向。由于 v 围绕零做不对称的振荡，这样每两步位置就会向零逼近一点。

我们不会改变数值计算法，而是要对公式进行修改，这样在 v=0 的时候摩擦力就连续了。一个方法就是利用粘滞阻尼（viscous damping）计算来代替库仑公式：

$$f=-v\mu_d N. \tag{2.7.11}$$

不幸的是，这种法将使得木块在高速的时候减速得过快，而在低速的时候又衰减得过慢。使用此公式的动画效果将显得很不自然。

另一个方法就是对摩擦力进行分段处理（参见[Abadie00]或是[Stewart00]），这样就有：

$$f=\begin{cases}-\dfrac{v}{v_c}\mu_d N & \text{when}|v|\leqslant v_c \\ -\operatorname{sgn}(v)\mu_d N, & \text{when}|v|>v_c,\end{cases} \tag{2.7.12}$$

其中 v_c 是一个大于零的速度门限值。这个公式可以解决高速情况下可能发生的反常现象，但是仍然无法阻止低速情况下粘滞阻尼现象的发生。此外，v_c 又是一个奇怪的数值，对每个特定的系统都需要进行调整。更重要的是，这些替换公式在进行滑动摩擦力与静摩擦力之间的转换时（不管是从谁转到谁）都将引起一些问题。

3. 第三个方法：处理不连续性

如果我们知道问题的所在，那么还是可以找到一个工作良好的数值方法的。在我们例子的问题中，轨迹 $x(t)$有两阶的连续度（如果一个函数 $x(t)$的一阶导数连续且处处存在，则我们称此函数具有连续度）：

$$x(t)=\begin{cases}v_0 t-\dfrac{1}{2}\operatorname{sgn}(v)\mu_d g t^2 & ,\text{when } t<t_s \\ x_s & ,\text{when } t\geqslant t_s\end{cases} \tag{2.7.13}$$

函数 $x(t)$以及导数 $v(t)$在点 t_s 都是连续的，但是 $v(t)$的一阶导数加速度 $a(t)$则是不连续的。通过欧拉法与其他高阶的方法都假设函数 $x(t)$在我们需要处理的这一段区域之内有一个泰勒展开[Gerald97]。但是此系统在 t 的一个邻域内却不满足这个条件，在此域之内 v 趋向于零而导致了不连续性的发生。因此，任何一个假设 $x(t)$有一个泰勒展开的方法都不会收敛到正确

Download at www.Pin5i.Com

的解上来。

假设当前的时间 $t<t_s$，且我们希望计算出下一个时间 $t+h<t_s$ 之处的位置。在这个时间段上，$x(t)$是有一个泰勒展开的，这样我们就可以以 h 为参量对 $x(t+h)$在 t 点进行展开了，可以得到：

$$x(t+h)=x(t)+h\frac{dx(t)}{dt}+\frac{1}{2}h^2\frac{d^2x(t)}{dt^2}=x(t)+hv(t)-\frac{1}{2}h^2\,\mathrm{sgn}(v)\mu_d g. \tag{2.7.14}$$

$v(t+h)$的泰勒展开则是：$v(t+h)=v(t)+h\dfrac{dv(t)}{dt}=v(t)-h\,\mathrm{sgn}(v)\mu_d g.$ （2.7.15）

将两个方程合并起来，我们就得到了一个泰勒法[Gerald97]。假设我们计算的是[t, $t+h$]这一段上面的值，而且 $t+h<t_s$，那么对于我们的木块在任意倾角的平面上滑动的情况来说，此方法得到的解就是精确的了。

在任一个区域内，如果下式成立，则 v 就将趋向于零：

$$h_s=\frac{v}{\mathrm{sgn}(v)\mu_d g} \tag{2.7.16}$$

满足

$$0\leqslant b_s\leqslant b \tag{2.7.17}$$

如果在某一步上 h_s 满足了以上的不等式，那么我们就必须将 v=0 了，而且可以将 $x(t+h_s)$设置为停止的位置。一般来说，如果木块的加速度为 $a(t)$，我们的方法将得到以下的形式：

$$x(t+h)=x(t)+hv(t)+\frac{1}{2}h^2a(t) \tag{2.7.18a}$$

$$v(t+h)=v(t)+ha(t) \tag{2.7.18b}$$

在每一步中，我们都将检查一下是否 h_s 满足：

$$0\leqslant h_s=-\frac{v(t)}{a(t)}\leqslant h \tag{2.7.19}$$

把此方法应用到我们的测试系统上以后，最后的结果很令人满意。实际上，从表 2.7.2 中可以看出 t_s 与 x_s 都是精确的，而且更重要的是，当 $t\geqslant t_s$ 的时候 $v(t)\equiv 0$。

表 2.7.2　　使用泰勒法计算得到的位置与速度

t	$x(t)$	$v(t)$
0.00000	0.00000	1.0000
0.02000	0.01804	0.8038
0.04000	0.03215	0.6076
0.06000	0.04234	0.4114
0.08000	0.04861	0.2152
0.10000	0.05095	0.0190
0.10194	**0.05097**	**0.0000**
0.12194	0.05097	0.0000

注意：使用泰勒法对本测试系统计算的位置与速度都是精确值。通过对中间时间值 h_s 的计算，我们就可以确保速度变成零，从而静止下来了。

Download at www.Pin5i.Com

4. 在静摩擦力与滑动摩擦力之间的转换

前面的例子只能处理滑动摩擦力。在斜坡上，我们必须判断出是否会发生静摩擦力与滑动摩擦力之间的转换。当 $v\neq0$ 的时候，按定义来说摩擦力是滑动摩擦，因此我们只考虑 $v=0$ 的转换情况。

请再回想一下图 2.7.3 中的例子。对于倾角$\theta > \phi$而言，木块将减速直至达到一个顶点，然后又会滑下来。但是对于$\theta \leqslant \phi$的情况而言，它将停止在顶点处保持静止状态。如果木块处于静止状态，而其上所有的力是随时间变化的，那么我们就必须定期对之进行检查看看静摩擦力是否可以抵消切向力以保持木块的静止状态。换句话来说，如果切向的力不满足下面的不等式：

$$-\mu_s N \leqslant F_t \leqslant \mu_s N; \tag{2.7.20}$$

那么木块就会开始滑动了。

然而，在此处有一个重要的细节问题不能被忽视。当木块从静摩擦力转移到滑动摩擦力作用的时候，它的速度是零，因此滑动摩擦力的方向无法确定。由于其趋向的运动方向与 F_t 一致，因此滑动摩擦力的方向就应该与 F_t 相反。

现在就很容易看出在从静摩擦力转换到滑动摩擦力的时候，为什么速度门限值会引起问题了。在滑动摩擦力的第一次作用中，速度将从 0 转换为某值 $v(t+h)$。如果$|v(t+h)|$小于我们允许的最小速度值 v_c，那么 v 就会又变成 0。然而，由于 $a(t)\neq0$，因此 $x(t+h)$就会继续增长。如果 v 一直达不到 v_c 以上，则木块将会慢慢地以一个常速度滑下斜坡。对于任何θ 、a 与 v_c 的组合来说，都存在一个最短的时间间隔 $h_{\min}$，在此时间间隔之下木块就会发生这种运动。

2.7.3 一个三维公式

现在我们可以得到三维空间里面的摩擦力公式：

$$\mathbf{f}_d = \begin{cases} -\mu_d N\hat{\mathbf{v}} & \text{when } \mathbf{v}\neq 0 \\ -\mu_d N\hat{\mathbf{F}}_t & \text{when } \mathbf{v}=0, \end{cases} \tag{2.7.21}$$

在其中 $\hat{F}_t$ 是作用在物体上面力的切向分量。此公式在从静摩擦转换到滑动摩擦的时候也是成立的。摩擦力 f_d 现在是一个矢量。我们假设速度没有法向分量，这样单位矢量 $\hat{\mathbf{v}}$ 就会与接触面相切。法向力为矢量 $\mathbf{N}=N\mathbf{n}$，其中 $\mathbf{n}$ 是单位法向矢量。

当 $v=0$ 的时候，静摩擦力将试图抵消切向力，而且其大小必须满足：

$$f_s = \|\mathbf{f}_s\| \leqslant \mu_s N. \tag{2.7.22}$$

以上所用的插值泰勒方法仍然适用，前提条件是我们能精确地预测出在计算的时间间隔里面 v 会不会变成零。

由于三维问题具有非线性的性质，因此我们不可能得到 h_s 的精确解，这样我们就必须找到一种方法能对此值进行估计。速度大小的平方随时间的变化率为：

Download at www.Pin5i.Com

$$\frac{d}{dt}\|\mathbf{v}(t)\|^2 = \frac{d}{dt}(\mathbf{v}\cdot\mathbf{v}) = 2\mathbf{v}\cdot\frac{d\mathbf{v}}{dt} = 2\mathbf{a}\cdot\mathbf{v}. \tag{2.7.23}$$

如果我们假设加速度 a 以及速度的方向在$[t, t+h]$这一段上面是保持不变的，那么就只有速度的大小会随时间改变了。我们可以使用方程 2.7.23 得到 h_s：

$$\int_t^{t+h} 2\mathbf{a}\cdot\mathbf{v}dt = 2\mathbf{a}\cdot\int_t^{t+h}\mathbf{v}dt = 2\mathbf{a}\cdot(x(t+h)-x(t)) = 2\mathbf{a}\cdot\Delta\mathbf{x}. \tag{2.7.24}$$

由于我们假设 v 的方向是一定的，而且其大小是按常数率递减的，这样就可以得到：

$$2\mathbf{a}\cdot\Delta\mathbf{x} = 2\mathbf{a}\cdot\frac{1}{2}\mathbf{v}h_s = \mathbf{a}\cdot vh_s. \tag{2.7.25}$$

如果变化率是一个负数，那么我们将得到：

$$h_s = -\frac{\mathbf{v}\cdot\mathbf{v}}{\mathbf{a}\cdot\mathbf{v}}. \tag{2.7.26}$$

当 **v** 与 **a** 在一个方向上面的时候此公式就将退化为一维情况下的公式了。

2.7.4　几何图形问题

1. 连续度

表现一个二维表面的常用方法是使用多边形网格。另一个方法是使用高程场（heightfield），在其中一个面的高程将被存储在一个定长分布的网格中。不幸的是，如果没有处理好连续性坡度的话，这两种方法都将导致收敛性的问题。

让我们来考虑一下图 2.7.4 中所示的例子。当木块与斜坡碰撞的时候，其引力将立刻改变，从只有法向分量变化到出现一个与摩擦力相反方向的分量，而在合力作用之下总的加速度将发生一个跳变，产生不连续性。如果这个问题没有被很好地处理，就会导致产生此处将要阐述的不收敛性的问题。

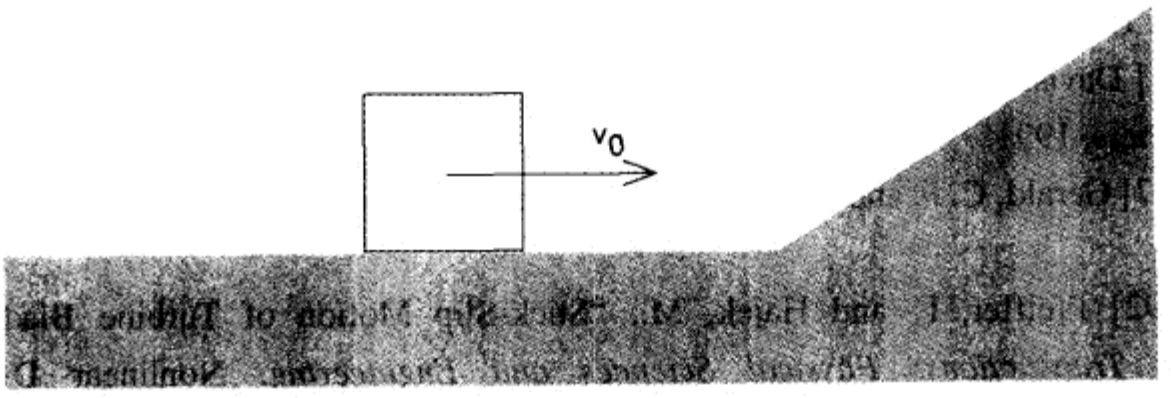


图 2.7.4　一个运动在平面上的木块在斜度突然改变的时候加速度会产生不连续点

一个补救方法就是在发生斜度变化的地方进行坐标与速度的插值。对于高程场来说，这是可以实现的，但是对于任意的多边形网格来说，它的效率可能会比较低。而且如果希望能使用通用的方法处理碰撞（这牵涉到速度的瞬时变化）的话，这种方法可能会带来一些问题。

另一个办法就是把表面连续化，这样它的方向导数就连续了（参见[Davis91]）。对于高程场来说，这需要延 x 方向和 y 方向进行二次样条函数的插值（参见[Watt00]）。对于任意的

Download at www.Pin5i.Com

多边形网格来说，我们需要使用子块划分模式。

最佳方法应该由应用程序的开发人员来决定，但是加速度的不连续性对于方法收敛性的影响必须要考虑到。

2. 曲率

虽然现在我们依据一个很好的模型来表达摩擦力在平面上的作用，但是如果想要把此模型扩展到曲面上，还是需要做很多工作的。如果你想对之进行扩展，请记住下一点：当曲面向上弯曲的时候，合力将使物体上升到斜坡上，而 v 也将有一个法向的分量。在处理这种情况的时候，只需要将物体的坐标重置一下，遵循前面的计算过程就可以将速度的法向分量给消去了。

不过，当曲面是向下弯曲的时候，我们就必须判断一下次物体是否会离开表面。虽然下面的方法不是很准确，但是你仍然可以使用一个门限值，将之与速度的法向分量比较。这或许是最通用的方法了。在使用这个方法的时候，你必须对门限值进行调整以得到预期的结果。

2.7.5 结论

我们通过一些简单的一维例子说明了摩擦力的库仑模型。此外，我们还探讨了某些数值方法不适用的原因，同时也开发了一个简单却有效的数值方法，你可以通过它来计算得到一个物体在常数坡度上滑行时的三维轨迹。现在，你就可以建构一个自己的运动场了！

2.7.6 参考文献

[Abadie00] Abadie, Michel, "Dynamic Simulation of Rigid Bodies: Modeling of Frictional Contact," *Impacts in Mechanical Systems: Analysis and Modeling*, SpringerVerlag, 2000.

[Beer62] Beer, F. P. and Johnston, E. R., Jr., *Mechanics for Enginerers: Statics and Dynamics*, McGraw-Hill, 1962.

[Davis91] Davis, H. F. and Snider, A.D, *Introduction to Vector Analysis*, 6th Edition, Wm. C. Brown Publishers, 1991.

[Gerald97] Gerald, C. F. and Wheatley, P.O., *Applied Numerical Analysis*, 6th Edition, Addison Wesley, 1997.

[Pfeiffer92] Pfeiffer, F. and Hajek, M., "Stick-Slip Motion of Turbine BladeDampers," *Philosophical Transactions: Physical Sciences and Engineering*, Nonlinear Dynamics of Engineering Systems, 1992: Vol. 338, No. 1651, pp. 503～517.

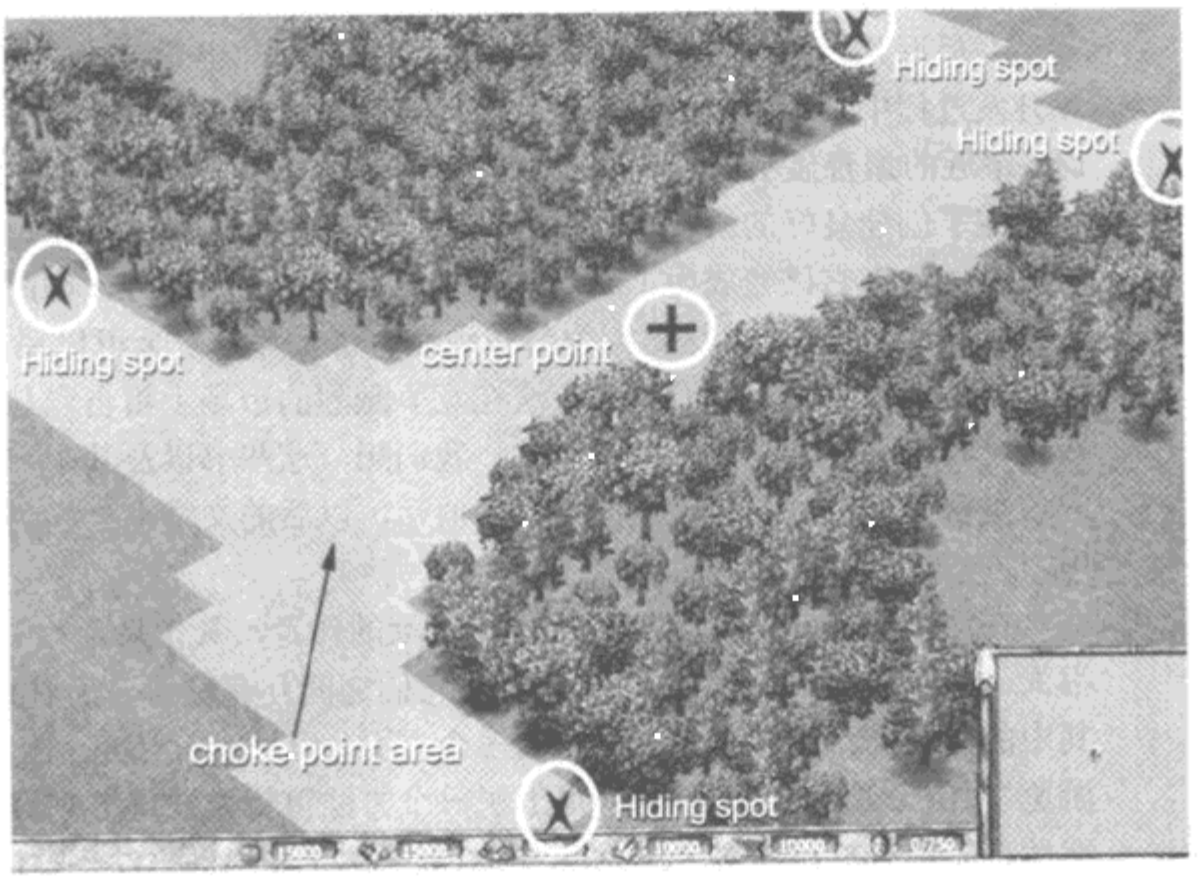
[Stewart00] Stewart, D., "Rigid-Body Dynamics with Friction and Impact," *SIAM Review*, 2000: Vol. 42, No. 1, pp.3～39.

[Watt00] Watt, A., *3D Computer Graphics*, 3rd Edition. Addison Wesley, 2000.

Download at www.Pin5i.Com

第 3 章

人工智能



Download at www.Pin5i.Com

简介

Steven WoodCock
Wyrd Wyrks
ferretman@gameai.com

各位读者，在此处我们为你准备了一道丰盛的大餐。

在《游戏编程精粹 3》的人工智能这一章里贡献给读者的文章是前一书《游戏编程精粹 2》中相应章节的优秀后继。它们全面地覆盖了一些有趣的 AI 中的主题，这些主题从如何避免硬编码来建构你的游戏以进行复杂的扩充，一直到长盛不衰的对游戏世界中人物的寻径进行改造，每篇文章都是作者从实践中得来的知识所提炼出的精华，而且其中大部分文章都是从作者最近作为一个游戏 AI 开发者的经验之中而来的。

在最近几年中，看到游戏 AI 的领域发展如此迅速，一直到其成为游戏开发过程中不可或缺的重要部分，对于开发人员来说这真是令人感到惊讶。最近的调查显示已经有很多时间、开发精力，以及 CPU 的时钟周期被花费在了创建良好的游戏 AI 上了。高质量的游戏 AI 再也不是皇帝女儿的陪嫁——现在它已经成为了游戏开发中至关重要的一部分，与图形或音效具有同样的地位。不管这是因为公司现在有了足够的资金可以写出好的 AI 程序，还是因为它们想通过此方法在竞争激烈的市场上将自己与类似的游戏区分开来——这总是一个好现象。我们想，这些书以及其中 AI 的文章应该是帮了一些忙的，而且如果幸运的话，以后的文章也会起到同样的作用。

在这里我们一共有 8 位为此作出贡献的作者，其作品覆盖了 AI 中的很大一部分领域。从表 3.0.1 可以看出它们之间互有覆盖，而且有所依赖，其中的几篇文章阐述了一个问题的多个方面，而且互有补充。例如，“矩形游览”（Board 和 Ducker 著）提出了一个方法可将游戏世界划分为区域，然后再利用分群技术对之进行游览。“快速游览网格的方法”（White 著）提及了类似的技术，不过他主要关注于对 A*算法的实现以及将世界中各个区域连接起来的门户。每个方法都有其优缺点，但是如果将之结合在一起，则它们阐述了一种可以处理含有巨型地图以及成百个人物游戏的重要方法。

Download at www.Pin5i.Com

表 3.0.1　　AI 的文章组织和应用

文　章	架　构	寻　径	战术式决策	地 形 分 析
经 GoCap 优化过的机器学习	Y		Y	
区域游览：对寻径模式的扩展		Y		
基于函数指针的内嵌式有限状态机（finite-state machine）	Y			Y
在 RTS 中的地形分析——一个隐藏的重要因素		Y	Y	Y
一个针对 AI 代理、对象，以及任务的可扩展触发器系统	Y			
基于 A*算法的战术式寻径		Y	Y	Y
快速游览网格的方法		Y		Y
在寻径与碰撞之间选择一种关系	Y	Y		

我们收集的文章表现出了这么一个显著的特点：游戏业界正在迅速地将注意力从所谓的“学院式”或者“高级的”AI 技术的试验上转移开来，而开始聚焦于那些经过实践而证明正确的原理上。在此处我们没有使用神经网络或是遗传算法，这些都是很有价值的工具，但只不过是还没有被当今的大多数开发人员使用上。不管是地形分析还是对寻径算法的补充，在此处呈现的文章都是基于任何一个游戏开发人员所熟悉的技术的——脚本、状态机，以及 A* 算法的寻径。第一眼看上去它们好像没什么了不起，但是一旦读者把每一篇文章都消化了之后，就很有可能会回问自己：“为什么我就没有想到这个方法呢？”

这并不是说游戏开发人员忽略了学术界和军方的技术——根本不是这样的。在游戏开发者大会上的会议和遍布于 Web 的各种讨论区都证明了游戏开发人员总是乐于学习新技术，而不管它们从何而来。唯一的区别就在于他们只有一堆可怜的硬件、一点紧巴巴的预算、一个紧张得不得了的开发进度，以及一些极其有限的资源。他们必须达到目标，不管这是否会意味着欺骗。因此，他们使用的技术必须能带来清晰而明显的好处，不然宁可不要使用它们——时间太紧张了。而学术界和军方一般都不会遇到这些限制，或者它们的目标与上述目标完全不同。

作为开发人员和编辑，我在此希望学术界、军方，还有游戏开发业界能继续互相学习并互相改进技术。正是在这种互相学习的精神中读者才能继续前行，所以请细心地阅读接下来的文章，同时也希望此书能指出一条到达此目标的道路。

Download at www.Pin5i.Com

3.1 经 GoCap 优化过的机器学习

Thor Alexander
Hard Coded Games
thor@hardcodedgames.com

机器学习是一个正在浮现中的技术，它将对未来的游戏开发产生深远的影响。站在产品的立场上来看，通过使用机器学习，我们就可以抛弃那些脆弱的、充满了特殊处理的 AI 算法了，而这些方法在当前的游戏中仍然被广泛使用着。通过使用对真人的观察来训练一个计算机控制的角色，一个专家级的玩家就可以极大地提升此角色表现出来的智能水平。游戏设计人员可以事先以各种角色的身份来玩游戏以表现出其性格特征，接下来这些资料就将被存储到库里面，以便日后能将这些信息导入各自的游戏中去，就如同使用传统的道具（content assets）一样。

本节展示的是一个 GoCap 的优化版，它是一种我们开发的通过观察来训练 AI 人物[Alexander02]的方法。你可以把它想成是“为了 AI 进行的动作捕获”。本节中的某些图例使用了 UML（统一建模语言）的术语。如果想要对 UML 有更深入的了解，请参阅[Booch98]。

3.1.1 GoCap 架构一览

在使用 GoCap 之前，我们需要对系统进行改造，要在一个真人玩游戏的时候记录其对系统的输入，然后在当前的仿真环境中把这些输入对应为玩家在游戏中选择执行的动作。如果想要使其具有实用价值的话，我们就需要在创建游戏的时候让一个真人的训练人员扮演游戏中的所有角色，包括敌人在内。要达到此目的，首先我们必须定义几个类以支持这种训练。

1. ActionState

`ActionState` 是有限状态机中的原子元素（atomic element）。它把自己对其他动作状态的合法转移都存储在了一个 `transitionList` 里面。`ActionState` 还维护了一个 `ActionRuleSet`，它列举出了所有使用此状态的规则，只有满足了这些规则，才能进入此状态。每个 `ActionRule` 都有一个自己的 `Evaluate()` 方法，此方法的作用是计算和返回某些与游戏相关的用来定义规则的值（参见图 3.1.1）。

Download at www.Pin5i.Com

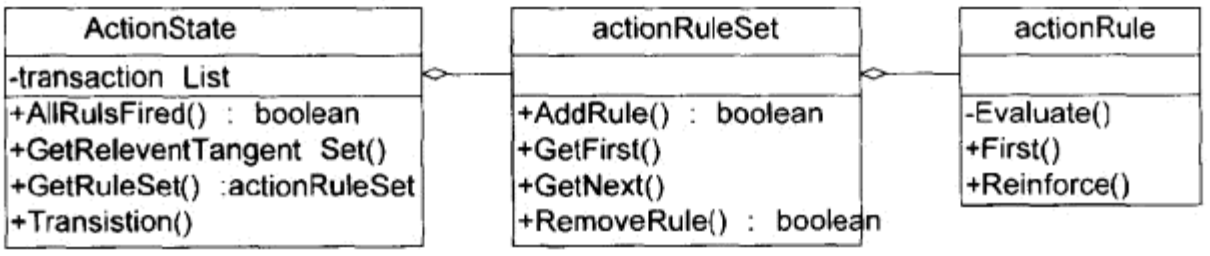


图 3.1.1　ActionState、actionRuleSet 与 actionRule 的类图

2. Actor 类

角色(actor)是可以与游戏中的环境进行交互的仿真对象。每个角色都有一个控制状态，它定义了当前角色是在玩家控制下，或是由 AI 控制的，抑或是在训练状态中。控制状态可以在运行时被修改，这样就可以转化为上述状态中的任意一种了。角色的 PerformAction() 方法用来代理对当前控制状态的处理，它决定了自己想要转移到的 ActionState。

3. ControlState

ControlState 定义了相关的角色可以从什么地方接受命令。在处于训练状态的时候，是由玩家来控制角色的，正如在用户控制态中一样。只有在此时，计算机才能进行观察，并从角色的动作中进行学习。

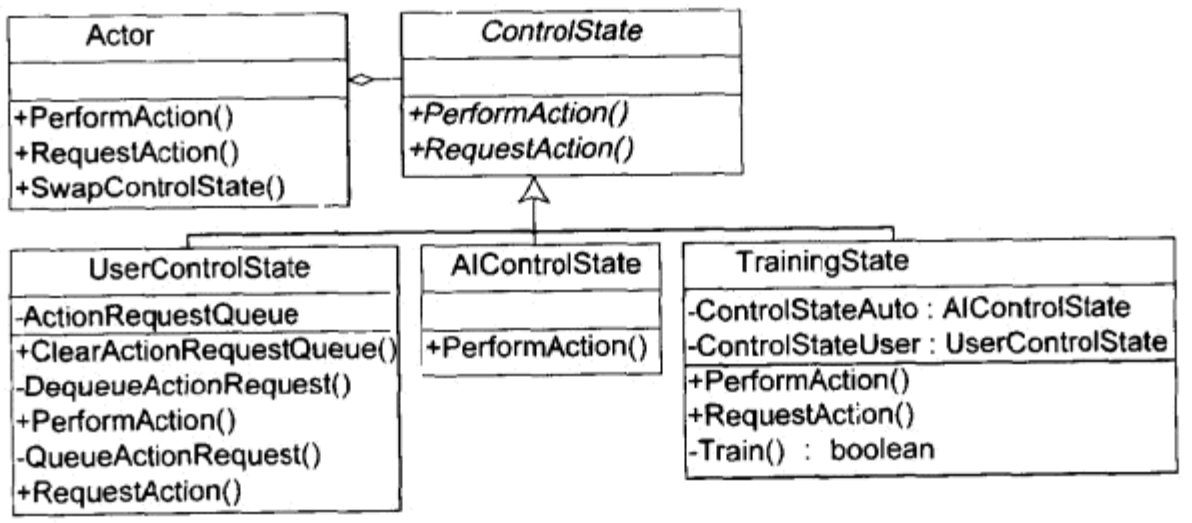


图 3.1.2　controlState 类的结构

4. UserControlState

当处在一个客户机/服务器环境下进行工作的时候，UserControlState 维护了一个队列，其中包含的是玩家发过来的尚未执行的动作。当此控制状态的 PerformAction() 方法被调用的时候，它就会从此队列中取出各个请求来。当训练环境是在本地，而没有使用服务器的时候，我们就可以使用一个即时请求处理的方法来取代这个队列了。

5. AIControlState

当处在此状态的角色调用 PerformAction() 方法的时候，由 AIControlState 来决定下面应该采取哪一个动作来执行。此处展示的例子实现了一个基于规则的 AI 决策系统，此系统使用了一个散列（hash）映射表进行规则的编码。

Download at www.Pin5i.Com

6. TrainControlState

TrainControlState 多加了一个私有的 Train()方法，它将使用此方法来学习在何种条件下需要转移到哪一个 ActionState 去。

3.1.2 训练开车

为了演示一下 GoCap 的工作方式，我们下面将要展示一个驾驶玩具车绕过一些障碍物的例子。在其中，小车将学习何时应该转弯以避开这些障碍物。

1. 设定动作状态

我们的玩具小车有 5 个基本的动作状态，它们定义了其可以进行的操作，还定义了这些操作之间转移的条件限制。表 3.1.1 列出了这些状态，而图 3.1.3 描述了它们之间的转移关系，用箭头表示的。

表 3.1.1　　小车驾驶的动作状态

动作状态	描　述
Stop	缺省状态，将小车从移动转为停止
MoveFwd	将小车朝面对的方向移动
TurnRight	将小车向右转弯
TurnLeft	将小车向左转弯
MoveBwd	将小车向后退

一旦我们定义好了这些状态，我们就需要知道它们之间的转移关系了，这样才能定义实现它们的类。图 3.1.4 显示了这些动作状态的类图，每个状态都可以被封装在它自己的 ActionState 实例里面。请注意从此实现可以得到一个包含状态之间合法转移的状态图，但是它并没有指出什么时候可以进行这些转移。

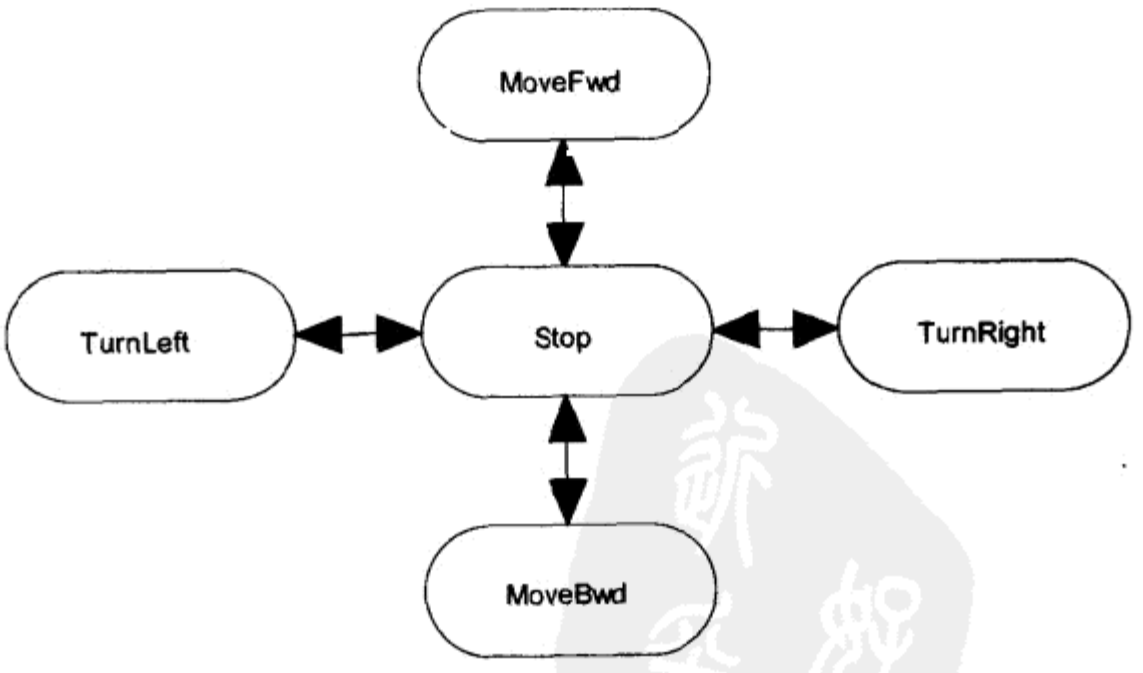


图 3.1.3　驾驶的动作状态及其之间的转移

Download at www.Pin5i.Com

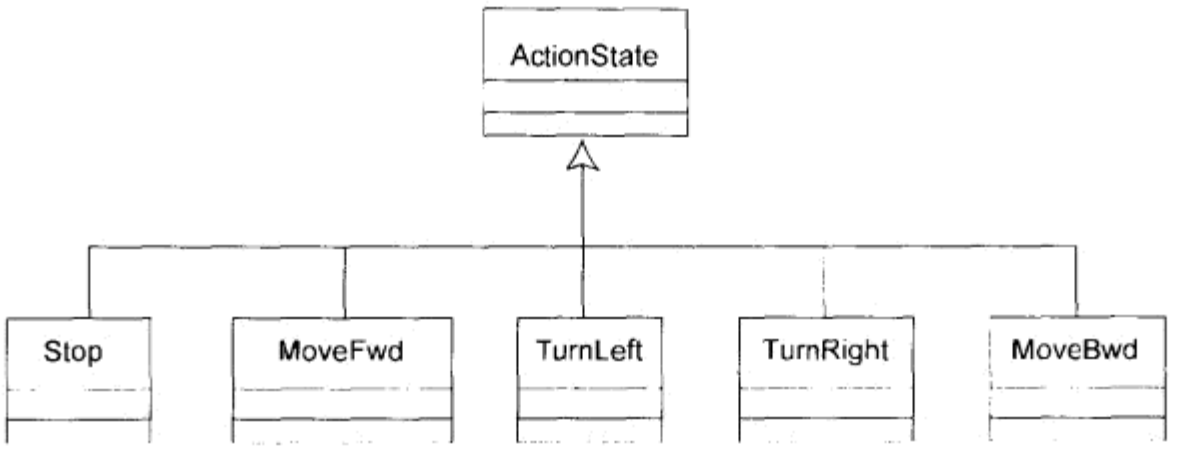


图 3.1.4　移动动作状态的类图

2. 设定规则

现在我们需要决定转移的规则了，这些规则详细说明了应该在什么时候我们的小车才能转移到这些动作状态下。每个 ActionState 都与一个 ActionRuleSet 对应，后者包含了我们可以使用此状态的规则（参见图 3.1.1）。每个 ActionRule 都有自己的 Evaluate() 方法用来计算和返回某些与游戏相关的值。对于我们的小车来说，我们将定义 6 个传感器以用来检测可能发生碰撞的邻域。每个传感器在实现的时候都将拥有一个 Evaluate() 方法，这些方法将以小车为原点发射一束光线，其方向就是传感器面对的朝向。如果传感器的光与一个障碍物发生了碰撞，那么此方法将计算出小车离障碍物有多远，然后返回一个以“1/距离”表示的相邻量，以确保此距离总是大于或等于 1 的。那么我们将在相邻量上得到一个近似的在 0.0 与 1.0 之间的浮点数。表 3.1.2 详细地描述了我们要使用的传感器。图 3.1.5 显示了这些传感器与小车结合的形式。

表 3.1.2　　碰撞检测的传感器

传　感　器	描　　述
FWD	对小车的正前方进行检测
FWD-L	对小车的左前方进行检测
FWD-R	对小车的右前方进行检测
LEFT	对小车的左方进行检测
RIGHT	对小车的右方进行检测
BWD	对小车的正后方进行检测

把 Evaluate() 方法封装到每条规则里面以后，我们现在就可以为每个 ActionState 创建 ActionRuleSet 了。图 3.1.6 描述了 TurnLeft 动作状态以及与其相关的 6 个传感器规则。图 3.1.7 显示了 TurnLeft 状态及其规则的类图。

3.1.3　学习规则

现在，我们已经做好了准备工作，可以来训练我们小车的规则了。要想达到此目的，其方法就是要把小车的角色变为训练控制状态，让一个真人玩家坐到驾驶员的位置上，正如此处描述的一样。下面我们提供了相应的伪代码以对此过程进行说明。

```
Actor.SwapControlState( TrainingControlState )
```

Download at www.Pin5i.Com

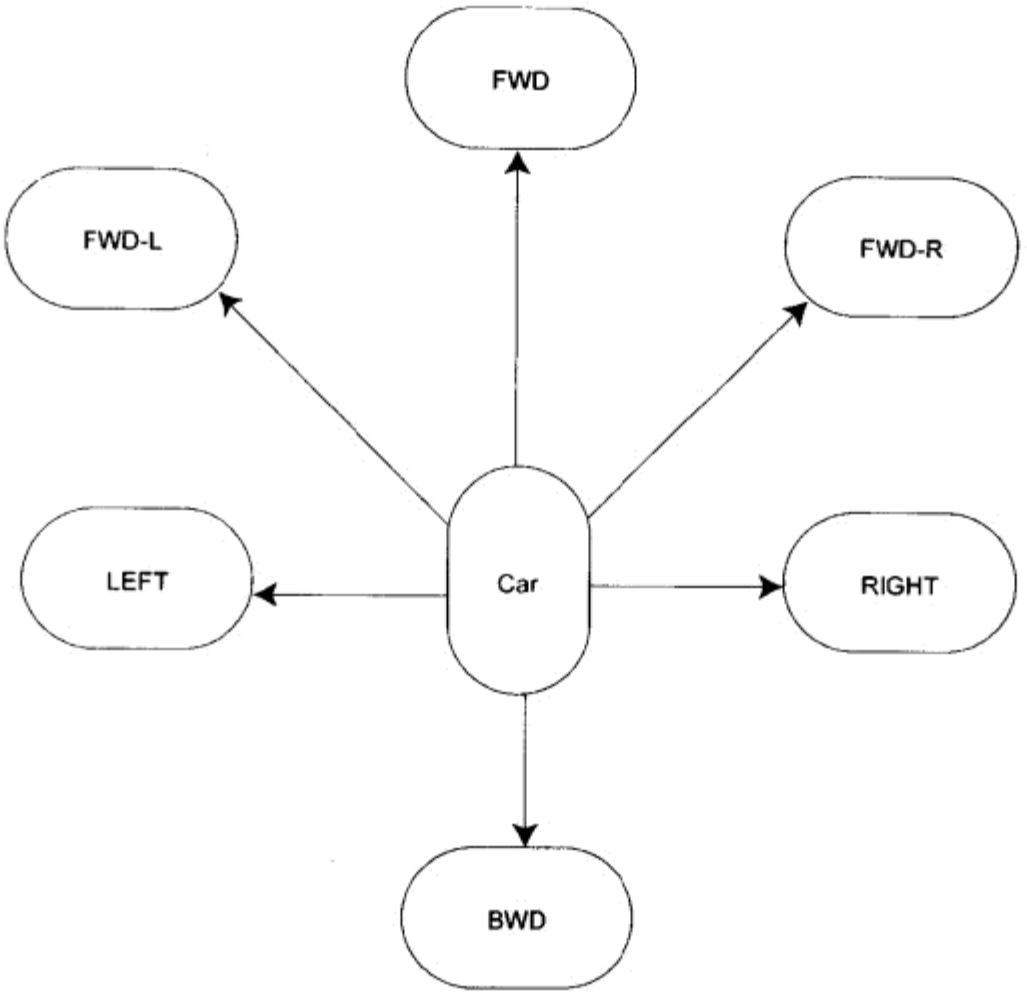


图 3.1.5 为小车加上传感器

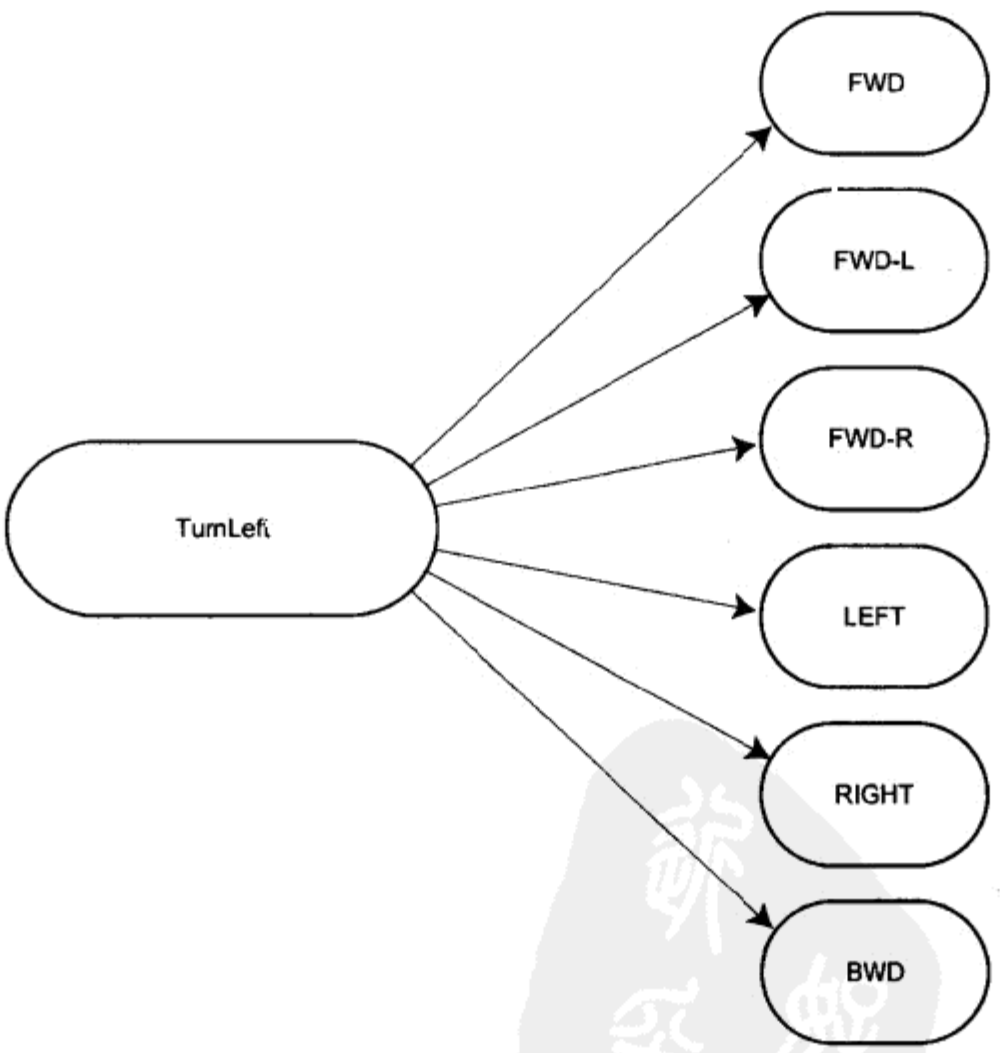


图 3.1.6 TurnLeft 行为的规则

Download at www.Pin5i.Com

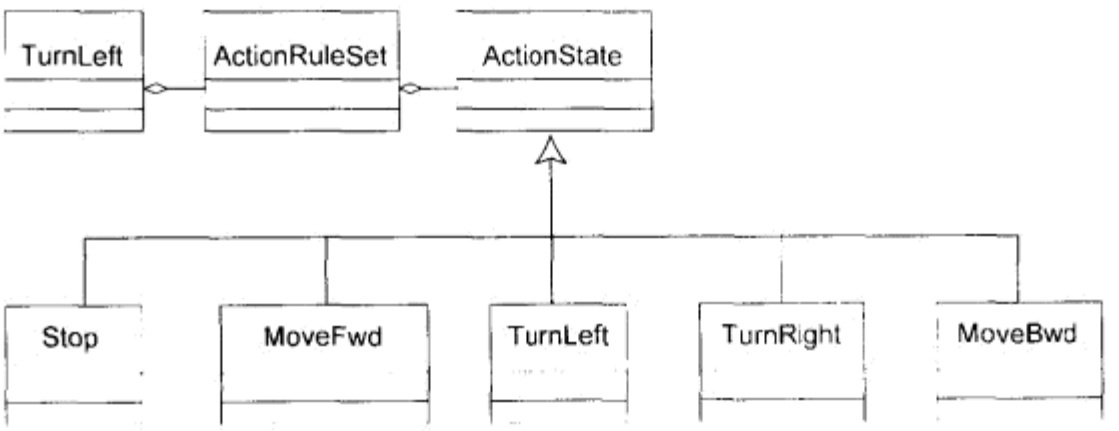


图 3.1.7　TurnLeft 动作状态规则的类图

当玩家为了避开障碍物而转弯的时候，训练角色就会得到此动作请求的通知。TrainingControlState 允许用户进行状态控制，和通常一样执行其请求的转弯动作。不过它同时会将得到的 ActionState 结果传入 Train()方法里面去。

```
TrainingControlState:PerformAction( Actor )
{
    ActionState = UserControlState.PerformAction()
    TrainingControlState.Train( ActionState )
    ...
}
```

1. 针对浮点数规则的映射簇

为了能动态地进行规则的训练，每个规则都有一个与之相关联的 ClusterMap 类。映射簇（cluster map）是一个一维的、进行空间划分的数据结构。此数据结构可以被划分为数个单元，每个单元对应相关规则的一个相应的域。为了覆盖 Evaluate()方法中从 0.0 到 1.0 的这个域，我们将使用一个有 10 个单元的映射簇。如果我们需要对这个规则所在的域进行更精细的划分，可以通过增加单元的个数来做到。一个由 10 个单元组成的映射簇可以用一个散列表来实现，此表使用一个在 0 与 9 之间的整数键值作为索引。图 3.1.8 中显示了相关的类。

ClusterMap 类提供了一个 CalculateIndex()的方法，从 Evaluate()方法返回的浮点数经过此方法作一个索引计算可以被转化为一个整数值。这个值就可以作为散列表中对应评估值的键值了。我们可以将此键值作为参数传递给 ClusterMap 的 Lookup()方法以得到对应于此单元的 Cluster 对象。映射簇的用途是将浮点数值进行取整，或者将之量化以得到其最近的单元。

```
TrainingControlState:Train( ActionState )
{
    For Rule in ActionState.GetRuleSet()
        ClusterMap = Rule.GetClusterMap()
        Value = Rule.Evaluate( Actor )
        Index = ClusterMap.CalculateIndex( Value )
            Key = ClusterMap.HashIndex( Index )
        Cluster = ClusterMap.Lookup( Key )
        Cluster.Reinforce()
```

Download at www.Pin5i.Com

```
    End For
}
```

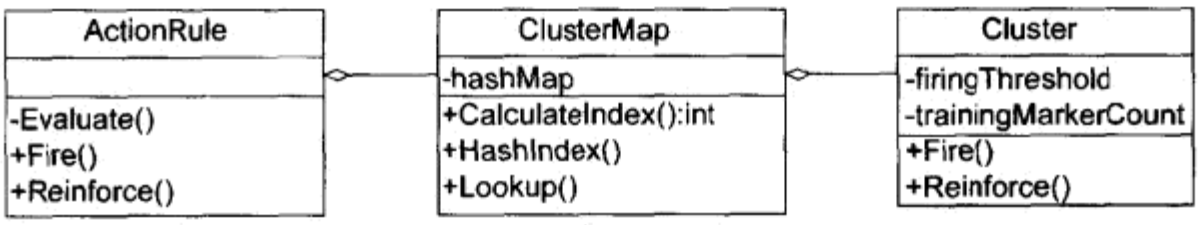


图 3.1.8 ClusterMap 的类图

Reinforce()是真正用来进行训练的地方。每个 Cluster 都有一个 traningMarkerCount 变量，它将在训练中递增。此标识值表征的是玩家使用上述的评估值激活了此动作规则的强度。当玩家对训练系统反馈了更多的输入以后，我们就将在此簇上得到更大的值。每次当我们对此簇进行一个 Reinforce()调用，我们就将对标识计数值加 1，一直等它达到了某个门限值为止。在此处我们使用门限值的原因是为了去除输入数据中可能带来的噪声或是错误。在实际应用中，你可能需要把这个门限值设置得相当高才好，如果你的训练数据中会有噪声或是错误的话。

```
Cluster:Reinforce()
{
    trainingMarkerCount ++
    If trainingMarkerCount > firingThreshold then
        trainingMarkerCount = firingThreshold
    End If
}
```

在我们训练的早期阶段，映射簇中只会有少数几个标识计数值。随着训练的继续进行，系统将得到更多的数据，也将学习到更多的规则。最终，我们将达到一个平衡点，在此时，我们的系统学习到了所有的与输入动作相匹配的规则。为了检测出当前系统是否达到了此平衡点，我们将对玩家的输入和 AIControlState 将要采取的动作进行比较。当我们发现它们连续匹配了一段时间以后，我们就可以通知玩家停止训练了。

2. 将控制交给 AI

现在我们已经拥有了一套完整的训练出的规则集，因此可以不再使用真人训练者，而以 AI 控制取而代之了。当一个角色执行它下一个动作的时候，它将调用其决策系统和新近训练出来的规则以决定采取什么动作。

```
Actor.SwapControlState( AIControlState )
ActionState = Actor.PerformAction()
If ActionState Is Not None Then
    CurrentState = Actor.GetCurrentActionState()
    CurrentState.Transition( ActionState )
End If
```

Cluster 的 Fire()方法现在就可以对评估值和映射簇中的训练标识值进行测试了。如果一条规则得到了一个落在其某簇上的评估值，而此簇已经有了足够的达到了门限值的标识

Download at www.Pin5i.Com

值，则此时这条规则就将被触发。如果一个规则集里面的所有规则都被触发了，那么相应的ActionState就将返回以将之激活。

```
AIControlState:PerformAction( Actor )
{
    fired = 0
    ActionState = Actor.GetCurrentActionState()
    For Action in ActionState.GetTransitions()
        For Rule in Action.GetRuleSet()
            ClusterMap = Rule.GetClusterMap()
            Value = Rule.Evaluate( Actor)
            Index = ClusterMap.CalculateIndex(Value)
            Key = ClusterMap.HashIndex( Index )
            Cluster = ClusterMap.Lookup( Key )
            If Cluster.Fire() Then
                fired++
            End If
        End For Rule
        If fired = Action.GetRuleSet().Count() Then
            Return Action
        End If
    End For Action
    Return None
}
```

3.1.4 结论

本节对GoCap的实现作了一个大体的勾画。此实现采用了散列表以进行规则簇的动态学习。这个增强的特性相对于使用稀疏数组的前一个版本来说，在性能上得到了极大的改善。AI决策循环中的嵌套结构对于将来的SIMD技术来说是一个很好的解决方案，但是这就不是本节讲述的内容了。

3.1.5 参考文献

[Alexander02] Alexander, Thor, "GoCap: Game Observation Capture," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

[Booch98] Booch, Grady, *The Unified Modeling Language User Guide*, Addison Wesley, 1998.

Download at www.Pin5i.Com

3.2 区域游览：对寻径模式的扩展

Ben Board，Mike Ducker
Dogfish Entertainment
Ben_board@yahoo.com, mike@ducker.org.uk

寻径是游戏人工智能中的一个重要组成部分，而且也是几乎所有现代电脑游戏中的一个主要关注点。一个简单的从 A 到 B 的寻径过程可能会产生一个很重的 CPU 负荷，这要视游戏世界的复杂度和对此任务分解的方式而定。传统的解决寻径问题的方法牵涉到两个方面：在游戏中放置节点和通过边将这些节点连接起来。在两个相邻的节点之间的直接路径连接中是没有障碍物的。

如何放置这些节点可以通过很多方法来解决：通过作为设计过程中一部分的关卡设计器（level designer）来解决；通过作为专家过程中一部分的过程式方法（procedural method）来解决；或是通过游戏世界中的内在特性来解决，在此世界中，环境有可能被分解为一组规范化的形状，例如一个分片地图，以表现出来。

一旦世界被分解为节点和连接以后，寻径就被转化为了某种形式的启发式搜索，此搜索的目的就是要找出一个连接列表，通过这些连接可以提供一条路径从人物出发点到达目的地。可以证明，现今电脑游戏中最流行的启发式搜索是 A*算法，其解释可以参见[Stout00]或是其他网站上的各种游戏网站。此算法说起来很简单，就是把当前列表中最好的节点加以扩展，一直到达到了目的地或是再也没有可扩展的有效节点为止（此即是说，此时企图进行的寻径失败了）。

本节提供了一个范式流程（paradigm shift）以用于将世界分解来达到寻径的目的。此模式没有使用节点作为寻径中的基本组件，而是将环境中的区域分解为各个更小的区域，每个都代表了一个搜索空间中的节点，而且当可以从一个区域游览到另一个区域的时候，我们就在它们之间建立连接。如果在游戏中实现了此模式，则我们将在下列方面得到极大的改善：

- 在速度和存储上会更加高效，这是因为在搜索算法中使用的节点数大大下降了。
- 更加真实的移动效果——我们现在可以从一个区域移动到另一个区域，而不需要精确地经过那些不必要的定位点。

Download at www.Pin5i.Com

3.2.1　辞旧

传统上来讲，寻径使用了定位点作为其遍历仿真环境的基本组件。通过将整个世界分解为一组相连的节点（当从一个节点到另一个节点可以不经障碍进行游览的时候，我们就将之连接起来），世界就可以看成是一个由顶点和边组成的图了，这样我们就可以直接对之使用状态空间搜索和图的理论了。有关图论的介绍可以参见[Sedgewick89]。

这种由一个复杂环境得来的抽象可能会涉及到对隔断地形元素的详细的 3D 造型。此抽象可以使用简单而强大的算法得到地理上分离的两点间最短的路径并将之连接起来。

虽然节点可以用来进行状态空间的搜索，但是由于它在图论中对应的顶点概念过于简单，在某些情况下如果直接使用的话，很容易导致低效。假设我们有一个最简单形式的环境——一个 2D 的分片地图。图 3.2.1 显示了一个简单的分片地图，其中包含了两种地形：可行走的（浅色的）地形和不可行走的（深色的）地形。为此地图生成节点的最简单的过程式方法就是在每片的中央设置一个节点，然后将之与可行走的水平方向和垂直方向的相邻点给连接起来，如图 3.2.2 所示。

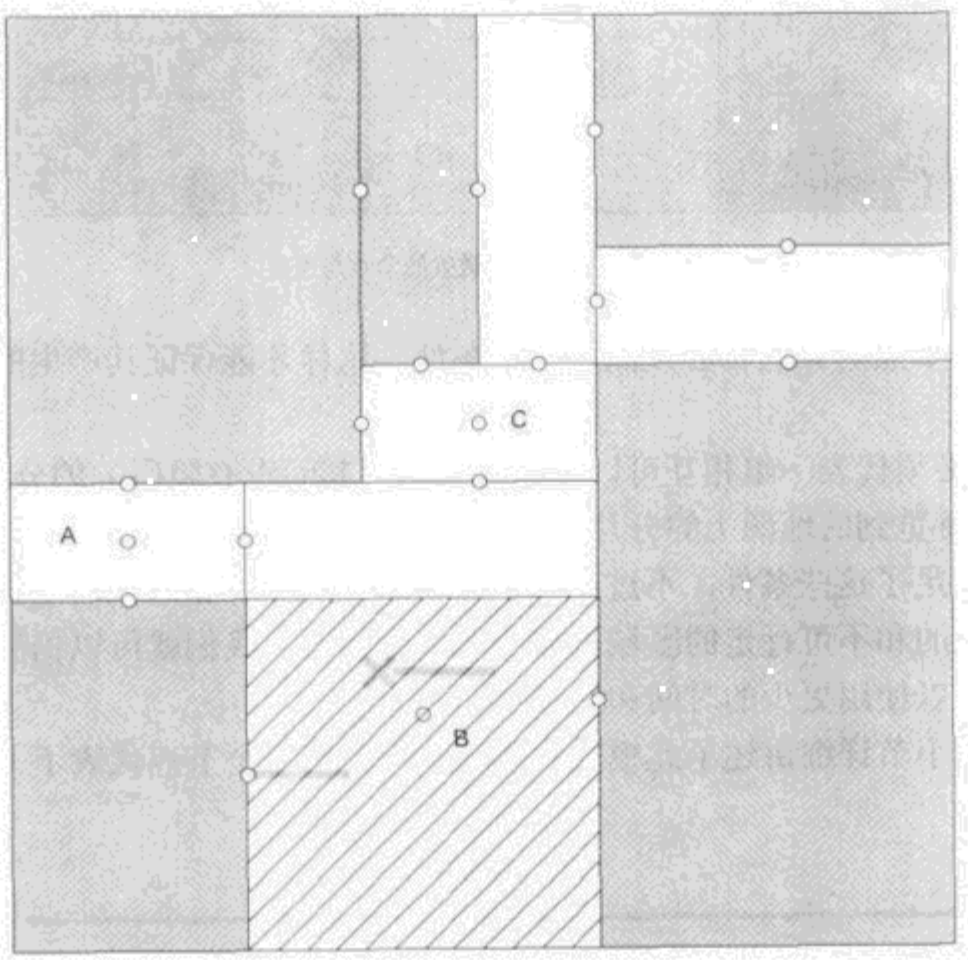


图 3.2.1　一个 2D 世界中简单的分片地图，浅色区域是可行走的，深色区域是不可行走的

当然了，使用此方法会在不可行走区域产生一些似乎是冗余的节点。不过如要想确认它们确实为真正冗余的点，那就要保证它们的类型（用来决定它们是否可以游览的特性）不会变化，而且在游戏世界中没有人物会使用到这些节点。

这样一个完整的节点系统保证了可以从任何一个可行走的分片找到一个没有障碍的路径通向另一个可行走的分片（如果存在此路径的话），但是每个节点（每个分片中的）的时间

Download at www.Pin5i.Com

和存储代价将会很大。我们应该使用更少的节点，精心将之挑选出来之后，在更少的空间里也能提供相同的可达性信息。

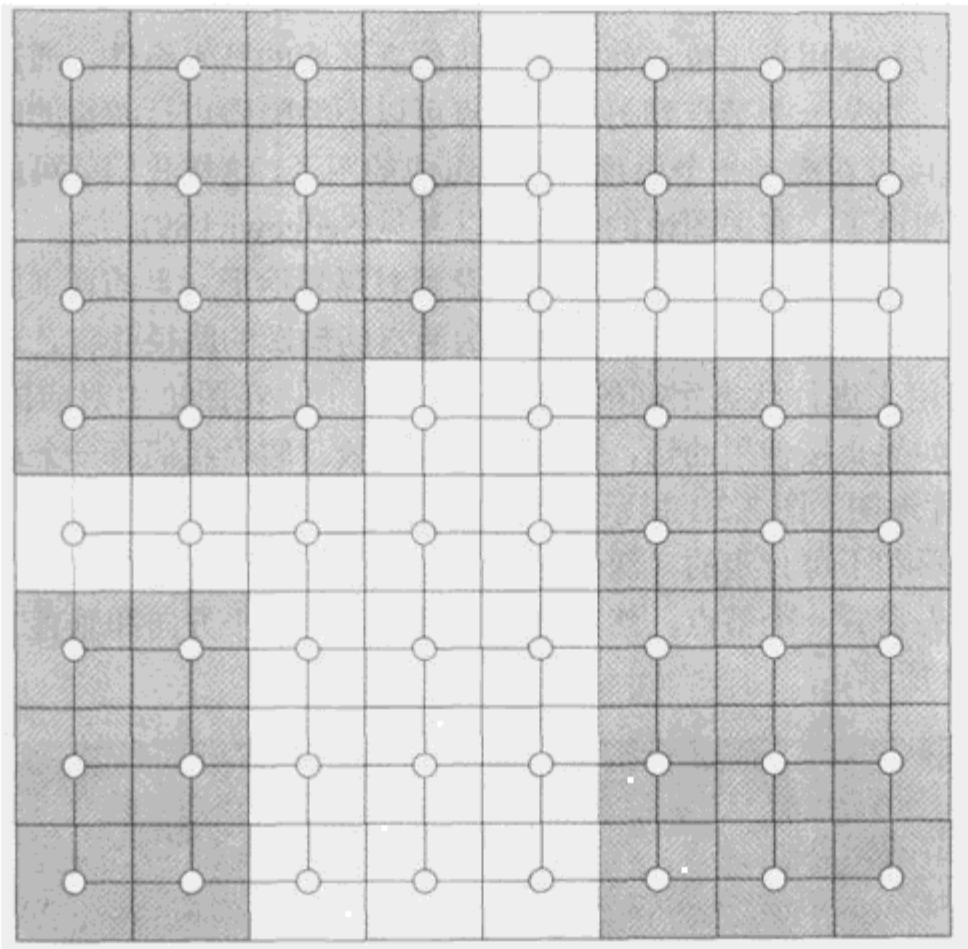


图 3.2.2 通过程序放置的节点与连接

任何一个使用节点的算法都必须满足以下条件，这样才能保证其产生的路径中是没有静态障碍物的：

- 每个节点必须代表一组相互可以互相游览到（其间没有障碍）的分片。
- 每个可能游览到的地图上的分片都必须有且仅有一个节点表征。

上述的方法满足了这些条件，不过产生了过高的冗余度。如果我们能找出一个最小节点集以分辨出可行走的和不可行走的区域以减少冗余度的话，我们就可以仍然享受图搜索方法的优点，同时还可以使用更少的时间和空间。

接下来的这个小节详细讲述了此模式的流程，在其间每个节点代表了不一样大的区域。

3.2.2 迎新

当着手解决寻径问题之前，我们必须首先要考虑一下以什么形式来表现其环境，这样以便对游戏进行特定的优化来编写出寻径算法。在分解世界的时候我们需要考虑两个基础性问题：

- 分解得到的节点数目：此数越小越好。如果节点数目越少，启发式搜索的速度也就越快。
- 结果节点对于路径遍历的有用程度：最好通过此节点得到的路径能充分利用环境中可行走区域的最大面积（这样一个人物就可以有更大的面积行走了）。

使用区域来代替点在上述的两个方面上都会表现得更加优越。

整个世界被划分为各个区域，每个区域都是一致可游览的（uniform navigability），而且在

Download at www.Pin5i.Com

其内部可以用一条直线穿过而不会遇到任何障碍物。如果一个人物能在两个区域中进行游览，则这两个区域就会用一个门户（portal）将之连接起来，此门户将坐落在两个区域的相邻部分上。在分片世界的例子当中，这些分片可以被划分为如图 3.2.3 所示的各个区域。需要注意的是我们的分片世界是长方形的，但是此处的新方法则更加通用，还可以应用到凸多边形上。

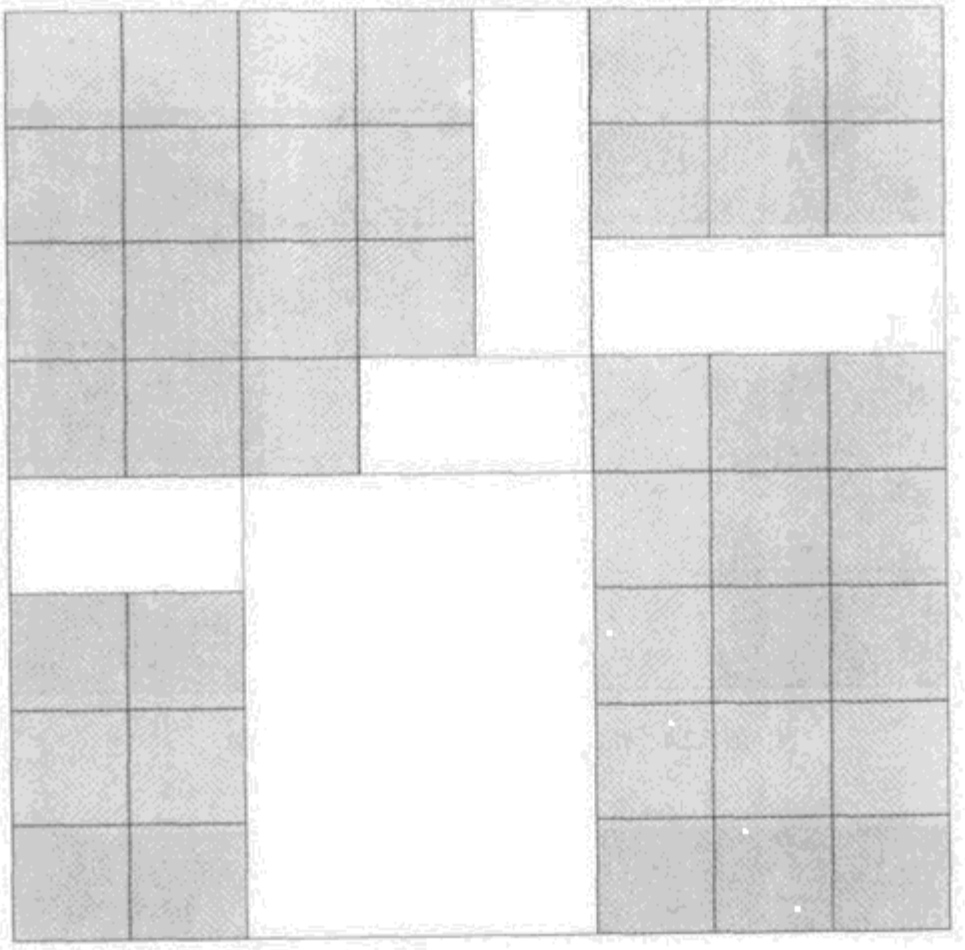


图 3.2.3　将世界划分为由门户连接的可行走的区域

正如图 3.2.3 所示，我们没有在寻径中使用固定的点。这也反映了大多数人在现实生活中用来解决此类问题的常识性的方法。当寻找一条路径从一个地点通向另一个地点的时候，通常你只会使用两个点：起点与终点。所有其他的区域都没什么不同，你观察世界的方式决定了你对这些区域划分的方法。当早上（或是下午刚开始那会）从卧室走向浴室，要设计要走的路径时，你可能会考虑到楼梯、阳台、走廊，或是甚至还有其他的房间。但是，你不会想一定要经过这些区域中某些特定的点，除非有其特定的理由（例如一旦到了楼梯上，你就没的选择，只好走下来了）。一般来说，在走廊上走的时候，你不会特意经过一个固定点，而仅仅是通过各个区域之间共享的门户最终到达目的地。任何在起点和终点之间的一个单独区域上的路径总是绕过动态障碍物（例如猫）的一条直线。

此处要提醒一下，使用区域并不会影响基本的启发式搜索算法。唯一需要做的改动就是如何计算从一个区域到另一个区域之间的距离。对于此种测量的最简单的解决方法就是使用区域中心点之间的距离作为需要的值。然而，如果使用一个比较现实的寻径算法，通过图 3.2.4 我们可以看出使用此值将得到一个比实际走过的距离更大的量。还有一个更好的方法，那就是使用进入区域的门户的中心点到离开区域的门户的中心点之间的距离作为其结果。例如，如果通过寻径算法的计算，你得到了一条从 A 经 B 到 C 的路径，从 B 到 C 的距离将是连接 A 与 B 之间门户到连接 B 与 C 之间门户的距离，如图 3.2.5 所示。

Download at www.Pin5i.Com

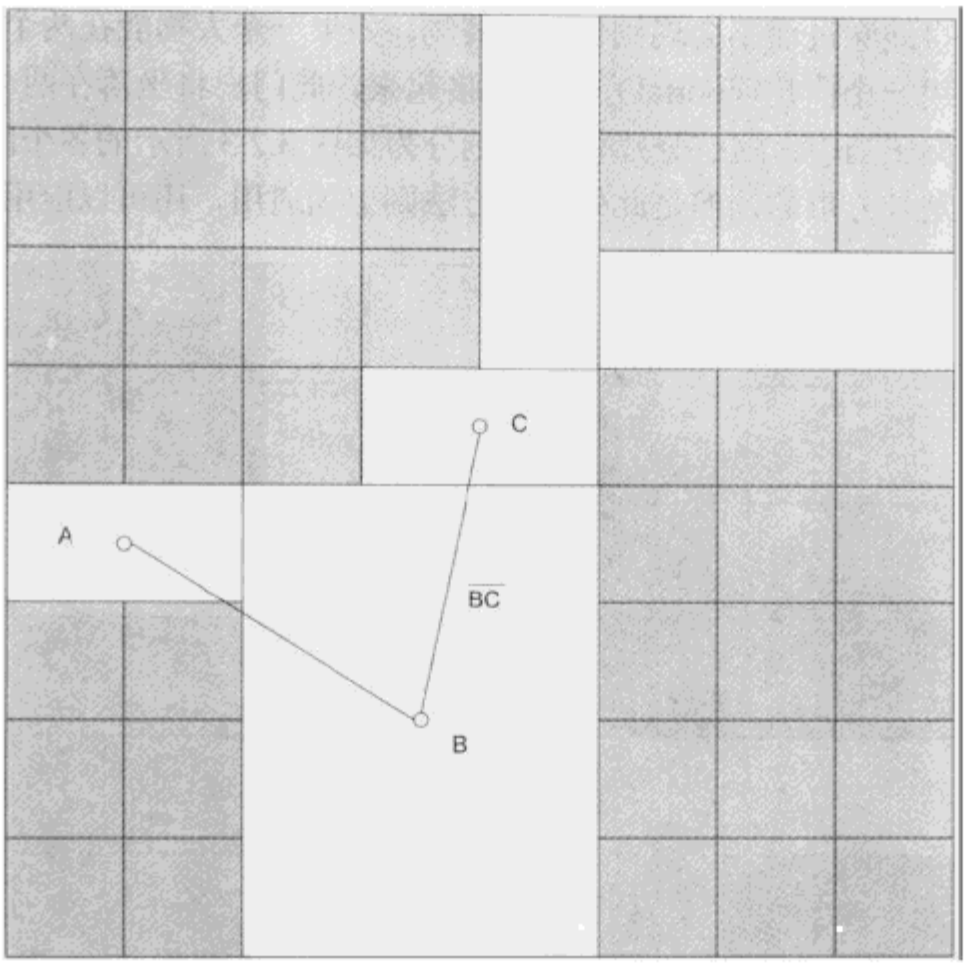


图 3.2.4 在区域间运动代价的一个简单估算

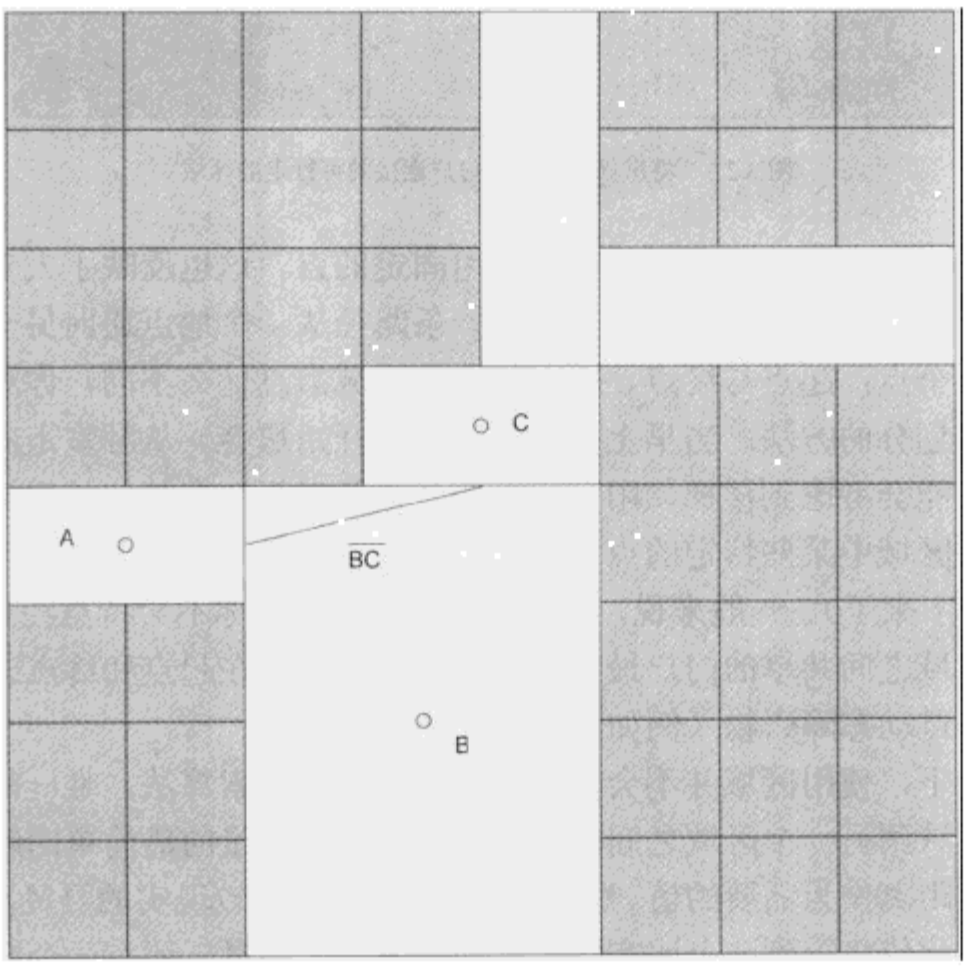


图 3.2.5 在区域间运动代价的一个真实估算

下面对此路径生成算法作一个小结：

Download at www.Pin5i.Com

- 在引入了基本概念以后，整个世界被划分为一组最佳区域，每个区域都是一致可游览的，而且其中任意两点都可以用一条不含障碍物的直线路径来遍历。
- 每个这样的区域在图中都表现为一个节点，而且如果区域中间有一个相通的游览区域（门户）的话，则这两个节点将通过一条边连接起来。
- 基本路径是如下生成的：首先找到起点与终点所在的区域，然后找到相对应的节点，接下来在图中进行搜索以找到两点之间的最佳路径。
- 一旦找到了由区域组成的路径，算法将利用各个相继区域中的门户创建一条路径。这就是路径遍历了，接下来我们就会谈到这一点。

下面的一小节还将继续讨论分片世界的例子，同时将阐述一个用来从环境中生成最佳区域的方法。

3.2.3　分而治之

从原始的地形数据里建构出区域来是一个与其所在的游戏关联性极大的问题。你有可能不得不从为各类地形分割的多边形中提取出经处理的区域，或是甚至要从一个全 3D 游戏的开放空间中提取区域。本小节将着重于对一个最简单的例子进行分析，然后展示一个将分片世界矩形化以得到如图 3.2.3 所示结果的方法。

虽然现在已经有很多方法可以用来从一个分片集合中生成一组矩形了（每个都有其自己的优缺点），但是如果专门针对寻径问题，则我们必须要满足以下的条件：

- 生成的矩形必须尽可能的大：其矩形越大，寻径过程就会越快，因为此时需要搜索的矩形比较少。
- 矩形必须尽可能的方：为了进行寻径，如果矩形的宽高比接近 1 的话就可以得到更好的路径。这可以帮助防止出现如图 3.2.4 所示的情况，从而得到更加符合实际情况的路径查找方法。
- 矩形化的算法必须很快：在地形可能动态变化，矩形化需要实时进行的游戏中，矩形化的算法必须要尽可能的快。

下面的伪代码可以用来将世界以较快的速度进行矩形化，会生成更方的矩形，而且可以对之进行性能上的调整以在为路径遍历进行矩形优化和速度优化中进行权衡。如果可以离线处理的话，速度优化就不用太过考虑了，而且甚至可以忽略掉以进行更好的矩形优化。此算法完全写下来会很长，因此在此处只给出其伪代码，完全的代码在 CD-ROM 上可以找到。

矩形化函数定义如下：

```
RectangularizeWorld( MapCellTypes cellType, int
    initialTestSize)
```

在此处参数 cellType 定义了此函数将要进行矩形化的环境类型（可行走的、不可行走的，或是还有其他什么类型），而参数 initialTestSize 则定义了将要测试的最大方块的尺寸，它是用来在矩形优化和速度优化中进行折中计算使用的。

Download at www.Pin5i.Com

此函数的第一步就是要计算所要求类型的单元的总数。如果通过计算发现在此区域内没有所需要的类型则此函数将退出。只有那些还没有被包含在一个区域里面的单元才会被计入此数。

```
int CellCount = number of free cells of type cellType
if CellCount = 0 then exit
```

下一步就是要确认此用来比较的方块的尺寸比所需类型单元个数的平方根小。如果此测试尺寸定义的区域大于所需类型的单元数所能占据的大小，那就没有办法在此方块中找到一个完全由这些单元构成的区域了。

```
if initialTestSize > sqrt(CellCount)
   then initialTestSize = sqrt(CellCount)
```

在矩形化过程的主循环中，首先设置方块的初始值为定义的最大尺寸，然后每进行一次迭代就将其减 1，一直到 0 为止。

```
for testSize = initialTestSize, testSize > 0, testSize = testSize - 1
```

在程序中将把此方块从地图的左下角移到右上角上去。

```
For startX = 0 to mapWidth - testSize
   For startY = 0 to mapHeight - testSize
```

此方块定义的是世界中的一个区域，我们将测试此（所需类型的）区域的一致性，以保证在此方块内部没有单元是属于其他区域的。

```
For testX = startX to startX + testSize
   For testY = startY to startY + testSize
      If cell[testX][testY] is not free or not of the required type, fail
```

如果此方块不满足上述条件，则我们将把它移到下一个测试点上去继续进行测试。如果此方块包含的单元都是所需类型，而且没有属于任何其他区域的话，我们将尝试着把此方块在北边、东边、南边和西边各扩张一个单元的大小，每次只尝试一个方向。每次扩张加入此区域的单元都将被永久性地加入到矩形中（如果它们都是所需类型，而且没有被其他区域包含的话）。此算法将继续在四个方向上进行扩张，一直到不能加入新的合法单元为止。

```
Fail = 0
While fail < 4
   For North, South, East, and West
   Expand the area by one cell width
      If the new rectangle is valid
         Fail = 0
      Else
         Fail = Fail + 1
      Contract the rectangle along this direction
```

在此处，我们就已经得到了一个经扩展矩形而得的区域，然后就要把它加入到区域列表里面去了。

```
For x = rectStartX to rectEndX
```

Download at www.Pin5i.Com

```
    For y = rectStartY to rectEndY
    Cell[x][y] set to unavailable

Add new area to area list

CellCount = CellCount - new area size
If CellCount = 0 then exit
```

通过改变初始时比较方块的最大尺寸，我们就可以在矩形优化和速度优化中进行权衡。如果初始化边长被设置为 1，那么我们做的矩形比较的次数就会到达最小值，这样就可以对速度进行优化。如果将之设置为最大值，例如设置为地图的宽度，则我们就要进行多得多的矩形比较了。然而，得到的一组区域将满足矩形优化的开始两个条件。这个需要开发人员根据自己游戏的要求进行权衡。

3.2.4 路径遍历

一旦生成了一条路径，对于一个人物来说，下一步就是要由此路径走到终点了。一个基本的路径遍历包含有两个方面：针对某些目标点的吸引力（attraction），还有针对某些障碍物的排斥力（repulsion）。如果人物已经走到了终点所在的区域的话，此吸引力的方向是指向路径的下一个门户或是指向终点的。排斥力则是由相邻的不可行走的区域产生的（按定义来说，一个区域内部是没有静态障碍物的）。

为了达到真实环境下任务走向下一个区域的行为，我们要结合着使用下面两种吸引力的简单模式（如图 3.2.6 和图 3.2.7 所示）：

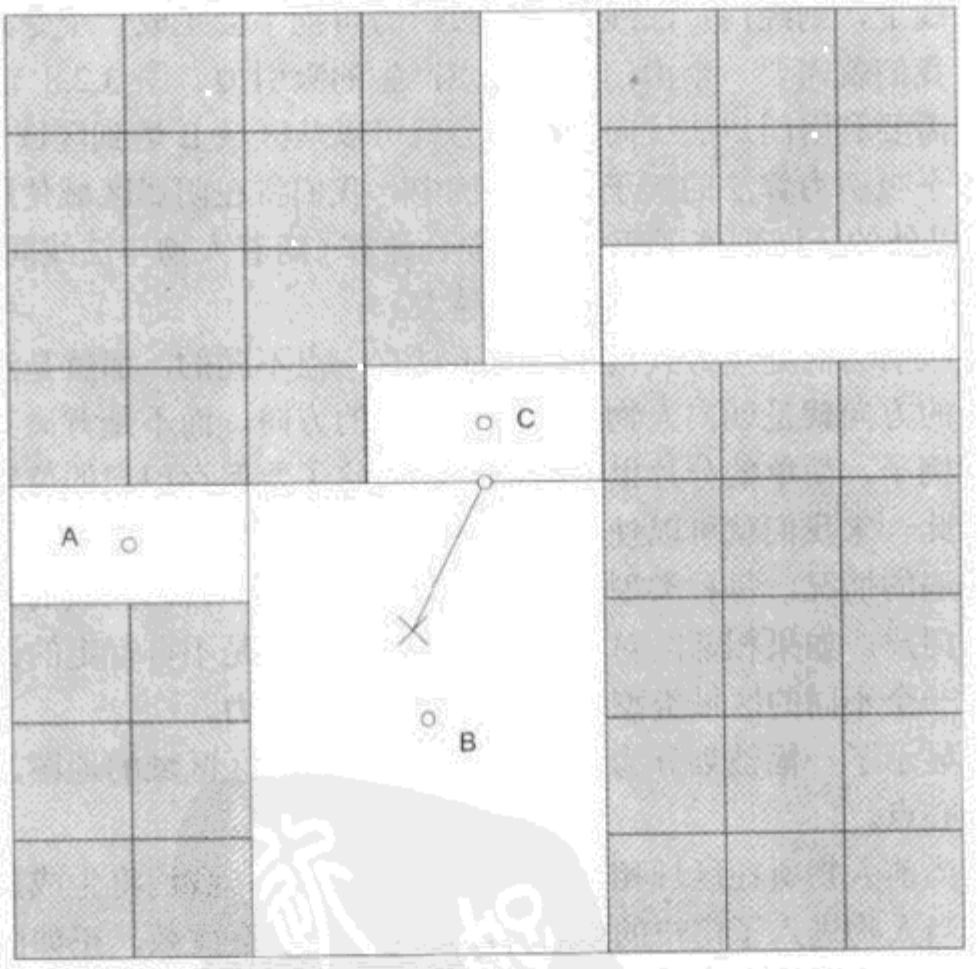


图 3.2.6 连接区域 B 与区域 C 的门户中心点对区域 C 的吸引力

Download at www.Pin5i.Com

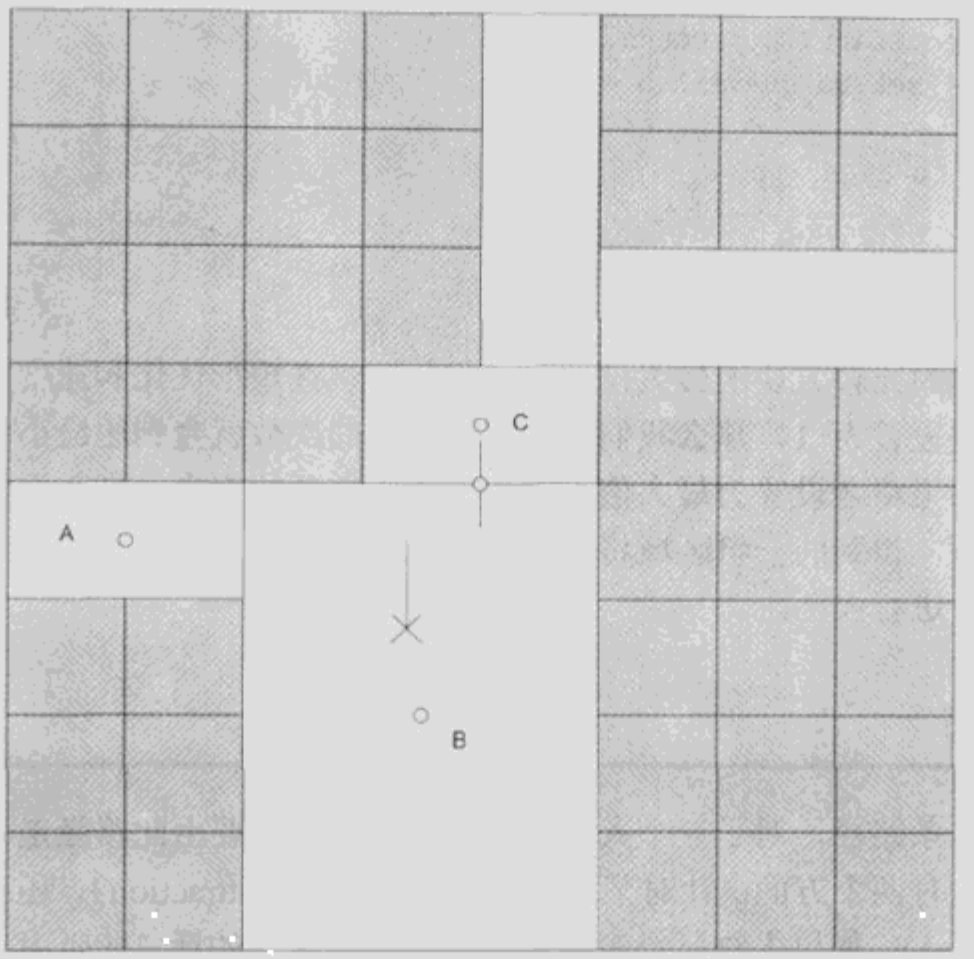


图 3.2.7 连接区域 B 与区域 C 的门户法线对区域 C 的吸引力

（1）由门户中心点产生的吸引力。

（2）在门户法线方向的吸引力（门户的法线是一个与门户矢量垂直的矢量）。

这些模式可以在不同程度上被使用，这取决于人物与门户之间的相对位置。如果人物没有正好处在门户法线上，则经门户法线方向的吸引力可能不会生成一个令人信服的到达新区域的路径。因此，我们就需要一个由门户中心点产生的吸引力。图 3.2.8 显示了区域 B 中的一部分，此部分只需要利用门户法线向的吸引力就可以从区域 B 达到区域 C 了。

我们可以举一个吸引力算法的例子，在此例中，我们将在阴影区域使用图 3.2.7 所示的法线矢量，在阴影以外的区域则使用两种矢量混合计算（随着人物与法线间的距离越来越远，中心点吸引力矢量在其中所占的比例也会越来越大）。

排斥力和法线吸引力的处理方式很相似，不过有一点不同的，那就是法线的方向是反过来的，这样此矢量的方向就是朝向人物当前所在区域的方向，而不是背离其区域所在的方向了。在我们特定的例子，简单的分片世界中间，我们除了要保存门户的位置以外，还要保存一个墙的列表，如此一来我们就可以使用墙来对人物进行排斥，以使之不进入不可行走单元了。如果考虑更普遍的情况，每种类型的单元都需要保存一个自己区域的列表，而所有的区域都与其相邻区有门户。如果相邻区对于某个特定人物来说是不可行走的话，那么我们就将在遍历算法中使用一个不同的区域类型来对此人物产生排斥力。

图 3.2.9 里面显示了一幅被划分为可行走区和不可行走区区域的地图，在其中的点表示的是每个门户的中心点。

对于每一个与当前人物所在区域相邻的不可行走区来说，我们将生成延门户法向的一个排斥力矢量。只有当人物进入了门户的范围时，此排斥力才会生效，正如图 3.2.10 中的阴影部分所示。

Download at www.Pin5i.Com

最后，一旦路径遍历完成，我们就可以使用以下方法来避开动态障碍物。我们可以使用基本的分群（flocking）技术，然后在路径遍历矢量中再加上由世界中的其他人物产生的排斥力矢量。如果想对分群技术有更多的了解，请参看[Reynolds87]和[Woodcock00]。

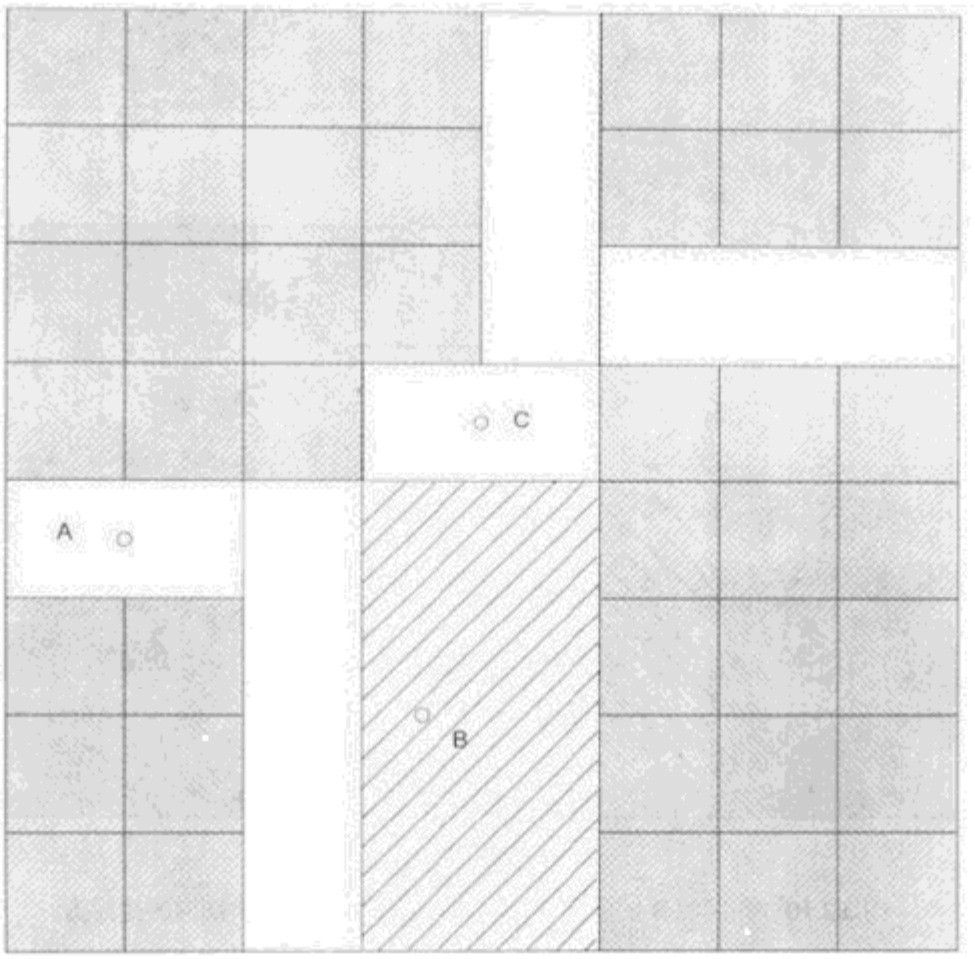


图 3.2.8　在阴影区域中，一个人物仅需考虑连接区域 B 与 C 的门户的法线吸引力（如图 3.2.7），就可以顺利地从 B 行走到 C 了

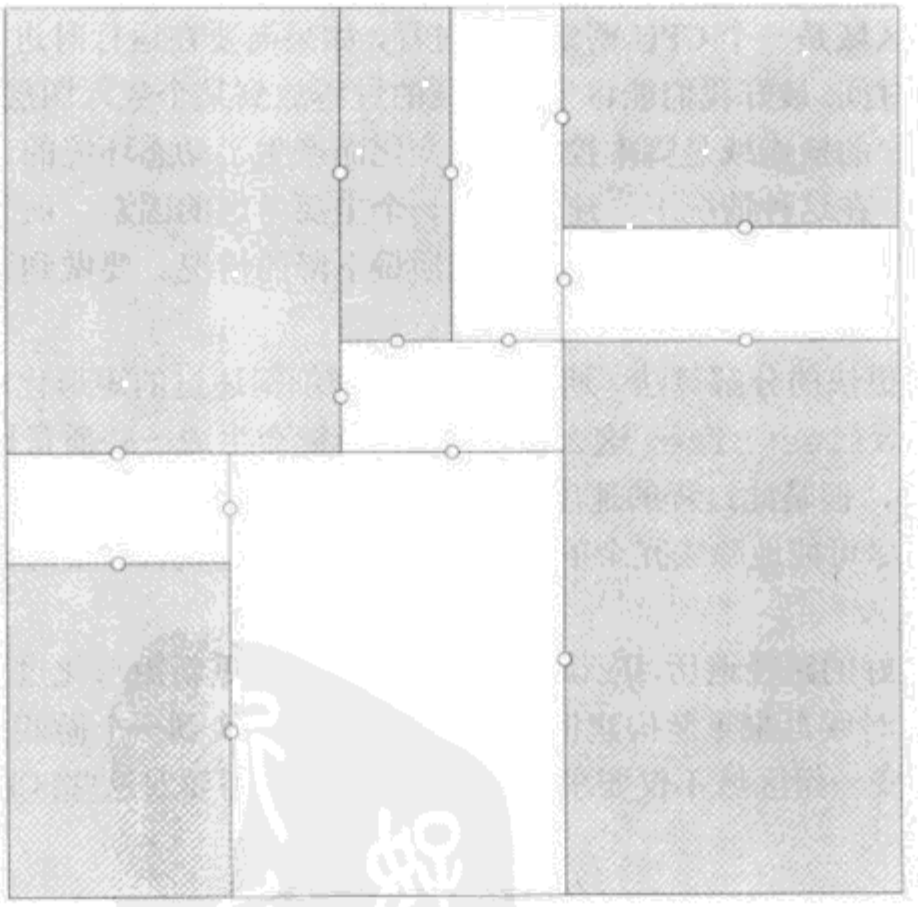


图 3.2.9　整个地图被划分为可行走区域与不可行走区域，它们由门户相连，门户的中心由圆圈表示

Download at www.Pin5i.Com

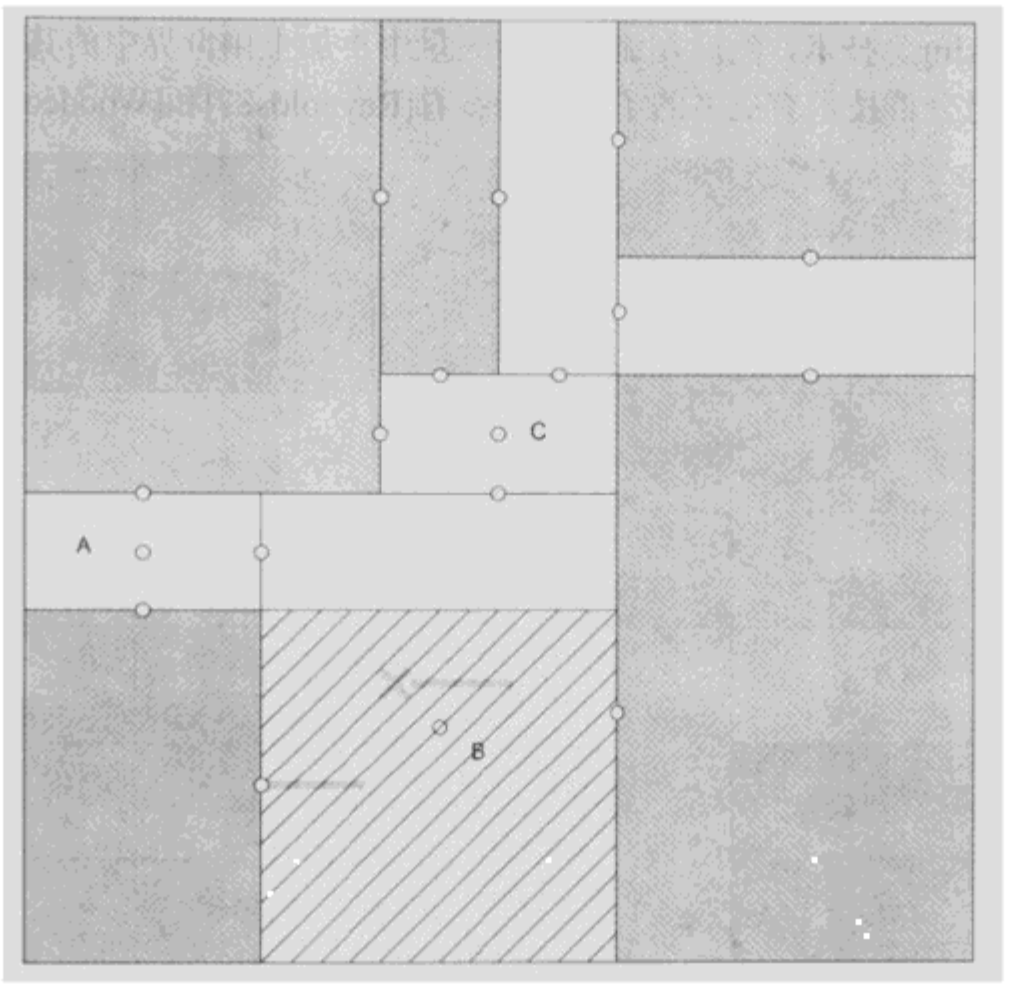


图 3.2.10 在区域 B 的阴影部分中，人物受到了延门户法向的排斥力

动态地形

把世界分解为区域是一个 CPU 密集型的过程，如果需要在运行时进行的话，则我们必须谨慎选择其运行的时间。最好我们能将生成区域的过程放到某个载入图层时的批处理过程中。然而，如果由于用户的操作或是脚本控制下的变化而产生了动态环境的话，则分解过程就必须在运行时进行了。在这种情况下，速度将是一个主要考虑的因素，而分解过程必须力求避免发生帧延迟，除非同时也发生了生成的区域质量下降的情况。要做到这一点，可以通过下面的方法来实现：

- 首先，使用更快的分解算法。通过使用上述详细阐述过的矩形化算法，选择一个等于 1 的 `initialTestSize`。这么做了以后，可能会生成一些质量比较低劣的用于路径遍历的矩形，但是此过程的速度将会提高到最大值。
- 在运行时，尽可能地除去冗余的矩形化过程（此即是说，不要对没有环境变化的区域进行矩形化）。
- 为了达到更好的路径遍历，应该在合适的时间进行重新矩形化过程，在用户不会发觉细小延迟的时候再来重新构建世界，例如在玩家切换到一个游戏里面的菜单时。一定要记住，改变一组区域不仅要求分解世界，而且还需要对这些区域里面的人物进行重新计算。

Download at www.Pin5i.Com

3.2.5　对此模式的扩展

虽然本节主要讨论 2D 分片世界这个简单的例子，但是此区域游览的方法对于任何游戏世界都是适用的（包括 3D 在内）。所有需要做的事情只不过是要使用一种方法将世界分解为有用的区域，这些区域能满足上面说过的条件，而且其相连的门户区域进行了重新定义。

3.2.6　结论

本节阐述了一个高效的算法，此算法用于在一个具有不同行走性的环境里创建和遍历路径，且此算法在本节中已经得到了成功的应用。由于在此算法中冗余度已经被减到了最小的程度，这样我们就拥有了更快的速度和更高的存储效率。我们首先要在环境里找出最佳的区域，在此区域上有一致的游览性，然后利用节点来代表这些区域，接下来再把相邻的可互相游览的区域连接起来。这样，即使针对很长的路径，我们也可以快速地生成一条路径了。为了支持此过程，我们还另外使用了一种方法以找出这些最佳的区域。此外，为了使用这些基于区域的路径以让人物能够进行游览，我们提出了一个路径遍历的方法。其最终结果是代价低且路径更真实。

3.2.7　参考文献

[Reynold87] Reynold, Craig, "Flocks, Herds and Schools: A Distributed Behavioural Model," Computer Graphics Proceedings(SIGGRAPH 1987): pp.25～34.

[Sedgewick89] Sedgewick, Roberts, *Algorithms,* Second Edition, Addison Wesley, 1989.

[Stout00] Stout Bryan, "The Basics of A* for Path Planning," *Game Programming Gems,* Charles River Media, Inc., 2000.

[Woodcock00] Woodcock, Steve, "Flocking: A Simple Technique for Simulation Group Behavior," *Game Programming Gems,* Charles River Media, Inc., 2000.

Download at www.Pin5i.Com

3.3 基于函数指针的内嵌式有限状态机

Charles Farris
VR1 Entertainment, Inc.
charles@vr1.com

本节的目的是要使用函数指针、继承和函数重载来创建一个 FSM（有限状态机）的实现。此实现需要满足三个设计上的要求：面向对象的实现、代码精简化、执行要迅速。

第一个要求，面向对象的实现，在现代游戏开发中非常重要。很多开发人员正在转向面向对象语言，例如 C++和 Java，这是因为这些语言支持抽象对象和继承这样的特性。为了要能在一个面向对象的编程环境中得到有效的使用，我们就需要为 FSM 的设计添上面向对象的成分。

第二个要求，代码精简化，是从下面一句软件工程中的基本原理得来的："保持简洁，易于使用（Keep It Simple, Stupid）"。相应地，有了这个特性以后，一旦想要添加 FSM 的功能，就只需要继承一个类然后为之添加一些成员函数和方法了。

最后一点，此实现使用了函数指针，这样就避免了在运行过程中进行代价甚大的 `if-then-else` 的比较[Calder94]。

3.3.1 什么是有限状态机

FSM 最通用的定义将之描述为了一个事件驱动系统的模型。一个系统的行为在 FSM 中会被表达为一组状态、一组输入事件和一组状态转移函数。我们在表 3.3.1 和图 3.3.1 中给出了一个电灯泡的例子。

表 3.3.1 电灯泡状态转移函数

当前状态	输入事件	状态转移
On	打开开关	
On	合上开关	Off
Off	打开开关	On
Off	合上开关	

如果想要对 FSM 和其在游戏开发中的应用有更多深入的了解，Eric Dysband 的《一个有限状态机的类》[Dysband00]和 Andre LaMothe 的《Windows 游戏开发大师的技巧》[LaMothe99]都是绝好的信息资源。

Download at www.Pin5i.Com

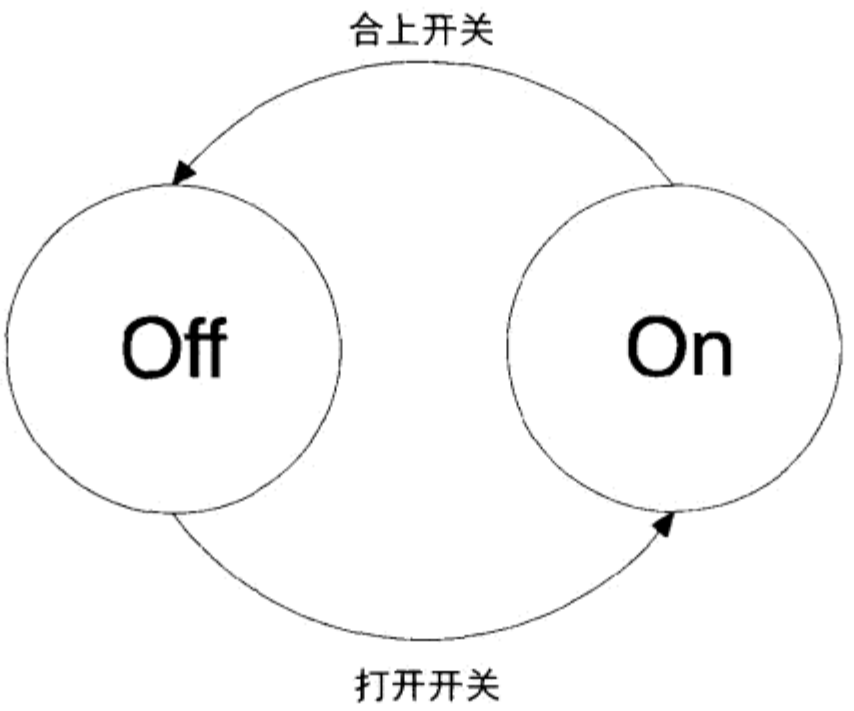


图 3.3.1　电灯泡 FSM

为何要使用 FSM?

在游戏开发中，FSM 一般被用来在运行时控制一个游戏对象的行为，这一般就是要根据当前的状态有选择地执行此对象的部分代码。虽然 FSM 可以被用来控制游戏中的任何对象，但是在 AI 开发中它们尤其受欢迎，其原因如下：

- FSM 可以用来将复杂的行为降解为较小的、较简单的行为。
- FSM 可以用来根据外部的事件调整 AI 行为，例如动画、声音，或是时钟。FSM 可以让开发人员将动画和 AI 行为集成在一起，同时也不必给绘画人员的艺术创作加上太多的限制。
- 与其他的 AI 技术，例如神经网或是遗传算法[Woodcock00, Woodcock01]，比较起来，FSM 要容易调试得多，也更容易进行性能调整。在执行的时候，FSM 产生的行为都是固定的，这是因为事先我们已经为指定的状态和输入事件定义好状态转移了。

3.3.2　FSM 的实现

首先，我们要对自己游戏中的对象加一些假设条件：每个对象都是由一个 C++的类来表示的，都有一个每个时钟周期就被调用一次的 `Update()` 函数，还有一组状态函数（它们将在 `Update()` 函数中被调用）。满足了这些假设条件以后，FSM 的实现还需要一个变量用来跟踪当前的状态，需要一些代码进行状态转移的处理。最重要的是，它需要一些代码将当前的状态映射到相应的游戏对象中的状态函数上去。

1. FSM 的集成

FSM 的实现可以以下面两种方式中的任意一种集成到游戏对象里面去：第一种，也是最简单的方法，就是在游戏对象里面加入一个 FSM 的实例；第二种方法是通过集成将 FSM 内

Download at www.Pin5i.Com

嵌到游戏对象中去。两种方法都各有其优缺点，但是本节将使用第二种方法，其理由是：首先，从 FSM 继承可以得到更大的灵活性，这是因为此时 FSM 的状态转移逻辑可以直接使用游戏对象的数据；其次，内嵌式的 FSM 在性能上稍微优越一点，这是因为在访问 FSM 的时候间接性更少了。

继承式 FSM 的最大缺陷在于 FSM 和游戏对象之间的关联性，这可能对日后的开发带来影响，因为任何对继承的 FSM 类的改动都可能需要对其派生类进行极大地修改。

2. Switch 实现

最常见的 FSM 的实现形式是 switch 实现。在此实现中，当前的状态被作为一个整数保存起来，然后我们将使用一个 switch 语句把当前的状态映射到状态函数调用上去[LaMothe99]。这种方法最大的好处就在于其实现比较容易，而且内存占用较低。

然而，通常的 switch 实现也有其缺陷。首先，switch 实现不是很面向对象[Sweeney00]。添加状态的时候必须要修改 switch 语句。当这些修改涉及到继承和函数重载的时候，其结果将是产生一堆“意大利通心面”式的混乱代码。另外，对于拥有多个开发人员的大项目来说，此 switch 实现中非面向对象化的特性将导致很严重的维护和调试方面的问题。

其次，此实现的性能依赖于 FSM 的大小。大多数 switch 语句将被编译器转化为 if-then-else 语句，这就很难对之进行优化了。对一个含有 n 个状态的 FSM 来说，平均下来，找到一个正确的状态函数调用需要进行 $n/2$ 次的 if-else-then 比较。如果 FSM 是在一个类层次结构上分布的，则除了要进行这些 if-else-then 比较以外，搜索相应私有状态函数的时候可能会牵涉到在类层次上数个虚函数的调用。

我们可以使用一个更加面向对象的方法来解决这个函数映射的问题，那就是将状态函数调用封装在当前的状态变量里面。由于状态函数一般都是和游戏对象绑定在一起的，因此在状态对象里面直接实现状态函数是不现实的。然而，我们可以使用函数指针将状态函数有效地存储在状态对象里面。

3. 函数指针

C 中的函数指针是很直接了当的，不过其语法却有点难懂。

```
<return type>(*<pointer>)(<arguments>)
```

在 C++中，几乎所有的函数调用都是类的成员函数。指向这些函数的函数指针在语法上就更麻烦了，因为此时函数都要和类绑在一起。相应地，其声明语法需要加入类的名称。

```
<return type>(<class>::*<pointer>)(<arguments>)
```

我们先假设 IsOdd()是一个叫做“CMath”的类的成员函数。IsOdd()的函数指针声明如下所示：

```
bool (CMath::*pfnFunction)(int)=&CMath::IsOdd;
```

针对成员函数的函数指针是特定于类的，这不仅仅体现在语法上。指向成员函数的函数指针不能指向一个派生类的函数，除非此函数已经在基类中进行了定义[Stroustrup97]。这是

Download at www.Pin5i.Com

一个严格的限制，因为这将要求所有的函数都要在基类中进行定义，会造成代码的极度膨胀。它也有好的一面，指向虚函数的函数指针将使用虚函数调用解析机制指向正确的虚函数调用。这就可以使成员函数的函数指针在派生类层次上得到应用（假设此函数已经在基类里面声明了）。

通过函数指针执行成员函数会带来一个问题。成员函数与一般的函数使用的是不同的调用规则，它们将得到一个类实例作为其隐含的参数（即 this 指针）。因此，通过函数指针执行成员函数需要得到一个类实例。下面的代码就阐述了通过 `pfnFunction` 函数指针执行 `IsOdd()` 成员函数的情况。

```
CMath Instance;
bool bResult=(Instance.*pfnFunction)(5);
CMath *pInstance=&Instance;
bool bResult=(pInstance->*pfnFunction)(3);
```

在讨论了这些限制以后，现在看来即使是在最佳情况下，使用函数指针来保存状态函数也是比较困难的。此时，我们先假设可以进行一些迂回得到结果，因此下面我们将继续考察在 FSM 实现中使用函数指针的情况（如果想得知更多关于函数指针的知识，Lars Haendel 的《函数指针教程》[Haendel01]是一个关于函数指针的绝好的参考资料，而且你可以从网上下载它）。

4. 函数指针的实现

在函数指针的实现中，每个状态都是被一个对象表示的。在这个对象里面，状态函数调用将保存在函数指针里，当状态初始化的时候函数映射就会生成。FSM 通过维护一个指向当前状态的指针对当前状态进行跟踪，而状态函数调用则直接通过状态对象的函数指针得以执行。

函数指针实现与 switch 实现相比，最大的优点就在于它的面向对象设计。函数映射现在成为了状态对象的一部分，而不是游戏对象的了。这个实现上的差异避免了将 FSM 的执行代码扩展到游戏对象类中去。我们现在不用在派生类里面重载 `Update()` 函数了，要添加新的状态，只需要把新状态对象的头文件包含进来就可以了。

此实现的另一个优点就是其性能与 FSM 中状态的个数是无关的。由于每个状态对象都非常清楚地知道应该调用什么函数，因此这里就不需要 `if-then-else` 比较了，也不需要在游戏对象的类层次结构中进行遍历了。

不过，此实现也有一些缺点。在实现上更困难了，这是由成员函数函数指针的内在特性造成的。此外，此实现明显需要更多的内存，这是因为状态函数映射现在被保存为一组函数指针，而不是被编译进代码里去了。

3.3.3　实现 CFSM

为了最好地满足设计需求，CFSM 的实现将由一个使用函数指针实现的基类组成，正如我们上面提到的一样。CFSM 的实现包括两个独立的部分：状态对象和 FSM。

1. 状态对象

在游戏对象的假设条件中，我们提到了游戏对象的每个状态都将由一组函数来表示。在

Download at www.Pin5i.Com

CFSM 的实现中，每个状态由 3 个函数表示。作为一个例子，我们先创建一个叫 CEnemy 的游戏对象类，它实现了空闲状态。

```
class CEnemy : public CFSM
{
    void BeginStateIdle();
    void StateIdle();
    void EndStateIdle();
};
```

其中 BeginStateIdle()和 EndStateIdle()函数是在状态转移的过程中被调用的，它们提供了很好的用来进行初始化工作和收尾工作的地方。StateIdle()函数是其主要的状态函数，而且每个时钟周期它都会在 Update()函数里面被调用一次。由于每个游戏对象的状态由三个函数构成，因此 CFSM 状态对象使用了三个函数指针来保存状态函数。

要创建一个状态对象，我们需要一个由两个类组成的层次结构。其基类，CState，提供了一个用来执行其保存的状态函数的通用接口。

```
class CState
{
public:
    virtual ~CState() {}
    virtual void ExecuteBeginState()=0;
    virtual void ExecuteState()=0;
    virtual void ExecuteEndState()=0;
};
```

第二个类，CStateTemplate，是从 CState 派生出来的，为了回避特定于类的函数指针的问题，我们把 CStateTemplate 设计为一个模板类。

```
template <class T>
class CStateTemplate : public CState
{
protected:
    typedef void (T::*PFNSTATE)(void);
    T *m_pInstance;
    PFNSTATE m_pfnBeginState;
    PFNSTATE m_pfnState;
    PFNSTATE m_pfnEndState;
public:
    CStateTemplate() : m_pInstance(0),
            m_pfnBeginState(0),
            m_pfnState(0),m_pfnEndState(0) {}

    void Set(T *pInstance,PFNSTATE pfnBeginState,
         PFNSTATE pfnState,PFNSTATE pfnEndState)
    {
        m_pInstance=pInstance;
        m_pfnBeginState=pfnBeginState;
        m_pfnState=pfnState;
```

Download at www.Pin5i.Com

```
        m_pfnEndState=pfnEndState;
    }

    virtual void ExecuteBeginState()
    {
        (m_pInstance->*m_pfnBeginState)();
    }
    virtual void ExecuteState()
    {
        (m_pInstance->*m_pfnState)();
    }
    virtual void ExecuteEndState()
    {
        (m_pInstance->*m_pfnEndState)();
    }
};
```

CStateTemplate 实现了三个状态函数的指针，而且还拥有一个指向类实例的指针。从 CState 类而来的执行函数都被 CStateTemplate 重载并实现。由于 CState 是 CStateTemplate 的基类，这样一个游戏对象就可以使用 CStateTemplate 实例来创建特定于类的状态对象了。不过，此 FSM 的实现可以把状态对象看成是通用的 CState 实例，这样在执行其状态函数的时候就可以不需要知道一个游戏对象的细节了。

2. FSM

现在既然我们已经定义了状态对象，CFSM 基类就可以使用它们以实现 FSM。

```
class CFSM
{
protected:
    CState *m_pCurrentState;
    CState *m_pNewState;
    CStateTemplate<CFSM> m_StateInitial;
public:
    CFSM();
    virtual ~CFSM() {}
    virtual void Update();
    bool IsState(CState &State);
    bool GotoState(CState &NewState);
    virtual void BeginStateInitial() {}
    virtual void StateInitial() {}
    virtual void EndStateInitial() {}
};
```

当前的状态，m_pCurrentState，是一个指向 CState 对象的指针。通过使用 CState 的指针，我们可以让 FSM 来完全实现 CFSM 中 Update()函数的状态函数执行代码了。除了当前状态的指针之外，还另外声明了一个 CState 的指针，m_pNewState。FSM 将对这个变量进行跟踪，一旦这个变量被赋值，FSM 就将进行一个状态转移。

CFSM 的实现中通过 CStateTemplate 类定义了一个初始状态，而且将之保存在状

Download at www.Pin5i.Com

态对象 m_StateInitial 里面。有了此状态以后，就可以保证 FSM 实现总有一个状态可以用来执行了。CFSM 的建构函数在初始化状态对象指针的时候也对此初始状态进行了初始化。

```
CFSM::CFSM()
{
    m_StateInitial.Set(this,BeginStateInitial,
            StateInitial,
            EndStateInitial);
    m_pCurrentState=static_cast<CState*>(
            &m_StateInitial);
    m_pNewState=0;
}
```

状态函数的执行是在 Update()函数中发生的。

```
void CFSM::Update()
{
    if (m_pNewState)
    {
        m_pCurrentState->ExecuteEndState();
        m_pCurrentState=m_pNewState;
        m_pNewState=0;
        m_pCurrentState->ExecuteBeginState();
    }
    m_pCurrentState->ExecuteState();
}
```

在进入 Update()函数的时候，FSM 将检查 m_pNewState 变量以得知是否需要进行状态转移。如果需要进行状态转移，则 Update()函数将调用 ExecuteEndState()和 ExecuteBeginState()函数，然后改变当前状态。

GotoState()和 IsState()函数是用来简化对状态对象处理的。GotoState()函数将设置新的状态变量，这将导致在下一个 Update()函数的调用中产生一个状态转移。

```
bool CFSM::GotoState(CState &NewState)
{
    m_pNewState=&NewState;
    return true;
}
```

IsState()函数提供了一个语法上友好的方法，通过它可以对当前状态与任何给定的状态进行比较。

```
bool CFSM::IsState(CState &State)
{
    return (m_pCurrentState==&State);
}
```

Download at www.Pin5i.Com

3.3.4 使用 CFSM

下面我们将通过一系列的例子创建一个模拟电灯泡的游戏对象，此游戏对象将使用于本节开始时谈到的 FSM，而 CFSM 类将用来提供 FSM 的功能。

1. 在类中加入 CFSM

首先，我们将创建一个叫做 CLightBulb 的类，而且将之从 CFSM 基类中继承下来。为了实现其中的状态，我们通过使用 CStateTemplate 类和相应的状态函数加入了两个状态对象。SwitchOnEvent()和 SwitchOffEvent()函数则是用来处理两个输入事件的。

```
class CLightBulb : public CFSM
{
protected:
    CStateTemplate<CLightBulb> m_StateOn;
    CStateTemplate<CLightBulb> m_StateOff;
public:
    CLightBulb();
    virtual void SwitchOnEvent();
    virtual void SwitchOffEvent();
    virtual void StateInitial();
    virtual void BeginStateOn();
    virtual void StateOn() {}
    virtual void EndStateOn() {}
    virtual void BeginStateOff();
    virtual void StateOff() {}
    virtual void EndStateOff() {}
};
```

建构函数里对 CFSM 基类进行了初始化，另外还初始化了两个状态对象。

```
CLightBulb::CLightBulb() : CFSM()
{
    m_StateOn.Set(this,BeginStateOn,StateOn,
         EndStateOn);
    m_StateOff.Set(this,BeginStateOff,StateOff,
         EndStateOff);
}
```

下面，我们将加入 SwitchOnEvent()和 SwitchOffEvent()函数用以处理输入事件和状态转移逻辑。

```
void CLightBulb::SwitchOnEvent()
{
    if (IsState(m_StateOff))
       GotoState(m_StateOn);
}
```

Download at www.Pin5i.Com

```
void CLightBulb::SwitchOffEvent()
{
    if (IsState(m_StateOn))
        GotoState(m_StateOff);
}
```

由于这个例子比较简单，因此其状态函数基本上没有什么东西。不过 BeginStateOn() 和 BeginStateOff()函数中有一些代码，它们是用来显示当前状态以便我们跟踪 FSM 的执行流程的。

```
void CLightBulb::StateBeginOn()
{
    cout << "State: On" << endl;
}

void CLightBulb::StateBeginOff()
{
    cout << "State: Off" << endl;
}
```

最后，StateInitial()函数被重载以用来跳入电灯泡的 FSM 中。

```
void CLightBulb::StateInitial()
{
    GotoState(m_StateOff);
}
```

2. 在一个派生类里面改变 FSM 的行为

让我们来创建一个新的类来模拟闪光灯。由于闪光灯和普通的电灯泡很相似，因此我们可以重用 CLightBulb 类，只不过要对之进行一下扩展。首先，我们需要从 CLightBulb 中派生出一个叫做 CFlashingLightBulb 的新类。接下来我们需要添加一个新的状态用来处理闪光，还需要重载 CLightBulb 中的 On 状态函数。

```
class CFlashingLightBulb : public CLightBulb
{
protected:
    CStateTemplate<CFlashingLightBulb> m_StateOnDim;
    unsigned int m_uTimer;
public:
    CFlashingLightBulb();
    virtual void SwitchOffEvent();
    virtual void BeginStateOn();
    virtual void StateOn();
    virtual void BeginStateOnDim();
    virtual void StateOnDim();
    virtual void EndStateOnDim();
};
```

就像上一个例子一样，新的状态是在建构函数里面初始化的。

Download at www.Pin5i.com

```
CFlashingLightBulb::CFlashLightBulb() : CLightBulb()
{
    m_StateOnDim(this,BeginStateOnDim,StateOnDim,
        EndStateOnDim);
}
```

闪光的行为是由在 On 和 On Dim 状态中来回循环来处理的，还有一个时钟用来控制闪光的间隔。从 CLightBulb 而来的 On 状态函数被重载了以适应新的行为。

```
void CFlashingLightBulb::BeginStateOn()
{
    CLightBulb::BeginStateOn();
    m_uTimer=10;
}

void CFlashingLightBulb::StateOn()
{
    --m_uTimer;
    if (m_uTimer==0)
        GotoState(m_StateOnDim);
}
```

同样地，我们实现了 On Dim 状态函数。

```
void CFlashingLightBulb::BeginStateOnDim()
{
    cout << "State: On Dim" << endl;
    m_uTimer=10;
}

void CFlashingLightBulb::StateOnDim()
{
    --m_uTimer;
    if (m_uTimer==0)
        GotoState(m_StateOn);
}
```

最后，我们更新了 SwitchOffEvent()函数以把新状态考虑进来。

```
void CLightBulb::SwitchOffEvent()
{
    if (IsState(m_StateOn) || IsState(m_StateOnDim))
        GotoState(m_StateOff);
}
```

在执行过程中，FSM 将在 On 和 On Dim 状态中进行循环，这样就模拟出了一个闪光灯的行为。这个例子演示了两个扩展 FSM 行为的方法：使用新状态和重载现存的状态。

3.3.5 结论

本节阐述了一个使用函数指针 FSM 的实现，它使得开发人员能给新的或是现存的游戏

Download at www.Pin5i.Com

对象加上 FSM 的功能。虽然本节 FSM 的实现形式也可以被开发人员使用，但是此处演示的实现是极简单的，对于一个想要创建其自己的 FSM 实现的开发人员来说，它只是一个起点而已。相应地，你还可以对之进行大量的修改，包括内存优化、更加复杂的状态转移逻辑、定制的状态对象，还有额外的错误处理。

3.3.6 参考文献

[Calder94] Calder, Brad, Dirk Grundwald, and Benjamin Zorn, "Quantifying Behvioral Differences Between C and C++ Programs," 网址为 http://www.cs.colorado.edu/deparment/publications/reports/docs/CU-CS-698-94.ps, January 1994.

[Dysband00] Dysband, Eric, "A Finite State Machine Class," *Game Programming Gems*, Charles River Media, Inc., 2000: pp.237～248.

[Haendel01] Haendel, Lars, "The Function Pointer Tutorials," 网址为 http://www.function-pointer.org. October 2001.

[LaMothe99] LaMothe, Andre, Tricks *of the Windows Game Programming Gurus*, Sams, 1999:pp.729～734.

[Stroustrup97] Stroustrup, Bjarne, *The C++ Programming Language, Third Edition*, Addison Wesley Longman Inc., 1997: pp.418～421.

[Sweeney00] Sweeney, Tim, "Unreal Technology FAQ," 网址为 http://unreal.epicgames.com/UnrealScript.htm, June 2000.

[Woodcock00] Woodcock, Steve, "Game AI: The State of the Industry," *Game Developer Magazine*, August 2000: pp.34～43.

[Woodcock01] Woodcock, Steve, "Game AI: The State of the Industry," *Game Developer Magazine*, August 2001: pp.24～32.

Download at www.Pin5i.Com

3.4　在 RTS 中的地形分析——一个隐藏的重要因素

Daniel Higgins
Stainless Steel Studios, Inc.,
dan@stainlesssteelstudios.com

当还处在孩童时期的时候（好吧，甚至已为成人），我们就努力想要理解自己身处的世界。而当我们迈入校门的时候，已经能很容易地识别和指认出很多东西了。我们知道什么是街道，什么是建筑，我们能识别出天空和海洋、山野和森林。甚至当我们还是什么也不知道的小孩的时候，即使我们还以为郊区家里房子后面的树木是延伸了数里的森林的一部分的时候，那时候我们就至少可以识别出这些地形元素，已经知道它们的很多特性了。

让我们再进入到虚拟世界中去，电脑游戏一直在努力，希望给玩家创造出一个世界，提供一个丰富多彩的游戏环境，在这个世界里面将充满了有趣的地形元素。正如一个少年可以自由决定是要穿过树林到朋友家去呢，还是要走街道去，开发人员也要利用同样的地形信息作出决策。开发人员可能会设计出一个虚拟少年，他在日落以后就不会穿森林走路了，或者如果是和一群朋友在一起的话，他才可能选择走森林。

对周围的世界环境收集信息的行为就叫做地形分析。在今天的电脑游戏里面，这是一个极其重要的因素，特别是在实时策略游戏（RTS）里。糟糕的是，这么重要的任务却有可能会被忽略或者在开发过程中被粗心地低估了。不要再犯错误了——编写一个地形分析引擎有可能是一个浩大的工程，这要取决于你到底希望要分析多少信息。不管如何，在一个 RTS 游戏里面，它是最重要的开发任务之一。

3.4.1　区域

地形分析的第一步就是规划。当选择究竟要分析什么地形元素的时候，我们必须考虑下面几点："我们需要了解什么样的区域？"我们需不需要识别游戏里面的森林、关隘或者海洋？进行一番头脑风暴式的思考，首先列出一个清单，在上面记录下你需要识别的区域。

在进行头脑风暴得到了区域清单以后，你可能会发现需要把区域分成两大类：静态区域和动态区域。先不管两者之间概念上的差异，静态区域

Download at www.Pin5i.Com

和动态区域在程序代码里面表现得很相像（即使不是完全相同的话）。由于两者很相似，因此你可以考虑编写一个通用的完全由虚函数组成的区域的基类，这将对以后的开发带来一些便利。

1. 静态区域

在游戏里面，静态区域是不会变化的。一般说来，静态区域会占据大多数点（或者分片），拥有大部分区域。与动态区域相比，静态区域必须在游戏开始之前就处理完毕。从这一点上来说，它比动态区域要好，因为动态区域可能在游戏的开始之前、进行过程中和结束以后都需要进行频繁的处理。

在运行时对一大片区域进行重新处理将把游戏拖慢得像蜗牛一样，甚至有可能使得游戏里的世界完全停止。对于一个游戏来说这真是一场灾难，而且显然应该避免。因此，为了减少一个游戏运行时处理的时间，静态区域就需要被单独提出来了。

下面是一些静态区域的例子：

大陆：在地形分析里面，大陆是最重要的区域之一。每张地图上的每个点都应该处在某种类型的大陆上，即使那个点是悬崖、海洋，或是瀑布。大陆是用来判断地图上不同部分的可访问性的。你可能有一个航海工具能在海洋大陆上航行；或者陆上工具能在陆地大陆上穿行；或者航空工具，能在所有类型的大陆上飞行。

山野：如果一个 RTS 引擎能考虑到使用视野，或者考虑到了在战斗中不同高度的影响，那么最好它能对山野地形进行分析。在 Empire Earth 中，我们把一座山定义为在某个高度之上的一组相邻点的集合。我们发现如果加入了山野的因素，那么就可以感觉出在游戏中电脑玩家能表现出更高的智能来。电脑玩家的弓箭手可能会撤到附近的山头上，从制高点进行攻击，或者电脑玩家会在一块资源附近选择一个山头作出色的防守据点。

海岸线：一个海岸线就是同时与海洋和陆地接壤的一部分区域。这就意味着一部分海岸线主要是在海洋里面的；而另一部分则主要是在浅水区的，在这里大部分陆上工具都可以行走。如果有海浪的话，游戏还可以利用海岸线来决定是否需要起一阵浪，应该在海洋多远的地方起浪，以及临近的海域应该是怎么样的。在决定一个运输船是否可以登陆或者载人的时候，海岸线也是非常有用的。

海域：海域就是那些离陆地有些距离的区域。如果一片区域离陆地有两片区域那么远的话，我们就将之保存在“海域 2”里。对于 RTS 游戏中那些占据了多片区域的物体来说，海域是很有用的。由于大物体的碰撞检测是一个很困难的问题，因此我们可以在海岸线边上设立一些海域，以此来防止巨大的船只，例如航空母舰，与陆地挨得太近。你可以使用任意数量的海域来防止各种船只过于接近陆地。当然如果不需要考虑把船与陆地分离开的问题，那么这可能并不是一个需要进行处理的区域了。

2. 动态区域

在运行时，动态区域是随着游戏世界的变化而改变的，这可以使得一个游戏不至于太单调，让游戏更耐玩。而且在一个变化的世界里面的，电脑玩家的行为会给真人玩家一种假相，似乎电脑真的拥有某种智能。另外，在每次玩游戏的时候，玩家都会得到一次全新的游戏体验。

在代码上，动态区域和静态区域的处理几乎是完全一样的，但是动态区域有一个内在的

Download at www.Pin5i.Com

问题。那就是假设一个区域很大的话，如果在游戏里重复处理它，这就将导致游戏运行效率低下甚至会冻结。通过使用一些创造性的技巧可以避免这种情况的发生，不过对动态区域进行高效再处理的话题并不在本节的讨论范围之内。

在一个 RTS 中有用的动态区域包括以下几种：

森林：森林就是一群彼此相邻的树木的集合。森林必须是动态的，因为如果玩家砍伐了一棵树的话，这棵树原来的位置就不应该再标识为属于森林区域了。如果游戏中的人物能够理解而且能识别出森林的话，那就很容易把森林用在游戏里面了。对森林的使用应该很直观。例如，如果玩家选择了一个游戏人物，然后点击森林中间的一棵树，那么玩家大概是想让这个人物(或者个体)去砍那棵被点击但是却接触不着的树的旁边的一棵树。如果我们已经对森林进行了分析的话，那么现在我们就有很多方法可以找到一棵符合条件的要砍的树了。一个方法就是直接找一棵离这个个体最近的树。另一个方法是首先计算出个体和被点击的树之间的路线，然后得到路线和森林相交的那一点上的树，最后就可以把个体指向那棵树走过去了。人工智能在这里还可以根据不同的任务作不同的处理，例如规划应该在什么地方建一堵墙或者在哪个灌木丛中埋伏作战单元。

城镇：在真实的世界中，大多数人都认为城镇就是一些相邻建筑物的集合。在一个 RTS 中间，这个概念也是基本一样的。有一点例外，那就是在游戏的世界中，城镇应该表示成一个凸包，在此凸包里面有许多的建筑物（参见图 3.4.1）。

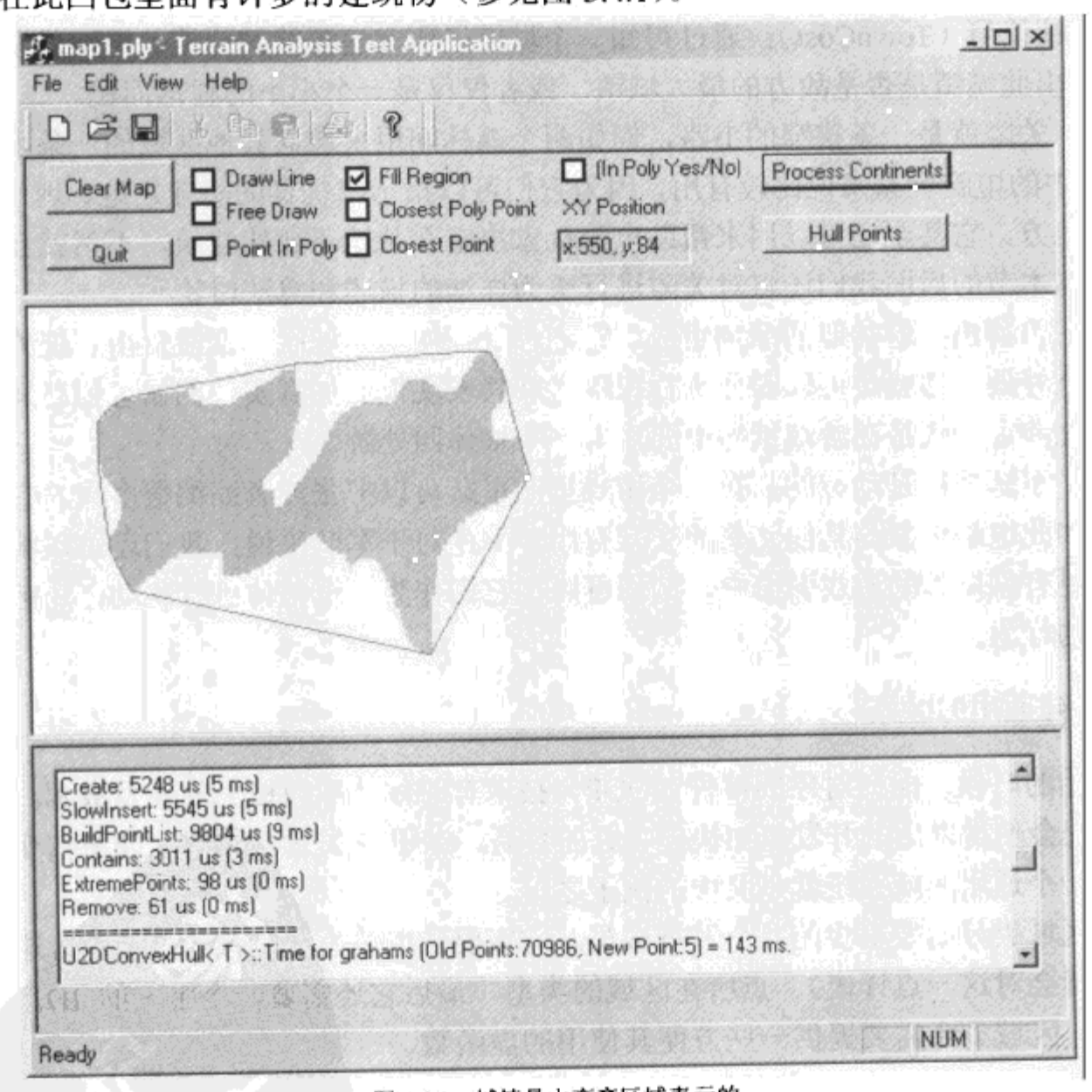


图 3.4.1　城镇是由高亮区域表示的

Download at www.Pin5i.Com

城镇是动态的，因为它们可以扩张、缩小，或者当一个建筑物被建起来或者推倒的时候，甚至可以合并起来。城镇和大多数其他的区域不太一样，因为这个包含所有点（即单独的一个建筑物）的凸多边形要比里面的点更加重要。在某种意义上，如果一个人想要知道是否一个点属于一个城镇，他们就需要得到对应的凸包，然后作一个检验，看看点是否落于多边形以内，而不是去对多边形里面的每个点进行检查。

当建筑物被建起来或者被推倒的时候，城镇也会同时被创建或者是被删除。要判断一个新建的建筑物是否属于一个城镇，首先就要检查此建筑物到每个城镇凸包的距离。然后根据这些距离，我们才能知道究竟这个新建筑是应该属于某个城镇；还是可以把两个或者多个城镇合并成一个大城镇；还是需要为这个建筑物新建立一个城镇。如果建筑物正好坐落在两个城镇间一个合适的位置，那么这两个城镇就可以合并起来，成为一个大城镇了。

除了可以用来标识出一组玩家的建筑物以外，城镇还有更大的用途。城镇的相关信息可以用来帮助一个人工智能个体作出智能化的决策：应该进攻城镇的什么地方呢？用什么作战单元进攻呢？接近一个城镇的最佳策略是什么呢？

城镇可以拥有很多信息，以下列出了其中的几个：

- 是否沿海（IsCoastal）：在很多情形下我们都需要知道是否一个城镇离海很近。这种信息最有用的一个方面就是可以帮助人工智能个体进行决策，决定是否可以调度海上作战单元来轰炸此城镇。
- 城镇价值（TownCost）：通过得知一个城镇里所有建筑物的价值，电脑玩家就可以判断出此城镇是否是敌方的最大城镇，或者仅仅是一个小小的村落而已。

关隘：关隘就是一条狭窄的小路，例如两个森林间用来通向其他区域的一条小路。狭径对于 RTS 中的电脑玩家来说比较有用，因为它们可以指示出优良的防守之地、埋伏点，还有建城墙的地方。它甚至还可以用来帮助电脑玩家决定在某地不要建建筑，不然就会把自己给堵死了。在本节的后面我们还将对关隘进行更为详细的描述和算法讨论。

兽群：兽群由一群类似的动物组成，它们都处在同一个地区，聚散自由。把兽群看成一种地形有个好处，那就是可以对之进行跟踪，使其表现得更加真实，例如它们之间也会因争夺地域发生冲突，或是在游戏世界里面作为一个整体四处游荡。

军队：军队当然是动态的。在一个游戏里面军队可以扩张或者缩编很多次。它们主要对电脑玩家和此电脑玩家与其他元素的交互有用，不过对于军队来说，我们还可以将之用到一些其他的更有创造力的游戏引擎中，例如可以把它们作为一个整体进行移动，还可以产生一些群体性的行为。

3. 创建通用的区域

在开始的时候，面向对象的编程（OOP）技术有时候会导致极大的开发成本。但是一般说来，它们会使得以后的开发更加快捷、更加容易。OOP 从长远看来是会使事情变得更容易的，设计一个通用区域系统就是其中的例子之一。

一个区域类只需要很少的几个数据成员。它需要知道其内所包含的点、包含这些点的凸包（马上就会对这一点详谈）、点所在区域的类型，最后它还需要一个唯一的 ID。除了数据成员以外，区域类还需要提供一些方便其使用的虚函数。

虚函数必须包括：

Download at www.Pin5i.Com

- Create()：创建此区域。
- GetClosestPoint()：得到离某个点最近的一个点。
- GetClosestPointToArea()：这个方法需要另一个区域作为参数，然后返回离这两个区域最近的点。
- GetClosestPointToHull()：这个方法与 GetClosestPoint()有所不同，因为它是使用凸包进行检查的。你可以在派生类的 GetClosestPoint()方法中调用 GetClosestPointToHull()。它对于城镇这样只包含很少几个有意义点（建筑物）的区域而言最有用。但是我们需要造成这样一种假相，那就是要让它看起来好像城镇中包含的每个点都存储在这个对象里面一样（而不仅仅只是那几个用来构成凸包的点）。
- GetRandomPoint()：这大概是和区域相关的最有用的方法了。这个方法可以用来得到电脑玩家的防守位置，这样就可以节省开发的时间，还可以让游戏看起来比较智能化。

优化的小技巧：当你派生出一个大陆区域类的时候，可以在大陆类里面包含很多其他的区域。例如，如果把关隘存储在大陆类里面的话，开发人员就可以迅速地定位其希望找到的关隘了。这种方法同样也可以使用到其他类型上面去，此处可以将其看成是在存储上的优化。你也可以把大陆区域看成是一个散列表，这样我们就可以减少任务花费在区域查找上的时间了，这些任务包括例如要找出用户究竟在哪一个森林的凸包上点击了一下。如果能让程序仅返回用户点击的大陆区域，只搜索那个森林，那你就可以节省下搜索的时间了。

通过从一个基类开始创建游戏的区域系统，我们可以让区域的开发变得更容易。即使不必对其所在的区域有更多的了解，只要对这些区域进行一些基本操作，游戏中的电脑玩家也可以做出智能化的决策了。

3.4.2　凸包

在模仿人类的智力行为中，有一个更加复杂的地方，那就是电脑没有视觉。只要瞟上几眼，人就可以从中获得丰富的信息，但是对于电脑来说这就太困难了。我们可以使用凸包这一简单然而强大的几何图形来对这种行为进行仿真。

凸包就是不会向内拐弯的闭合图形。如果某人沿着一个凸包的边缘顺时钟走动，那么他将发现自己将总是右转弯，但从不会左转。另外，如果你在凸包的这些顶点之间连线，连线将总被包含在凸包里面。

虽然也有多维的凸包，但是考虑到效率和大多数 RTS 中的实际情况，通常我们只需要使用 2 维的凸包就行了。当然了，开发一个多维的凸包系统也是有其用处的，不过这是在开发时间允许和真正需要的时候才行。

1. 为什么要使用凸包

凸包占用的内存很小，而且效率很高，此外还具有相当高的精确度。从事物的一个方面

Download at www.Pin5i.Com

考虑，你可以把所有点的列表存起来以得到很高的精确度，但是这样一来占用内存将极大，而且性能也将下降。凹包会更加精确，但是它也会使用更多的内存，而且使性能下降。从另一方面来说，你可以使用一个包含所有点的矩形或者是圆来近似模拟一个地形。这两种方法都占用极少的内存，而且会得到更高的性能，但是它们在精确性上就太差了。凸包结合了它们的优点，给我们以优良的精确度和优秀的性能，同时对内存的要求也比较小。

凸包几乎在 RTS 游戏引擎中的每一种区域上都可以得到使用。对于判定是否一个个体隐蔽在森林的某个邻近区域，或是一个个体是否穿越了城镇的边界这样的问题上，凸包是非常理想的。同时，它们也使得复杂的人工智能行为的创建更容易实现了，这是因为它们可以让开发人员快速地得到一个区域。作为一种特定的几何图形，它们也遵循一些数学规则，基于这些规则，人工智能可以进行优化并可假设某些条件存在（参见图 3.4.2）。

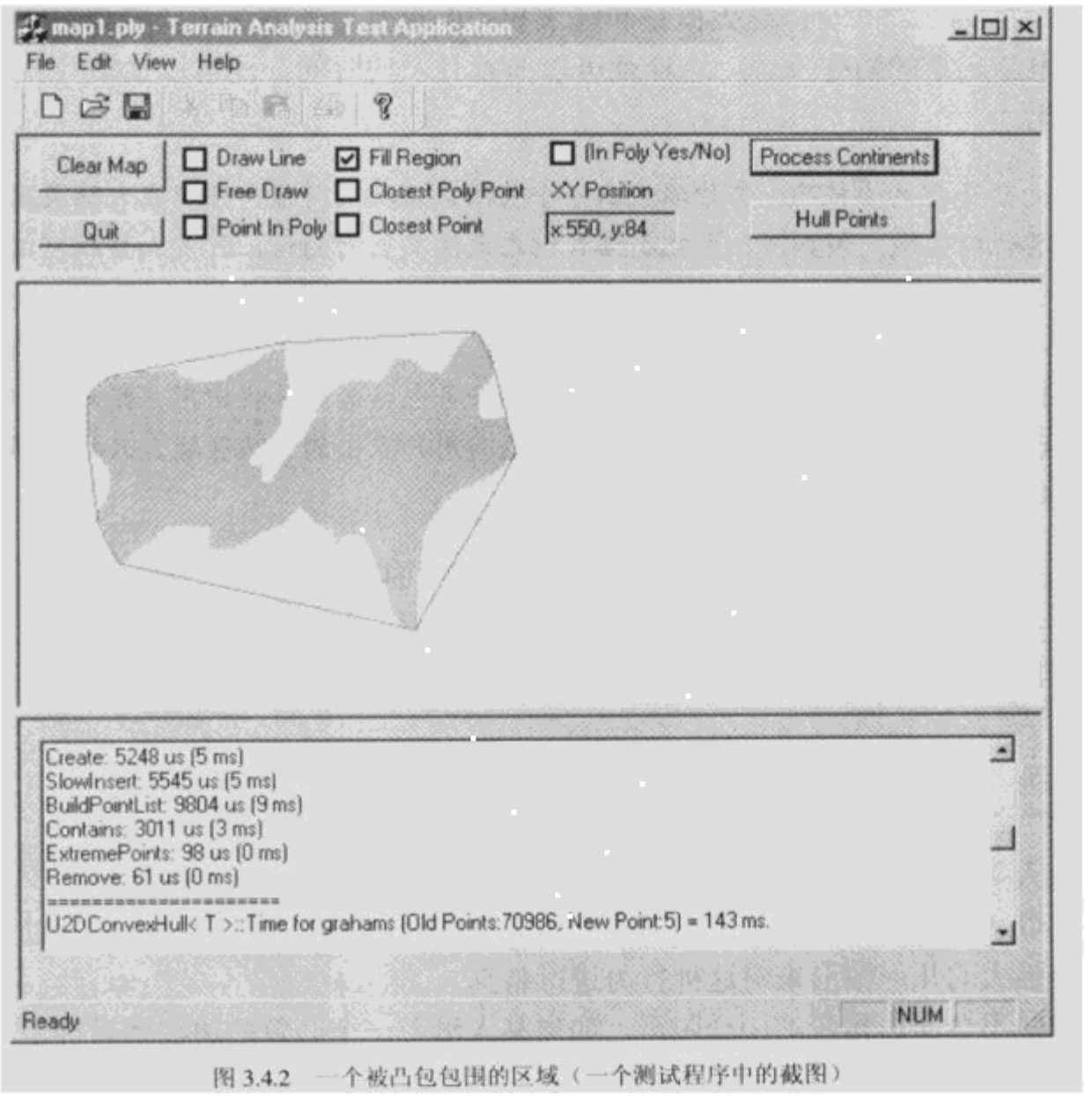


图 3.4.2 一个被凸包包围的区域（一个测试程序中的截图）

凸包的优点并不仅限于以上提到的几点。开发凸包需要的时间不会太长，因为编写 2 维凸包的代码只需要很少的数学知识（这就是说，即使开发人员不是数学专家，也可以很容易地实现凸包）。

2. 凸包里有些什么？

凸包其实就是一个满足某些特殊条件的多边形而已。因此，在创建凸包类之前，我们可以首先创建一个多边形类作为基类。还有一个好办法就是可以把多边形/凸包类做成模板，这

Download at www.Pin5i.com

是因为在基于 PC 的 RTS 游戏里面，大部分任务可以使用一个基于整数的凸包来提高性能。否则，我们就将使用浮点数的凸包了。

一个多边形类的数据成员大致如表 3.4.1 所列。

表 3.4.1　　一个多边形类的数据成员

类　型	变　量	描　述
vector<U2DPoint<T>>	mPoints	多边形所有的顶点
U2DRectangle<T>	mBoundingBox	对一个多边形区域的快速描述。它的区域包含了一个凸包或是多边形的最外的点。此矩形有两个点，一个在左上角，另一个在右下角
U2DPoint<T>	mCenterPoint	质点的中心。当需要快速计算出一个不那么准确的距多边形的距离的时候，或是对此多边形跟踪的时候，使用此数据将是非常方便的
Float	MArea	多边形内部的面积

请注意：U2DPoint<T>是一个 *xy* 点的模板类，而 U2DRectangle<T>是一个有上、左、右、下坐标的矩形模板类。

由于这只是一个基类，因此除了一般的 Get/Set 方法和建构函数/析构函数以外，它只需要提供很少的几个虚函数：

- CalculateArea()：用来得到一个多边形或者凸包的面积。
- ClearPolygon()：用来清空多边形的数据。
- Contains()：这是个内联函数，在调用更复杂的虚函数 ContainsInPoly()方法之前，它会先进行一个快速地判断，看看点是否在边界矩形内。

```
inline bool Contains(const U2DPoint<T>& inPoint) const
{ return (this->mBoundingBox.Contains(inPoint)) ?
        this->ContainsInPoly(inPoint) : false; }
```

- ContainsInPoly()：这个虚函数将对给定的点作一个点是否在多边形内的判定。
- Create()：此方法将创建一个多边形，另外此方法还有一个可选参数，它是一个标志值，此值可以用来对像凸包这样的图形进行优化。
- CreateCenterPoint()：此虚函数将使用适当的算法来计算出多边形的中心点。计算中心点的方法之一可以使用如下所述的重心计算方法：

```
U2DPoint<float> theSums(0.0f,0.0f);
long            theSize = this->mPoints.size();
long            theLoop;

/* Sum up all the points. */
for(theLoop = theSize - 1; theLoop >= 0; theLoop--)
{
    // cast to floats regardless of template type.
    theSums.SetX(theSums.GetX() +
    ((float)mPoints[theLoop].GetX() /(float)theSize));

    // cast to floats regardless of template type.
    theSums.SetY(theSums.GetY() +
    ((float)mPoints[theLoop].GetY() / float)theSize));
}
```

Download at www.Pin5i.Com

```
// cast it into the right format:(template function)
theSums.CopyTo(this->mCenterPoint); }
```

- Expand()：一个辅助性的函数，它可以用来将多边形成比例地扩展以增大其面积。一种简单的扩展算法就是可以从中心向每个顶点连一条线，然后把顶点沿线向外延伸以进行扩展。
- GetClosestPoint()：对于一个凸包来说，这是最有用的方法之一。GetClosestPoint()将返回一个可能是虚拟的[1]在凸包上的一个点。最好要有两种得到最近点的方法，一个用来返回最近的顶点，一个用来得到最近的在多边形或是凸包上的连线交点。
- GetClosestPoints()：使用此方法可以得到给定的两个多边形或是凸包的最近点。
- GetIntersection()：当两个多边形相交的时候返回 true，而且可以创建出一个多边形代表相交的区域。
- SortPoints()：这是一个模板方法，用来对点进行排序。

```
template<class K> // inFO is a function object.
inline void SortPoints(const K& inFO)
{ std::sort(mPoints.begin(), mPoints.end(), inFO); }
```

- TrimToVitalPoints()：在此方法中将执行创建凸包的算法。不过对于多边形来说此方法一般是空的。

当从 U2DPolygon<T>派生的时候，凸包类需要重载多边形基类中的一些方法，其中最重要的一个方法是 Create()。

Create()方法是凸包创建算法的入口点。使用不同的算法创建凸包会造成凸包创建时在性能上的极大差异。现在已经有很多公开的创建凸包的方法和辅助性函数了[O'Rourke98]。在开发人员中最流行的一个，同时也是在 Empire Earth 中使用的，就是 Graham 算法[O'Rourke98]。

优化的小技巧：在 Graham 算法中最花费时间的就是对点进行排序。在某些极罕见的情况下，当你已经有了一组已排序的点时，就可以传进一个“已经排序”的标志值，这样就可以节省一些 CPU 的时间了。

在地形分析中凸包是非常高效的一个元素，这是因为通过它们，开发人员可以在相对较短的时间里面创建出一堆令人难以置信的特性出来。强大的特性，加上低廉的开发成本，只要把握了这两点，设计人员、玩家们，还有开发人员都会非常高兴的。

3.4.3 重要的匹配器

能够找到一个区域的凸包真是太好了——除非你还需要从此区域中得到真正的点，如果你想得到一个真实的点（例如在一片大陆上的一个点，而不仅仅是凸包里的点），那就需要一

译者注[1] 可能是虚拟的，英文原文为 potentially fabricated。记住，一个凸包里面的点指的是其内包含的对游戏有意义的点，而不单纯是几何概念上的点。例如一个城镇（凸包）里的建筑物才是这个凸包真正有意义的点。所以最近的点可能在游戏中是没有意义的，因此称之为虚拟的。

Download at www.Pin5i.Com

个数据结构来存储此大陆上的所有点了。

天哪！大概在你的脑海里面马上就会浮现出成百上千，甚至是成千上万的点吧，当然了，巨型地图，甚至是某些较小的地图都会贪掉一大片内存的，除非你能使用更巧妙的技术。

我们需要的是一个内存超人，他能将所有这些魔鬼式的点挤进一个狭小的空间，同时也不需要占用过多的 CPU 时间。对此危机的解救者就是：主匹配器（Major 匹配器）——基于模板的 U2DMatchboxContainer 类。

这个 U2DMatchboxContainer 类，在本节的余下部分中被称之为“匹配器容器”的东西，其实是一组链表，它包含的是点的范围，这些范围将对点进行模拟。一个匹配器容器实际上是一个以 x 为索引的一维数组，其中包含了一个链表，链表中的节点是一组组起始点与终止点的 y 坐标。

我们先来对一个由单个陆地大陆组成的方形空白地图进行考察。如果地图大小为 400×400，那么此大陆将包含 160 000 个点。如果我们将这些点放进一个匹配器容器里面去的话，则这 160 000 个点将变成 400 个点了，因为我们对一个 x 值只需要保存一个点。其原因在于此大陆已经是一个凸包了（它是一个正方形）。如果我们研究一个更复杂的例子，把如图 3.4.3a 中的点保存在一个匹配器容器里面的话，则其最终结果可能会如图 3.4.3b 所示。

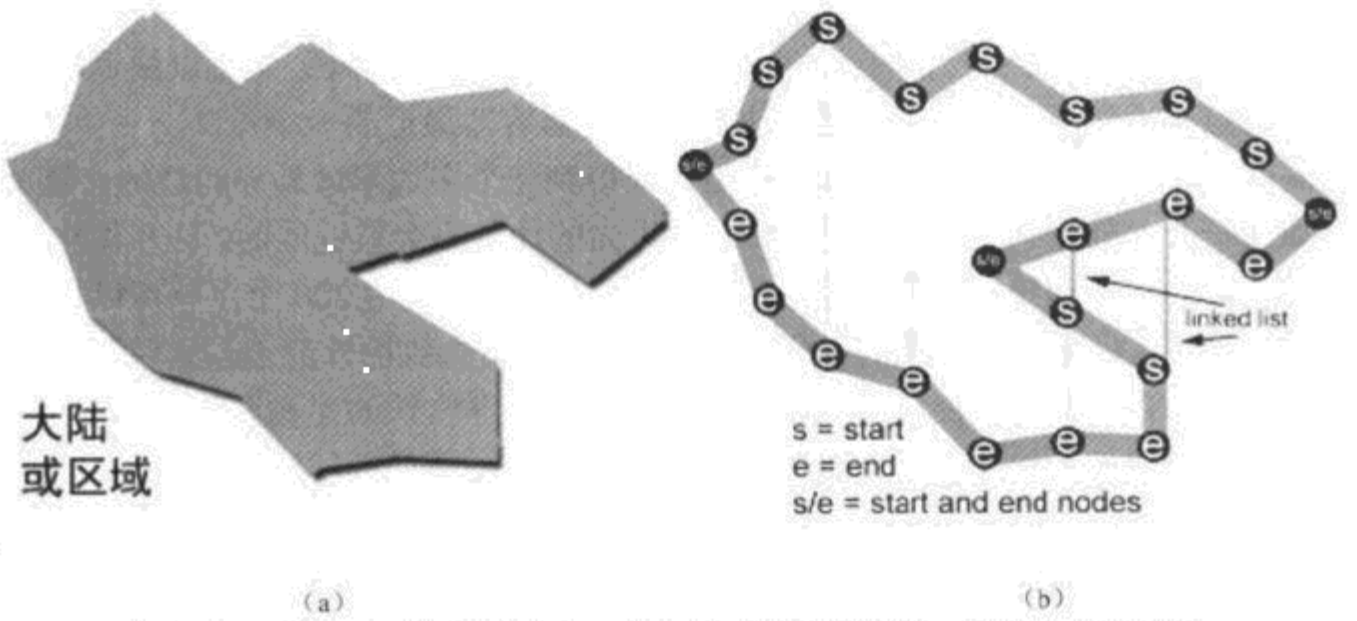


图 3.4.3　（a）一块大陆或区域（b）一个大陆的点是如何保存到一个匹配器容器里去的

1. 匹配器中有些什么？

在匹配器系统中有三种重要的结构——匹配器容器节点、匹配器容器列表头部，还有所有这些对象的管理者：匹配器容器。在匹配器容器里面有一个数组的头部节点，此数组表示的是 x 轴，虽然我们也可以用之来表示 y 轴，或者可以是动态决定的一个轴，如此以便于使用更少的内存。

匹配器容器的数据成员如表 3.4.2 所示。需要注意的是下列成员可以使用一个模板类来表示，但是为了简单起见，它们使用了一个整数容器显示出来。

Download at www.Pin5i.Com

表 3.4.2 匹配器容器的数据成员

类 型	变 量	描 述
Vector<MatchboxListHeader*>	MPointArray	对点在 X 轴上的表示。每个在此数组中的一个元素都是 y 轴上（起始一结束）点范围的链表
long	MOffset	很有可能 x 不是从 0 开始的，但是数组则要从 0 开始，因此我们把所有的 x 都按这个数量进行调整。这就是说，如果我们的点(x, y)的范围是从(10, 50)变成了(25, 99)，那么此处我们的偏移量（MOffset）就是 10
long	MSize	此容器内部的点数

下面列出了匹配器容器的一些方法：

- `BuildPointList()`：将传入的数组用我们所有的点来填充。在某种意义上来说，这就是要把我们所有的点变成一个数组的形式。我们可以传入一个递增量，对于使用模板的版本来说这是很重要的一点，因为如果容器使用了浮点数来代替整数的话，此递增量指示出在生成自己点的列表时应该使用什么数值来对每一个点的 x 坐标进行递增。
- `ComputeCenterPoint()`：此辅助函数使用了一个质心计算的方法来得到所有这些点的中心。
- `Contains()`：如果传入的点存在于容器里面的话（不管是真的存在还是有可能存在），此方法就将返回 true。
- `Create()`：由传进来的点创建我们的容器。
- `GetClosestPoint()`：很有可能最后你会需要这么一个方法，它可以快速得到离传进来的点最近的一个点，或是离传进来的匹配器容器最近的一个点。
- `GetOnlyExtremeEdgePoint()`：这是用来得到一个多边形最外界的边的。这就是说，如果一个大陆是一个环形的话，那么在里面的边是不会被返回的。这在优化一个凸包的计算时是很理想的。在 80 000 个点中，它只需要使用 200 个点就可以构建出一个凸包了，这将是一个极大的优化。这样，基于先前得到的外界点，然后再结合此存储结构中隐含的排序功能，我们就可以编写出一个非常快的建构凸包的方法了。
- `GetRandomPoint()`：通过传进来的随机数生成器，你就可以得到一个随机点生成的方法了，这个方法实际使用的次数可能比你想象需要使用的次数要多得多。
- `Insert()`：我们需要 3 个插入方法：一个方法使用点；另一个使用数组的点；第三个方法则使用匹配器容器。
- `Remove()`：依照传进来的点或是数组的点，此方法将把这些点从容器内部删除掉。
- 操作符：一个有用的方法，可以用来用来模拟数组的遍历。

2. MatchListHeader

此类包含的是一个链表，它里面是容器节点和数量。它主要的工作是用来进行插入、删除和合并节点。

3. 匹配器容器节点

这个匹配器容器节点包含起始 y 和结束 y 的坐标。它里面所有的方法都需要检查传入的

Download at www.Pin5i.Com

点或者坐标是否在其起始与结束范围以内。

一个 Contains()方法大致如下所示：

```
inline bool Contains(long inNum) const
{ return (inNum >= this->mStart && inNum <= this->mEnd); }
```

通过对匹配器节点、头部和容器的结合，你将会得到一个快速的数据结构，它仅用很少的内存就可以进行大量点的处理。对于优化地形分析和其他领域来说来说，它真是一个令人难以置信的有用工具。大多数游戏都使用点进行操作，因此它们都需要在自己的工具箱里添加一个这样的或是类似的结构。

3.4.4　关隘

如前所述，关隘就是那些用来通往其他区域的狭窄通道。在每个游戏里创建关隘的方法各有不同。在 RTS 游戏 Empire Earth 中，当两个或者更多大区域——例如森林、悬崖、海洋，或是地图的边缘——和其他区域离得很近但是却不互相接触的时候，我们就会创建一个关隘。

1. 场景

假设现在是一个明亮的晴天，你正想把一队骑兵和村民送到北部的平原去搭建一个居住地。在开始这段旅程的时候你会感觉愉快而且乐观。你期望着旅途顺利，因为看到沿路敌人的报告的数量极少。

正在旅途之中，马上就要到达一个两旁都是森林的通道了。旅伴们都沐浴在早晨的阳光之下，听着吟唱的鸟儿放声高歌，这条道路也显得非常安全。就在这条乡间小路的中央，鸟儿的歌声突然被一片咒语，还有那如雷的蹄声给掩盖住了，它们似乎铺天盖地，来自四面八方。这条静谧的小路突然消失了，包围你的是如雨的火箭与那冲锋的骑兵！当你命令自己的队伍后撤的时候，尖叫声已经震耳欲聋。你转身后撤，却也只能愁上心头。你只能无助地看着自己的后军在敌人的大部队之下分崩离析。军队在惊恐中溃散，而你只有眼睁睁地看着，震骇之下不能举步，看着本应在你保护之下的那些无助的村民。你用双手堵住耳朵，试图堵住那些曾经是欢声，曾经是笑谑的尖叫声。无路可逃，突然之间敌人到处都是，唯一的选择就是随便选择一方，向前或是向后，从尘土中冲过。你知道必须要做什么，而且立刻就发动了最后的攻击。你马上就成为了尖叫的人群中的一员，高举宝剑，放马前冲，在最后被砍下前的短短一瞬间，甚至还希望能再加一点分数。

2. 策划

电脑玩家是如何策划出这么一个阴险的埋伏呢？一个好的策划是不是碰运气产生的，还是只需要知道地形就可以了呢？可能你已经想到了，关隘对于策划一个埋伏来说是很有用的。让我们从头来一遍，看看电脑是如何策划此次进攻的吧。

首先，电脑必须选择一个关隘以进行埋伏。它或许会随机选择一个关隘，或是选择一个经常经过的关隘，或者选择一个在敌人（真人玩家）和主要城镇之间的关隘。先不管它是如

Download at www.Pin5i.com

何选择出来的，电脑可能会将其伏击部队分为四个作战单元，然后将其部署在关隘中所有相对的外边界点上，如图 3.4.4 所示。

值得注意的是，此中四个黑色 X 所在的地点是距离关隘最远的点。这就是电脑埋伏军队等待敌人通过的地方。它将一直等待，保持侦察，直到敌人走到了关隘的中央（+表示的地方），然后电脑就将发出冲锋号令，进行进攻了！

在 Empire Earth 中，通过关隘，再加上其包含的点和凸包，我们埋伏的策划和执行的任务就会变得很简单了。电脑玩家不需要做很多的 CPU 密集型策划，它只需要选择一个好地点，然后等待其对手的到来，最后发起进攻就可以了。

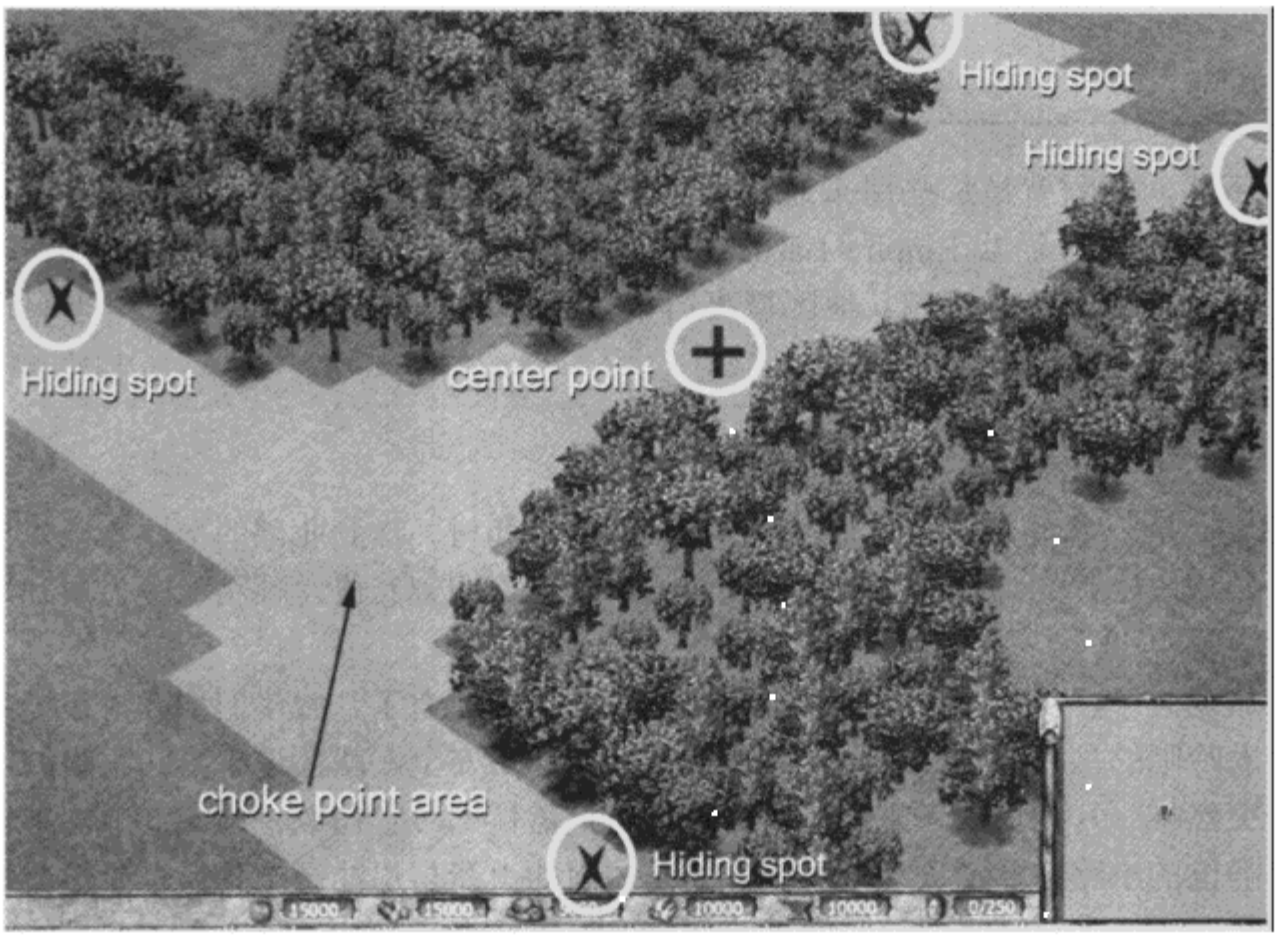


图 3.4.4 一个关隘处的埋伏

3. 查找关隘

查找关隘的算法需要临时性占用大量内存以及 CPU 工作时间，幸运的是，一旦找到了相应的关隘，所有这些就都可以不再需要了。大多数需要内存的地方是与提高算法的速度有关的，因此如果你希望的话，是可以降低性能换取内存的。

在我们描述此算法之前，一定要记住你需要相当多的（主要是整数的）数组，其大小要和对应的世界一样大。用不着害怕，这仅仅是临时存储量，当找到了关隘之后就会被释放的。如果你甚至不能临时分配这么大的内存，可以考虑一下首先把世界划分为各个大陆或是其他的区域来试试，然后使用这些区域或是大陆的边界框来作为临时的世界地图。

此算法的思想就是要给每个地形赋予一个影响力数值（在本节剩下的部分中将被称之为“氛围值”），此值可以增长。此算法执行的次数决定了此氛围值增长的具体数值。如果两个或是多个氛围值互相相交，那么它们就会形成一个关隘。

Download at www.Pin5i.Com

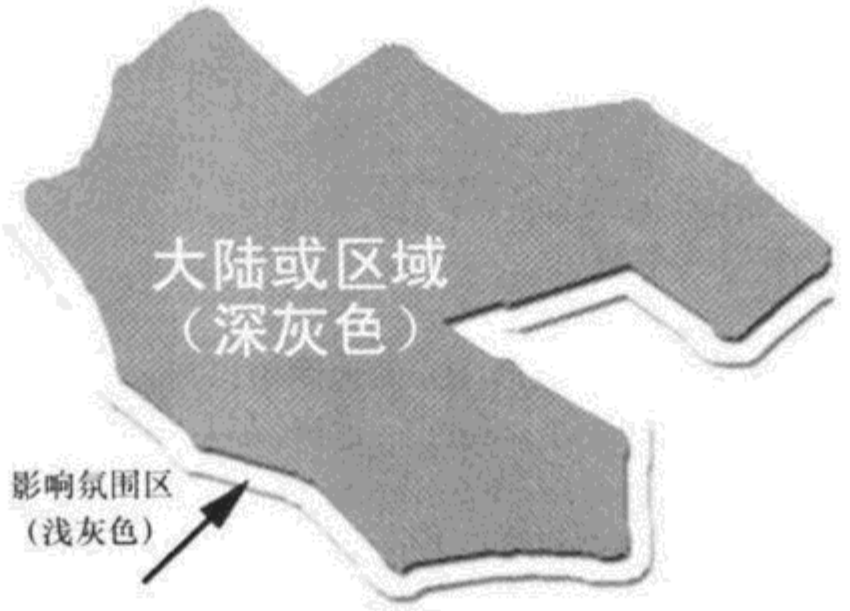


图 3.4.5 一个区域（深色）及其影响氛围（浅灰色）

此算法的主循环部分如下所示：

```
// copy the original map into choke point map.
CopyOriginalMapIntoChokePointMap(inMap,theOutMap);

// inPasses controls how much area's auras grow.
for(theLoop = inPasses - 1; theLoop > 0; theLoop--)
{
    // copy the map. (uses cached map from above
    // the loop)
    CopyOriginalMapIntoChokePointMap(inMap,
        theResultMap);
    // apply the algorithm
    ApplyChokePointAlgorithmToMap(theOutMap,
        theTempMap, theMaxX, theMaxY));
}
// create the final choke points.
CreateChokePointAreas(inMap);
```

在主循环开始以前，我们必须首先生成，然后缓存一个障碍物的地图。如果图中的某个分片中有障碍物，那么此片地图就会得到堵塞代价值（对于你游戏而言的任何适当的值），否则得到为 0 的值。

接下来对世界中的每片地图进行如下操作：

```
// cache the XY array position
long theXY = ((theXLoop * theMaxX) + theYLoop);

// if this tile is blocked, fill in the areas id
if(ChokePoint::IsBlockedTile(inMap->GetTile(theXLoop,
    theYLoop), theXLoop, theYLoop, theAreaID))
{
    // store the blocking area's id and set a cost.
    ChokePoint::sTerrainIDMap[theXY] = theAreaID;
```

Download at www.Pin5i.Com

```
        ChokePoint::sCostMap[theXY] = kBlockedCost;
    }
    else
    {
        ChokePoint::sTerrainIDMap[theXY] = 0;
        ChokePoint::sCostMap[theXY] = 0;
    }
```

把这些值缓存到静态的代价值/地形 ID 的地图中以后，下面 CopyOriginalMapInto ChokePointMap()的调用就只不过是对 sCostMap 进行 memcpy()了。

真正的算法在 ApplyChokePointAlgorithmToMap()方法中。这个方法必须对世界中的每一片（在此处，分片指的是有障碍物和边界上的分片地图）进行循环。如果一片地图中没有障碍物，那么对于此分片的每个相邻分片，我们将得到其所有非堵塞分片的代价值之和。要得到每个相邻片的代价值，我们只需要“当前代价图”就可以了。这就是说我们可以从上次算法的迭代中得到所有的代价值，通过使用前一次循环的代价值结果，氛围值就增长了。

当在相邻的分片中进行循环的时候如果我们得到了一个有代价值的分片，那么我们便会将其 ID 给缓存起来。这可以告诉我们氛围值是由于哪个分片而增长的。不过，这也仍然不够，因为我们还需要知道这个代价值是由我们的氛围值本身产生的，还是由其他来源产生的。

随着氛围值的增长，它们将对其影响到的分片加上一个标记。如果氛围值之间有重叠的部分，相应的分片就会被标记上一个“混合区域”的 ID。如果我们发现在相邻分片中有了一个混合区域的 ID 或是一个不是自己 ID 的氛围值 ID，那么这个代价值将被保留下来。否则，如果相邻分片里面没有来自混合区域氛围值的代价值，而且也没有来自其他区域氛围值的代价值，我们将把相应的代价值设置为 0。这可以防止一个区域创建一个只与自己相邻的关隘。

在我们分片循环结束的地方，每个分片将取得其相邻分片的代价值之和（如果不是从混合区域氛围值，也不是从其他氛围值而来，则此值为 0），然后将此值除以代价值不为 0 的相邻分片的数目。这么做以后，可以把关隘变成一个沙漏斗的形状，不过你也可以对之进行改造得到其他的形状。

在进行了所有这些需要的操作以后，你将会得到一个积分数组，其中的每个积分可能为 0、为堵塞值，或者为其他的某些积分。最后一步就是要收集相邻分片的积分，再根据它们创建一个关隘。为了解决此问题（以及其他的地形分析问题）有个非常好的工具，那就是在“人工智能的智慧”[Higgins02a]中描述的 A*算法。如果你不需要实现 A*算法，也可以使用填充算法来得到这些积分以创建此关隘区域。

此算法看上去好像很复杂，但是经过了正确的优化以后，它是可以运行得很快的。

3.4.5 进行地形分析

当你编写一个地形分析引擎的时候，成功在很大程度上取决于它的通用性和优化性。在 Empire Earth 中，我们使用了一个 A*算法，它是一个高度优化而且通用的工具，从寻径单元到创建森林区域以及其他的地形分析任务中，它都可以得到广泛的使用。

Download at www.Pin5i.Com

3.4.6　结论

当然了，在创建一个 RTS 引擎的时候，一个主要的任务就是地形分析。成功的游戏没有完全的地形分析也是可以发布的，但是如果开发人员得到的工具越多，游戏的特性也就会越丰富，越先进。地形分析需要一个高性能的、通用的引擎，而开发这么一个引擎则需要很多的时间。然而，如果做好了的话，而且如果对地形信息的使用也比较有新意的话，这个投资是会产生数倍的回报的。

3.4.7　参考文献

[Higgins02a] Higgins, Daniel F., "Generic Path-finding," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

[Higgins02b] Higgins, Daniel F., "How to Achieve Lightning Fast A*," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

[O'Rourke98] O'Rourke, Joseph, *Computational Geometry in C*, Second Edition, Cambridge University Press, 1998.

Download at www.Pin5i.Com

3.5 一个针对 AI 代理、对象，以及任务的可扩展触发器系统

Steve Rabin
Nintendo of America, Inc.
steve@aiwisdom.com

如果你的玩家玩腻了单机版游戏，继而想在网上搜索更多的关卡或是关卡编辑器，那么你应该早已为此做好准备，以便能够满足他们的要求了。可扩展的关卡与任务都是一个设计良好的游戏所拥有的显著特点，而且有了它们之后，你游戏的生命力也会更强。不管你的游戏是 RTS 游戏、RPG 游戏还是动作游戏，它们都应该能给玩家提供某种手段以让其能对关卡进行定制，能够创建新的征服之地。

Baldur's Gate、*StarCraft* 和 *Dungeon Siege* 都属于那种可以让玩家创建新任务的好游戏，而且在某种情况下，还能让玩家对其人工智能引擎进行修改和扩展。然而，玩家并非开发人员，而且你也不应该强迫他们去学习一种虚构的编程语言，甚至对其结果进行调试。如果你想要自己游戏中平均程度的玩家能改造游戏，那么简洁性是一个关键的因素，而通过实现一个可扩展的触发器系统，我们就可以达到此目标了。

3.5.1 触发器系统简介

触发器系统是专门做某件事情的一段集中的代码：它将对各个条件进行检验，继而执行相应的事件响应。如果有一组条件满足了，那么对应的一组反应就会被执行。这个简单的系统结构优秀、易于实现，而且很容易对之使用数据驱动的方法[Rabin00]。它可以用来解决各种不同的问题，尤其适合让设计人员和玩家进行修改。最棒的一点是，有了它以后，我们就可以更容易地创建一个好玩的、崭新的、交互式的环境让玩家来探险了。

假设有这么一个游戏，在其中你与一群勇士正在探险，在地下城中进行探索。在你走到地下墓穴的时候，一个柱子倒了下来，几乎把领队砸成了肉泥。当你走到一个恢宏的石门前面的时候，一股寒流突然扑灭了你的火把。在点燃了最后一个火把之后，你可以看到其上的篆文如下所示："历经此门之人，无不心思沉重"。经过一番考虑，你让队员们站到心形的地板上面，于是随着一阵响声，石门缓慢地打开了。

Download at www.Pin5i.Com

上述的每一个事件都可以使用触发器系统中的条件—响应模型进行描述。当队员走到一个特定的柱子对象旁边 1m 之内的时候，此对象对此事件的响应就是倒下。当队伍走到了门前 2m 以内的时候，相应的响应就是播放一阵刮风的音效，然后扑灭周围的火把。当每块心形地板上面都站上了一个玩家的队友之后，其响应就是打开石门，播放一阵石头摩擦的音效。

3.5.2　对象自有的触发器系统

虽然一般可能会用主触发器系统（正如[Orkin02]中所述），但是或许我们还可以使用一个更强大的架构，那就是可以考虑为每个代理（agent）、对象或任务加上一个其自有的触发器系统类。并不要求每个对象都有这么一个系统，不过如果能有这么一个触发器系统的实例的话，对象就可以把自己的数据封装在内部，这样就可以使得系统更加灵活，更加面向对象了。另外，也可以给特定的物体加上触发器，这也是一个很自然的想法，而且对于玩家来说也很容易掌握。

现在让我们来讨论一下那个当有人走近时就倒下的柱子。我们可以在关卡编辑器里面定义这么一个柱子，然后为之添加上一个触发器的定义，此触发器的行为就是使物体倒下。然后，设计人员或是玩家就可以在游戏里面放置很多这样的柱子了，它们的行为都出奇地相似，因为此对象给直接加上了一个触发器行为。通过使用这种方法，每个代理、对象，或是任务都可以加上一个触发器系统，而此系统则可以完全为此实体所拥有。

3.5.3　定义条件

条件可以是任何一个你在游戏中进行量化的事件或是状态。条件在可执行文件里面是固定的，但是可以对之使用参数或是关卡编辑器进行高度自由的配置。下面列出了一些可能的条件：

- 玩家离点(x, y, z)的距离在 R 之内；
- 玩家在一个闭合区域(x, y, z)的范围之内；
- 一个敌人逼近了玩家；
- 玩家的生命值低于 X%；
- 玩家的储备箱里有对象 X；
- 玩家装备了对象 X；
- 玩家被杀；
- 玩家达到了 X 等级；
- 玩家与人物 X 进行对话；
- 玩家杀死了敌人 X；
- 玩家收到了 X 消息。

3.5.4　使用布尔逻辑组合条件

为了得到更加灵活的条件限制，我们最好使用布尔操作符，例如 AND、OR、NOT，或

Download at www.Pin5i.com

是 XOR，将条件关联起来。举个例子，如果只有当你装备了冰剑、冰盾，以及寒冰盔甲的时候门才能开启的话，这些条件就需要用 AND 符连接起来。如果当你拿着银钥匙或者骷髅钥匙的时候门可以打开，那么这些条件必须用 OR 符连接起来。图 3.5.1 和图 3.5.2 使用一个树结构展示了这些条件的组合。

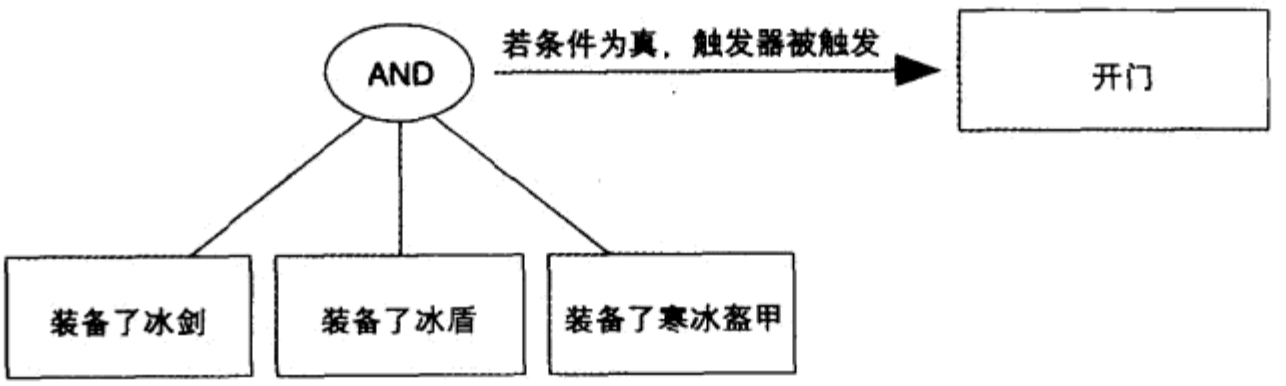


图 3.5.1 如果 3 个条件均为真，则开门

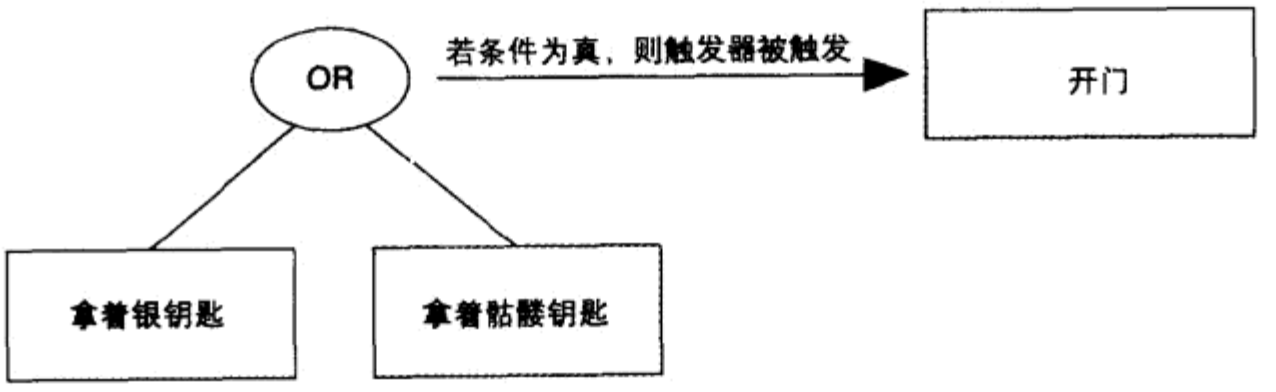


图 3.5.2 如果任一条件为真，则开门

如果条件是要求你装备了冰剑、冰盾，以及寒冰盔甲的同时还需要银钥匙或者骷髅钥匙的话，此时的情景就更有意思了。图 3.5.3 展示了这种配置。

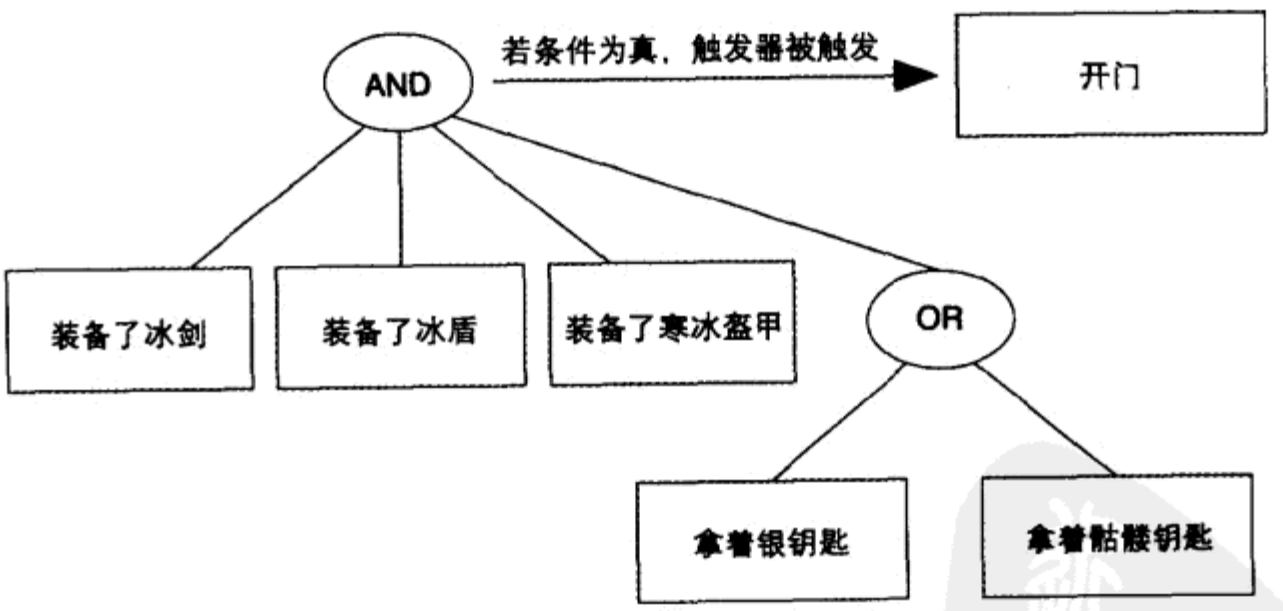


图 3.5.3 更复杂的一组开门条件

从这三张图中得到的视觉形象是很重要的，因为它可以为以后的代码编写提供一个结构。如果每个元素都是一个类，那一共就有两种类：Operator 类和 Condition 类。我们可以对 Operator

Download at www.Pin5i.Com

类进行设置让其在表现上和任何布尔操作符一模一样，它里面还有一个列表，其中包含的是指向 Operator 实例的指针或是 Condition 实例的指针，后者是一元布尔操作符的对象。Condition 类可以用来对任何需要检查的条件进行求值，且包含一些参数，以用来对其进行定制。

3.5.5　定义响应

响应可以是游戏中的任何一个你想改变的状态或是动作。和前面一样，这些也是在执行文件中设置好了的，但是它们也可以使用参数或者是关卡编辑器对之进行定制。下面列出了可能的响应：

- 关卡/任务结束了；
- 对玩家造成 *X* 点的伤害或是治愈 *X* 点的生命值；
- 给玩家 *X* 个经验值；
- 关门、开门、锁门以及打开门锁；
- 产生 *X* 个 *Y* 类型的生物；
- 杀死 *X* 个生物；
- 播放音效；
- 从一个列表中随机挑选音效进行播放；
- 有 *X*%的几率播放音效；
- 把玩家或是敌人 *X* 移动到位置(*x*, *y*, *z*)处；
- 在某地或是对玩家/敌人施加特殊效果；
- 让玩家/敌人 *X* 突然着火；
- 对玩家/敌人 *X* 下毒；
- 把玩家/敌人 *X* 打晕；
- 使玩家/敌人 *X* 进入无敌状态；
- 使玩家/敌人 *X* 进入隐身状态；
- 把开关扳回原位；
- 向玩家发送消息 *X*。

如果某组特定的条件得到了满足，相应的响应就会被执行了。不过，我们可以对之进行扩展，以让其响应包含一组响应，而不仅仅是一个响应。使用了这种方法之后，当条件被触发了以后，它就可以同时对多个事件造成影响，或是随机选择其中一个进行响应了。

3.5.6　求取触发器的值

当我们通过条件定义了一个触发器以后，我们还需要一个框架取得其值以得知何时应该触发这个触发器。第一个需要考虑的地方就是要决定一个特定条件应该是事件驱动的（触发器系统将等待得到一个事件的通知），还是应该对世界进行轮询（每隔几个游戏时钟周期就对条件的真值进行检查）。在实际环境中，你会希望能有比较大的灵活性，可以随意选择创建事件驱动或是轮询模式的条件。如果能把这两个模式都加入进来那就最好了。

对于事件驱动的条件来说，我们需要一个接口以让事件能进入此触发器系统。最简单的

Download at www.Pin5i.Com

方法就是使用事件消息。事件消息只是一个简单的通知以通报某个事件发生了，另外此消息还携带了相关的数据。如果想要对事件消息有更深的了解，请参阅[Rabin02]。

对于轮询类型的条件，我们可以在触发器系统内部调用一个更新函数以便让每个轮询条件自己去完成相应的工作。而事件驱动的条件则会忽略这个函数调用。

不管在触发器系统中进入了一条事件消息或是进行了一个轮询更新函数的调用，我们都需要采用一个方法将其与整个条件判断关联上。图 3.5.4 显示了一个例子，在此例中有一组同时需要事件消息和轮询的条件。左边的条件在等待发生一个碰撞事件，而右边的条件则在收到了轮询更新调用的时候对条件进行查询检查。

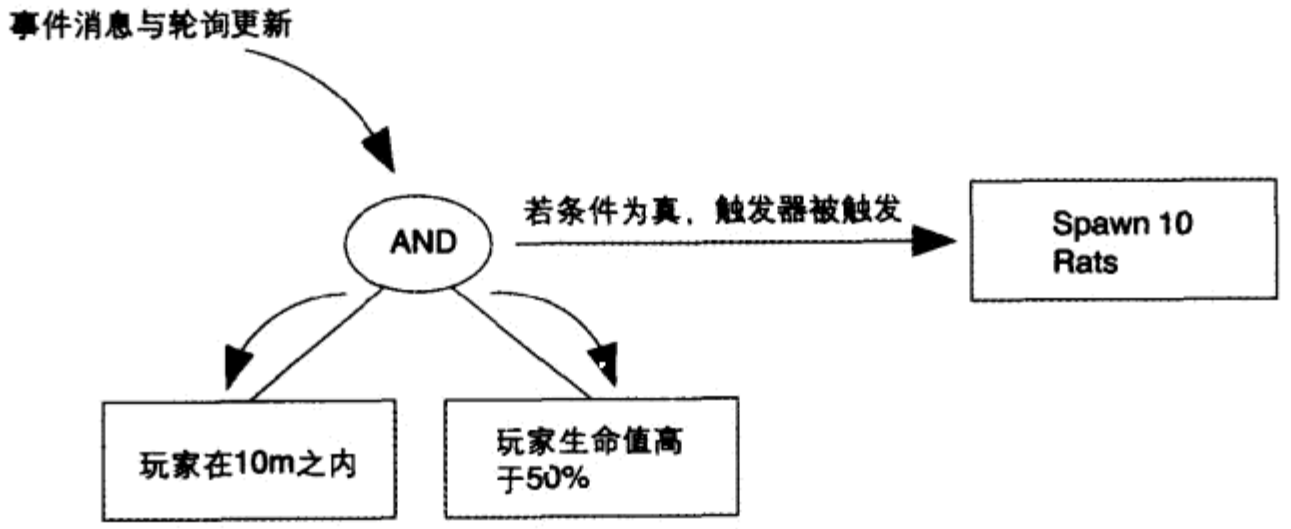


图 3.5.4 一个包含了事件驱动条件（左下）与轮询条件（右下）的触发器系统的例子

当事件消息或是轮询更新进入触发器系统的时候，它将被传递到每个触发器的根 `Operator` 实例中去。此 `Operator` 将把它传递给其子节点，期望得到一个“true”或是“false”的返回值。每个子节点还将把它传递给自己的子节点。当其达到了一个 `Condition` 实例的时候，子节点就会根据条件的新状态返回“true”或是“false”。接下来每个 `Operator` 实例都将对返回值进行求值运算，然后将结果返回给父节点。

一定要注意，`Operator` 类在给子节点传值的时候应该使用快速求值法。如果根据操作符有任何一个条件没有被满足的话，此触发器的检查就会被立即放弃掉。对于在图 3.5.4 中的例子来说，一个事件消息被传递到左边的条件上，而刚好得到了“false”的结果，这就应该导致此事件消息不再被送到右边去了。这将对减少处理量有所帮助。

在使用事件驱动的条件时还有另一个重要的事情，那就是一定要注意它们必须得记住其事件，一直到被重置为止。对于图 3.5.4 中的例子来说，如果玩家走到了 10m 的范围以内，此时就应该有一个碰撞事件被送到触发器中间去。此条件应该记住这个事件，而且在以后的事件消息或是轮询更新中保持返回“true”。

在某一个点上，为此特定触发器设置的条件将都返回适当的值，使得触发器被触发。当一个触发器被触发的时候，它就会记住这一点，以便以后不会再被重复触发。

3.5.7 一次性触发与载入次数

所有触发器都应该有两个额外的由设计人员定义的属性：

Download at www.Pin5i.Com

```
bool SingleShot;  //此触发器是否只应该被触发一次
float ReloadTime; //如果可以触发多次，那么需要多久才能被重置
```

这两个属性可以使得一个触发器被多次触发。SingleShot 属性决定了此触发器是否应该只触发一次还是可以触发多次。如果 SingleShot 是“false”，那么 ReloadTime 就决定了在触发器重置其所有条件和再次接受事件之前所需要的时间。

3.5.8　使用标志和计数器将触发器结合起来

只有当一个触发器能够到达某些中间状态，而且这些状态可以为此系统中的其他触发器所能访问到的时候，这些触发器才能结合起来。因此，每个触发器系统都在其内含有一组标志和计数器以用来跟踪那些被触发的触发器。为了得到一个尽可能通用的结果，我们可以让每个触发器创建任意的标志，通过字符串来访问它们。触发器系统将会在一个标志需要被访问或是被设置的时候创建它，然后将其保留下来一直到系统被销毁。

考虑一下下列的新条件：

- flag_name 是 true 还是 false?
- flag_name 是奇数还是偶数?
- flag_name1 逻辑与 flag_name2 是否为真?
- flag_name1 逻辑与 flag_name2 的逻辑非是否为真?
- flag_name1 逻辑或 flag_name2 是否为真?
- flag_name1 逻辑异或 flag_name2 是否为真?
- flag_name1 逻辑与 flag_name2 再与上 flag_name3 是否为真?
- flag_name 的计数是否为 *X*?
- flag_name 的计数是否多于 *X*?
- flag_name 的计数是否少于 *X*?

考虑一下下列的新响应：

- 对 flag_name 所表示的值加 1；
- 对 flag_name 所表示的值减 1；
- 将 flag_name 所表示的值设置为 *X*；
- 将 flag_name2 所表示的值设置为 flag_name1 所表示的值；
- 对 flag_name 所表示的值做逻辑非运算；
- 将 flag_name 所表示的值设置为 TRUE；
- 将 flag_name 所表示的值设置为 FALSE。

通过这些标志和计数器，我们就可以在触发器系统中做下标记或是对事件进行计数了，例如可以纪录玩家对一个特定的区域访问的次数。此外，现在的触发器系统中可以添加一种能对事件序列作出响应的触发器，例如以某种特定的顺序踩过三块地板的事件。由于这些标志和计数器都含有状态信息，因此现在我们就可以使用很多种触发器了。

在图 3.5.5 中的示例展示了当一个玩家无法通过某特定的门，而且在绝望之中反复访问几个区域的时候，游戏应该如何响应这种事件，给玩家一个提示信息。值得注意的是这里一共有三个分离的触发器，它们通过一个叫做“Visited”的计数器结合在一起。

Download at www.Pin5i.Com

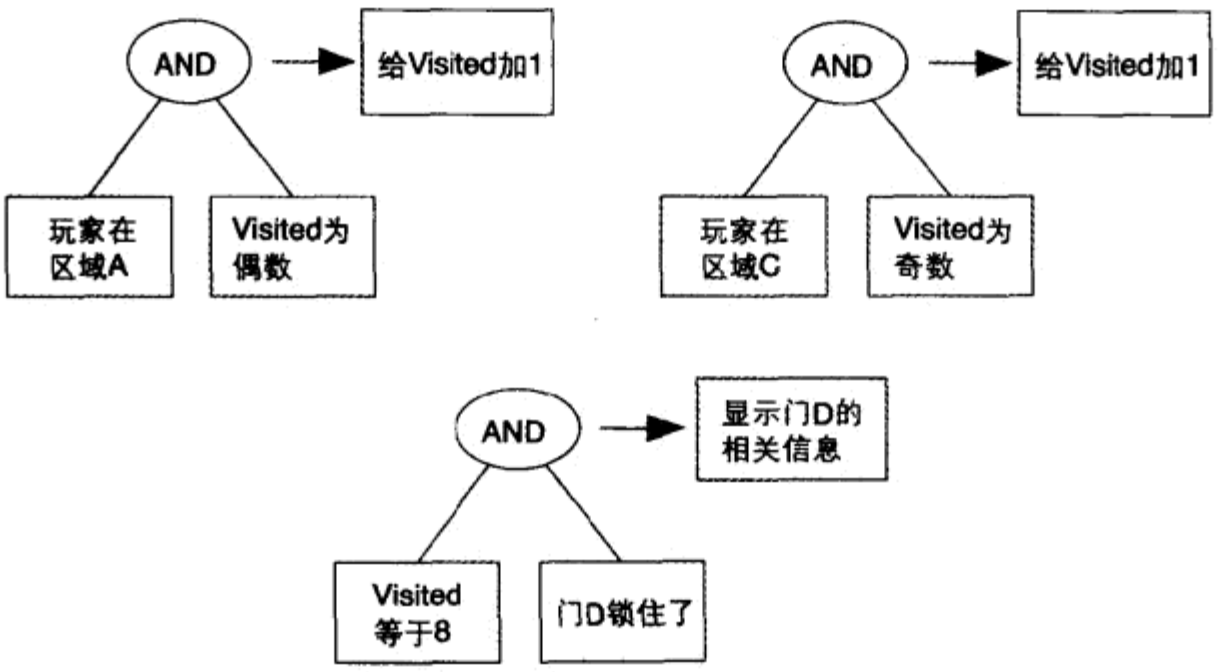


图 3.5.5 3 个访问 visited 变量的触发器如何一起工作的。若玩家来回访问了区域 A 与 C 共 8 次而没有开门，就显示一条提示信息

在加入标志和计数器之后，需要注意现在的触发器系统变成了一个非常类似黑板系统[1]的东西[Isla02]。标志和计数器构成了黑板，而触发器则是知识源（KS），它们对黑板数据进行操作。然而，由于触发器主要是与黑板外界来的数据打交道的，因此一般说来，触发器系统并不是一个黑板系统。

3.5.9 触发器系统与脚本语言的对比

或许你已经发现触发器系统的功能，尤其在加上了状态信息以后，越来越像脚本语言的功能了。虽然它们的功能有所雷同，但是只要经过一番思考，在触发器系统与一个全方位的脚本语言进行了比较以后，就可以得到下面列出的种种使用触发器系统的优点：

- 触发器系统能在 GUI 界面里面完全定制。脚本语言极少能够提供如此简单和健壮的编辑特性。而且你也不必担心什么语法错误，因为触发器已经定好了针对条件与响应的结构。
- 触发器系统对用户而言更方便。触发器的概念很容易理解，而且会有更多的人去使用它。
- 触发器系统是有边界限制的。由于触发器系统已经做了很好的限制，因此用户操作一般不会导致游戏的崩溃。这是因为触发器只会执行一组在很小范围内的、经过良好测试的、仅限于有限几种的动作（响应）。
- 触发器系统在实现和修改方面都很快。实现触发器系统相比使用完整的脚本语言的实现来说只需要很短的一部分时间。如果触发器系统需要几个星期来实现，那么脚本语言则需要几个月甚至几年才能实现[Tozour02]，[Brockington02]。
- 对触发器系统来说，其文档记录工作比较容易。相对而言，一个触发器系统在编写文

译者注[1] 黑板系统（black board）是人工智能中动态规则学习的一种系统，可以参见作者 Nilsson 的《人工智能》一书。

Download at www.Pin5i.Com

档和例子方面比较容易。这是因为如果你想在游戏发布了以后，用户能自己改造关卡，那么这份东西你总是要写的。

3.5.10　局限性

此处描述的系统的主要局限性就是扩展性不够好。不过，我们可以添加一些额外的代码来去除无关的触发器。其中相邻裁减（proximity culling）被证明是一种相当有效的方法。

此系统的另一个局限性在于定义条件和响应的词汇在执行文件中已经被固定死了。因此，如果想要进入代码内部则需要开发人员进行特意的改动。这也是一件好事，因为它能帮助你保护自己的游戏以防止随机的，或是恶意的改造行为。

3.5.11　结论

对于很多开发进度紧张的游戏来说，一个可扩展的触发器系统的开发可能不太现实，但是它确实是一个有用的特性，可以为你的游戏增值并增加其深度。此外，有了可扩展的触发器系统，关卡设计人员就不用非得成为熟练的开发者才能设计情节了。虽然乍看上去触发器系统的解决方案好像太简单了，但是我们的目标是要让设计人员和玩家能够使用它。如果越容易设计内容和关卡相关的情节，那么你的游戏也就越大，你的玩家也就会越高兴。

3.5.12　参考文献

[Brokington02] Brokington, Mark, and Mark Darrah, "How Not To Implement a Basic Scripting Language," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

[Isla02] Isla, Damian, and Bruce Blumberg, "Blackboard Architecture," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

[Orkin02] Orkin, Jeff, "A General-Purpose Trigger System," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

[Poiker02] Poiker, Falco, "Creating Scripting Language for Non-Programmer," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

[Rabin00] Rabin Steve, "The Magic of Data-Driven Design," *Game Programming Gems*, Charles River Media, Inc., 2000.

[Rabin02] Rabin, Steve, "Enhancing a State Machine Language Through Messaging," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

[Tozour02] Tozour, Paul, "The Perils of AI Scripting," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

Download at www.Pin5i.Com

3.6 基于 A*算法的战术式寻径

William van der Sterren
CGF-AI
william@cgf-ai.com

如果所有的游戏仅仅要求从点 A 到达点 B 的话，那么 AI 只需要提供一条最短的路径就可以了。一个标准的 A*算法就可以满足此要求。然而，除了到达点 B，经常还要求 AI 在途中避免被看到或是被射击到。此要求被称为战术式寻径（tactical path-finding），在此时 AI 需要进行权衡，在满足较短行走时间的同时还要考虑到避免被敌方侦察到以及被射击到（参见图 3.6.1）。不管是在处理坦克排撤退到山脊背后或是在驾驶 X 战机穿过密集火力封锁的时候，AI 都需要考虑到敌方的位置和他们的火力线。

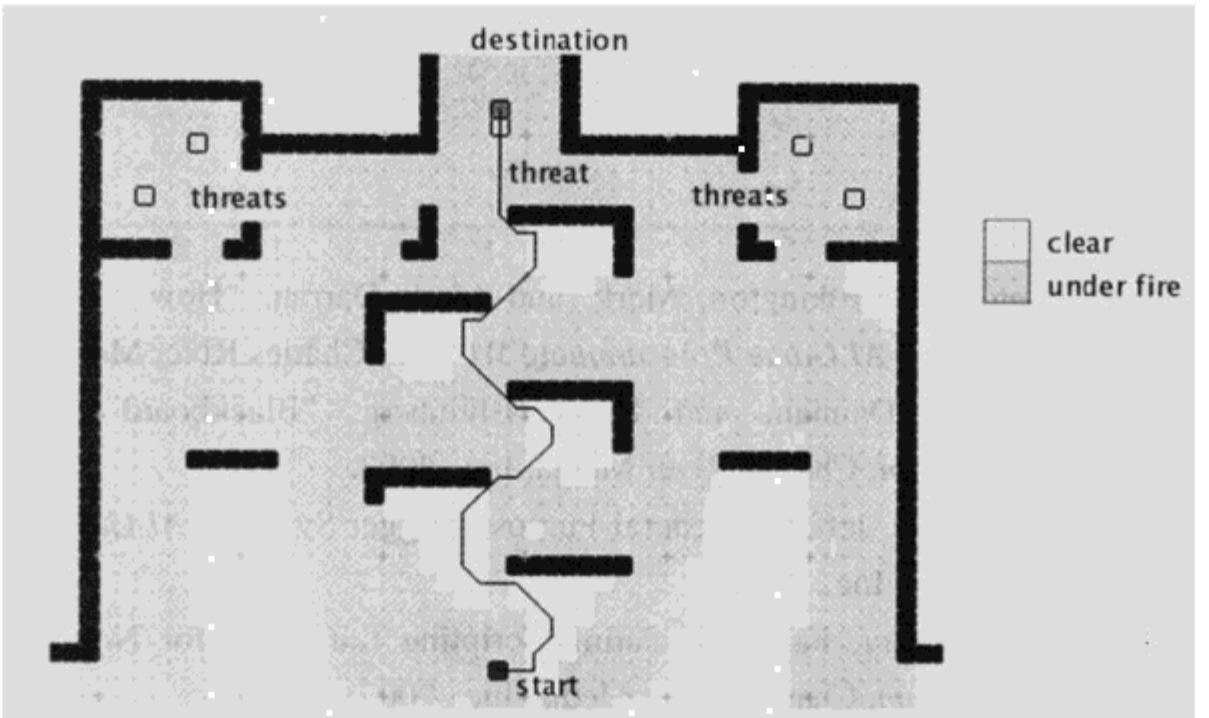


图 3.6.1 一条时而隐蔽、时而暴露的路径

使用了战术式寻径以后，不仅游戏显得更加真实了，而且对于玩家来说，其对手也更加精明，更加难以预测了。本节就将帮助你对 A*算法进行扩展以得到一个战术式的寻径算法。

首先，我们将对“最短路径”的 A*算法进行一点改造，对于那些暴露在敌方火力侦察或是覆盖之下的位置，我们会将其代价值增大。其结果就是，A*算法将生成一条尽量避免敌人侦察或是射击的路径。不过，这样产生的结果一般说来比较假，从战术上来说也并不安全。

Download at www.Pin5i.Com

接下来我们将对其中一些有缺陷的战术式路径考察一番，指出其缺陷的原因所在，以及在我们的 A*算法里面应该如何对之进行纠正。然后，我们还将比较一下战术式寻径与最短寻径之间计算代价值的异同。显然，战术式寻径将付出更多的代价，但是在 A*搜索和战术式寻径必须要计算的火力线估测方面，我们将采取一些技术与技巧以减少其开销。

ON THE CD

在 CD-ROM 上，我们为你提供了 James Matthews 的 A*探索器工具[Matthews01]的扩展版。通过此工具，你就可以对战术式 A*算法进行试验了（可以依照本节中的例子进行）。

3.6.1　有风险的 A*

A*是一个通用的搜索算法。为了在我们的 AI 中进行寻径，需要为之提供一个代价函数和一个启发式函数[Stout00]。对于寻径来说，代价函数计算的是从一个点到另一个点的所需的精确代价，这些代价是由通过的距离以及其地形允许的行走速度计算出来的。

一般说来，寻径算法的启发式函数提供的是一个到达目的地的估计代价值，例如它可以是到目的地的直线距离除以最大速度的值。代码列表 3.6.1 展示了一个代价函数与一个启发式函数。

LISTING 3.6.1　评价最短路径的代价函数与启发式函数

```
float MovementCostNodeToNode(node* aFromNode, node* aToNode) {
    float dist, fromMoveCost, toMoveCost;
    dist = (aFromNode->origin -
        aToNode->origin).Length();
    fromMoveCost = aFromNode->GetLocalMovementCosts();
    toMoveCost   = aToNode->GetLocalMovementCosts();

    // take the average
    return kTravelTimeFactor * dist *
        (fromMoveCost + toMoveCosts) / 2;
}

float HeuristicNodeToDestination(node* aToNode,
        node* aDestination) {
    float dist;
    dist = (aToNode->origin -
        aDestination->origin).Length();

    return kTravelTimeFactor * dist *
        (minimalMoveCostForAnyLocation);
}
```

使用如上的代价函数和启发式函数将能够找到并返回一条通往目的地的最低代价的路径。典型说来，AI 感兴趣的是那条能最快到达目的地的路径，因此代价值一般表现为时间。

现在，如果我们为那些暴露在敌人视线与火力线之下的位置再增加一些额外的代价值，那么生成的路径就会使玩家避免被侦察到或是被敌方火力射击到。通过使用不同类型的代价值[Reece00]，我们修正后的 A*算法就可以自动地在最短路径和通过风险区域之间取得平衡了。

Download at www.Pin5i.com

LISTING 3.6.2 战术式路径的代价函数

```
float TacticalCostNodeToNode1(node* aFromNode,
      node* aToNode) {
   float travelTime, riskFrom, riskTo, riskTotal;
   travelTime = MovementCostNodeToNode(aFromNode,
      aToNode);

   // use duration of move, and average risk of
   // both locations
   riskFrom  =
      GetRiskOfEnemyObservationOrFire(aFromNode);
   riskTo    =
      GetRiskOfEnemyObservationOrFire(aToNode);
   riskTotal = (riskFrom + riskTo) / 2 * travelTime;

   // return the weighted combination of travel
   // and risk costs
   return kTravelTimeFactor * travelTime +
      kRiskFactor * riskTotal;
}
```

在代码列表 3.6.2 中，你将发现一个更加战术化的 A*代价函数的例子。其额外的代价值是由每个节点上采样的风险以及从一个节点到另一个节点需要的行走时间决定的。如果两个节点之间距离不长，那么这个值就比较逼近从一个节点到另一个节点的总风险值。不然，你可能还需要在两个节点之间的某一个位置对风险值进行采样。

`GetRiskOfEnemyObservationOrFire()`函数返回的是一个给定节点的风险值，这需要考察从所有已知或是假设的敌方位置上是否可以对这一点进行侦察或是射击。一般来说，这需要在游戏世界的几何图形中进行一番投射操作。由于投射操作的代价较大，因此本节也将讨论如何创建较小的查找表以得到预先计算出来的火力线信息。

代码列表 3.6.2 显然没有提供一个战术式版本的启发式函数，这是因为我们无法预知路径中余下区域可能遇到的风险，而只能对剩余的时间进行估测。此处的`HeuristicNodeToDestination()`函数就是如此实现的，因此我们这里就不谈它了。

对图 3.6.2 产生的结果进行一番考察。在图 3.6.2a 中，你看到的是一个传统意义上的最短路径，它毫无畏惧地穿过了那些暴露在敌方下的区域（灰色区域）。在图 3.6.2b 中，你看到的是`TacticalCostNodeToNode1()`代价函数产生的结果。仅仅在为那些受火力压制的区域加上了风险代价之后，我们就可以得到一条更聪明、更可信的路径，在速度和火力覆盖之间取得一个平衡了。

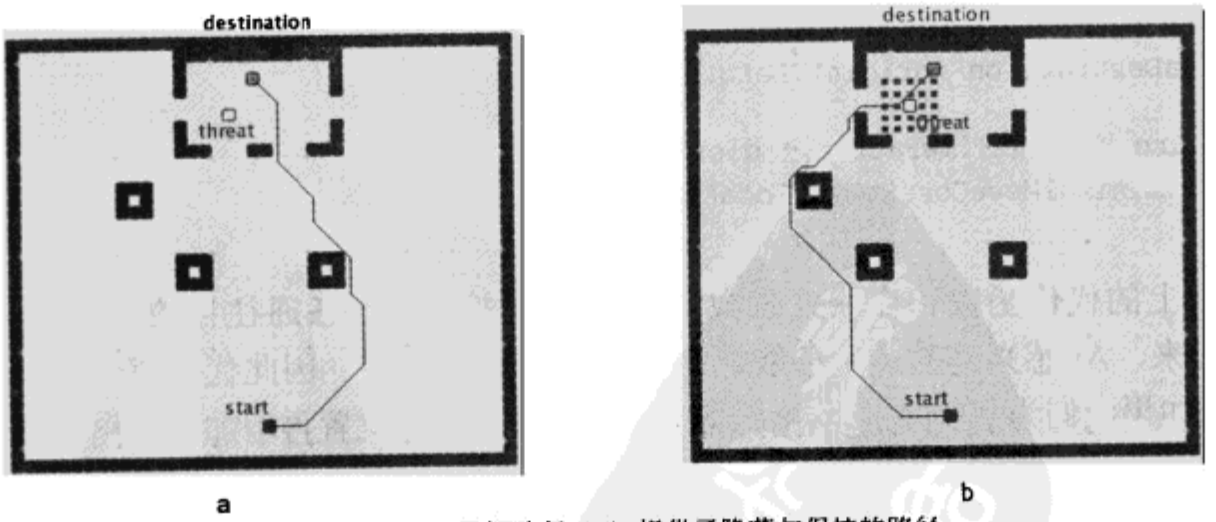


图 3.6.2 （a）最短路径（b）提供了隐蔽与保护的路径

Download at www.Pin5i.com

我们还没有达到最后的成功呢！在很多情况下，这个简单的战术式A*代价函数会产生战术上有缺陷的路径，从而打破了它的智能幻相。此算法还有一个问题，那就是在寻径中采用的“战术方法”将带来较大的（CPU）开销。

在本节剩下的小节中，我们将首先对代价函数中的一些战术性缺陷进行识别与纠正，接下来，对战术式寻径中的额外开销进行考察，然后得到限制此开销的一些方法。

3.6.2 对于有缺陷路径的战术式改良

即使计算机的 AI 算法只有一个缺陷，也足以暴露出其不足了。当玩家带领着僚机、小分队或是他的坦克排进入伏击区的时候，如果发现由于缺乏战术的理解而导致了其队伍被袭击，会让玩家非常恼怒的。不幸的是，我们前面定义的战术式代价函数在某些地方就刚好缺乏了这种理解力：

- 它不能对长期暴露在火力下和短期暴露在火力下的情况进行区分，只能简单地把所有的代价值加起来。
- 它假设在整条路径上火力威胁是静态的。

我们将对这两个问题都进行考察，然后得到相应的对战术式代价函数的改善以解决这些问题。

3.6.3 暴露时间与对敌人建模

假设有这么一个直升飞机，它需要通过一个被地对空导弹防御的区域。再假设此直升飞机可以通过同样长度的两条路径到达目的地。一条路径将使直升飞机仅仅暴露一次，不过时间将长达 20s。另一条路径将同样使直升飞机暴露 20s，但是要暴露 4 次，每次 5s，且每次暴露之间间隔 5s。对于我们的代价函数 `TacticalCostNodeToNode1()` 而言，两者的代价是一样的，因为此函数只是简单地把暴露的时间加起来而已。那么，直升飞机究竟应该选择那条路径呢？

对于敌方来说，直升飞机暴露的时间长度是非常关键的。对于单独的一段 20s 的暴露时间来说，可能已经足够导弹发射器侦察并锁定此直升飞机了，这样它就将发射一枚很准的导弹。然而，如果直升飞机走另一条暴露 4 次的路径，这种情况就不那么容易发生了。不管是导弹发射器需要时间进行锁定，还是狙击手需要时间来瞄准，或是卫兵会由于某些噪音而分神——他们都会希望其目标能有更长的暴露时间，而不是有更多的暴露次数。

因此，为了将敌方瞄准的因素考虑进来，我们需要对自己的结点和代价函数进行扩展。

LISTING 3.6.3 考虑了敌人瞄准质量的 A*节点

```
struct node {
    node(): aiming(0) { }; // default ctor: start
                           // with zero aim
    node(float anInitialAim) : aiming(anInitialAim) { };

    float  location[3];
    float  aiming;
};
```

在此代价函数中，我们使用并更新了瞄准信息，这是在代码列表 3.6.4 里面详细说明的。

Download at www.Pin5i.Com

LISTING 3.6.4 代价函数与战术式路径

```
float TacticalCostNodeToNode2(node* aFromNode,
        node* aToNode) {
    float travelTime, riskFrom, riskTo,
       riskTotal, aiming;

    // compute travelTime and riskTotal
    ...

    // update aiming quality based on risk,
    // and add it to risk
    aiming  = aFromNode->aiming;
    if ( riskTotal > 0 ) { // spotted,
                           // so increase aiming
       aiming = min(kMaxAiming, aiming + travelTime);
    } else { // not spotted, so decrease aiming
       aiming *= power(kAimingDamping, travelTime);
    }
    riskTotal += kAimingFactor * aiming;

    // store aiming quality at destination
    aToNode->aiming = aiming;

    // return the weighted combination of
    // travel and risk costs
    return kTravelTimeFactor * travelTime +
       kRiskFactor * riskTotal;
}
```

图 3.6.3 展示了一个简单的测试例子，它以直升飞机寻径问题为蓝本，图下面的图表展示了每条路径的暴露时间和瞄准的质量。

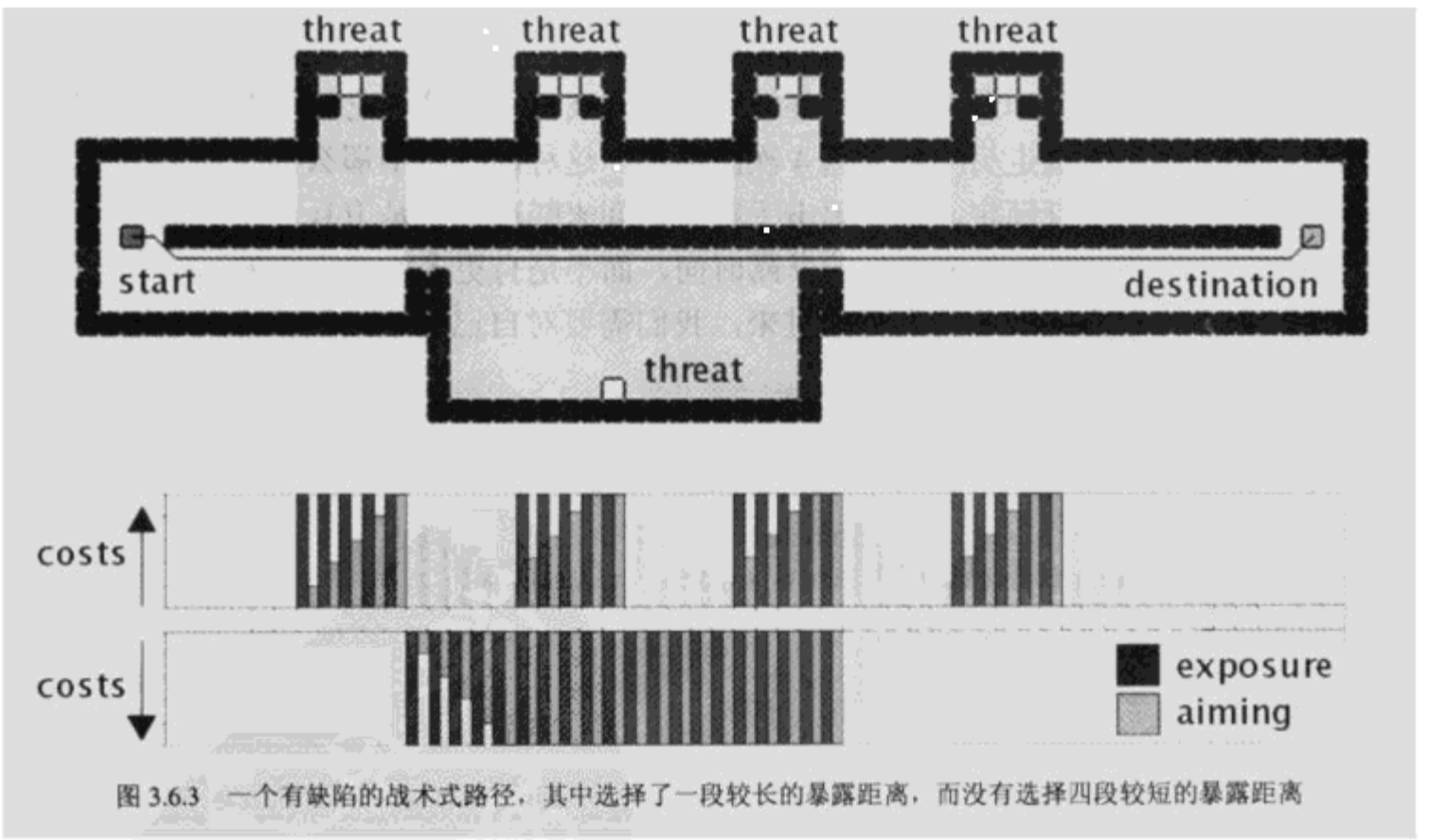


图 3.6.3 一个有缺陷的战术式路径，其中选择了一段较长的暴露距离，而没有选择四段较短的暴露距离

Download at www.Pin5i.Com

两条路径在敌方火力下暴露的时间都一样长——20s。在上面的一条路径中，暴露的部分（暗色的部分）由四个均匀的分段组成，每个分段之间的间隔与分段长度一致。在下面的路径中，暴露部分由一段 20s 的时间间隔组成。对于两条路径来说，有威胁的瞄准（灰色的条状物）是在进入暴露路径的时候就立即开始的。其瞄准质量是随着暴露时间的增长而增长的，直到达到了一个最大值为止。当一条路径不再暴露在火力下之时，瞄准（灰色的条状物）质量就立刻下降了。在一段时间里，敌方可能仍然可以预测出飞机的路线，但是其威胁是随着时间递减的。对于上面的路径来说，相对于下面的路径对于暴露而产生的“瞄准”的整体时间会相对较少。在每段暴露路径之间的间隔可以降低敌人的瞄准度，以防止此值保持在最大值上。

类似的，当 AI 开始搜索一条战术式路径，而其起点已经暴露在了敌方火力之下的时候，它应该假设敌人现在拥有最佳的瞄准质量。这就会促使算法尽快得到一条脱离暴露区的路径。

因此，在对敌人建模的时候，如果考虑到了它侦察和瞄准的能力，则在战术上我们就可以得到一条更好的路径了。我们的 A*算法现在更偏爱由长间隔时间打断的暴露路径，显然现在它表现得更加智能了。

3.6.4 威胁并不仅仅是静态的

不幸的是，在大多数游戏中，敌人的威胁并不是静态的。他们会在一个区域内巡逻或是在一条路径上每次移动一个单位。然而，我们当前的战术式 A*算法会傻乎乎地采用一条长达 10s 时间的路径，而使用的威胁位置则仅是在单独的某个时刻产生的。如果忽略了敌方的移动，那么这条路径的“战术质量”就只能听天由命了，而玩家对 AI 的能力大概也不过如此了。

为了能得到更好的战术式路径，我们必须对敌人的移动进行预测。这听起来挺复杂，但是实际上并非如此，即使不使用博弈中 AI 的最大最小算法，我们也能估测出敌人最有可能移动的地方。我们可以使用一个取代方法，假设敌人也占据了他们当前所在位置附近的一片区域。参看图 3.6.4：

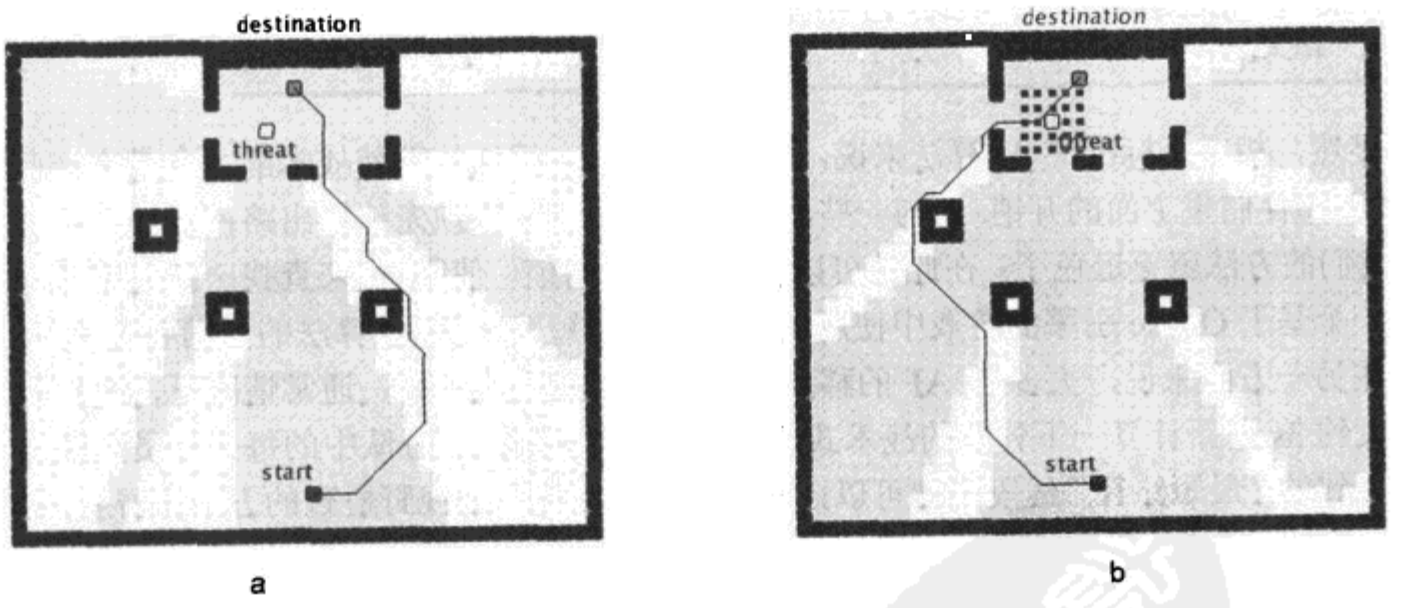


图 3.6.4 （a）假设威胁是静态的时候的战术式路径（b）假设敌人可以移动 2 步时的路径

Download at www.Pin5i.com

请注意在图 3.6.4a 中，其路径在多大程度上是取决于敌人起始时的位置的。其选择的路径对于敌人的移动来说并不健壮，而且它忽略了那些较大的障碍物和地图左边的掩护物。

当我们假设敌人覆盖了其开始位置附近两步区域的时候，实际上我们采取了一个更为悲观的假设。其结果，如图 3.6.4b 所示，对于敌人的移动来说更健壮，而且考虑到了左边障碍物可以利用的地方。对于 AI 来说选择这条路径显然更保险。

大多数游戏中 AI 的实现都提供了一个快速的方法，以找出敌方覆盖点附近几步之内的区域。这些区域一般都对应着出口点或是其相邻的节点。

3.6.5 更战术化的改进

在从点 A 到达点 B 的路径上，我们的 AI 现在已经能合理而有效地运用掩护物和隐身之处了。不过在战术上来说，除了掩护物和隐身之处以外，还有其他需要考虑的因素。下面，我们就将列出可以很容易地加入 A*的代价函数中的一些考虑因素。

并非所有的火力线都是一样的。如果 AI 算法只需要应付几个敌人，那么最好考虑一下在某一点上覆盖的火力线的数目，而不仅仅是对掩护物与危险地带进行区分。与敌人和敌方武器之间的距离在火力线问题中也可能是一个需要考虑的重要因素。

游戏中我们会对那些有阴影、植被，或是小物体的位置给予较高的评价，因为对于敌人的侦察来说它们能提供防护功能。如果代价函数对其他的区域增加了一个小小的代价值，寻径算法就会更倾向于选择这些地点了。

即使在没有已知敌人的情况下，也还是需要战术式寻径的。为了避免暴露在已知的和可能的火力线之下，现在重要的是我们要尽量避免处在战术上较为不利的位置上。例如，在没有必要的时候，坦克应该避免穿越山脊，因为其较为脆弱的底部可能会暴露出来，或是会在天空背景下给暴露出轮廓来。类似地，一个海军陆战队员应该避免爬梯，因为在梯子上此人的动作是可以被预测出来的，而且在爬梯的时候他还不能还手。我们可以分析和预先计算出一个地点上的很多战术属性，正如[Sterren01]里面所介绍的一样。给这些地点加上了额外的代价值以后，我们的战术式 A*寻径算法生成的路径就可以避免通过这些地点了。

3.6.6 性能

显然，相对于标准的 A*算法来说，此处描述的 A*算法的额外战术能力会带来一些额外的 CPU 和存储量上面的开销。而与一些游戏中使用的快速的预先计算出路径查找表的方法来说，我们的方法就更逊色了。在此处可以提供一个参考值，使用 A*来查找一条战术式路径要比在一个基于 Quake 引擎的游戏中使用的查找分队最短路径的 AI 算法的开销要大 10 倍。

在另一方面来说，大多数 AI 的移动不需要使用战术式路径。通常情况下，我们只需要每隔大约 3s 重新计算一下分队的战术式路径就可以了。然后，分队中的每个成员只要沿着路径上大致的“最短路径”线段走就可以，另外还可以使用一些视野检查的方法来寻找掩护物。

下面，让我们来看看这些额外代价值中某些值的起源，看看它们添加了一些什么东西，然后看看我们能做些什么工作来限制它们。首先，我们要考察一下火力线探测的方法，这一般是了最大开销的来源。其次，我们来审查一下 A*算法和它的战术式代价函数，还有其涉及

Download at www.Pin5i.com

到的更大的搜索空间。

3.6.7　有效的火力线以及视野的探测

在很多游戏中，视野和火力线（LOF）的探测都会消耗相当多的 CPU 时间。然而，对于战术式寻径来说，我们只需要使用对真实火力线的近似就可以了（AI 不需要找出所有的敌人位置，而且敌人也可能会移动）。在这种情况下，可能我们可以使用一个采样的火力线查找表来交换 CPU 时间。

如果地形不是很大的话，那么你可能会使用查找表，里面包含的是 1 位或者更多位表示的视野或是火力线的信息，正如[Lidén02]里面描述的一样。这么一个查找表会占用 $O(N^2)$级数的存储量。还有一个方法，你可以记录下在每个位置上一个分块中进入的火力线或是视线的数目，这只需要 O(*N*)级数的存储量。这种方法以及其对于我们 2D 情况下的例子产生的结果都在图 3.6.5 中展示了出来。

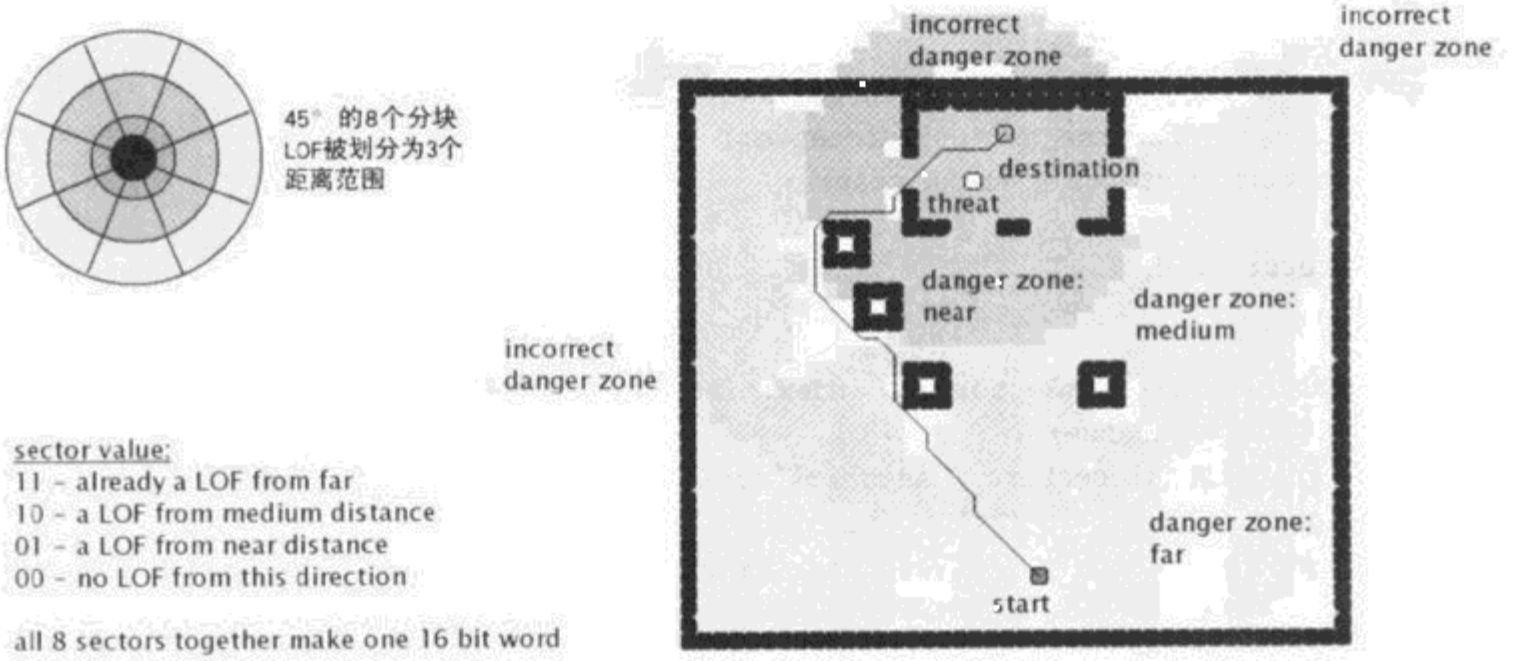


图 3.6.5　基于分块的 LOF 查找与得到的路径

对于每个位置可以使用 16 位的一个字来保存，这样信息就可以被预先计算出来，而且被存储下来了。对于每个位置来说，每个分块都给出了一个最坏的距离，在此距离上，一个处于相应分块的敌人可能对此位置会产生一条火力线。这个最坏距离是使用四个值来表示的，从“在此分块内根本没有火力线”一直到“在最大距离处产生的火力线”。值得注意的是此处保存的值是一个较为悲观的值：只需要有一条火力线经过了分块中的一个位置，那就足以把此分块标志为从此方向有火力线威胁了。

在一个 A*搜索的过程中，为了得到从某个地点发出的火力线的近似值，我们可以有效地使用这个基于分块的查找表（参见代码列表 3.6.5）。对于一个敌人和其位置来说，相应的 45° 角的分块以及距离都会被计算出来。然后查找表就可以告诉我们，是否在某点上可能会有火力线从敌人的分块中发射出来以及是否可能一条火力线会从此分块中进入。如果两者都有可能的话，则敌人就可能会产生一条火力线了。

显然，这么一个小小的基于分块的查找表是不能正确地找出所有火力线的。通过对图

Download at www.Pin5i.com

3.6.4 中的火力线和图 3.6.5 中的危险地带进行比较以后，你就可以轻易地看出这一点了。在很多情况下，此（悲观的）错误对应的是当敌人移动的时候，其可以轻易地得到的视线。在其他情况下，例如当危险地区处在考察范围之外的时候，这种错误就会更加严重了。不过，只需要再添加几个位和使用更小的分块，你就可以很容易地把这种错误降低。

对于 3D 环境和对于提供了不对称掩护物（这样在火力线中就失去了对称性）的游戏来说，你可能还需要存储更多的位数。尽管如此，这个基于分块的查找表的方法通常说来仍是很高效的，因为它只需要几万字节就可以保存 1000 个地点的视野信息了。

LISTING 3.6.5 使用基于分块的、预先计算好的 LOF 信息算出大概的风险

```
float GetApproximateRiskFromThreat(node* threat,
       node* location) {
   int   sector = GetSectorForLine(threat, location);
   int   reverse_sector = (sector + 4) % 8;
   float distance =
      GetDistanceForLine(threat, location);

   if (HasLineOfFireFromSector(location->id,
      sector, distance)
         && CanFireIntoSector(threat->id,
               reverse_sector, distance))
      return distance / kMaxDistance;
   else
      return 0.0;
};

bool HasLineOfFireFromSector(int index, int sector,
       float distance) {
   unsigned int allsectors = sectors[index];
   unsigned int mask  = (0x3 << sector);
   unsigned int value = ((allsectors & mask) >>
      sector);

   return (   (value == 3 && distance < kMaxReach   )
          || (value == 2 && distance < kFarReach   )
          || (value == 1 && distance < kMediumReach));
};

bool CanFireIntoSector(int index, int sector,
       float distance) {
   // assume firing is symmetric, otherwise
   // an extra table is needed
   return HasLineOfFireFromSector(index,
      sector, distance);
}
```

3.6.8 扩展的 A*算法的代价

我们在前面已经提过，在考虑了战术式因素之后，A*算法将会产生更大的 CPU 以及存

Download at www.Pin5i.com

储上的开销，主要表现在其代价函数以及 A*算法更大的搜索空间上。一个额外的开销来源是在战术式代价函数中使用的浮点运算导致的。如果使用整数运算来对浮点运算进行近似，那么你就可以提高性能了。但是，只有在你对之进行了验证，而且使用了最精确的浮点代价函数调整寻径算法以后，才能采用这种方法，不然可能会带来其他的问题。

一个战术式 A*算法搜索的空间比一个标准 A*算法搜索的空间要大得多，正如图 3.6.6 所示。由于在起点和终点之间的路径上一般都放置了一些火力线的掩护物，在对之进行考虑的时候就会导致搜索空间的扩大。A*算法会花费很长的时间来对尾部与边线进行计算，因为这些地点看起来更安全。而由于启发式函数无法考虑到这些新增的火力线掩护物，因此它也帮不上什么忙。

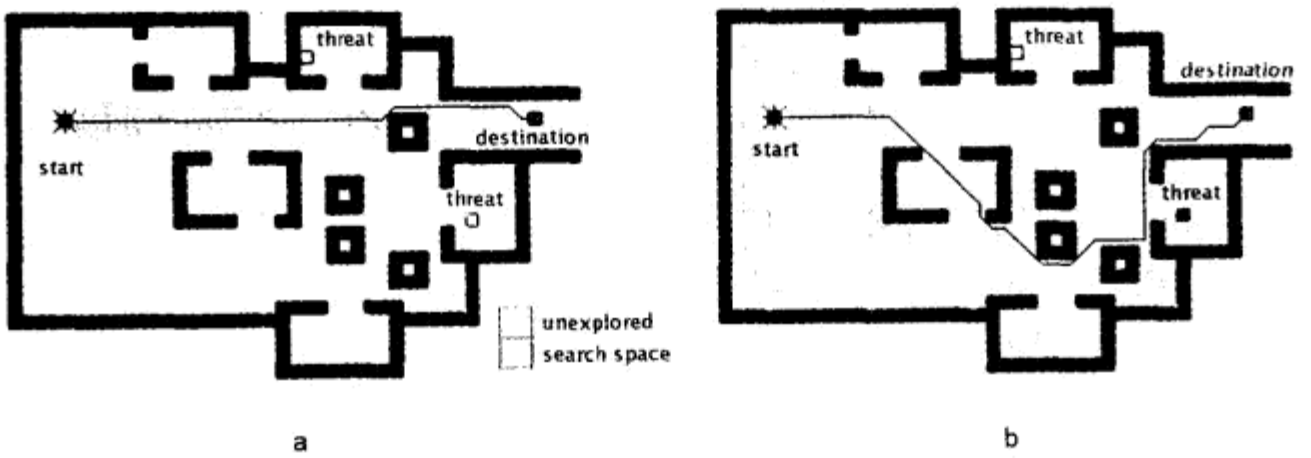


图 3.6.6 （a）最短路径的搜索空间（b）战术式路径的搜索空间

要想减小搜索空间，一个方法就是可以引进带来更大开销的虚拟障碍物。如果你希望自己的 AI 算法能找出一条通往目的地的战术式路径，为何不将其限制于尾部和边路之外的路径上呢？这只需要临时将这些点标志为“禁止进入”就可以了。也没有什么天条禁止你在算法中忽略这些区域。

另外一个减小搜索空间的方法就是使用启发式寻径方法，利用层来进行寻径。首先，在较高层生成一条地图上区域间的最短路径。接下来，选出此最短路径经过的区域和相邻的区域，然后在此区域中进行战术式寻径。Board/Ducker[BoardDucker02]与 White[White02]都在他们的文章里提到了区域游览的问题。

3.6.9 ASE 程序

如果想要做一下关于战术式寻径方面的试验，我们在 CD-ROM 上提供了一个 A*探索器（ASE）的应用程序。James Matthews[Matthews01]为这个 A*工具提供了一个基础。在本节中提及的战术方面的增强特性和对火力线的近似都加上了。请注意在此 ASE 中 A*算法的实现并不是一个效率很高的战术式 A*算法。不过，它在设计的时候考虑到了可以让用户自己提供不同的代价函数和相应的视觉效果。

3.6.10 结论

相对于把那些暴露在敌人火力之下的位置赋予一个较高代价值的方法来说，使用 A*来查

Download at www.Pin5i.Com

找战术式路径是比较复杂的。你应该从一个敌人的角度来考察这条路径，而且应该在代价函数中对敌人的瞄准和移动进行处理。通过这些特性，我们就可以确保路径能精明地对掩护物和隐身之处加以利用以预防受到敌人火力的攻击，即使敌人可能会移动。通过类似的方法，你也可以对 A*进行扩展，以使之在选择路径的时候，如果必须的话，与友好的区域或是敌对的区域保持接触。我希望，通过本节，你将能得到足够的帮助以将这些特性高效地应用到自己的游戏中去。

3.6.11 参考文献

[BoardDucker02] Board, Ben and Mike Ducker, "Area Navigation: Expanding the Path-Finding Paradigm," *Game Programming Gems 3*, Charles River Media, Inc., 2002.

[Lidén02], "Strategic and Tactical Reasoning with Waypoints," *AI Programming Wisdom*, Charles River Media, Inc., 2002.

[Matthews01], A* Explorer, 相应的网址为 *http://www.generation5.org*, 2001.

[Patel99] Patel, Amit J., "Amit's Thoughts on Pathfinding", 相应的网址为 *http://www-cs-students.stanford.edu/~amitp/gameprog.html*, November 18, 2001.

[Rabin00] Rabin, Steve, "A* Speed Optimization," *Game Programming Gems*, Charles River Media, Inc., 2000.

[Reece00] Reece, Doug, et al., "Tactical Movement Planning for Individual Combatants," Proceedings of the 9th Conference on Computer Generated Forces and Behaviral Representation, 对应的网址为 *http://www.sisostds.org/cgf-br/9th/*, 2000.

[Sterren01] van der Sterren, William, "Terrain Analysis for 3D Action Games," Game Developers Conference 2001 Proceedings, 在网址 www.cgf-ai.com 上面有对应的文章和演示，2001.

[Stout00] Stout,Bryan, "The Basics of A* for Path Planning," *Game Programming Gems*, Charles River Media, Inc., 2000.

[White02] White, Stephen, "A Fast Approach to Navigation Meshes," *Game Programming Gems 3*, Charles River Media, Inc., 2002.

Download at www.Pin5i.Com

3.7 快速游览网格的方法

Stephen White，Christopher Christensen
Naughty Dog
swhite@naughtydog.com
cchristensen@naughtydog.com

在视频游戏中，一个常见的问题就是应该如何在一个复杂的环境中操纵对象进行游览。对于 *Jak and Daxter: The Precursor Legacy* 来说，我们希望游戏中的生物都能在一个拥有极丰富细节的 3D 环境中移动自如，而在此环境中每一层都由数百万个多边形组成，且填满了生物和障碍物。我们希望生物能进行智能化的移动，而且能表现出几种不同的运动方式。由于此世界中的生物密度太大了，因此我们需要一个比通常的游览技术，例如说 A*，更快的系统。但是此系统仍然要保证灵活性和精确度，而且在运动中要表现得很聪明。为了解决这个问题，我们开发了一个自己的游览网格（navigation mesh）技术，此技术能很好地解决我们的问题，不过它也带来了一些有趣的限制。

3.7.1 静态障碍与动态障碍

在游览中最基本的问题就是要如何从点 A 到达点 B。在这两点之间最短的路线当然是一条线段。然而，如果在两者的直线路径上有障碍物的话，就不适合使用直线了。障碍物可以分为两类：静态障碍与动态障碍。静态障碍是指那些永不移动的障碍物。这是我们最喜欢的一种障碍物，因为已经有很多优化方法可以用来在此类障碍物中游览了。静态障碍物的例子包括悬崖、墙壁、树木，还有柱子。动态障碍指的是可以移动或是被移开的障碍物，例如其他的生物、玩家、移动的平台，还有箱子。动态障碍处理更为困难，因为它们可以移动，这就难以对之使用预先计算的方法了。更糟糕的是，一个动态障碍的移动可能会使得以前计算出来的路径变得没用了。由于静态障碍与动态障碍有如此多不同的优缺点，因此我们将分别对之进行讨论。

3.7.2 游览网格

在 *Jak and Daxter* 中最常见的一类静态障碍就是地形了。自然的边界，例如墙壁和悬崖，给出了一个生物可以在其内运动的区域。在这些边界内

Download at www.Pin5i.Com

部，有可能还有其他的静态障碍，例如树木，大块的岩石、柱子、长得无法跃过的沟壑，还有一些生物不能穿过的其他的东西。

为了描述出在一个区域内生物能够运动的地方，艺术人员建立了“游览网格”。这个网格是一个由三角形组成的集合，其中每一个三角形都表示了生物可以运动的一个有效区域。建模得出的网格将扩展到对象区域的边界，而网格中的空洞表示的是此生物在边界内部不能访问的区域。需要注意的是，我们的游览网格在本质上基本是二维的，而且它定义了一个生物可以运动的二维区域。然而，在我们的网格中也有三维的因素，这一点将在以后进行阐述。

作为一个例子，假设有这么一块空地，我们希望生物能在这片空地上进行游览（参见图 3.7.1）。在此空地中，有一块生物无法穿过的大岩石。艺术人员将建模出一个网格来，通过它既可以表现出生物游览的边界，也能在岩石旁边裁减出一个空洞（参见图 3.7.2）。在网格中的每个三角形都描述了一个此生物可以通过的区域。将此生物的运动范围限制在了网格三角形描述的二维空间之后，此生物就再也不能逃离此空地，或是穿过此岩石了。

此系统的优点表现在其简洁性上。通过一个简单的方法，我们就同时解决了区域边界和内部障碍的问题。即使是相当简单的游览网格也可以表现高度细节化和复杂化的地形。

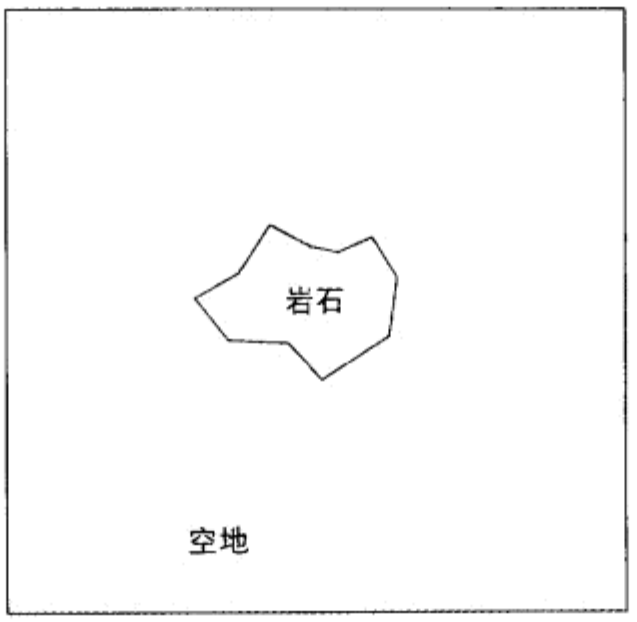


图 3.7.1 有一个静态障碍物的空地

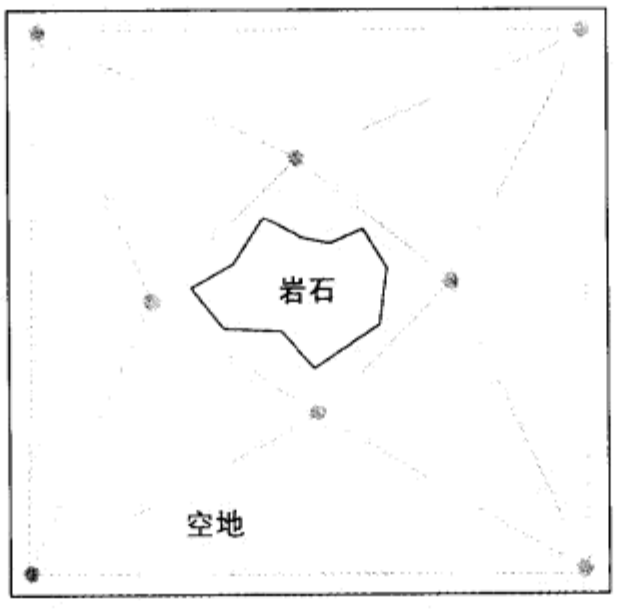


图 3.7.2 添加了游览网格的空地

Download at www.Pin5i.Com

3.7.3　门户

这些三角形的作用并不仅仅限于设定一个生物可以行走的区域——它们还定义了我们称之为“门户”（portal）的东西。门户可以看成是通往其他三角形的通道。在我们的游览网格中，门户是通过三角形的边来表示的。作为一个简化，我们规定这些三角形将不包含 T 形连接，这就意味着我们只允许一条边为最多两个三角形所分享。由于一个三角形有三条边，这就意味着一个三角形最多可以和其他的三个三角形连通。因此一个三角形最多只有三个门户通往连通的三角形。这个限制将会带来一些三维上的问题，但是它也将给我们带来一些优良的数据特性。举一个例子来说，我们可以只使用两个位来标明一个三角形的一个门户（第一个门户、第二个门户，或是第三个门户），而且还可以多一个值以做他用。

假设现在我们的问题是要如何从点 A 到点 B 去。如果点 A 表示的是生物，点 B 表示的是生物想要达到的目的地，而且两个点都在一个三角形以内，那么我们就知道生物可以径直运动到点 B 了，这是因为我们已知生物可以在一个三角形以内任意移动了。如果目的地和生物不在一个三角形以内的话，这就含有问题。在这种情况下，生物可能需要选择一个方向，在此方向上既能保证生物保持在游览网格之内，也能智能地使生物沿路径最终达到目的地，即使中间有可能暂时偏离终点。

因此，如果我们在一个三角形内，而目的地在另一个三角形内，那究竟该如何选择一条合适的路线呢？我们的方法是预先计算出一个二维数组出来，此数组标明了在网格中从任意一个三角形到另一个三角形需要通过的门户。请记住我们只需要两位来标识出一个门户，因此此表占用的存储量大致等于网格中三角形数目的平方再除以 4。这就意味着一个有 256 个三角形（在我们的实现中所允许的最大值）的网格只需要 16KB 的存储量。在实际情况中，我们的游览网格极少会超过 64 个三角形（1KB 的存储量），甚至通常会更少。另外，由于多个生物一般会共享一个游览网格，这样在总数上会有相对更少的游览网格，因此存储量方面的开销是很少的。现在，让我们考察一下使用这么一个二维表的好处。有了它之后，寻找应该使用哪个门户的过程就非常快了，因为这只是一次简单的表查找而已。只要使用源三角形的索引和目标三角形的索引就可以找到表中相应的项了。如果表是使用字节而不是两个位组成的话，那么 C 代码大致如下所示：

```
portalIndex = portalTable[destTriIndex][srcTriIndex];
```

当然了，使用了字节表以后存储量会翻两番，因此我们还有一个位表的代码，如下所示：

```
bitIndex = 2*(destTriIndex*triCount + srcTriIndex);
byteIndex = bitIndex / 8;
byteShift = bitIndex & 7;
portalIndex = (portalTable[byteIndex] >> byteShift)&3;
```

一旦我们知道了对给定三角形需要使用的门户，那就可以根据从点 A 到点 B 的矢量得到在门户上最接近我们所要求方向的点了。这样生物就可以向着这个门户上的点运动，直到到达这个点为止，在此处它也转移到了门户另一边所在的三角形内。如果点 B 已在生物最新所在的三角形内，则生物就可以直接达到点 B 了。如果点 B 仍然不在生物最新所在的三角形内，

Download at www.Pin5i.Com

那么我们将对表进行新的一次查找以决定下一个可以使用的门户。此过程将不断重复直到生物最终到达点 B 所在的三角形内为止。

图 3.7.3 是一个例子。点 A 所表示的生物所在位置位于三角形 0 以内，而点 B（其目的地）则在三角形 3 以内。由于点 A 和点 B 同不在一个三角形以内，因此此时对表进行了一次查找以决定将要使用哪一个门户，其返回值告诉我们要使用连接三角形 0 与三角形 1 的门户。在此门户上找到了一个最匹配指向点 B 方向的一个点，然后生物就将移向那个点。接下来生物就处在三角形 1 里面了。此种过程不断重复，直到生物进入了三角形 3，那个包含了点 B 的三角形。此时生物就可以直接运动到点 B 了。

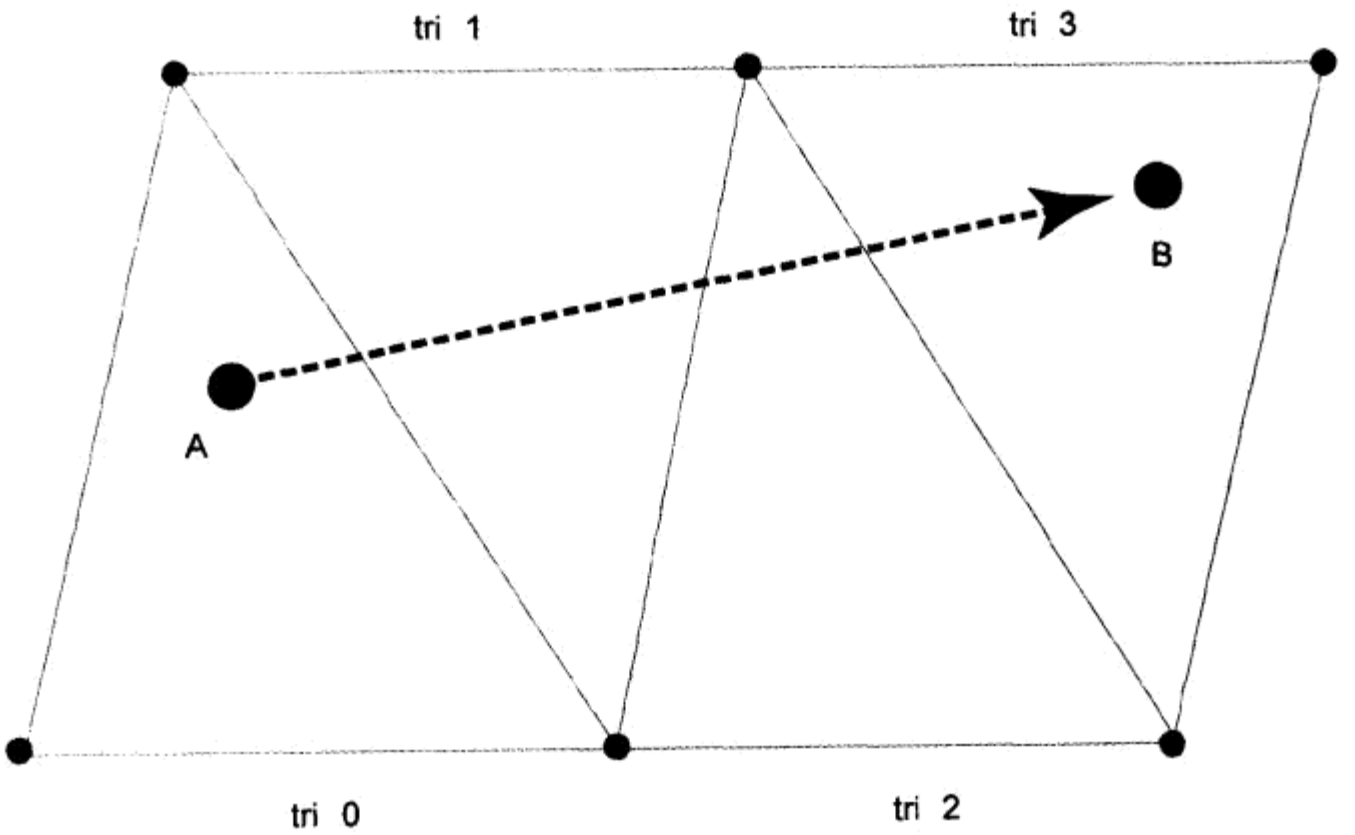


图 3.7.3 在一个简单的网格里从 A 浏览到 B

虽然上面的例子相当简单，但是对于更加复杂的情况我们也可同理分析。考察一下图 3.7.4，在其中显示了如何使用同样简单的逻辑对更为复杂的网格进行游览。

这个原理对于分叉和循环的路径都能使用。当有多于一个的门户可以通向目的地的时候，表中的项将给出哪一个门户代表了离目的地最近的路径。不过，这将是一个比较复杂的过程。一个三角形代表了一个区域，而不止是一个点。这就意味着从一个处在某个三角形之内的点到一个处在另一个三角形之内点的距离可能相差极大，这要视这些点处在其三角形内的位置而定。由于预先计算出来的表里面只能告诉我们如何从一个三角形到达另一个三角形，而没有告诉我们如何从一个三角形内的某点到另一个三角形内的某点，因此此预先计算出来的门户可能并不在通向目的地的最短路径上。然而，在实际情况中，这基本上不成问题，因为如果这些多于一条的路径距离相差不大的话，那么即使选择的路径不是最短，也没什么关系。

Download at www.Pin5i.Com

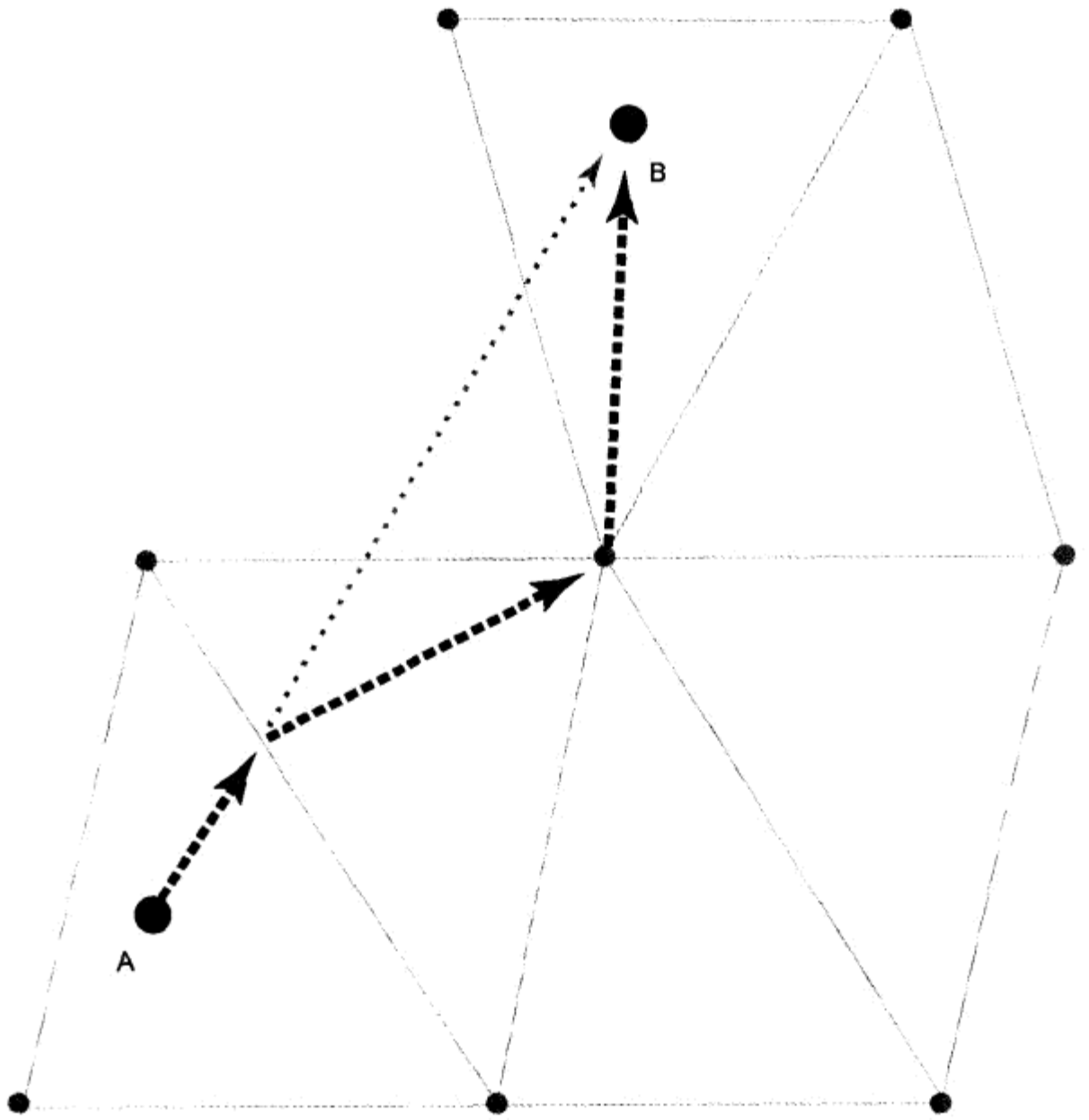


图 3.7.4　通过一个拐角处

3.7.4　建表

那么，应该如何计算出这个表呢？这里我们可以使用好些方法，但是最终我们采用的方法是要计算出一个点来代表每个三角形的中心。我们倾向于使用离三个角距离相等的那个点。接下来我们将使用一个简单的填充算法找出连通两个三角形的路径。如果在两个三角形中找到了不止一条路径，那么此算法将会丢弃那条较长的路径。

在创建游览网格的时候有很重要的一点需要考虑，那就是它代表了通往每个目的地的一条路径。对于任两个区域来说，每个门户都应该提供一条单独的、清楚的路径以产生一个可以接受的结果。如果提供的门户不止一个的话，那么其结果将会无谓地曲折。图 3.7.5 展示了这么一个例子，在此中，预先计算的时候没有搞清楚从点 A 所在的三角形到点 B 所在的三角形的最佳路径。在此例中，预先计算的时候只选择了一个门户，而此门户在运行时是不合适的，因为它没有生成一条从点 A 到点 B 的直线距离。图 3.7.6 展示了创建游览网格的正确方法，在其中总是会走直线距离的。

Download at www.Pin5i.Com

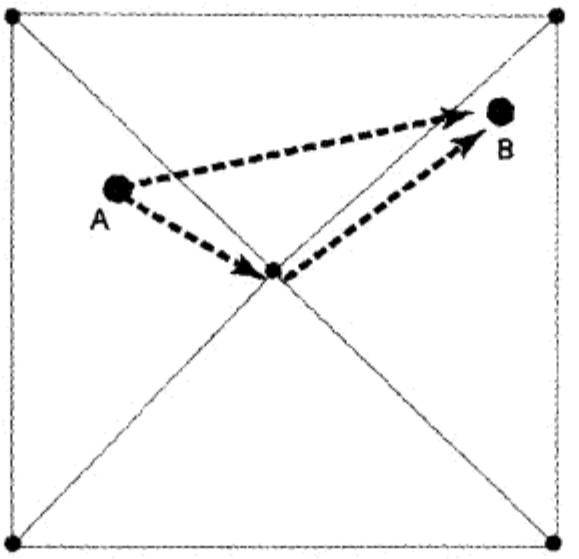


图 3.7.5 创建网格的错误方法

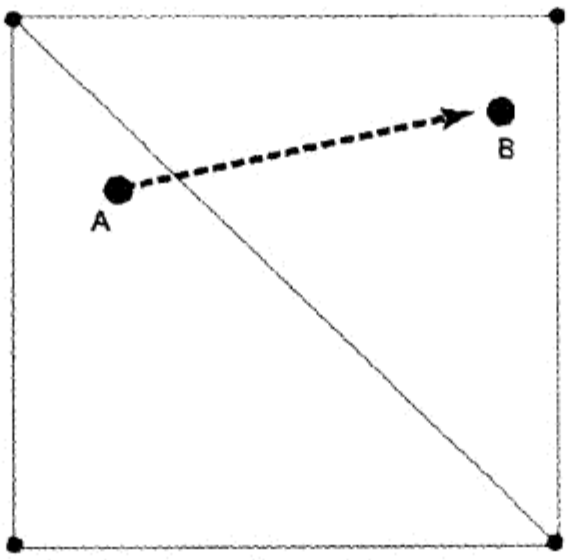


图 3.7.6 创建网格的正确方法

3.7.5 其他的门户相关问题

虽然使用门户的概念相当简单，但是在实现中仍然有一些复杂之处。一个复杂的地方就是当点 A 接近一个门户边缘的时候，如果要检测从点 A 到点 B 的矢量是否与门户相交则可能会不可靠。例如，浮点运算的舍入误差可能会认为点 A 在当前的三角形以外，导致此检测结果错误。另外，当从起点到终点的矢量和门户边缘几乎平行的时候，此相交检测的结果也可能会不可靠。为了避免这些问题，我们将要检查一下是否起点与门户边缘的距离小于一个选出的 delta 值。当此种情况发生的时候，我们就接着前进，继续寻找下一个路径上的门户，此门户与起点不那么接近，而且可以看成是我们要使用的门户。

另一个要考虑的问题就是当期望的方向矢量与门户不相交的时候应该选择的替代方向。可能究竟应该选择门户的哪一个端点作为最佳选择不太明显。如果只考虑选择的端点最接近期望的方向，这可能是不够的。举个例子来说，如果期望方向的矢量与门户垂直，而且方向刚好相反的话，那么门户的两个端点就一样地不适合了。浮点运算的精确度和其他的小误差可能会造成不希望出现的振荡现象。解决这种两难问题的方法就是继续向前进行门户的查找，

Download at www.Pin5i.Com

一直到找到了一个门户可以明显地给出究竟哪一个端点和未来的路径更为接近为止。这种“向前看”的技术还有另一个好处，那就是可以创建出一条更为直接的路径，避免生物在路上绕弯。由于这种技术的使用需求很少，而且通常也只需要向前看少数几个三角形，因此额外的 CPU 开销很小。

3.7.6　表示生物

通过使用游览网格，我们就能从网格中的一个点聪明地抵达网格中的另一个点了。不过，大多数生物可不止就占据一个点的空间，这就引发了一个问题：如何考虑生物的厚度呢？由于对于我们来说，效率比精确度更加重要，因此我们仅是简单地使用一个圆圈来表示生物在游览网格中的二维区域，其中圆圈半径表示的就是生物的厚度。使用了圆圈以后，对于那些不那么圆的生物来说可能会有问题。例如，当从上面俯瞰的时候，一匹马就不是那么圆。如果我们是用一个足以包含整匹马的圆圈来代表，则其他的生物就会避免接近这匹马的边缘，而且这匹马本身也将无法钻进一个视觉上看本是可以钻进的区域。如果你将这个圆圈变小一点以便其他的生物能接近这匹马的话，那么马的尾巴就会处于圆圈以外了，偶尔可能产生它与其他物体重叠的现象。

丢开这些问题不谈，我们觉得使用圆圈带来的好处还是比坏处要多。对于 *Jak and Daxter* 来说，我们总是可以找到一个合适的半径在两方之间取得平衡。另一个使用圆圈的好处在于在旋转的时候，我们不需要考虑与游览网格或是其他动态障碍的碰撞。

为了避免改造我们原先优良、简洁、点对点游览网格的处理以变成需要更大计算量的圆圈对圆圈的处理，我们更倾向于把游览网格进行预先收缩以适应可能会使用它的生物半径。这个方法比较简单，因为它没有对游览网格进行建模以精确地得到相邻的障碍物，取而代之的是，网格模型将对所有的障碍物保持一定的距离。图 3.7.7 展示了如何对游览网格进行建模以适应一个小生物，而图 3.7.8 展示了同样的场景，不过这回是对一个大生物的游览网格建模。如果我们能保证游览网格的边缘与所有静态障碍的距离都大于或等于使用此游览网格的生物，那么生物就可以使用一个点，而不是一个圆圈来表示了。

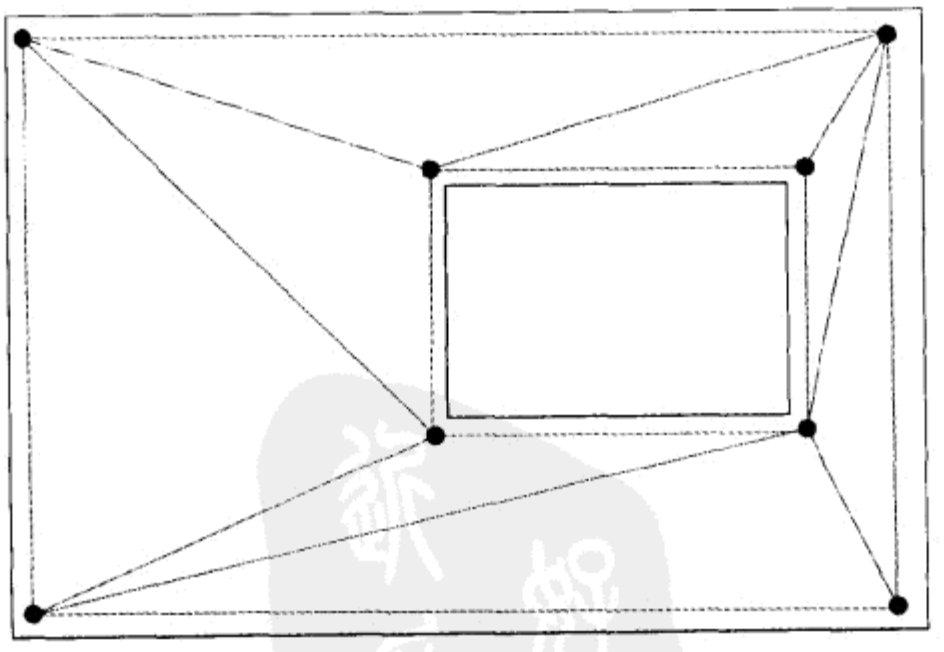


图 3.7.7　针对小生物的网格

Download at www.Pin5i.Com

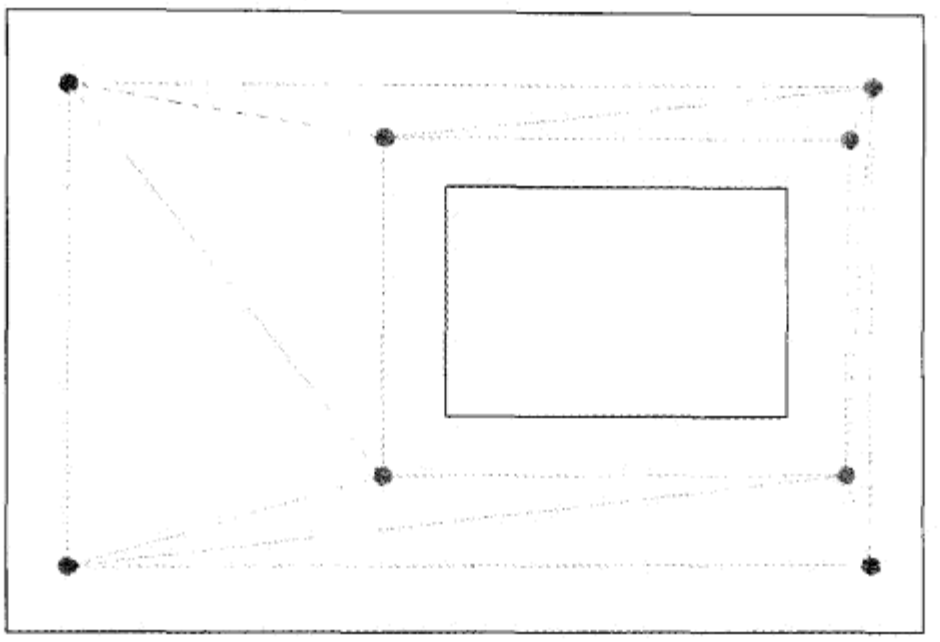


图 3.7.8 针对大型生物的网格

这是一个重大的优化，这是因为虽然它也会带来某些问题，但是在游览网格中移动一个点要比移动一个圆圈容易得多。最大的问题就是不同类型的生物可能会希望使用同一个游览网格，而其中的某些生物可能比另一些生物有大得多的圆圈。如果在创建游览网格的时候适应最大的生物，那小的生物就可能无法运动到静态障碍物的附近了。一般说来，对于我们来说这并不是问题，不过在有些时候，我们会遇到这种情况，那就是游览网格到静态障碍的距离是在太大了，以至于如果玩家站在墙边的时候，一个小一些的生物就会无法靠近墙壁对之进行攻击。为了修正这个预想之外的行为，我们不得不对游览网格和静态障碍之间的距离进行调整，直到我们找到一个折中的值。在此时较大的生物会和静态障碍有一点点的交叉，然而小生物就可以接触到玩家了。

我们第二大的问题就是，一般说来，在对一个游览网格进行建模的时候，很难判断应该使用什么距离。这个适当的距离应该基于网格中代表生物圆圈的半径，而生物的类型以及其圆圈的半径则会被偶然性的设计以及开发方面的变化所影响。

3.7.7 动态障碍

使用圆圈来代表生物之后，我们就能在游览网格中移动它们的时候把它们看作点来对待。当把这些生物在动态障碍，例如其他的生物，之间移动的时候这是很有用的。为了不使用计算量较大的处理来应付在圆圈之间移动圆圈的问题，我们可以使用一个简单的方法，那就是将周围的圆圈进行扩张，其增加的值等于当前正在移动的生物半径值。例如，图 3.7.9 中展示了一个半径为 r 的生物和两个拥有不同半径的动态障碍，而图 3.7.10 则展示了我们是如何将生物的半径加到障碍上面以将其看成一个点的。在移动一个生物之前扩展其周围的圆圈，通过此方法我们就仍然可以把生物表示为一个点，而将我们的游览问题降级为一个相对简单的问题了。后者只不过是要在网格里移动一个点，同时避免与周围的圆圈发生碰撞。

为了在这些圆圈里面移动这个点，我们要检查一下是否将要行走的方向会与周围的圆圈发生碰撞。如果不是的话，我们只要向期望的方向继续前进就行了。如果发现会有一个碰撞的话，那么我们将计算出两个新的矢量，它们将使我们避免发生碰撞（参看图 3.7.11）。

Download at www.Pin5i.Com

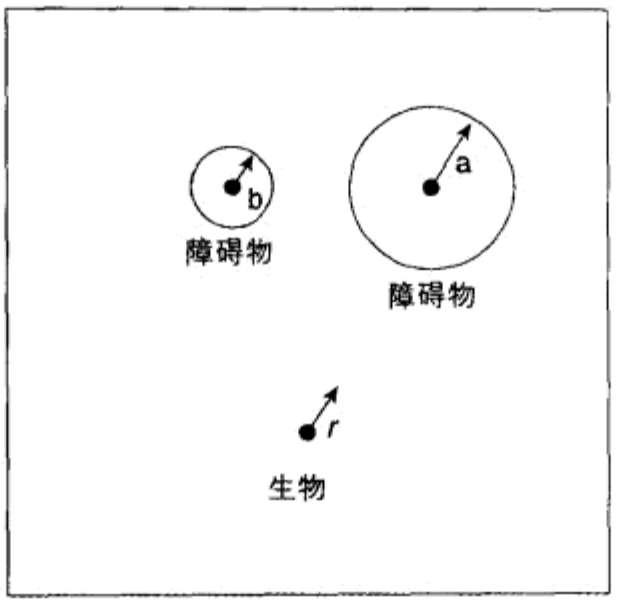


图 3.7.9　1 个简化为圆的生物与 2 个圆形的动态障碍物

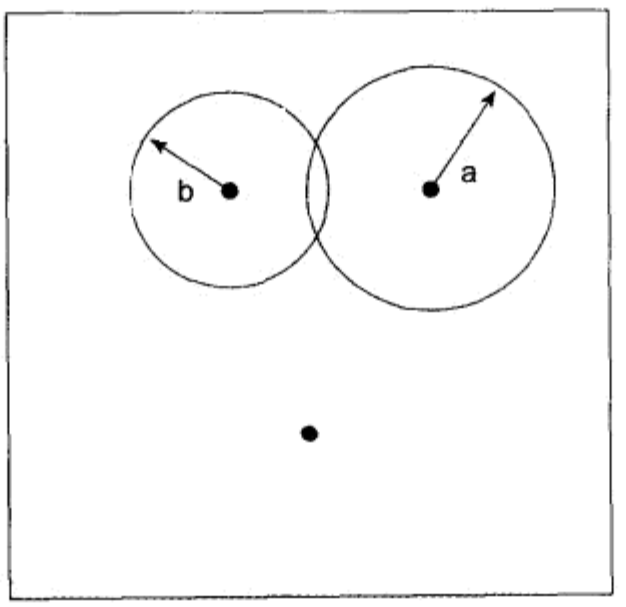


图 3.7.10　简化为点的生物与相应增大了半径的障碍物

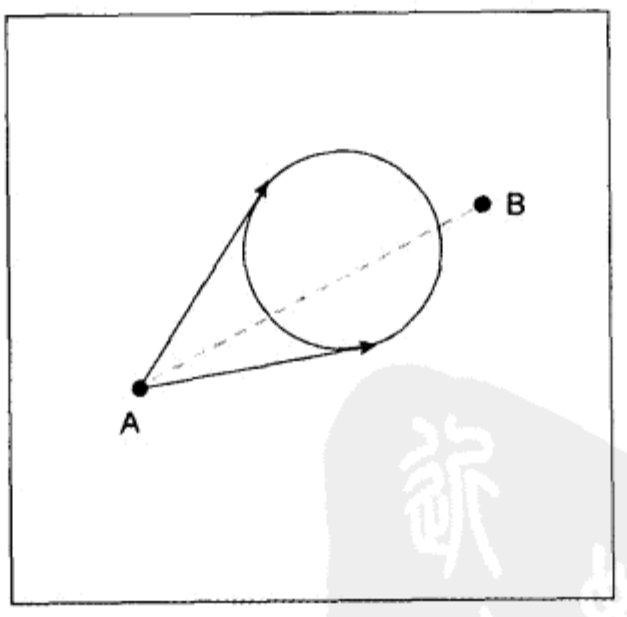


图 3.7.11　绕过一个与我们的路径相交的动态障碍物

Download at www.Pin5i.Com

这两个矢量可以看成是原矢量旋转得到的：一个旋转矢量指向障碍物的左边，一个旋转矢量指向障碍物的右边。接下来，我们将检查一下是否两个矢量会与任何其他的障碍物发生碰撞。如果是的话，我们将适当地把右边的矢量向右旋转或是把左边的矢量向左旋转，这样以避免任何其他的障碍物。一旦所有的障碍物都考虑过了以后，生成的两个结果矢量将显示出两条可以运动的方向，它们都可以避免动态障碍在原先期望运动方向上的格挡。为了避免发生不希望产生的圆周运动，而且也为了保持简洁性，我们一般会选择与生物上次运动方向最匹配的矢量。

使用此方法带来的一个问题就是生物偶尔会被困在其他的动态障碍之间。这通常是由于浮点运算的误差造成的，不过也有可能是其他的运动行为导致的。我们没有力图解决保证在任何情况下都不发生重叠的棘手问题，取而代之的是，我们将使生物有这么一种意识，那就是在发生了重叠以后，生物将力图远离与之发生重叠的障碍。基本来讲，我们就是先判断出生物和哪一个圆圈重叠得最厉害，计算出避免碰撞到其他圆圈的方向以后，我们将这些结果进行综合，得到一个远离重叠圆圈的方向，这是与重叠的大小成比例的。通过这种技巧，我们就可以以一种合理的方式解决此问题了。

另一个潜在的问题就是一个生物可能会完全被其他的动态障碍所包围，最后以至于左矢量和右矢量都和原先的方向完全相反。我们选择忽略对此情况的处理，这就意味着一个生物可能会运动到一个与周围动态障碍发生重叠的位置去。然而，在实际情况中，这对我们来说并不是问题，因为生物都是自然要远离重叠区域的。

3.7.8 在静态障碍与动态障碍之间进行游览

最终，我们处理生物运动的方法是相当简单的，它可以被分解为以下几个步骤：首先，我们将使用游览网格和它们的门户找到达到目的地的最佳方向；接下来，我们将检查一下是否产生的方向会和任何动态障碍（圆圈）发生碰撞，如果是的话，则我们将对方向进行修改以避免之；最后，我们将检查是否此方向将导致我们离开游览网格，若是的话，我们将进行运动的纠正以保持在游览网格以内。

最后进行运动的纠正以保持在游览网格以内的步骤是出于效率考虑的，而且需要一些小技巧。有个问题就是一个生物可能会被堵住而被迫停止。另一个问题是我们有时候允许一个生物的运动矢量沿着游览网格的边缘走而不是简单地被纠正。这个沿边矢量使得生物能继续运动，但是我们将不检查是否此矢量将导致生物与周围的圆圈发生重叠，因此一个生物可能会嵌到另一个生物里面去。由于我们避免圆圈重叠的处理逻辑将使得生物自然地想要远离重叠，因此一般说来，对于两个运动的生物这并不是问题。不过，我们确实遇到了一个运动生物与一个不运动的动态障碍，例如一个箱子，发生重叠的时候。为了消除这种现象，我们确保不运动的动态障碍，例如箱子，不会被摆放在游览网格的边缘处。

3.7.9 有关游览网格的其他想法

我们的游览网格使用了决定性的二维区域，在其中动态障碍可以进行运动。在大多数情况下这是可以满足要求的，因为大多数生物可以使用二维边界来描述。然而，偶尔我们会需

Download at www.Pin5i.Com

要使用三维的信息。例如，如果我们希望把一个生物放在一条螺旋上升的圆形阶梯上面的时候，仅仅使用二维游览网格来描述阶梯是不够的。对于此情况下的这种类型，我们还有一个从表面计起的最大高度值来表示此生物在对应游览网格三角形区域里面可以升到的高度。这就使得游览网格能够拥有一系列交叉的三维三角形，只要这些占据了一个二维空间的三角形在明显不同的高度上，就可以得知生物现在处于哪一个三角形中了。

另一个对我们的游览网格做的增强特性就是可以让生物跳过某些障碍以达到最终期望的目的地。例如，一个生物有可能会跳过一个小坑以追逐玩家。对这个坑进行三角形建模，然后把这些三角形标志为特殊的“沟壑三角形”之后，我们就得到此特性了。当一个生物穿过一个门户到达一个沟壑三角形的时候，它将搜索出在沟壑另一边的三角形，然后此生物将被告知可以跳过这条沟壑到达那个三角形去。为了防止生物降落到沟壑另一边的动态障碍上面，此生物将使用一个临时的圆圈来预留一个安全的着陆地点。

3.7.10 结论

我们的游览网格系统能极好地满足在 *Jak and Daxter* 开发中的需求（参见彩图 3）。我们可以创建各种表现各异的生物，同时能让它们在自己复杂的三维环境中进行智能漫游。对这个困难的问题，虽然仍存在着更健壮的通用解决方案以进行游览，但是我们的系统仍然最终胜出了，因为它表现得既快速，而且概念也很简单，为我们创建了一个两者皆佳的解决方案。

Download at www.Pin5i.Com

3.8 在寻径与碰撞之间选择一种关系

Thomas Young
PathEngine
thomas@pathengine.com

对于很多游戏来说，AI 只不过是要使人物在一个环境里面来回运动而已。如果当玩家移到一堆箱子后面的时候无须做出什么决策的话，那么就根本不需要拥有一个复杂的决策系统了。另一方面来说，如果一个人物能够正确地处理其所在环境中的障碍物，那么即使只有一个很简单的决策系统，也能带来令人印象深刻的 AI。

我们可以把寻径器看成是一个系统，其责任是要能理解碰撞。在创建一个能够理解如何对其所在环境的障碍物进行处理的人物时，寻径与碰撞系统之间的关系是一个需要解决的重要的结构性问题。在本节中我们将展示 3 个针对此结构的方法，还将阐述每种方法的意义。

3.8.1 在碰撞控制下的运动

我们可不想要人物能够穿越墙壁、桌子，或是其他的障碍物。为了防止这种情况的发生，人物的运动可以由一个碰撞子系统来控制。AI 在做决策实现人物的运动之前必须首先得到此子系统的决策。

这样就在生成人物运动的代码与裁决运动的碰撞代码之间产生了强烈的依赖性。其行为代码依赖于碰撞代码以确保能按预想的方式进行。假设有这么一种情况，行为代码认为一个人物能径直走向玩家，而实际上人物会被障碍物挡住。因此，行为代码必须将碰撞考虑进来。

3.8.2 对于寻径的碰撞模型

一个寻径器的责任就是要允许在某方向上的运动，同时也要禁止其他方向上的运动。这可以用来帮助我们理解碰撞是如何工作的。我们可以将之称为寻径器的碰撞模型。

我们可以使用寻径空间和障碍空间的术语对此碰撞模型进行描述。在一个基于分片或是单元的寻径器中，寻径空间就是由无障碍物的分片或是单元组成的。图 3.8.1a 展示了一个基于分片的寻径空间（白色的单元）和障碍空间（黑色的单元）。在一个基于可视点或是路标的系统中，寻径空间是显式地利用一组点来表示的。图 3.8.1b 展示了在此种情况下寻径空间可

Download at www.Pin5i.com

能的样子（请注意此寻径空间必须和一组使用到的点对应，否则寻径器就无法正常工作了）。当生成一个路径的时候，寻径器将允许对象在寻径空间中运动，但是不允许其在障碍空间中运动。

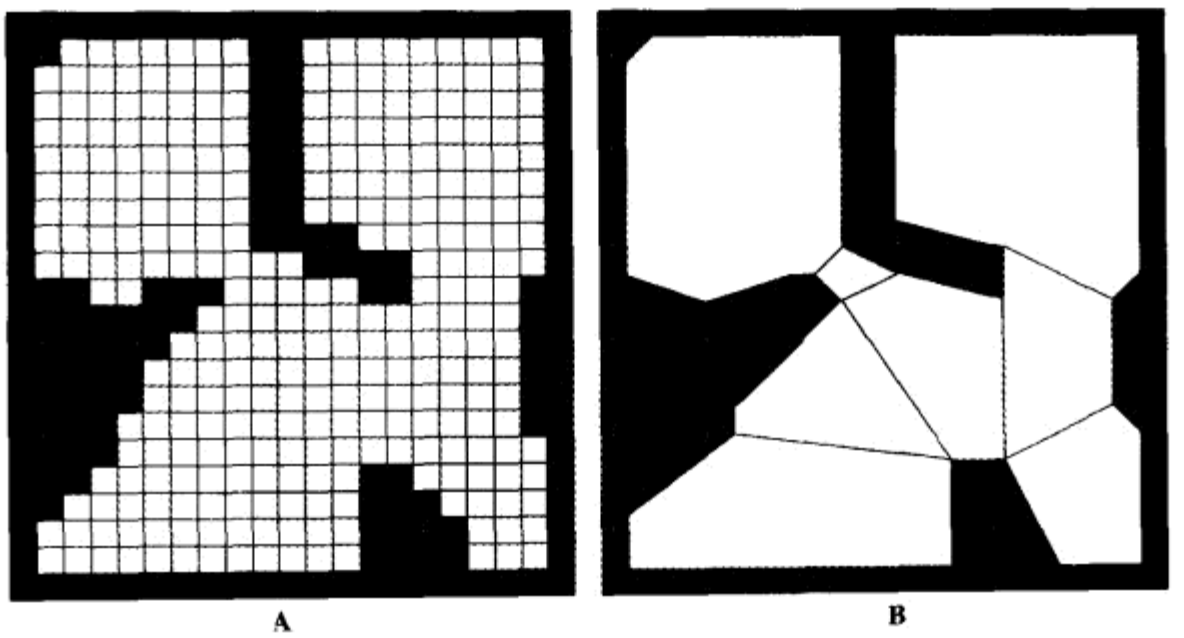


图 3.8.1　（A）基于分片系统的寻径空间（B）基于可视点系统的寻径空间

正是此用来寻径的碰撞模型与真正应用到运动中碰撞之间的关系造成了下面三种方法的不同。

3.8.3　方法 1：具有容错性的 AI

如果说在用来寻径的碰撞模型与真正应用到运动中的碰撞之间有确定关系的话，那么这就意味着在这两个系统中有着依赖性或是连接关系，这也意味着不管编写寻径系统的人是谁，他都需要及时得知碰撞系统中发生的任何改动，反过来也是如此。这样才能确保此种关系没有被打断。

从一个架构的角度来看，降低依赖性是一个很好的建议。人类的行为就并不需要依赖于理解我们环境中碰撞背后确切的物理现象。对于如何穿过此环境，我们只有一个大致的想法，然后得到反馈，进而避免碰撞。我们能不能在自己的 AI 中使用类似的方法以避免与碰撞系统发生直接的关系呢？

具有容错性的方法背后就有这样的想法，它认为寻径所需的碰撞模型只需要大致对应于真实的、更详细的针对运动的碰撞就行了。我们可能需要手工改动寻径器的表现形式以对碰撞进行模拟，或是自动使用碰撞使用的世界与对象就行，而不需要知道有关此碰撞的更深层的那些假设条件。寻径器可以用来生成具有指导意义的路径，接下来 AI 将使用容错性方法以确保此人物能在碰撞系统的指导下沿着这些路径行走。

在图 3.8.2 中，碰撞是在一个人物和画成黑色的障碍物之间发生的。寻径器将使用一个多边形寻径空间（画成白色的区域）对此进行模拟。其中的黑线显示了由寻径器返回的一条指导性路径。灰线显示的是人物必须移向的方向以避免发生（与黑色障碍物之间的）碰撞，同时还要沿着指导路径的大方向继续前进。[Reynolds97]描述了如何在对行为进行操控的同时得到此种类型的效果。

Download at www.Pin5i.com

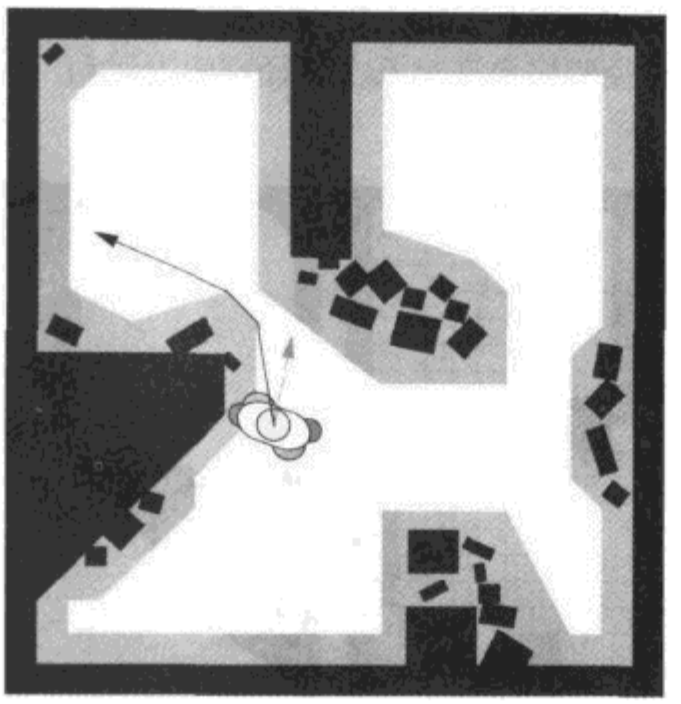


图 3.8.2 进行操控以避免碰撞

这种容错性方法是很有吸引力的，因为它可以产生一个从真实世界的行为看来很合理的结果。如果我们有一个简单的碰撞，还有一个相对较小的世界以测试自己行为的话，这种方法将会工作得相当出色。寻径空间对真实碰撞的模拟越接近，则此方法就会工作得越出色。然而，如果我们的碰撞比较复杂的话，那么就很难使用一个寻径器对之进行模拟了。如果我们的行为需要被应用到很多不同的情况中去，则此方法也会带来一些问题。

在某些情况下，找出正确的方向是极其困难的。对于在图 3.8.3 中的人物来说，它需要改动数次方向以通过此间隙。在图 3.8.3 中曲折的灰线表明了前进所需的方向。如果在类似的情况下无法找出正确方向的话，就可能产生人物面对障碍艰难前行的现象。在更极端的情况下，有可能人物会被困在这些障碍之间。

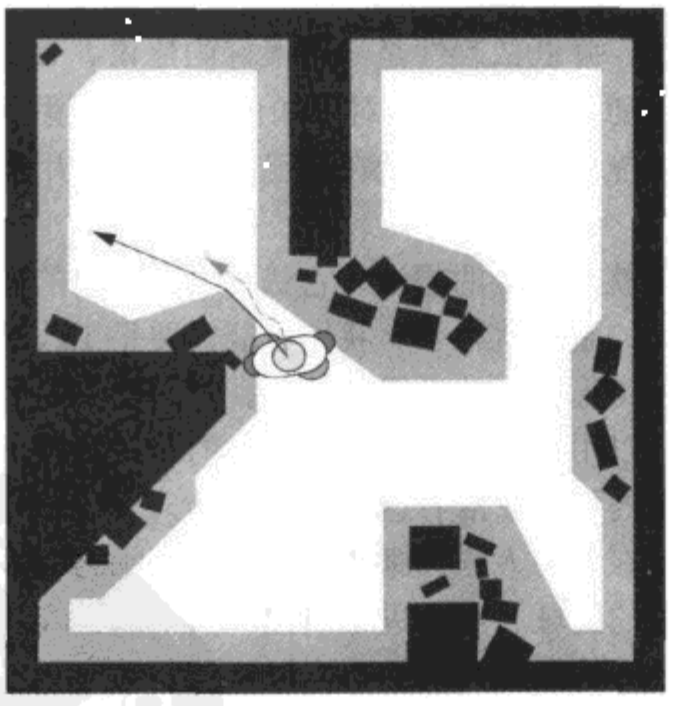


图 3.8.3 进行完全正确的操控是一个复杂的问题

Download at www.Pin5i.Com

为了能在复杂情况下找出正确的方向，就需要对未来的运动进行考虑。在图 3.8.3 中的灰线实际上指出了一条规划的运动路线，而不仅仅是一个方向矢量。我们需要结合一系列的路径分段来找出通过此间隙所需的运动。第一段应该能将此人物放置到正确的位置以继续沿第二段前进，依此类推。在此情况下，要找出正确的方向确实需要进行规划，而且需要对碰撞有一些详细的了解。

有时候，可能真的无法沿着指导路径前行。假设我们现在有一条比图 3.8.3 中稍微窄一点的间隙。在此情况下，我们需要人物预先就得知间隙是不可穿过的，从而可以选择另一条更合适的路径。再一次，我们需要进行规划，而且需要对碰撞有一些详细的了解。

或许我们可以对容错性方法进行扩展，通过使用从碰撞得来的反馈进行规划解决这些问题。当人物第一次到达图 3.8.3 所示的间隙，在使用了反馈式的行为试图找出一条通道的企图失败了以后，我们就可以在寻径中将其标志为堵塞状态。然而，这种容错性很难实现，而且仍然不能对行为的可靠性提供任何真正的保证。我们无法禁止一个人物陷入其无法走出的困境中。

另一个解决方案可以使用机器学习的技术，例如神经网络的训练，以从碰撞的反馈中建构相应的行为。这种技术能有效地自动建立碰撞与行为之间的连接——但是请注意此时仍然对碰撞有依赖性。这种类型的技术所产生的结果极大地依赖于碰撞的真实本质，因此任何对碰撞系统的改动都有可能会打断其与行为之间的联系。即便碰撞系统不发生任何变化，这些技术在面对与其训练环境不同的碰撞情况时也会表现不佳。

如果我们希望保证行为的可靠性，那么最好能接受 AI 与碰撞之间关联的事实。这样，我们就可以在自己的 AI 中建立对碰撞的理解了。

3.8.4 方法 2：在无障碍空间一个子集内的寻径

在这种方法中，我们可以在碰撞和寻径中建立一种关系，同时也避免了在它们之间产生依赖性。特别是，我们需要保证在寻径空间中所有的点都不会发生碰撞，或者换种说法，对于寻径器而言，其有效空间应该是真正的无障碍空间的一个子集。

真正的无障碍空间指的是这么一组位置，在这些位置上放置的人物不会发生碰撞。如果人物所处的方向会影响其是否会发生碰撞，那么此方向将为无障碍空间加入新的维度，这样在一个无障碍空间中的位置上还要加上方向作为组成部分。对于这种方法来说，寻径空间的维数将比真正的无障碍空间要少，而且寻径空间的边界会更加简单。最重要的是在寻径空间中从一个位置到达另一个位置的时候是保证不会发生碰撞的。

图 3.8.4 展示了如何使用此方法来建构一个寻径空间。在此处，碰撞的旋转维度被消掉了，这是通过把人物放在一个足够大的，足以消除人物旋转带来的不同之处的边框里面做到的。程序对障碍物进行了简化，其方法就是用凸包将其表示出来，然后对得到的几何图形进行扩张，以适应那些边框（参见[Young01]以获取更多的信息）。其结果就是我们可以保证，现在自己的人物在任何地点，不管如何旋转，都不会在寻径空间里面发生碰撞。

我们可以在基于分片的寻径中使用类似的方法。在这种情况下，我们将在寻径格网中进行其形状的扩张，然后将把任何与这些形状重叠的格网标志为堵塞状态。

Download at www.Pin5i.com

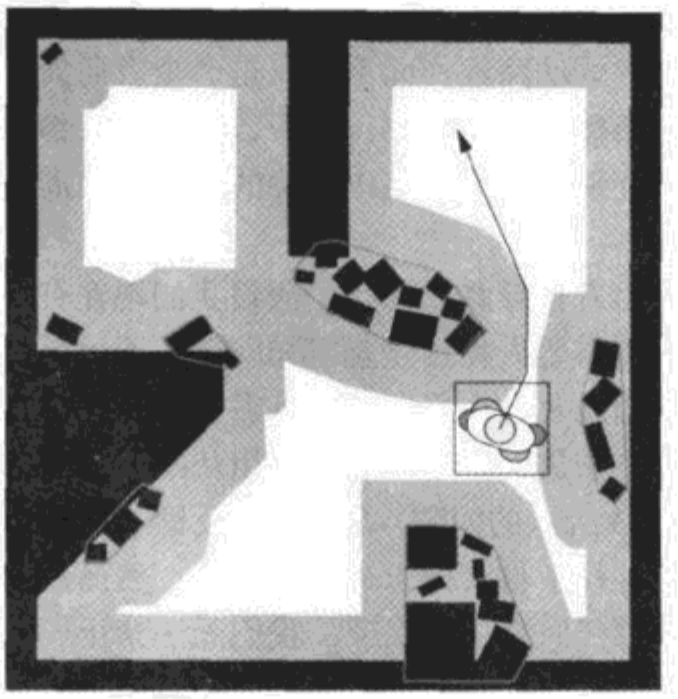


图 3.8.4 在一个无障碍的子空间里进行寻径

此方法的优点就是现在寻径器可以告诉我们如何在不发生碰撞的情况下进行运动。寻径器返回的路径可以直接使用，不再需要一层反馈行为了，这是因为我们已经知道这条路径是不会被堵塞的。AI 仍然不知道该如何穿过狭窄的间隙，不过现在这一点已经被集成进运动规划里面了，因此我们可以选择另一条路径或是采取任何适当的行动。如果我们希望人物能够进入环境中左上角的位置，则需要拓宽这条间隙。但是现在我们只能使用寻径器来自动把这种问题告诉我们。

由于寻径空间只是真正的无障碍空间的一个子集，因此我们必须考虑到，有可能人物发现自己已经处在了有效空间之外的一个区域。在使用由寻径器返回的路径时，人物应该绝对不能走到无效的位置上去。但是我们应该考虑到这种可能性，那就是人物可能会由于其他人物的关系，或是爆炸，或是其他什么原因，给推到寻径空间以外去。

寻径器无法告诉一个寻径空间之外的人物如何无碰撞地回到寻径空间里面来。一旦一个人物处在了寻径空间以外，则我们那就会遇到与前一个问题同样的麻烦。我们可以使用反馈式的方法重新回到有效位置来，但是如果一个人物被推到了狭径里面去的话，就有可能此被最终困住。这种情况是极少会发生的，因为人物不会自动进入这些间隙中去，因此，反馈式方法可以仅仅关注于如何把人物运回到一个有效的位置上去。这样编码就容易得多了，而且也更加有效。

一个优秀而实际的解决方法就是要使我们能保证当人物返回寻径空间的时候，人物的行为能对碰撞进行欺骗。如果每件事都运行正常，而且没有几何图形上的错误配置，那么我们就不需要这种保证了。不过如果在设计的时候加入了这种考虑的话，则我们在 beta 测试的时候就不会老来处理这种问题了。

1. 查找一个有效的位置

对一个抛弃在寻径空间之外的人物来说，其得到一个有效位置最简单的方法就是要持续跟踪那些最后已知的好位置。不过如果人物运动到了寻径空间外一个相当远的距离之后，这

Download at www.Pin5i.Com

种方法可能会得到一个较坏的结果（参见图 3.8.5）。此外，有时候我们还需要处理那些在寻径空间外的没有最后已知的好位置的地点。

另一个更精致的方法是要求寻径器提供一个查询功能，对于一个给定的在寻径空间外的点，它能返回一个最近的寻径空间内的点。这种方法相当好，而且其通用性更强。但是在某些情况下，有可能这个返回的点刚好在人物当前所在位置墙壁的另一侧（参见图 3.8.5b）。一旦发生了此种情况，此人物就无法向着那一点的方位前进以返回有效空间了，除非是进行碰撞欺骗。如果可以进行碰撞欺骗，那么一个人物就可以透过墙壁进入游戏中的另一个位置了。有时候这会产生一些很好笑的结果，但是无疑这是一个严重的程序错误。

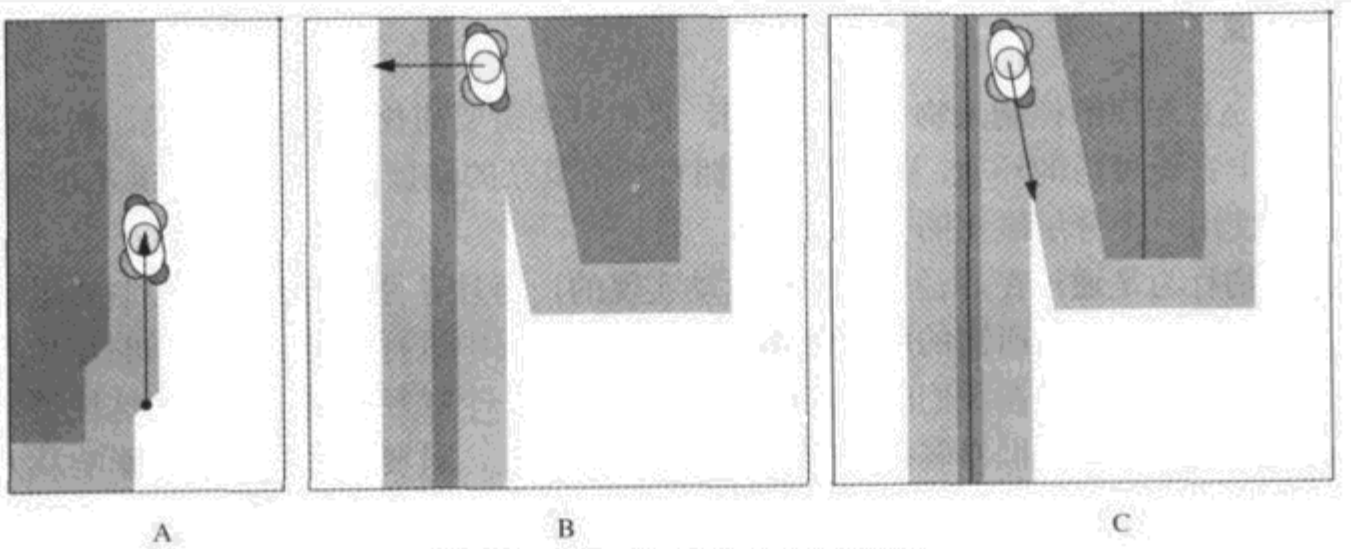


图 3.8.5　（A~C）寻找一个有效的位置

2. 为世界加上骨架

我们可以对最近有效点进行改造，方法是给世界加上额外的一层硬编码的碰撞 “骨架”（参见图 3.8.5c）。这些骨架可以被显式地表现出来，也可以基于现存世界的形象表现出来。如果此世界已经被分解为单元了，那么只要使用这些单元之间的边界作为骨架就够了。对最近有效点的查询可以被修改为对最近不穿越骨架点的查询。这就可以给我们一个更为合理的结果。

如果在人物返回寻径空间的过程中可以进行碰撞欺骗，那么在这种情况下，骨架提供了一种方法以让我们增强对游戏世界中连接度的限制。从理论上来讲，这意味着现在寻径需要绕开骨架，这样就可以维持我们的保证，以确保人物总能最终返回到一个有效的位置上。但是在实际情况中，这一般是没有必要的。如果我们真的需要实现绕开骨架的寻径，那么这将对层次性的寻径起到一定帮助（参见[Rabin00]）。

3.8.5　方法 3：使用寻径器本身处理人物碰撞

我们前面谈到的方法都比较复杂，其原因是碰撞系统中的障碍物可能会不为寻径器的简单模型所理解。为了避免这一点，我们在第三种方法中使用了寻径器本身来处理人物碰撞。此方法保证了人物不会被推到寻径空间之外，而且，因此也保证了寻径器总是可以了解一个人物当前所处的位置（而且寻径器返回的路径是可以直接使用的）。这将给我们带来一个较为简化的架构和可靠的人物运动方式，但是它也带来了代价，那就是我们不得不在人物碰撞中

Download at www.Pin5i.Com

使用一个简化的机制。

使用简化的人物碰撞还会带拉一些其他的好处。现在碰撞检测就会非常快速、健壮，而且可预测了。我们可以使用碰撞系统来实现一些约束，以限制游戏中的流程或是触发一些脚本，而且可以确信这些都将按我们所预期的执行。在另一方面来说，我们也将丧失很多潜在的在人物和世界之间可能发生的有趣交互，而这本来通过一个复杂的碰撞系统是可以提供的。

人物可能仍需要与处在寻径空间之外的位置打交道，为了这一点，我们仍然需要一个查询功能，以便找到最近的有效点。其优点就在于我们再也不需要依赖于那些方法以将人物从可能困住的地方给引导出来了。

分层碰撞

为了让 AI 能理解控制人物行走的机制，我们首先需要简化此机制。但是，如果我们希望创建一个丰富而可信的环境，则仍然需要拥有一个复杂的碰撞交互。要同时满足这些需求，我们就需要使用一个分层碰撞的架构。

如果人物行为是通过在寻径空间中的运动实现的，而且寻径器能够理解控制此运动的系统，那么我们就已经得到所需的所有保证了。在寻径空间中进行运动通常就指的是从人物的起始点开始进行的平移。在我们的分层架构中，专门有一个寻径碰撞层来处理这种类型的人物运动。我们可以在一个世界碰撞层中提供更复杂的交互，只要这些交互不会影响从人物起始点进行的运动。

世界碰撞层可以负责考虑进人物所在位置的额外维度信息（例如此位置的垂直坐标或是人物面对的方向），前提是如果这些维度与寻径器的碰撞模型无关。请注意，如果一个人物需要旋转到一个特定的方向才能运动的话，那么在将世界碰撞层应用到人物方向的时候一定要特别小心。世界碰撞层也可以提供一个人物的肢体与环境发生的交互，只要这些交互不会使得人物离开其所在的位置。

将某些事件标志为临时性或是异常性事件之后，我们就可以给系统添加一些更为有趣的交互行为了。在人物沿路径运动的时候，可以为之添加一些事件，产生一些影响。这样对于这种事件来说，我们允许世界碰撞层对寻径空间中的运动施加影响。这些影响包括：例如由一个爆炸产生的冲击波，或是与一个投射武器发生的交互。对于此类事件，只有一个限制条件，那就是这种交互不能使得人物运动到寻径空间之外，这样我们就可以保证人物以后仍然能够回到此处继续进行寻径。图 3.8.6 展示了这么一个例子，其中由一个 3D 交互产生的力量可能会反馈到寻径碰撞层中去。

图 3.8.6 碰撞层之间的交互

Download at www.Pin5i.Com

3.8.6 实现沿路的运动

对于第二种和第三种方法，我们假设一个沿路径行走的人物不会离开寻径空间。为了确保这一点，在实现一个沿路运动的时候，我们必须加以小心。

实现人物控制最常见的想法就是要使用一个转弯一移动的接口。在继续运动以前，我们可能会使用一个范围内的角度来探测一下，以确保转过的方向正对着下一个路径上的目标。然而，这将导致人物与顶角相交的问题（参见图 3.8.7a）。即使我们得到了完全正确的角度，一个人物也有可能由于越过路径目标而最终达到一个无效的位置（参见图 3.8.7b）。对于方法 2 来说，这意味着寻径器必须找到最近的有效点，但是运动到那一点会牵涉确定到沿路行走的平滑度。对于方法 3 来说，这可能会导致人物会被寻径碰撞所堵塞住，以至于无法沿路行走了。如果让人物沿着寻径空间的边缘行走，将在很多情况下有所帮助，但是也不能应付所有的情况。

如果想要在一个转弯一移动的接口里解决这些错误，其代码会非常复杂。我们可以对之进行简化，使用一个参数化的沿路运动的方法（参见图 3.8.7c）。转弯的限制在沿路运动之前就可以被解决掉了，但是对于路径中的转弯限制来说，我们最好对路径进行修正以满足这些限制条件（参见图 3.8.7d）。

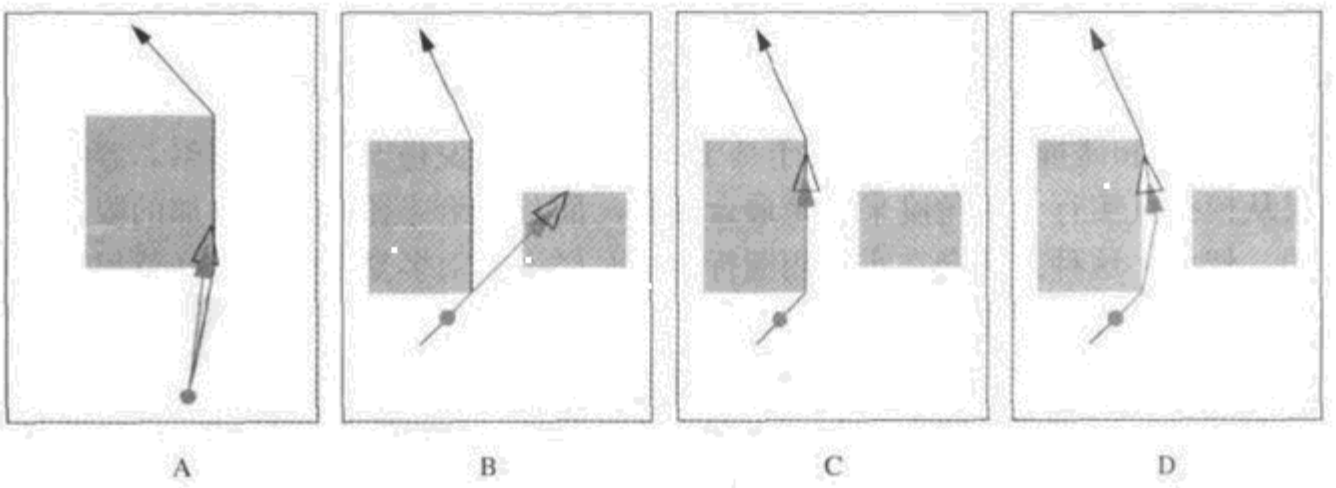


图 3.8.7 （A～D）沿着一条路径移动

即使是对于参数化的运动，仍然有一些对沿路的点进行近似的问题。最常见的问题就是一条路径将沿着寻径空间的一条对角线式的边界产生，如图 3.8.8a 所示。如果在距离此路径上的某处近似出一个点，则其很有可能会位于寻径空间之外。在这种情况下，我们将考虑到如何选择近似点的方向，这要视这条路径在此分段上之前和之后的路段方向而定。

然而，选择一个近似的方向并不能解决如图 3.8.8b 所示的情况下的问题。在此处，即使我们得到了相对于下一个拐弯处正确的点的方向，我们仍然会得到一个在障碍物中的近似点。还有一个更加通用的方法，那就是首先尝试着向第一次产生的近似点前进，然后，当失败的时候，再产生另一个近似点，不过这次再换一个方向来。

在处在如图 3.8.8c 所示的情况下时，可能在相应的路径分段上面没有有效的点。这种情况是相当罕见的。因此对于静态地图来说，我们可以在地图有效性检查的部分添加一些代码以检查是否有这种情况发生，一旦发生了，就对地图进行修改以消除这种问题。对于那些在运行时

Download at www.Pin5i.Com

可能改变的几何图形来说，我们就不能这么做了。有一个解决方案，就是可以将动态几何图形的边缘限制在垂直、水平和45°角上，这是因为对于这种边缘来说是不会产生此种问题的。

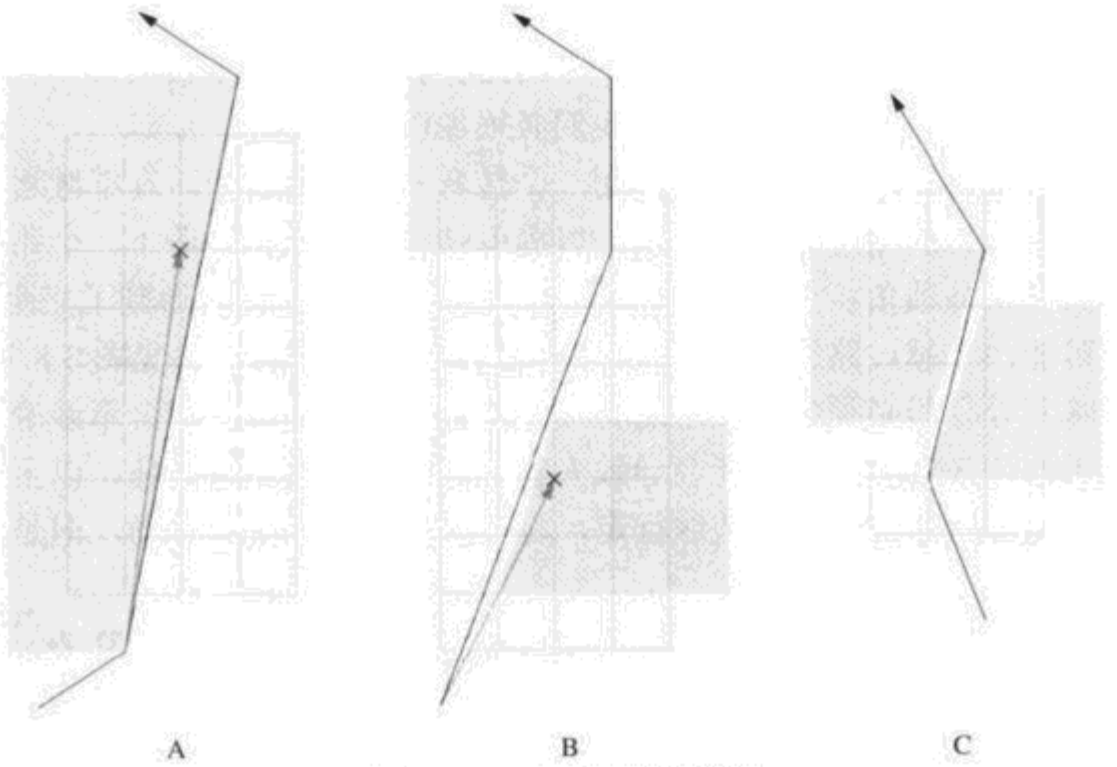


图 3.8.8 （A～C）问题的近似处理

3.8.7 结论

在AI、寻径和碰撞之间的关系对于基于运动的AI来说是一个关键因素。容错性方法产生的结果从模拟真实行为的角度来看是最合理的，而且从给寻径和碰撞添加的限制上来看也是最小的。然而，这种方法会在实现相应行为的时候添加很多复杂性，这样就很难创建出一个能可靠地工作在各种环境下的行为了。

在真正无障碍空间的一个子集里面进行寻径，在大多数时候都能给出可靠的行为，其代价是需要在寻径和碰撞子系统中加入一些极少的关联性。由于人物仍然可能处于寻径空间以外，因此我们仍然需要处理这种问题。如果我们能允许在某种情况下进行碰撞欺骗，那么这种方法可以对行为的可靠性进行保证。

通过使用寻径器本身来控制人物的碰撞，我们得到了最简单的架构和最可靠的行为。如果需要特别确保其行为结果的可靠性，那么这种方法就是你想要的了。通过加入一个分层的碰撞架构，我们可以提供更为有趣的碰撞交互。

第二个和第三个方法可以被看成是“控制异常情况发生”的方法。为了能让这些方法发挥最佳作用，我们需要对之进行扩展以得到实现沿路运动的方法。对于对真实行为的模拟来说，这些方法没有给出最合理的结果，但是对于那些对AI要求极高的游戏来说，它们提供了可以达到次优，而不至于导致过高代价的方法。

3.8.8 参考文献

[PathEngine] Path Engine, 对应网址为 http://www.pathengine.com, January 2002.

Download at www.Pin5i.Com

[Rabin00] Rabin, Steve, "A* Aesthetic Optimization," *Game Programming Gems*, Charles River Media, Inc., 2000.

[Reynolds97] Reynolds, Craig, "Steering Behaviors for Autonomous Characters," 对应网址为 http://www.red3d.com/cwr/steer/, September 6, 1997.

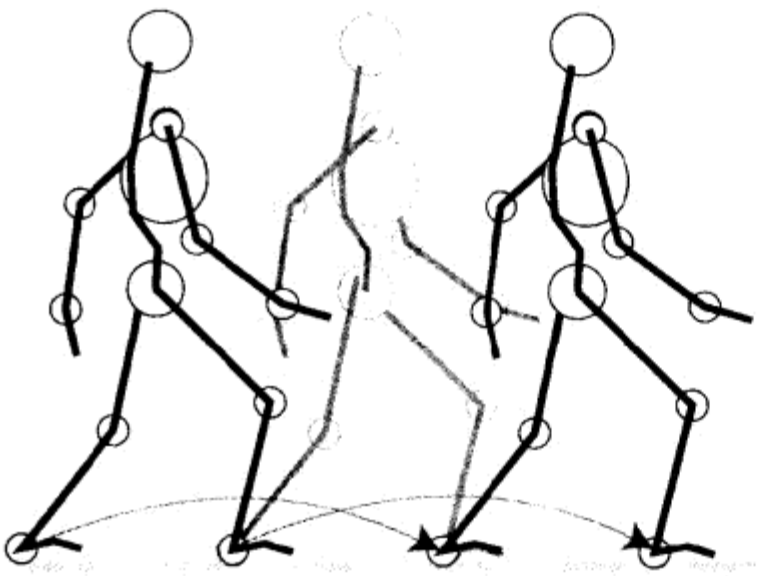
[Young01] Young, Thomas, "Expanded Geometry for Pointer-of-Visibility Path-finding," *Game Programming Gems 2*, Charles River Media, Inc., 2001.

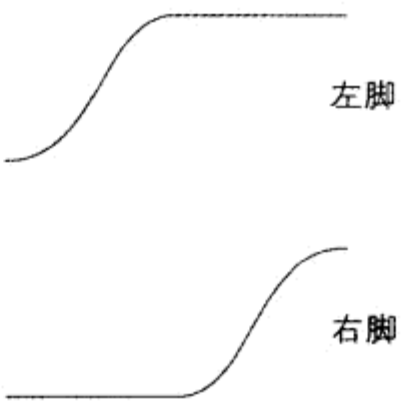
Download at www.Pin5i.Com

Download at www.Pin5i.Com

第 4 章

图形





Download at www.Pin5i.Com

简介

Jeff Lander
Darwin 3D, LLC
jeffl@darwin3d.com

对于游戏中图形部分的开发来说，我们正处在一个令人激动的时代。不管是在游戏机上，还是在个人电脑上，开发人员都可以利用强劲的图形硬件为自己的游戏创建出绚烂的图形效果，令人叹为观止。不过，游戏玩家也随之提出了更高的要求。玩家们总是希望下一个游戏比现在的游戏更美妙，而且期望开发人员能充分利用不同平台上的资源创造出更好的效果。

在最近几年里，游戏图形开发人员手头上的工具逐渐多了起来。所有的图形都是通过对屏幕上的点进行直接操作得到的。通常来说，一个图形开发人员总会有一个使用汇编编写的程序段，它能对屏幕上的图像数据块进行拷贝。如果在游戏中使用到了 3D 图形，那么在其中总会有简单的描画直线的函数、光照计算函数和平面着色或是带有纹理三角形的光栅化函数。我们将进行研究，看如何在系统的 CPU 里面进行快速运算以实现已知的 3D 渲染技术。

随着系统的改进，开发人员将对这些技术进行扩展，实现更为复杂的 3D 图形渲染算法。开发人员一直都在从视觉效果研究公司、教育科研机构，以及各种会议，例如 SIGGRAPH，中吸取新技术应用到自己的游戏中。

然而，现在的开发人员正面临着一个重要的选择。现有图形研究的方向已经与游戏系统的要求不再一致，在游戏上的图形开发人员面临的需求与其他计算机图形应用开发人员面临的需求也不再一致了。因此，轮到游戏的图形开发人员自己参与进来，贡献出自己的资产、力量和智慧以创建出一条新的道路了。

本章的这些文章可以说是在这条新路上的探索。其作者包括了开发人员、硬件供应商和研究人员，他们都对游戏图形开发人员面临的独特问题贡献出了自己的知识与专家级经验。这里的文章可以分为两类——与游戏场景几何模型处理相关的文章和与几何模型渲染相关的文章。

场景几何模型方面的问题包括要计算出渲染所需模型中的各个元素。我们这里开头就是 Eric Lengyel 的文章《消除 T 形连接与重新三角化》，在其中他给出了一个算法，解决了渲染中一个臭名昭著的问题。

Download at www.Pin5i.com

顶点法线的计算对于场景的光照来说是极其重要的。对于可变形的几何模型来说，如何高效地计算出这些数值更是重中之重。Jason Shankel 的文章《快速高程场法线的计算》和 Martin Brownlow 的《快速计算面片法线》，就是专门用来解决此问题的。

其中还有三篇文章谈及了数据组织以及数据操作的问题。渲染得最快的多边形实际就是你不需要画出来的多边形，正如《快速、简单的遮蔽剪裁》中讲到的一样，这篇文章是 Wagner Corrêa 等人贡献的。在 Carl Marshall 的文章，《三角形条带的创建、优化以及渲染》中，他给出了一些技巧用来对数据格式进行优化。最后，在《针对复杂数据集计算优化阴影体》这篇文章里面，Alex Vlachos 阐述了如何创建动态阴影的问题。

对于游戏来说，人物的运动是一个重要的因素。玩家要求更加真实的运动效果，而且此要求越来越迫切。在 William Leeson 的《针对人物运动的表面细分》一文中，针对人物所在的离散三角形网格的计算，他提出了一个取代方法。Jason Weber 的《改良的骨节变换计算》中，针对骨架变换的运动系统，他提出了一个方法用于解决其特有的伸缩扭曲问题。在处理人物的运动时，将不同的运动混合起来是很困难的一件事（但是仍然可以办到），这一点在 Thomas Young 的《针对真实人物运动的架构》一文中得到了阐述。

随着可编程图形硬件的出现，渲染程序，或称着色器，在图形开发方面日益成为不可或缺的工具了。不过，这些语言都相当底层，而且与 API 相关。在《可编程顶点着色器的编译器》一文中，Adam Lake 给出了一个方法，它可以用来创建编程语言，然后编译成与平台相关的代码。

某些几何模型处理的文章已经涉及到了渲染。在《画板光束》一文中，Brian Hawkins 采用了矩阵方法使用 2D 精灵来创建伪 3D 对象。另一个使用 2D 技术创建 3D 效果的例子是在《针对等测引擎的 3D 技术》这篇文章里面给出的。最后，对对象的渲染是相当复杂、逐点进行的，为了处理这种渲染，我们需要计算出法向地图来。为了得到它，Oscar Blasco 在《使用法向地图进行曲面模拟》一文中使用了一种密集型网格模型。

几何模型处理本身不可能创造出美丽的图形。我们还需要对这些地形进行渲染。这些文章以 Nathaniel Hoffman 的文章《动态的、具有照片效果的地形光照》开始，在其中他讲述了如何得到真实自然的户外光照效果。紧接着这篇文章的是 Kenneth Hurley 的《立体图光照技术》，它描述的是如何使用硬件渲染来得到真实的光照效果。

对于游戏开发人员来说，生成纹理数据是一件麻烦事。随着视觉效果的复杂化，我们需要更多的纹理了。从产品的角度上来看，要手工创建出这些数据是非常昂贵的，而且在游戏平台上往往会带来存储量与内存的问题。实际上，我们可以使用数学模型在需要的时候将纹理创建出来，正如 Mike Milliger 在“程序纹理”一文中提到的。下一篇文章，《独一无二的纹理》，提供了一种方法以动态地将纹理混合起来得到更复杂、更多样化的场景。

当使用复杂的数学对每一点进行计算的时候，我们就可以创建出很多计算机图形特效了。即便是可编程图形硬件，在快速进行这些数学处理的时候，也常常会让人觉得不够灵活。不过，很多数学计算都是可以使用查找表来进行近似计算的。我们可以很方便地将这些表保存在纹理地图里面。本章最后的文章都是围绕这一点展开的，而如何实现它则在《使用纹理作为查找表进行函像素光照计算》一文中由 ATI 小组指出。之后，Jan Kautz 对其进行了扩展，在他的文章里给出了一个更贴近艺术创作人员的方法，这是在他的文章《使用手工制作的着色模型进行渲染》中提到的。

Download at www.Pin5i.Com

这些文章都是在新道路上进行的探索，这条道路是专门为了在游戏中创造新的图形技术而铺成的。做出贡献的作者以及本人都诚挚地希望，所有的开发人员都能加入我们的行列，沿着这条道路前进，在游戏界尚未开发过的新边疆上大展身手。

Download at www.Pin5i.Com

4.1　消除 T 形连接与重新三角化

Eric Lengyel
Terathon Software
lengyel@terathon.com

假设在一景中包含了两个多边形，它们共享一条公用边，如图 4.1.1a 所示。如果两个这样的多边形是属于同一个对象，那么代表这条公用边的端点的顶点通常就不会被重复存储了。取代方法是两个多边形都使、用同样的顶点，这样可以节省空间和总线带宽。图形处理硬件现在已经有了这样的设计，那就是当相邻的多边形使用完全一样的坐标来表示共享边端点的时候，光栅线生成的点对于每个多边形来说都是和另一个多边形相吻合的。在共享边上，不会产生一个多边形的点与另一个的点重叠的情况。而且，更重要的是，在两个多边形之间是没有间隙的。

当相邻的多边形属于不同的对象时会产生一些问题，此时每个多边形都会有一份自己对共享边两端顶点坐标的拷贝。这些顶点的坐标在各自对象的本地坐标系里面可能会非常不同。例如，当这些顶点被转换到世界空间中的时候，浮点运算的舍入误差就可能对每个对象产生略有不同的顶点位置。由于这些顶点的位置再也不会保持一致了，这样就有可能在多边形光栅化的时候产生一条间隙。

当两个多边形的边在空间上共享一段线段，但并不共享相同的顶点时，会产生更严重的问题，如图 4.1.1b 所示。在这种情况下面，一个属于某个多边形的顶点会处于另一个多边形的边上。考虑到此种情况下这些边组成的几何形状，我们可以将之称为 T 形连接。由于这两条相邻的边并不共享相同的端点，因此在任何一个没有对之进行消除的游戏引擎里面都会产生可见的间隙。

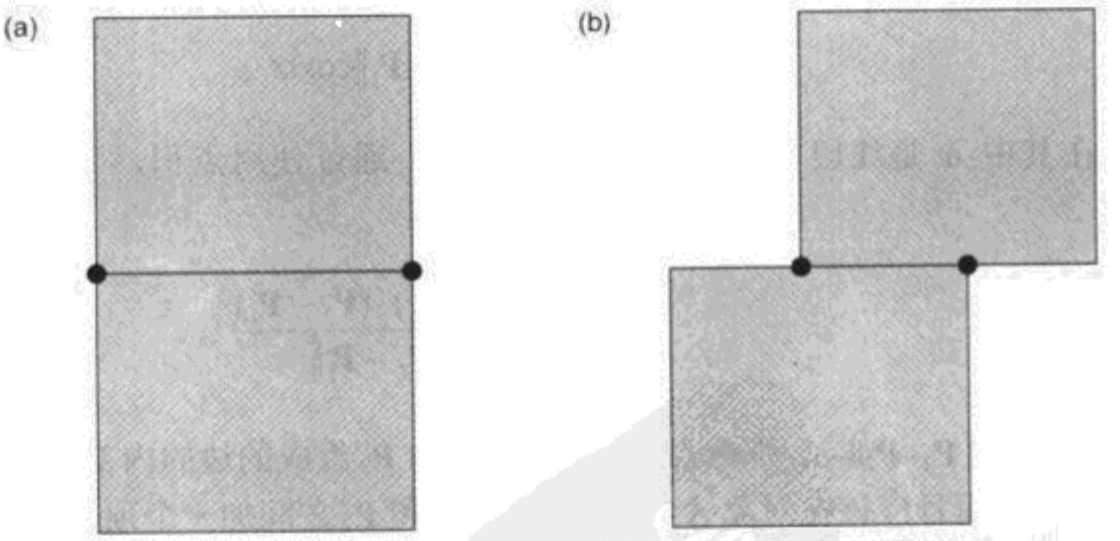


图 4.1.1　（a）相邻多边形共享一条边（b）相邻多边形的边重合，但是端点不一样

Download at www.Pin5i.Com

本节描述了应该如何在复杂的 3D 场景里面检查出这些间隙的来源，以及应该如何对静态的几何模型进行修正以避免产生这些可见的人为缺陷。由于 T 形连接的消除会给现存的多边形添加一些顶点（有可能并不是一个凸包），我们还将讨论一个方法以对任意的凹多边形进行重新三角化。

4.1.1 T 形连接的消除

在我们的世界中，对于一个给定的不可移动的对象 A，我们需要判断出是否存在任何其他不可移动的对象有一条与对象 A 相重叠的边。我们将仅仅考虑那些边界体（bounding volume）与 A 的边界体有重叠的对象。假设 X 是一个离对象 A 相当近的对象，而且有可能有相邻的边。我们假设两者边的数目都达到了上限值。在进行了 T 形连接消除的工作以后，我们将进行重新三角化，这样以避免创建多余的三角形。

在我们定位任何一个 T 形连接之前，让我们首先来看看是否对象 A 的任意一顶点会与对象 X 的任意一顶点非常接近。我们必须把两者的顶点都转换到世界空间里去，然后搜索那些距离在某个很小的常量 ε 以内的顶点。任何一个属于对象 A 但是却如此接近对象 X 的顶点 V_X 的顶点 V_A 都应该被去掉，这样一来，V_X 和 V_A 就可以用完全相同的世界坐标来表示了。这个程有时候就叫作接缝。

一旦现存的顶点被接缝了以后，我们就需要搜索出对象 X 上离对象 A 的某条边距离在一个很小值 ε 以内的顶点了，不过这个顶点距离对象 A 的任意顶点的距离应该都大于 ε。这可以告诉我们在何处发生了 T 形连接。假设 $\mathbf{P}_1$ 和 $\mathbf{P}_2$ 是对象 A 一条边的端点，再假设 $\mathbf{Q}$ 是对象 X 的一个顶点。点 $\mathbf{Q}$ 距经过了 $\mathbf{P}_1$ 与 $\mathbf{P}_2$ 直线距离的平方 d^2 可以使用下面的式子得到：

$$d_2 = (\mathbf{Q}-\mathbf{P}_1)^2 \frac{[(\mathbf{Q}-\mathbf{P}_1)\cdot(\mathbf{P}_2-\mathbf{P}_1)]^2}{(\mathbf{P}_2-\mathbf{P}_1)^2} \tag{4.1.1}$$

如果 $d^2<\varepsilon^2$，那么我们就知道点 $\mathbf{Q}$ 距离那条包含对象 A 的边的直线距离已经足够近了，但是我们仍然需要判断是否 $\mathbf{Q}$ 真的处在 $\mathbf{P}_1$ 和 $\mathbf{P}_2$ 之间。我们可以算出 $\mathbf{P}_1$ 和 $\mathbf{Q}$ 所在的线段对 $\mathbf{P}_1$ 和 $\mathbf{P}_2$ 所在边的投影长度 t。如图 4.1.2 所示，此长度由下式给出：

$$t = \|\mathbf{Q}-\mathbf{P}_1\|\cos\alpha \tag{4.1.2}$$

在其中 α 是线段 $\mathbf{QP}_1$ 与边所夹的角度。通过使用点积，我们可以算出此余弦值，如下：

$$t = \frac{(\mathbf{Q}-\mathbf{P}_1)\cdot(\mathbf{P}_2-\mathbf{P}_1)}{\|\mathbf{P}_2-\mathbf{P}_1\|} \tag{4.1.3}$$

如果 $t>\|\mathbf{P}_1-\mathbf{P}_2\|-\varepsilon$，那么点 $\mathbf{Q}$ 就不在 $\mathbf{P}_1$ 和 $\mathbf{P}_2$ 组成的边的内部，否则，我们就找出了一个 T 形连接，而且应该在对象 A 的多边形上 $\mathbf{P}_1$ 和 $\mathbf{P}_2$ 之间加一个顶点，正好落在点 $\mathbf{Q}$ 的位置上面。

Download at www.Pin5i.Com

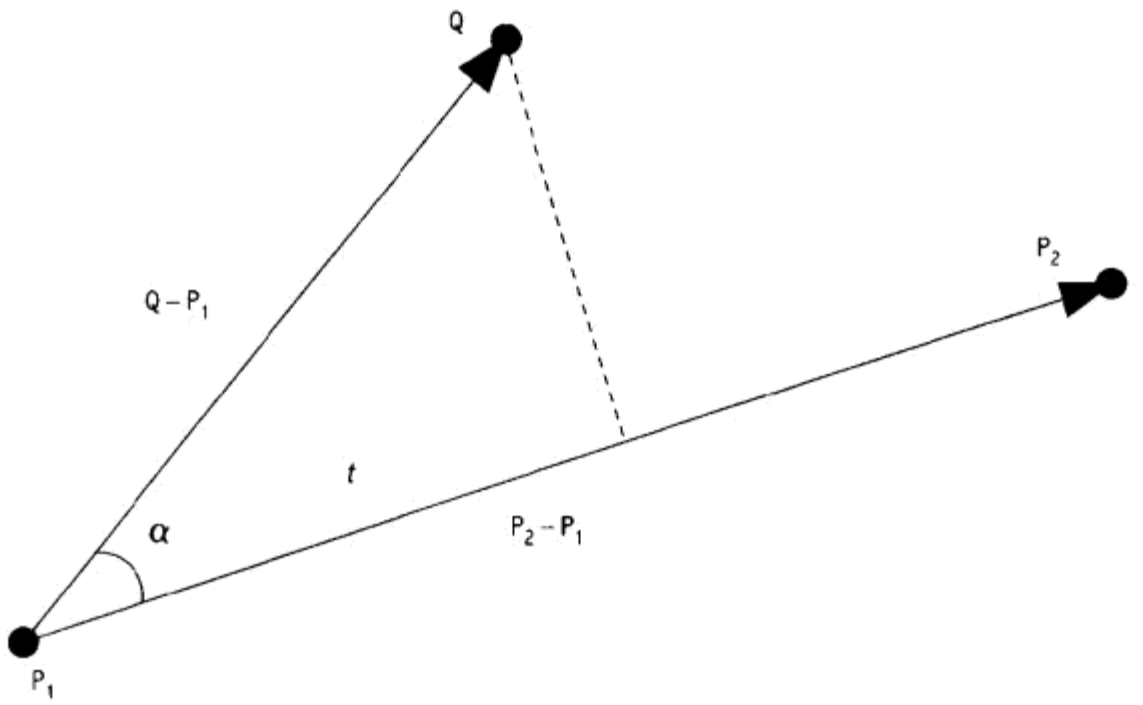


图 4.1.2　长度 t 等于 Q 点投影到 P_1P_2 上的点到 P_1 的距离

4.1.2　重新三角化

当所有静态世界的几何模型都被处理过以后，我们就必须对得到的多边形进行重新三角化，这样它们才能被送往图形处理硬件中去。任何一个为了消除 T 形连接而加入多边形的顶点都是与 T 形连接所在边上的顶点是同线的（或者至少几乎是同线的）。在消除了一个多边形所有的 T 形连接以后，其边上可能会含有几个处在同一条直线上的顶点。这就使得我们不能使用一般所常用的简单的扩展方法来对一个凸多边形进行三角化了。取代方案是我们将不得不把多边形看成凹多边形。

我们阐述的算法将使用一列 n 个逆时针分布的顶点作为输入，而且将产生一列 $n-2$ 个三角形。在每次循环中，我们都将搜索一组三个连续的顶点，它们形成的三角形将不会退化（也不会产生逆方向的三角形），而且不会包含任何多边形中余下的顶点。一旦找到了这样的三个顶点，其中中间的顶点就会被标出来以便在下面的循环中不再考虑它，而算法将继续重复进行直到最后剩下三个点为止。

为了判定是否三个顶点是逆时针方向排列的，我们必须事先知道包含此被三角化的多边形所在的平面。假设 $\mathbf{P}_1$、$\mathbf{P}_2$ 和 $\mathbf{P}_3$ 代表了这三个顶点。如果叉乘$(\mathbf{P}_2-\mathbf{P}_1)\times(\mathbf{P}_3-\mathbf{P}_1)$与法线 $\mathbf{N}_0$ 的方向一致，那么对应的三角形就是逆时针方向的。如果此叉乘几乎等于 0，那么此三角形就是退化的。因此只有在下面的不等式被满足的时候我们的三个条件中的两个才是成立的：

$$(\mathbf{P}_2-\mathbf{P}_2)\times(\mathbf{P}_2-\mathbf{P}_1)\cdot\mathbf{N}_0>\varepsilon \tag{4.1.4}$$

其中 ε 是一个很小的数（一般说来，$\varepsilon\approx0.001$）。

我们的第三个条件是要求三角形内部不能包含多边形其他的顶点。我们可以创建三个向内的法向矢量，$\mathbf{N}_1$、$\mathbf{N}_2$ 和 $\mathbf{N}_3$，分别对应于三角形的三条边，如下所示：

$$\begin{aligned}\mathbf{N}_1&=\mathbf{N}_0\times(\mathbf{P}_2-\mathbf{P}_1)\\ \mathbf{N}_2&=\mathbf{N}_0\times(\mathbf{P}_3-\mathbf{P}_2)\\ \mathbf{N}_3&=\mathbf{N}_0\times(\mathbf{P}_1-\mathbf{P}_3)\end{aligned} \tag{4.1.5}$$

Download at www.Pin5i.Com

正如图 4.1.3 所示，如果一个点 **Q** 在由 $\mathbf{P}_1$、$\mathbf{P}_2$ 和 $\mathbf{P}_3$ 组成的三角形内部的话，其充要条件就是对于每个 i=1、2、3 来说都满足 $\mathbf{N}_i \cdot (\mathbf{Q}-\mathbf{P}_i)>-\varepsilon$。

由于我们需要对每个三角形都计算由方程 4.1.5 给出的公式，因此可以将方程 4.1.4 的条件代入，这样就得到了等价的表达式，如下所示：

$$\mathbf{N}_1 \cdot (\mathbf{P}_3-\mathbf{P}_1)>\varepsilon \tag{4.1.6}$$

这可以判断出是否点 $\mathbf{P}_3$ 在连接 $\mathbf{P}_1$ 与 $\mathbf{P}_2$ 的边的正向上。

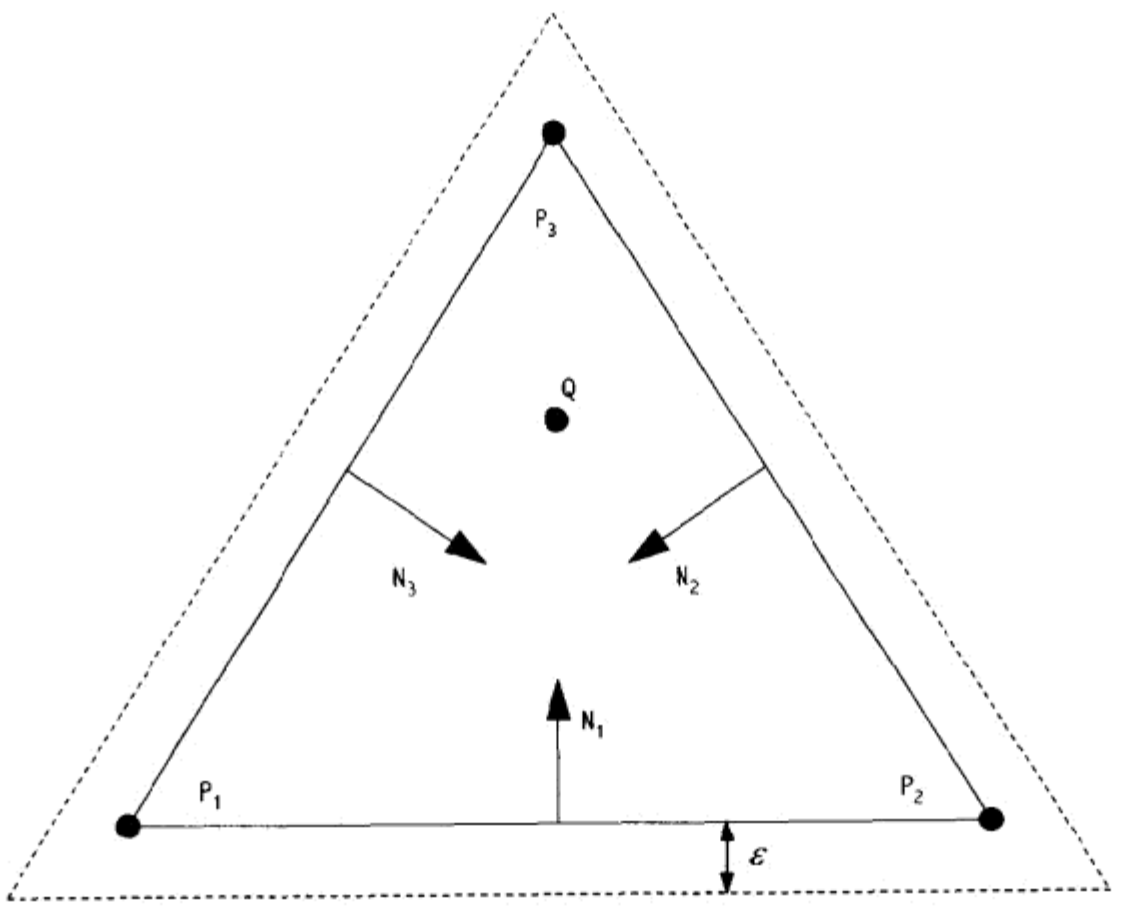


图 4.1.3 点 Q 位于三角形内的充要条件是它位于该三角条形三条边的正向上

4.1.3 实现

CD-ROM 上提供的源代码展示了一个重新三角化算法的实现。这个特定的实现维护了一个工作集，其中包含了 4 个连续的顶点，而且在每次循环中，将判断一下是否可以用开始三个或是结尾三个顶点形成一个有效的三角形。如果其中只有一组顶点可以组成一个有效的三角形，则此三角形将被忽略掉，算法将继续进行下一次循环。如果两组顶点都可以生成有效的三角形，那么代码就将选择那个拥有较大最小角的三角形。如果两组顶点都不能提供一个有效的三角形，则这组四个顶点组成的工作集就继续前进，直到可以使用此点集生成一个有效的三角形为止。

源代码中选用的方法是为了要在算法的输出中产生一组三角形条带和一组三角形扇状结构（triangle fans）。这样的三角形结构可以在现代的图形处理器上充分使用其顶点缓冲区。此实现还包含了一个安全机制。如果用户传入了一个退化的、自相交的，或是其他种类不可三角化的多边形，此算法就会及早退出，以防止陷入无限循环。在代码无法定位出一组能三

Download at www.Pin5i.Com

角化的连续顶点时就会发生这种情况。

4.1.4　结论

我们可以使用接缝技术连接相近的顶点，进行 T 形连接的消除以避免对例如相邻的对象之间的间隙进行渲染。

当这些操作作为预处理过程执行的时候，其得到的一组多边形可能会包含三个或者更多个同线的顶点。幸运的是，我们可以使用一种简单却健壮的算法来对这种多边形进行三角化，每次生成一个三角形，然后进行递归，对更小的子多边形进行三角化。

Download at www.Pin5i.Com

4.2 快速高程场法线的计算

Jason Shankel
Maxis
shankel@pobox.com

高程场（heightfields）是一个包含了高度信息的二维数组，它一般用来保存地形或是水面的地表数据，也通常用来对凸凹贴图进行计算。本节将描述如何能利用高程场网格的特殊性质来极大地优化顶点的法线计算。

4.2.1 一个任意网格上的法线

如果需要在最后的渲染结果中进行光照以及环境贴图计算的话，在高程场网格上的每个顶点都至少需要一个对应的表面法线。对于一个 3D 的网格来说一般有两种相关的法线——面法线与顶点法线。正如其名，面法线指的是每个网格面上的法线。而顶点法线就是每个顶点上的法线。

1. 面法线

计算面法线一般不是很复杂。首先得到面上共享一个顶点的两条边，然后定义两个矢量（v_1 和 v_2），它们的原点就在那个共享的顶点上，方向为沿着各自边的方向。面法线（n_f）就是一个单位矢量，其方向与 v_1 和 v_2 叉乘结果的方向一致。请注意，对于三角形来说任何两条边都是可以的，如图 4.2.1 所示。

$$n_f=(v_1\times v_2)/|v_1\times v_2| \tag{4.2.1}$$

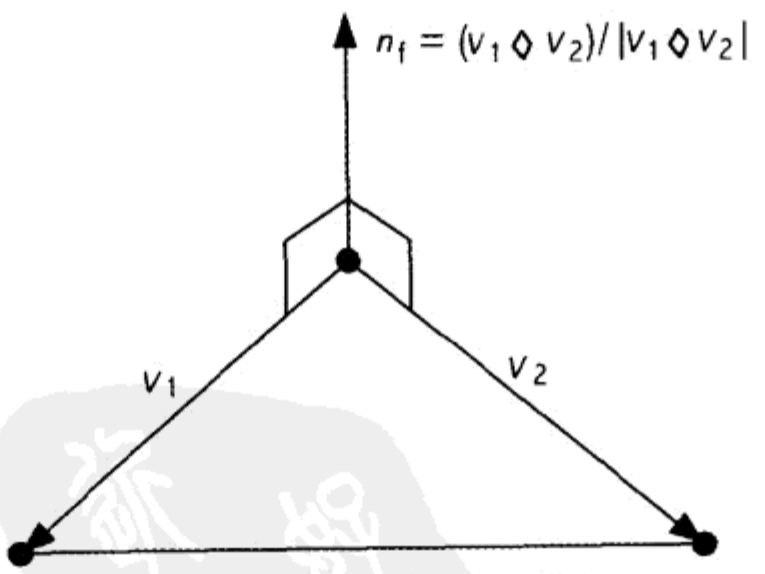


图 4.2.1 计算一个三角形的面法线

Download at www.Pin5i.Com

2. 顶点法线

顶点法线就不那么容易了。虽然对于一个给定的面来说只有一个唯一的面法线，但是一个给定的顶点却可能有好几个法线，每个都与一个面或是一组面相关联。然而，对于相当平滑的网格来说（对于高程场来说这是最典型的情况），我们可以对每个与此顶点相接触的面法线作一个平均，最后得到一个合理的唯一的法线。这个均值可以使用每个面在顶点的相对角来加权以防止过小或是过于分块化的面导致产生扭曲的结果。

假设$\{n_1, n_2, \ldots, n_n\}$是接触顶点 v 的面法线，假设$\{a_1, a_2, \ldots, a_n\}$是从第 1 到第 n 个面之间的夹角。

顶点 v 处的法线由方程 4.2.2 给出，如图 4.2.2 所示。

$$n_v = \frac{\sum_{i=0}^{n} n_i a_i}{\sum_{i=0}^{n} a_i} \tag{4.2.2}$$

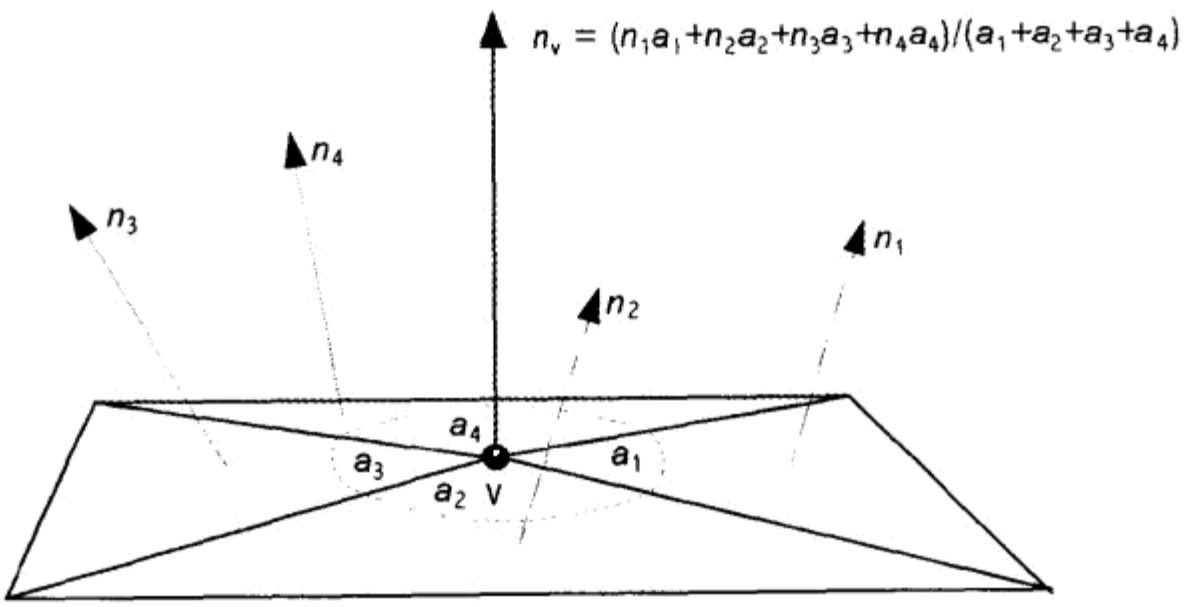


图 4.2.2　使用面法线计算顶点法线

4.2.2　高程场法线

一个高程场网格的顶点可以由如下 $v_{x,y}$ 的定义得到，其中

$$v_{x,y} = \{x, y, h(x, y)\} \tag{4.2.3}$$

在方程 4.2.3 中，x 和 y 是高程场格网均匀空间的坐标。而 $h(x, y)$则是在点 x, y 处的高度。对于一个高程场中给定的顶点 v 来说，我们可以将其相邻的顶点组织成如图 4.2.3 所示的样子。为了简单起见，$h_{1\ldots4}$ 表示的是相邻四个点的 h 值，以 v 为原点指向四个相邻点的

Download at www.Pin5i.com

矢量表示为 $v_{1...4}$。

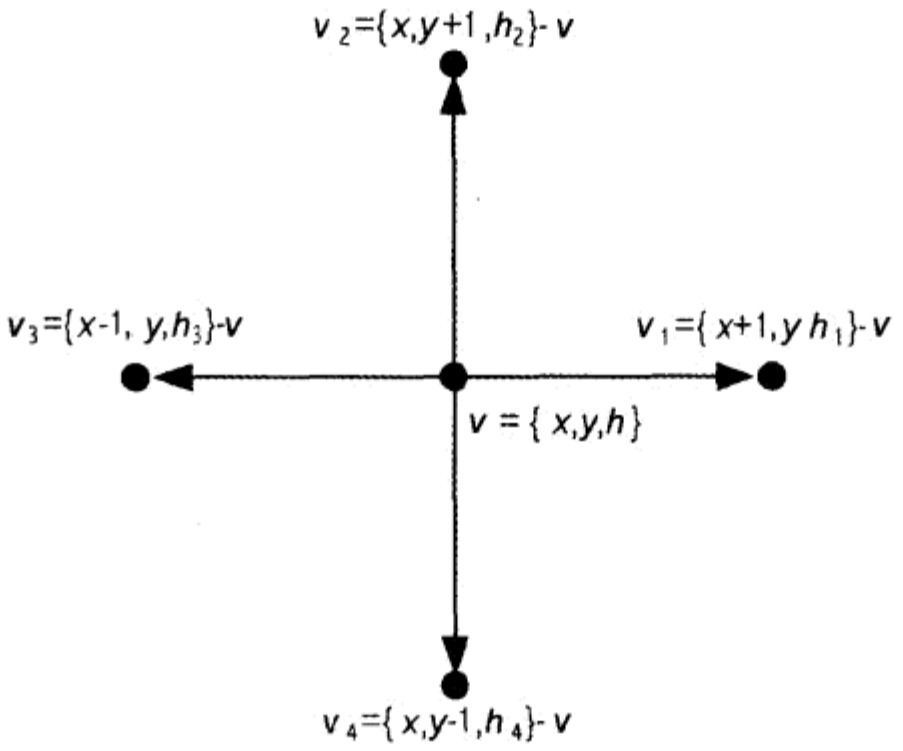


图 4.2.3 高程场中的相邻顶点

1. 利用假设进行简化

在得到了高程场顶点特殊的性质之后，我们就可以做一系列的假设以简化通用的顶点法线公式。

首先，我们可以假设在高程场中的每个顶点都准确地属于四个面。请注意，严格来说，有可能并非如此。我们假设网格是由方格组成，而其实最有可能是由三角形组成，然而这个假设对我们法线的质量不会产生负面的影响。

其次，我们可以假设一个顶点的每个面对顶点法线的贡献都是一样的，这样就不需要进行加权平均了。再一次，这个假设不是完全成立的，如果两个相邻的高度值相差极大，则它们对应面的角度就会与 90° 相差甚远，结果就会改变此面的“正确的”贡献。然而，当相邻高度值差异极大的时候，局部的顶点也就不再拥有唯一的法线了，这样一来，任何公式，包括“正确的”公式，其产生的结果都将不再可靠了。

最后，由于 x 和 y 的值是均匀分布的，当我们在 v 处设置原点的时候，相邻顶点的 x 和 y 坐标将变为 1、−1 或是 0。这些限制条件将极大地简化叉乘公式。

2. 数学运算

让我们从最原始的公式开始：

$$n_v = \frac{\sum_{i=0}^{n} n_i a_i}{\sum_{i=0}^{n} a_i} \tag{4.2.4}$$

Download at www.Pin5i.Com

接下来，由于此处只有 4 个面，而且每个对法线的贡献都是一样的，因此我们可以对均值进行简化：

$$n_v = (n_1 + n_2 + n_3 + n_4)/4 \tag{4.2.5}$$

现在，我们要去计算 $n_{1...4}$ 了，此时将发现叉乘已经被简化了：

$$\begin{aligned} n_1 &= v_1 \times v_2 = \{-h_1, -h_2, 1\} \\ n_2 &= v_2 \times v_3 = \{h_3, -h_2, 1\} \\ n_3 &= v_3 \times v_4 = \{h_3, h_4, 1\} \\ n_4 &= v_4 \times v_1 = \{-h_1, h_4, 1\} \end{aligned} \tag{4.2.6}$$

理想情况下，我们希望 $n_{1...4}$ 的大小都是一样的，因为如果它们大小不一将影响到均值公式。然而，由于只有在相邻点的高度值不一样的时候 $n_{1...4}$ 的大小才会不一样，因此，我们可以不再使用 $n_{1...4}$。

把它们加起来以后，可以得到：

$$n_v = (n_1 + n_2 + n_3 + n_4)/4 = \{2(h_3 - h_1), 2(h_4 - h_2), 4\}/4 \tag{4.2.7}$$

由于 n_v 的大小在此阶段无关紧要，我们可以将整个坐标乘以 2 以简化算术运算：

$$n_v = \{(h_3 - h_1), (h_4 - h_2), 2\} \tag{4.2.8}$$

由于 $h_{1...4}$ 极有可能只需要进行内存里的查找，因此显然此公式比对四个叉乘做平均来说要快得多了。有很重要的一点要记住，那就是 n_v 并不是一个单位矢量，因此可能需要对之进行归一化，这要视你的具体应用而定。

4.2.3　结论

我们经常使用高程场来存储地形和其他的静态对象。对于这些应用来说，计算顶点法线的速度可能并不是那么重要，因为法线极有可能由离线计算得到，或是只需要在数据载入的时候进行一次性的计算。

然而，对于使用动态高程场的应用程序来说（例如，为了模拟一片水的表面或是通过程序生成凸凹贴图的动画时），速度是一个极重要的因素。本节展示了我们如何能极大地改善标准顶点法线公式的性能，其方法就是充分利用高程场顶点的特殊性质。

4.2.4　例子程序

在例子程序中，我们在一个高程场中使用了快速的法线计算公式，此高程场是一段动画，在其中地形会在平面图形与分形随机图形中来回切换。此例子程序使用了 OpenGL 和 GLUT。读者可以参看彩图 4 看看此应用程序的屏幕截图。

Download at www.Pin5i.Com

4.2.5 参考文献

[Ebert94] Ebert, D., et al., *Texture and Modeling*, AP Professional, 1994.

[Fernandes00] Fernanders, António Ramires, "Terrain Tutorial," 对应网址为 *http://www.lighthouse3d.com/opengl/terrain/*, September, 2000.

Download at www.Pin5i.Com

4.3　快速计算面片法线

Martin Brownlow
Shiny Entertainment
mbrownlow@shiny.com

要想创建可以在各种层次的细节上渲染的平滑表面的话，表面面片（surface patch）是一个比较节省存储量的方法。然而，如果仅有一个光滑的表面，但是却不能对之进行正常的光照，那么它也就没什么用了。为了达到此目标，你需要得到每个顶点上的法线矢量。天啊，这需要实时地对每个点进行计算呢。

4.3.1　定义

虽然此处描述的方法对于任何基矩阵都是适用的，但是在本节中，我们将仅限于讨论双三次贝塞尔面片（以后简称为面片）。这些面片是由 16 个排列在一个 4×4 的格网里面的控制点来表示的。这些控制点构成了一个凸包，凸包里面就是真正的表面。只有四个角上的控制点是在表面上的。

一个表面法线（以后简称为法线）是一个单位矢量，其原点位于表面上，方向与表面垂直。此矢量有很多用处，其中最常用的是光照和碰撞检测。

一个在表面上的点是由两个参数座标所唯一确定的，它们通常被表示为 u 和 v，而且其有效范围在[0.0, 1.0]之内。不要把这个坐标和纹理坐标混淆了（虽然可能实际上一者是直接从另一者拷贝而来的）。如果我们将面片看成是一个 4×4 的控制点构成的格网，那么 u 值代表了格网上水平方向的坐标，而 v 值代表了垂直方向上的坐标。一个(u, v)点在真实世界中的坐标是完全由控制点决定的。实际上，如果没有均匀分布在世界空间上的话（这种情况的可能性比均匀分布的可能性要更大），那么在 u 方向上的连续分段值在世界中就会产生不同长度的分段值。

本节不讨论如何高效地进行分块和面片的渲染的问题，已经有很多出色的文章详细阐述了不同的画面片的方法（请参看[Farin96]、[Foley96]，以及[Gallier00]）。要单独讨论它们中的任何一个在此处都是不合适的，因为生成法线的方法对于大部分，即是不是全部的话，渲染面片的方法都适用。

4.3.2　传统方法

生成一个面片顶点法线最明显的方法涉及到要对其相邻点进行检查。

Download at www.Pin5i.Com

通过使用此面片在 u 和 v 方向上前向和后继的相邻顶点以后，你就可以得到近似的表面切线了。而法线只不过是这两个矢量的叉乘结果而已。

另一个方法是要使用面片方程的一次微分去直接生成两条切线矢量。然而，虽然这不需要任何相邻点，但是它仍然需要进行一次叉乘运算并对每个顶点进行一次归一化操作。

4.3.3 相关问题

为了要在现代的图形处理硬件上高效地处理面片，你必须使用所谓的“顶点着色器（shader）”。这基本上就是一个定制的程序，在程序中将对顶点流中的每个顶点调用着色器处理一次。它使用了一套最小化但却强大的指令集来编写，为了得到最大的并行化和吞吐量，一个顶点着色器被系统严格地限制为一次只能处理一个顶点。它除了当前正在处理的顶点以外不能访问任何其他的顶点。换句话来说，流中的每个顶点都应该包含了处理此点所需要的所有信息。另外我们还希望能把每个点的信息压至最小，这样就可以减少图形卡真正访问此数据所需要的时间，而且我们还希望将处理此点的时间最大化。显然，我们的目标是要在顶点流中让每个顶点只包含对应面片上的(u, v)坐标信息。由于 u 和 v 的坐标范围是[0.0, 1.0]，而且一般来说我们不会使用超过 100 个分段，因此就可以对之使用字节编码，这样每个顶点就需要在顶点流中使用两个字节了。

4.3.4 一个更简单的方法

上述两个方法假设我们只知道每个控制点的位置，在这些点上我们可以生成曲线方程和最终的法线。如果我们在开始的时候就知道每个控制点的法线，那会有什么结果呢？

有很多方法可以用来在每个控制点轻易地生成这些法线，包括前面我们提到的两个方法。这样一个控制点就有一个坐标和一条法线了。任何蒙皮（skinning）的代码，如果它能处理控制点的位置的话，那它也能处理其法线了。在知道了这一点以后，剩下我们要做的就是要在面片上的法线之间进行插值，正如我们对点的坐标做的处理一样。它还会带来一个非线性插值的好处，这样就可以更逼近真实的法线了。

这样我们所有的硬件要求就都被满足了。它只需要当前顶点的(u, v)面片坐标就可以正确地生成它的位置、法线和纹理坐标。这还有一个好处，那就是对于任何一个基系统都可以工作，而且可以使用完全一样的代码来生成位置，不过要使用控制法线来取代控制点而已。

4.3.5 其他的优点

此方法的另一个优点是通过使用不同的方法来生成法线之后，你就可以从曲面的连续性问题中消除那些着色错误了。如果我们将整个对象的控制网格看成是一个单一的连续的网格，那么我们就可以为每个控制点创建一条顶点法线，这与 Gouzaud 着色器横型中处理的方式是一致的。虽然对于曲面来说此法线并非绝对正确的，但是此方法将使模型看起来更加平滑，而同样的效果对于一般的建模过程来说则需要花费长得多的时间才能办到。如果法线是通过此方法生成的话，那么我们就可以在模型里解决连续性的问题了。此方法也可以消除任何蒙

Download at www.Pin5i.Com

皮过程中可能带来的着色问题，这是因为在蒙皮过程中，曲面会因为各自骨架的不同导致失去一致的连续性。

大部分现代的图形处理硬件架构都使用了一个基于矢量的处理器，它能够同时对四个数据元素进行操作。此方法使用了两个矢量，但是每个矢量只有三个元素。我们可以充分利用这一点，在面片上插入两个额外的值。这么做了以后我们就可以进行各种不同的处理了，例如任意的纹理映射或是为了得到更佳效果而使用不同的 alpha 值。

4.3.6　此方法的精确度有多大

一个贝塞尔曲线的控制点形成了一个边界网格，此曲线将整个限制在这个网格里面。这就意味着插值出来的法线的模绝对不会大于控制网格的法线模。然而，除非这段曲线退化为了一段直线，一个贝塞尔曲线绝对不会穿过中央的两个控制点。换句话来说，插值出来的法线的模总是小于或是等于控制法线的模。其结果是一个在线性和正确的弧线插值之间的值。在实际环境里，整个面片上面的法线不会差异太大，这意味着此插值与正确的值非常接近。

对于基于其他曲面的系统来说，此问题可能会变得更严重一些，法线会在表面上突变的锐角处变得很大。然而，由于现代的图形卡能自动地对法线进行归一化的处理，这个问题的严重性就小得多了。

4.3.7　结论

表面的面片对于通过使用极少的内存来创建一个平滑的、独立于分辨率的几何模型来说是很有用的。将每个控制点处的法线看成是第二个控制网格以后，我们就能快速地对正确的表面法线进行逼近了。虽然严格说来此结果并不完全正确，但是它们可以消除由于蒙皮引进的曲面不连续性导致的着色误差，从而得到相当好的结果。

4.3.8　参考文献

[Farin96] Farin, Gerald E., *Curves and Surfaces for Computer Aided Geometric Design: A Practical Guide*, Fourth Edition, Academic Press, 1996.

[Foley96] Foley, Van Dam, et al., *Computer Graphics: Principles and Practice, Second Edition in C*, Addison Wesley, 1996.

[Gallier00] Gallier, Jean, *Curves and Surfaces in Geometric Modeling: Theory and Algorithms*, Morgan Kaufmann Publishers, 2000.

Download at www.Pin5i.Com

4.4 快速、简单的遮蔽剪裁

Wagner T. Corrêa，Princeton University
James T. Klosowski，IBM Research
Cláudio T. Silva，AT&T Labs-Research
wtcorrea@cs.princeton.edu
jklosow@us.ibm.com
csilva@research.att.com

在很多图形应用程序里面，例如在建筑物里面行走，或是在第一人称游戏里，玩家将在一个虚拟的环境里面走动，计算机为玩家在各个位置上都创建出一个视角。对于任何一个给定的位置，一般情况下，玩家只能看到整个场景的一部分。因此，为了加快图形渲染的速度，应用程序应该避免对那些玩家看不到的对象进行渲染。一般来说，我们可以将算法分为几类，每类算法将对不同的对象进行裁剪。背景裁剪（backface culling）裁剪的对象是那些背对用户的物体。视界截面裁剪（view frustum culling）裁剪的是那些不在玩家视野范围以内的对象。遮蔽裁剪（occlucion culling）裁剪的是被其他对象挡住的对象。

虽然背景裁剪以及视界截面裁剪算法都比较简单，但是遮蔽裁剪算法非常复杂，而且经常需要花费大量时间进行预处理了。在本文中，我们介绍了两种遮蔽裁剪的算法，它们实践性强、效率较高，而且几乎不需要进行预处理。第一种算法是优先分层投影法（PLP， prioritized-layered projection），在一个给定的资源限制下，它能进行近似计算，得到一组可能可见的对象；第二种算法是 cPLP，它是 PLP 算法的稳健（conservative）版本，此算法能确保找出所有的可见对象。

4.4.1 可见性问题

在一个由各个对象模型以及一个视界截面构成的给定场景里面，我们需要进行判别以得知哪些对象的哪些部分是可见的，换句话说，我们必须找出所有的视线，这些视线连接的就是这些对象相应的部分，而且这些视线没有与其他对象相交[Dobkin97]。研究人员在这个问题上已经进行很多工作，且得到了很多解决此问题的方法[Cohen-Or01，Durand99]。在他们对可见性问题进行的研究中，[Cohen-Or01]将算法根据几条准则分成了不同的类别。我们将对这些与本文相关的类别作一个大致的描述。

Download at www.Pin5i.Com

1. 从视点出发与从区域出发

某些算法仅仅是从视点出发进行计算的，而其他算法则是从空间中的区域出发进行计算的。由于玩家总会在一个区域里面停留一段时间的，因此从区域着手进行处理的话，在一段多帧处理中我们就可以节省很多可见性的计算了。

2. 预处理与动态计算

很多算法都要求进行一些预处理计算，而其他算法则可以进行实时计算。例如，很多基于区域的算法都需要进行预处理，将场景划分为各个区域，然后计算区域可见性。

3. 对象空间与图像空间

某些算法计算可见性是在对象空间进行的，使用的是原始的、准确的 3D 建模对象。其他算法则使用图像空间进行计算，操作的是对象中离散的、光栅化了的各个部分。

4. 稳健算法与近似算法

几乎没有几个算法计算出来的是精确的可见性。大部分算法是稳健算法，其计算出来的可见对象一般都会偏多。其他的算法计算的则是近似的可见性，而且不能保证找出所有的可见对象。

4.4.2　PLP 算法

PLP [Klosowski00]是一个近似的、基于视点的、对象空间可见性算法，它只需要进行极少的预处理。我们可以把 PLP 看成是传统层次式视界截面（view-frustum）裁减算法[Clark76]的一个简单变体。在传统的算法中，它将对建模的层次化结构进行递归遍历处理，从根节点开始，一直到最后的叶节点。如果一个节点在视界截面之外，我们就将这个节点及其所有的子节点忽略掉。如果此节点在视界截面之内或是与之相交的话，我们就会递归遍历处理其子节点。此遍历最终将访问到视界截面里面的每个节点。

PLP 算法与上述的传统算法有几个不同之处。首先，PLP 不是使用一种预先定好的顺序对建模后的层状结构进行遍历的，而是将叶节点的结构保存在了一个称之为 front 的优先级队列中，然后在队列中按照优先级从高到低进行遍历。当我们访问一个节点（或是使用 PLP 的说法，对之进行投影的时候），如果它是可见的，那么我们就将之放入可见对象集合里面去。接着，我们会将之从 front 中删除掉，将与其相邻的未经访问过的节点加入到 front 中去（这就是本算法名称的由来，即优先分层投影法（prioritized-layered projection））。其次，PLP 不会对整个结构进行遍历，而是根据一个预算计算，当可见对象集合里面已经有一定数量的对象以后，此算法就将停止。最后一点，PLP 要求每个节点不仅知道其子节点，还需要知道其相邻节点。

PLP 的实现可简可繁，这要视我们如何给每个节点定义其优先级而定。某些方法需要事先计算出一个节点的“固态性（solidity）”，然后在遍历的时候还将对之进行累加。通过一个节点最终的“固态性”，我们就可以得知一个节点有多少可能性挡住其后的对象

Download at www.Pin5i.Com

[Klosowski00]。在本文中，我们将使用一个极其简单的启发式方法给每个节点赋予优先级。包含视点的节点优先级为-1，它的相邻节点优先级为-2，上述节点的相邻节点优先级为-3，依此类推。使用了这种方法之后，遍历过程就将从围绕着视点的节点开始展开。这种方法易于实现、非常快速，而且相当精确。我们展示其运行时结果的时候，也将同时展示其精确度的测量。此方法唯一需要的预处理就是要首先把这个层状结构建构出来。

我们将 PLP 作为硬件中 Z 缓冲算法的一个前端[Foley90]。对于一个给定的预算而言，PLP 将给出一组对象，这些对象在它看来是能得到最佳图像效果的。接下来，我们只需要将这些对象传递给图形硬件就可以了。读者可以在本书的 CD-ROM 上找到 PLP 的 C++实现。

4.4.3 cPLP 算法

虽然在实际环境中 PLP 对于大多数帧都是非常精确的，但是它并不能确保图像的高质量，因此在某些帧中可能会显示出本应被挡住的物体。为了避免产生这种问题，我们可以使用 cPLP [Klosowski01]，它是 PLP 算法的稳健版本。

cPLP 的主要想法就是利用 PLP 生成的可见对象集合作为初始数据，然后不停地对之添加数据，直到 front（即那个按优先级排列的队列）为空为止。这样，我们就可以确保其最后的可见对象集合是稳健的了[Klosowski01]。现在有很多实现 cPLP 的方法，其中包括了使用最新的与平台有关的硬件扩展进行可见性的计算。在本文中，我们将要阐述的实现方法使用了物件缓冲（itembuffezing）的技术，这个技术可以移植到任何一个支持 OpenGL 的系统上。

cPLP 的主循环由两个部分组成的。首先，我们需要找出在 front 中可见的各个节点。为了达到这个目的，我们为每个在 front 中的节点给出一个边界框，我们将使用与优先级对应的颜色对之进行平面着色。接下来，我们再把这些节点的颜色从缓冲区里面读出来，这样就知道哪些节点是可见的了。然后，为了要使得每一个找出来的节点都是可见的，我们将对之进行投影（有可能将其加入到可见对象集合中去），将其从队列的前面删除掉，然后将没有被访问过的相邻节点加入到 front 中去。我们将在主循环中不断迭代，直到 front 最终为空。这种使用物件缓冲技术实现 cPLP 的方法有个瓶颈，它出在把节点的颜色从缓冲区里面再读出来这一步上。为了避免在每一步都读出整个颜色缓冲区，我们把屏幕划分为一个个分片。在某一步中如果一个分片没有被修改，那么在接下来的几步里面我们就将忽略对其的处理。读者可以在本书的 CD-ROM 上找到 cPLP 的 C++实现。

4.4.4 讨论

PLP 与 cPLP 都是很有用的可见性判定算法，其理由如下：

- PLP 与 cPLP 是基于视点的算法，这就意味着它们不依赖于任何模型。与之形成对比的是，某些基于区域的算法要求模型由轴向对齐的房间和门户构成[Teller91，

Download at www.Pin5i.Com

Funkhouser93]，这往往会带来极大的限制。

- PLP 与 cPLP 几乎都不需要进行预处理。对于大部分启发式方法来讲，其预处理都由创建模型层次结构以及针对每个节点计算简单的统计数据构成，例如要计算出对象的总数量。即使是针对较大的模型，也可以很快完成。在另一方面来说，其他的技术[Teller91, Hong97, Zhang97]需要进行数小时甚至数天的预处理时间，即使是针对较小的模型而言。
- 虽然例如 PLP 这样的遮蔽裁减能避免渲染不可见的物体，但是它们仍然有可能渲染一些对最终图像影响极小的小个对象。正如[El-Sana01]所示，我们可以很方便地将 PLP 与分层细节的管理集成起来的。
- PLP 对于时间要求较高的渲染来说是比较适合的。即使我们使用的是最低级别的细节，在一帧里面需要渲染的可见对象仍然有可能超出一个低端图形卡的处理能力。通过使用 PLP 预算的概念，用户就可以很方便地在精确度和速度之间进行权衡了。在玩家行走的过程中，即使中间间或出现了一些不那么完全正确的图像，总也比游戏的帧率下降要好得多。

当我们需要高帧率更甚于绝对精确性的时候，PLP 就该登上舞台了——例如，当玩家快速移动，逼近某个目标的时候。从另一方面来说，如果游戏是不能出现错误的，那么 cPLP 就是更佳的选择了，例如当玩家达到了目标，仔细检查周围环境的时候。理想情况下，在游戏中，玩家应该能动态地在 PLP 与 cPLP 之间进行切换。

4.4.5　实验结果

为了展示一下 PLP 与 cPLP 究竟可以做什么，我们在一台奔腾 III，733-MHz 的机器上使用了 Nvidia GeForce2 图形卡针对一千三百万三角形的 UNC power-plant 模型进行了测试。我们针对一段共 500 帧长的片段对 PLP 和 cPLP 进行了统计，收集了统计数据。图 4.4.1 显示了在此片断中，使用了 140 000 三角形每帧的预算时的典型场景。

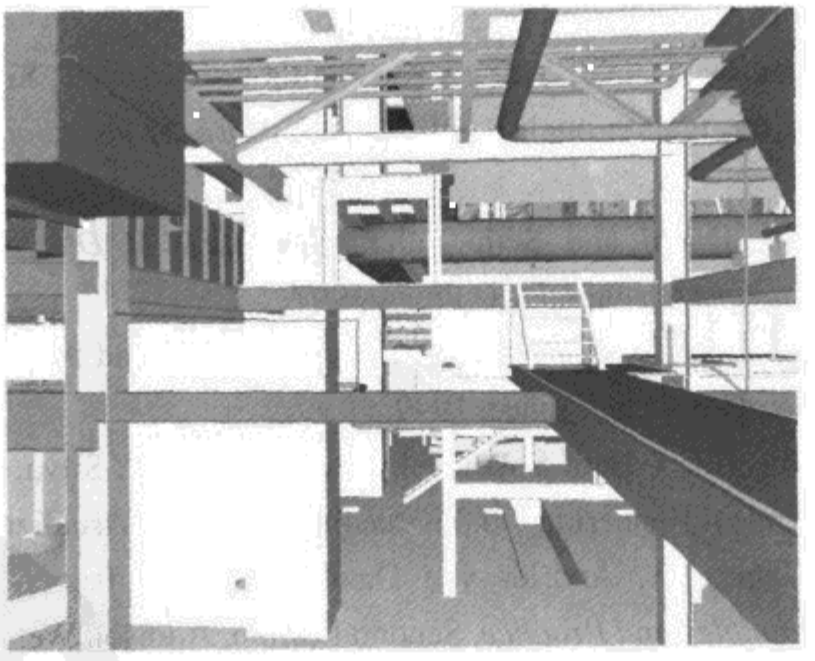


图 4.4.1　使用 PLP 在一个有 1300 万个三角形的 UNC power plant 模型中行走。配置为奔腾 III，755MHz，有一块 Nvidia GeForcez 图形卡。PLP 平均帧率 10.1Hz，平均准确度 96.3%

Download at www.Pin5i.Com

对于 PLP 来说，其平均帧率为 10.1Hz，在 75%的测试中，帧率在 9.3Hz 以上。对于 cPLP 来说，其平均帧率为 2.1Hz，在 75%的测试中，帧率在 1.5Hz 以上。虽然 cPLP 的帧率比 PLP 的帧率要低，但是其图像却可以保证是 100%正确的。我们对 PLP 的精确度进行了测量，方法就是对比生成的正确图像，计算图像中生成的错误像素数量。PLP 平均的正确率是 96.3%，在 75%的测试中，正确率都达到了 94.9%。

由于采用了分层模型的层次化遍历，因此错误像素离视点比较远。有时候错误是可以察觉到的，但是它们一般都可以容忍，对用户玩游戏不会造成什么影响。请再回想一下，我们使用的是极其简单的启发式方法，每次在层次化结构中只遍历一层就达到了这样的效果。我们相信，如果使用了更为复杂的启发式方法，其精确度还将更高。

4.4.6 结论

对于一般的可见性问题而言，PLP 与 cPLP 都是很实际的解决方法。PLP 可以让用户用精确度换时间。对于 PLP 来说，我们是不能保证图像质量的，但是在实际情况中，玩家都感觉不到缺陷，可以进行平滑的游览。如果我们真的需要 100%精确度的时候，我们可以将程序转入 cPLP 状态，这样我们仍然能够在较低的帧率下在模型中行走。

读者可以使用多种方法对本文中阐述的技术进行改良。首先，在计算节点的可见度这一点上，我们只给出了一种简单的方法。读者可以使用更为复杂的方法[El-Sana01]，这样就可以对之进行改善了。其二，这些算法还可以与层次化的细节管理（level-of-detail management）结合起来[El-Sana01]。最后一点，这些算法可以使用在缓冲架构之上，这是专门为那些在内存里面放置不下的模型准备的。

4.4.7 参考文献

[Clark76] Clark, James H., "Hierarchical Geometric Models for Visible Surface Algorithms," *Communications of the ACM*, 19(10): 547～554, October 1976.

[Cohen-Or01] Cohen-Or, Daniel, Yiorgos Chrysanthou, Cláudio T. Silva, and Frédo Durand, "A Survey of Visibility for Walkthrough Applications," *IEEE Transactions on Visualization and Computer Graphics*.

[Dobkin97] Dobkin, David and Seth Teller, *Handbook of Discrete and Computational Geometry*, Computer Graphics Chapter, CRC Press, 1997.

[Durand99] Durand, Frédo, "3D Visibility: Analytical Study and Applications," Ph.D. Thesis, Université Joseph Fourier, Grenoble, France, 1999.

[El-Sana01] El-Sana, Jihad, Neta Sokolovsky, and Cláudio T. Silva, "Integrating Occlusion Culling with View-Dependent Rendering," in Proceedings of IEEE Visualization, 2001.

[Foley90] Foley, James D., Andries van Dam, Steven K. Feiner, and John F. Hughes, *Computer Graphics: Principles and Practice*, Second Edition, Addison Wesley, 1990.

[Funkhouser93] Funkhouser, Thomas A. and Carlo H. Séquin, "Adaptive Display Algorithm for Interactive Frame Rates during Visualization of Complex Virtual Environments," Computer

Download at www.Pin5i.Com

Graphics Proceedings, SIGGRAPH 1993: pp. 247～254.

[Funkhouser96] Funkhouser, Thomas A., "Database Management for Interactive Display of Large Architectural Models," Proceedings of Graphics Interface '96: pp. 1～8.

[Hong97] Hong, Lichuan, et al., "Virtual Voyage: Interactive Navigation in the Human Colon," Computer Graphics Proceedings, SIGGRAPH 1997: pp. 27～34.

[Klosowski00] Klosowski, James T. and Cláudio T. Silva, "The Prioritized-Layered Projection Algorithm for Visible Set Estimation," in *IEEE Transactions on Visualization and Computer Graphics*, April-June 2000, 6(2):108～123.

[Klosowski01] Klosowski, James T. and Cláudio T. Silva, "Efficient Conservative Visibility Culling Using the Prioritized-Layered Projection Algorithm," in *IEEE Transactions on Visualization and Computer Graphics*, October-December 2001, 7(4):365～379.

[Teller91] Teller, Seth and Carlo H. Séquin, "Visibility Preprocessing for Interactive Walkthroughs," Computer Graphics Proceedings, SIGGRAPH 1991: pp. 61～69.

[Walkthru01] The Walkthru Project at UNC Chapel Hill, "Power Plant Model," 对应网址为 http://www.cs.unc.edu/~geom/Powerplant/.

[Zhang97] Zhang, Hansong, Dinesh Manocha, Thomas Hudson, and Kenneth E. Hoff III, "Visibility Culling Using Hierarchical Occlusion Maps," Computer Graphics Proceedings, SIGGRAPH 1997: pp. 77～88.

Download at www.Pin5i.Com

4.5 三角形条带的创建、优化以及渲染

Carl S. Marshall
英特尔实验室
Carl.S.Marshall@intel.com

在当前高性能的平台上，在需要表现和渲染几何模型的时候，三角形条带将成为首选对象。本节主要关注如何从任意的 3D 多边形模型生成三角形条带。我们将描述和提供开发长条三角形条带的源代码。在描述了三角形条带的算法以后，我们将解释一下三角形条带的优点、在创建时可能遇到的容易犯的错误，以及应该如何利用图形处理的 API 对之进行处理。此外，我们还将看看其他几个三角形条带的生成方法，并对之进行评价。

4.5.1 三角形条带

一个三角形条带（tri-strip）是一系列相互连接的三角形。这些三角形之间的连接使得顶点可以被缓存起来，这样就可以让图形卡能重用三角形之间的共享边了。图 4.5.1 展示了一个简单的三角形条带，其共享边为 V_2V_3 以及 V_3V_4。为了让一个三角形成为一个三角形条带中的一员，此三角形必须含有与此三角形条带中其他三角形一样的平滑组（smooth group）和材质组（material group）。平滑组指的是一组三角形，其中每个三角形的每个顶点都有一条法线；材质组指的是一组具有相同光照和纹理的三角形。

图 4.5.1 （A）一个简单的三角形条带，共享边为 V_2V_3 与 V_3V_4（B）三角形条带中三角形顶点的顺序在顺时钟方向与逆时钟方向中切换

Download at www.Pin5i.Com

1. 背景知识

使用三角形条带的技术由来已久。在此之前，将三角形传至 API 的一般方法就是显式地将每个顶点坐标、法线以及颜色直接传进去。但是由于三角形条带只需传送较少的顶点，这就使它们相对于一个纯三角形的图形处理 API 调用来说要优良得多了。在今天，顶点索引技术已经实际上取代了早期的处理过程，成为了当今向图形卡提交三角形的主流方法了。[Marselas00]中提到通过使用三角形条带与顶点索引技术，对于特定的网格来说，已经有可能把顶点：三角形的比值提升到 1：1 了。这就使三角形条带技术在降低数据量方面更有优势了。

2. 目标

创建三角形条带有 4 个目标：

（1）将三角形条带的数目最小化。

（2）将所需的重复顶点数最小化。

（3）将孤立的三角形数量最小化。

（4）将顶点缓存最大化。

这些目标经常会互相矛盾。例如，如果没有大量重复的顶点，一般来说是很难生成极长的三角形条带或是对相关的三角形条带进行缓存的。而且，我们宁可保留几个孤立的三角形（三个顶点的三角形条带），也不要得到几个四个顶点的三角形条带。这是因为你可以将这些孤立的三角形成批加入进索引后的顶点列表里面去。

3. 优点

使用三角形条带而不是独立的三角形会减少需要提交的顶点数或是顶点索引数。视三角形条带提交至图形处理硬件的方式而定，你可以在顶点数据、变换以及光照等方面得到极大的节省。使用三角形条带也带来了可以在图形卡上进行顶点缓存的优点。

4.5.2 三角形条带的创建

已经有好几种创建三角形条带的算法，但是每个算法都有其优缺点，这是因为要找出最优化的三角形条带是一个 NP 完全的问题。[Evan96a]使用了一个基于方块的网格在补丁中优化三角形条带；[Hoppe99]使用了一个倾向于缓存的三角形条带生成法。我们使用的方法是要对三角形条带的长度进行优化以减少图形处理 API 的调用开销。

由于不可能对任何情况下的网格都生成最优的三角形条带，我们的算法不得不针对特定的实现找出最优的三角形条带。我们使用的算法对于任何 3D 多边形模型来说都能极好地生成三角形条带。你可以将此三角形条带生成算法放到任何你最喜欢的 3D 编辑工具里面去，或者你可以将之作为一个单独的程序来运行。我们的目标就是要生成较长的三角形条带，然后将之写进一个易于渲染的格式里面去。彩图 5 展示了一个由两个模型产生的示例图像，它是在网格上运行了三角形条带算法以后生成的。

Download at www.Pin5i.Com

1. 定义

在我们讨论创建算法之前，最好先定义出几个术语。活动边指的是在条带中的三角形中可以添加新三角形的边。活动边是要加入三角形条带最后那个三角形的第二个和第三个顶点之间的边。图 4.5.2a 中的黑体边 DE 就是一条活动边，在此处可以加入一个拥有顶点 F 的新的三角形。图 4.5.2b 中显示了这么一种情况，其中 DE 仍然是活动边，但是下一个要添加的三角形却处在 CE 边上。为了添加上这个新的三角形，我们必须拷贝一个顶点 C，然后将之与顶点 E 交换。这样就保持了正常的时针方向，即逆时针方向。最后一个我们需要定义的术语就是翻转（flip），它指的是需要添加两个重复的顶点然后进行交换以添加一个新的顶点，如图 4.5.2c 所示。

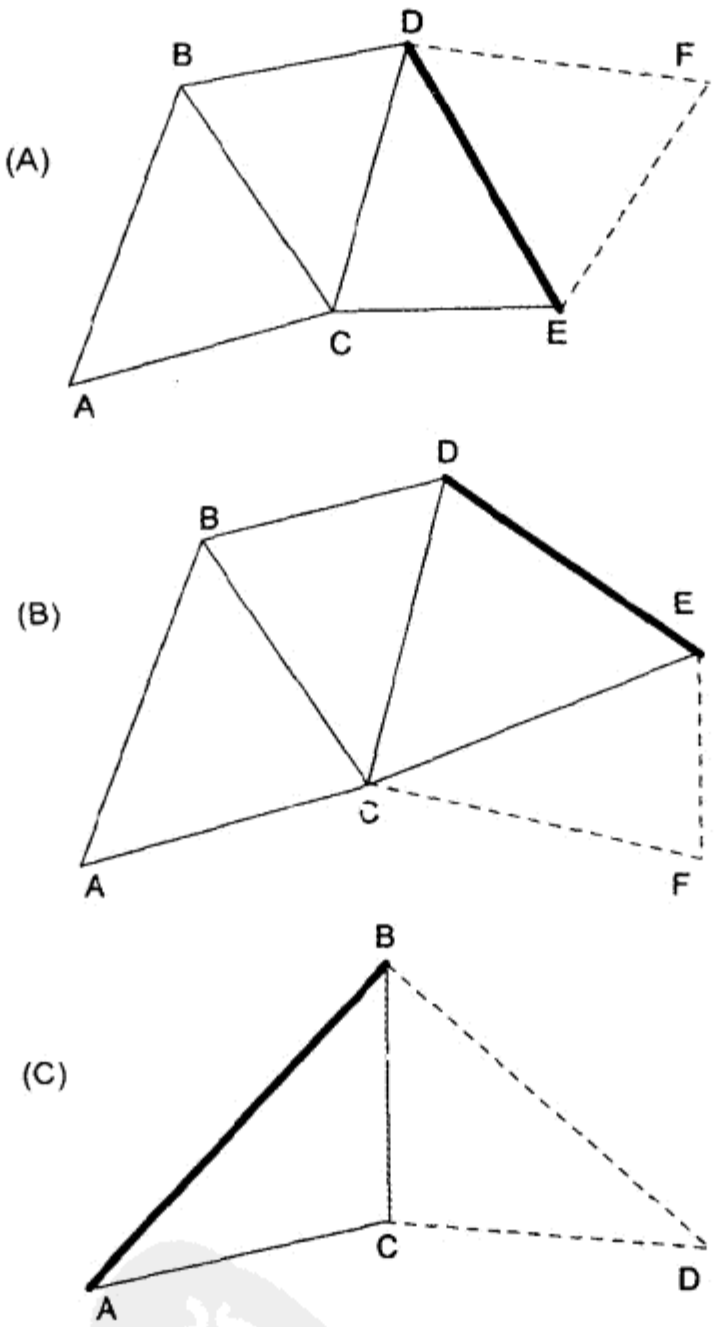


图 4.5.2 一个三角形添加到三角形条带中的不同方法

（A）三角形条带顶点顺序为 ABCDE，活动边为 DE，由于活动边与新三角形相邻，F 就可以加到条带尾部，得到 ABCDEF 了。（B）三角形条带 ABCDE 的活动边为 DE 为了将 F 包含进来，必须要复制 C，与 E 交换，这是因为新的三角形不与 DE 相邻。新条带为 ABCDCEF。（C）此时为了减少重复的顶点，必须将条带重新排序。如条带以 CAB 开始，则需要复制 B 与 C 以包含顶点 D。新条带为 CABBCD。如果活动边与第二个三角形不同侧则会产生复制

Download at www.Pin5i.Com

2. 预处理

预处理阶段用来创建生成高质量三角形的矩阵。

（1）找出具有最小面积的三角形，我们将之称为原点三角形。为了避免生成低质量的三角形条带，你可能需要得到网格中 10 个具有最小面积的三角形，然后在其中进行挑选得到原点三角形。

（2）从原点三角形中选出一个顶点，或是得到此三角形的质心，这将是三角形条带算法的起始点。

（3）为每个三角形创建一个质心。这仅需要把每个三角形的三个顶点的坐标平均就可以了（参见方程 4.5.1）。C 是质心的坐标，而 V1、V2 和 V3 是三角形顶点的坐标。

$$C=(V_1+V_2+V_3)/3.0 \qquad (4.5.1)$$

（4）对于每个三角形，计算出并存储下其质心到起始点的欧氏距离。

3. 创建算法

一旦我们结束了预处理过程，就可以生成三角形条带了。每当一个网格中的三角形加入到三角形条带以后，我们就将此三角形标志为无效。

（1）选择一个有效的三角形。此三角形将是三角形条带中的第一个三角形。

（2）如果此三角形有相邻的三角形，则选出那个距离最近的三角形，然后将之标志为当前三角形。否则，结束此三角形条带，跳到第 6 步。

（3）调整第一个三角形的方向，使其活动边与第二个三角形匹配。第二个和第三个顶点间的边应该匹配其相邻的三角形。

（4）找出最近距离的三角形作为当前三角形的相邻三角形，此距离值是在预处理阶段的第 4 步存储起来的。如果此三角形与活动边没有对齐，则需要拷贝一个顶点进行交换以继续创建三角形条带（参见图 4.5.2c）。

（5）跳到第 2 步。

（6）检查是否还有不在三角形条带中的剩余三角形。如果是，则跳到第 1 步。

对应的运行此三角形条带算法的高层伪代码如代码列表 4.5.1 所示。

LISTING 4.5.1　运行三角形条带算法的高层伪代码

```
void main( )
{
    Mesh *pMesh;

    LoadMesh(pMesh); // Load 3D polygonal mesh
    originTriangle = FindSmallestAreaTriangle();
    CalculateCentroidForEachTriangle();

    // Get Euclidean distance from centroid of
    // the origin triangle to the current
    // triangle origin
    CalculateDistanceFromToEachTriangle(
    OriginTriangle);
```

Download at www.Pin5i.Com

```
    // Generate the triangle strips
    TriStripGeneration(pMesh);

    // Run a second pass filter to see if any of
    // the previous triangle strips can be connected
    ConnectTriangleStrips();
    ConvertTriStrips(); //Use custom data structure
}
```

一旦创建算法结束了以后，我们就将输出 *N* 个三角形条带。每个三角形条带都将属于同一个平滑组，而且拥有相同的材质 ID。一旦三角形条带存储在内存以后，你就可以将其转化为任何你游戏所需要的输入、输出格式了。

4. 连接三角形条带

除此之外，还可以对三角形条带进行其他的处理，我们可以对之进行分析看看是否存在那些可以将这些三角形条带连接起来的点。在分析的开始，我们首先将找出每个三角形条带的起始边和结束边，以及对应于每条边的表面的索引值。接下来，我们将对这些起始边和结束边进行检查看看是否其中有相匹配的边。一旦找到了一条匹配的边，我们将检查这两个三角形条带分别处在什么样的相对位置（参见图 4.5.2）。在大多数情况下，这都需要进行一些顶点拷贝以合并这些三角形条带。最简单的情况就是一个孤立的三角形与一个三角形条带相匹配。在大部分情况下，我们都可以对三角形的顶点进行重新排序以适应匹配的三角形条带。

4.5.3 优化

由于开发三角形条带的目的是为了提高性能，因此对于任何一个三角形条带生成算法而言，优化都是一个关键的因素。优化可以在三角形开发的数个阶段上进行：预处理、生成和运行时。

1. 预处理

- 创建只有几个平滑组的网格（对于每个顶点不止一条法线）。
- 限制每个模型中材质组的数量。
- 对模型进行优化，以使多边形的面不会在不同的材质组和平滑组之间来回交换。如此就可以对三角形条带的长度加以限制了。
- 在提交到图形卡之前，利用材质组对三角形条带进行分类。
- 消除所有无用的面。当一个三角形中有两个或是更多点的坐标一样的时候就形成了一个无用面。这将造成三角形条带的翻转，而且可能导致在渲染的时候被裁剪掉。

2. 生成

- 将孤立的三角形成批地加入一个单独的编好了索引的缓冲区里面去。
- 尽可能地将具有相邻起始边和结束边的三角形条带合并起来。
- 取得硬件的缓存大小，然后进行相应的优化。

Download at www.Pin5i.Com

3. 运行时

- 不要纯粹按顺序提交顶点，取代方法是使用一个能接受经过索引的数组形式顶点的 API 调用。
- 减少状态切换和动态纹理调整的变化次数。

4.5.4　渲染

大多数图形处理 API 都支持对三角形条带的渲染。当把三角形条带提交至图形卡的时候，你通常可以选择好几种格式来提交。一种格式要求把三角形条带里面每个顶点的数据都送进去。此种方法开销极大，这是因为由于三角形条带之间有共享的边，因此会产生很多重复的顶点。另一种格式要求使用索引格式来提交三角形条带。索引格式是指的是你一次提交一个顶点池，三角形条带的顶点就是经过索引后放入进顶点池中的。对于大部分图形处理 API 而言，最大的问题就是一次 API 调用只能提交一个三角形条带。在某些 API 里面，你可以通过提交无用的顶点来提交多个三角形条带。[Neider99]和[Microsoft00]中都列出了针对三角形条带提交的 API 调用。OpenGL 使用原类型 `GL_TRIANGLE_STRIP`，而微软的 Direct3D 使用 `D3DPT_TRIANGLESTRIP` 类型。

4.5.5　倾向于缓存的三角形条带

另一个三角形创建的算法通过将顶点缓存失误（vertext cache misses）最小化的方法来创建针对缓存使用的三角形条带[Hoppe99]。[Nvidia00]使用了顶点缓存的模式，此模式将作为对上述创建好的三角形条带的后处理过程来运行，这样可以对缓存的使用进行优化。此方法的优点在于你可以针对特定的图形卡进行三角形条带的优化。当然了，如果你在另一块具有不同缓存大小的图形卡上运行同样内容的话，则此方法将会给你带来一些麻烦。

4.5.6　连续分层细节的三角形条带

如果要想为一个具有连续分层细节的网格创建三角形条带的话，这就是一件麻烦事了。一旦从网格中移走了一个顶点或是面，就将对那些在更高分辨率下创建的三角形条带造成极大的影响，因为这会打断此条带，而且会造成一些失效的面。对于这个问题，我们有这么几个解决方法。第一就是为此模型的每个分辨率都创建一个三角形条带。这实在是不太实际，而且会使程序的内存占用量大大提高。第二个方法就是实时地创建三角形条带，然后将之存储在内存里面，直至分辨率发生变化。我们选择了第二种方法，而且只有当一个三角形与三角形条带的当前活动边相邻时，或是只需要进行一次交换时，我们才允许此三角形加入到条带里面来。如果没有相邻的三角形，则三角形条带的创建就结束了。此处的关键是要对数据结构进行优化以能对相邻的三角形进行的查找与遍历。它带来的优点是可以让我们在动态的地形里面使用三角形条带了。然而，这将在存储量上带来一些额外的开销。

Download at www.Pin5i.Com

4.5.7 结论

本节描述了如何为你的游戏创建和优化三角形条带。当你考虑为你几何图形渲染图元（primitive）时，三角形条带可以为你提供一个比使用简单的三角形列表更高的速度。我们鼓励你用三角形条带后测试一下你的几何图形，然后自己比较一下差别。

4.5.8 参考文献

[Evans96a] Evans, Francine, Steven Skiena, and Amitabh Varshney, "Optimizing Triangle Strips for Fast Rendering," Visualization '96 Proceedings, IEEE, 1996: pp.319～326.

[Evans96b] Evans, Francine, Steven Skiena, and Amitabh Varshney, "Completing Sequential Triangulations Is Hard," Technical Report, Department of Computer Science, State University of New York at Stony Brook, 1996.

[Hoppe99] Hoppe, Hughes, "Optimization of Mesh Locality for Transparent Vertex Caching," Computer Graphics Proceedings, SIGGRAPH 1999:pp.269～276.

[Isenburg00] Isenburg, Martin, "Triangle Strip Compression," *Graphics Interface*, pp.197～204, 2000.

[Marselas00] Marselas, Herb, "Optimizing Vertex Submission for OpenGL," *Game Programming Gems*, Charles River Media, Inc., 2000.

[Microsoft00] *Microsoft DirectX 8.0 Software Development Kit*, 对应网址为 http://www.msdn.microsoft.com/downloads, 2000.

[Neider99] Neider, Jackie, et al., *OpenGL Programming Guide, Version 1.2*, Addison Wesley, 1999.

[Nvidia00] Nvidia NvTriStrip v1.1., 对应网址为 http://developer.nvidia.com/view.asp?IO=nvstristrip_v1_1, 2000.

Download at www.Pin5i.Com

4.6　针对复杂数据集计算优化阴影体

Alex Vlachos，Drew Card
ATI Research
Alex@Vlachos.com
DCard@ati.com

随着图形处理硬件性能的提高，在游戏业界里，阴影体（Shadow Volume）逐渐变得热门了。在本节中，我们将描述一个方法，通过此方法，你可以计算出在给定静态光源的情况下得到的精确可见的前景几何形状。这是从光线角度上来看的一个精确形状，它对于计算体积阴影是有帮助的。很多研究者已经在这个问题上已经作了一些前期工作，但是大部分方法不是可能导致无限递归（随着复杂的多边形模型），就是无法对互相覆盖的多边形求解。此处提到的方法对于互相交叉的多边形来说也是适用的。

4.6.1　前期工作

Weiler-Atherton 算法[Weiler77]对于前景形状的计算提出了一个有趣的方法。此方法的优点在于它不需要针对光源排序的完好的排序列表。此外，Weiler-Atherton 算法对于互相覆盖的多边形也能正确地求解。不过，即使针对稍微复杂一点的场景，在使用 64 位浮点数变量时，此方法也会产生无限递归的错误。本节中的方法从 Weiler-Atherton 算法吸取了一些长处，但是它解决问题的角度是不同的。

4.6.2　算法

要计算出在给定静态光源的情况下精确可见的前景形状，其步骤是比较复杂的。首先要从所有面对光源的截面上的前端多边形开始，将它们进行从后至前的排序。在此处，只需要基于每个多边形最近的顶点进行快速排序就可以了。在此时，你需要给每个多边形赋予一个唯一的 ID 以备后用。有可能你还需要把这些多边形存储起来以便进行快速查找。我们称这些多边形为“输入数组”。我们还需要一个输出数组，它将被初始化为空。

现在我们就需要对输入数组中的每个多边形进行一组操作了。首先，需要从光源到多边形创建一条光束（一个小截面）。这条光束 4 个面中的 3 个都是由光源与多边形的一条边构成的。第 4 个面则是多边形本身的面。所有我们在输出数组中存储的几何图形，如果它落在了这个光束里面的话，我们就将把其从中删除掉。剩下的部分则将被放回到输出数组中（参见图 4.6.1）。

Download at www.Pin5i.Com

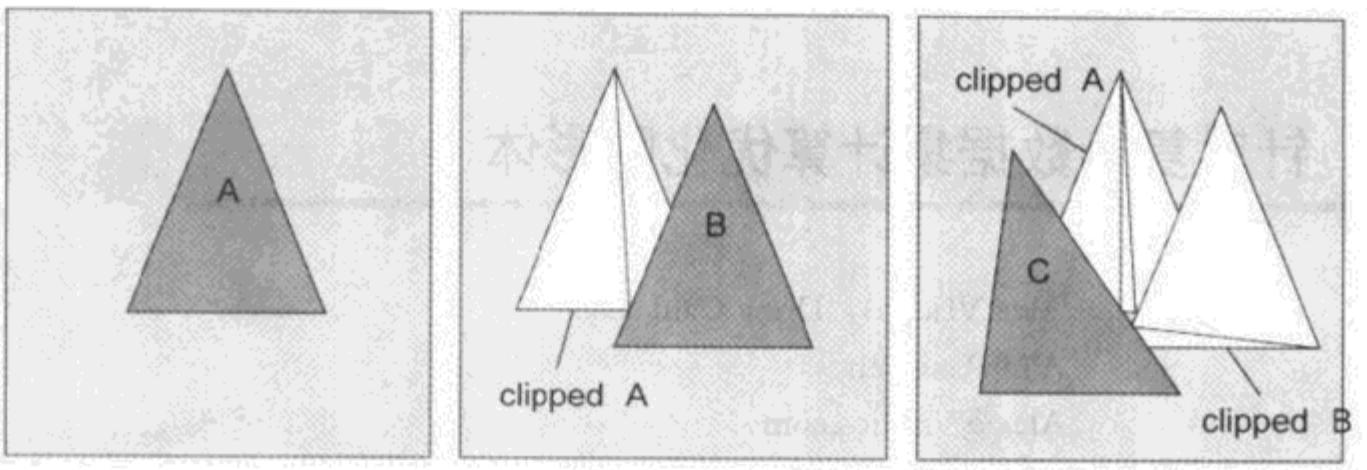


图 4.6.1 创建光束与输出多边形裁剪的说明。当前选择的是深灰色多边形，输出数组中有一个白色多边形。从光点（在此时为视点）创建出一条光束至选中的多边形。然后用光束对输出多边形进行裁剪

由于有时候不可能对多边形进行完全正确的排序（对于互相覆盖的多边形来说），因此在输出数组中将有一些多边形会挡住从光源射出到当前选中多边形的光线。为了对这种情况进行处理，我们对这些挡光的多边形将进行进一步的处理。我们将针对当前的输入多边形创建一个临时数组，然后将使用挡光多边形对此数组进行删减。在临时数组剩余的多边形将被复制到输出数组中去。如果没有挡光多边形，我们只需要把输入多边形复制到输出数组中就行了（参见图 4.6.2）。

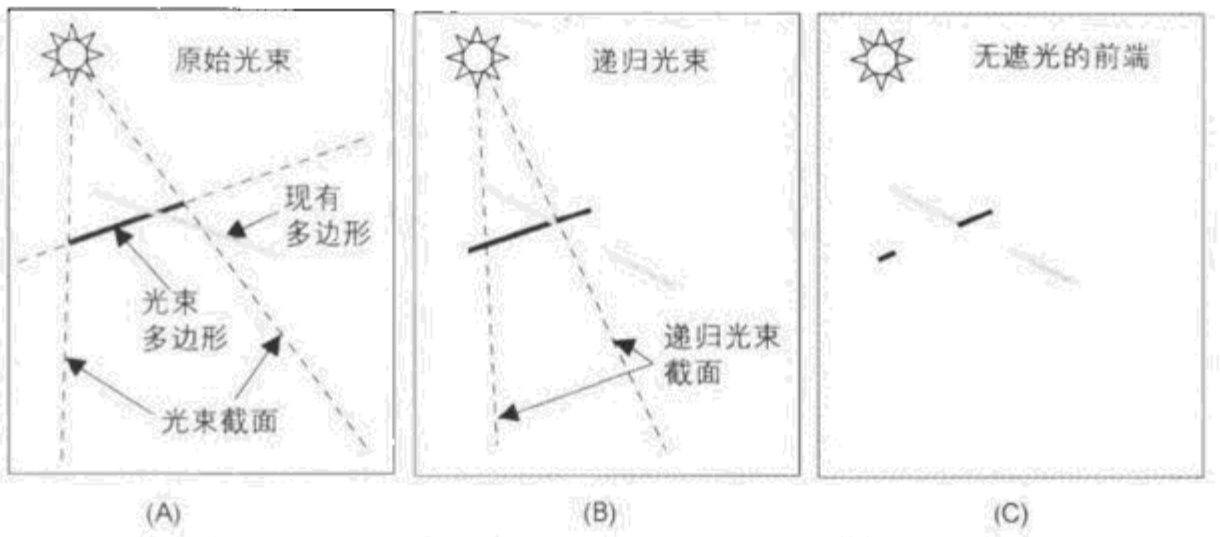


图 4.6.2 处理输出多边形遮光的情况。所示为正测图的顶视图。（A）光束为虚线，由（黑色）光束多边形包围（B）输出数组中的浅灰色多边形由原始光束截面从（A）进行裁剪。由于裁剪的多边形遮住了当前选中的（黑色）多边形，则进行调归创建一个有裁剪部分的光束截面，而原始的（黑色）多边形将根据它进行裁剪。（C）得到的是精确的前端几何图形

对每一个输入多边形进行了处理以后，我们还需要对输出多边形进行一番优化，以减少多边形分裂的数目。我们将在稍后介绍此算法。请注意，在每次主循环完毕了以后都必须进行这一步运算。

当把所有的输入多边形都处理完毕以后，还需要对最后输出数组中的多边形进行 T 连接消除以防止出现光栅错误。此时，我们已经得到了一个清晰的前景，接下来就可以很容易地生成出阴影体了。由于前景与视点光线出来的方向是一致的，因此我们可以复制最终前景的几何图形，将之投影到一个聚光灯截面的远端上以生成一个背景。我们可以将点光源分成 8 个象限以方便创建阴影体。

Download at www.Pin5i.com

4.6.3　优化算法

如前所述，我们需要使用一个优化算法以避免数值计算的误差并改善性能。在很多情况下，针对光柱截面进行不断地裁减会导致过多条形的多边形。为了避免这一点，由一个原始输入多边形生成的所有子多边形在循环中的每一遍都要分解成尽可能少的多边形。

在进行主循环之前，我们还可以对输入流进行优化，这样我们就可以避免在每次循环中对原始多边形进行优化了。对原始输入多边形进行成组的优化就可以做到我们要求的每一件事情。

我们可以使用两种算法来对多边形网格进行优化。我们将要在此处提及的这些算法提供了一种方法用来判定需要删除哪些顶点或是需要和合并哪些边。剩下的多边形需要使用一个稳定的镶嵌算法，例如[deBerg00]中的那种，来将之重新镶嵌成三角形。

1. 顶点删除

读者可以参见图 4.6.3 看看一个顶点是如何删除的。第一步要由所有对应原始的输入三角形（通过三角形 ID）创建出一个网格来。接下来，生成一个边表，里面包含了此网格中所有的边。对于网格中的每个顶点，选择出一个包含此顶点的三角形，使用边表，绕顶点进行顺时针旋转，走到相邻的依然包含了此顶点的三角形里面去。如此继续绕顶点行走，直到在此路径上再也没有三角形，或是走到了最初的三角形里面去。如果绕顶点的行走是成功的（就是说，最后完成了一个到起始三角形的循环），那么此顶点就可以被删除了。

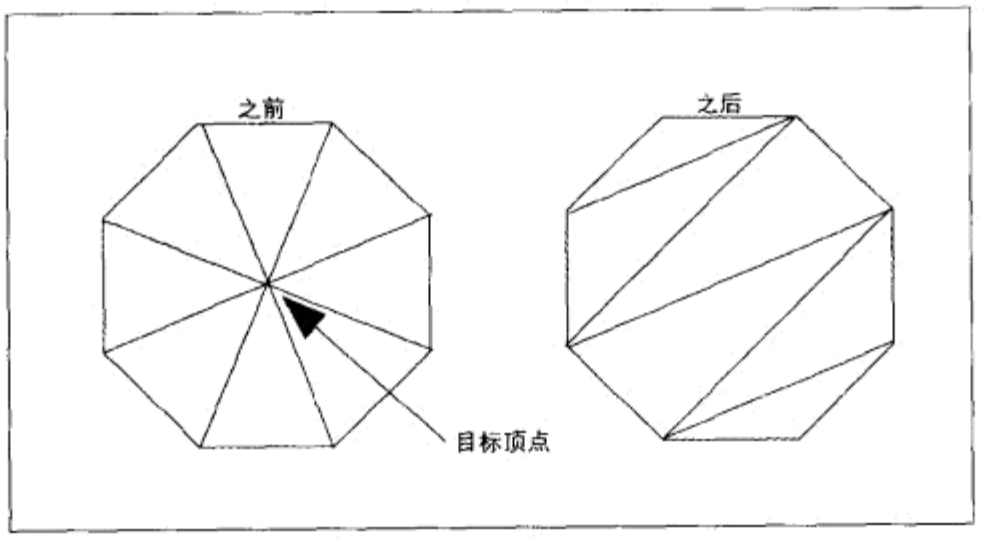


图 4.6.3　删除一个顶点

2. 边合并

读者可以参见图 4.6.3 看看一条边是如何被分裂的。与顶点删除算法类似，第一步要由所有对应原始的输入三角形创建出一个网格来。接下来，生成一个边表，里面包含了此网格中所有的边。对于网格中的每个顶点，选择出一个包含此顶点的三角形，使用边表，绕顶点进行顺时针旋转，走到相邻的依然包含了此顶点的三角形里面去。将当前三角形的起始边与最初的边进行同线的比较，使用一个圆周检测（或是将相关三角形的各个角加起来看看是否

Download at www.Pin5i.Com

等于 180°)。如果这两条边在一条直线上，那么就需要将共享的顶点删除，将两条相邻的边合并得到单一的一条边。

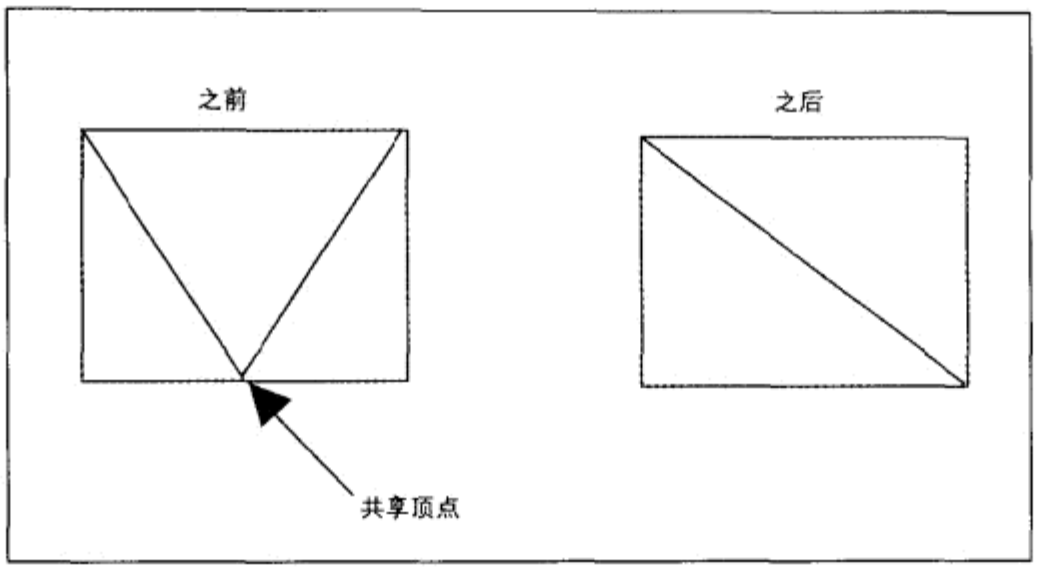


图 4.6.4 删除一个共享顶点合并一条边

需要注意的是，如果将输入三角形最初的边保存下来，我们可以减少边合并的计算时间。通过此方法我们可以通过边 ID 进行比较，而且可以消除由于浮点运算的不精确性而导致的可能误差。

读者可以参看彩图 6，在此处显示的是预先计算的阴影体以及其他渲染技术的混合运用。

4.6.4 参考文献

[deBerg00] de Berg, van Dreveld and Schwarzkopf Overmars, "Polygon Triangulation," *Computational Geometry Algorithms and Applications*, Second Edition, 2000: pp. 45～61.

[Weiler77] Weiler, K, and P. Antherton, "Hidden Surface Removal Using Polygon Area Sorting," *Computer Graphics*, SIGGRAPH, 1977: Vol.11, pp. 214～222.

Download at www.Pin5i.Com

4.7　针对人物运动的表面细分

William Leeson
Trinity College, Dublin
wleeson@indigo.ie

在最近几年中，随着高端渲染软件包的流行，通过细分（subdivision）建构物体曲面的方法也逐渐流行了起来。这在很大程度上是由于例如 Pixar 这样的公司创建的眩目的视觉效果造成的[DeRose98]。通过这些方法，我们可以解决很多其他与曲面相关的技术问题，例如 NURBS 曲面的问题。由于它们与多边形网格在处理方式上极为相似，因此它们的约束条件也相对较少。在表现人物皮肤的时候，我们甚至可以直接使用某些细分模式。对于其他种类的曲面来说，则需要对之进行一些修改以将其原始的网格很好地表现出来。

在本文中，我们介绍了一种曲面细分的方法，通过它，我们可以有效地改善游戏中人物的表现形式。首先，我们将提出几个可用的模式，主要关注于对曲面进行细分的两种实现方法。然后，我们将对一系列优化方法进行探讨，这些方法主要基于裁减与预处理。

4.7.1　各种细分模式

现在主要有两种曲面细分的类型[Kobbelt98]，名称分别是近似模式与插值模式。在本文中，与曲面细分相关的三种最重要的属性是：

- 效率；
- 仿射守恒性（即如果对控制点进行变换，则曲面也会相应得到变换）；
- 连续性（即要保持曲面的光滑性）。

细分模式使用了掩码来定义一组顶点以及相应的权重。对于每一种模式来说都有两种掩码：奇掩码与偶掩码。奇掩码用来生成新的顶点，而偶掩码则用来根据新的网格对旧的顶点进行调整。我们还可以对这两种掩码进行更细的分类，将之分为边缘掩码、折线掩码和法线掩码。此外，为了能表现出比较突出的特性，我们还需要创建特殊的折线掩码。对于三角形模式来说，具有 6 价（表示与之相连的顶点数目）的顶点被称为正常顶点，其他的则被称为特殊顶点。我们将对网格中的每个顶点使用掩码进行处理以得到一个新的网格。在多次使用掩码之后，网格将最终收敛为一个曲面（参见图 4.7.1 与图 4.7.2）。我们也可以使用同样的方法将这些掩码应用到纹理

Download at www.Pin5i.Com

坐标系里面去，这样一来我们就可以生成每个顶点的纹理坐标了。一般说来，近似模式会更快一些，因为它们受的约束更少，这是由于我们不必对之进行控制点的插值。不过，在处理边缘部分以及不连续的表面的时候，我们必须使用特殊的掩码（这些掩码将包括这些约束条件）。

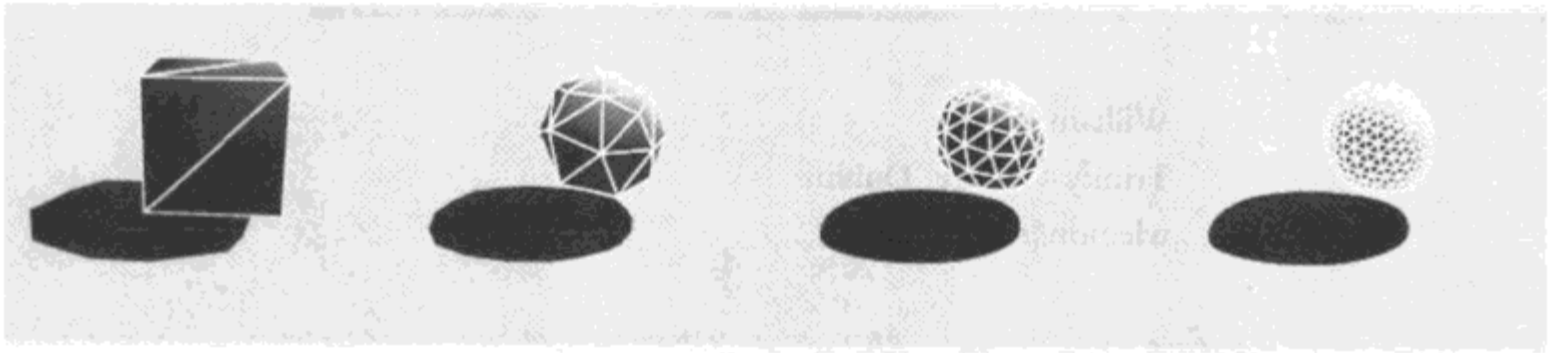


图 4.7.1 Loop 细分表面 4 次后的结果

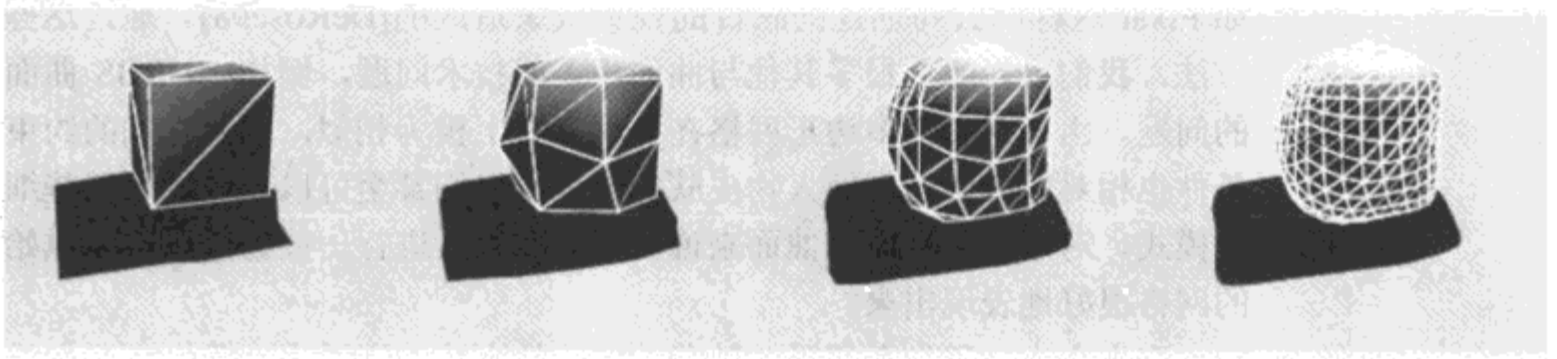


图 4.7.2 蝶形插值细分表面 4 次后的结果

1. 近似模式——Loop 细分

Loop 细分[Loop87]可能是最简单的细分模式了。它的支撑域（support area）比较窄。这种模式使用的掩码是由方程 4.7.1 给出的（参见图 4.7.3）。

$$v_i = \begin{cases} \frac{1}{8}(v_1 + v_2) + \frac{3}{8}(v_3 + v_4) & \text{偶} \\ \sum_{j=1}^{n}{}' \Omega v_j + (1 - n\Omega)v_i & \text{奇} \end{cases} \tag{4.7.1}$$

在其中，我们有

$$\Omega = \frac{1}{n}\left(\frac{5}{8} - \left(\frac{3}{8} + \frac{1}{4}\cos\left(\frac{2\pi}{n}\right)\right)^2\right) \text{或是} \quad \Omega = \begin{cases} \frac{3n}{8} & n > 3 \\ \frac{3}{16} & n = 3 \end{cases}$$

Loop 模式的优点之一就是我们可以使用以下公式计算出每个顶点的切向矢量：

$$t_1 = \sum_{i=0}^{n-1} \cos\frac{2\pi i}{n} v_i$$
$$t_2 = \sum_{i=0}^{n-1} \sin\frac{2\pi i}{n} v_i \tag{4.7.2}$$

Download at www.Pin5i.Com

其中，v_i 是我们希望得到的一个顶点几个法线中的一个。需要注意的是，此处我们使用的掩码与在边界处计算切向矢量使用的掩码是不同的[DeRose98]。由于 sin 与 cos 的计算都比较缓慢，因此我们需要使用一些优化手段（我们将在稍后提及这些方法）将之从计算公式中消除掉。

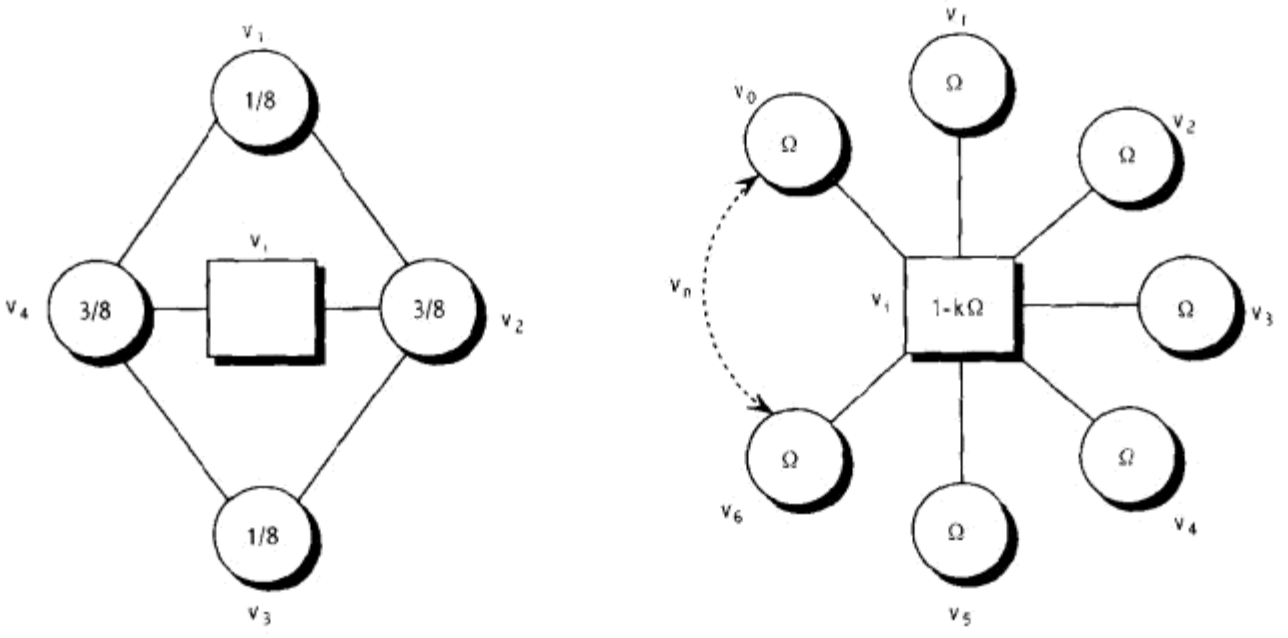


图 4.7.3　Loo 细分掩码

2. 插值模式——修正后的蝶形法

蝶形模式是由其掩码特殊的形状而得名的，当然了，此形状与一只蝴蝶颇为相似。原始的蝶形法[Dyn90]可能是最常用的插值模式了，但是它不能保证对任意的网格都能得到连续的曲面。如果我们再加上少量额外的工作，那么就可以使用一个此方法的修正版本[Zorin96]了。幸运的是，它的支撑域与原来的方法一样小。与大部分插值模式一样，此方法中也没有偶掩码。我们将重用原来的顶点，因为最终的曲面肯定会经过这些顶点。读者可以参看图 4.7.4，其中展示了一个示例掩码。

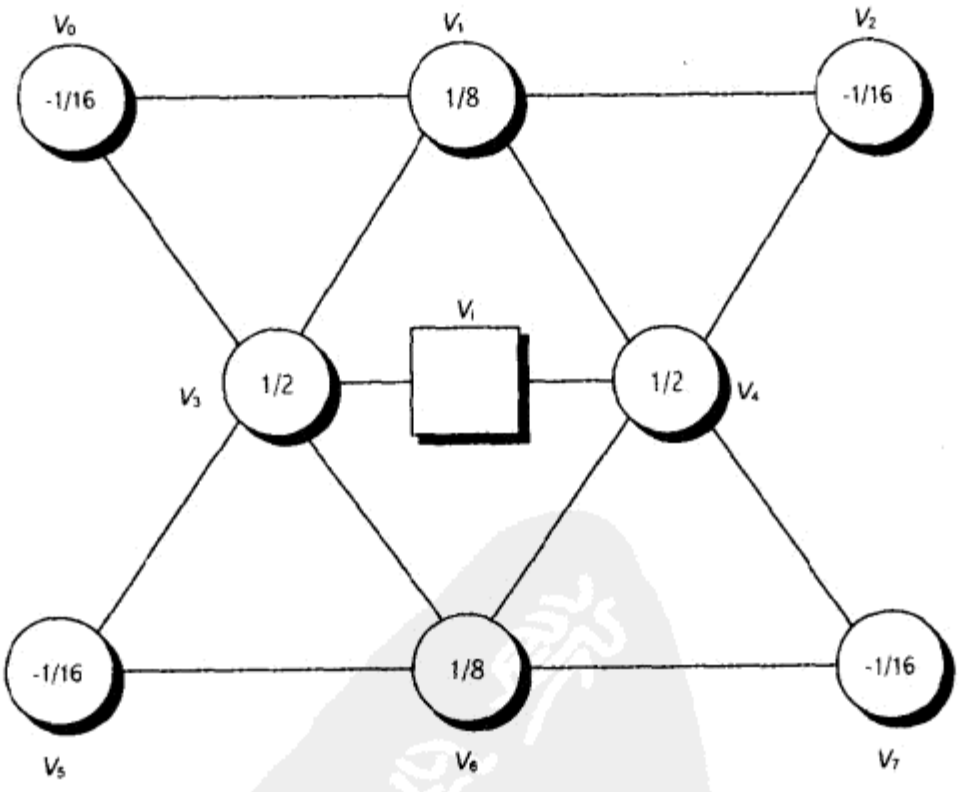


图 4.7.4　蝶形细分掩码

Download at www.Pin5i.Com

$$v_i = \begin{cases} v_i & \text{偶} \\ -\frac{1}{16}(v_1 + v_3 + v_6 + v_8) + \frac{1}{8}(v_2 + v_7) + \frac{1}{2}(v_4 + v_5) & \text{奇} \end{cases} \tag{4.7.3}$$

与 Loop 模式不同，这里计算切向矢量比较麻烦。从曲面来计算这些矢量或许会更快一些，在本文中我们就不讨论这一点了。

3. 层次化的半边缘网格

在使用一个细分模式的时候，最困难的一点可能就是要创建一个适当的数据结构[Weiler85]对节点进行遍历了，而只有在这个遍历中我们才能对每个节点使用细分规则进行处理。我们可以对每一层细分创建一个半边缘（half-edge）的数据结构。此半边缘的数据结构将使用一个顶点，一条边，还有一个面状结构以构成网格：

```
struct vertex
{
   edge *p_edge;     /* edge vertex starts */
};

struct edge
{
   edge *p_pair;     /* other half of edge */
   edge *p_next;     /* next edge in face */
   edge *p_prev;     /* previous edge in face */
   face *p_face;     /* face edge is part of */
   vertex *p_vertex; /* vertex that starts edge */
};

struct face
{
   edge *p_edge;     /* an edge in the face */
};
```

此数据结构与翼状边缘数据结构颇为相似。一个半边缘是一条被两个相邻面共享的边，每条边都有两个指针，分别指向其上一条边与下一条边（实际上，原来的半边缘数据结构中没有指向上一条边的指针，但是加上了这个指针以后，再进行某些查询时就更方便了）。此外，它还指明了其对应的是哪一个面，并指明了这条边的起始顶点（参见上述代码）。每条边只需要与一个顶点相关联就够了。每个顶点会指明由其开始的那条边，每个面会指明构成此面的一条边（参见图 4.7.5）。

即使在细分计算中没有直接使用到此半边缘数据结构，我们仍然可以使用它来快速、方便地计算出与掩码相关的顶点。这个半边缘数据结构是一个极好的检查网格其他属性的方法，例如连通性与错误检验（例如，是否有不只两个面共享了一条边）。为了保存这个层次化的信息，每个面都必须同时保存指向其子节点的指针。此外，每个子节点可以通过一个数组来保存，其地址可以表示为 $kn + i + 1$。其中 i 是子节点的下标，范围是 0 至 k-1； n 是父节点的偏移地址，k 是子节点的数目。关于此数据结构有一点很重要，那就是如果不对它进行修正，就不能处理一条边被两个以上的面共享的问题。如果在载入网格的时候没有对这一点进行检

Download at www.Pin5i.Com

查，那么就很有可能会出问题（可不要说我们没有提醒你）。在任何情况下，上述的两种模式都没有对这一种情况进行处理。

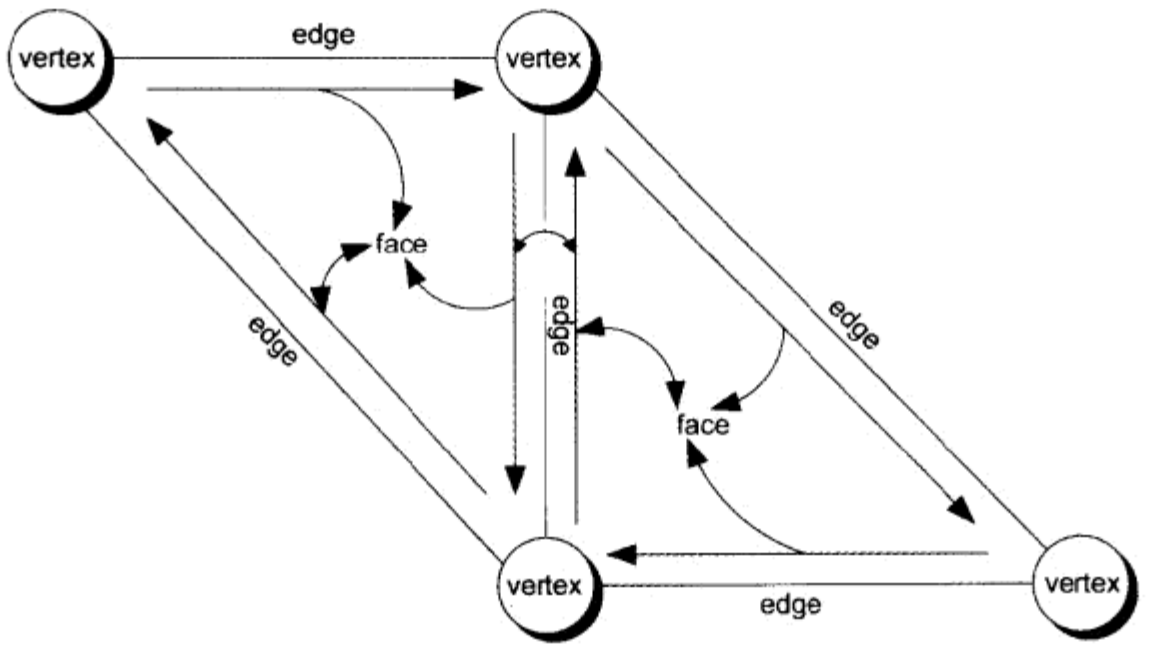


图 4.7.5　半边网格

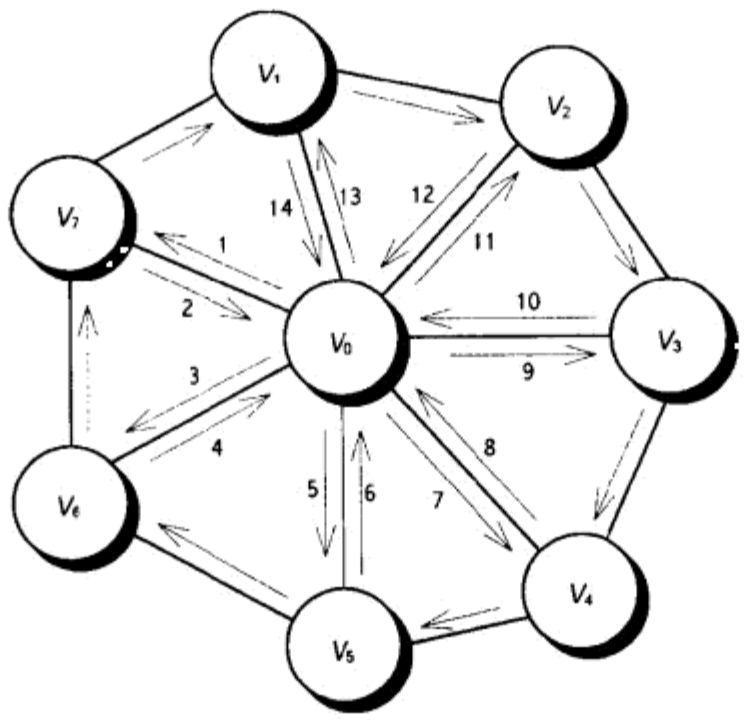


图 4.7.6　查找 v_0 相连顶点的遍历

4.7.2　骨节的层次化结构以及顶点积累缓冲

在生成一个人物的运动效果时，我们通常需要得到骨节的层次化结构。实现骨节的层次化结构并不很困难，因此在这里我们就不多说了。骨节层次化结构是一个变换的层次结构，在其中你可以对节点进行一系列的变换。随着处理层次的深入，每个节点及其顶点都会受到当前变换的影响。值得庆幸的是，如果我们使用了曲面细分的技术对人物的运动进行处理，那么我们只需要对控制点进行变换就足够改变皮肤的形状了。这显然是一个优点，因为此时我们变换的次数少得多。

Download at www.Pin5i.com

带权重的顶点积累缓冲

如果每个顶点都有一个权重值，那么对顶点进行变换的时候事情就会复杂得多。为了实现蒙皮，我们需要使用一个缓冲区来保存得到的积累下来的顶点。然后我们将把这个缓冲区作为细分模式使用的顶点缓冲区。随着处理层次的深入，每个顶点都会经过当前的变换矩阵进行变换，与相应的权重值相乘，然后被添加到顶点积累缓冲区里面去。如果使用这种方法，那么蒙皮过程就不会改变最初的细分网格。现在我们可以对细分网格随心所欲地进行改变，防止出现浮点误差积累的效应了。除此之外，它还有一个优点，那就是细分曲面与蒙皮过程成为了独立的两个步骤，不再会相互影响，这样，在实现的时候就更容易一些了。

4.7.3 优化

要同时处理多个细分的曲面，很重要的一点就是要减少此工作将带来的巨大开销。我们将阐述四种方法以用来减少开销，主要着眼在细分层次结构中的两个地方。

1. 层次化的背景裁剪

层次化背景裁剪使用的技术是基于层次化可见性裁剪集群技术的[Kumar96]，在其中各个面集中在一起进行分组处理。由于我们要处理的是细分表面，那么事先我们就已知子表面与其父表面的属性是很相似的了。因此，如果父表面及其相邻表面都是处在玩家视线的背面的，那么子表面当然也处在背面。这项技术是十分有效的，特别是在多边形数量比较大的时候。此外，我们还可以使用它来减少需要处理的子表面细分的数量，因为如此一来，我们就可以不用处理处在背面的表面细分了。感兴趣的读者可以参阅 Carlo Séquin 在 SIGGRAPH 中发表的文章[Séquin01]，在其中他对这项技术及行了很深入的研究。

2. 视界截面裁剪

另一个优化方法就是惰性空间细分方法，它使用了细分层次结构来略过对某些表面的处理。这种方法将使用 k-d（BSP）树或是八叉树将表面划分为可见的、被遮住的，或是部分可见的状态，这是由对视界截面分析的结果得到的。因此，如果玩家正在注视人物的脸，我们就只会对脸上的表面进行细分。如果要求得到最佳帧率的话，这种方法是很重要的。

每个细分网格都是包含在一个边界框里面。如果整个边界框都是可见的，那么此网格就将被勾画出来。否则，我们将把此框再次划分为四个小框，判定每个小框的可见性。此过程将不断进行，直到达到了最深深度，或者是达到了最大数量的表面数目。这些值的设定对算法的优化是很重要的，而它又是依赖于渲染方法或是使用的图形卡的。此技术带来的问题之一就是某些多边形可能会占据不只一个框。如此一来，可能会导致在程序中对此多边形针对视界截面进行多次判定与勾画。为了消除这种现象，我们在判断每个面的时候都会把它与当前的帧序号关联起来。然后，在处理其他节点的时候，已经作了关联的表面可以不再进行处理，这样一来就可以减少需要判定的表面的数量了。通过这个判定，我们可以去掉那些不需要进行细分的表面，只对在视界截面里面的表面进行细分。对于较小的网格而言，最好能直接使用对可见表面进行裁剪使用的视界截面。

Download at www.Pin5i.com

上述的两种方法都将表面划分为可见的与不可见的。然后我们将使用此数据减少由细分生成的子表面的数量，也可以在对某些表面进行渲染之前将之裁剪掉。

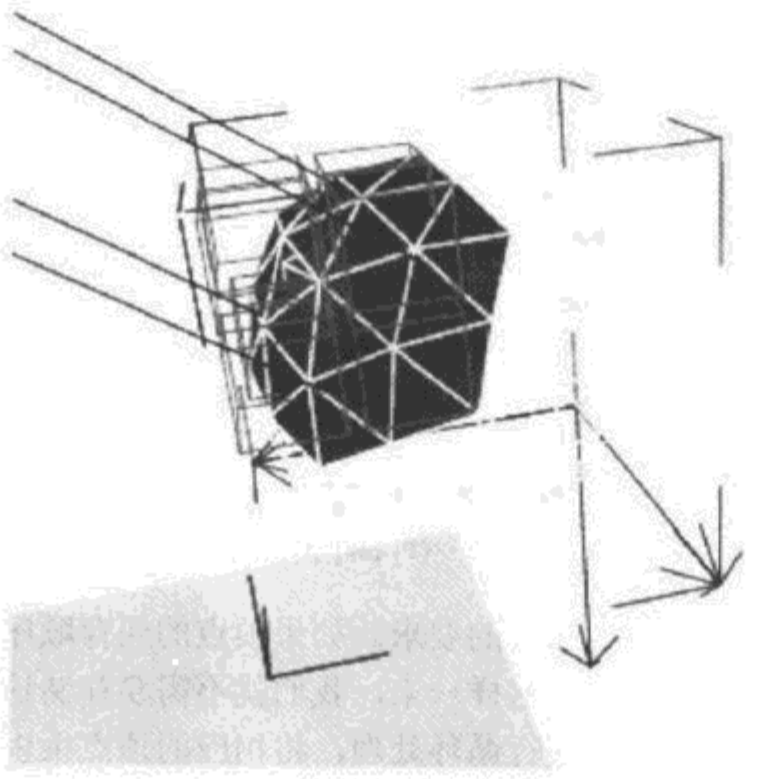


图 4.7.7　使用 BSP 树的惰性截面裁剪

下面的两种优化方法都需要对数据进行预处理，这样可以减少在遍历半边缘数据结构时产生的开销。不幸的是，这些技术可能会导致对内存的要求急剧增大，但是同时，它们也极大地提高了显示的速度和生成细分表面的速度。

3. 表面顶点索引预处理

在需要显示细分曲面上的一个表面时，对表面顶点索引进行预处理是很有用的。在这里我们只需要对表面的这个索引数组进行一次处理就够了。其主要的思想是要在使用细分曲面之前把构成表面的各个顶点的索引存储起来。这样一来，我们就不需要在每次渲染此表面的时候都重新计算索引了。接下来，我们将把它转化为一个顶点数组的一部分。此表面索引数组的形式如下所示：

$$\{(v_{11}, v_{12}, v_{13}), \ldots (v_{n_1}, v_{n_2}, v_{n_3})\} \tag{4.7.4}$$

4. 对权重值与顶点索引进行预处理

对权重值与顶点索引进行预处理主要有以下作用，那就是可以帮助生成偶顶点，同时也可以对奇顶点进行调整，这样我们就不需要再对半边缘网格进行遍历了。基本上来讲，我们将对每个顶点（i）计算出一组权重值（w_i）以及索引值（v_i），然后将其与最终结果的索引值（v_d）保存在一起。这样一来，我们只需要进行一次 for 循环就可以得到下一组顶点。我们同样可以使用这种方法在 Loop 模式和蝶形模式中计算出其切向矢量来，同时避免使用到 sin 与

Download at www.Pin5i.Com

cos。对于不同的细分模式而言，其具体处理方法也略有不同。蝶形模式需要的内存更少，这是因为对于每个掩码来说，其对应的顶点数目是不变的。这样，我们需要存储 8 个权重值与顶点的索引，同时还要加上最终得到的顶点的索引值。蝶形模式下的权重—索引数组形式如下所示：

$$n\times\{(v_0,w_0),\ldots(v_8,w_8),v_d\} \tag{4.7.5}$$

对于 Loop 模式来说，情况要稍微复杂一点。对于偶掩码来说，没有固定数目的顶点用来生成下一轮的顶点，因此，为了存储这些信息，我们需要添加一个域，在这个域里面保存的是一共存储的顶点的个数。值得庆幸的是，奇掩码的顶点数目是固定的，这样我们就可以使用与蝶形模式同样的方法来对之进行存储了，不过对于它来说，我们只需要保存四个索引值和权重值而已。一个 Loop 模式下偶掩码与奇掩码权重值与索引的数组形式如下所示：

$$n\times\begin{Bmatrix}((v_0,w_0),\cdots(v_4,w_4),v_d) & \text{奇}\\ (k,(v_0,w_0),\cdots(v_k,w_k),v_d) & \text{偶}\end{Bmatrix} \tag{4.7.6}$$

在此处我们还可以大大减少对存储量的要求。如果顶点的保存顺序总是一样的，那么对于奇掩码来说权重值就将保持不变了。这样一来，我们就不需要存储这些数值了。在使用这些数组的时候，只要在程序里面对之进行循环处理，将相应的顶点乘以权重值，最后将这些结果相加起来得到顶点就行。

4.7.4 系统集成

实现细分模式是很简单的。在本文的实现中，我们采用了一个多遍处理的方法，在每一遍处理中都会生成一层新的网格。使用了此方法以后，代码实现就相当简单了，如果需要的话，我们可以在每一遍循环中进行优化。在每一遍处理中，我们都将顶点保存在一个数组中，此数组的长度会根据所需顶点的数目增长。在最后一遍处理中，我们将会把顶点和表面放置在一个几何图形数组中，然后再把此数组送到渲染流水线上进行余下的处理。

1. 保存数据

在处理细分曲面的时候，我们将使用数组保存其数据。这既可以加快处理速度，也便于数据管理，而且相对来说，我们也更容易将此数组直接以顶点数组的形式传递给图形处理的 API 进行处理。数组的管理是非常方便的，因为我们只需要在开始分配一段足够大的内存就够了。然后，如果在使用过程中发现内存不够，我们还可以进行重分配以得到更多的内存。使用了这种方法以后，在游戏中，只有开始几帧会比较慢（一直到分配了足够的内存）。数组占用的内存也会比较小，因为此时只有简单的层次结构与线性存储结构了，我们也不需要使用指针对之进行处理。由于内存是保存在一整块连续的空间里面的，它的缓存吞吐量会更大，因此，对之进行访问也会更加快捷。基于数组的方法带来的另一个优点是在使用积累缓冲进行蒙皮的时候，表面索引与其他的参考数据不需要重新生成，这是因为每个生成的顶点都是保存在数组里面同样的相对位置的。

Download at www.Pin5i.Com

2. 删除不可见部分

为了节省时间，我们只对原始网格进行了可见性裁减，这是因为此项操作的开销很大。虽然判定一个面是否可见是相对容易的一件事情，但是要判定哪些表面需要生成子表面则会比较困难。为了完成此项任务，我们需要事先知道到底需要多少层细分。这是因为细分模式所需要的支撑域是互相重叠的，这就会导致产生一个有相互依赖关系的层次结构（请参见图 4.7.8）。需要的细分层数越多，要求的支撑域也就越大。如果你不想判定这些区域的大小，可以使用一种称之为“惰性估值”的方法，在需要表面的时候再生成它们。不幸的是，实践证明，这种方法效率比较低，这是因为此时我们访问的是分段的内存，而不是一整段内存。

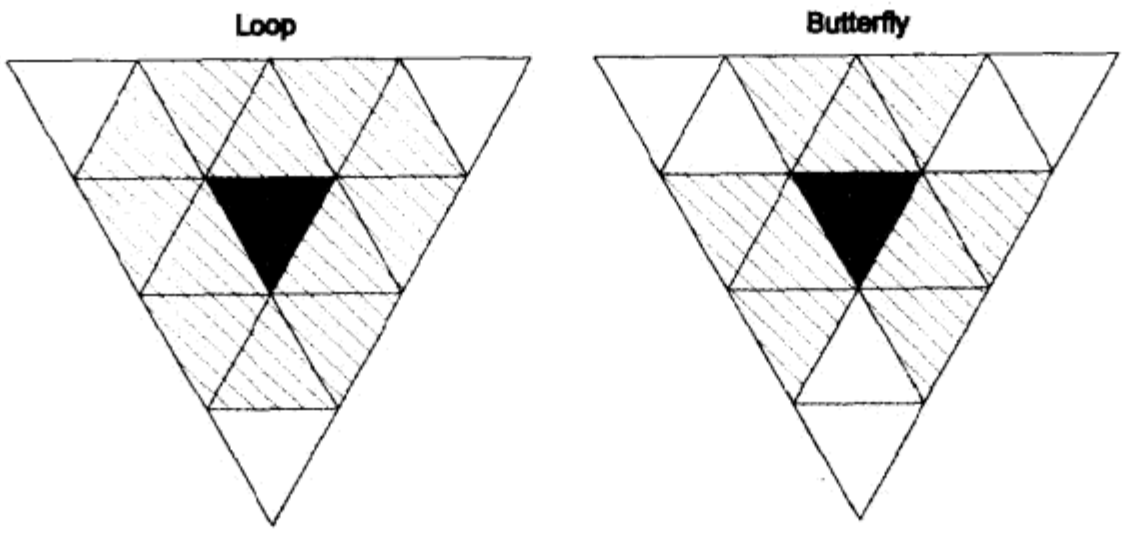


图 4.7.8　Loop 与蝶形细分要求的支撑域

值得庆幸的是，蝶形模式与 Loop 模式的支撑域是非常类似的。其中蝶形模式需要的区域会稍微小一点（参见图 4.7.8）。这样一来，除了得到在给定细分层次下的可见表面以外，我们还可以使用同样的方法得到其他的表面。同样，通过它，我们还可以清楚地看到，对于一个只有相对较少节点的网格来说，例如使用三角形构成的立方体，子节点对域网格中其他部分的依赖性还是相当大的。

3. 对帧进行渲染

为了渲染一个表面，我们需要把一组三角形、顶点、法线，还有纹理坐标都放置到一个顶点数组中，然后对之进行渲染。我们可以将渲染代码用一个简单的 API 封装起来，将之与其他的部分隔离开。这样，在对场景进行更复杂的操作与重新编排以进行更加快速的渲染时，我们的改动就可以最小化了。

4.7.5　源代码

在 CD-ROM 上有一个示例程序。

Download at www.Pin5i.Com

4.7.6 结论

我们可以使用表面细分来创建出非常细致、极其自然的人物来。我们也可以使用同样的方法对纹理参数进行处理以生成恰当的表面坐标。此外，很多当今的建模软件包也提供了对细分模式的支持。通过它们，我们可以很容易地得到一些增强细节效果的方法。不过，这些细分的方法没有提供方便的手段从原始的网格中减少网格的细节。在此时，渐进式（progressive）网格[Svarovsky00, Hoppe96]就显得更重要了。如果能将两者结合起来，那结果一定很棒——我们可以使用细分来增强细节，使用渐进式网格来减少细节。细分表面还可以减少蒙皮的时候所需进行的变换操作的数量，这是因为我们只需要对原始的网格进行修改就行了。现代的图形加速器都提供了 T&L[1]的支持以及顶点权重的扩展，这样我们就可以在细分中使用这些特性了。为了做到这一点，我们将对视界截面进行变换，而不是对网格的顶点进行变换。接下来，当我们使用这个修正后的视界截面进行了裁减以后，那些没有变换过的可见三角形就可以送到图形卡中进行变换与显示了。

4.7.7 参考文献

[DeRose98] DeRose, Tony, et al., "Subdivision Surfaces in Character Animation," Computer Graphics Proceedings (SIGGRAPH 1998): pp. 85～94.

[Dyn90] Dyn, Nira, et al., "A Butterfly Subdivision Scheme for Surface Interpolation with Tension Control," ACM Transactions on Graphics, Vol. 9, No. 2, pp. 160～190, 1990.

[Hoppe96] Hoppe, Hugues, "Progressive Meshes," Computer Graphics Proceedings (SIGGRAPH 1996): pp. 99～108.

[Kobbelt98] Kobbelt, Leif, et al., "Subdivision for Modeling and Animation," Course Notes (SIGGRAPH 1998).

[Kumar96] Manocha, Kumar, et al., "Hierarchical Visibility Culling for Spline Models," *Graphics Interface*, 1996: pp. 142～150.

[Loop87] Loop, Charles, "Smooth Subdivision Surfaces Based on Triangles," Master's Thesis, University of Utah, Department of Mathematics, 1987.

[Séquin01] Séquin, Carlo, et al., http://www.ce.chalmers.se/staff/tomasm/research/subdiv/, December 25, 2001.

[Svarovsky00] Svarovsky, Jan," View-Independent Progressive Meshing,"*Game Programming Gems*, Charles River Media, Inc., 2000.

[Weiler85] Weiler, Kevin, "Edge-Based Data Structures for Solid Modeling in Curved-Surface Environments," IEEE Computer Graphics and Applications, Vol. 15, No. 1, pp. 21～40, 1985.

[Zorin96] Zorin, Denis, et al., "Interpolating Subdivision for Meshes with Arbitrary Topology," Computer Graphics Proceedings (SIGGRAPH 1996): pp. 189～192.

译者注[1] T&L 指的是 Transform & Lighting，即坐标转换模块及光照模块。

Download at www.Pin5i.com

4.8 改良的骨节变换计算

Jason Weber
英特尔公司
jason.p.weber@intel.com

艺术创作人员可以制作出精美而真实的网格，在游戏中把各个角色表现出来。为了使之更加真实，我们需要为这些造型添加上各种行为动作。如果把每一帧运动中各顶点的位置都存储下来，这不仅在存储量上完全不可接受，而且人物的动作会被局限在艺术人员事先设置好的一套集合上面了。因此，在很多游戏里面，它们都使用了一层隐藏的线段，通过它们就可以得到更加真实的一套骨架动作了。

我们可以将网格上的各个顶点通过多个，而不是一个，矩阵进行变换，然后对得到的结果进行仔细的加权平均，这样一来我们就可以将这些网格平滑地变化到任意的位置，将得到的结果以任意的帧率播放了。通过使用这种方法，我们可以大大减少所需处理的运动数据量，而且骨架，因此也包括了网格，可以很快地调整成任何姿势，甚至可能在其环境中产生原本没有想到的动作。

不过，流行的变化算法如果不经改动就加以使用会带来一些问题。我们将展示一下，在较大偏转角的情况下关节将会如何变小，甚至有可能退化为一个点。值得庆幸的是，我们只要在出问题的关节处，例如肘部或膝盖处，加一段额外的骨节就可以解决此问题了。在仔细计算过这些关节处的权重值之后，我们可以复用同一段简单的核心变化算法，只需要额外处理一些骨节的计算就可以了。

4.8.1 背景知识

骨架结构基本上来说就是一组有次序的转换，正如一幕幕走景一样。在每个转换中间，我们都将定义一个骨节长度，实际上它就是延变换的 x 轴方向上的一段位移而已。在缺省情况下，每个子骨节的原点都处在父骨节的终点上。我们可以允许使用任意的位移量，但是在大部分情况下位移量为零，因为一般来说骨节都是一节节连起来的。

我们使用了一个参考坐标以描述骨架的层次结构，在此坐标中骨架处在一个没有变形的网格中（3ds max 中的 BiPad™称其为“figure mode”）。如果骨节的运动偏移了参考坐标，那么此运动可以用来将网格变换到任意的位置。这些骨节的运动是由某些驱动源产生的，例如动作捕获机制、事

Download at www.Pin5i.Com

先制作的动作，或是逆向动力学[Weber02]（若想对这些背景知识有更多的了解，请参看GDC2000 学报[Weber00]）。

4.8.2 简单的方法

读者可以在《游戏编程精粹 1》中的[Woodland00]中找到最基本的蒙皮算法，Jeff Lander 在 *Game Developer* 的文章中也对此算法进行很好的阐述。作为此技术的一个例子，我们来考虑一下在肘部处顶点的情况，此肘部的位置是由当前上臂与下臂骨节的共同变换决定的。当我们在两个变换中处理肘部位置的时候，其结果将得到不同两组顶点。如果接下来就对得到的结果进行加权平均，我们会得到一个折衷的位置。在此处有一个重要的准则，那就是这些顶点要能将 100%的上臂运动平滑地过渡成为 100%的下臂运动。否则，肘部的网格就会变得非常古怪，甚至有可能就像折断了一样。

随着下臂偏转角的增大和每个骨节位置差异的增大，有可能出现严重的异常现象。例如，最坏情况下面，你可能会把子骨节延胳膊的方向偏转 180°。两种变换得到的结果方向在肘部是完全相反的，因此如果对之进行对半平均，我们就会得到一个在肘部内部的点。其整体效果就好像是把毛巾拧了一个来回一样。图 4.8.1 与图 4.8.2 展示了这么一个出问题的例子。

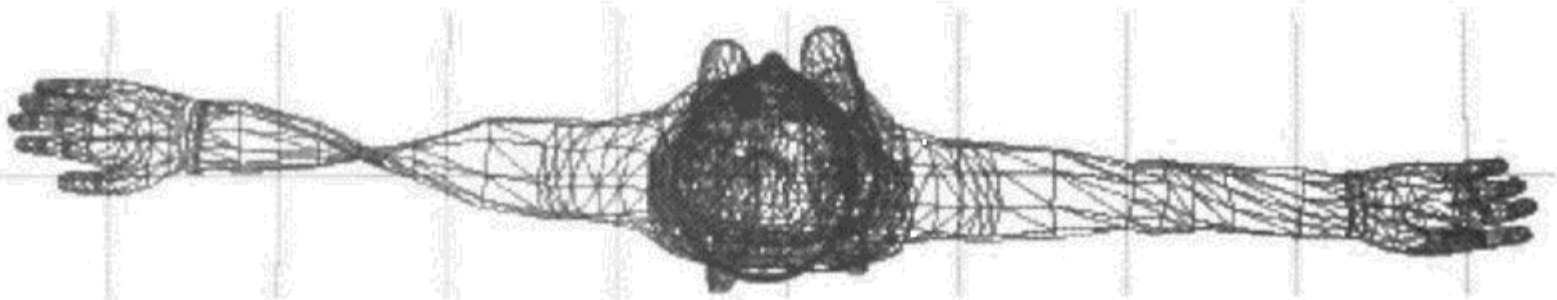


图 4.8.1 扭曲的肘部：左臂使用的是简单的蒙皮法，右臂使用的是改进的技术

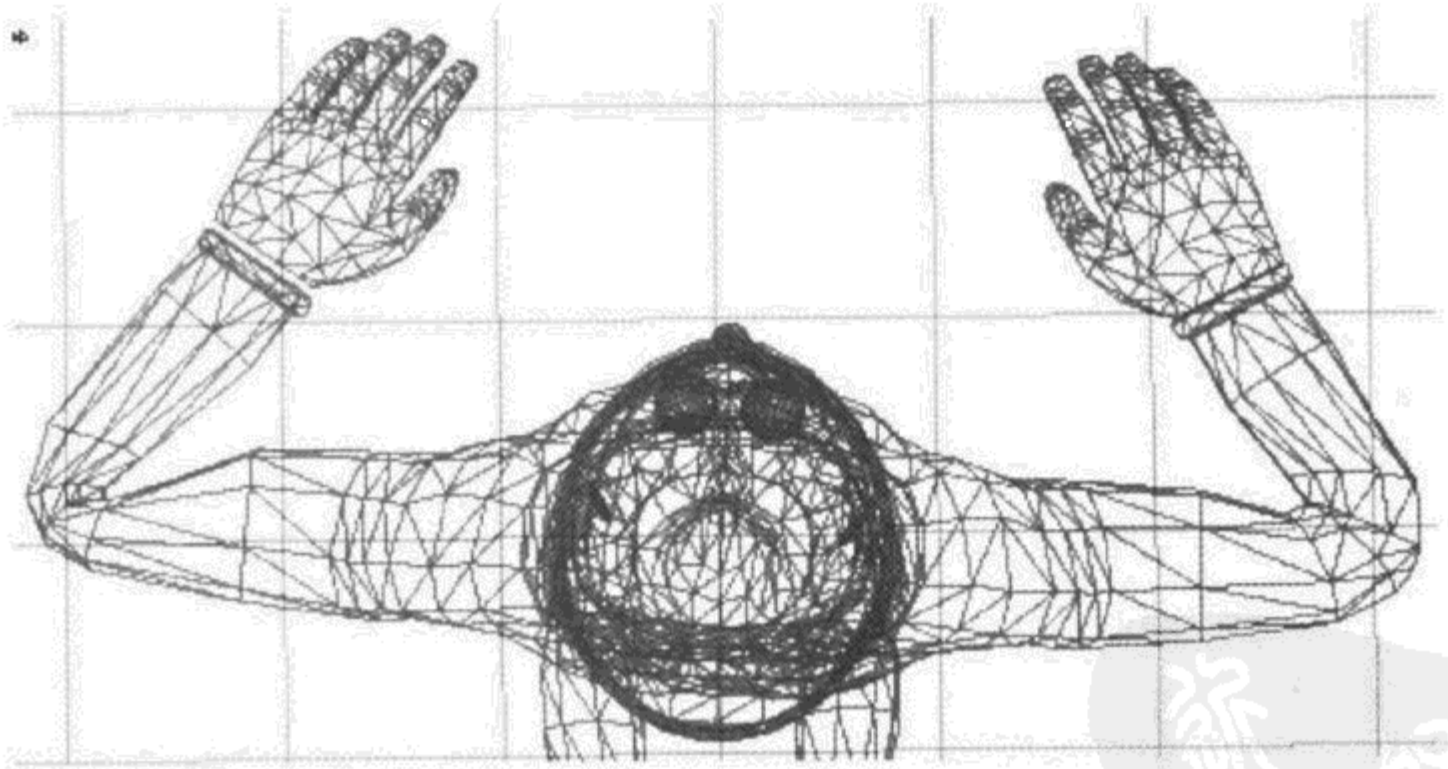


图 4.8.2 相应弯肘的效果。右臂使用了改进的技术

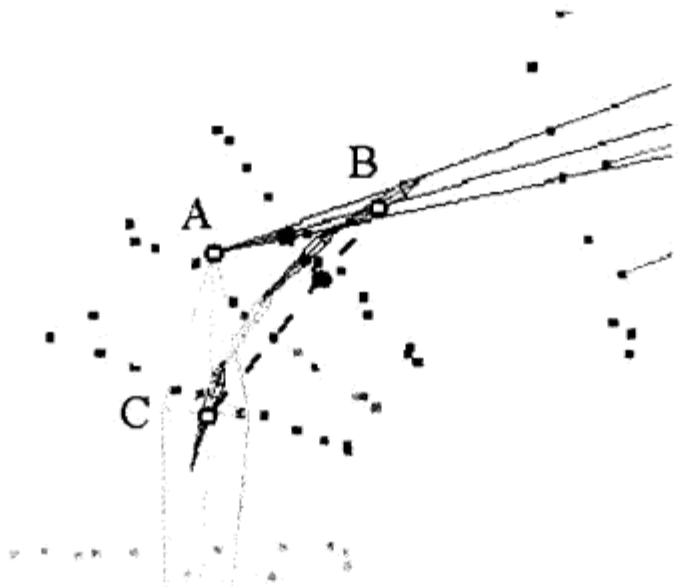
Download at www.Pin5i.Com

4.8.3　添加骨节

如果在大角度下可能出问题，那么一个解决方案就是对所有的偏转角进行限制，要求其更小一点。当然，在实际情况中，我们是不能对现有的骨节进行限制的，但是我们可以将较大的角度分布到几个放置在关节处的更小的骨节上。由于 60°角一般来说是安全的，因此只要加上三个或是更多的这种连接就可以了。而这些连接的长度可以留给建模人员来决定。我们可以使用一个缺省值，它可以从网格在关节处的跨节（cross-sectional）半径以及子骨节的长度计算得出：

```
total_linklength=0.3*child_length+1.5*joint_radius
```

骨节的摆放位置应该与样条曲线的形状差不多。骨节应该首尾相连，但是当关节弯曲的时候可以进行适当的滑动。我们并不需要真的把骨节的长度缩短，但是由于会产生重叠，因此它将很自然地把网格变小一点。读者可以在 CD-ROM 上 **GPGBoneNode.cpp** 中的 `GPGBoneNode::CalcBoneLinks()` 方法中找到我们的解决方案，它的工作方式如下所示：



（A）

（B）

图 4.8.3　（A）在关节处添加了一条骨节链（B）新的权重关系的例子

Download at www.Pin5i.Com

为了计算出此骨节的位置，首先让我们来考虑三个点，如图 4.8.3 所示——原始的关节连接点 A，第一个连接中心 B，还有最后一个连接中心 C。在关节弯曲的时候，它们将形成一个三角形。对于每个连接来说，取两个点，一个在 BC 上，另一个在 BA 或是 AC 上，这主要要看此连接是在整条链的前半段还是后半段而定。这些点在线上的位移与连接在整条链上的顺序成正比。一旦你得到了这两个点以后，就可以对之进行加权平均的运算，找到所需要的连接的中心了。对于 BC 上面的点来说，如果其处在第一个或是最后一个连接上，则权重为 100%，如果它在正中间的连接上，则权重就是 50/50 了。在计算连接的旋转时，我们可以将之转化为一个对整个角度变化的线性插值运算，这样总的偏转角就可以被平均分配下去。

创建连接以及对其权重的分配都只需进行一次。然而，连接的位置与旋转运动在每次父骨节或是子骨节进行运动的时候都需要进行运算，有可能在每帧都需要进行处理。

4.8.4 改变权重

在一个由骨节连接的关节上，大部分权重都需要重新赋值。其权重值应该能够平滑地从父骨节过渡到第一个连接，再沿着整条链传递下去，直到最后到达子骨节。如果在关节处没有分叉，而且所有原来的影响都是施加在父骨节与子骨节之上的，那么我们只需要使用轴向坐标（x）就可以找出一个骨节需要连接的两端的骨节了（此处的骨节包括那些用于连接的骨节）。如果我们将连接看成是一个整数排成的轴线，把父骨节看成是零点，把子骨节看成是 (number_of_links+2)，那么我们就可以沿着这条链将顶点的本地 x 坐标按照这条整数轴的尺寸进行定位了。我们可以找出一个顶点处在什么整数之间，然后根据它在轴向上的位置对之计算权重。比如说，如果我们的本地 x 坐标在骨节连接空间中定位为 2.3，那么这个顶点对于第二个顶点的权重就是 70%，对第三个顶点的权重就是 30%。

如果在骨节附近的空间里面有一个分叉，那么我们仍需要考虑更多的信息，即使对于某个特定的子骨节并没有明显的分叉子节点。臀部与肩部都是这样的例子，对于任何一个特定的连接链来说，我们使用它的目的都只不过是要将其父骨节的影响分担到它上面，然后将其余的影响通过它施加到余下的骨节上去而已。例如，在肩部附近的胸部之上的顶点可能会受到上臂运动的很大影响，但是同时它又是附着在一条或是多条脊骨之上的。当你在锁骨与上臂之间添加骨节的时候，我们并不需要去改变脊骨上的顶点。

为了得到相应的 x 坐标分量，我们需要得到与此子骨节具有同一父骨节的顶点的全部权重值。我们可以将它们看作是一个“竞争对手”，它们将分担一些弯曲量。为了确保此分量不会受到权重值处理顺序的影响，我们只会考虑那些在当前权重值后面的权重列表上的值。如果把所有竞争对手的权重都加起来，我们就可以得到此分量了，如下所示：

$$fraction = 1 - competitor / (competitor + childweight)$$

重新计算得到的权重值为：

$$fraction * parentweight + childweight$$

此时我们就可以像处理无分叉情况一样继续向下处理了。我们将现有存储下来的父节点的权重值就地修改。对于这两个新的权重值来说，我们首先把前一个子节点的权重值覆盖掉，

Download at www.Pin5i.com

它现在是一个被重写了的值，然后为第二个分量在权重值列表里面添加一个新的权重值。

图 4.8.4a 中显示了一个具有重新设置权重值的肩部关节的情况。在流图中我们可以看到顶点究竟是与哪一个连接相关加权的。在其中，我们并没有显示权重的具体大小或是任何其他骨节，例如脊骨，上面的权重值。一条由三个连接组成的链将锁骨与上臂连接了起来。在图中，我们在影响顶点的骨节中心处与相应的每个顶点都连上了一条线。其中，连线的灰度与骨节的灰度是一样的，但是为了将之区别开，它们相互之间是不同的。值得注意的是这些连接一直影响到了胸部的部分，这样，在手臂运动的时候，它就将产生一个联动的效果。

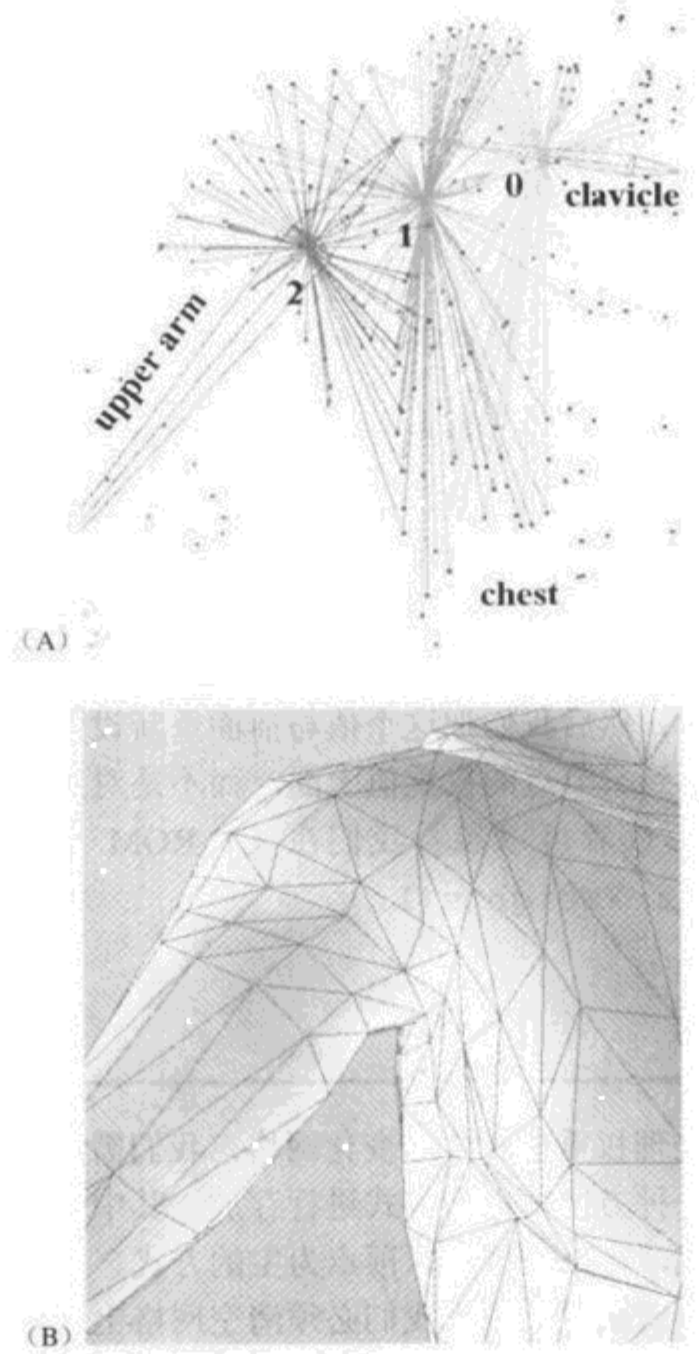


图 4.8.4　（A）添加了 3 个链后的新权重（B）对应的肩部

通过法线削弱过度的影响

在以上方法中我们必须消除其带来的一个副效应。由于我们是纯粹根据位置的远近进行权重值计算的，因此某些网格中直角上的关节有可能会不正常地放大。例如，肩部就有可能出现这个问题。当上臂向上移动的时候，胸侧就会向外鼓起来，好像它已经成为了上臂的一部分似的（请参见图 4.8.5）。

Download at www.Pin5i.Com

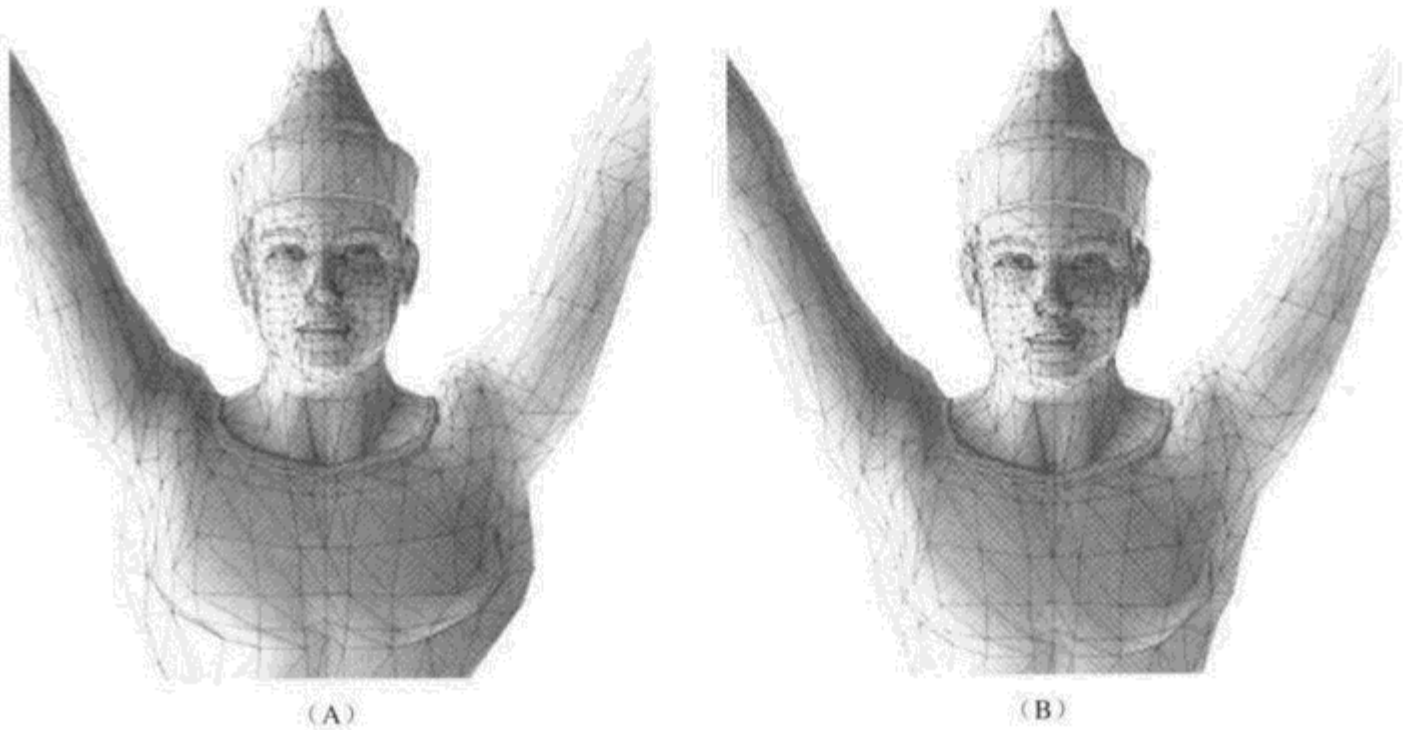


图 4.8.5 抬臂（A）没有消除法线效应（B）消除了法线效应

ON THE CD

我们可以使用这两部分法线的内在差异来消除这种过度联动的影响。首先，我们将顶点的法线与纵向的骨节轴线进行点积运算。然后，对得到的结果，我们再减去此顶点相对于骨节在 x 轴向上的位移量，接着将其结果除以关节部分的半径（两个中间结果都将大于 0）。相减的目的是为了将父骨节一侧对关节影响力的修正给独立出来。第二个结果是一个小数，其范围为 0.0 到 1.0，我们再对之平方，这样以方便比较。然后我们把这个值与前面重新设置的权重值相乘。这部分的权重值将会直接加到父节点的权重值上面，而不是对原来的权重值进行覆盖。关于这部分的代码，读者可以参看我们在 CD-ROM 上面 GPGSkin.cpp 文件中的 `GPGSkin::RelinkWeights()`。

4.8.5 系统集成与优化

为了能以最快的速度处理权重值并生成变化网格，我们需要注意很重要的一点，那就是运行时的数据结构应该以一种方便缓存的方式进行存放。其中，最重要的决策就是要判定究竟是使用以骨节为主的方式，还是要使用以顶点为主的方式。

对于以骨节的为主的方式来说，首先我们必须清空网格上所有的顶点位置及法线。接下来，对于每个骨节，你需要处理其影响到的顶点，把各分量累加起来将每个累加值保存在其相应的位置部分与法线部分上。在这种方法中，虽然当前矩阵可能会被保存在缓存中，但是它对内存的访问比较分散，效率较低。

在以顶点为主的方式中，我们将按顺序对顶点进行处理，需要的时候再载入矩阵。虽然这么一来访问矩阵的时间会比较分散，但是其优点仍是比较显著的。首先，我们不需要一个专门的时间段进行清空的工作，我们自己可以清楚地知道对顶点数据的第一次写入是发生在什么时候的。这样在访问的时候我们就可以使用直接设置的方式，而不需要进行相加了。由于一个顶点的权重值是集中存放的，因此我们可以将其积累值放在一个局部变量里面，每次

Download at www.Pin5i.com

在写入顶点数据的时候再一次性导出，而不必对每个顶点进行反复的读写操作。

以上的两种方法我们都做过试验，在我们的所有试验中，以顶点为主的方法都要比以骨节为主的方法至少快两倍，而且还可能会更快，因为此架构具有较好的灵活性，我们可以使用下面将要阐述的方法对之进行进一步的优化。

1. 变换矩阵

在骨架中的每个骨节都需要进行一个变换，其对象包括了新加上的骨节之间的连接。直到真正的变换阶段之前，我们都将使用四元数进行运算，因为对它们进行插值比较方便[Bobick98]。然而，对于原始的顶点变换来说，使用矩阵进行操作至少要快上四倍。因此，在进行核心变换处理之前，我们需要对一个排列整齐的 3×4 矩阵进行填充，每个骨节都有这么一个矩阵，每个矩阵都是将骨节参考变换的逆变换乘以骨节的当前变换得到的。通过这种方法，我们就可以从原始的、没有经过变换的网格中进行直接变换，而不需要保存每个顶点相对于骨节的偏移量了。

ON THE CD

核心变换处理大概只有 50 行的代码。读者可以参看 CD-ROM 上 GPGSkin.cc 文件中的 `GPGSkin::ComputeDeformedVerticesPacked()` 函数。

2. 压缩权重值

由于顶点的权重值是一个很大的数据结构，它一般比矩阵数组，甚至可能比网格数据还要大，因此我们需要让权重列表变得小一些，这样就可以减少在每帧中我们需要处理的数据量了。这么一来，不仅可以节省很多空间，而且可以对缓存的使用进行优化。但同时我们也要注意到，如果数据压缩得太多，有可能会导致字节不能对齐，这样会对处理效率产生负面的影响。

packweight 数据结构是一个字节数据块，它由一段段交替的顶点定义以及一个或多个骨节权重影响值组成。在其中，顶点定义包含的是顶点索引、一份变换前的顶点位置以及法线的拷贝，以及下面权重值的数目。骨节权重值域中，我们保存的是骨节在矩阵数组中的索引值以及其分到的权重值。在我们读取任一个数据块的时候可以很快地得到下一个数据块的位置以加速访问。

ON THE CD

需要注意的是，在权重列表重保存顶点的位置和其法线意味着我们可以对一个输出的网格进行连续的变换，同时不需要保存原有的变换之前的输入网格。如果我们希望能让外部对输入的网格进行修改，例如能让一个修改形状的代码对之进行修正，那么我们就不需要在权重列表里面保存这个值了，但是同时我们付出的代价就是要在每帧中从输入网格里面重新读入顶点。请参看 CD-ROM 上面的 GPGPackWeights.h 以熟悉这段代码。

3. 法线重新归一化

我们可以对多个法线进行加权平均，正如我们对位置数据一样，但是其得到的结果在绝对值上会变得小一些。这种差异很容易弥补。由于得到的法线大小在 0 到 1 之间，因此我们

Download at www.Pin5i.Com

可以使用一个大小适中的查找表以代替 sqrt()运算。如果不对这些法线进行重新归一化的话，最终光照密度就会下降。我们的观察表明其实这个差异是很小的，因此你可以完全不对其进行考虑，或是将之作为一个选项以得到更好的效果。

4.8.6 结论

基于骨节的运动对于减少动画开销，对于生成实时的、独特的行为来说都是非常关键的。我们可以对现有的变换技术进行改造，这样就可以使之大大加快，而且很容易进行扩展以克服某些内在的限制。

其他的主题将谈及最初顶点权重值的生成与操作。我们使用一个全自动化的过程来生成原始权重值、删除异常点、对其分布进行平滑处理，以及加上骨节连接，彩图 7 中显示了其最终效果。

在腰部的改善主要是由于重新生成的权重值产生的。在上臂处，我们使用了连接来减少过度的收缩，这也收到了极明显的改善效应。即使是肩部与膝部也从消除过度的伸展部分中得到了极大的改善。此外，胸部看上去更加真实了，因为在肩部使用了连接以后，我们就可以对其周围的网格施加更多的影响了。

4.8.7 参考文献

[Bobick98] Bobick, Nick, "Rotating Objects Using Quaternions," *Game Developer Magazine*, February 1998: pp. 34～42. 其对应网址为 http://www.gdmag.com.

[Lander98] Lander, Jeff, "Skin Them Bones: Game Programming for the Web Generation," *Game Developer Magazine*, May 1998: pp. 11～16.

[Weber00] Weber, Jason, "Run-Time Skin Deformation," Game Developers Conference Proceedings (GDC 2000): pp. 703～721. 其对应的 FTP 地址为 ftp://download.intel.com/ial/3dsoftware/animatedoc.pdf.

[Weber02] Weber, Jason, "Constrained Inverse Kinematics," *Game Programming Gems 3*, Charles River Media, Inc., 2002.

[Woodland00] Woodland, Ryan, "Filling the Gaps—Advanced Animation Using Stitching and Skinning," *Game Programming Gems*, Charles River Media, Inc., 2000: pp. 476～483.

此外，作者还将努力维持一个长期性的文档库与连接网站，通过它，读者可以访问其他相关资源，其网址为 http://www.imonk.com/baboon/bones.

Download at www.Pin5i.Com

4.9　针对真实人物运动的架构

Thomas Young
PathEngine
thomas@pathengine.com

在当前硬件的协助下，我们已经可以渲染出极其真实的人物来了。可信度极高的动作捕获系统在数据采集与供应方面已经得到了广泛的使用。然而，在大部分系统中间，每当人物开始运动的时候，这表面上的真实性就被完全粉碎了。人物的行走就像脚板贴着地面，身体在走动过程中会不自然地任意扭动，或者人物会一下子转换到完全不同的状态中去。

即使在游戏中只有一小段的贴地行走动作，也会被玩家迅速发现的。一旦我们在游戏中发现了这种现象，那立刻就意味着游戏中完全没有考虑到摩擦力的作用。如果在脚板与地面之间没有摩擦力，那么人物的前向运动就没有原动力了，因此一旦发生这种情况我们就马上能感觉到其不真实。

要想同时解决所有真实运动的约束问题是非常困难的。我们可以利用一个插值修正器来确保在运动中的平滑度。这个修正器将计算出运动中的中间位置。对于任意目标的运动问题来说，我们可以对每个动画片段进行修正以解决此问题。不过，如果简单地进行插值，我们仍然会得到贴地行走的结果。

在本文中，我们提出了一个对此问题的解决方案，它主要是基于对脚板的位置进行调整的，不过这些调整只有在它们处在运动状态中的时候才能进行。在此处我们将阐述一个架构以将这个点子应用到真实人物的运动问题上，方法就是要提供一个独立的修正器对骨架的不同部分进行修正。

4.9.1　问题：针对任意目标的运动

在 2D 时代的末期，已经有一些例如 *Prince of Persia* 以及 *Fade to Black* 这样的游戏开始出现了，它们都具有极其眩目的动画效果。其中的诀窍就在于游戏的环境是由单位长度的分片在一个固定的格网上面创建起来的。由于动画都是依照完全一样的单位长度构造出来的，因此我们就可以保证动画可以表现得非常漂亮了，它们其实不过是墙壁或是悬崖贴图上的一些像素而已。

当运动需要在此过程中进行任意角度的转向与向前运动的时候，情况就变得复杂得多，因为此时，我们就不能将其运动限定在一个固定的格网中了。在这里，我们将不得不努力解决人物需要行走任意距离，而且能转

Download at www.Pin5i.Com

向任意角度的问题。

在图 4.9.1 中的人物运动由开始行走运动、连续的行走周期运动，以及停止行走运动组成。在图 4.9.1a 中，仅处理两个行走周期无法到达目的地，而在图 4.9.1b 中，三个周期就会使人物超出目的地了。在图 4.9.1c 中，我们展示了在需要进行任意角度的旋转的时候所面临的同样的问题。在这种情况下，人物只有一个转动过程的运动。如果将此运动处理一次会不足，但是处理两次又有余。我们可以为人物提供更多的单位运动来解决此问题，但是其基本问题仍然是存在的。

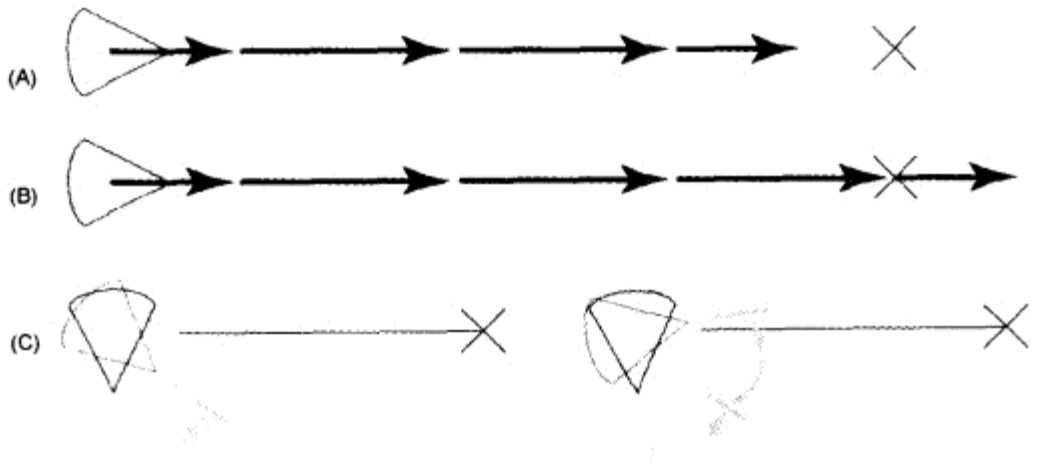


图 4.9.1 拥有固定运动的人物试图移到任意点与任意角度

（A）人物无法使用 1 个启动，2 个步长与 1 个停止到达目标（B）使用 1 个启动，3 个步长与 1 个停止会超出（C）使用 1 个或 2 个旋转都无法指向某角度

规划与修正

为了解决这个问题，我们可以在运行进行的过程中修正位移量或是旋转量。在图 4.9.2 中，我们将由程序从所有的运行中选出最接近此长度的一组值。我们将从出发点到目的地的距离进行划分，然后对每段运动进行修正以配合这段距离。现在，运动结束的时候就会正好停止在目的地上面了。

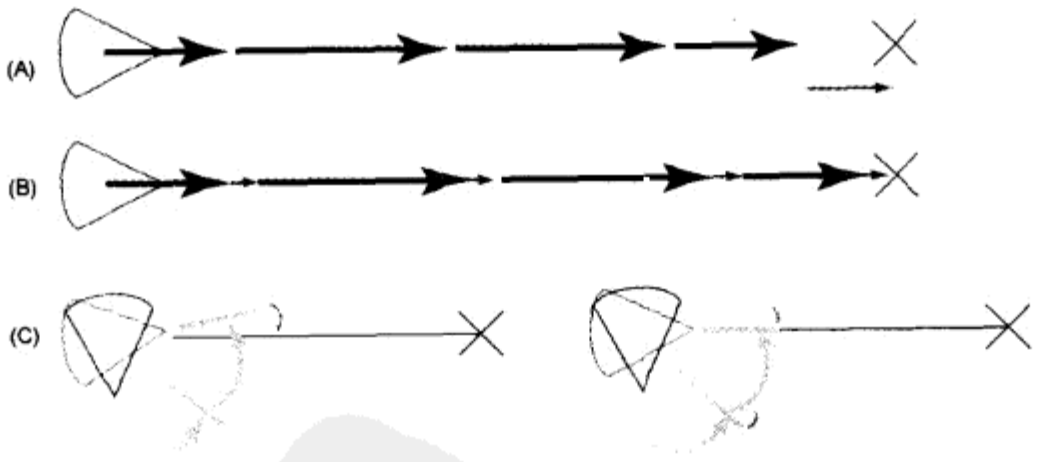


图 4.9.2 针对任意目标进行计划与修正（A）计算到达目标必需的调整

（B）对固定的运动步骤进行调整（C）对角度一样处理

为了能修改一段运动的位移量或是旋转角，我们只需要在动画播放的时候将原来的位移量或是旋转角乘以一个系数就可以了，这个系数的大小在 0 与 1 之间，而且这个过程必须进行得很平滑，这样就可以造成一个与原来数值不同的效果。然而，由于这个系数在运动过程

Download at www.Pin5i.Com

中会发生变化，例如在脚板暂时处于静止状态的时候，因此其结果将会导致贴地行走。

4.9.2　问题：运动之间的平滑过渡

不管其捕获对象的动作如何优异，从动作捕获而来的动画是很难正好停止在目的地上面的。如果仅仅关注于正确的位置这一个因素，这对运动的质量也会造成负面影响。如果捕获对象的动作非常正确，仅仅是由于其距离与所需的位移由少许的误差就抛弃了这段动画的话，那就有点太苛求与鲁莽了。在使用动作捕获的时候，我们需要对其捕获的运动进行平滑的变换以保证它不超出范围。即使使用手工绘制的动画，能很好地配合上这段距离，我们也同样需要这种变换的能力，在运动开始前就能预先算出结果。

1. 一种解决方法

如果我们能在一段运动结束的时候到下一段运动开始的时候计算出骨架的变换，其结果当然是最理想的了，这要比修正一段事先定死距离的运动要好得多。由于我们并不希望打断运动的连续性，因此必须在运动进行的过程中就将此变换计算出来。而为了得到最大的灵活性，我们不能在一段动画需要播放的时候再对之进行计算。基于这些约束条件，我们可以得到一个解决此问题的好方法，那就是修改每一段动画的开头部分，让其与上一段动画的结束部分一模一样，然后随着动画的播放，把此修正的影响逐渐变小。

在图 4.9.3a 中的人物展示了在上一段运动结束之时骨架的状态。图 4.9.3b 展示的是在下一段运动开始的时候骨架的状态，我们需要进行一次修正，将骨架从图 4.9.3b 中的状态转换为图 4.9.3a 中的状态。此外，在动画播放过程中，我们还需要将其转换到最初的位置上面去。

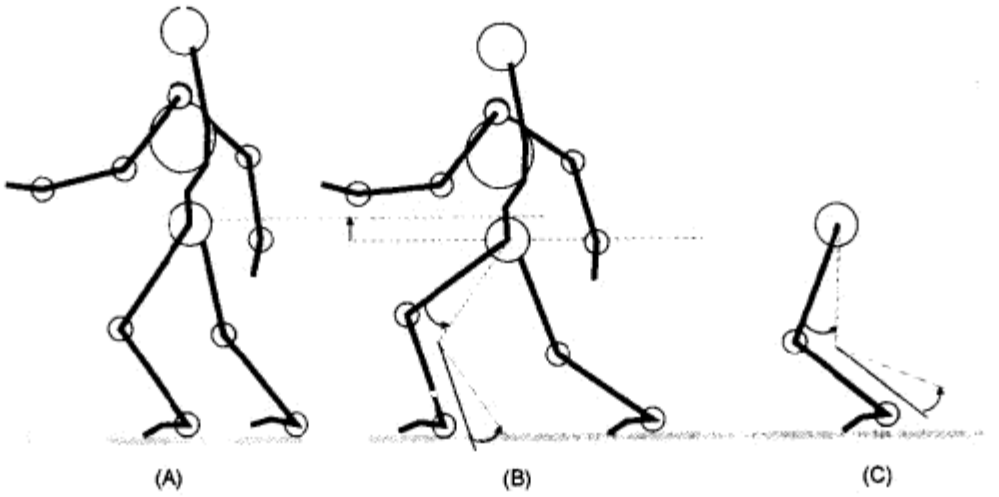


图 4.9.3　各步间的转换（A）前一动画结尾时的骨架（B）下一动画开始时的骨架（C）在后期进行修正会给一只脚带来问题

解决这种转换问题有一种通用的方法，那就是将骨架的状态保存为一组层次化的相对位置，然后再在这些位置中进行插值。我们一般使用四元数对这些位置进行球面线性插值（请参见[Shankel00]，它对四元数插值进行了详细的描述）。如果我们使用了这种插值方法，那么我们的形式就是在每个关节上使用一组四元数的偏移量，对之进行修正。为了在动画中将这种修正的影响逐渐减弱，我们将会在整段动画中把这些偏移量乘上从 1.0 到 0.0 的系数。在图

Download at www.Pin5i.Com

4.9.3b 中，我们展示了应该如何在关节处使用一组旋转，将骨架变换到期望的位置处。图 4.9.3 中，人物的原点是在臀部处的。由于原点的高度将随着运动而起伏，因此我们的修正中也应该包含一个对此高度值的修正。

2. 此方法的缺陷

这种直接插值方法的第一个缺陷就是对于任何一个可能影响脚板位置的修正来说，如果消除了这个修正，那么脚板的位置也有可能会受到影响。如果脚板的状态刚好是相对于地板的静止状态，那么我们就会得到贴地行走的效果。而这种贴地效果的时间长度是由原来时间间隔的大小以及修正被消除的过程占用的时间决定的。

第二个问题就是当修正器作用在运动晚期的骨架上时，又可能会带来一些意料之外的效果。在图 4.9.3b 中，我们展示了修正器对左腿施加旋转产生的效果。我们知道，当这些旋转应用到运动中第一帧的时候，我们将得到一个正确的骨架的位置，这是因为这个旋转是针对这个位置为了达到某个目的地设计的旋转。在图 4.9.3c 中，读者可以看到这个旋转在运动的后期作用在腿上产生的效果，就好像是脚板悬浮在了地板上面一样。这个脚板悬浮在地板上面的时间长度是由腿部的原点以及旋转共同决定的。如果我们对之进行修正，那么这个悬浮的动作就会被破坏，造成脚板悬浮在地板上的时间过长，或是甚至穿透了地板，或是对于地板来说脚板的位置被放在了错误的角度上，而在脚板应该静止的时候却会保持运动状态。骨架离原来的地点越远，此效果产生的误差也就越明显。我们可以将自己的修正衰减得更快一些，以此来改善这种情况，但是这将破坏动画的平滑度，使得贴地行走的效果在衰减阶段更加明显。因此，在我们解决两个运动之间的转换时，又再一次遇到了贴地行走的问题。

4.9.3 解决问题的一个架构：局部修正器与独立的插值系数

在动画播放的时候对之进行少量的修改以后，我们可以解决在人物运动中碰到的一些问题。然而，如果当脚板应该是静止的时候我们的修正器却改变了它的位置，那么问题就仍然没有得到最终的解决。

我们可以选择减少插值系数的时机。如果一个人物在运动中掠过了半空，那么我们需要延迟插值，直到人物的双脚都离开了地面，然后在人物着地的时候停止插值。这样，我们就可以解决在脚板仍处在地板上面的时候改变插值系数带来的问题。不幸的是，在大部分运动中，人物一般总有一只脚是放在地板上面的。

只有在脚运动的时候才对之施加影响

此处的技巧主要是在骨架的不同部分对之使用独立的修正器。这么一来，我们就可以在运动中的不同阶段消除每个修正器了。因此，为了在修正一个运动的同时不引进贴地效果，我们对每条腿使用了不同的修正器。在一段左脚先动，右脚紧跟的运动中，我们将在左脚前进的时候消除左腿的修正，然后一直等到右脚开始运动的时候在逐渐消去右腿上的修正。

我们可以对这个模式推而广之，应用到任何一个运动里面去，将判定何时进行每个修正器的过程自动化。在图 4.9.4 中，我们展示了一个只有两步的运动，在其中，每只脚都有自己的运动配置参数。我们可以从运动参数配置中直接得到每条腿的插值系数。在给定的一点，

Download at www.Pin5i.Com

此插值系数可以被设置为到这一点的运动距离，然后再除以整个动画中的总运动距离。

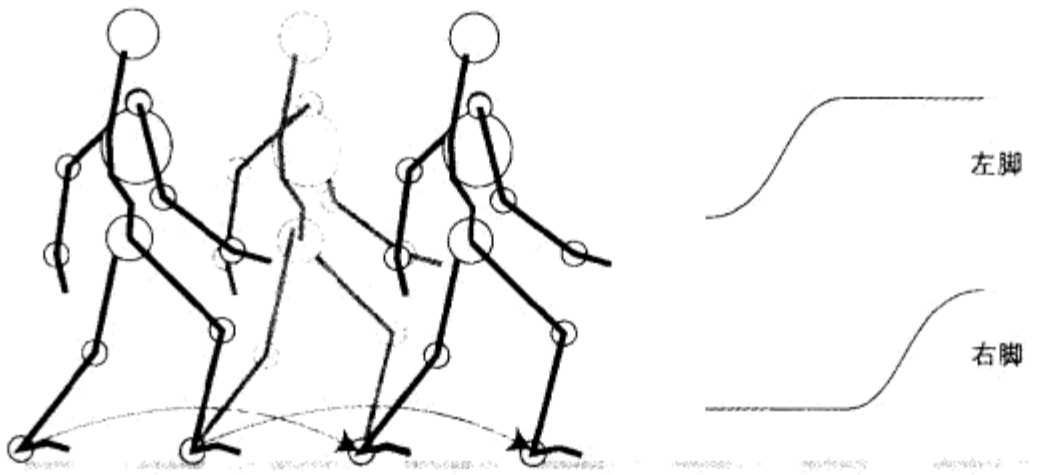


图 4.9.4　对脚有相应配置时的一个两步长的运动

如果在一段运动中有一只脚根本就没有运动，或是在此过程中的运动很少（这样插值的速率就会过快了），那么我们就可以选择一下，究竟是容忍在此段动画中出现贴地现象，还是要保持此修正值直到动画的结尾。

由于在骨架层次化运动传递中的误差积累以及原始动作捕获中的误差，脚板将会有轻微的偏移，因此我们需要设置一个脚板运动的门限值，这样就可以忽略掉任何在此门限值之下的运动了。

4.9.4　应用：处理任意目标的运动

对于运动到任意一个目标的问题来说，我们需要动画在开始的时候保持原样，在达到末尾的时候产生了一个均匀的偏移量。如果一开始在人物的原点上就加上了这么一个偏移值，那么一开始，我们就可以知道最终状态人物的腿脚所在位置了。然而，为了将我们的架构运用到此问题上，我们还需要使用某种方法对每只脚在不同的时候加上不同的偏移量。

解决之道就是要同时保持三个插值系数，其中一个系数用来控制一个全局偏移量，以将之作用在人物的原点上面，还有两个插值系数是从双脚的运动参数配置上面得到的，用来跟踪每只脚预期所达到的距离的。然后，我们将对每条腿使用一个修正器，根据此脚实际需要的插值系数，对全局修正器作用在其上的影响进行校正。在图 4.9.5 中，我们展示了应该如何把上述的技巧应用到图 4.9.4 中两步长的动画上面。在动画的中间，全局插值系数是 0.5，此时左脚已经结束了运动，因此它的插值系数是 1.0，而右脚此时尚没有运动，因此它的插值系数应该是 0.0。为了校正脚的位置，我们需要把左脚的系数修正为 0.5，右脚的系数修正为–0.5。

我们同样需要使用与全局修正器具有相同效果的旋转修正器或是位移修正器，这样当需要正值的时候我们就使用它们来生成这种效果，当需要负值的时候则使用它们来消除此效果。如果人物的原点处在其臀部处，那么我们就将使用一个旋转修正器对臀部进行旋转，这样就可以改变臀部关节的转向了。而位移修正器则要复杂得多，我们在实现的时候也有好几种选择。

Download at www.Pin5i.com

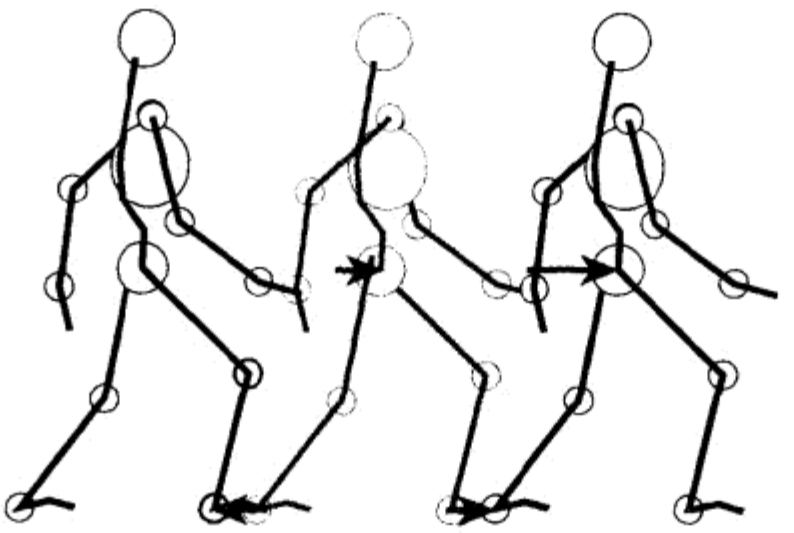


图 4.9.5 改变全局修正器。箭头指明了在一个两步长的运动中不同点处的修改。灰色的骨架表示的是中间步骤

4.9.5 位移修正器

位移修正器需要对脚板的位置设置一个偏移量，然后还要将腿部的其他部位进行相应的改动，同时确保不会改变臀部的状态。这是一个经典的逆向运动学（IK）问题（如果有兴趣，请参见[Talani00]）。使用了 IK 的方法以后，我们就可以解出腿部的约束方程，从而就可以将脚板放置到预想的位置了。我们必须有个思想准备，那就是 IK 有可能会失败。若是如此，在满足约束条件的前提下，我们可以将脚板放在离预想的位置最近的地方。在图 4.9.6a 中，我们展示了需要对脚板设置的偏移量。在图 4.9.6b 中，我们显示了如何使用 IK 得到这个偏移量在关节上所需要的旋转。

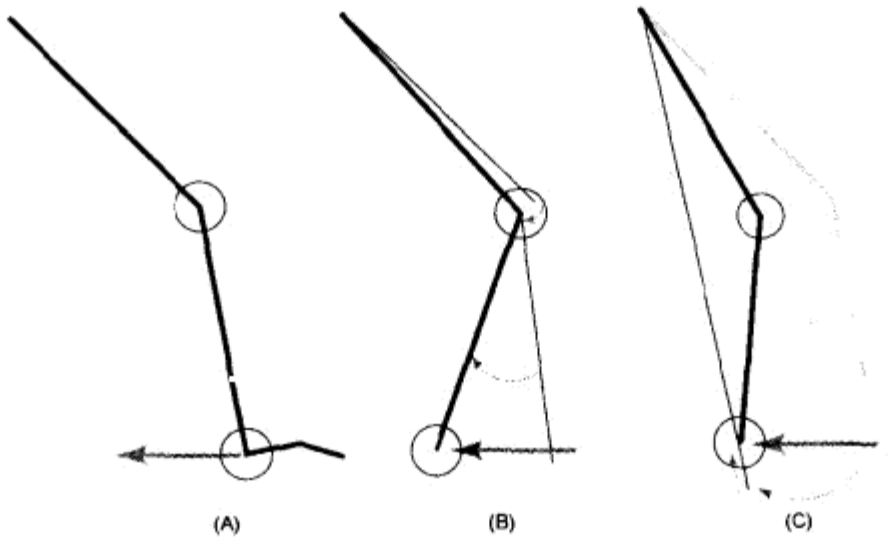


图 4.9.6 位移修正器。（A）要求的偏移（B）由 IK 得到的关节旋转
（C）更简单的方法是对整条腿进行旋转然后调整

我们还可以使用一个较简单的方法，那就是先将臀部关节进行旋转，将脚踝设置到期望的方向上，然后对腿部进行变换将脚踝放置到预期的位置上，如图 4.9.6c 所示。

乍看上去，IK 的解决方案似乎更好一些，因为它可以确保在整个动画过程中使得骨架的结构满足所有的约束条件，不过这种方法仍然有一些缺陷。如果仅仅使用 IK，无法保证帧之间的连续性。即使 IK 的目标位置变化很小，其引起的腿部变化也会很大，还会导致帧之间的不一致性。而这些帧之间的不一致性则会产生很不自然的姿态。

Download at www.Pin5i.Com

为了保证动画的一致性，最重要的一点就是修正器必须保证脚部位置的小变化只会导致腿部位置的小变化。为了在 IK 中加上这么一个约束条件，我们需要针对 IK 重新构造相应的方程。我们可以对问题进行一些修改，对腿部的角度进行某些修正，这样以满足脚部需要的偏移量。不幸的是，我们并不能确保在原始动画中腿部的位置能够满足 IK 的约束条件。

在实际情况中，我们更喜欢使用一种比较简单的方法（对臀部进行一次简单的旋转，然后再调整腿部的位置）。这样我们得到的动画会更加平滑，而且在腿部的每一点上我们都可以针对原始的位置进行更自然的调整。如果偏移量比较小，那么我们可以保证其修正值也比较小。而且在这种方法中，我们还可以保证膝部的角度不变。不过，由于我们没有对臀部加上约束条件，因此，在某些时候骨架可能会出现某些奇怪的姿态，而且虽然对腿部进行较小的调整不会被人发觉，但是只要调整稍大，就会出现古怪的形态。考虑到这些原因，我们必须避免使用较大的修正值，同时也要避免在人物静止的时候对值进行修正。

有的时候，即使调整很小也会导致出现蒙皮的问题。此时我们可以使用一个小技巧，那就是只对骨节伸展的方向进行调整，而不再在其他方向上进行缩放。

4.9.6　应用：变换

我们可以在动画的变换过程上使用同样的架构。我们可以对每条腿使用不同的修正器，这样就可以消除由于腿部静止而改变插值系数导致的问题了。不过，在人物原点运动的时候修正其腿部的角度仍然会为脚板带来一些问题。

其相应的解决方法是使用一个定位修正器。本质上来说，它与前面说到的位移修正器的作用是一样的，但是它还会影响到脚部的方向。在此处我们使用的不是偏移量，而是要为脚部在世界坐标系里面设置一个目标位置（或者也可以在人物的局部坐标系里面设置这个位置，只要它在动画播放的过程中不会发生变化就行）。这个针对定位修正器的目标位置是脚部在上一段动画里面结束的时候所在的位置。假设脚部被正确地放置在了这个针对地面的位置上，那么此修正器就能保证脚部的位置停留在这个位置上保持不变，直到它再次开始运动。一旦脚部开始运动，这个修正器就将被立刻消除。

使用了一个定位修正器以后，我们就可以确保脚部的位置会与上一段动画衔接正确了，但是它对腿部的其他部分并没有什么影响。即使臀部与腿部在变换期间不产生任何运动，如果腿部的位置在这些点上在变换期间没有对上号，那么动画就会显出抖动的效果。我们只要像前面一样对腿部进行球面插值就可以解决这个问题了，不过此时我们将会在结束的时候使用定位修正器对插值的结果进行处理。

4.9.7　其他细节

1. 单步动画

在一个两步运动中，两只脚在整个动画中是轮流运动的，这样，在动画播放的时候，我们就可以从容地将修正器从两腿上消除掉了。通过变换修正器，我们可以将此技术应用到单步动画上去，任何一个没有在动画结束的时候消除的修正器，在下一段动画开始的时候都可

Download at www.Pin5i.com

以被修正器处理。

2. 保持人物的运动状态

为了让人物看上去更加生动，我们必须让其保持运动。我们可以使用一系列的点上运动的方法让人物避免完全静止的站立状态。如果这些运动包括了脚稍微移动一下，则我们就可以让系统在此时把任何残余的修正器效应给消除掉。

玩家在人物停止运动的时候很有可能会观察到它姿态上不规则的地方。因此，人物停止的时候，正是一个绝好的消除剩余修正器的地方。如果游戏还为玩家提供了一个暂停键，那么我们也同样可以在暂停的时候使用这种技巧。

3. 限制修正器

退化的动作捕获将会导致超出目的地很多距离的结果，它将导致修正器出现很大的数值。如果在一段运动中一只脚完全不移动，也会导致在变换过程中出现很大数值的修正器。因此，最好我们对修正器的数值进行一下限制，这样就可以避免在这种情况下出现奇怪的现象了。

4.9.8 结论

为了解决人物运动中产生的问题，我们经常需要对动画进行修正。不幸的是，这些修正常常会打乱脚板与地板之间的相对位置，这样就会出现不真实的运动状态。通过使用独立的修正器，因此也针对每条腿使用了不同的插值系数，我们就可以在脚板运动的时候对每条腿进行修正，同时也避免了出现贴地行走的情况。

4.9.9 参考文献

[Shankel00] Shankel, Jason, "Interpolating Quaternions," *Game Programming Gems*, Charles River Media, Inc., 2000.

[Tolani00] Tolani, Deepak, et al., "Real-time inverse kinematics techniques for anthropomorphic limbs," 对应网址为 http://hms.upenn.edu/software/ik/ikan_gm.pdf, September 8, 2000.

Download at www.Pin5i.Com

4.10　可编程顶点着色器的编译器

Adam Lake
英特尔实验室
adam.t.lake@intel.com

在本文中，我们将探讨一下如何为可编程着色器实现一个编译器。为什么图形开发人员需要一个针对可编程着色硬件的编译器呢？这里有诸多的理由。有的时候，你需要确保针对可编程硬件编写的程序具有较高的可读性以及更好的可移植性。虽然底层的指令集有可能会变化，但是我们却可以避免重写这段程序。如果着色器是使用高级语言编写的，那么我们只需要换一个前端编译器，针对新的指令集使用新的代码生成器就行了。这样，我们就可以使用高级语言编写程序，与 OpenGL、DirectX，或是其他的内部软件渲染库中的着色器部分一起生成代码了。此外，如果使用一种高级的，类似 C 的语言来编写着色器的话，则我们在编写和阅读着色器代码的时候就更容易了。而且这么一来，对着色器库的改动也就更方便了。

在 CD-ROM 上，我们提供了一个简单的顶点着色器编译器的完整实现，并且带有相关的文档，说明了如何创建一个工作区间以生成编译器，此外还提供了一个例子，将 OpenGL 的光照方程转换为一个 DirectX 的顶点着色器。为了着重实效性，我们在本文中主要关注 DirectX8 中的顶点着色器实现。将来的实现可能会有所不同，但是我们提供的框架与基础架构仍然是适用的。

4.10.1　可编程顶点着色器

一个顶点着色器就是一段程序，它输入的是标准光照方程的参数，例如颜色、位置、法线，以及纹理坐标（一般被称为顶点流），然后将对最终结果进行计算，最后会将之提交到光栅化程序中去。材质与光照参数将通过一组常数寄存器送往顶点着色器，在这些常数寄存器中它们将保持不变，与此同时顶点着色器将处理一组特定的顶点流。DirectX8 拥有一个常用顶点着色器的实现，它包含有 96 个常数寄存器，12 个临时寄存器，以及 16 个顶点寄存器[Microsoft00]（参见图 4.10.1）。此程序的输出（最终的顶点属性）将被载入一组输出寄存器中，接下来，在经过了顶点着色器的变换与光照计算以后，它们将被传送到像素着色器中进行最后的显示。所有的

Download at www.Pin5i.Com

寄存器都有四个分量的矢量。单独的元素可以在一个指令中通过 x、y、z，以及 w 进行各分量的访问。在本文中，我们不会涉及到像素着色器，但是读者可以在参考文献中找到相关的资料。

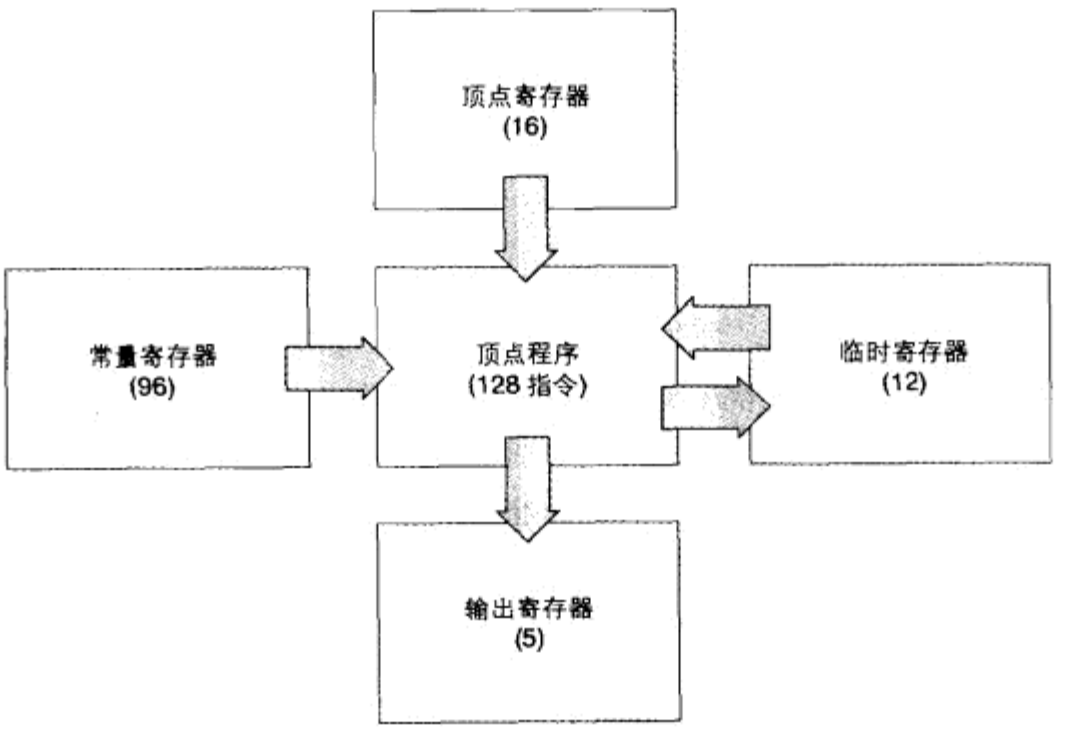


图 4.10.1 DirectX8 着色器的架构。有 16 个顶点数据寄存器，96 个常量寄存器，12 个临时寄存器，5 个输出寄存器。在 DirectX8 中，每个顶点程序不能超过 128 个指令。临时寄存器是唯一的读/写寄存器

如前所述，顶点着色程序将使用一组汇编指令对输入寄存器中的数值进行处理，并且算出输出寄存器中需要的值。它支持一些数学运算，例如 add、multi、max 以及 min，另外还有一组汇编指令，例如 dp3、dp4、logp，以及 dst（在[Microsoft00]与[Microsoft01]中列出了这些详细的指令集）。在代码列表 4.10.1 中，我们给出了一个示例的汇编程序。

LISTING 4.10.1 使用顶点着色汇编语言编写的卡通顶点着色器

```
;-----------------------------------------
; Vertex Transform
;-----------------------------------------
; rN is a temporary register
; vN is a vertex register
; cN is a constant register
; oPos and oTo are output registers
;-----------------------------------------
; Transform to view space
m4x4 r9, v0, c8;
; Transform to projection space
m4x4 r10, r9, c12;
; Store output position
mov oPos, r10;
;-----------------------------------------
; Lighting calculation (N.L dot product)
;-----------------------------------------
dp3 oT0.x, c20, v3;
```

Download at www.Pin5i.com

4.10.2　编译器

编译器由 6 个关键组件组成。它将把高级语言翻译成一组底层的指令集。此处，我们将介绍一下每个组件，然后在以后的各节中再给出更多的细节（请参见图 4.10.2）。

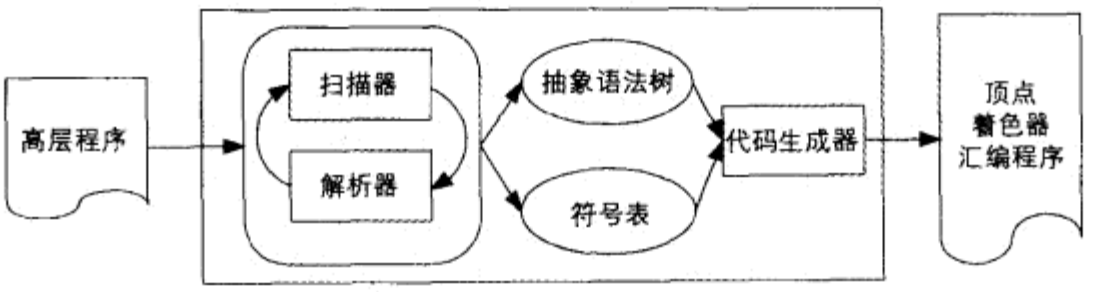


图 4.10.2　开始输入一个高层语言程序。首先，使用扫描器与解析器判定程序是否正确。在此过程中，我们将得到一个语法树与符号表。接下来，我们将其传给代码生成器，它将遍历语法树，给出一个顶点着色器的汇编程序

- **符号表**——它包含了所有的关键词与程序中使用到的变量，用来查找与保存某个变量所在的寄存器。
- **扫描器**——它与解析器共同工作，以决定接受或是拒绝某个程序。它将对输入扫描器的字符进行扫描，然后得到标识符或是符号。接下来我们将把这些标识符传送到解析器中，在符号表里面添加这些新的变量（*Lex* 是 *lexical* 分析器的简称。在实际环境中，我们将使用 *flex*，这个 gnu 版本的 Lex 生成我们的扫描器）。
- **解析器**——它将与扫描器共同工作，以决定接受或是拒绝某个程序。它将从扫描输入进来的标识符中建立语句，然后通过这些语句来创建一个抽象语法树（*Yacc* 代表的是 *Y*et *a*nother *c*ompiler *c*ompiler[Levine92]。在实际环境中，我们使用的是 *bison*，这个 gnu 版本的 Yacc，来创建我们自己的解析器）。
- **抽象语法树（AST）**——每当解析器生成一条语句，它就会将之添加到抽象语法树中去。这个抽象语法树将被解析器传送到代码生成器中。
- **代码生成器**——它将对 AST 进行遍历，根据 AST 中的数据生成顶点着色器的代码。
- **临时寄存器分配器**——它将管理代码生成器使用到的临时寄存器组。

与标准编译器（支持循环、分支，以及很多优化选项）相比，我们将要生成的编译器是很简单的。我们可以增加许多改进。这当然是将来的工作了，而且我们也鼓励你对 CD-ROM 上面的代码进行试验、优化和增强。

4.10.3　编译器的组成部分

我们的编译器被划分为 7 个部分：开发语言、扫描器、解析器、抽象语法树、符号表、临时寄存器组，以及代码生成器。

1. 开发语言

第一个任务就是需要确定我们将要对什么语言进行编译。由于我们希望用一种比较熟悉

Download at www.Pin5i.Com

的语言，因此我们将使用一个类似于 C 的过程式语言。最终，我们希望能支持参数传递、函数调用以及循环等特性。但是在第一个版本中，我们不会处理这些结构。我们也希望在将来能将其中的很多特性与关键词移植到一种渲染器的语言中来。虽然这些特性对于一个顶点着色器来说过于复杂了一点，但是在将来，针对完全的可编程图形流水线来说，渲染器语言几乎是不可避免的。

由于当前顶点着色器不支持循环或分支，因此我们也就不需要在语法中支持这些元素了。不过，我们仍然有一大组数值需要在顶点着色器中使用。我们将这些值在表 4.10.1 中都列了出来。其中的每个值都将被看成是常数，或至少在一个网格被渲染的时候不会改变。我们将把它们放置到常数寄存器组中的一个特定的寄存器中。我们将在应用程序与着色器中建立协议，确定将这些值具体放在什么寄存器中，应用程序与编译器将共同决定这个值的存放地点。例如，在我们的编译器中，我们将把第一条光照的方向，`LightDir0`，存放到常数寄存器 26 中。应用程序必须将这个光照方向放置到着色器的这个寄存器中，不然着色器就会读出寄存器中错误的数据了。编译器通过查找符号表将得知何处存放了 `LightDir0` 变量，然后将返回它所在的寄存器的索引号。在我们的例子中，这是一个在类 `CTempRegSet` 中的 `GetRegNumFromName(char *name)` 得到的。表 4.10.2 中列出了在我们的顶点着色器中使用的数学关键词。

表 4.10.1　　顶点着色器中使用的关键词

属性关键词			
顶点	光照	材质	输出
Pos	LightDir0~4	MatAmb	oPos
Normal	LightAmb0~4	MatDif	oColor
TexCoord0~4	LightDif0~4	MatSpec	oFog
	LightSpec0~4	MatShininess	oTexture
	LightPos0~4		

注意：在这个例子中，我们没有使用第二个色彩通道，也没有使用第二个纹理层，它们都是 DirectX8 输出寄存器中的一部分。

表 4.10.2　　数学关键词

数学关键词					
dot3	cos	clampTo1	floor	normalize	negate
dot4	sin	sqrt	ceiling	maxWidth0	

注意：在此处列出的数学关键词是除了+、-、*、/、(、以及)以外的操作符。请注意我们在此处只支持单元操作符。二元操作符显然可以作为一种将来的扩展。例如，max(x, y)。

2. 扫描器

扫描器也被称为“词法分析器”。这个词法分析器将与解析器共同工作以判定你输入的程序是否可以被接受。在此，我们的扫描器也是比较简单的。它能识别出符号表里面的标识符，以及在我们标识符列表中通过正则表达式识别的变量（那些不是关键词的字符串）、标点、浮点数字、数学操作符，以及注释。

在 CD-ROM 上，我们在本文的目录下提供了一个示例程序（参见 scanner.1）。我们使用了 Lex 来创建真正的 C 程序代码，然后将之进行编译连接得到这个扫描器。此过程的细节不在本文的讨论范围之列，但是读者可以参看其生成的 C 文件 lex.yy.c，从中领略到一点趣味。

Download at www.Pin5i.com

扫描器由三个部分组成。第一个部分是由{%与%}标识出来的，里面包含的是你的扫描器所需要的 C 的宏定义与类型定义。第二个部分由%%开始，也由%%结束，它包含的是扫描器的标识符匹配规则，其排列顺序是按照优先级从高到底排列。最后一个部分是从最后的%%开始的，它包含的是C/C++代码，这些代码可能会被上面的匹配规则使用到。一般来说，它们包含查找符号的函数、进行错误处理的函数，或是进行错误报告的函数。

为了从文件 scanner.l 中生成词法分析器，或说是扫描器，我们使用了 GNU 的工具 *flex*，读者可以从[Streett02]处得到它。在 CD-ROM 中没有带上它，你需要自行下载这个工具，编译这个例子编译器。

3. 解析器

解析器将与扫描器共同工作，以决定接受还是拒绝某个程序。扫描器的工作是识别标识符或符号，解析器的工作是接受或是拒绝一组符号，或是语句。例如，下述表达式 `AmbientLight = AmbientMaterial * LightAmb0;`就是一个语句。它由六个标识符组成——变量、算术操作符，以及语句末尾的分号。当扫描器扫描到这些符号的时候它首先将把 AmbientLight 识别成一个新的变量，将其添加到符号表中，然后将之传给解析器。解析器不能将仅仅一个变量名当成一条语句，但是它有一些以这个变量名开头的语句，因此它将会把这个变量名保存在一个符号栈中。在这个例子中，假设符号栈开始是空的，而且现在我们只需要识别这一条语句。接下来，=也将被识别出来，然后被放进去。最后，整个表达式都将被作为一组符号识别出来，因为它能与语言定义中的一种语句结构匹配。在此处，它匹配的语句是 Expr T_MULTIPLY Expr。所有这些元素都将被弹出栈外，然后解析器就会对下一条语句进行同样的处理了。如果程序结束的时候不只开始的符号保存在了栈底，那么我们就知道词程序在语法上面有问题了，接着整个解析过程就会停止下来。这个开始的符号是解析器文法描述中的第一个符号。在我们的例子文法文件 yaccer.c 中，这个开始的符号是 *Program*。

解析器的结构也是由三个部分组成，这与扫描器很类似。第一个部分是由{%与%}为首尾的，它包含的是与 C 相关的数据结构，这些数据结构在文法的动作部分是必需的。第二个部分包含在一对%%符号中，被称为文法。文法是一组解析器必须识别出来的语句。在文法中的每一条语句都有一个对应的动作。一个动作是一段 C 的代码，它描述的是当对应的语句匹配成功了以后将会采取什么行动。换句话来说，如果在栈上的一组符号中最近识别出来的符号（或是标识符）与一条语句匹配了的话，则解析器就会触发这组动作。在我们的解析器中，相应的动作就是将这条语句添加到抽象语法树中，如下所示：

```
Expr: Expr T_PLUS Expr
    {
        $$ = new CastNode("Expr");
        ((CastNode *)$$)->addChildNode(0,(CastNode *)$1);
        CastNode *pNewNode = new CastNode("T_PLUS");
        ((CastNode *)$$)->addChildNode(1,(CastNode *)pNewNode);
        ((CastNode *)$$)->addChildNode(2,(CastNode *)$3);
        }
```

Download at www.Pin5i.Com

在这个例子中，第一行包含的就是匹配规则。它说明的是，如果我们发现了两个表达式中间有一个加号的话，我们就可以把这些元素弹出符号栈，将栈中数据归纳为一个单独的标识符 Expr。除了进行这个归纳以外，我们还将执行在{和}之间的 C 代码。其中$$表示的是最左边的标识符 Expr。下面的$1、$2 以及诸如此类的符号表示的是右边的（RHS）标识符，其顺序是从左到右排列的。在此处，我们在 AST 中创建了一个新的节点，然后在这个节点中添加上了子节点，这些子节点对应于 RHS 的标识符。其中(CastNode *)这个类型强制转换是必须的，因为我们把每种这样类型的标识符都声明为了 void 类型（若想得知更多的信息，请参见[Levine92]）。

上述扫描器与解析器的描述只是为了向你提供足够的信息，这样，对在顶点着色器的编译器中如何使用这些工具就可以有个基本的理解了。如果你还想添加新的符号、改变符号的名称，或是改变它们对应的寄存器，上面提供的信息就应该足够了。不过，如果任何读者想对编译器作一些较大的修改（添加功能，例如类型检验，或是使用更复杂的符号表），我们则鼓励你去阅读一下 Lex 与 Yacc 的用户手册[Streett02]。对这些内容的描述另外还有一本经典的著作，是由 John Levine 写的[Levine92]。如果你想建构自己的编译器，则需要对上述的三者都有相当的了解才行。

4. 抽象语法树

抽象语法树是由解析器在解析过程中检验语句生成的那些数据结构。一旦某条语句被认为是合法语句，我们就会将之添加到 AST 中。当解析结束的时候，AST 就会被传送到代码生成器中去，后者将对语法树进行遍历，为树中的每条语句生成相应的代码。我们将之称作抽象语法树，这是因为它并没有包含文法中所有的语法元素[Aho86]。例如，分号就没有存储在树中，因为在生成代码的时候并不需要它们。与之形成对比的是，一个具体语法树则应该包含所有被解析出来的标识符。

5. 符号表

符号表中保存的是编译器使用到的符号。在初始化的时候，它将填充上所有的关键词与语言中用到的符号。在编译对程序进行解析的过程中，新的变量将被添加到符号表中。如果符号已经存在了，我们将在符号表中添加一个指针指向这个符号（参见表 4.10.3）。

表 4.10.3 一个符号表项

名称	类型	作用域	标识符	寄存器类型	寄存器索引	寄存器分量(x, y, z, w)	指向下一项的指针
“LightDir1”	“KeyWord”	0	T_STRING	eRegTypeConst	37	eRegCompAll	NULL

注意：一个表项包括符号的名称、符号类型、它的作用域，以及在扫描器中表示这个符号的标识符。一旦符号指定了一个寄存器以后，它还将包括寄存器索引、寄存器的类型、以及寄存器四个组成部分的情况。最后，我们还将放置一个指针，指向符号表中的下一个符号。在 CD-ROM 上的 CSymbolTable.cpp 中包含有一个符号表的例子。

Download at www.Pin5i.Com

6. 临时寄存器组

在图 4.10.1 中，我们展示了一组临时寄存器，它们用来在生成汇编指令的时候作为工作空间。我们创建了一个 CRegister 的类，它可以将一个寄存器标志为空或是满。通过这个类，我们可以创建一个 CTempRegisterSet 类，这个类用来管理临时寄存器。我们对寄存器可做的操作为：将其标志为空或是满，MarkAsFilled()或是 MakrAsEmpty()；要求得到一个特定的寄存器，RequestSpecificReg()；或是得到下一个寄存器，GetNextAvailableTempReg()。这个类用在代码生成器中。

7. 代码生成器

代码生成器是这个编译器的核心部件。代码生成器输入的是拥有正确文法的一个 AST。它将对 AST 进行遍历，为树中的每条语句生成相应的代码。在这一点上可以使用两种优化方法：第一种，也是最明显的一种，就是在遍历树的时候执行各种算法，每种算法进行一种优化；第二种就是生成一个中间语言描述，然后对这种中间语言执行算法。在[Muchnick97]中对这些技术进行了充分的讨论。本文的目的是要为了编写编译器提供一个出发点，然后你就可以自己对之进行扩展，为之添加增强特性，针对特定的平台进行优化了——在这里，你就可以开始为自己量身定制了！

4.10.4　结论

为了看到这些结果，你可以使用 CD-ROM 上的着色器作为输入编译这个示例顶点着色器的编译器。现在，假设你正使用一个全新的着色器，你是愿意调试高级语言呢，还是调试其生成的汇编语言呢？对某些应用我们总是希望能更加接近硬件，但是很多情况下，如果能有一个编译器来作这件事情会方便得多。这个编译器可以用来为使用 DirectX8 的应用程序生成着色器。和所有的项目一样，你仍然可以对之进行诸多的改良，而我们也欢迎广大读者为此做出的贡献。

4.10.5　致谢

如果没有英特尔实验室图形与 3D 技术组（G3D）的管理者的支持的话，这项工作是绝不可能完成的。在此，我特别感谢 Carl Marshall 以及 Stephen Junkins，他们不断地鼓励我，激励我的前进以完成这项任务，并且向我提出新的挑战。同样感谢 Jeff Lander 与 Mike Maxpherson，他们阅读了早期的作品，并提出了宝贵的反馈意见。

4.10.6　参考文献

[Aho86] Aho, Alfred, Ravi Sethi, and Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools,* Addison Wesley, 1986.

Download at www.Pin5i.Com

[Lake01] Lake, Adam, "Cartoon Rendering Using Texture Mapping and Programmable Vertex Shaders," *Game Programming Gems 2*, Charles River Media, Inc., 2001.

[Levine92] Levine, John, Tony Mason, and Doug Brown, *Lex and Yacc*, O'Reilly and Associates, 1992.

[Möller 02] Möller, Tomas and Eric Haines, *Real-Time Rendering*, Second Edition, A.K. Peters, Ltd., 2002.

[Microsoft00] *Microsoft DirectX 8.0 Software Development Kit*, 对应网址为 http://www.msdn. microsoft.com/downloads, 2000.

[Microsoft01] *Microsoft DirectX 8.1 Software Development Kit*, 对应网址为 http://www.msdn.microsoft.com/downloads, 2001.

[Muchnick97] Muchnick, Steven S., *Advanced Compiler Design and Implementation*, Morgan Kaufmann, 1997.

[Olano00] Olano, Marc "Interactive Shading Language, Language Description," Course Notes on Approaches for Procedural Shading on Graphics Hardware (SIGGRAPH 2000).

[Proudfoot01] Proudfoot, Kekoa, William R. Mark, Svetoslav Tzvetkov, and Pat Hanrahan, "A Real-Time Procedural Shading System for Programmable Graphics Hardware," Conference Proceedings (SIGGRAPH 2001).

[Streett02] http://www.monmouth.com/~wstreett/lex-yacc/lex-yacc.html. Flex 与 bison 向 Win32 平台的移植，包含了源代码以及文档。

[Upstill89] Upstill, Steve, *The Renderman Companion: A Programmer's Guide to Realistic Computer Graphics*, Addison Wesley, 1989.

[Woo99] Woo, Mason, et al., *OpenGL Programming Guide*, Third Edition, Version 1.2, Addison Wesley, 1999.

Download at www.Pin5i.Com

4.11 画板光束

Brian Hawkins
Seven Studios
winterdark@sprynet.com

对于很多游戏的图形部分来说，眩目的特效是一个非常关键的要素，而其中会反复出现的一种特效就是激光的光束。光束可以在很多地方得到应用，例如航空战舰的火炮或是魔咒效果，而且固态的光束可以用来表现建筑物中的结构性要素。

有好几种方法可以用来创建光束效果，每一种都有其自身的缺陷。最简单的方法就是在光束行进的路线上使用多个球面画板（spherical billboard）精灵来表现此效果。但是这么一来，我们就需要使用很多精灵来创建即使是很少的几条光束，而且在很多时候其视觉效果都不怎么好，例如在生成较大的光束的时候。更实际的技术是使用一个旋转的矩形围绕光束的轴线行进，使之尽量占据镜头的画面。这样，我们就可以创建一个从侧面来说很真实的光束效果了。但是如果直面正对光束的话，则光束的多边形特性就会显现无疑。

我们可以将这两种方法结合以得到一个更好的方法，这种方法既高效，也可以保证在大部分情况下拥有较好的视觉效果。我们将在镜头的两端采用两个三角形，然后在光束中采用更多的三角形。在本文余下的部分中，我们将更加详细地描述如何设置三角形的位置与纹理以创建一个具有三维效果的光束。

4.11.1 矩阵

对于每个端点来说，我们都要为其提供一个画板矩阵。要计算出这个矩阵，我们需要得到镜头—世界坐标系的矩阵，以及光束两个端点的位置。一个在镜头坐标系里精灵的普通画板矩阵与镜头—世界坐标系矩阵拥有同样的方向，但是其位置却不一样。然而，对一个条光束来说，其 billboard 矩阵应该与光束的屏幕方向保持一致。因此，前向的矢量应该与镜头—世界坐标系矩阵保持一致，但是上面与右面的矢量则需要进行修改。首先，我们需要从两个端点得到光束的方向矢量：

$$\mathbf{B}=\mathbf{B}_1-\mathbf{B}_2$$

接下来，我们将计算出一个方向矢量，它被称为视点矢量（eye vector），其方向是从镜头指向光束的一个端点处：

Download at www.Pin5i.Com

$$\mathbf{E}=[M_{03} \quad M_{13} \quad M_{23}]^{\mathrm{T}}-\mathbf{B}_1 w$$

然后，我们将对视点矢量与光束矢量进行叉乘，从而得到一个矢量，其方向与光束矢量在屏幕空间是垂直的：

$$\mathbf{P}=\mathbf{B}\times\mathbf{E}$$

最后，我们将把这个垂直矢量与镜头—世界坐标系的前向矢量作叉乘，然后将此结果进行归一化，得到画板矩阵的上端矢量：

$$\mathbf{F}=[M_{00} \quad M_{10} \quad M_{20}]^{\mathrm{T}}$$

$$\mathbf{U}=\frac{\mathbf{F}\times\mathbf{P}}{\|\mathbf{F}\times\mathbf{P}\|}$$

此时，我们就可以很容易得到右面的矢量了：

$$\mathbf{R}=\mathbf{F}\times\mathbf{U}$$

现在，每个端点的画板矩阵都可以从方向矢量与端点位置中计算得到：

$$\mathbf{M}_1=\begin{bmatrix} F_x & U_x & R_x & B_{1x} \\ F_y & U_y & R_y & B_{1y} \\ F_z & U_z & R_z & B_{1z} \\ 0 & 0 & 0 & 1 \end{bmatrix} \text{与} \mathbf{M}_2=\begin{bmatrix} F_x & U_x & R_x & B_{2x} \\ F_y & U_y & R_y & B_{2y} \\ F_z & U_z & R_z & B_{2z} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.11.2 顶点

在图 4.11.1 中，我们显示了两个用来组成光束端点的三角形，其半径由 s 决定，它是经过了适当的画板矩阵的转换以后得到的。顶点 $\mathbf{V}_1$、$\mathbf{V}_2$ 与 $\mathbf{V}_3$ 是由 $\mathbf{M}_1$ 变换而来的，而 $\mathbf{V}_4$、$\mathbf{V}_5$，以及 $\mathbf{V}_6$ 则是由 $\mathbf{M}_2$ 变换而来的。接下来，我们就可以使用顶点 $\mathbf{V}_2$、$\mathbf{V}_3$、$\mathbf{V}_4$ 与 $\mathbf{V}_5$ 来得到其他组成光束的两个三角形了。这两个端点处的三角形接下来将转向镜头，这当然也将导致另外两个三角形转向镜头了。

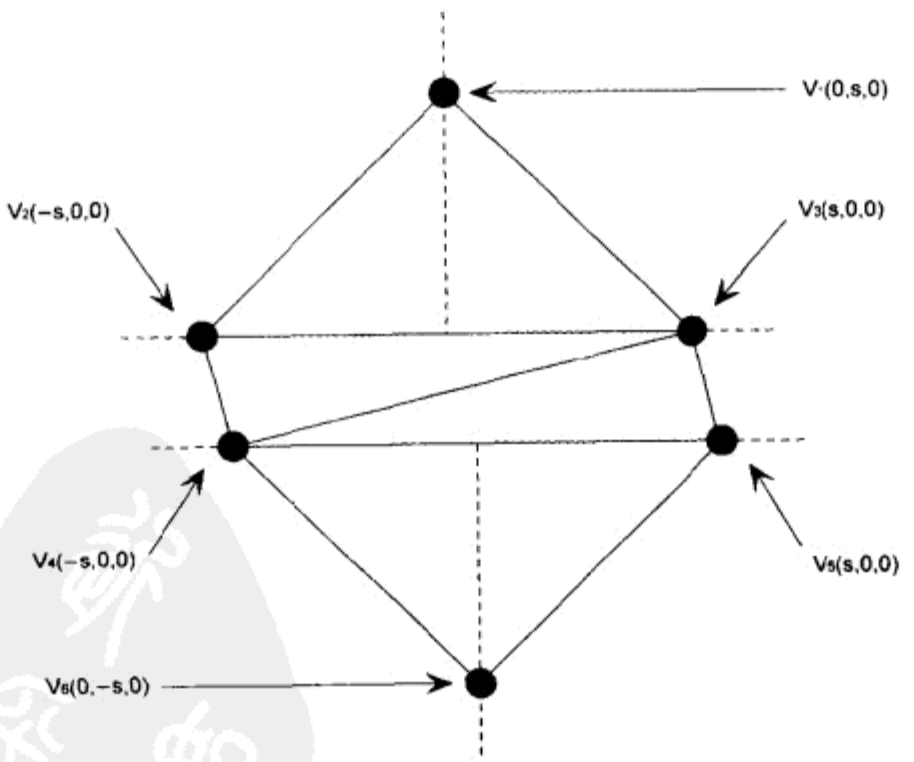


图 4.11.1 转换之前的顶点坐标

Download at www.Pin5i.Com

在大部分当代的机器架构中，我们还可以进行更多的优化，将这些三角形做成三角形条带传递进去，这样就可以减少每条光束需要传递的顶点数量了，其减少值大概在 12 到 6 之间。为了达到这个效果，我们需要从 $\mathbf{V}_1$ 到 $\mathbf{V}_6$ 把顶点传递进去，而那些共享的顶点则需要具有一样的属性。如果考虑到纹理映射的话，第二个要求是非常重要的。

4.11.3　UV 映射

在图 4.11.2 中，我们展示了在三角形条带生成过程中常用的两个纹理映射方法。顶点 $\mathbf{V}_1$ 到 $\mathbf{V}_6$ 将对应于纹理 $\mathbf{T}_1$ 到 $\mathbf{T}_6$。当选择使用什么设置的时候，我们需要考虑到两种方法：第一种方法如图 4.11.3a 所示，它与光束的真实形状更加匹配，因此，其图形上的误差也比较小。不过，它得到的纹理空间会被分成两个三角形，这样就难以为其他纹理所用；第二个方法如图 4.11.2b 所示，它在纹理空间的使用方面效率更高，但是它扩展纹理的方法却可能会在最终的图像上面造成视觉方面的瑕疵。对于大多数光束来说，这个瑕疵都是不可见的，因此，我们将采用第二种方法，除非真的遇到了视觉方面的问题。

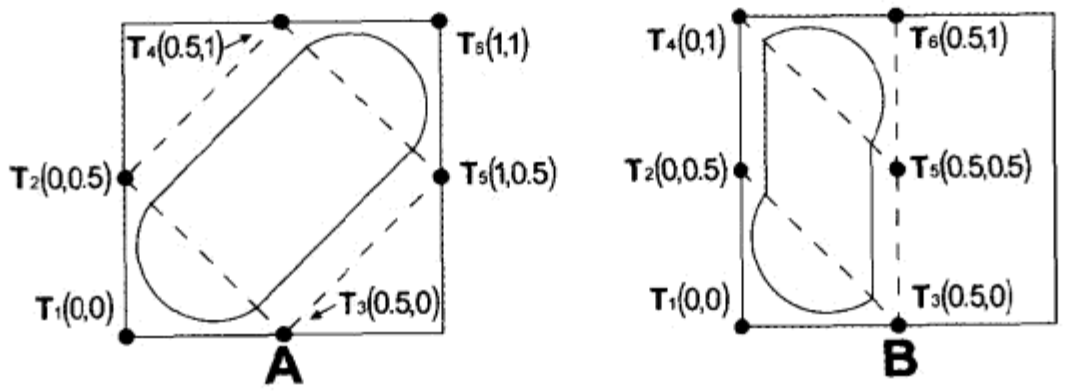


图 4.11.2　有纹理坐标的纹理的例子

4.11.4　结论

我们在此处阐述的算法提供了一个高效渲染光束的通用方法。读者可以在保持预期视觉效果的同时，利用特定平台架构的特性对算法进行进一步优化。下面，就等着看到眩目的效果吧。

Download at www.Pin5i.Com

4.12 针对等测引擎的 3D 技术

Greg Snook
Bungie Studios
gregsn@microsoft.com

等测引擎是二维图形技术功成身退之后为游戏界留下的一个宝贵财富。在本文中我们将提出一些 3D 的方法来增强一个本质上基于精灵的显示系统，它将尽力保持基于精灵图形的特性。虽然你可以使用 3D 建模来表现游戏中的主要对象以达到同样的视觉效果，但是本文中阐述的思想能够通过添加了一点技巧，保持原有的对精灵的使用，并且提供与 3D 建模一样的效果。这个概念对于其他的 3D 引擎也可能会很有用，读者可以使用它来代替平面的画板精灵，也可以使用它来表现距离较远分辨率较低的对象。

假设在一个等测（isometric）游戏引擎中，每个游戏的对象都是使用某些 3D 建模软件包创建的。游戏世界本身被划分为矩形的立方体，我们可以称之为“单元”。每个单元在游戏中使用一个 2D 的图像贴图表现。这些图像是单元中的内容以正交投影的方式映射到单元的前端表面的（请参见图 4.12.1 中生成的投影纹理）。由于我们在整个游戏世界中观看的内容都是通过一个正交镜头得到的，因此对游戏世界的渲染就转化为找出哪些单元应该显示在屏幕上，接下来我们就可以将每个单元内容的图像画到相应的屏幕坐标上面去了。在显示这些单元的时候我们不需要进行缩放或是角度的校正，因此我们的 2D 投影就可以完全代替原来 3D 的模型了。

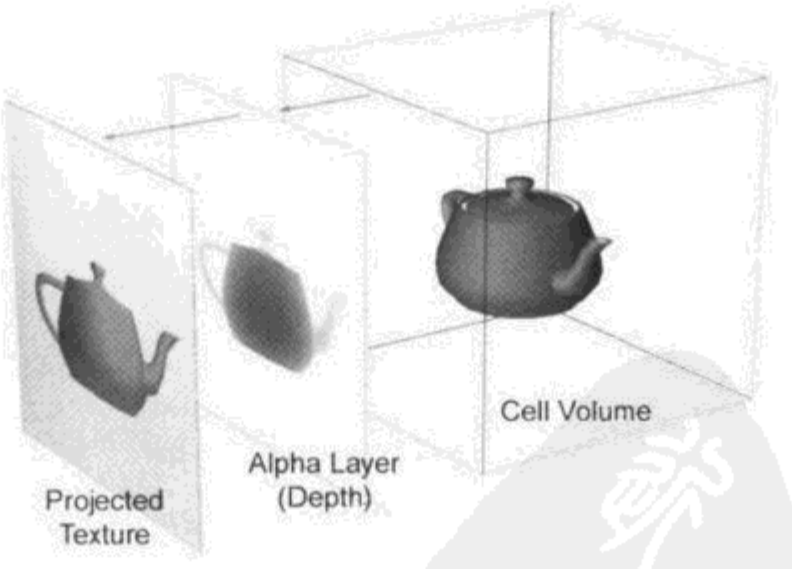


图 4.12.1 原始单元的正交投影生成了纹理图像与逐像素的深度信息，它们是保存在 alpha 通道的灰度中的

Download at www.Pin5i.Com

为了适应一个 3D 的环境，而且在其中角度表现正确无误，我们还需要添加一些 2D 投影缺乏的东西——深度信息。这样，我们就可以将 2D 图像转化为近似带有体积信息的表现形式。我们将探讨一些方法以在 3D 环境中正确地渲染出带有深度信息的图像来。

4.12.1　进入第三个维度

进入三维世界的第一步就是要修改我们的渲染方法。在一个 3D 系统中描画 2D 图像最常见的方法就是通过使用画板进行绘制。它们是一组平面的几何形状，我们将在其上映射出每个单元图像的纹理信息。如果我们在游戏中对每个单元图像都使用一个画板，而场景又是经过正交镜头渲染的，那么最终显示效果就会与原始的 2D 效果几乎完全一样了。现在，画板已经代替了传统的绘制方法，并且能够生成同样的效果。如果使用一个非正交的镜头来处理距离的缩放与视差效果，那么画板缺乏深度的缺陷就会暴露无遗。

为了在画板中能生成一种带有深度的影像，我们需要记录下每个纹理的体积特征。一定要记住，在画板上放置的每幅图像都是为了表现对象在一定体积空间中的状态的。这实际上就是将单元中的内容投影到一个平面上去。我们可以记录下每一个像素的深度信息，它代表的是从投影面到与相应单元内容相交的那一点的距离（请参见图 4.12.1）。下面我们将探讨用来从画板图像生成体积信息的三种方法，它们都将使用到上述每个像素的深度信息。另外，在计算每一个像素的纹理凸凹贴图时也是很有用的，这个技术可以与下面描述的方法套用。

4.12.2　方法 1：画板越多，效果越好

高程地图法最早是由 Sim Dietrich[Dietrich00]提出的，这种方法使用了多层画板来表现具有立体信息的空间。每个画板都会使用单元体积里面的一个分片进行纹理化，这样将之联合在一起，我们就可以在镜头前面模拟出原始的几何状态了。Dietrich 同时指出在一个单元组里面，所有的画板都可以使用同一种纹理进行映射，此纹理可以包含在 alpha 通道里单元中的深度信息。通过进行硬件支持的 alpha 测试以后，单元中纹理的各个深度层都可以绘制到每个画板上面去。这样一来，我们就可以使用很少的多边形与单独的纹理来生成伪立体的质感了（请参见图 4.12.2）。

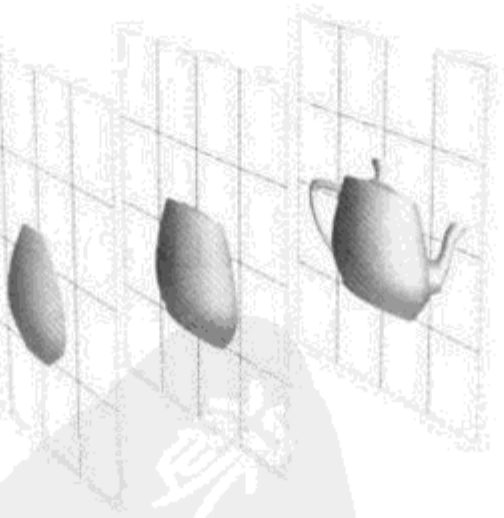


图 4.12.2　多深度切面视图，纹理被映射到不同层的画板上以模拟一个固态物体

Download at www.Pin5i.Com

这个深度值是放置在每个顶点散射颜色的 alpha 部分中的。只要在纹理化的过程中实行一个减法运算，我们就可以从纹理采样的 alpha 通道中得到这个值。然后我们可以对硬件进行设置，让其进行一个 alpha 测试，以将减法运算后所有小于零的像素抛弃掉。这么一来，我们就可以确保只有那些具有比顶点颜色要大的深度值的纹理才会被绘制出来。

在 DirectX8 中，这个设置只需要改变以下几个简单的渲染与纹理状态参量：

```
SetRenderState(D3DRS_ALPHATESTENABLE, TRUE);
SetRenderState(D3DRS_ALPHAREF, 0);
SetRenderState(D3DRS_ALPHAFUNC, D3DCMP_GREATER);

SetRenderState(D3DRS_ALPHABLENDENABLE, TRUE);
SetRenderState(D3DRS_SRCBLEND , D3DBLEND_ONE);
SetRenderState(D3DRS_DESTBLEND , D3DBLEND_ZERO);

SetTextureStageState(0, D3DTSS_COLORARG1, D3DTA_TEXTURE);
SetTextureStageState(0, D3DTSS_COLORARG2, D3DTA_DIFFUSE);
SetTextureStageState(0, D3DTSS_COLOROP, D3DTOP_SELECTARG1);

SetTextureStageState(0, D3DTSS_ALPHAARG1, D3DTA_TEXTURE);
SetTextureStageState(0, D3DTSS_ALPHAARG2, D3DTA_DIFFUSE);
SetTextureStageState(0, D3DTSS_ALPHAOP, D3DTOP_SUBTRACT);
```

视镜头类型（镜头使用来观看场景的）的不同，你可能需要从某些更大的角度上将单元重画一遍。高程地图在此时仍然可用，但是随着观看方向与画板越来越趋近平行，可能会在轮廓上出现某些误差。为了弥补这一错误，我们需要额外角度上的画板以应付更大的视角。由于每个画板需要的深度值存储在其顶点的散射颜色中，因此我们可以使用 Gouraud 着色方法对顶点颜色进行处理，这样就可以让每个分片在任何角度都能应付自如。

使用高程地图技术的另一个缺陷就是它把深度当成了高度来处理。每个像素只有一个高度值用来定义图像的体积要素，这样从某个角度看来就会使得最终的对象外貌凸显了出来。为了解决这个问题，我们还需要另外一个深度地图来定义对象的尾部位置，这样我们在一个像素上面就有两个值了：一个定义对象在前端的深度；另一个定义对象在后端的深度。

当我们的镜头放置在单元立方体对面的时候，这种方法生成的样子与我们当前的深度纹理是一样的。在渲染过程中，我们可以使用一个像素着色器进行第二次减法运算。不过这次我们使用的是第二个纹理里的深度信息。如果两个减法操作得到的都是正数，那么这个点的值就在前端和后端深度值之间，它将会被输出到屏幕上。如果有一个结果是负数，那么这个点就是处在对象的前面或是后面了，这样它就必须被抛弃掉。在CD-ROM上面的例子程序里面提供了这个技术的实现。

4.12.3 方法 2：变换纹理

虽然这种方法不是将单元的内容绘制到画板上的最快的方法，但是通过使用变换浮雕纹理（warped relief texture）的方法，我们可以避免使用额外的几何信息。这种方法的缺陷就是

Download at www.Pin5i.Com

需要进行预处理，这样在每次视角变换的时候就可以变换画板的纹理了。由于在性能上面这个开销会比较大，而且要求每个单元都有一个纹理，因此此方法只对游戏中罕有的对象有效。不过，对于独一元二的对象来说，例如玩家的人物或是老板级的关主，却可以从这种浮雕纹理中得到极强的深度感。

在此处，浮雕纹理的名词是从 Manuel Oliveira 与 Gray Bishop 借用的，他们在 Chapel Hill 的北卡罗来纳大学发明了这项技术[Oliveira99]。其思想基于如下的事实，即纹理中包含了单元中每个像素的深度信息，我们可以将其再度投影到画板平面上以模拟适应新的视角。在给定了镜头位置以后，纹理中的每个像素都可以进行偏移，基于其原始的纹理位置与深度值，在纹理中得到一个新的位置。最终纹理将在所有的像素偏移了以后生成，它给出了从观看者的位置上应该看到的单元内容（请参见图 4.12.3）。

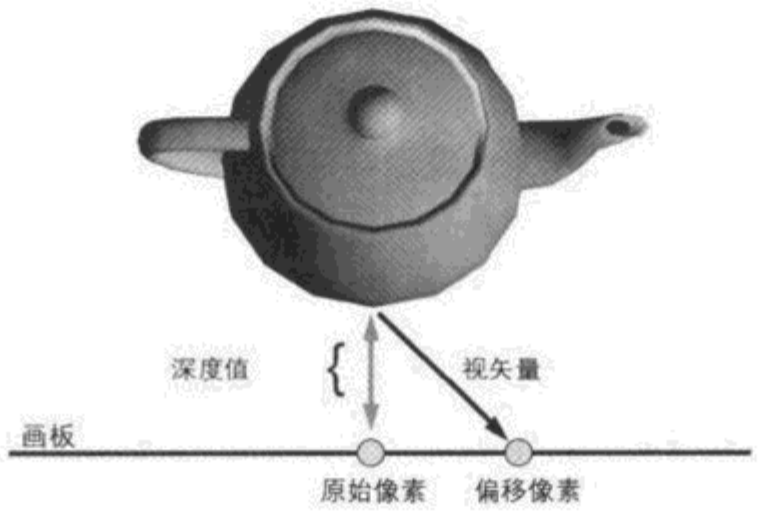


图 4.12.3　俯视图，展示了将像素再度投影到画板平面上。偏移位置由原始象素（x，y）在画板上的位置，以及编码在纹理的 alpha 通道中的深度信息计算得出

其麻烦就在于这是一个由两个过程组成的操作。首先，像素需要在纹理的 u 轴上进行位移，然后再在 v 轴上位移以得到最终的位置。从速度方面考虑，v 方向的位移一般都会被忽略掉，同时仍然能生成较真实的效果。大部分等测游戏对象（通常是瘦高的对象，例如双足）的自然特性与屏幕的高宽比可以帮助消除这个误差。在提供的例子程序里的纹理变换中，我们只对 u 进行了位移。

当进行 u 方向位移操作的时候，有可能会在图像中产生一个空洞（在此处没有像素的位移发生），而且消隐问题（在此处有不只一个像素偏移到此）也可能会发生。为了处理消隐的问题，纹理必须沿着像素位移的方向进行处理。这样我们就可以确保离镜头更近的像素覆盖掉较远的像素。例如，如果画板在镜头的左边，则像素就会向右位移。在这种情形下，纹理必须从左到右进行处理，这样就可以保证离镜头更近的（在图像中更右边的）像素覆盖更远的像素了。如果画板在镜头的右边，我们就要使用相反的方向了，纹理必须从右到左进行处理。在最坏情况下，画板将跨越镜头的两侧，纹理就必须从两个方向同时开始，一直到中间会合。

当进行纹理变化的时候原来纹理中两个相邻的点不再相邻，此时就会出现空洞。出现这种情况的视角是我们在原来对深度值的正交投影中没有考虑的。相应的解决方法为通过对两个像素的插值混合得到中间的像素。

Download at www.Pin5i.Com

最终的结果得到的纹理图像对于给定视角来说是很不错的。然后我们就可以直接将此纹理映射到画板上面进行显示了。在 CD-ROM 上面的例子程序里面提供了这个技术的实现。

4.12.4 方法 3：垂直插值的纹理

我们在此处阐述的最后的方法建立在前面提到的浮雕纹理基础之上。在基于合适的硬件使用了 DirectX8（或更高版本）的可编程像素着色器以后，在渲染过程中，我们可以在预先变换的纹理里进行插值混合。这样就可以模拟出不同视角的情况了。

我们可以把映射到画板上面的纹理看成是一张由针孔组成的纸，透过这些针孔玩家就可以看到单元里面的内容。每个纹理就是一个针孔，放置在此处的颜色就可以透过针孔看到。在此处，浮雕纹理的作用在某种程度上就是要计算出从不同的角度通过针孔看过去会看到些什么。

在这个最后的方法里面，创建出来的纹理将包含对一个单元四个不同的视图，这些视图是互相穿插，垂直交织的。通过一个像素着色器与一个二级的辅助性纹理，我们就可以把这些交织的条带在 GPU 里进行混合插值了。它可以提供与浮雕纹理方法几乎一样的视觉质量，但是此处就不需要进行反复的纹理变换了。其缺点就是在对各种图像采样进行组合的时候，某些 x 轴上的纹理分辨率会丢失掉。对于每一个在原始图像中的像素来说，它都必须有一个对应的四个像素组成的水平采样，每个都对应一个视角。我们可以将此方法扩展到纹理的 v 轴向上，但是其代价是会丧失更多的纹理分辨率（请参见图 4.12.4）。

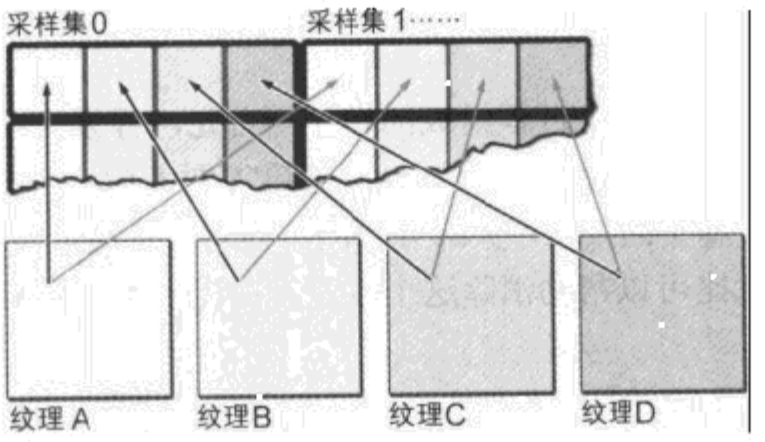


图 4.12.4 在源纹理相同的像素位置上进行采样，将之组织成 4 个一组，就可以生成交织的纹理了

像素着色器由有凸凹环境的地图指令（texbem）组成。它由两个步骤组成，首先，它将把输入的(u, v)坐标进行转换，将其指向期望的采样集合起始处。其次，它将会把新的 u 坐标偏移一下以适应集合里面的视角，它将使用到 texbem 指令。其结果是在我们四个视角中选择两个进行插值混合，最后组合，形成纹理。

着色器的第一步就是要使用我们前面说到的辅助性纹理。此纹理包含的是一个查找表，它将把我们输入的 uv 坐标对转换到第二个纹理开始的地方。此纹理与交织的纹理宽度应该一致，但是它将在每个像素的 u 轴上填充$(u*4)$的值。如果画板得到了 u 方向上 0.00 到 0.25 之

Download at www.Pin5i.com

间的纹理，那么这个辅助性纹理就可以将输入值变换到[0.0, 1.0]的域上了。这将导致每个输入的纹理坐标都指向我们四个像素采样集合中开始的地方。

第二步是要把此值进行位移以在采样集合中选出想要的像素。这个位移值的范围是[0.0, four_pixel_span]，此值可以通过顶点着色器计算出来，然后针对每个顶点写入第二个 *uv* 的集合中去。上述的 four_pixel_span 是纹理空间上四个像素的宽度。我们可以使用 4 来除以图像的像素宽度得到这个值。

最后，texbem 指令将从采样集合的开始处进行位移操作，偏移到预期的采样位置，在两个最接近镜头的视角视图中进行混合插值。在 CD-ROM 上面的源代码里面展示了定点着色器与像素着色器的实现细节。此外，彩图 8 中显示了从一个 2D 图像源中生成伪 3D 图像的三种方法。

4.12.5　结论

虽然任何一种画板方法都无法完全取代一个复杂的 3D 模型，但是通过这些技术，读者可以看到在一个 3D 环境中，预渲染的图像仍然是很有用的。只要在生成内容的时候小心一点，就很有可能创建出一个与原始模型几乎无法分辨的对象来。不论你是想更新自己的等测精灵引擎，还是需要得到一种简便的方法以在 3D 世界中生成更多的对象，我们都希望此处提到的思想能对你有所帮助。

4.12.6　参考文献

[Dietrich00] Dietrich, Sim, “Elevation Maps,” 在 Nvidia's 的开发者网站上有对应的网址：http://developer.nvidia.com/docs/IO/1334/ATT/ElevationMaps2.doc, January 2000.

[Olivera99] Olivera, Manuel M. and Gary Bishop, “Relief Textures,” 对应网址在 UNC 的基于图像渲染的网站，网址为 http://www.cs.unc.edu/~ibr/projects/RT/RT.html, April 1999.

Download at www.Pin5i.Com

4.13 使用法向地图进行曲面模拟

Oscar Blasco
Aside Software
ascar@asidesoft.com

当代游戏中的模型离真正真实的效果距离尚远。我们一般使用的方法是事先做好光照效果的纹理，用低分辨率的模型对曲面进行模拟。其实当前硬件的水平已经足以解出每个像素级别的光照方程了（不管是一般的还是比较好的）。在本文中，我们将阐述如何使用表面法线，通过凸凹贴图去模拟出一个较密集的曲面。我们将通过此方法增强视觉效果，同时确保不会丧失速度，也不用抛弃传统的增强细节的方法（例如，手工制作的凸凹贴图，请参见图 4.13.1 与彩图 9）。

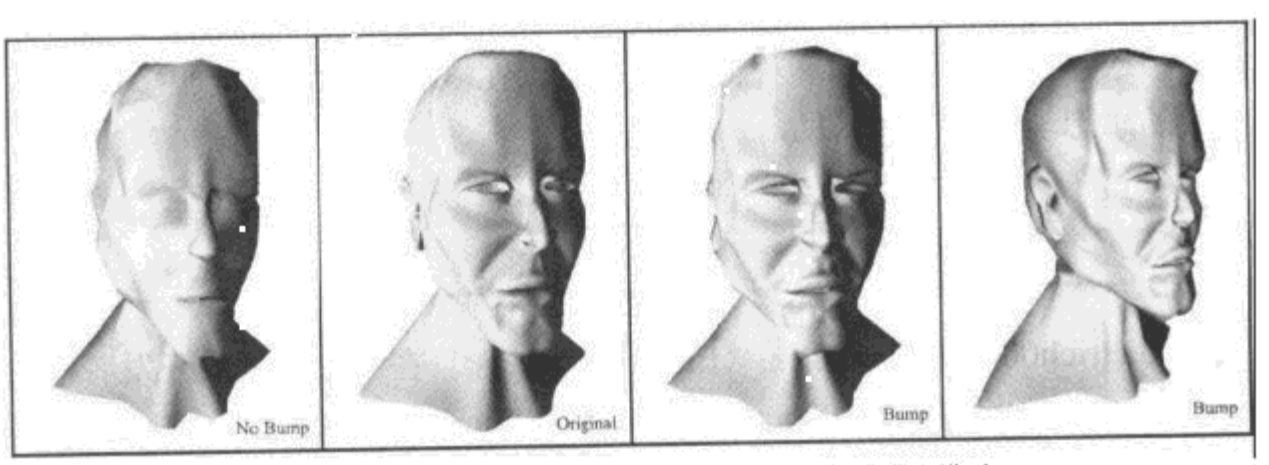


图 4.13.1 凸凹贴图的结果，从左至右依次为：低分辨率模型、原始模型、以及两个使用凸凹贴图的最终结果

4.13.1 法向地图

凸凹贴图是一个逐像素进行的过程，它将改变当前点的光照密度值。这就意味着我们可以将正在渲染的点变亮或是变暗。换句话来说，我们就可以逐像素计算出光照方程，而不用针对每个顶点计算方程了。方程 4.13.1 是一个简单而常见的光照方程，它带有一个镜面分量：

$$color=(n \cdot l) \cdot Surface_Color+(n \cdot h)^8 \qquad (4.13.1)$$

在我们的例子里面，n 作为一个纹理采样存储在纹理里面（它的形式是一个矢量的坐标，使用的是 RGB 编码），我们称之为法向地图（normal map）。硬件负责计算出 n 与 l 的（还有 n 与 h 的）点积，然后再使用其结果作为调制系数改变表面的颜色。请注意，在此处 l 将被变换到局部空间里面来，而点积则是在纹理空间里进行的（稍后我们再详细讨论这一点）。

Download at www.Pin5i.Com

4.13.2　整个过程的纵览

曲面模拟算法展示了一个通用的模式，以用来获取密集模型中的细节，这样就可以使之用在较低分辨率的模型上进行实时渲染了。通过在低分辨率的表面上进行光照调制，我们就可以模拟出捕获下来的细节了。

第一个模型是一个密集型网格，看上去像是为非实时渲染建模的。第二个，也是一个类似的模型，则有相当数量的三角形以用来渲染具有交互性的场景的。它同样也有进行纹理化所需要的 *uv* 坐标。较低分辨率的模型不必非要是密集模型的简化版，这样此项技术就有用武之地了。它可以让建模人员对低分辨率的网格进行修改以解决一些问题，或是直接使用一个没有直接关系的网格也行。

正如我们所说的，我们的目标是要把低分辨率的网格中没有表现出来的细节保存到一个法向地图中去。为了做到这一点，我们需要把原始模型的曲面捕获下来，同时也要考虑到低分辨率模型也有自己的曲面。这个曲面由密集型模型中的法线给出，因此我们需要把这些法线给捕获下来，将之保存在地图中。

在运行的时候，生成的纹理将改变低分辨率网格上每一点的法线。这样就会改变光照方程中的点积（方程 4.13.1），继而生成对曲面的模拟。我们给每个纹理采样都加上了一些随机的噪声信号，这样其表面就与原始对象在此表面处的点更相近了（请注意，我们的法向地图其大小必须是有限的）。此算法的大致步骤如下：

- 预处理；
- 计算出地图；
- 后处理。

而计算地图则需要：

- 对每个多边形进行光栅化，将之放到地图中（通过使用每个顶点的 *uv* 坐标）；
- 在每个纹理采样处计算出受扰的法线。

4.13.3　数据准备

在开始之前，我们还需要以下的数据：

对于高分辨率的模型来说，我们需要：

- 在每个顶点的法线，这些法线在这个模型上是光滑的。

对于低分辨率的模型而言，我们需要：

- 在每个顶点处得到其切向空间基（包括光滑的切线、副法线、以及法线）；
- 每个顶点的 *uv* 坐标。

我们之所以需要切向空间基是因为要把计算得到的法线变换到纹理空间里面去。

为什么对象需要进行 *uv* 的映射是显而易见的，不然的话，我们就无法知道如何将低分辨率的网格地图映射到纹理上面去了（请参见“已知的问题”这一小节，这样读者就可以对 *uv* 坐标常见的问题有更清楚的认识了）。

Download at www.Pin5i.Com

4.13.4 投影线

现在我们就可以讨论如何在每个纹理采样上计算出加扰后的法线了。由于两个网格的顶点不是共享的，因此我们不得不从低分辨率网格的表面上进行投影，看看它在何处与密集模型的表面相交。

Sander，et al[Sander]探讨了两种方法以用来选择投影线的方向：第一种，也是最简单的方法就是使用当前表面的法线（最近点的方法）；第二种方法要求沿着表面对法线进行插值，以此作为投影线的方向（法线投射的方法）。

正如你在图 4.13.2 中所见的一样，对法线投射进行参数调整以后，我们可以得到更好的结果，而最近点法则会引起不连续性问题。

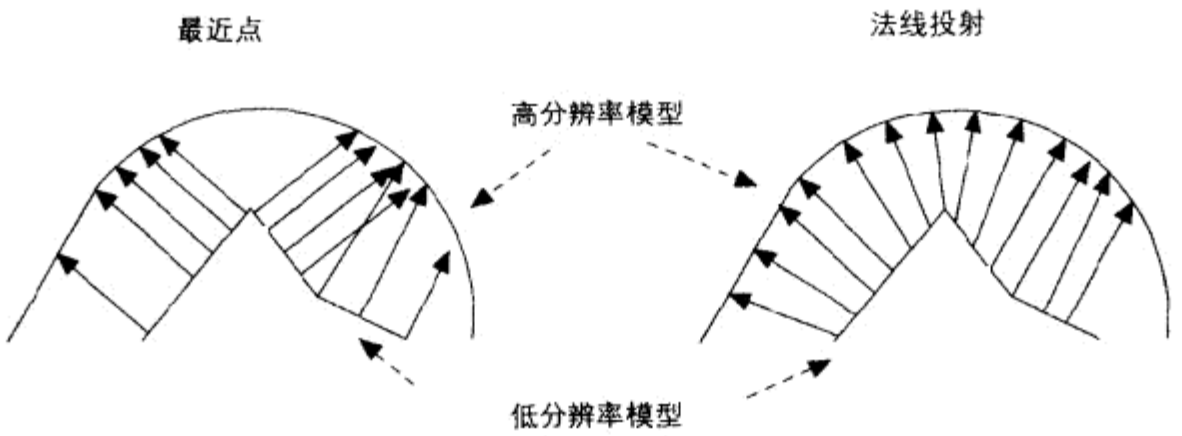


图 4.13.2 使用最近点与法线投射跟踪投影线

对于每条投影线来说，一旦我们选择将此线进行投影，得到了其与密集模型表面的相交点以后，我们就可以使用在三角形里面的重心坐标计算出此点的法线了。

重心坐标是一种用来定义一个三角形内部点的有效方法。假设 P 是我们在对象空间里面找到的交点，v_0、v_1、以及 v_2 是三角形的三个顶点，那么，我们就可以使用下面的方法将其计算出来：

$$u=\frac{\|(v_2-v_1)\wedge(P-v_1)\|}{\|(v_1-v_0)\wedge(v_2-v_0)\|} \tag{4.13.2}$$

$$v=\frac{\|(v_0-v_2)\wedge(P-v_2)\|}{\|(v_1-v_0)\wedge(v_2-v_0)\|} \tag{4.13.3}$$

$$w=1-(u+v) \tag{4.13.4}$$

这里的重心坐标相对于由交点与三角形顶点定义的子三角形区域（请参见图 4.13.3）。

在碰撞检测中它们是很有用的，因为任何一个对每个顶点都可以定义的属性，都可以使用 u、v 和 w 来插值得到。例如，此处的交点就可以如下计算得到：

$$\vec{P}=u\cdot v_0+v\cdot v_1+w\cdot v_2 \tag{4.13.5}$$

Download at www.Pin5i.Com

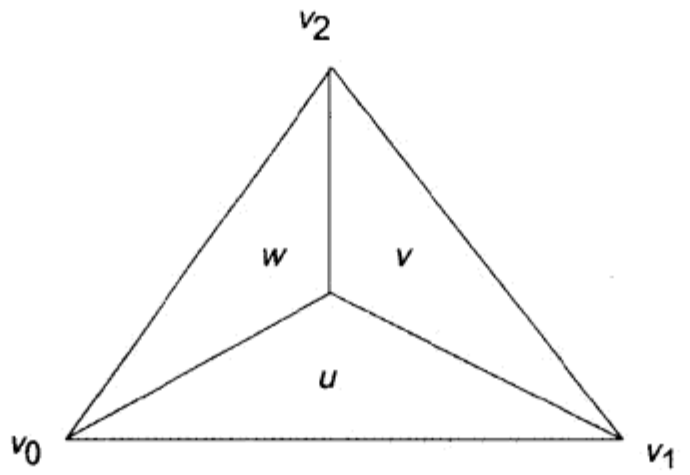


图 4.13.3　重心坐标与每个子三角形的面积有关

在此小节中，我们讲到的各个步骤可以如下所示：首先，对于每一个低分辨率网格上的点来说，我们需要沿着插值得到的法线投影出一条射线，得到射线与高分辨率模型中一个三角形的交点；其次，计算出它的重心坐标，这样我们就可以插值得到高分辨率网格中这一点的法线了。正如你所看到的一样，我们已经得到了想要的东西了——那就是一个法向矢量。下面我们必须将之保存在正常的地图里面去。

4.13.5　得到细节信息

由于我们已经决定了在纹理空间里对光照进行处理，因此我们就需要对捕获的法线进行一次空间变换（请参见图 4.13.4）。

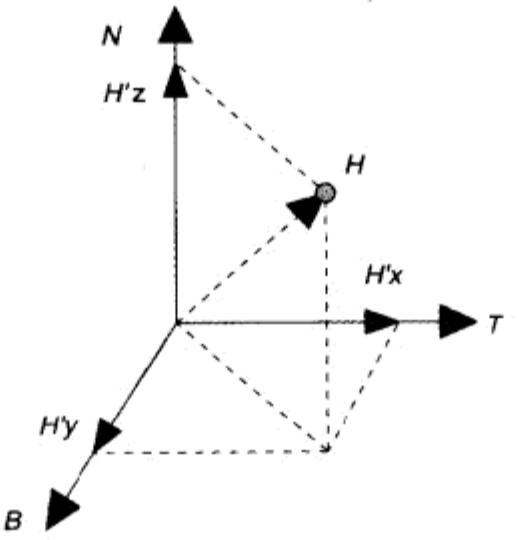


图 4.13.4　捕获的法线（*H*）被转换到纹理空间中去（*H'*）

在表面上任意一点的切线、副法线和法线构成了一个矩阵（请参见方程 4.13.6），它是从模型空间到纹理空间的变换矩阵（方程 4.13.7 是变换后的法线）。*T*、*B*，还有 *N* 都被称为切向空间基[Lengyel01]。

$$\mathbf{M}=\begin{bmatrix}T\\B\\N\end{bmatrix}=\begin{bmatrix}Tx & Ty & Tz\\Bx & By & Bz\\Nx & Ny & Nz\end{bmatrix} \tag{4.13.6}$$

Download at www.Pin5i.com

$$\vec{H}' = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} Tx & Ty & Tz \\ Bx & By & Bz \\ Nx & Ny & Nz \end{bmatrix} \cdot \begin{bmatrix} Hx \\ Hy \\ Hz \end{bmatrix} \tag{4.13.7}$$

4.13.6 后处理

在计算出了法向地图以后，其中仍然有很多空余的区域。在光栅化的过程中，由于存在着这些法向地图中的空余地带，纹理过滤程序将改变我们使用的法线。为了解决这个问题，我们可以将法向地图中空余的区域填充浅蓝色——RGB(127, 127, 255)或矢量形式的(0.0, 0.0, 1.0)。

一般来说，使用蓝色就可以了，但是我们还可以做更多的工作，得到更好的效果。利用边缘扩展过滤器，我们可将多边形边缘的颜色扩展到纹理中去。这样就可以解决硬件过滤带来的问题了。

一定要使用一种不会改变纹理采样的图像文件格式来保存纹理。例如，JPEG 就不行，因为它使用的是有损压缩算法，会破坏法线的归--性。

4.13.7 已知的问题

在使用此处提到的投影跟踪方法中，我们可能会遇到一些问题，这有两个原因：第一个是 *uv* 坐标；另一个则与错误的投影线或是相交点有关。

如果模型之间差异较大（例如，投影线与密集网格相交了两次），我们就会得到错误的法线。一定要让建模人员避免这种情况的发生（请参见图 4.13.5）。

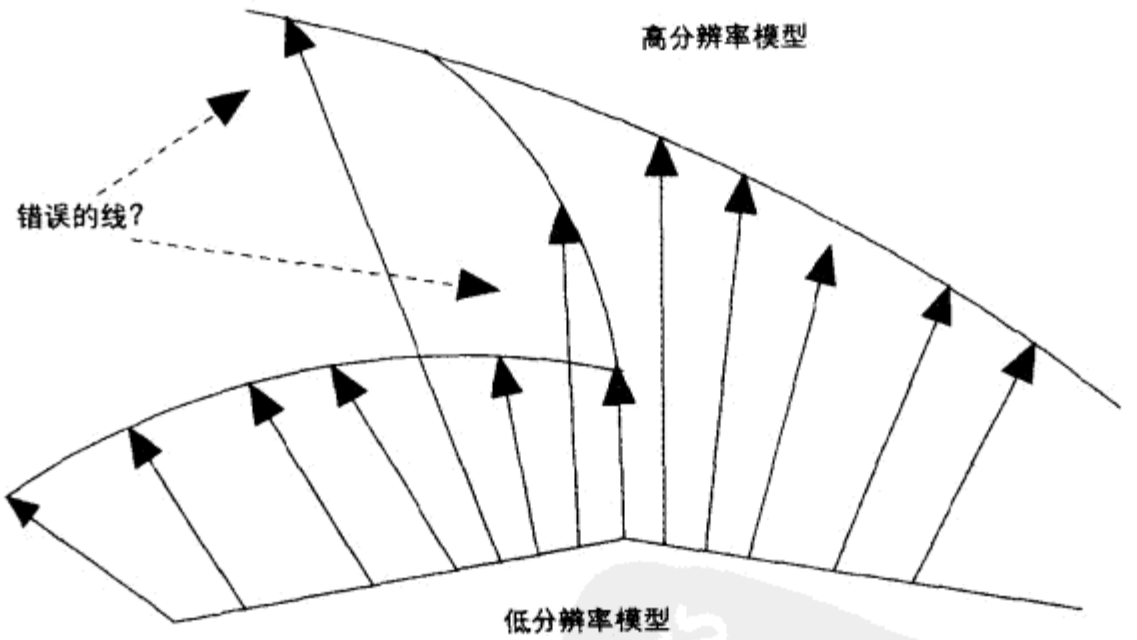


图 4.13.5 投影线算法不知道哪个交点是错误的

对于 *uv* 坐标来说，有两个必须说明的问题：

- *切向空间基的不连续性*。这个问题对于任何一个在纹理空间计算光照的凸凹贴图来说

Download at www.Pin5i.Com

都是存在的。在纹理空间里面进行光照处理可以让我们在改变几何形状的时候仍然能使用同一个法向地图。其问题就在于由于需要 *uv* 映射，切向空间基在模型表面上就不连续了，这样就会产生光照矢量不连续的问题。如果仅在模型空间里进行光照处理，那么我们就可以避免这个问题。但是它会带来其他的问题：如果法线是在模型空间里面定义的，那么我们就需要在每次几何形状改变的时候随之对其进行改变。不过，你可以将光线变换到改变后的几何形状定义的新空间里面去，这样就可以解决这个问题了。不论如何，这是依赖于引擎设计与期望达到的质量的。

- *错误的映射*。此方法对 *uv* 坐标有极大的依赖性。此对象需要进行小心的映射以避免产生预料之外的结果。

4.13.8　其他方法

再来探讨一些其他计算法向地图的方法是非常有意思的，特别是当模型具有类似空间特性的时候。我们需要考虑到，低分辨率的网格是直接从密集的网格由某些简化算法得来的，而且此算法仅仅使用到了原始模型中的顶点。此外，原始的模型已经是经过 *uv* 映射的，或是通过某些模式已经加上了此特性的。考虑到这几点，我们就可以很容易地对每个纹理采样在模型之间建立一种直接的关联了，因为它们的顶点是一致的（具有同样的 *u* 坐标与 *v* 坐标）。

如果我们不需要将法线转换到纹理空间，则此模式还可以进一步简化。由于这与直接把密集模型的法线绘制到法向地图上是完全一样的，因此我们可以使用 3D 硬件的光栅化处理实现此模式。如果我们可以通过 *uv* 进行模型的映射，那么这方法将会很有用。

4.13.9　结论

在本文中，作者尽力使得在此处讨论的算法更加通用化，虽然一般说来，通用化会带来不够精确的结果与较低的速度。在此处，通用的意思就是说此方法不依赖于两个模型中任何的特殊关系，最主要的目标是让艺术创作人员就能通过此方法修改低分辨率网格中的顶点。其他用于得到原始曲面的算法则期望模型之间的顶点是共享的（而且还期望其属性是共享的，例如 *uv* 坐标）。一种使用边缘分解的简化算法经常用来从密集模型中获取低分辨率的网格。此方法的缺陷是一望而知的：经过简化以后得到的表面是不均匀的，而且网格也很难卷曲起来（通过 *uv* 坐标进行映射）。此外，如果原始的对象已经经过 *uv* 映射了，则其映射通常会被破坏掉。尽管如此，在某些情况下你还是可以使用此方法的。

在生成了法向地图以后，我们仍然可以添加手工制作的图块以添加细节部分。实际上，我们也鼓励这样做。在一个密集的网格里对细节进行建模是比较困难的，而且完全是低效的。如果我们添加手工制作图块的话，投影线跟踪的精确度就比较小了。我们可以在生成的法向地图上通过简单的法向地图合并代码将之添加进来。这只需要在纹理里面保存一个法线的加扰量就可以了。

Download at www.Pin5i.Com

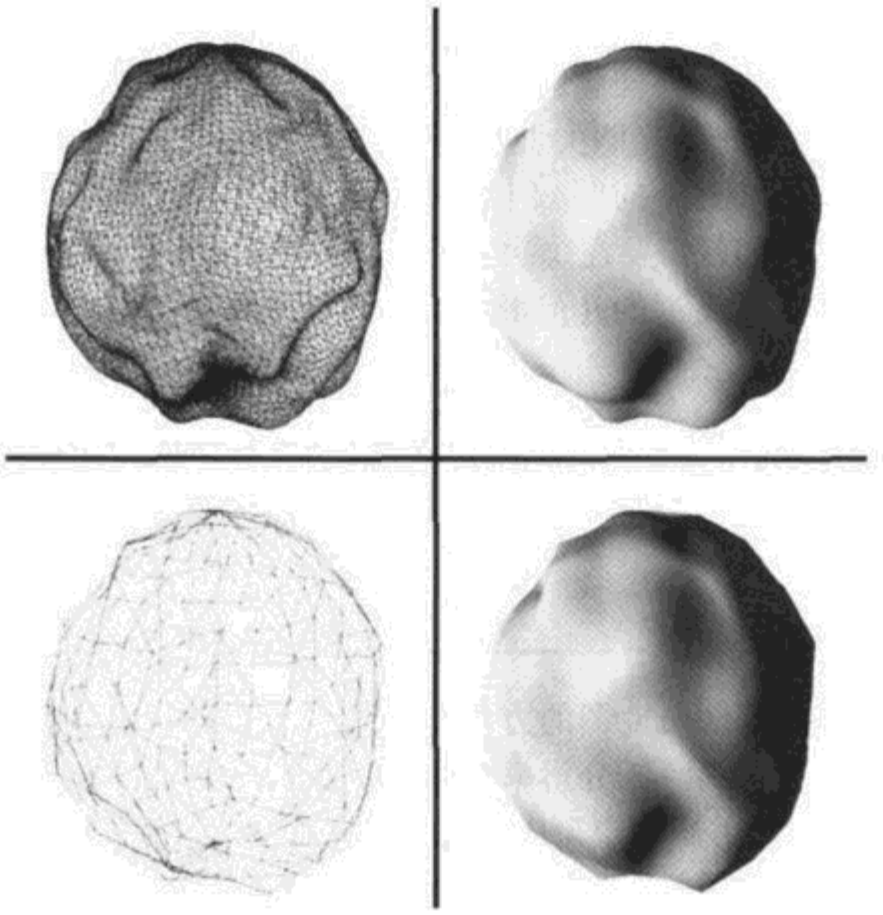


图 4.13.6 一个加扰后的球体。上面两个图像是原始对象，下面两个是使用此算法后得到的结果

4.13.10 致谢

在此，我谨向 Julio Cesar 表达谢意，他为本文创建了所有的模型（特别是图 4.13.1 中的头部造型），此外他在使用与测试此技术的过程中提出了宝贵的反馈意见。另外，我还要感谢 Ignacio Castaño，他给我提供了建议与支持。

4.13.11 参考文献

[Ebert] Ebert, David S., et al., "Texturing and Modeling: A Procedural Approach," (SIGGRAPH).

[Kilgard] Kilgard, Mark J., "A Practical and Robust Bump-Mapping Technique for Today's GPUs," 对应的 nVIDIA 网址：http://developer.nvidia.com/docs/IO/1329/ATT/bumpmap.pdf.

[Lengyel01] Lengyel, Eric, *Mathematics for 3D Game Programming & Computer Graphics*, Charles River Media, 2001.

[Sander] Sander, Pedro V., et al., "Silhouette Clipping," SIGGRAPH 2000 Proceedings: pp. 327～334. 对应的网址为 http://research.microsoft.com/~hoppe/.

[Watt92] Watt, Alan H. and Mark Watt, *Advanced Animation and Rendering Techniques*, Addison Wesley, 1992.

[Wynn] Wynn, Chris, "Implementing Bump-Mapping Using Register Combiners," 对应的网址为 http://developer.nvidia.com/docs/IO/1273/ATT/BumpMappingWithRegisterCombiners.pdf.

Download at www.Pin5i.Com

4.14 动态的、具有照片效果的地形光照

Naty Hoffman，Kenny Mitchell
Westwood Studios
naty@westwood.com
kmitchell@wstwood.com

在当前的渲染技术下，我们已经能够通过使用大量三角形，在游戏中创建广阔而细致的户外场景了。不过，为了表现户外场景中复杂的视觉效果与魅力，有个重要的部分是我们所不能忽视的，那就是光照，而不仅仅是其几何形状。如果光照是静态的，那么我们可以很容易地事先计算出一个高质量的光照解决方案来——但是如果光照是动态变化的时候，我们又应该怎么办呢？在本文中，针对动态光照条件下的地形，我们将提出几个方法用来生成高质量的、基于物理原理的光照方案。

4.14.1 背景知识

第一步就是要对光线进行正确的量化。光线是处在电磁辐射波频谱范围中可见光部分（大约在 400～700nm 波长范围内）的波。光线测量中最常用的是辐射量，或说是一条光线中的光通量。为了帮助读者理解辐射量的定义，假设在光线通过的地方有一个矩形（区域 A），其中心处在光线通路之上，其面与光线方向垂直（请参见图 4.14.1a）。

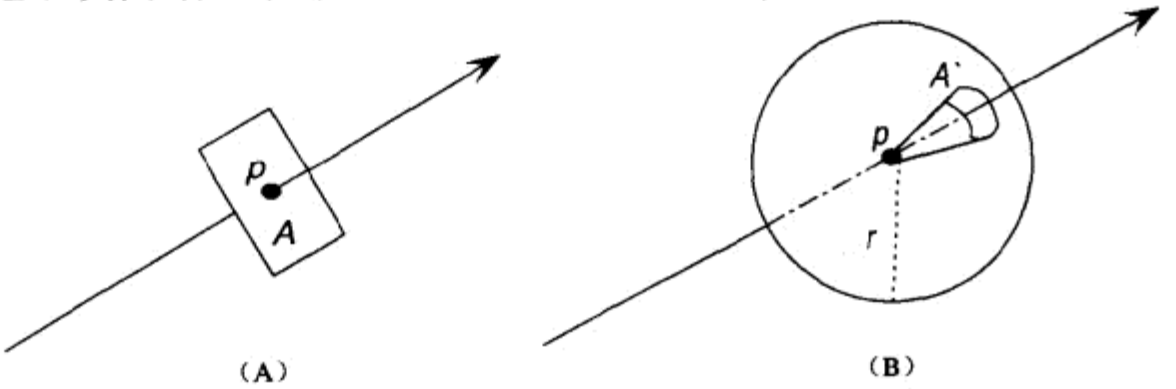


图 4.14.1 （A–B）沿光线方向在区域 A 上的光通量以及球面 A′上的光通量

光能将在各个方向与频率上持续不断地通过此矩形面。在 RGB 空间中，此能量（以及任何由之派生出来的量）是通过三个分量计算的。大致上来讲，你可以把它看成由三个范围组成，其范围划分主要基于频率。在某个时间点上，我们可以计算出此时间段上的平均能量来（光功率，或称其为辐射通量 Φ）。通过此矩形的通量与矩形的面积有关，而且它并没有真实地反映出光通路上通过的光线。我们可以计算得到在此截面 p 上平均

Download at www.Pin5i.Com

面积的通量值（$M = d\Phi / dA$，或称平均面积上的功率，其单位是瓦特每平方米）。不过，M 包含的是所有方向上面的光线，而不仅是此光通路上的光线——我们还需要考虑到方向的因素。在图 4.14.1b 中，我们看到有一个球面（半径为 R），其中心处在 p 处，此外在球面上还有一小块区域（区域 A'）围绕着光线。在 M 中有一部分是会通过此区域的，我们称其为 M'——M' 与球面半径的大小无关，仅仅与球面这部分区域的大小有关系。在此处，它就代表了方向上的分布，我们称之为立体角（$\omega = A' / R$，其单位是球面角）。这样，在光通路方向上面的光能就可以如下计算了：$L = dM' / d\omega$，最终结果的单位是瓦特每平方米每球面角。辐射量是单位球面角上单位投影面积的功率——投影面与光通路的方向是垂直的。如果我们使用的面与光通路不是正交的（例如，当我们观察从一个面上反射回来的光线的时候），那么我们就需要使用一个修正系数了，此系数与表面法线以及光线的点积成正比。

在户外场景中，其辐射量之间的差别可以达到 6 个数量级之多[Debevec98]。但是在大部分硬件设备上面，我们是无法满足这个要求的。在本文中，我们将把所有的光照值压缩到[0, 1]这个范围上。这样就引起了一个相关的问题，那就是在像素值与显示的辐射量之间会产生一个非线性的关系。人类眼睛对光线的响应会导致其他的问题。我们在此处暂时不考虑这些问题，仅仅讨论线性辐射值的问题。

在测量光线的问题上，我们还将使用另一个量，那就是 *irradiance*，E。它与单位面积上面的通量，即上面提到的 M，非常类似，而且具有同样的单位，不同的是它测量的不是在某个点上射出的光线，而是在这个点上进入的总的光能。由于这也是在一个表面上面进行测量的，因此在光线进入的点 p 上，我们需要使用一个半球面 $H(p)$（此球体的直径就是表面法线 $\mathbf{N}(p)$）对之进行积分：

$$E(p) = \int_{\mathbf{V} \in H(p)} L_i(p, -\mathbf{V})\mathbf{N}(p) \cdot \mathbf{V} d\Omega \qquad (4.14.1)$$

在此处，$\mathbf{V}$ 是半球面 $H(p)$上的单位径向矢量，$L_i(p, -\mathbf{V})$是在方向$-\mathbf{V}$ 上进入点 p 的辐射量，而 $d\Omega$则是差分立体角，它是用来进行积分的。点积得到的就是投影面积修正参数。

在本文中，我们仅处理朗伯（Lambertian）地形表面。对于这种表面来说，射出的辐射量 L_o 在各个方向上都是一样的，而且大小就等于辐射量乘以颜色值 $C(p)$，再除以 π：

$$L_o(p) = \frac{C(p)}{\pi} E(p) \qquad (4.14.2)$$

在其中，颜色值指的就是 RGB 三原色的值，每个值都在 0 与 1 之间。当颜色值等于 0 的时候，入射能量将被全部吸收，在相关频谱范围里面没有任何反射；当颜色值等于 1 的时候，所有能量都将被反射出去。

在此处我们需要解决的问题就是要在一个地形图的高程场上面，在给定的分辨率之下，考虑到光照变化的限制条件计算出所有点的 $L_o(p)$来。由于地形是 Lambertian 类型的，因此我们可以假设 $C(p)$是已知的，这样，问题就转化为如何计算 $E(p)$了。为了得到它的值，我们需要知道在 $H(p)$上各个方向入射的辐射总量。我们可以将之分为三类：从日照方向上进入的、从天空方向上进入的，还有从其他地形地物里面进入的。我们假设在任何时间都可以得到相应的数据：太阳的方位、角度大小、辐射量。我们可能还需要得到云层方位的信息以及其他的数据。此外还请注意，在天空的不同方向上，其辐射量也有可能不一样。

现在，我们需要考虑从其他地形地物里面进入的辐射量了，它是一个难题。为了得到这

Download at www.Pin5i.Com

些方向上进入的辐射量，我们需要首先得到对其他地形地物出射的辐射量。如果太阳与天空的方位与辐射量都是静态的，那么我们就可以使用一种迭代算法，例如使用辐射穿透量，来预先计算出一个解了，但是在动态环境中，这种方法就不现实了。

4.14.2　解的分类

我们将提出一系列的技术来解决辐射量计算的问题。这些技术在预先计算量、存储结果需要的空间、各自的局限性与假设条件，以及最终结果与正确值的接近程度上面都会有所不同。它们其中的大部分技术需要的预先的计算都太多了，在可变形的地形中无法使用（我们将指出哪些算法有这个弱点）。由于主要有两种光源——太阳与天空——因此我们可以将整个问题分解为“仅考虑日照”的子问题与“仅考虑天空光照”的子问题，以便分而治之。大部分技术都只能解决这些子问题中的一部分，因此我们可能需要使用两种技术，最后将其结果加起来得到最终值。

其中的某些解决方法牵涉到在 CPU 上计算出光照值，以及将光照地图载入图形卡上的问题。在这种情况下，一定要特别小心，确保不会占用过多的 CPU 时间与带宽。由于光照值一般说来变化得都很慢，因此我们就可以过数帧再更新一次了。我们曾经试验过使用多线程，不过，最后还是觉得单线程加上双缓冲的模式比较好。我们使用它来计算与载入当前需要的纹理数据，同时使用以前的纹理进行渲染。在当前的纹理处理完毕以后，我们将切换到下一个纹理，渲染当前纹理，然后又开始了一次循环。

4.14.3　日照：地平角、椭圆阴影以及 PTM

在本小节中，我们将解决“仅考虑日照”的子问题。大部分日照子问题的解决方法都忽略了从其他地物反射回来的光线。我们的实验结果显示，只要在天空光照子问题中考虑到了互相之间反射的问题，得到的结果就不会太差。既然如此，我们应该尽可能地考虑到日照问题中互相之间反射的问题，而且我们将展示一个考虑到了此方面的技术。如果不考虑互相反射的问题，那么在 p 点处日照的贡献就是：

$$E_{\text{sun}}(p)=\int_{\mathbf{V}\in S\cap H(p)} L_{\text{sun}}\mathbf{N}(p)\cdot\mathbf{V}d\Omega \tag{4.14.3}$$

在其中 S 是一组有日照分布的点的集合。由于太阳的立体角 ω_{sun} 是很小的，因此我们可以假设点积的结果基本是一个常量，这样我们就可以推导出：

$$E_{\text{sun}}(p)=O_{\text{sun}}(p)\omega_{\text{sun}}L_{\text{sun}}\mathbf{N}(p)\cdot\mathbf{V}_{\text{sun}} \tag{4.14.4}$$

在其中，$\mathbf{V}_{\text{sun}}$ 的方向是指向太阳中心的，而 $O_{\text{sun}}(p)$ 则是日照中处于 $H(p)$ 之外，或是遮蔽在 p 点之内的那部分分量。这两个量在不同的地物点上是有所差异的，它们分别是点积修正值与遮蔽系数。为了减轻计算点积的工作量，我们需要对所有地物上面的法线进行量化，将之在预处理阶段中分配到一个划为 256 段的法线表里面去。接下来，对每一帧，我们都将计算出法线表中每一个法线与当前日照矢量之间的点积。这样，我们就不需要对每一地物点计算点积，而只需要进行一下表的查找就可以了。我们还可以使用另外一种方法，那就是使用一个法向地图加上在硬件中进行的 dot3 纹理混合。

Download at www.Pin5i.Com

我们可以使用多种方法来计算遮蔽系数。其中的一种方法使用到了地平角映射，它是Max 在[Max88]中提出来的。此方法主要的思想就是针对一组给定的角，把其中所有点的地平角都保存下来（请参见图 4.14.2a）。如果太阳的运动局限在一个跨过天顶的角度上，那么我们只需要两个地平角就足够了。在得到了这个信息以后，如果想要得知一个点是否在阴影部分，只需要将太阳的角度与当前方向的地平角进行一下比较就足够了。如果太阳角 θ 比地平角 φ 要小，那么当前的点就在阴影之内（$O_{sun}(p)=0$），不然的话，它就不在阴影之内（$O_{sun}(p)=1$）。我们可以使用两个太阳的角度（θ_{top} 与 θ_{bottom}，分别是太阳的顶部角与底部角）来处理模糊阴影，将之与地平角进行比较。如果 $\varphi \geqslant \theta_{top}$，那么 $O_{sun}(p)=0$；如果 $\varphi \leqslant \theta_{bottom}$，则有 $O_{sun}(p)=1$；如果 $\theta_{top} > \varphi > \theta_{bottom}$，则就有 $O_{sun}(p) = (\theta_{top} - \varphi) / \alpha$ 了，在其中 α 是太阳的角直径。在我们的实验中，不论是以 16 位定点数的形式存储地平角，并在 CPU 中进行计算，还是在 8 位的纹理通道中存储地平角，并在像素着色器（pixel shader）中进行计算，都得到了很好的结果。考虑到效率问题，定义地平角的方向应该对应于高程场数组中的每一行。这样在计算地平角的时候会比较方便。你应该在高程场的每一边扫描相当长的一段距离，因为当太阳比较倾斜的时候，山峰会有比较长的一段阴影面积。值得注意的是，地平角应该确保在 $H(p)$ 上进行计算得到，不然就会产生错误（请参见图 4.14.2b）。

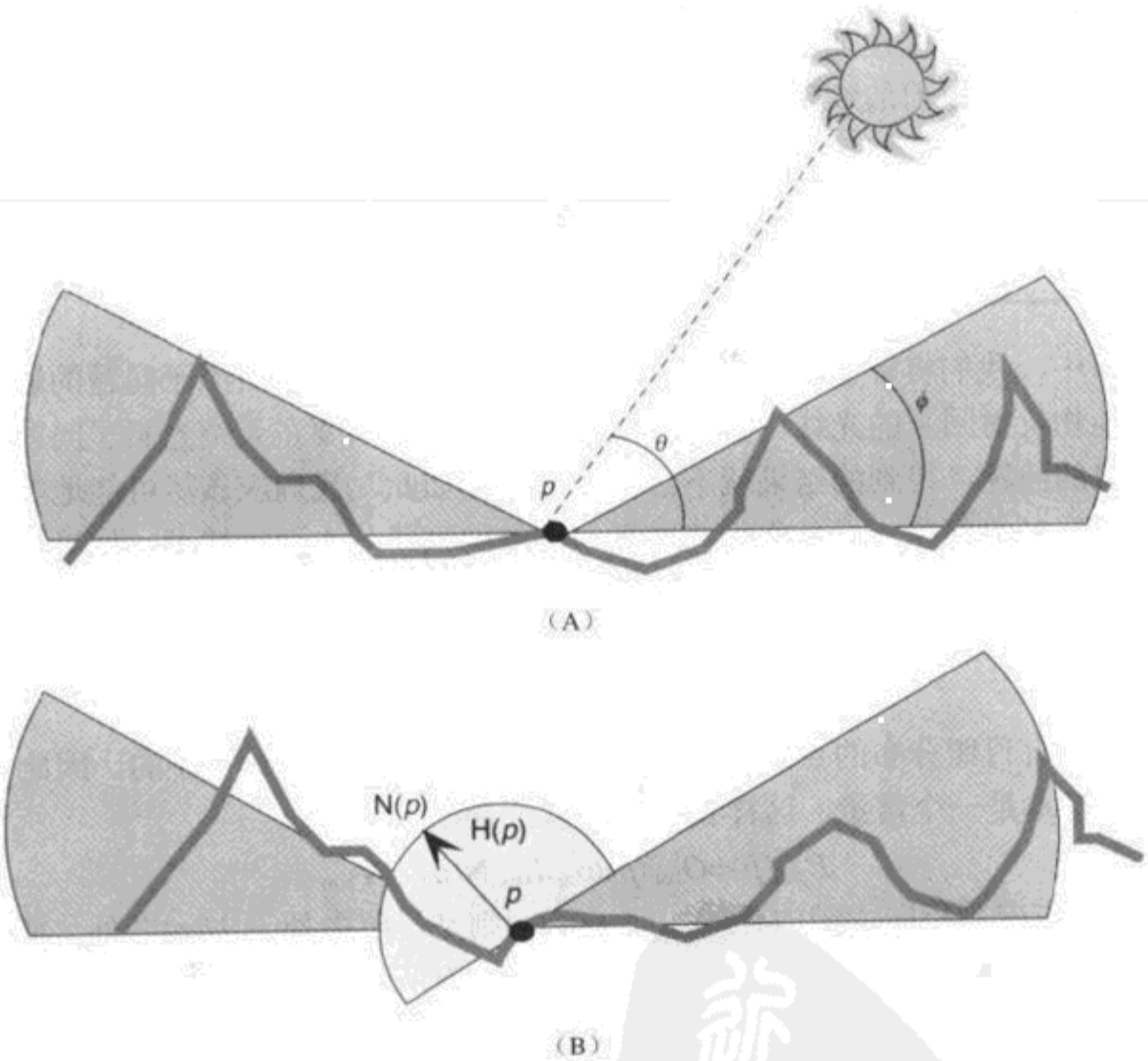


图 4.14.2 （A）拥有太阳角 θ 与地平角 φ 的地形（B）地平角必须确保在 $H(p)$ 上得到

在所有的日照计算方法中，地平角映射需要进行的预处理是最少的。如果高程场中出现了局部的较小变化（例如小坑），这也是可以得到处理的，我们重新计算一下受影响区域两侧

Download at www.Pin5i.Com

附近一段距离的地平映射图就行了。

有时候我们可以使用椭圆阴影来进行遮蔽的计算，它是在[Heidrich00]中提出的。在此处，我们将使用一个椭圆来覆盖每个地物点上的可视角度。我们可以通过 6 个参数来调整这个椭圆，它们是存储在两个 RGB 的纹理图中的。接下来，我们就可以使用软件、寄存器组或是像素着色器来计算每一点上太阳的遮蔽了。这种方法处理模糊阴影是很麻烦的。与地平角方法相比较，它的主要优点就在于它可以处理任意角度的太阳方位（请参见[Heidrich00]，其中有更多的细节，另外在[Kautz00]中有使用 OpenGL 通过硬件实现此方法的细节）。

多项式纹理图（Polynomial texture maps，PTM）是在[Malzbender01]中提出的，它也可以用来计算日照。在少数几个可以处理地形里相互反射的日照的技术中，它是其中的一种。为了创建一个地形的 PTM，我们首先需要使用计算辐射穿透量的软件包来渲染一系列的 N 幅图像，这些图像都是地形图像，在其中太阳分别处于不同的方位。在此处，我们不像地平角映射方法中的那样，要求太阳只能处在天顶的一段弧上，但是其运行轨迹应该可以使用一段一维弧线描述。我们使用的太阳颜色是 RGB(1, 1, 1)。接下来，将得到的 N 幅图像放入一个 PTM 中。第一步就是要将每幅图像中的颜色与亮度信息分离开。然后，使用一个二次一元多项式$(a_0x^2 + a_1x + a_2)$来得到每个像素的 N 个亮度值，在其中我们使用了太阳在弧线上的方位作为其变量。最后，生成两个 RGB 纹理：一个保存的是每个像素上的颜色信息（通过对所有的图像进行平均以后得到的），另一个保存的是每个像素上的亮度多项式的三个系数。在运行期，我们将使用当前太阳的方位计算出多项式的值，不管是通过软件还是像素着色器都可以。得到的结果将与颜色值合并，并利用当前太阳的颜色进行调制。（读者可以参阅[Malzbender01]以得到更多的信息，如果想实现这些技术，可以到相关 Web 地址的下载部分去下载实现它们的工具。值得注意的是相关的论文中使用的是二元多项式，这样就一共有 6 个系数了，对于本文来说，这种技术就太复杂，太昂贵了。）

如果光照条件仅仅依赖于一个变量（例如一天中的时间），那么我们就可以使用 PTM 来得到完全的光照方程的解了。在这种情况下，我们可能需要使用一个单独的多项式来存储每个 R、G 与 B 值以跟踪颜色的变化，这样在每个像素上我们就需要 9 个值，而不是 6 个值（三个值再加上一个 RGB 颜色）。

4.14.4　天空光照：辐射透过量的近似与分块

本小节中的探讨主要是为了解决“仅考虑天空光照”的子问题。首先，我们需要处理整个天空辐射分布一致的情况。这个近似得到的结果其实还是不错的。在使用过程中，我们得到了相当好的效果。在这种情况下，我们只需要使用 RGB(1, 1, 1)作为天空辐射量来计算地形光照，然后把得到的结果保存在一个 RGB 纹理中就可以了。在运行期，我们将把这个纹理与当前天空的 RGB 辐射量相乘以得到期望的结果。其纹理的颜色相对来说还是比较细致的，因为我们在彩色地形中考虑到了互相反射的情况。如果空间特别宝贵的话，我们还可以将纹理转换为每个像素 8 位的灰度图，其在质量上的损失也是很小的。

如果预处理的时间与空间都不是问题，那么生成天空纹理最好的方法就是使用一个处理辐射穿透量的软件包了，其输入参数是地形与一个呈半球状分布的光源。如果这个方案不太现实的话（出于时间或是空间方面的考虑，或者是地形会发生变化），那么我们还可以使用一

Download at www.Pin5i.Com

个有效的近似以在极短的时间里面生成一个非常类似的结果。Stewart 与 Langer 在[Stewart97]中提出，在一个由散射半球状光源照明的场景中，一个点 p 会比较容易“看到”场景中其他具有相近辐射量的点。因此，我们可以假设对于所有在 $H(p)$中的 $\mathbf{V}$ 来说，如果在此处可以看到其他的地物点，那么就有 $L_i(p, -\mathbf{V}) = L_o(p)$。Stewart 与 Langer 指出通过这种近似方法引入的误差是很小的。这种方法的意义就在于它打破了不同地物点之间的相关性，这样我们就可以用 $E(p)$的闭式表达式了。Stewart 与 Langer 使用地平角来计算地形覆盖了哪些方向，得到了一个闭式表达式。他们的公式使用到了任意数量的地平角。我们在实际中使用了 8 个（在计算方面来说这个数字比较好，因为它对应了高程场数组中的行、列与对角线）就得到了很好的效果。假设一共有 8 个地平角（对于阳光来说是垂直方向，而不是水平方向）$\varphi 0$、$\varphi 1$、$\varphi 2$、$\varphi 3$、$\varphi 4$、$\varphi 5$、$\varphi 6$、$\varphi 7$，天空对辐射量的贡献在 p 点是（已经考虑到了相互之间的反射）：

$$E_{\text{sky}}(p) = \frac{L_{\text{sky}} I(p)}{1 - C(p)\left(1 - \frac{1}{\pi} I(p)\right)} \tag{4.14.5}$$

$$I(p) = \frac{1}{2}\mathbf{N}(p) \cdot \sum_{i=0}^{7}\left(\left(\varphi_i - \frac{\sin 2\varphi_i}{2}\right)\Delta\sin_i\left(\varphi_i - \frac{\sin 2\varphi_i}{2}\right)\Delta\cos_i \frac{\pi}{4}\sin^2\varphi_i\right) \tag{4.14.6}$$

$$\Delta\sin_i = \sin\left(\frac{\pi}{4}(i+1)\right) - \sin\left(\frac{\pi}{4}i\right) \tag{4.14.7}$$

$$\Delta\cos_i = \cos\left(\frac{\pi}{4}i\right) - \cos\left(\frac{\pi}{4}(i+1)\right) \tag{4.14.8}$$

（若想得知方程 4.14.5–8 的详细推导过程，请参见原始论文[Stewart97]或是[Hoffman01]中的总结。）在此处使用到的地平角可以在较短的距离上得到，这与太阳阴影的处理不太一样。当然了，如果你已经计算得到了太阳阴影的地平角，在这里就可以直接用它们来代替 8 个角中的 2 个角了。由于这种技术使用的预处理相对来说比较少，因此它可以用来处理对高程场偶尔的较小的编辑（例如，小坑），这只要在受影响的区域及其附近重新计算一下太空光照的纹理就可以了。

如果假设天空辐射量一直没有得到理想的效果，那么我们就需要将“仅考虑天空光照”的子问题分解为更小的子问题了。例如，我们可以将天空分为 N 个分块，这样就需要计算 N 个天空光照纹理，每个纹理的计算仅与相关分块照到的地形有关。在运行期，每个天空光照的纹理都要使用相关分块的辐射量进行调制，最后再将这些结果加起来。这些天空光照纹理分块实际上都可以看成是得到最终光照结果的一个基函数（basis function）。我们可以使用辐射透过量或是 Stewart 与 Langer 的方法（在[Stewart98]中有相关的数学推导）来计算出这些基函数来。另一个办法是使用[Sloan02]中的方法来计算，它可以处理非常复杂的天空光照分布情况。

4.14.5 活动的云层阴影

在户外场景中是很少没有云层的，而云层造成的阴影也是一个真实场景中所不可或缺的

Download at www.Pin5i.Com

一部分。地形中云层的阴影是通过太阳光线的遮蔽造成的。随着云层密度的增加，阳光被遮蔽得就越多，这样地表上的阴影区域颜色就越深。因此，我们就可以利用下述的简单模型将阴影加入场景中了：我们将把阳光乘以云层密度的倒数，然后把此值加上天空光照的那部分贡献：

$$E(p)=E_{sky}(p)+[1-D(p)]E_{sun}(p) \qquad (4.14.9)$$

在其中 $D(p)$是云层密度投影到 p 点上的等效数值。严格说来，云层与其在地表上阴影的关系是一个斜射，是太阳成一定角度斜射到地表曲面上造成的。不过，由于太阳的距离与阴影区域的尺寸在大小上相差极远，因而我们可以完全将之看成是一个平面投影。在纹理映射坐标与云层阴影中另外还有一个常常假设成立但是精确度比较低的前提条件，那就是太阳是垂直位于地表上方的。如果这个条件成立，我们就可以进行一些优化，直接使用地表平面世界空间中地形顶点的方位（G_{pos}）得到其纹理坐标了，而其他需要的参数只有比例系数与偏移系数而已：

$$uv=G_{pos}Scale+Offset \qquad (4.14.10)$$

如果没有这个假设条件，我们就需要一个更复杂的纹理映射矩阵了，这样在每个顶点上就将花费更多的处理时间。这个与顶点成比例的开销对于地形网格来说是非常重要的，因为它拥有大量的顶点。不过，当对一天之内的时间进行建模的时候，我们应该将太阳的高度角在计算纹理映射的时候考虑进来。

如果不考虑特殊情况，例如暴风云或是暴风雨，云层一般说来是延直线行走的。因此，我们只需要在云层行进的方向上更新纹理的偏移量就可以生成活动的云层了。不过，在投影过程中，我们还需要非常仔细以避免生成超出范围的纹理坐标。

在 CD-ROM 上，我们提供了一个 DirectX8 的 PC 云层映射示例程序，它演示了顶点着色与像素着色硬件在上述方法中的使用情况。首先，顶点处理程序将把世界坐标系中的顶点位置变换到屏幕坐标系中间来。在做这一步的时候要尽可能的早，这样就可以让图形硬件判断是否需要通过裁减去掉顶点了。

```
dp4 oPos.x, VPOSITION, c[CV_VIEW_PROJECTION_0]
dp4 oPos.y, VPOSITION, c[CV_VIEW_PROJECTION_1]
dp4 oPos.z, VPOSITION, c[CV_VIEW_PROJECTION_2]
dp4 oPos.w, VPOSITION, c[CV_VIEW_PROJECTION_3]
```

地形的颜色纹理是作为一个平面投影进行处理的，而且使用的是世界空间顶点的方位在地表平面进行处理。然后我们就可以使用单个的乘加指令（**mad**）对之进行比例变换与偏移变换了。

```
mad oT0.xy, VPOSITION.xz, [CV_BASE_TEX_PROJ].xy,
  c[CV_BASE_TEX_PROJ].zw
```

日照的纹理也是通过同样的方式进行映射的，但是投影则需要重做一次，因为在此处重新计算要比把这些结果存储下来然后复制到输出纹理寄存器中要高效一些。

```
mad oT1.xy, VPOSITION.xz, c[CV_BASE_TEX_PROJ].xy,
  c[CV_BASE_TEX_PROJ].zw
```

Download at www.Pin5i.Com

接下来，我们将对云层进行映射。这又是一次比例变换与偏移操作，不过这里还要加上纹理偏移的动画处理。

```
mad oT2.xy, VPOSITION.xz, c[CV_CLOUD_TEX_PROJ].xy,
  c[CV_CLOUD_TEX_PROJ].zw
```

这第二个云层投影将以同样的方式应用到第四组输出纹理坐标上，其比例系数与偏移系数则有所不同。值得注意的是顶点处理程序唯一的输入流就是顶点在世界空间中的方位。其他的任何数值都是从它经程序计算出来的。这样，我们就可以减少顶点处理器所需要的带宽了（这又是一个高密度网格，例如地形关心的问题）。

在像素着色器中，每个纹理都将根据方程 4.14.9 进行混合，得到最终渲染的结果。在声明了纹理以后，我们就可以计算出两个云层的平均密度了。

```
add_d2 AVG_CLOUD_DENSITY, TEX_CLOUD_LAYER_0,
   TEX_CLOUD_LAYER_1
```

然后，我们将得到的结果减去 1，再使用其相反数乘以日照纹理的贡献。

```
mul CLOUD_SHADOW_LUM, TEX_SUNLIGHT,
  1-AVG_CLOUD_DENSITY
```

接下来，我们再把地形纹理颜色乘上一个天空光照的常量系数。

```
mul SKYLIGHT, TEX_TERRAIN_COLOR, CP_SKYLIGHT
```

最后，我们会将天空光照的贡献加入日照阴影部分，乘以地形颜色。

```
mad OUTPUT_REG, CLOUD_SHADOW_LUM, TEX_TERRAIN_COLOR,
  SKYLIGHT
```

在这个例子中，我们使用 Terragen[Terragen01]事先计算出了天空辐射系数与太阳辐射纹理的常量。它们是从 Terragen 中通过如下方法得到的：

- 对于颜色纹理来说，将太阳垂直放在地形上，不考虑地形阴影。
- 对于天空光照系数来说，在给定的太阳高度角上使用灰度面来渲染地形，对图像中最暗的地段进行采样。
- 对于日照纹理来说，将天空纹理系数从上面的图像中抽取出来。

云层的密度是手工生成的。此纹理也可以用来渲染云层本身。

4.14.6 基于视频的解决方案

有一种生成动态地形光照的蛮力法，那就是使用先进的离线渲染工具渲染出事先做好的视频序列，然后在视频纹理中回放此序列。任何独立于视角的光照效果都可以完全通过视频纹理表现出来。某些与视角相关的效果，例如雾化，还有可能使用传统的技术加入到视频纹理中来。此外，通过在较小范围内对地形使用光照视频纹理，我们还可以获得一种比较新奇的、更加细致的凸凹贴图的效果。

如果没有时间的限制，我们可以在这种方案里面得到无限的复杂度。不过，这种生成视

Download at www.Pin5i.Com

频序列的方法将导致非常高昂的离线开销。在开发过程中，你必须事先计算好这个颇费时间的过程的预算。只要能记住这一点，实施这项技术剩下来的障碍就是如何在 3D 场景中高效地回放此段视频纹理了。

PC 与游戏机的硬件对 3D 环境下视频纹理的支持越来越好了，虽然其步伐还没有顶点处理与像素处理方面的快。例如，在 PlayStation2 上面的图像处理单元（IPU）就可以加速 mpeg 帧的解压。然而，视频数据必须要首先通过 IPU 然后再到达视频显存的传输路线显然比较费劲[Hoffman01]。在 PC 上，虽然先进的视频解压硬件已经出现了一段时间，但这种支持还是直接归于 3D 中的纹理化几何模型支持部分的。

硬件解决方案中一个有吸引力的部分就是可以在像素着色之前从显存里面解压一段视频纹理格式。这样一来，我们就可以大大地减少带宽，加快解压过程了。

4.14.7　非地形对象

在渲染拥有动态光照的户外场景时，有一个重要的问题需要考虑，那就是如何表现出其中的对象，例如在地形中的建筑物与车辆。我们可以使用很多方法来实时地渲染一般的阴影[Haines01]，而这些方法也可以应用到各种环境下的地形阴影中来。

在其中的一种技术，直绘阴影纹理投影（decal shadow texture projection）中，纹理将被重新生成（或是通过将阴影渲染到一个纹理表面上来生成），然后将被映射到地形上去。我们需要一个较小的 z 方向上的偏移以在投影的直绘网格与底层的地形网格不匹配的时候避免发生 z 缓冲区的冲突[McNally99]。这种方法可以生成细致的模糊阴影，能够快速地生成大量静态对象（例如树木）的阴影，而且可以很容易地与一个分层阴影架构进行集成。

对于动态的对象来说，其使用的技术上至投影阴影（drop shadow），下至模板缓冲或是阴影缓冲法。需要选什么主要取决于硬件的性能及限制。不过，在所有的方法中都需要能对任意地形网格进行快速投影。我们可以进行优化，假设太阳的方位是轴对称的，而且对于高程场来说，则假设其有一个轴对称的网格。

4.14.8　结论

我们探讨了各种方法以用来在实时环境中、变化的光照条件下，达到具有照片效果的、高质量的地形光照。若想看一下实时的、具有照片效果的地形光照的示例，请参看彩图 10。不同的方法具有不同的优缺点，以适应不同类型的游戏、平台以及开发过程。如果你正在开发一个具有户外环境的游戏，我们希望至少其中的某种方法能对你有所帮助。

4.14.9　参考文献

[Debevec98] Debevec, Paul, "Rendering Synthetic Objects into Real Scenes: Bridging Traditional and Image-Based Graphics with Global Illumination and High Dynamic Range Photography," Computer Graphics Proceedings (SIGGRAPH 1998), pp. 189～198.

[Haines01] Haines, Eric, et al., "Real-time Shadows," GDC 2001 Conference Proceedings, 对

Download at www.Pin5i.Com

应网址为 http://www.gdconf.com/archive/proceedings/2001/haines.pdf.

[Heidrich00] Heidrich, Wolfgang, et al.,"Illuminating Micro Geometry Based on Precomputed Visibility," Computer Graphics Proceedings (SIGGRAPH 2000): pp. 455～464. 对应网址为 http://www.cs.ubc.ca/~heidrich/Papers/.

[Hoffman01] Hoffman, Naty, et al., "Photorealistic Terrain Lighting in Real-Time," *Game Developer Magazine* (July 2001): pp. 32～41.

[Kautz00] Kautz, Jan, et al., "Bump Map Shadows for OpenGL Rendering," 对应网址为 http://www.mpi-sb.mpg.de/~jnkautz/projects/shadowbumpmaps/.

[Malzbender01] Malzbender, Tom, et al., "Polynomial Texture Maps," Computer Graphics Proceedings (SIGGRAPH 2001): pp. 519～528. 对应网址为 http://www.hpl.hp.com/ptm/.

[Max88] Max, Nelson, "Horizon Mapping: Shadows for Bump-Mapped Surfaces," *The Visual Computer* (July 1988): pp. 109～177.

[McNally99] McNally, Seamus, "Treadmarks Engine." 读者可以通过以下网址得到电话交谈的信息：http://www.vterrain.org/.

[Sloan02] Sloan, Peter-Pike, et al.,"Pre computed Radiance Transfer for Real-Time Rendering in dynamic, Low-Frequency Lighting Environments," to appear, Pro-ceedings of SIGGRAPH2002. Available online at http://research.microsoft.com/~ppsloan/.

[Stewart97] Stewart, James, et al., "Towards accurate recovery of shape from shading under diffuse lighting," *IEEE Transactions on Pattern Analysis and Machine Intelligence* (September 1997): pp. 1020～1025. 对应网址为 http://www.cs.queensu.ca/home/jstewart/papers/.

[Stewart98] Stewart, James, "Fast horizon computation at all points of a terrain with visibility and shading applications," *IEEE Transactions on Visualization and Computer Graphics* (March 1998): pp. 82～93. 对应网址为 http://www.cs.queensu.ca/home/jstewart/papers/.

[Terragen01] Fairclough, Matt, "Terragen." 对应的软件所在网址为 http://www.planetside.co.uk/terragen.

Download at www.Pin5i.Com

4.15　立体图光照技术

Kenneth L. Hurley
NVIDIA Corporation
khurley@nvidia.com

立体图（Cube Map）是在 DirectX7 中为游戏开发者们引入的。在 1999 年，NVIDIA 推出了 Geforce256，从而把立体图推向了广大的 PC 游戏市场。直到现在，它们主要仍是用来渲染有光泽的对象的。不过，我们可以将立体图的使用范围进一步扩大。在本文中，我们描述了一些方法，用来对不同的光照条件以及其他立体图中的属性进行编码。本文将对立体图的属性以及如何对之建立索引进行一番纵览。某些后退法（fallback）使用的参考文献也将被列出来。本文是围绕 DirectX 展开的，但是同样的原理也适用于 OpenGL。

4.15.1　立体图的物理属性

立体图在硬件中是使用 6 个纹理来表示的。每个纹理代表的都是立方体上的一个面。一般说来，在编写代码的时候就会确保可以使用一个反射矢量来对立体图进行索引编号。接下来，我们将把一个有三个分量的矢量传到硬件里面去，这样硬件就得到相应的信息，可以选出正确的面与对应于这个面的 *uv* 坐标来了（请参见图 4.15.1）。硬件使用的是三个分量中最长的一个来选择面。例如，如果我们把矢量(0.4f, 0.5f, 0.7f)传给了硬件，它将选出立体图在正 *z* 方向上的面，因为 0.7 是最大的分量。接下来，硬件将使用其他的两个分量(0.4f, 0.5f)来选出 *uv* 坐标。在立体图与一般的纹理选址中间的差异之一就在于其 *uv* 坐标是有符号的值，而纹理地址的选择则要使得立体面的中央位于 *uv* 坐标的(0, 0)点上。一般的纹理映射则使用左上角作为 *uv* 坐标中的(0, 0)点。

Download at www.Pin5i.Com

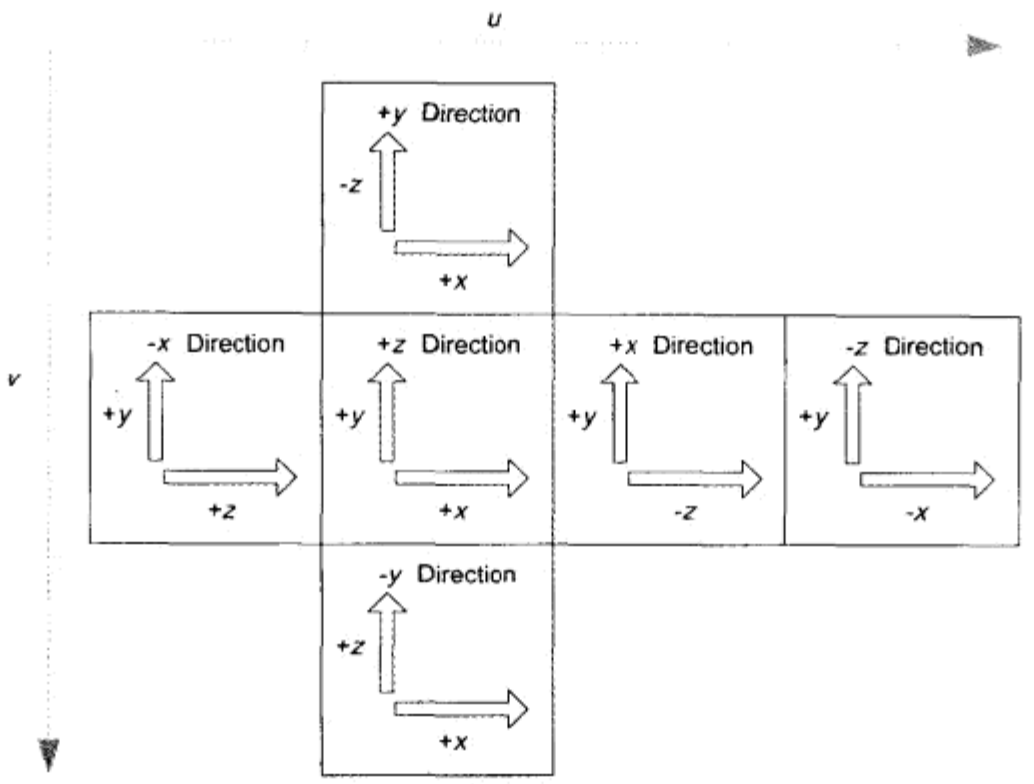


图 4.15.1 一个立体图在硬件中的表示

4.15.2 如何与立体图进行数据交换

在 DirectX8 中载入立体图可以通过几种方法来实现。最简单的方法就是使用微软的 DirectX Texture 工具来创建一个.dds 的立体图文件，这个工具是 DirectX8 自带的。下面的代码展示了如何从一个.dds 文件中载入立体图：

```
LPDIRECT3DCUBETEXTURE8 m_pCloudTexture;

D3DXCreateCubeTextureFromFile(m_pD3DDev, "cloudtex.dds",
    &m_pCloudTexture));
```

一定要确保在对 LPDIRECT3DTEXTURE8 指针进行操作的时候使用的是 D3DXCreateCubeTextureFromFile 而不是 D3DXCreateTextureFromFile，因为 DirectX8 对这个调用不会报错，但是在渲染的时候会产生奇怪的结果。为了渲染选出的纹理，只要在纹理化的某个阶段中设置一下纹理就可以了。下面的代码展示了如何做到这一点：

```
m_pD3DDev->SetTexture(0,(LPDIRECT3DBASETEXTURE8)m_pCloudTexture);
```

载入纹理的另一种方法就是从 6 个独立的位图文件中将其载入。这需要调用 D3DXCreateCubeTexture 来创建相应的立体图纹理，然后再调用 GetCubeMapSurface 来载入立体图的每个表面。此外，你还可以使用 D3DXLoadSurface*系列的函数来载入纹理图。

4.15.3 使用立体图进行渲染

设置一个立体图的 3D 纹理坐标可以通过几个途径来办到。在 DirectX8 推出之前，一个开发人员会使用类似于下面的代码来设置对立体图的渲染：

Download at www.Pin5i.Com

```
for( LONG i = 0; i < cV; i++ )
{
    // eye vector (doesn't need to be normalized)
    FLOAT fENX = m_vEyePt.x - pVIn->v.x;
    FLOAT fENY = m_vEyePt.y - pVIn->v.y;
    FLOAT fENZ = m_vEyePt.z - pVIn->v.z;

    FLOAT fNDotE = pVIn->v.nx*fENX + pVIn->v.ny*fENY +
                   pVIn->v.nz*fENZ;
    FLOAT fNDotN = pVIn->v.nx*pVIn->v.nx + pVIn->v.ny*pVIn->v.ny +
                   pVIn->v.nz*pVIn->v.nz;
    fNDotE *= 2.0;

    // reflected vector
    pVIn->v.tu = pVIn->v.nx*fNDotE - fENX*fNDotN;
    pVIn->v.tv = pVIn->v.ny*fNDotE - fENY*fNDotN;
    pVIn->nz = pVIn->v.nz*fNDotE - fENZ*fNDotN;

    pVIn++;
}
```

这段代码将根据每个顶点计算反射矢量，然后将这些矢量转化为 3D 纹理坐标，这些坐标将被传给硬件处理。

DirectX8 中给出了新的用来处理立体图的方法。现在，我们可以设置顶点缓冲区来使用固定函数处理流程（fixed function pipeline），或是使用顶点和像素着色器了。DirectX8 的固定函数处理流程与 DirectX7 的非常类似。你可以通过设置不同的纹理化阶段的状态与渲染状态来对之进行设置。在固定函数处理流程中，立体图特定的渲染状态是 D3DRS_LOCALVIEWER 与 D3DRS_NORMALIZENORMALS。需要考虑的纹理化阶段则是 D3DTSS_TEXCOORDINDEX。在本文中，我们将主要关注 D3DTSS_TCI_PASSTHRU 这个纹理化阶段。它将提示硬件直接使用纹理坐标，而不需要对其进行任何变换。

在 DirectX8 及其更高版本中，顶点着色器都可以用来改变基于旋转的法线。接下来，顶点着色器将把法线放在输出纹理坐标中，此坐标对应的是立体图被选出来的纹理化阶段。下面的代码片段展示了如何做到这一点：

```
;transform normal, put into texture coordinate output
dp3 oT0.x, srcNormal, c[CV_ROTATION_X]
dp3 oT0.y, srcNormal, c[CV_ROTATION_Y]
dp3 oT0.z, srcNormal, c[CV_ROTATION_Z]
```

4.15.4　对云层进行编码

把云层（cloud cover）编码成一个立体图并不如想像中的那么简单。分数布朗运动（fBm）以及 Perlin 噪声[Perlin95]都被广泛应用于创建 2D 云层纹理的工作中。这项技术相当好用，而且可以使一般的纹理进行无缝接边。不过，这些图像在载入立体图的时候会出现明显的缝

Download at www.Pin5i.Com

隙。出现缝隙的原因是 2D 图像的纹理只在平行的边上匹配。顶端与底部的边只能互相匹配，但是却不能与侧面的表面进行匹配，因为它们顶上与底下的边不是一样的。其解决方法就是要使用 3DfBm 或是 Perlin 噪声函数将云层编码为立体图。下面的代码就起到了这样的作用：

```
D3DXVECTOR3 tVec;

// now loop through each pixel of the cube map, gathering the fBm
for (i=0; i<CUBEMAP_TOTAL_DIR; i++)
{
    dest = m_CubeFacesData[i];
    for (pos.y=0; pos.y < m_CubeSize.cy; pos.y++)
    {
        for (pos.x = 0; pos.x < m_CubeSize.cx; pos.x++)
        {
            GetCubeVector(i, &pos, &m_CubeSize, &tVec);
            *dest++ = fBm(tVec);
        }
    }
}

//=============================================================================
// Function:    GetCubeVector
// Description: returns a 3 vector given a from an x,y, face of a cube
//                          map
// Parameters: face = face of cube map to calculate vector from
//             tInfo = x,y, width and height of cube map face texture
//             vecOut = 3 vector pointer for output
// Returns:    pointer to vecOut
//=============================================================================
D3DXVECTOR3 *CCubeMapDoc::GetCubeVector(DWORD face, const CPoint *pos,
                                        const CSize *size, D3DXVECTOR3 *vecOut)
{
    float s, t, sc, tc;

    // move pixels to center
    s = ((float) pos->x + 0.5f) / (float) size->cx;
    t = ((float) pos->y + 0.5f) / (float) size->cy;
    sc = s*2.0f - 1.0f;
    tc = t*2.0f - 1.0f;

    switch (face)
    {
    case CUBEMAP_POS_X:
        vecOut->x = 1.0f;
        vecOut->y = -tc;
        vecOut->z = -sc;
        break;
    case CUBEMAP_NEG_X:
        vecOut->x = -1.0f;
        vecOut->y = -tc;
```

Download at www.Pin5i.com

```
            vecOut->z = sc;
            break;
        case CUBEMAP_POS_Y:
            vecOut->x = sc;
            vecOut->y = 1.0f;
            vecOut->z = tc;
            break;
        case CUBEMAP_NEG_Y:
            vecOut->x = sc;
            vecOut->y = -1.0f;
            vecOut->z = -tc;
            break;
        case CUBEMAP_POS_Z:
            vecOut->x = sc;
            vecOut->y = -tc;
            vecOut->z = 1.0f;
            break;
        case CUBEMAP_NEG_Z:
            vecOut->x = -sc;
            vecOut->y = -tc;
            vecOut->z = -1.0f;
            break;
        }

        D3DXVec3Normalize(vecOut, vecOut);
        return vecOut;
    }
```

为了在一个天空的球面上使用这个立体图，我们需要创建一个代表世界大气层的球面，使用坐标在立体图中进行查找。我们不需要把坐标归一化，因为在立体图中进行查找并不需要归一化的矢量。下面就是一段顶点着色器的代码，它展示了这种技术：

```
vs.1.1

;transform position
    dp4 oPos.x, srcPosition, c[CV_WORLDVIEWPROJ_0]
    dp4 oPos.y, srcPosition, c[CV_WORLDVIEWPROJ_1]
    dp4 oPos.z, srcPosition, c[CV_WORLDVIEWPROJ_2]
    dp4 oPos.w, srcPosition, c[CV_WORLDVIEWPROJ_3]

; Output texture coordinates
    mov destTexCoord, srcPosition
```

要使云层活动起来是相当简单的。通过 DirectX8，你只要将旋转矩阵传给顶点着色器，对天空进行旋转就可以了。

经云层编码为立体图的一个副效应就是云层产生的阴影可以描画到地形上面了。我们可以云层的反色灰度图减去天空的颜色得到 alpha 值。我们同样可以通过一个像素着色器得到颜色值，只需要从 1 中减去原始云层立体图中的颜色就可以了。地形的位置可以被用来作为立体图的索引，而 alpha 分量则可以用来使地形变暗。这样我们就得到了附加值，可以从事

Download at www.Pin5i.Com

先绘制好的云层中生成具有半透明边缘的模糊阴影了。而由太阳导致的光照的计算也可以使用这个值来改变。例如，光照计算可以与 alpha 通道值的一半进行混合，得到具有半透明效果的云层。

4.15.5 在一个立体图中对光源进行编码

动态光源与静态光源都可以通过很多方法编码到一个立体图中去。无限远的静态光源，例如阳光，可以使用前面描述的技术进行预渲染加入索引编码到立体图中去。顶点的法线可以作为索引加入立体图中以得到立体图中渲染的光源。对于无限远的静态光源来说，这种模式是相当好用的，但是对于动态光源来说就不太一样了。动态光源一般说来都需要进行多重的纹理处理。其中主要的思想就是要使用 90° 角的视场在立体图中渲染移动的光源，然后再在立方体的每一面放置镜头。在镜头中要渲染的位置就是被光照对象所在的位置。这种技术对于下面的场景是适用的，如在一个迪厅里面，大概一共会有 100 个光源。渲染迪厅与在屋里面的对象可以使用这种技术来完成。要是这种技术成功运用，所需要的立体图也不用太大，因此在渲染的时候记住这一点就够了。

4.15.6 在立体图中渲染散射光照

使用上面的代码在立体图中进行遍历生成云层的纹理，我们同样也可以将具有多个光源的场景与散射图集成在一起。对于立体图上某个面的每个 *uv* 坐标来说，我们都将计算出一个矢量 *v*（法线）来。对于矢量 *v*，我们还将从每个立方体表面的每个 *uv* 坐标上计算出另一个矢量 *l*（光照）来。接下来我们将把这两个矢量进行点积，以得到两个矢量夹角的余弦。这与标准的顶点光照中的计算是一样的。如果点积的结果是负数，那么我们就知道矢量 *l* 对矢量 *n* 没有贡献了，否则，得到的点积就将作为一个权重值，用来将立体图上的像素混合在一起。这些被选出来进行混合操作的像素是从用来计算出矢量 *l* 与 *n* 的 *uv* 坐标中得来的。它实际上就是对一个半球面上的像素进行了采样，这个半球面的中心就是矢量 *n*。在图 4.15.2 中的立体图是通过此采样得到的结果。为了将这个立体图使用到光照中间去，我们还需要对采样值进行 gamma 校正，这是因为在转化到较低动态范围的过程中我们丢失了一些区域。

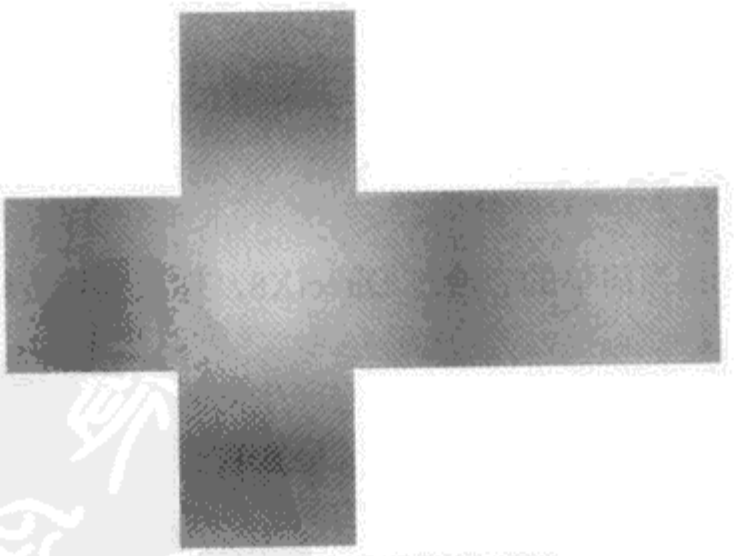


图 4.15.2 编码后的散射立体图

Download at www.Pin5i.Com

对于镜面图的编码与散射图的编码是大致一样的，都需要对立体图进行遍历，然后在每个方向上进行采样，如前所述。不同的是，我们不需要进行点积以得到权重值，而是要得到点积的第 n 次方根，在此处 n 被称为镜面指数。在这个阶段上可能需要进行一些人工干预，因为镜面指数在不同的材质之间是不一样的。

4.15.7　将日夜循环编码进立体图中

日夜循环可以与云层立体图一起使用，使用的就是在生成云层立体图中讲过的技术，只需要对云层的灰度信息进行编码。在对天空的球面进行光照处理的时候，将点积与一个数值关联起来，这个数值代表的就是天际线上变换颜色的点中位置最低的一点。我们将天空的颜色作为光照计算的一部分传递给像素着色器，然后，随着太阳的旋转，我们可以在顶点所在的存储区里面写入数据改变天空球面的颜色。随着太阳逐渐接近地平线，天空的颜色也随之由蓝色变成了橙色。当太阳处在地平线之下以后，我们将基于时间把颜色从橙色变为黑色。将点积与这个数值关联在一起之后，我们就可以基于这个数值得到任意颜色了。

4.15.8　结论

除了渲染有光泽的对象以外，立体图还可以用来生成诸多的效果。在本文中，我们描述了一些立体图的有趣的应用。值得注意的是，即使硬件不能支持立体图，你仍然可以通过使用球面图或是双抛物面图[Heidrich99]分享这些技术带来的好处。在这种情况下，我们就要使用纹理坐标生成或是 CPU 来计算 uv 坐标以进行处理了。

以上技术的几个演示程序的源代码在 CD-ROM 上都有。此外，还有一个在地形上的云层动画立体图与一段散射光照（envlight.nvp）代码片段。另外，还有彩图 11 的演示。

4.15.9　参考文献

[Debevec97] Debevec, Paul E. and Jitendra Malik, "Recovering High Dynamic Range Radiance Maps from Photographs," (SIGGRAPH 1997): pp. 369～378.

[Elias02] Elias, Hugo, "Perlin Noise," 对应的网址为 http://freespace.virgin.net/hugo.elias/models/m_perlin.htm, January 5, 2002.

[Heidrich99] Heidrich, Wolfgang and Hans-Peter Seidel, "Realistic, Hardware-Accelerated Shading and Lighting," Computer Graphics Proceedings (SIGGRAPH 1999): pp. 171～178.

[Perlin85] Perlin, Ken," An Image Synthesizer,"*Computer Graphics Proceedings* (SIGGRAPH 1985): pp. 287～296.

Download at www.Pin5i.Com

4.16 程序纹理

Mike Milliger
2015, Inc.
mikem@2015.com

程序纹理（procedural texturing）是一个正在展开研究的领域，它包含的内容从傅立叶综合至随机建模，都是希望能基于物体的化学成分创建其纹理。Darwyn Peachy 在 Texturing & Modeling 中将程序（procedural）定义为一种“用来区分使用程序代码而不是数据结构描述的实体[Ebert98]”的方法。在游戏中与本文中，我们主要关注的是如何通过数学函数将一个纹理图像映射到一个表面上去。

不管在什么时候，当我们与艺术创作人员讨论程序纹理映射时，几乎无一例外，他们的第一反映就是：“怎么，你是想让我失业么？”现在，所有的内容创作人员都暂时息怒吧。最终看来，手工制作的纹理总是要比纯粹通过程序进行的纹理合成要美观得多的。在一个作品里面纯粹使用程式化的系统不仅不利于开发过程的任务均衡分配，而且系统最终从美学的角度来看也难以令人满意。只有将程序方法与传统方法结合起来，我们才能加强游戏的艺术性，使得游戏中的世界更加真实。我们不仅可以通过程序生成独特而崭新的纹理，而且还可以操纵生成的图像的众多属性（色彩、方位、宽高比等等）。本文的目标就在于引入新的概念，以用来使用程序对图像为游戏进行操纵与合成，并为实现给出一些指导性的原则。

4.16.1 参数与函数

创建程序纹理需要两个要素——函数，用来生成纹理；参数，用来控制函数。

1. 参数

传递给生成函数的参数控制了输出的纹理。在数学函数中对变量进行操作通常就可以在程式化的纹理中得到不同的变化。就像一个音频工程师可以使用各种电子处理仪修正音频信号一样，视觉效果的艺术创作人员也可以操作各种信号发生器与处理函数以创建一个独特的纹理。一般说来，我们使用的是同样的术语。信号（或称基本纹理）将在不同的波长上被创建出来，然后通过不同的控制器进行过滤与混合。对于程序算法而言，常见的参数包括了“频率”、“幅度”以及“偏移量”这样的术语。然而，我

Download at www.Pin5i.Com

们操纵的不是声音，这些函数是用来操纵纹理属性的，例如色彩值与透明度。

2. 函数

在游戏中使用到的程式化的效果主要出自 Ken Perlin 提出的一种被称为“噪声”的算法。伪随机数字生成器（PRNG）是大部分噪声函数的核心，而这种随机性可以帮助生成具有自然表现力的效果。噪声是程序纹理的主要组成部分，也是游戏中最实用、应用最广泛的函数之一。在其中又有很多种噪声——数值噪声、梯度噪声、数值梯度噪声、点阵卷积（lattice convolution）、稀疏卷积。所有这些函数都将伪随机数转变为图形图像纹理源的变体之一[Macri00]。PRNG 可以生成一个点阵。在此处，点阵指的是一个 n 维的、平滑的网格，在其上均匀分布着纹理空间上面的点，每个点都包含着一个随机浮点数。这些点之间的间隔决定了频率，多个不同频率的噪声源可以合并在一起，每个噪声源（或称是音阶）都可以被加在一起使用各种数学方法得到最终的图像。然后我们就可以仔细地调整这些方法以创建特殊效果了。例如，Kim Pallister 在《游戏编程精粹 2》[Pallister01]的文章中就阐述了如何使用噪声，通过将各个音阶加在一起得到湍流噪声来生成动态的、程式化的云层。

噪声算法是生成一个程序纹理源的直接方法。另一个生成纹理的方法是要得到一个纹理源，然后通过某个算法函数随着时间的不同对之进行相应的修正。其中一个这样的算法被称为自动机（automaton）[Macri00]。当用来生成程式化纹理的时候，原始的纹理将被分成一组分离的采样，称为“单元”或“纹理采样”。图像中每个纹理采样处的色彩都定义了当前单元自动机系统的状态。在函数的每一次迭代中，纹理中的每个纹理采样都会根据一组规则得到修正。这些规则一般牵涉到设置每个纹理采样为一组特定的相邻纹理采样的加权平均值。只要使用这些简单的系统，我们就可以得到很多不同的效果了。例如，设置每个单元的值，使其等于这个单元之下的各单元的平均值的话，我们就可以得到火焰一样的效果了。在系统的每次迭代中，纹理中虚拟的火焰效果都会上升一点[Macri00]。

迭代过程，例如单元自动机，是直接式方法，例如噪声函数，的有效补充。举个例子，我们可以使用一个伪随机噪声过程来生成一组随机分步的点，然后我们可以使用这些点来作为迭代式系统的原始状态开始运行。

4.16.2　进入游戏世界

程序纹理技术已经在很多计算机图形程序里面得到了应用，例如科幻可视化效果以及视觉效果。在本文中，我们将主要关注于这些技术在游戏程序中的应用。

1. 程序纹理的使用

对于主流游戏开发来说，程序纹理仍然还处在其早期发展阶段。在以前游戏的大部分效果中，所使用到的仅是简单的水流或是火焰效果。不过，程序纹理不仅能应用到自然现象的表现上面，还可以用来生成木头、大理石、白雪、草地、星辰以及具有体积形状的效果，例如云层[Pallister01]。这些都是游戏中所需要使用到的常见的表面类型。

生成的纹理可以被静态地使用，就如其他类型的纹理一样。通过使用当代图形硬件的纹理变换能力，我们可以使纹理在图像的表面上移动。我们可以生成一系列的纹理，然后将之

Download at www.Pin5i.Com

作为动画纹理回放出来。这在很多游戏中都是很常见的情况。不过，在这种情况下，在表面上就很容易察觉出其运动模式，而且在动画重放的时候也很容易察觉出来。在某种程度上，这会使得玩家对游戏产生厌倦感。程序纹理真正的力量在于它可以实时地在游戏中生成纹理。由于这些过程都是数学的而且可以产生无限的变化，因此我们就可以消除重复的现象了。

我们除了可以将程序纹理当成色彩的原材料以外，还可以将之在多纹理系统里面作为立体地图、散射地图、镜面图，或是动态环境地图来使用。我们可以将程序纹理作为一种手段来检验表面上的缝隙或是凹痕，这只需要使用一个函数来对纹理空间的法线进行检查就可以。带有噪声的散射地图可以打破大片同样的色彩带来的单调性，使之得到一些变化（大片同样色彩的区域比较显眼，而且看起来很不真实）。我们经常使用分形地形作为一个噪声图像的高程场（位移图）放置在一个较为密集的网格中。很多户外图形引擎都使用这项技术来为单一的地形生成几何模型，而不用把所有的数据都保存下来。由于我们使用的是带有随机种子的PRNG 生成的噪声图像，因此当玩家返回同一区域的时候，我们仍然能够生成同样形状的地形。

2. 程序纹理的优点

在你的游戏里面使用程序纹理有很多好处。在决定究竟是否要使用它们的时候，最终总是会归结为一个古老的计算机科学上的问题：究竟应该保持更小的程序空间，还是应该拥有更快的运行速度？虽然我们需要一些 CPU 的时间来创建纹理，但是基本上程序纹理是不需要耗费什么内存的。此外，程序纹理没有什么带宽上的限制，你不需要不断地载入或是引用纹理内存以得到像素的色彩。由于纹理在每一帧都会被重新生成，因此图像将始终保持一致。它在第一人称视角的射击游戏里面特别适用，因为此时视点与表面之间的距离可以很小。随着玩家接近一棵树，这棵树上的树皮纹理可以在每一帧都重新生成一次，随着观看者的接近而提高其细节的表现力。

虽然大部分程序纹理都是使用高度优化的汇编语言编写的，但是在一个场景里面对大量的程序纹理进行渲染仍然会降低游戏的帧率。在游戏的某一关中，我们可以在很多表面上使用程序纹理，只要它们不会在同一时刻可见就行。把屏幕上看不见的纹理裁减下来，忽略其计算是很简单的。程序纹理可以被重用，而我们也可以在多个对象上面使用同一个纹理，例如我们可以在一个通道里面为所有的火炬使用同样的火焰纹理。虽然由于这些过程比较难以控制且具有随机性，因而比较难以调试，但是它也可以给你一个惊喜（那种突然发现了宝藏一样的心情）[Ebert98]。尽管如此，我们仍然可以通过参数对这些过程加以某种程度上的控制，使之改变输出的纹理，而不至于总是生成同样的东西。

程序纹理可以是一维、二维或是三维的。三维的纹理可以作为新的工具给艺术创作人员和开发人员生成新的效果而使用。一个经过程序生成的三维纹理可以作为一个切面以任何角度切入色彩空间里面去，与其他色彩空间的纹理相匹配。此项技术也被称之为“立体纹理”（solid texturing）。色彩空间使用了色彩值组成的 3D 数组保存纹理，加上了纹理的对象就处在这个色彩空间之内。可分解的环境在游戏中越来越流行了，而立体纹理则可以使得这种场景中的对象更加真实。假设有一个柱子，它的程序纹理是大理石类型的，它不小心被错放到火箭发射的场地上去了，当然它会被分解为一个个石块。在这种情况下，只要我们能保证它们仍在同一个色彩空间之内的话，不仅柱子的内部仍然是大理石纹理的，而且被分解后的石

Download at www.Pin5i.Com

块也将仍然是大理石纹理的。我们可以使用同样的技术生成侵蚀物或是云层。程序纹理的区域并不固定，而且可以覆盖任意尺寸的区域，而不会导致重复或是出现缝隙。为了给较大的区域加上更多的细节，只要增加函数中的层数就可以了。

3. 在游戏中使用程序纹理

你可以在游戏中使用两种类型的程序纹理：实时纹理或是事先生成的纹理。事先生成的程序纹理可以在内容类的程序里面使用，例如 Lightwave 或是 Photoshop，也可以在专业的程序纹理软件包里面使用，例如 DarkTree，然后导入到传统的方法里面去（这样就丧失了实时计算的好处了）。DarkTree 可以让用户可视化地修改参数，其手段是将函数串联起来以生成不同的效果。其结果可以实时地显示出来，这样用户就可以在此效果上得到最快的反馈了。同样你也可以把动画纹理按帧的顺序保存起来。

如果你希望在自己的游戏引擎中实现一个实时程式化系统，那么我们可以在这里给你一些指导性的信息。第一步就是要创建一个相当简单的固定流程，最重要的就是要让非开发人员也能看懂此架构。如果能实现一个带有滑动条与预览窗口的编辑工具，那就一定要实现它。其中的关键就在于可以让内容创作人员发挥其特长，自由地进行创作，而不用在文本编辑器里面输入各种古怪的参数。在编辑工具里面应该把一个表面上所有的参数和过程名都保存在一个着色文件里面去。当相应的关卡或是区域被载入的时候，如果要绘制出此表面，只需要将此工具生成的参数传递给相应的过程就可以了。另外要注意为表面赋予合理的缺省参数值，当有表面描述值存在时再将其覆盖掉。

将来，游戏在纹理与着色这一点上可能会使用一个类似 Renderman 的环境。大部分游戏引擎都已经拥有或是正在制作着色系统。如果针对游戏能有一套标准的着色语言，那我们就可以使用更好的方式来定义参数，为各种材质编写函数了。

4. 程序纹理的其他应用

虽然本文主要关注的是图像的合成，但是大部分程式化技术都可以扩展到游戏开发中的其他方面去。我们已经讨论了使用噪声来生成地形的问题。对于动画来说，用户可以在基于 IK 的死亡场景的动画中使用噪声，这样，在同一个房间里面倒下的几个坏蛋就可以死态各异了。使用造型目标时，我们可以添加一点随机因素，这样就可以得到不同的结果了，例如随机的闪烁效果或是不同的海豚摆尾。Perlin 已经展示了使用噪声生成面部表情的效果[Ebert98]。程式化建模对于游戏开发人员来说也是一个逐渐开始感兴趣的领域，特别是在某些内存较少的架构中。树木与其树枝可以程序化地生成，这样就可以得到不同的植物形态，而不用保存大量的几何模型数据了。

4.16.3　硬件加速

在运行期对每个像素计算纹理将要花费很多计算时间。如果我们能将逐像素计算的工作转移到图形处理硬件上，而将主处理器的负担减轻的话，我们就可以减少程序计算的开销了。现在，随着最新的消费型图形硬件的诞生，我们可以在游戏中得到实时的程序纹理，而不会给主处理器增加很大的负担了。如果你没有这样的图形处理芯片组可以进行逐像素的操作与

Download at www.Pin5i.Com

上色效果的话，那么这样的计算就会花费主处理器的时间了。

实时生成程序纹理的主要问题就是看图形处理硬件能不能直接对一个表面进行渲染。如果有合适的硬件的话，我们是可以通过 Direct3D 或是 OpenGL 做到这一点的。举个例子来说，伪随机函数可以被用来生成几个噪声源纹理，各自具有不同的频率。接下来，这些源噪声纹理可以合并在一起，使用现有的硬件混合与过滤模式将之渲染到一个表面上去。对于支持多纹理渲染的图形处理硬件来说，我们可以在一个通道里面对多个纹理进行合并。经过这些操作得到的纹理可以在游戏场景中应用到一个对象上面去。

除了这种直接的程序纹理创建之外，我们甚至可以在硬件里面直接创建迭代式的单元自动机系统。只要有一个硬件能支持同时四个纹理的混合，然后将结果再反馈回系统里面去，我们就可以生成火焰与流水的动画效果了。

4.16.4 结论

对于生成新的纹理与修改现存的纹理来说，程序纹理是一种强大而灵活的方法。现在此项技术尚未得到广泛的使用，虽然函数可以增强游戏的艺术效果。这主要是因为进行逐像素操作生成纹理花费的时间比较长。但是现在的消费级硬件已经可以在图形芯片组上处理像素操作了，这就将主处理器从巨大的开销中解脱了出来，因此在将来的游戏中，我们期望着可以看到更多的程式化生成的效果。毫无疑问，其中最流行的将是噪声系统与单元系统，它们将领导潮流。

附带例子中的代码是由 Simon Green（NVIDIA）提供的，它展示了如何使用图形处理硬件来创建动态的程序 3D 纹理。它使用了单个的预先计算出来的 3D 噪声纹理，然后通过纹理矩阵对之进行多次访问以控制其频率，让寄存器组在每一层上进行加权与加和，生成最终的色彩。

此处的 3D 纹理是 64×64×64 的纹理采样，使用了单通道的亮度格式，因此大概会花费 256KB 内存。值得注意的是虽然这段纹理的周期较短，但是一旦将之混合起来以后，观察者很难发现重复的地方。此项技术新奇的地方就在于我们事先使用了立体过滤器对噪声纹理进行了过滤，这样就避免了某些可能发生的错误，如果我们使用随机纹理采样，然后让硬件在中间进行线性插值的话就有可能出现这些错误。此外，我们还可以预先计算出 Perlin 的“湍流”方程中需要的绝对函数。

为了得到最终的分形噪声模式，我们在噪声纹理中使用了 4 个或是更多的 3D 纹理查找表。每一层都被看成是一个音节，因为每次频率大概就会翻一倍，这与音乐里面的音阶很相似。我们使用纹理矩阵将每一组纹理坐标进行缩放，这样就增大了每个音阶的空间频率，但是使用一个顶点处理程序就可以轻易做到这一点。我们还可以使用不同的比例通过平移或是旋转每层的纹理得到更有趣的动画效果。这比真正的四维噪声可要容易得多了。当每个音阶的动画速度与空间频率成一定比例的时候，似乎就可以得到最佳的效果了。

最后，寄存器组可以被用来给每个音阶的数值加权，方法就是修正其幅度，然后将所有的音阶加起来。一旦我们拥有了最终（标量形式的）加和起来的噪声值，我们就可以使用多种方法来从中生成色彩值了。对于简单的色彩效果来说，我们可以使用寄存器组基于噪声值在两个或是更多的色彩之间进行插值，但是对于非线性比较明显的模式来说（例如有脉纹的

Download at www.Pin5i.Com

大理石），我们则可以使用颜色表。要达到这样的效果，一种方法就是将场景渲染到一种纹理上，然后在第二次处理的时候，使用 dependent_gb 纹理着色器与一个一维的纹理绘制出一个与屏幕对齐的方块来，这个一维的纹理是用来将噪声映射为色彩值的。图形硬件的 alpha 测试函数也可以被用来对象素进行判定，抛弃掉那些高于或是低于某门限值的像素，得到有趣的像是被“侵蚀”了的图像。将来的可编程硬件应该可以一次就处理完这些操作（甚至更多的操作），而且可以提供更加灵活的程序纹理的机制。

4.16.5　致谢

在此要感谢 Simon Green 与 NVIDIA，他们给我提供了演示的代码与彩图 12 以及彩图 13。

4.16.6　参考文献

[Ebert98] Ebert, David, et al., *Texturing & Modeling*, Second Edition, AP Professional, 1998.

[James01] James, Greg, “Operations for Hardware-Accelerated Procedural Texture Animation,” *Game Programming Gems 2*, Charles River Media, Inc., 2001. 在下面的网址上有相应的演示程序 http://developer.nvidia.com.

[Macri00] Macri, Dean, and Kim Pallister, “Procedural 3D Content Generation,” 对应的网址为 http://cedar.intel.com/, February 8, 2001.

[Pallister01] Pallister, Kim, “Generating Procedural Clouds Using 3D Hardware,” *Game Programming Gems 2*, Charles River Media, Inc., 2001.

Download at www.Pin5i.Com

4.17 独一无二的纹理

Tom Forsyth
Mucky Foot
tomf@muckyfoot.com

在真实世界里，没有两个表面是完全一样的。它们之间具有无数细微的差异——木头的脉纹、表面的纹理、泥垢的痕迹、磨损之处、褪色的油漆、足迹、乱画的痕迹，以及诸如此类的东西。尽管如此，计算机游戏还是在不断地重用同一段纹理。在以前的日子里面，这么做是出于现实的原因，显存比较小，而且在一个场景里为每个表面使用不同的纹理将导致过量的纹理页面交换，甚至会导致出现图像画面的暂时静止。

现在，显存相对来说便宜了很多，而且数量也比较大了，但是又产生了一些新的问题。我们需要的是更高分辨率的纹理，而这要求的内存则又大得惊人。此外，我们需要创建巨大的地图，这样一来，就不可能仅由单独一组艺术创作人员来制作出每个表面特有的数据，即使我们有足够的磁盘空间可以把这些数据都给保存下来。

很多此类的问题都可以使用程序生成纹理来解决，而不需要让艺术创作人员来手工制作出每个纹理采样。在本文中，我们的核心思想就是要如何使用相对较少的艺术创作人员生成的源纹理，将它们在运行期使用各种混合方法、分形方法以及随机数的方法将之合并起来，创建出场景所需要的纹理。

4.17.1 程序纹理

正如 Mike Milliger 在本章的文章中描述的一样[Milliger02]，程序纹理使得艺术创作人员做更少的像素级的工作，而可以埋头专心生成不同类型表面的描述信息。这些描述信息可以应用到很大的区域中以及此区域的对象中去。由于这些描述信息使用的是伪随机数来生成某些数值的，因此每个示例都将在精确的表现形式上略有不同，正与在真实世界中的情况一样。最重要的一点就是要让艺术创作人员能对此过程中的任意一个阶段进行完全的控制——对于某些重要的对象，他们可能希望将其锁定在某个状态，让其不再变化。不过，对于一个游戏世界中的几乎所有地形与场景，这种控制都是没有必要的。

由于每个表面生成的纹理都是独一无二的，因此每一个都可以与其映射到的对象所在的几何形状相适应。最明显的好处就是纹理之间的缝隙再

Download at www.Pin5i.Com

也没有了——连续的纹理，例如岩石或是木头，可以连续地分布在任何几何形状里面去。艺术创作人员一直为传统的工作方法所折磨，因为要想避免产生缝隙而同时又要求避免产生过量的纹理采样变换是非常困难的。此外，暴露在外的边角会被磨平，藏在里面的边角会沾染灰尘，而任何暴露在阳光之下的表面则会有所褪色。

4.17.2　智能纹理缓存

在典型的图形处理系统中，其核心部分就是一个纹理缓存。图形处理系统可以将不同的源纹理智能地合成起来以得到最终的结果。在某些情况下，这种合成可能会相当复杂而缓慢，因此一定要注意不要出现缓存频繁交换的现象，这样才可以与那些不是很智能但是却比较快的缓存协同工作（例如，Direct3D 中的纹理缓存）。

如果你认为或是已经发现了纹理缓存的速度比较慢，我们还可以做一些修正以提高性能。例如，你可以针对某个对象做出其需要的最高分辨率的纹理序列。这通常需要对一个常量（离线计算的）做一次除法，我们只需要把这个常量除以对象的距离，然后对之使用以 2 为底的对数就可以了。这个结果得到的是一个比较保守的估计，而且还需要假设纹理与观察者的视线是垂直的。不那么保守的估测则还需要考虑表面与观察者的方向，但是弹门的计算开销也比较大（例如，在一个曲面上，很有可能某些曲面总是大致与观察者垂直的）。

4.17.3　合成模型

最简单的方法就是 Photoshop 的栈式法——将一组 RGBA 的层放在上一次某类型混合的结果之上，在每一层中都有一个源纹理。不过，任何有实际意义的由随机数控制的混合都要求使用一种更复杂的系统，其使用的是树状混合，而不是栈式混合。除此之外，你通常还需要将不同的 alpha 通道合成起来，然后使用最终得到的 alpha 通道进行色彩的混合。由于这些仅有 alpha 通道的混合很常见，因此我们需要给层与混合都加上标记，标志其为仅有 alpha 通道的，这样就可以确保不用总是使用 32 位的 RGBA 纹理了——这样，我们的内存访问效率就更高了。

这种树状模式在大部分情况下只需要进行最多 10 次混合。我们将仍然使用层与混合的术语，而不使用树节点与叶节点，不管其内部表现形式具体是什么样的。

4.17.4　层的映射与变换

一个层最基本的功能就是可以将之映射到几何形状上去。这通常是由 3D 软件包来处理的，它可能会使用显式的 *uv* 坐标，也可能使用某种形式的平面或是球面投影。它们包括了代表源纹理的那些层、中间层以及最重要的目标层——此纹理由合成过程生成，随后将被传递给图形处理芯片，最终成为图像的纹理。

在源与目标之间的映射不需要在两者之间有任何关系。一个常见的情况，单独一个源可能会被映射到两个目标纹理或是多个目标纹理的关联处，以确保在它们之间进行平滑的过渡。一般说来，源层上的数据和进行映射的数据都是逐像素确定的，而不是逐三角形确定的，这

Download at www.Pin5i.Com

样，纹理就可以组合起来在任何表面上平滑过渡了。在很多情况下，在一个特定的顶点上，某些层可能是不连续的（例如墙壁或是车辆上的画面），而且它部分则是连续的（例如污迹或是刻痕）。在设计数据结构的时候一定要记住，在大部分情况下，如果每个材质都有一个纹理，是不能简单地将一个对象放到另一个不同的材质中去的。一旦完成了合成，生成了目标纹理，把数据传递给底层的网格渲染系统之后，这些数据就会进行上述的处理，但是在较高的层次上是不使用此模式的。

除了原始的层次化 *uv* 映射数据以外，此外的数据可以被保存在每个顶点上。这些数据控制了混合的一些属性，而且将被线性插值到顶点之中去。例如，控制混合的随机数可能会通过每个顶点的数值被缩放或是偏移，这么一来，艺术创作人员就可以给每个顶点做上标志，说明其是否脏了，是否平滑，以及诸如此类的属性。如何把这些数据在 3D 模型中取出来是某些 3D 软件包没有处理好的一个问题（顶点上色的支持比较弱，而多通道的支持更麻烦）。在某些情况下，这些数值可能需要被编码到其他的层中去，而且可能会在某个预处理阶段再转换出来放到每个顶点中去。

层也可以有自己的 *uv* 映射，这些映射可以被各种其他的方法改变或是决定。例如，旋转、缩放、扭曲以及所有这些可能的动作（通常是由随机数决定的），这些方法可以是复杂的硬编码方法——例如，一个砖墙可能由很多独立的砖头源纹理合成而成，每块砖头都是从一个大约 20 块砖头的集合中随机选出来的。在这种情况下，砖头的源就是一个随机数，它被量化划分为 20 个位置，而目标则是交错排列的砖墙模式。这一般来说需要进行两次变换，第一次是将选出的源砖头放到左上角，第二次是将它放置到目标纹理中合适的位置去。在运行期，这两次变换可以合成为一次完整的 uv 变换，在此时我们还没有读进任何纹理采样。

4.17.5 层的源与过滤器

层可以由很多方法得到。例如，艺术创作人员可以提供一个位图，或是通过程序生成的数据，例如 Perlin 噪声[Perlin]，或是单元自动机（CA）来生成苔藓或是木头的脉纹[Ebert94]。所有这些不同方法的参数都可以由逐像素或是逐顶点的系数得到，这些系数则是由艺术创作人员或是提供的信息中的伪随机数提供的。

我们也可以对源进行过滤。最有用的过滤就是对 RGB 值或是 alpha 值进行缩放与偏移操作，特别是当其应用到噪声函数上或是其他类似的重映射上时，例如 S 形的曲线或阶状函数。如果在其中进行 1° 到 4° 角的样条曲线插值，而且可以让艺术创作人员在一幅图中控制其强度、对比度以及亮度，也是很有用的。这些方法中的绝大多数仅会生成含有 256 项的查找表（或是有时候会生成三个不同的 RGB 表），接下来这个表将会被应用到图像上去。其他常见的过滤器还有模糊过滤与边缘加强过滤，以及类似的 2D 基于卷积核的技术。这些都是通过单源混合来实现的。

4.17.6 合成方法

大部分 Photoshop 的标准混合都应该被支持，例如标准 alpha 混合、乘法混合、加法混合、减法混合等等。表 4.17.1 中列出了某些基本的 Photoshop 混合方法，还有其等价的数学以及

Download at www.Pin5i.com

alpha 的参数设置。值得注意的是这些混合将会频繁被通过 alpha 混合使用，而不是通过多纹理使用。虽然在很多情况下，使用多纹理的方法在理论上比多个 alpha 混合通道的效率要高，但是在实践中要满足多纹理的某些限制条件是很麻烦的。

表 4.17.1　　Photoshop 混合方法以及等价的多通道 alpha 混合设置

Photoshop 混合	数学运算	Alpha 混合（源、组合、目标）
正常	Arg1*alpha + Arg2*(1-alpha)	SRCALPHA, ADD, INVSRCALPHA
乘法	Arg1*Arg2	DESTCOLOR, ADD, ZERO
加法	Arg1+Arg2	ONE, ADD, ONE
减法	Arg1-Arg2	ONE, SUBSTRACT, ONE
屏幕	Arg1+Arg2-Arg1*Arg2	INVDSTCOLOR, ADD, ONE
加亮	Max(Arg1, Arg2)	ONE, MAX, ONE
变暗	Min(Arg1, Arg2)	ONE, MIN, ONE

在某些情况下，我们还需要进行三源混合——对三个源进行混合。例如，在我们对两个 alpha 通道进行混合的时候，还需要使用使用一个由第三方 alpha 通道决定的因数。有的时候，这种需求可以通过使用双参数的多重混合解决，但是这种方法比较慢，而且不灵活，因此我们需要让混合系统保持开放，这样一个混合就可以接受多个输入了。如果有相应硬件的话，三源混合则可以通过多纹理处理的硬件来实现，这样就可以避免生成多余的临时纹理了。

4.17.7　对数字的控制

由于使用了独特纹理的系统，艺术创作人员需要使用越来越复杂的方法以生成数字，这些数字将在混合因数或是映射变换中使用。首先他们需要在系统中有对应的可见对象——他们可以移动的滑动条，一个拥有固定的缩放系数与偏移系数的伪随机数字生成器等等。接下来，他们可能需要生成很多类似的对象，然后通过一个滑动条控制系统中所有同样的参数。

你将发现他们还需要变量与数学运算，此时你可以为此提供一整套的编程语言（Python、LUA、LISP 等，选一个你喜欢的就行）。不过，艺术创作人员比较偏爱可视化编程——有变量、数学运算等的窗口。因此，你可能需要提供一个图形化的表现形式。在实际情况中，我们发现使用一个类似文件树的树状视图来计算数值是最好的。我们还在其间添加了可拖放的规尺线，这样就可以显示出结果是在混合或是其他运算的哪一步中得到了应用。

4.17.8　动态纹理

另外一项强大的技术可以根据当时游戏的情景改变层。最明显的例子就是足迹、血迹、子弹孔以及爆炸留下的痕迹。这些一般都是比较简单的层，可以实时地计算出它们的 *uv* 坐标来。这些方法相对于传统的几何形状纹理花印技术的优势是比较明显的：我们不需要考虑 z 方向上的问题，在每帧里面没有额外的几何形状需要绘制出来的，更加宽泛的效果和纹理变化，同时也不会降低批处理的效率，另外也可以得到更智能化的混合效果。

动态纹理混合可能在纹理修正图里面有用。简单地对多个 dot3 的纹理修正图进行 alpha 混合是不会得到很好的效果的。它们会互相重叠（例如沙滩上的足印），而且一般说来它们之

Download at www.Pin5i.Com

间的相互关系不是很复杂——对一个足印来说，只要完全把另一个足印覆盖掉就可以了。不过，我们需要把足迹的不同的高程场合成起来，手段就是使用一个最大混合，它将得到两个高度值（或是凹陷值）中间较大的值。一旦它们被合成以后，我们就将结果加到原始表面的高程场上。只有在这个时候完整的高程场才会通过一个过滤器转换为一个 Dot3 的法向地图。仅仅通过标准的花印进行 alpha 混合传递给帧缓冲是不行的，这种合成是不可能实现的。

此方法的另一个用途就是将多个阴影图合成在一个场景里面去。例如，在一个森林里面，树木的阴影图并不需要精确地与数目的形状完全一致——只要使用三个或是四个地图就可以提供足够的变化，让肉眼看不出来了。这种方法可以提高纹理缓存的性能。除此之外，如果需要在游戏中销毁一些树木的话，阴影图还可以进行重组。此外，在太阳移动的时候，我们可以使用新的光源方向将阴影图计算出来。

4.17.9 可扩展性

可扩展性与可移植性在 PC 平台与游戏机平台上都是很重要的要素。在玩家可以观察到的地方，我们都需要花费时间与空间来处理这类问题。

独特纹理的系统已经有内置的可扩展性了。基于距离，使用简单的对纹理序列等级最大尺寸的估测，我们就可以仅对可见的纹理采样进行合成。这样，我们就可以对 CPU 的时间进行优化，使得显存的使用更加高效，以针对可见的对象生成更高分辨率的纹理。

使用一个可扩展的系统也可以在需要的时候得到更高的真实性。例如，在一个激烈的战斗中，玩家会比较关心帧率，而不是纹理的质量。当战斗结束的时候，玩家则会开始欣赏自己的战斗成果。在运动比较少的时候，缓存里面每秒收到的纹理会比较少，帧率就不是问题，因此此时系统可以花费更多的时间处理纹理。

为了增强可扩展性，可以做如下的工作：

- 减少源纹理的尺寸，使用较小的纹理序列。这可以提高缓存使用的效率，减少按需提交的磁盘存取或是虚拟内存系统引起的磁盘访问。
- 减少目标纹理的尺寸。这就意味着使用较少的纹理采样来合成，减少显存里面纹理的交换。
- 简化采样过滤器。不要使用高斯过滤器或是三线性过滤器，使用双线性过滤器或是 even-point 过滤器。
- 简化图像过滤器。光雾效果或是边缘增强效果可以一并省略掉，使用较大卷积核的过滤器可以用质量较差的较小卷积核替换掉。
- 减少混合的数目，方法是给某些混合加上细节层次化的因数。例如，细节比较多的纹理或是花印纹理可以被删除或是被削弱，同时也不会导致太大的质量上的损失。当场景的细节要求提高的时候把它们再加进来也不是什么难事。

上面提到的这些因素可以有选择地应用到远距离的纹理上。

有些事情要注意不要去做，包括在场景细节级别改变的时候不要将所有的纹理重新合成，这将导致两个问题：首先是在场景中的纹理重组的时候，将导致很明显的帧画面静止；其次，所有的纹理都会发生明显的变化，导致一个很讨厌的、很突然的效果。我们可以使用一个优越得多的方法，那就是只对屏幕之外的纹理进行重组。不论是出于什么理由，如果一

Download at www.Pin5i.Com

个在屏幕上的纹理非要进行重组的话，至少要在前后两个版本的画面之间进行 alpha 混合，以消除视觉上的突变。

4.17.10　使用 CPU 还是图形芯片进行合成运算

理想情况下，所有的合成运算都应该是由图形芯片来处理的。这本来就是图形芯片该干的活——得到几个纹理，对之进行 *uv* 坐标的变换，对之施加某些像素特效，然后将它们与现有的图像进行混合。图形芯片设计的时候就考虑到了要将这些运算并行进行，得到最佳的吞吐量。不幸的是，这中间需要的某些过滤或是混合实在是很复杂，而且很难或是基本不可能在大部分图形芯片上进行处理。此外，如果把源纹理与目标纹理在显存之间来回切换，还会带来一些问题或是会降低速度，而且在图形芯片上进行合成会降低生产效率。通常，艺术创作人员总是非常需要在项目的末期进行一次混合，这是无法在图形芯片上处理的。出于这些考虑，我们必须时时准备好一种完全使用 CPU 的合成处理，即使很多操作仍然会被移到图形芯片上去做，以提高速度。

1. PC 机

PC 机的 CPU 是比较擅长合成处理的。MMX（SIMD 小整数型）指令相当强大而且快速，大部分混合运算与过滤的瓶颈是体现在总线带宽上，而不是 CPU 处理能力上。内存本身是非常大的，因为在其背后还有虚拟内存支持，这本身就是一种缓存技术。

使用图型芯片来处理合成对于 PC 机来说，难度要大于游戏机。PC 机常带有各种图形卡，这使得事情更加难办了。渲染到纹理的能力在各种图形卡上相差极大（即便是最新的图形卡上），下至根本没有，上至完全支持。此外，可用的混合与过滤的种类也是非常多的，很多处理都需要进行多遍处理，降低了速度。

2. XBox

XBox 与 PC 机的情况有些类似。CPU 是一样的，而且它有一些非常快速的 MMX 指令以用来处理图像合成，因此部分流水线是可以共享的。不过，图形芯片相当强大，它可以自行完成 90%的混合与过滤而不需要 CPU 的协助。最妙的是，图形芯片与 CPU 使用的是同一片内存，这就意味着在合成的每个阶段，我们可以调度较快的一方来进行处理。由于图形数据是完全保存在系统内存里面的，因此我们就不需要考虑把数据来回拷贝的问题了。不幸的是，这会引起一些带宽与利用率的问题。

4.17.11　演示程序

CD-ROM 上提供的演示程序显示了一个非常简单的程序景物图。在此处仅仅使用了一种源纹理，对不同的地形类型上的各个顶点，我们为其使用了不同的着色。在纹理应用到每个顶点之前，我们还对其进行了随机化。在合成某个特定的纹理的时候，纹理上面的 25 个顶点的每种地形类型都是同时混合的。然后，每种花印（在此处是简单着色的 splat）都在顶端进行 alpha 混合。花印的数据是与最近的景物顶点关联在一起的，因此得到某个特定顶点的花

Download at www.Pin5i.Com

印列表是很快的。

如果按下 Shift 键，读者就可以看到使用到的不同纹理序列的伪彩色图——所有的纹理序列的计算都已完毕，就好像镜头是放在景物的正中一样。中央的纹理序列由 256×256 的纹理采样组成，随着距离的增大其数量会倍减。

这个演示程序说明了几件事情：如果使用到的源纹理数量较少（在我们的程序里面只有一个），则无缝纹理就会简单而高效；表面使用大量的花印带来的速度降低并不很大，这与传统的帧缓冲区的合成不一样；智能的纹理缓存与简单的、基于距离的纹理序列可以极大地减少合成的数量与纹理空间的要求。最后，即使使用软件来实现合成也可以是快速而高效的——本演示程序没有使用任何硬件来进行合成。

4.17.12 结论

独一无二的纹理包含了许多有趣的特性。其主要的目标是要使得整个世界中的每个纹理都是独一无二的，将斧凿的痕迹完全去掉。我们也可以使用同样的技术来得到更好的效果。

我们可以根据表面所在的游戏世界中的位置来改变它，这样对玩家就更有吸引力了。一般使用细致的纹理、光照图以及花印生成的额外的细节是可以被更有效、更方便、更具有灵活性地添加进来。即使是在极大的、极复杂的世界中，艺术创作人员也不用再去进行大量繁琐的像素处理了，这样他们就有更多的时间来对场景中真正重要的东西进行调整与精化了。

最后，如果使用脚本语言来驱动纹理组合引擎的话，艺术创作人员与开发人员就可以拥有极大的灵活性与更多的机会来创建一个在当今的电脑游戏中很少见到的世界了。

4.17.13 参考文献

[Ebert94] Ebert, et al., *Texturing and Modeling: A Procedural Approach*, AP Professional, 1994.

[Milliger02] Milliger, Mike, "Procedural Texturing," *Game Programming Gems 3*, Charles River Media, Inc., 2002.

[Perlin] Ken Perlin 关于噪声的网址为 http://mrl.nyu.edu/~perlin/doc/oscar.html.

Download at www.Pin5i.Com

4.18 使用纹理作为查找表进行逐像素光照计算

Alex Vlachos, John Isidoro, Chris Oat
ATI Research
Alex@Vlachos.com
jisidoro@atil.com
coat@ati.com

最新一代的图形处理硬件已经将实时应用的逐像素（per-pixel）计算能力带上了一个新台阶[Microsoft02]。尽管如此，它们仍然受到了其指令大小与类型的限制。由于有了这种限制，开发人员需要使用一套新技巧针对每个像素使用更高级的图形处理算法。在本文中，我们将提出一些新方法，使用纹理图作为查找表以调用相应的函数，主要集中在光照处理上。使用这个技巧可以节省宝贵的像素着色指令，而且在很多情况下，为了在当前的硬件环境下实现某些逐像素的特效，它也是唯一可行的方法。

4.18.1 不使用立体图进行 *h* 归一化（*n.h*/*h.h* 映射）的镜面逐像素光照

在像素着色器里进行镜面光照，常见的一个问题就是当中间矢量 ***h*** 在一个多边形上进行线性插值的时候会失去归一性。标准的解决方法之一就是使用一个归一化的立体图对此中间矢量进行重新归一化[Baker01]。归一化的立体图有个缺点，那就是它需要额外得到一个纹理图，此外还要耗费很多纹理存储空间。如果不使用一个 1D 的镜面纹理图的归一化立体图，那么我们就需要在着色器里面使用一个 *n.h*/*h.h* 图作为查找表了。这种优化技术把每个像素需要的纹理图从两个减到一个。尽管如此，我们仍然需要使用归一化的立体图对目标空间光照矢量 *l* 进行归一化。

- 从法向图（凸凹贴图）中得到每个像素上面的法线 ***n***。
- 把中间矢量 ***h*** 作为 3D 纹理的坐标保存下来，对其在多边形上进行插值，这样它将会在每个像素上失去归一化的性质。
- 将 *n.h* 保存在纹理坐标的第一个分量里面。
- 将 *h.h* 保存在纹理坐标的第二个分量里面。
- 使用上述的 2D 纹理坐标得到 *n..h*/*h.h* 图。得到的纹理采样就是镜面光照量的某个常数 *k* 指数次方。

Download at www.Pin5i.Com

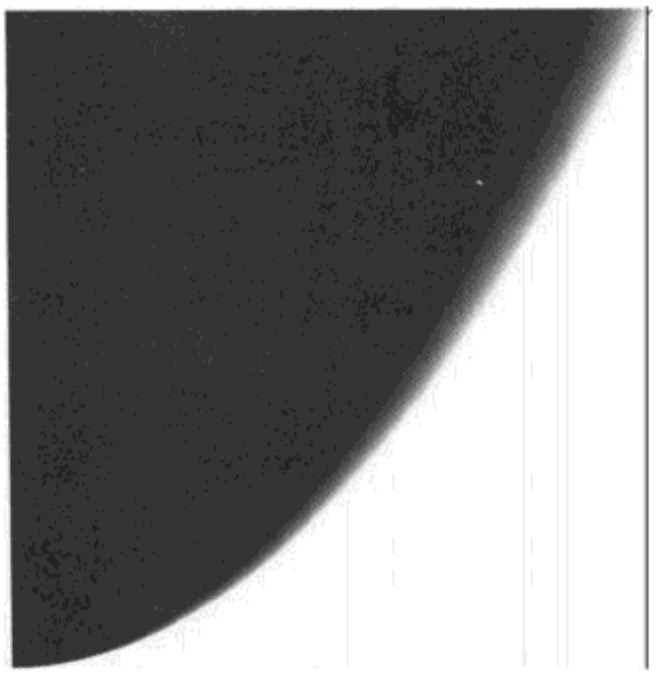


图 4.18.1 n.h/h.h 图（$k=32$）。一个程序创建 n.h/h.h 图的例子，它使用了镜面指数为 32 的逐像素镜面光照

得到 *n.h/h.h* 图相对来说还比较简单（请参见图 4.18.1）。此图是一个单色的图像，它使用的是内存只占到一个归一化立体图（它有 6 个带 RGB 色彩的面）所需的纹理内存的 1/18 而已。这里的 *h.h/n.h* 图只是下面公式中使用的一个查找表而已。

$$f(s,t)=\left((s.t)/\left((t.t)^{0.5}\right)\right)^k \tag{4.18.1}$$

它的等价形式是

$$((n.h)/\|h\|)^k \tag{4.18.2}$$

下面就是一个 DirectX8.1 中的像素着色器（1.4 版），它进行的就是 *n.h/h.h* 的映射：

```
;(0.0, 0.5, 1.0, 1.0)
SetPixelShaderConstant 0 psCommonConst
SetPixelShaderConstant 1 ambient
SetPixelShaderConstant 2 diffuse
SetPixelShaderConstant 3 specular
StartPixelShader
    ps.1.4
    texld r0, t0 ;normal map n
    ;tangent space H (not necessarily normalized)
    texcrd r2.rgb, t2
    ;tangent space L (normalizer cube map lookup)
    texld r4, t1
        dp3_sat r4, r0_bx2, r4_bx2 ;(n.l)
        dp3_sat r3.r, r0_bx2, r2 ;(n.h)
        dp3_sat r3.g, r2, r2 ;(h.h)
    phase
    ;Base map
    texld r1, t0
    ;(n.h)/(h.h)^k map lookup
    texld r2, r3
```

Download at www.Pin5i.Com

```
        ;light = n.l * diffuse + ambient
        mad r5.rgb, r4, c2, c1
        ;basemap * light
        mul r5.rgb, r1, r5
        ;(specular * color) + (basemap * light)
        mad_sat r0.rgb, r2, c3, r5
        +mov_sat r0.a, c0.b
EndPixelShader
```

4.18.2　使用一个$(n.h)^k$图的逐像素镜面指数

如果使用一个如上所述的方法，我们就有可能逐像素得到唯一的镜面指数了。这需要在着色器里面添加一些新的数学运算。此外，还需要得到一个镜面指数（光洁度）图并创建一个$(n.h)^k$的纹理。灰度镜面指数图一般是存储在基本纹理的 alpha 通道中的。这就是每个艺术创作人员绘制出来，在每个像素定义镜面指数的东西。

- 从法向图（凸凹贴图）中得到每个像素上面的法线 n。
- 从指数（光洁度）图中得到 k。
- 把中间矢量 h 作为 3D 纹理的坐标保存下来，对其在多边形上进行插值，这样它将会在每个像素上失去归一化的性质。
- 将$(n.h)^2$保存在纹理坐标的第一个分量里面。
- 将$(k*h.h)$保存在纹理坐标的第二个分量里面。
- 将$(h.h)$保存在纹理坐标的第三个分量里面。
- 我们需要在$(n.h)^k$图中得到一个经过投影的、衍生出来的纹理。经过此投影，前两个分量将会被第三个分量除，得到$((n.h)^2/(h.h),\ k)$的结果。通过这些坐标得到的纹理采样就是镜面光照量的逐像素上的 k 指数次方。

就像 $n.h/h.h$ 图一样，$n.h^k$ 图也是用来减少你使用其他方法会给像素着色器带来的数学运算的（例如对 h 进行归一化，或是取 $n.h$ 的 k 指数次方）。在这种情况下，$k=[0.0, 1.0]$，其中 0.0 对应的是在$(n.h)^k$纹理创建过程中 k 的最小值，而 1.0 对应的是在$(n.h)^k$纹理创建过程中 k 的最大值。在得到这个衍生出来的纹理的过程中，我们将使用投影除法（pzojective divide）(在 D3D 中是_dz)，即用前两个分量除以第三个分量。投影除法将导致最终的纹理查找坐标成为：

$$s=((n.h)*(n.h))/(h.h)=(n.h)^2\|h\|^2 \tag{4.18.3}$$

$$t=k$$

对于 $n.h^k$ 映射，纹理查找表将计算出下列的函数：

$$f(s,t)=s^{0.5*t} \tag{4.18.4}$$

从这个图得到纹理采样以后，使用上述的坐标，此方程将变换为：

$$((n.h)/\|h\|)^k \tag{4.18.5}$$

结果将得到标准的 Phong 镜面光照分量的 k 指数次方。这项技术的优美之处在于投影纹

Download at www.Pin5i.com

理采样可以同时对 h 矢量进行归一化与进行指数化。彩图 14 中底部排列的矩形使用的就是 $(n.h)^k$ 图进行的渲染，这样可以得到逐像素不同程度的镜面光照。镜面指数在每个像素上都是不一样的，这是由图 4.18.4 中定义的指数图决定的。图 4.18.3 是一个示例，在其中，自动生成的 $(n.h)^k$ 图可以逐像素改变镜面光照，其逐像素的指数范围是[0, 64]。

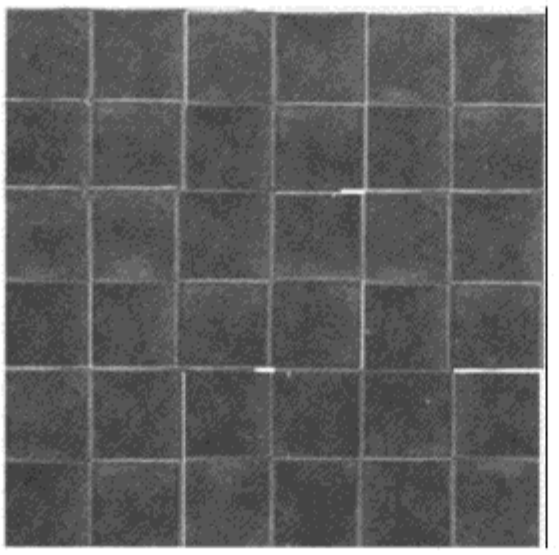


图 4.18.2 没有任何光照的 RGB 分片表面纹理

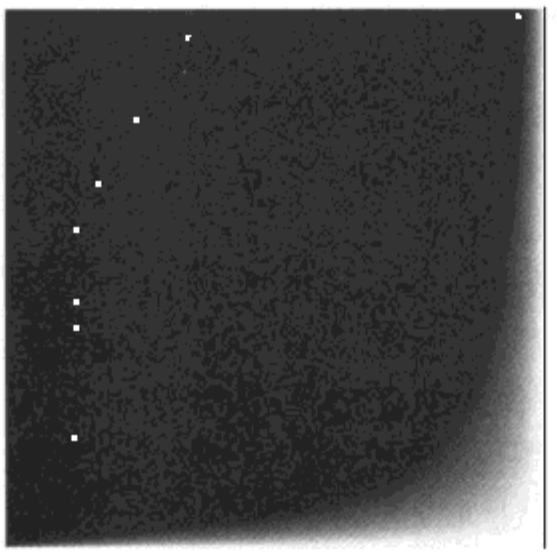


图 4.18.3 程序生成的一个（$n.h$）^{k}（k=[0,64]）图，它在每个像素上采用了不同的镜面光照指数

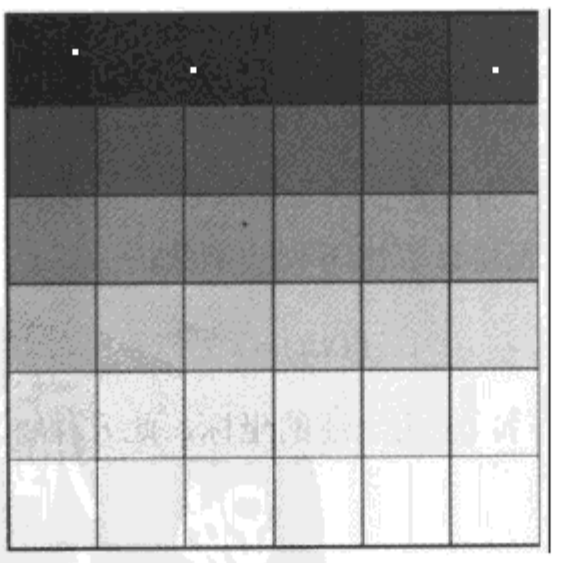


图 4.18.4 灰度镜面指数图与图 4.18.3 中的纹理混合，以得到艺术创作人员可编辑的逐像素的镜面指数

Download at www.Pin5i.com

```
;(0.0, 0.5, 1.0, 1.0)
SetPixelShaderConstant 0 psCommonConst
SetPixelShaderConstant 1 ambient
SetPixelShaderConstant 2 diffuse
SetPixelShaderConstant 3 specular
StartPixelShader
   ps.1.4
   ;dot 3 map (specular exponent stored in alpha)
   texld r0, t0
   texcrd r2.rgb, t2 ;tan H
   ;tangent space L (normalizer cube map lookup)
   texld r4, t1
      dp3_sat r4, r0_bx2, r4_bx2 ;(n.l)
      dp3_sat r3.b, r2, r2 ;(h.h)
      dp3_sat r3.r, r0_bx2, r2 ;(n.h)
      mul  r3.g, r3.b, r0.a ;k*((h.h))
      mul  r3.r, r3.r, r3.r ;(n.h)^2
   phase
   ;Base map
   texld r1, t0
   ;Attenuated (n.h)^k map
   texld r2, r3_dz
      ;n.l * diffuse + ambient
      mad r5, r4.r, c2, c1
      ;diffamb = basemap * (n.l * diffuse + ambient)
      mul r5.rgb, r1, r5
      ;diffamb + (specular * specular color)
      mad_sat r0.rgb, r2, c3, r5
      +mov_sat r0.a, c0.b

   EndPixelShader
```

4.18.3　色彩偏移的光晕

另一个使用纹理查找表的渲染效果就是色彩偏移的光晕效果（color-shift iridescence）。这种光晕效果可以出现在昆虫的翅膀、某些玻璃器皿的装饰上以及类似珍珠的物体上。此算法需要仿真的这种效果是基于经验得来的，那就是一个物体上某点的色调会随着视线矢量与表面法线矢量之间夹角的变化而变化。通常情况下，这是因为在物体的表面上有一层很薄的半透明膜，它在不同的方向上会散射不同频率的光线。其基本算法会首先进行 Phong 光照处理，然后得到镜面高亮色彩，再从一个 1D 的基于色调的梯度纹理中得到纹理采样，将此采样与高亮色彩相乘。纹理采样的坐标是$(n.v)$，其中 n 是每个像素上的法线，而 v 是切线空间的视线矢量。请参见彩图 15，在其中有一些基于色调梯度的图像，还有使用色彩偏移光晕效果制作的物体。经过下面像素着色器的处理，我们就可以得到色彩偏移光晕效果了。请注意，这个例子使用的是上一小节中提到的$(n.h)^k$技术。

```
;(0.0, 0.5, 1.0, 1.0)
```

Download at www.Pin5i.Com

```
SetPixelShaderConstant 0 psCommonConst
SetPixelShaderConstant 1 ambient
SetPixelShaderConstant 2 diffuse
SetPixelShaderConstant 3 specular
StartPixelShader
    ps.1.4
    ;dot 3 map (specular exponent stored in alpha)
    texld r0, t0
    ;tan H
    texcrd r2.rgb, t2
    ;tangent space L (normalizer cube map lookup)
    texld r4, t1
    ;tangent space V (normalizer cube map lookup)
    texld r5, t3
        dp3_sat r4.gb, r0_bx2, r4_bx2 ;(n.l)
        dp3_sat r4.r, r0_bx2, r5_bx2 ;(n.v)
        dp3_sat r3.b, r2, r2 ;(h.h)
        dp3_sat r3.r, r0_bx2, r2 ;(n.h)
        mul  r3.g, r3.b, r0.a ;k*((h.h))
        mul  r3.r, r3.r, r3.r ;(n.h)^2
    phase
    ;Base map
    texld r1, t0
    ;Attenuated (n.h)^k map
    texld r2, r3_dz
    ;Iridescent map 1D tex map lookup
    texld r3, r4
       ;Iridescent * Specular
        mul_sat r0.rgb, r2, r3
        ;n.l * diffuse + ambient
        mad r5, r4.g, c2, c1
       ;diffamb = basemap * (n.l * diffuse + ambient)
        mul r5.rgb, r1, r5
        ;diffamb + specular * specularcolor
        mad_sat r0.rgb, r0, c3, r5
        +mov_sat r0.a, c0.b
EndPixelShader
```

4.18.4 拥有正确的逐像素衰减的逐像素点光照

对于纹理查找表来说，另一个有用的应用就是正确的逐像素光照衰减。要计算点光照的距离衰减，传统的做法是对顶点进行采样，然后对距离进行线性插值以得到逐像素的效果。这种方法的缺陷在于当光照与一个大多边形的表面很接近的时候，没有顶点会得到离光源较近的光照。

此算法的做法就是要计算出每个像素距离光源的距离，而不是顶点与光源的距离。这样我们就可以基于一个光源得到精确的衰减值了，因此也就避免了传统的基于顶点的方法带来的毛病。

顶点着色器将在归一化光照空间（*normalized light space*，NLS）里面计算出顶点的方位，

Download at www.Pin5i.Com

而不是使用顶点方位之间的距离来直接计算。NLS 变换是一个增量为-lightPos 的平移变换，然后再加上一个 1/lightFallOff 的单位变换。这样，我们就得到了每个顶点在 NLS 空间里面的方位。接下来，像素着色器就可以计算出每个像素上距离的平方了，其方法是将插值得到的顶点方位自乘。然后，我们将使用 NLS 距离的平方作为纹理坐标得到 1D 纹理的采样。这样你就可以使用任何方法通过距离来改变光照的强度与色彩了。此算法逐像素处理的特性可以解决任何有关顶点采样的问题。由定义可知 NLS 的方位是线性的，因此这种方法是非常完美的。此方法把逐像素求平方根的运算放在了纹理采样的步骤中进行（参见图 4.18.5）。

图 4.18.5　一维点光照纹理，其色调与密度是由当前位置至点光源的距离决定的

4.18.5　拥有正确的逐像素衰减的逐像素聚光灯与方向性光照

逐像素光照衰减的自然推广就是对聚光灯与方向性光照进行建模。这种技术与点光照着色器是很相似的，但是它需要一些稍有不同的顶点与像素着色操作。在这里我们不需要为光照创建 1D 纹理，但是需要一个 2D 纹理。纹理的 u 轴使用的是与点光照一样的距离的平方，然而，纹理的 v 轴则根据聚光角（NLS 顶点方位矢量与聚光灯方向之间的夹角）对衰减值进行编码。

为了使用这个 2D 纹理，我们对归一化光照空间进行了扩展以表示一个旋转，而不仅是能表示一个缩放因子与一个平移。旋转矩阵是由光照的标准正交基作为 x、y 轴，以光照方向作为 z 轴组成的。

在顶点着色器中，与点光照类似，顶点方位将变换到归一化的光照空间中去。在像素着色器中，我们将把这个新方位作为一组纹理坐标载入到一个寄存器中去，然后对其自身做一个点积运算，以得到 NLS 距离的平方（与上面点光照的处理一模一样）。NLS 方位将同时会被载入到另一个寄存器中去进行投影修正，导致前两个分量被第一个分量所除。这样就可以得到投影方位了。在 NLS 中，这个投影将导致被 z 除的操作，这样就将得到聚光灯的锥状视图。接下来再对之进行一个自乘点积运算，我们就得到了一个缩放过的、平方后的 NLS 的 z 轴上的距离，其范围就如同角度值一样在 0～1 之间。如果点积的时候没有进行修正，那么这个值将代表一个 0° 角到 90° 角的范围。

然后，我们将利用这两个值进行 2D 纹理的查找，此纹理在视觉上是一个很逼真的仿真，同通常的聚光灯距离与角度方程相乘的值近似，假设其输入是平方值的话。

如果我们仅仅做一个如下所示的 dp3 运算，那么 2D 纹理图代表的就是一个 0° 到 90° 之间的光照截面值。这就意味着为聚光灯创建的纹理必须对整个 0° 到 90° 的范围进行编码。对于具有更大或是更小角度的光照锥面来说，我们可以选择一个合适的范围更好地使用纹理采样。通过使用缩放因子的指令，2D 纹理还可以用来代表其他角度的截面。截面角的公式是 2*arctan(1/scale factor)。在表 4.18.1 中，我们列出了可用的缩放因子及其相应的截面角。

表 4.18.1　　可用的缩放因子及其相应的截面角

缩放因子	_d8	_d4	_d2	无	_x2	_x4	_x8
截面角（度）	165.7	151.9	126.8	90.0	53.1	28.1	14.3

Download at www.Pin5i.Com

通过对照截面角来得到缩放因子，纹理查找表 y 方向上的分辨率可以为较小的截面下降一点，因为此时处理的空间要小得多。如果所期望的截面角没有列在表中，只要选用一个最接近的大于所需截面角的角度就可以了，为了减小差异，可以在 y 方向上将查找表缩小一点。

```
;(0.0, 0.5, 1.0, 1.0)
SetPixelShaderConstant 0 psCommonConst
StartPixelShader
    ps.1.4
    ;NLS pos
    texcrd r1.xyz, t1.xyz
    ;NLS pos in xy plane scaled by 1/z
    texcrd r2.xy, t1_dz.xyz
       ;distance squared from light
       dp3 r1, r1, r1
       ;Set z = 0
       mov r2.b, c0.r
       ;(dist in xy plane scaled by 1/z)^2
       dp3 r1.g, r2, r2
    phase
    ;Base
    texld r0, t0
    ;Light attenuation
    texld r1, r1
       ;Base*light
       mul r0, r0, r1_x2
       ; * N.L
       mul r0, r0, v0
EndPixelShader
```

从第二个 texcrd 指令中去掉_dw 可以得到一个圆筒型方向的光照截面。这样在 *xy* 平面上就消除了距离上 1/z 的因子，使光照区域与圆筒一样（参见图 4.18.6）。请注意此光照仍然有一个方位以及从此方位产生的衰减。如果我们不需要基于方位距离产生的衰减，那么只需要删除掉第一个点积，使用第二个点积作为 1D 纹理的索引就可以了。

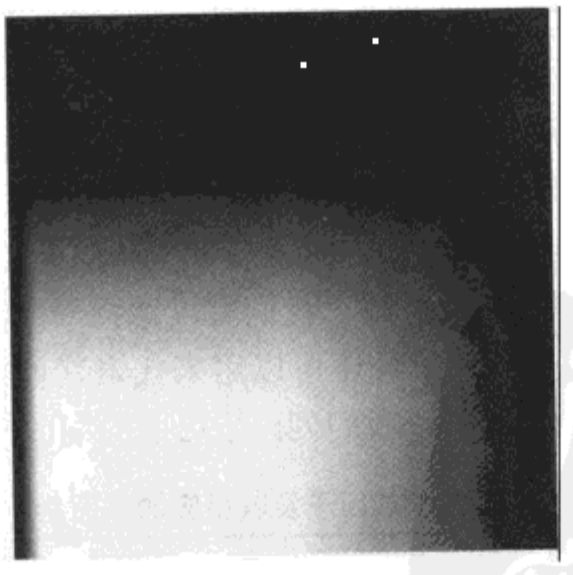


图 4.18.6 二维的方向性光照纹理，其色调与密度是距光源距离的函数（在一个轴向上）。在另一个轴向上光照是光矢量角度的函数。在此例中假设使用的是 90° 的截面，我们可以使用 53.1° 的截面对之压缩以节省纹理

Download at www.Pin5i.Com

4.18.6　结论

我们展示了很多使用纹理作为函数查找表生成的效果。通过这个方法，很多需要逐像素处理技术的图形算法现在真的可以进行逐像素的处理了。当然，这些例子仅仅包含了使用此技术可以得到的一小部分有用的效果。

4.18.7　参考文献

[Baker01] Baker, D. and C. Boyd, "DirectX 8.0 Shader Applications (Per Pixel Lighting)," DirectX Developer Day, 2001, 对应网址为 http://www.microsoft.com/corpevents/gdc2001/developer_day.asp.

[Microsoft02] MSDN.Microsoft.com, "DirectX 8.1 Pixel Shader Reference," 对应网址为 http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dx8_c/directx_cpp/Graphics/Reference/Shader/Pixel/Instructions/Instructions.asp, February 2002.

Download at www.Pin5i.Com

4.19 使用手工制作的着色模型进行渲染

Jan Kautz
Max-Planck-Institut für Informatik
kautz@mpi-sb.mpg.de

人们已经提出了相当多的技术来使用硬件为游戏实现更复杂、更真实的着色模型了[Heidrich99，Kautz99]。但是，这些技术很少被使用，可能是出于两个原因：实现上的复杂以及着色模型使用的参数不太直观。我们提出的是一个比较简单的称为“法线分布函数（normal distribution function）着色”的技术，它可以让一个艺术创作人员手工制作着色模型，把其中高亮区域的形状与色彩都保存在位图里面。此技术使用到了逐像素着色，你还可以将之与凸凹贴图一起使用，而且还可以创建不等测的着色模型。

4.19.1 着色模型

着色模型决定了一个表面究竟会反射多少光线，这取决于光源的方向 l，观察者的方向 v 以及表面的法线方向 n。

到现在为止，大部分交互式渲染系统都使用 Blinn-Phong[Blinn77]模型，这是因为它比较简单：

$$L_o = k_d(n \cdot l) + k_s(h \cdot n)^N \qquad (4.19.1)$$

其中的参数 k_d 是散射系数，通常存储在一个彩色纹理图中间。k_s 则是镜面系数，一般这是全局一致的，但是有时候会把它存放在一个光泽图中。参数 N 是镜面指数，它将用来改变材质的光洁度。矢量 h 是归一化的中间矢量，它处在观察者与光照方向之间。

正如你所看到的一样，此模型主要使用的是简单的操作，除了一个指数运算以外。在当代的图形处理硬件上面，它可以通过像素着色器来实现[Kilgard00，Mitchell01]，这是因为点积与衍生纹理（dependent texture）的读取都是现成可用的。

尽管如此，被光照的物体通常看上去都像光亮的塑料一样，这几乎不可能使得一块布与真实的布一模一样。对于不等测着色来说，他们将用来对擦过的铝材料进行渲染，这时候就根本不能使用 Blinn-Phong 算法了，因为它是完全等测的（就是说它是与方向无关的）。

Download at www.Pin5i.Com

以前用来得到更好模型的方法

有人提出过一些可以得到更好的着色模型的方法[Heidrich99，Kautz99]，但是这些方法通常都有局限性，例如其参数不直观，或是需要一些较少使用的测量数据。

4.19.2　基于微表面的着色模型

基于微表面（microfacets）的着色模型（请参见[Cook81]）假设表面是由很多指向不同方向的镜面构成的（图 4.19.1）。这些微表面是如此的细小，以至于大家无法察觉到它们，但是它们方向的总和却决定了镜面高亮区域的形状。因此，所有基于微表面的着色模型都要使用到微表面法线 n_m 的分布。我们将这个 NDF 定义为 $p(n_m)$。这个分布函数能告诉我们什么呢？如果我们使用 $p(h)$来查找一个值，则它将给出指向 h 方向的微表面所占的百分比。

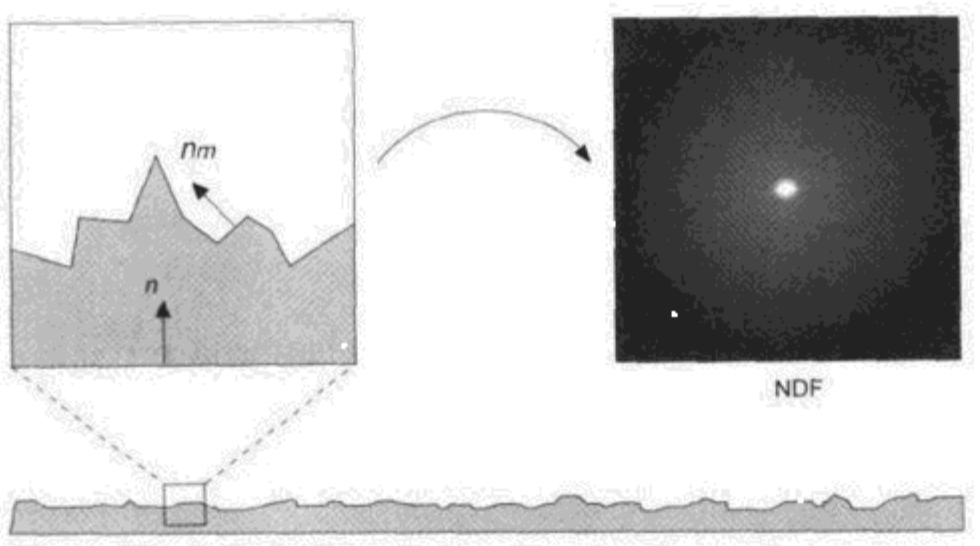


图 4.19.1　由微表面构成的面及其法线分布函数

正如前面所提到过的一样，微表面模型假设所有的微表面都是很小的、完美的镜面。若在视线与光照矢量之间的中间矢量与这个完美镜面的法线一样的话，我们就可以看到光源的反射了。

对于由微表面构成的表面来说，这基本上是正确的。如果微表面的法线 n_m 等于视线与光照矢量之间的中间矢量，我们就可以看到从光源反射回来的光线了，但是其量级也仅是微表面量级的。分布函数 $p(h)$告诉我们的是入射光线有多少会被反射回去，这是因为并非所有的微表面都是朝着一个方向的。

Blinn-Phong 模型的镜面部分可以看成是一个简单的基于微表面的着色模型，它假设微表面的方向分布是余弦的 N 次方。

4.19.3　NDF 着色

我们希望使用的着色模型仅是原来 Blinn-Phong 模型的一个改进而已：

$$L_o = k_d(n\cdot l) + k_s p(h_{local}), h_{local} = (h\cdot t, h\cdot b, h\cdot n) \tag{4.19.2}$$

Download at www.Pin5i.com

正如你看到的一样，其唯一的区别就是我们没有使用固定的余弦 N 次方的分布，而使用了一个任意的法线分布函数。其表面切线 t、副法线 b 以及法线 n 定义了局部表面坐标系的框架，此外凹凹贴图也需要它。如果法线分布函数恰好是 $p(n_m) = (n_{m,z})^N$，那么我们就退化到了最初的 Blinn-Phong 模型。然而，由于法线分布现在是取决于完全局部中间矢量的而不仅仅是其 z 坐标（仅是 $h.n$），因此我们现在还可以创建非等测高亮区域了。

1. 如何存储 NDF

我们如何存储 NDF 以让图形处理硬件使用到它呢？我们将之保存在 2D 纹理图中去了。假设表面的微结构是一个高程场，微表面的方向就只会在上球面上有所不同了。换句话说，微表面法线 n_m 的 z 坐标就肯定是正数了。因此，我们只需使用下面两个坐标就可以得到 2D $p(n_m)$纹理：

$$u_x = \frac{n_{m,x} + 1}{2}, \quad u_y = \frac{n_{m,y} + 1}{2} \tag{4.19.3}$$

在图 4.19.1 中展示了一个通过此方法保存的法线分布函数的例子。假设有一个球面，其中布有所有的法线分布函数的值。现在，我们将此球面投影到一个正方形上面去，这就得到了图 4.19.1 中所示的情况，同时这也阐明了其映射的细节。

2. 渲染/伪代码

那么，我们究竟应该怎样使用这个着色模型对物体进行渲染呢？其实，它的使用方法比标准的 Blinn-Phong 模型麻烦不到哪里去。如果我们已经通过存储法线分布函数创建了一个纹理，那么我们只需要依照下面的步骤就可以对镜面高亮区域进行渲染了：

给定：

一个方位为 $\mathbf{p}_{light}$ 的光源

一个方位为 $\mathbf{p}_{viewer}$ 的观察者

绑定 NDF 纹理

对于一个多边形的每个顶点 $\mathbf{v}_i$：

- 计算出从 $\mathbf{v}_i$ 到 $\mathbf{p}_{light}$ 的归一化后的矢量 **l**。
- 计算出从 $\mathbf{v}_i$ 到 $\mathbf{p}_{viewer}$ 的归一化后的矢量 **v**。
- 计算出中间矢量 **h** = **v**+**l**。

对 **h** 进行归一化

得到切线以及幅法线

- 计算出 $u_x = (\mathbf{h}*\mathbf{t}_i+1)/2$。
- 计算出 $u_y = (\mathbf{h}*\mathbf{b}_i+1)/2$。
- 设置纹理坐标为(u_x, u_y)。

对多边形进行渲染

我们可以以后再考虑散射的影响，或者如果支持多纹理的话也可以现在对之进行处理。如果需要的话，我们还要提供一个额外的光泽图。

Download at www.Pin5i.Com

3. 生成 NDF 纹理

那么我们应该怎样创建这样一个 NDF 纹理呢？我们将使用一个画图程序对法线分布进行建模处理，它与绘制高亮区域本身是等效的。图 4.19.2 中展示了一些高亮区域，而且还给出了它们的外观形态。

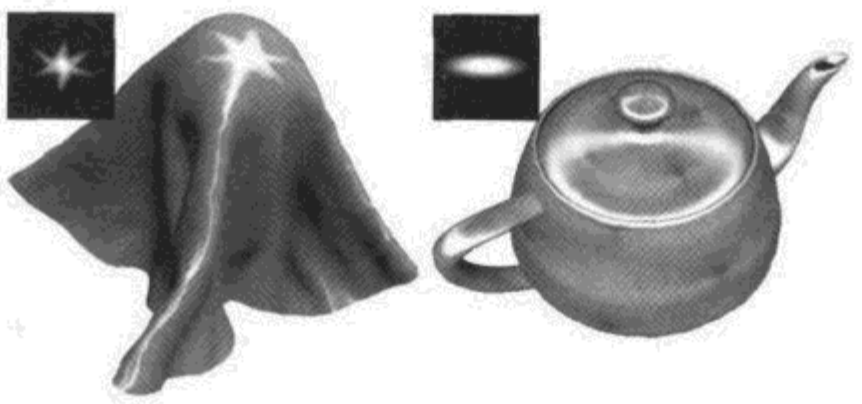


图 4.19.2　用在布料与茶壶上的 NDF

4. 用途

现在我们就会建议应该如何使用散射与崭新的镜面 NDF 以生成有趣的着色模型了。如果我们需要一个标准的高亮区域（就像最初 Blinn-Phong 模型中的一样），那么这个 NDF 纹理将可能是灰度图，因此你就可以根据光源改变其色彩了。纹理中不应该包含方向性的散射分量。在图 4.19.2 中，我们将在几何图形中应用两种 NDF 和一个标准的散射分量。

我们可以加入一个彩色的方向性散射分量以得到更有意思的着色效果。与普通的散射模型不同的是，方向性散射是依赖于 $h.n$ 的。因此，我们可以将之包含在 NDF 中。这个方向性散射当然可以是彩色的，这样就使得它很有趣了。

在彩图 16 中的左边，读者可以看到一块布料，它使用了带蓝色的方向性散射 NDF 并加上了一个红色的散射分量。这块布料基本是紫色的，但是在折角上红色的散射分量比蓝色的方向性分量要大，使得布料基本上是红色的。在中间一图中，你可以看到一个 NDF，它从深红色逐渐过渡到中间矢量所在折角的亮红色，这与一般的 NDF 的外观刚好相反。将这种 NDF 应用到布料上将使之得到天鹅绒的质感。在右边的图中，我们生成了一个非等测的 NDF，它从红色变换到了蓝色。此时，布料看上去就像非等测的绸缎一样。

4.19.4　使用 NDF 的凸凹贴图

NDF 着色可以与凸凹贴图很方便地结合在一起，只要你可以读取衍生纹理。对于凸凹贴图来说，它与 Blinn-Phong 模型的处理几乎是一样的，只是你需要计算出纹理的坐标，u_x 与 u_y，以及查找需用的切线与幅法线。这样，我们就需要将之也保存到纹理图中去了。下面就是一个用来做此用途的像素着色器（DirectX8.1 中的称呼）：

```
ps.1.4
texld r1, t0  ; Normal (tex1)
texcrd r2.rgb, t1 ; Tangent Space l vector
texcrd r3.rgb, t2 ; Tangent Space Halfangle vector
```

Download at www.Pin5i.Com

```
texld r4, t0  ; Binormal (tex4)
texld r5, t0  ; Tangent (tex5)

dp3_sat r1.xyz, r1_bx2, r2 ; r1 = max(n*1,0)

dp3 r2.x, r4_bx2, r3 ; r2.x = b*h
dp3 r2.y, r5_bx2, r3 ; r2.y = t*h
add_d2 r2.xy, r2, one ; r2 = (r2+1)/2

phase
texld r0, t5   ; tex0.rgb = k_d and tex0.a = k_s
texld r2, r2   ; NDF is in texture unit 2

mul r0.rgb, r0, r1  ; k_d * (Nn*l)
mul r1.rgb, r0.a, r2 ; k_s * NDF()
add r0.rgb, r0, r1  ; k_d * (Nn*lL) + k_s * NDF()
```

4.19.5 扩展

此着色模型可以进行 Fresnel 扩展，还可以进行所谓的自阴影（self-shadowing）上的扩展，此处指的是微表面的自阴影，这在更为细致的微表面模型中是常常会用到的[Cook81]。虽然这两种扩展会改变最终的着色，但是在很多情况下它不会对着色模型有什么较大的改变，特别如果使用的是点光源的话。

另一个扩展就是要包括进突发光照的透视缩短（foreshorting）区域以加强镜面光照效果：

$$L_o = k_d(n \cdot l) + k_s p(h_{local})(n \cdot l) \qquad (4.19.4)$$

这将对着色带来很大的影响，特别是在 NDF 纹理中包含了方向性反射的话。尽管如此，最后使用的扩展主要是看你希望生成什么效果或是材质而定了。

4.19.6 结论

在本文中，我们提出了一个着色模型，它易于实现也易于使用。我们可以用它来模拟多种材质，一个艺术创作人员则可以仅凭绘制纹理中的高亮区域以及方向性散射就能直观地修改一个物体的外观。请参见彩图 19，其中显示的就是一个 Standford Buddha 模型加上了散射彩色使用 NDF 着色的例子。虽然此着色技术背后有一些物理原理，但是它实际上并没有模拟任何真实世界中的表面。尽管如此，使用此着色模型得到的视觉效果仍然是很丰富的。

4.19.7 参考文献

[Blinn77] Blinn, J., "Models of Light Reflection for Computer Synthesized Pictures," Computer Graphics Proceedings (SIGGRAPH 1977), pp. 192～198.

[Cook81] Cook, R. and K. Torrance," A Reflectance Model for Computer Graphics,"Computer

Download at www.Pin5i.Com

Graphics Proceedings (SIGGRAPH 1981): pp. 307～316.

[Heidrich99] Heidrich, W. and H. P. Seidel, "Realistic, Hardware-Accelerated Shading and Lighting," Computer Graphics Proceedings (SIGGRAPH 1999): pp. 171～178.

[Kautz99] Kautz, J. and M. McCool, "Interactive Rendering with Arbitrary BRDFs Using Separable Approximations," Tenth Eurographics Workshop on Rendering, pp. 281～292, June 1999.

[Kautz01] Kautz, Heidrich W. and H. P. Seidel, "Real-Time Bump Map Synthesis," Eurographics/ SIGGRAPH Workshop on Graphics Hardware, August 2001, pp. 109～114.

[Kilgard00] Kilgard, M., "A Practical and Robust Bump-Mapping Technique for Today's GPUs," 对应网址为 http://developers.nvidia.com, July 2000.

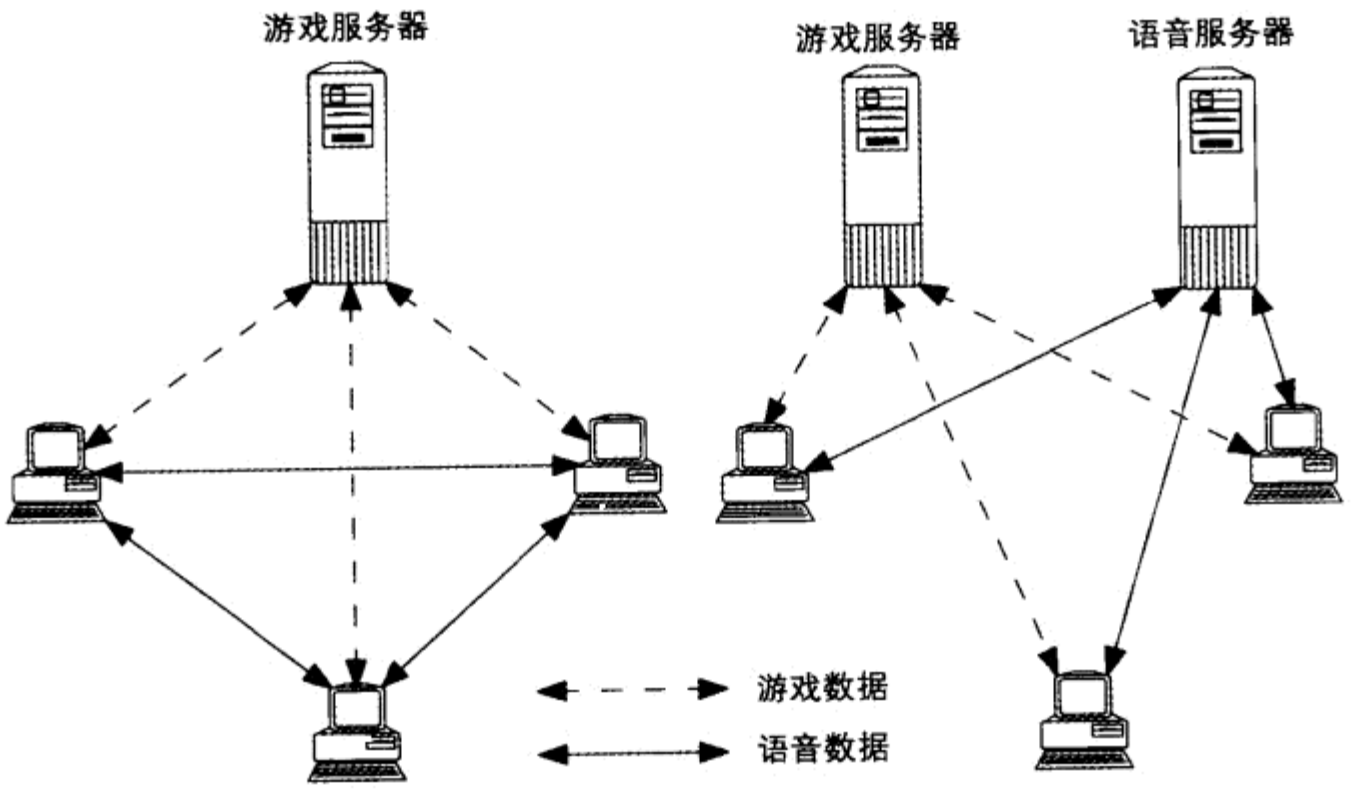
[Mitchell01] Mitchell, J., "Advanced Vertex and Pixel Shader Techniques," 对应网址为 http://www.ati.com/developer, September 2001.

Download at www.Pin5i.Com

Download at www.Pin5i.Com

第 5 章

网络和多玩家游戏



Download at www.Pin5i.Com

简介

Andrew Kirmse
LucasArts Entertainment Company
ark@alum.mit.edu

在本书中加入网络这一章本身就是一个证据，它证明了在线游戏的时代正在到来。很多顶尖的 PC 游戏都是多玩家的，或是只有在线形式的，而网络游戏正在向游戏机与移动平台上面扩展。本章中的各篇文章讨论的是网络——从很小（David Fox 的《使用 Java 微型版开发无线游戏》）到极大（Justin Randall 的《对多玩家游戏进行扩展》以及 Thor Alexander 的《一个针对巨量多玩家游戏的灵活的仿真架构》），我们都有。

实时策略（RTS）游戏在网络的发展史中有其特殊的地位，它将在线游戏带入了广阔的市场。Jim Greer 和 Zachary Simpson 在《将实时策略游戏中的延迟最小化》一文中阐述了他们为 RTS 游戏开发的事件锁策略，而 Jan Svarovsky 则在《实时策略网络协议》中给出了实际实现上的一些建议。这些文章都是从实际商业游戏的开发中提炼出来的宝贵经验。

在线游戏才刚刚起步——在一个游戏的底层网络架构上还需要做很多选择。在许多客户机/服务器的游戏中，安全性是一个很重要的问题。Pete Isensee 的《安全套接字》阐述了 IPSec 的各个部分，此协议为微软的 XBox 游戏机所使用。另外一个方法是使用微软的 DirectPlay，最近才刚刚重写过。Gabriel Rohweder 对它的特性做了一个概述，包括其语音通信功能，这些内容都在《使用 DirectPlay8.1 创建多玩家游戏》一文中。在更高一层的抽象上来看，Jason Beardsley 在《基于模板的对象序列化》一文中展示了一个可以使 C++的类之间自动通信的方法。本人的文章，《一个网络监控与模拟工具》，可以被用来在一个局域网上对一个较差的网络环境进行模拟，以测试底层的方法。

这些文章记录了我们在实现实时分布式系统中的第一步尝试。在我们得到了更多建构在线游戏的经验之后，就可以尝试着将这些早期的成果转化为可移植、可重用的组件了。当网络普及化了以后，我们就可以着眼于更高层的系统，最终得到更快、更好的游戏。

Download at www.Pin5i.Com

5.1　将实时策略游戏中的延迟最小化

Jim Greer, EA.com ， Zachary Booth Simpson, Mine Control
jamesfgreer2@yahho.com
oink54321@yahoo.com

相对于动作类游戏来说，多玩家实时策略游戏的网络要求是大不一样的。它不会传递玩家每毫秒中的动作，但是却必须要管理数百个半自动的个体。这样，其网络协议的优化方法就和动作类游戏的方法完全不同了。在本节中，我们将描述一下事件锁定，一个很适合实时策略游戏的时间同步方法。此方法还可以应用到模拟类游戏和其他非打斗类的游戏上。

5.1.1　帧锁定与事件锁定

在早期一般为局域网（LAN）编写的网络游戏中，采用了一个称为“帧锁定”的技术。通过此种技术，客户机每帧都会发送一个更新信息，它至少包含在此帧的时间里面用户的所有输入。这个更新信息可能会被送往所有其他的客户机（在对等用户的情况下）或是一个对之进行处理然后将其结果转发到其他客户机的中央服务器去。在实践中，某些这种帧锁定的游戏在发送一个更新信息之前便可以渲染不止一帧。但是更新信息一般是以固定的时间间隔发送的。虽然这种方法对于 LAN 而言是比较适合的，但是它不能很好地扩展到因特网范围去，即使只要求低级至中级延迟（150～300ms）。

现在让我们来考虑有一个中央服务器的网络架构。只要所有的客户机都按照同样的间隔发送其更新信息，那么每件事都将工作正常。然而，如果出于网络延迟或是其他的什么原因，有某个客户机的更新信息没有被及时收到，那么服务器和其他的客户机就不得不暂停运行，等待更新信息的到来了。如果它们不等待延迟客户机的输入信息，那么游戏很快就会失去同步——本地的玩家可以看到自己动作的结果，但是却看不到其他玩家的动作结果了。

图 5.1.1 显示了一个这样的示例，它是按事件的发生时间排序的。对于在时间 0 的第一次更新来说，客户机 A 和 B 都把自己的更新信息发送到了服务器。服务器几乎同时收到了两个更新，然后立刻将结果发送了出去。然而，对于第二次更新来说，由于网络的延迟，客户机 B 的更新花费了更多的传输时间。服务器于是开始等待，客户机 A 也是如此。

Download at www.Pin5i.Com

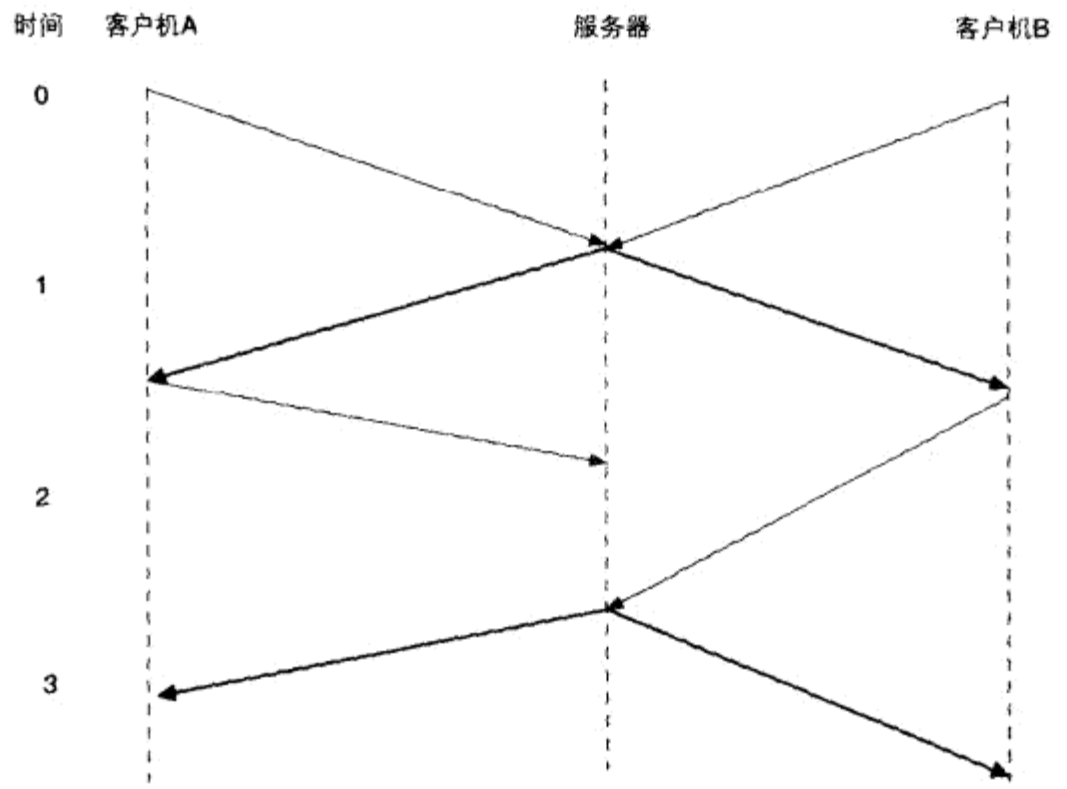


图 5.1.1 帧锁定情况下的等待：服务器与客户机 A 都在等待客户机 B

这种行为使得帧锁定只能适用于 LAN 上的游戏，不过经过修正以后，它可以处理更长的延迟。例如，Age of Empires[Bettner01]使用了一个对等用户的架构，其中有一个自适应的通信往返时间。当不能收到某台机器的通信信息时会产生一个停顿，但是往返时间马上就会增大以适应此种情况了。以后不会有更多的停顿，但是对玩家的响应速度会稍微慢一点。不幸的是，游戏的性能仍然会被玩家中最慢的网络连接所限制住。

1. 事件锁定

在考察了帧锁定之后，我们希望能在新方法里避免最慢的客户机拖慢所有的人。相对于这一点，在这里，我们认为事件锁定是一个有效的方法。通过这个方法，每个客户机都可以发送对事件的请求，而事件则由服务器来负责，一旦服务器认为可以发送，那么此事件将同时对所有的客户机广播。

例如，假设有一个典型的实时策略游戏，在其中一个玩家决定要移动一个坦克。于是玩家在自己的客户机上面发出了移动命令，在执行之前还需要对之进行合法性的检查。如果合法的话，客户机将向服务器发送一个 RequestMoveTank 包。接下来服务器将进行自己的合法性检验，一旦认为此移动是合法的话，就将向所有的客户机发送一个 MoveTank 的包，其中也包括了那个发出请求的客户机(如果想要知道更多一些如何选择什么动作作为事件的信息，请参看[Dinkinson01])。

请注意并不是所有的客户机都会同时收到此事件包的，因此必须在事件里面包含相应的信息，以便于客户机能在适当的时候开始执行事件。例如一个移动的事件，比方说上述的 MoveTank 包，可能会包含一系列的地点，其中每个都有一个相应的到达时间。一个收包比较迟的客户机可以将坦克直接移到当前应在的地点，或是当差异不大的时候，只需要将其动画加速一下就可以了。

看上去这种安排会给玩家带来无法接受的延迟。如果居然花费了 300ms 才使自己的坦克

Download at www.Pin5i.com

移动起来，玩家会不会发火呢？答案就是要给玩家以即刻的“请求反馈”。在最简单的情况下，反馈可能就是在客户机上播放一段动画或是音效，或者对于移动请求来说，如果服务器很有可能会同意此请求的话，可以在客户机上即时就开始“非法的”移动。例如，在 MoveTank 的例子中，假设服务器在绝大部分时候都是会批准此移动的，坦克驾驶员可能会说“遵命，长官！”坦克就立刻开始移动了。图 5.1.2 显示了这个例子。

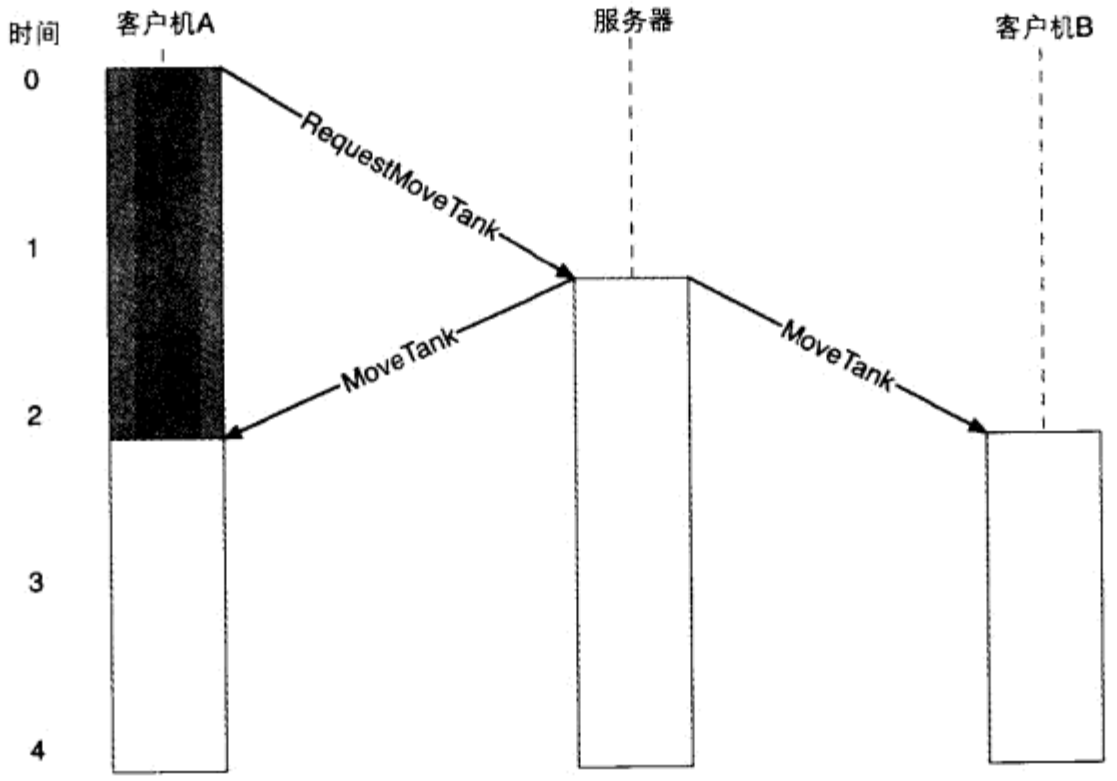


图 5.1.2　坦克移动的例子——坦克在块状区域里移动（灰色表示的是未检验的移动）。所有玩家都可同时看到坦克的移动

有可能服务器会驳回客户机的请求或是安排一条与客户机生成的不同的路线。例如，假设刚好在客户机 A 发出一个 RequestMoveTank 的请求之前，客户机 B 发出了一个在此坦克附近起建一个建筑物的请求。客户机 A 可能生成了一条路线，把坦克先移到北边，然后再开始走，而且同时等待服务器的批准。然而，由于服务器知道现在在坦克北边的路线上有了一个建筑物，于是就将坦克指向了西边。当客户机 A 收到此正确的路线的时候，它必须将坦克从非法的路线上移开，转到正确的路线上来，而玩家有可能根本就不会察觉到。不过，除非延迟非常的高，不然明显的改向将会很少见。此外，对于大部分游戏而言，它们宁可要异常然而却生动的改向，也不要没有异常的但是却死寂的暂停。此外，这种转向只有较慢连接上的玩家才会感受得到。

由于在客户机上有可能会有处于非法位置的个体，这就意味着所有重要的决策都应该由服务器来处理。对于那些没有一个最终权威的游戏来说，同步失调的程序错误将是一个大问题[Bettner01]。在很多实时策略游戏中，其中的某个客户机也是服务器。一般说来，个体的创建与销毁、寻径以及目标瞄准的决策都是由服务器控制和广播的。然而，移动和进攻都是在每台客户机上独立完成的，不会产生任何网络流量。

下面的代码展示了寻径包可能会如何在客户机与服务器之间进行交换。

```
void MoveableUnit::moveRequest( Coord destination ) {
```

Download at www.Pin5i.Com

```
    // GENERATE a temporary path and
    // begin traversing it. This will
    // probably be the same as the path the
    // server finds but not guaranteed:
    setActivePath(generatePath(destination));

    // SEND a RequestMovement the server,
    // even if this client also happens
    // to be the server:
    sendRequestMovement( destination );
}

void handleRequestMovement(
    MoveableUnit &unit,
    Coordinate destination
) {
    // This is the handler for the destination
    // request packet. Note that all clients send
    // requests, even the client who also happens
    // to be the server

    // This can only run on the client who
    // is also the server
    assert( thisClientIsAlsoTheServer );

     // ENCODE which unit is to move and compress
     // the path into a compact form:
    Path path= unit.generatePath( destination );
    MovementPacket p = makeMovementPacket(unit, path);

    // SEND the packet to every client
    // including the original sender
    // even if that sender is also the server:
    broadcast( p );
}

void handleMovement(MoveableUnit &unit, Path &path) {
    // This function is called in response to
    // a MovementPacket.

    // Note: setting the path may induce warping if the new
    // path is not the same as the temporary path on
    // the requesting client
    unit.setPath( path );
}
```

2. 事件锁定的传输层——TCP

在实现事件锁定的时候我们假设使用的是可靠传输协议，例如 TCP[Postel80-2]。在其他的游戏协议中——例如那些当个体移动的时候就产生网络流量，可能会用在飞行模拟器中的

Download at www.Pin5i.Com

协议——丢包是没什么了不起的，因为更新的数据马上就会到来以更正此丢失产生的错误。然而，在事件锁定的实现中，服务器只会把重要的数据发送一次。因此传输层必须保证其可靠性。我们采用了 TCP，而且对之甚为满意。一些游戏开发人员在 UDP 之上编写了自己的可靠传输协议[Postel80-2]，认为他们能改善 TCP 算法以得到更小的延迟。我们强烈反对这种做法。不仅是因为此项工作的复杂性往往被大大低估了，而且也因为其效果不佳。整个因特网的顺利运行都是建构在定义良好、经过很好测试的 TCP 的流控算法上的，如果企图重写自己的流控算法，其结果可能会导致路由器传回灾难性的反馈，而且可能会导致暂时的拒绝服务（如果游戏开发人员实施了自己的中央服务器的话，这种拒绝服务的现象最有可能发生在他们自己的路由器上）。最后一个反对 UDP 的理由就是一般它会被公司网的防火墙挡住，因为它很难保证其安全性。这就会导致开发人员将考虑把 TCP 连接看成是其最后的退路，即使有可能他们已经实现了一个自己的基于 UDP 的系统。

5.1.2　时间同步

通过对客户机进行时间同步，我们可以极大地改善事件锁定的性能。例如，假设当一个个体移动的时候，服务器广播了这么一条消息："对个体 X 进行移动以使之在未来的 T 时刻移动到位置 P。"如果没有时钟同步，那么在某客户机上的 T 时间可能会和另一台客户机的 T 时间相差当前包的延迟这么长（这有可能比平均的延迟要长得多）。在这种情况下，一个有较低延迟的玩家会有比较大的优势，因为他们看到的世界与服务器的世界更接近，从而其请求也更容易通过。

1. 现存的时钟同步协议

时钟同步是一个比较重要的课题，而且现在已经有一些开发得很好的协议了。最简单的就是简单网络时间协议（SNTP）[Mills96]。在此协议中，客户机向服务器发送一个分组数据包（UDP），然后服务器马上会返回一个当前自己的时间。虽然它比较简单，但是当对精确度要求比较高，而且延迟可变的时候，SNTP 算法就没有什么用了，因为它并不测量延迟，也不对延迟进行补偿。

不像 SNTP，网络时间协议（NTP）[Mills92]会使用复杂的统计算法对延迟进行补偿。不幸的是，NTP 实在是太复杂了，而且更重要的是，它在求取精确时间差的时候收敛得很慢。这样，当玩家希望能立即生效，而且不想要那么精确的时间时，NTP 对这些网络游戏就不那么适用了。

还有更麻烦的事情，NTP 和 SNTP 都使用了 UDP 而不是 TCP 以避免引进 TCP 隐含的重传可能产生的不定的网络延迟。如上所述，UDP 经常被很多因特网服务提供商的防火墙挡住，特别是公司网挡住，因此是不能使用它的。

2. 一个简单的取代方法：高阶消除法

我们希望能为游戏找到一个取代 NTP 和 SNTP 的方法。理想状态下，此协议应该足够精确（150ms 或是更小）、快速收敛、实现简单，而且应该能在基于流的协议上运行，例如 TCP。

我们提出了以下的算法，我们称之为"消除了高阶模式的流式时间同步"：

Download at www.Pin5i.Com

（1）客户机在一个时间请求包上加上当前本地时间，然后将之送往服务器。

（2）服务器收到了以后，将自己的时间加上然后返回此包。

（3）客户机收到了以后，计算出一个时间差 delta=(currentTime-sentTime)/2。

（请注意，直到此刻为止此算法仍然和 SNTP 非常相似。）

（4）第一个收到的包被立即用来更新时钟，因为这样做了以后，至少可以将本地时间调整到一定范围以内了。

（5）客户机重复第 1 到第 3 步，循环 5 次或者更多次，每次暂停几秒。可以在其间发送其他的网络数据，但是最好不要多了，这样才能得到最佳的结果。

（6）每个包的时间差都将被存储起来，按照从低延迟到高延迟的顺序排列。中值可以从这个排序列表的中间取一个采样值得到。

（7）在所有上述的采样值中，所有大于中值 1.5 倍的数值都被抛弃，剩余的值使用算术平均得到一个平均值。

此算法中唯一需要注意的就是在所有的采样值中，所有大于中值 1.5 倍的数值都被抛弃这一点。这么做的目的是为了消除那些被 TCP 重传的包带来的影响。为了给出一个形象一点的描述，假设一组 10 个采样包使用 TCP 传输了出去，而刚好没有发生重传。此时，延迟柱状图将会有一个一致的分布，围绕在中值延迟附近。现在，假设在另一个测试中，第 10 个包被重传了。这个重传的值将使此采样落在延迟柱状图的极右端，一般会处在是延迟中值 2 倍远的地方。只要我们简单地把所有距离中值比较远的采样都抛弃，那么这种零落分布的现象就会被很容易地消除了，只要假设它们不会是统计数据的主体的话，而且一般说来它们也的确不是的。

在时间同步中一个十分重要的问题就是当同步正在运行的时候，时间有可能会倒流！在时间同步里任何基于时间的检验（例如，在启动时候的动画或是启动超时时的动画）不要盲目地使用被同步的时间。如果没有意识到这一点，其结果将得到奇怪的“只出现在某地”的程序错误，这些错误只有当试玩阶段玩家处在不同的时区里面的时候才会被发现，而且在开发人员的工作间里面将神秘地不能被重现出来。

这个比较基础的算法在 NetStorm: Islands At War，一个实时因特网策略游戏中测试过，此游戏是由 Titanic Entertainment（1997）的人员共同开发的。结果令人满意，而且一般同步误差都在 100ms 以内。在大型测试中的证据表明，由于重传导致的同步错误是较少的，即使发生了一般说来也是由少见的因特网上的坏连接导致的，此连接将最终导致更严重的错误（例如掉线），这样时间同步的错误反而显得微不足道了。

下面的例子代码演示了我们使用的统计方法：

```
// GLOBAL variables holding time samples:
//-------------------------------------------------------
typedef double Time; // or appropriate type for platform
Time timeSamples[MAX_TIME_SAMPLES];
int numTimeSamples;  // Num valid samples in the timeSamples


// CODE that calculate the time correction:
//-------------------------------------------------------
```

Download at www.Pin5i.Com

```
assert( numTimeSamples > 2 );

// FIND the median:
sort( timeSamples, numTimeSamples );
Time median = timeSamples[ numTimeSamples/2 ];

// FIND mean of samples less than 1.5 times the median:
sum = (Time)0.0;
int count = 0;
for( int i=0; i<numTimeSamples; i++ ) {
    if( timeSamples[i] - median < (Time)1.5 * median ) {
        sum += timeSamples[i];
        count++;
    }
}
Time correctedDelta = sum / (Time)count;
```

5.1.3　结论

在使用事件锁定的方法中我们取得了巨大的成功。我们的第一个 RTS，*NetStorm: Islands at War*，于 1997 年由 Activision 发布，虽然它负担很重，需要实时处理 8 个客户机上成百个个体的动画，但是却只需要在服务器端有一个 9 600 波特的调制解调器就可以了，而在客户机端，其要求只会更小。*Next Generation* 杂志在描述此游戏在网络上玩的流畅性时称之为“如丝绸般地平滑”，即使其网络环境很糟糕。我们希望你也能得到同样的效果。

5.1.4　参考文献

[Bettner01] Bettner, Paul and Mark Terrano, “GDC 2001: 1500 Archers on a 28.8: Network Programming in Age of Empires and Beyond,” Game Developer Conference, 2001, 对应网址为 http://www.gamasutra.com/features/20010322/terrano_01.htm.

[Dickinson01] Dickinson, Patrick, “Instant Replay: Build a Game Engine with Reproducible Behavior,”Gamasutra.com, July 2001, 对应网址为 http://www.gamasutra.com/features/20010713/dickinson_01.htm.

[Mills92] Mills, David,“Network Time Protocol(Version 3) Specification, Implementation and Analysis,” University of Delaware, March 1992, RFC-1305, 对应网址为 http://www.eecis.udel.edu/~mills/ntp.htm.

[Mills96] Mills, David, “Simple Network Time Protocol(Version 4),” University of Delaware, October 1996, RFC-2030, 对应网址为 http://www.eecis.udel.edu/~mills/ntp.htm.

[Postel80-1] Postel, J.,“User Datagram Protocol, STD 6,”USC/Information Sciences Institute, August 1980, RFC-768.

[Postel80-2] Postel, J., “Transmission Control Protocol, STD 6,” USC/Information Sciences Institute, August 1980, RFC-761.

Download at www.Pin5i.Com

5.2 实时策略网络协议

Jan Svarovsky
jan@svarovsky.com

本节的目标是要阐述一个简单而实用的系统，它可以用来在因特网上连接多达10台计算机玩动态策略游戏、经营类游戏或是格斗类游戏。StarTopia[StarTopia01]是一个基于因特网的实时策略游戏（RTS），它可以同时支持4个在线玩家。玩家之间的交互包括放置和维护设备、与单独的人物进行交互以及在战场上指挥部队。本节主要基于在游戏中我们所使用的简单的网络协议。虽然本节主要是对基本原理的一个描述，但是它也包含了我们在实现一个真实的商业游戏中的经历，其中既有成功的经验，也有失败的教训。

首先，我们将大致讨论一下我们用到的一般的协议，然后将谈及基本原理，在此之后是对协议的精炼、对一些我们发现有用的模块的描述以及遇到的一些容易犯的错误。我们还附上了一个简化的示例游戏。StarTopia中的部分系统在CD-ROM上有。

5.2.1 其他的协议

此处我们关心的主要是能支持10个玩家的游戏使用的系统，而不是那些巨量多玩家在线游戏。

1. 客户机/服务器：第一人称视角射击类游戏

对于第一人称视角的动作游戏而言，其最常用的协议就是要求每个玩家都有一个“客户机”程序，它几乎可以被看成就是一个哑终端；有个服务器用来运行游戏，概念上，它处在任何客户机的远端，即使在实际情况中它可以是和某个客户机在同一台机器上的。服务器告诉每个客户机能看到什么，每个客户机告诉服务器它希望发生什么事情，这由玩家的输入决定。另外还有一些工作是用来增加游戏的平滑性的，主要是在客户机上对服务器可能的反应作一些预测。

这个协议的好处在于它的简单性，以及能有效地防止欺骗（因为服务器可以禁止某些非法的动作）。客户机可以随时加入或退出，但是服务器上的游戏仍然存在。不过，它要求客户机上可见的世界状态要比较少，因为这样就可以不停地重传了。出于这个限制，我们就不能在实时策略游戏中

Download at www.Pin5i.Com

使用它，因为此时游戏的状态实在太多了。

2. 对等用户

在对等用户协议中，每台计算机都同样重要。每个计算机都拥有着游戏状态的一部分，而且能完全决定在这一部分上发生的事件（一般说来，此部分包含游戏玩家的实体和几个 AI 的代理）。

此协议的好处就在于由本地计算机拥有的这一部分可以得到立即刷新。在理想状态下，你的计算机将拥有离你最近的那部分 AI 代理，因为对于其他较远的玩家来说，它们就不那么重要了。其坏处就是在游戏中的任何两个人物或是对象之间要发生交互都需要有一个对应的网络包，这样才能在两台不同的计算机之间产生一样的输出。当然，随着游戏状态的扩张，根据网络通信状态而定的需要发送的包数也会成平方增长。

3. 锁步协议

正如其名，一个使用此协议的游戏是和所有牵涉到的计算机同步运行的。它们从同一个游戏状态开始，其游戏时间则分为若干周期。在每一个周期里面，每台计算机都在自己的游戏世界里接受一个动作以前进一步。在此周期的结尾，每台机器都会告诉其他所有的机器用户究竟做了一些什么动作。由于所有的机器都知道所有的用户在做什么，因此它们就可以共同得到在下一个游戏周期的世界，从而保持同步了。

这个协议很容易实现，而且其带宽也相对较低，因为需要传输的东西只有用户的输入（有可能数帧才有一个输入）。其带宽与游戏的状态也是无关的，这对于实时策略游戏来说是一个很大的优点。

这个协议的一个缺点就是每秒游戏的周期是受到往返网络延迟的限制的。在每个游戏周期的结尾，这些机器都必须至少收到从所有其他机器来的一个确认信息，这样才能继续得到下一个游戏周期的世界。要保持这些机器的同步也是相当困难的，你必须保证用户的输入不会直接影响到游戏，只能间接地通过网络的代码来影响游戏。这就确保了所有的游戏都是在处理相同的信息，而且必须时刻遵循这条规则。类似地，你还必须对随机数生成器保持同步。

与客户机/服务器系统有些类似，一些欺骗在此处是很难实行的，因为任何对游戏状态的非法改动都会使得此机器与其他机器失去同步。在另一方面来说，由于每台机器上都有整个游戏的状态，另外一些欺骗就得以实施了——例如，修改游戏以能看见比平常更多的位置。

5.2.2　我们的协议

我们的协议是基于锁步（Lockstep）系统的，另外还加了一些修正以克服其缺点。一台机器将被设置为服务器以使得网络流量能保持与玩家数目成线性比例，否则网络流量就会与玩家数目的平方成正比了。针对每个事件，每个客户机只（向服务器）发送一个包，而不是向其他每个客户机都发送一个包。使用了这种方法之后，当一台机器掉线时，服务器就可以负责调整游戏状态了。

对于锁步来说，在每个游戏周期之间，客户机将从其他客户机处收集输入信息。然而，我们可以消除这种周期之间的网络往返带来的延迟。服务器只要定时广播“前进到下一个周

Download at www.Pin5i.Com

期”的包就可以了，而当服务器收到了每个客户机发来的用户输入之后，就会将之广播到所有的客户机。这就意味着每个用户输入的事件都必须要首先传到服务器，然后再传回，之后才能得到处理，不过同时游戏的周期仍然在继续前进（参见图 5.2.1）。

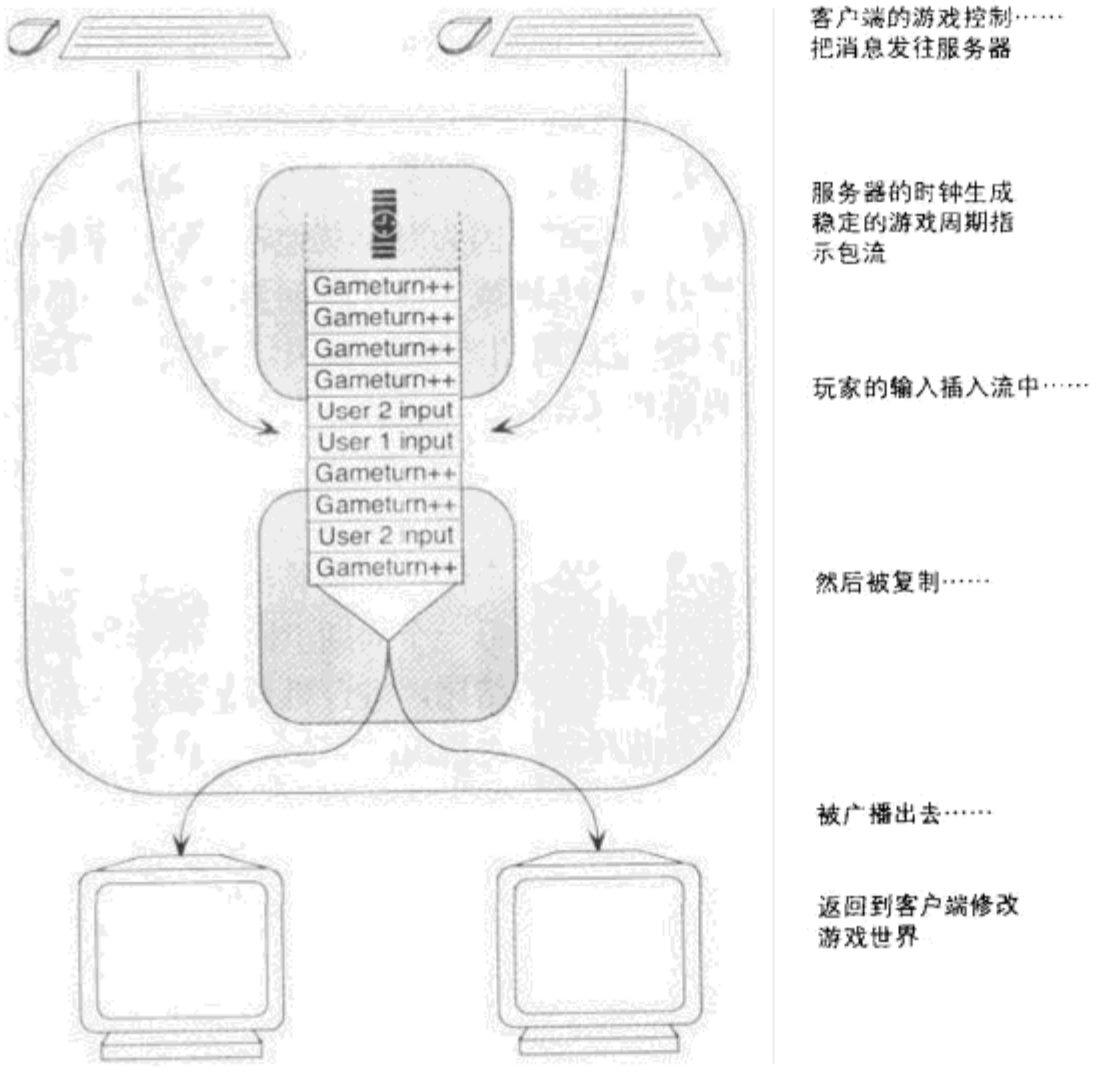


图 5.2.1 服务器/客户机与游戏周期/时间的关系

此系统的主要特性可以归纳如下：

- 在游戏的开始，所有机器上的游戏状态都是一样的。
- 接下来服务器将发送一系列相同的包给客户机，这是固定不变的“前进到下一个周期”的数据包流，中间还夹杂着用户的输入。
- 只需基于服务器发过来的信息，游戏就可以继续前进了，而此信息对于所有客户机来说都是一样的。因此，在整个网络上游戏都保持同步。
- 由于此原因，我们需要把用户输入从本地游戏状态中剥离出来，这是很重要的一点。用户的输入被传输到服务器，在此又被广播到所有其他的机器以使得同样的动作能得到执行。
- 游戏周期不必绝对同时地发生在所有的机器上，但是游戏事件的顺序必须都是一致的。
- 在用户输入和其效果最终显示在屏幕上，中间的时间至少是一个往返延迟（我们将在下一小节讨论此话题）。
- 所有的网络包都必须保证传输的可靠性。你希望所有的用户动作都得到处理，因此所有发向服务器的包都必须到达。同样，所有服务器发出去的包也应该按序到达，这样所有的客户机才能收到同样的事件流。

Download at www.Pin5i.Com

每秒的游戏周期数目

在决定每秒应该有多少游戏周期之前，我们必须考虑以下几个因素。正如要在下一小节讨论的那样，图形引擎并不会按照游戏状态的本来面目将之画出来，它画的只不过是其插值产生的结果而已。这就意味着每秒的周期数不会直接被玩家感觉出来，因此我们需要其他因素的帮助以进行此设计决策。

当然了，在这一点上有个平衡的问题。较低的游戏周期数会带来以下的优点：

- 绝大多数时间里面，网络上流通的只有服务器发出的游戏周期更新包，因此如果周期的数目更少，则需要的带宽也就更小。
- 大多数游戏周期的处理对于每个周期来说都是一个常数开销，这与每秒钟有多少周期是无关的，最简单的例子就是一个以常数速度移动的对象。降低了游戏周期速率的话，对游戏建模的 CPU 开销就会更小。

在另一方面来看，较高的周期速率也是有其优点的：

- 某些游戏处理每秒钟的开销是一个常数，而不是每个周期开销为常数。例如，如果你的机器人要每秒对周围的区域扫描一次，那么将每秒的周期数目减少一半就会导致每个周期的扫描次数变为原来的 2 倍。在极端情况下，对世界的处理会在平滑的渲染中加入一个可见的暂停现象。在 StarTopia 中，你可以在一个配置很低，然而却有一个高端显卡的计算机上运行游戏，然后在游戏的区域中产生很多动作，从而重现此效果。此时帧率较高，但是计算机却会在游戏的处理上忙得焦头烂额。
- 当用户输入到达的时候，其效果只有在游戏的下一个周期中才会被实现。这就意味着，平均来说，游戏中的响应时间会被加上一半的游戏周期。如果游戏周期划分得更多的话，那么此延迟效果就会被消弱了。

在 StarTopia 中，我们最后采用的是 6 周期每秒。而 Dungeon Keeper 小组[Keeper01]使用了 4 周期每秒的配置。在两种情况下都对不同的数值进行了测试，而且事实上在游戏开发过程中还在不停地变化。对于大多数实时策略游戏来说可以使用类似数目的每秒周期的数值。

5.2.3　精炼

一旦实现了基本的协议以后，我们就可以对之进行一些修正以使游戏的可玩度提高了。

1. 在游戏周期之间进行插值

首先，客户机将会对游戏周期何时从服务器到来有个期望值，而且会尝试着在周期之间平滑地插入一些场景。在实际情况中，所有发过来的东西只不过都是从服务器而来的批准而已，它是用来对下一周期的世界进行建模的，另外还可能有一些新的用户输入。

虽然世界每秒的更新次数是一个常量，但是每个客户机都可以尽可能快地将屏幕场景渲染出来。有着更快速度计算机的玩家会得到更平滑的显示效果，但是游戏对于所有的玩家而言都是一样的。

当然了，要想正确地预测出下个周期精确的到达时间是不可能的，因此只要预测系统估计得足够接近就可以接受了。我们发现使用对前几次到达时间的加权平均值是一个比较合适的值。

Download at www.Pin5i.Com

这个修正并不需要预测未来游戏的状态，你只不过是要在最后两个游戏周期之间进行插值而已。此插值对于大部分事物来说可以是线性的，例如对象的位置，即使是在每秒只有 4 个周期的时候。理想状态下，系统将能正确地预测出下一个游戏周期发生的时间，因此你可以继续把插值进行下去。这将在每个游戏周期上导致一个额外的延迟，此延迟有可能会比较大，但是它的游戏效果却会更好。

2. 隐藏延迟

一个余留的主要问题是在游戏世界里玩家的输入和响应之间会有一个可察觉到的延迟。只有在事件从服务器再返回来了以后，用户的输入才会产生效果，这样就增加了一个游戏周期的延迟和一个额外的用于插值的延迟值。

在一个经营型的游戏里面，你可以把游戏分为界面、网络部分以及游戏状态，以把这个延迟给隐藏起来。游戏状态是唯一一个必须要同步的部分。此处的界面含义较广，它尽可能多地包含了游戏对用户的响应。

我们将之划为界面的游戏状态包括下面几个例子：

- 在一个游戏里，如果玩家能创建房间或是建筑物，完成了部分设计的房间布局都是在本地处理的，只有最后完整的房间设计才会被广播出去。
- AI 对命令的响应有个延迟，但是用户界面对命令的响应，例如在目标上闪烁的光圈，则可以立即在本地处理。即使你的坦克实际上过了四分之一秒才对目标开始开火，玩家也不会察觉到。
- 假如你正在看的地方不会对游戏造成影响的话，镜头的移动通常不需要传输过去。
- GUI 的交互只应该在本地机器上处理。例如，当我们人物正在交谈的时候，其头部可以随着屏幕上鼠标的位置转动。

事实上，可以在与世界交互量更大的游戏中使用此协议，例如在第一人称视角射击类游戏里面，但是其效果就不怎么好了。一个例子就是玩家人物的移动。一个“左移”的包可能会变成一个“我希望在游戏周期 x 时已经左移了”，显示给本地用户看的影像将会被更新，就好像此移动已经生效了一样。每个从服务器收到了此事件包的客户机都要判断此移动是否有可能。然而，此判断在所有的机器上都是一样的，因此一旦没有批准则本地机还必须负责将无效的动作给撤销掉。

3. 主机迁移

由于有一台机器被分配成为了服务器，因此当那台机器退出游戏的时候就会产生问题。这是很有可能发生的，因为一般来说，玩家不希望由于其他人的退出而导致自己也退出。

为了解决这个问题，你必须首先确保在服务器上的状态已经尽可能地少了。上述的系统是很理想的，因为此服务器只不过是一个时钟而已，它仅仅被用来对游戏周期计数，然后就是一个用来放置用户输入包的临时的场所了。

剩下的客户机必须决定要使用其中的哪一个来放置新的服务器。不同的客户机可能从服务器收到了不同的事件流。其中我们应该使用最新的版本，因为收到了最新事件的客户机已经不能撤销此效果了。

为了得到一个更加无缝的迁移，客户机应该检查一下是否它向服务器发送了一些包，而

Download at www.Pin5i.Com

服务器还没有将之放在新的事件流里广播出去。这些包应该再被重传一次。存储 5 秒以内的网络包大概还不会占用很多开销，因为包的尺寸都比较小。

在 *StarTopia* 中，对于服务器的退出我们还没有这么好的支持。由于大多数游戏都是 1 对 1 的，因此服务器退出的情况要比预想的少得多。我们有一个频繁的自动存盘系统（大概每分钟存一次）。如果由于某种原因服务器退出了，剩下的客户机仍然可以从最近的自动存盘中恢复回来重新开始，这样对游戏的影响就很小了。

4. 对较慢计算机的处理

服务器将以固定的频率生成游戏周期，即使只有一小会，此值对某个客户机来说也可能太快了。如果游戏周期的更新请求是每秒 4 次，而客户机则需要半秒来处理一个游戏周期，那它将永远也跟不上，最终会被冻结住，无法跟上游戏的进程。

我们添加了一种额外的包，是用来让客户机发向服务器的，此包不会被广播出去。这个包将告诉服务器处理一个游戏周期占用客户机 CPU 的百分比。服务器会相应地减慢游戏速度，允许较慢的客户机跟上来，确保 CPU 处理的开销对所有的客户机都降到了一定水平之下。

5.2.4　有用的模块

接下来将谈一下我们觉得有用的 4 个其他的技术。

1. 在单人游戏里模拟延迟

即使是在单人游戏里也应该有模拟的网络延迟。对于我们来说这是很有用的，因为游戏开发和测试的大部分都是基于单人模式来做的。当我们开始测试网络游戏的时候，测试人员非常担心网络延迟可能会给游戏的可玩度带来的影响。在此时，我们很得意地指出其实他们一直都是带着延迟来玩的。游戏在一开始开发和测试的时候就考虑到网络延迟的问题。

2. 从指针到对唯一的 ID

虽然游戏的状态可能是完全同步的，但是指针可不会在不同的机器上都完全一样，因此任何需要被网络包引用的（C++的）对象，例如“我正在点击这个单元”，都需要一个唯一的 ID。相对来说这是不花什么力气的，只要将所有的对象都从一个包含一个唯一数字 ID（以及一个指针到数字转化的函数）的基类派生出来就行了，另外还需要一个查找表以进行从数字到指针的转化。

在此处有一些需要提防的小陷阱。例如，当对象被销毁，而新的对象被赋予了同样的数字时，可能还有一些网络包引用的是老对象。可以通过给每个 ID 加上一个较小的额外数字对此缺陷进行修正，当对象被销毁，而此 ID 又被用到了新对象上面的时候则此值就会加 1。还有一个取代方法，那就是使用一个很大的数作为对象的 ID，而且绝不重用 ID，或者以最近最少使用的原则来重用 ID。这将增大 ID 到指针查找表的尺寸，或是需要将之转化为一个散列表。在 *StarTopia* 中，在一个典型的会话里，我们将创建和销毁数目惊人的一堆对象。

3. 调试同步失调的问题

在网络游戏的调试中，其主要的工作就是要进行跟踪，找出为什么客户机会和其他的机

Download at www.Pin5i.Com

器同步失调。检测游戏是否同步失调了可以只需要一个校验值系统就够了。这个校验值将包含对象的数目和位置以及一点其他的东西。

一旦游戏同步失调了，仅仅查看游戏状态的异同是没有什么用的。因此，最好的方法是在代码里面乱放一些 printf，特别是在那些代码中的关键点上。然后你就可以比较两台机器的输出看看究竟它们是在什么地方失调了。

当然了，使用 printf 代价是比较大的。因此，我们打印的大多数都是数字和常数字符串，我们保留了一个很大的包含小结构体的队列，在结构体里面是指向字符串或是数字的指针。printf 的操作只不过是把结果写到此队列的尾部而已，然后只有当游戏同步失调的时候，指针才将被解引用以得到“同步输出”。为了简单起见，输出将被写到一个网络驱动器上的文件里面，此文件名是依赖于各个机器的。

4. 丢包

如前所述，在我们的系统里面，所有的网络消息都应该确保按正确的顺序到达。我们发现还需要一个自己的消息保障系统，它处在此处描述的协议的下一层。对于每个客户机来说都要有一个消息流发向服务器，其中包含的是用户的输入和一些维护性的数据包。从服务器到每个客户机也有一个流，包含的是递增的游戏周期和用户事件。虽然每个客户机都收到的同样的信息，这些流却是被分别处理的。

5.2.5 在 StarTopia 中容易犯的错误

我们将描述以下在实现此系统中遇到的最常见的问题。当然了，本小节是针对 StarTopia 而言的，但是我们得到的教训希望也能对你的项目有所帮助。

1. 同步失调

如前所述，就目前来说这是我们遇到的最大的问题，但是它并非是不能克服的。虽然在游戏中有很多的细节，但是我们已经将同步失调的错误减少到了在测试中无法发现的地步。此后，我们利用宏定义去掉同步失调的调试模块，加入一条消息建议玩家可以从最新的自动存盘重新开始。这条消息迄今还没有被发现（除非我们作弊将之显示了出来）。

我们同步失调错误的主要原因是因为，在本来不需要的情况下，在网络协议加入之前，我们已经编写了游戏的很大一部分了。因此，偶尔遗留的一些代码会打乱协议的规则。由于此原因产生的最常见的错误包括[Ireland01]：

- 直接改变游戏的状态，而不是通过一个应该广播到所有客户机的网络包来办这件事。
- 游戏中的代码直接引用了玩家机器上的玩家。
- 游戏中的代码引用了镜头。
- 游戏中的代码直接使用了系统时钟进行计时，而不是游戏周期。

这些问题本来可以通过严格的界面与游戏状态的分离办到，例如控制游戏状态的代码应该完全不知道当前机器的玩家，也不应该知道他正在看向什么地方。一旦你遵循了此条原则，那么剩下来的导致同步失调的常见原因还有：

- 没有初始化的数据。

Download at www.Pin5i.Com

- 不同版本的数据文件。一旦游戏发布了以后，这就不是一个问题了，但是在开发过程中，当人们可能会运行很多版本的时候，那就需要有一个所有载入游戏数据的校验和。
- 不同的可执行文件。再一次，此问题最简单的解决方法就是在游戏里要求所有的玩家都使用完全一样的可执行文件，虽然这将导致一些额外的测试工作。

2. 事件包变得没有用了

这个问题或者错误典型的表现就是一个事件包引用了一个对象，而此对象在此包送到服务器再返回来的途中被销毁掉了。或是，一个包描述了对某个对象的改变，而此对象已经不再在适当的状态以接受此改变了（有可能有那么一会儿，我们会雇用或者解雇一个在此期间已经死亡的人！）。

当然了，这种情况的测试是比较简单的——上述的唯一 ID 系统就可以捕获对象销毁事件。此外，你应该保持检查自己的对象，看看它是否处在一个可以对某事件进行响应的状态。这个问题之所以成为问题是因为当玩家发送很多包或是多个玩家同时使用一个对象的时候它才出现。这种情况在游戏开发的晚期出现得更加频繁，当时游戏已经快要接近尾声，将要进行一个较大的演示了。由此可见，这种错误可能会隐伏很长的一段时间。

5.2.6　示例游戏

在 CD-ROM 上已经加入了同步失调的调试代码。我们鼓励你去阅读这些代码和注释。更加完整的模块，例如 syncdebug，已经给添加了上来，因此你可以使用它们作为一个实际实现中的良好基础。我们还鼓励你去阅读它的代码和注释。

5.2.7　结论

此协议实现简单，而且在我们的游戏中工作正常。它给了我们自由发挥的空间以编写出一个充满了很多同时事件的游戏，而不必担心它们对网络带宽的影响。如果你想要试一下我们的协议的话，希望本节能对你有所帮助。

我要在此感谢 StarTopia 小组中其他的成员，特别是 Tom Ireland，在我写作此文之时，他给我提供了一些有用的信息。

5.2.8　参考文献

[Ireland01] Ireland, Tom, list compiled during game development, July 2001.

[Keeper01] Bullfrog, *Dungeon Keeper II*, PC game, 对应网址为 http://www.dungeonkeeper.com, June 1999.

[StarTopia01] Mucky Foot, *StarTopia*, PC game, 对应网址为 http://www.muckyfoot.com, July 2001.

Download at www.Pin5i.Com

5.3 一个针对巨量多玩家游戏的灵活的仿真架构

Thor Alexander
Hard Coded Games
thor@hardcodedgames.com

巨量多玩家（MMP）游戏为成百上千的玩家提供了乐趣和令人着迷的游戏体验，他们在线上互相战斗或是加入队伍去战胜共同的敌人。在因特网上创建如此大量和持续性（persistent）的游戏场将是游戏开发中，甚至是一般的软件开发中，最大的一个挑战。一个 MMP 由很多组件组成，包括网络组件、渲染组件、数据库访问组件，还有游戏仿真组件。本节将关注于游戏仿真，我们阐述了一个解决方案，它提供了一个灵活的仿真架构，此架构可以被重用以创建很多不同风格的在线游戏。

商业的 MMP 游戏更像是一个服务，而不是一个产品。一个成功的在线游戏服务应该至少有 5 年的生命期。在此期间，必须维护代码基，而且要对之进行及时的修改以加入到一个产品中，或是一个实时的环境中，在其间会有很多讨厌的玩家对每个修改进行批评。为了能在这样一种环境中生存，此代码基必须建构在扎实的工程基础之上。为了达到此目的，我们将同时从设计模式工具和 UML 中吸取技术。

设计模式是近年来非常有用而且经过考验的软件工程的工具。一个模式就是一个针对标准问题的可重用的解决方法。若想知道更多的关于模式的信息，请参看[Gamma94]。建模语言（UML）是一个将软件系统可视化和规范化的设计过程。本节将展示一些 UML 类图和序列图。若想对 UML 有更深的了解，请参看[Booch98]。

5.3.1 架构一览

首先，让我们来对这个针对 MMP 的架构作一个总览。

1. 客户机/服务器组件

从本质上来说，MMP 就是一个客户机/服务器的系统。它提供了一个网络层用于在因特网上的客户机和服务器之间传输消息数据包。这些消息数据包会被游戏仿真层接收和解析。此层负责维护在客户机和服务器上面的游戏状态空间，而在状态空间中仿真细节的物理表现则是由物理层来负责的。由于客户机的渲染层只会表现出玩家接收到的数据，因此在任何时刻，客户机上需要仿真的对象都只是服务器上面对象集的一个子集而已。

Download at www.Pin5i.Com

这样就使得客户机能够比服务器的物理层在物理仿真上呈现出更细致的表现力。请参看图 5.3.1 以对架构的总体有个印象。

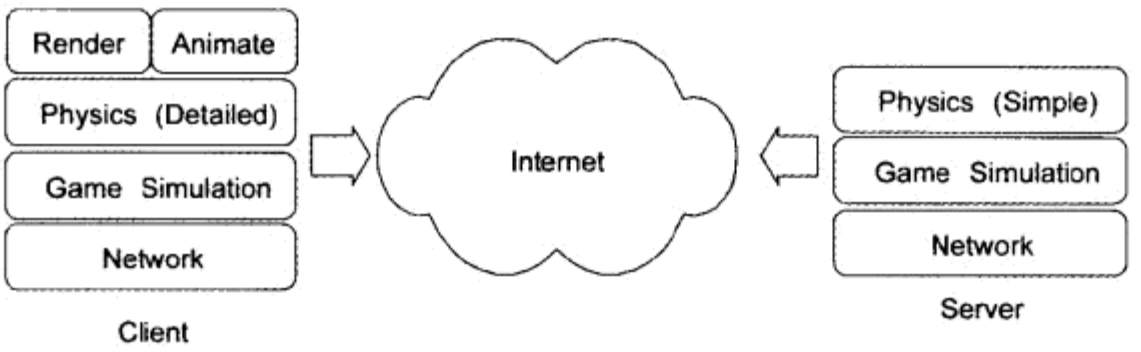


图 5.3.1　客户机/服务器游戏的架构

2. 代理进行的仿真

在服务器上面的游戏仿真层掌握了一个完整而真实的状态空间。如果有任何一个客户机同步失调了，那就它必须以指导者的身份出现以再次同步。服务器将把状态空间中角色和对象的改变以仿真事件的形式广播出去。而客户机则利用这些事件去更新其代理对象（proxy），此对象负责维护一个状态空间的本地拷贝。用户的交互首先被作为动作请求送到服务器，然后被服务器的仿真层通过了以后才能使相应的动作生效。图 3.5.2 描述了这个请求/响应流程。考虑到安全性的问题，在一个 MMP 的设计中必须绝对不要信任客户机上状态空间表现的有效性。如果想对代理仿真有更细致的了解，请参看[Perlin96]中的虚拟仿真小节。

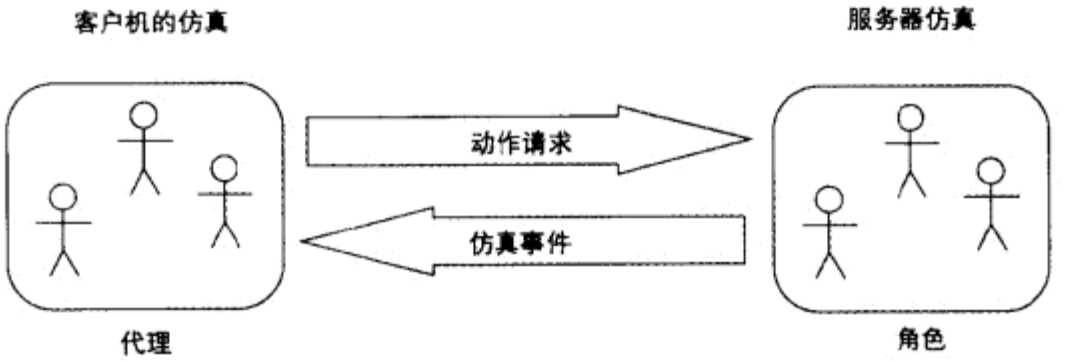


图 5.3.2　经过角色与代理仿真的客户机/服务器通讯

5.3.2　支持类

在我们讨论自己架构中的核心类之前，还需要预先定义一些支持类，它们将是建构系统核心类的基础。

1. 字典/散列表

字典是一个抽象数据类型，它保存了一些子项以及与其相关联的值。其基本操作有 AddEntry、LookupEntry 和 RemoveEntry。一个较好的字典实现方法是使用散列表，它是一个关联性数组，在其中一个键值可以通过一个哈希函数映射到数组中的某个位置上。图 5.3.3 显示了这么一个字典的类图。此字典将成为架构中主要的数据结构。

Download at www.Pin5i.com

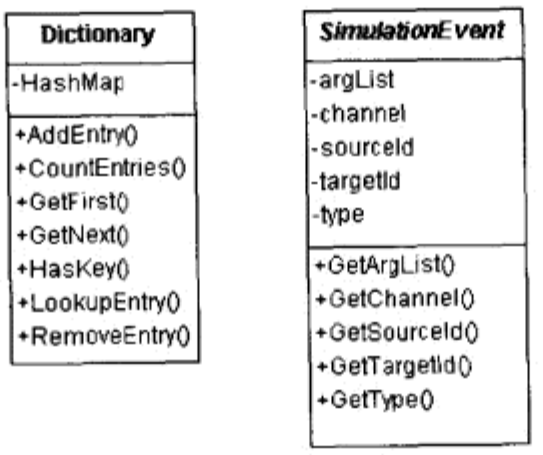


图 5.3.3 支持类的类图

2. 仿真事件

仿真事件是系统中的交易对象。当一个角色通过动作与环境进行交互的时候，这些动作的结果将向其他能识别此事件的角色进行广播。一个事件对象包含了事件类型、源角色 ID 以及一系列特定于此类型的参数。此外，它还有一个频道属性，此属性可以被事件的接收者用来对其进行过滤，得到自己感兴趣的事件。

3. SimulationState

SimulationState 是我们对状态模式的一个实现。为了清晰起见，我们在此处展示了一个简化后的模式版本，它既没有包含状态机，也没有包含状态管理器（若想得到一个更细致、更健壮的实现，读者可以参看[Boer00]与[Dybsand00]）。SimulationState 在我们的架构中是所有状态的基类，其基本操作有 CanTransition 以及 Transition。其基类中的 CanTransition 是一个用来预先检验的方法，它将确认是否其上下文（context）对象能够触发一个转移达到此状态。其派生类可以实现额外的检验以满足其自己的需求。而 Transition 方法则是真正发挥作用的方法，每个子类都必须实现此方法，并且要提供任何进入此状态时所需要进行的特定行为。

4. ActionState

ActionState 是一个游戏仿真中的典型操作，例如击打一个对手或是打开一扇门。每个 ActionState 都有自己的 durationTime 属性，它详细描述了此动作需要执行多长时间。一般说来，此属性是用来对服务器的仿真与客户机的动画回放时间进行同步的。ActionState 还维护了一个 startTime 属性。在一个角色已经进入了一个 ActionState 之后，如果此时进来了一个作为旁观角色的观众，那么这个属性就可以用来计算此时动画的播放时间了（参见图 5.3.4）。

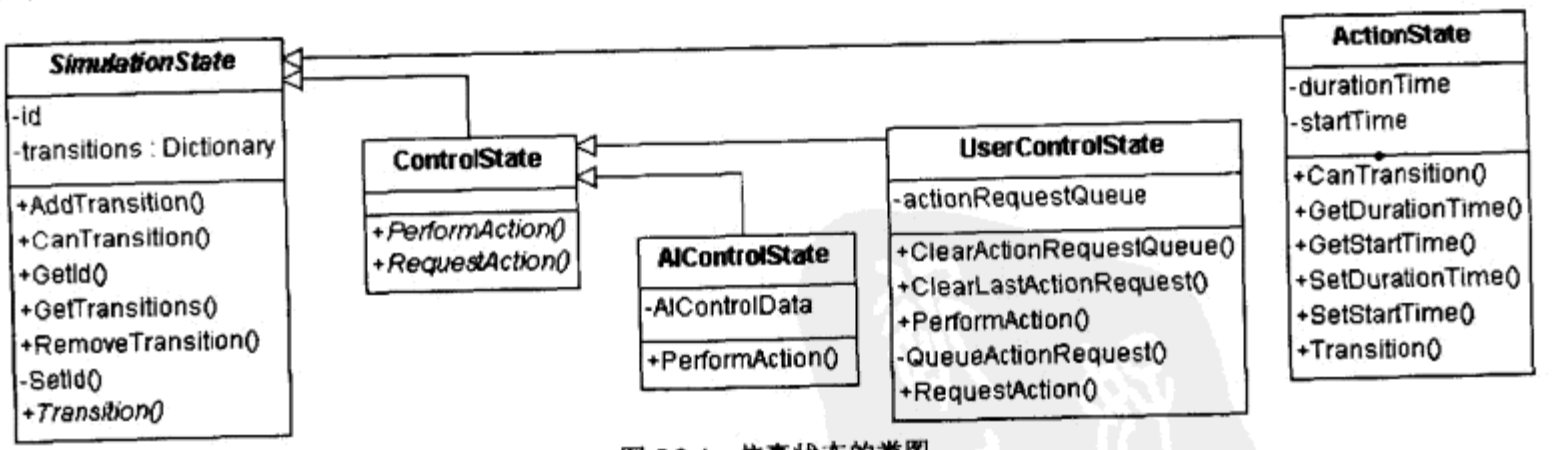


图 5.3.4 仿真状态的类图

Download at www.Pin5i.Com

5. ControlState

一个 ControlState 定义了相应的角色能从何处接受命令。这就使得角色可以处在玩家的控制之下、AI 的控制之下，或是其他的模式控制之下了，例如在一个脚本状态控制下的游戏内部的过场动画。ControlState 还可以在运行时进行交换以使得角色在上述的几个状态之间切换。ControlState 的另一个用处就是可以利用观察进行 AI 训练。当处在一个训练状态控制之下的时候，玩家会和平时的用户控制状态一样对角色进行控制，但是此时计算机将会对角色进行“窃听”，而且可以从其动作中得到学习（若想对通过观察进行训练有更多的了解，请参看[Alexander02a]）。

6. UserControlState

UserControlState 维护了一个队列，其中包含的是玩家发过来的尚未执行的动作。当此控制状态的 PerformAction()方法被调用的时候，它就会从此队列中取出各个请求。

7. AIControlState

当处在此状态的角色调用 PerformAction()方法的时候，由 AIControlState 来决定下面应该执行哪一个动作。此决策过程应该使用 AI 技术来实现，而此技术应该是对仿真游戏的机制适应得最好的技术，例如有限状态机、神经网或是模糊逻辑。读者可以参看[Alexander02b]，里面描述了其中的一个技术。

5.3.3　核心类

现在我们可以来讨论此架构中的核心类了。

1. SimulationObject

SimulationObject(SOB)形成了所有仿真涉及到的类的基类，包括角色、区域、物件以及障碍物。图 5.3.5 显示了其类层次。SOB 有自己的 ID，此 ID 是用来唯一标识此对象的。SimulationObject 通过发送仿真事件与其他的 SOB 进行通信，它们可以向其他的仿真对象预订其感兴趣的事件。每个 SOB 都维护了一个预订对象的字典，而且通过此字典向这些预订对象发布生成的事件。

一个 SOB 可能会包含其他的 SimulatonObject。每个 SOB 都维护了一个内容字典，其中就是自己包含的对象，还有一个 ownerId 用来指向包含自身的 SOB。SimulationObject 的容器是一个抽象的概念，有了它就使很多子类特性的实现更方便了。角色子类可以用它来实现一个储备箱物件的管理系统。物件也可以成为游戏内部的容器，例如箱子或是包，它们也可以包含其他的物件。作为一个抽象空间标识对象的区域可以使用容器来跟踪其他的在其内进出的 SimulationObject。

与容器类似，SimulationObject 还维护了一个连接向其他 SOB 的字典。连接在 SOB 中提供了一个聚合的概念，或是“整体/局部”的关系。这就提供了一个简单但却是强大的方法，它可以用来使对象能够接收从临时的组成部分而来的事件。连接可以用来作为区域之间可关可开的门户或是通道。复合物件就可以通过将物件连接起来的方法创建出来。

Download at www.Pin5i.Com

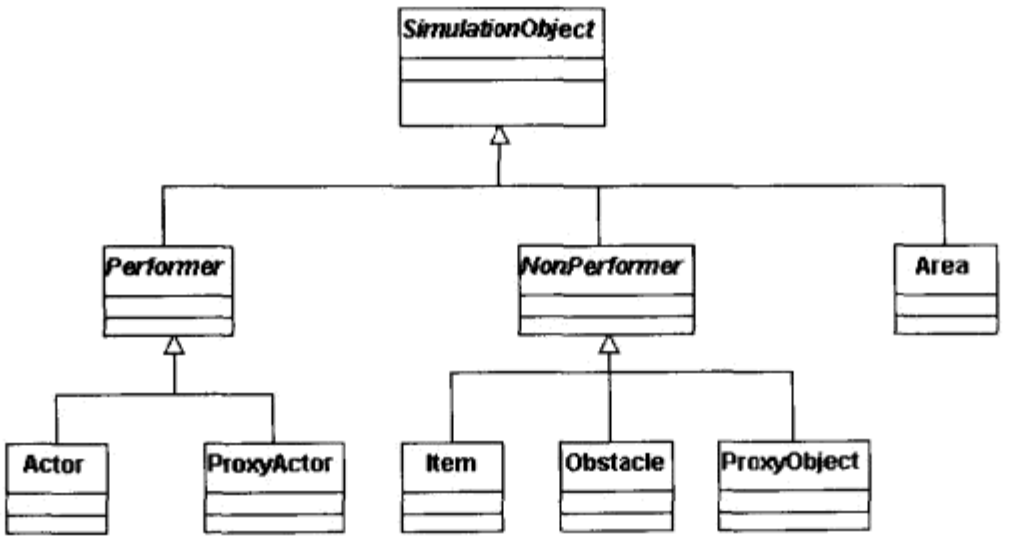


图 5.3.5 核心类的层次结构

SimulationObject 另外还提供了一个属性字典用来存储游戏中特定于系统的属性，这些属性是针对于单个对象的，SOB 将利用数据驱动的形式来使用它们。在需要的时候，每个 SimulationObject 的子类都可以在字典中添加自己的属性。字典的内容可以使用被复制到客户机上相关的代理对象中去，不过必须要使用智能而即时的方法以将网络流量的开销最小化。例如，属性中可以包含对象在世界空间中的坐标、运动速度、生命值、魔力值等东西。

最后一点，SimulationObject 提供了一种持续性的机制，它可以用来让仿真层以一种通用的方式与存储系统打交道。每个 SimulationObject 都有一个脏标志值，当持续性的属性或是数据被改变了以后，此标志就将设置为 true。接下来，在需要的时候，仿真层就会调用 SOB 的 `Store` 方法。`Store` 将检查一下脏标志是否为 true，如果是的话，就将存储此对象。`Restore` 方法刚好是 `Store` 的逆过程，它将把对象载入到仿真中去。这些方法在目标应用程序里面实现的时候则可以将对象保存为普通文件的格式，例如 XML 或是 Oracle 或 MS-SQL 的数据库中（参见图 5.3.6）。

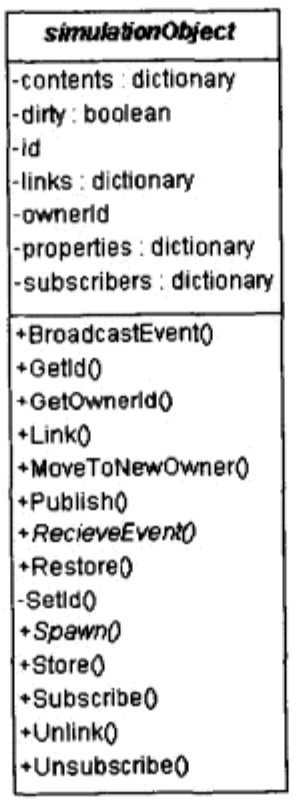


图 5.3.6 SimulationObject 的类图

Download at www.Pin5i.Com

2. Performer

抽象的 Performer 核心类提供了共享的客户机/服务器的功能，还有 Actor 和 ActorProxy 仿真对象类的通用接口。执行者类（Performer）维护了一个 schedulePriority 的属性，它是给仿真进行调度用的。一个执行者还维护了对象的 currentActionState 属性，它代表的是对象当前正在执行的动作，此属性是由 PerformAction 方法来设置和决定的。在实现较高级仿真的时候，可以对此对象进行扩展，在各个互不影响的活动层上给角色添加多个并行的动作状态，例如移动活动层、姿态活动层、对话活动层等等[1]（参见图 5.3.7）。

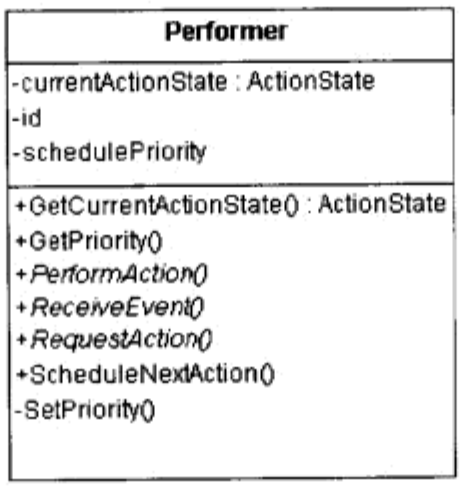


图 5.3.7　Performer 的类图

3. Actor

Actor 就是一个在服务器端的仿真对象，它可以与仿真的环境进行交互。角色有一个 ControlState，这可以让它们通过很多不同的代理（agent）来控制，包括玩家、AI 以及脚本化的过场动画。PerformAction 和 RequestAction 方法都会把调用转交给当前的控制状态，在此状态中控制代理将负责提供相应的实现。Actor 类还维护了一个 eventQueue 属性，它是为 ReceiveEvent 方法使用的（参看图 5.3.8）。把事件放入队列里面去以后，Actor 就可以把它们先放置起来推迟处理，直到下一次调用 PerformAction 方法再说。

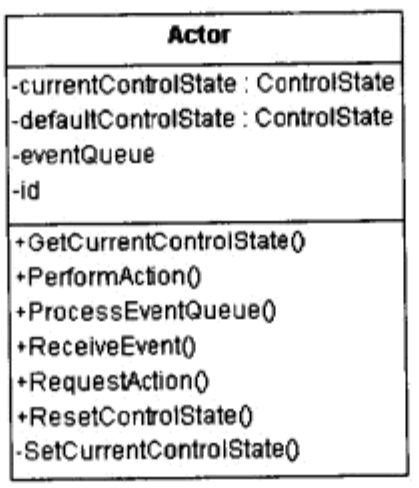


图 5.3.8　Actor 的类图

译者注[1]　作者举的例子是说在一个场景里面角色可以在移动活动层中挥舞宝剑，在姿态活动层中骑马，在对话活动层中大声叫骂。这些层互不干扰，但是可以协作表现一个角色在多层里面的动作状态。

Download at www.Pin5i.Com

通过这个被动的，即时的事件处理机制，我们就可以让系统避免在每次一个 Actor 收到了事件的时候就要做出需要做什么事情的决定了。在一个事件极多的仿真中，例如 MMP 游戏中，此处角色将需要对每一件周围发生的事情进行关注，此时这就是一个极大的改善了。ReceiveEvent 方法还可以过滤掉那些需要即使处理的事件，绕过事件队列，一收到事件就进行相应的处理。

4. ActorProxy

ActorProxy 是 Actor 在客户机上的对应对象。它与 Actor 具有同样的 SimulationObject 的 ID，而且复制了角色的相关数据。客户端的仿真将把所有输入的事件转发到相应的角色代理对象，在那里它将被 ReceiveEvent 方法处理。此代理对象还提供了一个 RequestAction 方法，此方法将把所有输出的动作请求都转发给服务器端相应的角色来处理。最后，它还有一个 PerformAction 方法用来处理任何只与客户机有关的行为，这些行为将不会被服务器的仿真复制走，例如动态的音轨选择，或是触发了一个 UI 元素（参见图 5.3.9）。

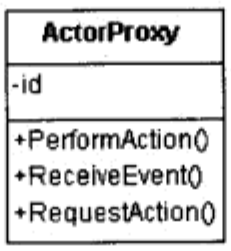


图 5.3.9 Actor Proxy 的类图

5. 非执行者

比执行者更简单的其他核心类并不直接与仿真环境打交道。这些对象并不执行动作，也不会从仿真中接收任何调度处理的时间安排。所有的事件都必须被相应的接收者直接处理。这些对象包括：

- 物件——较小的可以捡起来、可以移动、可以被角色丢弃的游戏对象。
- 障碍物——在游戏中不能移动，不能捡起来的对象。
- 代理对象——客户机端物件和障碍物的对应对象。
- 区域——抽象的空间表现，它用来将仿真空间划分为可管理的各部分。区域将定义本地可能被看见的一个集合，此集合将被用来过滤出什么人可以看到（或是如果需要的话，还有听到）仿真事件，什么人不能看到仿真事件。

5.3.4 管理器与工厂

管理器和工厂都是被作为单实例类来实现的[Gamma94]。当我们需要一个单个的全局对象能被多个不同的类和对象访问的时候，就需要使用单实例类。当仿真层初始化的时候它们将被创建，而且将一直提供服务，直到仿真层关闭。在客户机和服务器上都维护了并行的互相独立的一组管理器。

Download at www.Pin5i.Com

1. SOBFactory

SOBFactory 负责创建仿真对象并确保它们都有一个唯一的 ID。为了达到此目标，SOBFactory 是唯一维护 nextSobId 的对象。此工厂提供了一个 Create 方法，它有一个 SOB 类型的参数指明需要创建 simulationObject（角色、区域、物件等等）的什么派生类。如果客户机端仿真需要创建本地的仿真对象，则它可以维护一个自己的 SOBFactory，这个对象必须保证客户机上的 SOB ID 与服务器上生成的 ID 不会冲突。

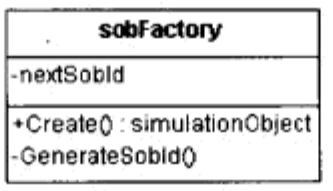


图 5.3.10　SOBFactory 的类图

2. SOBManager

SOBManager 负责维护一个仿真对象的字典。此管理器的主要功能就要从 SOB ID 通过 LookUpById 方法生成相应对象的引用。此管理器还提供了 Store 和 Restore 方法用来监控所有对象的同名方法。这两个方法将调用真正特定仿真对象对应的持续性函数的实现。这样，我们只需要使用统一的同一个调用就可以透明地从仿真层存储或是载入所有其包含的对象了（参见图 5.3.11）。

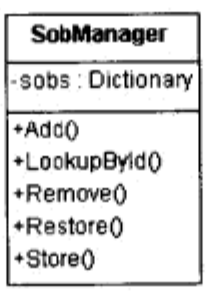


图 5.3.11　SOBManager 的类图

3. ScheduleManager

ScheduleManager 负责的是对所有当前在仿真中的 Performer 仿真对象进行 PerformAction 方法调用的调度（参见图 5.3.12）。另外，它还提供了 ProcessTasks 方法，通过输入的时间分片参数，它将调用在此时间分片上它能处理的所有等待调度的函数，这些函数是使用 Performer 的 schedulePriority 方法进行排序的。虽然可以使用字典进行有效的调度，如此处所展示的那样，但是使用优先级队列的解决方案则在性能上会更好（若想要参看实现优先级队列的经典文章，可以参见[Nelson96]）。

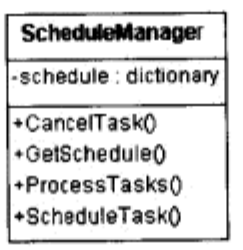


图 5.3.12　ScheduleManager 的类图

Download at www.Pin5i.Com

4. LookupManager

LookupManager 提供了一个快速有效的方法用来在游戏运行时访问静态的游戏数据（参见图 5.3.13）。一般说来，此数据将会保存在关系数据库或是对象数据库中，在仿真开始的时候载入进来。此数据将被映射到一个嵌套的字典中，其中有一个 table 主键和一个 Entry 副键。静态游戏数据的例子包括初始的 ActionState 数据、SimulationEvent 的类型，还有 SimulationObject 的属性。

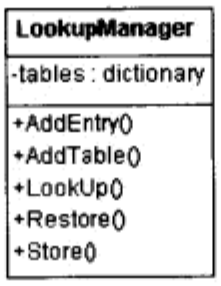


图 5.3.13 LookupManager 的类图

5.3.5 把它们都组合起来

现在我们已经定义完毕了支持类、核心类和管理类，下面就需要将之封装在一个优良的顶层接口里面，以用来管理客户机/服务器的仿真层。BaseSimulation 类提供了这么一个通用的接口，它包含了所有对管理器单实例类对象的引用，以及对根部仿真对象的引用。此对象代表的是整个仿真环境，还提供了一个指针以放置所有的顶层区域对象。仿真维护了两个 SOBManager 的实例，一个是 areaManager，一个是 actorManager。这种仿真对象的分离对于调试、维护和文件归档来说都比较好（参见图 5.3.14）。

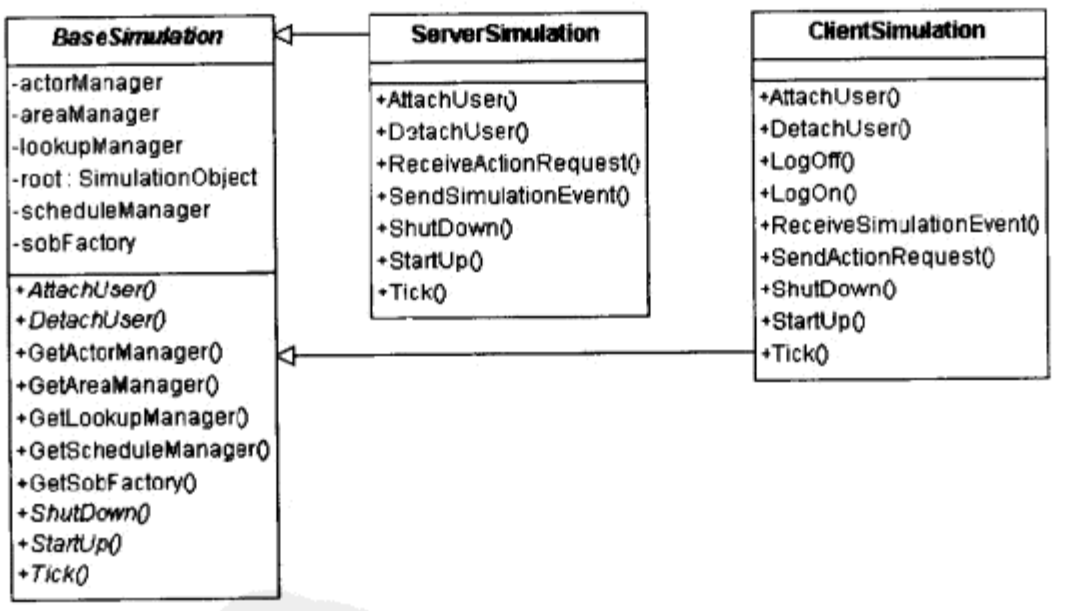


图 5.3.14 Simulation 的类图

1. 将用户与角色关联起来

ServerSimulation 和 ClientSimulation 类分别实现了 `AttachUser` 和 `DeleteUser` 方法，提供了一个机制以将调用的用户映射到一个在服务器上面的特定 Actor 实例还有其对应的在

Download at www.Pin5i.com

客户机上的 ActorProxy 实例上面去。将一个用户与一个角色相关联就可以使得用户能够发出动作请求和接收仿真事件。当前目标角色的的控制状态决定是否此角色可以接受这个关联的请求。这样一个关联机制可以用来支持一些高级特性，例如允许多个用户关联到同一个角色。这个特性在 MMP 中的用处有很多，其中包括允许客户支持人员接手一个制造麻烦的玩家的人物控制，或是当一个新玩家遇到麻烦而请求帮助的时候，可以完全从玩家的角度来看问题。

2. 动作请求

一旦与一个服务器上的用户关联上了以后，客户机的主要输出通信就来自于动作请求了。ClientSimulation 的 `SendActionRequest` 方法将得到一个 `ActionStateId` 和一个代表用户想要执行的动作的参数列表，然后将把它们传到服务器的仿真层去。接下来，服务器会通过一个责任链的形式将这些请求从相应关联的角色转发给它的 currentControlState，在此处它将被处理或是抛弃。图 5.3.15 阐明了这个动作请求的整个序列。

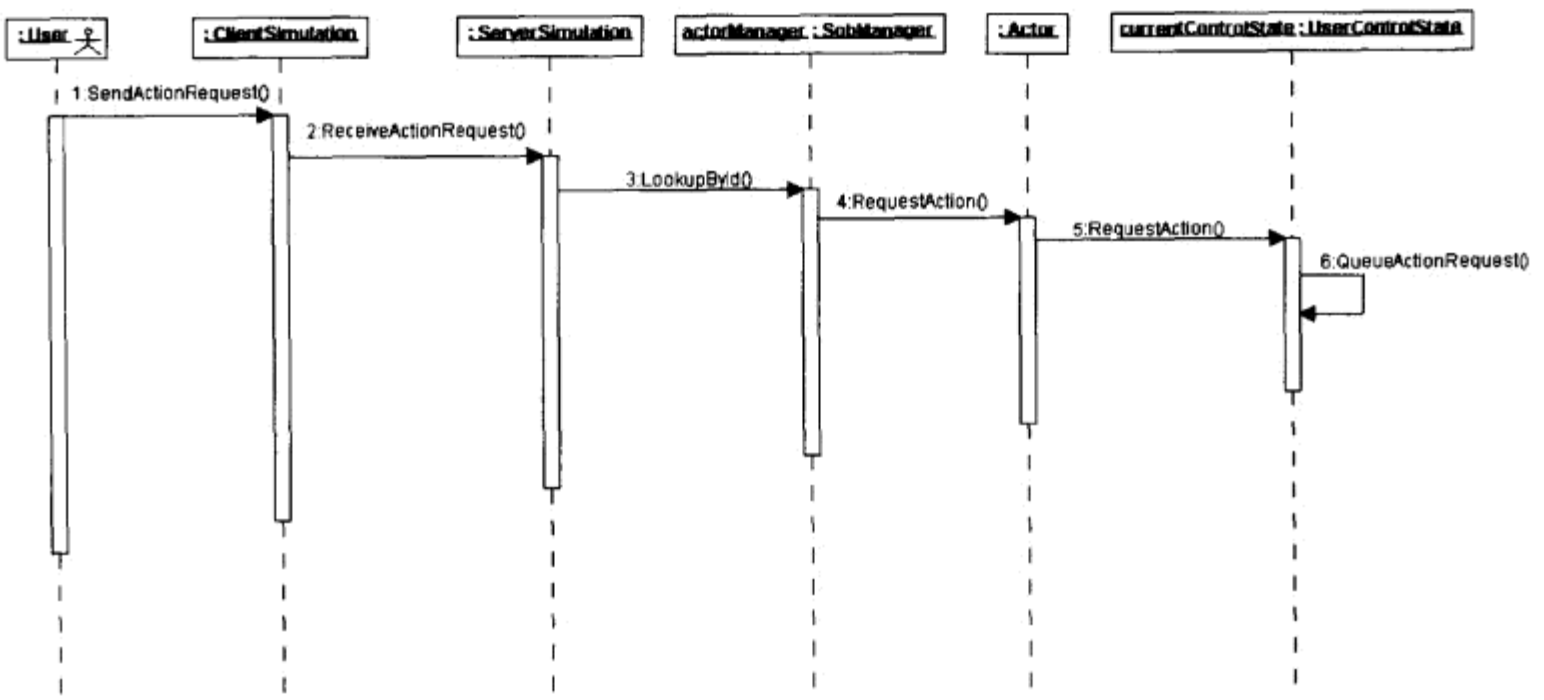


图 5.3.15　动作请求的序列图

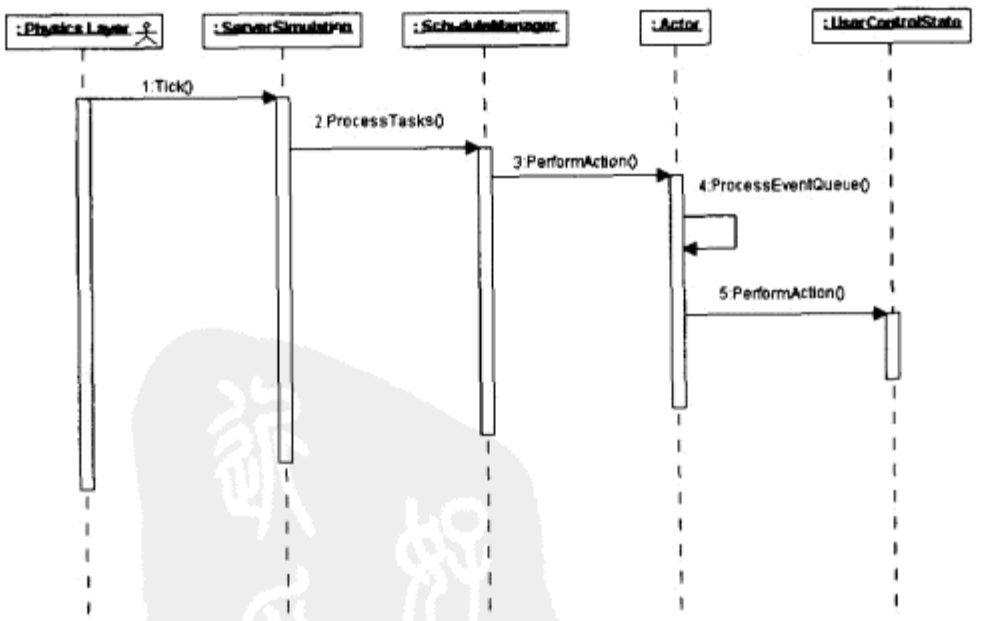


图 5.3.16　动作调度的序列图

Download at www.Pin5i.Com

3. 动作调度

ServerSimulation 提供了一个 Tick 方法，它可以在仿真层的外部被调用以触发对所有等待中动作的处理。此方法负责计算出可用于仿真处理的时间分片，然后会将之通过 ProcessTasks 方法传递给调度管理器。由于服务器的物理层一般要求处理的频率要高于仿真层，因此 Tick 方法可以作为一个很好的回调函数，我们可以将之在作为游戏主循环的物理层里面进行注册以得到调用（参见图 5.3.16）。

4. 对事件的广播和处理进行优化

ControlState 的 PerformAction 方法决定了调用此方法的角色企图转移到的下一个动作状态。如果这个动作状态通过了预测试的 CanTransition 方法，那么我们就将调用其 Transition 方法来做一些很繁重的工作了。每个动作状态都需要提供自己特定的实现。一般说来，这个实现需要同时对其他有关此状态转移的仿真对象给出一个通知。这是通过仿真事件完成的。做出动作的仿真对象维护了一个预订者的字典，其中包含了其他因对其动作感兴趣而注册的对象。这些预订者都会将通过它们的 ReceiveEvent 方法调用得到通知。然后，此接收者可以决定是否需要对此事件给予立即的关注，或是可以将其暂时搁置。在前一种情况下面，一旦事件被接收到了以后就会进行处理；在后一种情况下面，它将被放进队列中，在接收者的下一次 PerformAction 调用里面处理。图 5.3.17 展示了相应的序列图。

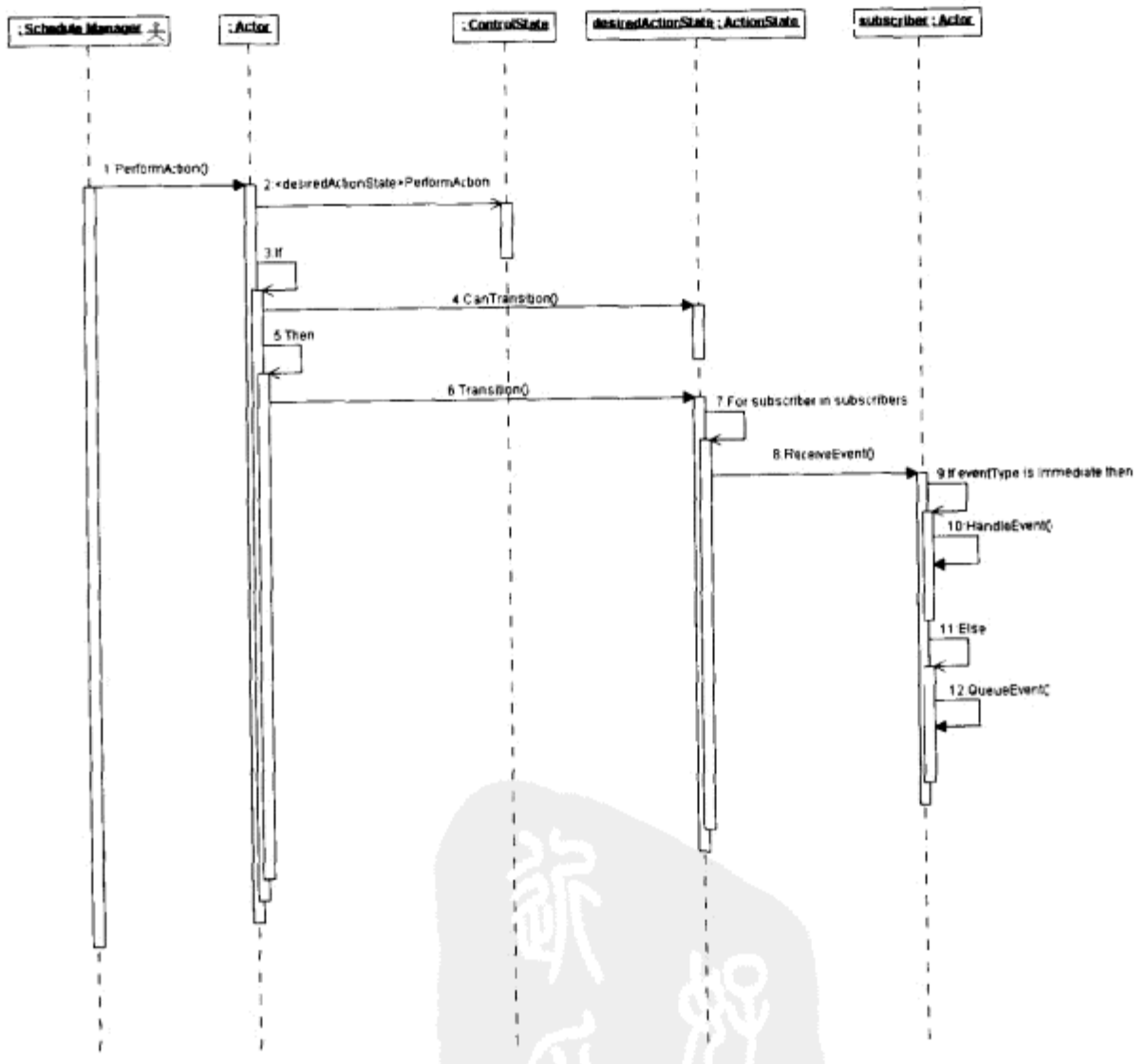


图 5.3.17 事件广播的序列图

Download at www.Pin5i.Com

5.3.6　结论

开发一个 MMP 游戏是一个浩大的工程，而且会带来很多特有的挑战。你可以从一个坚实且符合良好工程规范的基础开始，例如此处描述的这个，然后就可以继续前进，并能够将更多时间花费在创新游戏机制上，而不是到处修改错误和打补丁了。

5.3.7　参考文献

[Alexander02a] Alexander, Thor, "GoCap: Game Observation Capture," *AI Game Programming Wisdom*, Charles River Media, Inc., 2002.

[Alexander02b] Alexander, Thor, "An Optimized Fuzzy Logic Architecture for Decision-Making," *AI Programming Wisdom*, Charles River Media, Inc., 2002.

[Boer00] Boer, James, "Object-Oriented Programming and Design Techniques," *Game Programming Gems*, Charles River Media, Inc., 2000.

[Booch98] Booch, Grady, "The Unified Modeling Language User Guide," Addison Wesley, 1998.

[Dybsand00] Dybsand, Eric, "A Finite-State Machine Class," *Game Programming Gems*, Charles River Media, Inc., 2000.

[Gamma94] Gamma, et al., "Design Patterns," Addison Wesley Longman, Inc., 1994.

[Nelson96] Nelson, Mark, "Priority Queues and the STL," *Dr. Dobb's Journal*, 对应网址为 http://www.dogma.net/markn/articles/pq_stl/priority.htm, January 1996.

[Perlin96] Perlin, K. and A. Goldberg, "Improv: A System for Scripting Interactive Actors in Virtual Worlds," Computer Graphics Proceedings(SIGGRAPH 1996) ACM, 1996.

Download at www.Pin5i.Com

5.4 对多玩家游戏进行扩展

Justin Randall
Sony Online Entertainment
logic@jrlogic.dyndns.org

几年以前，多用户地下城（MUD）游戏中经常可以看到 50 到 100 个玩家同时在其中进行交互。即使是在这些早期的不太复杂的多玩家游戏阶段，玩家们就受到了延迟与服务器资源紧张的困扰。服务器管理员、开发人员和向导们经常要与程序错误、欺诈者、恼怒的玩家，还有在玩家机器上的“机器人”进行搏斗。往日的问题今天仍需面对，但是其规模更大了，麻烦度也更大了。游戏系统及其设计的复杂性使服务器暴露在了更多可能的攻击下。为了为上千个玩家仿真出交互式的 3D 世界，其需要的吞吐量会带来更大的延迟。随着科技的进步，那些足以破坏游戏体验的工具也更加高级了，例如那些解密、自动化、窃听、欺诈、当机或是其他的工具。我们仍然要与同样的敌人搏斗，但是它们却更强悍了，我们需要一些不同的战术来取得胜利。

本节将描述一些用来改善游戏公平度的策略以及在改进服务进程性能的同时对进程进行集群和优化系统以降低所需吞吐量的方法。

5.4.1 改善游戏公平度的策略

决不要信任客户机会向服务器发送回正确的数据。首先要假设每个包都是错误的。玩家可能会发送错误的包以在游戏中移动得更快、得到更大的伤害力，或是为了使服务器当机。他们这么做可能只是想看看是否有可以利用的数据复制的程序错误，或是他们只是想给服务器管理员或者其他的玩家带来麻烦。不管是出于什么原因，他们都会发送一些垃圾——而服务器则要准备好丢弃这些垃圾。

对一个多玩家游戏的攻击可以分为两类：

- 从客户机发送过来的数据——客户机给服务器以虚假的数据告诉它自己现在在做什么，或是对网络上的其他玩家进行攻击。
- 在客户机上的数据——玩家对数据进行偷窥以谋取利益。

想要对得不到的数据加以利用是不可能的。游戏的客户机通常使用的是与维护世界仿真有关的信息。如果客户机上的信息比较多，那就有利于防止对象的突然出现，或是用来防止客户机等待仿真数据到来而产生的延迟。这个可能有关的信息可以被玩家加以利用，这样以得到通常玩游戏的

Download at www.Pin5i.Com

时候无法得到的好处。客户机只会表现出信息的一部分，而玩家则可以利用某些其他的手段（例如，第三方的软件）得到其余的信息。

1. 策略 1：　不要发送此数据

只有当数据已知是必需的时候才向客户机发送。在客户机上的数据越少，相应可加以利用的机会就越小，带宽就越低、性能也就越好。

例如，在打斗中，如果得知敌方的生命值的话，则客户机端的仿真就会比较好，但是如果把所有可能的敌方生命值都发过去的话，则玩家就可以得到它们，在其中选择一个攻击目标，而无需直接与游戏交互了。像 ShowEQ[ShowEq01]这样的程序和在第一人称视角射击类游戏中的代理机器人就是这样的例子，恶意破坏的用户通过它们利用发向客户机的多余信息来加以利用。如果没有数据的话，则此类攻击就无用武之地了。

2. 策略 2：暂时锁住数据

当然了，在有限带宽环境中玩游戏时的现实情况要求事先要把一些数据发送过去，以便改善在客户机上仿真的体验。不过我们还有其他的方法，另外一个策略就是利用一个不会送往客户机的随机生成的密钥对事先缓存的数据进行加密。

当这些数据到达客户机的时候它们是在密文里面的。由于没有用来解密数据的密钥，客户机就只能对加密本身发起攻击了。使用蛮力破解对于一个密码学而言的强力加密，例如 Twofish[Schnier98]，来说需要的时间将远远超过事先缓存的数据在使用之前保存的时间。毕竟，只有当客户机可能在将来不久的时间（可能是下一个 60s 以内）里面使用此事先缓存信息的时候我们才会发过去。

当解密密钥（16 个字节一般已经够长了）发送出去之后，数据就会魔术般地出现在客户机上。此时不再需要发送上万或是上十万字节的数据了，当发送这几个用于解密事先缓存信息的字节的时候，其数据才变得相关起来了。

某些信息，例如如何画一个对象，可以放在明文里面，作为事先缓存的信息要求进行后台载入。而像经验值、生命值、对象的名称以及类似的游戏信息则应该放在密文里面。这样我们就可以在网络发送延迟、磁盘读取延迟、CPU 载入延迟以及公平游戏中取得一个平衡点了。

3. 策略 3：客户机发送命令而不是状态

如果客户机对于位置、生命值、战斗结果，或是不管什么东西没有决定权的话，那么玩家就不可能说“我有 65536 点生命值”，或是“Joe 死亡。”此时，客户机只能允许发送命令，例如“行走”、“跑步”、或是“攻击 Joe”。

4. 策略 4：服务器确认客户机的输出

玩家或许不能发送类似“攻击 Joe”的消息，或是可能发送一个非法的垃圾包。如果服务器收到了一个非法的命令，它应该准备好将此包丢弃，或者通过断线连同客户机一起丢弃。

Download at www.Pin5i.Com

5. 策略 5：绝对不要告诉客户机另一个客户机的网络地址

如果一个玩家知道了另一个玩家的网络地址，那么他就可以使用任意的手段对远端的客户机进行攻击了。例如，玩家正在与 Joe 搏斗，而策略 3 已经实现，这就意味着客户机只能发命令。然而，此玩家可能对 Joe 在搏斗中使用一个拒绝服务的攻击（针对游戏的端口），Joe 被有效地冻结住了，不能发送任何命令，而他正在一个决斗之中，此拒绝服务的攻击导致他的电脑断线，于是恶意破坏的玩家赢得了这场搏斗。

游戏的设计中可能会牵涉到高带宽的网络服务，例如语音通信。在这种情况下，对等用户的通信方式对于吞吐量问题和控制的开销来说都是一个吸引人的解决方案。不过，在其中有一个较低的操作成本和用户体验之间的权衡。可以让玩家自己选择激活对等用户的通信，在激活的时候告诉他们其危险性，而且让他们可以选择与何人通信，这可能是大家都可以接受的一个折中方法。

5.4.2 设计可扩展的服务器

有一点很重要，那就是要在服务器的代码中做好准备以扩展成为高吞吐量的应用程序。数据传输的效率应该很高。在游戏的代码中使用网络系统应该既简单又安全。高效性、简洁性和安全性都是通常应达到的目标，但是在实现巨量多玩家服务器的时候，这些目标却常常被遗忘了。

1. 高效性——使用现代系统接口

每个人都希望代码具有良好的可移植性。只要使用了 select、connect、accept、listen、send 还有 recv，再加上几个#ifdef 的封装以用来处理 Winsock 和 BSD 套接字中间的不兼容性，一个网络子系统只需要几个小时用很少的代码就可以写出来了，而且能具有相当好的可移植性。

对于那些处理几十个连接的低容量网络服务来说，这已经足够了，但是要编写一个处理数百或者上千个当前的对话时，它的扩展性却不太好。大多数现代操作系统都提供了扩展的 API，它们比基本的 BSD 套接字服务更加高效。

2. poll()与 POSIX2 AIO

POSIX2 定义了一个对于大吞吐量应用来说更好的异步 I/O 系统（AIO）。不幸的是，并非所有的平台都能给之以很好的支持。截至 2001 年止，glibc 为 AIO 提供了一些基本的功能，而 Linux 只对使用 lseek()的文件提供了 AIO 的支持。不过，总有一天 AIO 会成为高容量网络服务使用的首选方法的。

但是在今天，游戏仍然不得不在真实世界中经过测试的、稳定的平台上编写代码。取代 POSIX2 AIO 的备选方案是 UN*X 系统上面的 `poll()` 系统调用。`poll()` 与 `select()` 有着密切的关系。文件描述符（特别是轮询（poll）文件描述符）将被传入此调用中，而它则返回有事件等待的描述符的数目。

然而，它并不能告诉应用程序哪些描述符已经准备好了。应用程序必须直接对描述符列

Download at www.Pin5i.Com

表进行循环以找出其中的哪些有事件发生了。

poll()比 select()要更好，这是因为事件将被直接送到描述符列表中。网络系统不需要离开用户空间去查找有等待事件的描述符。在一个成百甚至是上千的套接字组中查找有等待事件的套接字现在只需要几个毫秒，而且不会痛苦地因为要对每一个套接字查询其事件而进行系统调用。

下面的代码就是使用轮询调用的一个例子：

```
int result = poll(fds, count, 0);
int c = 0;
for(i = 0; i < count, c <= result; ++i)
{
    if(fds[I].revents == POLLIN)
    {
        c++;
        // .. receive data
    }
}
```

3. Win32 I/O 完成端口与重叠式 I/O

微软在 Winsock2 中引入了 IOCP（I/O 完成端口）。IOCP 是一个异步 I/O 的 API，它可以高效地将 I/O 事件通知给应用程序。与使用 select()或是其他异步方法不同的是，一个套接字与一个完成端口关联了起来，然后就可继续进行正常的 Winsock 操作了。然而，当一个事件发生的时候，此完成端口就将被操作系统加入一个队列中。然后应用程序可以对核心层进行查询以得到此完成端口。微软指出这是实现高容量网络服务器的最佳方法[MSDN00]。下面的代码大致描述了 IOCP 的使用方法：

```
void foo()
{
    SOCKET s = socket(AF_INET, SOCK_STREAM, 16);
    HANDLE iocp = CreateIoCompletionPort(
         s, g_iocpGroup,
         0, 0);
}

void updateNetwork()
{
    bool success = true;
    while(success)
    {
        int ok = GetQueuedCompletionStatus(
        iocp, // completion port of interest
        &bytesTransferred, // number of bytes sent or
                           // received
        &completionKey,
        &overlapped,
        0 // timeout immediately if there are no
```

Download at www.Pin5i.Com

```
            // completions
            );

            if(ok)
            {
                success = true;
                // handle event
            }
            else
            {
                success = false;
            }
        }
    }
```

4. 安全性——消息序列化

一个利用强类型语言编写的服务器在错误进入运行的应用程序之前，当编译的时候就能找出消息类型匹配的错误了。为网络加入类型安全检查比起保存数据到磁盘上时进行的安全检查其难度并不会更高。事实上，对于基于网络的数据同步应采用基于文件的存储使用的设计与策略，只有少数例外。

当把数据存储到磁盘的时候，程序通常会认为此数据将被在同一主机上写入数据的同一程序给重新读出来。但是当把数据写到网络上去的时候，这就不一定对了。如果只是简单地把包含了一些结构体或是类的原始数据直接发到网络上肯定会出问题，即使两边的主机都使用同样的字节序。本地系统与远程系统可能都是 x86 的系统，但是运行的应用程序则可能在编译期使用了不同字节对齐选项（Dev Studio 缺省是 8 字节对齐，而 gcc 缺省是 4 字节对齐）。客户机可能使用的是低位在前字节序的 x86，而服务器则可能是高位在前字节序的 64 位的 Alpha 机（而且还关闭了对低位在前字节序的模拟）。可能在今天它们还是同样的架构，但是明年说不定服务器就会升级。当与网络打交道的时候，根本就无法保证其字节对齐和字节序会是什么样的。

我们希望对网络消息、文件 I/O 和数据库存储（如果使用了数据库的话）都使用同样的序列化机制。通用的序列化可能可以用来增强对象数据的特性，也可以用来建构整个对象。例如，一个处理数据库的进程可能得到了一个对象位置更新的网络消息，它会将之翻译成 SQL 语句，然后仅仅将对象的位置写入后台的数据库中。这样一来，数据库就可以被当作与其他的服务器一样能接收更新消息了。接口都是一样的，但是实现则各有不同。

加入类型安全性从长远的角度看来是会节省时间的，它提供了一个更加稳定的应用程序，而且在开发过程中使得代码的重用性更好了。带类型的消息系统的最大用途之一就是可以为服务器系统分发数据。

5. 简洁性——消息分发

一般说来，一个协议会使用一个较短的 1 至 4 字节的头部来标识消息的类型。当收到消息的时候，头部将被取出来，接下来会有更多的数据从包中抽取出来，直到整个消息处理完

Download at www.Pin5i.com

毕。这种情况经常是用一个单独的函数处理的，其中会有一个很大的 switch 语句，它将把一个网络缓冲区送到其他的函数进行进一步的处理。

此方法既能完成此任务，也具有较高的效率，而且很容易理解。但是要为它添加新的消息类型也很痛苦。当系统中引入了太多消息的时候，这个 switch 语句可能会长达上万行，而且很难阅读。每当为消息系统添加新消息的时候，这个 switch 的实现都会需要极长的编译时间。更重要的是这个消息分发的代码与每个新消息都是一个紧耦合的关系。新消息的代码无法使用老代码去实现新行为。必须更新老代码去理解新的消息数据。而无用的数据处理流程可能会遗留在代码库中而无人注意。

上述带类型的消息系统可以用来避免其中的某些麻烦。如果应用程序能告诉分发系统“当连接 X 发送消息 Y 的时候，调用我的 Z 成员函数来处理它”，那就会比较好。

使用了这样的分发系统，就不需要预先知道什么类型的对象在请求一个消息，也不需要知道正在处理的消息的类型了。一旦分发系统编好了以后，它就不需要再被改动，而且其代码长度会比一万行要小得多！

它是如何工作的（参见 CD-ROM 上提供的完整实现的细节）：

- 一个连接对象有一个分发对象。
- 在系统中的另一个对象将自己注册到了分发对象中。
- 注册对象提供了一个“connect”方法，它有一个参数对应的是其监听的分发对象的引用——分发对象将把“this”指针传进去以调用“connect”方法，另外还有一个注册对象成员函数的指针，当请求的消息发出的时候，此函数就将被执行。
- 连接对象将把请求者的分发对象与回调函数加入到一个根据接收者的类型排列的容器里面去。
- 当分发系统收到了一个网络消息的时候，它将创建此消息，然后将此消息“发出”，而相应消息已经在编译期就与相关的分发方法关联了起来。
- 请求对象的成员函数被调用了，其参数是此相应的消息。

如果你使用的是 C++，此处就需要使用一些元语言编程的技术，例如模板。它提供了一个类型安全的消息分发，而且当消息类型（对于请求的方法）无法解析的时候会产生编译期的错误。

具体对于网络消息系统的环境来说，一个连接将收到一个网络缓冲区，创建一个特定类型的新消息，然后将之分发：

```
void Client::Client(Connection * connection) :
myCallbackObject()
{
    myCallbackObject.connect(
        connection->getEmitter(),
        this,
        &Client::onDisconnect);
}
```

Download at www.Pin5i.com

```
void Client::onDisconnect(DisconnectMessage & d)
{
   cleanup();
}

void Connection::onReceive(const Archive::Stream & data)
{
   Archive::Stream::ReadIterator r = data.begin();
   Message & m = MessageFactory::create(r);
   emitMessage(m);
}
```

它可以不仅仅用在网络消息上。假设在服务器进程中有一个怪兽对象，当怪兽死去的时候，在所有能看见怪兽的客户机上都应该将之删除。

```
void Client::onCreateMonsterOnRemote(Monster * m)
{
   myCallbackObject.connect(
      m->getEmitter(),
      this,
      &Client::onKilledMonster);
}

void Client::onKilledMonster(KillMessage & k)
{
   RemoveObjectMessage r(k.getMonster()->getId());
   send(r);
}

void Monster::onKilled(Character * destroyer)
{
   KillMessage k(this, destroyer);
   emitMessage(k);
}
```

在开发的后期，设计人员可能会希望人物能知道是否其杀死了一个怪兽。在这种情况下，人物只想知道他在搏斗中杀死的怪兽。可能所有参与了与怪兽搏斗的人物都会得到一些经验值。使用上述的分发系统，原先在老代码中怪兽发出的单独的被杀消息可以用来触发一个新人物对象的新代码了。

```
void Character::onEnterCombat(Monster * m)
{
   myCallbackObject.connect(
      m->getEmitter(),
      this,
      &Client::onKilledMonster);
}

void Character::onKilledMonster(KillMessage & k)
{
```

Download at www.Pin5i.Com

```
        experiencePoints +=
            k.getMonster()->getExperienceValue();
    }
```

如果你在实现分发系统的时候使用的是 C++，就还会有额外的一个好处，那就是作为回调函数的函数指针可以是类的保护类型的成员函数，这样就对外隐藏了接口的细节，但是在需要处理特定消息的时候，它们仍然可以被有选择地暴露出来。没有谁能够直接调用此保护类型的成员函数，除非是那些定义了此函数的对象在实现的代码中把它们显式地暴露了出来。

5.4.3　分布负载

为了确保服务器系统拥有良好的可扩展性，能够处理任意数量的连接，这就常常要求引入对主机系统的集群，以及同时运行多个服务器进程。拥有多个进程的游戏服务将带来技术设计方面的极大挑战。客户机怎样找到一个正确的服务？如何让这些服务合作无间以表现出一个完整的世界仿真？世界在进程开始与结束之间应该在何处保存游戏数据？进程是如何生成的？当客户机需要与多个服务器上的对象进行交互的时候会发生什么事情呢？对每个分布式游戏服务器而言，这只是需要解决的问题中的一小部分。

一旦确立了解决方案，再需要满足游戏的需求的时候，面对的就不是技术问题，而是经济问题了。理想情况下，在服务器端的集群中加入更多的硬件就会得到更大的容量。

1. 使用一个前端进程与客户机打交道

在解决服务器定位（一个客户机如何找到正确的服务）和提供一个完整的世界仿真的问题上，第一步就是可以将与客户机的通信和其他的服务器分离开。当一个客户机连接上服务器的时候，实际上它是在与一个前端进程（FEP）打交道。可以使用几个前端进程来合作，以增加一个服务器集群可以处理的连接数量。

2. 客户机如何找到正确的服务?

常用的设计使用的是一个公开的登录服务，此服务将对客户机进行认证，然后给它几个可能需要与之交互的服务器。每个服务器都是一个前端进程，它们将客户机与后台的游戏进程给分离开了。一旦客户机连接上了一个 FEP，此进程就将与其他的后台进程联系以将一个玩家的人物加入到世界仿真中，这样它就可以开始玩游戏了（参见图 5.4.1）。世界如何表现给此客户机以及哪一个后台进程应该与此客户机交互，都是游戏设计的问题。如果在一个角色扮演游戏中，每个城市都是一个进程代表的，那么玩家上次所在的城市就是新客户机所应该连接的进程了。

后台可能还有一些其他的定位服务。哪些对象在哪些服务器上？哪些玩家在哪些服务器上？此关系是客户机看不到的，它只与 FEP 打交道。一个客户机完全不需要直接连接这样的定位服务或是任何一个后台进程。后台定位器需要和每个 FEP 建立一个单独的连接，这样通过它就可以成功地将客户机的通信转发到后台的某个世界仿真服务上了。

Download at www.Pin5i.Com

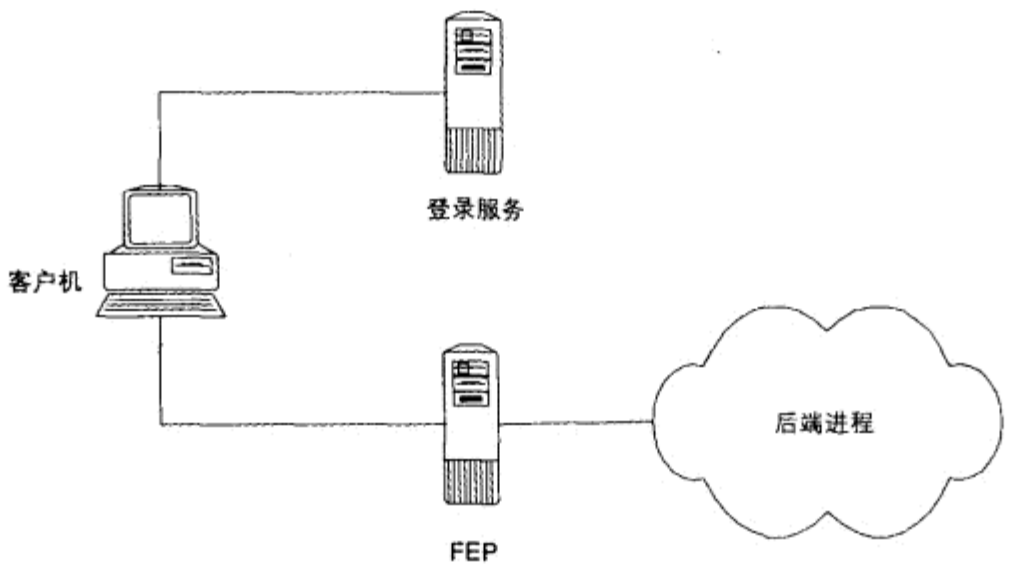


图 5.4.1 使用登录服务与一个前端进程（FEP）的连接图

3. 如何让服务合作无间以表现出一个完整的世界仿真?

FEP 方便了一个完整的世界仿真的表现。一个完整的仿真指的就是在游戏世界中，仿真表现得好像就是一个服务器在一个连接上提供了一个连续的仿真。如果一个客户机需要从一个后台进程转移到另一个后台进程，对于客户机来说并没有重新连接的问题。这整个过程都是在 FEP 和游戏服务器进程之间完成的。

FEP 引进了一个单独的进程用来过滤数据包。一个 FEP 可能用来处理客户机连接上的压缩和加密，得到原始的、未压缩的明文再传到后台进程去以为其他的活动（例如 AI 或是物理层）节省宝贵的 CPU 资源。由于 FEP 是用户数据进出的唯一关口，数据包的有效性检验也可以在 FEP 上进行，而不需要进行协议的升级（例如，如果一个客户机发送了一个错误的包以使游戏服务器当机）。

前端进程同时保证了客户机不会受后台游戏服务崩溃的影响。如果系统还实现一个恢复机制，那么客户机就完全不需要知道服务器曾经狠狠地崩溃过一次了。当后台启动一个新的游戏服务，将客户机转接到新进程上的时候，FEP 和客户机之间的连接仍然可以继续保持着。同时，开发人员还可以查看内核转储（core）文件，而玩家仍然能继续在服务器上玩游戏。

当然了，每种设计都有其缺陷。在数据从网络上读入继而被分发到相应后台的连接上，然后交给网线，最后通过后台进程读入此数据的过程中，使用 FEP 会引入额外的延迟。另外还有一个进程来管理服务器的维护和诊断。

将服务器进程分布化并不必须引入 FEP。对某些涉及到自治游戏服务器（意思就是此处不需要提供一个完整的世界仿真），需要近乎实时延迟的游戏设计来说，它们可能并不需要使用前端进程。

4. 进程间交互

当玩家跨进程进行交互的时候，就会出现一个技术上很大的可被利用的弱点。一个与另一个进程中的 NPC 或是玩家通过物件交换金子的玩家就是一个很好的利用复制谋取利益的例子。假设 Joe（在服务器 A 上）正在接手一个 Jane（在服务器 B 上）的全金属盔甲，如果

Download at www.Pin5i.com

Joe（或是其他的某人）能够使服务器 B 崩溃，那么他就有可能复制出一个全金属盔甲，即服务器 A 已经提交了这个交易，而同时服务器 B 还没有提交此交易的时候就崩溃，那么这种情况就发生了。Jane 仍然保有此全金属盔甲，而现在 Joe 也有了。现在世界中就有了两个全金属盔甲，而在此之前是只有一个的。下一次他们就可以一人得到 2 个盔甲，接下来就是 4 个、8 个、16 个，等等，直到他们有了无数的盔甲为止。

通过使用 UML 顺序图[Larman01]，我们就可以看出这个进程间交互的弱点。当创建流图的时候，可以假设在此进程间交互的某时间点上一个或是两个服务器都崩溃了。当这种情况发生的时候，看看游戏的状态会是什么样的。

当创建顺序图的时候，要考虑到与超负荷的服务器交互的时候会发生什么问题，而这些问题又会如何影响游戏。如果将一个玩家锁定在某个交易场所中，而此交易却在等待一个决不会返回的服务器，这样就很不公平了。在进程之间发送的消息可能不会得到立即的处理。假设一个或是更多的进程都处在 CPU 利用率 100%的情况下，而且它们有可能永远没有时间来处理一个交互。如果使用异步模式来设计一个系统，那么用户就会觉得它的性能更好了。这样一来，产生不公平现象的几率就小得多，而且它还有利于创建一个负载均衡的系统，以在一个服务器集群中得到更平均的负荷。

5.4.4　优化

在处理一个场景中什么是可见部分的时候，三维处理的客户机将对送往一个渲染流水线上多边形的数量加以限制。这么做的目的是为了节省画出一个场景所需要进行的处理负荷。大型分布式服务器的负荷非常大，它需要处理的对象数目比一个客户机需要处理的多边形的数目还要多。在一百万个对象之间进行 N×N 阶的操作，当分布在 20 个或者更多进程中的时候，其数量如此之大，以至于也需要对其在服务器上进行对象的裁减了！

在一个客户机上，裁减一般是在一个场景中的一个镜头里面发生的。在一个服务器上，问题的规模则大得不成比例了。每个对象都对世界有自己的一个视角！在一个游戏客户机上工作良好的策略对于复杂的服务器而言就不那么适合了——服务器可不能把大部分的 CPU 时间都交给排序和裁减算法，而只留一小部分给 AI、消息处理、物理层、数据存储以及其他很多需要完成的任务。

我们希望，甚至认为是必须，要对每个对象的交互加以限制，将其交互的对象限制在一个世界对象中很小的集合里面。如果系统能快速地对对象进行排序与更新，而不管一个对象涉及的视角，而且对 CPU 的使用在 10%之下，那么这种情况就比较理想了。

这样的系统是存在的，而究竟那一种最适合一个特定的服务器则要视仿真而定。本节将主要关注一个进行子系统划分的系统，它是在《游戏编程精粹 2》中 John Ratcliff 的文章——球状树（Sphere Tree）[Ratcliff]里面提到的。对于可以使用一个基于距离的裁减系统的对象来说，它也提供了一个视角策略。

球状树

对于那些拥有很大的区域，在整个世界空间中分布着一群群人口的大型世界来说，基于距离的裁减是最适合的。交互仅在相互距离较近的对象中发生，对于那些距离很远的对象来

Download at www.Pin5i.Com

说，它们之间就没有交互了。而球状树正是一个理想的基于距离的空间排序算法。

利用球状树，你可以进行快速的距离查询，能很快地返回在任意球状区域中的一组对象。它们还能对运动中的对象进行极快的重新排序。我们可以利用这些特性来维护一个独立于视角的排序以及子划分系统，这样就能在世界仿真中限制住对象之间的交互了。

球状树是利用层次化的球体来对对象进行组织的。其中有一个根节点，它包含了所有的子球体（参见图 5.4.2）。子节点继而又可以被分为更小的球体直到树中充满了合适大小的球体。当一个对象在一个球体中移动的时候，它将检查其父节点的边界。如果超出了边界，父节点就会改变尺寸以适应其新位置，或是当达到了某个最大值的时候，此对象就将进入其父节点的父节点的球体里面去，同时一个新的子球体又将被创建出来了。

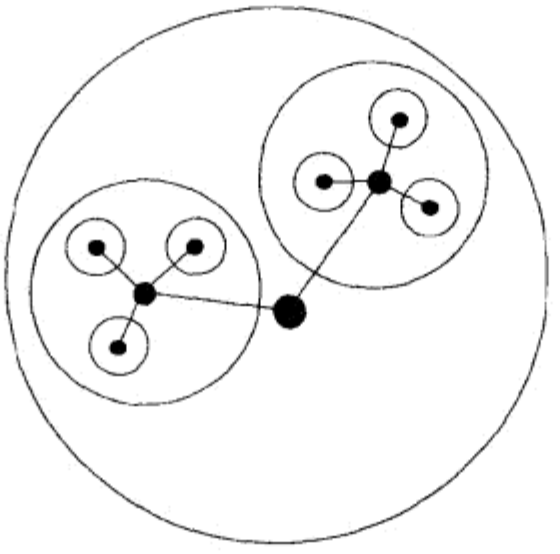


图 5.4.2 一个 3 层球状树，它描述了 1 个根节点，2 个超球，6 个子球

球体对球体的检验是一个很快的过程，它只需要检查矢量距离。如果直接检查距离的平方的话，此过程还将得到优化，因为此时就避免了缓慢的 sqrt() 函数的调用了。由于在一个球体树里面的对象是针对其边界球体进行了排序的，因此当对此树进行基于范围的查询的时候，只需要对树中最顶层节点进行检验。那些与此范围相交的球体将进一步被查询。其子球体将被此查询访问。这个过程将在此树中递归进行，直到最后找到了满足此范围查询的对象为止。

查询的性能要视要求范围的大小而定。查询范围越小需要进行递归查询的顶层节点就越少。对于基于距离的裁减来说，这是球体树带来的第二个好处。

每个对象都有一个边界球体，这样它就可以在树中进行正常的排序了。此外，它还有一个感兴趣球体，在此中，基于某个距离，它包含了自己对世界的视角。当对象移动的时候，这个感兴趣球体也会随之移动。此球体是用来对球体树进行查询的。当查询中返回了一个新的需要交互的对象时，相对于原对象的移动来说，它就处在了其视野范围之内，这个新对象就会被放入一个可能会交互的对象集合中去了。

当服务器检查对象之间的交互时，它只需要检查相应的可能会交互的对象集合。相对于世界中对象的总数量来说，这个集合就小得多了（具体要视兴趣半径而定）。

当新对象加入的时候，服务器可能还需要进行检查看其是否是一个在世界中漫游的玩家人物。如果是的话，那么当此对象被添加到可能会交互的对象集合之中的时候，此新对象的

Download at www.Pin5i.Com

基本信息将会被送到客户机那里去。如果对象移动了或是改变了，客户机就可以收到关于此对象的更新信息。对于那些可能会交互的对象来说，服务器只需要向客户机发送更新信息就可以了。类似地，当对象从可能会交互的对象集合中删除的时候，它们可能会从客户机上给卸载掉。

你可能希望给客户机的人物对象上加上一个兴趣半径，这样就使得客户机能对此交互集合进行查询了。不过，如果给其他的对象加上交互集合则会更有效。一个建筑物可能会有一个较大的交互范围（在游戏世界空间中它可能会有一个几百米的可见范围），而一个建筑物旁边的灌木丛则可能只有几米的可视范围。

对被观察者的交互集合进行排序，而不要对观察者的交互集合进行排序，这样更新信息就可以被那些被观察的对象主动推给观察者了。这可以防止过度的轮询，只当有事件发生的时候才会占用 CPU 的时钟周期。

在大部分情况下，对象是处在静止状态的，因此它们不会对球体树进行查询以更新其交互集。在运动中的对象才需要与静态对象的交互集做相交操作。当人物走向城市的时候，建筑物将希望得知人物何时会走近到看得见自己的距离以内。

交互球体也有可能是球体树中的边界球体。在球体树中进行碰撞检测是很容易的，当产生了一个碰撞的时候，目标可能会收到通知，将源对象（移动对象）加入到自己的交互集中间来。通过这个方法，静态对象例如建筑物或是闲置的非玩家人物就可以加入到一个客户机的世界视图中了，而且当它们的状态发生变化的时候客户机还将得到相应的通知。

在服务器与服务器的裁减中也可以使用同样的方法。降低服务器之间的通信量，其重要性至少是与降低服务器与客户机之间的重要性是一样的。但需要交互的时候，服务器将需要对远端的对象加以处理。如果要求所有的服务器对世界里所有的对象加以处理，则服务器集群的性能立刻就会急剧下降了。

对于空间排序和交互的裁减来说，球体树并非唯一的解决方案，但是它们对于那些注重距离的世界来说却是一个较好的解决方法。四叉树和八叉树对于大型世界来说都是很好的解决方案，但是它们也有自己的问题。如果某些世界中含有互相连接的空间，那么对它们而言，可能最好是要选择一个基于门户的系统以对交互进行裁减。门户、四叉树、八叉树、球体树以及许多其他的算法都处在开发人员的选择之下。困难不在于如何实现一个正确的方案，而在于找到一个正确的方案来实现。

5.4.5　结论

本节描述了几种对于设计一个可扩展的多玩家服务器有用的技术。一定要确保你充分地使用了服务器平台上的网络 API，即使这将导致不可移植的问题，只有这样服务器才能将可处理的连接数目提到最大值。架构上的特性，例如前端进程以及针对裁减的球体树，则提供了高性能的服务器，而且将后台服务器与客户机器的直接连接给分离开了。在开发的早期投入更多的时间去建构简单、安全、可重用的 API，例如消息分发和消息工厂，在项目接近尾声的时候就可以确保游戏本身的开发会更快了。

Download at www.Pin5i.Com

5.4.6 参考文献

[Larman01] Larman, *Craig, Applying UML and Patterns: An Introduction to Object-Oritented Analysis and Design and the Unified Process*, Prentice Hall, 2001.

[MSDN00] Microsoft Corporation, "Windows Sockets 2.0: Write Scalable Winsock Apps Using Completion Ports," 对应网址为 http://msdn.microsoft.com/msdnmag/issues/1000/Winsock/Winsock.asp, MSDN, October 2000.

[Ratcliff] Ratcliff, John W., "Sphere Tree for Fast Visibility Culling, Ray Tracing, and Range Searching," *Game Programming Gems 2*, Charles River Media, Inc., 2001.

[ShowEq01] HackersQuest, "HackersQuest" 的对应网址为 http://www.hackersquest.org/, March 2002.

[Schnier98] Schnier, Bruce Twofish, "A 128-bit Block Cipher," 对应网址为 http://www.counterpane.com/twofish-paper.html, Counterpane Labs, 1998.

Download at www.Pin5i.Com

5.5　基于模板的对象序列化

Jason Beardsley
NCSoft Austion
jbeardsley@ncaustion.com

对于网络游戏来说，不管是客户机/服务器还是对等用户模式，都需要一种有效的方法将对象转化为网络形式，再从之转化为对象，这种网络形式一般来说就是一个原始字节流。此过程被称为序列化。

即使是对于不使用网络的游戏来说，序列化也可以用来与文件系统打交道（例如，可以从磁盘载入资源或是保存游戏的状态）。然而，基于文件的模式经常会用更大的空间换取一些在网络环境中无法实行的能力，例如随机位置的访问、本地修改、一个层次化的目录结构以及版本控制。这样，它们对于网络通信来说就不太适宜了。

本节阐述了一个专门为网络而用的序列化方法。在处理所有类型对象的时候，它都具有较高的灵活性和一致性，例如处理内置的数据类型、STL 容器类以及用户自定义的类时。

5.5.1　现存的解决方案

序列化本身并不是一个新鲜的话题。其中一个经常使用的方法就是定义一个结构体，里面包含了要进行序列化的数据，对之进行填充，然后简单地利用 memcpy ( ) 将其转化为适合网络传输的一种数据格式。对于很多应用来说，这种方法既快捷也足以满足需求，但是它有一些内在的缺陷：

- 只有当位一级的数据表现形式和网络一样的时候它才能正常工作。对于 C++的类来说这并不总是对的——特别是对于那些使用了动态存储或是内置指针的类，或是任何一个有虚方法的类而言。
- 当生成结构体的时候一定要特别仔细，以减少编译器带来的对齐问题。在一个跨平台的环境里面必须保证所有的编译器都有同样的字节对齐选项。
- 在拷贝之前必须显式地进行字节序的转化。
- 对于变长数据的处理，例如字符串或是数组，即使在最好的情况下也只能是很笨拙的。

总的来说，这个“结构体/memcpy ( )”的序列化方法只有当处理包含普通数据类型的定长结构时才能使用。另一个标准的方法涉及到使用一个对象来对每个类型进行存储和读取的处理。这个类可能会和下面的结构差不多：

Download at www.Pin5i.Com

```
class Serializer
{
public:
    // plain ol' data
    bool Put(int val);
    bool Get(int& valOut);
    bool Put(float val);
    bool Get(float& valOut);
    // standard classes
    bool Put(const std::string& val);
    bool Get(std::string& valOut);
    // STL containers
    template <typename T>
        bool Put(const std::set<T>& val);
    template <typename T> bool
        Get(std::set<T>& valOut);
    // et cetera
};
```

请注意 Put()和 Get()都被重载了，而没有使用不同的名字对不同的类型进行处理（例如，PutInt()或是 GetString()）。对于 STL 的支持来说，这是必要的一部分。如果方法没有被重载，那么用于处理一个 STL 集合的代码就无法泛型化了（这就是说，无法调用特定于各自元素的 Put()和 Get()方法了）。

当处理用户自定义的类的时候，我们则会处在一个两难的境地之中——应该对 Serializer 进行扩展以直接支持每个类呢，还是应该让每个类都提供自己的序列化方法？对于第一个问题的回答是一个响亮的“不是”，原因如下：首先，Serializer 必须要能访问到任何重要类的内部数据（这需要使用友元）；其次，每当增加了一个新的类型支持以后，就需要进行不必要的大量的重编译工作；此外，Serializer 的接口很容易就会变得无法控制了。简而言之，这将带来维护上的灾难，更不用说此方法本身的笨拙了。

因此，序列化的支持应该直接成为每个类的责任。一般说来，这需要定义一个具有纯虚函数的抽象接口类以支持序列化：

```
class Serializable
{
public:
    virtual bool PutInto(Serializer& s) const = 0;
    virtual bool GetFrom(Serializer& s) = 0;
}
```

接下来，需要对 Serializer 进行扩展以支持序列化的对象：

```
bool Serializer::Put(const Serializable& obj)
   { return obj.PutInto(*this); }

bool Serializer::Get(Serializable& obj)
   { return obj.GetFrom(*this); }
```

这种方法并没有什么错误，但是它有一些缺陷。它要求对类进行修改以支持序列化，如

Download at www.Pin5i.Com

果可能的话，这确实是挺好的。然而，如果这个类是由第三方的库提供的，而且无法改变的话那又如何呢？另外还有性能上的问题，它将导致一个本来是很简单的、独立的类成为一个继承层次结构中的一员。对一个游戏来说（此处速度占有极重要的地位），对一个简单的类引入虚方法可能会付出无法接受的代价。

除了要求要用户类从 Serializable 继承，而且引入一些较小的性能问题以外（例如，可能声明 Serilizable 的头文件会包含所有的 STL 容器类的头文件，导致编译时间的加长），这个方法还能满足一个一致的、可扩展的序列化系统的要求。问题在于，能不能将之进一步精炼以消除（或是削弱）这最后几个小问题呢？答案是“是”，但是在我们详细阐述最后的方法之前，还需要先讲讲背景知识。

5.5.2　可移植性

网络游戏，尤其是那些客户机/服务器模式的游戏，通常都是跨平台应用的。客户机可能会运行在 Windows、Macintosh 或是 PlayStation2 上面，而服务器则可能是 Linux、Solaris 和 Windows 的混合体，运行在不同的处理器架构上。因此，任何一个序列化库都应该具有高度可移植性，这就意味着需要在每个目标平台上对之进行移植（以及测试）。

在编写跨平台代码的时候，得到的第一个教训就是 C++语言并没有强制所有的内置类型都有同样的大小，例如 int 和 float。对于序列化来说，这会造成一些问题，因为我们必须得知一个对象的大小，否则一个值可能会被写成 32 位但却读出 64 位，从而导致不可预测的结果。最新的 C 语言标准使用了<inttypes.h>头文件来解决这个问题，但是现在并非所有的编译器提供商都实现了此要求。那么退而求其次，我们可以定义（通过 typedef）一组固定大小的原始类型（int8、int16、int32、int64、uint8、…、uint64、float32、float64），然后在序列化的时候只使用这些类型。如果一个类型可能会被隐式地转化到其中好几种支持类型之一，处理时要特别小心。在存储或是读取的时候，枚举类型和无类型的常量应该被显式地转化到相应的数字类型。

下一个跨平台代码的问题就是字节序问题。不同的芯片架构在内存里面保存多字节值（包括整数和浮点数）的时候可能使用“高位字节在前”也可能使用“低位字节在前”的顺序[Kernighan99]。这么一来，序列化系统就必须将数据在一种标准字节序间进行转化以保证底层的数据格式是真正可移植的。具体要选择那一种字节序（高位字节在前或是低位字节在前）并没有很大关系，只要保持一致就可以了。一个比较好的原则就是统一成服务器端的原始字节序，因为一般都认为客户机上的 CPU 会更加空闲。不管如何，字节倒序的问题应该不会给用户带来可见的影响。因为我们使用的都是自己的原始类型，因此也必须提供一套自己的转化函数，而不能依赖于标准的 `htonl()` 及其相关函数了。

谈到原始类型，浮点数可能会给可移植性带来极大的障碍，这主要取决于目标平台。幸运的是，几乎所有当今生产的处理器都使用 IEEE 标准格式，因此对浮点数在位一级的拷贝事实上是可移植的。一定要记住浮点数也有字节序的问题，在这一点上它和整数是一致的，因此可能需要交换其字节序。编写一个程序检查某个特定的架构是否和 IEEE 兼容不是件难事，不过对于一个不兼容平台应该如何处理的问题就将作为练习给那些不那么幸运的读者去课下完成了。

Download at www.Pin5i.com

5.5.3 Serializer 类

序列化系统有如下的几个目标：

- 对所有类型的数据提供一个一致的接口进行数据存储和读取。
- 支持原始类型、STL 容器类以及用户自定义类，对于后者来说要求相应的改动最小化。
- 尽可能地高效，同时要保持可移植性。

从本节的标题得知，要满足这些要求，其关键就在于模板的使用——特别是成员函数模板的使用。

假设我们在 Serializer 内部定义了如下的 Put()和 Get()接口：

```
class Serializer
{
public:
    template <typename T> bool Put(const T& obj);
    template <typename T> bool Get(T& obj);
};
```

接下来，所需做的事情就是要支持任意 T 类型了，这只要为每个函数定义其偏特化版本就可以了。例如，对于标准 C++的字符串类来说：

```
template <>
bool Serializer::Put(const std::string& obj)
   { /* implementation */ }
template <>
bool Serializer::Get(std::string& obj)
   { /* implementation */ }
```

遗憾的是，正是在这一点上我们得到了跨平台开发中的第二个教训——并非所有的编译器都生来一模一样。虽然上述的代码是标准 C++，但是并非所有的编译器都支持它。例如，微软的 Visual C++6.0 就不支持在类声明以外出现的成员模板函数。对于网络开发人员来说，他们需要绕开编译器的不同之处，这已经成了标准的操作过程了。此问题与其他很多开发问题一样，其解决方法就是要引进新的一层封装。

如果有相应的辅助函数可以处理对任何类型 T 存储(PutInto())与读取(GetFrom())在位一级的底层细节，那么我们就可以将之放在 SerializerHelper 的名字空间里面去，这样以避免可能出现的命名冲突，接下来就可以很容易地使用它们来实现 Put()和 Get()了，如下所示：

```
template <typename T> bool Put(const T& obj)
{
   return SerializeHelper::PutInto(*this, obj);
}

template <typename T> bool Get(T& obj)
{
```

Download at www.Pin5i.Com

```
    return SerializeHelper::GetFrom(*this, obj);
}
```

下面的任务就是要对所有需要的数据类型都定义这么一个 PutInto()与 GetFrom()函数，它们包括整数、浮点数、STL 容器类、等等等等。由于名字空间可以被任意扩展，也可以放在不同的编译单元里，因此我们就可以很容易地为用户自定义类型提供有效的支持了。

1. 输入输出存储

Serializer 使用了一个字节缓冲区的辅助类来处理那些麻烦的对内存读取和写入的细节，包括可以在写入的时候自动扩大自己的尺寸。下面是此类的部分接口：

```
class Buffer
{
public:
    bool Write(const void* data, int len);
    bool Read(void* dataOut, int len);
    const void* GetData() const;
    int GetLength() const;
};
```

2. 处理简单的定长类型

整数、浮点数以及布尔值都可以使用两种方法来处理——使用 SerializerHelper 名字空间里面的函数或是直接使用 Serializer 本身（就是对 Put()和 Get()进行重载）。由于这些类型是最基本的，因此我们将使用后一种方法，因为它会使编译的时间短一点。

处理每种类型的代码都差不多。将其值转化为网络字节序（如果需要的话），然后进行按位拷贝。对于布尔类型来说，首先将其转化为 uint8（true=1，false=0），因为底层对其使用的表现形式可能是一个大整数。

3. 处理简单的变长类型

在简单的变长类型中有两个类型· C 风格的字符串和无类型的字节数组。由于字符串是空字符结尾的，因此我们可以将其与结尾的空字符一起存起来，而不需要 Pascal 风格的长度域。尽管如此，将其长度显式地存储进去（同时忽略结尾符）总的来说会更加安全，即使它的空间效率不是很高。string 对象和 char*都可以被传进 Put()方法里面去。不过，我们没有提供 Get(char*)的函数，因为其输出的长度是不定的。

宽字符的字符串存在一个移植性的问题，因为 wchar_t 的大小是没有被语言严格定义的。即使是在同一个平台上，其准确大小也可能不一样，这要视编译器的选项而定。因此，我们没有提供内置的宽字符支持。不过你可以使用一个临时方法，那就是定义一个应用程序特定的宽字符串类（可能可以基于 basic_string 模板实现），然后为将其加入序列化的支持。

原始字节数组也带来了一些问题。Put()接口只有一个参数，但是它却必须要知道到底有多少项要写入（也要知道最大有多少项要读出）。幸运的是，我们引入了一个和字节数组表

Download at www.Pin5i.Com

现得一样的类（Buffer），而它的序列化是很简单的。然而，在某些时候仅使用 Buffer 是不可能的，或者会涉及到一些拷贝的工作。我们提供了额外的两个函数 PutRaw()和 GetRaw()，来提供对序列化流的低层次的访问。这些方法需要一个长度作为参数，它分别表明了要写入或是要读出的字节数。

4. 处理 STL 的容器类

保存一个 STL 容器类内容的过程是很直截了当的。首先，写下元素的个数，然后从容器的 begin()走到 end()，把每一个元素都写入进去（通过调用 Put()）。对于所有的容器类来说其实现代码都是一样的，自然我们也就只需要写一次就知道了。

熟悉 STL 的开发人员都知道要对 STL 容器类（这是 STL 的精髓部分）的元素进行访问，其关键就是要高效地使用迭代器[Meyers01]。STL 附带的那么多算法（例如 find_if()）都几乎只用到了迭代器。序列化也不例外，而 Put()和 Get()虽然表面上与容器类打交道，实际上使用的都是一对操纵迭代器的函数——PutRange()与 GetRange()。这两个函数处在 SerializerHelper 的名字空间里面，而且只依赖于 Serializer 的公共接口。

下面就是 PutRange()的实现。请注意它可以利用 distance()算法来得到元素的个数，但是一般说来调用者使用 size()来计算此数会更加高效，然将之作为一个参数传进。

```
template <typename Iterator>
bool SerializeHelper::PutRange(Serializer& s,
                               const Iterator& begin,
                               const Iterator& end,
                               uint32 size)
{
    if (!s.Put(size)) return false; // store size
    for (Iterator it = begin; it != end; ++it)
        if (!s.Put(*it)) return false; // store item
    return true; // success
}
```

得到一个容器的元素会稍微困难一点。整个过程中会有两个小障碍——首先，我们必须知道其包含的对象的类型（例如．对于 set<T>来说，我们必须知道 T 是什么）。其次，一旦一个元素被恢复了以后，它必须被再插入到容器里面去。如此一来，我们就要首先读出元素的个数，然后把每个元素一个个的读出来，同时再把它们加入到容器里面去。幸运的是，STL 的设计者已经在 iterator_traits 类中提供了我们需要的功能，以及插入迭代器的概念[Josuttis99]、[Austern98]。

```
template <typename Inserter>
bool SerializeHelper::GetRange(Serializer& s,
                               Inserter it)
{
    uint32 size;
    if (!s.Get(size)) return false;
    for (uint32 i = 0; i < size; ++i)
```

Download at www.Pin5i.Com

```
    {
        iterator_traits<Inserter>::value_type obj;
        if (!s.Get(obj)) return false;
        *it = obj; // inserts into container
    }
    return true; // success
}
```

此处，我们再一次遇到了编译器的差异（或者更准确一点来说，是 STL 的差异），它又使得相应的代码无法移植了。此处的问题在于 iterator_traits，它不是在所有平台上都能完全工作的。要绕过这个问题，我们需要一些黑客式的技巧。我们需要的是一个能得到容器内对象的类型的方法。调用 GetRange()的代码显然是知道的，这样它就可以传进一个对应类型的空指针，这个参数就是我们需要的另一个模板参数。其结果（不同之处用黑体标出来了）如下：

```
template <typename T, typename Inserter>
bool SerializeHelper::GetRange(Serializer& s,
                               Inserter it,
                               T* /* unref */)
{
    uint32 size;
    if (!s.Get(size)) return false;
    for (uint32 i = 0; i < size; ++i)
    {
        T obj;
        if (!s.Get(obj)) return false;
        *it = obj; // inserts into container
    }
    return true; // success
}
```

通过 PutRange()和 GetRange()，我们就可以直接开始编写针对 STL 容器类的辅助函数了。对于 list 而言，其代码如下所示：

```
template <typename T>
bool SerializeHelper::PutInto(Serializer& s,
                              const list<T>& l)
{
   return PutRange(s, l.begin(), l.end(), l.size());
}

template <typename T>
bool SerializeHelper::GetFrom(Serializer& s,
                              list<T>& l)
{
   l.clear(); // destroy any existing contents
   return GetRange(s, back_inserter(l), (T*)0);
}
```

Download at www.Pin5i.Com

其他 STL 容器类的处理都是类似的，其不同之处只有模板参数的个数（例如，map 需要两个类型——键值和数值）以及在调用 GetRange()时插入迭代器的创建。顺序类型的容器使用一个 back_inserter 适配器（它调用 push_back()），而其他的则使用一个普通的 inserter（它调用 insert()）。

为了提供对关联容器（map、hash_map 等）的完全支持，我们还必须为辅助模板类 pair 定义序列化函数。请注意对于任何可序列化类（包括嵌套容器）的 STL 容器来说，只要其类有一个缺省建构函数，则我们就可以提供序列化的支持。

5. 处理用户自定义类

本系统最后一个要处理的就是用户自定义类（这是指你任何一个想要序列化的类）。如果我们可以通过一个类（或是结构体）的公共接口就能读出和存储其完整的状态，而且它没有继承关系的话，那么相应的处理就很简单了：定义出相应的 PutInto()和 GetFrom()就行了。根本不需要对类进行任何改动。

当一个类拥有私有数据的时候，SerializerHalper 函数则需要成为其友元函数。如果把它看成是外部成员函数的话，这种做法也不算是打破了类的封装。实际上，这和为一个类添加 iostream 的支持是差不多的。

当此类处在一个继承层次结构中的时候，事情就比较有趣了。例如，假设我们有一个抽象基类、Message 类以及一些具体派生类，MoveMessage 和 ChatMessage。基类有一些需要序列化的成员，派生类也是一样。由于在将来可能会加入更多的消息类，因此网络层（在此处消息将被序列化，以便送到网线上去）只知道 Message 对象。此层应该如何对任意的 Message 进行序列化呢？它又如何从一堆收到的字节中创建任意的 Message 呢？

对于序列化来说，其答案就是要再引进可序列化的概念。直接把基类 Message 定义成可序列化的（就是为其定义 PutInto()和 GetFrom()）。然后在这些函数里面调用一些纯虚方法，而这些方法应该是 Message 的派生类必须定义的，以处理自己特定的数据。在接收的时候，使用工厂模式[Gamma95]在恢复之前先创建出正确类型的 Message 对象。为了让接收方能知道到底发送了什么类型的消息，以知道要处理什么类型，我们还需要加上一个类型信息的头部（例如一个消息类的 ID）。

6. 指针与数组

对指针的处理是很简单的——只要解引用然后调用其相应的方法就行了。专门定义一个针对指针的 Put()和 Get()方法是没有用的，因为编译器会以为需要进行选择以实例化相应模板（Put<T>(T*)或是 Get<T>(T*)）。这是一个二义性的错误，编译器不喜欢选择，其结果就意味着指针的 STL 容器是无法自动序列化的。

我们必须改变接口以支持数组，因为我们必须要知道到底需要在 Put()中写入多少个元素，在 Get()中需要读出的元素数目的最大值又是多少。由于重载的一个目的就是为了保持一个一致的接口，因此这是我们不希望看到的现象。然而，与原始的数组打交道会带来较高的效率，因此我们提供了模板化的 PutArray()和 GetArray()函数（需要一个长度参数）——但是一般说来，使用 STL 的 vector 类是与数组一样快的。正如指针一样，一个数组的 STL 容器也是不能序列化的，虽然这基本上不算一个重要的限制。

Download at www.Pin5i.Com

5.5.4　扩展与优化

1. 一个变长的计数值

字符串、缓冲区以及 STL 容器都将其尺寸保存在了序列化输出的结果中。那么就有了这么一个问题：这个长度域应该使用什么样的整数来保存呢？为了将输出的大小最小化，它应该尽可能的小。如果将长度使用一个（假设）uint16 来保存，则我们就无法序列化大型的集合了，这可能会对用户的代码造成影响。使用一个 uint32 当然就足够了，但是在大多数情况下它只会带来空间的浪费。

要将输出的大小最小化的同时还能照顾到较大的长度域，有个思想就是将长度用变长字节来保存。每个字节包含了 7 个数据位，还有一个截止位。为了对一个具有 N 个有效位的计数值进行编码，则我们总共需要（[(N-1)/7] + 1）个字节。这种格式将使用 5 个字节对一个具有 29 个或是更多有效位的值进行编码，但是如果有这么多的元素需要存储的话，则额外字节带来的影响就可以忽略不计了。

你可能会想对所有的多字节整数都使用这种技术。然而，如果要存储的值在相应类型的整个取值空间上是均匀分布的话，这种方法就很不好了，因为此时平均的编码长度将比固定长度还要大。只有当大部分需要序列化的值比较小的时候编码才是有用的。

2. 使用键值的 Serializer

脚本语言一般都有一个内置的通用散列表，它可以用来存储从任何对象到另一个对象的映射关系——其中 Perl 的关联数组和 Python 的字典类型就是两个例子。通过使用序列化，我们也可以在 C++中实现同样的概念。我们的目标是要创建一个类，它负责维护一个从某已知类型 T 到任意类型对象的映射——例如一个不对其值类型限制的 STL 的 map 类。你可能已经想到了，这个类是一个模板（还有成员模板函数），其接口如下：

```
template <typename KEY>
class KeyedSerializer
{
public:
    template <typename T>
        bool Put(const KEY& key, const T& value);
    template <typename T>
        bool Get(const KEY& key, T& value);
};
```

此实现实际上相当简单。它的内部有一个从 KEY 到 Buffer 的映射，而这两个类型在前面已经定义了。Put() 首先把 value 序列化为一个 Buffer，然后将其加入映射。Get() 则进行一个逆过程（没有改变映射——因此它可以多次被同一个键值调用）。KeyedSerializer 有一个有趣的特性，那就是它不像 Serializer，可以允许对其存储的对象进行随机访问。如果一个对象总是没有被读出，则其恢复代码（有可能很复杂）就决不会被调用。最后一点，KeyedSerializer 本身也是可以被序列化的，这样，它在网络环境中就有实用价值

Download at www.Pin5i.Com

了。

3. 部分序列化

网络开发人员都知道带宽就是金钱。在一个网络游戏中，任何能减小在服务器和客户机之间（或是在对等客户机之间）发送消息尺寸的事情都值得去尝试。从这个角度来看，由于KeyedSerializer类可以用来在网络上发送部分更新的消息，因此它就会很有用了。

例如，假设一个游戏对象有这么几个属性：位置、方向、速度、大小、颜色、类型，如果一个对象的更新消息里面包含了所有这些内容，而且每过一段常数时间就会发送一个更新，则显然带宽在此处就被浪费了——对象的大小、颜色和类型即使会变化的话，也通常不会变得很频繁。显然，我们可以使用单独的消息来标识每个属性的更新，但是随着属性数量的增加，可能的消息（包括相关属性的组合）数量就会急剧膨胀了。不过，如果一个对象的更新消息中包含了一个KeyedSerializer（或许会使用uint8作为键值），那么一个消息就足以处理任何属性的组合了，而这种方法也没有产生其他的分支处理，并且开销也是最小化的。

4. 对象标识

Serializer有一个潜在的严重的问题。那就是类型安全——其实上它没有类型安全。如果两个对象序列化后的字节数一样的话（例如float32与uint32），那么在程序中有可能会存入的是一种类型，而读出的又是一种类型。当然了，这是上层应用程序的错误，而不是序列化库的错误，但是实际情况就是此错误发生了，而且没有任何提示信息（除非是其操作造成的其他奇怪的现象）。

对于此问题的解决方案是引入一种安全手段，在每个对象前面加上一个标识值。在读出的时候，程序将检查此标识值，看其是否与期望值一样。如果标识值不匹配，那么就将返回一个错误。为了计算出一个给定类型的标识值，我们需要使用在SerializerHelper名字空间里面的另一个函数（此函数必须支持所有的类型）。在使用单字节的标识值时，下面就是相应的代码：

```
template <typename T> bool Put(const T& obj)
{
    uint8 tag = SerializeHelper::ComputeTag(obj);
    return WriteTag(tag) &&
          SerializeHelper::PutInto(*this, obj);
}

template <typename T> bool Get(T& obj)
{
    uint8 expected = SerializeHelper::ComputeTag(obj);
    return ReadAndCheckTag(expected) &&
          SerializeHelper::GetFrom(*this, obj);
}
```

我们上面没有提到这个方法会加大序列化输出结果的尺寸。然而，我们可以将标识值设置为可选项（例如只在调试版本中使用或是在运行时可以进行调整），这样就可以消除这个额外的开销了。其辅助性函数WriteTag()与ReadAndCheckTag()隐藏了所有的细节，在

Download at www.Pin5i.Com

此处就可以进行标识值的选项处理。如果不使用标识值，则这些函数就是一些空操作，直接返回 true 就是了。

5.5.5　未来的工作

你可以对 Serializer 及其相关的类做很多方面的扩展。下面就是其中可能的一些改动：

- 将存储机制通用化（不要老是使用 `Buffer`），使用一个抽象接口，实现对文件的序列化，包含二进制形式和可阅读形式的（对于调试很有用）。
- 加入对你喜欢的脚本语言的支持。
- 编写相应的适配器，加入流操作符（<<与>>）的支持，这样看上去更符合语法。
- 编写一个代码生成器，将创建可序列化类的工作自动化，当然要基于一个有关类型和域命名的规范了。
- 在序列化层中加入加密以及压缩的支持。
- 将代码移植到一个新的编译器上或是平台上去。

5.5.6　结论

本节阐述了一个对象序列化的系统，它具有高效性和可扩展性，而且对原有系统的修改也很少。由于使用了强大而具有表现力的模板，它在一个相对较小的代码里面提供了很多的功能。

5.5.7　参考文献

[Austern98] Austern, Matthes H., *Generic Programming and the STL*, Addison Wesley, 1998.

[Gamma95] Gamma, Erich, et al., *Design Patterns*, Addison Wesley, 1995.

[Josuttis99] Josutti, Nicolai M., *The C++ Standard Library: A Tutorial and Reference*, Addison Wesley, 1999.

[Kernighan99] Kernighan, Brian W. and Rob Pike, *The Practice of Programming*, Addison Wesley, 1999.

[Meyers01] Meyers, Scott, *Effective STL*, Addison Wesley, 2001.

Download at www.Pin5i.Com

5.6 安全套接字

Pete Isensee
微软公司
pkisensee@msn.com

在一个多玩家游戏中进行欺骗和攻击是一个常见的足以毁掉游戏兴致的问题了。防止欺骗的一个方法是使用密码，对网络数据进行加密和认证。本节探讨了因特网协议安全（IPSec）标准，展示了游戏可以如何使用此标准以保护网络数据包，而且防止伪装、窃听以及重放（replay）攻击。

5.6.1 IPSec

IPSec 是一个由因特网工程任务组基于学术以及政府研究提出的因特网标准。[RFC2401]至[RFC2411]之间的文章构成了此标准的正文，此外其中还包含了[IPSec2]。IPSec 为认证、完整性与信任度提供了服务，而且在虚拟专用网上得到了广泛的应用。IPSec 是用来在网络层（网络栈）中实现的，而且一般说来对上层应用是不可见的。不幸的是，对于大多数游戏平台而言都没有提供 IPSec 的内置选项支持。即使 IPSec 在某些操作系统上已经有了（例如 Windows2000），但是它并没有被设置为缺省值，而且也没有要求应用程序对之给予支持。尽管如此，游戏开发人员仍然能从 IPSec 中学到很多东西，而且很多 IPSec 中的特性都可以被加入到游戏的网络代码中去。

认证就是验证是谁发送了一个消息。完整性是指确保一个消息没有被篡改。IPSec 使用了键值加密散列算法（Keyed cryptographic hash），例如 MD5[RFC1321]以及 SHA-1[RFC3174]。“信任度”这个术语是用来描述数据遮蔽的，这一般指的是加密。IPSec 使用了对称加密算法（Symmetric encryption algorithm）例如 DES[RFC2405]以及 AES[AESDraft]。对于游戏数据来说，通常会使用一个高性能的算法。需要进行更高层次加密的数据（例如玩家造成伤害的数据包，比赛的分数等）可以使用更强大然而也更缓慢的算法（若想对算法的性能有个总体的了解，请参看[WeiDai01]）。

仅仅只有加密是不能防止重放攻击的。一个被认证过和加密过的消息可能会被一个攻击者截获住，然后再重新发送出来，而同时仍能通过认证与解密。因此，IPSec 还引入了一个机制用来防止重放攻击，其方法就是使用序列号和滑动重放窗口。

Download at www.Pin5i.Com

5.6.2　警告

在创建一个安全系统的各步骤中，最关键的就是密钥的管理。本节没有讲述有关密钥交换的技术，也没有涉及生成、终止以及更新密钥的合适方法。此处我们假设在一个安全连接的两端有同样的密钥，而且密钥的生成和交换都是通过安全的形式进行的。另一个对安全系统至关重要的过程就是如何为密钥、共享的密文以及初始化的数组生成真正随机的数值[Isensee01]。本节将假设密钥使用的是密码学上有意义的随机数值。

IPSec 标准要求实现者在数据包的载荷中加入一份 IP 包头部的拷贝，此拷贝必须加以加密与认证。这样，接收者就可以确认真正的 IP 包头没有被篡改了。实现这一层的认证非常复杂，而且一般也没有必要，因为对载荷本身的认证就证明了此数据包是从一个知道认证密钥的人而来的。本节将不讨论 IP 认证。

IPSec 的实现与 IP 协议的包分片（fragmentation）机制有着极为紧密的联系。为了避免讨论这个问题，本节假设所有的数据包都是使用不分片的 UDP 包传输的，虽然此假设不合常理。本节还将假设所有的消息都短于最大传输单元（MTU），对于那些大于 MTU 的消息来说，应用程序必须自己实现其序列号机制。

5.6.3　安全连接

安全连接（SA）是出于安全目的而创建的逻辑连接。SA 定义了当数据从一个节点传输到另一个节点的时候如何保证其安全性。SA 包含了加密和认证密钥、模式、算法以及其他的定义了此数据包格式的数据。本节中的 `SecurityAssociation` 类包含了如表 5.6.1 所示的信息。

表 5.6.1　　安全连接数据

数　据	描　述
认证算法	对认证使用的加密散列算法（例如 MD5、SHA-1）
认证密钥	对认证使用的对称密钥数据
加密算法	对数据包加密和解密使用的对称加密算法（例如 DES、AES）
加密密钥	加密算法使用的对称密钥数据
序列号	在本 SA 上下一个将要发出的包使用的序列号
最近序列号	在本 SA 中接收到的最大序列号
重放窗口	一个用来为了防止重放攻击而作为滑动窗口的位掩码
IV 尺寸	加密初始化数组（2～8 个字节）的尺寸
ICV 尺寸	完整性检验值的尺寸（8～12 个字节）
最大填充块	额外随机填充块的最大数量（0～4 块）。每块的尺寸是由加密算法定的

SA 几乎总是成对出现，其中一个发送端的 SA 精确地匹配于一个接收端的 SA。在一个客户机—服务器的游戏中，服务器可能有一个 SA 的列表，其中每个 SA 都对应了一个客户机，而每个客户机都有一个 SA 用于匹配服务器上的 SA。在一个对等用户的游戏中，每个用户都将有一个 SA 的列表。由主机维护的安全连接列表称为“安全连接数据库”（SAD）。对

Download at www.Pin5i.Com

于游戏来说，此数据库一般都存在于游戏或是游戏服务器的内存当中。

在发送或是接收一个安全数据之前，通信的双方必须先建立一个安全连接。本节将不讨论发送方和接收方之间 SA 数据交换的方法。其中的某些数据，例如算法，可以固化在代码中。其他的数据，例如密钥，应该出于最佳安全的理由定期进行改变，而且应该使用安全协议例如 EKE、Kerberos 或是 Diffie-Hellman[Schneier96]进行交换。为了保证得到最佳的安全，要确保为认证和加密使用不同的密钥。

5.6.4 包格式

实现安全套接字就是要对数据进行加密，然后在其上加一个散列值，然后把它发出去，在接收的时候对之进行认证与解密。表 5.6.2 显示了一个安全数据包的格式，详细指出了数据包的哪一部分被认证和加密了。

表 5.6.2 安全数据包的格式

数　据	大小（字节）	缺省大小（字节）	经过认证	经过加密
安全参数索引（SPI）	1～4	2	是	否
序列号	4	4	是	否
初始化数组（IV）	2～8	4	是	否
载荷	变长	变长	是	是
填充数据	0～255	0～255	是	是
填充长度	1	1	是	是
完整性检验值（ICV）	8～12	10	否	否

1. 安全参数索引

安全参数索引（SPI）是一个在安全连接数据库中唯一标识了一个安全连接的句柄。一个 SPI 在每个安全数据包中都将被传送，这样客户机就可以选择相应 SA 下的数据包进行处理了。SPI 的大小是取决于游戏的，而且还要视在同一时刻 SA 的最大数目而定。在 CD-ROM 上包含的示例实现生成的是随机的唯一两个字节长的 SPI。此 SPI 没有，而且也不应该被加密，因为它是用来标识定义了相关算法和密钥的 SA 的。如果攻击者修改了 SPI，则数据包的有效性检验就无法通过，因为 SPI 总是要被认证的。

2. 序列号

序列号是一个总在增长的计数值。在一个给定的安全连接中发送的第一个数据包的序列号是 1，下一个包将是 2，如此类推。接收者使用序列号以确保忽略重复的包和过期的包。发送方必须确保如果有可能序列号返回 0 的话，则连接的双方要生成新的一对安全连接，同时在返回 0 之前要终止老的 SA。序列号没有被加密，这样无需解密就可以发现重放攻击了。这也可以降低拒绝服务攻击可能造成的影响。如果一个攻击者修改了序列号，数据包的有效性检验就会失败，因为序列号总是要被认证的。

Download at www.Pin5i.Com

3. 初始化数组

一个初始化数组（IV）是一堆用来初始化对称加密算法的二进制随机数据。IV 确保了即使需要把同一份明文发送多次，其每次加密得到的密文也各不相同。IV 是发送方通过随机数据初始化而成的，随之就会把它送给加密算法。接收方使用同样的 IV 对其解密函数进行初始化。每个加密算法都定义了一个标准的 IV 长度，而大部分对称算法都使用 8 个字节长的 IV。为了牺牲安全性以得到更小的带宽要求，在实现中可以使用更小的 IV。我们的例子使用的是缺省 4 字节的 IV。在加密和解密过程中，IV 剩下的部分将被设置为序列号。

4. 载荷

载荷指的是原始的明文数据。此数据将被加密，放到数据包中，最后被传送到网线上。

5. 填充数据

填充数据是在加密之前被添加到载荷后面的。填充数据的使用是出于两个原因：第一，加密算法可能需要使用某个特定的块大小。例如，很多对称加密算法使用 8 字节的块；第二，可能需要添加一份随机大小的填充数据以隐藏载荷的真实大小。填充数据是使用一组整数值（从 1 开始）进行初始化的，这些数据可以被接收方验证，从而提供了更高的安全性。

6. 填充长度

填充长度是一个 1 字节的域，它用来存储数据包中填充数据的长度。

7. 完整性检验值

完整性检验值（ICV）是一个截断过的、基于散列函数的消息认证编码（HMAC）。一个 HMAC 是一个混合了标准加密散列算法和秘密对称密钥的关联散列算法[RFC2104]。正如一个指纹能唯一地标识出一个人一样，一个 HMAC 也唯一地标识出了一个安全数据包。而对 HMAC 加以截断则是一个众所周知的安全方法。在 CD-ROM 上的示例实现缺省将把散列值截断为 8 个字节。

发送方通过对加密后的包数据使用 SA 的认证密钥计算出 ICV 值。接收方也将进行同样的计算，然后将计算出的 ICV 与发送的 ICV 进行比较。如果两者一致，则数据包就是有效的——它是某个知道匹配 SA（认证）的人发出的，而且在传输中没有被篡改（完整性）。不然的话，此包就是伪造包，应该被丢弃掉了。

5.6.5　发送数据

发送数据简单说来就是对载荷进行加密，然后生成一个将要发出的数据的加密散列值。下面的各个小节描述了示例实现中的相关细节。

Download at www.Pin5i.Com

1. 建立一个安全连接

在发送安全数据之前，发送方要与接收方建立一个安全连接。一旦 SA 与 SPI 都设置好了以后，通信的双方就可以得到其数据，能对两端之间的传输流进行加密、解密以及认证了。

2. 建构头部

头部包含 SPI、序列号以及加密后的 IV。SPI 对应的是用来对载荷进行编码和认证的 SA。当前的序列号在 SA 中也可以找到。一旦一个序列号被用过了，它就将在 SA 中加 1。加密后的 IV 是随机生成的。如果 SA 的加密算法要求一个比发送的 IV 更大的 IV，则剩余的字节就用序列号来填充。

3. 生成填充字节

填充字节是基于明文载荷的大小和 SA 加密算法生成的。此外，可能还需要添加额外的随机填充字节以隐藏载荷的真实大小。而随机填充数据的数量则是可以配置的。填充字节是使用一组从 1 开始的整数进行初始化的。

4. 对载荷加密

数据包的加密部分包含着原始的载荷、填充字节以及填充长度字节。对称加密使用上面生成的 IV 进行初始化，而加密的载荷则利用 SA 对称加密算法与 SA 对称加密密钥生成。得到的密文将被放置到头部之后。

5. 生成认证编码

在发送数据之前的最后一步就是要生成 ICV。HMAC 算法使用 SA 加密散列算法和认证密钥进行初始化。散列算法接下来对包头部与加密后的载荷以及填充数据进行处理。得到的散列值被追加到密文之后。此散列值是没有加密的。

6. 发送安全数据包

现在我们就可以使用任意 UDP 的套接字方法把安全缓冲区发送出去给游戏用了，一般此方法是 send()。

5.6.6 接收数据

在接收方，其操作的顺序对于安全性与性能是很重要的。接收方必须检验数据包中 SPI、ICV 以及序列号的有效性。然后接收方可以对数据包进行解密，检验填充字节的有效性。只有当所有的有效性检验都通过了之后，接收方才能来处理数据包的载荷。如果其间有任何一个有效性检验失败了，则此包就必须给丢弃。如果接收方是一个游戏服务器，那么服务器可能还需要对失败的事件进行检查以作为一种探测和跟踪欺诈行为的一种方法。

Download at www.Pin5i.Com

1. 接收安全数据

安全数据包可以使用任何 UDP 的套接字方法来接收以为游戏所用，一般这是 recv() 函数。

2. 检验数据包的有效性

对数据包可能做的最简单的检验就是对 SPI 的有效性进行检验。如果 SPI 与接收方安全连接数据库中的安全连接值不匹配，那么此数据包就将被视为无效。假设 SA 匹配了，那么接下来就可以对块大小做一个快速检验，这是基于安全数据包中的最小数据大小做的。

下面一步就是要检验数据包中 ICV 的有效性。HMAC 算法将使用 SA 的加密散列算法及其密钥进行初始化。然后散列算法将对数据包头与加密后的载荷进行处理。得到的散列值被截断以后再与接收到的 ICV 进行比较。如果两者匹配，则此数据包就通过认证了。

在对数据包解密之前的最后一步就是检查序列号。如果此数据包是一个重放包或是太老了，我们就不需要对之进行解密了。示例实现使用了一个 64 位的滑动窗口以拒绝重放攻击，同时还能允许 UDP 中可能出现的乱序包的处理。窗口的右边代表的是在此 SA 上当前已经验证过的最高的序列号。如果一个数据包的序列号小于窗口的左边的数值，则此数据包将被丢弃（它们太老了）。落在窗口中的数据包则将对比已收到的数据包进行检验。如果相应的数据包已经收到了，那么这就是一个重放包，应该被忽略掉。新的数据包则会在窗口中设置相应的一位。而如果一个数据包的序列号大于窗口右边的数值，则窗口就向右滑动了。

3. 对载荷解密

解密的第一步就是要取出 IV。如果需要的话 IV 将被追加上序列号，然后它就可以被用来对解密算法进行初始化了。载荷与填充数据是利用 SA 对称解密密钥进行解密得到的。其最终的明文包含原始的载荷、填充字节以及填充数据长度。

4. 检验填充数据的有效性

我们需要检验填充数据大小的有效性以确保它在有效范围以内。接下来再对填充字节进行有效性检验以确保它们包含了一组整数值。最后所有的填充信息都被从明文后删除掉，只剩下原始的载荷了。

5.6.7　示例实现

本节讲过的 CD-ROM 上的示例代码中含有一些 C++的类，它们对加密函数、安全连接以及安全缓冲区进行了封装。表 5.6.3 展示了我们提供的类。

表 5.6.3　　安全套接字类

类　名	描　述
CryptoContext	Win32 CryptoAPI 加密服务提供商
Key	加密密钥与算法

Download at www.Pin5i.com

续表

类　名	描　述
Cipher	加密与解密函数
Hash	散列算法
Buffer	std::string<unsigned char>
SecurityAssociation	安全连接数据
SADatabase	std::map<SpiType, SecurityAssociation>
SecureBuffer	用于对载荷加密、解密以及认证的函数

关键的实现细节在 SecurityAssociation 和 SecureBuffer 之中。下一小节描述了如何在一个游戏里面使用这些类来进行安全数据的发送与接收。

1. 创建一个安全连接

创建一个安全连接，将之加入安全连接数据库（SAD）中去：

```
// One-time association of SAD to SecureBuffers
SADatabase sad;
SecureBuffer::SetSADatabase( &sad );

// Generate random keys. This example uses the
// DES cipher for encryption and MD5 for hashing
Key keyAuth( CALG_DES );
Key keyCrypt( CALG_DES );

// Create a new SA using the specified keys
// and algorithms; other values get defaults
SecurityAssociation sa( keyAuth, keyCrypt, CALG_MD5 );

// Generate a new SPI and add the pair to the db
SpiType nSPI = sad.GenNewSPI();
sad.Insert( nSPI, sa );

// Securely exchange nSPI and keys with other end
// of connection . . .
```

2. 发送一个安全数据包

生成与发送一个安全数据包：

```
// Associate SecureBuffer with SA via SPI
SecureBuffer sb( nSPI );

// Generate the encrypted and authenticated buffer.
// Encryption and hashing happens here.
sb.Create( "payload", 7 );

// Send the secure packet
send( sock, sb.GetDataPtr(), sb.GetSize(), 0 );
```

Download at www.Pin5i.Com

3. 接收一个安全数据包

接收与处理一个安全数据包：

```
// Receive the secure packet
char pData[ 1024 ];
int n = recv( sock, pData, 1024, 0 );
if( n == 0 || n == SOCKET_ERROR )
  return false;

// Validate the packet
SecureBuffer sb( pData, n );
if( !sb.IsAuthentic() )
  return false;

// Extract the original payload
if( !sb.GetPayload( pData, &n ) )
   return false;

// Adjust the replay window
sb.SetAccepted();
return true;
```

5.6.8　CryptoAPI

此处使用的加密算法和密钥管理使用了 Windows CryptoAPI。所有加密的的代码都模块化到 `crypto.cpp` 里面去了，这样你就可以使用其他的实现（例如，Crypto++）[WeiDai01]替换底层的加密函数了。CryptoAPI 一个令人恼怒的方面就是它没有提供对密钥数据的直接访问。没有相应的 API 可以让你直接设置或是得到一个密钥。不过，微软的 KnowledgeBase 中的文章 Q228786[KB228786]讨论了一个直接访问密钥数据的方法，而 `Key` 类就使用了此方法。

出于 CryptoAPI 特殊的设计——它使用了加密服务提供商的概念——并非所有版本的 Windows 都有同样的算法或是密钥长度。因此一定要检查返回值以确保没有发生错误。

5.6.9　性能

在使用安全套接字的时候有两个主要的性能问题。第一个是它需要额外的带宽。在使用缺省设置和一个 8 字节块长的加密算法（例如 DES）的情况下，对于示例实现，最小的额外数据包开销是：

2 (SPI) + 4 (SeqNum) + 4 (IV) + 0–15 (Padding) + 1 (PadLen) + 10 (ICV) = 21–36 bytes

这 21 到 36 个字节是除了标准 UDP 数据包以外的数据开销。你可以改变 SA 参数以得到更小的开销，但是代价就是牺牲了安全性。为了减少开销带来的影响，可以考虑把较小、较频繁的载荷传输改为较大、较少的载荷传输形式。为了避免不必要的填充，在发送的时候确保载荷的大小加 1 的数值可以被 SA 加密算法要求的块长所整除（加 1 是为了考虑到填充数

Download at www.Pin5i.com

据长度字节）。

第二个性能问题就是加密/解密与认证了。表 5.6.4 显示了对于 1k 字节的载荷数据进行创建、认证和解密带来的开销，它表明其性能还是可以接受的。在实际工作中最重要的因素就是算法的速度和每个包的大小。你可以选择相应的算法和密钥长度以在可接受的性能之下提供合理的安全性。

表 5.6.4 性能

载荷大小（字节）	处理 1kB 载荷的时间		
	创建（ms）	认证（ms）	解密（ms）
64	2.07	0.90	1.07
128	1.06	0.45	0.57
256	0.59	0.24	0.34
512	0.35	0.14	0.23
1024	0.23	0.09	0.16

这些值是在一台奔腾 III，866MHz，Windows 2000 系统，使用缺省 SA 设置和 DES 加密（64 位长的密钥）以及 MD5 散列函数生成的。“创建”一栏指的是重复调用 `SecureBuffer::Create` 对给定的载荷大小以生成总载荷数据所花费的毫秒数。“认证”栏显示的是调用 `SecureBuffer::IsAuthentic` 花费的时间，而“解密”栏显示的是调用 `SecureBuffer::GetPayload` 花费的时间。随着载荷大小的增大，性能也随之上升，因为对于大包来说每个包的相对开销就小了。

5.6.10 安全性

本节的目的是要给出一个实现，在造成很大的包开销的极端健壮安全性与最小包开销的合理安全性之间作出一个权衡。所提供的安全性的等级主要是取决于使用算法的强度的。决不要使用 XOR 作为加密或是认证的算法，以为可以让你的数据包不会被别人解出来。选择一个公开的标准算法经过考验的实现，例如加密方面的 DES、Triple-DES、Blowfish 以及 AES，散列计算方面的 MD5 与 SHA-1。要理解在这些算法中安全等级与性能的折中考虑。当算法支持的时候，尽量使用较大的密钥（128 位的）。定期改变密钥。不要减少 IV 的长度到 4 个字节以下，也不要把 ICV 的长度减到 8 个字节以下，不然安全性就会大大下降了。

虽然从技术上来讲，你可以不使用加密而仅仅使用认证，但是要注意没有加密的认证数据包会使你的游戏暴露在窃听攻击之下，这将导致游戏欺骗的结果，而这和伪装攻击效果是一样糟糕的（而且更难被察觉）。

5.6.11 结论

随着越来越多的游戏开始使用竞赛或是大奖赛的特性，安全通信不仅可以减少欺骗行为，而且也成为了游戏中不可或缺的部分。IPSec 规范包含了一些可以直接使游戏受益的安全因素。虽然数据包的开销并非不可忽视，但是性能仍然较高，而且其提供的安全数据的优点也是值得考虑的。

Download at www.Pin5i.Com

5.6.12　参考文献

[AESDraft] “The AES Cipher Algorithm and Its Use With IPSec,” 草案对应网址为 http://www.ietf.org/10.html, November 2001.

[Howard02] Howard, Michael and David LeBlanc, *Writing Secure Code*, Microsoft Press, 2002.

[IPSec2] “IP Encapsulating Security Payload(ESP),” 更新后的草案对应网址为 http://www.ietf. org/ID.html, November, 2001.

[Isensee01] Isensee, Pete, “Genuine Random Number Generation,” *Game Programming Gems 2*, Charles River Media, Inc., 2001.

[KB228786] “How to Export/Import Plain Text Session Key Using CryptoAPI,” 对应网址为 http://support. microsoft.com; q228786, July 2001.

[RFC1321] “The MD5 Message-Digest Algorithm,” 对应网址为 http://www. ietf.org/rfc/rfc1321.txt, April 1992.

[RFC2104] “HMAC: Keyed-Hashing for Message Authentication,” 对应网址为 http://www. ietf. org/rfc/rfc2104.txt, February 1997.

[RFC2401] “Security Architecture for the Internet Protocol,” 对应网址为 http://www. ietf.org/rfc/rfc2401.txt, November 1998.

[RFC2403] “The Use of HMAC-MD5-96 within ESP and AH, ” 对应网址为 http://www. ietf.org/rfc/rfc2403.txt, November 1998.

[RFC2404] “The Use of HMAC-SHA-1-96 within ESP and AH,” 对应网址为 http://www. ietf.org/rfc/rfc2404.txt, November 1998.

[RFC2405] “The ESP DES-CBC Cipher Algorithm with Explicit IV,” 对应网址为 http:// www.ietf.org/rfc/rfc2405.txt, November 1998.

[RFC2406] “IP Encapsulating Security Payload(ESP),” 对应网址为 http://www.ietf. org/rfc/rfc2406.txt, November 1998.

[RFC3174] “US Secure Hash Algorithm(SHA1),” 对应网址为 http://www.ietf.org/ rfc/rfc3174.txt, September 2001.

[Schneier96] Schneier, Bruce, *Applied Cryptography*, Second Edition, John Wiley & Sons, 1996.

[Schneier99] Ferguson, Neils and Bruce Schneier, “A Cryptographic Evaluation of IPSec,” 对应网址为 http://www.counterpane.com/ipsec.html, February 1999.

[WeiDai01] Dai, Wei, “Crypto++ Cryptographic Class Library,” 对应网址为 http://www.eskimo.com/~weidai/cryptlib.html, November 2001.

Download at www.Pin5i.Com

5.7 一个网络监控与模拟工具

Andrew Kirmse
LucasArts Entertainment Company
ark@alum.mit.edu

大多数在线游戏都是在一个理想的高带宽低延迟局域网的环境下开发出来的。与之形成对比的是，在线游戏通常是在一个低带宽高延迟在机器之间有着不可预测的网络行为的环境里面运行的。本节描述了一个叫做“NetTool”的基于 Windows 的模拟工具，你可以用它来使自己的局域网表现得更像因特网。在 CD-ROM 上有完整的 Win32 源代码与二进制文件。

5.7.1 界面

图 5.7.1 显示了 NetTool 界面的一个截图。在顶部的一栏里设置的是一组过滤器，每个过滤器都描述了一对网络地址（IP 地址与端口号）之间的行为。面向连接的 TCP 过滤器可以允许通信在两个方向上进行，因此在任何两个主机之间都只有一个过滤器。UDP 过滤器是单向的——点击了“Add Reverse”按钮之后，你就可以得到当前选择的过滤器，将之两端交换，创建出一个新的过滤器了。一般当主机使用相同的端口进行数据的发送与接收的时候这是比较有用的。

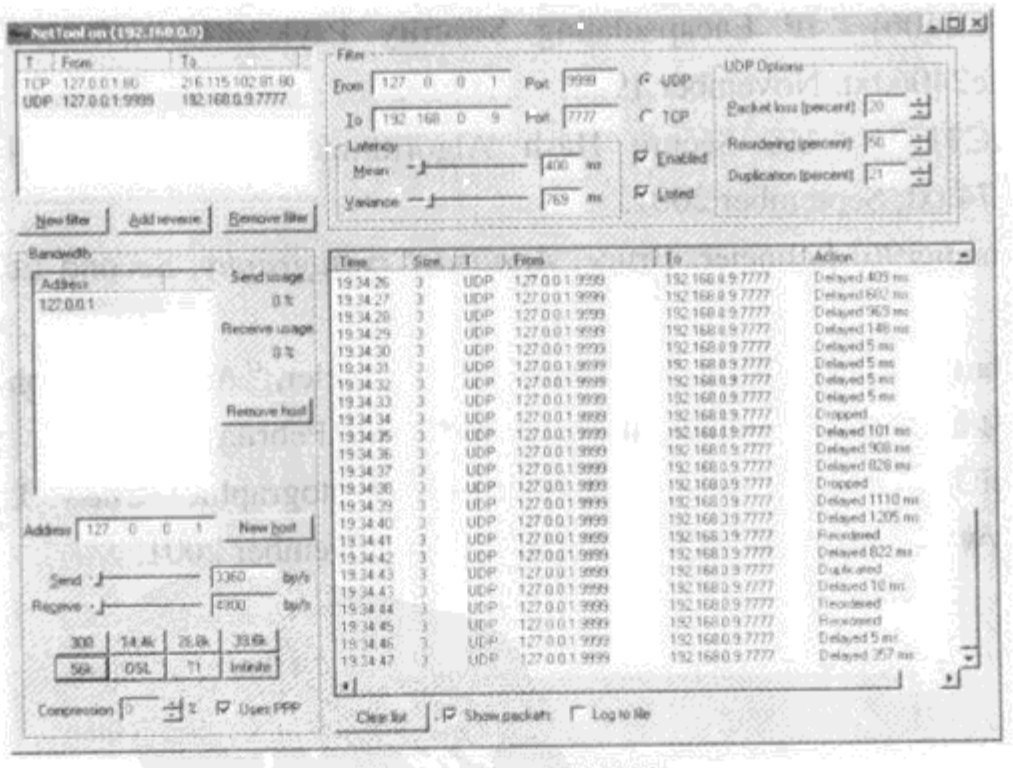


图 5.7.1 NetTool 的界面

Download at www.Pin5i.Com

NetTool 基于每个包进行网络延迟的模拟，它使用的是高斯分布。在界面上你可以设置每个过滤器延迟分布的均值与方差。

对于每个过滤器来说还有其他的一些控制可用。只有当 Enabled 复选框被选中的时候传进来的数据包才能被转发，这样你就可以选择性地限制住从某个主机而来的流量了。如果选中了 Listed 复选框，则传进来的数据包将被显示在屏幕右下角的那个大列表框里面。UDP Options 里面的设置只对 UDP 有效，这一点我们稍后就会谈到。

标题栏里面显示的是运行 NetTool 的机器上的所有 IP 地址。

5.7.2　网络监控

当数据包到达的时候，NetTool 将根据其对应的过滤器设置对之进行分发。如果 Show packets 复选框被选中了，那么每个数据包的信息将显示在界面右下角的大列表框里。其中将显示出数据包的到达时间、载荷大小、目的地址与源地址以及 NetTool 对之采取的行动。这些行动列表如下所示：

- Sent Immediately：对于此包没有延迟要求，因此它将被立即发送出去。
- Delayed X ms：网络模拟要求此包在转发之前进行延迟。延迟的数值是由过滤器的延迟设置和主机带宽模拟（稍后就会谈到）一起决定的。
- Dropped：此包没有被转发。这只对 UDP 的数据包有效，当过滤器的 Packet loss 设置不为零的时候会发生。
- Reorder：此包在另一个先前到达的包之前被发送出去了。这只对 UDP 的数据包有效，当过滤器的 Reordering 设置不为零的时候会发生。
- Duplicated：此包发送了两次。这只对 UDP 的数据包有效，当过滤器的 Duplication 设置不为零的时候会发生。
- Wait for connect：对一个 TCP 的过滤器来说，数据在远程连接完成之前到达了。一旦连接成功，此包就将被发送。

5.7.3　TCP 模拟

TCP 是一个可靠的、基于流的网络协议，数据按发送时的顺序达到，其数据块大小则是任意的。NetTool 将把所有的到达数据放在一个单独的队列里。然后它将对此队列进行轮询，在发送方和接收方带宽模拟设置允许的情况下转发尽可能多的数据。

在 NetTool 中有几个需要注意的细节，它们是由 TCP 面向连接的本质引发的。例如，从发送方而来的数据可能会在接收方尚未处理完连接的时候就来了，或是发送方可能在之前发送的数据尚未被转发之前就关闭了连接。这些情况都在程序里面考虑到了，而且应该对应用程序是透明的。

5.7.4　UDP 模拟

UDP 是面向消息的，这就是说到达的数据和发送的数据其数据块都是一样的。UDP 是

Download at www.Pin5i.Com

不可靠的，这就意味着在传输中可能会丢掉一个包，包到达的顺序和发送的顺序可能会不一样，或是甚至可能到达多次。在一个局域网里面，UDP 包几乎完全不会丢掉。但是在因特网上，丢包的现象则很普遍。

NetTool 是根据 Packet loss 框里面设置的概率进行丢包的。这些包就是简单地被丢弃了，如同一个因特网上的路由器在超负荷运行的时候表现得一样。

如果一个包没有被丢弃，那么它还有可能被复制（发送两次），其概率等于 Duplication 框里面显示的数值。延迟对这两个复制包的影响是独立的。如果还需要进行乱序，则有可能这两个包不会连续到达。

包乱序只会在某些特定的情况下出现。假设有两个 UDP 包，A 和 B，到达了同一个端口。延迟模拟将对每个包单独进行，导致 A 和 B 进入调度以备传输。如果其结果是 A 在 B 之前被调度的话，则没有乱序的发生。然而，如果 B 在 A 之前被调度，则 NetTool 将根据 Reordering 设置里面的概率值看是否需要进行此乱序。如果是为了测试的目的要强制加上乱序的话，则需要给延迟加上一个较高的方差以使得第二种情况出现的概率加大。

5.7.5 主机带宽模拟

出了网络可能会引起延迟以外，在一个连接的两端，由于其带宽限制，也有可能引入延迟。例如，在一个 300bit/s 的调制解调器的连接上面，则需要整整 1 秒钟来传输 30 个字节（假设 8 位每字节，而且还需要一个开始位和一个结尾位）。在界面的左下方，你可以设置每个主机可用的带宽。发送和接收带宽可以是不同的，实际上这对于 56kbit/s 的调制解调器来说正是如此（它只能有 33.6kbit/s 的上载速率），对于不对称 DSL 连接来说也是一样的。在带宽滑块的下面有几个预定义的带宽设置值的按钮，不过这些值只是一些大概的值。

NetTool 在带宽计算中考虑到了每个数据包头部的大小。对于 UDP 来说，头部是 28 个字节（20 个字节的 IP 头部加上 8 个字节的 UDP 头部），如果选中了 Uses PPP 的话这个值可以减到 7 字节，因为 PPP 将对 IP 头部进行压缩。对于 TCP 来说头部一般是 40 个字节，但是大部分消费用的连接使用 VJ 头部压缩，这可以把头部大小变成 8 字节。此外，很多调制解调器还有自己的压缩选项，你可以使用左下角的压缩比来对之进行模拟。此处的带宽模拟比较简单，但是它已经足够对一个客户机的连接进行模拟了。

5.7.6 结论

NetTool 是一个用来在局域网上模拟消费者因特网的方便的工具。它特别适合用来进行较差网络环境中的压力测试。一个很好的应用就是可以将之用来对 UDP 上的可靠通信进行测试。如果你对之做了改善，请将你的改动发给作者，以便与大家分享。

Download at www.Pin5i.Com

5.8　使用 DirectPlay8.1 创建多玩家游戏

Gabriel Rohweder
微软公司
grohwed@hotmail.com

给一个游戏添加多玩家支持是一件棘手的事情。一般说来，开发人员会为一个游戏的需要专门设计一个网络引擎。而要在另一个游戏里面重用这段代码往往比为新游戏重写一段代码更令人头痛。DirectPlay 则为这些问题提供了一个解决方案，它提供了一套通用的、可扩展的网络游戏 API，这套 API 不仅易于使用，而且也很容易维护。本节将讲述一些使用 DirectPlay 的时候比较复杂的细节，同时也为如何避免一些常见的错误提供了一些提示。如果你还未体验过 DirectPlay，那就请在阅读本节之前先浏览一遍 DirectX 开发包的示例程序。DirectPlay 支持两种网络架构——客户机/服务器式与对等用户式。虽然现在市面上有很多成功的基于对等用户架构的游戏，但本节将主要关注客户机/服务器架构。

5.8.1　DirectPlay 内幕

要想使 DirectPlay 发挥最大的效用，开发人员首先要熟悉一下这套 API 是如何组织的。DirectPlay 由 3 个独立的层组成：核心层、协议层与服务提供者层。

开发人员的应用程序是直接与核心层打交道的，如图 5.8.1 所示。核心层负责很多事务，例如初始化及维护玩家列表。DirectPlay 使用推模式进行信息交换，这就意味着应用程序需要在核心层里面注册一个可重入的[1]回调函数，当相关的消息从协议层传到核心层的时候，核心层就会调用这个回调函数。

协议层负责数据包的组包和解析。DirectPlay 在每个数据包前面都加上一个称为“包头”的 4 字节长的数据。由于 DirectPlay 是使用 UDP 作为下层的传输协议的，因此这 4 个字节将会紧接在 UDP 包头和 IP 包头的 28 个字节的后面。接收方只有通过解析此包头里面包含的信息才能正确处理数据包。协议还负责对数据进行流量控制（这一点我们下文就将讲到）和确认进来的数据包格式是否是正确的。

译者注[1] 可重入的（reentrant）函数是指在不同的线程里面可以同时调用这个函数而不会引起错误。例如一个会使用静态变量的函数就很有可能不是一个可重入的（reentrant）函数，而一个只使用局部变量的函数就是一个可重入的函数。

Download at www.Pin5i.Com

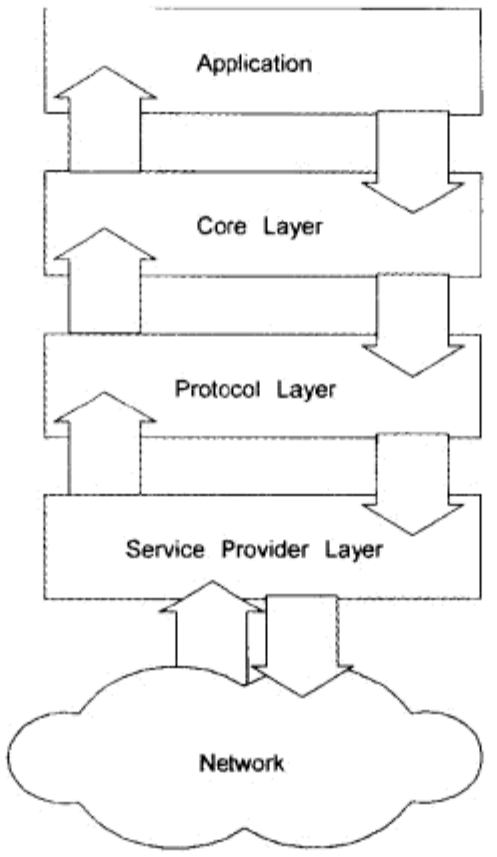


图 5.8.1 DirectPlay 的消息流

服务提供者层为会话[1]提供了传输的功能。它可以支持各种联网方式，例如 TCP/IP，串口通信和 IPX。

DirectPlay 中另一个值得一提的组件是 DPNSVR。DPNSVR 是一个可选的服务，它负责维护一个列表，此列表里面包含的是程序所在机器上面所有的游戏服务器。有了它就可以保证应用程序不会使用冲突的端口创建服务器端的会话，也能使得客户机程序能通过查询特定机器的 6073 号端口得到它上面所有开启的游戏服务器。当前 DPNSVR 只支持 IPX 和 IP 协议。

5.8.2 数据传输

此前曾经提到过 DirectPlay 是一套通用的 API。在这里通用指的是 DirectPlay 不是仅为某一类游戏设计的。当前市场上最流行的一类游戏是第一人称视角射击(first-person shooter, FPS)类游戏。通常 FPS 游戏需要很频繁地传输小块的数据。由于 FPS 需要很快的动作响应，它就需要使用不可靠的消息传递进行大多数数据传输。如果一个数据包在传输过程中丢失，发送方重新发送数据包再传到的时候可能数据已经完全没有意义了。由于这个原因，大多数 FPS 网络层使用了 UDP 协议。UDP 的数据传输速度比较快，但是它不保证数据传输的顺序是正确的，甚至不能保证数据一定会传到。然而有时候开发人员也需要保证数据被指定的接收方收到，而且还要是按序收到，比如玩家死亡的时候和级别变化的时候。DirectPlay 能满足开发人员的所有这些需求，因为它能让开发人员指定一个数据包是否应该可靠，还是可以不可靠，是否要保证顺序，还是可以乱序。典型的 ClienT-side 调用 DirectPlay 发送数据的例子如下所示：

译者注[1] 会话（session）在一般网络书籍里面指的是接收方和发送方进行数据交换处理一个事务的过程，例如一个 HTTP 会话可以指一次网上购物的过程。在此处则指的是玩家从加入或者创建网络游戏直到退出或者关闭游戏的整个过程。

Download at www.Pin5i.Com

```
MYMSG_OBJECT        msgShoot;
DPN_BUFFER_DESC     buffDesc;
DPNHANDLE           hAsync;

msgShoot.dwType       = MYMSGTYPE_SHOOT;
buffDesc.dwBufferSize = sizeof(MYMSG_OBJECT);
buffDesc.pBufferData  = (BYTE*)&msgShoot;
pClient->Send(&buffDesc,1,0, NULL,&hAsync,0);
```

缺省情况下面，DirectPlay 发送数据使用的是不可靠传输和按序传输。Send()调用的最后一个参数是一个 dwFlags 参数。这个参数是用来指定发送配置的。例如，如果你想进行一次可靠的但是可以是乱序的发送，可以使用如下的编码：

```
DWORD dwFlags = DPNSEND_GUARANTEED |
                DPNSEND_NONSEQUENTIAL;
pClient->Send(&buffDesc,1,0,NULL,&hAsync,dwFlags);
```

在调用了一次 Send()或者 SendTo()函数以后，程序的数据就被放置在了 DirectPlay 内部的发送队列里面等待被发送。这么做是因为 DirectPlay 对数据进行了流量控制，根据链路的状态和接收方接收数据的速度，它可以让协议层增加或者减少数据的发送频率。发送队列是和每个会话里面的每个连接关联在一起的。这可以保证在一个会话里面一个慢速的连接不会使其他所有玩家的连接都变慢。尽最大的力量提高网络数据的处理速度对于应用程序效率的提高非常重要（请参照“可重入的回调函数”此段看如何使得程序的回调函数对数据的处理更有效）。开发人员可以通过调用 GetSendQueueInfo()这个方法得到当前发送队列的状态，如下面的代码所示。服务器端的相应接口方法多一个参数，用来指定要返回哪一个玩家的发送队列信息。

```
DWORD dwNumMsgs; //number of msgs in the send queue
DWORD dwNumBytes; //number of bytes in the send queue
pClient->GetSendQueueInfo(&dwNumMsgs, &dwNumBytes, 0);
```

只要一个消息还没有被发送出去或者是超时，它就将一直被保存在发送队列里面。程序可以给每个消息指定一个超时值，用来指明这个消息在发送队列里面能最多保存多长时间。如果一个消息过了一段时间以后就将失去意义，正如我们前面说的那样，那么最好就给这个消息加上一个超时值。下面的代码展示了如何设置一个 30ms 的超时值。

```
DWORD dwTimeOut = 30; //time in milliseconds
pClient->Send(&buffDesc,1,dwTimeOut,NULL,&hAsync,0);
```

如果应用程序发现自己的发送队列实在变得太大了，一般来说有两种可能：一种是接收端的接收线程被堵塞住了，导致 DirectPlay 无法处理传过来的数据；另一种可能是应用程序发送数据的速度过快，使得接收方无法及时处理，从而导致数据包的丢失。如果发送队列的消息数量变得很多，但是队列里面的字节数目相对比较小的话，程序可以试试把数据给组合起来。组合意味着把两个或者更多的消息连起来得到一个更大的消息。DirectPlay 不会为程序做这种事情，因为它是一套通用的 API，因此它不可能知道如何把一个程序的数据按程序

Download at www.Pin5i.com

逻辑连起来。在组合数据的时候开发人员必须注意的一件事情是不要把消息的类型给混淆了。举一个例子，现在有一条告诉服务器玩家现在看不见的消息和一条表示玩家正在用轨迹枪(railgun)开火的消息，你大概并不想把它们组成一条消息，因为：如果你使用不可靠的方式发送这条消息，那么你就有可能冒风险丢失这条表示玩家看不见的消息，因为这个数据包有可能会丢失而且还不会被重传；从另一方面来说，如果你使用可靠的方式来发送这条消息，当前的玩家就可能会在游戏里一个莫名其妙的时候突然开了一枪。

当消息接收到了以后，接收方会把一个 ACK 数据包回送给发送方。ACK 数据包是一个告诉发送方表示接收方已经收到了此消息的特殊的数据包。如果发送方没有收到某个消息的 ACK 包，它就会认为这个消息已经丢失了。可靠传输和不可靠传输的区别就在于发送方是如何处理一个丢失的数据包的。对于一个不可靠消息来说，发送方会忽略这个丢失的消息，继续传输内部发送队列里面的消息。而对于一个可靠消息来说，发送会不停地尝试重发这个消息，直到接收方收到了它或者这个消息超时。值得注意的是这里的超时值和上面说到的超时值是不一样的。当一个可靠的消息在这种情况下面超时的时候，DirectPlay 就认为当前的连接已经中断了，于是就会相应地产生 DPN_MSGID_TERMINATE_SESSION 和 DPN_MSGID_DESTROY_PLAYER 消息。在 DirectPlay 中间，ACK 数据使用了“捎带”(piggyback)技术，就是和一个正常的数据包一起组合起来传送，以得到更大的网络吞吐量。如果暂时没有数据包可以捎带的话，DirectPlay 就会生成一个特殊的 ACK 数据包把它传送回发送方。虽然 DirectPlay 为双向传输进行了优化，但是它生成特殊 ACK 数据包的速度也相当快（大概是 100 毫秒左右）了，足够防止发送方在进行单向数据传输的时候不必要地进入流量控制状态。

另外一类现在很流行的游戏是巨量多玩家在线角色扮演游戏（massively multiplayer, online role-playing game, MMORPG）。虽然这种游戏与 FPS 有相似之处，但是它们之间的差异也相当的多。一个 MMORPG 游戏要能应付成千的玩家同时在一个可扩展的虚拟环境中进行交互。当前实现这种模式的方法不是将每个玩家的动作都传输到网络上去，而是传输状态信息到网上去。如果你曾经玩过 MMORPG 游戏的话，你可能会发觉在战斗中，其结果并不像在 FPS 中那样取决于自己对键盘鼠标的灵活操作，而是取决于你人物的属性和能力。玩家向服务器发送一个消息，里面包含了他正在进入战斗状态的信息和其当前目标的信息，剩下的就是服务器的工作了。状态信息也可以包含其他种类的向服务器发送的数据，例如玩家的移动。当传输这种信息的时候，玩家需要一个可靠的数据传输。假设玩家的机器显示现在他们正在努力战斗，与一个山中巨人(mountain troll)殊死搏斗，但是服务器却以为玩家正在发呆地站着，任由一个山中巨人把自己撕成碎片，吃得津津有味，无疑这会使某些玩家相当愤怒。同时管理上千个连接也是 MMORPG 游戏的一个大问题。如果使用得当，DirectPlay 会带来很好的扩展性，而且会给开发人员一些方便之处让事情变得更容易一点。一个方便之处就是分组。

分组

分组使得开发人员可以降低数据发送的次数，因为这时候数据可以同时传送到一组玩家，而不是分别向他们重复传送一样的数据。在 MMORPG 中间，整个场景会被划分为多个地域或者地区。这就是一个对玩家分组的绝好方法。游戏可以将一个地区里面的玩家划分为

Download at www.Pin5i.Com

一组，然后向那组玩家发送数据。给一个组添加玩家和删除玩家会带来额外的负担，因此在分组的时候必须小心仔细。在 FPS 中，把玩家根据地区来分组不会有什么好处，因为在一个平均大小的 FPS 地图上一个玩家可能每隔几秒就会从一个地区转移到另一个地区。取代方法是游戏可以利用战斗小组或者其他的方式来进行玩家的划分。下面的代码示范了如何在 DirectPlay 中间添加一个分组：

```
DPN_GROUP_INFO dpnGrpInfo;
DPNHANDLE hAsync;
ZeroMemory(&dpnGrpInfo, sizeof(DPN_GROUP_INFO));
dpnGrpInfo.dwSize        = sizeof(DPN_GROUP_INFO);
dpnGrpInfo.dwInfoFlags   = DPNINFO_NAME;
wcscpy(dpnGrpInfo.pwszName,"ThievesGuild");
pServer->CreateGroup(&dpnGrpInfo,NULL,NULL&hAsync,0);
```

当程序发送一个消息的时候，DirectPlay 会在内部复制一份这个消息，这个复制会带来一个额外的负荷。为了提高性能，程序可以在 Send() 或者 SendTo() 调用中指明 DPNSEND_NOCOPY 标志，这可以禁止 DirectPlay 进行内部复制。这也会带来一个负面影响，那就是这段待发送的数据一直到发送结束的时候都必须保持有效。如果是在函数栈上面给消息分配的内存，开发人员就不能使用异步的 Send() 操作，因为当函数返回的时候包含这段内存的函数栈也将被视为无效。取代方案是使用堆来分配内存，当异步发送结束的时候再释放这段内存。为了得到最佳效果，开发人员可以使用一个缓冲池以代替为每个消息不停地分配和释放内存的策略。如果使用同步方式发送消息则不会产生这些问题，因为此时 Send() 调用只有当消息被发送完毕的时候才会返回。

5.8.3　可重入的回调函数

对于任何程序来说，多线程都是最难应付的问题之一。这不仅仅是因为编写多线程程序很麻烦，而且调试这样的程序也非常令人头痛。同样，由于 DirectPlay 需要大量使用可重入的回调函数，多线程的问题对于开发人员而言也是最大的问题之一。开发人员在编写程序的时候也一定要很小心，不然一个出人意料的线程切换就有可能导致其程序崩溃。当设计程序数据结构（例如链表）的时候，必须要牢牢记住它们有可能会被 2 个不同的线程同时访问。开发人员可以使用引用计数的方法防止资源共享冲突。在程序的回调函数里访问任何一个全局变量或者其他共享变量的时候都要注意应该使用临界区来进行保护。除了注意要防止死锁以外，还要注意等待同步锁或者在回调函数里面进行其他耗时操作带来的负面影响，那就是这些操作会使得 DirectPlay 暂时无法使用线程处理其他的数据，从而导致发送方进入流量控制状态，人为地降低了网络的吞吐量。

为了防止 DirectPlay 线程被拖慢，有个方法就是你可以把接收到的消息放到一个队列里面去，使用一个独立的线程来专门处理它们，这样 DirectPlay 的线程就可以腾出手来作更多的工作了。在处理 DPN_MSGID_RECEIVE 消息的时候，开发人员可以返回 DPNSUCCESS_PENDING 值，这样就可以避免程序把数据复制到队列中去，减少一点操作，因为此时 DirectPlay 会把这块数据的所有权转移到应用程序上面来。如果你想这么做的话，当处理完消息以后，必须要调用 ReturnBuffer() 函数以便 DirectPlay 能释放这些资源。

Download at www.Pin5i.Com

```
case DPN_MSGID_RECEIVE:
{
    PDPNMSG_RECEIVE pRcvMsg;
    pRcvMsg = (PDPNMSG_RECEIVE)pMessage;
    //lock the queue to avoid race condition
    LockMyQueue();
    //pass the hBufferHandle to the queue because
    //you will need that in your
    //call to ReturnBuffer()
    AddToMyQueue(pRcvMsg->pReceiveData,
                 pRcvMsg->hBufferHandle );
    UnlockMyQueue();
    return DPNSUCCESS_PENDING;
}
```

一旦队列处理完了这个消息，就可以调用 ReturnBuffer()函数了：

```
pClient->ReturnBuffer(hBufferHandle,0);
```

如果你的应用程序需要进行扩展以应付大量的连接，只使用一个队列有可能会使程序的性能下降。在这种情况下你可能希望使用更加复杂的方法以减少回调函数里面消息处理的操作。

值得注意的是，不释放 DirectPlay 的接收线程也不总是件坏事。有可能网络部分处理数据的速度会比使用 CPU 的接收程序还要快。如果此时把数据都放在一个队列里面的话，这个队列就会无限增长。解决这个问题的方法之一是设置一个门限值，当队列里面的消息个数超过了这个门限值以后程序就堵塞住 DPN_MSGID_RECEIVE 的回调，当所有的线程都被堵塞住了以后，发送方就会进入流量控制状态了。

开发人员可以调用 SetSPCaps()函数来设置程序线程池的大小。缺省情况下，DirectPlay 会创建 2n+2 个线程来回切换，其中的 n 是系统里面 CPU 的个数。如果系统的 CPU 仍有空闲的时候队列却还在增长的话，开发人员就应该考虑增加线程池的大小了。大多数情况下，程序里面的很多线程可能只是在等待进行磁盘读写或者是等待临界区，因此，开发人员可以创建更多的线程来处理 DirectPlay 的消息而不会导致系统 CPU 的开销过大。另一方面，如果应用程序占据了太多的 CPU 资源，这时候可能就应该减少 DirectPlay 使用的线程数了。当设置 DirectPlay 应用程序里面线程数的时候，有一点要记住，那就是一旦调用了 Host()、Connect()或者 EnumHosts()函数以后，线程池就只能增大不能减小了。在调用这些函数之前，线程池可以为开发人员随心所欲的设置成任何大小。线程池是一个进程里面所有的接口所共享的。开发人员可以调用 GetSPCaps()函数，这样就可以得到 DirectPlay 拥有的可用来处理接收到消息的线程个数了。

```
DPN_SP_CAPS dpnCaps;
pClient->GetSPCaps(&CLSID_DP8SP_TCPIP,&dpnCaps,0);
dpnCaps.dwSize = sizeof(DPN_SP_CAPS);
if(dpnCaps.dwNumThreads < 16)
{
    dpnCaps.dwNumThreads = 16;
}
pClient->SetSPCaps(&CLSID_DP8SP_TCPIP,&dpnCaps,0);
```

请参照图 5.8.1。当一个消息到达的时候，服务提供者层会通知协议层，协议层会通知核

Download at www.Pin5i.Com

心层，核心层则会调用程序设置的回调函数，所有这些操作都是在服务提供者层的线程里面进行的。这样可以减少浪费在线程切换上的时间，使得消息一被接收到以后就能尽快传送到应用程序里去。

DirectPlay 为开发人员提供了一些方法，它们可以帮助开发人员解决多线程问题和回调函数里面的性能问题，其中之一就是标识值。开发人员可以将一个玩家标识值传递给 DirectPlay，然后 DirectPlay 可以通过调用程序相应的回调函数把这个值传递回来。为什么要这么做呢？假设你有一个程序，它有两个用户接口，分别是数据接口和语音接口（在本节稍后你可以看到，一个 DirectPlay 的语音连接需要一个客户机或者一个对等用户接口）。尽管有两个接口，但是你可能希望为它们写一个公用的回调函数，这样当核心层调用程序的回调函数的时候，程序就无法知道新到的消息应该属于哪一个接口。如果在调用 `Initialize()` 函数的时候设置了一个标识值，则此时就可以知道新到的消息应该是谁的了。在一个会话里面，程序可以为每个玩家都设置一个标识值。这个玩家标识值对于优化回调函数的速度来说有很大的帮助。当一个客户机连接到一个会话里面去以后，你就可以通过响应 DPNMSG_INDICATE_CONNET 或者 DPNMSG_CREATE_PLAER 来设置相应的标识值。所有以后那个客户机发过来的消息都会有一个指针，指针指向的就是那个玩家的值。在下面的代码里面，一旦一个玩家加入会话以后程序就为他设置了一个标识值，以后，当从同样的玩家收到了伤害力改变的消息以后，服务器就可以使用相应的玩家标识值作为索引查找到玩家了。这可以节省很多处理的时间。

```
HRESULT WINAPI DPClientMsgProc(PVOID pvUserContext,
                               DWORD dwMessageType,
                               PVOID pMessage)
{
HRESULT hr = S_OK;
switch(dwMessageType)
{
    case DPN_MSGID_CREATE_PLAYER:
    {
        //add player to your list and
        //set their user context value
        PDPNMSG_CREATE_PLAYER pMsg;
        pMsg = (PDPNMSG_CREATE_PLAYER)pMessage;
        MYPLAYER mpPlayer;
        mpPlayer.dpnid = pMsg->dpnidPlayer;
        LockMyPlayerList();
        pAddress = AddPlayerToList(&mpPlayer);
        UnlockMyPlayerList();
        pMsg->pVUserContext = pAddress;
        break;
    }
    case DPN_MSGID_RECEIVE:
    {
        PDPNMSG_RECEIVE pRcvMsg;
        pRcvMsg = (PDPNMSG_RECEIVE)pMessage;
        MYPLAYER * pPlayer = NULL;
        LockMyPlayerList();
```

Download at www.Pin5i.Com

```
            pPlayer = GetPlayerFromList(
                pRcvMsg->pvPlayerContext);
            if(pPlayer == NULL)
            {
                hr = DPNERR_GENERIC;
                UnlockMyPlayerList();
                break;
            }
            MYMSGTYPE * pMsg = NULL;
            pMsg = (MYMSGTYPE*)
            pRcvMsg-> pReceiveData;
            switch( pMsg->dwType )
            {
                case MYMSG_QUADDAMAGE:
                {
                    pPlayer->DamageFactor *= 4;
                    break;
                }
            }
            UnlockMyPlayerList();
            break;
        }
    }
    return hr;
}
```

DirectPlay 给开发人员提供的另外一个好处就是回调函数的串行化。串行化的意思是说 DirectPlay 保证每个玩家的任两个消息都不会同时被不同的线程处理。举一个很好的例子，一个服务器收到了一个玩家要加入一个会话的消息，然而加入会话以后，玩家马上就想退出了。如果在服务器完成玩家的初始化之前 DPN_MSGID_DESTROY_PLAYER 的消息就已经到了的话，服务器就会访问玩家列表里面已经删除了的一个玩家。而 DirectPlay 的回调串行化机制就可以保证这种情况不会发生。

5.8.4 使用 DirectPlay 发送语音

不管是讨论战术、嘲弄敌手或是和朋友聊天，对于任何一个多玩家游戏来说，通信都是一个重要的部分。不幸的是，在玩游戏的时候使用文本方式聊天会让玩家分神，而当去听成百遍事先录制好的语音时，玩家也会觉得索然无味。幸运的是，我们还有其他的选择。如果你使用 DirectPlay 作为网络层的话，你就可以轻而易举的为自己的程序添加上语音通信的功能了，这也只需要多做一点点工作而已。

DirectPlayVoice 接口可以让开发人员使用网络发送语音数据，相应的可以使用 DirectPlayClient 或者 DirectPlayPeer 连接来传输它。开发人员只需要在调用 `Initialize()` 函数的时候注册一个回调函数，这和使用客户机接口的时候是一样的。如果程序已经在 DirectPlay 会话里面为语音数据设置好了传输会话，使用和运行 DirectPlayVoice 就很简单了。下面的代码足够让你为一个语音服务器添加一个会话了：

Download at www.Pin5i.Com

```
//pServer is your IDirectPlay8Server interface pointer
//and must be currently hosting a session
pVoiceSrvr->Initialize(pServer,SrvrProc,NULL,NULL,0);

DVSESSIONDESC dvSesDesc;
ZeroMemory(&dvSesDesc, sizeof(DVSESSIONDESC));
dvSesDesc.dwSize      = sizeof(DVSESSIONDESC);
dvSesDesc.dwFlags     = DVSESSION_NOHOSTMIGRATION;
if(fMixingServer == TRUE)
{
    dvSesDesc.dwSessionType      =
        DVSESSIONTYPE_MIXING;
}
else //Forwarding server
{
    dvSesDesc.dwSessionType      =
        DVSESSIONTYPE_FORWARDING;
}
//this value must be retrieved by a call to
// GetCompressionTypes()
dvSesDesc.guidCT = MyCodecGuid;
dvSesDesc.dwBufferQuality  =
     DVBUFFERQUALITY_DEFAULT;
dvSesDesc.dwBufferAggressiveness =
     DVBUFFERAGGRESSIVENESS_DEFAULT;
pVoiceSrvr ->StartSession(&dvSesDesc, 0);
```

初始化一个语音客户机然后把它连接到一个服务器会话上面也一样简单。下面的代码将为语音客户机进行初始化、连接和设置传输目的地的工作：

```
//pClient is your IDirectPlay8Client interface pointer
//and must currently be joined to a session
pVoiceClient->Initialize(pClient,VoiceProc,NULL,0,0);
DVCLIENTCONFIG dvClientConfig;
ZeroMemory(&dvClientConfig, sizeof(DVCLIENTCONFIG ));
dvClientConfig.dwSize  = sizeof(DVCLIENTCONFIG );
dvClientConfig.dwFlags                =
     DVCLIENTCONFIG_AUTOVOICEACTIVATED |
     DVCLIENTCONFIG_AUTORECORDVOLUME;
dvClientConfig.lPlaybackVolume        =
     DVPLAYBACKVOLUME_DEFAULT;
dvClientConfig.dwBufferQuality        =
     DVBUFFERQUALITY_DEFAULT;
dvClientConfig.dwBufferAggressiveness =
     DVBUFFERAGGRESSIVENESS_DEFAULT;
dvClientConfig.dwThreshold      = DVTHRESHOLD_UNUSED;
dvClientConfig.lRecordVolume    = DVRECORDVOLUME_LAST;
DVSOUNDDEVICECONFIG dvSDConfig;
ZeroMemory(&dvSDConfig, sizeof(DVSOUNDDEVICECONFIG));
dvSDConfig.dwSize = sizeof(DVSOUNDDEVICECONFIG);
```

Download at www.Pin5i.Com

```
dvSDConfig.guidPlaybackDevice   =
    DSDEVID_DefaultVoicePlayback;
dvSDConfig.guidCaptureDevice    =
  . DSDEVID_DefaultVoiceCapture;
dvSDConfig.hwndAppWindow        = hDlg;
pVoiceClient->Connect(&dvSDConfig,&dvClientConfig,0);
DVID dvid = DVID_ALLPLAYERS;
pVoiceClient->SetTransmitTargets(&dvid, 1, 0);
```

DirectPlay 使用 DirectSound 来处理语音，这样就可以让开发人员在语音的接收端进行效果处理，例如让语音变形或者是音唇同步。DirectPlay 同样支持 3D 声效，这可以使得 3D 游戏里面的语音传输得到更真实的效果。

DirectPlayVoice 可以使用三种配置方案来设置：

- 对等用户方案可以让所有的玩家把语音直接送到目的地。这样做的好处在于不需要一个专门的服务器来处理语音数据。这种方案对于玩家比较少的游戏是不错的，但是它几乎没有可扩展性。
- 转发方案是将语音数据先传输到一个中央服务器，然后服务器再将数据传送到接收玩家去。使用这种方案的话，一个客户机只需要一次性的把数据和所有希望其接收的目标列表传送出去就可以了，减少了需要的带宽。但是在服务器端来说带宽要求就会高得多了，因为它需要向不同的玩家发送数个同样的语音消息。
- 第三个方案是混合服务器。如果你有专门的硬件设备可以对收到的语音数据作混合处理的话，这就是一个最佳方案了。和转发服务器类似，混合服务器也允许客户机把语音数据和接收方列表一起进行传输，但是它们的不同之处在于混合服务器会把所有传向一个接收方的数据组合起来作为一个音频流发送出去。这减少了客户机和服务器的带宽要求，但是需要更强的处理能力。

如果你想使用对等用户方案传输语音数据，你必须要现有一个对等用户的会话。不过你并不需要使用同一个会话来传输游戏的数据。图 5.8.2 显示了两种可能的使用了语音的客户机/服务器架构的游戏：一个使用的是对等用户方案；另一个则使用了独立的语音转发或者语音混合方案。

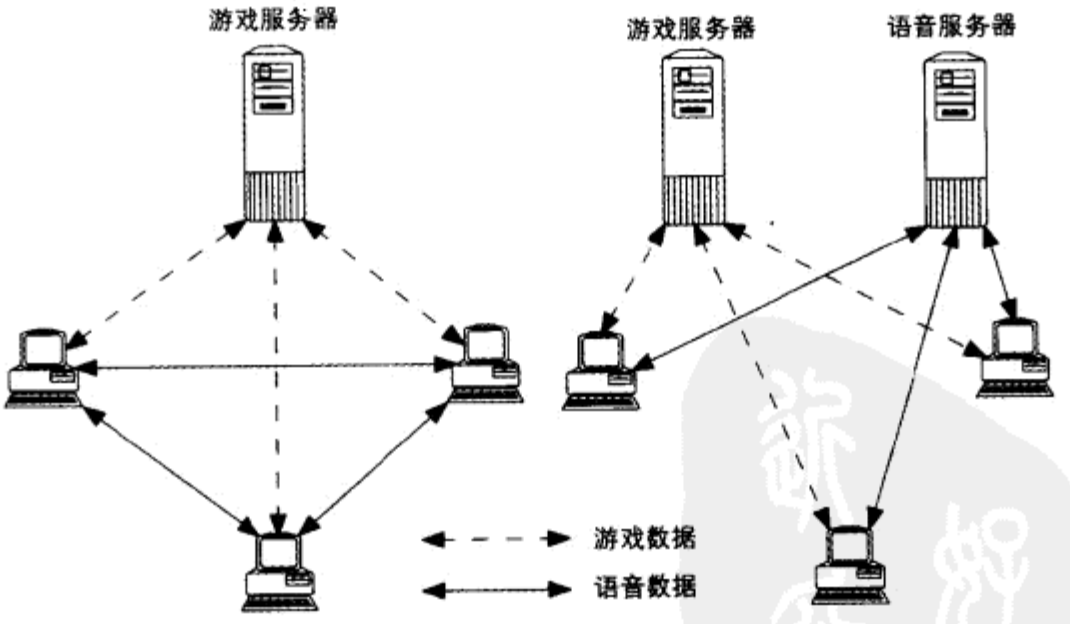


图 5.8.2 左：对等语音方案；右：客户端/服务器语音方案

Download at www.Pin5i.com

5.8.5　相关资源

如果你使用 DirectPlay 时碰上了什么困难的话，也许其他人也会遇到同样的麻烦。在因特网上有很多相关的资源可以为 DirectPlay 网络编程提供帮助，其中最著名的是 microsoft.public.directx.networking 和 microsoft.public.win32.programmer.directx.networking 的 Web 新闻组。DirectPlay 开发小组的人和有经验的 DirectPlay API 用户经常会光顾这些新闻组，因此它们对你的游戏开发来说将是一个非常有用的资源。

Download at www.Pin5i.Com

5.9 使用 Java 微型版开发无线游戏

David Fox
Next Game
davidfox@ureach.com

在一个拥有超大屏幕、有着具有照片效果的怪兽的丰富的 3D 世界中，为无线设备编写游戏看起来好像太没劲了：屏幕太小了——对大多数移动电话来说小于 100×100；可使用的内存太小了——对大部分电话而言只有一个 32kB 的堆；网络带宽也太小，大部分第二代无线网络是以 9.6kbit/s（千位每秒）的速度来发送数据的；而且无线设备上的处理器则要比一般的桌面计算机的处理器几乎要慢上百倍。

从好的方面来看，现在几乎人人都携带着某种具有网络功能的设备了。截止 2001 年底，在世界范围内有超过 6 亿的手机用户。如果一个迷人的无线游戏风行了起来，它的用户群将是巨大的。

虽然从表面上看，要在一个微型屏幕上得到一个丰富而有意义的表现形式是一个很愚蠢的想法，但是移动电话游戏却有一个即使是最好的显示也无法企及的优点：它们总是与你同在，而且可以在任何你地方玩。另外，很多无线网络实现了某种形式的移动定位。当这个功能更加普及的时候，游戏就可以精确地跟踪现实世界中玩家的行踪了。无线游戏不仅方便携带，而且只要再加上一些聪明的设计，它就可以提供一个与现实世界无缝集成的虚拟世界，从而带来全新的游戏体验。

另一个崭露头角的领域是为现在的游戏创建其无线版本。例如，你可以为一个通常在大屏幕上具有全视觉效果的多玩家游戏世界加入一个移动界面。一个移动玩家可以随时得到进攻的通知，在会议间偷偷地登录进去以改变设置或是作出重要决策。

其困难就在于如何在给定的极其有限的资源下得到合理的网络交互。

5.9.1 网络特性

延迟就是一个数据包从某一点到另一点所花费的时间，它通常是取决于两点间的距离以及它们之间的跳[1]数。带宽就是一秒内可以被发送出去的数据量，它通常取决于用来传输数据用的物理硬件。

多玩家游戏开发人员总在与因特网的高延迟与一般家庭用户的低带宽

译者注[1] 跳（hop）指的是在整个网络传输路径中一个网络节点到另一个网络节点之间的一次传输，例如在路径上个一个路由器和一个主机之间的一次数据转发。

Download at www.Pin5i.Com

做持续的搏斗。不过，与第二代（2G）无线网络比起来，因特网已经是快得吓人了。

欧洲移动电话主要使用的是移动通信全球系统（GSM），其发送数据的速度为 9.6kbit/s。而在北美、南美、俄罗斯、以色列、东亚以及中非的大部分地区使用的则是码分多址（CDMA）标准，其运行带宽为较高的 14.4kbit/s。某些网络使用时分多址（TDMA），它与 GSM 很相似，其上限也为 9.6kbit/s。

在日本、香港、韩国以及新加坡的运营商使用的是被称为 2.5 代（2.5G）的网络，其方式是使用一种 CDMA 的变体（IS-95B），它可以达到 64kbit/s 的数据传输速率。在美国，CDMA2000 已经在一些试验区开始实施，最终想要达到高达 144kbit/s 的速度。此外，大多数 GSM 网络正在使用通用组包无线业务（GPRS）进行升级，其理论上限为 170kbit/s。

第三代（3G）网络正在逐渐形成中。大多数 3G 标准都是建构在因特网协议（IP）服务上的，这样可以得到高速的移动访问，其计划是要逐渐地对网络进行升级，从 2.5G 技术开始逐渐达到 300kbit/s 的速度，最终得到 2400kbit/s（2.4Mbit/s）的速率，此速率甚至要比当今的家庭 DSL 连接的速度还要快。然而，3G 的实施将极其昂贵，而且会面临很多挑战。我们不可能期望在不远的未来看到 3G 网络的普及。

广域无线网也具有很高的延迟，这是由无线电波通信本身所固有的干扰和噪音所致。大部分无线网络都要求数据包要经过很多高延迟的路由器。基于卫星的无线网络一般说来会造成更大的延迟。如果网络延迟有 1～2s，这是很正常的。

那么，在开始的时候，无线游戏开发人员就应该意识到他们是在一个完全高延迟、低带宽的连接上工作的。

5.9.2　Java 微型版

Java（Sun Microsystems）正在成为无线设备的标准语言。Java 是运行在一个虚拟机上的，这就意味着只要开发人员遵循正确的流程，那么同样的 Java 字节码就可以运行在任何支持的平台上。Java 也特别容易使用。它是面向对象的，没有指针、没有复杂的内存管理，具有自动的垃圾收集功能。最重要的是，Java 小应用是无法访问到其安全“沙箱”以外的任何功能或是内存的，这就意味着使用它基本不可能写出恶意的代码或是病毒。

Java2 微型版（J2ME）[J2ME01]的目标就是要抽取出标准 Java 中最有价值的方面，为了更小的设备将其剥离出来，这些设备包括移动电话、传呼机以及手持个人助理。最主要的几个移动电话制造商都加入了 Sun 的阵营以建构 CLDC（面向连接的受限设备配置）[CLDC]和 MIDP（移动信息设备框架）[MIDP]。这样,一个为移动电话写的 Java 小应用就被称之为“MIDlet”了。

各个设备制造商为 J2ME 发布了不同的扩展 API。例如，Siemens 有一个建构在 MIDP 之上的游戏扩展 API。NTT DoCoMo 并不使用 MIDP，但是它有一个单独的叫做“I-Appli”的 Java 框架。

Qualcomm 创建了一个称为针对无线设备的二进制运行时环境（BREW）的虚拟机和语言，其基础是 C++。Qualcomm 已经把 BREW 嵌入了为 CDMA 准备的芯片组中去了。虽然在某种意义上来说 BREW 是 J2ME 的一个竞争者，但是你也可以用 BREW 来编写一个 Java

Download at www.Pin5i.Com

的虚拟机。

移动电话制造商正在热情地拥抱 Java，其程度比起 PC 制造商是有过之而无不及。Java 正在成为移动设备的备选平台。

5.9.3 J2ME 网络精髓

在 Java 标准版的世界里面，庞大而复杂的 java.io.*与 java.net.*的包使用得很广。这些包包含了几乎所有你所需要的网络类，例如 Socket、DatagramSocket 或者 ServerSocket。每个类都有不同的方法和不同的使用方法。

然而，在 J2ME 的世界里面，你可没有这么全的宝藏了。对新手而言，我们不知道究竟一个电话使用的是什么网络传输协议。对于在电路交换网络中工作的设备来说，它使用的是基于流的不间断连接，例如传输控制协议（TCP）。然而，包交换的网络可能只能处理分离的、无保证的数据包这样的网络数据，使用的协议则是用户数据包协议（UDP）。

CLDC 中的 Connection 就是要尽力提供一个尽可能通用的接口。Connection 类是一个包罗万象的类，它在理论上来说是可以处理任何网络连接的。而一个特殊的 Connector 类则可以加入任何一个想实现 Connection 接口的 CLDC 类里面去。

每个 Connector 的 open 方法都接受一个具有形式“protocol:address:parameters”的字符串作为参数。例如，若想打开一个典型的 HTTP 连接：

```
Connector.open("http://java.sun.com/developer");
```

要打开一个套接字：

```
Connector.open("socket://123.123.111.000:9000");
```

打开一个数据报连接：

```
Connector.open("datagram://www.myserver.com:9000");
```

接下来你就可以创建一个数据报，然后如下发送出去了：

```
Datagram dgram = dc.newDatagram(message, msglength,

    "datagram://www.myserver.com:9000");
dc.send(dgram);
dc.close();
```

当然了，其远程的“服务器”也可以是另一个移动设备。大部分无线设备可以很容易地通过对等用户的方式与其他设备进行通信。对等用户的延迟要更低一点——因为此处不需要中间人。然而，在对等用户游戏中，其中的某个用户会充当服务器的角色，其他的则是客户机。而成为服务器所需要的额外代码使得这种模式无法实行。为了让一个 MIDlet 能处理收到的数据流，只要创建一个无限循环，监听一个端口，等待数据的到来就可以了：

```
DatagramConnection dc = (DatagramConnection)Connector.open
    ("datagram://:"+receiveport);
```

Download at www.Pin5i.Com

```
while (true)
{
    dgram = dc.newDatagram(dc.getMaximumLength());
    dc.receive(dgram);
    reply = new String(dgram.getData(), 0,
  dgram.getLength());
      }
```

由于数据报协议是一个标准协议，我们可以用 Java 标准版（或是任何其他的语言）编写一个非常简单的服务器组件在任何一台 PC 上运行，例如：

```
DatagramSocket receiveSocket = null;
DatagramPacket receivePacket = new DatagramPacket(bytesReceived,
  bytesReceived.length);
receiveSocket.receive(receivePacket);
```

正如你所看到的，有了 MIDP 规范，发送数据和接收数据就方便得多了。不幸的是，很多无线设备都不支持数据报。如果在你使用的设备上不支持相应的协议，那么你就会得到一个 ConnectionNotFoundException 的异常了。

MIDP 设备必须支持的唯一协议是 HTTP 协议。因此，我们推荐你在设计无线游戏的时候至少要向最低标准看齐——使用 HTTP。

5.9.4　HTTP 的限制

HTTP 是一个受到广泛支持的协议，而设计 Java servlet 或是其他的 Web 技术来处理 HTTP 是极其容易的事情[Fox01]。此外，HTTP 是建构在 TCP 之上的，因此你决不会遇到乱序或者丢包的问题，你也不需要使用额外的代码来编写自己的 MIDlet。

然而，HTTP 的简单性也是其弱点所在。HTTP 是一个请求—响应协议，在开始设计时它是用来传输静态内容的。首先建立一个 TCP 连接，然后一个浏览器要求得到一个文档（请求），一个 Web 服务器提交此文档（响应），最后连接关闭。HTTP1.1 版稍有改进，可以允许传输任意大小的数据块，也可以让连接保持激活状态。但是在很大意义上，HTTP 仍然是一个半双工的协议——你不可能同时进行双向的传输。

如果你的游戏是一个简单的回合制游戏，那么可能根本不需要全双工，延迟和带宽也不会造成严重的影响。一次只能有一个玩家采取行动，当一个玩家行动完毕之后，新的游戏状态会被送给所有的玩家，然后下一个玩家就可以行动了。

然而，如果你的游戏比较高级，经常有不可预测的数据包的到来，那就麻烦了。而当你希望基于 UDP 来创建一个复杂的、具有完整特性的游戏服务器的时候，那又怎么办呢？

1. 使用多个连接

如果要进行全双工通信，一个方法就是让你的 MIDlet 客户机创建多个连接——一个用来向服务器发数据，另一个用来保持激活状态，准备收数据。第一个连接应该使用成块数据传输。服务器创建一个打开的连接，给它赋一个唯一的 ID，然后此 ID 将被送给客户机。

Download at www.Pin5i.com

现在客户机就可以创建一个非常大的 Content-length[1]的客户机—服务器的连接了，其中带有相应的 ID。接下来服务器将对客户机发过来的请求数据进行处理，向那个打开的服务器—客户机连接发送相应的响应数据。

某些服务器或是代理可能不能处理一个超长内容长度的请求，而且可能会暂时将之缓存起来等待所有的报文结束再一起传输。在这种情况下，客户机可以把每个消息分成小块作为单独的请求进行传输。客户机应该在每个请求里面加入其 ID，可以将之作为一个用户定义的头部元素，也可以将之作为载荷的一部分。然后服务器就可以解析出 ID 数值，把相应的响应送还给客户机了。

2. 代理的能力

创建健壮的无线通信的另一个方法就是使用任何你希望使用的协议来编写游戏服务器，然后通过一个 HTTP 代理提供游戏的服务。此系统就相当简单了——MIDlet 与一个代理进行通信，而不是直接与游戏服务器通信。当然，它将带来额外的延迟。不过，通过这个方法，你可以为支持更好的网络协议的设备提供更好的游戏体验。例如，支持数据报的设备可以尝试着直接和你的游戏服务器进行连接。如果抛出了一个 `ConnectionNotFoundException` 的异常，则它可以再去通过代理使用 HTTP。

5.9.5 优化数据包

那么我们究竟应该怎样使用 HTTP 在一个高延迟、低带宽的网络上创建有意义的游戏数据呢？实际上，这种创建无线游戏的挑战与任何其他多玩家游戏面临的挑战是一样的，数据包必须尽可能的小，而且根据网络的限制设计游戏本身也是一个挑战。

在基于 TCP/IP 进行设计的时候，你需要考虑到很多网络开发的细节和基本规则。不管如何，这些细节在无线游戏里面都可以忽略——延迟和带宽才是重中之重。

1. 更好的延迟

下面给出的游戏场景描述了一个与延迟相关联的基本问题：老鼠 A 和老鼠 B 同时看到了一片奶酪，两者都疯狂地向它冲了过去。由于需要至少 1 秒才会使得每个玩家注意到另一者的位置，因此两个老鼠都认为自己将第一个得到美餐。

有很多技术可以用来解决此问题。位置推测法[Aronson01]就是用来推断和猜测一个游戏人物运动的方法。我们也可以通过在每一步锁帧来对付游戏的延迟问题[Bettner01]。然而，这两个技术都需要每个客户机对游戏中的每个实体都保有详细的仿真信息。它要求的内存可能比无线设备有的要多得多。

有可能你需要在游戏服务器上对接收的包进行延迟的测量，然后进行相应的处理。一个典型的多玩家射击类游戏服务器大概每秒发送 10 帧的数据（fps）。这对于一个无线游戏来说几乎是不可能的。由于无线设备上的动画频率比较低（一般只有 10fps），考虑到所有方面都

译者注[1] content-length 是 HTTP 包头里面的一个域，它指明了包里纯内容的长度。对于 HTTP1.1 来说，它可以很长，这样有利于实现大量的数据交换，可以参见本文的上一个小节。但是 HTTP1.0 里面的不应该很长，因此下文说到了某些服务器是不能处理超长内容的。感兴趣的读者可以参见 http://www.ietf.org/rfc/rfc2616.txt（HTTP1.1）中的 3.6 小节。

Download at www.Pin5i.com

比较慢，因此网络速率只要有 2fps 到 3fps 就可以了。

2. 更好的带宽

带宽的重要性一般来说没有延迟大，因为游戏的数据包通常都很小。最重要的准则就是，和以前不一样，每个位都要仔细考虑。

你不能每帧都发送一遍数据，最好是把信息进行增量压缩，只传输变化了的数据。此外，MIDP 不支持浮点运算。大多数涉及到图形的游戏都使用长整型来代替浮点运算。你可以尽可能地将之缩小并简化到 8 位或是 16 位。最简单的数据压缩方法就是字节压缩了——不要使用一整个字节来传输一个布尔值，而应该对每个位进行操作，一次性地传输 8 个标志值。

此外还存在着更先进的数据压缩方法，能将大个的数据包压缩至原大小的 70%或更小。其中就有 gzip。不过，压缩数据包的缺点是会使本来已经很高的延迟变得更高了，因为需要时间来进行压缩与解压。而处理压缩的代码可能会占用过多的 MIDlet 空间。因此，在大多数情况下，只要把数据压缩到字节就够了。

游戏服务器应该对消息进行优先级的分类。某些消息对于游戏的正常运行起着至关重要的作用。其他的消息，例如聊天消息或是计分更新，则没有这么重要。如果一条消息不是和当前的游戏场景紧密相关的，你就不应该将它们一起发送。

把消息符号化，越简单越好。换句话来说，千万不要发送“你的坦克被击中了!”这样的消息，而要发送一个单字节的宏，它可以映射到客户机上一个硬编码的语句列表里面去。

5.9.6　从服务器获取图像

MIDlet 版本 1 只能显示 PNG 文件格式的图像。不幸的是，没有方法可以直接从网络上得到一个图像。事实上，createImage 方法只接受两种类型的参数：一个（文件名的）字符串或是一个字节数组：

```
createImage(byte[] imageData, int imageOffset, int imageLength);
```

使用 MIDP 获取图像的方法就是要把一个图像的二进制内容给下载回来，然后将此数组直接传给 createImage 方法：

```
public Image grabImge(String url)
{
    InputStream is = null;
    HttpConnection hc = null;
    Image img = null;
    try {
    hc = (HttpConnection)Connector.open(url);
    // If we've got a connection...
    if (hc.getResponseCode() ==
        HttpConnection.HTTP_OK) {
        // Open the stream
        is = hc.openInputStream();
        // How big is the file?
        int len = (int)hc.getLength();
```

Download at www.Pin5i.com

```
            // Create a byte array that size
            byte[] data = new byte[len];
            // Read in the file
            int actual = is.read(data);
            // Create an image from the raw data
            img = Image.createImage(data, 0, len);
            }
        } catch (Exception e) {
            System.out.println("IO Exception+"+e);
        } finally {
            if (is != null) {
            try {
               is.close();
            }
            catch (Exception e) { }
            }
            if (c != null)
            {
            try {
                c.close();
            }
            catch (Exception e) { }
            }
        }
        return img;
    }
```

如果你希望图像能在各种移动设备上都尽量美观，那么我们推荐你创建一个图像服务器。这样一来，你的游戏就可以把设备的屏幕大小、颜色能力以及其他的参数发送到服务器去，图像服务器就可以将大幅的彩色图像实时地转化为微型的灰度 PNG 图。Java 高级图像 API[JaveImage01]和其他类似的包可以为你作大部分的工作。

5.9.7 结论

无线网络面临的最大挑战不是网络限制本身，而是要在有限的设备空间、速度、内存以及高级的网络技术之间进行权衡。虽然从理论上来讲，使用位置推测法、锁帧、压缩数据包以及其他的技术是很有效的，但是很多移动设备只能允许你的字节码占用 10kB 到 50kB 的空间。要把游戏代码和网络代码都塞到这么一个小盒子里面去通常是一个痛苦的经历。

聪明地设计游戏以适应设备的最上限，这通常是最好的选择了。例如，你可以在一个战争游戏里面使用坦克来代替士兵，因为坦克移动得比较慢，只能偶尔发一炮，而且还需要很长的时间来转向。当然了，使用这种方法，你也只能做到这样了。总的来说，某些类型的游戏就是不行的，除非有了更好的无线网络并出现了更好的设备。

不过一旦解决了这些挑战，其结果也将物有所值。无线设备提供了时时在线的网络和方便携带的交互性。考虑到手持设备的普及和其将来可能拥有的史无前例的广大用户，一个真正具有原创性的无线游戏可能会给娱乐界带来一场伟大的革命。

Download at www.Pin5i.com

5.9.8　参考文献

[Aronson01] Aronson, Jesse, “Dead Reckoning: Latency Hiding for Networked Games,” *Gamasutra*, 对应网址为 http://www.gamasutra.com/features/19970919/aronson_01.htm, September 19, 1997.

[Bettner01] Bettner, Paul and Mark Terrano, “1500 Archers on a 28.8: Network Programming in Age of Empires and Beyond,” Game Developer’s Conference Proceedings, 2001. 对应网址为 http://www.gdconference.com/archives/proceedings/2001/terrano_1500arch.doc.

[CLDC01] Sun.com “CLDC Information Page,” 对应网址为 http://java.sun.com/products/cldc/.

[Fox01] Fox, David, “Creating Games Using J2ME,” *Gamasutra*. 对应网址为 http://www.gamasutra.com/ resource_guide/ 20010917/fox_01.htm, September 17, 2001.

[J2ME01] Sun.com, “J2ME Information Page,” 对应网址为 http://java.sun.com/j2me/.

[JavaImage01] Sun.com, “Java Advanced Imaging API,” 对应网址为 http://java.sun.com/products/ java- media/jai/.

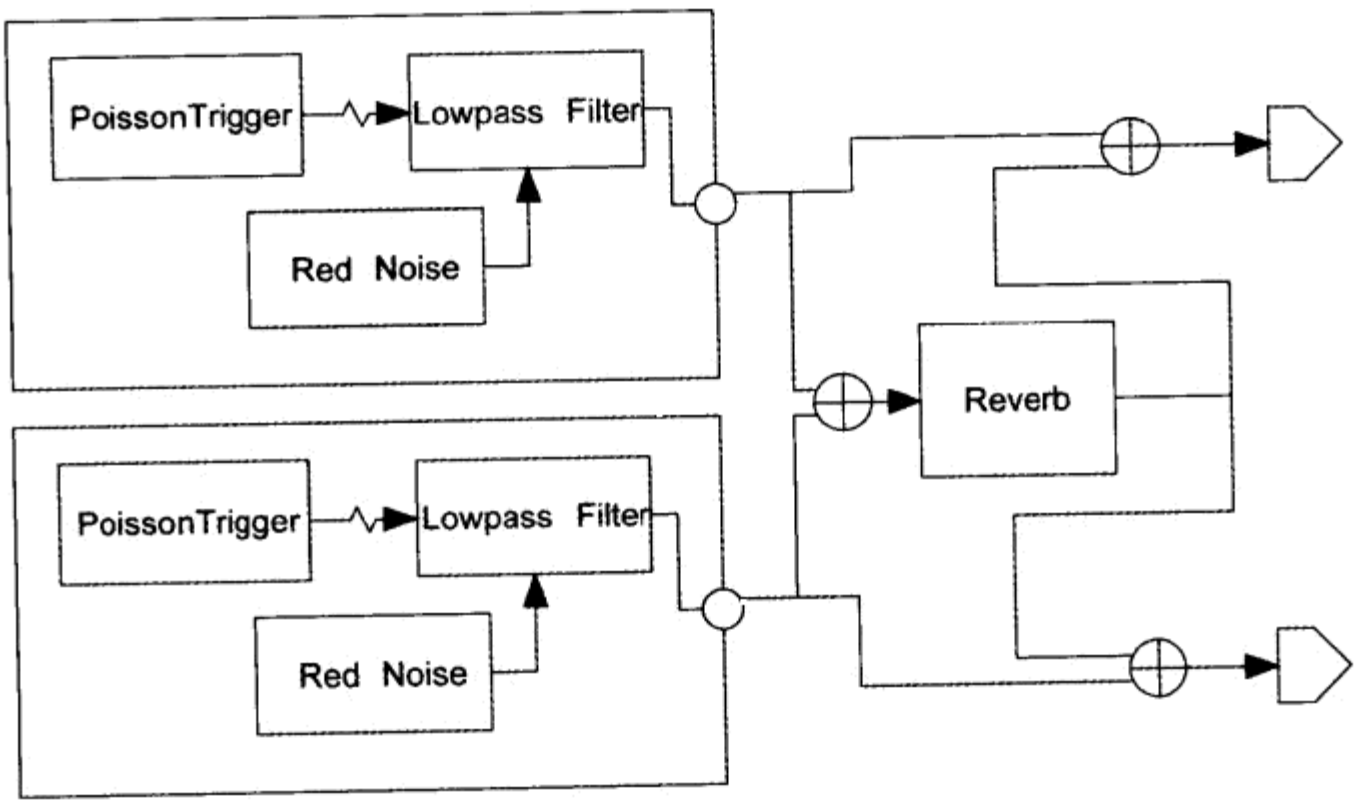
[MIDP01] Sun.com, “MIDP Information Page,” 对应网址为 http://java.sun.com/ products/ midp/.

Download at www.Pin5i.Com

Download at www.Pin5i.Com

第 6 章

音频处理



Download at www.Pin5i.Com

简介

Scott Patterson
Next Generation Entertainment
scottp@tonebyte.com

为游戏编写音频部分的程序与一般的游戏编程非常相似。例如，游戏的设计与目标平台将决定开发的决策。对于音频开发人员来说，首先开发出基本功能，然后对之进行增强也是很典型的。游戏的设计在决定音频中使用何种增强效果是最重要的，而目标平台则决定了何种增强是可行的。在今天，游戏音频的计算机处理能力使得很多增强效果都成为了现实。在本章中，我们的任务就是要推荐这些增强效果中的思想与实现。

在游戏中通过压缩技术以得到更多的内容是很常见的，而高质量的音频压缩算法直到现在都非常少见。Jack Moffitt 在他的文章《使用 Ogg Vorbis 进行音频压缩》中给我们介绍了心理声学压缩。Ogg Vorbis 听上去好像是一个科幻小说中的人物，但实际上它是一个开源的、无版权的、通用的音频编解码器。Jack 描述了游戏中典型的音频压缩过程，并且演示了如何使用 Ogg Vorbis 的 API。

很多游戏平台现在都包含了处理 3D 音效的能力。Garin Heibert 在他的文章《创建一个美妙的 3D 音频环境》中给我们介绍了一些 3D 音频处理的实际经验。他详细地阐述了 OpenAL 与 DirectSound3D 的 API 编程技术。在《使用轴对齐的边界框设置音障》一文中，Carlo Vogelsang 提供了一个有效的技术以用来计算出一个障碍物在声源与听者之间会造成多大的路径偏差。

在游戏中，我们可以使用实时数字音频处理技术得到更多的声音回放。在《使用双二次共振滤波器》一文中，Phil Burk 介绍了一种有用且通用的滤波器。此外，他还阐述了滤波器实现中的重要环节，例如改变算法或是对浮点数值进行反归一化。在《语音压缩与音效的线性预测编码》中，Eddie Edwards 阐述了如何使用一种称为声音编码器的特效来改变人物的嗓音，此外还可以同时得到可观的压缩比。

通过自动生成技术，我们可以得到实时的控制与复杂的音频响应。Phil Burk 在他的文章《复杂声音的随机合成方法》中展示了如何使用随机合成技术自动生成交互式的声音。在 Frank Luch 的文章《针对游戏的实时模块化音频处理》中，他介绍了一个配置系统以用来自动生成与处理音频信号。通过使用这样的系统，我们就可以在游戏音频未来的辉煌世界中灵活地生

Download at www.Pin5i.com

成与处理音频，以得到更美妙、更吸引人的音响效果。

一旦你的小金库里有了这些技术，你就可以对自己游戏中的音频部分加入更多的体验、增强效果以及绚烂的特性了。你可以在同样的空间里得到更多的采样；你的生物可以发出空间中动态的声音，可以让它听起来就如同从墙角边拐弯过来的一样；你的机器人说话的时候会使用具有机械效果的声带；你的引擎将会叮当作响、牢骚不断，还会使用更复杂的方式与响应咆哮起来；即使是风也会在新系统里面呼啸得面目一新。如果你能将这些创新带入游戏的音频世界，你也就同时把这些新意给带入整个游戏之中去了。

Download at www.Pin5i.Com

6.1 使用 Ogg Vorbis 进行音频压缩

Jack Moffitt
Xiph.org Foundation
jack@xiph.org

自从计算机可以发声的那一刻起，声音，不管是以什么形式出现，就成为了游戏的一部分。在最开始的时候，可用的只有主板喇叭的嘀嘟声。接下来就可以使用 MIDI 了，它可以在很小的空间里面包含很多的声音信息。然后，随着声卡成为主流，MOD 类型的文件更加流行了，它提供了具有序列化和有限音效功能的声音采样。一旦 CD-ROM 成为了游戏发布的标准介质，Redbook 音效（正常的 CD 音效）就开始流行。最终游戏使用的是高品质音乐。

不幸的是，随着音乐效果的增强，音乐文件也增大了。尽管 CD 可以很容易地存放一个小时的高品质音效，但是此时也没有什么多余的空间了。ADPCM 是一个在商业游戏中使用的压缩标准，它希望能减小音乐文件的尺寸，同时保留原有的音质，但是它节省的空间实在不大。高品质音乐现在对于任何游戏而言都是不可或缺的，而由于游戏开发人员希望继续使用原有的方式在游戏里面包含音效和音乐，因此音乐文件必须再变小一点。

下面，让我们开始探讨心理声学压缩。

6.1.1 心理声学压缩

心理声学压缩的原理就是要消除所有人耳听不到或是影响很小的那部分音效。与其他的压缩形式不同，心理声学压缩是有损压缩。虽然对于大多数耳朵而言它们没什么不同，但实际上原始信号被不可逆地改变了。

心理声学压缩的压缩比一般在 10:1 和 20:1 之间。这比其他声音压缩的方法要高得多。

1. 它是如何工作的

心理声学有几个基本原理。首先，存在一个听力的绝对门限值，这可以用图 6.1.1 中的图表表示出来。

这个听力的绝对门限值表示的是对于一个给定范围的频率，一个声音究竟要有多大人耳才能听见。任何一个在曲线之下的声音都是听不到的。如果你从原始声音中把曲线之下的所有声音都给拿掉，其得到的声音听起来也应该是一样的。

Download at www.Pin5i.Com

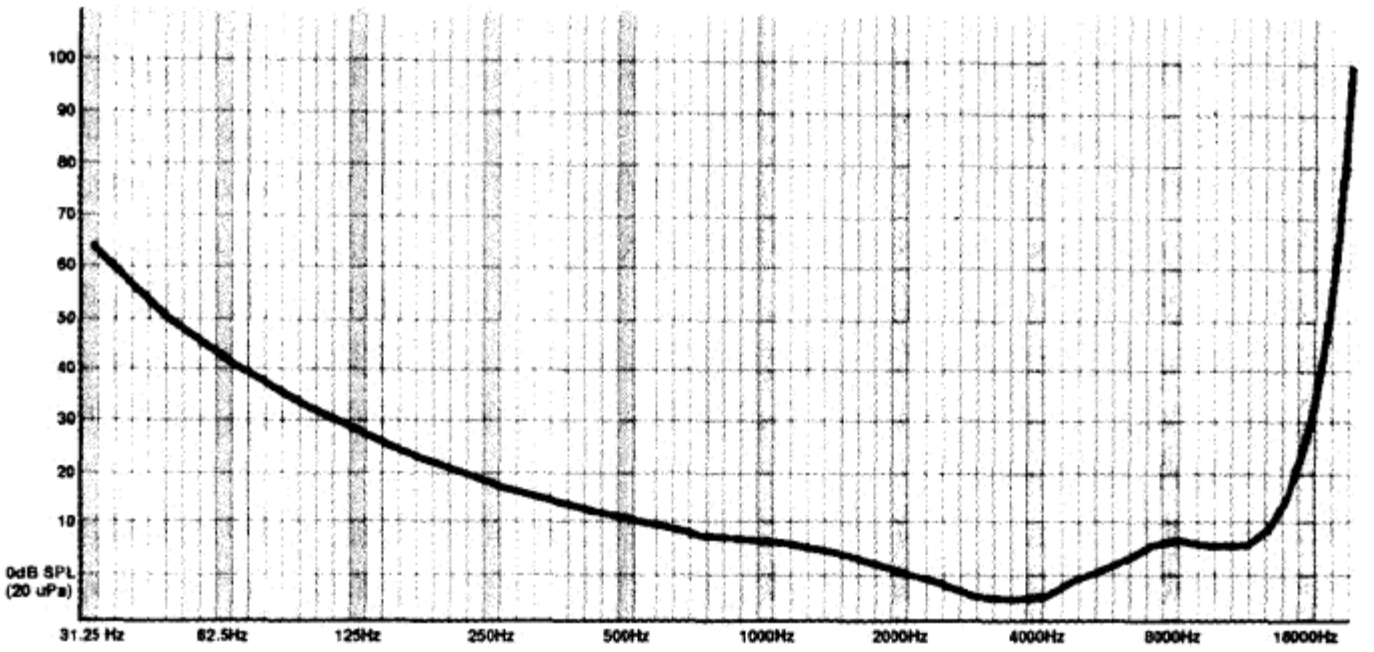


图 6.1.1　听觉的绝对门限值。X 轴是频率，Y 轴是音量

心理声学的另一个基本概念就是掩盖（mask）。正如一个明亮背景光源会掩盖住前景对象一样，较大的声音也会掩盖较小的声音。一个强音会掩盖住频率较高和较低范围内的弱音。即使一个强音消失了，它仍然会掩盖住一些声音，这是由暂时掩盖引起的。由于相当多的声音都被掩盖住了，因此心理声学编码器就不会对之进行编码了，而人耳也不会察觉到。

2. 如何比较音频压缩技术

现在既然我们已经对心理声学压缩的工作原理有了一个基本的了解，那么接下来再让我们将之与其他更为熟悉的音频压缩方法进行一下比较。音频一般是以原始的 PCM 采样形式传输的。它是未经压缩的，因此每个立体声 16 位采样都要耗费 4 个字节。对于 CD 音质的音频来说，每秒有 44 100 个采样，或者说每分钟有 10 MB 的音频数据。

一般用来减少音频数据的方法包括向下混音与重采样。向下混音把立体声频道减少至一个频道，将整个数据量减了一半。重采样在每秒使用了更少的采样以表现声音。一般的重采样率是 22 050Hz 和 11 025Hz。这两种方法都将大大减少文件的尺寸，但是其代价是降低了音质，牺牲了立体声。

ADPCM 是一个 4:1 压缩比的方法，它是对连续采样之间的不同点进行压缩。有时候在游戏中会使用它以减少音频尺寸，同时也避免了向下混音和重采样带来的音质的损失。不过，ADPCM 也是有损压缩，会降低音频质量，而且它只能将音频压缩到原来的 1/4，这就意味着一个小时的音频可能仍然还有几乎 200MB。

另外还有一些无损压缩方法。虽然它们不会损失任何音质，但是其压缩比则不怎么理想。一般的无损压缩方法在通常情况下得到的是 1.5:1 至 2:1 之间的压缩比，其中的很多可能会很低。就它们的压缩比这一点来看，在大部分场合就不适合使用了。

3. 为何使用 Ogg Vorbis

心理声学压缩看起来似乎是音乐压缩的唯一选择了。高压缩比和几乎不可察觉的质量损失使得它具有相当的吸引力，但是问题仍然存在——应该在广泛使用的编解码器中使用哪一

Download at www.Pin5i.Com

个呢？

Ogg Vorbis[OggVorbis02]对于游戏和其他应用程序而言是最佳选择。它的免费版权使其成为一个比其他竞争编解码器都便宜的候选对象。它的开源实现给开发人员在必需的时候可以定制和修改的自由。此外，它广泛的平台支持确保了同样的代码在 Windows、Machintosh(传统的，还有 OS X)、Linux、BeOS 以及甚至是游戏机平台上面都可以正常运行。

除了这些优点以外，Ogg 还有另外几个特性，使得它对游戏开发人员特别有吸引力。它支持将音频数据与其他类型的信息一起进行调制。已经有人为同步设备控制实现了 MIDI 和 CD 音质的混合效果了。它另外还支持多通道音效，这意味着你可以编码得到并回放环绕立体声或是更高级的一些音响效果。

例如，通过使用 Ogg，一个游戏开发人员可以为压缩的嗓音加上同步的嘴唇运动，这样在说话和唱歌中就可以得到更加真实的感受了。对于其他的方法来说，这就很困难，而且没有直接的支持方法。

如果游戏开发人员希望能紧跟当前流行形式，要求音频和声音不仅要紧凑，而且要很灵活。这就是其他的很多编解码器的弱点，而正是 Ogg Vorbis 的强处了。

6.1.2 使用压缩的情况

通过使用 Ogg Vorbis 作为实现的选择，我们下面就可以讨论如何最好地将压缩的音乐加入到游戏里面，同时还要考虑到几个速度方面的要求和使用的音频类型的可能性了。虽然主要将讨论 Ogg Vorbis，但是此处展示的思想对其他的编解码器也应该是适用的。

当在一个游戏里面使用一个编解码器的时候，我们需要考虑好几件事情。大多数游戏对速度的要求都比较高，因此解码音频所需占用处理器的时间数量就是一个很重要的因素。对于奔腾 III 系列的机器来说，桌面计算机一般进行心理声学压缩音频的解码需要占用总处理器时间的 2%到 5%。

另外，需要解码的声音类型也是同样重要的。例如，背景音乐一般只有一个立体声声道，它将在两个声道中来回切换；更复杂的音乐可能需要将声道叠加起来，要求同时对几个文件进行解码；音效一般是比较短的声音，它与事件关联在一起；在事件发生与声音播放之间的时间差是非常重要的；可能同时需要播放多个音效。

在考虑到这些条件的情况下，我们将规划出一些用来将压缩音频有效地集成到游戏中的方法。

1. 情况 1：实时解码

在一个游戏中使用 Ogg 的最常用的方法就是实时地对音频进行解码。这将占用一点 CPU 的时间，但是它就可以让文件保存在一个大容量存储设备上，例如一个硬盘上。

在此情况下，一个音频流将完全编码成为 Ogg 格式，然后在游戏运行的同时被实时地解码和播放出来。用户不需要等待整个文件解压完毕了以后才播放，而且在玩家的硬盘上也不会出现大个的未压缩音频文件。此外，如果使用这种集成方式，玩家也很容易在游戏中添加他或她自己的音乐。

Download at www.Pin5i.Com

2. 情况 2：解码到缓存

在某些情况下，把解码的结果输出到缓存会更加合适。如果需要同时播放多个音乐，那么把它们都实时地解码出来可能是不现实的，或是会占用太多的处理器时间。在这种情况下，当进入新一关或是在启动的时候可以把文件先解码到缓存去。这样，在实现游戏的时候仍然可以让所有的音频数据保持压缩，而不用在玩游戏的时候即刻解码出来了。

这也是一种产生音效的好方法。游戏可以在第一次使用它们的时候将之实时地解码出来，然后就可以在以后的请求中播放已经解码的音频了。如果解码会带来一定的延迟，那么音效可以在关卡变动的时候或是启动的时候先解码好，正如前面说到的一样，这样就可以在播放的时候根本不带来解码延迟了。

3. 情况 3：压缩传输

当前面所说的各种情况都不可能的时候，压缩仍然可以在点到点之间的传输中用上。游戏发布的 CD-ROM 以及可下载的包可以使用 Ogg 把下载文件压缩得更小一些，这样以给图形和代码留出更多的空间。当游戏或是演示安装的时候，所有的音乐都可以被解压到硬盘上去。这情况对于游戏补丁和游戏所携带的其他的包也是适用的。

6.1.3　使用 Ogg 的代码示例

通过几个代码的示例，你就看出使用 Ogg Vorbis 是多么容易了。如果要使用 vorbisfile 库，你必须在 API 的头部文件里面加上这么一行代码：

```
#include <vorbis/vorbisfile.h>
```

ON THE CD　然后在连接的时候加上 Vorbis 的库文件 vorbis 和 ogg。在 Windows 下，这两个库可以被静态或是动态连接进来，这要视你的个人偏好而定（请参见 CD-ROM 上的 SDK 文件和 API 参考手册以得到更详细的信息）。

1. 把文件解压到内存

使用 vorbisfile 的 API 把文件解压到内存是很简单的。首先，我们声明一个 OggVorbis_File 变量：

```
OggVorbis_File vf;
```

接下来，我们必须使用一个已经打开的文件指针去打开此 Ogg Vorbis 文件。请注意在打开 vorbisfile 文件的时候一定要使用二进制方式打开。在某些平台上面，二进制文件在像 fopen() 这样的函数中是缺省打开方式，但是在 Windows 上就不是这样了，因此你必须显式地将之以二进制方式打开。

```
int err;
err = ov_open(fp, &vf, NULL, 0);
```

此中的 fp 是一个我们已经打开的文件指针。而 vf 则是 OggVorbis_File 的结构体，

Download at www.Pin5i.Com

其他两个参数一般被设置为 NULL 和 0，正如我们例子里面使用的一样（如果你很想知道他们是什么意思，以及应该在什么时候用上它们，可以参见 API 的参考手册）。

成功的时候 ov_open 将返回 0，失败的时候将返回一个负数。如果失败了，此返回值将有一个对应到 API 中标准错误代码的值（你可以在 API 参考手册中找到其对应的解释）。

现在 Ogg 文件已经被 vorbisfile 库打开了，当前库拥有此文件。一旦它完成了任务，就将把此文件关闭，而你也不能在外部的文件操作里再使用此文件指针了。一旦文件打开了以后，就要从中读取信息了：

```
vorbis_info *vi;
vi = ov_info(&vf, -1);
```

Vorbis_info 结构体将包含此打开的 Ogg 文件的信息，例如它的采样频率、比特率以及声道数目。函数的第二个参数指明了应该返回哪一个逻辑比特流的信息（-1 代表当前的逻辑比特流）。此外，你还可以调用 ov_comment() 函数读出此文件里面的注释。

在我们把 Ogg 文件解压到内存中之前，首先必须要为其分配一些空间。若想得知其需要的总空间数，我们就要调用 ov_pcm_total() 以得到其中 PCM 采样的总数，然后将此数乘以声道数和每个采样占用的字节数（对于 16 位采样来说一般是 2 个字节）。

```
int size;
char *buffer;

size = vi->channels * 2 * ov_pcm_total(&vf, -1);
buffer = (char *)malloc(size);
```

现在我们就已经分配了一个缓冲区以供整个解码输出所用了，下面就要在一个循环中不停地调用 ov_read() 以将 PCM 采样放置到缓冲区中去。

```
int eof = 0;
char *buf = buffer;
int current_section;

while (!eof) {
    long ret = ov_read(&vf, buf, 1024, 0, 2, 1,
               &current_section);

    if (ret == 0) {
        /* 0 return value means end of file */
        eof = 1;
    } else if (ret < 0) {
        /* < 0 return value indicates an error */
    } else {
        /* advance the buffer pointer on success */
        buf += ret;
    }
}
```

上述的代码是相当简单的，它没有做任何错误检查（虽然在使用 Ogg 的时候，大部分错误一般都不会有严重的后果，因此可以被忽略掉），也没有检查是否有溢出现象，最复杂的工

Download at www.Pin5i.Com

作都是由 vorbisfile 库来代你做了。

一旦文件的处理结束了，你必须调用 ov_clear() 以收尾。

```
ov_clear(&vf);
```

2. 实时解码

实时解码与上述的解码到一个内存缓冲区是很相似的，只不过它没有一次性把所有的数据都解出来，实时解码是一次解一点的。当调用 ov_read() 得到了每个数据块以后，代码就将其送到下一级流水线上。在最常见的情况下，下一级就是一个音频输出设备。

为了简单性起见，有关音频输出系统的细节这里就不提了。取代方法就是我们假设有那么一个 audio_output 函数，其参数是一个缓冲区的采样及其尺寸，然后就可以将之播放出来了。

在上述代码的基础上，唯一需要修改的就是那个解码循环。在循环的开始处，我们仍需调用 ov_read()：

```
char pcmout[4096];

while (!eof) {
    long ret = ov_read(&vf, pcmout, sizeof(pcmout),
                       0, 2, 1, &current_section);
    /* ... */
}
```

请注意现在缓冲区是固定大小的，因为代码现在只处理小块的输出。

在前一个例子的 if 块中的成功情况下，我们将把指针向前移动一个缓冲区大小。由于在实时解码中，我们一次只能处理一个数据块，因此我们只需把当前的数据块交给 audio_output 就可以了：

```
audio_out(pcmout, ret);
```

在成功的时候，ret 放置到缓冲区里面的字节数。

3. 编码

在你的游戏中，如果要全面地理解心理声学压缩的使用方法，最后一点就是编码和生产了。在压缩 Ogg 文件的时候有几个问题需要说明一下，一旦理解了这些问题，你就将在游戏里得到更好听的音乐和音效了。

在 Ogg 中音质与大小是紧密相关的，而两者是一个问题的两个方面。在大部分编解码器中，你可以调整大小参数以此隐式地影响音质。在 Ogg 中，你可以调整音质而隐式地影响到大小。一般说来，你的文件越大，那么声音就越好听。同样地，你需要的音质越差，文件也就越小。当然，其中还有一些实际的限制。

压缩音频的大小是由其比特率决定的：用于存储一秒音频所用的比特数。一般的玩家听不出这些细微的音质损失，因此当比特率超过了一定数值后其边际效果就趋向缩小了。Ogg 文件在 64k bit/s（千位每秒）的时候听起来相当好，而且应该和原始的 128k bit/s 的音频听起

Download at www.Pin5i.Com

来没有什么差别。

由于对于大部分游戏来说，音乐并不是玩家主要关心的地方。低比特率的 Ogg 文件应该足够给玩家一个很棒的游戏体验了。

其他两个问题是音频的声道数与采样频率。虽然它们对音质有所影响（例如，单声道就没有立体声好），但是其对音质的影响与大小的改变不成线性比例。如果你把立体声换成单声道声音以减少一半的大小，在大部分情况下其音质只会很少地下降一点。同样地，如果你把采样率下降一半，声音听起来会比较差，但是也不至于差了一半。如果空间对你而言很重要，那么你可以在利用 Ogg 压缩之前将其向下混音和重采样。这样你就可以得到更小的文件，同时音质损失也是可以接受的。

6.1.4 结论

随着游戏音频重要性的增强，很显然我们应该使用更好的压缩技术以减小音频的尺寸。由于很多游戏的资金都比较紧张，因此选择一个开源而有免费版权的方案就很自然了。因此，在你的下一个项目里，可以考虑使用 Ogg Vorbis 来进行心理声学的压缩。

6.1.5 参考文献

[OggVorbis02] Ogg Vorbis, 对应网址为 http://www.vorbis.com, July 2001.

[XiphHome02] Xiphophorus, http://www.xiph.org, June 2001.

[XiphName02] Xiphophorus 名称以及商标，对应网址为 http://www.xiph.org/xiphname.html, June 2001.

Download at www.Pin5i.Com

6.2　创建一个美妙的 3D 音频环境

Garin Hiebert
Creative Labs, Inc.
garinh@hibyte.com

本节将提供为你的游戏创建一个美妙的 3D 音频环境所需要的技术，而你只需要在音频引擎设计和采样的时候遵循一些简单的规则就可以达到此效果了。我们将介绍一些在 3D 音频渲染中使用到的基本概念，提供一些提示信息以让你的音频引擎得到最好的效果，然后将提供一些例子展示如何使用这些信息。DirectSound 3D 与 OpenAL 是当前两个出色的 3D 音频处理 API，但是如果你使用的是其他的 API，此处的概念也应该同样适用。完整的示例代码在 CD-ROM 上有。

6.2.1　3D 音频的核心概念

很多现代游戏的假设场景都是 3D 世界，即使是那些在过去使用 2D 世界的游戏。3D 音频引擎的基本概念就是要使用同样的音视频组件数据，在游戏的各个方面表现出 3D 的世界。下面的各个主题对于 3D 定位音频渲染来说都是很基本的，而且对你的音频引擎设计也很重要。

1. 唯一的听众

在游戏中有一个唯一的听众对象，它表示的是在 3D 环境中的什么地方可以听到声音。听众在游戏世界空间中有一个位置坐标和方向参数，它可以用来计算扬声器的输出。

2. 多个音源

在环境中的每个发声对象都是用一个音源来表示的。所有的声音都在听众的位置上进行混音，然后再表现出来，而每个都对应了声卡上的一个声音通道。

3. 多缓冲区

一个缓冲区对象包含了实际的音频数据。在 DirectSound 3D 中，一个缓冲区和一个音源是合并在一个对象里面的。在 OpenAL 中，音频数据是与音源分离的，一般说来分配的缓冲区数量会比音源数量要多。

Download at www.Pin5i.Com

4. 声音管理器

我们必须实现一个架构以在声卡的声音通道上分配音频数据（以缓冲区的形式）。游戏经常会要求同时播放很多声音从至于系统无法支持。而声音管理器就可以解决这个问题，其方法是基于距离或是优先级调用，以决定在任一给定时间应该播放什么声音。在 DirectSound 3D 中，一个缓冲区和一个音源是同一个对象，此时在其内部使用了一个声音管理器的架构来决定在当前的硬件上什么缓冲区可以被播放。在 OpenAL 中，一个音源对象直接对应一个音频硬件上的声音通道，而此时则可能有很多缓冲区需要在有限的音源上播放。在这种情况下面，声音通道的分配策略是由开发人员来决定的。

5. 决定使用哪一个扬声器输出

最终用户可能会有很多种扬声器输出的设备配置，而其中的任何一种都可以用来很好地表现出 3D 音频世界。为了进行输出，音频渲染系统（例如 DirectSound、OpenAL）必须至少指明对于听众的位置来说每一个音源应该具有什么样的相移和衰减。我们可以使用听众的相对位置和方向以及每个音源来决定扬声器的相移。每个音源和听众之间的距离决定了该音源的衰减，而其得到的音频将被混音，继而播放。为了得到更好的音频效果，我们还可以在混音之前使用回响与过滤（可以在全局范围内使用或是针对每个音源使用）的技术。

6.2.2 有效地使用你的音频引擎

下面的提示信息（其顺序是按它们在音频处理中的顺序排列的）将帮助你尽量得到一个最好效果的 3D 音频引擎。

提示 1：使用归一化的采样

在把一个采样输入你的引擎之前，要确保已经对之使用了归一化的方法，这样以保证其能充分利用采样比特数的幅度范围。否则，你可能会发现有某个采样声音总是太弱，然后就不得不调整每个采样的幅度以适应它。如果在一开始就进行了归一化，那么这个问题就可以避免了。

提示 2：使用纯采样

如果你希望对自己的音频添加音效，那就要确保原始的采样没有使用音效（没有音效的采样被称为纯采样）。如果在早期阶段就加入了回响或是过滤，那么在运行时处理的时候就不可能将之从音效中恢复出来了。

提示 3：决定单位

要确保你确切地知道在游戏世界设计中使用的是什么单位。很重要的一点就是要给底层的 API 提供正确的信息，这样它才能正常地将音频给渲染出来。这就意味着音频 API 需要知道游戏世界中的物理参数。如果音频世界的单位是米，但是游戏世界中却是英尺，那么某些音效（例如，多普勒效应或是空气吸音）可能会产生出人意料的效果，导致你使用另一套单

Download at www.Pin5i.Com

位重新校正音效。在整个引擎中使用一致的单位就可以让 API 工作正常了。

提示 4：理解与发挥底层 API 的能力

先选择出一个底层 API，然后作出计划以发挥其最大的优势。除了 3D 位置以外，底层的 API（例如 DirectSound 3D、OpenAL）还将有全局属性和针对每个音源的一些属性，例如滚降参数或是每个音源的增益。做好准备，向底层的 API 提交此数据。例如，如果你遵循了提示 1，对音频数据进行了归一化，那么下面就要为每个音源设置相应的参考距离了。OpenAL 和 DirectSound 3D 都有方向性音频的概念——此特性在两个 API 都没有被充分利用。

表 6.2.1 和 6.2.2 列出了一些对于 OpenAL 和 DirectSound 3D 都适用的全局属性和针对每个音源的属性。

表 6.2.1　　部分 OpenAL 与 DirectSound 3D 的全局属性

描　述	OpenAL 的属性	DirectSound 3D 的属性
主增益	AL_GAIN	主音量，使用混音 API 设置
声音速度	AL_DOPPLER_VELOCITY	DistanceFactor，针对听众
多普勒效应	AL_DOPPLER_FACTOR	DopplerFactor，针对听众

表 6.2.2　　部分 OpenAL 与 DirectSound 3D 中针对音源的属性

描　述	OpenAL 的属性	DirectSound 3D 的属性
音调乘法器	AL_PITCH	Frequency
增益	AL_GAIN	Volume
位置	AL_POSITION	Position
速度	AL_VELOCITY	Velocity
方向	AL_DIRECTION	ConeOrientation
朝向的相对/绝对模式	AL_SOURCE_RELATIVE	Mode
参考距离	AL_REFERENCE_DISTANCE	MinDistance

提示 5：添加音效

对音频进行处理以适应游戏环境。使用一个对应的 API 加入音效以增强玩家的体验，例如 EAX[1]、隔离、音障、回响、相移以及其他的效果。如果没有相应的硬件加速，那么系统也不必仅仅局限在纯音频上，而可以使用过滤或是额外的衰减。例如，在一个第一人称视角射击类游戏中，中间格挡的墙壁应该进行消音处理。使用 EAX 加上隔离或是使用一个低通滤波函数加上额外的衰减，都可以使得场景中的声音更加逼真，同时也给玩家提供了周围环境的信息。

6.2.3　实现

在 CD-ROM 上我们提供了一个演示程序，其音频引擎使用了上述的提示信息。在很多方面来说，这个演示程序都是很简单的，但是它仍然表现了典型的音频引擎的工作流程，带有四个高层事件。

译者注[1] 中文为环境音效功能扩展集。

Download at www.Pin5i.Com

1. 载入音频数据

在程序启动的时候，所有的音频数据都被载入了进来。在大部分游戏里面，音频数据是在每个关卡变换的时候，在载入其相应关卡其他的数据（例如几何形状、纹理等等）过程中同时载入的。但是一般说来，音频数据的载入是一个不那么频繁的事件。提示 1 与提示 2 都可以使用到此事件上。载入的音频数据应该进行归一化，而且应该是纯音频。

2. 设置全局参数

全局参数是在音频数据载入之后一次性完全设置完毕的。一个游戏可能会有不止一套的全局参数设置，但是与其他的音频事件比较起来，这个事件的发生频率一般也不高。例如，游戏世界中的物理参数，比方说多普勒参数和主音量就不应该变个不停。

提示 3 和提示 4 都对此事件适用。任何参数的设置都应该使用正确的单位，而参数的选择则应该基于 API 提供的功能。

3. 调整针对每个音源的参数

针对每个音源的参数在每帧图像显示的时候只设置一次，这与大部分游戏的操作是一样的。某些游戏设置音频参数的频率要比视频更新的频率低，但是此事件仍然是程序主循环的一部分。

提示 3 与提示 4 都对此事件适用。任何参数的设置都应该使用正确的单位，而参数的选择则应该基于 API 提供的功能。

4. 调整音效参数

在演示程序中，我们同时使用了全局音效和针对每个音源的音效。如果检查出系统有 EAX 的功能，则在演示程序运行的时候全局的 EAX 框架将会被改变。此外，此演示程序会不停地侦测是否可能给每个音源加上 EAX 的效果。任何一个游戏都会同时使用全局音效和针对每个音源的音效，视此效果是否可能而定。此事件是程序主循环的一部分，它是与针对音源参数变化的事件一起发生的。

提示 3 与提示 5 都对此事件适用。某些效果依赖于游戏世界中物理属性的正确设置，而程序将尽可能利用系统的能力来生成相应层次的效果。

6.2.4 结论

为你的游戏创建一个美妙的 3D 音频环境是一个需要考虑很多细节的艰难的任务。通过使用本节中提出的 5 个提示，这项工作将会变得轻松一点，花费的时间将更少一点，而游戏玩家听到的音频也会更迷人一点。

6.2.5 参考文献

[Creative02] Creative Technology, Ltd., “Creative Labs—Developer Central,” 对应网址为

Download at www.Pin5i.Com

http://developer.creative.com, March 2002. 其中的“Games”小节中有 EAX 和 OpenAL 的信息，里面包含了文档和 SDK。

[Loki01] Loki Entertainment Software, “OpenAL | Open Source Audio Library,” 对应网址为 http://www.openal.org, March 2002. 在其上有 OpenAL 的源代码和邮件列表。

Download at www.Pin5i.Com

6.3 使用轴对齐的边界框设置音障

Carlo Vogelsang
Creative Labs Inc.,
cvogelsang@creativelabs.com

本节描述了在你的游戏中，如何使用轴对齐的边界框来简化任意障碍物所造成的正确音障的计算。在这里我们将描述 EAGLE™与 EAX-Manager 中实际使用的技术，它是由 Creative Labs 的 Keith Charley 开发的。其结果是一个优秀的音频体验，它的计算代价较小，而且也有足够的灵活性以在更通用的场合下使用。

6.3.1 问题

当声音不能被直接听到的时候，这就意味着音波不能直接从音源传输到听众。我们称之为音障（obstruction）[Jot99]，因为直接的路线被障碍物挡住了。图 6.3.1 显示了无障碍和有障碍的音源。

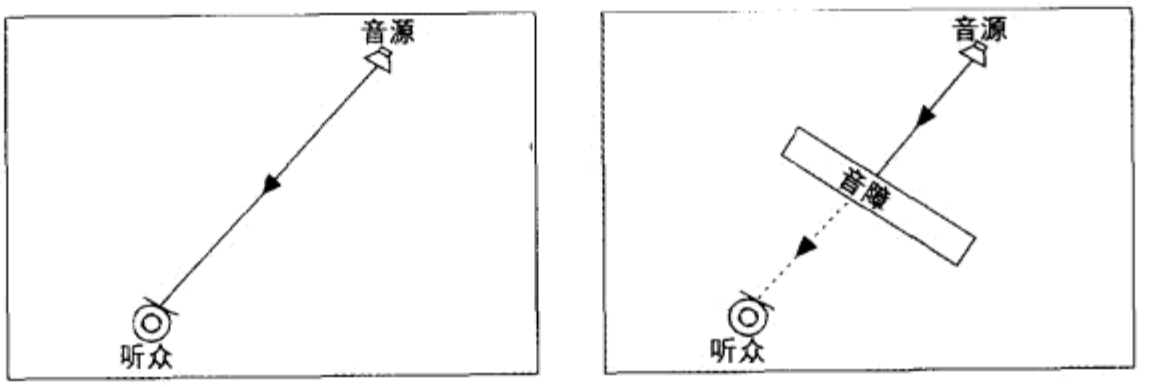


图 6.3.1 无音障（左图）与有音障（右图）的音源

我们希望得到一个数值来表示此障碍的具体影响。现在已知具有较大波长（或说是较低频率）的音波在固态物体上的衍射效应比较小波长的（或说是较高频率）音波更明显。因此，一旦我们得到了一个数值可以用来表示此障碍的具体影响，那么我们就可以使用这个数值来决定针对音源的低通滤波器的增益了[I3DL2]。这个滤波器是用来模拟高频音波通过固态物体衍射后衰减的情况的。

在下面的各个小节中，我们将阐述一个算法，它可以用来得到一个从 0（静音，此声音将完全被消去）到 1（全声，此声音将完全通过）的数值。

Download at www.Pin5i.Com

6.3.2　解决方法

在本节中提出的解决方法牵涉到对衍射路径的模拟，此路径是当声音绕过障碍物的边角时所形成的。我们选择用来计算的边角是离音源最近的那一个角。为了计算出这个弯路，我们假设角度 α 为下面两条连线的夹角，一条从音源到边角，一条从边角到听众。当此角度达到最大值（180° ）的时候，中间就没有障碍物。此角度越小，则声音被阻隔得也就越多[Kutt00]。你可以参见图 6.3.2，在其中你可以看到其结果就是一个与障碍物相交的矢量。一条精确的路线当然是不会相交的，但是我们在这里是在用精确性交换速度。

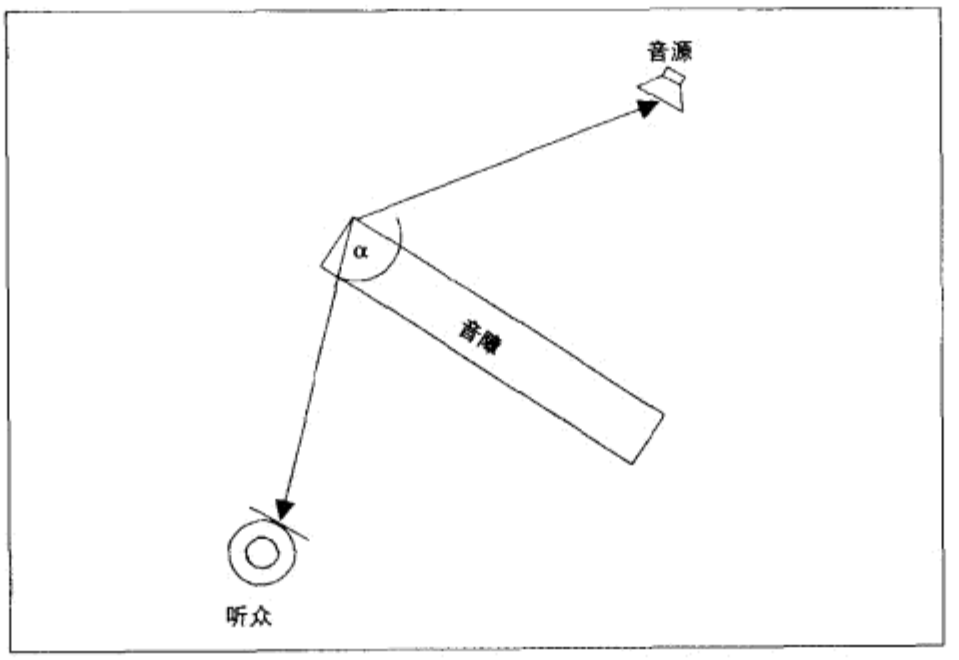


图 6.3.2　最大角/最短路径算法

为了加快此过程，我们将使用一个轴对齐的边界框（AABB）。这时我们将引进一些小误差，但是其计算时间则大大地减少了。你可以对比图 6.3.2 与图 6.3.3 看看它引进的误差。任何一个不是立方形和轴对称的物体都将导致在 α 上产生一些误差。图 6.3.3 中显示了在使用 AABB 的时候引进的误差。

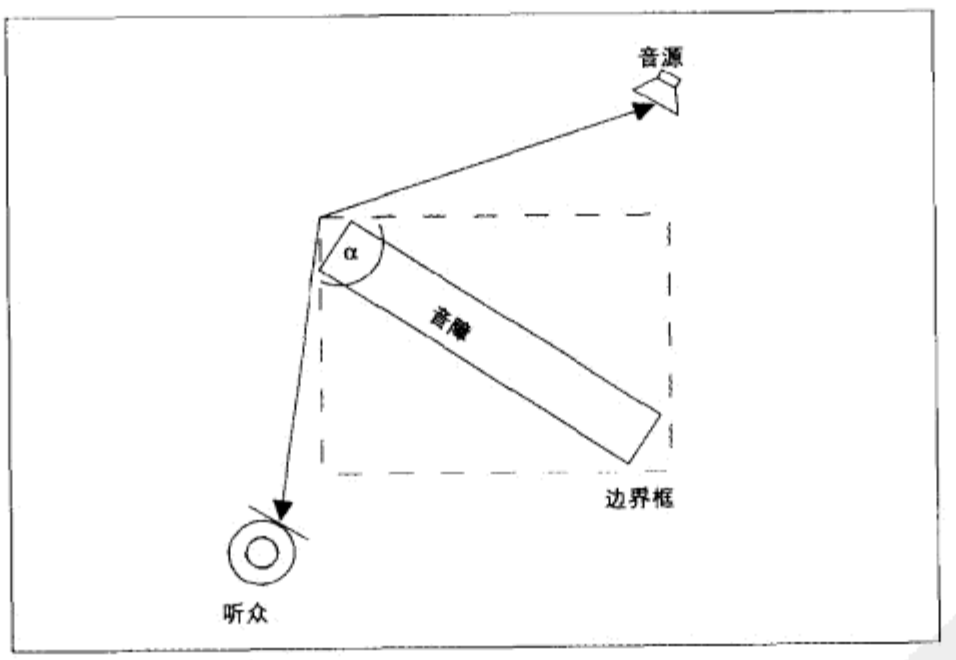


图 6.3.3　由轴对称的边界框覆盖的音障

Download at www.Pin5i.Com

当给定了音源位置、听众位置以及障碍物边界框的时候，如果要来计算此角度，则我们要首先计算出两个比特矢量，一个是为音源计算的，一个是为听众计算的。我们使用了 6 个比特来表示这个 3D 边界框的 6 个面。如果一个比特被设置了，则就意味着音源或者听众就在这个面的前面。下面的代码示例是用来计算对应于正面和背面针对听众的比特位的：

```
if (ListenerPosition.Z < BoundingBox.vMin.Z)
   lListenerFlags |= 0x02;
else if (ListenerPosition.Z > BoundingBox.vMax.Z)
   lListenerFlags |= 0x01;
```

通过使用这两个比特矢量，我们就能迅速得知是否有一个障碍了。如果两者的比特域具有相同的真值位，那么中间就不可能有障碍物，因为音源和听众在同一个面的前面。这个检查可以如下进行：

```
if (lListenerFlags & lSourceFlags)
   continue;
```

如果我们判断出中间有可能有一个障碍物，那么接下来就可以创建一个列表包含所有离音源最近的边界框的面了。要做到这一点，我们可以使用比特矢量来看音源究竟落在了哪一个区域（在边界框以外有 26 个可能的区域，在边界框里面一共有 38 个可能的区域，加起来一共有 2^6=64 个区域）。在外面的 26 个有效区域其中一个的 switch 处理代码如下所示。38 个内部的区域是无效的，因此使用了 default 来处理。

```
switch (lSourceFlags & 0x3f)
{
case 0x01:// Behind bounding box
   vPlanePoint[0] = BoundingBox.vMax
   vPlaneNormal[0] = {0, 0, 1};
   lPlaneType = BACK;
   lNumPlanes = 1;
   break;
   ..
default: // Invalid combination (inside)
   lNumPlanes = 0;
   break;
}
```

在 switch 语句之后，我们可能会一共挑出 3 个边界框的面。对于每个面，我们都要检查是否它和音源/听众的矢量相交了。如果发现了相交的情况，则我们就将计算出交点：

```
for (i = 0; i < lNumPlanes; i++)
{
   SVector svIntersection;
   FindPlaneIntersectionPoint(..,&svIntersection);

   if (PointInPlane(svIntersection,..))
   {
   ...
   }
```

Download at www.Pin5i.Com

```
}
```

现在我们知道在边界框的外表面上有一个交点，下面就需要找出这个点，然后把它移到最近的边上。下面的代码正是起的这个作用：

```
switch (lPlaneType)
{
    case FRONT:
    case BACK:
    if (Listener->fY < Box.vMin->fY &&
    Listener->fY > Box.vMax->fY)
    {
        fMidPoint = Box.vMin->fX - Box.vMax->fX;
        fMidPoint /= 2;
        fMidPoint += Box.vMax->fX;
        if (svIntersection.fX < fMidPoint )
            svIntersection.fX = Box.vMax->fX;
        else
            svIntersection.fX = Box.vMin->fX;
    }
    else
    {
        fMidPoint = Box.vMin->fY - Box.vMax->fY;
        fMidPoint /= 2;
        fMidPoint += Box.vMax->fY;
        if (svIntersection.fY < fMidPoint )
            svIntersection.fY = Box.vMax->fY;
        else
            svIntersection.fY = Box.vMin->fY;
        }
        break;
    default:
        break;
    }
```

由于交点已经在边界框的边上了，因此我们就可以计算出此角度 α 了。它是由交点——音源矢量和交点——听众矢量构成的。

```
long lLevel = MAX_OBSTRUCTION;

CVector cvListener(*psvListener);
cvListener -= svIntersection;

CVector cvSource(*psvSource);
cvSource -= svIntersection;

cvSource.Normalize();
cvListener.Normalize();

float fAngle = cvListener.DotProduct(cvSource);
```

Download at www.Pin5i.Com

```
if ( fAngle >= 0 )
   return lLevel;

fAngle += 1;

float fMaxAngle = (float)m_lDifAngle;
fMaxAngle /= 90.0f;

if ( fAngle >= fMaxAngle )
   return lLevel;

lLevel = (long)(((float)lLevel * fAngle) / fMaxAngle);
```

现在既然已经知道了 α，因此也就知道了此障碍边的具体影响了，我们将把最大的值（也就意味着最小数量的障碍）找出来。请注意，此算法只关注与音源最接近的边。如果边界框的厚度很小（例如墙壁与箱子等），那么这个算法是相当合适的。对于更大的边界框来说，离听众最近边的影响则会带来极大的误差。

6.3.3 实现

在 CD-ROM 上我们提供了一个示例实现，它包含了对一个边界框的所有计算。这是从 EAXMan 中提取出来的实际的代码，它是由 Creative Labs 的 Keith Charley 提供的。

6.3.4 结论

在本节中，我们展示了如何利用轴对齐的边界框简化音障的计算。这可以极大地减少所需的计算量，而且其结果精确度依然很高，并给游戏增加了很好的音频特性，给玩家带来了好处。

6.3.5 参考文献

[I3DL2] IASIG, "Interactive 3D Audio Rendering Guidelines Level 2.0," 对应网址为 http://www.iasig.org, Los Angles, CA, 1999.

[Jot99] Jot, J. M., "EAX 4.0 Specification," Creative Labs, Inc., Milpitas, CA, 2001.

[Kutt00] Kuttruff, H., *Room Acoustics*, Fourth Edition, Spon Press, London, U.K, 2000.

Download at www.Pin5i.Com

6.4　使用双二次共振滤波器

Phil Burk
SoftSynth.com
philburk@softsynth.com

本节将要谈论的是双二次无限冲击响应（IIR）滤波器，它是一个通用的共振滤波器。通过改变滤波器的系数，它可以进行各种滤波操作，例如低通滤波、带通滤波、高通滤波、带阻滤波以及其他滤波。我们将描述一下数字滤波器是如何工作的、如何实现一个高效的双二次滤波器以及如何针对不同的功能计算不同的系数。同时我们也将提供几个声音滤波方面的技术和合成的技巧。

你可以使用一个滤波器来改变一个声音的特性。声音可以看成是由各种频率的正弦波组成的。滤波器可以根据这些正弦波各自的频率来改变其幅度。你可以加大高频或是低频的份量以得到更响亮的声音或是更低沉的声音。

当对采样进行处理的时候这是相当方便的，因为它们听起来都差不多。然而，如果当一个采样通过滤波器的时候你改变其参数的话，那么每次播放采样的时候，你就可以改变播放的声音了。你可以动态地改变这些滤波器的参数，这样就可以对游戏的状态进行跟踪了。如果你对声音进行相应的改变以反映当前游戏的状态，那么游戏的真实感就会大大增强。（使用一个双二次滤波器来创建复杂的声音效果的例子在 6.6 节中也有，节名为“复杂声音的随机合成法”，作者为 Phil Burk。）

6.4.1　数字滤波器的工作原理

当你把同频率的正弦波进行叠加的时候，它们可能会互相抵消，也可能会得到增强，这要视它们的相对相位而定。如果把一个复杂信号延迟一段时间，例如一个采样的时间，则其所有频率分量的相位都将会得到不同的改变。一个采样的延迟对一个低频信号而言，造成的相移是很小的，因为它的周期很长，而一个高频信号则会有较大的相位改变，因为此延迟对于它的波形周期来说是一个很大的值。这样我们就可以得到一个方法来对不同的频率进行不同的改变了。

一个典型的数字滤波器包含有一条延迟线，它是由连续的一个采样的延迟，或称是拍（tap），组成的。原始的输入信号叫做 x_n。因此这些拍将使用 x_{n-1}、x_{n-2} 等等来命名。我们将对这些拍进行量化，然后将之相加或是

Download at www.Pin5i.Com

相减以得到不同频率波形的相消或是增强。

例如，一个均值滤波器输出的 y_n 就是当前输入采样与上一个输入采样之间的均值。对一个简单的低通滤波器而言，其方程是：

$$y_n = 0.5 * (x_n + x_{n-1}) \quad (6.4.1)$$

由于较低频率的相位不会对一个采样的延迟敏感，因此相对而言，它们经过了滤波器以后改变会比较小。然而，高频波则会产生相当大的相移，甚至会互相相消。实际上，波的频率越高，相消的就越多。由于此滤波器通过的低频波比高频波要多得多，因此我们可以把这个简单的滤波器称为低频滤波器。

要建构一个高频滤波器，你只需要把低频滤波器方程中的＋用－代替就可以了。这么一来，频率较低相移较小的波形就会相消了。这个简单的高通滤波器的方程是：

$$y_n = 0.5 * (x_n - x_{n-1}) \quad (6.4.2)$$

6.4.2 IIR 与 FIR 滤波器

有限冲击响应（FIR）滤波器只对输入的声音进行处理。因此，当输入为零的时候，在几个采样之内输出也会变为零，这要视输入有多少拍而定。

无限冲击响应（IIR）滤波器则会同时使用以前的输入和输出。使用了这种反馈以后，滤波器的输出就能永远振荡下去了，甚至当输入已经变为零的时候。电子音乐特有的回响滤波器就是 IIR 滤波器。

6.4.3 双二次滤波器的实现

双二次滤波器在输入上使用了三拍，在输出上使用了两拍[Dodge97，p.214]。此滤波器的方程是：

$$y_n = a_0 * x_n + a_1 * x_{n-1} + a_2 * x_{n-2} - b_1 * y_{n-1} - b_2 * y_{n-2} \quad (6.4.3)$$

通过改变系数 a_0、a_1、a_2、b_1 以及 b_2，你就可以让这个滤波器变成一个低通、高通、带通、带阻、峰值均衡或是其他种类的滤波器了。

我们可以采取多种措施以对此滤波器进行优化。对于滤波器操作来说，其最佳优化位置就是在循环里面。大部分编译器都会将变量一次性载入寄存器，这样就可以不用再载入它们以进行计算了。下面就是一段双二次滤波器的实现，它是从 CD-ROM 上的 Unit_BiquadFilter.cpp 里面摘抄出来的。

```
for( i=0; i<GGSYNTH_FRAMES_PER_BLOCK; i++)
{
// Generate outputs by filtering inputs.
    xn = inputs[i];
    yn = (a0 * xn) + (a1 * xn1) + (a2 * xn2)
        - (b1 * yn1) - (b2 * yn2);
    outputs[i] = yn;

// Delay input and output values.
```

Download at www.Pin5i.Com

```
        xn2 = xn1;
        xn1 = xn;
        yn2 = yn1;
        yn1 = yn;
    }
```

6.4.4　改变变量

对于双二次滤波器而言，另一个优化就是可以通过改变算法来减少延迟线上数据的移动。首先，我们对循环进行修改，这样在一次循环里面我们就可以进行两次滤波了。我们可以不移动数据，而是改变方程，把数据放在原地就可以使用其正确的值了。下面的代码就是上述代码的一个改进版。为了把这个情况讲得更清楚一点，请参看下面代码中的第二个 IIR 方程，在其中我们进行了$(a_2 * x_{n1})$的乘法。一般来说，我们可以进行 $x_{n2}=x_{n1}$ 的操作以实现延迟线，然后再进行$(a_2 * x_{n2})$的操作。然而，我们可以取一条捷径，直接进行$(a_2 * x_{n1})$的操作，从而可以得到同样的值。某些编译器可以自动实现此功能，但是如果我们自己做了就更清晰了。

```
for( i=0; i<GGSYNTH_FRAMES_PER_BLOCK; i+=2)
{
// Generate outputs by filtering inputs.
        xn = inputs[i];
        yn2 = (a0 * xn) + (a1 * xn1) + (a2 * xn2)
            - (b1 * yn1) - (b2 * yn2);
        outputs[i] = yn2;

// Permute filter operations to reduce data movement.
// Just substitute variables instead of xn1=xn, etc.
        xn2 = inputs[i+1];
        yn1 = (a0 * xn2) + (a1 * xn) + (a2 * xn1)
            - (b1 * yn2) - (b2 * yn1);
        outputs[i+1] = yn1;

// Only move a little data.
        xn1 = xn2;
        xn2 = xn;
}
```

另外，很多种滤波器的许多系数都是一样的或是零。因此，你可以消除其中的某个乘法。例如，假设 a_0 等于 a_2，那么$(a_0 * x_{n2}) + (a_1 * x_n) + (a_2 * x_{n1})$ 这一项就变成$(a_0 * x_{n2}) + (a_1 * x_n) + (a_0 * x_{n1})$了，它还可以进行合并得到$(a_0 * (x_{n2} + x_{n1})) + (a_1 * x_n)$。

6.4.5　避免异常情况

依赖于前一次调用结果的递归算法容易出现某种令人恼火的问题。假设下面的式子将执行很多次：

Download at www.Pin5i.Com

```
x = 0.99 * x;
```

其中的 x 值将变得越来越小，最终它将小得超出浮点数的有效范围。如果在一个游戏中你暂停很长时间就有可能出现这种情况。当这种情况发生的时候，某些 FPU 将会中断 CPU 的任务请求进行处理，这就将给计算机带来额外的开销。

幸运的是，我们还有一个简单的解决方法，那就是只需要给滤波器加入一点点能量，避免其衰减到这种数值就可以了。你可以加入一个极小的脉冲，其效果可能是用户不会察觉的，但是其值已经足够防止这种情况的出现了。当你使用前面的循环计算了一组采样以后，只需要在延迟值中再加上一个很小的数就可以了。这么一个小峰值可以预防滤波器衰减到接近零的地步。下面就是一个可能的解决方法：

```
yn1 += 1.0E-26; /* prevent derormalization */
```

6.4.6 控制滤波器

我们可以使用两个参数来控制滤波器的行为：截止频率和共振系数（Q）。对于一个低通滤波器来说，超出截止频率的分量将会被逐渐消弱。对于一个高通滤波器来说，低于截止频率的分量则会被逐渐消弱。共振系数（Q）可以用来增强反馈回滤波器的值。这可以使得截止频率附近的曲线更陡峭。当 Q 值足够高的时候，滤波器甚至可能会发生振荡。当 Q 等于 1 的时候，一般认为就是正常值了。

6.4.7 计算此滤波器的系数

对于不同种类的滤波器使用的系数，我们是参照 Robert Bristow-Johnson 在音乐数字信号处理的邮件列表上发布一本的 cookbook 选取的[RBJ]。请注意，我们代码中 A 与 B 的顺序与他的书中的顺序是颠倒的，这么做的目的是为了和一般教科书上的写法一致。系数的计算牵涉到三角函数的运算，这种计算开销比较大，因此一般应尽量避免，除非输入参数发生了变化。

数字滤波器的操作对象很简单，就是一组数字或是采样。对于一组给定的系数和采样来说，不管采样率是多少，其输出总是一样的。因此在计算滤波器之前，我们需要把截止频率的单位由赫兹改为角速度的 omega，后者和采样率总是保持一定的比例的。omega 的正弦和余弦在计算中会使用到多次，因此我们只会事先计算一次（sin_omega 与 cos_omega）。

```
omega = (2.0 * PI * frequency) / sampleRate;
sin_omega = sin( omega );
cos_omega = cos( omega );
```

所有的系数实际上都扩大了一样的倍数。我们就不做多余的乘法，直接把此倍数计算到系数中去。

```
alpha = sin_omega / (2.0 * Q);
scalar = 1.0 / (1.0 + alpha);
```

Download at www.Pin5i.com

6.4.8　低通滤波器

低通滤波器将消弱一个声音中的高频含量，让其中的低频含量通过。

```
/* Coefficients for LowPass Filters */
A0 = 0.5 * (1.0 - cos_omega) * scalar;
A1 =  (1.0 - cos_omega) * scalar;
A2 = A0;
B1 = -2.0 * cos_omega * scalar;
B2 = (1.0 - alpha) * scalar;
```

当我们将反射功能与低通滤波器结合起来以后，它就可以生成一种听起来像从远处来的声音了。这样，当一个声音处在一堵墙背后，或是处在一个盒子里面，或者在一个隧道里面的时候，我们可以使用这种技巧让声音听起来比较模糊。回响低通滤波器可以生成一种特别古怪的尖啸声或是合成的武器噪音。此外，我们还可以将白噪声（white noise）通过一个滤波器，同时缓慢地、随机地改变滤波器的截止频率和 Q 值以得到呼啸的风声。

6.4.9　高通滤波器

高通滤波器可以消弱一个声音的低频分量，让其中的高频部分通过。

```
/* Coefficients for HighPass Filters */
A0 = 0.5 * (1.0 + cos_omega) * scalar;
A1 =  -(1.0 + cos_omega) * scalar;
A2 = A0;
B1 = -2.0 * cos_omega * scalar;
B2 = (1.0 - alpha) * scalar;
```

高通滤波器可以用来生成一种尖细的声音，就好像是一个电话或是电台的声音一样。此外我们还可以用它来使一个声音更响亮。

6.4.10　带通滤波器

带通滤波器将同时消弱一个中心频率两侧的频率分量。我们称其为中心频率而不是截止频率，这是因为如果你观察过一个带通滤波器的频率响应曲线的话，就可以发现这个频率是在峰值的中间的。

```
/* Coefficients for BandPass Filters */
A0 = alpha * scalar;
A1 = 0.0;
A2 = -A0;
B1 = -2.0 * cos_omega * scalar;
B2 = (1.0 - alpha) * scalar;
```

Download at www.Pin5i.Com

6.4.11 将滤波器进行串联

你可以将低通滤波器与高通滤波器进行串联，把它们的输入输出首尾相连就行了。每个滤波器都将对上一个滤波器的输出进行裁减。此外，你还可以将低通滤波器与高通滤波器串联起来，生成一个定制的具有较大通过带宽的带通滤波器。

6.4.12 将滤波器进行并联

我们可以将几个滤波器进行并联，把它们的输出进行混合或是相加就可以了。这样一组滤波器被称为滤波器组。举个例子，在你的立体声音响里面的图形均衡器中就有可能有这么一组带通滤波器组。

6.4.13 软件

在 CD-ROM 上我们提供了一个键盘驱动的 C++程序，你可以使用它来实验一下本节提到的三种滤波器。你可以输入白噪声或是一组训练脉冲，听听它们的输出声音。你也可以改变一下滤波器的共振系数。滤波器的频率是由一个正弦波控制缓慢地上移与下降的。此软件的更新情况将在网址 http://www.softsynth.com/gamegems/上公布出来。

这些示例程序使用了 PortAudio，它是一个简单跨平台的音频 API。因此，此程序应该在 Win32、Macintosh、带有 OSS 的 Unix、SGI 以及其他的平台上都能运行。若想了解 PortAduio 针对你所在平台上的更新情况，请访问他们在 Web 上的网址 http://www.portaudio.com/。

6.4.14 结论

通过使用双二次滤波器，我们就可以在不占用大量内存的情况下使用一个简单的方法对声音的特性进行修改了。它可以让我们用较少的采样得到更多的结果，甚至可以不使用任何采样就生成有趣的声音。本节给我们展示了如何高效地实现一个滤波器，而示例软件给我们展示了如何对声音进行过滤以及播放其结果。

6.4.15 参考文献

[Dodge97] Dodge, *Jerse, Computer Music Synthesis, Composition and Performance,* Simon & Schuste, 1997.

[Moore90] Moore, F. Richard, *Elements of Computer Music,* Prentice Hall, 1990.

[RB] Bristow-Johnson, Robert, "Cookbook Formulae for Audio EQ Biquad Filter Coefficients," 对应网址为 http://www.musicdsp.org/。

Download at www.Pin5i.Com

6.5　语音压缩与音效的线性预测编码

Eddie Edwards
eddie@tinyted.net

语音合成器是用来改变语音的设备，它由于那首 Cher 的歌曲 Believe [Cher98]而一举成名。语音合成器最出名的应用就是它们可以生成机器人的声音，从 Dr. Who 系列中的 Dalek[Nation63]一直到 Star Wars 中的战争机器人[Lucas99]，但是它们也可以生成古怪奇异的声音，还能生成吉他音乐与类似的效果。此外，语音合成器是几个语音压缩方法的基础，包括 GSM 数字移动电话标准[ETSI00]。

虽然语音音效和语音压缩好像是不同的两个问题，但是对两者却可以使用同样的方法。在 CD-ROM 上的程序演示了语音音效，但是此方法还有一个副效应就是每秒只需要 500 个字节的数据——这比 CD 音频要小 350 倍！完整的 GSM 算法是对公众开放的[Degener00]，它可以获得相对于 CD 音频来说 200:1 的压缩比。

这种魔术式的效果是由语音合成器工作的方式决定的。现在有好几种不同类型的语音合成器，但是它们的工作原理却都是一样的：将声音分解为几个音素（分析），对音素进行处理，然后把它们再重新合到一起来（重新合成）。接下来我们就可以在处理这些数据的时候创建音效，或是对之进行压缩了。显然，我们还可以保存和读取数据，这样分析和合成就不必非要在一起了——甚至不必非要在同一台机器上。在这种意义上来说，一个语音合成器与任何一个编解码器都是一样的，后者在编码阶段和解码阶段中就有明显的逻辑上的分别（参见图 6.5.1）。

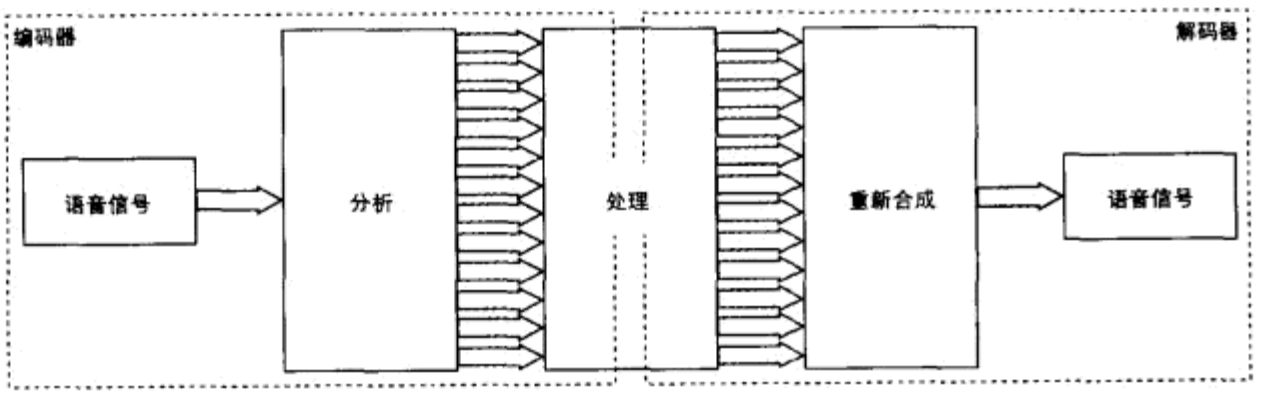


图 6.5.1　语音合成器

我们可以来比较一下这种方式和一个 JPEG 压缩的过程。在压缩过程中，每个像素块都被转换到频域上（分析），在此处更容易对之进行压缩。

Download at www.Pin5i.Com

当 JPEG 解压的时候这些像素点就又被转换回来了（重新合成）。

一个语音合成器最终是由其使用的用来分析与重新合成的算法所定义的。当此算法为快速傅立叶变换（FFT）的时候，我们得到的是相位语音合成器。在此处我们展示的算法是线性预测编码（LPC），因此我们就得到了一个 LPC 语音合成器。

6.5.1 对语音建模

LPC 试图对语音在人身体里面生成的过程进行建模，但是其使用的方法是很通用的，因此它也可以对不同环境下声音的生成进行建模。基本模型只包含两个部分，其术语分别是振荡器与共振器（参见图 6.5.2）。

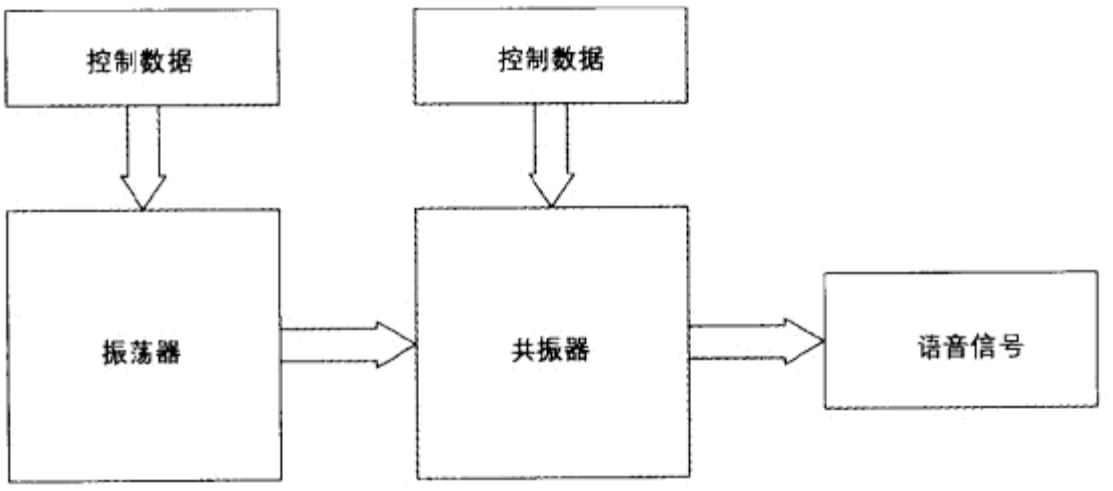


图 6.5.2 LPC 模式

没有振荡器你是不可能生成语音的——这是一个可以生成音波的振动的物理物体。任何一个制造声音的东西都含有某种形式的振荡器，从一个正在演奏的小提琴（琴弦）到一个掉下桌子的铅笔（铅笔的振动）。人类的语音也不例外。人类的声带就是一个振荡器。振荡器的概念很容易理解，而且在计算机上对之进行模拟也很简单。

很多系统都可以制造某种形式的声音，不过某些是专门从事此工作的（不管是逐渐演进而来的还是新创的）。这些系统的区别就看它有没有一个共振器。所有的音乐仪器都有这个东西——例如，手拨吉他的中间是空心的以用来放大琴弦的声音（电子吉他是实心的，因此它的琴弦声是基本听不见的，除非你给它加上了一个放大器）。然而，从任何方面来说，共振器都不仅仅是一个简单的放大器。它对不同频率的声音给予不同程度的放大，这要视此共振器的特性而定。对于小提琴来说，共振器的作用实际上是减弱了琴弦发出的非谐音。正是一个小提琴里的共振器才决定了弦乐器和一个雅马哈电子琴的分别。在铜乐器中，要产生不同的音符，唯一的方法就是改变共振器的配置，因为振荡器（吹口）是不能改变音调的。当按下了一个键的时候，演奏者就在共振器边上开启了一个空气入口，改变了其特性，生成了一个新的音符。

对于人类的语音来说，共振器是由头部形成的——特别是由头部的内部空腔构成的，声音将从声带经过此处最终到达嘴部。这里的共振器当然不是固定的，而且通过舌头、下颚、嘴唇的运动，谈话者就可以快速地改变共振器的特性了。如果你振荡自己的声带把嘴张大缩

Download at www.Pin5i.Com

小的话，就可以听出效果来了——显然，大部分词语的发音都是从这种机制而来，而不是从声带来的（起作用的还有其他的一些机制，我们稍后就会谈到）。

由于共振器可以随时间改变其特性，因此我们将把此模型分解为一个个帧，每帧大约 20ms 长。假设在每帧之中共振器的特性是不变的。对于 LPC 算法来说，每帧的长度是其中的一个参数。

6.5.2　软件仿真

利用软件对此模型进行仿真是很直接了当的。振荡器可以用一个采样播放器仿真，而共振器则可以用一个数字滤波器进行仿真。当振荡器把采样发给滤波器的时候，话音就产生了。

在原理上，这个数字滤波器是很简单的。它将得到一系列的输入采样，然后生成一系列的滤波后的输出采样。滤波器的工作方式是将前 N 个输入采样的加权平均值赋予当前的输出采样。这些权重就决定了此滤波器的特性。通常实现的时候，它是一个具有单个参数的函数，其输入是最近的一个采样，其他的最近 N 个采样将被作为一个内部的状态矢量给记住。

在 LPC 中，我们使用的是一个稍微有所不同的公式，在本质上其物理偏向比数学偏向要更强。我们将共振器看成是一个由不同宽度的各段组成的长管道。当一个音波通过其中两段的时候，音波中的某部分将会被反射回去。在每一点上反射回去的数量是由一个反射系数决定的。一组这样的系数就描述了整个通道，而系数的数目则是 LPC 算法中的一个参数。滤波器将保留一个对应每段中音波压力的状态矢量。这就使得反射回来的音波可以真正返回到仿真的管道中了。下面给出的函数就可以做此项工作。

```
// s = input sample
// FFS[] = forward filter state
// rc[] = reflection coefficients

for (int ii = order; ii--;)
{
    // s -= FFS[ii] * rc[ii]
    // FFS'[ii + 1] = FFS[ii] + s * rc[ii]
    s -= FFS[ii] * rc[ii];
    FFS[ii + 1] = FFS[ii] + s * rc[ii];
}
return s;
```

在此处有一个有趣的地方，那就是此函数是可逆的，而这正是 LPC 的关键（参见图 6.5.3）。如果我们将滤波器逆转过来，让话音通过它，那么我们就可以得到振荡器的输出信号了（当我们如此运行滤波器的时候，此振荡器的信号被称为是残差（residual）信号）。此逆函数如下所示：

```
// s = input sample
// IFS[] = inverse filter state
// rc[] = reflection coefficients
```

Download at www.Pin5i.Com

```
float ifsp = s;

for (int ii = 0; ii < order; ii++)
{
    // IFS'[ii + 1] = IFS[ii] + s * rc[ii]
    // s += IFS[ii] * rc[ii]
    float tmp = IFS[ii] + s * rc[ii];

    s += IFS[ii] * rc[ii];
    IFS[ii] = ifsp;
    ifsp = tmp;
}
return s;
```

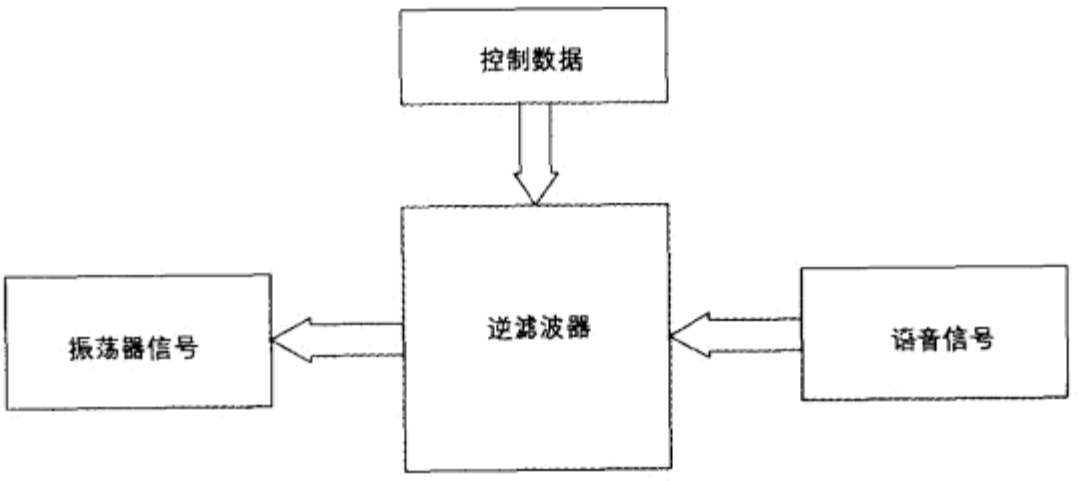


图 6.5.3 逆滤波器

这正是我们所需要的。在分析阶段，我们得到了话音信号，但是没有振荡器的信号。现在我们就可以抽取出振荡器的信号（residual）以为重新合成而用了。现在我们需要的就是反射系数了。问题在于任何一组系数可以用在这里。当运行上述代码的时候，每组系数都可以生成某种形式的“振荡器信号”。我们需要的是选出一组系数，它能给我们带来某种意义上“最佳的”振荡器信号。

LPC 将最佳残差定义为有最小能量的一个残差信号。这是很合理的一个条件，因为它意味着逆向滤波器已经消去了信号的高能量部分——也就是最重要的那部分。而正向滤波器则当然把这些部分在信号里重新生成了。

解决此问题现在就转化为找出一组很大的联立线性方程组，然后得到其最小均方解的问题了。对此方法进行详细的描述不是本节的任务，其基本概念可以在任何一本合适的线性代数书里面找到。CD-ROM 上的代码使用的是 Schur 算法[Degener94]，我们已经针对本问题的特殊情况对之进行了高度优化。其过程可以重现如下：

```
// G[], H[] = working arrays
// AC[] = autocorrelation vector for input signal
// rc[] = (output) reflection coefficients

for (ii = 0; ii < order; ii++)
{
```

Download at www.Pin5i.Com

```
    G[ii] = H[ii] = AC[ii + 1];
}

// one iteration per coefficient
for (ii = 0; ii < order; ii++)
{
    // calculate rc and update error
    float r = -H[0] / error;

    rc[ii] = r;
    error += H[0] * r;

    // update G & H
    for (int m = 0; m < order - ii; m++)
    {
        H[m] = H[m + 1] + r * G[m];
        G[m] = H[m + 1] * r + G[m];
    }
}
```

在 linearpredictor.cpp 里面的代码处理了所有的这些任务，为 LPC 编码提供了一个黑箱式的解决方案。现在我们就剩下两步了——分析与合成——它可以把原始的话音采样精确地重新生成出来（将带有一些舍入误差）。唯一的参数就是每帧的时间（对采样来说）和滤波器的复杂度（反射系数的数量）。测试代码使用的是 20ms 的帧（160 个采样，8 kHz）和 8 个反射系数。

6.5.3　取代声带

一旦 LPC 编码器完成了其任务，我们接下来就希望做一些工作以创建自己的机器人或是怪声了。最简单的方法是用一个合成的声音来代替振荡器信号。此合成的声音将被滤波，其最终的结果就好像合成器的声音是从一个扬声器中传出来的，而不是从它自己的嗓子里出来一样。

这里要记住一件重要的事情，那就是合成的声音仅仅被滤波了一次，因此如果某个声音的频率在此之前没有，则在此之后也没有。如果我们使用一个简单的正弦波，那么它将仅仅含有一个频率，不会生成有趣的声音。这就好像是使用单色的激光灯来照亮一个彩色的壁画一样——只有一种颜色被反射回来了。因此我们必须使用一个具有丰富频谱内容的语音。

最佳选择是使用脉冲波或是锯齿波，因为两者都包含了所有的谐音。一个方波包含了所有的奇数次谐音。使用这些波形都将带来一个相对智能一些的机器人的话音，而且它们都能很快地使用代码来实现。

此外，一个重低音波会带来一个重低音的语音。而一个只带有某些特定频率的波形将只能在这些频率进行调谐，这将生成语音调制。采用这些方法都会得到一种异类声音，更奇异的输入波形将导致更奇异的话音——虽然你再也不可能听出它们到底在说什么了！稍微复杂一点的波形可以通过 FM 调制得到。

一个采样可以简单地被回放出来。如果此采样是残差信号，则我们将得到极高品质的结果

Download at www.Pin5i.com

（如果可以允许舍入误差的话，则可以称之为完美）。如果不是的话，例如是一个吉他的即兴演奏，我们将得到一个古怪的好像视吉他在说话似的效果：其结果不再完美，而是充满了低音。

另一个有趣的选择是使用白噪声。当我们私语的时候，我们根本不会使用自己的声带，而振荡器的信号则极其微弱。同时似乎我们的声调也降低了。如果使用一个均匀分布的大声的噪音的话，则它将产生一种混杂的私语的效果，听起来特别深沉。

6.5.4 控制重新合成器

生成新的振荡器信号中唯一困难的一步就是要得到正确的音调和音量。这些可以从残差信号中提取出来（会有一些困难）。

音量是最重要的因素，因为需要在正确的地方保持沉默。它也是最容易计算的值，只需要对残差信号中语音的每帧进行一次均方根运算就行了。在输出的时候，最好对音量进行插值，这样声音就不会忽大忽小了。

音调则更难以处理。在 CD-ROM 上使用的方法如下所述。首先，假设音调是小于 1kHz 的，这样就可以把残差信号经过一个低通滤波器进行滤波以消去任何高于此频率的波段。然后，对波形进行归一化，中心对称对之进行折叠。这将使波形变形，得到一系列锥状的波形。接下来，将此波形与一系列脉冲波形进行比较，选出最佳匹配（通过最小均方的比较）。最后，对于每一帧，对最后的三帧进行计算取得中值（这将使得结果更平坦）。此方法并不完美——它只是有点取巧而已。不管如何，它可以得到相当好的结果——振荡器现在似乎和输入的语音相当合拍，而且一些不真实的效果也确实是我们期望的！

每帧的处理可以参见图 6.5.4。每帧的音调和音量现在控制了合成器，它是用来创建一个波形的，创建的依据就是针对特定扬声器选择的参数。然后此波形将通过滤波器以得到最后需要的重新合成的话音。请注意在图中我们忽略了一个可能——输出时的帧大小可能会与输入的帧大小不一样。这么做的目的是可以相对输入加快或是放慢话音的速度。

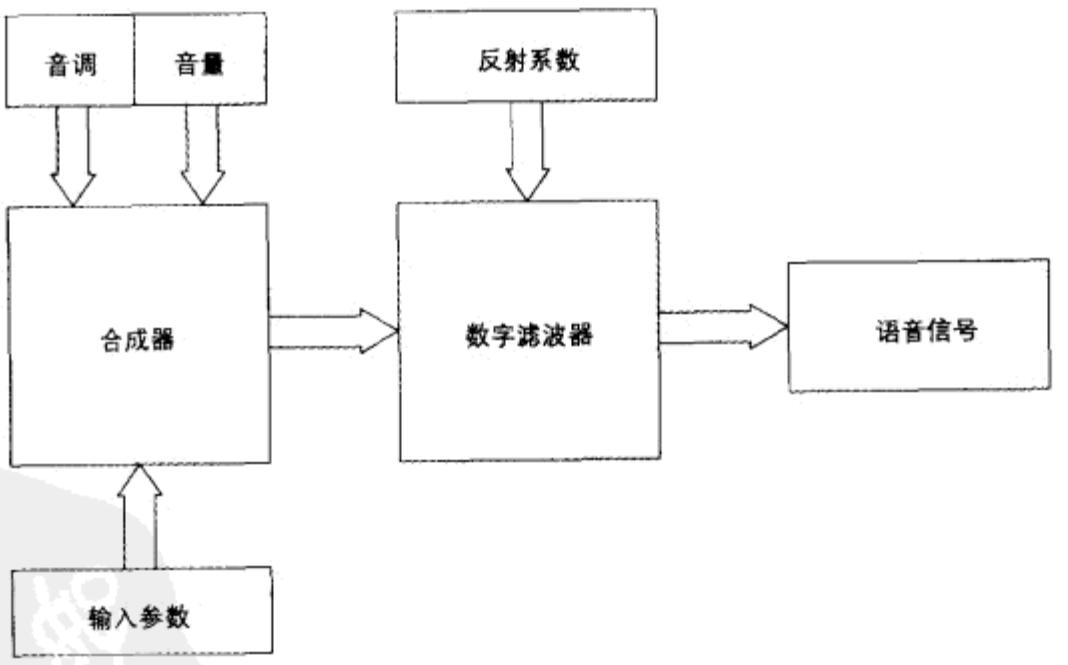


图 6.5.4 语音重新合成器

Download at www.Pin5i.Com

需要注意的是话音的参数（音调、音量和反射系数）与扬声器的参数（它们控制了合成器的参数和速度）是分离的。因此，在原理上来讲，同样的话音数据可以被不同的扬声器使用。例如，每个机器人的音调可能会互不相同，这样玩家就能将它们分辨开了；或是某些机器人说话的速度比其他的机器人要慢。而当对机器人造成伤害的时候也会影响到这一点，这样一个损坏的机器人可能会说得很快。

6.5.5　增加扬声器的深度

由于我们的反射系数反应的是一个共振器的物理模型，因此我们可以通过改变系数的方法来改变共振器。最简单的作法就是把系数增多，这样它就更“深”了。如果我们开始时有 8 个系数，那么现在就可以在其中插入 0 来得到 16 个系数。其结果就是共振更深了（会低一个音阶），听起来更深沉。

另外，如果把系数减少，那么滤波器就更“浅”了——话音就会变得模糊不清，就好像说话的人刚刚看过牙医似的。与处理残差信号比起来，处理反射系数的效果就不那么明显了，但是理论上来讲，应该是可以得到很多效果的。

6.5.6　对数据编码

在得到了音调、音量以及 8 个反射系数以后，我们现在在每一帧上就一共有 10 个数字了，它们给出了每一帧生成一个机器人话音或是怪声所需的所有条件。如果我们将每个数字量化成一个字节，这就将得到每秒 500 个字节的数据，或是 30KB 每分钟，正如前述。

对于某些游戏而言，我们做到这一步也就够了，其他的游戏则可能需要高品质的话音，或是更加自然、更逼近人声的话音。或许我们可以试试由一个源采样来生成各种不同的男子嗓音。在这种情况下，主要的问题就是编码。

残差信号很难被编码，因为它含有很多“丢失的”信息。例如，大部分话音使用的是声带，但是某些话音则无声的，这就意味着声带是松弛的，只有一点点空气释放到了发生的空腔里面去。字母 s 就是一个很好的例子。在这些情况下，残差信号就像是白噪声一样。我们可以针对这种情况检测出白噪声以改进编码。

其他的声音，例如 p，是由很多不同的过程共同作用产生的——在这种情况下，空气通过嘴唇进行“爆破”得到了 p 的声音。这些声音被称为爆破音。爆破音不是很适合使用这种发声－声带－加上滤波器的模型，因此大部分信息最后还是在残差信号里面。如果有足够的时间，我们有可能会对这些声音进行建模。

为了简化这些细节问题，我们应该对整个残差信号进行编码。这样的话，重新创建的话音就会和真实的声音尽可能地接近了。我们可以有很多方法来达到此目的——例如矢量量化、峰值检测或是 ADPCM 编码。我们甚至可能会将整个残差信号保存为一个 PCM 波形。这样我们就不能节省空间了（正好相反），但是如果能很方便地进行某些效果的处理的话，这也是值得的。[Degener94]描述了 GSM[ETSI00]如何将残差信号编码到 1.5k bit/s(千位每秒)。

Download at www.Pin5i.Com

6.5.7 速度

这种算法不仅给我们带来了更低的数据比特率，而且其性能也相当好（即使是在两代前的机器上）。虽然编码很难实时进行，但是解码则真是非常快的。

此重放算法的核心就在于这个滤波器。针对每一帧的每个系数，它将进行两次载入操作、两次加法运算以及两次乘法运算。假设一共有 8 个反射系数，这样大概每个采样会耗费 50 个周期，或者说在 8kHz 的时候是每秒 400 000 个周期。当然了，一个没有经过优化的版本可能会更慢一点。但是一个矢量化的版本则将要快得多（如果硬件支持的话）。

在一个经过优化的实现中，波形生成器将和滤波器放在同一个内部循环里面，另外还有一个包络生成器（envelop generator），大概每个采样会花费几个周期。因此，创建一个话音大概需要每秒 500 000 个周期——对于一个 16MHz 的处理器来说这只占用了它 3%的时间。

对于游戏来说，编码就不那么重要了，除非你想在因特网上使用 LPC 来传输话音。在这种情况下，在编码端需要做一些工作，但是一个当前主流的机器就应该能很轻易地处理这项任务了。

6.5.8 实验

在 CD-ROM 上的程序几乎把本节讲到的内容都给演示了一遍，因此请花时间看一下。你可以把话音载入到程序中去（提供了一个例子），然后程序将使用 LPC 算法对之进行转化。你可以使用一个残差信号或是程序中定义的语音来进行回放。我们提供了几个语音的例子。此程序包含了完整的源代码（微软 Visual C++），在 ReadMe.txt 里面还提供了完整的指示。

6.5.9 参考文献

[Cher98] Cher, *Believe*, WEA/Warner Bros., 1998.

[Degener94] Degener, Jutta, "Digital Speech Compression," *Dr. Dobbs Journal*(December 1994), 对应网址为 http://www.ddj.com/documents/s=1012/ddj9412b/.

[Degener00] Degener, Jutta, "GSM 06.10 Lossy Speech Compression," 对应网址为 http://kbs.cs.tu-berlin.de/~jutta/toast.html, July 2000.

[ETSI00] "Digital Cellular Telecommunicatio.ns System(Phase 2+); Full Rate Speech; Transcoding," (GSM 06.10 version 5.2.1 Release 1996), Third Edition, European Telecommunications Standards Inistitute, 2000. 对应网址为（需要免费注册）http://webapp.etsi.ord/pda/.

[Lucas99] Lucas, George, *The Phontom Menace*, 20th Century Fox, 1999.

[Nation63] Nation, Terry, *Dr. Who: The Dead Planet*, British Broadcasting Corporation, 1963.

Download at www.Pin5i.Com

6.6　复杂声音的随机合成方法

Phil Burk
SoftSynth.com
philburk@softsynth.com

随机合成是一种使用随机数的音频合成方法。你可以在底层使用随机数来生成噪声。然后此噪声可以被用来作为一个音源或是一个调制信号。你还可以利用随机数在高层进行声音的控制或是触发声音，那么这些随机控制或是随机触发的声音就可以被用来创建音景了。

大多数游戏使用数字音频采样来生成声音。使用采样的好处之一就是它们可以精确地将原声给复制出来。然而，使用采样有一个坏处，那就是它们比较单调。你可以改变采样的回放速度和其幅度，但是这仍然无法消除其单调性。真实世界的声音每次听来都是不同的。例如，拿一条狗的叫声来看，每次吠声都会略有不同。如果一条狗每次都叫得一模一样，这就会让人生疑。

有时候我们利用采样来生成连续的声音，例如风声。不过，这种方法最终将使用采样进行循环，此时声音的重复就可以听出来了。解决此问题的一个方法就是直接合成此声音。使用合成的好处之一是声音的设计者可以对更多的参数加以控制，你可以改变合成风声的特性让它呼啸，或是让它波动得更快。此时就不仅仅限于幅度和音调的改变了。

我们将提供一些例子，包括风声、声纳回声、雨声、火箭发动机的声音以及直升机螺旋桨的声音。这些例子是使用被称为单元生成器的简单的、基本的单元生成的。这些单元包括滤波器、振荡器或是音调生成器以及效果处理器，例如反射器。

6.6.1　线性同余算法

使用软件你是无法生成真正随机的数的。这是因为计算机只能生成可重现的、确定性的结果。不过，正如大部分游戏开发人员一样，我们也不免要自欺欺人一下。我们可以生成数列，它们听起来将很随机，即使实际上它们并非如此。

最常见也是最简单的伪随机数生成方法是线性同余算法（LCA）。它的基本原理就是进行乘法和加法，然后生成一个大得足以溢出计算机一个字长的数值。如果能事先精心地选择方程的系数，则我们就可以得到一个不重复的长达 2^{32} 的数列了[Chamberlin80]。如果我们要生成一个 44 100Hz

Download at www.Pin5i.Com

的采样则需要27个小时才会重复一遍。

为了使用此处描述的 LCA 函数，你就首先要给函数传入一个起始值，然后将结果反馈回函数以生成一系列的随机数。

```
/* Calculate pseudo-random 32 bit number using
 * the linear congruential method. */
static unsigned long GenerateRandomNumber(
   unsigned long previous )
{
   return ((previous * 196314165) + 907633515);
}
```

此函数返回结果的高位要比低位更加随机。因此，如果你想生成一个 16 位的随机数，那就不要对低16位进行位掩码，而要进行右移以得到高16位。

6.6.2 噪声种类

线性同余算法其直接的结果是一个白噪声，它听上去就好像是静态的“唏唏”声一样。白噪声在整个频谱上有同样的能量密度。而通过修改此算法，你就可以得到不同频谱的噪声了。

你可以每过几个采样就生成一个新的随机数，然后在它们之间进行插值以得到干涉采样，这样就可以生成红噪声了。这个方法在下面的代码和图 6.6.1 中展示了出来，它可以让你对频率内容进行很简单的控制。

```
// phase ranges from 0.0 to 1.0
phase += phaseInc;
if( phase >= 1.0 )
{
// grab a new random target whenever the phase wraps
    source = target;
    target = randomFloat();
    delta = target - source;
    phase -= 1.0f;
}
// linear interpolation between random values
outputs[0] = source + (phase * delta);
```

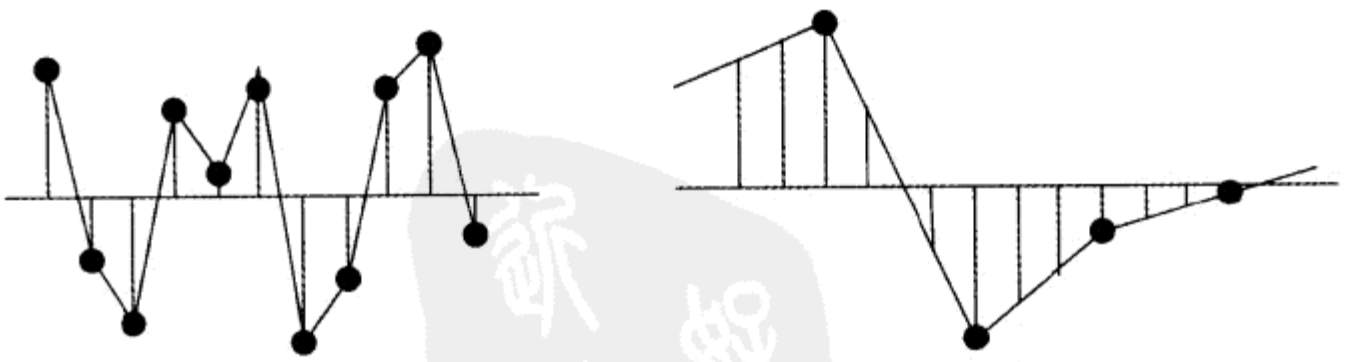


图 6.6.1 左图为每个采样均随机的白噪声，右图为间隔了随机数的采样，为红噪声

Download at www.Pin5i.Com

你也可以使用分形数学来生成噪声信号（其反馈函数是非线性的）。其中一个例子就是有了反馈的简单的正弦波[Roads96]。循环调用以下代码，你就可以生成采样或是控制信号了。

```
phase += phaseIncrement + (feedback * x );
if( phase > 1.0 ) phase -= 2.0;
else if( phase < -1.0 ) phase += 2.0;
x = sin( PI * phase );
```

在这个例子里面，相位范围是从－1.0 到 1.0 的。随着其反馈系数的提高，你就将得到一个更噪音、更混沌的信号。

6.6.3　软件示例

下面的各个小节描述了使用上述技术的真实的例子。其中提到的各种组件的源代码在 CD-ROM 上都有。

1. 生成一阵风声

你可以使用一个白噪声生成器（WhiteNoise）作为音源来生成风声。此噪声信号将通过一个门限值为 1000Hz 的低通滤波器（LowPassFilter）。如果把滤波器的 Q（品质系数）增加到 16 左右，你就可以得到一个很好的呼啸风声了。

参见图 6.6.2 以对此处提到的风声示例有一个总览。一个红噪声生成器（RedNoise）控制了滤波器的截止频率，对风声的随机起伏进行模拟。modRate 参数控制了风声起伏的频率。modDepth 控制了起伏的大小。接下来你就可以将 RedNoise 的输出乘以 modDepth，将其结果加上频率值以生成真正滤波器使用的截止频率了。如果将几个这样的生成器进行混合（或是相加），则我们就可以生成更为逼真的声音了。你可以把 modRate 设置为 0.6，将 modDepth 设置为 350 看看。

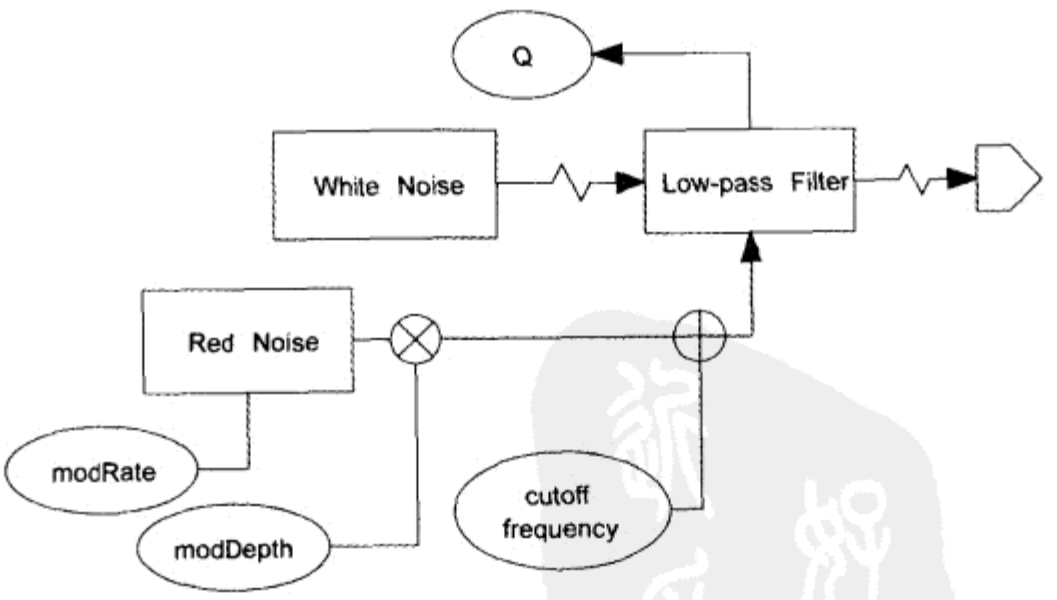


图 6.6.2　风声的例子

Download at www.Pin5i.Com

2. 生成声纳回声

一个声纳的回声由一个短暂的正弦波爆发并由其导致的水下各种物体的回声组成。此声音将随着时间的流逝逐渐消失。我们可以利用一个脉冲波输入一个声纳低通滤波器来生成正弦爆发音，就像使用一个锤子敲钟一样。为了对此进行实现，你需要把滤波器的截止频率设置到 700Hz 左右。为了让滤波器能持续一段较长的时间，把 Q 值设置得更大一点，例如 1000。如果想增强回声的强度，可以将脉冲的幅度设置为 7。

到此为止我们得到的仅仅是一个纯声而已。为了让它听起来好像是从水下的物体反射回来的一样，我们需要把它变得杂一点。我们可以加入大量的反射器，但是另外还有一个更简单的方法，它可能会对游戏开发人员更有吸引力。我们可以使用一个红噪声生成器对低通滤波器的截止频率进行调制。你可以将红噪声生成器的频率设置为 180Hz，将其幅度设置为 80 左右以得到期望的结果。

此电路与图 6.6.2 中显示的风声示例很相像，只是在图中要用一个 `ImpulseOscillator` 来代替 `WhiteNoise`。

3. 生成雨声

雨声是由数以百万计的雨点打到汽车上、窗户上、狗身上等的声音组合而成的。我们不可能生成每个雨点声，但是可以对其进行近似。当每个雨点撞击的时候，它将生成一个小小的扑通声。我们可以对每个扑通声使用一个振荡器模拟，但是这样需要的振荡器就太多了。因此，我们只能使用熟悉的回声滤波器来生成这个扑通声。使用回声滤波器的优点之一就是我们可以使用很多小脉冲输入滤波器，其结果与向很多滤波器中的每一个输入一个脉冲的效果是一样的。此滤波器的频率被设置为了 250，其 Q 值为 2。

每次我们输入的时候回声滤波器得到的结果都是一样的，除非对其频率进行调制。我们可以使用一个快速的红噪声生成器对滤波器的频率进行调制，正如上一个示例里面我们所做的一样。我们将把 `modRate` 设置为 10，把 `modDepth` 设置为 90 以得到期望的结果。

下一个技巧就是要生成很多随机分布的脉冲来输入滤波器了。我们可以使用一个比较器对一个随机信号进行观察。只要信号超过了门限值，我们就输出一个随机值，正如下面代码所示。否则，我们就输出零。通过此方法，我们就可以得到一个偶然的随机脉冲以用来激发此滤波器了（参见图 6.6.3）。

```
threshold = 1.0 - rate;
excitation = (nextRandom() > threshold) ?
    nextRandom () : 0.0;
```

我们可以调节上面 `rate` 的值来调整雨的猛烈程度。

回声滤波器可以生成离我们很近的单个雨点的声音。现在，我们还需要周围环境数百万雨点造成的回声，它们组成了雨声中烦闷的背景声。我们可以使用一个反射效果处理器以反射一些雨点的回响，这样就能生成很多雨点的声音了。反射器是由 6 个梳状滤波器和一个全通滤波器构成的[Moore90]。梳状滤波器有一个低通滤波器用来消除其他雨点造成的声音，让其听起来好像在远方一样。反射器还使用了一个比平常大得多的延迟，它在 200ms 到 350ms 之间，这样回响效果就不那么明显了。接下来我们将近处的雨点声与反射回的声音混合起来，

Download at www.Pin5i.Com

就得到一个完整的雨声了。如图 6.6.3 所示。

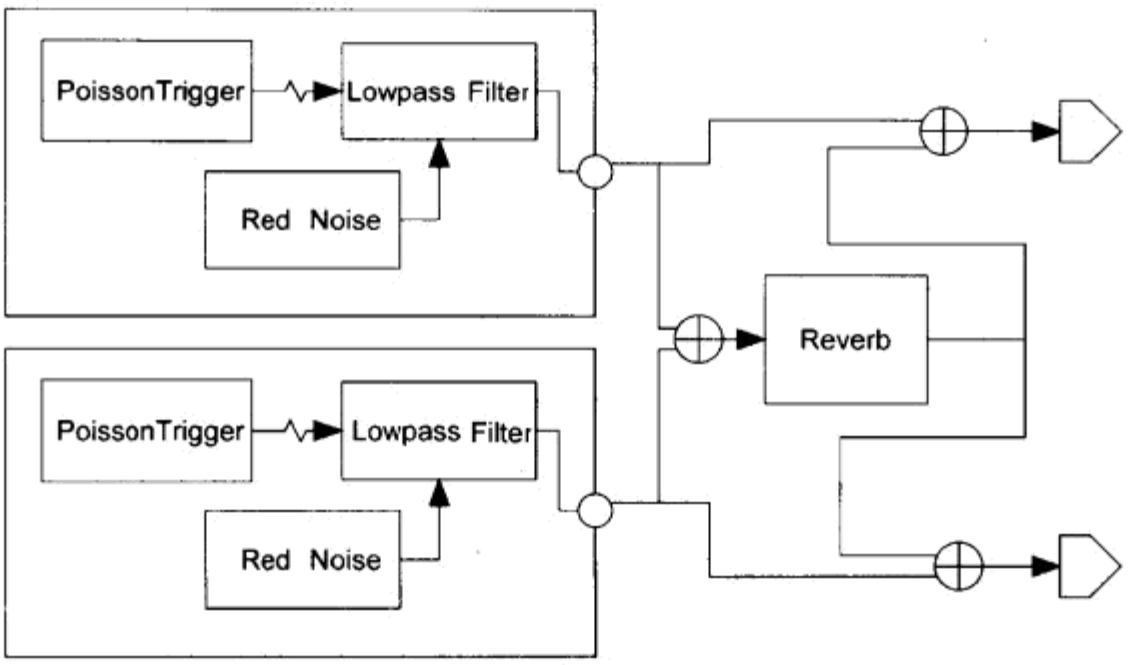


图 6.6.3　雨声的例子。两个雨滴发生器共享一个回声滤波器

你可以在图中添加不同的共振器以模拟雨点打到锡皮屋顶的声音或是打到玻璃天窗的声音。只要把低通滤波器替换下来，使用更复杂的滤波器组或是交叉耦合滤波器就行了。

4. 生成火箭发动机的声音

一个逼真的火箭发动机的声音只要使用两个红噪声生成器就可以生成了。一个红噪声生成器只能生成一个清晰的音调，没有一个真实火箭声音所特有的嘈杂声。因此我们需要使用一个红噪声生成器来对另一个生成器的频率进行调制。你需要把载波（即你听到的红噪声）的中心频率设置为 1300Hz，将调制器的频率设置到 731Hz 左右。避免使得调制器的频率成为载波频率的一个因数，另外还要设置调制器的幅度在 600Hz 左右。

为了模拟火箭开关的情况，我们还需要一种方法来将声音的大小迅速增大与缩小。我们可以利用一个信号变化限速器来做到这一点。此限速器将跟踪其输入，但是其速度不会超过一个事先给定的速度，正如下面的代码所示。这样我们就可以突然将幅度设置到 0，但是其实际的值则将更缓慢地降下来。

```
if( input > value )
{
    value += increment;
    if( value > input ) value = input;
}
else if( input < value )
{
    value -= increment;
    if( value < input ) value = input;
}
```

接下来我们就可以通过将红噪声信号与 SlewRateLimiter 的值相乘以控制电路的输出幅度了。图 6.6.4 显示了火箭发动机声音的逻辑电路。

Download at www.Pin5i.Com

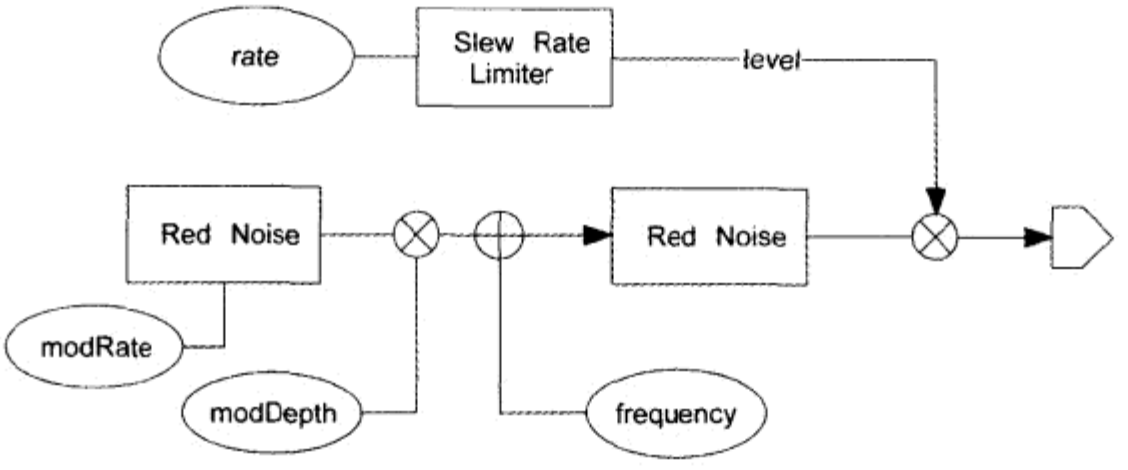


图 6.6.4 使用红噪声对与包络（由限幅器生成）生成火箭发动机的声音

5. 生成直升机螺旋桨的声音

一个直升机产生的声音主要由两个主要的部分：发动机的噪音和螺旋桨的噪音。螺旋桨将滑过空气，产生一种嗖嗖声。由于螺旋桨会旋转，因此其声音时而向我们逼近，时而又远离我们。这将导致产生多普勒效应，使得嗖嗖声的音调升高或降低。假设一辆汽车在路从你身边驶过。当它接近你的时候，其声音将越来越高，而当它远离你的时候，其声音则会越来越低。这是因为当汽车向你驶来的时候空气被压缩了，而波长越短则频率越高。

此声音将使用滤波后的白噪声作为音源，正如风声示例中的一样。接下来它将把声音经过两个不同的延迟传出。这两个延迟是使用一个正弦波调制器来回交换的。如果将正弦波相位翻转 180°，则我们将得到两个相对的桨片。因此，当一个延迟增大的时候，另一个就会减小。如图 6.6.5 所示。

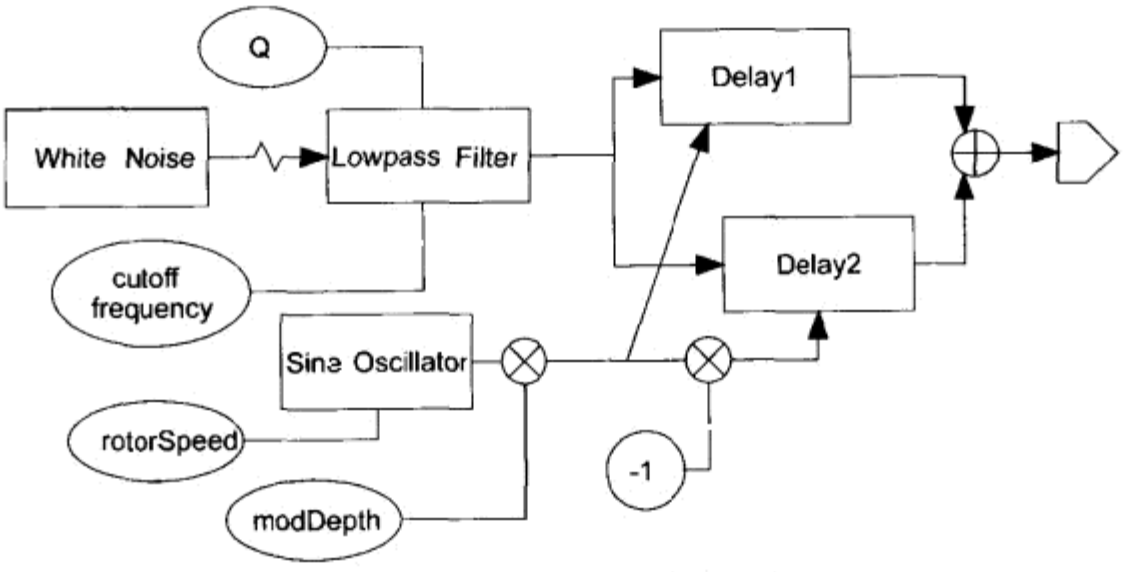


图 6.6.5 使用 180° 相移多普勒频移生成直升机螺旋桨声

下面，我们就可以根据螺旋桨的直径来调整其延迟时间了。当桨片最接近我们的时候延迟是最小的，反过来也是如此。

```
delayRange = rotorDiameter / speedOfSound;
modDepth = delayRange / 2.0;
```

一个 UH-1 Huey 直升机的螺旋桨直径为 48 英尺（约 14.6m）。此声音的速度大概是 1100 英尺/S（约 335m/s），因此多普勒效应相应的 modDepth 为大概 0.022 秒。使用上述的代码

Download at www.Pin5i.com

就可以得到此相应的 modDepth 值。

6.6.4　软件

在 CD-ROM 上提供了一个键盘驱动的 C++程序，它可以让你对这些声音做做实验，你可以打开或关闭声音，交互式地改变其参数。请参看 index.html 文件以对软件的设计有更多的了解。如果想了解此软件的更新情况，请访问[Softsynth02]。

此示例程序使用了 PortAudio，它是一个简单的、跨平台的音频 API。因此，此程序应该在 Win32、Macintosh、带有 OSS 的 Unix、SGI，以及其他的平台上都能运行。为了了解 PortAduio 的更新情况和得到针对你所在平台上的源代码，请访问[PortAudio01]。

6.6.5　结论

只需使用几个简单的单元生成器，你就可以无需采样合成出各种声音了。这样，你将能对声音参数进行更多的控制，可以使你摆脱采样循环带来的不真实性。随机函数在此处特别有用，因为它们不需要很多额外的计算量就可以增添很多复杂度。使用一个红噪声生成器对单个的滤波器进行调制的技术非常方便，你可以使用它来代替多个没有调制的滤波器。在更高层次上，随机过程也是很有用的，它们可以用来触发短时的声音事件，例如雨点声。你可以使用回响来对其处理，其效果就好像是你在电路中使用了比实际多得多的滤波器来生成此声音一样。我们希望这些技术能被你用来在下一个游戏中增添更多的音响效果。

6.6.6　参考文献

[Chamberlin80] Chamberlin, Hal, *Musical Application of Microprocessors*, Hayden, 1980.

[Dodge97] Dodge, Jerse, *Computer Music Synthesis, Composition and Performance,* Simon & Schuster, 1997.

[Moore90] Moore F. Richard, *Elements of Computer Music*, Prentice Hall, 1990.

[PortAudio01] Bencina, Ross, "PortAudio—an Open Source Cross-Platform Audio API," 对应网址为 http://www.portaudio.com/, 2001.

[Roads96] Roads, Curris, *The Computer Music Tutorial*, MIT Press, 1996.

[Softsynth02] Burk, Phil, "Biquad Filters and Stochastic Sysnthesis," 对应网址为 http://www.softsynth.com/gamegems/, 2002.

Download at www.Pin5i.Com

6.7 针对游戏的实时模块化音频处理

Frank Luchs
Visiomedia Software Corporation
gemsaudio@visiomedia.com

本节将阐述如何将模块化的音频处理集成到游戏中，这样以能在实时环境中通过程序生成声音。通常游戏会一次性或是循环地播放一段采样来在游戏中发出声音。虽然这一般也能用，但是这些静态的一次性或是循环采样的播放局限性较大，很难对用户的动作进行不同的响应。通过程序使用模块化的音频处理来生成声音则可以为开发人员提供更多的控制与变化，这样，由于在游戏与音频之间有了更为丰富的交互，在声音方面我们就可以得到更真实的效果了。

6.7.1 模块化音频处理

正如在那些古老的模块化合成系统一样，在软件中，我们也可以使用虚拟的模块与线缆来创建出一个音频处理系统。在两个系统中，信号处理的处理流程都是不固定的。你可以在各个单元之间进行连接得到一个音频处理流水线来生成一个声音，例如振荡器、滤波器以及包络生成器。通过使用单元的组合来生成声音，在合成各种声音的时候我们就可以得到最大的灵活性了，同时可以避免把所有这些可能的连接情况都使用硬编码来固化在程序里面。

音频处理的数据流是由单元之间的连接状况决定的。这些单元都是一个个封装的实体，它们可以对数据流进行各种特定的操作。

四十年前，合成器相当之大，需要一整间屋子才能放下，唯一保存模块（patch）及其连接的方法就是把它整个刻到一片偏振片上去。今天，纯软件的系统已经可以让你在硬盘上直接创建与保存上百万个模块了。将 DSP 处理单元加入处理流水线以后，我们就得到模块了。复杂的组件是架构在更简单的模块之上的，它使用的单元里面进行的是常见的数学操作的处理。

正如古老的硬件信号处理一样，在此处，我们也不需要你对软件的数字信号处理有多深的了解，因为你只要一边通过连接模块做实验，一边用耳朵来听就可以学到了。

Download at www.Pin5i.Com

6.7.2　通过程序生成声音

我们有很多通过程序生成声音的方法，而其中的许多种在电子合成方面都有一段相当长的历史。在此处，我们将描述一些通用的技术，这些技术已经为很多合成器使用过了，它们对于当前的实时软件合成而言不仅有效，而且实际。

经典的合成器使用的是线性合成的概念。典型的线性合成器有一个振荡器用来生成基本的音频信号，一般说来此信号是一个含有丰富谐波信号的声波。这个声波将通过一个低通滤波器，消除高频谐波，最后再通过一个放大器。在每个阶段都有相应的调制器。因此随着时间的改变，声音在音调、音色以及大小方面都会改变。

通常我们使用的有两种调制器。一种是低频振荡器（LFO），它是一个在次声波频段（20Hz之下）运行的振荡器。它生成的是周期性的调制信号。当你将之连接到一个音频振荡器的时候，就会得到颤音；当你将之连接到一个滤波器的时候，得到的将是“呜呜”的声音；当你将之连接到一个放大器的时候，其结果则是一个带有震动的颤声。另一种调制器是包络生成器，它是针对一次性调制的，通常由键盘触发。包络生成器可以生成一个声波的外包络。

另一种重要的合成方法是 FM 调制，它是在声频范围内对频率（或是相位）的一种调制。当你需要从很少几个振荡器得到一个丰富的频谱信号的时候，它就很有用。当然了，振荡器也可以用来生成简单的线性插值波表信号（如果想要了解有关合成技术的历史应用，请参看[Smith91]）。

6.7.3　Sphinx MMOS 系统

在下面几个小节中，我们将介绍一个称为“Sphinx MMOS”的系统，它可以用来在游戏中通过程序生成声音。你可以在 CD-ROM 上找到此系统的源代码与可执行文件。

Sphinx MMOS 是建构在 PD[PDRef]、MAX[MAXRef]以及 Generator[GENRef]之上的。此外，它还有一个可扩展的面向对象模型，用户可以通过此模型来在不同层次上创建声乐效果。它使用了知名的模块模型来简化声音应用的开发。

对于本系统中单独的各个元素，我们使用了一些不同的术语。表 6.7.1 中对 Sphinx MMOS 中使用的术语与其他常见的术语做了一番比较。

表 6.7.1　　在 Sphinx MMOS 中使用的术语

Sphinx MMOS 中的术语	其 他 名 称
处理器	单元、滤波器、块、节点、（DSP）模块
引脚	输入/输出、入口/出口、槽、端口
信号	数据、媒质、流、采样、载荷、信息
模块	流图、处理链、网络

Sphinx MMOS 是一个面向数据流的系统。在这样一个系统中，你将把各个原始单元连接起来以创建一个数据处理的网络。每个单元（此处称其为“处理器”）都有输入引脚，可以从上游连接的处理器接收数据；还有一个或多个输出引脚，可以把处理后的结果发送到下游的

Download at www.Pin5i.com

处理器去。为了效率方面的考虑，所有的数据都是成块处理的，而在 Sphinx MMOS 中，块的大小缺省是 256 个采样。如果采样率为 44 100Hz，则我们的缓冲区时长就是大约 6ms 了，对应的帧率大约为 172Hz。

6.7.4 处理器

在 Sphinx MMOS 中有很多种可选的处理器。最常见的类型就是振荡器、噪声生成器、包络生成器、滤波器以及混音器。为了对声音触发加以控制，我们还有针对脉冲的生成、序列化以及选择的处理器。每个处理器都有其自己特定的连接要求。

表 6.7.2 展示了在 Sphinx MMOS 系统中的的某些处理器。我们将在下一个小节讨论一下如何将这些处理器连接起来。

表 6.7.2 在 Sphinx MMOS 中的部分处理器

振荡器	噪声生成器
CGASineOscillator	CGARandomGenerator
CGATriangleOscillator	CGANoiseGenerator
CGASquareOscillator	其他类型处理器
CGASawtoothOscillator	CGAAverager
CGASinCosOscillator	CGAGlide
CGASineXOscillator	CGAInterleaver
CGASawtoothOscDSF	CGAThreshold
CGASawtoothOscBlit	触发器与选择器
CGAWaveTableOscillator	CGAImpulseGenerator
包络	CGASequencer16
CGATriangleEnvelope	CGASelector
CGAHalfCosineEnvelope	混音器
CGAGuassEnvelope	CGAMixer2,3,4,6
CGARampEnvelope	CGABalance
滤波器	
CGAButterworthLPF	
CGAResonator	

6.7.5 模块文件简介

Sphinx 的模块文件是你可以通过文本编辑器进行编辑的简单文本文件。下面让我们看看一个简单的模块。它描述的是一个基频为 200Hz，使用一个 LFO 在 3.5Hz 进行调制的振荡器。FMAttenuation 被设置为 0.1，这就将调制深度限制在了+/一一个音阶以内。所有音频处理流水线上的处理器都列在了这个模块里面。此外，所有的参数都放在了这个对象的括号里。

```
CGAPatch "Sine Oscillator with LFO Modulation"
{
    CGASineOscillator LFO
    {
        Frequency = 3.5;
    }
```

Download at www.Pin5i.Com

```
    CGASawtoothOscillator OSC
    {
        Frequency = 200;
        FMAttenuation = 0.1;
    }

    Connection = LFO, OSC, SampleOut, FM;
}
```

模块中的最后一行把 LFO 的采样输出与 VCO 的（双极）频率调制输入连接了起来。下面的代码是一个例子，它说明了如何使用硬编码在游戏里面实现上述类似的功能：

```
void SamplePatch()
{
    IGAPatch* pPatch = CreatePatch(
        "Sine Oscillator with LFO Modulation");

    // 0 modulator
    IGASineOscillator* pLFO =
        PCREATE(pPatch,SineOscillator);
        pLFO ->SetFrequency(3.5f);

    // 1 oscillator
    IGASawtoothOscillator* pSaw2 =
    PCREATE(pPatch,SawtoothOscillator);
    pSaw2->SetFrequency(200.0f);
    pSaw2->SetFMAttenuation(0.1f);

    // connections
    pPatch->Connect(0,1, SampleOut, FM);
}
```

第一行产生了新的模块，然后将之加入应用程序的模块列表中。通过下面的两行 PCREATE 调用，我们创建了两个处理器。PCREATE 是一个方便使用的宏，它用来将处理器的类型转化为 CID 指明的类型，创建对象，然后将其添加到我们模块中的处理器列表里面去。创建的顺序很重要，因为它定义了流水线中的处理顺序。最后一行将两个处理器连接了起来。其参数是处理器在流水线里面的索引号与它们的引脚。在此处，我们将振荡器 0 的 SamplOut 引脚与振荡器 1 的 FM 引脚连接了起来。

下面所示的是对 Sphinx MMOS 进行初始化、创建以及播放此模块，最后收尾的步骤。

```
g_pGASystem->Initialize()

SamplePatch();

g_pGASystem->Start();

// run application loop here and handle
// start/stop and parameter setting somewhere
```

Download at www.Pin5i.Com

```
// in your event handling
RunLoop();

g_pGASystem->Terminate()
```

其中的 g_GASystem 是一个全局变量，它代表了在我们的应用程序中的单实例音频对象。如果没有特殊声明，所有的渲染都是针对缺省流进行的。对于音频渲染来说，当前版本的 Sphinx MMOS 使用了它自己的一套包装类，对出色的 PortAudio 系统进行了封装[Burk01]。

6.7.6 模块文件的应用

现在我们已经对基本的模块定义语言进行了一番描述，下面就可以创建复杂的声音了。由于我们可以对处理器与其之间的连接进行描述得到任何的信号处理路线，因此理论上可以生成无限种模块。此外，这些模块可以被游戏控制，因为我们可以对一个给定的模块选择不同的游戏变量来代替其中某个处理器，这样我们就可以在结构上控制数字声音的生成与信号处理的过程了。甚至，我们可以将任一游戏数据中或是用户界面给出的动态值连接到一个 CGAController 上，这样就可以在不改变采样率的情况下修改音频流了。

为了展示我们是如何将模块系统应用到游戏中去的，下面我们就将描述一下应该怎样设计模块配置以对发动机的声音进行模拟。不过我们将不会覆盖到相关模块的每个细节，而只是着重描述其中某些重要的特性。此外我们还将展示如何利用复杂的模块逐步对发动机模拟的质量进行改进。

下面各个小节中描述的模块都可以在 CD-ROM 提供的模块中找到。

1. 机动车辆

一辆车的声音是一个由数个音源造成的复杂信号。其中有很多确定性与随机性的因素同时在起作用。确定性的因素是与发动机的每分钟转速有关的。随机性因素则主要是由进气系统与排气系统的空气湍流造成的。当速度比较低的时候，声音的特征主要是由确定性因素组成，当时当速度比较高的时候，随机性因素则占了主导地位。

我们在一个发动机模块中使用的主要概念就是对声音的几个阶段进行模拟。为了生成复杂而难以预测的声音，我们混合使用了合成声音的循环采样和简单的一次性采样。我们使用了一个音序器控制声音的时间。脉冲生成器用来触发一个选择器的开关。每个输入选择器的脉冲将选择下一个采样的来源，同时还将重置采样的初始值。在某种程度上，这与鼓盒的开发差不多，其区别仅在于速度。我们将加快此循环以进入音频范围。一个鼓音序列大概是每分钟 120 拍，而我们的发动机则需要达到 4000RPM 的转速甚至更高。

- PowerStroke 模块是用来模拟一般的四缸四冲程发动机的。这些反复运行的四冲程被称为进气、压缩、释放和排气。我们将使用采样来模拟其中的三个阶段。在模块文件的开始，我们将把采样载入到 CGASignal 对象中，为它们赋予相应的名称（进气、脉冲、排气）。这些名称（并非文件名）将被作为参考值为一个 CGAWavetableOscillator 的 WaveTable 参数所用。

Download at www.Pin5i.Com

波表振荡器可以播放循环采样或是一次性采样。我们希望从一个 CGAImpulseGenerator 中触发信号波，因此就将 Repeat 参数设置为 1。脉冲生成器的速度，即 RPM，是由一个低频振荡器控制的。每个脉冲都将在 CGASelector 中进行下一个输入源的选择。为了得到四个出入，我们将 Selection 参数设置为 15(1 + 2 + 4 + 8)。对于压缩阶段来说，我们没有相应的输入源，这就意味着这个阶段将是无声的，而且选择器将用零来填充相应的输入。

- PowerStroke with Muffler 模块有额外的两个 CGAResonator 类型的滤波器，以用来模拟笑声器的回声。原始信号与共振器的信号在一个 CGAMixer3 对象里面进行混音。为了得到更高速度上的轰鸣声，我们把共振器的输入增益设置得很高，但是使用了速度控制器来控制 AM 参数。
- PowerStroke 2 Cylinders with Muffler 模块在整个范围 RPM 上的声音之间加入了混合衰减。在整个 RPM 范围上，声音的组成应该有一点点不同，这样就可以生成更加真实的音质上的改变了。

2. 直升飞机

- Turbine 模块播放的是循环的涡轮采样。此采样同时包含了尖锐的声音和噪声因素。与一般的采样编辑不同的地方是，此处的循环点是特意选出来以表现周期性特性的。

  CGAWavetableOscillator 通过一个平滑后的 LFO 源进行频率调制。为了得到具有不同泛音的复杂而响亮的声音，我们将给 CGAWavetableOscillator 加入一点对音频进行的 FM 处理。此处还需要一个 CGAMixer 以将由 LFO 和 VCO 出来的调制信号进行混合。其结果就是一个 Turbine AudioFM1 的模块。如果我们为调制采样的 VCO 加入一些 FM 调制，那么就可以得到更有趣的结果，其结果就是一个幅度调制，而它将增强缓慢旋转的音效（Turbine AudioFM2）。
- Rotor-Enveloped Sample 模块演示的是一个由 CGAGuassEnvelope 调幅后的循环采样，此处使用 CGAGuassEnvelope 是为了得到柔和的加速声与减速声。采样 m_propeller01.wav 由一个很强的噪声部分以及一个低频的变形后的正弦波组成。包络是由一个 CGAImpulseGenerator 触发的。触发点与循环的起始点相互独立，这样声音的变化就比较小了。
- Turbine + Rotor 是一个 AudioFM2 Turbine 与包络转子（enveloped rotor）的混合产物。涡轮的频率与包络的触发频率是由同一个 LFO 控制的。
- Helicopter with Turbine, Rotor and Flap Fx 模块使用了序列化的波表以及反馈 FM 的技术对发动机以及空气震动进行模拟。正如在车辆发动机上一样，我们使用了 CGAMixer 的平衡参数在其输入中进行混合衰减。此模块的组成部分有以下几个：Jet/Turbine 发动机的进气与排气声、基本的带有桨参数的主转子声与尾转子声、一个额外的生成器用来增强在更高速度上的空气噪声以及为了生成特有的鼓翼声效的一个边缘脉冲。在远处听来，直升飞机的声音带有很强的幅度变化，还有由空气湍流产生的间接的消音效果。对于较短的爆发声来说，声音有时候会完全消失或仅仅是一个单独而强大的脉冲。在我们的模块中，这种特有的互扰音效是在原始的鼓翼采样上加上四毫秒的延迟生成的。而延迟时间是由一个 LFO 缓慢调制的。其结果就是在音频频谱中加入了一

Download at www.Pin5i.Com

个移动的切槽，这样就得到了一个很好的鼓翼音效。

- Turbine Controlled by Keypad 是一个交互式的演示，它展示了如何使用小键盘来控制涡轮的采样。循环的频率是由数字键盘上赋予某值的一个键来控制的。你可以按数字 1 到 9 来切换不同的涡轮速度。对于切换效果，我在 `CGNumpadController` 与 `CGAWavetableOscillator` 之间连接了一个 CGAG lide。

3. 潜水艇

一个潜水艇的声音是由机械与设备的噪声、舱壳的回声以及推动器生成的噪声组成的。其中宽带部分是水流噪声、气泡噪声和海洋的自然噪声背景。为了生成具有潜水艇特征的声音，我们使用了两个特殊的振荡器来生成粉色噪声调制的正弦波。它可以模拟湍流噪声气泡和含有船体离散回声频率的音调噪声中的不规则与随机的过程。这段连续频谱跨越的最大区域大概在 50Hz 到 100Hz 之间，但是真实的频率值可能会达到 1000Hz。

推动器的声音是使用一个含有更高频率的循环采样模拟的。当推动器的激活频率等于舱体的自然频率时，我们就可从舱体得到较强的声音。此外，当涡街（Vortex-shedding）频率与桨翼的频率匹配的时候也会引起振鸣。这是一个在窄带速度范围之内发生的与速度相关的反馈，而且在不同速度都可以发生。

当船体通过水流的时候，沿着舱体会产生湍流噪声。当速度增加的时候，这种水文噪声也会随之增加。气泡与水流噪声的频率可以高达 10 kHz。

- Sonar-Noise Modulated Sine 模块是一个 2000Hz 的正弦波，其幅度是一个斜线下降的波形调制的，而频率则是一个噪声波调制的。噪声调制的包络生成的是逐渐衰减的效果，这样音调在开始是清晰的，但是随后就会越来越模糊了。
- Submarine-Engine Only 是一个带有发动机循环的调幅采样。其中使用了一个 CGASinCosOscillator 以生成衰减效果。接下来的湍流模块则使用了两个特殊的振荡器来生成音调噪声，CGANoiseMSinceOsc 使用的是缺省的配置，而 CGASine XOscillator 则使用的是针对更高范围湍流的 860Hz 的频率设置。
- Submarine-Engine, Turbulence 是一个混合对象，它由发动机采样、由 CGANoiseMSine Osc 生成的低频噪声以及由 CGASineXOscillator 生成的高频噪声组成。我们使用了一个三通道的混音器，即 CGAMixer3，而且将高频的增益设置得很低，这样就可以得到一个真实的滤波后的水下效果了。本模块不需要额外的滤波器。
- 在 Submarine-Engine, Turbulence, Sonar 的模块中间，我们包含了以上声纳模块中的元素，此外还为发动机采样增加了额外的频率调制，这样就可以模拟多普勒效应了。再加上一个 400Hz 的低通滤波器，我们就可以得到一个更加真实的水流效果了。

6.7.7 源代码

在 CD-ROM 上，你将找到 Sphinx MMOS 系统的源代码、可执行文件、模块文件以及示例程序。一些模块文件演示了各种处理器，一些则包含了上述各个小节讨论到的发动机模拟。读者可以访问 Visiomedia 的网址[Visiomedia]以得到更新信息。

Download at www.Pin5i.Com

你可以使用三种方法将音频系统集成进自己的应用程序里面：

1. 编写新的 C++类进行音频处理和管理，使用定制的源代码建构自己版本的函数库。

2. 编写一个插件，里面含有使用 Sphinx 基本接口，IGAProcessor 的 C++类，然后把你的插件与原始的 Sphinx MMOS 的插件一起载入进去。

3. 直接使用系统，只创建新的模块文件。

6.7.8　结论

本节提供了一些经验之谈，通过它们，读者可以将模块化的音频处理与程序化的声音合成结合起来，以在游戏中生成各种不同的车辆声音。在通过程序实时生成音频的系统与传统的播放系统之间有个主要区别，那就是它们在游戏环境中复杂的管理和声音数据处理的响应速度有所不同。我们将以音频生成的采样与常见的合成方法结合了起来，这样读者就可以为自己的交互式应用程序创建特有的声音效果了。

6.7.9　参考文献

[Ackermann94] Ackermann, Philipp, "Design and Implementation of an Object-oriented Media Composition Framework," 对应网址 http://citeseer.nj.nec.com/ackermann94design.html, 1994.

[Burk01] Burk, Phil and Ross Bencina, "PortAudio - An Open-Source Cross-Platform Audio API," 对应网址为 http://www.portaudio.com, 2001.

[GENRef] Generator by Native Instruments, 对应网址 http://www.native-instruments.net/.

[MAXRef] MAX / MSP by Cycling 74, 对应网址 http://www.cycling74.com/index.html.

[PDRef] Puckette, Miller, "Pure Data Dot Org," 对应网址为 http://www.pure-data.org/.

[Rabin00] Rabin, Steve,"Classic Super Mario 64 Third-Person Control and Animation,"*Game Programming Gems 2*, Charles River Media, Inc., 2001.

[Sim-Schmitz99] Sim, Ben and Fredric Schmitz, "Acoustic Phasing and Amplification Effects of Single-Rotor Helicopter Blade-Vortex," Proceedings of the 55th Annual National Forum American Helicopter Society, 1999.

[Smith91] Smith, Julius O., "Viewpoints on the History of Digital Synthesis," Proceedings of the International Computer Music Conference, October 1991: pp. 1～10.

[Visiomedia] Updates to the Sphinx MMOS system, 对应网址为 http://www.visiomedia.com/rooms/ labor/src/sphinxmmos/index.htm.

[Wang99] Wang, Geng, "Prediction of Rotorcraft Noise with a Low-Dispersion Finite Volume Scheme," 对应网址为 http://www.ae.gatech.edu/~lsankar/CERT/ 1999.

Download at www.Pin5i.Com

Download at www.Pin5i.Com

索　引

FILE：27
LINE：27, 28, 29
3D 音频环境（3D audio environment）：510, 519, 522
A*算法：204~205, 214, 250, 260~269
A*探索器（A Star Explorer, ASE）：261, 269
Aaron Nicholls：85
ActionState：206, 442
Actor：207, 445
ActorProxy：446
Adam Lake：295, 353
ADPCM：512, 513, 543
AIControlState：207, 443
Allen Pouratian：34
Andrew Kirmse：424, 487
ASE：*请参见* A*探索器
AST：*请参见* 抽象语法树
BaseSimulation：450
Ben Board：59, 214
Bison：84
Blinn-Phong 模型：417~419
Brian Hawkins：40, 117, 295, 361
Byon Garrabrant：133
Carl S. Marshall：4, 316
Carlo Vogelsang：510, 524
CCD：*请参见* 循环坐标推演
CFD：*请参见* 计算流体动力学
CFSM：228
Charles Farris：226
Chris Oat：407
Christopher Christensen：271
CLDC ：*请参见*面向连接的受限设备配置
ControlState：207, 443
Conway 的生命游戏（Conway's Game of Life）：181
cPLP 算法（cPLP algorithm）：312
CProfileIterator：139
CProfileManager：137~138
CProfileNode：138
CProfileSample：137
CryptoAPI：484, 486
Cygwin：34, 39
David Fox：424, 501
DirectPlay8.1：490
DirectSound 3D：519~521
Don Hopkins：106
DPNSVR：491
Drew Card：323
DTD：100
Eddie Edwards：510, 535
Eric Lengyel：294, 297
Eric Robert：123
FEA：*请参见* 有限元分析
FEP：*请参见* 前端进程
Flex：84
Frank Luchs：550
Garin Hiebert：519
GoCap：213, 452
Goertzel 算法：157
Graham 算法：244
Greg Seegert：99
Greg Snook：364
HMAC：*请参见* 基于散列函数的消息认证编码
ICV：*请参见* 完整性检验值
IIR：*请参见* 无限冲击响应

Download at www.Pin5i.Com

IME：*请参见* 输入法编辑器
IPSec：*请参见* 因特网协议安全
Jack Moffit：510, 512
Jan Kautz：294, 416
Jan Svarovsky：424, 432
Jason Beardsley：424, 466
Jason Shankel：295, 302
Jason Weber：173, 295, 337
Java2 微型版（Java 2 Micro Edition (J2ME)）：502
Jeff Lander：294, 338, 359
Jim Greer：424, 425
John Byrd：142
John Isidoro：407
Justin Randall：424, 453
Kenneth Hurley：295
Kim Pallister：2, 395, 399
LCA：*请参见* 线性同余算法
Lex：39, 77, 80~84, 360
List::splice：51
LOF：*请参见* 火力线
LookupManager：449
LPC：*请参见* 线性预测编码
Mark Zarb-Adami：169
Martin Brownlow：54, 295, 307
Michael Harvey：4
MIDlet：502
MIDP：*请参见* 移动信息设备框架
Miguel Gomez：193
Mike Milliger：295, 394, 400
n.h/h.h：407~409
Naty Hoffman：377
NetTool：487
NDF：*请参见* 法线分布函数
NLS：*请参见* 归一化光照空间
NTP：*请参见* 网络时间协议
Ogg Vorbis：510, 512~515, 518
OpenAL：520, 521, 523
Oscar Blasco：295, 370
Paul Kelly：76
Performer：445
Perlin 噪声：389, 393, 395, 402
Pete Isensee：44, 424, 477
Phil Burk：510, 529, 543
PIMPL：119
POSIX2：455
PRNG：*请参见* 伪随机数字生成器
Remez 交换算法：164
Robin Green：155
RTS：*请参见* 实时策略游戏
Schur 算法：538
Scott Patterson：510, 13
SimulationObject：443
SimulationState：442
SOBFactory：447
SOBManager：447
Søren Hannibal：64
Sphinx MMOS 系统：551, 552, 556
SPI：*请参见* 安全参数索引
Stephen White：271
Steve Rabin：23, 252
Steven Woodcock：204
STL：*请参见* 标准模板库
TaskSys-t：17, 18
TCP：428, 503
Thomas Demachy：67
Thomas Young：147, 282, 295, 345
Thor Alexander：206, 424, 440
Tom Forsyth：400, 180
T 形连接：273, 294, 297~299
UDP：429, 488, 503
UML：*请参见* 统一建模语言
Unicode：90, 98
UserControlState：207, 211, 443
Van der Sterren：260, 270
Wagner T. Corrêa：310
William Leeson：295, 327
XML：100, 105, 116

Download at www.Pin5i.Com

Yacc：39, 77, 80, 82, 84, 360
安全参数索引（Security Parameters Index，SPI）：479
安全连接（Security Associations）：478
安全套接字（secure sockets）：424, 477
八叉树（octrees）：464, 183
白噪声（white noise）：533, 540, 544, 545, 548
半边缘的数据结构（half-edge data structure）：330
贝塞尔（Bézier）：307~309
背景裁剪（back-face culling）：310, 332
被动场景（passive scenery）：182
本地化（Localization）：3, 85, 92~98, 101, 102
逼近函数（approximating function）：163, 167
边合并（Edge Collapse）：325
编解码器（Codecs）：510, 513, 514, 517, 535
编译期常量（Compile-time constants）：25
编译期断言（Compile-Time Assert）：26
标识符（token）：36, 38, 79~85, 358
标识符分析器（tokenizers）：36
标志器规则（Flag Policy）：119
标准模板库（Standard Template Library（STL））：102, 117
表面细分（subdivision surface）：295, 327, 332, 336
饼状菜单（pie menu）：100~120
不精确结果（Inexact Result）：65
布尔操作符（Boolean operators）：253, 255
参考坐标（reference pose）：174, 337
残差（residual）：537~542
产品包装阶段（Wrapping Stage）：14, 20
宠物小精灵（Punkemon）：109
抽象语法树（abstract syntax trees（AST））：355, 358
触发器系统（Trigger Systems）：205, 252~259
触摸屏（touchscreens）：116
词法分析器（lexer）：36, 77, 78, 356
大型整数（Big Integers）：153
代价函数（cost function）：261~270
带通滤波器（band-pass filter）：533
单元自动机（cellular automata）：180, 395, 398, 402
等测的（isometric）：416
低通（low-pass）：142, 521, 524, 529~534, 540, 545~547, 551, 556
地平角（Horizon Angles）：379
地形分析（terrain analysis）：205, 237~251
第一人称视角射击类游戏（First- Person Shooter）：432, 436, 454
调度器（scheduler）：4~11
调试标志（Debug flags）：117~119
顶点积累缓冲（vertex-accumulation buffer）：331, 332
顶点法线（vertex normal）：295, 302, 303~305, 307, 308
定位 new 操作符（placement new operator）：47
定位修正器（anchored modifier）：351
定位音频渲染（positional audio rendering）：519
定制的分配器（custom allocators）：44, 45, 51
丢包（packet loss）：104, 429, 438, 504
动态区域（dynamic areas）：238
动态障碍（dynamic obstacles）：278, 271, 280
动作捕获（motion capture）：206, 337, 345, 347, 349, 352
对等（peer-to-peer）：425, 426, 433, 455, 466, 475, 478, 490, 496, 499, 503
对流（Convection）：188
对数（logarithms）：61, 76
多叉树（N-ary trees）：134
多线程（Multithreading）：10, 11, 379,

Download at www.Pin5i.Com

494~496
多项式纹理图（polynomial texture maps）：381
法线分布函数（normal distribution function（NDF））：416
法向地图（normal maps）：295, 370~376, 379, 404
分块化的（tesselated）：303
分数矢量（vector fractions）：150
浮点异常（floating-point exceptions）：3, 64~66
符号表（symbol tables）：174, 355~358
辐射量（irradiance）：378
辅助函数（utility function）：46, 246, 469, 472
概念阶段（Concept Stage）：13
高程场（heightfield）：201, 295, 302~305, 378, 380, 382, 385, 396, 404, 418
高级语言（high-level languages）：2, 353, 355, 359
高通（high-pass）：529, 530, 532~534
格网（grid）：303, 307, 345
共振器（resonator）：536, 537, 541, 547, 555
古怪奇异的声音（alien voice）：535
骨架的（skeletal）：174, 287, 309, 337, 345, 347, 348, 350
骨节（bone）：169, 173~177, 295, 331, 337~344, 351
关隘（choke point）：237, 240, 241, 247~250
光束（beam）：295, 323, 324 , 361~363
光晕（Iridescence）：411
归一化光照空间（NLS （normalized light space））：412
滚降（roll-off）：521
函数绑定（function-binding）：34, 35, 38
函数指针（function pointers）：101, 205, 226~235
合成器（synthesizers）：535, 536, 539~541, 550, 551
核心处理模型（Core Processing Model）：184
核心类（core classes）：443
红噪声（red noise）：544~549
后台进程（back-end process）：460, 461
花印（decal）：403~406
滑动摩擦力（dynamic/kinetic friction）：193, 194, 198, 200
画板（billboard）：295, 361, 362, 364~369
缓冲区规则（buffer policy）：120
火箭发动机的声音（rocket engine sounds）：543, 547, 548
火力线（LOF （line-of-fire））：260~262, 266~269
机器学习（Machine learning）：205, 206, 285
基于函数的（function-based）：14, 16
基于距离的裁减（distance-based culling）：462, 463
基于散列函数的消息认证编码（hash-based message authentication code（HMAC））：480
基于帧的（frame-based）：5, 7, 14~16
几何模型（geometry）：294, 295, 298, 299, 309, 316, 385, 396, 397
计算流体动力学（computational fluid dynamics （CFD））：192
寄存器（registers）：65, 66, 146, 153, 353~359, 383, 398, 413, 530
加密（Encryption）：58
接缝（welding）：298, 301
解密（Decryption）：453, 454, 477~485
解引用（Dereferencing）：41~43, 438, 473
静摩擦力（static friction（stiction））：193, 195, 196, 198, 200
静态区域（static areas）：193
静态障碍（static obstacles）：216, 221, 271, 272, 277, 278, 280
镜面图（specular map）：393, 396
巨量多玩家游戏（massively multiplayer

Download at www.Pin5i.Com

game）：424, 440
具有容错性的 AI（Fault-Tolerant AI）：283
聚光灯（Spotlights）：324, 413
可扩展的服务器（scalable servers）：455
可玩性阶段（Playability Stage）：14
客户机/服务器（Client/Server）：207, 424, 432, 440, 441, 445, 449, 466, 468, 490, 499
库仑摩擦力（Coulomb friction）：193
跨平台（multiplatform）：26, 64, 66, 468, 534, 549
类图（class diagram）：70~74, 207~212, 440, 442~449
立体图（cube map）：295, 387~393, 407, 408
滤波器（filter）：529, 551, 552
路径遍历（path transversal）：221
轮廓区域（Silhouette regions）：148, 149
蒙皮（skinning）：308, 309, 332, 338, 351
密钥管理（key management）：484
面法线（Face normals）：302
面向连接的受限设备配置（CLDC）：502
瞄准（aiming）：263~365
逆向运动学（inverse kinematics）：173, 350
欧拉角（Euler angles）：57, 175
偶掩码（Even masks）：327, 329, 334
碰撞检测（collision detection）：4, 11, 182, 183, 209, 238, 288, 307, 372, 464
平衡锁定（Gimbal lock）：175, 177
奇掩码（Odd masks）：327, 334
启发式函数（heuristic function）：261, 262, 269
前端进程（front-end process（FEP））：460, 461, 464
嵌套的迭代（nested iterations）：61
窃听（sniffing）：443, 453, 477, 485
球状树（sphere tree）：462, 463
区域游览（area navigation）：205, 214
曲面模拟（curvature simulation）：295, 370, 371
燃烧物质（burning materials）：180, 189
认证（authentication）：480, 485
日夜循环（Day/night cycle）：393
日志服务（Journaling Services）：123
三角函数（trigonometric function）：155, 161, 532
三角形条带（triangle strip（tri-strip））：295, 300, 316~322, 363
散列表（Hash Tables）：211, 213, 241, 437, 441, 474
散射（diffuse）：366, 382, 392, 393, 396, 411, 416, 418~420
扫描码（Scan codes）：91
舍入误差（round-off errors）：7, 147, 276, 297, 539, 540
设计模式（design pattern）：15, 59, 63, 117, 119, 440
声纳回声（sonar pings）：543
时区（time zone）：430
实时策略游戏（Real-Time Strategy, RTS）：237, 424, 425~427, 432, 433, 435
世界市场（world market）：85~98
事件管理器（Event managers）：5, 6, 8
事件锁定（event-locking）：425~431
视点矢量（eye vector）：361, 362
视界（view frustum）：310, 311, 332, 336
视界截面裁减（view frustum culling）：310, 332
收敛性（convergence）：167, 201, 202
手持设备（Handheld devices）：112, 507
兽群（Herds）：240
输入法编辑器（Input Method Editor, IME）：86, 87
输入事件（input events）：103, 226, 227, 233
数据包压缩（packet compression）：506
数字滤波器（digital filter）：529, 532, 537
衰减（attenuation）：184, 187, 198, 348, 412~414, 520, 521, 524, 532, 555, 556
四元数（quaternion）：169, 343, 347
随机合成（stochastic synthesis）：510, 529,

Download at www.Pin5i.Com

543
泰勒级数（Taylor Series）：143, 163~165, 171
填充块（padding）：478
贴地行走（foot-sliding）：345, 347, 348, 352
同步失调（out of synch）：427, 437~439, 441
同线的（collinear）：299, 301, 325
统一建模语言（Universal Modeling Language, UML）：67, 68, 206
凸凹贴图（bump map）：407, 409
凸包（convex hull）：241
完全可分解的世界（totally destructible world）：183
完整性检验值（Integrity Check Value, ICV）：479, 480
网络时间协议（Network Time Protocol, NTP）：429
网络协议（network protocols）：424, 425, 432, 438, 488, 505
微表面（Microfacets）：417, 418, 420
伪随机数字生成器（Pseudo-Random Number Generator, PRNG）：395
伪装（Spoofing）：477, 485
纹理采样（texel）：366, 370~375, 395, 398, 400~406, 409~413
无线设备（wireless devices）：501~507
无限冲击响应（IIR）：530
无限循环（Infinite loops）：25, 503
析构函数（destructor）：46, 47, 49, 52, 61, 137
细分（subdivision）：295, 327~336
下溢异常（Underflow Exception）：65
线性插值（Linear interpolation, Lerp）：159, 160, 308, 340, 347, 398, 402, 407, 412, 501
线性同余算法（Linear Congruential Algorithm, LCA）：543, 544
线性预测编码（linear predictive coding, LPC）：510, 535, 536
相邻的单元（neighbor cells）：181~184, 187, 191
向下混音（downmixing）：513, 518
小精灵（sprite）：100, 109, 111
协作流图（collaboration diagram）：68, 70, 71, 73
心理声学（psychoacoustic）：510, 512~514, 517, 518
性能评测（profiling）：133
性能评测树（profile tree）：134
序列号（sequence numbers）：478, 479
序列化（serialization）：424, 457, 466~476, 512, 552, 555
序列图（sequence diagram）：440, 450, 451
学习曲线（learning curve）：107
寻径（path finding）：148, 151, 204, 205, 214~219, 250, 260~269, 282~290
循环坐标推演（cyclic coordinate descent, CCD）：173, 174
移动电话（mobile phones）：501~503
移动信息设备框架（MIDP）：502
移植阶段（Conversion Stage）：14
异步 I/O（asynchronous I/O）：455, 456
异常处理（exception handling）：64~66
异常情况（Denormalization）：290, 531
因特网协议安全（Internet Protocol Security, IPSec）：477
音唇同步（lip synching）：499
音频压缩（audio compression）：510, 512~513
音障（obstruction of sound）：521
映射簇（Cluster Maps）：211
硬编码（hard-code）：34, 35, 73, 88, 93, 100, 102, 402, 506, 550, 553
游览（navigation）：204, 205, 214~219, 225, 269, 271~281
游览网格（navigation meshes）：271, 272
游戏开发的各个阶段（Development stages）：13
有限冲击响应（FIR）：530

Download at www.Pin5i.com

有限元分析（finite element analysis, FEA）：180
有限状态机（finite-state machine）：205, 206, 226, 443
语法分析器（parser）：76~83
语音合成器（vocoders）：535, 536
语音通信（voice communications）：424, 455, 497
语音压缩（Voice Compression）：510, 535
原型阶段（Prototype Stage）：13
云层（cloud cover）：378, 382~384, 389~393, 395, 397
云层阴影（cloud shadows）：382, 383
载荷（payload）：479, 480, 551
噪声生成器（noise generator）：552
粘滞阻尼（viscous damping）：198
战术式寻径（tactical pathfinding）：205, 260~269
遮蔽（occlusion）：295, 310, 379~383
遮蔽裁减（occlucion culling）：310
振荡器（oscillator）：552
帧锁定（frame-locking）：425
支撑域（support area）：328, 329, 335
支持类（Support Classes）：441
直升飞机的声音（helicopter sound）：555
智能的纹理缓存（smart texture cache）：406
智能指针（Smart pointers）：42
重采样（resampling）：142, 513, 518
重放攻击（Replay attacks）：477~479, 482
重力（gravity）：11, 112, 194, 196
重心坐标（barycentric coordinates）：372, 373
重绑定（rebind）：48
轴对齐的边界框（axis-aligned bounding boxes）：510, 524, 525, 528
逐像素（per-pixel）：364, 370, 397, 398, 401, 402, 407~415
主动场景（active scenery）：183
着色器（shader）：295, 308, 353~359, 366, 368, 369, 380, 381, 384, 389, 391, 393, 399, 407~409, 411~413, 416, 419
字体（Fonts）：86
自定义数据文件（Custom data files）：76~84
自动列表（autolists）：59~63

Download at www.Pin5i.Com